AKKODiS

Engineering a Smarter Future Together



PROVEtech:TA

OPERATING INSTRUCTIONS

VERSION: 2025 SE



PROVEtech Tool Suite

© Akkodis Germany Consulting GmbH, Sindelfingen 2025 - all rights reserved

No part of this publication may be reproduced, stored in a retrieval system, or transmitted, in any form or by any means, electric, mechanical, photocopying, recording, or otherwise, without the prior written permission of Akkodis Germany Consulting GmbH

Akkodis Germany Consulting GmbH
Flugfeld-Allee 12
D-71063 Sindelfingen

https://www.provetech.de
provetech@akkodis.com
Tel.: +49(0)7031 686-3333

https://www.akkodis.com

Table of contents

Chapter 1 **Notes on the manual** **1**

1.1 Organization and structure 1
1.2 Conventions and formatting 2

Chapter 2 **Installing and starting the program** **5**

2.1 The installation 5
2.2 Licensing 9
2.2.1 Computer-bound licensing 9
2.2.2 Floating licensing 18
2.2.2.1 License Borrowing 19
2.2.2.2 The license information dialog 22
2.3 Login 24
2.4 Configurations of the test system 29
2.5 Overview 30
2.5.1 Workpage 31
2.5.2 Test Manager 31
2.5.3 Diagnostics 32
2.5.4 Fault simulation 32
2.6 Placing the functional areas 32
2.7 Command line parameters 34
2.7.1 Calculation of the TOTP command line parameter 38
2.8 PROVEtech:TA Runtime version 39
2.9 PROVEtech:TA DAQViewer 40
2.10 32-bit and 64-bit 40
2.11 System requirements 42
2.12 Working with floating-point numbers 46
2.13 Creating a problem report 47
2.14 Processing and protection of personal data 51
2.15 Legal notice 52

Chapter 3 **The menu bar and toolbar** **79**

3.1 The file selection dialog 82
3.2 The Application menu 83
3.2.1 New login 84
3.2.2 Load configuration 84
3.2.3 Reset signals 84

3.2.4 Restart model....85
3.2.5 CCP/XCP configuration....86
3.2.6 Load CCP/XCP parameter file....87
3.2.7 Settings....88
3.2.7.1 Common....91
3.2.7.2 Configurations....93
3.2.7.2.1 Event macros....96
3.2.7.2.2 Replacement variables....100
3.2.7.2.3 Models....102
3.2.7.3 Workpage....139
3.2.7.4 Data acquisition....144
3.2.7.5 Test Manager....148
3.2.7.6 Database....150
3.2.7.6.1 Configure an Oracle ODBC connection....153
3.2.7.6.2 Configure a Microsoft SQL Server ODBC connection....156
3.2.7.6.3 Test ODBC connection and further database checks....157
3.2.7.6.4 Windows Authentication....158
3.2.7.6.5 Configure the database backup....161
3.2.7.6.6 Re-index the database....164
3.2.7.7 CAESAR diagnostics settings....165
3.2.7.8 MCD-3 D diagnostics settings....167
3.2.7.8.1 DoIP configuration....169
3.2.7.8.2 D-PDU API Monitor....172
3.2.7.9 Fault simulation....173
3.2.7.9.1 PROVEtech:µHiL fault simulation....177
3.2.7.9.2 Softing Engineering & Solutions fault simulation....178
3.2.7.9.3 ETAS ES4440 fault simulation....180
3.2.7.9.4 dSPACE FullSize fault simulation via CAN....182
3.2.7.9.5 dSPACE MidSize or FullSize fault simulation via RS232....188
3.2.7.9.6 dSPACE SCALEXIO fault simulation....189
3.2.7.9.7 dSPACE DS5394C1 fault simulation....190
3.2.8 Lock User Interface....191
3.2.9 Exit....193
3.3 The Workpage menu....193
3.3.1 New cockpit, load cockpit and save cockpit....193
3.3.2 New workpage, load workpage and save workpage....194
3.4 The Data Acquisition menu....195
3.4.1 Converting data acquisitions....199
3.5 The Macro menu....202
3.6 The Window menu....205
3.6.1 PROVEtech:TA Log....207
3.6.1.1 The menu bar and the toolbar....209
3.6.1.2 Settings....210
3.7 The Help menu....212
3.7.1 Technical Support....212

3.7.2 The Info dialog 213
3.8 Hotkeys overview 214
3.8.1 Hotkeys for the workpage 214
3.8.2 Hotkeys for the Test Manager (Tree, Explorer-tab, Info/Version-tab) 215
3.8.3 Hotkeys for the test parameters 216
3.8.4 Hotkeys for the Basic editor 217

Chapter 4 **Workpage** **223**

4.1 Managing the workpage 224
4.1.1 View scaling 227
4.2 The Signal Selection window 228
4.2.1 Signal area 229
4.2.1.1 Attribute selection 231
4.2.2 Column headers 233
4.2.3 Measurement signal selection, clipboard and info field 234
4.2.4 Selecting control types, adding controls 235
4.2.5 Changing CCP/XCP event channel numbers 238
4.3 Managing controls 240
4.3.1 Context menus 240
4.3.2 The property dialog for pages and group controls 247
4.3.3 Selection of controls 249
4.3.4 Alignment of several controls 250
4.3.5 Moving and duplicating controls 250
4.3.6 Sizing controls 252
4.3.7 Printing or adding controls to the test result 254
4.3.8 Find and replace 256
4.3.9 The hotkeys 257
4.3.10 Managing bitmaps 258
4.3.11 Hexadecimal representation of signal values 261
4.4 The display and control elements 261
4.4.1 General properties 262
4.4.1.1 The units converter 265
4.4.1.2 Import of text tables 266
4.4.2 The strip chart and multi-strip chart controls 266
4.4.2.1 The property dialog of the strip chart control 269
4.4.3 The XY strip chart control 273
4.4.3.1 The property dialog of the XY strip chart control 275
4.4.4 The slider / bar display control 278
4.4.4.1 The property dialog of the slider / bar display control 279
4.4.5 The switch / LED control 281
4.4.5.1 The property dialog of the switch / LED control 282
4.4.6 The number control 285
4.4.6.1 The property dialog of the number control 286
4.4.7 The bitmap control 288

4.4.7.1 The property dialog of the bitmap control 289
4.4.8 The bitswitch control 293
4.4.8.1 The property dialog of the bitswitch control 294
4.4.9 The gauge....296
4.4.9.1 The property dialog of the gauge....296
4.4.10 The grid control....297
4.4.10.1 The property dialog of the grid control....299
4.4.11 The surface graph control 300
4.4.11.1 The property dialog of the surface graph control 301
4.4.12 The custom control....302
4.4.12.1 The property dialog of the custom control 302
4.4.12.2 Bitmap / Text 305
4.4.12.3 Numeric Field....308
4.4.12.4 Button 310
4.4.12.5 Invisible Button....312
4.4.12.6 Macro Button....313
4.4.12.7 Slider....314
4.4.12.8 Combo Box 315
4.4.12.9 Color Display....316
4.4.12.10 Text Message....317
4.4.12.11 Text Signal Array 318
4.4.12.12 Basic Handler....319
4.4.13 The special controls 322
4.4.13.1 The property dialog of the special controls 329
4.4.14 The message window 331
4.4.14.1 The property dialog of the message window 342
4.4.14.2 Handling of Ethernet 356
4.4.15 The oscilloscope display 357
4.4.15.1 The property dialog of the oscilloscope display 358
4.4.16 The group control....360
4.4.17 The widget 360
4.4.18 The video control 365
4.4.18.1 The property dialog of the video control 366
4.5 The offline mode 368
4.5.1 The overview graph, evaluation of measure data 369
4.5.2 The property dialog of the overview graph 373
4.5.3 Extract signals from messages 375
4.5.4 Calculated signals....377

Chapter 5 **Test Manager** **381**

5.1 The structure of Test Manager....384
5.2 The objects of the Test Manager 386
5.2.1 Different types of tests....386
5.2.1.1 The system test....389

5.2.1.2 The elementary test 389
5.2.1.3 The object 390
5.2.1.4 The class 390
5.2.1.5 The text object 390
5.2.1.6 The user-defined test object 391
5.2.2 Projects and libraries 391
5.2.2.1 The project 391
5.2.2.2 The library 393
5.2.2.2.1 The Library in the Test Manager tree 394
5.2.2.2.2 The User Library in the Test Manager tree 394
5.2.3 The database object 394
5.2.4 The trash folder 395
5.2.5 Read-only test objects 397
5.2.6 Access rights 399
5.2.7 Object types in comparison 408
5.3 Managing tests 409
5.3.1 Creating new tests 409
5.3.2 Renaming tests 411
5.3.3 Changing the type 413
5.3.4 Recording a new test 414
5.3.5 Copying and moving tests, generating links 415
5.3.5.1 Copying and moving with the context menu 416
5.3.5.2 Copying and moving with drag and drop 419
5.3.6 The tab “Explorer” 421
5.3.7 The tab “User groups” 423
5.3.7.1 The user group 424
5.3.8 The tab “Users” 427
5.3.8.1 The user 428
5.3.8.2 The dialog “Additional attributes templates” 433
5.3.8.2.1 The attribute type “Choice” 434
5.3.9 The tab “Info/Version” 436
5.3.9.1 The tab “Description” 437
5.3.9.1.1 The HTML Text Edit control 438
5.3.9.2 The tab “Parameters” 446
5.3.9.2.1 The parameter type “Choice” 449
5.3.9.2.2 The dialog “Show inheritance” 450
5.3.9.3 The tab “ToDo list” 451
5.3.9.4 The tab “Documents” 452
5.3.9.5 The tab “Link” 453
5.3.9.6 The tab “Version description” 453
5.3.9.7 The tab “History” 454
5.3.10 Exporting and importing tests 455
5.3.11 Displaying the properties of tests 462
5.3.11.1 The tab “Info” in the property dialog 463
5.3.11.2 The tab “Database settings” in the property dialog 464

5.3.11.3 The tab “Extended” in the property dialog 467
5.3.11.4 The tab "Security" in the property dialog 469
5.3.12 Deleting tests 471
5.3.13 Comparing and merging test code 473
5.3.14 Linked test objects 479
5.4 The Basic editor 483
5.4.1 The toolbar 484
5.4.2 The hotkeys 488
5.4.3 The output window 492
5.4.4 Object and procedure selection 493
5.4.5 The edit window 493
5.4.6 The find and replace dialog 496
5.4.7 The method selection window 498
5.4.8 The dependencies window 502
5.4.9 Edit references as text 504
5.4.10 Locking the Basic code 505
5.4.11 Encryption of Basic code 507
5.5 Executing tests 512
5.5.1 The status display 515
5.5.2 Breakpoints 516
5.5.3 Skipping test objects 517
5.5.4 The timeout value 518
5.5.5 Pre-process the next test object for execution 520
5.5.6 Post-process the test result after execution 521
5.6 Test results 522
5.6.1 Displaying the test result in a dialog 523
5.6.2 The Comment 525
5.6.3 The test protocol 530
5.6.3.1 The test protocol of an elementary test 530
5.6.3.2 The test protocol of a system test or a project 532
5.6.3.3 Writing test protocols in HTML format 534
5.6.3.4 Test Automation references 535
5.6.3.5 Writing test protocols in XML and formatting via XSL-T 541
5.6.3.6 Test protocols formatting using post-processing 543
5.6.4 Links 549
5.6.5 Runtime errors 549
5.6.6 ECU errors 550
5.6.7 Keyword values 551
5.6.8 Parameters 552
5.6.9 Attachments 552
5.6.10 The history of a test result 557
5.7 Managing test results 558
5.7.1 Displaying the list of test results 558
5.7.2 Test results of older test versions 562
5.7.3 Deleting test results 563

5.7.4 Restoring deleted test results 564
5.7.5 Validating test results 566
5.7.6 Comparing test results 566
5.7.7 The tab "Overview" 568
5.8 Versioning 570
5.8.1 Release a test version 574
5.8.1.1 Pre-process before creating a release version 577
5.8.2 Create a branch 578
5.8.3 Restore a test version 579
5.8.4 Delete a test version 580
5.8.5 The version number 582
5.8.6 Executing older test versions 582
5.8.7 The Branch Selection 583
5.8.7.1 Define the branch selection by context menu 584
5.8.7.2 The branch selection overview dialog 586
5.9 User-defined test objects and extending the context menu 586
5.9.1 The structure in the path for the extended context menu 587
5.9.2 Creating a new user-defined test object 590
5.9.3 Extending the context menu 590
5.9.4 Executing user-defined test objects 594
5.9.5 Embedded editors 596
5.10 The Quota mechanism 598
5.11 Create and restore a database backup 600
5.12 Database consistency checks 603
5.13 Multiple databases in the Test Manager tree 607
5.14 The remote test sequencer 609
5.14.1 Register the remote test sequencer 611
5.14.2 Working with the remote test sequencer 613
5.14.3 The remote test sequencer and access rights 615
5.14.4 Automatic control of the sequencer order 616
5.14.5 The schedule 618
5.15 Access from foreign software via URI 623
5.16 Supported URL formats 625
5.17 ToDo list dialog 626
5.18 Find, filter and replace 630
5.18.1 Finding tests and test results 630
5.18.1.1 Examples of find options 637
5.18.1.2 Finding by Basic handler 639
5.18.2 Filter tests and test results 641
5.18.3 Find and replace in test programs 642
5.18.4 Regular expressions 644

Chapter 6 **Diagnostics** **647**

6.1 CAESAR diagnostics 648

6.1.1 Basics 648
6.1.2 The diagnostics dialog 649
6.1.2.1 Static commands of the diagnostics dialog 650
6.1.2.2 The diagnostics tree 652
6.1.2.2.1 The context menu of a diagnostic object 653
6.1.2.2.2 Searching in diagnostics tree 655
6.1.2.3 The data window of diagnostics dialog 657
6.1.2.3.1 Data types used in data window 658
6.1.2.4 The trace window of the diagnostics dialog 659
6.1.2.5 User defined selection of diagnostic objects 661
6.1.2.5.1 Adding a diagnostic object to the user selection 661
6.1.2.5.2 Removing a diagnostic object from the user selection 662
6.1.2.5.3 Apply or reset the current user selection 663
6.1.3 ECU ID 664
6.1.4 ECU errors 665
6.1.4.1 Environment data (sub errors) 666
6.1.4.2 Extensions for the keyword protocol 2000 667
6.1.4.3 Extensions for the UDS protocol 668
6.1.5 Diagnostic services 672
6.1.5.1 Sending a user defined request message 674
6.1.6 Coding 675
6.1.6.1 General features 675
6.1.6.2 Loading and Saving .AED files 679
6.1.7 Flashing an ECU 681
6.1.7.1 Changing the priority of a flash table 682
6.1.7.2 Flashing a single flash table 683
6.1.8 Variants of an ECU 684
6.1.9 Communication parameters 685
6.1.10 Extended and default diagnostics session 688
6.1.10.1 Switching the diagnostic session type 689
6.1.11 ECU settings 690
6.2 MCD-3 D diagnostics 694
6.2.1 Basics 694
6.2.2 Terms in the MCD-3 D diagnostics 696
6.2.2.1 Measurement Calibration Diagnostics (MCD) 696
6.2.2.2 MCD-3 D 696
6.2.2.3 D-PDU API 697
6.2.2.4 Modular Vehicle Communication Interface (MVCI) 697
6.2.2.5 Simulating diagnostic communication 697
6.2.2.6 DoIP – Diagnosis over IP 698
6.2.2.6.1 Standard DoIP configuration 698
6.2.2.6.2 Manually configuring DoIP 699
6.2.2.6.3 Hints for DoIP configuration 700
6.2.2.7 Open Diagnostic Data Exchange (ODX) 701
6.2.2.8 Packaged ODX (PDX) 701

6.2.2.9 Java Archive (JAR) 701
6.2.2.10 Jobs In Diagnostics 701
6.2.2.11 Service In Diagnostics 702
6.2.2.12 Semantic As Attribute 702
6.2.2.13 Vehicle Information 702
6.2.2.14 Logical Link 702
6.2.2.14.1 Special Case Diag Over IP (DoIP) 702
6.2.2.15 Variant 703
6.2.2.16 Session 703
6.2.2.17 Activation 704
6.2.2.18 The Fault Memory (DTC) 705
6.2.2.19 Variant coding 705
6.2.2.20 Flashing 705
6.2.2.21 Communication parameters 706
6.2.2.22 QuickTest 706
6.2.2.23 Monitoring 706
6.2.2.24 DoIP monitoring 708
6.2.3 Diagnostics MCD-3 D window 708
6.2.3.1 Action toolbar 709
6.2.3.2 Control toolbar 711
6.2.3.3 Filtering the views 712
6.2.3.4 Navigation area 713
6.2.3.4.1 Navigation hierarchies 715
6.2.3.4.2 Favorites 721
6.2.3.4.3 Navigation area context menu 725
6.2.3.4.4 Find in navigation area 726
6.2.3.4.5 The properties dialog 728
6.2.3.5 The request view 730
6.2.3.5.1 Name in request view 732
6.2.3.5.2 Value in request view 733
6.2.3.5.3 Unit in request view 733
6.2.3.5.4 Type in request view 733
6.2.3.5.5 Semantic in request view 734
6.2.3.5.6 Upper and lower limit in request view 735
6.2.3.5.7 Special case: variable parameters 735
6.2.3.6 The response view 736
6.2.3.6.1 The result of an execution 738
6.2.3.7 Communication view 739
6.2.3.8 The ECU element 740
6.2.3.8.1 The setting Alias 741
6.2.3.8.2 Variant identification and selection 741
6.2.3.8.3 Check variant on activation 741
6.2.3.8.4 The attribute Logical Link 742
6.2.3.8.5 Initial session and current session 742
6.2.3.8.6 The context menu on ECU level 743

6.2.3.9 The coding 744
6.2.3.9.1 Prerequisites for the coding 745
6.2.3.9.2 The VCD qualifier 746
6.2.3.9.3 The context menus of the coding 746
6.2.3.9.4 The AED file dialog 748
6.2.3.9.5 The ECX file dialog 751
6.2.3.9.6 The SGDE file dialog 761
6.2.3.10 The DTCs 764
6.2.3.10.1 Reading and clearing DTCs 765
6.2.3.11 Artificial elements 765
6.2.3.12 Services and jobs 766
6.2.3.12.1 Special case: ComParams (service ProtocolParameterSet) 767
6.2.3.12.2 Special case: the FlashJob 767
6.2.3.12.3 Flash window 769
6.2.4 Button to terminate the MCD-3 D server 770

Chapter 7 **Fault simulation** **773**

7.1 PROVEtech:µHiL fault simulation 774
7.1.1 Architecture 774
7.1.2 Wiring faults 775
7.1.2.1 Preselection of cables 776
7.1.2.2 Shorts and breaks 777
7.1.3 Fault patterns and reset 780
7.2 Softing Engineering & Solutions fault simulation 781
7.2.1 Architecture 781
7.2.2 Wiring faults 782
7.2.2.1 Preselection of cables 783
7.2.2.2 Shorts and breaks 785
7.2.2.3 Contact resistances and external devices 787
7.2.3 CAN fault settings 789
7.2.4 Fault patterns and reset 790
7.3 ETAS ES4440 fault simulation 791
7.3.1 Architecture 791
7.3.2 Wiring faults 792
7.3.2.1 Preselection of cables 793
7.3.2.2 Shorts, breaks and current measurement 794
7.3.2.3 Contact resistances, duration and loose contacts 797
7.3.3 Special cases 798
7.3.4 Fault patterns and reset 799
7.4 dSPACE FullSize fault simulation via CAN 800
7.4.1 Architecture 800
7.4.2 Wiring faults 801
7.4.2.1 Preselection of cables 802
7.4.2.2 Shorts and breaks 804

7.4.2.3 Contact resistances and external devices 806
7.4.3 CAN settings 808
7.4.3.1 CAN topology 809
7.4.3.2 CAN fault settings 810
7.4.4 Bus faults 812
7.4.5 Fault patterns and reset 814
7.5 dSPACE FullSize fault simulation via RS232 814
7.5.1 Architecture 814
7.5.2 Wiring faults 815
7.5.2.1 Preselection of cables 816
7.5.2.2 Shorts and breaks 817
7.5.3 Fault patterns and reset 820
7.6 dSPACE MidSize fault simulation 820
7.6.1 Architecture 820
7.6.2 Wiring faults 821
7.6.2.1 Preselection of cables 821
7.6.2.2 Shorts and breaks 823
7.6.3 Fault patterns and reset 826
7.7 dSPACE SCALEXIO fault simulation 826
7.7.1 Architecture 827
7.7.2 Wiring faults 829
7.7.2.1 Preselection of cables 829
7.7.2.2 Shorts and breaks 830
7.7.3 Fault patterns and reset 832
7.8 dSPACE DS5394C1 fault simulation 833

Chapter 8 **Troubleshooting** **835**

8.1 Problems concerning the model 836
8.1.1 Problem analysis concerning dSPACE systems 836
8.1.2 Problem analysis concerning the communication via TCP/IP (includes dSPACE SCALEXIO) 839
8.1.3 Real-time violations 842
8.1.4 Problems concerning the TOI block-set 843
8.1.5 Which MATLAB/Simulink versions are supported 844
8.2 Problems concerning the workpage 845
8.2.1 Trouble while parsing an .a2l file 845
8.2.2 CCP/XCP signals do not get updated 846
8.2.3 Button to configure CCP/XCP disabled 848
8.2.4 The maximum amount of measurement signals is not sufficient 848
8.2.5 Issues with the video capturing 849
8.3 Problems concerning the Test Manager 849
8.3.1 The “Test Manager” tab is not displayed 849
8.3.2 Logging in to the Test Manager does not work 850
8.3.3 The Test Manager cannot show or save the special characters correctly 853

8.3.4 Start script....854
8.3.5 Different time zones and daylight saving time....855
8.3.6 Exporting including all dependencies....855
8.3.7 The test object got lost....856
8.3.8 Data security and avoiding data loss in the Test Manager....859
8.4 Problems concerning the database....863
8.4.1 Problems when connecting to the database....863
8.4.1.1 Oracle error messages....864
8.4.1.2 PostgreSQL error messages....867
8.4.1.3 Microsoft SQL Server error messages....870
8.4.1.4 Problems related to LoadLibraryEx....872
8.4.2 Problems caused by lost connection to the database....874
8.4.3 Problems when installing the database....875
8.4.4 The locally installed PostgreSQL server no longer starts....877
8.4.5 The database schema must be updated....877
8.4.6 The database schema is internal information....879
8.4.7 The database behaves very strange....880
8.4.8 The database is very slow....880
8.4.9 There is not enough storage space in the file system left for the database....884
8.4.10 How can I determine the actual storage space of my database?....887
8.4.11 Failed to create a database backup....887
8.4.12 Transfer the data to another database instance....889
8.4.13 PostgreSQL error message “Out of shared memory”....891
8.4.14 New created database IDs violate unique index constraint....891
8.4.15 SQL Server error message about Transaction log....892
8.4.16 The database is busy....893
8.5 Problems concerning test programming....894
8.5.1 A strange WinWrap Basic license error dialog pops up....895
8.5.2 How to access COM / .NET Assemblies / Word / Excel / CANoe / ActiveX / OLE / third-party components in your test program....895
8.5.3 Error message when accessing a Basic object or a Basic class....897
8.5.4 Problems concerning “Go To Definition Of...”....897
8.5.5 Alternative test languages instead of Basic....898
8.5.6 Output parameters in Basic methods....899
8.5.7 How can I uniquely reference a test object?....900
8.5.8 Resolving cyclic references....900
8.5.9 '#Language "WWB.NET" is not working....901
8.5.10 The execution time of the Basic script takes very long....901
8.6 Problems concerning the AL/RTAE....902
8.6.1 RTAE link overflow....902
8.7 Problems with CAN (both CAESAR and MCD-3 D)....903
8.7.1 TX Error....903
8.7.2 RX Error....904
8.8 Problems concerning the eCOM....904

8.8.1 General eCOM problems 904
8.8.2 Malfunction between eCOM and PC 905
8.8.3 Malfunction between eCOM and PROVEtech:TA 906
8.8.4 Malfunction between eCOM and ECU 907
8.8.5 eCOM not recognized 907
8.8.6 Conflict between CAESAR and MCD-3 D 908
8.8.7 Version mismatch between firmware and toolkit 908
8.8.8 Further problems 909
8.9 Problems concerning the CAESAR diagnostics 909
8.9.1 No matching resources to open a channel 909
8.9.2 No hardware present 910
8.9.3 Diagnostics interface of an ECU 911
8.9.4 Nexiq Device / Part J / Pass-Thru 911
8.9.5 CAESAR Part Y and Part F 913
8.9.6 TIMEOUTP2CAN 913
8.9.7 Activation without matching variant 913
8.9.8 Activation of your ECU 914
8.9.9 Missing service, coding fragment, etc. 914
8.9.10 Update of CAESAR Firmware 915
8.9.11 CAESAR release authentication 915
8.9.12 .ccf files 915
8.9.13 Application is hanging after CAESAR call 916
8.9.14 “Cannot send message” on low speed CAN 916
8.9.15 Coding problems 916
8.9.16 Channel init failed 917
8.9.17 Application crashes due to data version mismatch 918
8.10 Problems concerning the MCD-3 D diagnostics 918
8.10.1 Configuring MCD-3 D servers 918
8.10.1.1 Installing MCD-3 D 918
8.10.1.1.1 Installing DTS (Softing) 919
8.10.1.1.2 Installing Prodis and VW kernels 920
8.10.1.2 Basics 921
8.10.1.3 Working without hardware 921
8.10.1.3.1 Softing kernel without hardware 922
8.10.1.3.2 DSA and VW kernels without hardware 922
8.10.1.4 Configuring DTS8 and DTS9 922
8.10.1.5 Configuring DSA PRODIS and VW kernel 928
8.10.2 Trace recording with MCD-3 D servers 930
8.10.2.1 Trace recording with DTS 930
8.10.2.2 Trace recording with DSA/VW 931
8.10.3 Known issues with MCD-3 D servers 932
8.10.3.1 Softing DTS 932
8.10.3.1.1 Sporadic MCD-3 D server crashes or deadlocks when using it 932
8.10.3.1.2 License Problems with DTS8 933
8.10.3.1.3 Device Explorer blocks initialization 933

8.10.3.1.4 Conflicts between different versions of D PDU-API 934
8.10.3.1.5 Conflicts With CAESAR Module 935
8.10.3.1.6 Loosing contact with ECU 935
8.10.3.1.7 MCD-3 D System Crashes due to wrong file versions 935
8.10.3.1.8 Flashing Problems 936
8.10.3.1.9 Non-standard Behavior 937
8.10.3.1.10 Problems Concerning Flashing 937
8.10.3.1.11 Star3 problems 938
8.10.3.1.12 Problems with SecureSession 938
8.10.3.2 DSA PRODIS and VW kernel 938
8.10.3.2.1 Problems with Java Virtual Machine 938
8.10.4 Graphical user interface of MCD-3 D diagnostics 939
8.10.4.1 When starting PROVEtech:TA for the very first time the diagnostics will not start 939
8.10.4.2 Client is in use 939
8.10.5 Communication issues with the ECU 939
8.10.5.1 Execution Of A Specific Service Or Job Fails Or Returns Incorrect Results 939
8.10.5.2 Start of a MCD-3 D configuration with PROVEtech:TA fails with “DSA; Database format version file missing for project xy” 940
8.10.5.3 Execution of a service or activation of an ECU fails with “PDU CreateComLogicalLink failed” 941
8.10.5.4 TX-Timeout (PDU_ERR_EVT_TX_ERROR) 941
8.10.5.5 RX-Timeout (PDU_ERR_EVT_RX_ERROR) 941
8.10.5.6 '181A;DSA: Data type mismatch for ProtocolParameter […]’ 942
8.10.5.7 CompuMethod Error (eSYSTEM_COMPU_METHOD_FAILED) 942
8.10.5.8 Access to diagnostic databases not possible 942
8.10.5.9 Table entry not found 943
8.10.5.10 Java runtime error after job execution 943
8.11 Problems concerning the fault simulation 943
8.11.1 The ECU does not react as expected 943
8.11.2 Error frames in the CAN communication 944
8.11.3 Critical PROVEtech:TA Message 944
8.12 Other issues 945
8.12.1 Transferring a complete PROVEtech:TA configuration 945
8.12.2 PROVEtech:TA crashes or hangs 945
8.12.3 PROVEtech:TA runs very slowly 946
8.12.4 Memory overflow 947
8.12.5 Screen scaling 951
8.13 Tips and tricks 952

Chapter

1

Notes on the manual

1.1 Organization and structure

This manual will teach you how to work with PROVEtech:TA, a software application for the control and automation of test systems. The manual will guide you through the operations that you perform on the test system.

The manual is arranged into the following chapters:

1 Notes on the manual This first chapter gives general advice on working with the manual.

2 Installing and starting the program This chapter describes how to install PROVEtech:TA and obtain a license file. Furthermore, it explains how to log in to PROVEtech:TA and how to select the desired configuration. There is an overview of the PROVEtech:TA user interface, the command line parameters, the PROVEtech:TA Runtime version with hidden user interface, the PROVEtech:TA DAQViewer (Data Acquisition Viewer), how to create a problem report and legal notices.

3 The menu bar and toolbar This chapter describes the menu bar and the toolbar of PROVEtech:TA.

4 Workpage The workpage is used to configure different display and control elements for your signals. This chapter shows you how to use the many settings and management options that are available.

5 Test Manager The Test Manager is used to manage tests and test results. You learn how to create test programs, have them executed automatically and evaluate the test results at the end.

6 Diagnostics In this chapter you will find out how to read the fault memory of ECUs, how to code and flash ECUs and how to perform diagnostic services.

7 Fault simulation This chapter explains how to set errors in the cabling of the vehicle's electrical system, the sensors/actuators, and the CAN bus so that they have an effect on the various ECUs. You can then examine how the set defects affect the function of the ECUs.

8 Troubleshooting In this chapter you can find typical problems and questions to each of the above mentioned chapters including possible causes, solutions and answers.

The ***Test Language*** Manual is integrated in this document too. Here you can read about the test language extensions to WinWrap Basic that are added especially for PROVEtech:TA.

Also included is the ***Automation Library*** manual where you can find information about how to program a real-time test script.

The ***Administrator's Guide*** explains the entries of all configuration files. You can set them in the Settings dialog of PROVEtech:TA.

1.2 Conventions and formatting

The manual uses particular signs and formatting to refer you to important points in the text:

Italic type	Text in italic type indicates the name of a menu item.
Bold type	Text in bold type emphasizes a term.
`Buttons`	Buttons, radio buttons, check boxes, etc., are described using a monospaced font.
Cross references	Cross references to a different chapter are shown in this way.

Click	This means pressing the left mouse button.
Drag and drop	Drag and drop means clicking on a user interface element (list entry, tree entry, etc.), dragging it to the desired position and dropping it.
Copy and paste	Copy and paste means copying objects to the Windows clipboard and inserting these objects at a different position.



This sign indicates references to possible sources of error and malfunctions.



This sign indicates warnings.



Examples and tips that may be of additional help to you are indicated by this symbol.

Chapter

2

Installing and starting the program

2.1 The installation

After starting “TASetup.exe” you might need to enter the password of the Windows administrator. After that the following dialog appears.

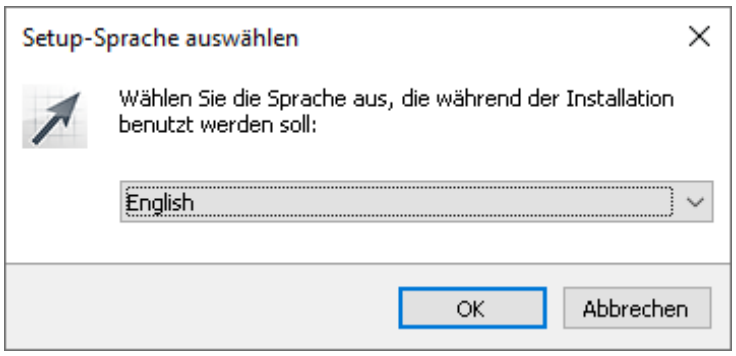


Figure 1 – Choose Setup language

If it is a new installation the language selected here is the one for the installation dialogs as well as PROVEtech:TA itself. However, it is possible to change it user dependently after the installation. From here on, simply follow the installation instructions and always click on `Next` to go to the next page.

The setup program not only installs PROVEtech:TA from scratch but also updates existing installations. To do an update, select an existing PROVEtech:TA directory on the page “Select Destination Location” and click on `Next`. Normally the page automatically displays this path in case PROVEtech:TA is already installed.

When you install PROVEtech:TA from scratch, the following page is displayed after you select the destination directory:

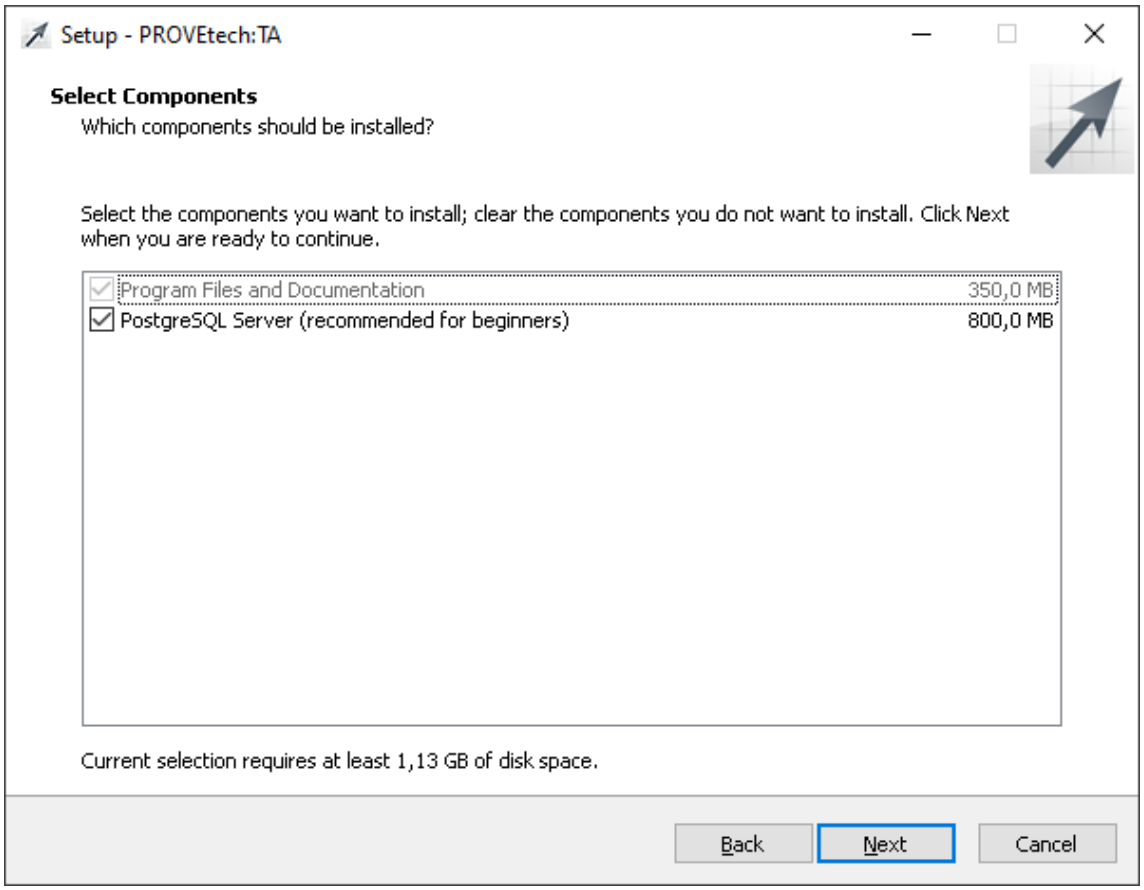


Figure 2 – Select Components

If you want to use the Test Manager (see chapter ***2.5.2 Test Manager***), you need a database server. This can either run on your local computer or a remote server. PROVEtech:TA supports three different database systems: Oracle, Microsoft SQL Server and PostgreSQL. The last one is a free system, for the others you need a separate license from Oracle or Microsoft. Only the “Express” versions of these servers are free.

Select `PostgreSQL Server` for automatically installing the PostgreSQL server locally. In case you do not know what you need, it is recommended to activate this component. Below you get more information about this component.

PROVEtech:TA accesses the database server via a so-called ODBC driver. The setup automatically installs fitting ODBC drivers for all three supported systems without changing any Windows settings.



The communication with databases (also local databases) always works via network access. Please make sure that no firewall (also no personal firewall) blocks the network access. In case of using a Windows standard installation with the Windows standard firewall no additional settings are necessary. However, when using a remote server the server must accept the incoming access.

Use the next two pages of the installation program to select the link creation in the start menu and on the Windows desktop.

The following page of the installation program shows a summary of your settings. Click on `Install` to start the installation.

At the end, click on `Finish` to close the setup program.

In case you selected the “PostgreSQL Server” above, another window appears after the PROVEtech:TA installation for the PostgreSQL Server installation:

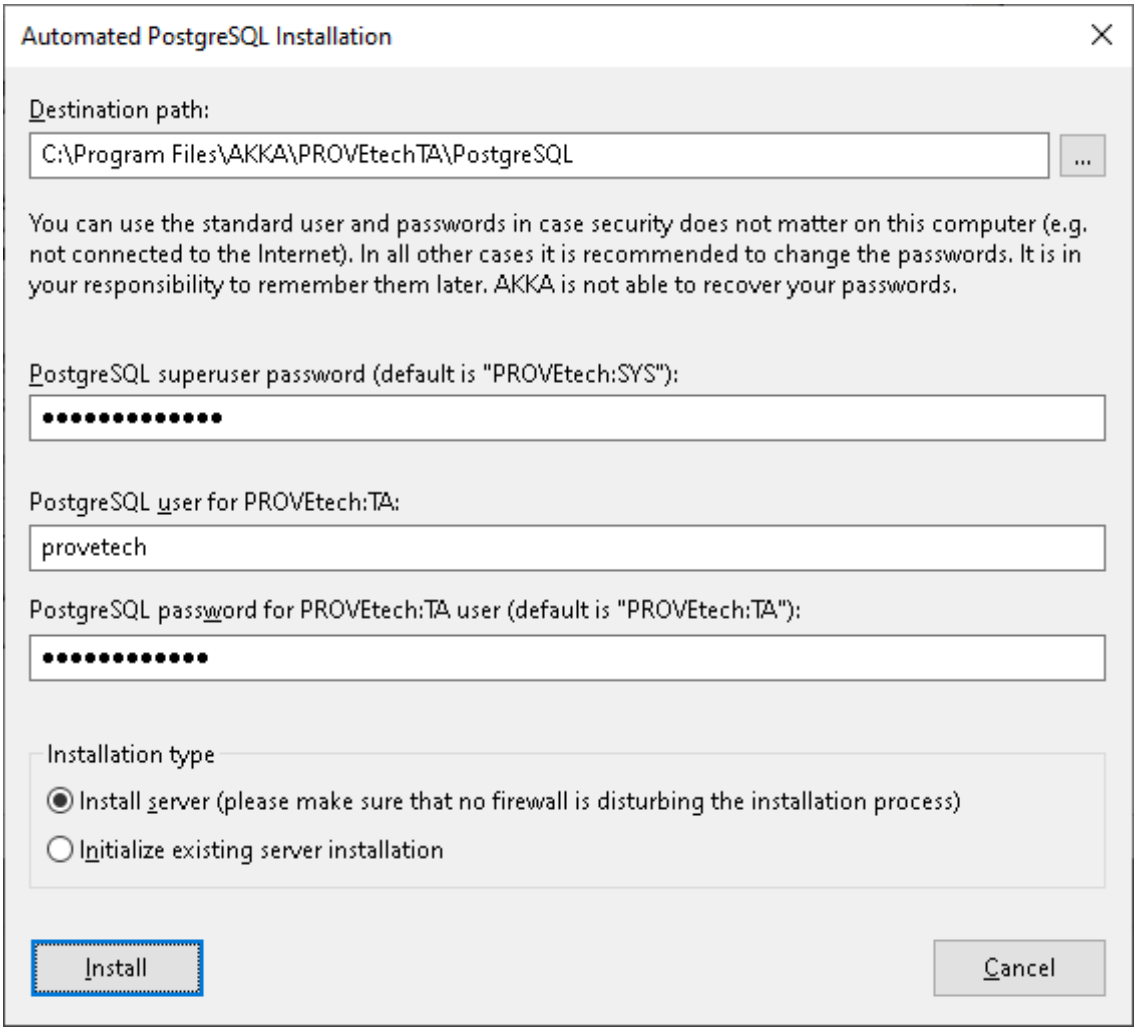


Figure 3 – PostgreSQL Server installation

Use the field `Destination path` to choose a different installation path than for PROVEtech:TA, e.g. on a different volume.

The installation program creates two uses in PostgreSQL: A so-called superuser (administrator) with full access rights. This one always has the name “postgres”. You can modify the password via this window. Another user is for the PROVEtech:TA access with limited access rights. For that one you can choose the name and password via this window. You should change the passwords in case the computer has a network access. However, you must remember your passwords as the PROVEtech Support cannot reset them.

These users are independent from the users of the Test Manager (see chapter ***5.3.8.1 The user***).



If you want to update an existing PROVEtech:TA installation to a newer version, please consider the following issues:

- You might need a new license file
- Read the history containing information about changes to avoid compatibility issues (https://www.provetech.de/ta/history)
- Please care about backups (e.g. database backup)

After that you can immediately start the installation. It is not necessary to uninstall PROVEtech:TA before that. You can update any previous version, for example from version 2011 SE directly to the current version.



The parallel installation of several .PROVEtech:TA versions is possible but not directly supported by the installation program. The installation program copies the PROVEtech:TA configuration files into the directory "<CommonDocuments>\PROVEtechTA" where "<CommonDocuments>" typically is „C:\Users\Public\Documents" since Windows Vista. The Windows localization influences the exact path.

In case the installation program finds files in that path for a new installation (not when updating an existing installation), you get a warning message.

2.2 Licensing

You can activate PROVEtech:TA in two different ways:

- ***2.2.1 Computer-bound licensing***
- ***2.2.2 Floating licensing***

2.2.1 Computer-bound licensing

When you start PROVEtech:TA via the Windows desktop or the start menu for the first time, the following license dialog is displayed:

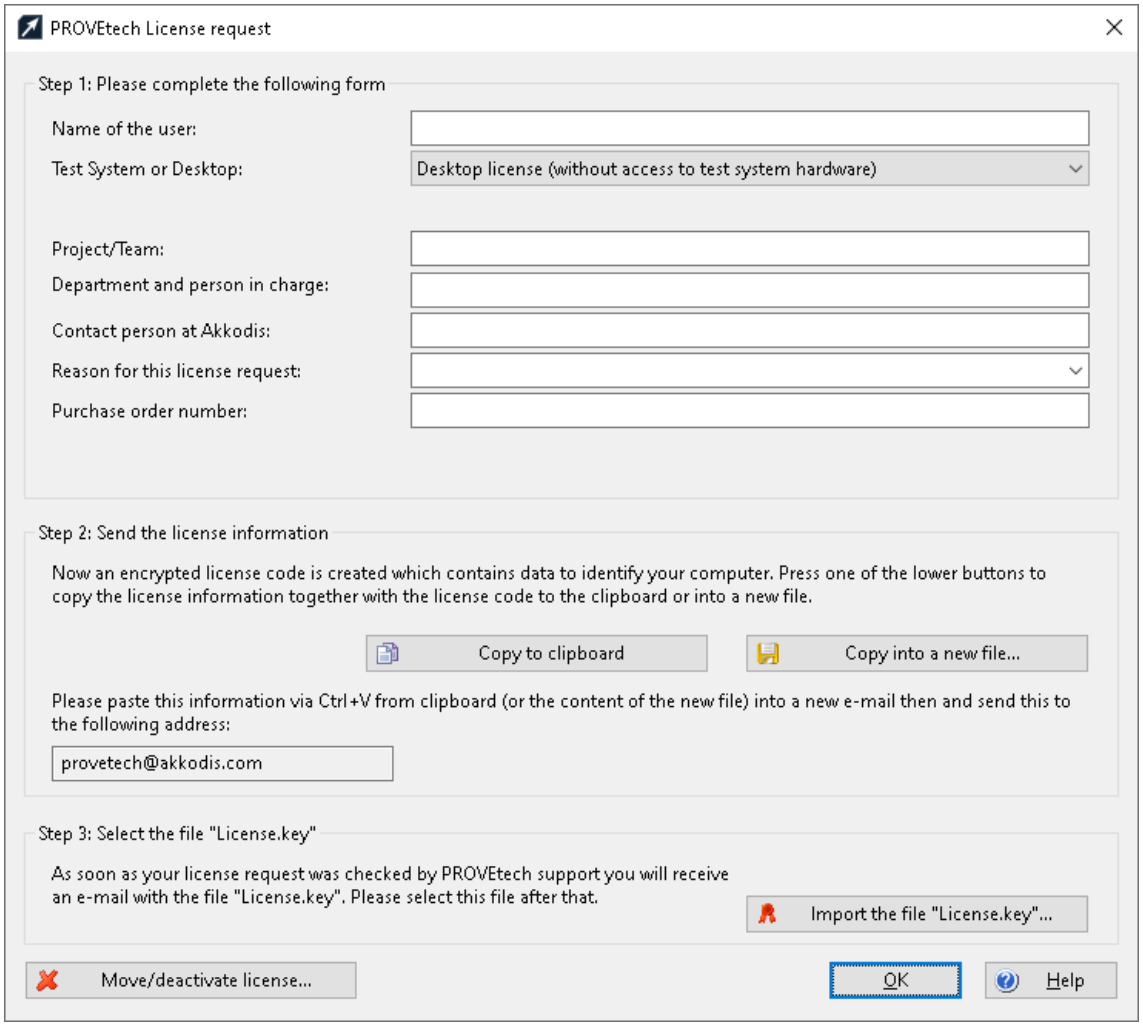


Figure 4 – The license dialog

Just follow the instructions of the dialog. Depending on your license contract, you immediately receive the license file “License.key”. So please do the following steps:

1. Complete the form of “Step 1”. The PROVEtech Support needs this information to synchronize the data of your license purchase order.

2. As described in “Step 2”, click on the button `Copy to clipboard` to copy your data and the corresponding license code as text into the clipboard. You can also click on the button `Copy into a new file...` if e.g. there is no e-mail program installed on your computer or you do not have network access.
   Important: Make sure that you correctly set the computer time on your local computer. Otherwise, the PROVEtech Support cannot create a license file for you and you have to repeat the license process from start.

3. Create immediately a new e-mail in your mail tool and set the subject, e.g. “License request”.

4. Paste the text (e.g. via hotkey Ctrl+V) from the clipboard into your e-mail or attach the currently saved file from step 2.
5. Copy the e-mail address of the License dialog into the addressee field of your e-mail.
6. Send your e-mail.
7. After your request has been reviewed by PROVEtech Support, you will immediately receive the license file “License.key” by e-mail. Save it on your hard disk.
8. If you temporarily have finished PROVEtech:TA, start again and run “Step 3”. Click the button Import the file “License.key”… and choose the license file “License.key” that you just saved to hard disk. Now the license file is copied and later you can delete or archive the original file.
9. After that, PROVEtech:TA is starting, and the licensing is finished.



Licensing problem or license expired

If the license dialog appears again (e.g., due to a licensing problem or because the license has expired), repeat the entire process. PROVEtech support can use the new licensing code from your e-mail to analyze your licensing problem.

Please do not send any screenshots, as these do not usually contribute to the analysis.

The file “License.key” specifies the expiry date, the activated PROVEtech products, the version binding and the activated modules of PROVEtech:TA. Furthermore, it enables or disables any hardware access to the test system (test system license). Without hardware access your license is a desk license, which allows you to, for example, edit your test programs or evaluate your test results.

Apart from a new installation, the following are reasons why the license dialog may appear at PROVEtech:TA start-up:

- *New installation of PROVEtech:TA.* PROVEtech:TA cannot be used immediately after installation. First, a valid computer-dependent license file “License.key” is needed in order to run PROVEtech:TA on this computer. This License.key can only be granted by the PROVEtech Support of Akkodis. Please note (for example, in the case of a customer’s demonstration installation), that it may take some hours before you receive your file “License.key”. The PROVEtech Support needs this time to verify the PROVEtech:TA license. The file “License.key” can also be generated a few days before PROVEtech:TA is

installed. For this option, you should contact your Akkodis representative. However: The more detailed the information you provide, the quicker the license file can be generated.

- *The license expired and has to be renewed*. The PROVEtech:TA licenses are often set to expire at the end of the calendar year, after which they must be yearly renewed. To serve as a reminder and not to block the users, the license dialog starts appearing three weeks before the license expires. During this three-week period PROVEtech:TA can still be started, despite the license dialog. Now you should renew your support agreements and request the new license file.

- *The license file does not cover the installed PROVEtech:TA version.* If you update your PROVEtech:TA version, then a new license file "License.key" could be required. You should synchronize this with PROVEtech Support before you update your PROVEtech:TA version.

- *The license file "License.key" was accidentally deleted or overwritten by a different computer's license file within the last week.* Sometimes a new license file is accidentally chosen on a computer for which it was not intended. It is therefore advisable, that you create copies of your license files and carefully structure and manage them. However, a new file "License.key" can be requested from the PROVEtech Support at any time.

- *The computer time was changed.* This case could occur if your license file "License.key" has a global expiration date (in contrast to a version-bounded license file). If the computer time is altered by more than a certain margin (e.g. set back by one day or a few hours), then the license dialog will appear. The license mechanism already accounts for different time zones as well as daylight saving time. In case you move your computer from one time zone to a different one, it is important that you change the Windows time zone and not the system time.

- *The Windows operating system was reinstalled.* When you change your operating system (e.g. from Windows 10 to Windows 11) a new PROVEtech:TA license file is needed. The same applies when the operating system is damaged and must be reinstalled on the computer.

- *The hardware changed within the last week.* The license file is bounded to your computer's hardware. When your hardware is altered PROVEtech:TA ignores the change for week thus accommodating temporary hardware changes. However, after the week is up the license dialog will start appearing. PROVEtech:TA recognizes a deactivation of hardware in Windows as a 'hardware change'. Permanent changes in your hardware require a new license file "License.key" which can be requested from PROVEtech Support.

- *The license synchronization path does not contain a valid license file.* If the user defined the license synchronization path in "`TA.ini`" configuration file (see ***Administrator's Guide***), the remote location should contain the file

"COMPUTERNAME_License.key", where COMPUTERNAME is the computer name provided in Windows Control Panel System dialog. If either the license does not exist, or either, does not contain a valid license for the current computer, contact with your IT Administrator in order to clarify it.

If the license file "License.key" has expired and must be renewed, the license dialog will appear in a modified form.

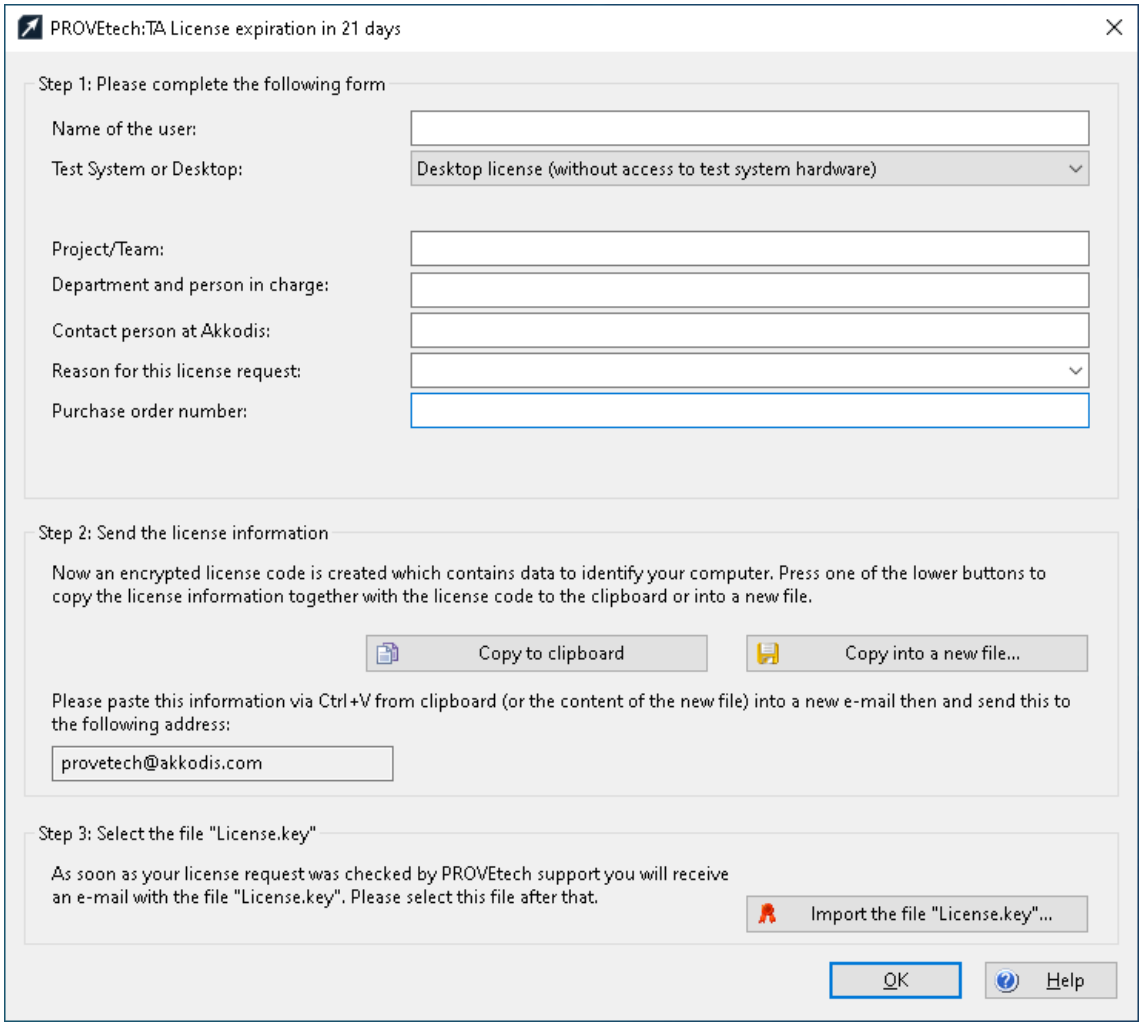


Figure 5 – The license dialog with expiration time

In the example above PROVEtech:TA will fully launch after you click OK on this license dialog. However, per this example, in 21 days it will cease to function unless a new license file "License.key" is obtained. You should now carry out as soon as possible the license renewal to ensure a timely creation of the new license file "License.key".

Move a license to a different computer

To receive the authorization to move a license to a different computer you first must de-activate the license on the old computer.

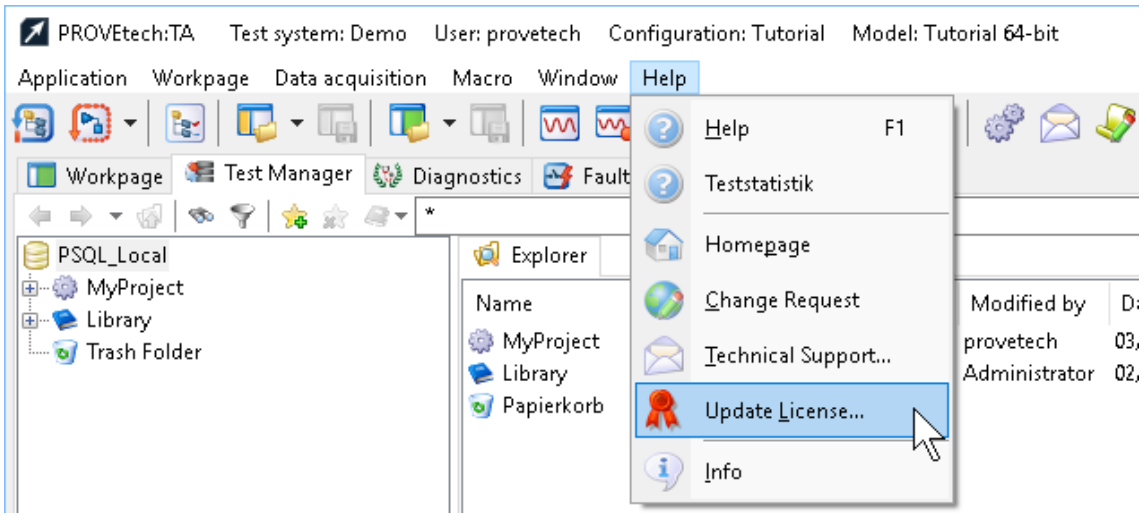


Figure 6 – Main menu "Update license…"

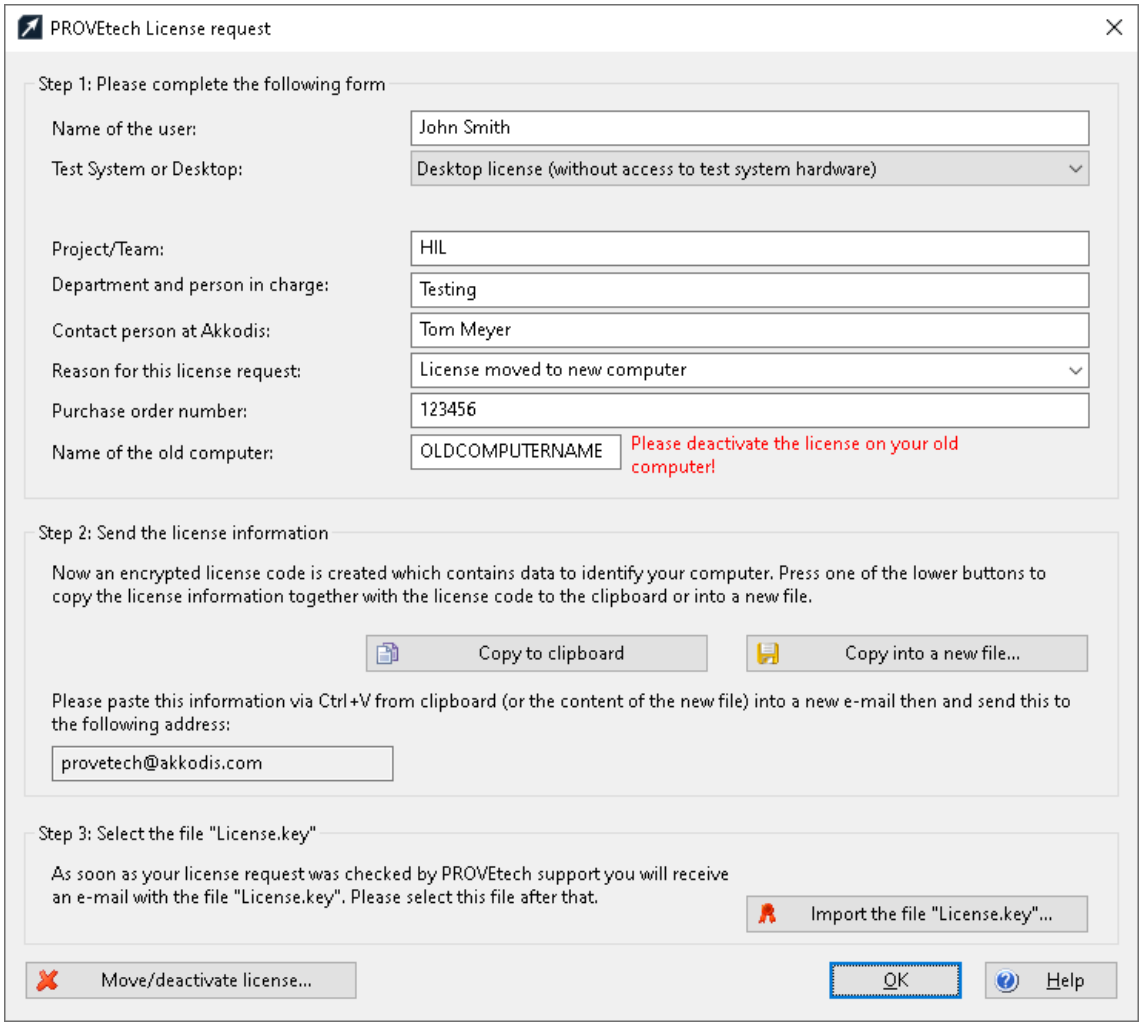


Figure 7 – Move/Deactivate license…

Please proceed the following steps to move a license to a different computer:

1. Start PROVEtech:TA on the old computer.
2. Choose the item *"Help|Update license…"* in the main menu (see ***Figure 6***). After that, the license dialog is opened.
3. Select the button `Move/Deactivate license…` (see ***Figure 7***) and send the license code to PROVEtech Support.
4. After the installation of PROVEtech:TA on the new computer please request a license as described above. Select "License moved to new computer" as the reason for the license request. Enter the name of the old computer.
5. Based on point 3. the PROVEtech Support can now grant you the move of this license and send you the License.key file for the new computer.



The license file “License.key” is bound to a specific computer. Therefore, the license information you send to PROVEtech Support contains computer-specific data. PROVEtech:TA uses this data for license encryption and decryption. The computer-specific data is not public and, except of arrangements with the user or customer, this data must not be passed to third parties under any circumstances.



In the Info dialog you can see the information about your PROVEtech:TA license, e.g. the expiry date or whether it is bound to a specific version (see ***3.7.2 The Info dialog***). If a license is version-bound, all service packs for this version are still included.



Choose the item *“Help|Update license…”* in the main menu (see ***Figure 6***) to request additional modules from PROVEtech Support (see chapter ***3.7 The Help menu***).



If you reinstall PROVEtech:TA, then you can use the old license file “License.key” again. This is also possible when you copy your TA\bin directory within your computer. If you reinstall your Windows Operating System, then you must request a new license file from PROVEtech Support of Akkodis.



If you have installed several different PROVEtech products, a common license file “License.key” can activate them at once.

PROVEtech:TA knows two basic license types:

- Test system license: This license type contains HiL hardware access, so it is possible to access the model running on a real-time computer, fault simulation hardware and diagnostics hardware.
- Desk license: With this license type it is not possible to access HiL hardware. Access to the model of the real-time computer, fault simulation hardware and diagnostic hardware is impossible. This license type supports creating test programs and workpages on your desk, later you use them on your test system including HiL hardware access. Furthermore, with desk license you can analyze your test results “offline” without wasting valuable time at the test system.

As an alternative to computer-based licensing via the License.key file, PROVEtech:TA can also be activated via floating licensing. (see chapter ***2.2.2 Floating licensing***)

There are no hardware dongles (USB stick) supported. Company licenses are possible, but even then, you must request a license file "License.key" for each single computer separately.

The file “License.key” can provide you the possibility to run PROVEtech:TA on a single computer multiple times by different Windows users (**Multi Instance License**). You may need this if you have PROVEtech:TA installed on a Windows server and you want to work via Windows Remote Desktop Services (Windows Terminal Services/Server). Please contact PROVEtech Support (provetech@akkodis.com) for further information.

Attention: Please also clarify the license terms of third-party software that is integrated into PROVEtech:TA but is not a product component of PROVEtech:TA (e.g. diagnostic software).

Attention: In case you run several PROVEtech:TA instances in parallel, please make sure that the path to the temporary directory is reset to the default value. Otherwise you may get some strange side-effects. By default, this path points into the Windows user profile.

2.2.2 Floating licensing

As an alternative to computer-bound licensing via the License.key file (see ***2.2.1 Computer-bound licensing***), PROVEtech:TA can also be activated via floating licensing.

Floating Licensing, also known as Concurrent Licensing or Network Licensing, is a software licensing approach in which a limited number of licenses for a software application, in this case PROVEtech:TA, is shared among a larger number of users over a longer period of time. When an authorized user wants to run PROVEtech:TA, they request a license from a central license server. If a license is available, the license server allows PROVEtech:TA to run. When they finish using PROVEtech:TA or the allowed license period has expired, the license is re-claimed by the license server and made available to other authorized users.

The license server is usually located in the customer's network and is set up and maintained by their IT. PROVEtech:TA licenses are installed via a "license certificate" (a signed file containing the license information). This file is available from PROVEtech support and contains for each module the maximum number of licenses that can be issued simultaneously to the different PROVEtech:TA clients. Therefore, if a PROVEtech:TA client requests or returns a license, the corresponding computer must be connected to the license server via network.

Floating licensing in PROVEtech:TA

For the floating license model, the third-party component FlexNet is integrated in PROVEtech:TA. The license server must be installed on a server computer by the customer's IT department. The license server contains a so-called Vendor Daemon, which makes license management available specifically for PROVEtech:TA. Administrative tools are also available to manage and monitor the license server.

The individual PROVEtech:TA modules such as Test Manager or Diagnostics, which can be activated via the license server, are also called "Features" in FlexNet lingo. PROVEtech:TA distinguishes between modules with or without hardware access (test system licenses or desk licenses). In addition to the maximum number of concurrent licenses, a PROVEtech:TA version number may be set as a limitation for each feature. Furthermore, the license certificate contains a global time limitation.

The administration of the used license features is always done by assigning them to the combination of the username and the computer name. This means in particular that in the case of multi-user operation on a computer, the licenses are always marked as 'in use' for each user. This ensures that the number of license features used always corresponds to the actual circumstances.

Configuring the path to the FlexNet license server

Please enter the path or URL to the FlexNet license server manually in the License.ini file under the `FNP_LicenseServer` key. If the license server cannot be found when PROVEtech:TA is started, FlexNet tries alternative path specifications via the Windows Registry. See also the ***Administrator's Guide*** regarding the License.ini file.



In addition to PROVEtech:TA, PROVEtech:RE can also be activated via network license.



In the Info dialog you can see information about your PROVEtech:TA license (see ***3.7.2 The Info dialog***).



If the complete Windows operating system is reinstalled, the network license will continue to work, unlike the computer-bound License.key file.

2.2.2.1 License Borrowing

License borrowing is an advanced variant of floating licenses that allows a user to run software on a computer that is not permanently connected to the license server via the network.
During a license borrowing request, the computer must be connected to the license server via the network. After the license is borrowed, the user can disconnect the computer from the network and continue to use the software for the duration of the borrowing period (which is usually set by the software vendor). During the entire borrowing period, the borrowed licenses are removed from the pool of available licenses on the license server.
After the borrowing period has expired, the licenses are then checked back into the pool and the software can no longer be executed. This happens regardless of whether the computer is connected to the network at that time or not. The license server organizes the withdrawal of the borrowed licenses without the intervention of the computers concerned.

Borrowing license features is done by a command line call from PROVEtech:TA. The call parameter '-b' is available for this purpose. The borrowing parameter follows this syntax:

```
-b[:[<ExpirationDate;FeatureNameA;FeatureNameB=3;...>][install_service][unin-
stall_service][#<SilentOutputFilePath>]]
```

There are four possible operations of the -b parameter:

1. Borrow license features
2. Give back license features
3. Installing the Licensing Service
4. Uninstalling the Licensing Service

Please note that no other instance of PROVEtech:TA must be running to execute the borrowing parameter via the command line. After execution of the borrowing parameter PROVEtech:TA is not started, only the borrowing operation is executed and PROVEtech:TA closes automatically afterwards. Furthermore, on each computer that is to use license borrowing, the so-called Licensing Service must be installed once. For this one-time installation administrator rights are required.

1. Borrow license features

```
-b:<ExpirationDate;FeatureNameA;FeatureNameB=3;...>
```

This parameter defines those license features that should be borrowed from the license server for offline use until a certain expiration date. To perform this operation, a connection to the license server must be available on the computer.

(ExpirationDate):
The expiration date must be specified according to ISO 8601 in the form 'yyyymmdd' or 'yyyymmddThhmmss' (e.g.: 20190916, or 20190916T174500). The 'T' is an obligatory separator when using the time in addition; seconds will not be considered. The date/time is interpreted b<y the license server as current local time.
If no time is specified, the time 23:59 of the specified day is automatically used.

(FeatureName):
Then follow (each separated by a semicolon ';') the names of the features to be borrowed. Here is the list of all available feature names of the PROVEtech ToolSuite (see also the ***Administrator's Guide***):

PTA_Base	PTA_Base_DESK
PTA_Runtime	PTA_Runtime_DESK
PTA_TestManager	PTA_TestManager_DESK
PTA_RealtimeAutomation	PTA_RealtimeAutomation_DESK
PTA_RemoteSequencer	PTA_RemoteSequencer_DESK
PTA_DiagnosticsMCD3D	PTA_DiagnosticsMCD3D_DESK
PTA_FaultSimulation	PTA_FaultSimulation_DESK
PTA_XCP	PTA_XCP_DESK
PTA_CCP	PTA_CCP_DESK
PTA_OSLCplus	PTA_OSLCplus_DESK
PTA_Plugins	PTA_Plugins_DESK
PTA_MATELOplus	PTA_MATELOplus_DESK
PTA_CodeEncryption0xHHHH	

PTA_CodeEncryptionExec0xHHHH

PRE_Base
PRE_CAN
PRE_LIN
PRE_MOST
PRE_MultiIO
PRE_SimulinkModel
PRE_FlexRay
PRE_GenericEthernet
PRE_GenericSOMEIP
PRE_AutomotiveEthernet
PRE_FPGA
PRE_Bluetooth
PRE_WIFI

The number of borrowed features can be defined by adding an '=' followed by a number directly after the FeatureName. This is currently only used for the feature 'PRE_Bluetooth'.

Note: Of course, only features that are currently available for License floating can be borrowed. Features that are not available or no longer in stock cannot be borrowed.

During the execution of the borrowing operation, all features already borrowed are given back to the license server before the new features are borrowed. This also means that a cumulative call of the borrowing operation is not possible. Example: If a required feature was forgotten to be borrowed, all required features must be borrowed again (including the previously forgotten one).

After executing this operation, a message box appears displaying the number of features borrowed from the license server and the number of feature names returned to the license server (or, if `<SilentOutputFile-Path>` is defined, the message box is suppressed and the information is written to the specified file).

Examples:

```
> TA.exe -b:20210819T210000;PTA_Base;PTA_TestManager;PTA_XCP;PTA_CCP
```

```
> TA.exe -
b:20210819T210000;PTA_Base;PTA_TestManager#C:\Temp\SilentOutputFile.txt
```

2. Give back license features

If no identifier is specified after the -b parameter, all license features already borrowed are given back to the license server. To perform this operation, there must be a connection to the license server on the computer.

After executing this operation, a message box appears showing the number of feature names given back to the license server (or outputting this information to the specified `<SilentOutputFilePath>` file).

Example:

```
> TA.exe -b
```

3. Installing the Licensing Service

As mentioned above, the FlexNet Licensing Service must be installed on the computer to use license borrowing. The installation on the computer has to be done only once. The installed service remains on the computer and is automatically started with every Windows restart.

You can install this service with the 'install_service' parameter.

The computer must have administrator privileges (at least to perform this operation) to install this service. A network connection to the license server is not necessary for this operation.

Example:

```
> TA.exe -b:install_service
```

4. Uninstalling the Licensing Service

If the FlexNet Licensing Service needs to be uninstalled from the computer (which is usually not required), the 'uninstall_service' parameter can be used.

The computer must have administrator rights to uninstall this service. A network connection to the license server is not necessary for this operation.

Example:

```
> TA.exe -b:uninstall_service
```



When setting up the license borrowing function on a computer for the first time, the FlexNet Licensing Service must be installed. Administrator privileges are required for this.

2.2.2.2 The license information dialog

If the key `FNP_LicenseServer` is set in the License.ini file, the menu item *"Help|Update license..."* in the main menu changes to *"Help|License information"*. There the user can open a dialog that shows detailed information about the current license floating and license borrowing.

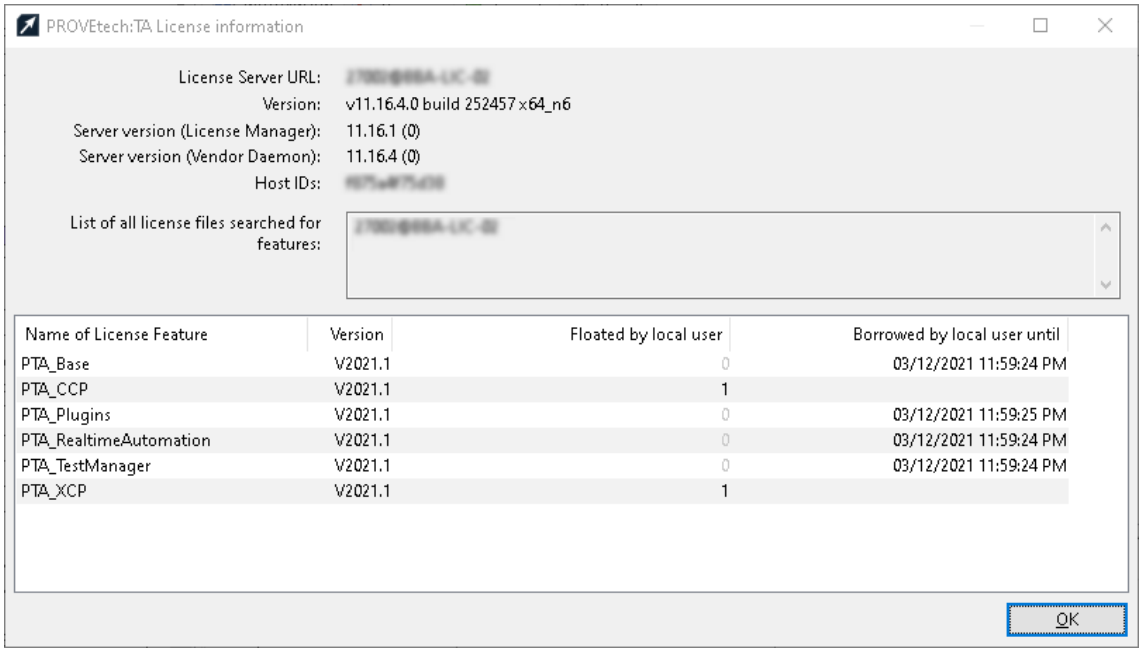


Figure 8 – License information dialog

If this dialog is opened by a PROVEtech:TA non-administrator user, license information concerning only the user itself are displayed. In the upper area, general information can be viewed, such as the URL and various version information of the license server, the local host ID, and license certificates used.
In the lower area, the user sees a list of all license features used by him. A distinction is made between those features that are bound as floating licenses and those that are borrowed.

When this dialog is opened by a PROVEtech:TA administrator user, much more information is displayed in the lower area. Then all license features available on the license server and their current usage by all existing PROVEtech ToolSuite users can be viewed.

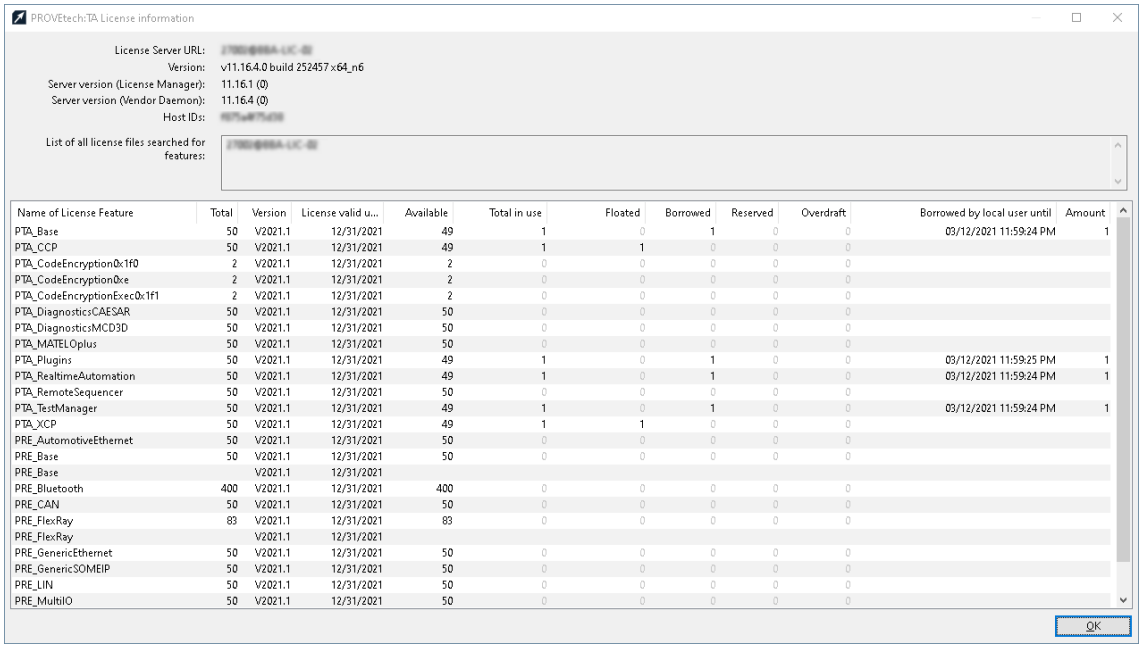


Figure 9 – License information dialog for administrator users

2.3 Login

After you activate the PROVEtech:TA license and start PROVEtech:TA via the Windows desktop or the start menu, the following login dialog is displayed in simple view or advanced view.

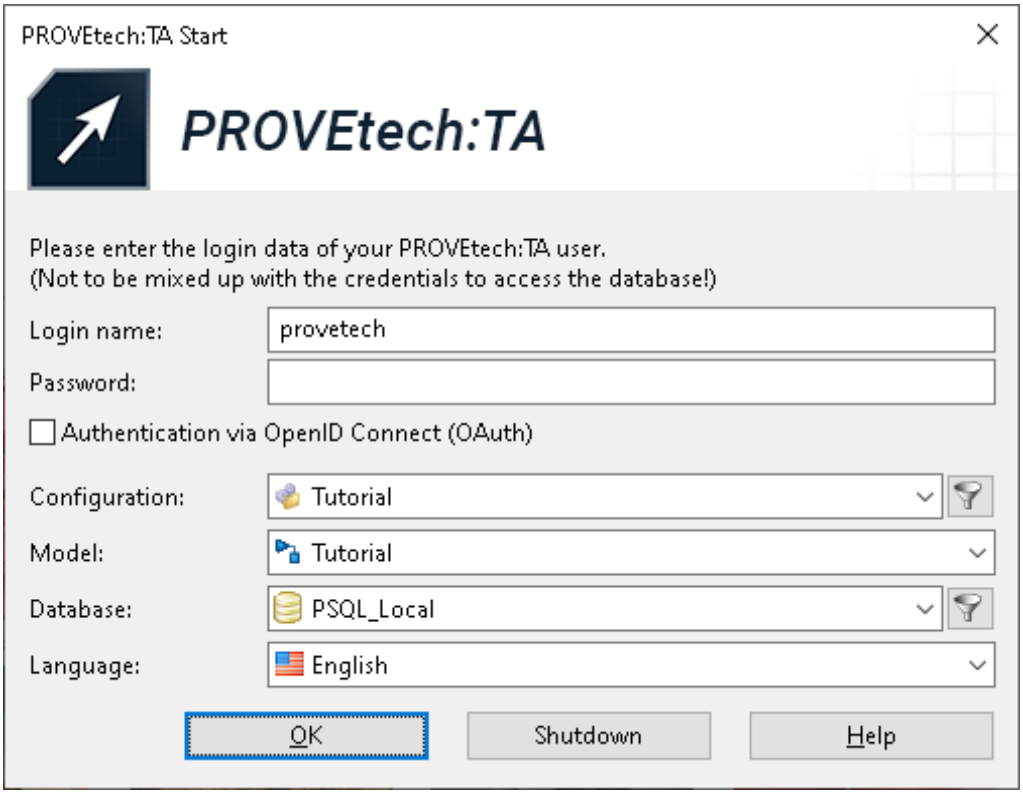


Figure 10 – The login dialog

Enter your login name and password. Both can be obtained from your PROVEtech:TA administrator. After the PROVEtech:TA installation, the PROVEtech:TA user 'provetech' has the empty password, which must then be changed. The username is case-insensitive, but not the password. If there are problems logging in with your PROVEtech:TA user, please refer to chapter ***8.3.2 Logging in to the Test Manager does not work***.

In PROVEtech:TA there are three login options for each user, which can be configured in the user dialog: PROVEtech:TA user with encrypted password stored in the PROVEtech:TA database, PROVEtech:TA user with Windows password and login via OpenID Connect (OAuth), OIDC for short. PROVEtech:TA also supports login via two-factor authentication. As a second factor, you can activate TOTP (Time-based One-time Password Algorithm), which is also stored in encrypted form in the PROVEtech:TA database. Please read also in chapter ***5.3.8.1 The user***.

When `authenticating via OpenID Connect (OAuth)`, the login name and password field are disabled, as these are only entered in the subsequent OpenID Connect dialog. The OpenID Connect dialog is a company-specific Microsoft dialog and can contain several authentication factors. The PROVEtech:TA administrator must first configure the connection data for the login procedure via OpenID Connect. These are saved in the database. The selected database connection is therefore also important for

this. Please read also in chapter ***5.3.11.2 The tab "Database settings" in the property dialog***. The login via OpenID Connect must be configured for the user both in the operating system and for the PROVEtech:TA users (see chapter ***5.3.8.1 The user***).

By default, the last chosen configuration, model, database and language is selected. You can find more information about configurations in chapter ***2.4 Configurations of the test system***, ***3.2.7.2 Configurations*** and ***3.2.7.6 Database***. Use the combo box `Language` to select the PROVEtech:TA display language for that user. Then click on `OK`.



If there is already a PROVEtech:TA session running when starting PROVEtech:TA then you switch automatically to this running session. To be able to start PROVEtech:TA multiple times on your computer, you must have a multi-instance license and use the /multi command line option (see chapter ***2.7 Command line parameters***).



Only the PROVEtech:TA administrator or a user with Manage access rights can set up new users in PROVEtech:TA. Read more about this in chapter ***5.3.8.1 The user*** and chapter ***5.2.6 Access rights***.

The terms 'PROVEtech:TA user', 'database user' and 'Windows user' refer to completely unrelated users. Much in the same way, the PROVEtech:TA administrator is in no way connected to the Windows administrator or the database administrator.



Exception: You can configure PROVEtech:TA users to take the Windows user password for PROVEtech:TA log in. Then PROVEtech:TA uses the Windows API function *LogonUser* with the login name, the configured domain and the parameters LOGON32_PROVIDER_DEFAULT and LOGON32_LOGON_NETWORK to check the password. For more details, please refer to the Microsoft documentation (MSDN), or chapter ***8.3.2 Logging in to the Test Manager does not work***.



The password of the PROVEtech:TA user, his secret TOTP key and the password of the database user are stored in encrypted form in accordance with current standards and transmitted in encrypted form between the DLLs and interfaces (SHA-512, AES-256, see as well in chapter ***8.3.8 Data security and avoiding data loss in the Test Manager***).

PROVEtech:TA also supports login via two-factor authentication, also known as 2FA, multi-factor authentication or MFA. As a second factor, you can activate TOTP (Time-based One-time Password Algorithm). Please read also in chapter ***5.3.8.1 The user***.

For cyber security reasons, failed login attempts are logged at the root object when logging is switched on (see in chapter ***5.3.9.7 The tab "History"***).

If a combo box contains a lot of entries, then the filter button appears on the right side.

After clicking the filter button, the filter dialog appears. Now you can find the correct entry per substring or regular expression (see ***5.18.4 Regular expressions***).

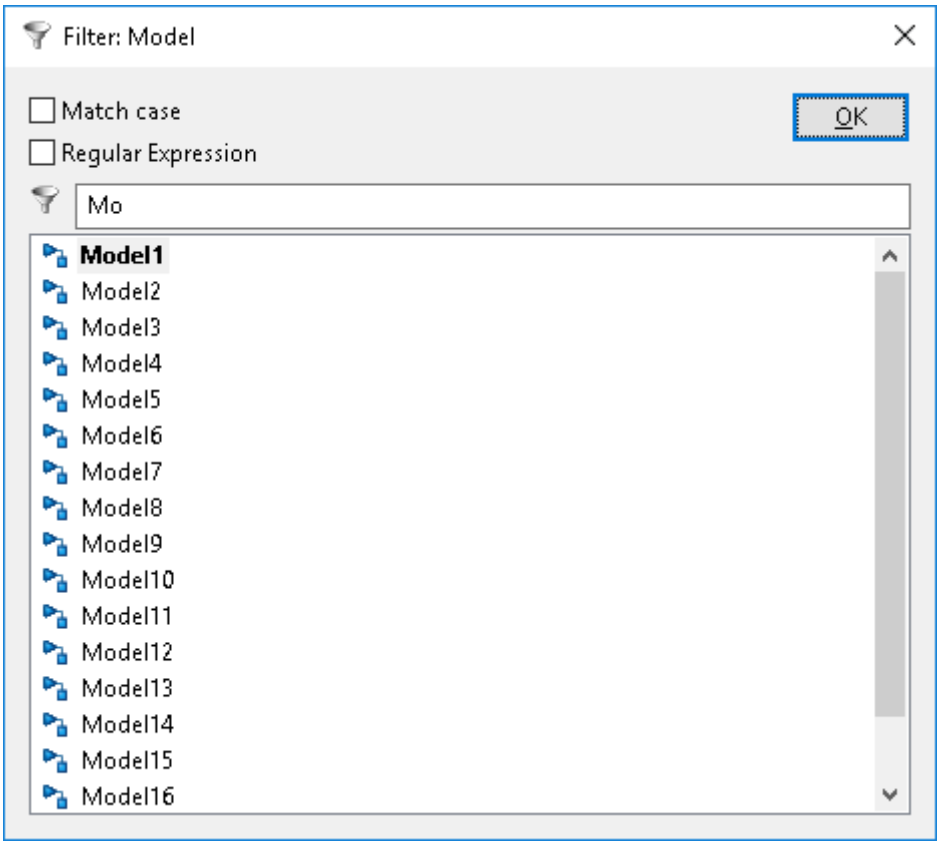


Figure 11 – The filter dialog



In the list field of the combo box as well as in the Filter dialog the last chosen entry is marked in bold font. E.g. in case of the model this could be a hint which model is actually running (see chapter ***3.2.7.2.3 Models***).



You can also find the filter button and the filter dialog next to many other combo boxes in PROVEtech:TA.

The following list offers a rough overview of the actions that are performed when PROVEtech:TA is started:

1. Read command line
2. Read Base.ini
3. Read TA.ini
4. Check whether PROVEtech:TA is already running and switch to this instance if necessary
5. Check whether PROVEtech:TA should be updated (UpdateVersion)
6. Display start dialog with progress bar
7. Open PROVEtech:TA Log
8. Execute start file, if specified in TA.ini
9. Check if all DLLs are there which are loaded dynamically (e.g. diagnostics and fault simulation)
10. Read Cfg.ini, User.ini and TM.ini
11. Display login dialog and wait for user confirmation ; if the Test Manager is deactivated (via TA.ini or because of licensing or error message), the fields `Password` and `Database` are grayed out in the login dialog and steps 12. to 14. are skipped
12. Connect to database (database connection was selected in the login dialog); Connection data from TM.ini as ODBC connection string to the database (database server computer, database name, database port, database user, database password (database password is not necessary in case of Windows authentication, see as well in chapter ***3.2.7.6.4 Windows Authentication***)
13. PROVEtech:TA checks whether the database structure is compatible with the PROVEtech:TA version. If an update of the database schema is required, corresponding message boxes are displayed.

14. PROVEtech:TA user login; the PROVEtech:TA user and its password have been specified in the login dialog; these are sent encrypted to the database; in a database table the PROVEtech:TA user and the hash code of the password are checked;
    if the PROVEtech:TA user is marked as Windows user, the password is checked directly in the Windows operating system by calling the Windows API function *LogonUser* (see further up in the chapter);
    if the login happens via OpenID Connect (OAuth), the user is initially an empty string and is only determined via the OpenID Connect dialog; if the OpenID Connect login is successful, PROVEtech:TA checks whether the user is in fact registered in the PROVEtech:TA database as an OpenID Connect user;
    if two-factor authentication is activated, the 6-digit PIN is then checked (see chapter ***5.3.8.1 The user***)
15. Language is set (depending on the user from the User.ini)
16. Set configuration (the configuration was specified in the login dialog)
17. Load model (the model was specified in the login dialog)
18. Initializing WinWrap Basic
19. Create main window and main tabs
20. Load cockpit and workpage
21. Load and display Test Manager tree
22. Load and display diagnostics
23. Load and display fault simulation
24. Make main window visible and position it
25. Execute various start macros if specified in cfg.ini

2.4 Configurations of the test system

In configuring the test system, you are selecting those ECUs and models which you need for the desired test. For example, if you wish to simulate a car with shift transmission, then the model for automatic transmission and the relevant ECU need not be included in the simulation environment.

A test system configuration contains the following components:

- ECUs which are included in the configuration environment

- Computer models of the other vehicle components, e.g. models of the engine, the electrical system or the air-conditioning
- The screen configuration (the appearance of the workpage). A workpage configuration consists of various control and display elements for signals (see chapter ***4 Workpage***).

You can structure your configurations hierarchically too, and separate them in configuration groups.



You can manage your configurations via the page `Configurations` of the PROVEtech:TA settings dialog (see chapter ***3.2.7.2 Configurations***).

2.5 Overview

The following PROVEtech:TA user interface is displayed after start-up:

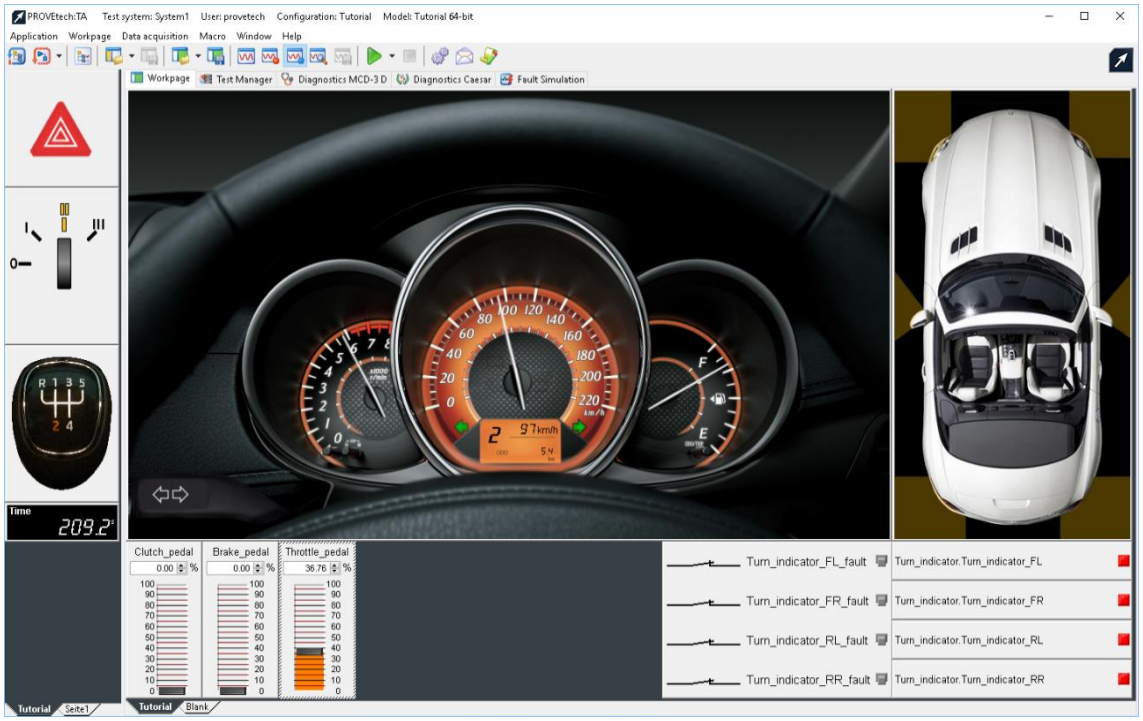


Figure 12 – The user interface

The title bar shows information about the program, the version number, the 32-bit or 64-bit variant (see chapter ***2.10 32-bit and 64-bit***), the test system name, the user, the selected configuration and the selected model:



Figure 13 – The header

The menu bar and below that the toolbar (see chapter ***3 The menu bar and toolbar***) remain visible on the interface at all times. Use them to select central PROVEtech:TA functions.

The left area below the toolbar with the gear shift is called cockpit. You can place controls on the cockpit, which are visible all the time. Right of the cockpit there are tabs to select the current functional area. The next sub chapters describe these functional areas. In the upper image, the workpage functional area is selected. Cockpit and workpage belong to the same module.

You can lock the whole user interface while running tests and data acquisitions to prevent other computer users from interfering. See ***3.2.8 Lock User Interface*** for more details.

2.5.1 Workpage

The workpage is used to display and stimulate signals. You may select signals from the complete signal pool and assign them values (see ***4.2 The Signal Selection window***). A signal can be displayed as different element types, such as slide controls or buttons). The temporal progression of signals is shown by means of strip chart controls. Additionally, you can display message traces. The complete workpage can be conveniently operated using drag and drop, copy and paste or via the context menu with the mouse. Numeric values can also be entered via the keyboard. The chapter ***4 Workpage*** contains all the information on this topic.

The cockpit area on the left side has the same features as the workpage, but in contrast to the workpage it always stays visible. You may want to display the special elements there, such as the gearshift element and the gauge cluster.

2.5.2 Test Manager

With this function area you can automate the execution of tests. Tests are programmed in the WinWrap Basic language (a VBA dialect), which has been expanded to include commands specific to hardware-in-the-loop. This test language is used to control the entire system. All manual interventions can be automated. You can combine several tests (elementary tests) to create system tests and thus define complex test sequences. The

test results are created and stored in a database automatically. For further information refer to the chapter ***5 Test Manager***.

2.5.3 Diagnostics

The ECUs check the signals at their inputs in a plausibility test. If this test gives a negative result or an error occurs as the signals are being processed in the ECU, then an error code is noted in the internal error memory (On-Board Diagnostics). Upon occurrence of the error, the environment data is also recorded. The Diagnostics function area can be used to read the error memory and subsequently clear it. You can additionally execute the respective diagnostic services and the coding specific to the ECUs or flash the ECU with new firmware. The chapter ***6 Diagnostics*** describes this topic.

2.5.4 Fault simulation

Here you have the possibility of simulating errors in the cabling of the vehicle's electric system and the connected components such as sensors and actuators. You can simulate the error on individual ECUs or also globally on several ECUs. It is also possible to build in interruptions in the bus system of the vehicle. You can select from numerous error classes and set different errors and error combinations for each ECU. For further information refer to the chapter ***7 Fault simulation***.

2.6 Placing the functional areas

To show several functional areas simultaneously you can drag and drop a tab out of the tab control and place it as a separate window.

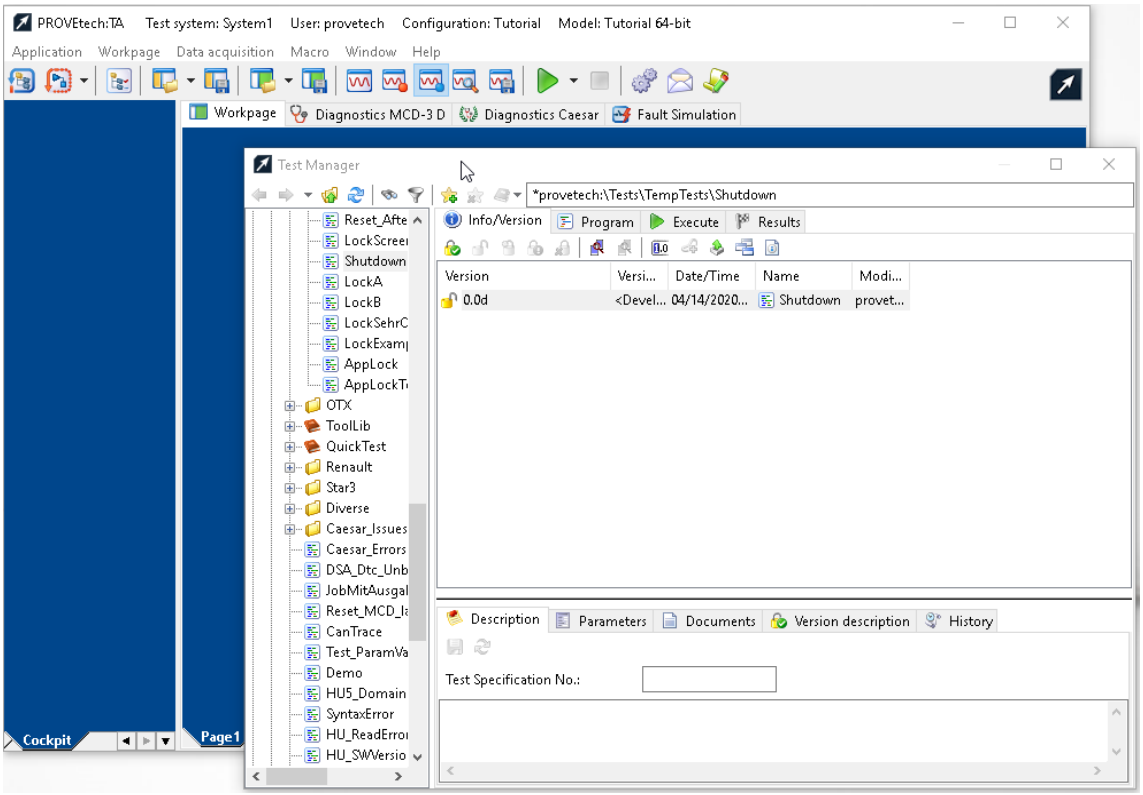


Figure 14 – Drag a functional area from the tab control

If you close this separate functional area then the window docks back as a property page of the tab control.

The only exception is the Workpage. The main Workpage is always docked to the main window but a separate Workpage can be opened.

You can dock back all functional areas by choosing “Dock all” in the context menu of the tab control (see ***Figure 15***). See also ***3.6 The Window menu***.

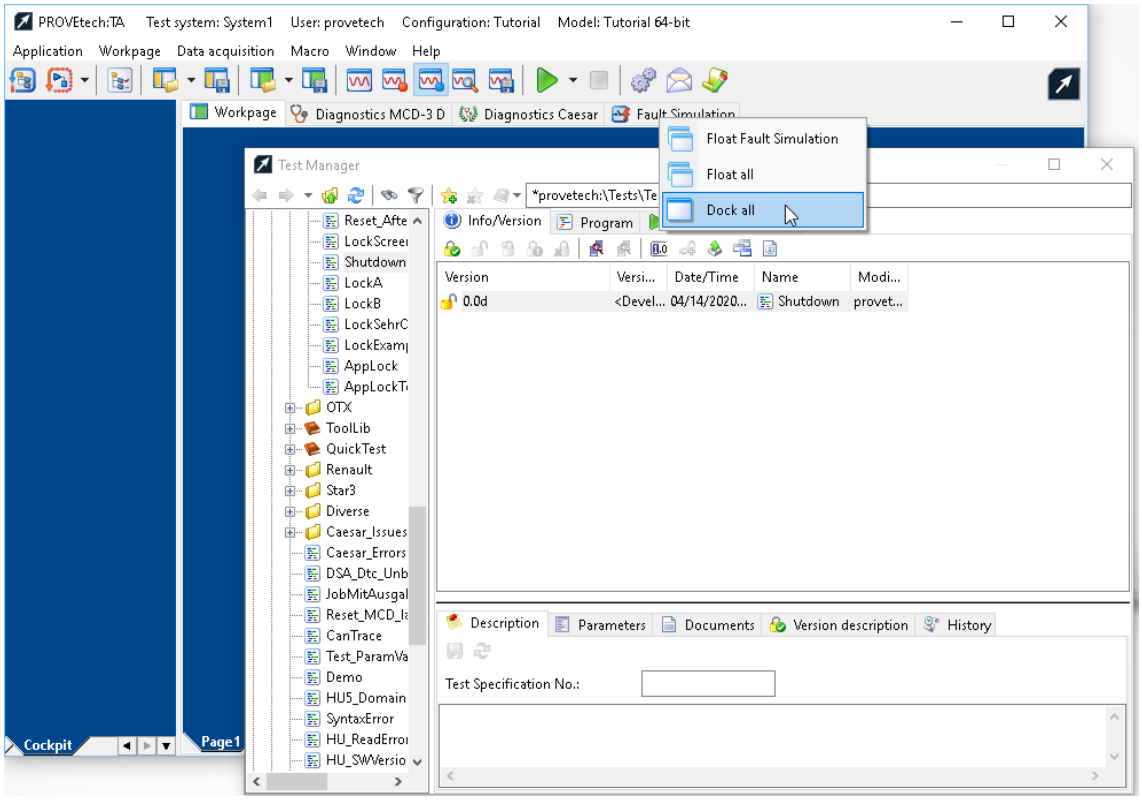


Figure 15 – Context menu "Dock all"

2.7 Command line parameters

You can start PROVEtech:TA via the command line, e.g. to automate the PROVEtech:TA login. To do this use the following syntax and parameters:

```
TA.exe [/u:<User>] [/p:<Password>] [/totp:<HexSha256>] [/oidc:<token>]
[/c:<Configuration>] [/m:<Model>] [/mnr] [/d:<Database>] [<DataAcqui-
sitionFile>] [/a:<TAIni>] [/Automation] [/e:<TestForExecution>] [/Em-
bedding] [/HideSplashScreen] [/gRPC:<Settings>] [/l] [/s] [TestAutoma-
tion:Path=<Path>&Action=<Action>[&Line=<Line>]] [/v:<VersionUpdate-
Path>] [/x] [/b[:[<Date;FeatureA;FeatureB=N;…>][install_service][unin-
stall_service]][#<SilentOutputFilePath>]] [/?]
```

`/u:<User>`	Optional PROVEtech:TA user; otherwise login dialog
`/p:<Password>`	Optional login password; otherwise empty password (Attention: The password must be given unencrypted, which is a certain security risk)

`/totp:<HexSha256>`	Optional TOTP code in case of using two-factor-authentication. We do not recommend passing it via command line for security reasons. See chapter ***2.7.1 Calculation of the TOTP command line parameter*** for the calculation of `<HexSha256>`.
`/oidc:<token>`	Optional token for automated login via OpenID Connect (OAuth); quotation marks must be protected; please also specify database connection. See chapter ***5.3.11.2 The tab "Database settings" in the property dialog***
`/c:<Configuration>`	Optional name of configuration; otherwise configuration of the last session
`/m:<Model>`	Optional name of the model; otherwise model of the last session; the model is always reloaded (exception: /m: with empty string skips the model loading)
`/mnr`	Optional, avoids reloading the model if the model is already loaded
`/d:<Database>`	Optional name of the database connection; otherwise last database connection
`<DataAcquisitionFile>`	Optional path to a data acquisition file (.hdr file) that is shown in the offline mode after PROVEtech:TA is started
`/a:<TAIni>`	Optional path to TA.ini; otherwise the Base.ini defines the base path to TA.ini
`/Automation`	Can be set explicitly or is set automatically if PROVEtech:TA was started as COM server; then no login dialog appears (see chapter ***2.3 Login***), but PROVEtech:TA waits for the call of `Application.Init` (see chapter ***Access from foreign software via COM*** in the Test Language section)
`/e:<TestForExecution>`	Optional path of a test that will be executed after starting PROVEtech:TA. If omitted, no test will be executed. A path starting with '*' is interpreted as a path to a test object in the Test Manager tree; otherwise it is a system file path to a test macro; this corresponds to the option `TestAutomation` with `Action=Execute`

`/Embedding`	Same as `/Automation`

`/gRPC:<Settings>`	Optional gRPC settings additional to the ones in the TM.ini file. You can pass this command line parameter several times for multiple gRPC settings. Each is like an own gRPC section in TM.ini. The `<Settings>` string consists of key/value pairs identical to the TM.ini key/value pairs. Key and value are separated by the = character and the pairs are separated by the | character. For more information about the keys and values please have a look in the ***Administrator's Guide*** describing TM.ini. Full example (all in one line):
	`/gRPC:Listen=127.0.0.1:9876`
	`|PrivateKey1=C:\Certs\server_priv.pem`
	`|CertChain1=C:\Certs\server_pub.pem`
	`|RootCerts=C:\Certs\client_pub.pem`
	`|ClientCertificateRequestType`
	`=GRPC_SSL_REQUEST_AND_REQUIRE_CLIENT_CERTIFICATE_AND_VERIFY`

`/HideSplashScreen`	The startup screen is not displayed when starting and exiting this application; the parameter is only valid in combination with `/Automation` or `/Embedding`
`/l`	Displays the license dialog (including the Move/Deactivate button)
`/multi`	In case you have an according license, you can start several independent PROVEtech:TA instances with that. As an example, each instance can handle an independent device under test. The PROVEtech:TA title bar shows the instance number.
`/s`	Switch to Remote Test Sequencer mode after starting PROVEtech:TA

`TestAutomation:`
`Path=<Path>&`
`Action=<Action>`
`[&Line=<Line>]`
	Optional Test Manager path or file path (e.g. .hdr data acquisition file) together with an action ("Select", "Execute", "Open"); in case of "Select" and a WinWrap Basic macro you can set a line number; the order of path, action and line does not matter; special characters in Test Manager paths can be escaped by %25 for the % character and %26 for the & character; see chapter ***5.15 Access from foreign software via URI***

`/v:<VersionUpdate-Path>`	Optional path to update your PROVEtech:TA version automatically
`/x`	Suppresses the message box for updating PROVEtech:TA and keeps the current version
`/b[:[<Date;FeatureA; FeatureB=N;...>] [install_service] [uninstall_service]] [#<SilentOutput-FilePath>]`	Borrows several license features from the license server for offline usage until a given expiration date. (Date): The expiration date must be specified according to ISO 8601 in the form 'YYYYMMDD' or 'YYYYMMDD**T**HHMMSS'. (Feature): Then follow, separated by semicolons (';'), the feature names that are to be borrowed. The amount of borrowed features can be defined by adding a '=' followed by a number N. If no identifier is specified after the /b parameter, all borrowed license features are given back to the license server. For redirecting the message box output into a file, use the hash character (#), directly followed by a complete and absolute file path. Whitespace characters are not supported. If the defined file cannot be generated, the corresponding error message is still displayed in a message box. When this parameter is used, TA will automatically terminate after the borrowing operation. Refer to chapter ***2.2.2.1 License Borrowing*** for available license feature names and additional information of the /b parameter.
`/?`	Optional help information



When opening a data acquisition file (.hdr file) PROVEtech:TA starts automatically by command line and displays the data acquisition in the offline mode (see chapter ***4.5 The offline mode***).



Do not forget to protect special characters and whitespaces in paths by using double quotation marks.

If PROVEtech:TA is started (also by command line), although another PROVEtech:TA instance is already running, PROVEtech:TA is not started a second time, but it is switched to the running instance. Command line parameters are processed, or an error is displayed via message box, if for example the configuration does not match.

You can also use the "Windows Task Scheduler" to run a test in PROVEtech:TA at a specific time, or when Windows is idle.

Regarding the interaction between PROVEtech:TA and external software, please refer to the chapter ***5.15 Access from foreign software via URI*** and chapter ***5.6.3.4 Test Automation references***, as well as the chapter ***Accessing foreign software*** and ***Access from foreign software*** in the book **Test Language** which is part of this document.

2.7.1 Calculation of the TOTP command line parameter

For an automated PROVEtech:TA login via command line while two-factor-authentication is active, it is necessary to pass the 6 digit TOTP code (see chapter ***2.3 Login***). We explicitly recommend not to do this. However, customers want to circumvent this recommendation. A security mechanism prevents at least copying the code to another computer and reading the code in plain text – even if the code is only valid for a few minutes. This is why passing it works as a hash value, which contains the serial number of the system drive beside the TOTP code. To be more precise: The /totp: command line parameter passes a hexadecimal string of a SHA256 hash value (64 characters). The calculation of the SHA256 value uses 14 bytes. The first 8 bytes are a hexadecimal string (upper case ASCII) of the system drive serial number followed by the 6 byte TOTP code (ASCII).

Here is the calculation as Visual Basic .NET function (transferable to C#):

```
Imports System
Imports System.Security.Cryptography

Declare Unicode Function GetVolumeInformation Lib "Kernel32.dll" _
    Alias "GetVolumeInformationW" _
    (lpRootPathName As String, lpVolumeNameBuffer As IntPtr,
    nVolumeNameSize As UInteger, ByRef lpVolumeSerialNumber As UInteger,
    ByRef lpMaximumComponentLength As UInteger,
    ByRef lpFileSystemFlags As UInteger, lpFileSystemNameBuffer As IntPtr,
    nFileSystemNameSize As UInteger) As Integer

Function GetHash(uTOTPCode As UInteger) As String
    Dim aData() As Byte
```

```
    Dim strInput As String
    Dim uSerialNo As UInteger

    'get serial No. of Windows system drive
    If GetVolumeInformation("C:\", 0, 0, uSerialNo, 0, 0, 0, 0) = 0 Then
        Return "" 'error
    End If
    'create byte array from serial No. and uTOTPCode
    strInput = $"{uSerialNo:X8}{uTOTPCode:D6}"
    aData = Text.Encoding.ASCII.GetBytes(strInput)
    'calculate SHA256 of aData
    Using sha As SHA256 = SHA256.Create()
        aData = sha.ComputeHash(aData)
    End Using
    'return hexadecimal string
    Return BitConverter.ToString(aData).Replace("-", "")
End Function
```

The function `GetHash` gets the TOTP code as number (ignore leading 0s) and returns the hash value as string. The command line parameter looks this way as an example:

```
/totp:E74E7044BC2944CFCA086F244B26F5604F346B776D182C43B15664F8D3644BBB
```

This manual does not explain the TOTP code calculation. TOTP is a standard and you can read the according standard documents to find out how it works or use a 3rd party software to calculate it.



If there are still problems logging in and the dialog for the second factor still appears, you can enter the transferred 6-digit TOTP code directly in the dialog for problem analysis.

2.8 PROVEtech:TA Runtime version

In the Runtime version of PROVEtech:TA the user interface is not visible. This version is used, for example, to execute test scripts out of a self-programmed user interface or an external software application. The license mechanism controls whether PROVEtech:TA can only be started in the Runtime version. Because of the non-visible user interface and therefore the functional limitations, it is offered with an more attractive price.

For example, the following use cases are possible:

- Via command line parameters, PROVEtech:TA is started and a certain test script is executed. In this test script, a dialog window is programmed that controls the operations (see chapter ***2.7 Command line parameters***).

- PROVEtech:TA is started as a COM server in a background process. Test scripts are created, executed and test results are analyzed by using the PROVEtech:TA Test Language (see the manual for the ***Test Language***)

2.9 PROVEtech:TA DAQViewer

The PROVEtech:TA DAQViewer (Data Acquisition Viewer, formerly called PROVEtech:TA Light) is a free version of PROVEtech:TA. This version is limited to operations of the offline mode (see ***4.5 The offline mode***). You can use it as a data acquisition viewer and may freely copy it. When you start PROVEtech:TA DAQViewer, it will immediately and exclusively display the offline mode. There is no login dialog, no PROVEtech:TA user or special configuration. You also need no database installation. Data acquisitions are handled in your file system. Furthermore there are no Windows administrator rights necessary for installation and execution.

2.10 32-bit and 64-bit

Beginning with PROVEtech:TA 2024 only the 64-bit variant exists (32-bit is obsolete). It only runs on Windows 64-bit.

On Windows a 32-bit process cannot load 64-bit components (DLLs) and a 64-bit process cannot load 32-bit components. This means when switching from the 32-bit to the 64-bit variant of PROVEtech:TA, all used foreign components within the process must also be available as 64-bit components.

The following list helps you to identify issues:

- In case you are using Part J with CAESAR (see chapter ***6.1 CAESAR diagnostics***), you need a 64-bit Pass-Thru DLL (see chapter ***8.9.4 Nexiq Device / Part J / Pass-Thru***). For Part P (eCOM) the PROVEtech:TA setup contains all necessary files (but not the driver for the USB2LAN adapter).
- In case you use Simulink DLLs as models, they must be 64-bit (see ReadMe.txt for Simulink DLLs on PROVEtech:TA installation medium).
- For using dSPACE PHS bus systems the file “dsp32.dll” must be located in the “C:\Windows\System32” directory of your 64-bit Windows. If it is missing, you need to update your dSPACE installation to a newer version.
- For the Generic TCP platforms with platform abstraction DLLs as well as Seed&Key DLLs for CCP/XCP and the Opal-RT API DLL, PROVEtech:TA automatically launches a separate 32-bit or 64-bit process fitting to the DLL for the execution. This means that these DLLs cause no trouble.

- The MCD-3 D diagnostics server (see chapter ***6.2 MCD-3 D diagnostics***) runs in an own Windows process beginning with PROVEtech:TA 2025. For Softing DTS PROVEtech:TA supports a 32-bit process as well as a 64-bit process for that. Therefore, 32-bit is no problem anymore. For DSA PRODIS.MCD there is only the 64-bit variant as customers do not use the 32-bit variant anymore. DSA provides a 64-version beginning with PRODIS.MCD V3.6 and Softing beginning with DTS 8.13 COS.
- Normal Basic scripts using just the internal PROVEtech:TA commands are not affected by the 32-bit/64-bit differences. However, you should identify all foreign components used in your Basic scripts and check if the suppliers of these components also ship 64-bit versions. This is not necessary for components running in own processes (e.g. out of process COM servers like Microsoft Excel). It might be necessary to modify your Basic scripts to switch to a 64-bit component.
- In case of using `Declare` statements in your Basic scripts, you must also check if you use variant dependent types or function names. As an example, pointers and Windows handles depend on the used variant (32-bit or 64-bit). To make your *Declare* statement compatible to both variants, use the data type `PortInt` (in .NET mode: `System.IntPtr`). Example:

  ```
  Declare Function GetActiveWindow Lib "user32" () As PortInt
  ```

  In case a DLL exports functions with so-called “decorated names”, the export names include the parameter and return value types. 32-bit and 64-bit DLLs use different conversions for that. To solve this you can create a `Declare` line for both of them and select the right one at runtime via the `Win64` constant which is only `True` if the process is 64-bit. Example:

  ```
  Declare Function MyFunction32 Lib "MyLib.dll" _
      Alias "?MyFunction@@YGFXZ" () As Integer
  Declare Function MyFunction64 Lib "MyLib.dll" _
      Alias "?MyFunction@@YAFXZ" () As Integer

  'wrapper for 32/64-bit call
  Function MyFunction() As Integer
      If Win64 Then
          Return MyFunction64()
      Else
          Return MyFunction32()
      End If
  End Function
  ```

- .NET assemblies compiled with the option “Any CPU” can be used in 64-bit processes as well as 32-bit processes.

- In case you are using the Test Manager plug-in interface (embedded editor), your plug-in must also support 64-bit. If the plug-in is compiled as .NET assembly with the option “Any CPU”, it should work with both variants.

2.11 System requirements

PROVEtech:TA runs on 64-bit Microsoft Windows 10 and Windows 11. We test PROVEtech:TA with the most recent official version of Windows Professional available at that point of time when releasing PROVEtech:TA. We do not test Windows releases of the Microsoft “Insider Program” or Home, S, N or other “special” editions. We recommended to always install the latest security patches from Microsoft. Please note, that it is impossible to test with all existing Windows versions and patch levels. The chances for a dedicated PROVEtech:TA issue due to a Windows update are low though. It is more likely, that a Windows update creates a common issue not only affecting PROVEtech:TA, but other software as well. In the past Windows compatibility issues were very rare as long as the Windows major version number did not change. It can happen though, that some features require an up-to-date Windows version.

PROVEtech:TA is a 64-bit application for x86 processors. We do not test it on ARM processor versions of Windows.

The minimum supported CPU is an Intel Core i3-2100 or AMD FX-4100. For Windows 11 Microsoft defined the minimum supported processors on operating system level. In general, the CPU speed has a direct influence on the reactivity of PROVEtech:TA. Your test programs have the main influence on the CPU requirements. PROVEtech:TA also profits from more CPU cores. Four cores are typically sufficient, but if you execute several test programs in parallel, more cores might suit you. Please keep in mind that other software processes running on your computer might produce additional CPU load.

The graphic resolution should not be below 1280x768 pixels with 32-bit color depth and a screen scaling of 100 %. With a higher screen scaling you also need an according higher minimum resolution. The graphics card must support DirectX 11.1. When using the video recording/display feature, please use a graphics card with sufficient video encoding/decoding hardware acceleration. Use an up-to date graphics card driver from the graphics card manufacturer. Do not use Microsoft’s driver. See also chapter ***8.12.5 Screen scaling***.

Very often customers use PROVEtech:TA with several monitors. This works fine if you do not use different screen scaling factors for the monitors. Please select the same scaling factor for all monitors in Windows.

8 GB of RAM are sufficient for a normal operation. The RAM usage mainly depends on the data you load (e.g. model signals, diagnostic data, fault simulation data, test

protocols, etc.). In addition, executed test programs and other software processes can use a lot of RAM.

For the PROVEtech:TA installation itself 500 MB of hard disk space is sufficient. The amount of necessary hard disk space can increase significantly when using a local database for saving test programs and test results. If, for example, only a local PostgreSQL server is installed for demo purposes, then approximately 1500 MB of additional hard disk space should be provided. Additionally, your model data, diagnostic data, etc. might use a lot of hard disk space.

The hard disk speed might be important for handling large files like AUTOSAR-XML files or video files or acquiring many signals with a high sampling rate or many messages. We recommend using a fast SSD.

Please keep in mind that the used Hardware-in-the-Loop (HiL) technology has additional software and hardware requirements defined by the vendor.

In case of using a separate database server, there are additional requirements for that which depend very much on the working scenario.

The login via OpenID Connect (OAuth) needs an installation of Microsoft .NET 8.0 Desktop Runtime 8.0 (Minimum) and an up-to-date Microsoft Edge WebView2 on the computer. Beside the OpenID Connect (OAuth) login PROVEtech:TA uses Microsoft Edge WebView2 at several points to display HTML. We do not recommend using the fallback to Internet Explorer, which is discontinued by Microsoft.

System requirements for a database server

PROVEtech: TA supports the latest versions of PostgreSQL, MS SQL Server and Oracle at the time of publication. PostgreSQL is license-free, whereas the full versions of MS SQL Server and Oracle require a license fee. It is also possible to start with the free version of MS SQL Server Express or Oracle Express, but with restrictions (see on the Internet). The exact version and compatibility information regarding PROVEtech:TA can be found in the list of release notes at https://www.provetech.de/ta/history.

In the following list below, you can find a rough estimation for the system requirements (server and network) for a database server:

- To cover peaks, the data throughput to the server should be at least Fast Ethernet, which is 100Mbps. This means, for example, to store a PROVEtech:TA signal recording of compressed 100 MB into the database, you then need about 10 seconds.
- Very important is the ping time, which is related to factors such as firewall, virus scanner, router, the database server itself, the local distance, and other circumstances. Experience has shown that with a ping of 40ms to the server, you

can work reasonably smoothly with a PROVEtech:TA client. The corresponding ports must be enabled by the firewall.

- The data of the PROVEtech:TA database instance should be backed up daily (e.g. in daily incremental backups and weekly full backups) and archived at least once a year and then cleaned up by the PROVEtech:TA user. Storage space must be available on appropriate storage media for the server installation, the data of the database instance, backup and archiving (see below in data growth per year).

Here are two examples of hardware requirements regarding the database server for a small test system and for a large test system:

Example 1: Database is accessed by 3 HiL operating computers and 8 desktop computers, which mean 12 PROVEtech:TA clients in sum:

- A rough estimate results in a total of 500 MB per day average traffic, depending on user activity, time of day, weekend, vacation situation, etc.
- You can expect a total data growth of 50 GB per year.
- The database server with no other programs running should have a processor with at least 4 cores and 16 GB of RAM.

Example 2: Database is accessed by 30 HiL operating computers and 70 desktop computers, which mean 100 PROVEtech:TA clients in sum:

- A rough estimate results in a total of 5 GB per day average traffic, depending on user activity, time of day, weekend, vacation situation, etc.
- You can expect a total data growth of 400 GB per year.
- The database server with no other programs running should have a server processor with at least 8 cores and 64 GB of RAM.

PROVEtech:TA customers often use a central database server (SQL Server or Oracle) to share the data, as well as local PostgreSQL servers at each PROVEtech:TA installation to test things out.

Which database system should a customer choose?

If the customer is already using database systems from Oracle or Microsoft SQL Server, it is of advantage to set up an additional database instance on an existing server. This means that administration, database backup, maintenance and licensing are the responsibility of the customer's IT department and not the PROVEtech:TA user.

If it is a larger test system (5 to 10 PROVEtech:TA installations or more), the customer can consider whether to start with an Express version first and later switch to a

commercial Oracle Server or MS SQL Server. It is advisable to start with Oracle Express or SQL Express in order to keep the data migration as simple as possible later on.

If the above requirements do not apply or the customer wants to save costs, the license-free PostgreSQL Server is recommended. However, it should be regulated who is responsible for administration, backup and maintenance.

If worldwide use with a shared data is desired, database replication is an option. PROVEtech support recommends the use of MS SQL Server and the SymmetricDS replication software. You can also order the administration, monitoring and maintenance of database replication via the PROVEtech team.

The advantages and disadvantages of a central database server compared to several local database servers are listed below:

Advantages:

- Central data storage of Test Manager data
- Shared use of test libraries
- Multi-user operation on the test system
- Remote sequencer access possible
- Easier preparation of test scripts and post-processing of test results possible at the desk computer
- The test system is blocked less frequently as offline work at the desk computer is possible; this leads to better workload of the test system
- Better backup strategies possible
- Greater reliability thanks to better hardware on the server computers; however, if the database server fails, everyone is blocked
- Administration and maintenance via IT
- Data does not have to be synchronized between the different databases

Disadvantages:

- Database access could fail in the event of network problems or changes to firewall settings
- Physical distances or firewall settings can have a negative impact on database performance (read/write speed)
- The database connection must be configured for each PROVEtech:TA client
- Mutual overwriting of data in multi-user operation is possible with a shared database; however, this is intercepted in many places by PROVEtech:TA's internal lock mechanism
- Higher costs, as a central database server is subject to licensing; local database servers are often license-free

2.12 Working with floating-point numbers

Working with floating-point numbers is tricky. It is based on the IEEE 754 standard. Floating-point numbers use 32-bit or 64-bit processor registers where mantissa, exponent and sign share the available bits. However, they do not use the decimal system familiar to humans, but the binary system.

This results in complications mainly for the decimal places as very often a value can only be represented approximately. As an example, values like 0.5, 0.25, 0.125, etc. or sums of such values are no problem as they are correctly represented by 1/2, 1/4, 1/8, etc.. A value like 0.001 which is simple in the decimal system causes issues though, as for a 64-bit floating-point value it results in something like 0.0010000000000000002. Additionally, calculations with such values sum up the rounding errors.

To make things worse, there are different variants for rounding/approximating the numbers – depending on the settings. When converting a floating-point number into a string for the output in the decimal system, rounding is necessary very often. Microsoft changed the default rounding algorithm beginning with Windows 10 2004 to banker's rounding, used by the financial sector very often (e.g. rounds 2.5 to 2 and 3.5 to 4). However, this is not very common in technical sectors where people use PROVEtech:TA. There, rounding "away from zero" is more common (rounding of 2.5 to 3). Therefore, we changed the rounding behavior to that via our compiler settings beginning with PROVEtech:TA 2022 and do not use the default behavior of the operating system. However, this has no influence on 3rd party components like MCD-3 D servers which use own settings or the operating system default.

The combination of the upper two topics (no possibility to exactly represent decimal places and additionally rounding for the output) even makes it more complex. Example: We want to output the value 0.0011111111111115 with 16 decimal places. As the value on decimal place 17 is 5, you would expect an output of 0.0011111111111112. However, a 64-bit floating-point value cannot exactly represent 0.0011111111111115 and the approximated value is about 0,00111111111111114992. So, an output with 16 decimal places actually results in 0,0011111111111111.

You also need to be careful when comparing floating-point values. In many cases it is necessary to plan with inaccurate values. As an example, the following code line is dangerous, which compares a signal value:

```
If System.GetSignal("MySignal") = 0.001 Then …
```

Instead, a range-check is recommended, e.g.:

```
If Abs(System.GetSignal("MySignal") - 0.001) < 0.000001 Then …
```

This line checks if the difference from 0.001 is less than 0.000001. It considers that it is not possible to represent both values in an exact way.

IEEE 754 defines special values for + and – infinite and “Not a Number” (*NaN*). Comparisons with *NaN* are tricky. The standard defines it this way: All comparisons of values (any value, including + and – infinite and *NaN* itself) with *NaN* always result in *False*. Exception: The unequal operator which always results in *True*. Examples:
1 > *NaN* → *False*
1 < *NaN* → *False*
NaN = *NaN* → *False*
NaN <> *NaN* → *True*
1 <> *NaN* → *True*

This means, to check if a floating-point number is *NaN*, compare it with itself. Only in case it is *NaN*, the result is *False*. A precondition for this is that the used environment (compiler, processor, etc.) support the IEEE 754 standard.



PROVEtech:TA supports 64-bit integer signal values beginning with version 2022 SE. Please be aware that workpage controls can only display these values accurately in a textual form. For displaying signal values in strip chart controls, gauges or bar controls, PROVEtech:TA must convert signal values to pixel positions or degrees by using the 64-bit floating-point data type (52-bit mantissa). Converting a 64-bit integer value to a 64-bit floating-point value automatically reduces the precision if the value does not fit 100% into the 52-bit mantissa. Only a textual output of such signal values is accurate.

2.13 Creating a problem report

This chapter describes what to do when PROVEtech:TA behaves unexpectedly, and you suspect the cause of this behavior to be a bug in the software. It also describes how you can inform PROVEtech Support when PROVEtech:TA crashes.

Communicating errors in the software is of interest to both the software user as well as the developers. It plays an important part in developing stable software. The following paragraphs describe how to recognize a bug in PROVEtech:TA when one exists, and which information you should forward to PROVEtech Support (provetech@akkodis.com) so that the bug can be quickly removed. By suspicion of a bug in PROVEtech:TA, please follow these steps:

1. Check for any warning or error messages in the PROVEtech:TA Log (see ***3.6.1 PROVEtech:TA Log***). The PROVEtech:TA Log runs in a separate process independent of the PROVEtech:TA application. The PROVEtech:TA Log continues to function even after a crash in the program. The last output in the PROVEtech:TA Log displays the module with which the crash is most likely associated.

2. If you are uncertain whether a particular behavior of PROVEtech:TA is intended or unintended (bug or a feature), then please consult with the relevant chapter in the manual. If you select a window or tab and then press the F1 key, the chapter containing the relevant information about this item will be automatically displayed. If PROVEtech:TA crashes or PROVEtech:TA does not react anymore (dead lock or endless loop) then it is a software bug in many cases. Exception: You ran into an endless loop in your WinWrap Basic test script.

3. Please check out if you use the latest Service Pack to your PROVEtech:TA version. In Service Packs we exclusively fix software bugs. There are no new features implemented.

4. If your program code modules (e.g. your WinWrap Basic Code) link with components external to PROVEtech:TA (such as COM object, ActiveX components, .NET assemblies, etc.) please check whether one of these components is the cause of the software crash (e.g. try to debug in the Basic editor) and contact the relevant software support service. If CAESAR or WinWrap Basic technical assistance is needed PROVEtech Support can also provide you with help.

5. In the main menu of PROVEtech:TA (see ***Figure 16***) as well as in the PROVEtech:TA Log menu, you can find the menu item *Help|Technical Support…* (see ***3.7.1 Technical Support***). Additionally, there is a toolbar button for it in PROVEtech:TA as well as in the PROVEtech:TA Log (see ***Figure 17***). After selecting it a window pops up where you can write a short description of the error as well as the task which led to the error (see also ***Figure 18***). Use the button `Save report…` to save the problem report (without additional compression because of possible firewall settings) to your local hard disk and e-mail the problem report to the address provetech@akkodis.com

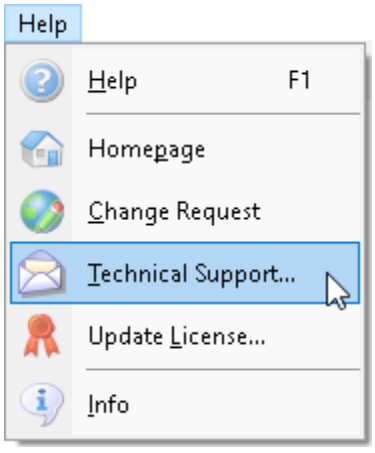


Figure 16 – Menu item "Technical Support…"

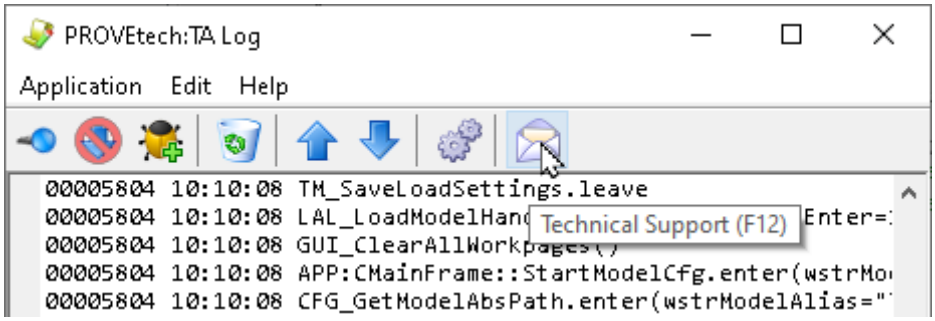


Figure 17 – Toolbar button "Technical Support" in the PROVEtech:TA Log

---

If PROVEtech:TA is still in the starting process, or PROVEtech:TA is not responding anymore (dead lock or a different reason why the application did not return from the operation) then you should first create a problem report via the **PROVEtech:TA Log** before you kill the PROVEtech:TA process. Only then the PROVEtech:TA software development can analyze the dead lock in the source code.

Below is an example of a typical problem report:

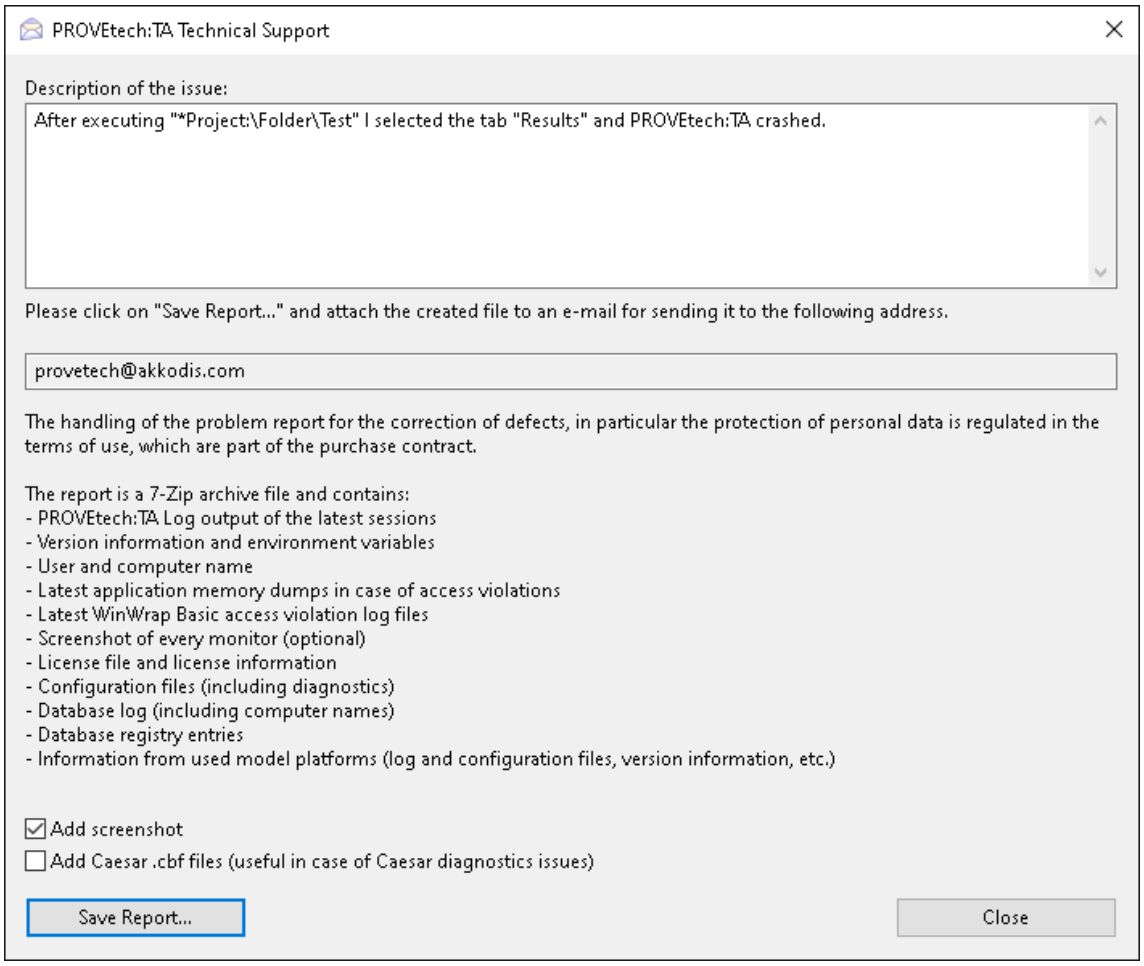


Figure 18 – Creating a problem report

Screenshots are often very helpful for analyzing issues. However, a screenshot might contain personal/sensitive information. PROVEtech:TA creates a screenshot of all monitors before opening the upper dialog. We recommend minimizing/hiding windows with sensitive information before opening the dialog. However, if you feel uncomfortable with adding the screenshot to the report, simply uncheck the `Add screenshot` check box.

Activate `Add Caesar .cbf files` in case you have an issue with the CAESAR Diagnostics (see chapter ***6.1 CAESAR diagnostics***).

The report file is a 7-Zip archive file containing a lot of information for analyzing the issue (the lower list in ***Figure 18*** describes the content). You can open it using 7-Zip (see www.7-zip.org) and have a look at the packed data.

The technical support collects the critical errors and fixes the bugs via service packs. In the event of a “showstopper” bug, a new hotfix will be immediately posted. Superficial

bugs are usually addressed in the next version of PROVEtech:TA or with the next service pack.



The Akkodis e-mail server deletes all attachments with the .zip file extension. The problem report files have the extension .7z0 which passes the server. So please attach problem report files to your e-mails without additionally packing them or modifying the file extension.



PROVEtech Support uses the data of the problem report only for solving the problem. This data is not passed to third parties under any circumstances, except of arrangements with the user or customer.

The handling of the problem report for the correction of defects, in particular the protection of personal data is regulated in the terms of use, which are part of the purchase contract.

2.14 Processing and protection of personal data

PROVEtech:TA cares about the protection of personal data. The following is a list of information and advice in this regard:

- In the database of PROVEtech:TA's Test Manager, personal data is only stored in a central location in the user data record. The user ID is now used internally to link a username with the author of a test object or with the tester in the case of a test result. The lock mechanism, the history tab and the validator of a test result work in the same way. This means that when you delete a user from the database, only the user ID is displayed instead of the name. Read more in chapter ***5.3.8.1 The user***.
- The user attributes (full name, e-mail, phone) are stored in the database in plain text and can be viewed directly if the login credentials of the database user is known. The user attributes can only be read via PROVEtech:TA if the *arManage* access right to the root object has been granted. See also chapter ***5.2.6 Access rights***.
- Also in the test object's description, version description and in the comment of a test result, corresponding replacements take place internally (e.g. userid#123 instead of the user name in plain text).
- .mac files and data acquisitions no longer contain user data in the header; .dbi.7z files (Test Manager's export files) contain no personal data anymore in the info.txt file

- There are two test language commands ***TestItem.ErasePersonalData*** and ***TestResult.ErasePersonalData*** for deleting personal data, for example from subtrees of older HIL projects or series (see in section ***Test Language***). The user is set to "userid#0" at all positions, leading to the display of "Unknown".
- **Attention:** Old PROVEtech:TA versions (PROVEtech:TA 2020 and older) stored the user ID as a plain text string in the database (e.g., "josmith"). These remain in the database even if the user was deleted with a current PROVEtech:TA version. Only with the new PROVE-tech:TA version, "Unknown" is displayed as user. The reason for this is that it would be too time-consuming to search and replace all occurrences in the complete database.
- It is not possible to remove only a single user from a test object in the database if this user is still stored in plain text in the database from an old PROVEtech:TA version.
- When exporting and importing a PROVEtech:TA archive file (.dbi.7z file, see chapter ***5.3.10 Exporting and importing tests***), regarding the user data at the author (test object), executor and validator (test result), usernames that are still in plain text in the database are converted to the new format (e.g., userid#123), or userid#0 if the user no longer exists. If arManage access rights to the root object are available, user data and user groups can also be exported in order to have the user ID assigned to the username. After the import, a user is only displayed in the Test Manager for the author of the test object and for the executor and validator of the test result if the identifier from userid#123 also exists in the new database, or if the mapping of the identifier to the user exists in the export archive and the username exists in the new database. Please remember to delete the PROVEtech:TA archive file once it is no longer needed.
- The processing of personal data, in particular with regard to the sending of problem reports (see chapter ***2.13 Creating a problem report***) for analysis, is regulated and described in the license agreement in accordance with the legal requirements. (see German: Datenschutzgrundverordnung (DSGVO/DS-GVO), French: Règlement général sur la protection des données (RGPD), English: General Data Protection Regulation (GDPR))

2.15 Legal notice

PROVEtech:TA uses 3rd party software modules. Here are their license texts in alphabetical order.

7-Zip (https://www.7zip.org)

PROVEtech:TA uses “7za.dll” and “7za.exe” of the “7-Zip Extra” package.

```
  7-Zip Extra
  ~~~~~~~~~~~
  License for use and distribution
  ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~

  Copyright (C) 1999-2025 Igor Pavlov.

  7-Zip Copyright (C) 1999-2025 Igor Pavlov.

  The licenses for files are:

    - 7za.exe:
         - The "GNU LGPL" as main license for most of the code
         - The "BSD 3-clause License" for some code
         - The "BSD 2-clause License" for some code
    - All other files: the "GNU LGPL".

  Redistributions in binary form must reproduce related license information from this
file.

  Note:
        You can use 7-Zip Extra on any computer, including a computer in a commercial
        organization. You don't need to register or pay for 7-Zip.

  It is allowed to digitally sign DLL and EXE files included into this package
  with arbitrary signatures of third parties.


GNU LGPL information
--------------------

    This library is free software; you can redistribute it and/or
    modify it under the terms of the GNU Lesser General Public
    License as published by the Free Software Foundation; either
    version 2.1 of the License, or (at your option) any later version.

    This library is distributed in the hope that it will be useful,
    but WITHOUT ANY WARRANTY; without even the implied warranty of
    MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.  See the GNU
    Lesser General Public License for more details.

    You can receive a copy of the GNU Lesser General Public License from
    http://www.gnu.org/



BSD 3-clause License in 7-Zip code
----------------------------------

  The "BSD 3-clause License" is used for the following code in 7za.exe
    -  ZSTD data decompression.
       that code was developed using original zstd decoder code as reference code.
```

```
       The original zstd decoder code was developed by Facebook Inc,
       that also uses the "BSD 3-clause License".

  Copyright (c) Facebook, Inc. All rights reserved.
  Copyright (c) 2023-2025 Igor Pavlov.

Text of the "BSD 3-clause License"
----------------------------------

Redistribution and use in source and binary forms, with or without modification,
are permitted provided that the following conditions are met:

1. Redistributions of source code must retain the above copyright notice, this
   list of conditions and the following disclaimer.

2. Redistributions in binary form must reproduce the above copyright notice,
   this list of conditions and the following disclaimer in the documentation
   and/or other materials provided with the distribution.

3. Neither the name of the copyright holder nor the names of its contributors may
   be used to endorse or promote products derived from this software without
   specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND
ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED
WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE
DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT HOLDER OR CONTRIBUTORS BE LIABLE FOR
ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES
(INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES;
LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON
ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT
(INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS
SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.



BSD 2-clause License in 7-Zip code
----------------------------------

  The "BSD 2-clause License" is used for the XXH64 code in 7za.exe.

  XXH64 code in 7-Zip was derived from the original XXH64 code developed by Yann Col-
let.

  Copyright (c) 2012-2021 Yann Collet.
  Copyright (c) 2023-2025 Igor Pavlov.

Text of the "BSD 2-clause License"
----------------------------------

Redistribution and use in source and binary forms, with or without modification,
are permitted provided that the following conditions are met:

1. Redistributions of source code must retain the above copyright notice, this
   list of conditions and the following disclaimer.

2. Redistributions in binary form must reproduce the above copyright notice,
   this list of conditions and the following disclaimer in the documentation
```

```
   and/or other materials provided with the distribution.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND
ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED
WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE
DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT HOLDER OR CONTRIBUTORS BE LIABLE FOR
ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES
(INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES;
LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON
ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT
(INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS
SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

Boost Library (https://www.boost.org)
PROVEtech:TA uses the regular expression parser of the Boost Library.

Boost Software License - Version 1.0 - August 17th, 2003

Permission is hereby granted, free of charge, to any person or organization obtaining
a copy of the software and accompanying documentation covered by this license (the
"Software") to use, reproduce, display, distribute, execute, and transmit the Soft-
ware, and to prepare derivative works of the Software, and to permit third-parties to
whom the Software is furnished to do so, all subject to the following:
The copyright notices in the Software and this entire statement, including the above
license grant, this restriction and the following disclaimer, must be included in all
copies of the Software, in whole or in part, and all derivative works of the Soft-
ware, unless such copies or derivative works are solely in the form of machine-exe-
cutable object code generated by a source language processor.
THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED,
INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A
PARTICULAR PURPOSE, TITLE AND NON-INFRINGEMENT. IN NO EVENT SHALL THE COPYRIGHT
HOLDERS OR ANYONE DISTRIBUTING THE SOFTWARE BE LIABLE FOR ANY DAMAGES OR OTHER
LIABILITY, WHETHER IN CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN
CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.
```

CAESAR

PROVEtech:TA uses the software CAESAR. All rights are with the Daimler AG and the I+ME ACTIA GmbH. The XS PDU API and the SDconnect Toolkit are installed as well.

cppdap (https://github.com/google/cppdap)

```
                                 Apache License
                           Version 2.0, January 2004
                        http://www.apache.org/licenses/

   TERMS AND CONDITIONS FOR USE, REPRODUCTION, AND DISTRIBUTION
```

1. Definitions.

   "License" shall mean the terms and conditions for use, reproduction,
   and distribution as defined by Sections 1 through 9 of this document.

   "Licensor" shall mean the copyright owner or entity authorized by
   the copyright owner that is granting the License.

   "Legal Entity" shall mean the union of the acting entity and all
   other entities that control, are controlled by, or are under common
   control with that entity. For the purposes of this definition,
   "control" means (i) the power, direct or indirect, to cause the
   direction or management of such entity, whether by contract or
   otherwise, or (ii) ownership of fifty percent (50%) or more of the
   outstanding shares, or (iii) beneficial ownership of such entity.

   "You" (or "Your") shall mean an individual or Legal Entity
   exercising permissions granted by this License.

   "Source" form shall mean the preferred form for making modifications,
   including but not limited to software source code, documentation
   source, and configuration files.

   "Object" form shall mean any form resulting from mechanical
   transformation or translation of a Source form, including but
   not limited to compiled object code, generated documentation,
   and conversions to other media types.

   "Work" shall mean the work of authorship, whether in Source or
   Object form, made available under the License, as indicated by a
   copyright notice that is included in or attached to the work
   (an example is provided in the Appendix below).

   "Derivative Works" shall mean any work, whether in Source or Object
   form, that is based on (or derived from) the Work and for which the
   editorial revisions, annotations, elaborations, or other modifications
   represent, as a whole, an original work of authorship. For the purposes
   of this License, Derivative Works shall not include works that remain
   separable from, or merely link (or bind by name) to the interfaces of,
   the Work and Derivative Works thereof.

   "Contribution" shall mean any work of authorship, including
   the original version of the Work and any modifications or additions
   to that Work or Derivative Works thereof, that is intentionally
   submitted to Licensor for inclusion in the Work by the copyright owner
   or by an individual or Legal Entity authorized to submit on behalf of
   the copyright owner. For the purposes of this definition, "submitted"
   means any form of electronic, verbal, or written communication sent
   to the Licensor or its representatives, including but not limited to
   communication on electronic mailing lists, source code control systems,
   and issue tracking systems that are managed by, or on behalf of, the
   Licensor for the purpose of discussing and improving the Work, but

excluding communication that is conspicuously marked or otherwise designated in writing by the copyright owner as "Not a Contribution."

"Contributor" shall mean Licensor and any individual or Legal Entity on behalf of whom a Contribution has been received by Licensor and subsequently incorporated within the Work.

2. Grant of Copyright License. Subject to the terms and conditions of this License, each Contributor hereby grants to You a perpetual, worldwide, non-exclusive, no-charge, royalty-free, irrevocable copyright license to reproduce, prepare Derivative Works of, publicly display, publicly perform, sublicense, and distribute the Work and such Derivative Works in Source or Object form.

3. Grant of Patent License. Subject to the terms and conditions of this License, each Contributor hereby grants to You a perpetual, worldwide, non-exclusive, no-charge, royalty-free, irrevocable (except as stated in this section) patent license to make, have made, use, offer to sell, sell, import, and otherwise transfer the Work, where such license applies only to those patent claims licensable by such Contributor that are necessarily infringed by their Contribution(s) alone or by combination of their Contribution(s) with the Work to which such Contribution(s) was submitted. If You institute patent litigation against any entity (including a cross-claim or counterclaim in a lawsuit) alleging that the Work or a Contribution incorporated within the Work constitutes direct or contributory patent infringement, then any patent licenses granted to You under this License for that Work shall terminate as of the date such litigation is filed.

4. Redistribution. You may reproduce and distribute copies of the Work or Derivative Works thereof in any medium, with or without modifications, and in Source or Object form, provided that You meet the following conditions:

   (a) You must give any other recipients of the Work or Derivative Works a copy of this License; and

   (b) You must cause any modified files to carry prominent notices stating that You changed the files; and

   (c) You must retain, in the Source form of any Derivative Works that You distribute, all copyright, patent, trademark, and attribution notices from the Source form of the Work, excluding those notices that do not pertain to any part of the Derivative Works; and

   (d) If the Work includes a "NOTICE" text file as part of its distribution, then any Derivative Works that You distribute must include a readable copy of the attribution notices contained within such NOTICE file, excluding those notices that do not pertain to any part of the Derivative Works, in at least one

of the following places: within a NOTICE text file distributed as part of the Derivative Works; within the Source form or documentation, if provided along with the Derivative Works; or, within a display generated by the Derivative Works, if and wherever such third-party notices normally appear. The contents of the NOTICE file are for informational purposes only and do not modify the License. You may add Your own attribution notices within Derivative Works that You distribute, alongside or as an addendum to the NOTICE text from the Work, provided that such additional attribution notices cannot be construed as modifying the License.

You may add Your own copyright statement to Your modifications and may provide additional or different license terms and conditions for use, reproduction, or distribution of Your modifications, or for any such Derivative Works as a whole, provided Your use, reproduction, and distribution of the Work otherwise complies with the conditions stated in this License.

5. Submission of Contributions. Unless You explicitly state otherwise, any Contribution intentionally submitted for inclusion in the Work by You to the Licensor shall be under the terms and conditions of this License, without any additional terms or conditions. Notwithstanding the above, nothing herein shall supersede or modify the terms of any separate license agreement you may have executed with Licensor regarding such Contributions.

6. Trademarks. This License does not grant permission to use the trade names, trademarks, service marks, or product names of the Licensor, except as required for reasonable and customary use in describing the origin of the Work and reproducing the content of the NOTICE file.

7. Disclaimer of Warranty. Unless required by applicable law or agreed to in writing, Licensor provides the Work (and each Contributor provides its Contributions) on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied, including, without limitation, any warranties or conditions of TITLE, NON-INFRINGEMENT, MERCHANTABILITY, or FITNESS FOR A PARTICULAR PURPOSE. You are solely responsible for determining the appropriateness of using or redistributing the Work and assume any risks associated with Your exercise of permissions under this License.

8. Limitation of Liability. In no event and under no legal theory, whether in tort (including negligence), contract, or otherwise, unless required by applicable law (such as deliberate and grossly negligent acts) or agreed to in writing, shall any Contributor be liable to You for damages, including any direct, indirect, special, incidental, or consequential damages of any character arising as a result of this License or out of the use or inability to use the Work (including but not limited to damages for loss of goodwill, work stoppage, computer failure or malfunction, or any and all other commercial damages or losses), even if such Contributor

```
   has been advised of the possibility of such damages.

9. Accepting Warranty or Additional Liability. While redistributing
   the Work or Derivative Works thereof, You may choose to offer,
   and charge a fee for, acceptance of support, warranty, indemnity,
   or other liability obligations and/or rights consistent with this
   License. However, in accepting such obligations, You may act only
   on Your own behalf and on Your sole responsibility, not on behalf
   of any other Contributor, and only if You agree to indemnify,
   defend, and hold each Contributor harmless for any liability
   incurred by, or claims asserted against, such Contributor by reason
   of your accepting any such warranty or additional liability.
```

DTS MCD-3 D Server of Softing (https://www.softing.com)

The DTS MCD-3 D Server of Softing is no part of the PROVEtech:TA product. No component is integrated in the installation package of PROVEtech:TA. You can find legal notice of the DTS MCD-3 D Server of Softing on the Homepage of Softing.

Edit Control HTML/JS

PROVEtech:TA uses a third party edit control template that may be freely modified and published under the MIT license.

Copyright (c) 2015 Akshay Nair (@phenax)

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

FlexNet (https://www.revenera.com)

This product uses FlexNet Publisher which is a registered trademark of Revenera. All rights reserved. You can find legal notice of FlexNet on the Revenera Homepage.

Licensee informs Licensee's end users of the Licensee FNL Products of the connection, collection and dissemination of information between such end users and/or Revenera, as applicable. Licensee will reproduce with the redistributables all applicable trademark and copyright notices that accompany the FlexNet Licensing Software and/or redistributables, but Licensee may not use Revenera’s name, logos or trademarks to market Licensee's products.

gRPC (https://grpc.io)

Copyright 2014 gRPC authors.

Licensed under the Apache License, Version 2.0 (the "License"); you may not use this file except in compliance with the License. You may obtain a copy of the License at

http://www.apache.org/licenses/LICENSE-2.0

Unless required by applicable law or agreed to in writing, software distributed under the License is distributed on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the License for the specific language governing permissions and limitations under the License.

ICU License - ICU 1.8.1 and later (icu-project.org – used by MAT file API)

COPYRIGHT AND PERMISSION NOTICE

Copyright (c) 1995-2010 International Business Machines Corporation and others

All rights reserved.

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, provided that the above copyright notice(s) and this

permission notice appear in all copies of the Software and that both the above copyright notice(s) and this permission notice appear in supporting documentation.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT OF THIRD PARTY RIGHTS. IN NO EVENT SHALL THE COPYRIGHT HOLDER OR HOLDERS INCLUDED IN THIS NOTICE BE LIABLE FOR ANY CLAIM, OR ANY SPECIAL INDIRECT OR CONSEQUENTIAL DAMAGES, OR ANY DAMAGES WHATSOEVER RESULTING FROM LOSS OF USE, DATA OR PROFITS, WHETHER IN AN ACTION OF CONTRACT, NEGLIGENCE OR OTHER TORTIOUS ACTION, ARISING OUT OF OR IN CONNECTION WITH THE USE OR PERFORMANCE OF THIS SOFTWARE.

Except as contained in this notice, the name of a copyright holder shall not be used in advertising or otherwise to promote the sale, use or other dealings in this Software without prior written authorization of the copyright holder.

All trademarks and registered trademarks mentioned herein are the property of their respective owners.

IdentityModel (https://github.com/IdentityModel, used for OIDC/OAuth login)

The OIDC/OAuth login (OpenID Connect) uses several .NET DLLs from the IdentityModel collection with Apache-2.0 license (https://github.com/IdentityModel/IdentityModel/blob/main/LICENSE). They depend on several Microsoft DLLs with MIT license (https://licenses.nuget.org/MIT). You can find these DLLs and their license texts in the installation directory “bin\IdentityModel”.

InnoSetup (https://www.innosetup.org)

```
Inno Setup License
==================

Except where otherwise noted, all of the documentation and software included in the Inno Setup
package is copyrighted by Jordan Russell.

Copyright (C) 1997-2025 Jordan Russell. All rights reserved.
Portions Copyright (C) 2000-2025 Martijn Laan. All rights reserved.

This software is provided "as-is," without any express or implied warranty. In no event shall the
author be held liable for any damages arising from the use of this software.
```

```
Permission is granted to anyone to use this software for any purpose, including commercial
applications, and to alter and redistribute it, provided that the following conditions are met:

1. All redistributions of source code files must retain all copyright notices that are currently in
   place, and this list of conditions without modification.

2. All redistributions in binary form must retain all occurrences of the above copyright notice and
   web site addresses that are currently in place (for example, in the About boxes).

3. The origin of this software must not be misrepresented; you must not claim that you wrote the
   original software. If you use this software to distribute a product, an acknowledgment in the
   product documentation would be appreciated but is not required.

4. Modified versions in source or binary form must be plainly marked as such, and must not be
   misrepresented as being the original software.


Jordan Russell
jr-2020 AT jrsoftware.org
https://jrsoftware.org/
```

MATLAB MAT file API (www.themathworks.com - from the MATLAB license text for the .mat file export and import of data acquisitions)

6.2. MAT files. Licensed User may create and distribute Applications that read MAT files using the MAT file API, however, if such Applications are distributed to Third Parties, they must also implement MAT file write capability.

Microsoft SQL Server (https://www.microsoft.com/sql)

The Microsoft SQL Server is no part of the PROVEtech:TA product. Only the Microsoft SQL Server ODBC Driver is integrated in the installation package of PROVEtech:TA. You can find legal notice of Microsoft SQL Server on the Microsoft SQL Server Homepage. Additionally, there is a text files in the “ODBCMicrosoft” sub folder of the PROVEtech:TA installation.

Microsoft WebView2 (Edge on Chromium Interface, https://www.nuget.org/packages/Microsoft.Web.WebView2)

Copyright (C) Microsoft Corporation. All rights reserved.

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

* Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.

* Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.

* The name of Microsoft Corporation, or the names of its contributors may not be used to endorse or promote products derived from this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

NATIONAL INSTRUMENTS SOFTWARE LICENSE AGREEMENT FOR TDM DLL

INSTALLATION NOTICE: THIS IS A CONTRACT. BEFORE YOU DOWNLOAD THE TDM DLL AND/OR COMPLETE THE INSTALLATION PROCESS, CAREFULLY READ THIS AGREEMENT. BY DOWNLOADING THE TDM DLL AND/OR CLICKING THE APPLICABLE BUTTON TO COMPLETE THE DOWNLOAD PROCESS, YOU CONSENT TO THE TERMS OF THIS AGREEMENT AND YOU AGREE TO BE BOUND BY THIS AGREEMENT. IF YOU DO NOT WISH TO BECOME A PARTY TO THIS AGREEMENT AND BE BOUND BY ALL OF ITS TERMS AND CONDITIONS, CLICK THE APPROPRIATE BUTTON TO CANCEL THE DOWNLOAD PROCESS AND DO NOT USE THE TDM DLL.

1. Definitions. As used in this Agreement, the following terms have the following meanings:

   A. "You." Means you the individual using the TDM DLL (the "TDM DLL") as well as your employer if you are using the TDM DLL within the scope of your employment.

B. "NI." Means National Instruments Corporation, a company organized under the laws of the State of Delaware, U.S.A.

C. "TDM DLL." Means the computer libraries provided with this Agreement and which are being downloaded together with all accompanying documentation, utilities, and driver interface software

2. Grant of License; Obligations. NI grants you a limited, non-exclusive right to download, use and distribute the TDM DLL pursuant to the terms and conditions of this Agreement. You may use the TDM DLL in your product in order for the product to access (i.e., read and write) TDM and/or TDMS files. If your application reads TDM files using the TDM DLL, you must support writing TDM files as well. If your application reads TDMS files using the TDM DLL, you must support writing TDMS files as well. You may distribute the TDM DLL as part of your product with such limitations as set forth below.

3. Restrictions. You may not: (i) reverse engineer, decompile, or disassemble the TDM DLL (except to the extent such foregoing restriction is expressly prohibited by applicable law); (ii) sub-license, lease, or rent the TDM DLL; (iii) modify, or create derivatives of the TDM DLL; and (iv) directly or indirectly, export, re-export, download, or ship the TDM DLL in violation of the laws and regulations of the U.S.A. and the laws and regulations of the applicable jurisdiction in which you use or are downloading the TDM DLL. Under no circumstance is "floating" or shared use permitted under this Agreement. You agree not to use the TDM DLL to make or distribute your own or a third party's application, a principal purpose of which, as reasonably determined by NI, is to perform (by itself or in combination with other products) the same or similar functions as (or are otherwise intended to replace or supplant any component of) the TDM DLL or any other software of NI.

4. Transfer/Distribution. Your license is for your own use only. Therefore, the license is non-transferable and you may not distribute or otherwise provide the TDM DLL to any third party except as part of your own product which you provide to third parties. Such third parties may not redistribute the TDM DLL and you agree to include such a prohibition in your form of license agreement. In distributing the TDM DLL as part of your product, you must include the copyright notice: "Portions used under license from National Instruments Corporation; copyright (c) 2007 National Instruments Corporation.

5. Multiple Versions (CD-ROM Media). If the TDM DLL is distributed on a CD-ROM containing multiple versions of the TDM DLL, you may only use one version of the TDM DLL.

6. Copyright. The TDM DLL is owned by NI or its suppliers and is protected by applicable copyright laws and international treaty provisions. Therefore, you must treat the TDM DLL like any other copyrighted material. You may, however, copy the TDM DLL solely for backup or archival purposes. All rights not expressly granted to you in this Agreement are reserved to NI.

7. Patent and Trademark Notice For patents covering National Instruments products, refer to the appropriate location: Help>>Patents in your software, the patents.txt file on your CD, DVD, or ni.com/patents. ComponentWorks, CVI, FieldPoint, IMAQ, Lookout, LabVIEW, LabWindows/ CVI, Measurement Studio, NI-488.2, NI-CAN, NI-DAQ, NI-FBUS, NI-VISA, NI-VXI and TestStand are the trademarks of National Instruments Corporation. DIAdem and DASYLab are the trademarks of National Instruments (Ireland) Limited. Further, all other product and company names used herein are (or may be) trademarks or trade names of their respective companies.

8. No Warranties. YOU ASSUME THE ENTIRE RISK AS TO THE RESULTS AND PERFORMANCE OF THE TDM DLL. NI WILL NOT UPDATE THE TDM DLL NOR WILL NI SUPPORT THE TDM DLL. THE TDM DLL IS PROVIDED "AS IS" WITHOUT WARRANTY OF ANY KIND, AND NO WARRANTIES, EITHER EXPRESSED OR IMPLIED ARE MADE WITH RESPECT TO THE TDM DLL, INCLUDING BUT NOT LIMITED TO ANY IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE OR NON-INFRINGEMENT, OR ANY OTHER WARRANTIES THAT MAY ARISE FROM USAGE OF TRADE OR COURSE OF DEALING. NI DOES NOT WARRANT, GUARANTEE, OR MAKE ANY REPRESENTATIONS REGARDING THE USE OF OR THE RESULTS OF THE USE OF THE TDM DLL IN TERMS OF CORRECTNESS, ACCURACY, RELIABILITY, OR OTHERWISE AND DOES NOT WARRANT THAT THE OPERATION OF THE TDM DLL WILL BE UNINTERRUPTED OR ERROR FREE. NI EXPRESSLY DISCLAIMS ANY WARRANTIES NOT STATED HEREIN.

9. Limitation On Liability. To the maximum extent permitted by applicable law, in no event shall NI and its licensors, distributors, and suppliers (including its and their directors, officers, employees, and agents) be liable for any damages, including, but not limited to, any special, direct, indirect, incidental, exemplary, or consequential damages, expenses, lost profits, lost savings, business interruption, lost business information, or any other damages arising out of the use or inability to use the TDM DLL or related hardware even if NI or its licensors, distributors, and suppliers has been advised of the possibility of such damages. You acknowledge that the applicable fees and prices reflect this allocation of risk. Because some states/jurisdictions do not allow the exclusion or limitation of liability for consequential or incidental damages, the above limitation may not apply. If the foregoing limitation of liability is not enforceable because the TDM DLL or the hardware is determined by a court of competent jurisdiction in a final, non-appealable judgment to be defective and to have directly caused bodily injury, death, or property damage, in no event shall NI's liability for property damage exceed the greater of $5000 or the license fee paid for the TDM DLL.

10. INDEMNIFICATION. YOU SHALL DEFEND, INDEMNIFY, AND HOLD NI HARMLESS FROM AND AGAINST ALL LIABILITY, DAMAGES, AND COSTS (INCLUDING COURT COSTS AND REASONABLE FEES OF ATTORNEYS AND OTHER PROFESSIONALS) ARISING OUT OF OR

RESULTING FROM YOUR DISTRIBUTION OR USE OF THE TDM DLL; EVEN IF DUE IN PART TO NI'S CONCURRENT NEGLIGENCE, OTHER FAULT, OR STRICT LIABILITY.

11. Warning. (1) NI PRODUCTS ARE NOT DESIGNED WITH COMPONENTS AND TESTING FOR A LEVEL OF RELIABILITY SUITABLE FOR USE IN OR IN CONNECTION WITH SURGICAL IMPLANTS OR AS CRITICAL COMPONENTS IN ANY LIFE SUPPORT SYSTEMS WHOSE FAILURE TO PERFORM CAN REASONABLY BE EXPECTED TO CAUSE SIGNIFICANT INJURY TO A HUMAN. (2) IN ANY APPLICATION, INCLUDING THE ABOVE, RELIABILITY OF OPERATION OF THE SOFTWARE PRODUCTS CAN BE IMPAIRED BY ADVERSE FACTORS, INCLUDING BUT NOT LIMITED TO FLUCTUATIONS IN ELECTRICAL POWER SUPPLY, COMPUTER HARDWARE MALFUNCTIONS, COMPUTER OPERATING SYSTEM SOFTWARE FITNESS, FITNESS OF COMPILERS AND DEVELOPMENT SOFTWARE USED TO DEVELOP AN APPLICATION, INSTALLATION ERRORS, SOFTWARE AND HARDWARE COMPATIBILITY PROBLEMS, MALFUNCTIONS OR FAILURES ELECTRONIC SYSTEMS (HARDWARE AND/OR SOFTWARE), UNANTICIPATED USES OR MISUSES, OR ERRORS ON THE PART OF THE OF ELECTRONIC MONITORING OR CONTROL DEVICES, TRANSIENT FAILURES OF USER OR APPLICATIONS DESIGNER (ADVERSE FACTORS SUCH AS THESE ARE HEREAFTER COLLECTIVELY TERMED "SYSTEM FAILURES"). ANY APPLICATION WHERE A SYSTEM FAILURE WOULD CREATE A RISK OF HARM TO PROPERTY OR PERSONS (INCLUDING THE RISK OF BODILY INJURY AND DEATH) SHOULD NOT BE RELIANT SOLELY UPON ONE FORM OF ELECTRONIC SYSTEM DUE TO THE RISK OF SYSTEM FAILURE. TO AVOID DAMAGE, INJURY, OR DEATH, THE USER OR APPLICATION DESIGNER MUST TAKE REASONABLY PRUDENT STEPS TO PROTECT AGAINST SYSTEM FAILURES, INCLUDING BUT NOT LIMITED TO BACK-UP OR SHUT DOWN MECHANISMS. BECAUSE EACH END-USER SYSTEM IS CUSTOMIZED AND DIFFERS FROM NI'S TESTING PLATFORMS AND BECAUSE A USER OR APPLICATION DESIGNER MAY USE NI PRODUCTS IN COMBINATION WITH OTHER PRODUCTS IN A MANNER NOT EVALUATED OR CONTEMPLATED BY NI, THE USER OR APPLICATION DESIGNER IS ULTIMATELY RESPONSIBLE FOR VERIFYING AND VALIDATING THE SUITABILITY OF NI PRODUCTS WHENEVER NI PRODUCTS ARE INCORPORATED IN A SYSTEM OR APPLICATION, INCLUDING, WITHOUT LIMITATION, THE APPROPRIATE DESIGN, PROCESS AND SAFETY LEVEL OF SUCH SYSTEM OR APPLICATION.

12. U.S. Government Restricted Rights. If you are an agency, department, or other entity of the United States Government ("Government"), the use, duplication, reproduction, release, modification, disclosure or transfer of the TDM DLL, or any related documentation of any kind, including technical data or manuals, is restricted

in accordance with Federal Acquisition Regulation 12.212 for civilian agencies and Defense Federal Acquisition Regulation Supplement 227.7202 for military agencies. The TDM DLL is commercial computer software and the related documentation is commercial computer software documentation. The use of the TDM DLL and related documentation is further restricted in accordance with the terms of this Agreement, or any modification hereto. The Contractor/Manufacturer is National Instruments Corporation, 11500 North Mopac Expressway, Austin, Texas, U.S.A., 78759-3504.

13. Compliance. You agree to make all applicable records available for review by NI during your normal business hours so as to permit NI (upon reasonable notice to you) to verify your compliance with the terms and conditions of this Agreement. Further, if you are a business or other entity, you agree that upon the request of NI or NI's authorized representative, you will promptly document and certify in writing to NI that your and your employees' use of the TDM DLL complies with the terms and conditions of this Agreement.

14. Termination. NI may terminate this Agreement at any time by delivering notice to you. You may terminate this Agreement by destroying or erasing all copies of the TDM DLL. Further, this Agreement shall automatically terminate upon failure by you to comply with its terms. Upon termination of this Agreement, the license granted to you in Section 2 above will immediately terminate and you agree to destroy or erase all copies of the TDM DLL that you have made. However, the destruction requirement does not apply to copies distributed prior to the effective date of termination.

15. General.

    A. This Agreement is governed by the laws of the State of Texas, U.S.A., exclusive of any provisions of the United Nations Convention on the International Sale of Goods, and without regard to principles of conflicts of law and (ii) the non-exclusive venue for all actions under this Agreement shall be in the courts located in Travis County, Texas, U.S.A. and the parties agree to submit to the jurisdiction of such courts.

    B. This Agreement constitutes the complete agreement between you and NI regarding the TDM DLL and it supersedes any oral or written proposals, prior agreements, purchase orders or any other communication between you and NI relating to the subject matter of this Agreement.

    C. If any action is brought by either party to this Agreement against the other regarding the subject matter hereof, the prevailing party shall be entitled to recover, in addition to any relief granted, reasonable attorney fees and court costs. If any provision of this Agreement is held invalid, the offending clause will be modified so as to be enforceable and, as modified, shall be fully enforced, and the remainder of this Agreement will continue in full force and effect. If you are downloading the TDM DLL, you represent and warrant that you are not located

in or under the control of any country which the export laws and regulations of such country or of the United States prohibit the exportation of the TDM DLL to. Product and company names used herein are trademarks or trade names of their respective companies.

© 2007 National Instruments Corporation. All Rights Reserved.

netcoredbg (https://github.com/Samsung/netcoredbg)

```
MIT License

Copyright (c) 2017 Samsung Electronics Co., LTD

Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in
all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
SOFTWARE.


linenoise.cpp
=============

Copyright (c) 2010, Salvatore Sanfilippo <antirez at gmail dot com>
Copyright (c) 2010, Pieter Noordhuis <pcnoordhuis at gmail dot com>

All rights reserved.

Redistribution and use in source and binary forms, with or without
modification, are permitted provided that the following conditions are met:

  * Redistributions of source code must retain the above copyright notice,
    this list of conditions and the following disclaimer.
  * Redistributions in binary form must reproduce the above copyright
    notice, this list of conditions and the following disclaimer in the
    documentation and/or other materials provided with the distribution.
  * Neither the name of Redis nor the names of its contributors may be used
```

```
    to endorse or promote products derived from this software without
    specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS"
AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE
IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE
ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT OWNER OR CONTRIBUTORS BE
LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR
CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF
SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS
INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN
CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE)
ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE
POSSIBILITY OF SUCH DAMAGE.


wcwidth.cpp
===========

Markus Kuhn -- 2007-05-26 (Unicode 5.0)

Permission to use, copy, modify, and distribute this software
for any purpose and without fee is hereby granted. The author
disclaims all warranties with regard to this software.



ConvertUTF.cpp
==============

Copyright 2001-2004 Unicode, Inc.

Disclaimer

This source code is provided as is by Unicode, Inc. No claims are
made as to fitness for any particular purpose. No warranties of any
kind are expressed or implied. The recipient agrees to determine
applicability of information provided. If this file has been
purchased on magnetic or optical media from Unicode, Inc., the
sole remedy for any claim will be exchange of defective media
within 90 days of receipt.

Limitations on Rights to Redistribute This Code

Unicode, Inc. hereby grants the right to freely use the information
supplied in this file in the creation of products supporting the
Unicode Standard, and to make copies of this file in any form
for internal or external distribution as long as this notice
remains attached.


json.hpp
```

```
========

MIT License

Copyright (c) 2013-2018 Niels Lohmann

Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in
all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
SOFTWARE.
```

Oracle (https://www.oracle.com)

The Oracle Server is no part of the PROVEtech:TA product. Only the Oracle ODBC InstantClient is integrated in the installation package of PROVEtech:TA. You can find legal notice of Oracle on the Oracle Homepage. Additionally, there are according text files in the “ODBCOracle” sub folder of the PROVEtech:TA installation.

PostgreSQL (https://www.postgresql.org)

PostgreSQL Data Base Management System is no part of the PROVEtech:TA product, but it is included in the PROVEtech:TA installation package.

Portions Copyright (c) 1996-2025, PostgreSQL Global Development Group Portions Copyright (c) 1994-1996 Regents of the University of California

Permission to use, copy, modify, and distribute this software and its documentation for any purpose, without fee, and without a written agreement is hereby granted, provided

that the above copyright notice and this paragraph and the following two paragraphs appear in all copies.

IN NO EVENT SHALL THE UNIVERSITY OF CALIFORNIA BE LIABLE TO ANY PARTY FOR DIRECT, INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES, INCLUDING LOST PROFITS, ARISING OUT OF THE USE OF THIS SOFTWARE AND ITS DOCUMENTATION, EVEN IF THE UNIVERSITY OF CALIFORNIA HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

THE UNIVERSITY OF CALIFORNIA SPECIFICALLY DISCLAIMS ANY WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE. THE SOFTWARE PROVIDED HEREUNDER IS ON AN "AS IS" BASIS, AND THE UNIVERSITY OF CALIFORNIA HAS NO OBLIGATIONS TO PROVIDE MAINTENANCE, SUPPORT, UPDATES, ENHANCEMENTS, OR MODIFICATIONS.

PostgreSQL ODBC driver (https://odbc.postgresql.org)

PostgreSQL ODBC driver is no part of the PROVEtech:TA product, but it is included in the PROVEtech:TA installation package.

License: GNU Library General Public License, see upper paragraph: GNU LGPL information (https://opensource.org/license/lgpl-2-0/)

PRODIS MCD-3 D Server of DSA (https://www.dsa.de)

The PRODIS MCD-3 D Server of DSA is no part of the PROVEtech:TA product. No component is integrated in the installation package of PROVEtech:TA. You can find legal notice of the PRODIS MCD-3 D Server of DSA on the Homepage of DSA.

Protocol Buffers (https://github.com/protocolbuffers/protobuf)

```
Copyright 2008 Google Inc.  All rights reserved.

Redistribution and use in source and binary forms, with or without
```

```
modification, are permitted provided that the following conditions are
met:

    * Redistributions of source code must retain the above copyright
notice, this list of conditions and the following disclaimer.
    * Redistributions in binary form must reproduce the above
copyright notice, this list of conditions and the following disclaimer
in the documentation and/or other materials provided with the
distribution.
    * Neither the name of Google Inc. nor the names of its
contributors may be used to endorse or promote products derived from
this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS
"AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT
LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR
A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT
OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL,
SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT
LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE,
DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY
THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT
(INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE
OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

Code generated by the Protocol Buffer compiler is owned by the owner
of the input file used when generating it.  This code is not
standalone and requires a support library to be linked with it.  This
support library is itself covered by the above license.
```

QR Code Generator

Copyright © 2025 Project Nayuki. (MIT License)
https://www.nayuki.io/page/qr-code-generator-library

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

* The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

* The Software is provided "as is", without warranty of any kind, express or implied, including but not limited to the warranties of merchantability, fitness for a particular purpose and noninfringement. In no event shall the authors or copyright holders be liable for any claim, damages or other liability, whether in an action of contract, tort

or otherwise, arising from, out of or in connection with the Software or the use or other dealings in the Software.

Roslyn (https://github.com/dotnet/roslyn)

```
The MIT License (MIT)

Copyright (c) .NET Foundation and Contributors

All rights reserved.

Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in
all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
SOFTWARE.
```

Scintilla + Lexilla (https://scintilla.org)

```
License for Lexilla, Scintilla, and SciTE

Copyright 1998-2021 by Neil Hodgson <neilh@scintilla.org>

All Rights Reserved

Permission to use, copy, modify, and distribute this software and its
documentation for any purpose and without fee is hereby granted,
provided that the above copyright notice appear in all copies and that
both that copyright notice and this permission notice appear in
supporting documentation.

NEIL HODGSON DISCLAIMS ALL WARRANTIES WITH REGARD TO THIS
SOFTWARE, INCLUDING ALL IMPLIED WARRANTIES OF MERCHANTABILITY
AND FITNESS, IN NO EVENT SHALL NEIL HODGSON BE LIABLE FOR ANY
```

```
SPECIAL, INDIRECT OR CONSEQUENTIAL DAMAGES OR ANY DAMAGES
WHATSOEVER RESULTING FROM LOSS OF USE, DATA OR PROFITS,
WHETHER IN AN ACTION OF CONTRACT, NEGLIGENCE OR OTHER
TORTIOUS ACTION, ARISING OUT OF OR IN CONNECTION WITH THE USE
OR PERFORMANCE OF THIS SOFTWARE.
```

SymmetricDS (https://www.symmetricds.org)

You can use SymmetricDS by Jumpmind to replicate your PROVEtech:TA database. You can find legal notice at https://www.symmetricds.org/about/license. If you are interested or if you want to purchase additional support for database replication, you can find more information in the setup package in *Goodies\Databases\Replication\PROVEtechTA_SymmetricDS_howto.pdf* or please contact our PROVEtech Support via provetech@akkodis.com.

Two Level Segregated Fit memory allocator

Written by Matthew Conte, based on the original documentation by Miguel Masmano.
https://github.com/mattconte/tlsf

This implementation was written to the specification of the document, therefore no GPL restrictions apply.

Copyright (c) 2006-2025, Matthew Conte
All rights reserved.

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

* Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
* Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.
* Neither the name of the copyright holder nor the names of its contributors may be used to endorse or promote products derived from this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE

DISCLAIMED. IN NO EVENT SHALL MATTHEW CONTE BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

WinWrap Basic (https://www.winwrap.com)

This product uses WinWrap® Basic, Copyright 1993-2025 Polar Engineering, Inc., https://www.winwrap.com/

XL Driver Library (https://www.vector.com)

PROVEtech:TA accesses the XL Driver Library of Vector Informatik for controlling fault simulation hardware via CAN. The software is not part of the PROVEtech:TA installation package tough.

zlib (https://www.zlib.net)

Copyright (C) 1995-2025 Jean-loup Gailly and Mark Adler

This software is provided 'as-is', without any express or implied warranty. In no event will the authors be held liable for any damages arising from the use of this software.

Permission is granted to anyone to use this software for any purpose, including commercial applications, and to alter it and redistribute it freely, subject to the following restrictions:

1. The origin of this software must not be misrepresented; you must not claim that you wrote the original software. If you use this software in a product, an acknowledgment in the product documentation would be appreciated but is not required.
2. Altered source versions must be plainly marked as such, and must not be

misrepresented as being the original software.
3. This notice may not be removed or altered from any source distribution.

Jean-loup Gailly Mark Adler
jloup@gzip.org madler@alumni.caltech.edu

Microsoft, Windows, SQL Server, Internet Explorer, Edge, Roslyn, Word and Excel are registered trademarks of the Microsoft Corporation. All rights reserved.

Polar Engineering and WinWrap Basic are registered trademarks of Polar Engineering and Consulting. All rights reserved.

The Mathworks, MATLAB, Simulink, Real-Time Workshop, Simulink Coder and Simulink Real-Time are registered trademarks of The Mathworks, Inc. All rights reserved.

Speedgoat is a registered trademark of the Speedgoat GmbH. All rights reserved.

National Instruments, LabVIEW, LabVIEW-RT and VeriStand are registered trademarks of National Instruments. All rights reserved.

Oracle is a registered trademark of the Oracle Corporation. All rights reserved.

Softing and DTS are registered trademarks of the Softing AG. All rights reserved.

DSA and PRODIS are registered trademarks of the DSA GmbH. All rights reserved.

Google is a registered trademark of Alphabet Inc. All rights reserved.

I+ME ACTIA is a registered trademark of the I+ME ACTIA GmbH. All rights reserved.

Mercedes-Benz, eCOM and CAESAR are registered trademarks of the Mercedes-Benz Group AG and Daimler Truck Holding AG. All rights reserved.

Opal-RT and RT-LAB are registered trademarks of Opal-RT Technologies Inc. All rights reserved.

ETAS, ES4440, INCA and LabCar are registered trademarks of the ETAS Group. All rights reserved.

Softing Engineering & Solutions is a registered trademark of the Softing Engineering & Solutions GmbH. All rights reserved.

Vector Informatik, XL Driver Library, CANape, CANoe, CANcardX, CANcardXL, CANcardXLe, CANboardXL, CANcaseXL and the VNxxxx family are registered trademarks of the Vector Informatik GmbH. All rights reserved.

Synopsys and Silver are registered trademarks of the Synopsys, Inc. All rights reserved.

dSPACE, ControlDesk, ControlDesk NG, MicroAutoBox, DS1005, DS1006, DS5394C1, FullSize, MidSize, SCALEXIO, PHS-bus, VEOS and RTI are registered trademarks of the dSPACE GmbH. All rights reserved.

Chapter

3

The menu bar and toolbar

As in other Windows applications, the menu bar (see ***Figure 19***) and the toolbar (see ***Figure 20***) offer direct access to many functions of the program.

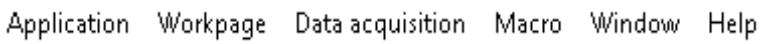


Figure 19 – The menu bar



Figure 20 – The toolbar

Menu item	**Toolbar**	**Chapter**
Application		***3.2 The Application menu***
- New Login…		***3.2.1 New login***

- Load Configuration…		***3.2.2 Load configuration***
- Reset Signals		***3.2.3 Reset signals***
- Restart Model		***3.2.4 Restart model***
- CCP/XCP Configuration		***3.2.5 CCP/XCP configuration***
- Settings…		***3.2.7 Settings***
- Lock User Interface…		***3.2.8 Lock User Interface***
- Exit		***3.2.9 Exit***
Workpage		***3.3 The Workpage menu***
- New Cockpit…		
- Load Cockpit…		
- Save Cockpit		
- Save Cockpit As…		
- New Workpage…		
- Load Workpage…		
- Save Workpage		
- Save Workpage As…		
Data Acquisition		***3.4 The Data Acquisition menu***
- Start Data Acquisition without saving to file		
- Start Data Acquisition with saving to file		
- Stop Data Acquisition		
- Offline		
- Display Data Acquisition from file…		

- Save Data Acquisition as…		
- Convert Data Acquisition…		***3.4.1 Converting data acquisitions***
Macro		***3.5 The Macro menu***
- Record Macro…		
- Stop Macro Recording…	 (if active)	
- Start Macro…		
- Stop Macro		
- Edit Macro		
Window		***3.6 The Window menu***
- Signal Selection		***4.2 The Signal Selection window***
- Workpage		***4 Workpage***
- Test Manager		***5 Test Manager***
- Diagnostics MCD-3 D		***6.2 MCD-3 D diagnostics***
- Diagnostics Caesar		***6.1 CAESAR diagnostics***
- Fault Simulation		***7 Fault simulation***
- Float all		***2.6 Placing the functional areas***
- Dock all		
- PROVEtech:TA Log		***3.6.1 PROVEtech:TA Log***
Help		***3.7 The Help menu***
- Help		
- Homepage		
- Change Request		

- *Technical Support…*		***3.7.1 Technical Support***
- *Update License…*		***2.2.1 Computer-bound licensing***
- *Info*		***3.7.2 The Info dialog***

3.1 The file selection dialog

PROVEtech:TA provides a file selection dialog that extends the normal windows file selection dialog.

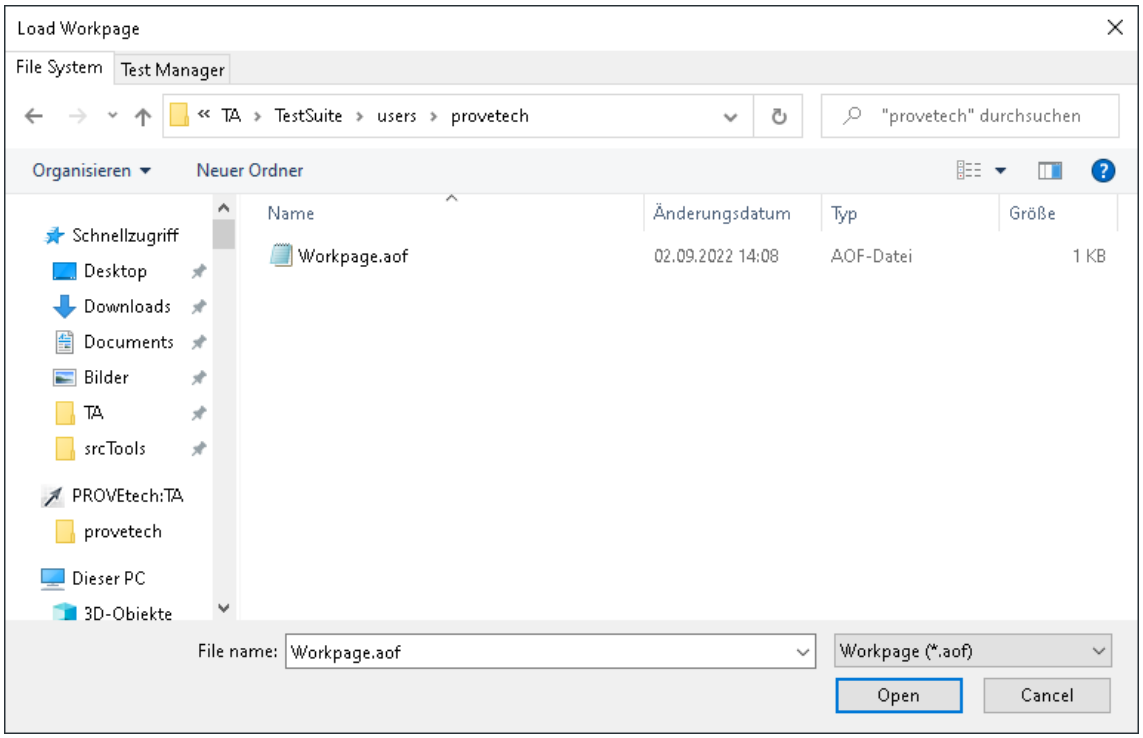


Figure 21 – File selection dialog

The tree on the left has an additional entry representing the user directory of the currently logged in user, in this case “provetech”. So, clicking on the entry directly selects that directory. The user directories are typically in the following folder (example for Windows 10):

C:\Users\Public\Documents\PROVEtechTA\users\

PROVEtech:TA creates a sub directory for each user there. Use these directories to store user specific data like workpages and cockpits.

For workpages, cockpits, hotkey files, additional protocol files and many configuration files there is additionally the property page `Test Manager` in the file dialog. There you can use test documents of the Test Manager tree to store these files. After that, a Test Manager path is stored to reference this test document. So, you can choose between saving your configuration in the file system or in the database.

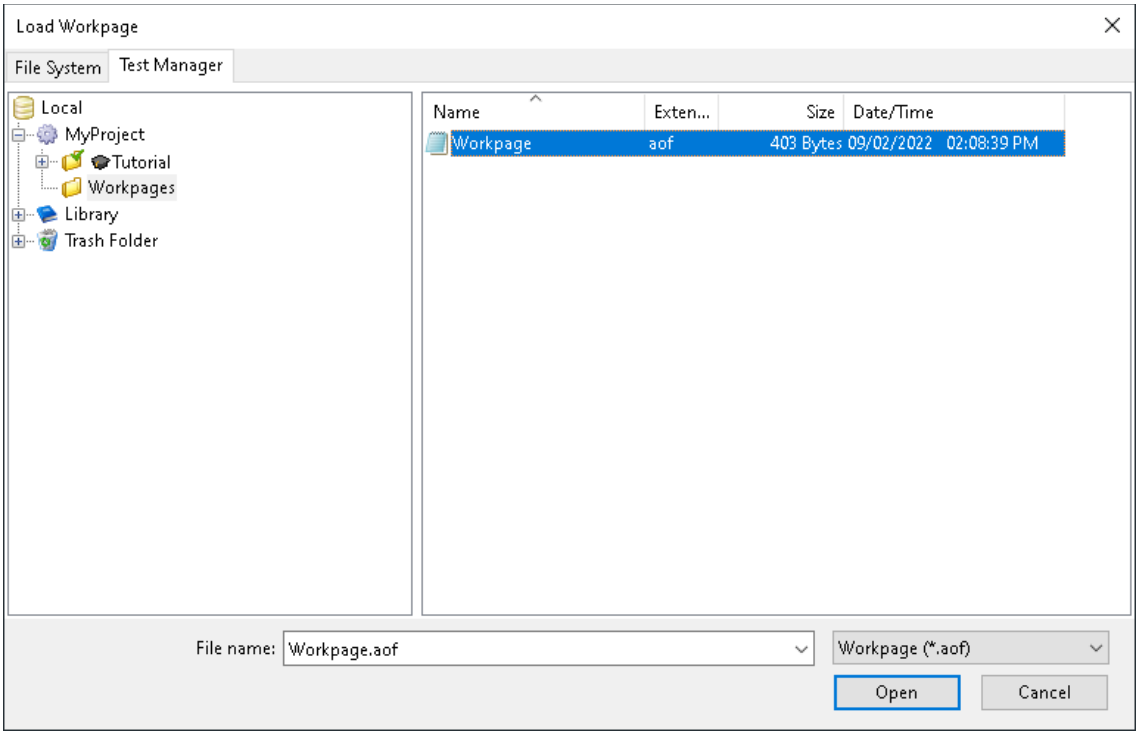


Figure 22 – Test Manager tab of the file selection dialog

Such paths have the following format:

*MyProject:\Workpages*doc*Workpage.aof

3.2 The Application menu

The *Application* menu (see ***Figure 23***) allows you to login as a different user or with a different configuration, to reset all signals, to restart the model, change the settings or to close PROVEtech:TA.

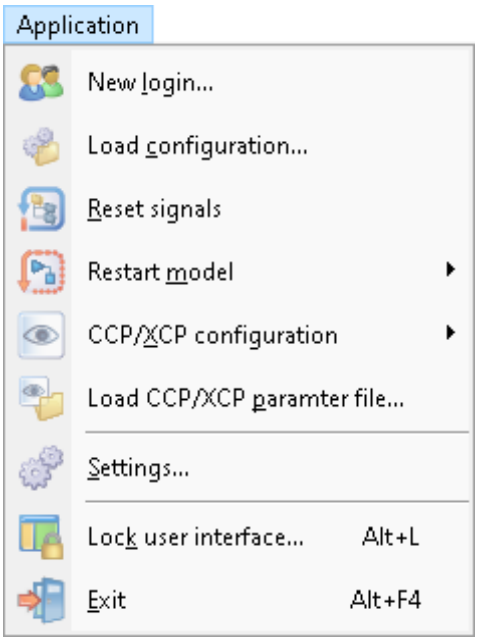


Figure 23 – The Application menu

3.2.1 New login

Select *New Login…* to choose a new user, configuration, model, database connection or language. After selecting *New Login…* the dialog of chapter ***2.3 Login*** appears.



When you select *New Login…* the current user is automatically logged off. So you must log in again to continue working.

3.2.2 Load configuration

Choose *Load Configuration…* to choose a configuration and model. After that the dialog of chapter ***2.3 Login*** appears, where non-relevant controls are disabled.

3.2.3 Reset signals

During a simulation you change values for the input signals of the system. *Reset Signals* returns these signals to their initial values. PROVEtech:TA can only reset the signals for which it knows the initial values. This depends on the used platform and model settings. For TOI (Test & Operating Interface) signals created with the TOI library in Simulink,

PROVEtech:TA always knows the initial values (see directory "<PROVEtechTA_SetupDirectory>\Goodies\TOI Library").

The button  on the toolbar corresponds to the menu item *Reset Signals*.

3.2.4 Restart model

The simulation is started when you log on to the system and select the test system configuration. The simulation typically runs on the real-time computers of the test system. It supplies input signals for the ECUs and process their output signals. They form your test model, which you stimulate by setting parameters for the selected signals.

The models of the vehicle environment and the ECUs interact in the test system in such a way that, ideally, a stable working point is achieved. Loops between models and ECUs can be opened by the program properties of the models, permitting exact setting of the desired states in the model via special model inputs, the control variables. This possibility of directly controlling certain model variables does, however, open up the danger of obtaining implausible simulation conditions. In extreme cases this can cause the model to crash, after which you will need to reset and restart. This function corresponds to a restart of the model, comparable with logging on to the system. All internal parameters and variables will then be reset to their initial status.

If models with different properties are to be managed, this constitutes a special case. On a test system with several models, the *Restart Model* menu item displays a list that allows you to select the desired model (see ***Figure 24***). Otherwise only one option is available for selection.

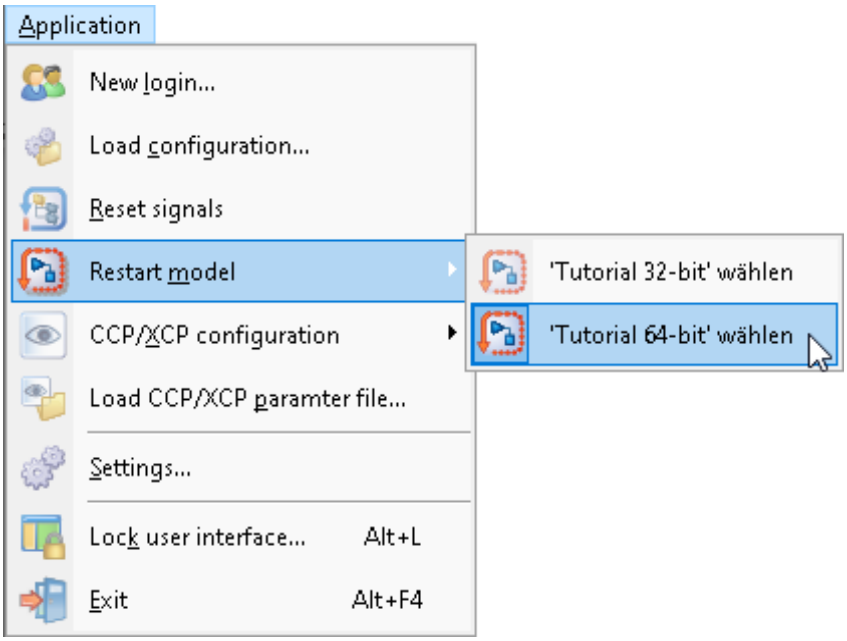


Figure 24 – Selection of one of several models

The button  on the toolbar provides an alternative to this menu item. Click on the arrow to display the above-mentioned list (see ***Figure 25***). To restart the current model, do not click on the arrow but on the button symbol instead.

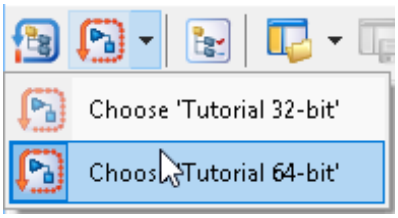


Figure 25 – Selection of a model via the toolbar

3.2.5 CCP/XCP configuration

This menu item is only visible in case you have a CCP or XCP license. You can find more information about CCP and XCP in the chapter ***3.2.7.2.3.13 Configuration of CCP/XCP***.

Every model configuration also has a CCP and XCP configuration which can be empty. The chapter ***3.2.7.2.3 Models*** describes how to configure your models. This is great for a static CCP/XCP configuration fitting to your model. However, there might be cases where you want to modify your CCP/XCP configuration dynamically without switching or re-loading your model. For these cases you can use the popup menu *CCP/XCP Configuration*:

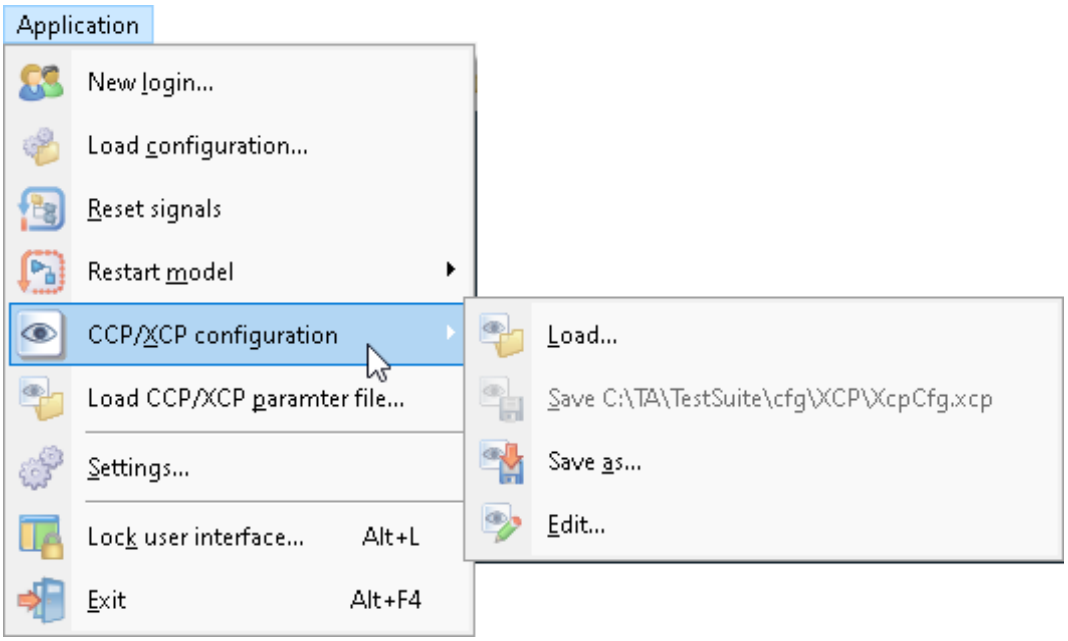


Figure 26 – Popup menu CCP/XCP configuration

The upper menu items *Load <file path>* show the most recently used CCP/XCP configuration files and you can select those menu items to directly load one of them. With *Load…* the file selection dialog (see chapter ***3.1 The file selection dialog***) pops up to select a file which can also be located in the Test Manager database as a test document.

Save <file path> saves your current modifications in the currently loaded file. *Save As…* opens the file selection dialog to choose a new file path for saving. The saved file includes your event channel number changes made in the signal selection window (see chapter ***4.2.5 Changing CCP/XCP event channel numbers***).

Edit… opens a dialog for editing the current CCP/XCP configuration. This dialog is the same as the one described in chapter ***3.2.7.2.3.13 Configuration of CCP/XCP***. However, when clicking `OK` the modifications are directly applied.

All modifications are only active until you load a model. Save your CCP/XCP configuration in a file if you want to use it again later. Model configurations can also reference these files to activate them immediately when loading the model.

3.2.6 Load CCP/XCP parameter file

This menu item is only enabled in case a CCP or XCP instance is running. It opens this dialog:

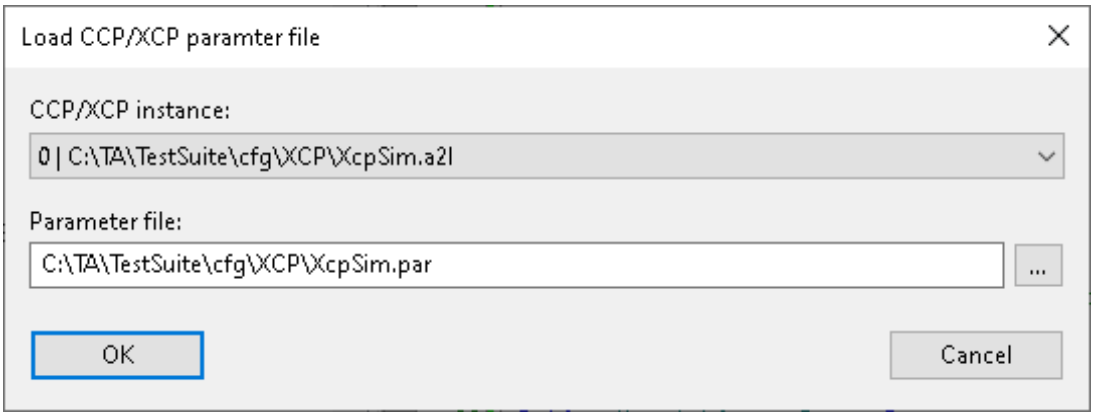


Figure 27 – Load CCP/XCP parameter file

It lets you select a `CCP/XCP instance` (displays the according instance number and .a2l file) and a `Parameter file`. A parameter file is a text file with logical signal values for writable signals. The format is from Vector CANape. The signal names, signal dimensions und data types must match the ones of the .a2l file. Clicking `OK` starts reading the file and writing the values. This can take some time depending on the size and transport layer. Meanwhile, an additional dialog shows up. The dialogs close when writing is done. In case of an error an error message shows up. The PROVEtech:TA Log (see chapter ***3.6.1 PROVEtech:TA Log***) has additional warnings in case a signal of the parameter file does not exist in the .a2l file or if something does not match. While writing, the status signal of the CCP/XCP instance has the value 5.



PROVEtech:TA does not support signals that do not start/end at byte boundaries. It only supports version 3.1 of the parameter file format. It does not work on old dSPACE PHS bus systems.

3.2.7 Settings

The `Settings` dialog contains possibilities to manage common settings, configurations, the workpage settings, data acquisitions, the Test Manager, users, projects, the diagnostics and the fault simulation. You can modify several settings as PROVEtech:TA administrator or manager only as you could destroy important data when using the dialog improperly. Some settings apply immediately when pressing `OK`, for others you have to restart PROVEtech:TA. In this case an according message box gets displayed.

You can modify a lot of file and directory paths. The entry “BasePath” in the section [General] of the file “<PROVEtechTADirectory>\bin\Base.ini” configures the base path for the configuration entry file “TA.ini”. In case this entry or the whole file is missing, the “<PROVEtechTADirectory>\bin” directory is the base path. The ***Administrator’s Guide*** has detailed information about the configuration files.

It is a good idea to specify file paths relative instead of absolute. Relative paths always refer to the path of the file that contains the relative path. Example:
The file “C:\Users\Public\Documents\cfg\FaultSim.ini” contains the relative path “..\data\cab1.csv”. PROVEtech:TA loads “C:\Users\Public\Documents\data\cab1.csv” in this case.

For Windows environment variables in path statements use the following format: %VariableName%. Enter %% to specify the % character.

Storage location of the configuration files

Changes in the Settings dialog are stored in the configuration files after clicking `Save and close`. The storage location of the configuration files are initially predefined by the PROVEtech:TA installation (see ***2.1 The installation***), but you can change them afterwards. The default installation of PROVEtech:TA uses the following paths:

- **<ProgramFiles>\PROVEtechTA\bin:** In addition to the application binaries and the documentation you can find the file “Base.ini” here (see ***Administrator’s Guide***) which contains the base reference to the configuration files. Usually, this area is only writeable for the administrator. So in this directory, you should not locate files that must be changed while running PROVEtech:TA.
- **<CommonDocuments>\PROVEtechTA:** “<CommonDocuments>” typically is „C:\Users\Public\Documents“ since Windows Vista. Every Windows user has write access inside this folder. PROVEtech:TA installs the configuration files here.

You can control the access to the configuration files by using the Windows access rights.

Copy and move of a complete configuration to another computer

In PROVEtech:TA there are no menu items or tools to export and import an entire configuration. The configuration file structure is very flexible and thus a little bit confusing for new users. But if you adhere to the recommended file structure of the PROVEtech:TA installation, moving or copying the configuration is quite simple. Basically you only need to handle the directory **<CommonDocuments>\PROVEtechTA**. In detail, the following directories are sufficient to copy or move:

- <CommonDocuments>\PROVEtechTA\cfg (e.g. configuration of the model and platform)
- <CommonDocuments>\PROVEtechTA\data (e.g. fault simulation files)
- <CommonDocuments>\PROVEtechTA\users (one working sub directory for each PROVEtech:TA user, e.g. for workpage and cockpit files and data acquisitions)

It is important to always use relative paths instead of absolute paths. When sticking to that rule moving the whole PROVEtech:TA configuration even works in case the base directory changes. Just the storage of “recently used” files gets invalid in this case (e.g. last loaded workpage file), but this is not critical.

Also using Windows environment variables might make moving the configuration files easier.

Please notice that moving or copying the PROVEtech:TA configuration might only be one task, but typically you have dependencies to diagnostics and model data which is just referenced by the PROVEtech:TA configuration. This data is also referenced by non-Akkodis software modules, so you have to consider the recommendations of the according suppliers.

Using shared network directories is a good idea for sharing PROVEtech:TA configurations across several computers. In that case exchanging a computer is very easy. You just have to exchange the “Base.ini” file after the installation, which references a network path in that case. However, it is necessary to create the configuration file structure manually once. If some files are very big (e.g. model files or diagnostics data) it is recommended to do a local synchronization of these files. This way starting PROVEtech:TA is much faster as these files are rarely modified and PROVEtech:TA does not load them via the network each time. You can ask the PROVEtech Support (provetech@akkodis.com) for help.

Configuration files are stored in the file system, Test Manager data is stored in the database

Although PROVEtech:TA stores the Test Manager data in a database, the configuration files are by default located in the file system and not in the database. This leads to advantages and disadvantages:

- **Advantage:** On each test system the user can locate the configuration files individually.
- **Advantage:** You can work with PROVEtech:TA even when the database connection is lost or not available.
- **Advantage:** Some third-party components need their configuration in the file system (e.g. diagnostic data).
- **Advantage:** You cannot manage the database connections inside the database, so you would still need some files.
- **Advantage:** Avoiding a mix of configuration in the file system and the database.

- **Advantage:** You can work with several databases without the need to synchronize your configuration between the databases.
- **Disadvantage:** You cannot use the versioning feature of the Test Manager to control your configuration files. First you must import them into the database.
- **Disadvantage:** You cannot use the database backup to back up the configuration; but you can back up most of the configuration by creating a problem report (see ***2.13 Creating a problem report***). Additionally, you can import the files recurrently as test documents into the database.
- **Disadvantage:** The consistency of configuration and Test Manager data is not completely ensured.

3.2.7.1 Common

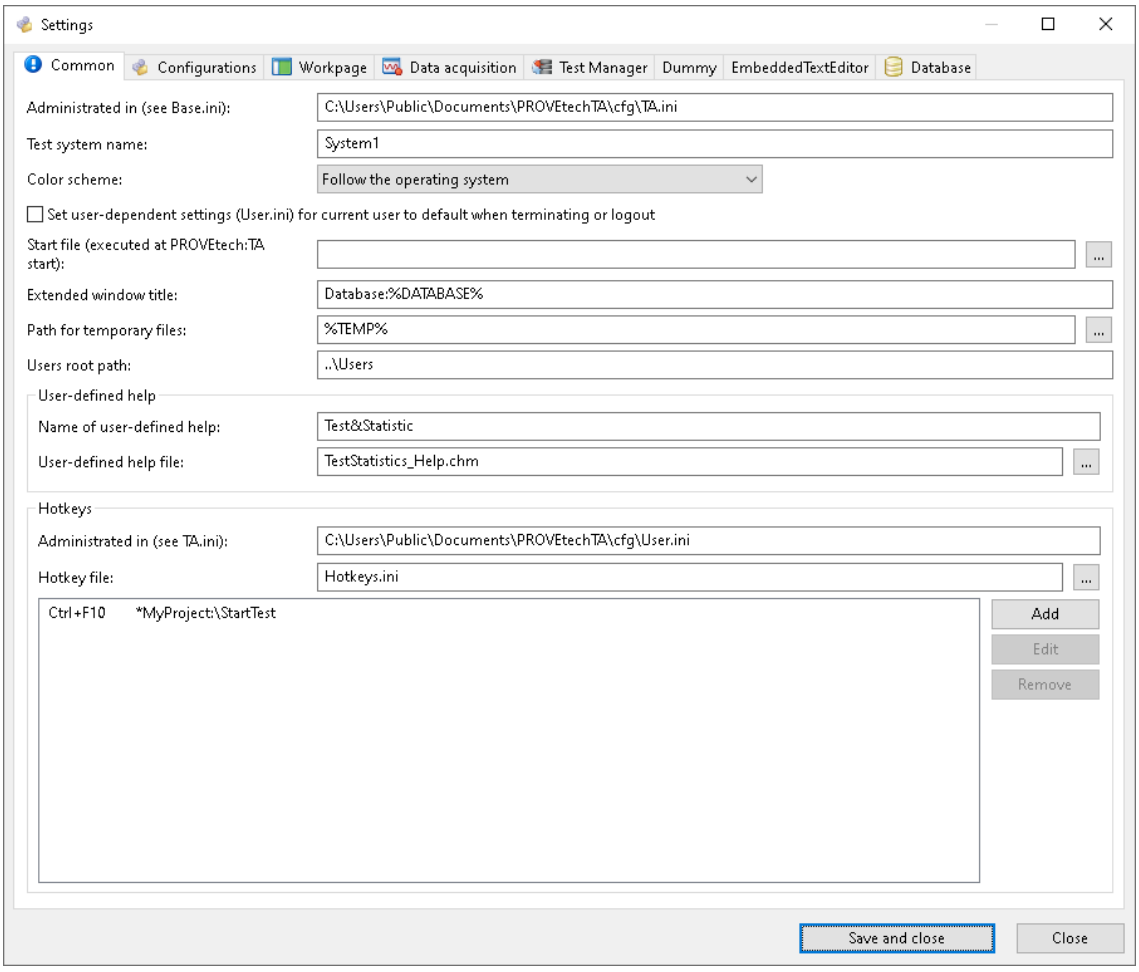


Figure 28 – The Common property page

The `Administrated in` box of the property page `Common` displays the name of the file that contains these settings.

The property page Common contains the following settings:

Test system name: PROVEtech:TA displays this name in the title bar of the main window. Only the PROVEtech:TA administrator can edit the test system name.

Color scheme: Color scheme for PROVEtech:TA, you can choose from:
- Follow the operating system
- Light
- Dark

Remark: In case the high contrast mode is active in Windows, PROVEtech:TA ignores this setting and uses the high contrast mode. In case the Windows version is older than Windows 10 22H2, using the dark mode is impossible. PROVEtech:TA then uses the light mode or, if active, the high contrast mode.

If you want to reset all user-dependent settings (e.g. your recently used workpages and window positions) you can click Set user-dependent settings (User.ini) for current user to default when terminating or logout.

Start file (Executed at PROVEtech:TA start): PROVEtech:TA starts this file at startup (e.g. for special initializations). Leave the text box empty to start no program at startup.

Extended window title: This text appears in addition to the standard PROVEtech:TA window title. You can also use environment variables (format is %VariableName%).

Directory for temporary files: During one session PROVEtech:TA needs a path to store temporary files. All the files are deleted when finishing PROVEtech:TA.

Users root path: In this directory each user gets automatically his / her own directory. Each user can manage the content by himself / herself.

User-defined help: Choose a Name of user-defined help: and a User-defined help file: (.chm file) to add an additional entry in *Help* of the main menu.

In the group box Hotkeys you can specify hotkeys for starting macro files. These hotkey settings are stored user-dependently in the Hotkey file or in a test document in the Test Manager (see chapter ***3.1 The file selection dialog***).



The hotkeys of the General tab are disabled when the focus is in the WinWrap Basic editor.

3.2.7.2 Configurations

Use the property page `Configurations` (***Figure 29***) to manage the entries of the file CFG.ini. The path of this file is displayed in `Administrated in (see TA.ini).`

In case the changed (renamed, copied or deleted) configuration is referenced in the configuration-dependent user settings, the relevant section of the USER.ini is also changed accordingly.

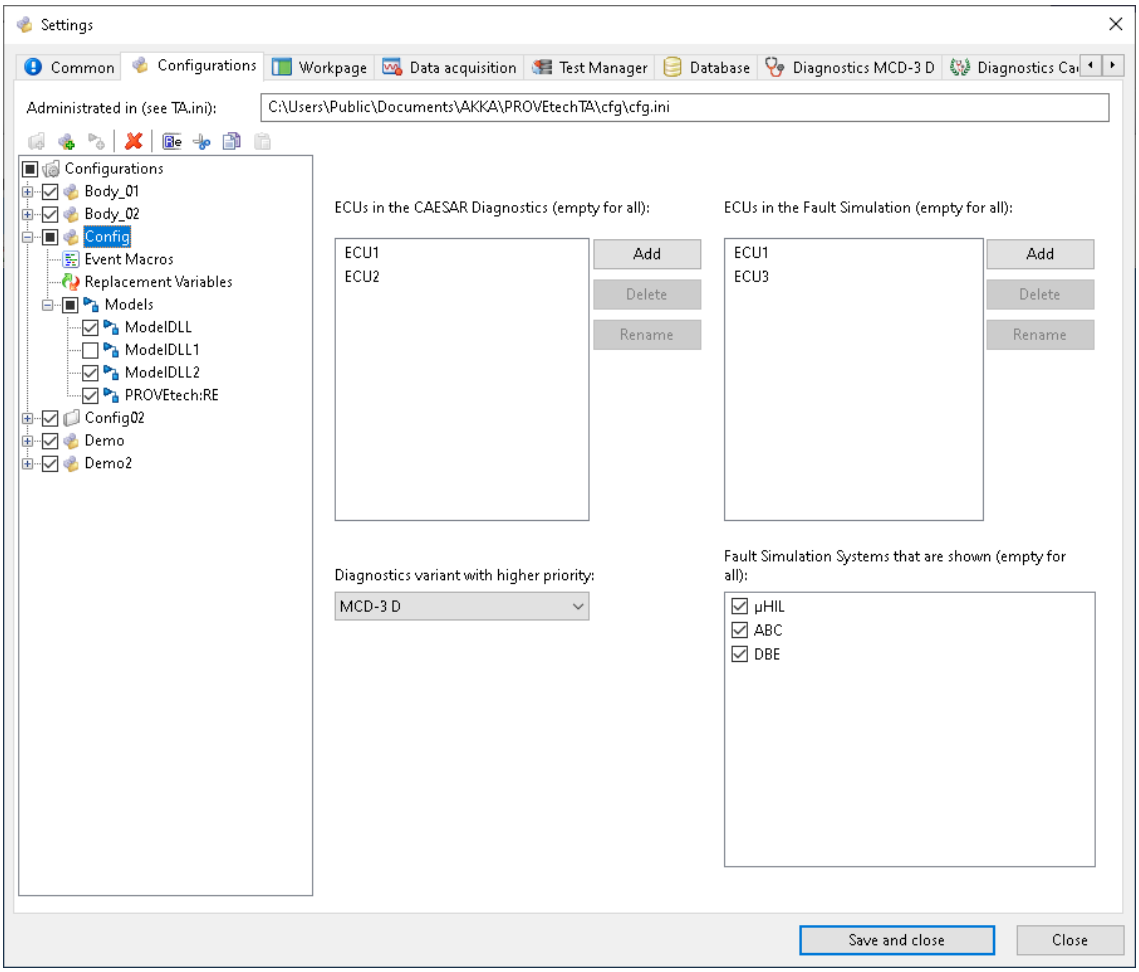


Figure 29 – The Configurations property page

In the configurations tree on the left side, you can create several configurations and each configuration can contain several models. You can arrange and group configurations hierarchically by using configuration groups. You can hide configurations and models in PROVEtech:TA using the checkboxes on the left of the configuration tree. These then only appear in the settings dialog and cannot otherwise be selected.

The toolbar placed on the top of the configurations tree allows to perform several operations on the configurations tree:

Symbol	Hotkey	Operation
	-	**Add new configuration group**: Adds a new configuration group to the selected item in the configurations tree. Available only at root level (Configurations) or configuration groups.
	Ins	**Add new configuration**: Adds a new configuration to the selected item in the configurations tree. Available at root level (Configurations), configuration groups or configurations. If the selected item is a configuration group, the configuration group adds the new configuration as a child in that group. If the selected item is a configuration, the configuration is added as a sibling configuration.
	Ins	**Add new model**: Adds a new model to the selected item in the configurations tree. Available only for 'Models' item or when another model is selected.
	Del	**Delete**: Removes the selected item from the configurations tree, including all their child items. The possible items to be deleted are configuration groups, configurations, and single models. In the latest case, the model can only be deleted if the configuration contains more than one model.
	F2	**Rename**: Renames the selected item in the configurations tree. Rename is active only for configuration groups, configurations and models.
	Ctrl+X	**Cut**: Cuts out the selected item in the configurations tree and places it in the clipboard. The selected item is marked in the configurations tree as marked to cut.
	Ctrl+C	**Copy**: Copies in the clipboard the selected item in the configurations tree.
	Ctrl+V	**Paste**: Pastes the content of the clipboard on the selected item in the configurations tree. The Paste operation is only available for configuration groups and configurations.

The operations on the configurations tree are available in the context menu using the right click on the corresponding tree item.

While logging in (see chapter ***2.3 Login***) or changing the configuration (see chapter ***3.2.2 Load configuration***) the according dialog shows the configuration groups and configurations in the same way as the configurations tree.

When you select a **configuration** you can edit the following settings:

The `ECUs in the CAESAR Diagnostics` box displays the ECUs used for the CAESAR diagnostics (only visible if the CAESAR diagnostics is active). If you do not select particular ECUs, all ECUs for which .cbf files exist are used.

In case you use both diagnostics variants (CAESAR and MCD-3 D) with the same hardware interface, the combo box `Diagnostics variant with higher priority` is relevant for you. As it is not possible to use the same hardware interface from both diagnostics variants in parallel, this combo box selects which diagnostics variant to start when loading this configuration. The other one goes into the shutdown state. However, you can switch the variants at run-time by shutting down the currently started one and starting up the other one after that. The combo box has three entries:

- **None:** This is the default setting. You can use it for all cases beside the one where you use both diagnostics variants with the same hardware interface.
- **CAESAR:** This starts the CAESAR diagnostics and shuts down the MCD-3 D diagnostics when loading this configuration.
- **MCD-3 D:** This starts the MCD-3 D diagnostics and shuts down the CAESAR diagnostics when loading this configuration.

The combo box is only visible in case any of the diagnostics variants is available.

The `ECUs in the Fault Simulation` setting works appropriately (only visible if the fault simulation is active). If you do not set any ECUs in this control, all ECUs defined in the fault simulation configuration files are used. In the fault simulation configuration you have the possibility to rename ECUs (see chapter ***3.2.7.9.4 dSPACE FullSize fault simulation via CAN***). PROVEtech:TA tries to match the entries in the list `ECUs in the Fault Simulation` with the renamed ECU names. In case you want to match with an original name, just prefix the list entry with ~.

In `Fault Simulation Systems that are shown` you have the possibility to choose different Fault Simulation Systems for each configuration (only visible if the fault simulation is active).

You can assign MCD-3 D diagnostics settings to special configurations (only available if the MCD-3 D diagnostics is active, see also ***3.2.7.8 MCD-3 D diagnostics***).



When creating or duplicating a configuration please consider that common files for cockpit and workpage will be used. This means, a change of the cockpit or workpage will be propagated to the cockpit or workpage in other configurations using the same files. This can lead to data loss. To avoid this, you can create an individual copy of cockpit and workpage files in any configuration (see ***3.3 The Workpage menu***).

The configuration group name and the configuration name cannot contain the full stop character (‘.’). The model name cannot contain the pipe character (‘|’).

3.2.7.2.1 Event macros

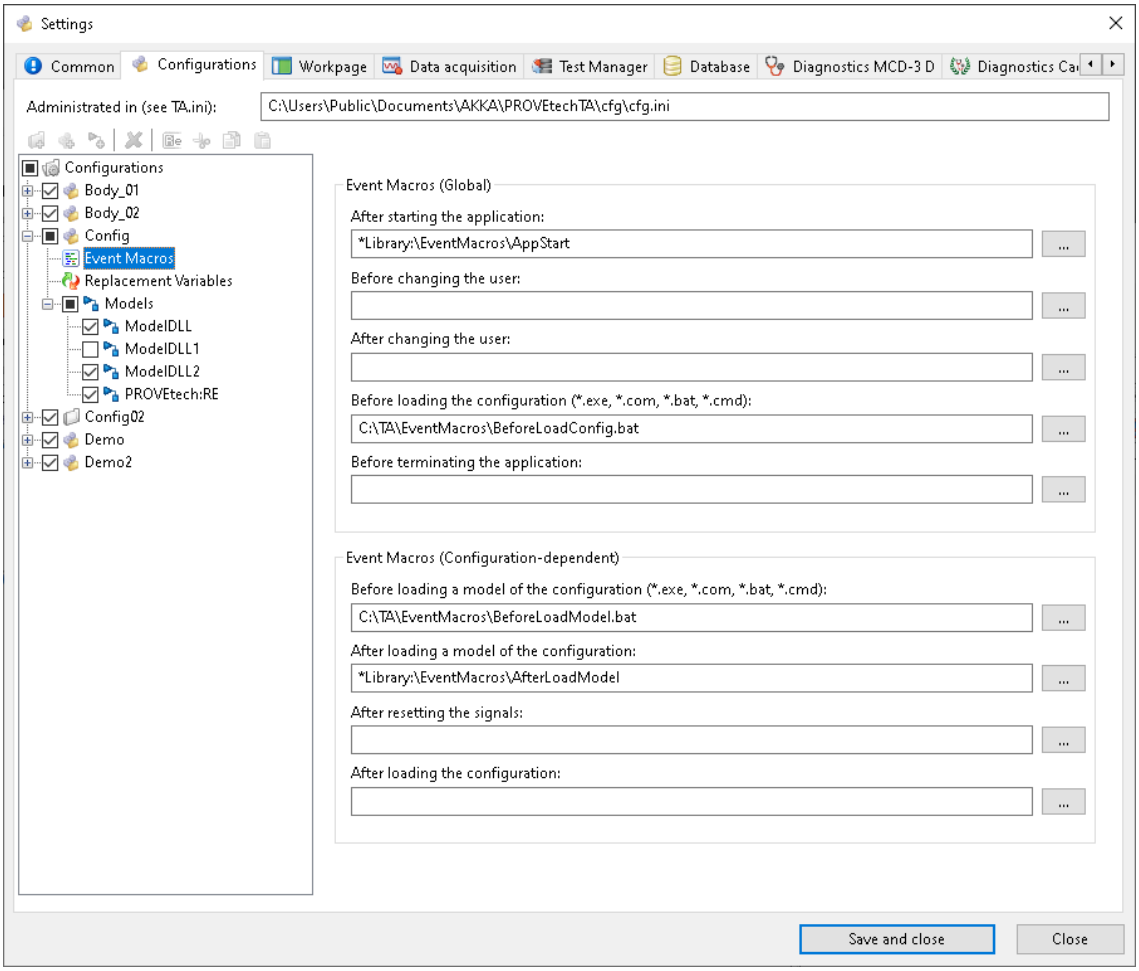


Figure 30 – Event macros

Event macros are test macros that are executed when the corresponding event occurs. The macros may be located in the Test Manager tree or in the file system. Global event macros apply to all configurations. When using configuration-dependent event macros you can specify different paths for different configurations. Several events may occur consecutively, in which case the event macros are executed consecutively.

Here follows an overview of each single **global event macro**:

After starting the application
This event macro is executed as one of the last actions when starting the application. If an error occurs, it will be displayed and the process will continue.

Before changing the user
This event macro is executed before the logon dialog is displayed to switch to a different user. If an error occurs, it will be displayed and the process will be canceled.

After changing the user
This event macro is executed as one of the last actions after logging on as a different user, that means, after the logon dialog. If an error occurs, it will be displayed and the process will continue.

Before loading a configuration (.exe, *.com, *.bat, *.cmd)*
The provided executable (.bat batch file or .com, .cmd, or .exe file) is triggered just before loading a configuration in PROVEtech:TA. PROVEtech:TA automatically attaches the configuration name and the model name as command line parameter after the provided executable file path. **Caution:** Running WinWrap Basic macros is not allowed for this event.
If you have for example set "C:\MyFolder\MyBatch.bat" and you want to load the model „NewModel“ of the configuration „CurrentConfig“, then the following shell command is executed:

"C:\MyFolder\MyBatch.bat" "CurrentConfig" "NewModel"

Inside your batch file you access the command line parameters by %0-%9. %1 contains "CurrentConfig" and %2 respectively "NewModel".
In addition, all global replacement variables are provided to the new process as Windows environment variables. The environment variables get the prefix *_REP_VAR_* (see chapter ***3.2.7.2.2 Replacement variables***).

If an error occurs when executing the event macro, or if the event macro returns an error code other than 0, the entire operation is aborted and the configuration is not loaded. The following error codes are treated as an exception:

• 901-905: The operation continues, but no connection to the model is established and no diagnostics or fault simulation data is loaded; this can be done later

• 906-910: The error code is ignored (for possible future extensions)

Before terminating the application
This event macro is executed as one of the first actions after asking the user to save his changes. If an error occurs, it will be displayed and the process will continue.

Here follows an overview of each single **configuration-dependent event macro**:

Before loading a model of the configuration (.exe, *.com, *.bat, *.cmd)*
The provided executable (.bat batch file or .com, .cmd, or .exe file) is triggered just before loading a model in PROVEtech:TA. PROVEtech:TA automatically attaches the configuration name and the model name as command line parameter after the provided executable file path. **Caution:** Running WinWrap Basic macros is not allowed for this event.
If you have for example set "C:\MyFolder\MyBatch.bat" and you want to load the model „NewModel“ of the configuration „CurrentConfig“, then the following shell command is executed:

"C:\MyFolder\MyBatch.bat" "CurrentConfig" "NewModel"

Inside your batch file you access the command line parameters by %0-%9. %1 contains "CurrentConfig" and %2 respectively "NewModel".
In addition, all configuration specific replacement variables are provided to the new process as Windows environment variables. The environment variables get the prefix *_REP_VAR_* (see chapter ***3.2.7.2.2 Replacement variables***).

If an error occurs when executing the event macro, or if the event macro returns an error code other than 0, the error message is displayed in the PROVEtech:TA Log and then the configuration is still set. The following error codes are treated as an exception:

• 901-905: The operation continues, but no connection to the model is established and no diagnostics or fault simulation data is loaded; this can be done later

• 906-910: The error code is ignored (for possible future extensions)

After loading a model of the configuration
This event macro is executed just after successfully loading the model, corresponding workpage and cockpit. If the model was not reloaded (due to an error or it was already loaded) then the event macro is not executed. You can load the model when starting the application, changing the user, changing the configuration manually, reloading the model manually or by test language.

If an error occurs, it will be displayed and the process will continue.

After resetting the signals
This event macro is executed after resetting the signals, the workpage and the cockpit to its default signal values. You can do this manually by main menu or automatically by test language. When restarting the model this event macro is not executed.

If an error occurs, it will be displayed and the process will be canceled.

After loading the configuration
This event macro is executed as one of the last actions when loading the configuration. You can load the configuration when starting the application, changing the user, changing the configuration manually or by test language.

If an error occurs, it will be displayed and the process will continue.



There is also a handler mechanism before a test object is executed. Please read chapter ***5.5.5 Pre-process the next test object for execution*** and chapter ***3.2.7.5 Test Manager***.



No test result is generated in the Test Manager when an event macro is executed.

Examples:

a) The application is started, a user logs on, the configuration and the model is loaded. After login dialog the following event macros are executed in this order if specified:

1) *Before loading a configuration*
2) *Before loading a model of the configuration*
3) *After loading a model of the configuration*
4) *After loading the configuration*
5) *After starting the application*

b) One user logs out, a different user logs on and loads a new configuration with a new model.
The following event macros are executed in this order, if specified:

1) *Before changing the user*
2) *Before loading a configuration*
3) *Before loading a model of the configuration*
4) *After loading a model of the configuration*
5) *After loading the configuration*
6) *After changing the user*

c) The application is terminated.
After asking the user to save his data, the following event macros are executed in this order if specified:

1) *Before terminating the application*

3.2.7.2.2 Replacement variables

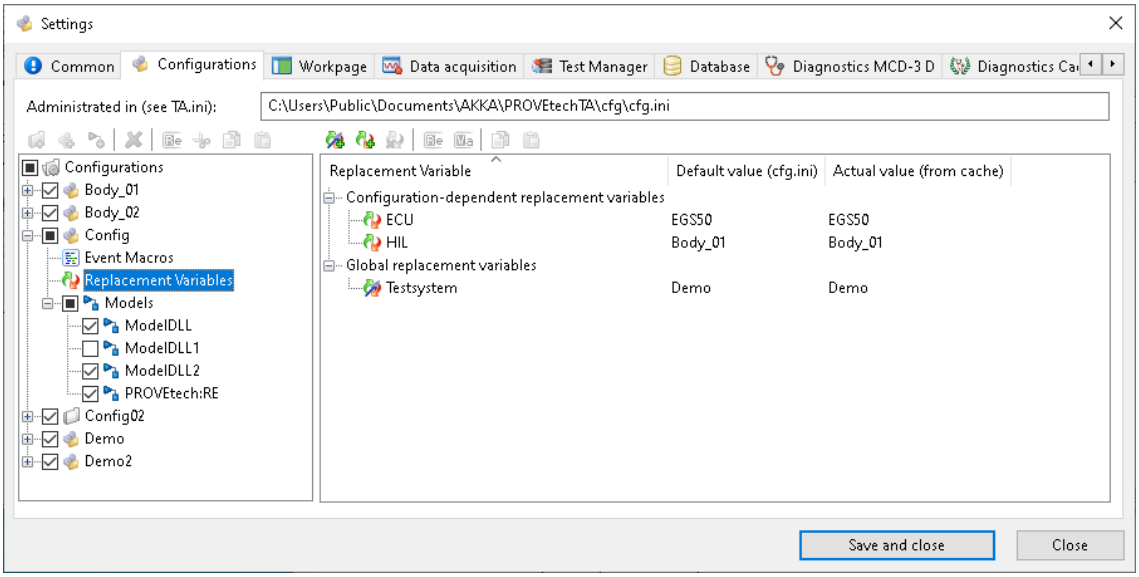


Figure 31 – Replacement variables

With the aid of replacement variables you can construct your Test Manager paths in a variable way (global or configuration-dependent). E.g. in '#uses-statements or in ***TM.Execute*** commands, just use the variable name in angle brackets <*VariableName*>. It is replaced by its value. This provides you to include library functions in your test script depending on the current configuration.

The values of replacement variables can embed further replacement variables again (surrounded by angle brackets <*Variable_2*>). Furthermore, you can use the content of environment variables, preceded and followed by the character %, p.e. *%USERNAME%*.

A global variable is valid for any configuration, except it is overwritten by a configuration-dependent variable of the same name.

The replacement variables can be read and also temporarily set or changed by the script language. All temporarily changed variables will be reset to the original values at (re)loading a configuration, also when editing the replacement variables in the configuration window resets them.

The dialog shows the replacement variables and their values, which are currently stored in the cfg.ini. For the presently loaded configuration, however, these may differ from the current values, since these are already loaded and held in the working memory when PROVEtech:TA is started. In addition, these can also be temporarily overwritten (see Test Language manual, ***System.SetReplacementVar***). Therefore, an additional column for the values from the working memory is displayed for the currently loaded configuration.

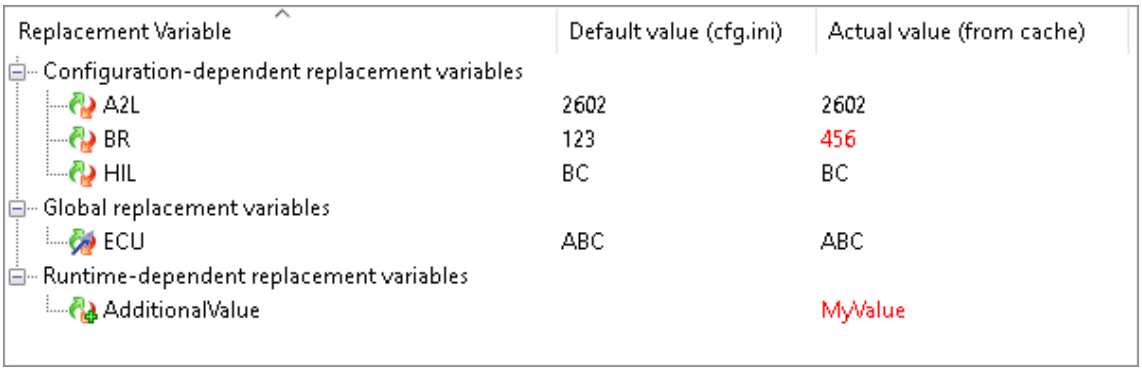


Figure 32 – Replacement variables for the currently loaded configuration with additional column

3.2.7.2.3 Models

Select a model in the configuration tree below the node “Models” to change its settings:

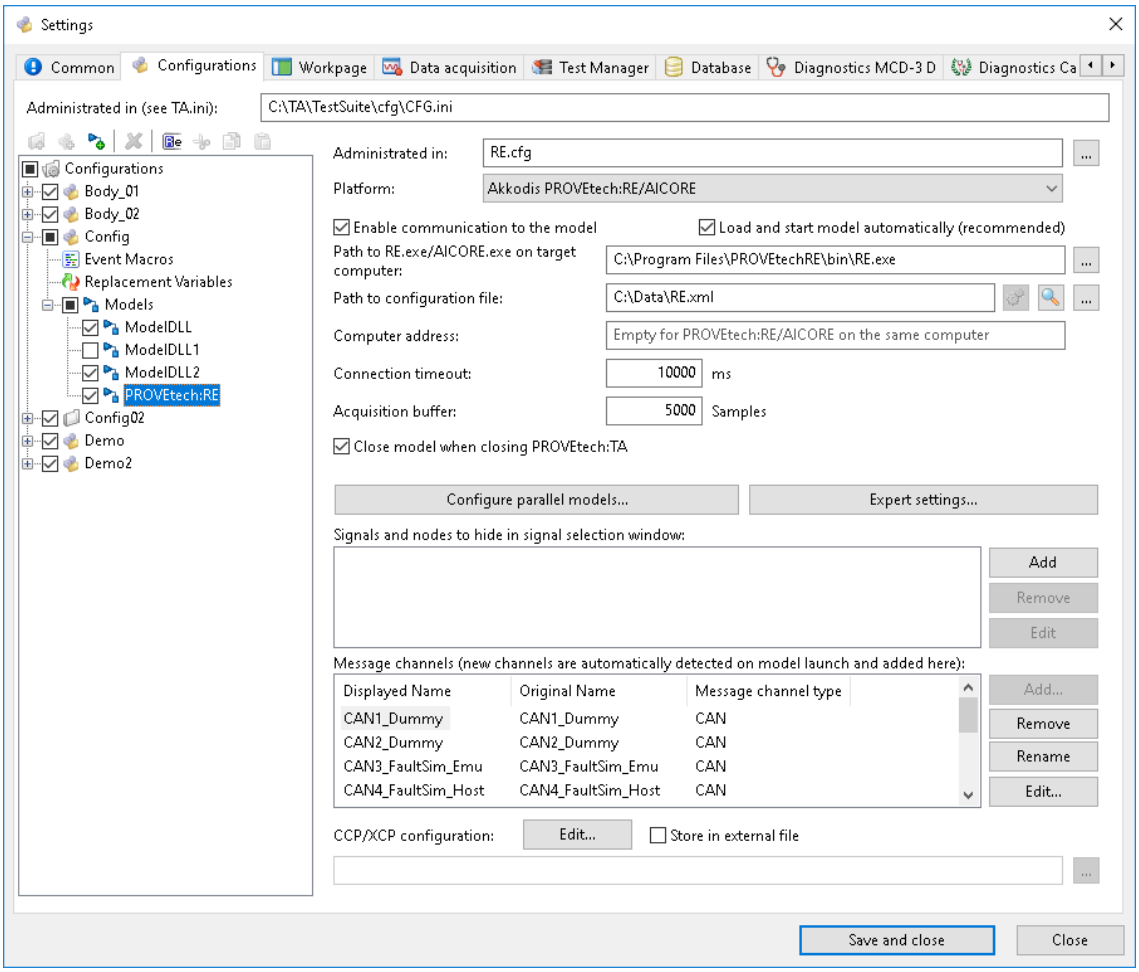


Figure 33 – The model in the tab Configurations

The settings of the page displayed in ***Figure 33*** are stored in a separate file. You can select the path to this file via the edit box `Administrated in`.

Use the combo box `Platform` to select the platform where your model runs. The lower paragraphs describe the settings for the different platforms.



PROVEtech:TA needs a software module called ModelCom in the model for communicating with it. You must link it to every model. This works different for every platform type. On the PROVEtech:TA installation medium you can find the folder “Goodies\ModelComAndRTAE”. There is a sub folder for each platform type within that folder containing a file “ReadMe.txt” with detailed instructions how to do it.

For real-time tests you need the Real-Time Automation Engine (RTAE). Its installation works the same way.



Security information: The communication with the model works via network in many cases. Please use an exclusive network subnet for that. The firewall settings must ensure that only PROVEtech:TA can access ModelCom (and also ModelLauncher for PROVEtech:RE/AICORE). The ModelCom module is optimized for the highest possible performance thus directly accesses the memory of the model process. Invalid usage of ModelCom by foreign software over the network might harm the system and inject viruses as an example. See also chapter *8.1.2 Problem analysis concerning the communication via TCP/IP (includes dSPACE SCALEXIO)*.

Click on `Configure Parallel models…` to run one or several Akkodis PROVEtech:RE, Akkodis AICORE, National Instruments LabView or Generic TCP as additional asynchronous model nodes in parallel to your model configured on this page. It opens the dialog described in chapter ***3.2.7.2.3.15 Configuration of parallel running models***. This does not work for all model constellations and only in case the model communication is active (needs a test system license). The button is disabled in case it is not possible.

The button `Expert settings…` opens another window with a list of additional expert settings. The displayed name of each setting corresponds to the name in the .cfg file referenced by `Administrated in`. The ***Administrator’s Guide*** explains each setting. You can click on the values in the window to edit them, or by selecting an entry and using the F2 key. Even if you acknowledge the window with `OK`, saving the changes only happens by clicking on `Save and close` on the parent window.

The list `Signals and nodes to hide in signal selection window` exists for all platform types. This list specifies signals and nodes you rarely need so you can hide them in the signal selection window. Nevertheless you can always access the signals via the test language and you can show them in the signal selection window by

selecting the menu item *Show Hidden Signals* in the context menu (see chapter ***4.2.1 Signal area***).

The list `Message channels` also exists for all platform types. However, it only contains entries when your model platform supports the handling of messages in combination with PROVEtech:TA. On the first connection to such a model, PROVEtech:TA automatically detects the message channels and adds them to this list. In case a message channel does not exist anymore when loading the model again, the list displays it gray. Select it and click on `Remove` to delete its configuration permanently. The left column of the list represents the name displayed in dialogs and used in the test language. Click `Rename` to change it. The right column is the original name provided by the model.

In case you have disabled the model access (check box `Enable communication to the model`) or you do not have the license for accessing the model (desk license), the `Add...` button is enabled and you can manually add message channels of all the supported types. This is useful in case you want to set up filters or program tests at your desk before going to the test system.

Select a channel and click `Edit...` or use the double click to open a dialog for configuring the channel. Depending on the channel type you can parameterize different information: For CAN channels it is a .dbc or AUTOSAR-XML file and other information (see chapter ***3.2.7.2.3.12 Configuration of CAN channels***). For LIN channels you can select a .ldf or AUTOSAR-XML file, for FlexRay an AUTOSAR-XML or FIBEX file, for MOST channels a MOST Function Catalog file and for generic channels you can configure names for user defined Ids. For IP/Ethernet channels you can select an AUTOSAR-XML file. If you additionally specify the path for .proto files, the message window switches to a mode for SOME/IP where the payload uses Google Protocol Buffers. PROVEtech:TA searches all sub directories of the specified path for .proto files and matches the contained messages to the AUTOSAR-XML SOME/IP PDUs by name.



The message processing feature of PROVEtech:TA depends on its support by the platform and the model. The ModelCom module linked to the model provides an interface for adding message channels. Additionally, the model needs a direct access to the message hardware for getting the messages.

The fields at `CCP/XCP configuration` provide the possibility to control ECUs via the CCP/XCP protocol for accessing ECU internal signals. Click on the nearby `Edit...` button to open the dialog described in chapter ***3.2.7.2.3.13 Configuration of CCP/XCP***. The CCP/XCP configuration is typically stored in the same file as the rest of the model configuration referenced via the edit box `Administrated in`. Activate `Store in external file` to save it in a different file and just reference it by the model

configuration. The lower edit field specifies the according file path. The files correspond to the ones handled by the main menu; see chapter ***3.2.5 CCP/XCP configuration***.



The CCP/XCP feature requires an additional license, so the button is disabled in case you do not have the according license. This feature also does not work in case your `Hardware type` is “DLL”.

The base technology of PROVEtech:TA is independent from the used platform. Nevertheless not all platforms support all features of PROVEtech:TA. Limitations can have the following reasons:

- The platform does not have the according technology at all, e.g. Simulink has no direct support for CAN/LIN/etc. messages thus you cannot send/receive messages only with a Simulink DLL.
- The platform does not provide the necessary public API to get the according access on the target computer, e.g. no API for spying all FlexRay messages. In this case it could be possible to add the according feature if the producer of the platform and Akkodis can agree on the usage of an according API.
- The connection to the real-time computer is too slow to transport all the data, e.g. you cannot transport the data of a 10 MBit/s FlexRay bus via a 5 MBit/s connection.
- The feature is not implemented yet on the according platform as nobody has asked for it yet.

Accessing signals provided by the platform works on all platforms. Limitations always have to do with accessing raw messages. This concerns the functionality of the message window (see chapter ***4.4.14 The message window***) and CCP/XCP (see chapter ***3.2.7.2.3.13 Configuration of CCP/XCP***) among other things.

The description of the platform dependent settings follows. These chapters also contain information about possible limitations.

3.2.7.2.3.1 Akkodis PROVEtech:RE/AICORE

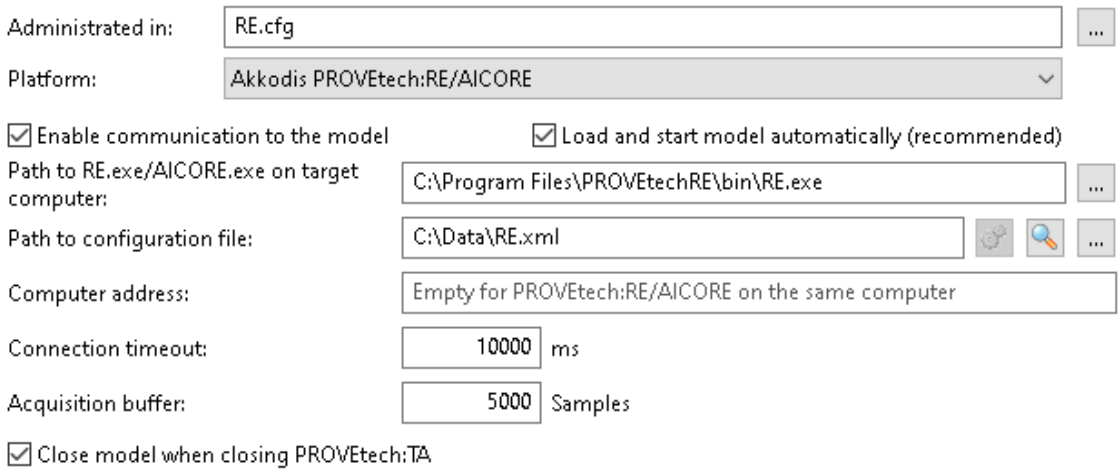


Figure 34 – Property page for PROVEtech:RE/AICORE

In case you use the PROVEtech:RE or AICORE platform, you always need a PROVEtech:RE or AICORE installation accessible via PROVEtech:TA even when using the desk mode without hardware access.

The check box `Enable communication to the model` determines if PROVEtech:TA is in the test system or desk mode. When you deactivate this check box, you can still access all the signal names to program tests in the office without test system hardware (see chapter ***3.2.7.2.3.16 Simulation mode***). In case you have no test system license, you cannot activate this check box.

`Path to RE.exe/AICORE.exe on target` is the file path to the executable file RE.exe of PROVEtech:RE or AICORE.exe of AICORE on the computer referenced via `Computer address`.

The text box `Path to configuration file` specifies the .xml configuration file loaded by PROVEtech:RE at startup. The button with the gears icon opens this .xml file with the PROVEtech:RE configuration tool. This only works in case the local computer can access the specified path. The button is only available in case the relative path "..\cfg\cfg.exe" starting from the RE.exe path exists. The button with the magnifying glass icon opens the .xml file with the default application assigned to the .xml file extension.

For AICORE it is an .ini file instead.

`Computer address` specifies the address of the PROVEtech:RE/AICORE computer (which might be the same that runs PROVEtech:TA – just leave the field empty in that case). The PROVEtech:RE/AICORE computer must also permanently run "ModelLauncher" installed together with PROVEtech:RE/AICORE used by PROVEtech:TA to launch and terminate models.

In case PROVEtech:TA and PROVEtech:RE/AICORE run in a Windows Remote Desktop Services environment (also known as Windows Terminal Services), you should run ModelLauncher with the -l command line parameter. In that case please do not specify anything for the field `Computer address` as several PROVEtech:RE/AICORE and ModelLauncher processes with different local computer addresses run at the same time and automatically assign the correct address to PROVEtech:TA.

The edit box `Connection timeout` specifies the number of milliseconds to try to open the command connection to the model after starting it. It might be necessary to increase this in case it takes a long time after the startup until it really runs, e.g. for initializing the model configuration.

The edit box `Acquisition buffer` specifies the number of samples to store temporarily on the model computers while data acquisitions are running. A small value can cause abortions of data acquisitions because of buffer overruns. The bigger the value, the more memory data acquisitions need on the model computers.

Normally HIL models continue running when you close PROVEtech:TA. However, in this environment closing the model might be desired. To do this activate the check box `Close model when closing PROVEtech:TA`.

Deactivate the check box `Load and start model automatically (recommended)` only in special occasions. In case it is deactivated PROVEtech:TA always tries to connect an already running model. In this case you must load and start the model via a different mechanism.

Only for AICORE the dialog shows the field `Video transmission`. There you optionally specify a video source to transmit to AICORE for image recognition (see also chapter ***3.2.7.3 Workpage*** for the configuration of video sources). This way both tools can work with the same video in parallel. Several AICORE instances can analyze different video sources in parallel.



PROVEtech:RE supports all PROVEtech:TA features.

3.2.7.2.3.2 Akkodis Offline Information File

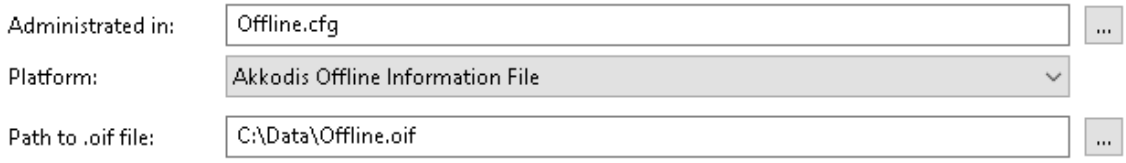


Figure 35 – Property page for Offline Information File

This platform type is a possibility to work without HiL hardware but with the signal information, e.g. for creating test programs. You specify the path to a file created with PROVEtech:TA on a real HiL system with a real model in the field `Path to .oif file`. To create the file use the context menu of the signal selection window, see chapter ***4.2.1 Signal area***.

Working with an .oif file corresponds to deactivating the check box `Enable communication to the model` for other platform types. However, there you always need the original model files and in case of PROVEtech:RE its installation. For National Instruments VeriStand this variant even does not exist. Using an .oif file is an alternative.

3.2.7.2.3.3 Opal-RT RT-LAB

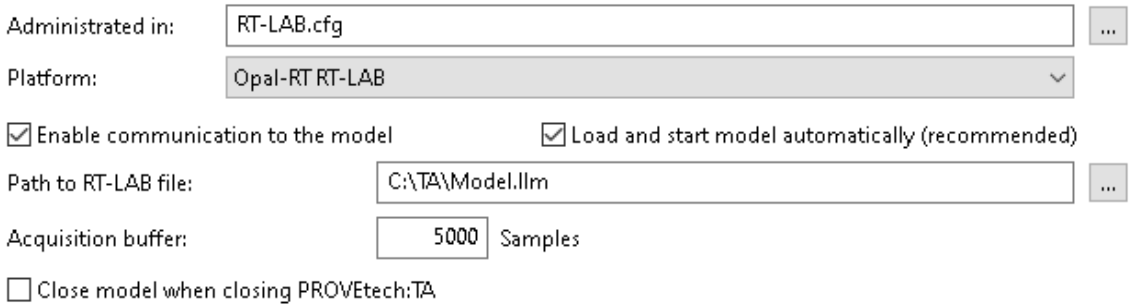


Figure 36 – Property page for RT-LAB

Activate the check box `Enable communication to the model` when your system has a model computer. When you deactivate this check box, you can still access all the signal names to program tests in the office without a model computer (see chapter ***3.2.7.2.3.16 Simulation mode***). In case you have no test system license, you cannot activate this check box.

Deactivate the check box `Load and start model automatically (recommended)` only in special occasions. In case it is deactivated PROVEtech:TA always tries to connect an already running model. In this case you must load and start the model via a different mechanism.

`Path to RT-LAB file` is the file path to the RT-LAB model configuration with the extension .mainctrl (old RT-LAB version) or .llm (newer RT-LAB version).

The edit box `Acquisition buffer` specifies the number of samples to store temporarily on the model computer(s) while data acquisitions are running. A small value can cause abortions of data acquisitions because of buffer overruns. The bigger the value, the more memory data acquisitions need on the model computer(s).

Activate the check box `Close model when closing PROVEtech:TA` to terminate the model at PROVEtech:TA shutdown.



This platform currently only allows accessing CAN raw messages and no other raw message types. For CAN channels the message type “Remote” is not available, thus selecting it has no effect. For the message type “Event” only ErrorFrame events are available. CCP/XCP on CAN, UDP and TCP works completely.

3.2.7.2.3.4 ETAS LabCar

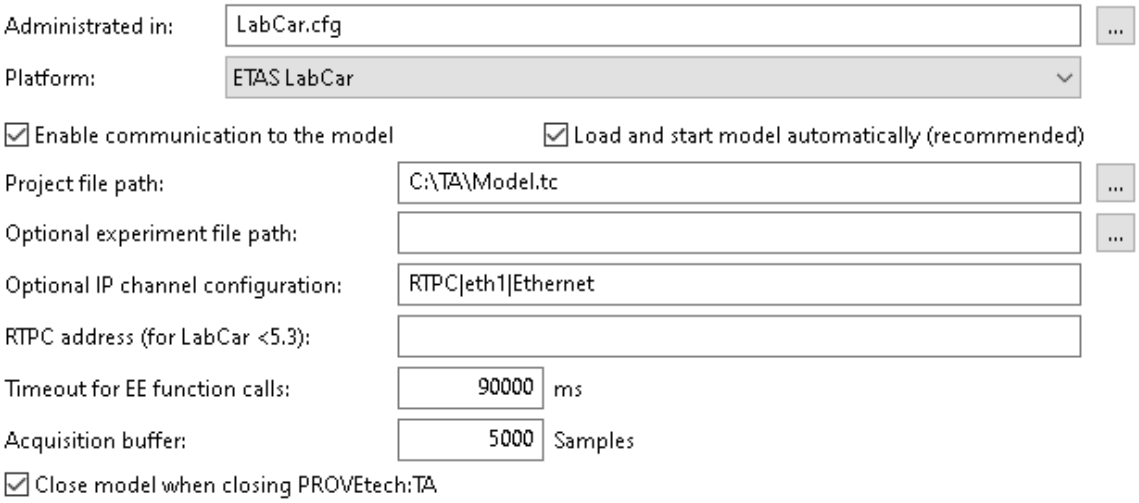


Figure 37 – Property page for LabCar

Activate the check box `Enable communication to the model` when your system has a model computer. When you deactivate this check box, you can still access all the signal names to program tests in the office without a model computer (see chapter ***3.2.7.2.3.16 Simulation mode***). In case you have no test system license, you cannot activate this check box.

Deactivate the check box `Load and start model automatically (recommended)` only in special occasions. In case it is deactivated PROVEtech:TA always

tries to connect an already running model. In this case you must load and start the model via a different mechanism.

`Project file path` is the path to the LabCar project file.

`Optional experiment file path` specifies an .eex file to use when loading and staring the model in LabCar (e.g. for loading additional parameter sets). In case you do not specify this field, PROVEtech:TA creates a new empty experiment.

To activate monitoring of UDP raw messages, please enter a string in the field `Optional IP channel configuration` (IP = Internet Protocol). It is a two dimensional list. A special character separates the list elements. For the first dimension it is the asterisk * and for the second dimension it is the vertical line |. The first dimension has an element per model node on which the monitoring shall run. Per model node there is a | separated list beginning with the model node name and then the alternating an Ethernet port name and the according PROVEtech:TA channel name:

Node1|Ethernet1|Channel1|Ethernet2|Channel2|…*Node2|Ethernet3|Channel3|Ethernet4|Channel4|…*Node3|…*…

Here "…" means any further extension of the list. The names "Node?" correspond to those in the upper specified project file. "Ethernet?" corresponds to the typical interface name of Linux on the according RTPC, e.g. "eth1". "Channel?" is the name in PROVEtech:TA displayed by the message window as an example. See chapter ***4.4.14 The message window***. It is possible to combine several Ethernet ports to the same channel in PROVEtech:TA on the same node by specifying the same channel name.

`RTPC address (for LabCar <5.3)` specifies the IP address or name of the LabCar RTPC. This setting is not necessary anymore for LabCar version 5.3 or later.

PROVEtech:TA starts models via the COM interface of ETAS EE (Experiment Environment). `Timeout for EE function calls` defines the maximum time in milliseconds PROVEtech:TA shall wait for an answer per function call. In case the time is up PROVEtech:TA cancels it with an error message.

The edit box `Acquisition buffer` specifies the number of samples to store temporarily on the model computers while data acquisitions are running. A small value can cause abortions of data acquisitions because of buffer overruns. The bigger the value, the more memory data acquisitions need on the model computers.

Normally HIL models continue running when you close PROVEtech:TA. However, in this environment closing the ETAS Experiment Environment and this way also the model might be desired. To do this activate the check box `Close model when closing PROVEtech:TA`.

This platform currently only allows accessing CAN, LIN and UDP (only monitoring and any IP defragmentation is limited to 20 fragmented UDP packets in parallel and correct order of the fragments) raw messages and no other raw message types. For CAN channels the message types "Event" and "Remote" are not available, thus selecting it has no effect. For CAN Tx messages there is no acknowledgement when and if the message was physically sent. Instead of this the SW timestamp represents the time when the driver got the message and the HW timestamp is always 0. For LIN and UDP the Rx/Tx differentiation is not available – all messages are Rx. CCP/XCP on CAN, UDP and TCP works completely.

3.2.7.2.3.5 Simulink DLL

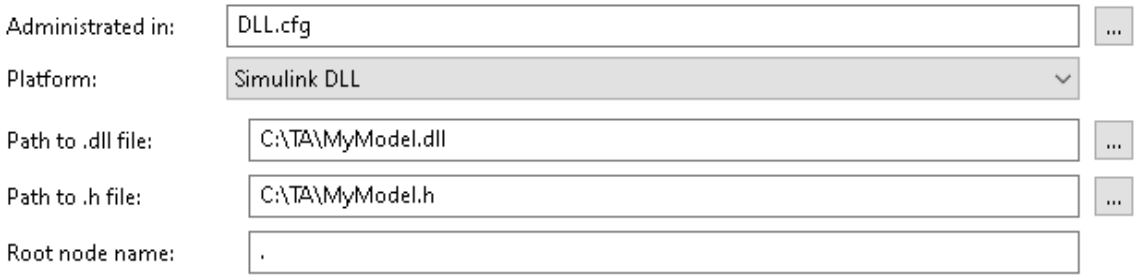


Figure 38 – Property page for DLL

`Path to .dll file` specifies the file path to the compiled model, a result of the Simulink build process. The C header file specified via `Path to .h file` is also a result of the Simulink build process. The file normally resides in a sub directory of the model directory; its name is the same as the model's name.

You can name the root node in the signal hierarchy via the text box `Root node Name`. Use the empty string to remove the root node or the dot "." to use the model's name for the root node name.

Read the chapter ***3.2.7.2.3.16 Simulation mode*** for some more information.

Any raw message access and CCP/XCP are not available on this platform.

3.2.7.2.3.6 Simulink Real-Time for Speedgoat

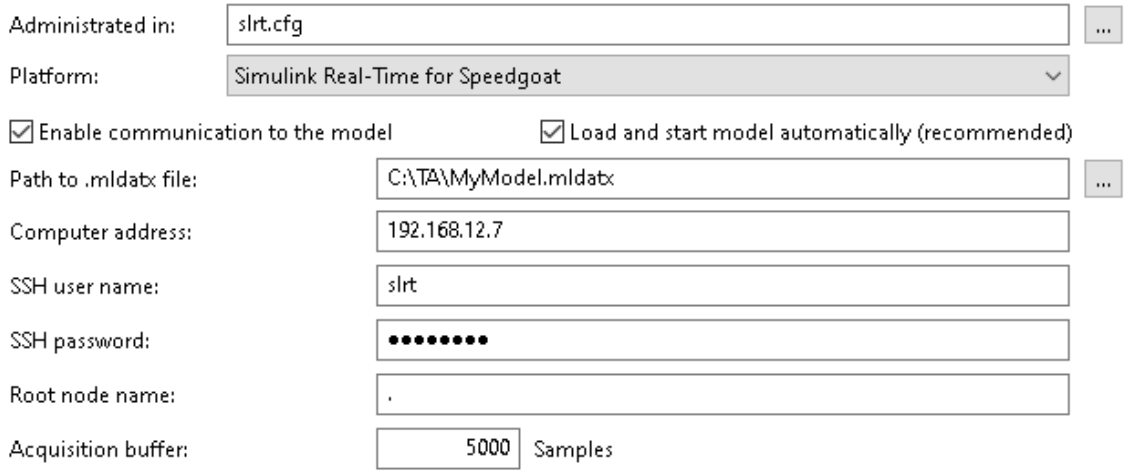


Figure 39 – Property page for Simulink Real-Time for Speedgoat

Activate the check box `Enable communication to the model` when your system has a model computer. When you deactivate this check box, you can still access all the signal names to program tests in the office without a model computer (see chapter ***3.2.7.2.3.16 Simulation mode***). In case you have no test system license, you cannot activate this check box.

Deactivate the check box `Load and start model automatically (recommended)` only in special occasions. In case it is deactivated PROVEtech:TA always tries to connect an already running model. In this case you must load and start the model via a different mechanism.

`Path to .mldatx file` is the path to the completely built Simulink Real-Time model.

`Computer address` specifies the address of the real-time computer.

Loading and starting the model works via the SSH protocol. Therefore, PROVEtech:TA needs the login data (`SSH user name` and `SSH password`, the default value is "slrt" for both). Storing the password in the .cfg file is encrypted in case it differs from the default.

You can name the root node in the signal hierarchy via the text box `Root node Name`. Use the empty string to remove the root node or the dot "." to use the model's name for the root node name.

The edit box `Acquisition buffer` specifies the number of samples to store temporarily on the model computer while data acquisitions are running. A small value can cause abortions of data acquisitions because of buffer overruns. The bigger the value, the more memory data acquisitions need on the model computer.



This platform currently has no possibility for PROVEtech:TA to access raw messages. Therefore, the message window and all other features depending on that have no function on this platform. CCP does not work and XCP only works on Ethernet channels.

3.2.7.2.3.7 Synopsys Silver

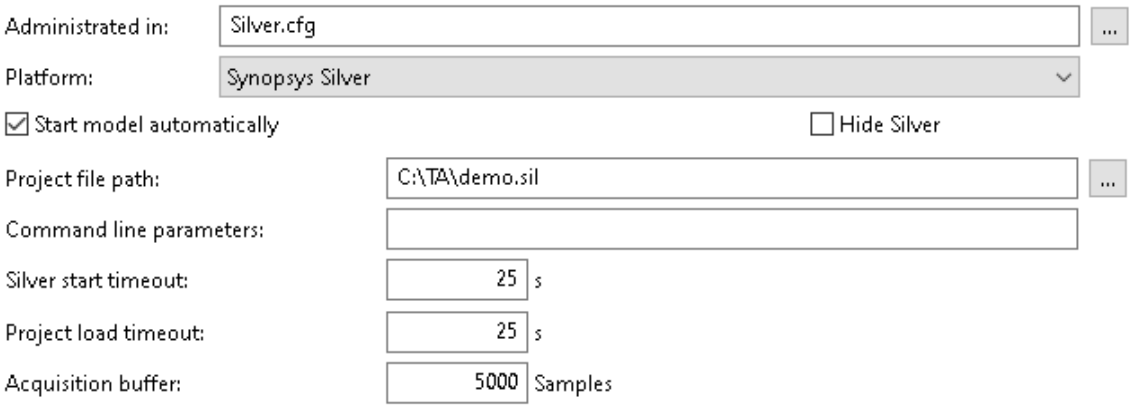


Figure 40 – Property page for Silver

Activate `Start model automatically` to start the model simulation automatically after loading the project. Deactivate the check box to simulate single steps as an example. The option `Hide Silver` starts “Silver.com” instead of “Silver.exe” with the command line parameter --noGui if it is not already running.

`Path to project file` is the path to the Silver project file with the file extension .sil.

You can optionally specify additional command line parameters for starting “Silver.exe” via `Command line parameters`. Embed parameters with space characters in quotation marks.

`Silver start timeout` is the maximum amount of time for starting Silver. In case it takes longer or there is an error, PROVEtech:TA cancels the operation.

`Project load timeout` is the maximum amount of time for loading and starting the project. In case it takes longer or there is an error, PROVEtech:TA cancels the operation.

The edit box `Acquisition buffer` specifies the number of samples to store temporarily on the model computer while data acquisitions are running. A small value can cause abortions of data acquisitions because of buffer overruns. The bigger the value, the more memory data acquisitions need on the model computer.

Any raw message access and CCP/XCP are not available on this platform.

This platform currently only simulates signals and no messages, thus the message window and all other dependent features have no function on this platform. Additionally, all Silver signals have the type “double”, even boolean signals.

3.2.7.2.3.8 National Instruments LabVIEW

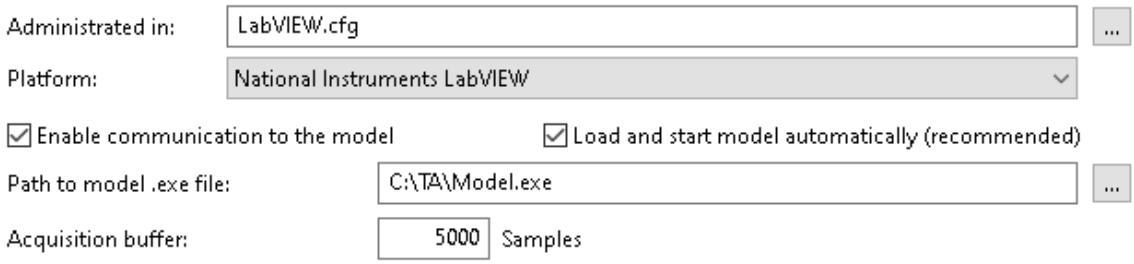


Figure 41 – Property page for LabVIEW

In case you use the LabVIEW (not LabVIEW-RT) platform, you always need a LabVIEW model as an .exe file accessible via PROVEtech:TA.

Activate the check box `Enable communication to the model` when your system has test system hardware. When you deactivate this check box, you can still access all the signal names to program tests in the office without test system hardware (see chapter ***3.2.7.2.3.16 Simulation mode***).

Activate the check box `Load and start model automatically (recommended)` to specify the LabVIEW model in form of an .exe file via the edit box `Path to model .exe file`. In this case PROVEtech:TA starts and stops this model automatically. Otherwise it expects an already running model (e.g. in the LabVIEW development environment). In case you have no test system license, you cannot activate this check box.

The edit box `Acquisition buffer` specifies the number of samples to store temporarily on the model computer while data acquisitions are running. A small value can cause abortions of data acquisitions because of buffer overruns. The bigger the value, the more memory data acquisitions need on the model computer.



Any raw message access and CCP/XCP are currently not available on this platform.

3.2.7.2.3.9 National Instruments VeriStand

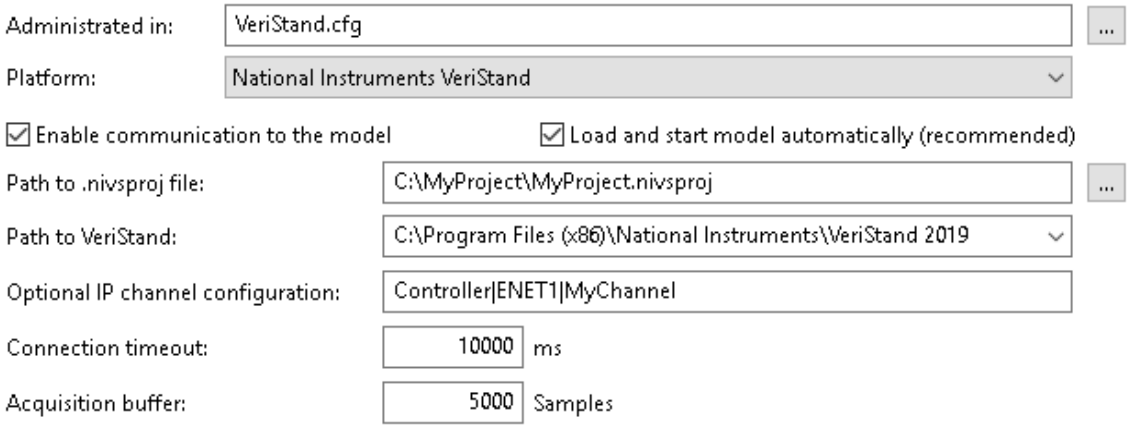


Figure 42 – Property page for VeriStand

Activate the check box `Enable communication to the model` when your system has a model computer. Deactivating this check box makes no sense for VeriStand systems. To work without a model computer please read ***3.2.7.2.3.2 Akkodis Offline Information File***. In case you have no test system license, you cannot activate this check box.

Deactivate the check box `Load and start model automatically (recommended)` only in special occasions. In case it is deactivated PROVEtech:TA always tries to connect an already running model. In this case you must load and start the model via a different mechanism.

`Path to .nivsproj file` is the file path to the VeriStand project file with the extension .nivsproj.

`Path to VeriStand` is the path to the VeriStand installation directory to use for loading the model. In case you do not specify this (empty field), it uses the path from the Windows registry key "HKEY_CLASSES_ROOT\VeriStandProject\shell\open\command".

To activate monitoring of UDP raw messages, please enter a string in the field `Optional IP channel configuration` (IP = Internet Protocol). It is a two dimensional list. A special character separates the list elements. For the first dimension it is the asterisk * and for the second dimension it is the vertical line |. The first dimension has an element per model node on which the monitoring shall run. Per model node there

is a | separated list beginning with the model node name and then the alternating an Ethernet port name and the according PROVEtech:TA channel name:

Node1|Ethernet1|Channel1|Ethernet2|Channel2|…*Node2|Ethernet3|Channel3|Ethernet4|Channel4|…*Node3|…*…

Here "…" means any further extension of the list. The names "Node?" correspond to those in the upper specified project file. "Ethernet?" corresponds to the typical NI XNET Ethernet interface name, e.g. "ENET1". "Channel?" is the name in PROVEtech:TA displayed by the message window as an example. See chapter ***4.4.14 The message window***. It is possible to combine several Ethernet ports to the same channel in PROVEtech:TA on the same node by specifying the same channel name.

The edit box `Connection timeout` specifies the number of milliseconds to try to open the command connection to the model after starting it. It might be necessary to increase this in case it takes a long time after the startup until it really runs, e.g. for initializing the model configuration.

The edit box `Acquisition buffer` specifies the number of samples to store temporarily on the model computer(s) while data acquisitions are running. A small value can cause abortions of data acquisitions because of buffer overruns. The bigger the value, the more memory data acquisitions need on the model computer(s).



This platform currently only allows accessing CAN, LIN and UDP (only monitoring and any IP defragmentation is limited to 20 fragmented UDP packets in parallel and correct order of the fragments) raw messages and no other raw message types. For LIN channels the message type "Event" is not available, thus selecting it has no effect. CCP/XCP on CAN, UDP, TCP and SxI (via VISA/RS232) works completely.



On VeriStand every signal has the data type "double", even boolean signals. Additionally, the signals have no description or other attributes. VeriStand is responsible for this behavior, not PROVEtech:TA. Only for Simulink signals, there is a mechanism to get additional information.

3.2.7.2.3.10 dSPACE System

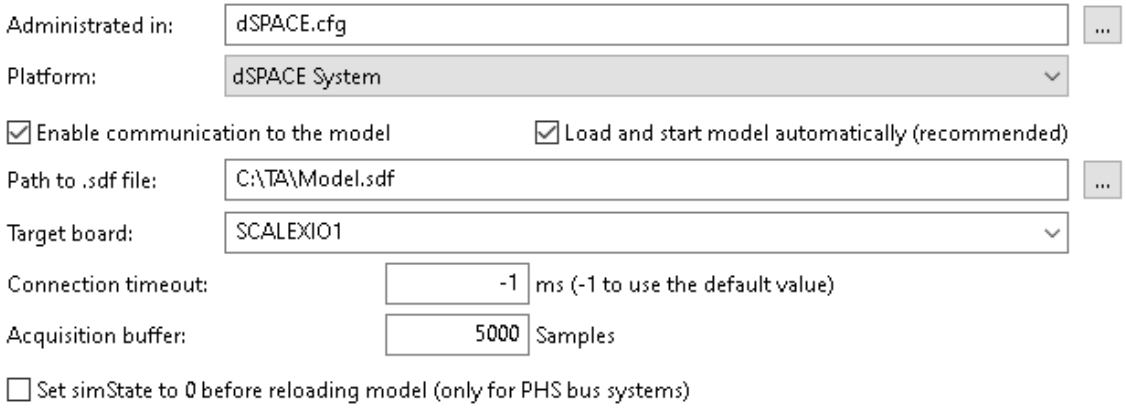


Figure 43 – Property page for a dSPACE system

Activate the check box `Enable communication to the model` when your system has a model computer. When you deactivate this check box, you can still access all the signal names to program tests in the office without a model computer (see chapter ***3.2.7.2.3.16 Simulation mode***). In case you have no test system license, you cannot activate this check box.

Deactivate the check box `Load and start model automatically (recommended)` only in special occasions. In case it is deactivated PROVEtech:TA always tries to connect an already running model. In this case you must load and start the model via a different mechanism.

A compiled model consists of several files. You only need to enter the path to the file with the extension .sdf via the edit box `Path to .sdf file`. PROVEtech:TA expects the other files in the same directory.

In case you have enabled the communication to the model, enter the processor or processor group in the edit box `Target board` where you want to run your model (e.g. “multiprocessor” or “ds1005”). The combo box provides all possibilities of the dSPACE configuration on your computer. This entry is ignored for dSPACE VEOS models.

The edit box `Connection timeout` specifies the number of milliseconds to try to open the command connection to the model after starting it. It might be necessary to increase this in case it takes a long time after the startup until it really runs, e.g. for initializing the model configuration.

The edit box `Acquisition buffer` specifies the number of samples to store temporarily on the model computers while data acquisitions are running. A small value can cause abortions of data acquisitions because of buffer overruns. The bigger the value, the more memory data acquisitions need on the model computers.

To explicitly stop the model before restarting it (see chapter ***3.2.4 Restart model***) by setting the signal “simState” to 0, you can activate the check box `Set simState to 0`

`before reloading model`. This is only available in case you have enabled the communication to the model, and it only works on PHS bus systems.



DS100x currently only allows accessing CAN and LIN raw messages and no other raw message types. For CAN channels the message type "Event" is not available, thus selecting it has no effect. “Remote” is only available on SCALEXIO/MicroAutoBox III. For LIN channels, the message type “Event” is limited to “Error”.

SCALEXIO (but not MicroAutoBox III) additionally supports Ethernet message access (see chapter ***4.4.14.2 Handling of Ethernet***).

The bandwidth between the real-time computer and the host PC is limited very much for DS100x platforms. This might lead to an increased amount of buffer overflows, particularly while doing data acquisitions.

CCP/XCP on CAN works. XCP on UDP and TCP only works on SCALEXIO and MicroAutoBox III.

Any raw message access and CCP/XCP are currently not available on the MicroAutoBox I.

For information concerning VEOS systems, please read the according ModelCom integration instruction.

On this platform type you have the possibility to run several models in parallel and control them from the same PROVEtech:TA instance. Each model has an own .sdf file and can control several nodes (CPUs/cores). It is possible to control SCALEXIO and DS1006 models in parallel. To configure this, specify a list of .sdf file paths in the text field `Path to .sdf file`. Use the | character to separate the paths. In the field `Target board` specify the target for each .sdf file and in the field `IP addresses` specify the IP addresses for each .sdf file if necessary. The | character separates the entries again. There are additional settings for an advanced configuration. The ***Administrator's Guide*** describes them.

PROVEtech:TA loads the .sdf files one after the other. Typically, the models run asynchronously. For data acquisitions across the models, PROVEtech:TA optionally has a loose synchronization mechanism where one node transmits its clock permanently to one node of each other model via Gigalink. The other nodes permanently calculate their clock difference and store it in a specific variable. PROVEtech:TA uses this difference at the beginning of a data acquisition for the synchronization. Have a look in the directory "Goodies\ModelComAndRTAE" on the PROVEtech:TA installation medium for information about how to install it. Please keep in mind that the synchronization happens at the beginning of the data acquisition. The asynchronous timer interrupt clocks of the models might drift over the time, so the synchronization might suffer.

The PROVEtech:TA license does not cover dSPACE products. The dSPACE products have own license agreements you need to consider. You get information about necessary dSPACE licenses from dSPACE.

3.2.7.2.3.11 Generic TCP

"Generic TCP" is a set of generic settings for PROVEtech:TA to access platforms PROVEtech:TA does not know directly.

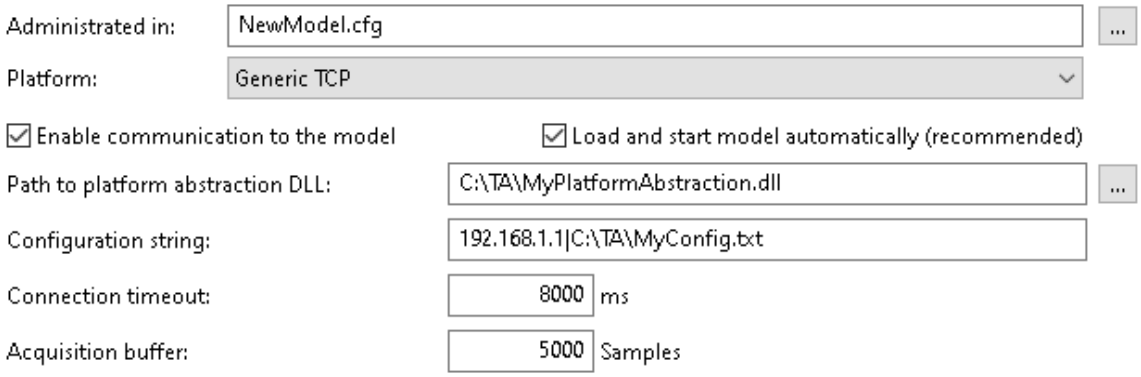


Figure 44 – Property page for Generic TCP

You can use this configuration to integrate your own hardware platform in PROVEtech:TA. A precondition for this is the usage of the TCP version of ModelCom. It provides functions to register signals and message channels. Optionally you can use compiled Simulink models running on your platform and access their signals. The only thing missing is a mechanism to start your model via PROVEtech:TA. For this PROVEtech:TA defines a plug-in interface you need to implement in a form of a DLL. In the field `Path to platform abstraction DLL` you can specify its path. The `Configuration string` is passed to this DLL when loading a model. Your DLL defines its format.

In case you are interested in this plug-in interface, please contact PROVEtech Support (provetech@akkodis.com).

Activate the check box `Enable communication to the model` when your system has a model computer. When you deactivate this check box, you can still access all the signal names to program tests in the office without a model computer (see chapter ***3.2.7.2.3.16 Simulation mode***). In case you have no test system license, you cannot activate this check box.

The edit box `Connection timeout` specifies the number of milliseconds to try to open the command connection to the model after starting it. It might be necessary to increase this in case it takes a long time after the startup until it really runs, e.g. for initializing the model configuration.

The edit box `Acquisition buffer` specifies the number of samples to store temporarily on the model computers while data acquisitions are running. A small value can cause abortions of data acquisitions because of buffer overruns. The bigger the value, the more memory data acquisitions need on the model computers.

The PROVEtech:TA features supported on this platform depend on the 3rd party model integration.

3.2.7.2.3.12 Configuration of CAN channels

When you select a CAN channel on the model configuration page (see ***Figure 33***, chapter ***3.2.7.2.3 Models***) and click on the button `Edit...` or double click on a CAN channel, the following configuration dialog pops up:

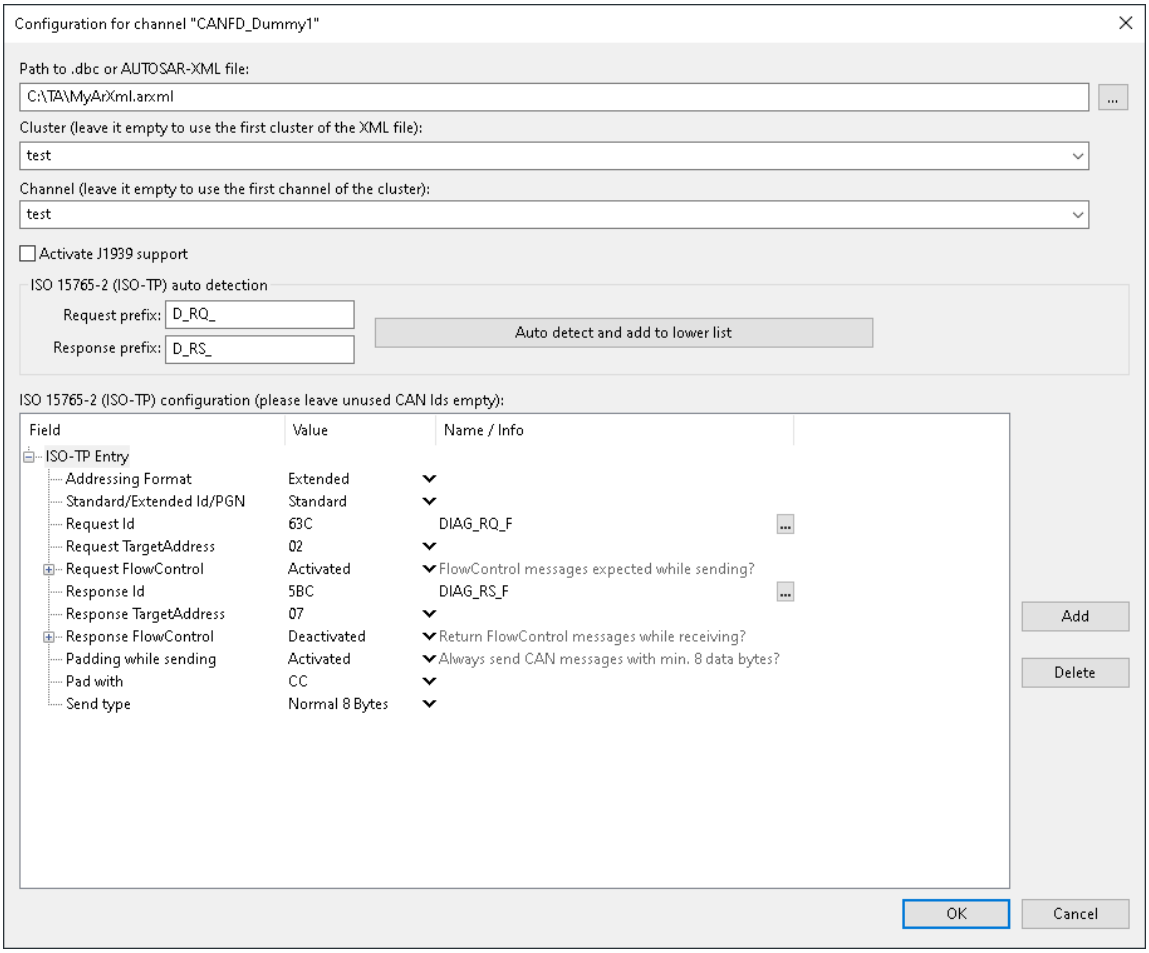


Figure 45 – CAN channel configuration dialog

In the text field `Path to .dbc or AUTOSAR-XML file` you can optionally specify a .dbc or AUTOSAR-XML file for this CAN channel. Some features only work with a correct data file, e.g. displaying message names or disassembling a message into the contained signals. In case of AUTOSAR-XML file please specify a contained CAN cluster name and a CAN channel that is part of the cluster (fields disabled in case of a .dbc file). Or leave the fields empty to use the first cluster and first channel.

In case you want to use the J1939 (SAE standard, mainly for trucks) features of PROVEtech:TA, please activate the check box `Activate J1939 support`. In case it is activated, PROVEtech:TA considers every CAN message with an extended identifier as a J1939 message where the Id consists of priority, parameter group number, destination address (only PDU1 format) and source address. Also, in the test language every CAN message with an extended identifier is received as *J1939Msg* object.

In case you activate the J1939 support, you also deactivate the automatic GMLAN (General Motors Local Area Network) detection for extended identifiers described next. When selecting a .dbc file which has the attribute “SourceId” for at least one ECU, each message with the attribute “Prio”>0 and an extended Id gets detected as a GMLAN message. The mapping from received extended Id to the according .dbc data works by ignoring the priority and source Id bits in that case.

The rest of the dialog handles the optional configuration of the ISO 15765-2:2016 transport protocol via CAN (ISO-TP, e.g. for diagnostics). ISO-TP can transport big data packages via CAN messages. A CAN message contains a maximum of 8 bytes of payload (64 bytes for CAN FD). So ISO-TP needs several CAN messages to transport a single ISO-TP message if it does not fit into a single one.

The information needed for this is not part of the data file specified above, so you need to configure additional information. The lower tree list handles this configuration. Normally there are special CAN Ids for ISO-TP. Two together are a pair: one for sending (request) and one for receiving (response) messages.

Click on `Add` to create a new pair. This inserts a new `ISO-TP Entry` on the root level. Use its sub entry `Addressing Format` to choose one of the following settings: `Normal`, `Extended` and `Mixed`. When using `Extended` or `Mixed` Mode, the first byte of a CAN message is an additional address information (together with the CAN Id). To have a complete address information, please enter this byte hexadecimal. To do this for `Extended` the fields `Request TargetAddress` and `Response TargetAddress` and for `Mixed` the fields `Request AddressExtension` and `Response AddressExtension` are available.

At `Request Id` and `Response Id` you can enter the according CAN Ids. Use the field `Standard/Extended Id/PGN` to select if they are standard or extended Ids (or a J1939 PGN, see below). As an alternative you can click the arrow button in the column `Name / Info` to select an Id by name.

In case you select `J1939 PGN` you get the new entry `J1939 PGN` instead of `Request Id` and `Response Id`. “PGN” is the abbreviation for “Parameter Group Number” and it corresponds to the bits 8-25 of the extended CAN identifier while using the J1939 protocol. When specifying this, PROVEtech:TA uses the ISO-TP protocol for all extended CAN Ids matching the PGN. It automatically generates the according response Id from a request Id and vice versa by switching the source and destination address part of the extended CAN Id. This only works in case there is a destination

address. So, it is only possible to specify hexadecimal PGNs of the type PDU1 for this entry: 0000-EF00 where the lower byte must be 0. Optionally you can set the “Data Page” and the “Reserved” bit.

Beside the addressing information PROVEtech:TA needs information about the flow control. ISO-TP defines special messages (FlowControl messages) to control the communication speed between two nodes. PROVEtech:TA needs this information only in case it is used as an active sender or receiver on the CAN bus. Change `Request FlowControl` from `Deactivated` to `Activated` to let PROVEtech:TA wait for FlowControl messages from the receiver when PROVEtech:TA sends ISO-TP messages. The parameter `SeparationTimeMin Offset` defines a positive or negative offset for the SeparationTimeMin (minimum time between two Consecutive Frames) sent by the opponent in milliseconds. As PROVEtech:TA runs on Windows and sending CAN messages depends on the used model platform, it cannot guarantee an exact timing. Use this parameter to influence the timing. PROVEtech:TA sends ISO-TP messages with flow control works directly on the model computer on systems others than dSPACE DS100x. However, the timing still might be inaccurate due to bus load, communication with the CAN card, etc.

Activate the `Wait STmin after FlowControl` option to force a wait for the time STmin specified by the FlowControl message after receiving the FlowControl message before PROVEtech:TA sends the next message.

When you change `Response FlowControl` from `Deactivated` to `Activated`, PROVEtech:TA sends back FlowControl messages in case the other node sends ISO-TP messages. Please make sure that there are no other nodes on the bus that send these FlowControl messages. Via the sub entry `BlockSize` you can specify, how many Consecutive Frames should be received between two FlowControl messages. The value 0 means that no additional FlowControl message follows. `SeperationTimeMin` specifies the expected minimal time between two Consecutive Frames. In case of a CAN FD channel, PROVEtech:TA uses the same frame variant for sending flow control frames as the other node: CAN, CAN FD without Bit Rate Switch or CAN FD with Bit Rate Switch.

Activate `Padding while sending` to fill all sent CAN messages up to at least 8 bytes. Otherwise, set it to `Deactivated`. For CAN FD padding is mandatory in some cases as frames larger than 8 bytes cannot have any size. As an example it is not possible to send a 9 byte frame, you must send at least a 12 byte frame instead and use padding.

The `Pad with` field defines the hexadecimal byte value for padding. The ISO 15765-2:2016 standard suggests using CC (= 204 decimal) to prevent unnecessary stuffing bits. J1939 defines FF as some kind of default for any message.

In case of a CAN FD channel, there is the additional option `Send type`. When PROVEtech:TA sends ISO-TP messages, this specifies the maximum CAN FD frame

size and type to use. You can send normal CAN frames with 8 bytes or CAN FD frames with 8-64 bytes. For CAN FD frames there is additionally the possibility to activate the BRS (Bit Rate Switch) while sending.

To understand all these mechanisms it is useful to read the ISO 15765-2:2016 documentation.

To simplify the configuration, you can at least fill the lower list with some CAN Ids automatically. The diagnostics messages which use ISO-TP very often have prefixes in the name. Enter these in the fields `Request prefix` and `Response prefix` and click on `Auto detect and add to lower list` to fill the list with these Ids.



ISO 15765-2:2016 extends the maximum message size from previously 4095 to now 4294967295 bytes. PROVEtech:TA currently limits it to 2097152 bytes (2 MB) for self-protection.

3.2.7.2.3.13 Configuration of CCP/XCP

In addition to automating the manipulation of the ECU environment to verify ECU functionalities, HiL testing very often requires measurement and calibration of the ECU memory. The standardized protocol for doing this is called XCP (the predecessor is CCP). You can use the XCP feature to access XCP signals like any other signal in PROVEtech:TA: perform measurements, use the signals on the work page and within test scripts.

Not all platforms support CCP/XCP. The previous platform configuration chapters contain information about this separately for each platform.

In case your license contains CCP or XCP, the configuration dialog for CCP/XCP pops up when you click the according button in the model configuration page (see chapter ***3.2.7.2.3 Models***).

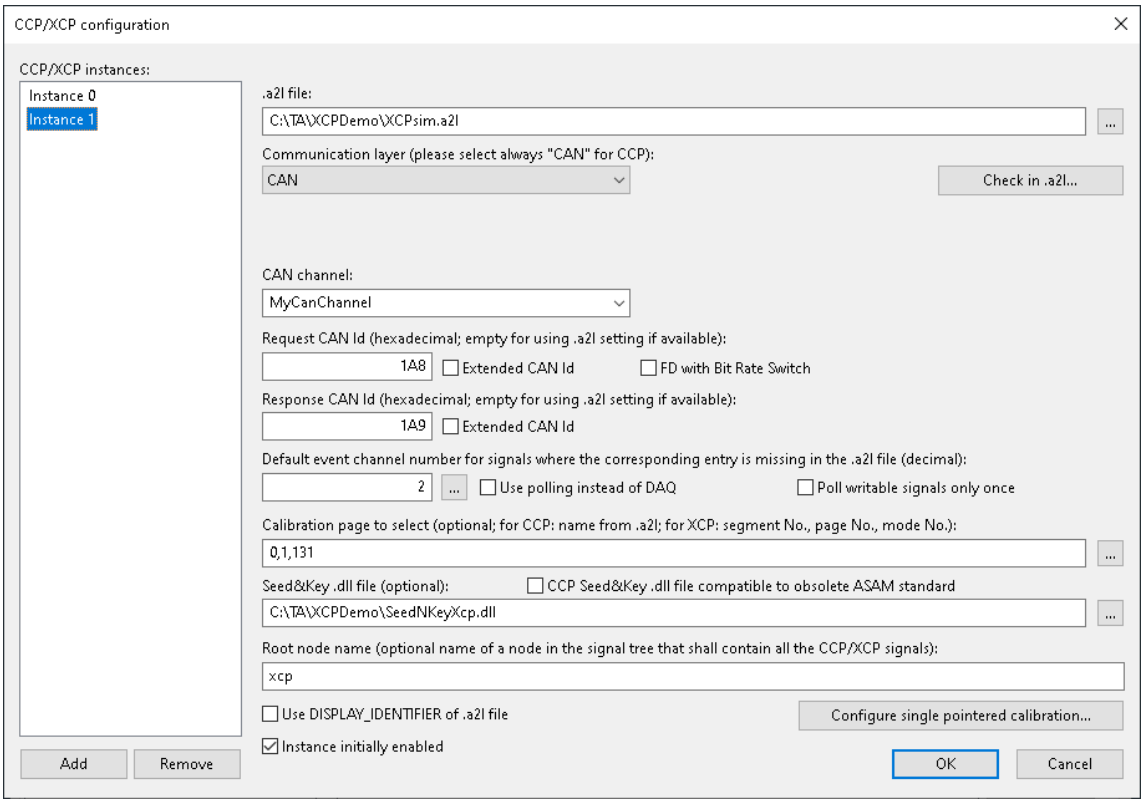


Figure 46 – CCP/XCP on CAN configuration dialog

The left list displays all CCP/XCP instances. You can fill and clear the list via the buttons `Add` and `Remove`. You need an CCP/XCP instance for every CCP/XCP ECU you want to access. Some ECUs contain several logical units for which you need several CCP/XCP instances. The right fields display the configuration of the instances selected in the left list.

The text box `.a2l file` specifies the path to the ASAM A2L file containing the signal information. PROVEtech:TA detects automatically if it is CCP or XCP via the .a2l file. For XCP you can select if the communication works on CAN, FlexRay, SxI (serial interface), TCP or UDP (Ethernet) via the combo box `Communication layer`. For CCP you must select CAN.

Click on `Check in .a2l...` to load the selected .a2l file and display a list of supported communication layers and their parameters according to the file.

The field `CAN channel` specifies the message channel used for the CCP/XCP communication in case you have selected CAN in the upper combo box. It must be a channel already configured for the model (see chapter ***3.2.7.2.3 Models***), because the CCP/XCP instances run on the model computer and use its channels.

The `Request CAN Id` specifies the CAN Id the CCP/XCP instance uses for sending messages. The `Response CAN Id` corresponds to the opposite direction, so it is the Id used by the ECU for sending. Activate `Extended CAN Id` to use 29-bit CAN Ids

instead of 11-bit CAN Ids (corresponds to set bit 31 for the CAN Id in the .a2l file). Activate `FD with Bit Rate Switch` for CAN FD channels to send FD messages with activated Bit Rate Switch (corresponds to set bit 30 for the CAN Id in the .a2l file). Otherwise PROVEtech:TA sends normal CAN messages without FD. This setting is not available for `Response CAN Id` as the XCP master does not care if it is an FD message or not upon reception. Only the Id is important. `Extended CAN Id` and `FD with Bit Rate Switch` have no meaning if the according Id field is empty. You can leave these fields empty if the .a2l file contains the correct information. In case you fill the fields, the according .a2l file information is ignored.

`Default event channel number` specifies the event used for acquiring signals where this setting is missing in the .a2l file. In the ECU typically several event channels with different cycle times are available for measuring CCP/XCP signals. PROVEtech:TA needs to select an event channel for every signal. For XCP this information can be part of the .a2l file. In that case, you do not need to change the field `Default event channel number`.

For CCP the .a2l file typically has no signal specific event channel entries thus PROVEtech:TA uses the `Default event channel number` for all of them. However, PROVEtech:TA supports the same keywords in the .a2l file as for XCP.



In case your XCP slave only supports static DAQ lists with fixed event numbers, PROVEtch:TA sorts the signals according to their event channel numbers into the static DAQ lists. However, this way static DAQ lists are very inflexible compared to dynamic DAQ lists. DAQ lists with other event channel numbers possibly stay unused or the DAQ list with an often-used event channel number is too small to acquire all selected signals.

This is why PROVEtech:TA supports an additional mode for DAQ lists with fixed event channel numbers: Enter 65535 in the field `Default event channel number` to activate it. This way PROVEtch:TA ignores the individual signal event channel numbers and just fills the available static DAQ lists from the first to the last (order determined by the .a2l file).

Read chapter ***4.2.5 Changing CCP/XCP event channel numbers*** to learn how to change CCP/XCP signal event numbers at runtime.

In case the ECU does not support the DAQ mode (data acquisition with cyclic sending of signal samples), you can activate `Use polling instead of DAQ`. In this case there is a cyclic polling of each selected signal (UPLOAD instead of DAQ for all CCP/XCP signals on the workpage, in data acquisitions and RTAE tests). This happens without any time relation, the ping-pong mechanism happens as fast as possible. The

more signals the longer it takes for one round. Attention: The polling mode can increment the communication busload significantly, but it can also avoid limitations of the DAQ mode.

Some ECUs do not support reading writable signals via DAQ. Additionally some ECUs limit the amount of signals readable via DAQ very much. In that case, activate `Poll writable signals only once`. This way PROVEtech:TA reads (UPLOAD) writable signals only once after each ECU connection establishment.



When activating `Poll writable signals only once` PROVEtech:TA does not read back written values. This means if an ECU positively acknowledges writing a signal value, PROVEtech:TA displays the signal on the workpage with the new value. In case the ECU actually uses a different value internally, you will not see that. Also in case such a signal changes via a different mechanism in the ECU (e.g. via diagnostics access), the workpage still displays the old value.

In case you specify the optional text box `Calibration page to select`, the CCP/XCP master sends a SELECT_CAL_PAGE (0x11) command for CCP or a SET_CAL_PAGE (0xEB) command for XCP at the end of the connection phase. All following calibration commands (setting a CCP/XCP signal) work on that page on the ECU. For CCP you can use the ... button to select a page by name from the .a2l file.

For XCP there are no names in the .a2l file and everything is much more complex as there can be several segments. PROVEtech:TA only supports sending one SET_CAL_PAGE command. It has three data bytes which you enter as a comma separated list of decimal values (0-255). The first value is the segment, the second one is the page and the third one is the mode. The mode has three used bits according to the XCP specification:
Bit 0 (=1): The ECU application shall use that page.
Bit 1 (=2): The XCP slave shall use that page.
Bit 7 (=128): The specified segment number (first value) shall be ignored and instead of that all segments shall point to that page.

The other bits are reserved and should be 0. Entering the mode value is optional. The default value is 3. Another typical value is 131 (all three upper bits are set). Unfortunately we cannot tell you, which overall setting is “correct”. It is not possible to read this from the .a2l file. The ECU developer should know the exact behavior. In the past very often the following two settings worked correctly:
0,1,3
0,1,131

If you have a different tool where calibration already works, you can try to get the working values from a bus trace of that tool. Just search for the SET_CAL_PAGE command and take over the three bytes following the command byte (0xEB). Attention, the order is different for the command: mode byte, segment byte, page byte.

In case the ECU uses the Seed&Key algorithm for enabling CCP/XCP (security mechanism) you can specify the according DLL for Windows of the manufacturer via the field `Seed&Key .dll file`.

The Seed&Key interface for CCP used by PROVEtech:TA has the following declaration:

```
BOOL __cdecl ASAP1A_CCP_ComputeKeyFromSeed(BYTE *seed, unsigned short
    lenSeed, BYTE *key, unsigned short maxLenKey, unsigned short *lenKey);
```

In case you activate the check box `CCP Seed&Key .dll file compatible to obsolete ASAM standard`, PROVEtech:TA uses the following declaration:

```
BOOL __cdecl ASAP1A_CCP_ComputeKeyFromSeed(BYTE *seed, BYTE *key);
```

In that case the seed must have 4 bytes and the key 6 bytes according to the obsolete ASAM standard.

For XCP it is the following interface and the check box has no influence:

```
DWORD __cdecl XCP_ComputeKeyFromSeed(BYTE privilege, BYTE lenSeed,
    BYTE *seed, BYTE *lenKey, BYTE *key);
```

The `Root node name` is optional and defines the name of a newly created node in the signal tree containing the signals of the .a2l file (see chapter ***4.2 The Signal Selection window***). For a better overview it is a good idea to enter a name here. The dot is the hierarchy separator, e.g. “XCP.KLA”.

Activate the check box `Use DISPLAY_IDENTIFIER of .a2l file` to use the DISPLAY_IDENTIFIER setting of the .a2l file for each signal instead of the original signal name if available. Please be aware that all DISPLAY_IDENTIFIER must be unique in that case.

PROVEtech:TA automatically adds a writable boolean signal with the name “Enable” to your signal tree within your specified root node. If this signal is 0, the communication to your ECU is stopped otherwise it is activated. Use the `Instance initially enabled` check box to specify the initial value of that signal. Please consider that changing the “Enable” signal takes some time until the change gets executed.

Please stop the communication to an ECU before accessing it with a different CCP/XCP master than PROVEtech:TA. It is not possible to access an ECU with several CCP/XCP masters simultaneously.

For TCP or UDP the dialog differs slightly; here is an example for UDP:

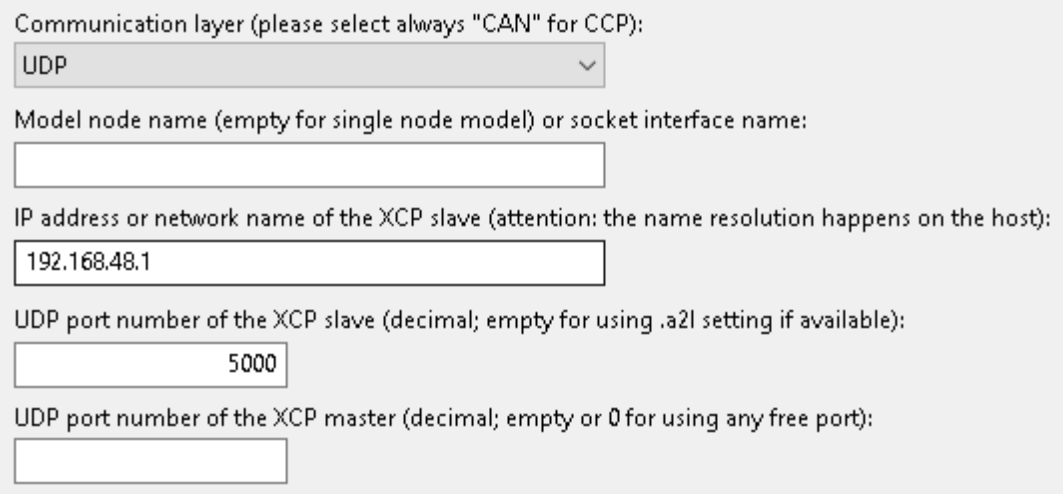


Figure 47 – XCP on UDP configuration dialog

In case your model works on several computer nodes, please enter the name of the node where the XCP instance shall run in the field `Model node name (…) or socket interface name`. This node must be connected to the XCP slave via a network port. You can leave this field empty in case your model runs on a single node. The XCP master within ModelCom uses the operating system's socket commands in such a configuration. However, if your model provides own socket interfaces for ModelCom and the XCP master, enter the according socket interface name to use in that field. This is interesting for SiL scenarios without real network traffic or in case of using special network cards that do not use the operating system's TCP/IP stack.

You can leave the next two fields empty in case your .a2l file correctly contains the according information. You can use the fields to overwrite the .a2l file settings.

Specify the address of the XCP slave via `IP address or network name of the XCP slave`. In case you enter a name PROVEtech:TA resolves it on the host PC, not on the model node.

`UDP port number of the XCP slave` specifies the UDP port used by the XCP instance for sending messages. For TCP this is the TCP port number. The `UDP port number of the XCP master` is the counterpart. In case you enter 0 there or leave it empty, the XCP instance just uses any free port. For TCP this 2nd port setting does not exist.

For SxI (XCP on serial interface) the dialog part changes like this:

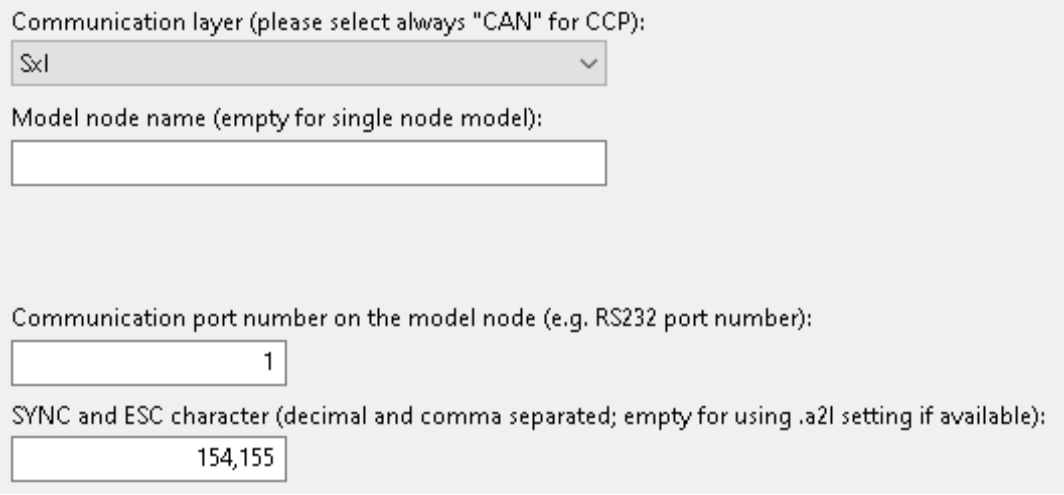


Figure 48 – XCP on SxI configuration dialog

In case your model works on several computer nodes, please enter the name of the node where the XCP instance shall run in the field `Model node name`. This node must be connected to the XCP slave via a serial interface. You can leave this field empty in case your model runs on a single node.

The `Communication port number on the model node` is a logical number of the port to use managed by the operating system or access library.

Use the field `SYNC and ESC character` to enter the decimal numbers of two special characters for synchronizing the beginning of XCP packets on the serial port (see XCP specification). Leave the field empty to use the according setting of the .a2l file if available.



PROVEtech:TA currently only supports SxI communication with a SYNC character, a checksum byte and a header with one byte length. The checksum calculation does not include the header.

For FlexRay there are no more FlexRay specific settings beside the selection of the FlexRay channel. This means, the .a2l file must provide all necessary parameters correctly. Using FlexRay currently only works in combination with the PROVEtech:RE platform and only with FlexRay cards which have the necessary software interfaces.

After accepting the settings via `OK` and loading and starting the model, you can see the new signals in the signal tree. For the user there is basically no difference between using “normal” signals and CCP/XCP signals within PROVEtech:TA. You can use the

CCP/XCP signals on the workpage, in test scripts and even in real-time tests (RTAE, but there you can only read but not write these signals). However, you should consider that an additional communication layer exists in comparison to “normal” signals.

PROVEtech:TA manages CCP/XCP signal pools over all CCP/XCP signals used in the whole system. For CCP/XCP PROVEtech:TA must transfer the used signals to the according ECUs, so they can measure the signals cyclically and transfer their values to the CCP/XCP instances. The sampling rate within the ECUs can differ from each other and also differ from the model rate. So the updates only happen in the raster specified by the ECU. Every time a signal pool within PROVEtech:TA changes, a new signal list gets transferred to the ECU and the new signal values get transferred asynchronously to the model computer later. This way it can take some time until PROVEtech:TA gets the new signals and displays old values or 0 (never read so far) in this time.



This means when PROVEtech:TA starts a data acquisition or a real-time test program (RTAE), the CCP/XCP signal values at the beginning might be old or 0 (never read so far). It might take some seconds until the ECUs send new values.

Additionally the data acquisition within the ECUs gets interrupted for a short time when the signal pools change (e.g. a signal gets deleted from the workpage). This is caused by the retransmission of the DAQ lists to the ECUs. This can cause discontinuities in the signal waves.

PROVEtech:TA cyclically checks whether the ECUs are still responding. In case they do not respond (e.g. the battery voltage is powered off), you can still read the CCP/XCP signals in PROVEtech:TA. However, their values are old or 0 (never read so far) in this case. When the cyclic check detects that an ECU responds again, the DAQ lists get transferred automatically and the signal updates are activated. An additional signal with the name “Status” within the specified root node displays the internal status of the according CCP/XCP instance:

Value	Meaning
0	There is no connection to the ECU (either the ECU is not responding or the “Enable” signal is 0).
4	There is a connection to the ECU but not all writable signals are retrieved yet once (only happens in case you activate `Poll writable signals only once`).
3	There is a connection to the ECU but the DAQ lists are not transferred and started yet.

2	There is a connection to the ECU and the DAQ lists are transferred and started but the ECU did not send all signal values yet.
1	There is a connection to the ECU, the DAQ lists are transferred and started and the ECU has sent all signal values at least once. Only in this state you see valid signal values.
5	There is a connection to the ECU and it currently writes the signal values of a parameter file (see chapter ***3.2.6 Load CCP/XCP parameter file***).

The test language command ***System.GetSignal*** is a special case. When you read a CCP/XCP signal with it, a direct communication with the ECU follows for getting the value (UPLOAD). In this case the signal pools are not used.

PROVEtech:TA can also write CCP/XCP signals (calibration). This either works via the test language command ***System.SetSignal*** or the controls on the workpage. Writing within real-time tests (RTAE) works via the command ***RT.Assign***. Its execution must run on the same model node as the according CCP/XCP instance though. Just like ***System.GetSignal*** writing signals uses a direct communication with the slave (DOWNLOAD) and the signal pools are not used.



The more CCP/XCP signals are on the workpage, used in data acquisitions and in real-time tests, the more data must be transferred by the according ECU. There are limits in the ECUs and on the communication channel (e.g. CAN). In case you exceed these limits, error messages and unexpected behavior might happen. To avoid these limitations of the DAQ mode, you can switch to the polling mode.

PROVEtech:TA currently does not support pre-defined DAQ lists (different than static DAQ lists) or a different address granularity than 1. It currently ignores the GROUP, VARIANT_CODING, TRANSFORMER and TYPEDEF blocks of the .a2l file. Additionally, it ignores all conversion types other than IDENTICAL, LINEAR, RAT_FUNC (without the coefficients “a” and “d”), TAB_INTP and TAB_VERB.



The following applies to TAB_INTP: There is a limit of maximum 128 value pairs. The output values (physical values) must rise or fall strictly monotonously, because otherwise no inverse function can be formed. PROVEtech:TA supports DEFAULT_VALUE_NUMERIC for TAB_INTP, but not DEFAULT_VALUE. If the logical value is outside the table when converting to the physical value, PROVEtech:TA uses DEFAULT_VALUE_NUMERIC. If this does not exist, it extrapolates the table values. The A2L standard has no information for this case. Thus, tools from different vendors can behave different concerning this. When calibrating (writing), PROVEtech:TA ignores DEFAULT_VALUE_NUMERIC as described in the A2L standard. If the physical value is outside the table, it extrapolates the table values. The A2L standard has no information for this case. Thus, tools from different vendors can behave different concerning this. Please be aware that integer raw/logical values can overflow. PROVEtech:TA does not check that.

The CCP and XCP “standards” are just a sum of all used technologies in the market. The “standard” documents leave open many questions and allow a huge variation of configurations. This means, it might happen that PROVEtech:TA does not support the specific implementation of your slave/ECU. Please contact the PROVEtech:TA support in case you want to discuss possible solutions for that.

In case your XCP slave only supports static DAQ lists but these are not defined in the .a2l file, the XCP master tries to get that information via the commands GET_DAQ_PROCESSOR_INFO and GET_DAQ_LIST_INFO. The XCP slave must support these commands though.

When sending the XCP command SET_DAQ_LIST_MODE, the XCP master must assign an event number and a priority to a DAQ list. According to the standard each event number has a fixed priority number though. Some XCP slaves require to set exactly that one correct priority number. Unfortunately, very often that assignment between event number and priority number is missing in the .a2l file or it is incorrect. Therefore, PROVEtech:TA has the following fallback mechanisms:

- The entry “Priorities” in the PORVEtech:TA XCP configuration file (see ***Administrator’s Guide***) overwrites the .a2l file settings.
- In case the entry “Priorities” and the .a2l file settings are missing, the XCP master tries to get the priorities via the commands GET_DAQ_PROCESSOR_INFO und GET_DAQ_EVENT_INFO from the slave. In case that also fails, the master uses the fixed priority number 1.

3.2.7.2.3.14 XCP AUTOSAR Single Pointered calibration

The AUTOSAR Single Pointered calibration is a special way of calibrating via XCP and ignoring the ASAM standard. This means, neither the XCP nor the A2L ASAM standards have information about it. Only the AUTOSAR standard touches the topic a little bit, but it mainly describes the implementation in the ECU and not the usage via the XCP protocol or the settings in the according .a2l file. This means that the implementation in PROVEtech:TA makes a lot of assumptions and uses A2L parts set by Vector Informatic but might be different for other manufacturers.

In the ECU/slave the access on parameters does not happen via their memory addresses but via a pointer table. During startup the ECU/slave first copies the pointer table from a read-only memory to writable memory. The initial values point to so-called parameter element groups in the read-only memory. Each parameter element group contains one or several parameters.

So, if you want to calibrate a parameter (overwrite it), you first must copy the according parameter element group into the writable memory. Then you must overwrite the according pointer in the pointer table and then you can overwrite the parameter in the writable memory.

Restarting the ECU/slave also resets the pointer table and the written parameters are lost.

The reason for this complicated procedure is limited writable memory that cannot hold all parameters. This means, the user must first select which parameter element groups are available for calibration later and which are not.

In PROVEtech:TA this selection happens via the button `Configure single pointered calibration...` (see ***Figure 46***). When clicking the button PROVEtech:TA first loads the specified .a2l file and checks if it fulfills all requirements for the single pointered calibration (see below).

Then it opens this dialog:

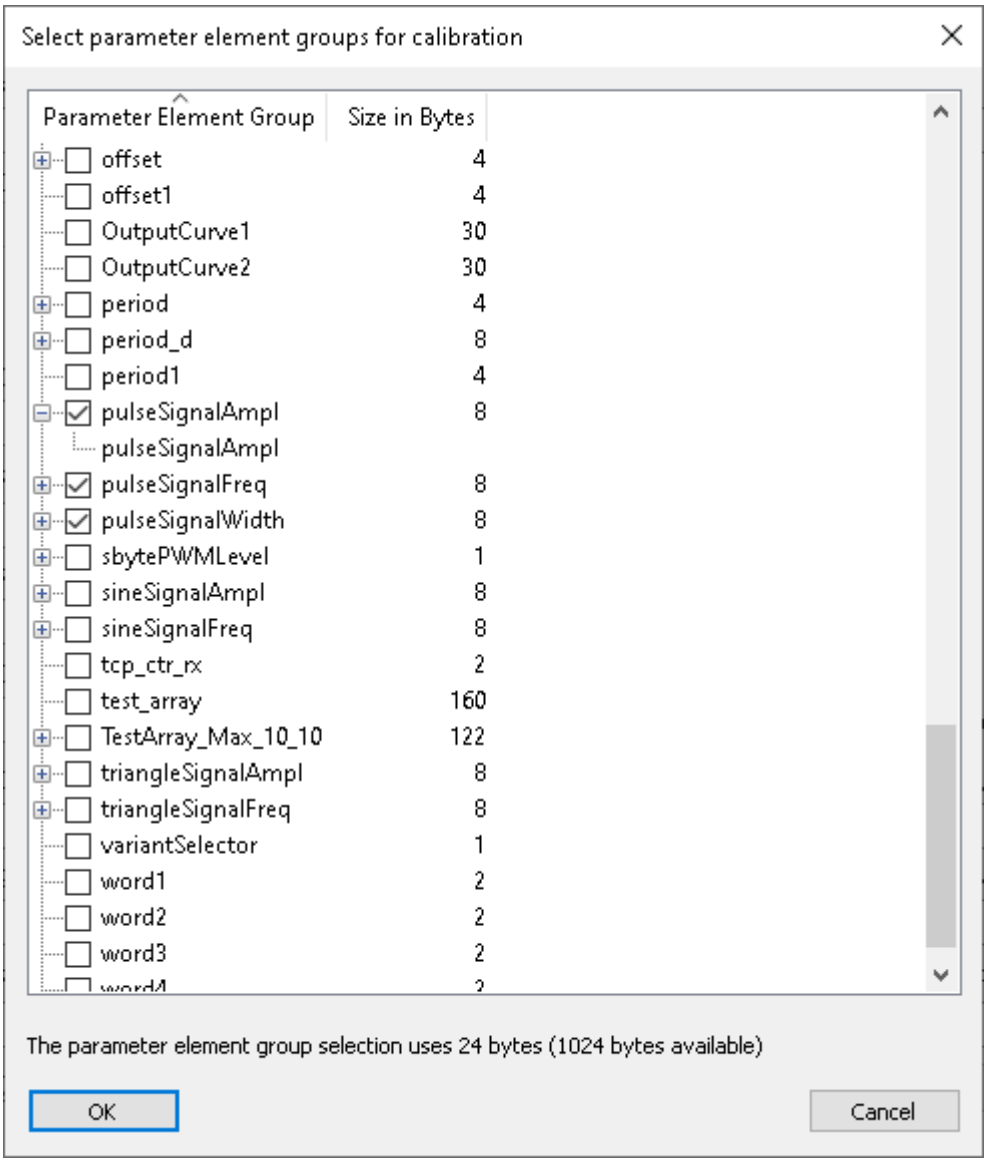


Figure 49 – Selection oft he parameter element groups

The first tree level shows the names of all parameter element groups and the second level the according contained parameters (writable CHARACTERISTICs in.a2l file).

The text below informs about the necessary memory for the selected groups and the overall available memory (upper limit).

PROVEtech:TA only switches to the single pointered mode if you select at least one group. PROVEtech:TA creates read-only signals for all parameters outside the selected groups

PROVEtech:TA needs the following information from the .a2l file:

- There must exist a PROJECT\MODULE\MOD_PAR\MEMORY_SEGMENT with the name “PoolRAM”. It must have an IF_DATA XCP\SEGMENT for the “address extension” information. “PoolRAM” is the available memory for the calibration.
- There must exist a PROJECT\MODULE\MOD_PAR\CALIBRATION_METHOD with the name “AUTOSAR_SINGLE_POINTERED” and 4 values within CALIBRATION_HANDLE: address of the pointer table in the writable memory, size of the table in bytes (must be dividable by 4), size of an entry in the table (must be 4), address of the pointer table in the read-only memory. Unfortunately, the “address extensions” are missing for the two addresses and PROVEtech:TA must use the “address extension” of the “PoolRAM” and one of the element groups for that instead.
- There must be one or several CAL_PARAM_GROUP within PROJECT\MODULE\IF_DATA with this format:

  /begin CAL_PARAM_GROUP
  NAME "*Name*"
  ADDRESS *Address*
  SIZE *SizeInBytes*
  LINK_MAP
  "*NameIgnoredbyPROVEtech:TA*"
  AddressIgnoredbyPROVEtech:TA
  AddressExtension
  IgnoredbyPROVEtech:TA
  IgnoredbyPROVEtech:TA
  /end CAL_PARAM_GROUP

Please be aware that the PROVEtech:TA master performs the following steps every time when establishing a new connection with a single pointered slave:

1. Copy the selected parameter element groups into the writable memory.
2. Copy the pointer table into the writable memory and change the pointers of all calibratable parameter groups while doing that.

These actions can take a lot of time and the “Status” signal stays 0 while doing that. Additionally, this always resets the parameters every time when establishing a new connection.



PROVEtech:TA does not align the parameter element groups in the writable memory. This means the slave must support any access on byte level.

3.2.7.2.3.15 Configuration of parallel running models

To open the dialog described below just click on `Configure Parallel models…` (see ***Figure 33***).

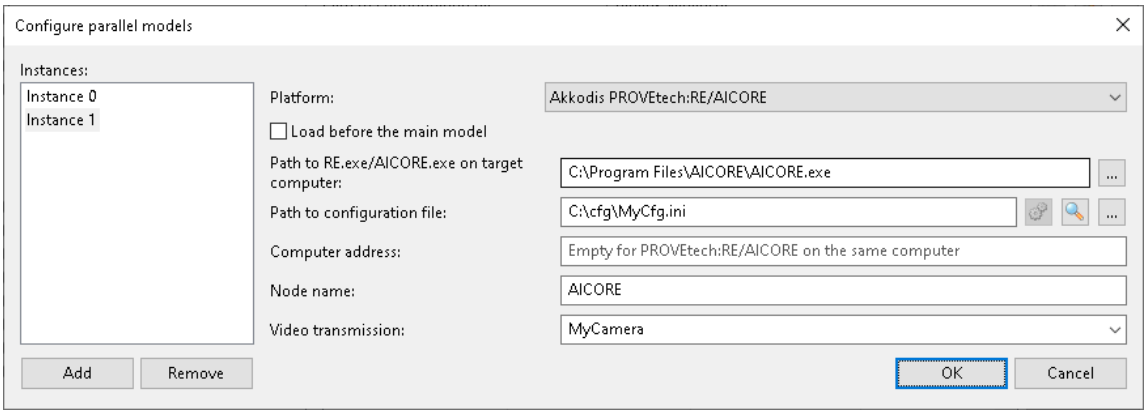


Figure 50 – Configuration of parallel running models

Use this dialog to configure one or several model nodes running asynchronously in parallel to your main model for additional tasks. In PROVEtech:TA you can access them like any other model node, which means that it looks like a big model while working with the whole setup. This configuration does not work for all model constellations and only in case the model communication is active (needs a test system license).



Please be aware that there is no synchronization between the models. PROVEtech:TA starts data acquisitions in parallel as good as possible, but they can drift apart over the time. Signal values displayed in the offline mode at the same time were not acquired at the same time in reality for sure. The RTAE (Real-Time Automation Engine) instances running in parallel models can only access signals of other model nodes if you configure additional communication channels for that (see ***Administrator’s Guide***, entry “RtaeLink” in the .cfg file).

Click on Add to add new instances and on Remove to remove the selected instance.

Select either PROVEtech:RE/AICORE, LabView or Generic TCP in the Platform combo box to choose the type of your model. The check box Load before the main model determines the model loading order:
1. All instances with active check box in the instance order of this dialog
2. The main model
3. All instances with inactive check box in the instance order of this dialog

The other fields correspond to the ones of the "normal" configuration page for the according platform type. See chapters ***3.2.7.2.3.1 Akkodis PROVEtech:RE/AICORE***, ***3.2.7.2.3.9 National Instruments VeriStand***, ***3.2.7.2.3.11 Generic TCP***.

3.2.7.2.3.16 Simulation mode

In the simulation mode the communication with the model computer(s) is deactivated. This happens if you do not have a test system license or in case you deactivate the according check box on the model's configuration page or if you use a DLL model (see chapter ***3.2.7.2.3.5 Simulink DLL***).

In the simulation mode all signal access works on a local memory area of the PROVEtech:TA process created for simulating signals. In case of a DLL model you access the memory of that model, in other cases you simply access a big memory block. Simulated message channels create no messages.

The execution of the simulation works in bursts. At the beginning the simulation gets the current PC time and uses this as simulation time 0. About every 100ms the simulation compares the current PC time with the previous one and executes as many ticks necessary to increase the simulation time about the difference. This way the simulation time increases in the same speed as the PC time.

The artificially inserted writable signal "TimeScale" corresponds to the factor which is used to increase the simulation time faster or slower than the PC time. The default signal value is 1, so the speed is identical. A value greater than 1 increases the simulation speed, a value between 0 and 1 decreases it. Use 0 to pause the simulation.

If the value gets too big, PROVEtech:TA limits it as otherwise it would use the whole CPU performance and PROVEtech:TA would stop working.

The RTAE (Real-Time Automation Engine) and data acquisitions run synchronously to the simulation, so these functions also work in the simulation mode.

3.2.7.3 Workpage

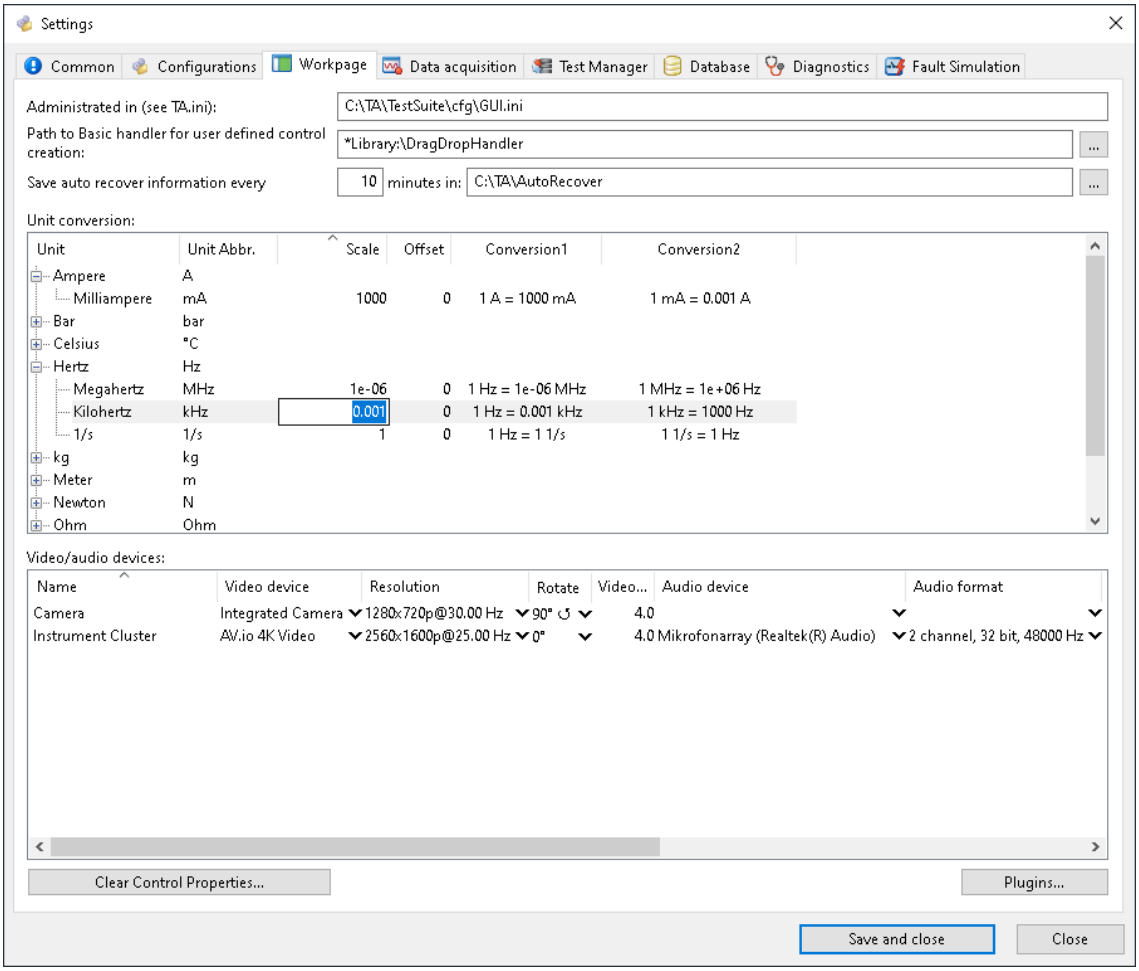


Figure 51 – The Workpage property page

The `Administrated in` box of the property page Workpage (see ***Figure 51***) displays the file path that contains these settings.

The edit box `Path to Basic handler for user defined control creation` optionally defined a file or Test Manager path to a Basic script. This Basic script must define the following function:
Function SignalsToControls(ByRef straSignals() As String) As String()
PROVEtech:TA calls this function when you copy and paste or drag and drop signals to the workpage and in the signal selection window the automatic control type selection is activated (see chapter ***4.2.4 Selecting control types, adding controls***). This function specifies what the new controls look like. The function has a string array parameter (signal names) and also returns a string array (control definitions). The function checks all incoming signal names and if it wants to change the default control creation for a signal or several signals, it returns array elements for them and overwrites the according signal name(s) in the “straSignals” array with empty string(s). PROVEtech:TA creates default controls for only the signals not overwritten by empty strings. The returned control

definitions are equal to those in .aof or .egr files. So, the best way is to edit the control on the workpage and save it to a file. Then you can load the file in a text editor and copy the according line into a Basic module. In the Basic module you can replace the control position and signal names dynamically. The test language manual has an example for this handler. Changing the Basic module has no effect until you restart PROVEtech:TA or change the path in the configuration dialog.

The `Save auto recover information` fields specify a cycle time in minutes and a destination path to automatically save your workpage and cockpit file in case of modifications in the background. When PROVEtech:TA does not close in a normal way and you restart it and it finds such files, it opens the following window where you can recover your files:

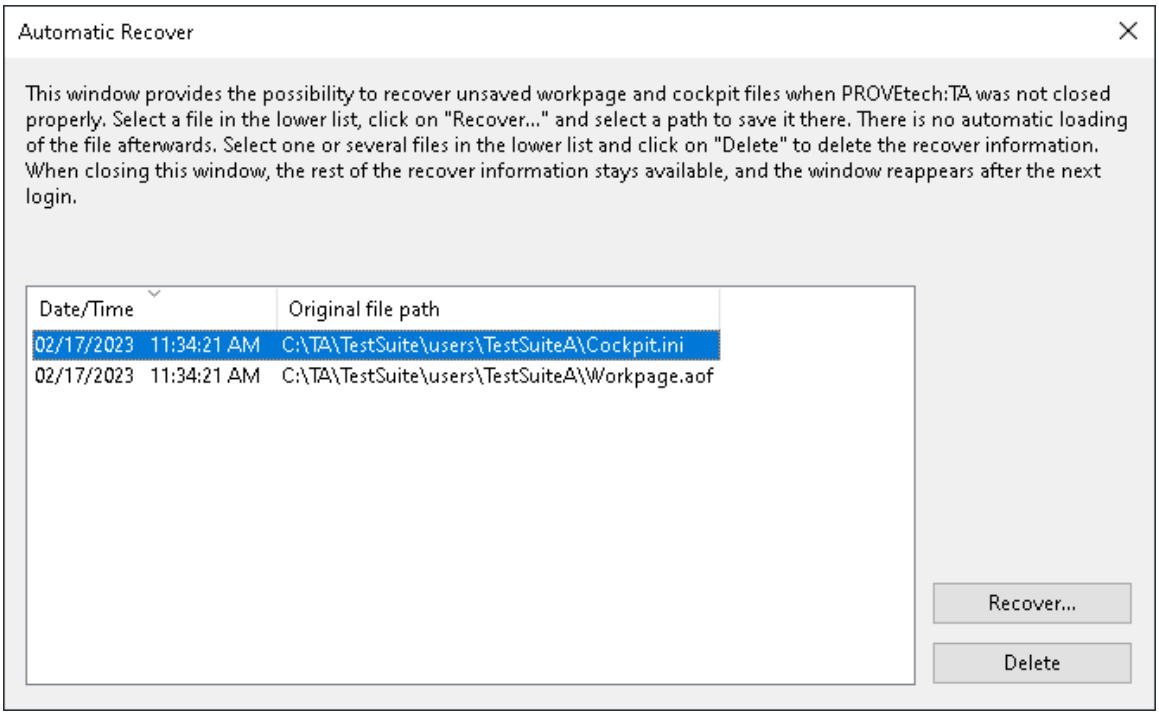


Figure 52 – The automatic recover window

The list shows the date and time when PROVEtech:TA saved a file and the original file path. Select one entry and click on `Recover...`. This opens the file selection window to recover your file. It uses the original path but adds "_recovered" to the file name as default. You can change this to overwrite the original file, but we recommend saving it with the modified name to compare the files later. After recovering, the list entry disappears. Select one or several entries of the list and click on `Delete` to delete the according recover information without the possibility to get it back. If you close the window and there are entries left, the widow reappears if you login the next time.

The recover information is user dependent, but it is all in the same directory for all users. Please do not use a network directory for the recover information. Several

PROVEtech:TA processes cannot share the same directory in parallel and saving on a local drive is faster and also possible if the network is down. Enter 0 for the cycle time to disable the auto recover function. PROVEtech:TA deletes according recover information in case you normally save your cockpit or workpage file or if you choose to not save them when PROVEtech:TA asks if you want to save them (e.g. when loading a different file or switching the user).

Use the table `Unit conversion` (***Figure 51***) to set mathematical relations between units via a scale and an offset. Some control types of the workpage can do unit conversions automatically using this table (see chapter ***4.4.1.1 The units converter***).

The table has two hierarchical levels: the base unit (e.g. “Second”) and the units under it (e.g. “Millisecond” and “Minute”) that have a relation to the base unit. Use the context menu to create and delete units and base units.

The table consists of six columns:
`Unit`: The name of the unit (e.g. “Second”)
`Unit Abbr.`: The abbreviation of the unit name (e.g. “s”)
`Scale`: The factor for the conversion
`Offset`: The number to add for the conversion
`Conversion1`: Conversion formula to convert between the base unit and the unit
`Conversion2`: Conversion formula for the contrary direction

You can edit the first four columns by selecting the context menu item *Edit Field* or by double clicking on the field.

Use the list `Video/audio devices` to configure video/audio recording. Please notice that the recording only works on Windows 10. Additionally PROVEtech:TA uses Microsoft Media Foundation for processing video/audio thus your source devices must support that.

Right-click to create new entries (menu item `Add` in the context menu) or modify/delete existing entries. Each entry represents a video and/or an audio device. The following columns are available:

`Name`: Enter a unique name here used for referencing the dive in workpage files, data acquisitions and the test language.

`Video device`: Use this column to select the video device by clicking on it. The column identifies the device via a friendly name. When opening the device list it additionally displays a long unique identification. After the selection this appears in the column `Video Id`. In case the friendly name is not unique, PROVEtech:TA uses this unique identification to map the device.

`Resolution`: Select the resolution to use here after selecting the video device. The entries have the following scheme:
<WidthInPixels>x<HeightInPixels><p/i>@<Frequency> Hz
“p” means “progressive” and “i” means “interlaced”. “p” is typically the better choice. This list shows all possible resolutions reported by the device. The list can change in case you change something on the device, e.g. streaming a new resolution to a frame grabber.
This entry is only relevant if you also select a `Video device`. In case you select no resolution, PROVEtech:TA uses the default resolution reported by the device.

`Rotate`: Use this to optionally rotate the video by 90° clockwise, 90° anticlockwise or 180°.

`Video compression rate`: This is a value >0 and ≤200. The bigger this value is, the higher compression of the video you get during recording which reduces the amount of data and the quality. PROVEtech:TA uses this value to calculate the target bit rate for the video encoder. The formula is:
BitRate = WidthInPixels * HeightInPixels * Frequency / CompressionRate
This entry is only relevant if you also select a `Video device`.

`Audio device`: Use this column to select the audio device by clicking on it. The column identifies the device via a friendly name. When opening the device list it additionally displays a long unique identification. After the selection this appears in the column `Audio Id`. In case the friendly name is not unique, PROVEtech:TA uses this unique identification to map the device.
Currently PROVEtech:TA can record audio, but not play it. However, you can play the created video/audio file in the Windows Media Player as an example to hear the audio.

`Audio format`: After selecting the audio device use this setting to select an input format for the recording. You can choose one of the formats reported by the audio device. This includes the number of channels, the number of bits per sample and the sampling rate.

`Delay [s]`: When starting a data acquisition PROVEtech:TA tries to synchronize the video/audio data with the signals and message time stamps in such a way, that the time 0 of the measurement data corresponds to a time stamp of the video/audio file. Recording the video/audio happens on the PROVEtech:TA Windows computer while signal data and messages come in on one or several model computer(s). There is no hardware synchronization of the used time sources. Starting the video/audio recording happens shortly before starting the data acquisition on the model computer(s). Additionally, there is typically a delay of the video/audio data up to the time when arriving in the video/audio file due to the used hardware (e.g. a camera) and the data processing in the driver and encoder. PROVEtech:TA has no chance to automatically detect this delay. So, you can enter it manually in the column `Delay [s]` in seconds. PROVEtech:TA additionally adds a delay happening due to the asynchronous start of the recording. And in the offline mode (see chapter ***4.5 The offline mode***) you can adjust it also afterwards.

Please read chapter ***4.4.18 The video control*** to get more information about video/audio processing. The next chapter ***3.2.7.4 Data acquisition*** describes how to select the video/audio sources actually recorded in data acquisitions.

Click on `Clear Control Properties...` to open an additional dialog. On every `OK` button click of a properties dialog (see chapter ***4.4.1 General properties***) of a standard control (not the custom control or the special controls), PROVEtech:TA stores the settings user dependent in the file "Elements.edb" of the user directory. When you create the displayed signal with the same control type on the Workpage again, PROVEtech:TA uses these settings for the control. In the course of time a lot of settings are stored and the file grows permanently. You can use the `Clear Control Properties` dialog to reduce the file content.

The `Plugins...` button opens an additional dialog to configure plugins for video/audio sources, data acquisition import and export formats. For video/audio plugins you just select according DLL paths. There are plugins for different camera vendors like Basler and Daheng or infrared cameras from FLIR. They do not support the Windows standard camera interfaces and have own APIs. Thus, you need a plugin DLL providing a PROVEtech:TA specific interface to translate between the camera specific interface and Windows.

For a new data acquisition format, you specify the according plugin DLL path as well as the unique file extension (without the dot) fitting to the format. PROVEtech:TA uses that file extension to know which DLL to use. A plugin DLL has a PROVEtech:TA specific interface.

After changing any of the plugin DLL lists you must restart PROVEtech:TA to use them. For video/audio, the according drop-down list in the upper `Video/audio devices` list should display additional entries for added and correctly configured plugins.

For data acquisition formats the file open/save dialogs for data acquisitions should display your newly added file extensions.

3.2.7.4 Data acquisition

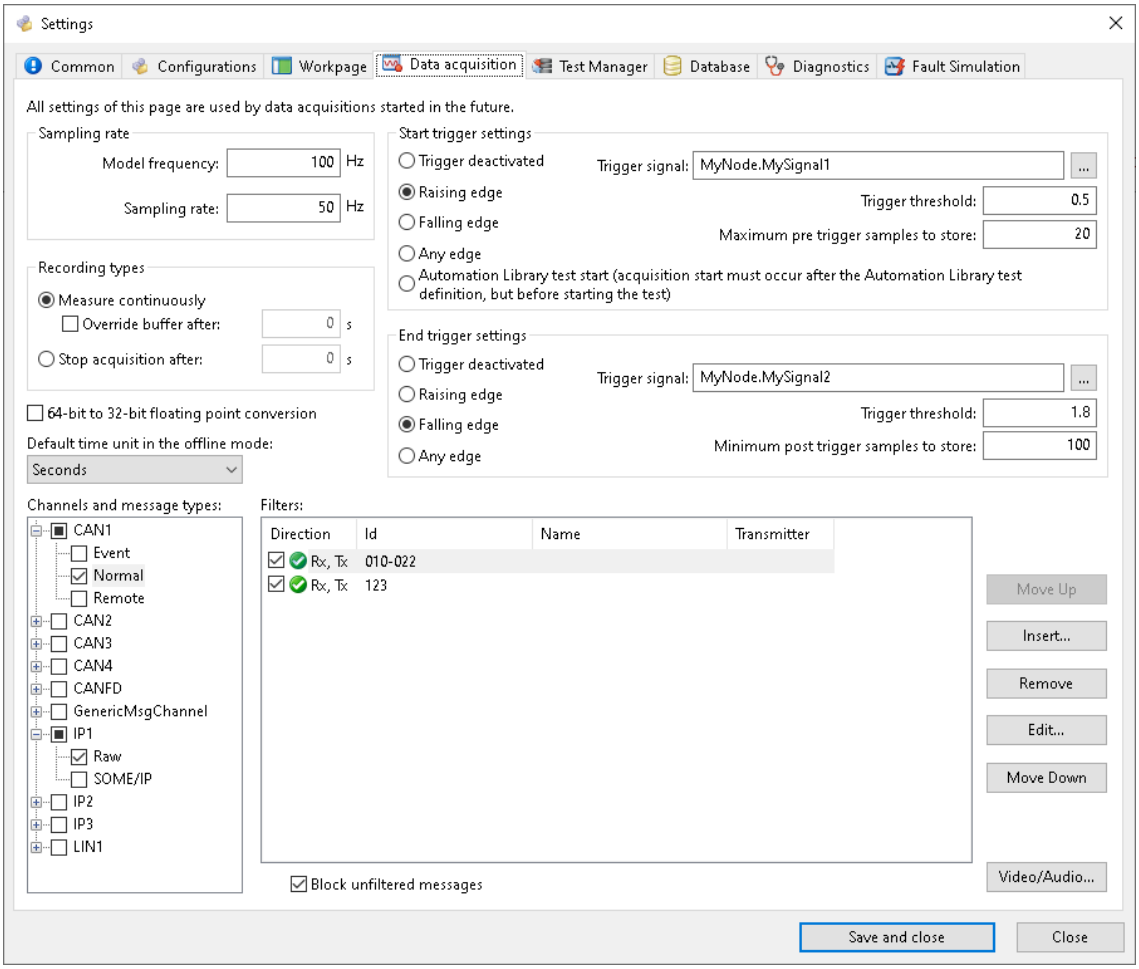


Figure 53 – The Data Acquisition property page

It is important that changes on the `Data Acquisition` property page only work on data acquisitions started later on and not on currently running data acquisitions or already finished data acquisitions. The settings are valid as long as you do not change them and they are stored user and configuration dependent. For additional information about data acquisitions have a look at chapter ***3.4 The Data Acquisition menu***.

Use the `Sampling rate` edit box to select the default data rate (sampling rate) of the signal transfer from the model computer to PROVEtech:TA. Data acquisitions happen on the model nodes where each model node can have an own sampling rate. That is the base rate (fastest possible rate) of available signal acquisition rates for the signals of a node.

It is possible to sample signals slower by using down-sampling (acquire only in every 2nd, 3rd, 4th, etc. model step) to reduce the necessary bandwidth and memory. As an example, if a node has a base rate of 100 Hz, it is possible to select 50 Hz, 33.333 Hz, 25 Hz, 20 Hz, etc., but not 55 H. PROVEtech:TA would convert 55 Hz to 50 Hz. You can overwrite the default sampling rate entered here per signal in the signal selection

window (see chapter ***4.2.3 Measurement signal selection, clipboard and info field***). Please be aware that old dSPACE PHS bus systems only support one sampling rate for all signals per node. PROVEtech:TA uses the fastest one across all signals per node.

PROVEtech:TA knows three different recording types:

- Continuous measurement (radio button `Measure continuously`):
  The measurement is terminated when you stop it manually, via a test program, via an end trigger or when the hard disk is full.
- Continuous measurement with cyclic buffer:
  The measurement data gets stored in a ring buffer on the hard disk, with the oldest data being overwritten by new data when the buffer is full. Activate the check box `Override buffer after` additionally to the radio button `Measure continuously` and enter the capacity of the ring buffer in seconds in the edit box.
  The measurement is terminated as described for the normal continuous measurement above.
- Fixed duration measurement (radio button `Stop acquisition after`):
  Specify the desired duration of the measurement in seconds in the edit box. A data acquisition with saving to file that has been started is automatically stopped after the specified time. The events described above (continuous measurement) can also stop it earlier.

The `Default time unit in the offline mode` combo box determines, which time unit to use when saving/converting the data acquisition in/to the PROVEtech:TA format and opening it in the offline mode (see ***4.5 The offline mode***). However, you can also change the time unit afterwards directly in the offline mode via the combo box below the overview graph. The PROVEtech:TA data acquisition format stores this in its header.

A start trigger can be set for acquisitions whose measurements are recorded on the hard drive (group `Start trigger settings`). After the start of a data acquisition, the measurement data gets transferred from the model node(s) to PROVEtech:TA. When `Trigger deactivated` is set, the hard drive recording gets immediately started. In any other case the hard drive recording starts if the set `Trigger signal` satisfies the trigger condition. By doing so, PROVEtech:TA checks changes in the signal value and if it raises above (`Rising edge`) or falls below (`Falling edge`) the set `Trigger threshold`. When `Any edge` is selected the direction is irrelevant. For integer or boolean signals, the set `Trigger threshold` should lie between two integer numbers. For boolean signals use 0.5 as the threshold.

In case you select `Automation Library test start`, the start trigger occurs when an Automation Library / RTAE test gets started on the model computer. There is no need to specify a trigger signal and threshold in this case. For getting a correct

triggering, the start of the data acquisition must occur after defining the test using the Automation Library (**RT.EndNode**) and before starting it via **AL.AsyncRun** or **AL.Run**.

The setting `Maximum pre trigger samples to store` defines the pre-trigger time, a recording-period for samples before the start trigger condition is satisfied. The duration of a sample here corresponds to the sampling rate of the according trigger signal. When an acquisition with pre-trigger time is observed in the offline mode (see chapter ***4.5 The offline mode***), this period (the pre-trigger time) shows up in the negative region of the time-axis. If the specified period exceeds PROVEtech:TA's internal buffer size, or the trigger condition is satisfied too soon after the start of the acquisition, then the period is automatically shortened.

In the `End trigger settings` group you can define a trigger that automatically stops a data acquisition. This works on both data acquisition types: with or without saving to a file. The radio buttons, `Trigger signal` and `Trigger threshold` work the same way as for the start trigger.

The setting `Minimum post trigger samples to store` defines the post-trigger time, a recording-period for samples after the end trigger condition is satisfied. The duration of a sample here corresponds to the sampling rate of the according trigger signal. As PROVEtech:TA internally works with blocks of samples, it is not possible to stop a data acquisition after each sample, but only after a whole block. So, this setting is a minimum value of samples rounded up to the end of a block.

In the lower part of this property page you can edit the message channels you want to record and set filters for specific messages. The possible settings here are equal to the settings for message windows (see chapter ***4.4.14.1 The property dialog of the message window***). The only difference is, that you can only record raw messages of the lowest protocol layer. For example it is not possible to record MOST AMS messages as they are assembled using CMS messages. However, when you record CMS messages, you will still see AMS messages in the offline mode (see chapter ***4.5 The offline mode***) as the offline mode assembles the AMS messages by using the CMS messages.

Message recording is only available when your model platform supports the handling of messages in combination with PROVEtech:TA.

Click on the button `Video/Audio...` to open a separate window displaying a list of all video/audio devices configured (see previous chapter ***3.2.7.3 Workpage***). Activate the according check box per entry to activate the recording of that device when starting data acquisitions. Then click on `OK`.



The recording of video/audio in one or several .mp4 files always starts when starting a data acquisition. In opposite to signal and message recordings neither a start trigger nor “continuous measurement with cyclic buffer” has an influence on the file size. The data acquisition only stores a synchronization time between .mp4 file and signals/messages: Which time within the .mp4 file corresponds approximately to the time 0 of the signal/message recording.



Checking the trigger signals happens within PROVEtech:TA and not on the according model nodes. This means, the checks work with the sampling rate selected during the data acquisition for the according trigger signal.

3.2.7.5 Test Manager

In the property page `Test Manager` you can make all settings for the Test Manager and Basic Editor.

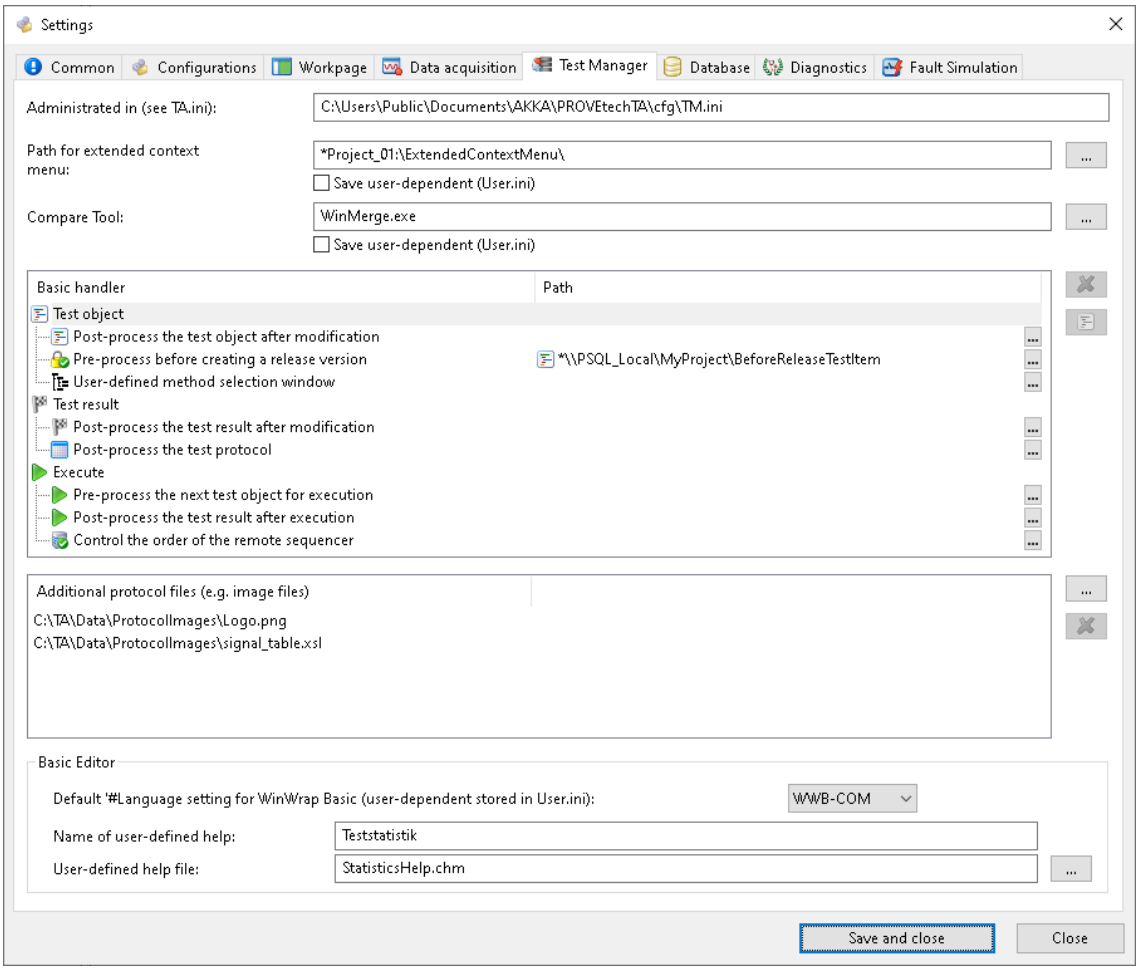


Figure 54 – The Test Manager property page

The box `Administrated in (see TA.ini)` displays the file path where these settings are stored.

In `Path for extended context menu` you can choose a Test Manager path for acting on user-defined test objects and inserting additional user-defined menu entries to the context menu of the Test Manager tree (see also ***5.9 User-defined test objects and extending the context menu***). You can optionally `Save user-dependent (User.ini)` this setting. Then you overwrite this setting in the TM.ini.

In `Compare Tool` you can set the path to a comparison tool for comparing test code or test results. By default, PROVEtech:TA searches for WinMerge.exe, which is freely available at (https://www.winmerge.org), for example. This comparison tool must accept two files or folders in the last two command-line options, which are compared (see also ***5.3.13 Comparing and merging test code*** and ***5.7.6 Comparing test results***). You can

optionally `Save user-dependent (User.ini)` this setting. Then you overwrite this setting in the TM.ini.

Basic handler

In the `Basic handler` control you can specify paths to test macros that are triggered by certain user actions or test language calls to adjust objects or control the action. These test macros are called automatically, processed in the background and do not generate a test result. Click the button  to create a sample code in the Test Manager tree, which contains the interface and comments. This sample code can later be adjusted into the corresponding macro after closing the settings dialog.

Generally, the macro can only be initialized as a basic handler by starting PROVEtech:TA, clicking on `Save and close` it in the settings dialog, using the ***TM.ClearCache*** test language or the keyboard shortcut Ctrl+F5 (Refresh). Changing the Basic code and saving it afterwards is not enough, because the code is not yet transferred to the Basic instance.

To `Post-process the test object after modification` and `Post-process the test result after modification`, certain operations can be determined for which the Basic handler is called. For example, you could set a certain emoji as name prefix if a special test parameter is set.

In the Basic handler `Pre-process before creating a release version`, you can specify, for example, whether the release version should actually be created (see chapter ***5.8.1.1 Pre-process before creating a release version***).

Per `User-defined method selection window` you can add further entries to the method selection tree and control their behavior when clicked (see chapter ***5.4.7 The method selection window***).

If you want to post-process test protocols, e.g. for graphical formatting, then you can set in `Post-process the test protocol` a special elementary test that defines the handling. Read more about this in ***5.6.3.6 Test protocols formatting using post-processing***.

In `Pre-process the next test object for execution` you can specify a Test Manager path to modify the test object to be executed next (see also chapter ***5.5.5 Pre-process the next test object for execution***).

`Post-process the test result after execution` allows you to perform actions on test results of elementary test objects after the final saving of the test result (see also chapter ***5.5.6 Post-process the test result after execution***).

To control the execution order in the Remote Sequencer list by a WinWrap Basic handler, you can set a Test Manager path in `Control the order of the remote`

test sequencer. You find more about this in chapter ***5.14.4 Automatic control of the sequencer order***.

Additional Protocol Files

When you work with html-code in your test protocols then you can define Additional Protocol Files (e.g. image files) of the file system or test documents of the Test Manager (see chapter ***3.1 The file selection dialog***). These files are copied to the temporary protocol directory when PROVEtech:TA starts (see also ***5.6.3.1 The test protocol of an elementary test***). Please note that PROVEtech:TA uses a set of standard files to display the RTAE protocols, which are also copied to this temporary protocol directory. To avoid a possible name conflict with your own files, please avoid the following file names: dummy.html, edge_missing_XSL.xsl, empty.bmp, I.bmp, img_colormap.gif, img_selectedcolor.gif, item_error.bmp, item_ok.bmp, item_unknown.bmp, last.bmp, minus.bmp, minus.png, minus_last.bmp, minus_last.png, node.bmp, node_error.bmp, node_ok.bmp, node_unknown.bmp, plus.bmp, plus.png, plus_last.bmp, plus_last.png, T.bmp, TextEditCtrl.Template.001.html, w3color.js.

Basic editor

In the combobox Default '#Language setting for WinWrap Basic you can set which language directive ('#Language) should be used when creating a new Basic script.

The edit control Name of user-defined help file: and User-defined help file: in the group box Basic Editor may contain the name and path of an additional .chm file. In which case, an additional button is inserted in the Basic editor toolbar for the new help file.

3.2.7.6 Database

In the tab Database you can configure and test different database connections. You choose one of these database connections when you log in to PROVEtech:TA (see ***2.3 Login***). Furthermore, you can configure your settings of the database backup.

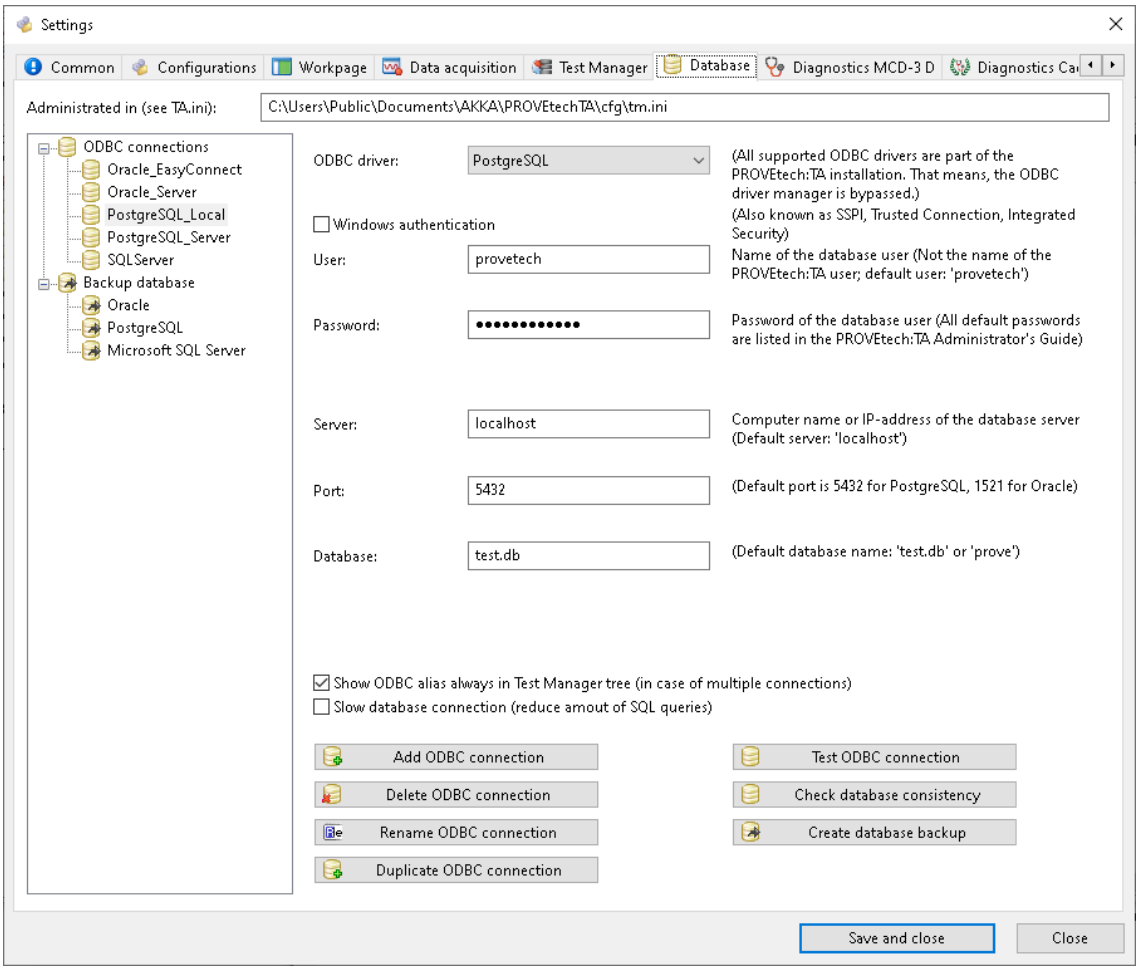


Figure 55 – PostgreSQL in the property page Database

The field `Administrated in` of the `Database` property page displays the file path that saves these settings.

In the list box on the left side, the names of the ODBC database connections are located which you can choose at logon. Use the buttons `Add ODBC connection`, `Delete ODBC connection`, `Rename ODBC connection` and `Duplicate ODBC connection` to manage the connections. Furthermore, you can set up the database backup, which is useful for example with a PostgreSQL server installed locally on the same computer as the PROVEtech:TA application (see also further below).

All ODBC drivers supported by PROVEtech:TA (these are from PostgreSQL, Microsoft SQL server and Oracle) are part of the installation and they are listed in the combobox `ODBC Driver` (see also ***3.2.7.6.1 Configure an Oracle ODBC connection*** and ***3.2.7.6.2 Configure a Microsoft SQL Server ODBC connection***).



PROVEtech:TA installs all supported ODBC drivers separately inside the specified installation directory. That means, the ODBC Driver Manager of your Windows control settings are skipped.

The `User` and `Password` fields refer to the database user, but not the PROVEtech:TA user. The database username is set to `provetech` by default (Default password is `PROVEtech:TA`). For security reasons your database administrator should change this name and password.

To connect to the database with your current Windows user instead of a database user, select `Windows Authentication`. This avoids entering a database user plus password and transfer these inside the ODBC connection string. Please read also in chapter ***3.2.7.6.4 Windows Authentication***.

In case of PostgreSQL the properties `Server`, `Port` and `Database` are necessary in order to locate the PROVEtech:TA database instance and establish communication. You should be able to access the server domain address, and the firewall on server and client side must not block the port.

The checkbox `Show ODBC alias always in TM tree (in case of multiple connections)` allows you to display the test structure of multiple databases simultaneously in the Test Manager tree. Please read more about this in chapter ***5.13 Multiple databases in the Test Manager tree***.

If the database connection is very slow (e.g. due to a distant server) then you can check `Slow DB connection (Reduce amount of SQL queries)`. This blocks operations that are not mandatory for the basic functionality but produce much network traffic (e.g. the Overview tab, the History tab, comparing whole subtrees or the Extended tab in the property dialog of a test object). Caution, the execution time of a single SQL query is not influenced by this parameter.

Test ODBC connection

Clicking on the `Test ODBC connection` button initiates an attempt to connect to the selected database. You can find more about this in chapter ***3.2.7.6.3 Test ODBC connection and further database checks***.



If the attempt to establish a database connection is unsuccessful, you can find more about possible reasons in chapter ***8.4.1.2 PostgreSQL error messages***.

Check database consistency

In addition, PROVEtech:TA administrator has activated the button `Database consistency`, which checks the database integrity (consistency of data) and test the compatibility of existing data with current PROVEtech:TA version (see also chapter ***5.12 Database consistency checks***).

Database backup

You can back up a selected database using the button `Create database backup` (see ***Figure 55***). Typically, this is very useful if your database server is not administrated by your IT, e.g. in case of a local database server on your computer. Please read in chapter ***3.2.7.6.5 Configure the database backup*** and chapter ***5.11 Create and restore a database backup*** for more information.



You can read more about global database settings in chapter ***5.3.11.2 The tab "Database settings" in the property dialog***.

3.2.7.6.1 Configure an Oracle ODBC connection

If Oracle is selected, then the property page `Database` looks like the following.

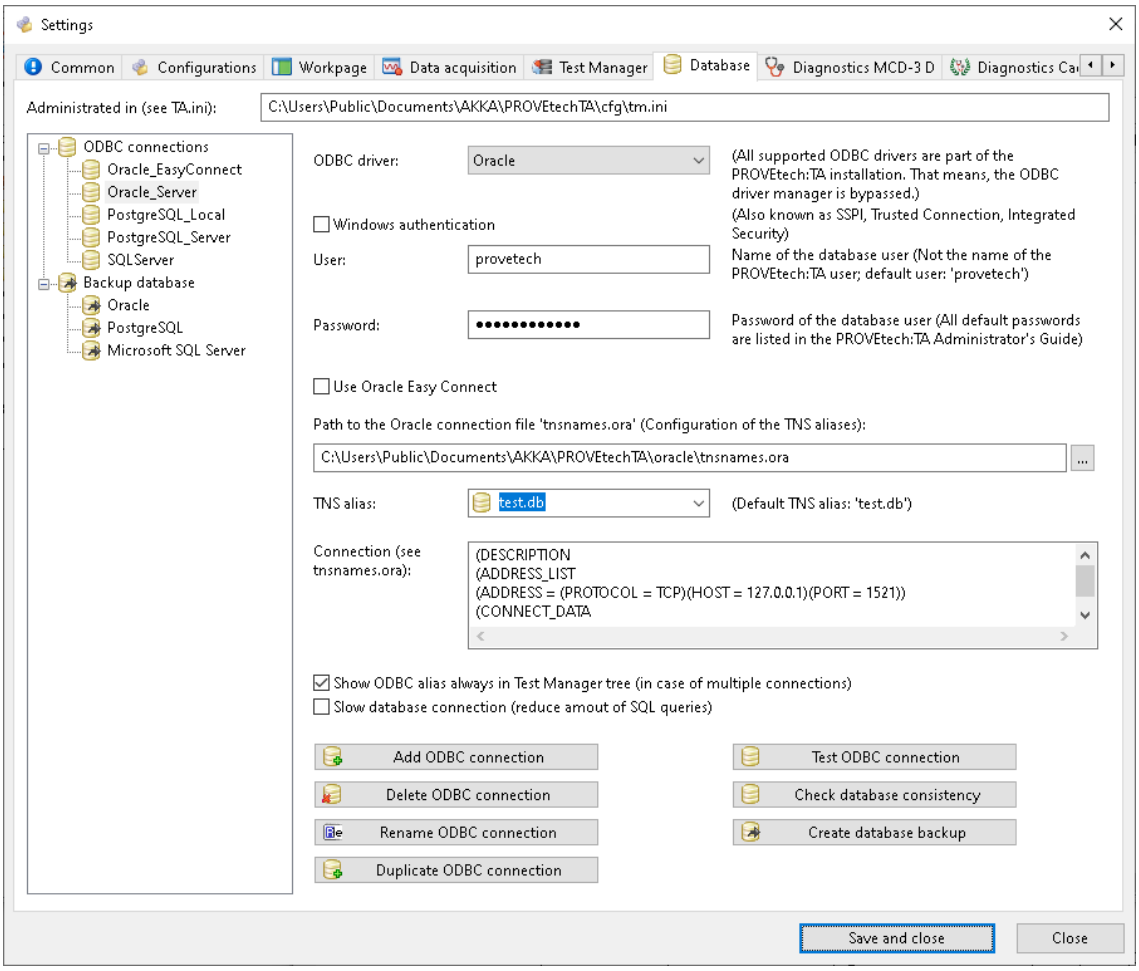


Figure 56 – Oracle in the property page Database

To connect to the database with your current Windows user instead of a database user, select `Windows Authentication`. This avoids entering a database user plus password and transfer these inside the ODBC connection string. Please read also in chapter ***3.2.7.6.4 Windows Authentication***.

You can use Oracle tools to configure the computer address, the SID of your database instance and the communication port of your database connection and store this in the Oracle file tnsnames.ora. After that, you only need to set the `Database Alias` in the PROVEtech:TA setting dialog. If the path to the file tnsnames.ora is not automatically found, you can set it by clicking the button "…". In the tnsnames.ora the entries typically have the following form:

```
<Database Alias> =
  (DESCRIPTION =
    (ADDRESS_LIST =
      (ADDRESS = (PROTOCOL = TCP)(HOST = <Computer name with domain or IP ad-
dress of the server>)(PORT = <Port number>)
    )
    (CONNECT_DATA =
      (SID = <Database name>)
    )
  )
```

If you want to connect to an Oracle server without using the tnsnames.ora file and database alias, then you can `Use Oracle Easy Connect` (see ***Figure 57***). You only need to specify the host name of the server, the port for the TCP/IP connection and the database name (SID) directly in PROVEtech:TA. As a requirement, *Oracle Easy Connect* must be enabled on the Oracle server.

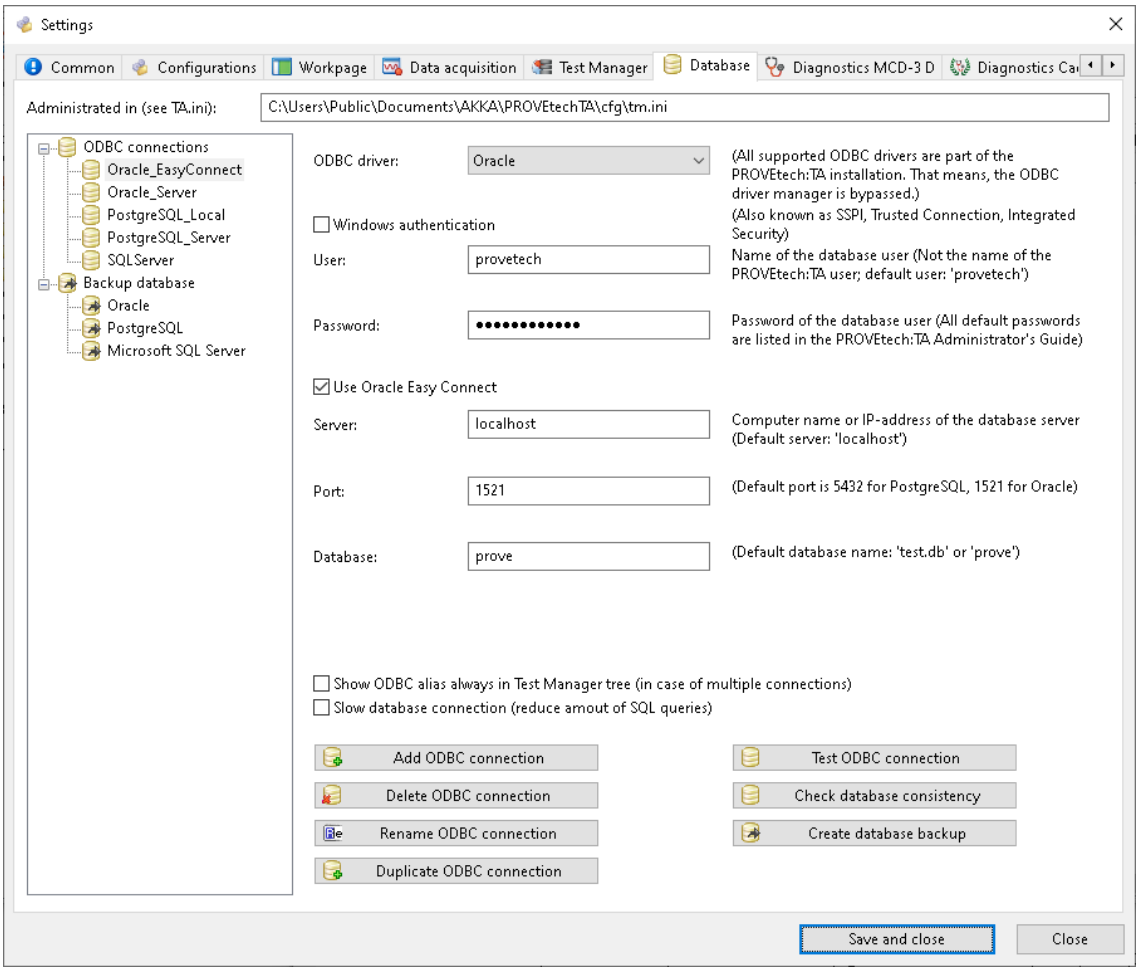


Figure 57 – Oracle connection by using Oracle Easy Connect

If the attempt to establish a database connection is unsuccessful, you can find more about possible reasons in chapter ***8.4.1.1 Oracle error messages***.

3.2.7.6.2 Configure a Microsoft SQL Server ODBC connection

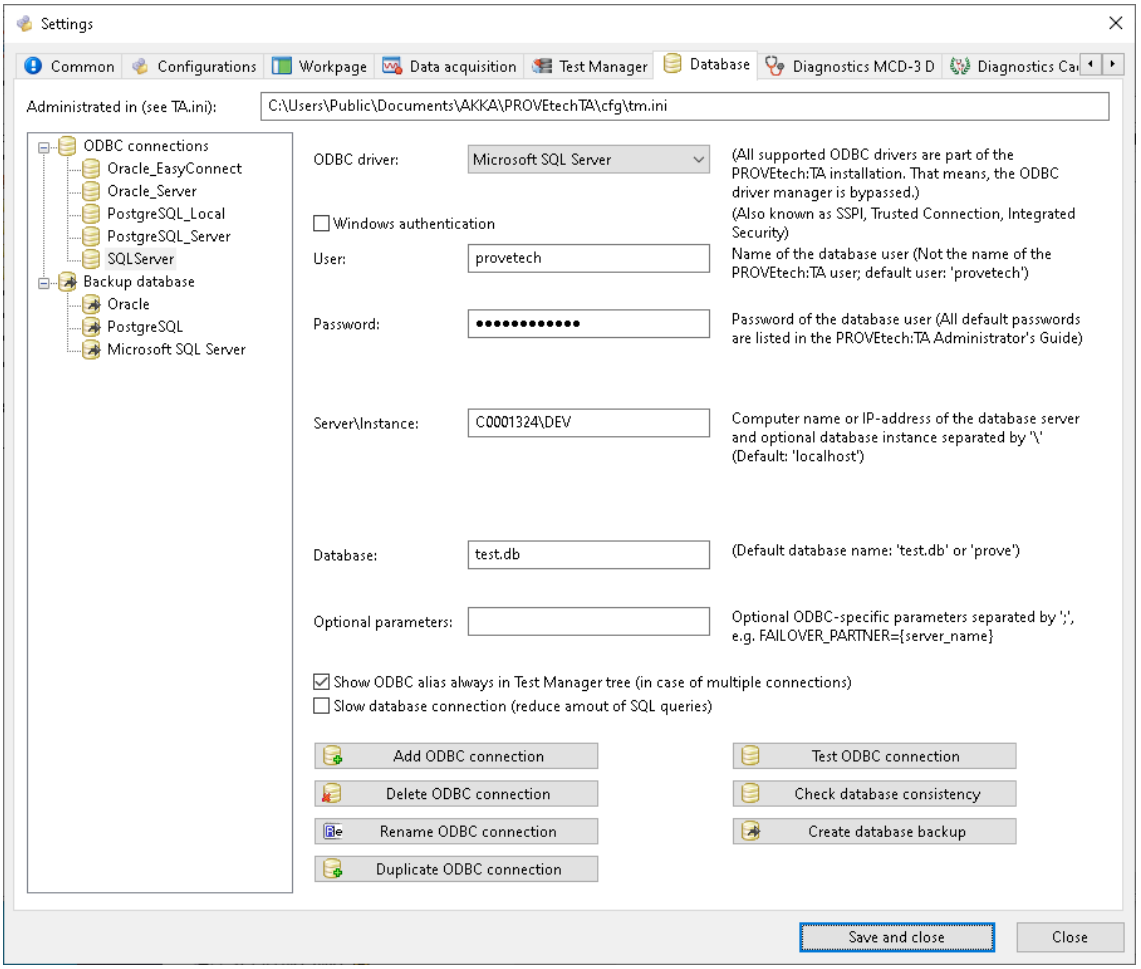


Figure 58 – The Microsoft SQL Server in the property page Database

If you want to connect to a Microsoft SQL Server database then you choose (similar to PostgreSQL) the `ODBC Driver`, `User` and `Password` of the database user, the `server` (possibly the database `instance` including, separated by '\') and the name of the `Database`.

To connect to the database with your current Windows user instead of a database user, select `Windows Authentication`. This avoids entering a database user plus password and transfer these inside the ODBC connection string. Please read also in chapter ***3.2.7.6.4 Windows Authentication***.

You can define additional ODBC parameters in `Optional parameters`. An example of this would be `TrustServerCertificate=No;Encrypted=Yes`.

You can find more information in chapter ***3.2.7.6 Database***.



If the attempt to establish a database connection is unsuccessful, you can find more about possible reasons in chapter ***8.4.1.3 Microsoft SQL Server error messages***.

3.2.7.6.3 Test ODBC connection and further database checks

Clicking on the `Test ODBC connection` button on the `Database` tab of the settings dialog initiates an attempt to connect to the selected database. In addition, the database server and client version are displayed, as shown in the following dialog after successful database connection.

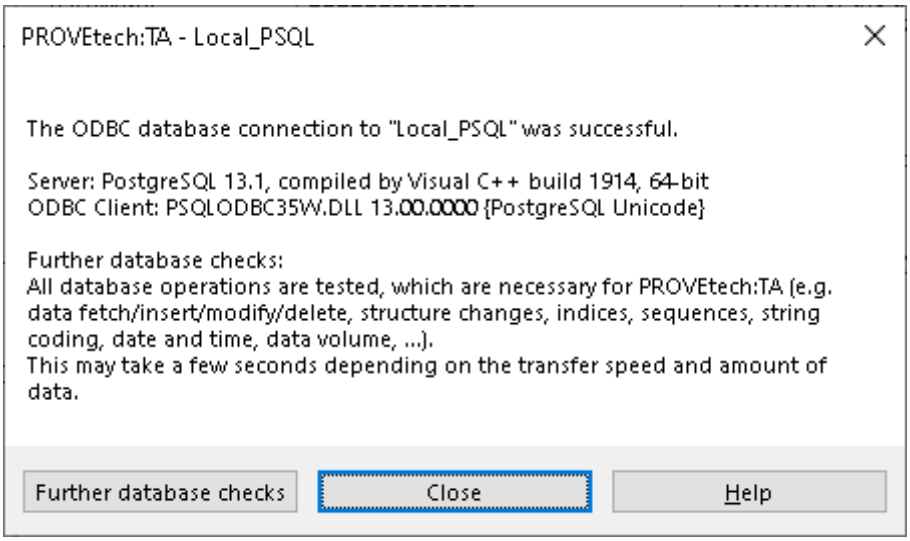


Figure 59 – Test database connection

By clicking the button `Further database checks`, PROVEtech:TA performs additional operations to check the health of the database. This involves all basic functions for reading and writing data, as well as database tables, data types, server settings, tablespaces, character encoding, indices, sequences, timestamps, PROVEtech:TA client connections, large objects, storage space and others.

Further database checks are also carried out immediately after setting up the database schema, after the first PROVEtech:TA client has connected to the database. It is recommended that any error messages and warnings that occur are followed up immediately before the database is filled with data.



If the attempt to establish a database connection is unsuccessful, you can find more about possible reasons in chapter ***8.4.1 Problems when connecting to the database***.

3.2.7.6.4 Windows Authentication

When connecting to a database via ODBC, as it is the case with PROVEtech:TA, a so-called ODBC connection string is transmitted to the database server for login. This string historically contains a username and a password, which, depending on the database system, are sometimes even transmitted unencrypted over the network. This creates the risk that, for example, packet sniffers can read the logon credentials (see also in chapter ***8.3.8 Data security and avoiding data loss in the Test Manager***).

Windows Authentication is about eliminating the transmission of logon information in the ODBC connection string over the network. This means that there is no longer a user and password in the ODBC connection string. The logged-in Windows user of the client computer is also registered on the server and is used for the logon. If the client and server are in the same Windows domain, and it is assumed that the server and clients are trusted, Windows Authentication significantly reduces the risk that credentials can be found out over a non-trusted network.

The disadvantage of this method, however, is that registering Windows users or user groups on the database server creates additional administrative and synchronization overhead.

To connect to the database with your current Windows user instead of a database user, select the checkbox `Windows Authentication`. This avoids entering a database user plus password, and it avoids transferring these inside the ODBC connection string. Windows Authentication is also known as SSPI (Microsoft's Security Support Provider Interface), Trusted Connection or Integrated Security.

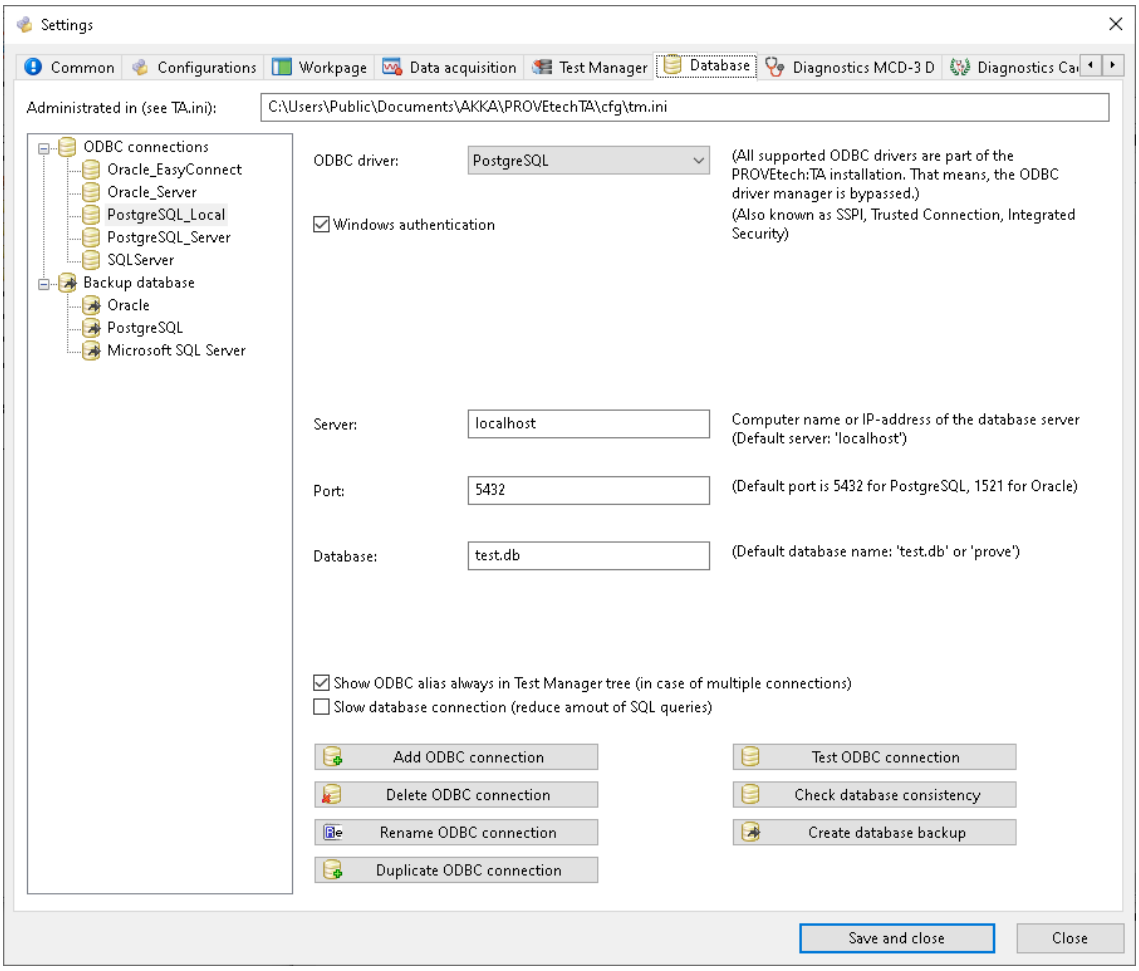


Figure 60 – Database logon via Windows Authentication

Depending on the database system, the following requirements must be met for Windows Authentication.

PostgreSQL

The connection to the database is done via SSPI authentication. This authentication method must be specified in the pg_hba.conf file, and the mapping between system and database usernames must also be specified.

```
# TYPE   DATABASE      USER          ADDRESS         METHOD
host     test.db       all           0.0.0.0/0       sspi map=SSPI
```

In pg_identity.conf, the domain control of the connected user must be set up by a regular expression.

```
# MAPNAME SYSTEM-USERNAME PG-USERNAME
SSPI      /^(.*)@MyDomain$     \1
```

You must create a database user for each domain user and set the appropriate permissions for that user. For example:

```
CREATE ROLE "john.smith" WITH
LOGIN
NOSUPERUSER
NOINHERIT
NOCREATEDB
NOCREATEROLE
NOREPLICATION;
GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA public TO "john.smith";
GRANT ALL PRIVILEGES ON ALL SEQUENCES IN SCHEMA public TO "john.smith";
```

Oracle

With SYSDBA role by SQL, you need to create a database user for each domain user and set the appropriate permissions for that user. (Beware of case-sensitivity.)

```
CREATE USER "OPS$<DOMAIN>\<USER>" IDENTIFIED EXTERNALLY;
GRANT CONNECT,RESOURCE TO "OPS$<DOMAIN>\<USER>";
GRANT ALL PRIVILEGES TO "OPS$<DOMAIN>\<USER>";
```

The file sqlnet.ora in the Oracle configuration directory should contain the following entry:

```
SQLNET.AUTHENTICATION_SERVICES=(NTS)
```

This value enables the Network Authentication Services (NTS) available in Windows.

Set in TM.ini according to the ***Administrator's Guide*** the appropriate current database schema for the database connection with PROVEtech:TA, e.g. PROVETECH.

```
[DBCurrentSchema]
MyOracleAlias=PROVETECH
```

(Please read also on https://stackoverflow.com/questions/8837458/oracle-connection-string-with-windows-authentication)

Microsoft SQL Server

The SQL Server has two authentication modes:

- Windows Authentication
- SQL Server and Windows Authentication

In both modes, you can login to the database through a domain user. No special configuration is required. You need to create database login and database user. This could be a domain user or domain group. For example:

```
USE [master]
CREATE LOGIN [<DOMAIN>\<user or group>] FROM WINDOWS WITH
DEFAULT_DATABASE=[master]
```

```
USE [test.db]
CREATE USER [<DOMAIN>\<user or group>] FOR LOGIN [<DOMAIN>\<user or
group>]
USE [test.db]
ALTER USER [<DOMAIN>\<user or group>] WITH DEFAULT_SCHEMA=[dbo]
ALTER ROLE [db_owner] ADD MEMBER [<DOMAIN>\<user or group>]
```

You can find more information at https://www.mssqltips.com/sqlservertip/6702/sql-server-windows-authentication-with-users-and-groups/

Error messages

The following error messages indicate that the connection establishment via ODBC connection string using Windows Authentication does not work:

PostgreSQL

```
ODBC Error: fe_sendauth: no password supplied  (lNativeError=101,
wstrSqlState=08001)
```

Oracle

```
ORA-01017: invalid username/password; logon denied  (lNativeError=1017,
wstrSqlState=28000)
```

SQL Server

```
[SQL Server]Login failed for user '<domain>\<user>'. (lNativeEr-
ror=18456, wstrSqlState=28000)
```

3.2.7.6.5 Configure the database backup

Please keep in mind to regularly perform a database backup if your database server is not administrated by your IT, e.g. in case of a local database server on your computer.

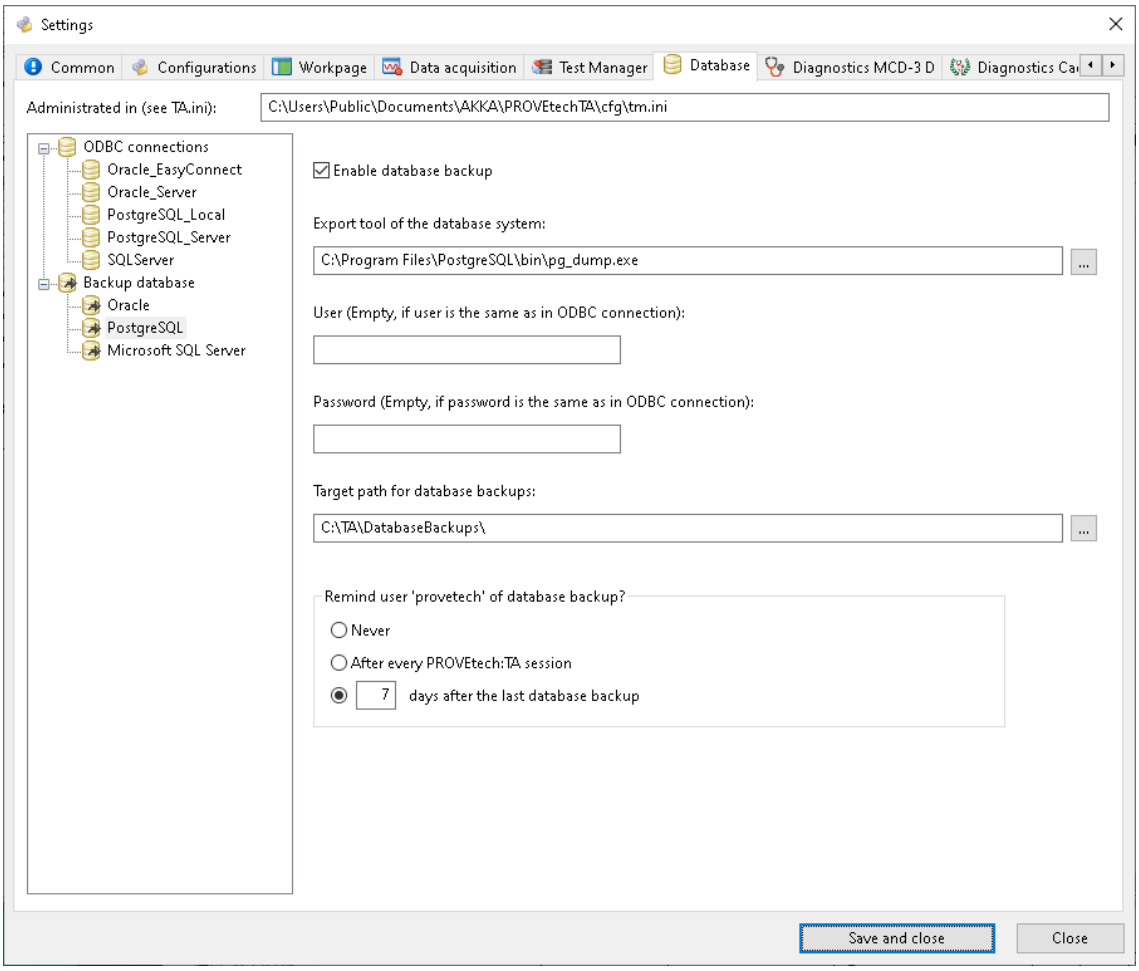


Figure 61 – Database backup in the property page Database

In the subtree `Database Backup` you can configure the backup settings depending on the database type (see also ***5.11 Create and restore a database backup***). Once you've defined an ODBC connection, its database type is displayed in the backup subtree.

In `Activate database backup` you can activate or deactivate the database backup reminder for this type of database. The `Export tool of the database system` is automatically set, however it can also be set manually. The following table shows the default tools for the database backup in PROVEtech:TA:

Database system	**Tool**
Oracle	exp.exe
PostgreSQL	pg_dump.exe
MS SQL Server	sqlcmd.exe

The database `User` and `Password` should not be confused with the PROVEtech:TA user and password. Backup files and their .log files are stored in the path indicated in `Target path for database backups`.

In case of problems see chapter ***8.4.11 Failed to create a database backup***.



If you are using **Microsoft SQL Server**, then the `Export tool of the database system` refers to the server computer. That means, the backup file is stored on the server computer if the path exists and the appropriate access rights are set.

`Remind user '...' of database backup` is a user-specific setting and is stored in the user.ini. When you terminate PROVEtech:TA, the application checks when the last backup was logged in the database.

PROVEtech:TA provides the possibility to automatically create backup files of Basic code, description text and test parameters.

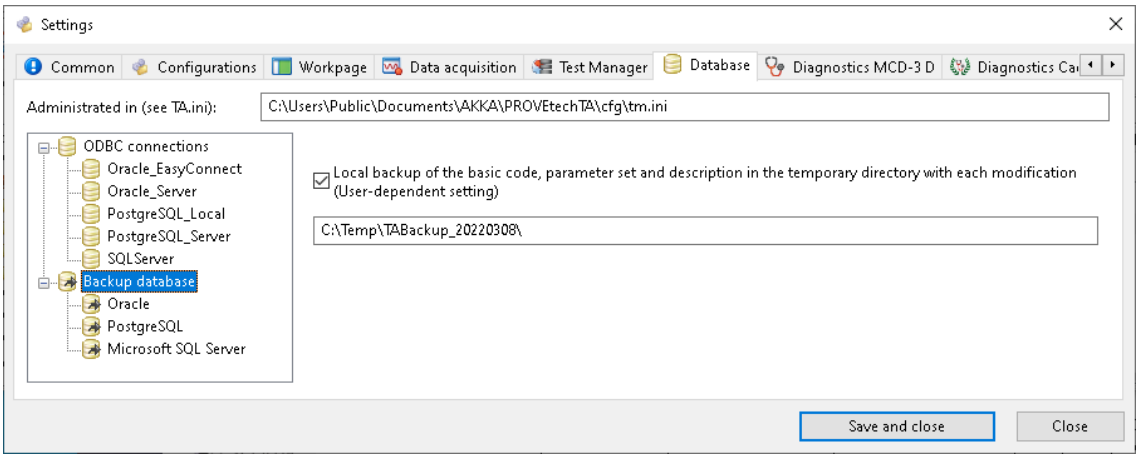


Figure 62 – Automatic backups of Basic code and description

The backup files are stored in the temporary directory of PROVEtech:TA (see ***Administrator's Guide***, chapter "TA.ini", section [Path], entry "Temp", as well as chapter ***3.2.7.1 Common*** in the field `Path for temporary files`) and kept for 7 days. Each time you perform a save operation the old and new content is stored. This option is stored user-dependent in the User.ini.

3.2.7.6.6 Re-index the database

With large amounts of data in the database it can happen that despite fast network connection the response time of the database is slow. A reason for this could be that the database already exists for several years and that because of earlier PROVEtech:TA versions the indexes on the database tables are not correctly set. Fragmentation of the indexes is also possible, as these have grown over a longer period.

This can be checked via the button `Further database checks` (see chapter ***3.2.7.6.3 Test ODBC connection and further database checks***). If you receive warning messages, you can consider re-indexing the database.

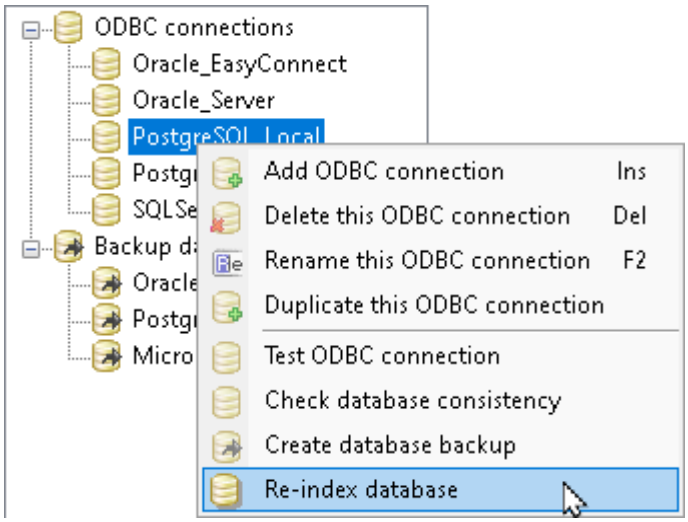


Figure 63 – Re-index the database

To do this, log in as PROVEtech:TA administrator 'admin' and start this operation. This operation may take several minutes. Before you start, you should make sure that a timely database backup exists (see chapter ***5.11 Create and restore a database backup***).

See also chapter ***8.4.8 The database is very slow***.



For re-indexing directly by the PROVEtech:TA client, the database user via which you are connected to the database (that means, not the PROVEtech:TA user) must have access rights to change the database schema, at least, temporarily. The reason for this is that all PROVEtech:TA indexes are deleted and recreated.



Re-indexing can also be carried out when database replication is active. In this case, re-indexing may have to be performed separately on each replication node.

3.2.7.7 CAESAR diagnostics settings

If you have installed the CAESAR diagnostics on your computer, the property page `Diagnostics` corresponds to the description of this chapter (see also chapter ***6.1 CAESAR diagnostics***).

A description of the property page for the CAESAR diagnostics follows (***Figure 64***):

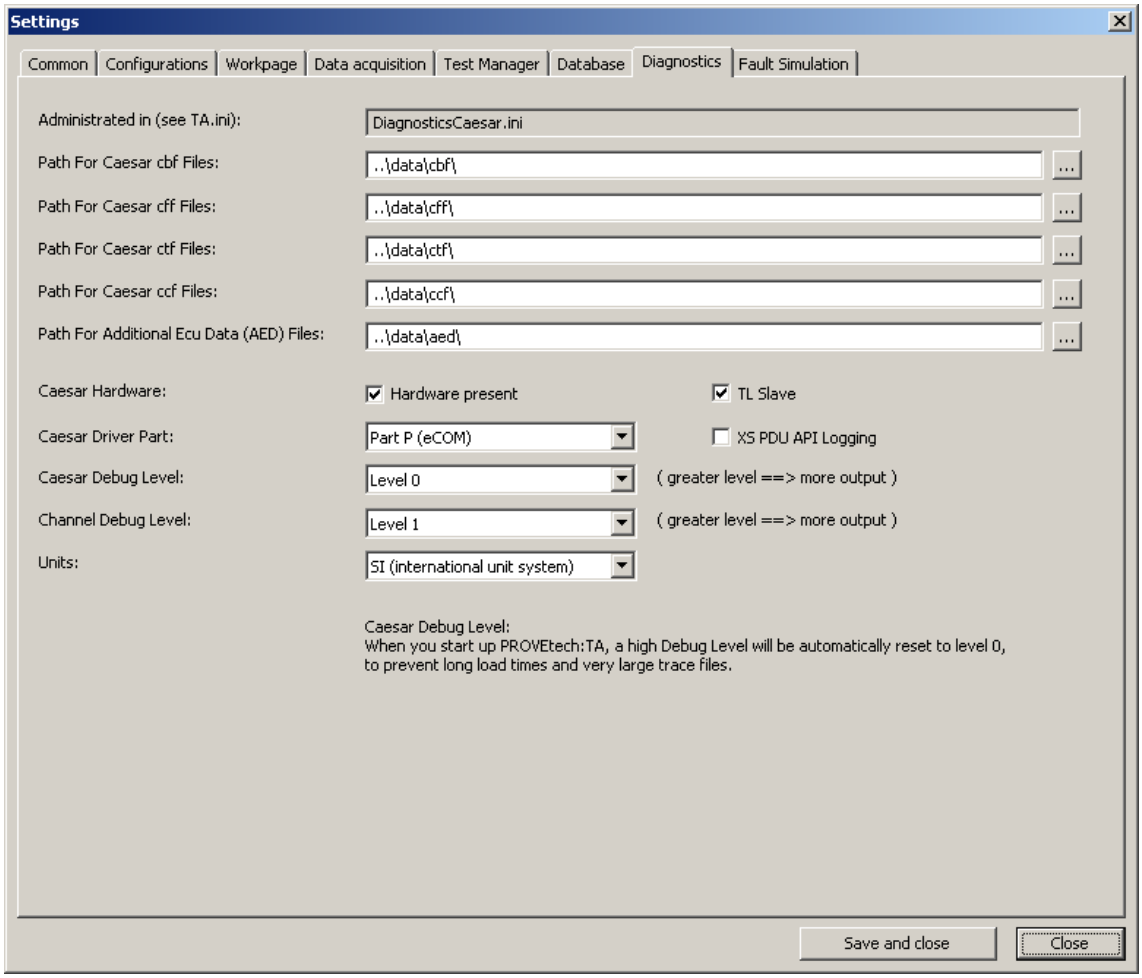


Figure 64 – The diagnostics property page for the CAESAR diagnostics

`Administrated in (see TA.ini)`: This file contains the diagnostics settings.



The paths for CAESAR driver and .gbf files are not configurable. They are in <PROVEtechTADirectory>\bin\Caesar.

`Path For Caesar cbf Files`: A .cbf file contains the CAESAR parameters for an ECU. You can specify the directory containing these files in this text box.

`Path For Caesar cff Files`: A .cff file contains the information for flashing an ECU. This path setting tells PROVEtech:TA where to find the .cff files.

`Path For Caesar ctf Files`: A .ctf file contains CAESAR text files which are the translations of the texts of the CAESAR binary files. This path setting tells PROVEtech:TA where to find the .ctf files.

`Path For Caesar ccf Files`: A .ccf file contains the varcoding data for coding the ECU. This path setting tells PROVEtech:TA where to find the .ccf files.

`Path For Additional ECU Data (AED) Files`: An .aed file contains the varcoding strings for the ECU. This path setting tells PROVEtech:TA where to find the .aed files.

`Caesar Hardware`: Select the option `Hardware present` if there is a CAESAR hardware, installed on your computer. If you do not select this option, the diagnostics module cannot access the hardware. Select the option `TL Slave`, if you want to use the improved capabilities of the so called TL Slave firmware. Some proprietary protocols and ECUs are not supported in this mode. In case the option TL Slave is deactivated, CAESAR uses the firmware CaesarGO, which does not support some current protocols.

`Caesar Driver Part`: Select the correct driver type for your CAESAR hardware. Nowadays this is typically Part P for eCOM or Part J for Nexiq / Pass-Thru (see chapter ***8.8 Problems concerning the eCOM*** and ***8.9.4 Nexiq Device / Part J / Pass-Thru***).

`XS PDU API Logging`: With this option you can turn the logging of the XS PDU API on or off. This action alters the .ini file of the used XS PDU API. When the logging is activated logging files are created in the folder that is entered in the .ini file.

`Caesar Debug Level`: With this combo box, you can control the amount of additional messages of the CAESAR database component, which are displayed in the PROVEtech:TA Log window (see chapter ***3.6.1 PROVEtech:TA Log***). The higher the selected level, the more information is reported. When PROVEtech:TA starts, the CAESAR Debug Level is automatically limited to level 0.

`Channel Debug Level`: With this combo box, you can control the amount of additional messages of the CAESAR communication component, which are displayed in the PROVEtech:TA Log window (see chapter ***3.6.1 PROVEtech:TA Log***). The higher the selected level, the more information is reported. When PROVEtech:TA starts, the Channel Debug Level is automatically limited to level 0.

`Units`: With this combo box, you can toggle between SI and US units.

3.2.7.8 MCD-3 D diagnostics settings

If you have installed the MCD-3 D diagnostics on your computer, the property page `Diagnostics` corresponds to the description of this chapter (see also chapter ***6.2 MCD-3 D diagnostics***).

The MCD-3 D diagnostics property page allows to handle several diagnostics configurations. Every TA configuration can be assigned to a diagnostics configuration.

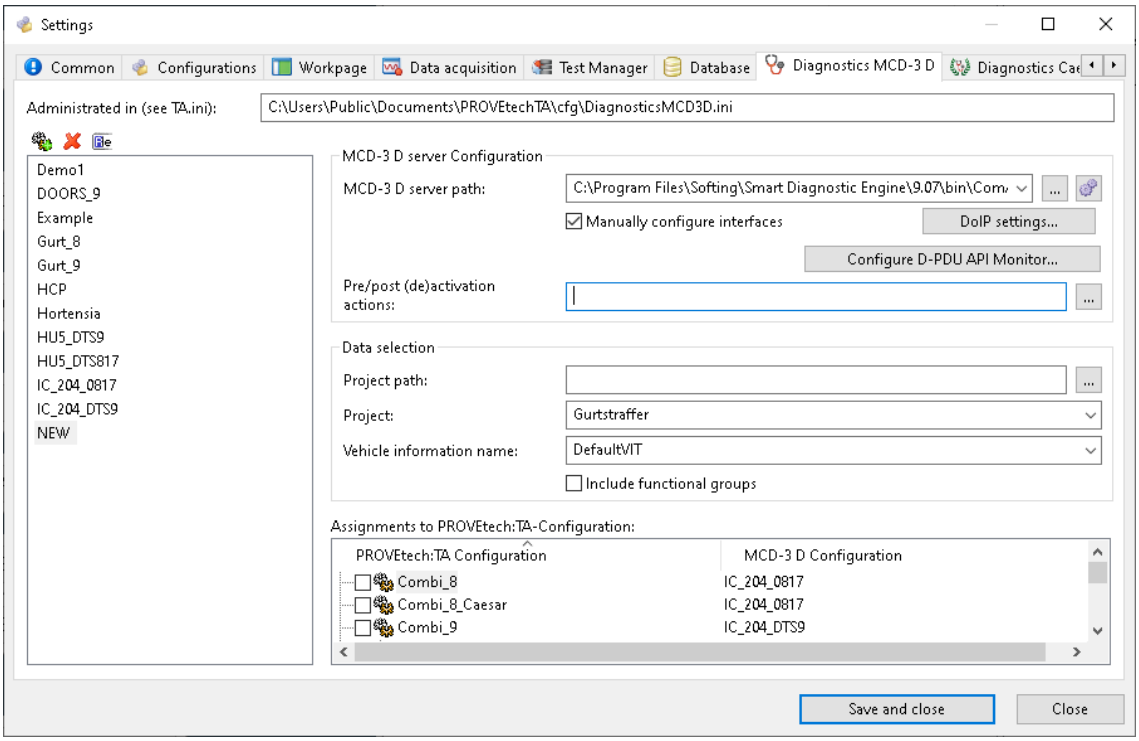


Figure 65 – The property page for MCD-3 D diagnostic configurations

You can generate a new configuration via the context menu item `Add configuration` in the left list or via the keyboard with the “Ins” key when the focus is in the list.

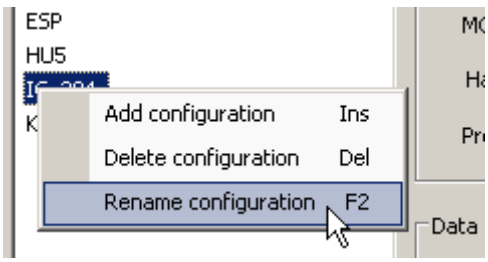


Figure 66 – Options in the context menu

The default name is CONFIG*n*, with *n* representing the number of the currently existing configurations. You can rename your configuration via context menu, hotkey F2 or . Exceptions are the keywords and special characters of the DiagnosticsMCD3D.ini.

Use the combo box `MCD-3 D server path` to enter the DLL path of the MCD-3 D server. For the Softing DTS8 server the DLL has the name DTS8.dll, for DTS9 it is ComApi.dll and for the DSA PRODIS server it has the name AsamDsaDCom.dll. In case of the VW kernel, the file's name is McdKernelDCom_<Version>.dll. The drop down list of the combo box initially contains the paths found via the Windows registry and in the configuration file. Relative paths reference the "bin" directory of PROVEtech:TA.

The button  next to the server selecting combo box opens the configuration of the selected diagnostics server. In case of Softing the SstemConfiguration.exe is started, in case of DSA or VW a text editor opens the current configuration file. Please refer to ***8.10.1.4 Configuring DTS8 and DTS9*** and ***8.10.1.5 Configuring DSA PRODIS and VW kernel*** for details about configuring diagnostics servers. Please consider that you need to restart the diagnostics module (in case of DSA or VW) or the whole program PROVEtech:TA (when DTS is in use) for configuration changes to take effect.

If an interface is to be configured manually (e.g. when using Ethernet or switching to different CAN channels), set the check mark in the check box `Manually configure interfaces`. In case of Ethernet (DoIP) further entries are needed via the `DoIP settings` button, as described in ***3.2.7.8.1 DoIP configuration***.

The button `Install D-PDU API monitor` allows to activate a monitoring channel (see chapter ***3.2.7.8.2 D-PDU API Monitor***)

With "Pre/post (de)activation actions" you can set a test or object in the Test Manager or a MAC file in the file system. Here you can define actions to be automatically executed before and after activation or deactivation. See chapter ***6.2.2.17 Activation*** for details.

Special case: if you use DTS8, you can temporarily overwrite the DTS project path by entering it into the text field `Project path`.

By clicking on one of the combo boxes `Project` or `Vehicle information name` the project information of the currently selected MCD-3 D server, if not already done, will be loaded. This might take some time. You may select a project with the opened combo box `Project`, that you have configured within the MCD-3 D server before. Subsequently you may select the preferred vehicle information with the corresponding combo box. For each project several vehicle information can exist.

In the list `Assignments to PROVEtech:TA-Configuration` you may establish links between the MCD-3 D configuration and the PROVEtech:TA configurations (see chapter ***3.2.7.2 Configurations***). By loading the PROVEtech:TA configuration the corresponding MCD-3 D configuration with the selected project and vehicle information will be loaded as well. Set a check mark for every PROVEtech:TA configuration you want to link to the currently selected MCD-3 D configuration.

The check box `Include Functional Groups` determines if the so called functional groups should be included in the ECU list or if they should not be available (see also chapter ***6.2.3.4.1.2 Functional group***).

3.2.7.8.1 DoIP configuration

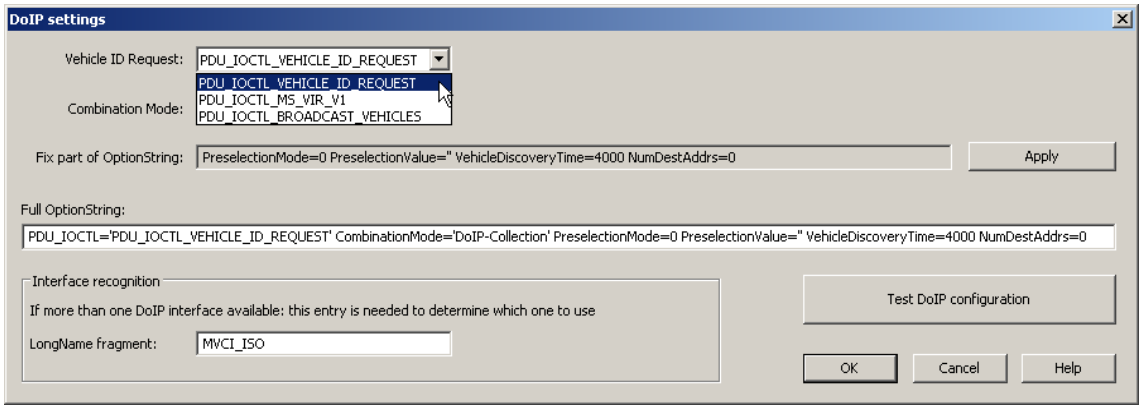


Figure 67 – DoIP configuration

The leading role in DoIP is played by the so-called OptionString, which is to be passed by the MCD3D kernel to the D-DPU API during interface initialization (see also ***6.2.2.14.1 Special Case Diag Over IP (DoIP)*** and ***8.10.1.4 Configuring DTS8 and DTS9***). Diverse features and aspects of the communication depend on different parameters included in the OptionString. Details about those parameters are defined in ISO 22900 and ISO 13400.

If OptionString and name fragment stay empty, PROVEtect:TA tries to activate the interface with minimal settings. With some older D-PDU API and earlier DoIP-capable ECUs this might be sufficient.

The first two parameters – PDU_IOCTL and CombinationMode – have to be adapted for every version of D-PDU API and different ECUs. The dropdown boxes `Vehicle ID Request` and `Combination Mode` offer lists of valid values. Which values are to be used in your system – this fundamental question can be answered by the developer of the ECU or found out by try & error.



Some wrong parameter values might cause a crash of the D-PDU API (e.g. “DoIP-Collection” used with Bosch D-PDU API ver. 5.24). This is not an error of PROVEtech:TA and cannot be handled by our application. Therefore, we recommend to save your configuration temporarily before you start the test of the DoIP configuration.

The remaining parameters do not need to be changed very often. You can leave them blank or fill with default values. This `Fix part of OptionString` can be defined for all diagnostics configurations in the file *DiagnosticsMCD3D.ini* in section *[General]* under the key *DoIPOptionStringFix*.

Caution! earlier versions of DTS (before 8.17) do not follow the ISO standard in naming the parameters of the option string. In case of an upgrade to DTS 8.17 or later, it might be necessary to replace some parts of the string. Otherwise the initialization of the interface will not work. Especially, replace
VehicleDiscoveryTime=2000 NumDestAddr=0
with
VehicleDiscoveryTime=‘2000‘ DestinationAddressCount=‘0‘

Besides the possibility to combine the OptionString using predefined values, you can edit the resulting string at your will in the edit line `Full OptionString`. Clicking the OK button will save the content of this edit line in the configuration. Please keep in mind that wrong parameter values or formatting errors will result in error message when trying to initialize DoIP.

The `Name fragment for interface identification` is needed when the MCD-3 D system contains more than one DoIP interface (e.g. Bosch D-PDU API ver. 5.28 ff might offer two separate interfaces differing in the ISO 13400 obedience). As there is no explicit method to identify the type of the interface, sometimes it might be necessary to distinguish the proper interface by a fragment of the LongName (this string is case-sensitive).

By clicking the button `Test DoIP configuration` you can start the initialization routine with the currently set parameter values. The D-PDU API software tries to establish an Ethernet connection to the ECU(s), which must be therefore connected and in a proper state (e.g. rest bus simulation active).

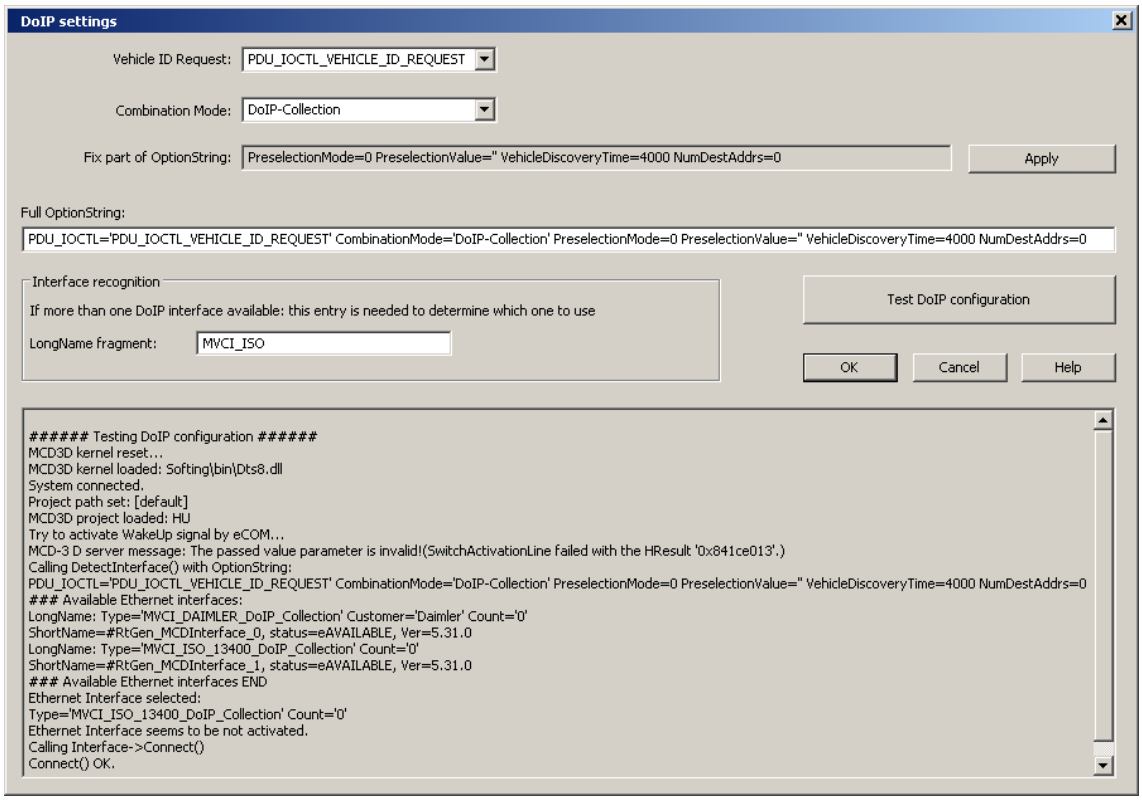


Figure 68 – testing the DoIP configuration

After the configuration test (which can last up to half a minute) you find in the lower window some entries, which can be helpful to find errors and establish the proper configuration. If you need help from support or a DoIP expert, you can copy the text and paste it into an email. To understand all entries, some basic DoIP knowledge is necessary. A simple decision – OK or not OK – is most often not possible.

These are the most important information you get from the log:

- Has the MCD-3 D system been started and the project loaded
- Is the WakeUp signal switched on (only when eCOM hardware is in use)
- Did the software accept the OptionString (i.e. there are no format errors)
- Are there one or more interfaces detected, are they active
- Which one (of several interfaces) has been selected for usage.

Hint 1: use only one D-PDU API

All D-PDU API are entered into the RDF file when installed (see ***8.10.1.4 Configuring DTS8 and DTS9*** for more details). If the RDF contains more than one DoIP interface, it cannot be determined in which order they get initialized and if there happen conflicts between them. We recommend to create a separate RDF file for PROVEtech:TA usage

only and leave only the one interface you want to use for DoIP (and an additional interface for CAN communication if needed).

Hint 2: monitoring the network

A monitoring tool (e.g. Wireshark®) can be used to check the network traffic during initialization. In this phase a significant increase of messages should be visible. A Wireshark trace might be helpful when problems occur.

Hint 3: use direct network connections only

We detected that using USB2Lan adapters might lead to problems with DoIP. To exclude all possible error causes it is recommended to connect the ECU directly with the Ethernet card of the PC and disconnect the PC from intranet and all other networks during configuration. After finding the proper OptionString and establishing a stable communication you can try in a next step to use other connectors and networks.

3.2.7.8.2 D-PDU API Monitor

Clicking the button `Configure D-PDU API Monitor…` (see ***Figure 65***) opens the following dialog:

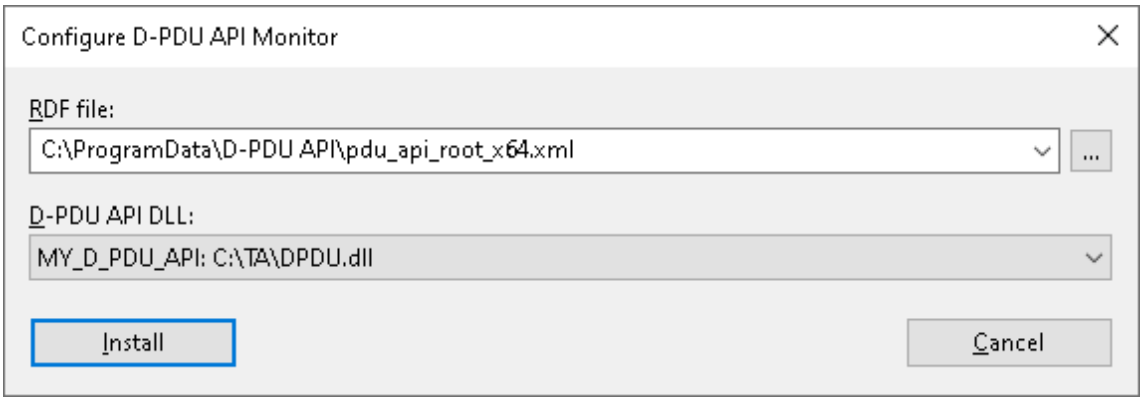


Figure 69 – Installing D-PDU API Monitor

This helps you to install or uninstall a D-PDU API monitor. It is a DLL placed between the MCD-3 D server and the D-PDU API. It grabs all messages (typically UDS protocol) exchanged between the MCD-3 D server and the original D-PDU API and forwards them to the message processing of PROVEtech:TA. It creates a new a dummy model node and a new generic message channel, both with the name “D-PDU API Monitor”. Please be aware that it is a “generic” message channel as the D-PDU API is independent from the physical transport layer like CAN or Ethernet.

The Id of these generic messages is just an incrementing number per created logical link of the D-PDU API and the payload are the bytes exchanged between MCD-3 D and the D-DPU API. Please be aware that messages created by the D-PDU API internally stay hidden, like “tester present” messages as an example.

For installing the monitor, you must change the path to the D-PDU API DLL in the RDF file (XML file containing all installed D-PDU APIs) used by the MCD-3 D server. In the dialog you select that RDF file path via the upper combo box. It tries to detect the RDF path automatically, but that might fail, and you can change that path also manually.

The lower combo box contains all found D-PDU APIs from the selected RDF file. It shows the <SHORT_NAME> and the DLL path of each entry. Additionally, it displays "(Monitor active)" at the end of the entry in case the monitor is already installed for that D-PDU API.

Select the entry to change and then click on `Install` to install the monitor. The button changes to `Uninstall` if you select an entry where it is already installed. With that you can uninstall it. An installation changes the RDF file to point to our "DPDUAPIMonitor.dll" copied to the directory "<PublicDocuments>\PROVEtechTA\DPDUAPIMonitor". This also contains "DPDUAPIMonitor.ini" with all the original D-PDU API DLL file paths.

After installing the monitor and reloading the MCD-3 D project, you can see the generic messages in the message window (see chapter ***4.4.14 The message window***) and you can also fetch them via the ***MsgQueue*** test language class.



Using the D-PDU API monitor requires a loaded model with a TCP/IP connection to ModelCom. It does not work with a dSPACE PHS bus model or an offline model.

3.2.7.9 Fault simulation

The `Fault Simulation` property page described in this chapter is only available when using the fault simulation (see chapter ***7 Fault simulation***).

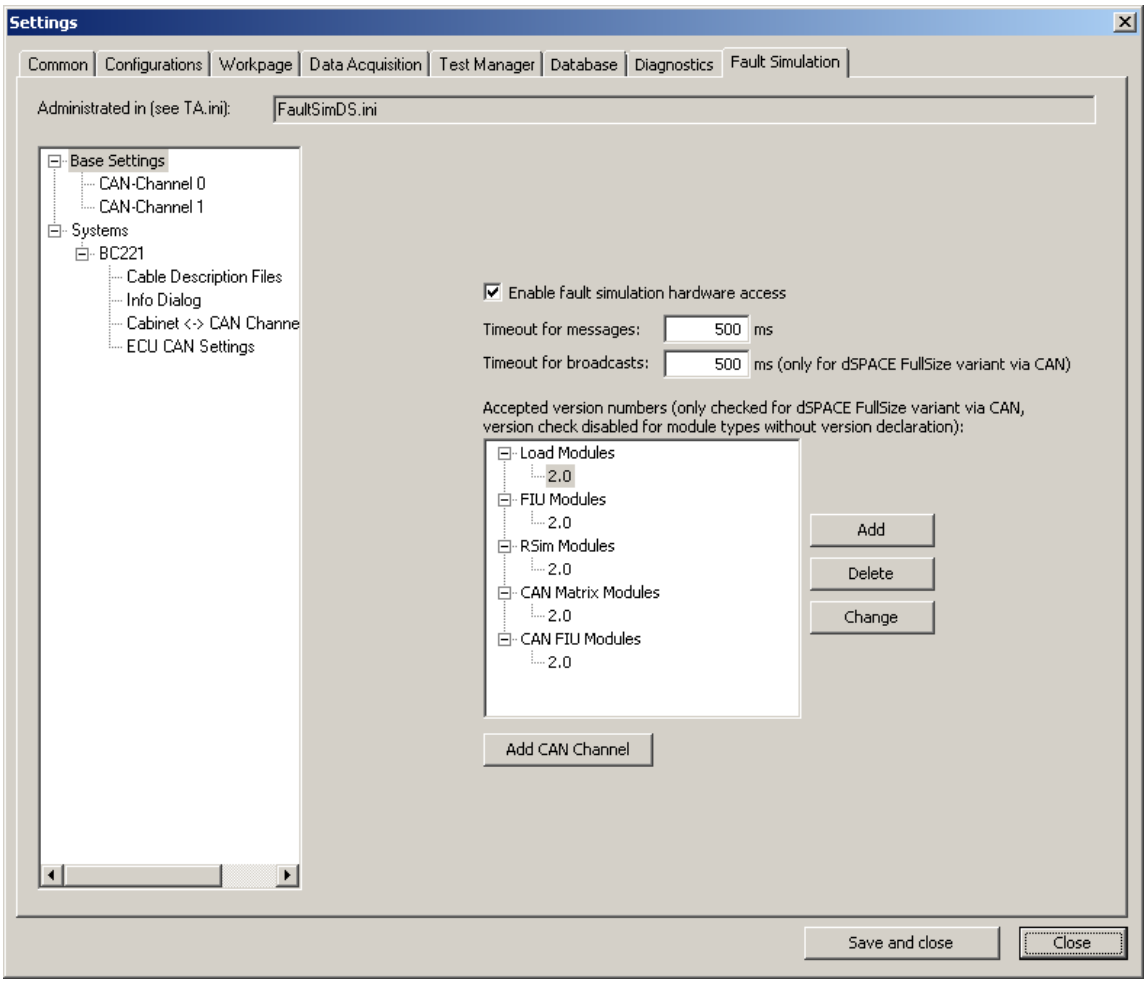


Figure 70 – The fault simulation property page

The `Administrated in` text box of the `Fault Simulation` property page (see ***Figure 70***) shows the name of the file that stores these settings. You can change the file path in the file “TA.ini” with a text editor.

Use the left tree to select the different settings pages of the fault simulation. The lowest level contains the entries “Base Settings” and “Systems”. The context menu has items to create and delete CAN channels and systems and to rename systems.

Base Settings

This page determines the hardware preconditions. When you uncheck the `Enable fault simulation hardware access` check box, PROVEtech:TA supports a simulation mode without hardware access. So you can use the fault simulation user interface and its test language extension, but nothing really happens.

`Timeout for Messages` determines, how long PROVEtech:TA waits for response messages from the fault simulation modules (not used for dSPACE MidSize or dSPACE SCALEXIO variant).

There are additional fields for the usage of dSPACE FullSize fault simulation modules controlled via CAN:
`Timeout for Broadcasts` determines, how long there must be silence on the fault simulation bus, before PROVEtech:TA detects a finished the broadcast.

Enter all version numbers in the `Accepted version numbers` tree that PROVEtech:TA shall accept for the different dSPACE fault simulation modules. PROVEtech:TA deactivates the fault simulation when a module reports an invalid version number.

CAN-Channel x

PROVEtech:TA controls some fault simulation hardware via one or more CAN channels. For each channel you create an entry in the left tree and configure it on the according page.

You can configure PROVEtech:TA to initialize one or several CAN cards of the PC for directly sending and receiving CAN messages. This works via the Vector XL Driver Library thus only supports cards controllable by that library. Select the used hardware type via the combo box `CAN device`. The combo box `Device index` specifies which device to use if there are several ones of the same type installed (0 is the first one). `Channel No. on this device` determines the used hardware channel on the device (0 is the first one). For an easier configuration click on `Select Vector Channel…` to auto-detect all available channels and select one of them.

Instead of selecting a device by its hardware type and index, you can select it via its serial number instead. To do this choose “Detect device via serial number” in the `CAN device` combo box and enter the serial number in the according field.

As an alternative choose the entry “A CAN channel of the model” in the combo box `CAN Board`. Then you enter the name of the according CAN channel provided by your model for sending and receiving the fault simulation messages in the text box `Channel name in the model`. The name corresponds to an entry of the column `Displayed Name` in the list `Message channels` of the model configuration (see ***Figure 33*** of the chapter ***3.2.7.2.3 Models***).



The PROVEtech:TA installer does not install the Vector XL Driver Library. You must download it from the Vector Informatik GmbH Internet page and install it if you need it. PROVEtech:TA searches for “vxlapi64.dll” first in its application directory and then in the Windows system directory.

Systems

Normally the fault simulation only controls one system. But when you want to connect several test systems controlled by one PC, you can enter several systems here. Every system must have a unique name (in this example it is “BC221”), which is also used for the test language. The fault simulation user interface displays the systems in different tabs. The test language distinguishes between them via the unique names.

PROVEtech:TA knows several different fault simulation hardware types. Clicking on `Add System...` opens the following dialog:

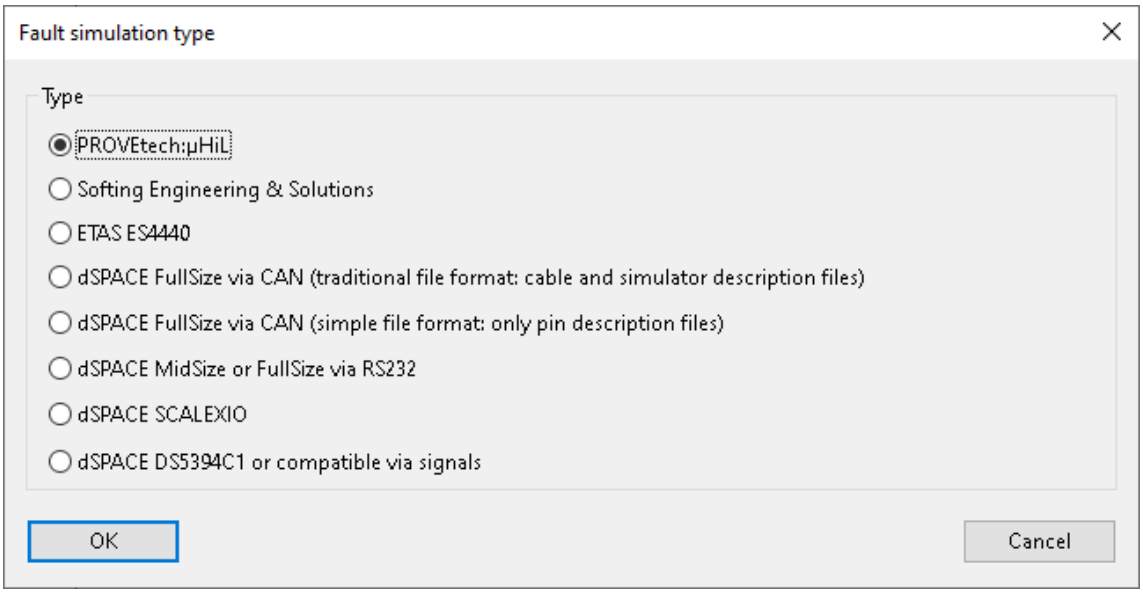


Figure 71 – Fault simulation type

The configuration pages are also different: ***3.2.7.9.1 PROVEtech:µHiL fault simulation***, ***3.2.7.9.2 Softing Engineering & Solutions fault simulation***, ***3.2.7.9.3 ETAS ES4440 fault simulation***, ***3.2.7.9.4 dSPACE FullSize fault simulation via CAN***, ***3.2.7.9.5 dSPACE MidSize or FullSize fault simulation via RS232***, ***3.2.7.9.6 dSPACE SCALEXIO fault simulation*** and ***3.2.7.9.7 dSPACE DS5394C1 fault simulation***. When using the dSPACE FullSize variant via CAN, you have additionally the possibility to choose between the traditional file format and the simple file format. The ***Administrator’s Guide*** describes these file formats. However, the hardware is identical.

3.2.7.9.1 PROVEtech:µHiL fault simulation

PROVEtech:TA displays the following settings page when clicking on a PROVEtech:µHiL system in the left tree:

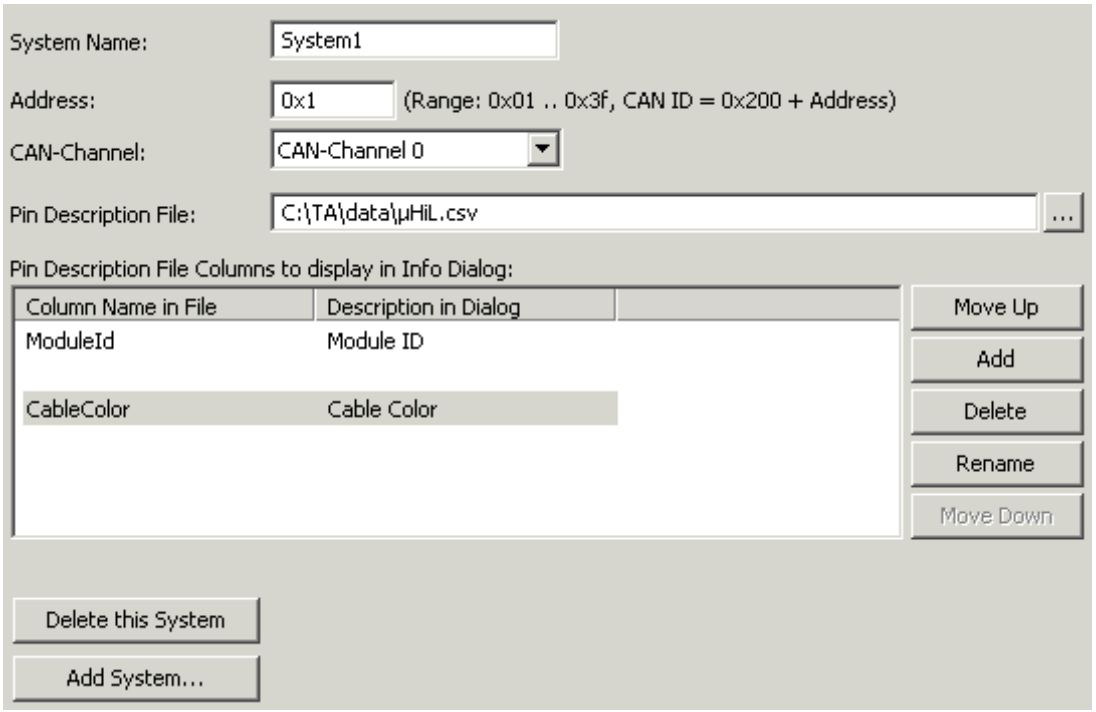


Figure 72 – Fault simulation settings for a PROVEtech:µHiL system

The text box `Address` specifies the address of the µHiL to use for the fault simulation. PROVEtech:TA uses the CAN ID Address+0x200 to control the µHiL on the CAN channel.

The combo box `CAN-Channel` sets the CAN channel previously configured in the left tree under the entry “Fault Simulation CAN Channels”. Normally you only have one CAN channel, so you do not need to change this setting.

The `Pin Description File` is a .csv file that contains all information about the ECU pins connected to the µHiL. You can find more information about the file structure in the ***Administrator’s Guide*** document.

Enter the column headings you want to see in the info dialog of the fault simulation and on demand as additional columns on the fault simulation panel (see chapter ***7.1.2.1 Pre-selection of cables***) in the list `Pin Description File Columns to display in Info Dialog`. PROVEtech:TA reads the information from the pin description file. The left column describes the column heading contained in the file and the right column the name displayed in the info dialog.

In case the µHiL is connected to PROVEtech:RE via XCP (Ethernet), the entered `Address` has no meaning. PROVEtech:RE creates a virtual CAN channel for each µHiL for PROVEtech:TA in this case (has the name of the corresponding PROVEtech:RE configuration entry for the µHiL). Create a CAN channel for each µHiL in the fault simulation configuration and map it to the corresponding virtual CAN channel of PROVEtech:RE.

3.2.7.9.2 Softing Engineering & Solutions fault simulation

PROVEtech:TA displays the following settings page when clicking on a Softing Engineering & Solutions system in the left tree:

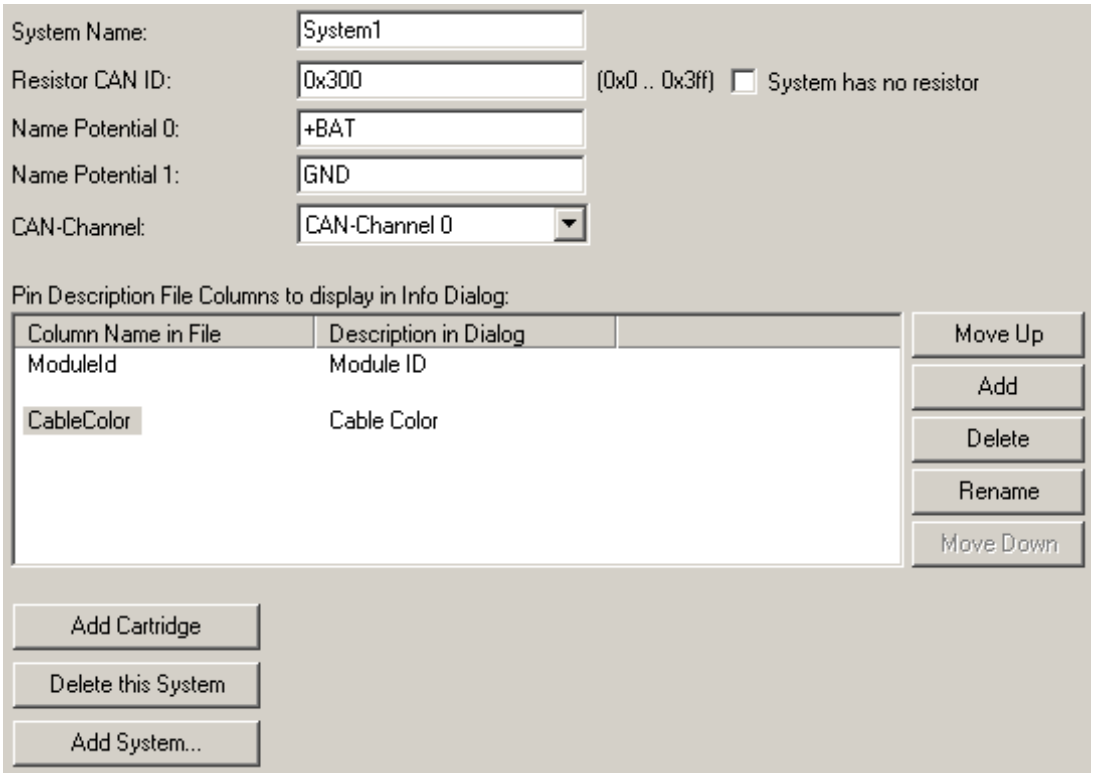


Figure 73 – Fault simulation settings for a Softing Engineering & Solutions system

In case your System has an adjustable resistor, enter its CAN ID in the field `Resistor CAN ID`. Otherwise activate `System has no resistor`.

You can connect the rails of the fault simulation to two different potentials. Use the text boxes `Name Potential 0` and `Name Potential 1` to change their names. Normally there is no need to change the default names as potential 0 is the battery power and potential 0 is the ground. These names are displayed on the user interface and also the test language uses them.

The combo box `CAN-Channel` sets the CAN channel previously configured in the left tree under the entry “Fault Simulation CAN Channels”. Normally you only have one CAN channel, so you do not need to change this setting.

Enter the column headings you want to see in the info dialog of the fault simulation and on demand as additional columns on the fault simulation panel (see chapter ***7.4.2.1 Pre-selection of cables***) in the list `Pin Description File Columns to display in Info Dialog`. PROVEtech:TA reads the information from the pin description files (see below). The left column describes the column heading contained in the file and the right column the name displayed in the info dialog.

A system consists of one or several cartridges. A cartridge is a logical unit that contains a main control unit and two internal rails. Normally there exists one cartridge per ECU. Click on `Add Cartridge` to create a new cartridge. An according entry will be displayed in the left tree.

Select a cartridge in the left tree to see the following page:

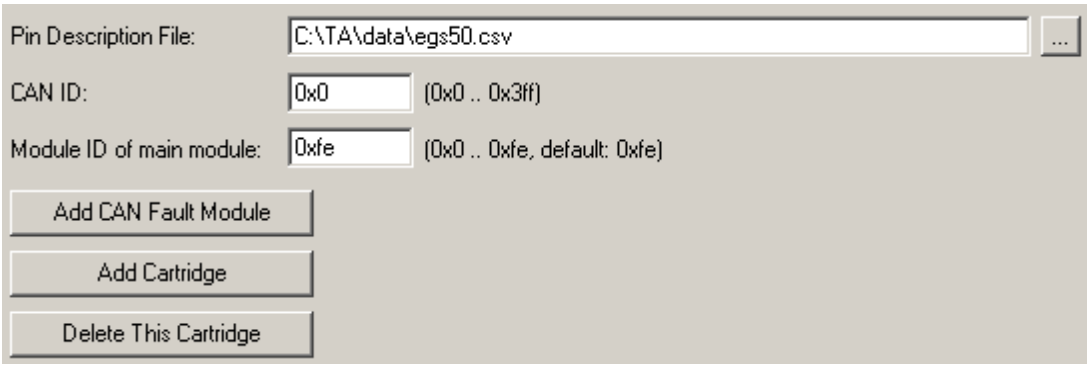


Figure 74 – Configuration page for a cartridge

The `Pin Description File` is a .csv file that contains all information about the ECU pins connected to the cartridge. You can find more information about the file structure in the ***Administrator’s Guide*** document.

Enter the CAN ID of your cartridge (control module) in the text box `CAN ID`. Every cartridge additionally has a main module to control the relays of the rails. The text box `Module ID of main module` contains its module ID (1st byte of a CAN command message).

Every cartridge can additionally contain several CAN fault modules. Click on Add CAN fault module to create a new one in your configuration that will be displayed in the left tree.

Select a CAN fault module in the left tree to see the following page:

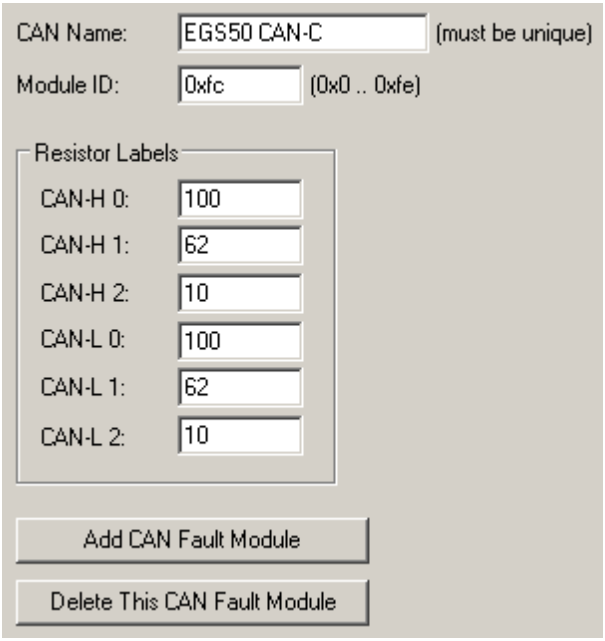


Figure 75 – Configuration page for a CAN fault module

The entered `CAN Name` must be unique across all systems and is used for the user interface and for the test language.

Additionally every CAN fault module has a `Module ID` (1st byte of a CAN command message) that is unique on the cartridge.

Enter the values of the six hard wired resistors of the module in the fields `Resistor Labels`. You can enter any text here, e.g. “1kOhm”. Only the user interface uses this text, but not the test language.

3.2.7.9.3 ETAS ES4440 fault simulation

PROVEtech:TA displays the following settings page when clicking on a ETAS ES4440 system in the left tree:

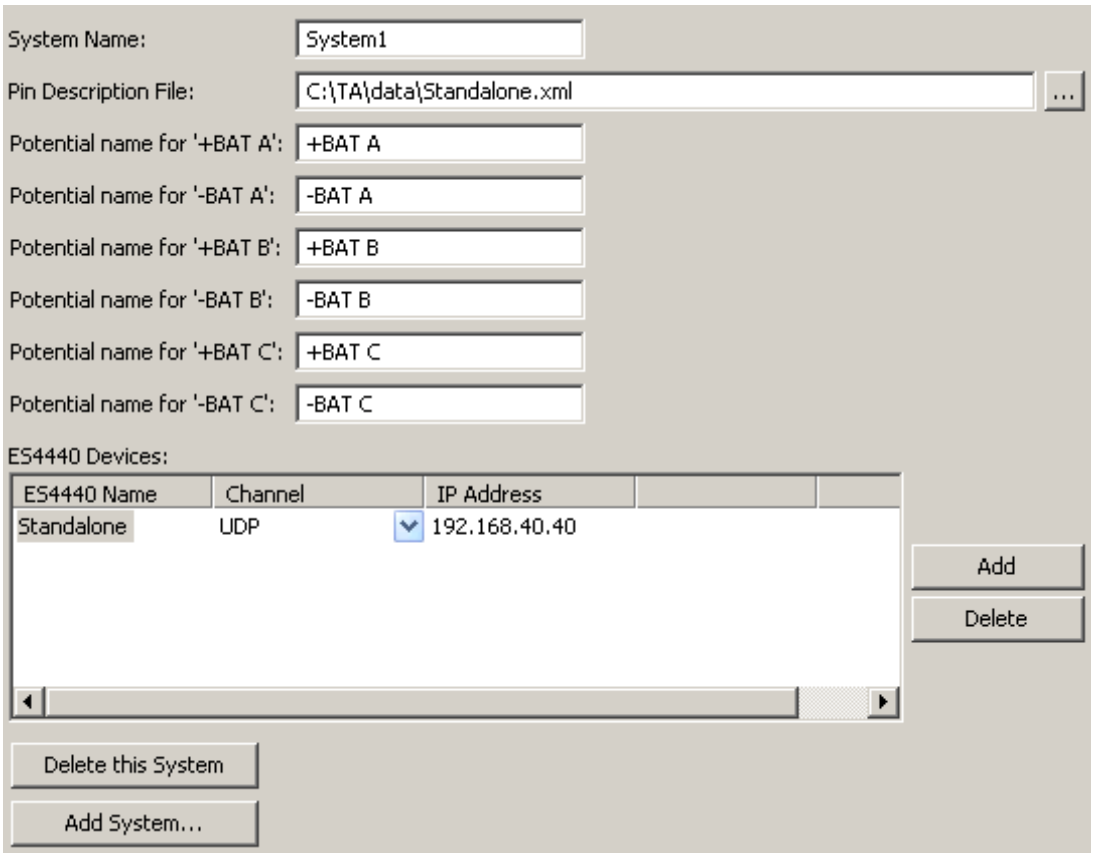


Figure 76 – Fault simulation settings for an ETAS ES4440 system

The `Pin Description File` specifies the path to a .xml file containing the names of the ECU pins and their hardware assignment. ETAS has defined the file format and they call the files “wiring harness files”.

The six fields for the potential names define unique identifiers of the six potentials that you can connect to an ES4440 box. This setting influences the user interface as well as the strings used in the test language. You can connect the first four potentials only to high current pins and the lower two only to high voltage pins.

In the list ES4440 Devices you can specify the used ES4440 boxes and how to control them. Use the buttons `Add` and `Delete` to increase/decrease the amount of ES4440 boxes to use. The column `ES4440 Name` corresponds to the name in the pin description file. As long as there is only on ES4440 box, the name is always “Standalone”. Otherwise the topmost entry is “Master” and the others are “Slave1”, “Slave2”, etc.

Use the combo box in the column `Channel` to select if you want to control the according ES4440 box via UDP (Ethernet) or a previously specified CAN channel (see ***3.2.7.9 Fault simulation***).

In case of UDP just fill the column `IP Address`. For CAN you need to enter the CAN Ids for Request and Response (both hexadecimal) and the bit rate.

3.2.7.9.4 dSPACE FullSize fault simulation via CAN

The dSPACE FullSize variant supports two different file formats (described in the ***Administrator's Guide***). In the left tree the according nodes look like this:

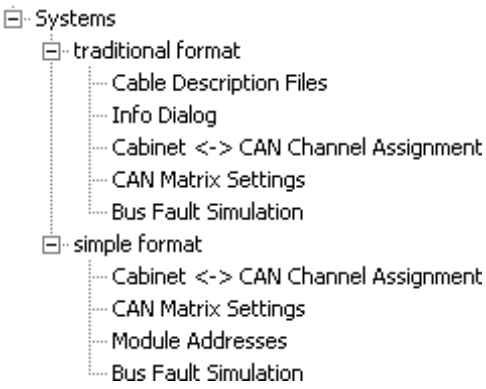


Figure 77 – Configuration pages in the tree

Traditional Format

This paragraph describes the settings pages for the traditional format. PROVEtech:TA displays the following settings page when clicking on a dSPACE FullSize system in the left tree:

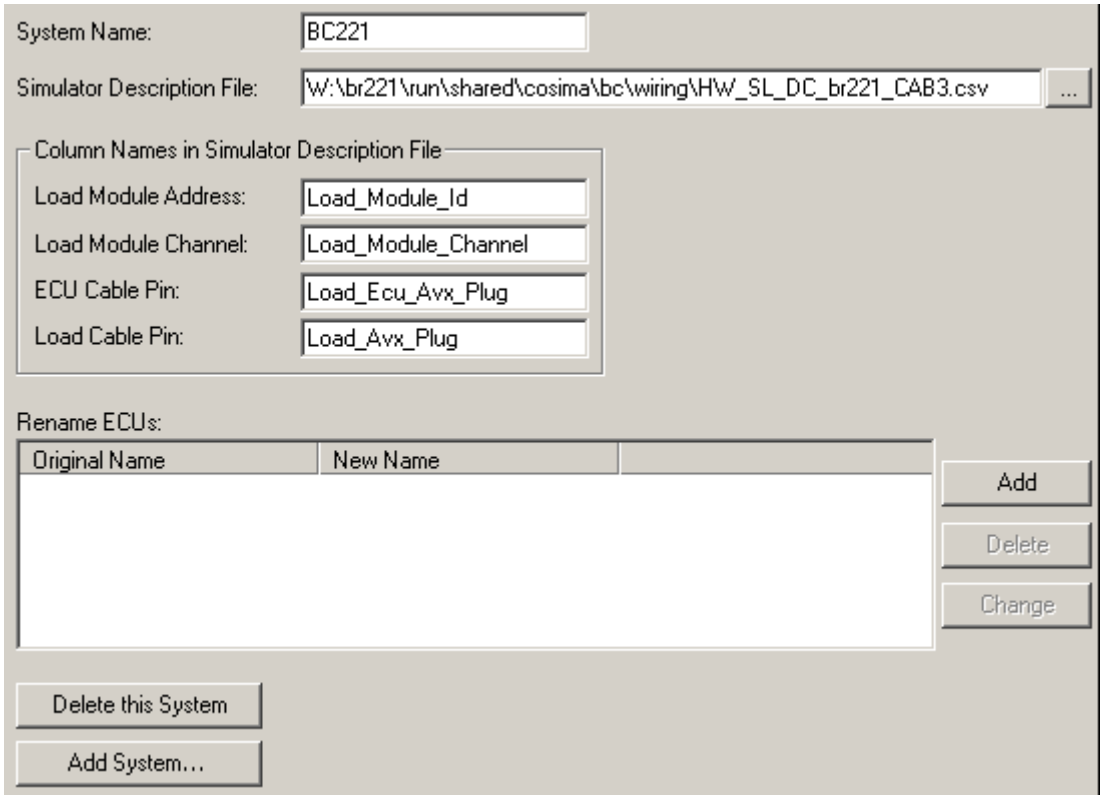


Figure 78 – Fault simulation settings for a dSPACE FullSize system via CAN

For every system dSPACE provides an Excel file that contains a table for every cabinet with information about the cabinet construction. Please export all tables to the csv format before using them in PROVEtech:TA (German region settings for the export where the semicolon is the separation character). One of the tables contains additional information, among other things the links to the other tables. Enter the path to this .csv file in the `Simulator Description File` edit box. PROVEtech:TA needs four columns of the tables which names you can edit in the four according edit boxes.

You need the table `Rename ECUs` only in very special cases where the displayed ECU name(s) shall differ from the name(s) in the .csv files. PROVEtech:TA matches the ECU list specified for each configuration (see chapter ***3.2.7.2 Configurations***) to the renames ECU names. To match them to the original ECU names prefix the list entries in the configuration onfigwith ~.

For every system the tree on the left side displays four additional entries described in the following paragraphs.

Cable Description Files

For every ECU you need a file in the csv format containing a table that describes the ECU connection cables and the allowed fault settings for every ECU pin. Enter these file paths and the column names that are relevant for PROVEtech:TA in the page *Cable Description Files*.

Info Dialog

Enter the column names you want to see in the info dialog of the fault simulation and on demand as additional columns on the fault simulation panel here (see ***7.4.2.1 Preselection of cables***):

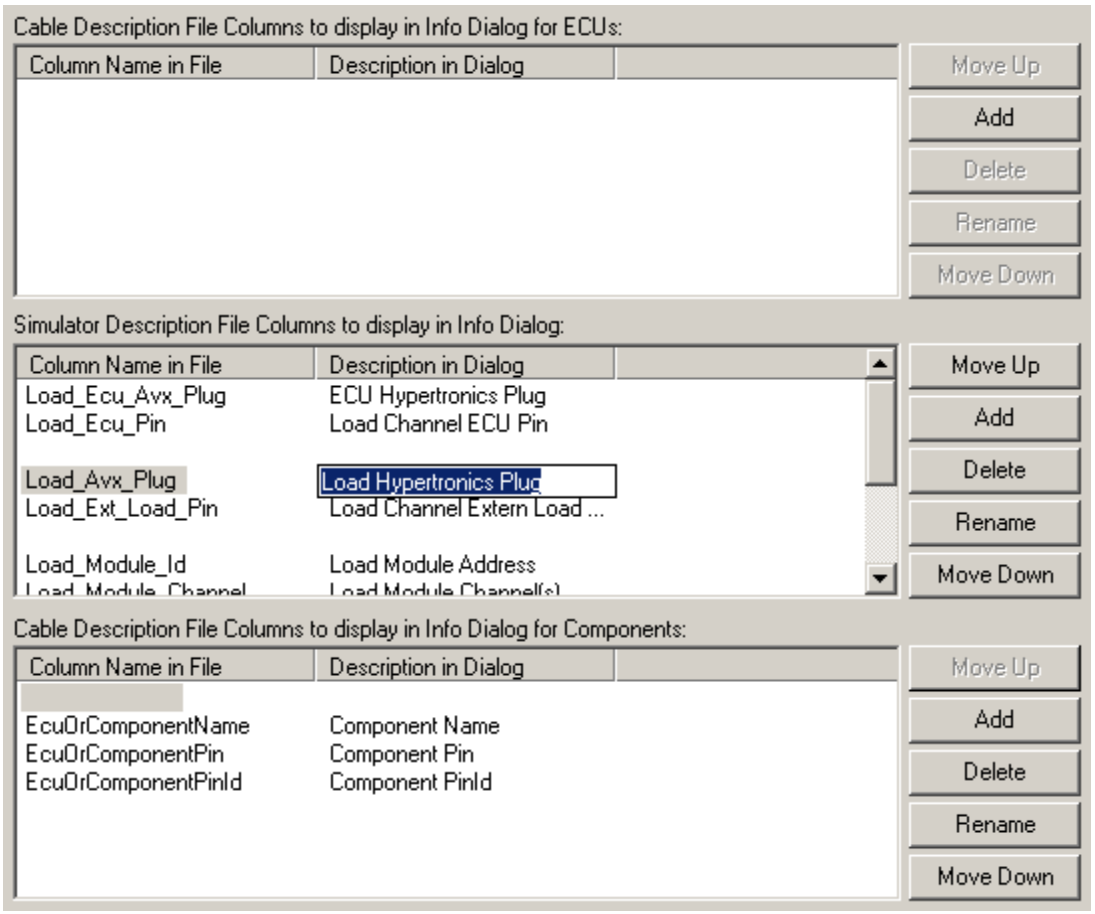


Figure 79 – Settings for the info dialog

There are three tables for the path ECU → cabinet → component (sensor/load) available. The left column contains the name of the column in the .csv file and the right column contains the name to be displayed in the info dialog. An empty line is a separation line in the info dialog.

Cabinet <-> CAN Channel Assignment

There are 15 combo boxes on this page to select which CAN channel controls which cabinet. The CAN channels you have configured under the entry “Fault Simulation CAN Channels” in the left tree are available here. Normally a system does not consist of 15 cabinets. PROVEtech:TA ignores irrelevant entries.

CAN Matrix Settings

This page displays the settings of the CAN matrix (see chapter ***7.4.3.1 CAN topology***). Just leave all settings empty, when you do not have a CAN matrix.

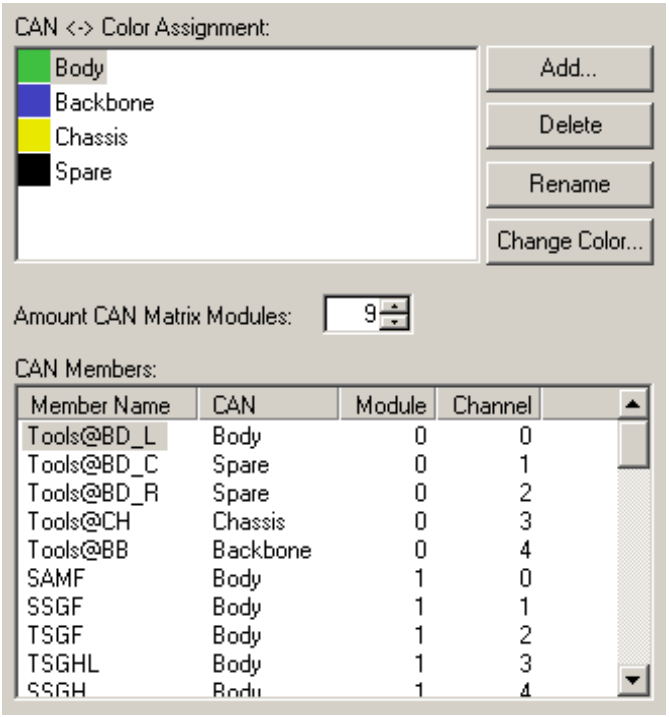


Figure 80 – Page for the ECU CAN settings

In the upper list you can configure the colors for the different CAN busses of your system. The fault simulation user interface frames the CAN member symbols with these colors.

After that you enter the amount of CAN matrix modules in your system. PROVEtech:TA extends the lower list with 5 entries per module, because every module can support 5 CAN members. Edit the names of the CAN members (1st column) and their CAN bus association (2nd column) in the lower list. The two rightmost columns display which CAN member is connected to which CAN module and channel.

Bus Fault Simulation

This page displays the settings of the bus fault modules (see chapter ***7.4.4 Bus faults***). Just leave the list empty if you do not have bus fault modules. The controlled bus fault modules are single dSPACE DS295 modules. Originally this module was designed for inserting faults in a connection between a CAN member and the according CAN bus (see CAN matrix). But you can use it also for inserting FlexRay, LIN or Ethernet faults as long as the electrical properties of the board are sufficient for that.

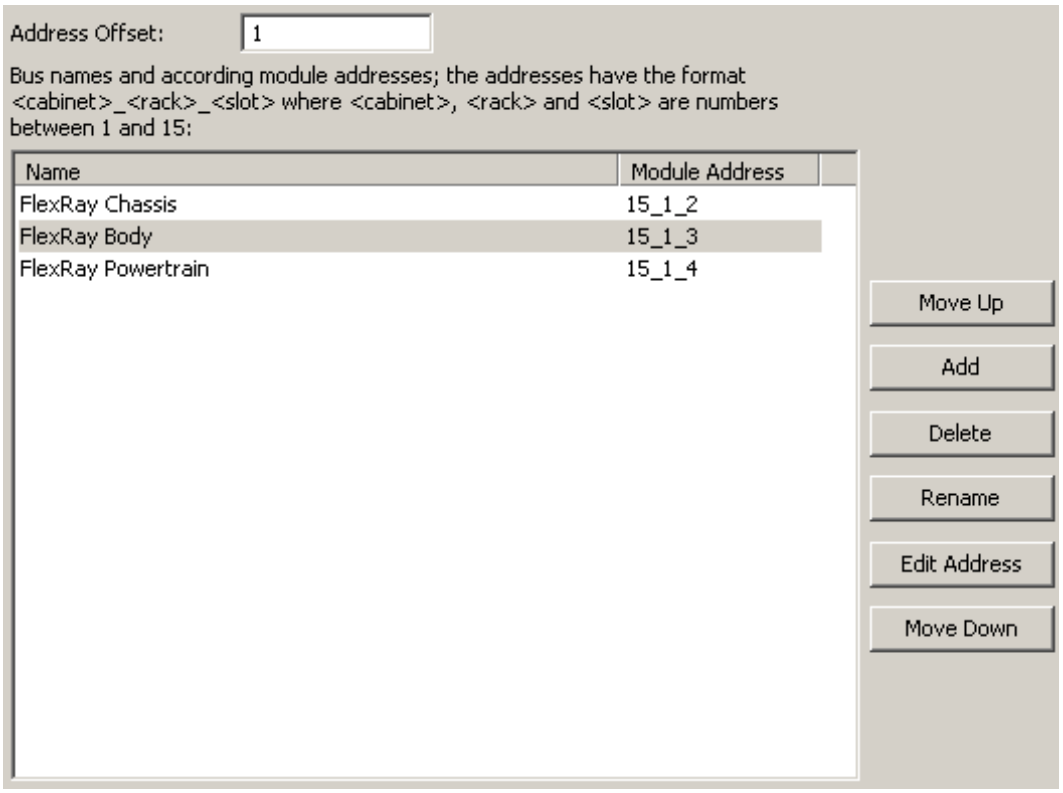


Figure 81 – Page for the bus fault simulation

Use the field `Address Offset` to specify the DIP switch setting of the DS295 modules. Bit 0 is set as a default.

In the lower list you can add single buses (DS295 modules) via the `Add` button. Specify a unique name (button `Rename`) and the according DS295 address (button `Edit Address`) for each entry. You can change the order of the entries via the buttons `Move Up` and `Move Down`. Click on `Delete` to remove an entry.

Simple Format

PROVEtech:TA displays the following settings page when clicking on a dSPACE FullSize system using the simple file format in the left tree:

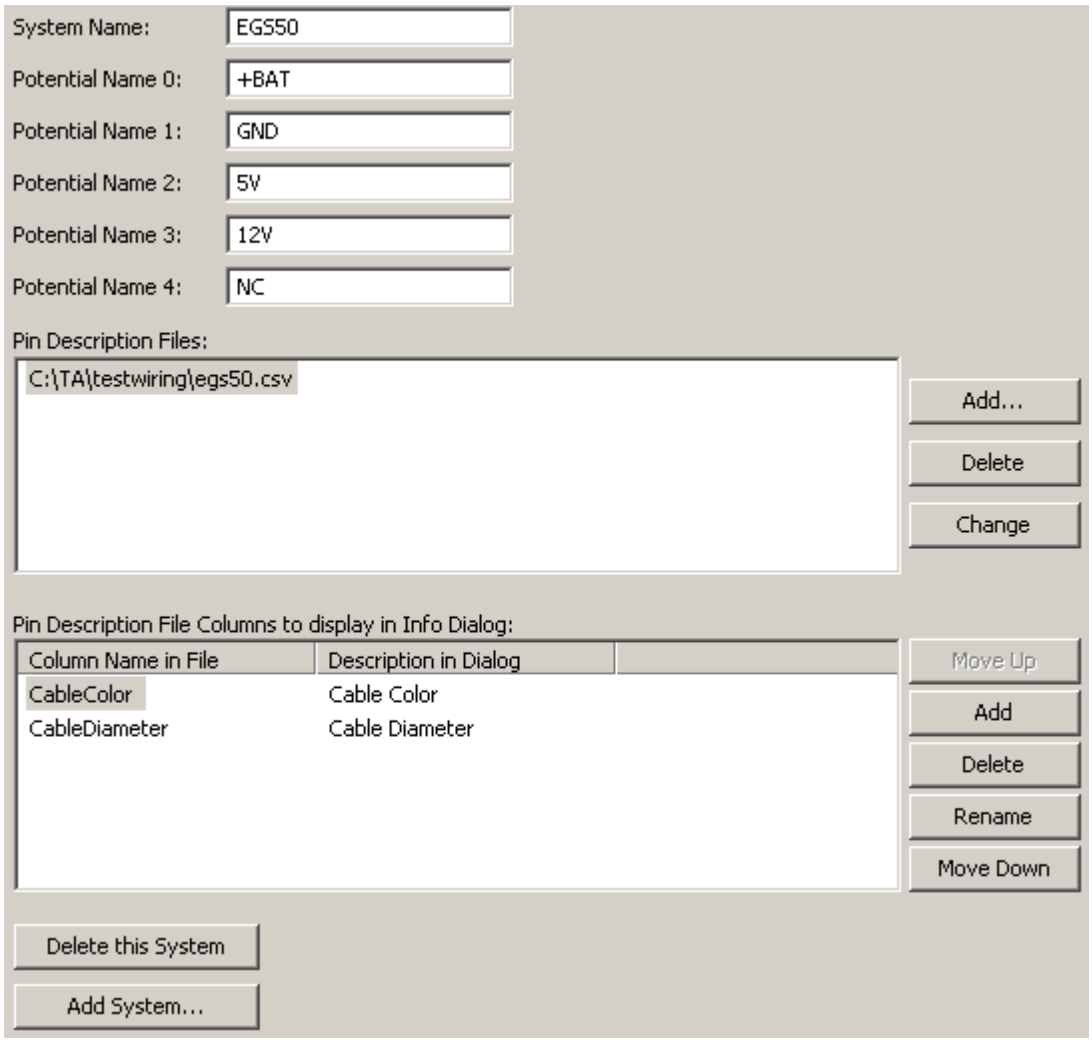


Figure 82 – Fault simulation settings for a dSPACE FullSize system via CAN

The text fields `Potential Name x` specify the names of the five existing potentials for shorts in your system. The user interface shows them in the according combo boxes and also the test language uses these names. In opposite to the traditional format the simple format only one .csv file type (see ***Administrator's Guide***). Use the list `Pin Description Files` to specify the path to one or several files of this type. The list `Pin Description File Columns to display in Info Dialog` corresponds to the settings of the page *Info Dialog* of the traditional format (see above). As there is only one file format in this case, you only need one list instead of three.

Module Addresses

On this page (***Figure 83***) you specify all addresses of the dSPACE modules beside the ones of the load modules as they are already specified via the pin description files. Please read the upper text of this settings page. The `CAN Matrix` part is only available

when you have set the amount of CAN matrix modules on the page *CAN Matrix Settings* before (in case your System has a CAN matrix at all).

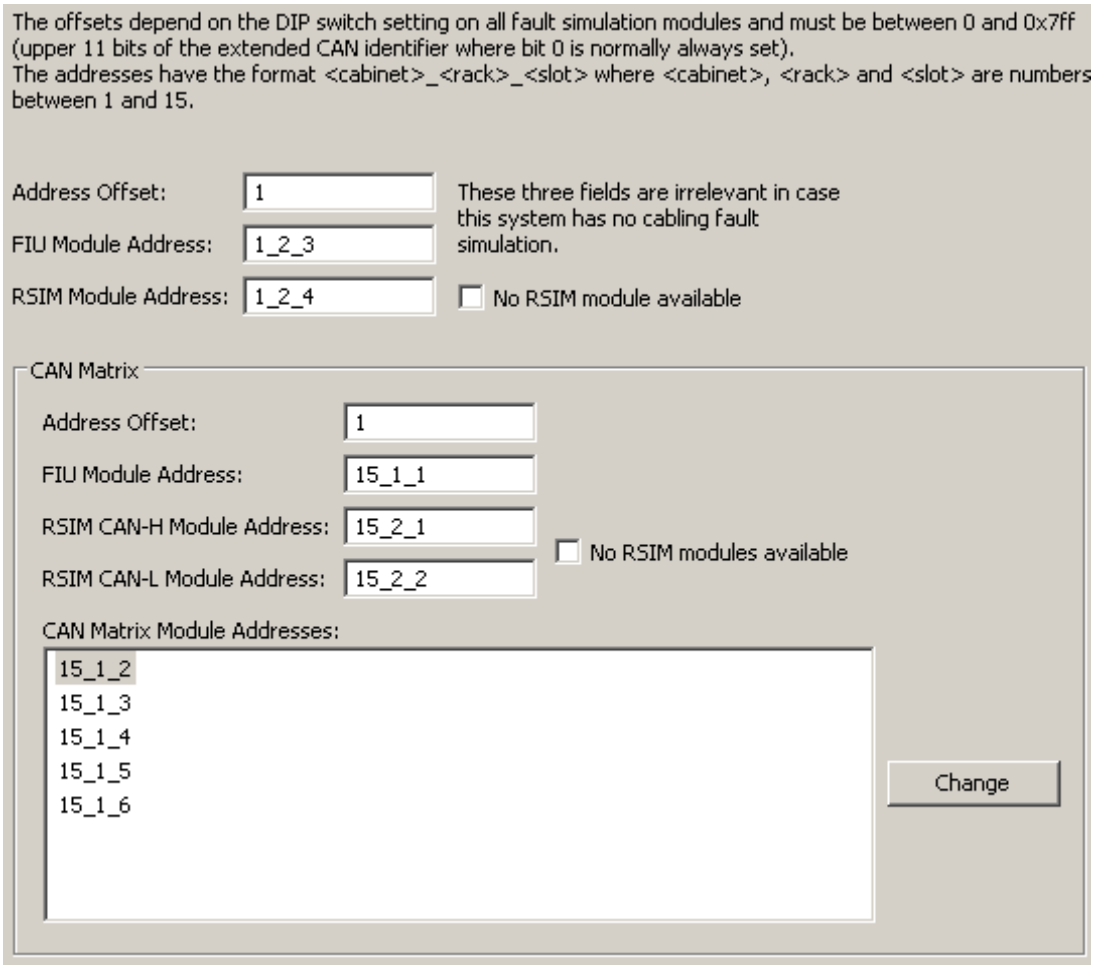


Figure 83 – Page for the module address settings

The pages *Cabinet <-> CAN Channel Assignment*, *CAN Matrix Settings* and *Bus Fault Simulation* are identical to those of the traditional format (see above).

3.2.7.9.5 dSPACE MidSize or FullSize fault simulation via RS232

PROVEtech:TA displays the following settings page when clicking on a dSPACE MidSize or FullSize system via RS232 in the left tree:

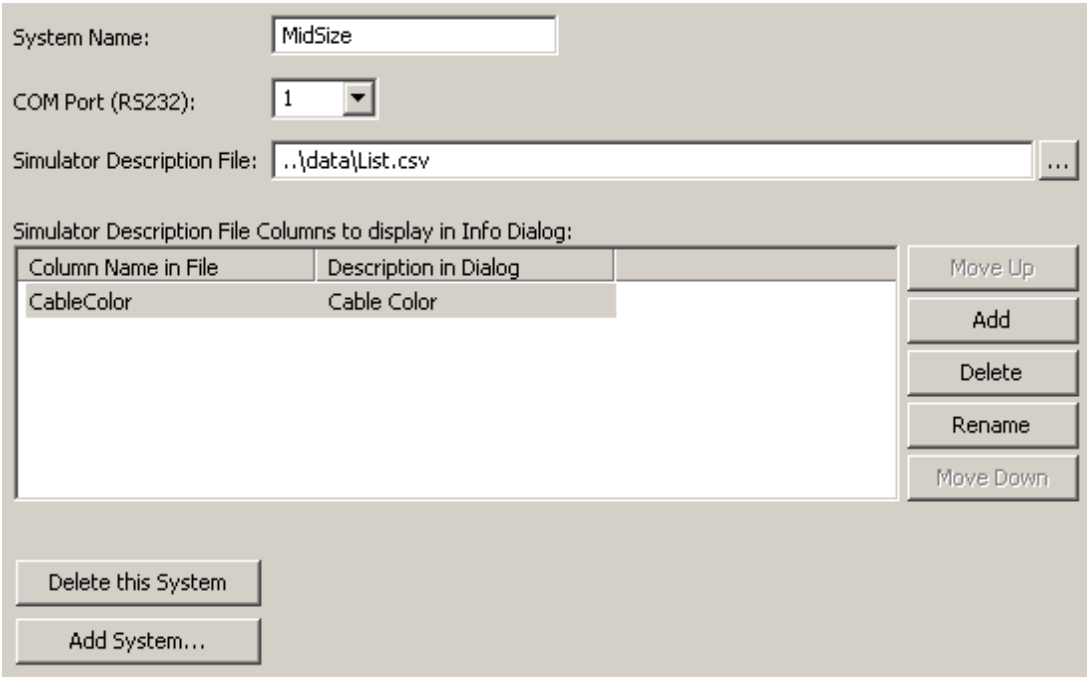


Figure 84 – Fault simulation settings for a dSPACE MidSize or FullSize system via RS232

Please select the COM/RS232 port of your computer where the fault simulation hardware is connected via the combo box `COM Port (RS232)`.

For every system dSPACE provides an Excel file that contains the settings for the MidSize cabinet. Please export this table to the csv format before using it in PROVEtech:TA (German region settings for the export where the semicolon is the separation character). Enter the path to this .csv file in the `Simulator Description File` edit box. You can find more information about the .csv file structure in the ***Administrator's Guide*** document.

Enter the column headings you want to see in the info dialog of the fault simulation and on demand as additional columns on the fault simulation panel (see chapter ***7.5.2.1 Preselection of cables*** or ***7.6.2.1 Preselection of cables***) in the list `Simulator Description File Columns to display in Info Dialog`. PROVEtech:TA reads the information from the simulator description file. The left column describes the column heading contained in the file and the right column the name displayed in the info dialog.

3.2.7.9.6 dSPACE SCALEXIO fault simulation

PROVEtech:TA displays the following settings page when clicking on a dSPACE SCALEXIO system in the left tree:

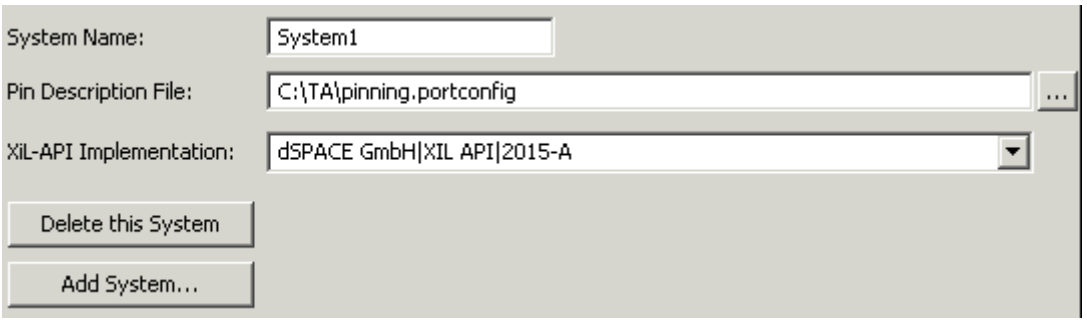


Figure 85 – Fault simulation settings for a dSPACE SCALEXIO system

Controlling the SCALEXIO fault simulation works via the dSPACE XiL-API. Enter the path to your .portconfig file fitting to your dSPACE fault simulation configuration in the field `Pin Description File`. Older dSPACE releases had the command line tool EESPortConfiguration.exe to create this file fitting to a model configuration. Newer dSPACE releases do not have this tool anymore. Please read the dSPACE documentation to get to know how to create this file or let dSPACE deliver it to you together with the model.

The combo box `XiL-API Implementation` lists all XiL-API installations of dSPACE. Please select the matching dSPACE release used for creating the model.



PROVEtech:TA uses the dSPACE XiL-API to control the SCALEXIO fault simulation. This API uses the running model and needs a .portconfig file. Please ensure that the .portconfig file always fits to the currently running model to avoid any damage. Update the .portconfig file after every relevant model change and do not switch to a different model without ensuring that the fault simulation runs with the matching .portconfig file.

3.2.7.9.7 dSPACE DS5394C1 fault simulation

PROVEtech:TA displays the following settings page when clicking on a dSPACE DS5394C1 system in the left tree:

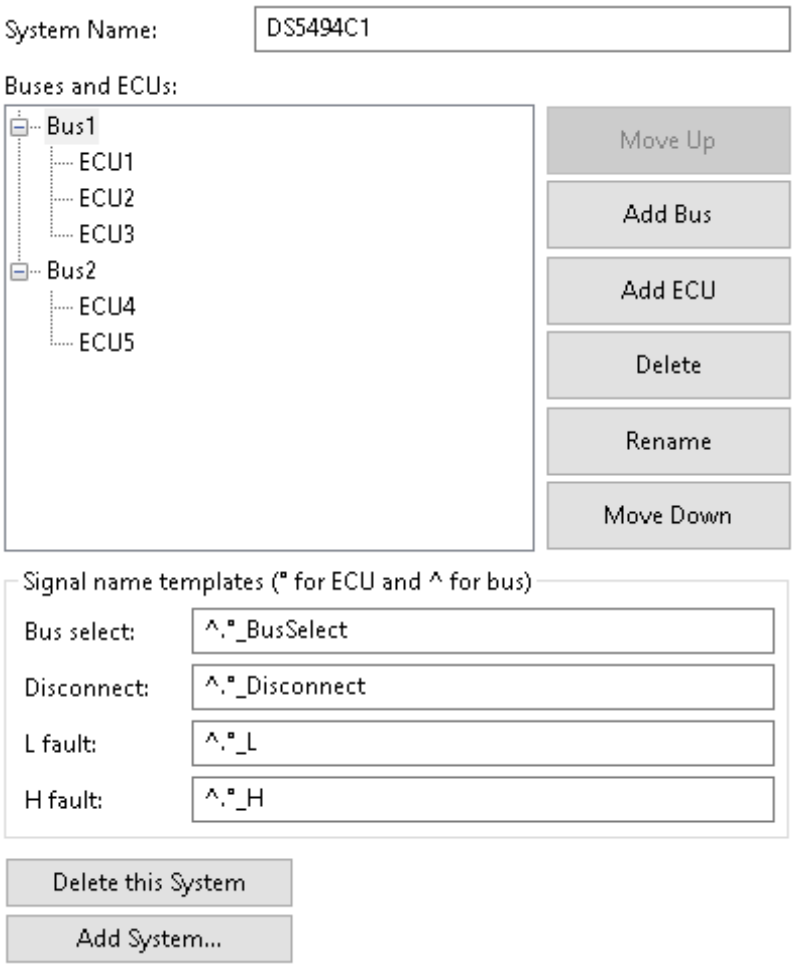


Figure 86 – Fault simulation settings for a dSPACE DS5394C1 system

In the tree `Buses and ECUs` you edit the topology of your system. The first level represents the buses and the second level the ECUs. Use the buttons on the right side or the context menu to change the tree entries.

The names must correspond to the signal names in your model. In the group `Signal name templates` you enter the 4 signal names of your model per ECU. The `^` character is an optional place holder for the bus name (from the upper tree) and `°` is a place holder for the ECU name (from the upper tree). The first occurrence of the special character will be replaced. The signal names must not use these 2 characters otherwise.

Read the chapter ***7.8 dSPACE DS5394C1 fault simulation*** to see the according graphical user interface.

3.2.8 Lock User Interface

This command (or the shortcut Alt-L) makes the application's windows invisible and prevents from interacting, while started data acquisition and tests are continued. Only an

extended login window stays visible. The current user or the admin can unlock the application again. It is also possible to change users, but the Basic scripts that are still running will be stopped. The window contains a list of currently active scripts and an optional message from the user.

Without the proper password it is only possible to shut down the application. This causes running scripts to be stopped.

You can find more in chapter ***2.3 Login***.

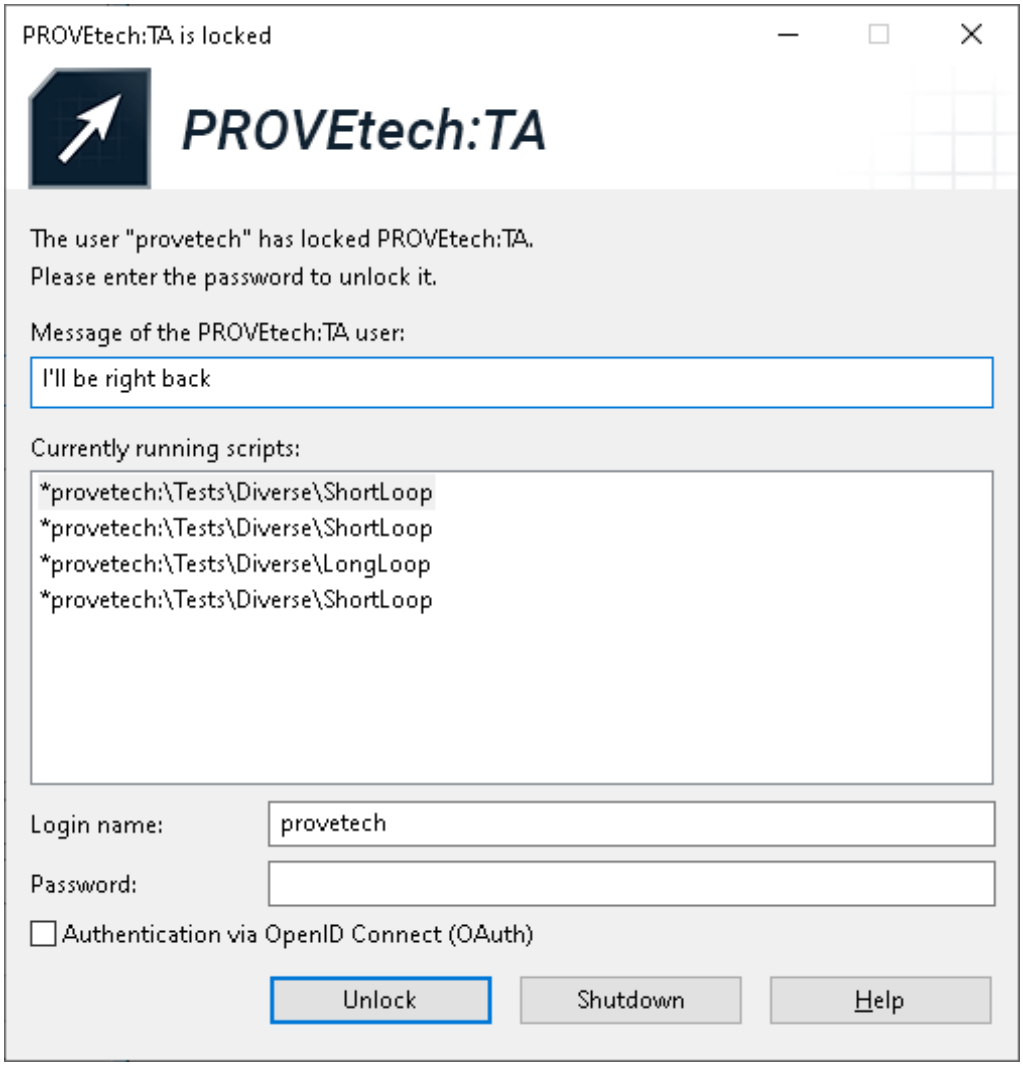


Figure 87 – login window to unlock the application

Please keep in mind that locking the application does not prevent the running Basic script from generating windows for user interaction, except scripts running in debug mode, which will be paused while the application is locked.

3.2.9 Exit

Exit closes the program. When exiting the program, PROVEtech:TA will ask you to save the changes in your configuration (e.g. the currently loaded workpage or unsaved data acquisition).

3.3 The Workpage menu

The *Workpage* menu (see ***Figure 88***) allows you to load and save cockpits and workpages.

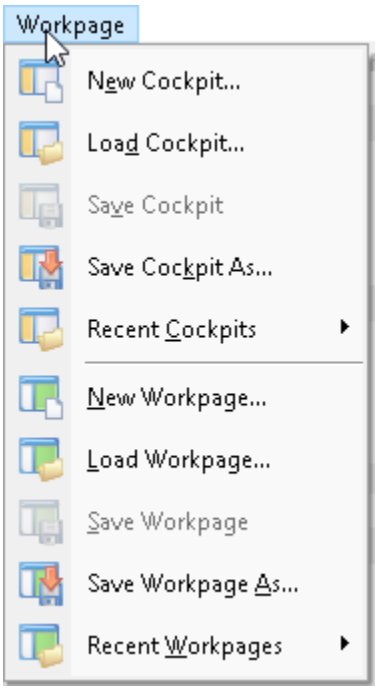


Figure 88 – The Workpage menu

3.3.1 New cockpit, load cockpit and save cockpit

The cockpit area on the left side of the PROVEtech:TA main window has the same features as the workpage, though it always stays visible. The cockpit menu items correspond to the ones of the workpage, which are described in the chapter ***3.3.2 New workpage, load workpage and save workpage***. Cockpit files have the file extension **.ini**.

3.3.2 New workpage, load workpage and save workpage

The workpage loaded determines the appearance of PROVEtech:TA. It contains selectable display and control elements that represent the signals of your test system. At the file level, workpages are identified by the suffix **.aof**. The path of the currently loaded workpage is displayed at the bottom in the PROVEtech:TA status line. For further information on workpages refer to the chapter ***4 Workpage***.

Select the *New Workpage…* menu item to save a new empty workpage. The file selection dialog (see chapter ***3.1 The file selection dialog***) is displayed where you can choose the saving location. You can choose between the file system and the database of the Test Manager (as a test document of a test object, see chapter ***5.3.9.4 The tab "Documents"***).

When you select the *Load Workpage…* menu item, the file selection dialog (see chapter ***3.1 The file selection dialog***) appears, and you can select the workpage to be loaded.

The *Save Workpage* menu item saves all the changes you have made to the current workpage to the associated .aof file. The previous version of the workpage contained in this file is lost.

Select the *Save Workpage As…* menu item to save the current workpage under a different file path. The file selection dialog (see chapter ***3.1 The file selection dialog***) is displayed for this purpose.

With the *Recent Workpages* item you can load most recently used workpages without using the file selection dialog.

If, after starting the workpage, PROVEtech:TA is unable to find the signals to which elements refer, then these elements are disabled (see ***Figure 89***). You may assign a new signal to these elements via the context menu (right mouse button), load a different workpage, or delete these elements.



Figure 89 – Disabled element



If you do not wish to lose changes made to the current workpage, you should save them before loading an other workpage. Use either the *Save Workpage* or *Save Workpage As…* menu item to save the changes.

3.4 The Data Acquisition menu

The *Data Acquisition* menu (***Figure 90***) is used to create, display and convert data acquisitions. Data acquisition is the measurement of the signal values of one or several signals over time. They are acquired via a buffer in the real-time computer and therefore have an exact time behavior.

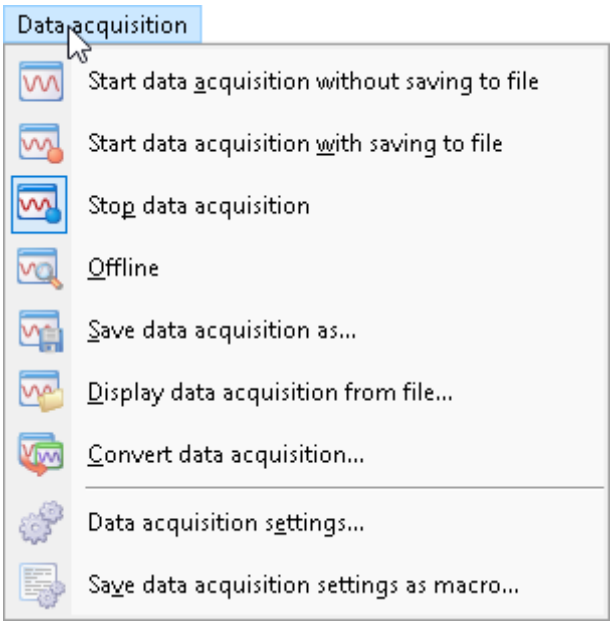


Figure 90 – The Data Acquisition menu

Before you perform a data acquisition, select those signals from your signal pool that you wish to acquire. For information on this, refer to chapter ***4.2 The Signal Selection window***.

The chapter ***3.2.7.4 Data acquisition*** describes how to configure data acquisitions. This includes the selection of which messages to acquire and which video/audio sources to record.

Data acquisition is started via *Start Data Acquisition without/with saving to file*. Data acquisition without saving to file (corresponds to the button) only fill up the strip chart controls on your workpage (see chapter ***4.4.2 The strip chart and multi-strip chart controls***), and **there is no way** to evaluate this data later on. Data acquisition with saving to file (corresponds to the button) additionally store the acquisition data in a temporary file on the hard disk. The menu item *Stop Data Acquisition* ( ) terminates a data acquisition. Subsequently you can use the *Offline* ( ) menu item to change to the offline mode in order to evaluate your data acquisition (see chapter ***4.5 The offline mode***) if it is a data acquisition with saving to file.

One of the 4 menu items / toolbar buttons *Start Data Acquisition without saving to file*, *Start Data Acquisition with saving to file*, *Stop Data Acquisition* and *Offline* is always marked. In the upper ***Figure 90*** it is *Stop Data Acquisition*. It shows which of the 4 different states PROVEtech:TA is currently in. When running a data acquisition, the according toolbar button has a red background. During a data acquisition some actions like loading a model are disabled on purpose to prevent disturbing the data acquisition.

Display Data Acquisition from file… opens the file selection dialog (see chapter ***3.1 The file selection dialog***) to select an already saved data acquisition and show it in the offline mode.

Select the menu item *Save Data Acquisition as…* () to save your data after stopping a data acquisition with saving to file. First, the file selection dialog (see chapter ***3.1 The file selection dialog***) pops up where you select the destination path of your data acquisition. Additionally you can use the combo box `Files of type` in the file selection dialog to select the file format of the data acquisition, e.g. PROVEtech:TA, MATLAB, MDF (Measurement Data Format), MF4, PCAPNG, TDM, TDMS, DLT or ASCII table. Attention: Only the PROVEtech:TA format supports all features. For example, the CANalyzer ASCII log file format only supports messages, but no signals. PCAPNG only supports Ethernet messages.

You find more details in chapter ***8.13 Tips and tricks***.

In case the destination format supports a title or remarks, the following dialog shows up after that:

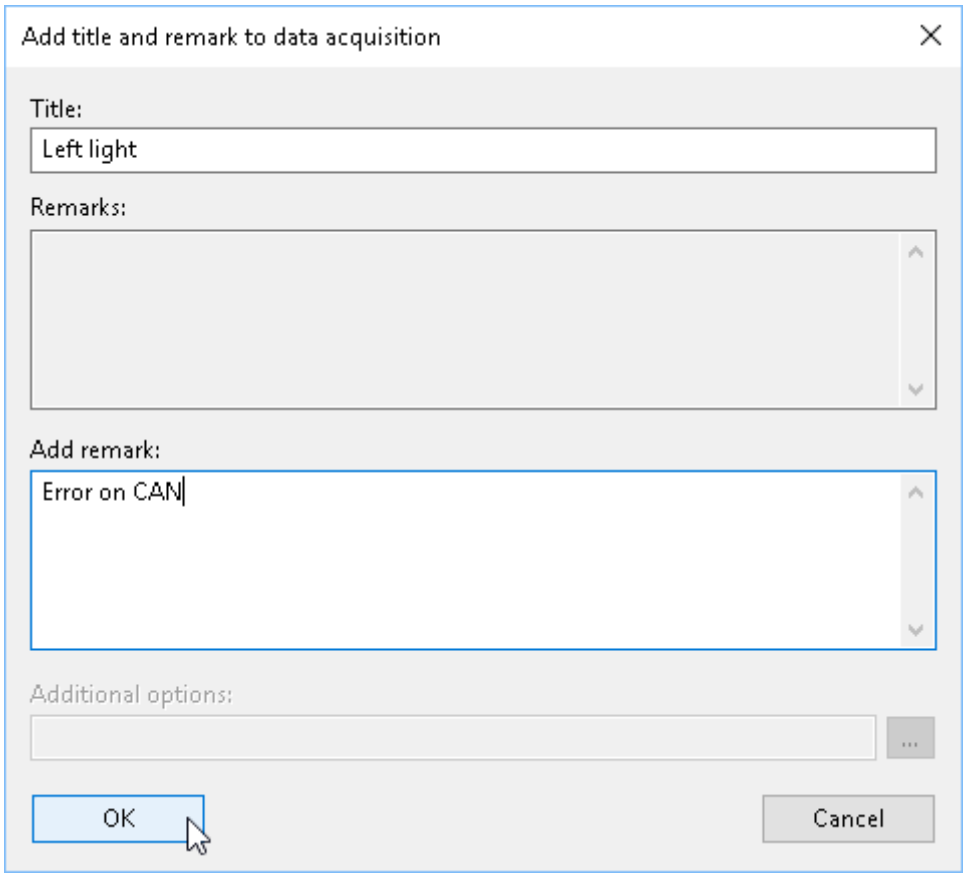


Figure 91 – Add remark to data acquisition dialog

Enter a title and a remark for your data acquisition. The edit box `Remark` contains any error messages, which have been automatically generated, for example, when a buffer overflow on the real-time computer due to performance problems has caused the data acquisition to be aborted.

Depending on the destination format you can configure additional settings via the field `Additional options`. Use the button … of the field to open a dialog for configuring the additional options. In case the destination format does not support additional options, these controls are deactivated. In case the destination format does not support a title and a remark only the dialog for additional options pops up.

The menu item *Convert Data Acquisition…* () first opens the file selection dialog to select a saved data acquisition to convert. Then the file selection dialog pops up again to select the destination file. Then it shows the dialog described in the chapter ***3.4.1 Converting data acquisitions*** to convert the selected data acquisition.

The menu item *Data acquisition settings…* () opens the settings dialog for the data acquisition (see chapter ***3.2.7.4 Data acquisition***).

The menu item *Save data acquisition settings as macro…* () opens the file selection dialog to specify the destination for the created .mac file. The file is a macro that calls the test language commands ***Measure.SetMessageFilter***, ***Measure.SetVideoAudio***, ***Measure.SetParameters*** and ***Measure.SetSignals*** to set all data acquisition settings to the ones active while saving the macro. Use the menu item *Start macro…* to run it (see chapter ***3.5 The Macro menu***).



- Once a data acquisition has been stopped, it cannot be continued.
- You can switch directly from a data acquisition with saving to file to a data acquisition without saving to file or the offline mode. This will stop the data acquisition, and the measurement cannot be resumed.
- Starting a data acquisition with saving to file automatically clears a previously performed data acquisition with saving to file unless you first save it.



When doing a data acquisition with signals and messages at the same time, PROVEtech:TA synchronizes the data. In the offline mode you can have a look at the result (see chapter ***4.5 The offline mode***). Please be aware, that due to technical reasons messages can be missing for the last signal sample of the data acquisition in case you use a downsampling rate (see chapter ***3.2.7.4 Data acquisition***).



When doing data acquisitions, PROVEtech:TA relies on the timer(s) and the synchronization of the model platform. So, the synchronization and timing information of the acquired data is only as good as provided by the model platform. In case of a multi model setup where the models run asynchronously, PROVEtech:TA tries to start the data acquisition across the nodes in parallel as good as possible to synchronize the data loosely. The different asynchronous timers might drift apart anyway during the data acquisition and PROVEtech:TA has no chance to detect this. Signal values displayed in the offline mode at the same time were not acquired at the same time in reality for sure.

In case of recording video/audio additionally, there is just a very inexact synchronization. The recording of video/audio data happens on the Windows computer while the signal and message data comes in asynchronously on one or several model computer(s). This involves totally different time sources which work asynchronously and additionally can drift away from each other or jitter. Additionally the video/audio data appears delayed in the according file (delayed by hardware, driver and encoder). PROVEtech:TA just tries to synchronize the start time of all the data as good as possible. For that enter a manually and empirically detected delay per device (see chapter ***3.2.7.3 Workpage***). In the offline mode (***4.5 The offline mode***) it is possible to manually adjust it afterwards.



Please be aware that when stopping a data acquisition, video/audio data at the end of the acquisition might be missing as stopping of everything happens at nearly the same time while the incoming video/audio data of that point of time is from the past (see upper delay).

Using start triggers and data acquisitions with fixed buffer length (cyclic data acquisitions) cannot work with video/audio data. Instead of that video/audio recording always starts when starting a data acquisition and PROVEtech:TA just calculates the synchronization time (time 0) accordingly.

PROVEtech:TA always saves video-/audio data in the MP4 file format and uses H.264 video compression/encoding and AAC audio compression/encoding. You should use a graphics card supporting hardware H.264 encoding and the latest Windows drivers. Additionally PROVEtech:TA uses Microsoft Media Foundation for processing video/audio thus your source devices must support that.

3.4.1 Converting data acquisitions

You get to the following dialog via the menu item *Data Acquisition*|*Convert Data Acquisition…* or by using the context menu of the overview graph described in chapter ***4.5.1 The overview graph, evaluation of measure data***:

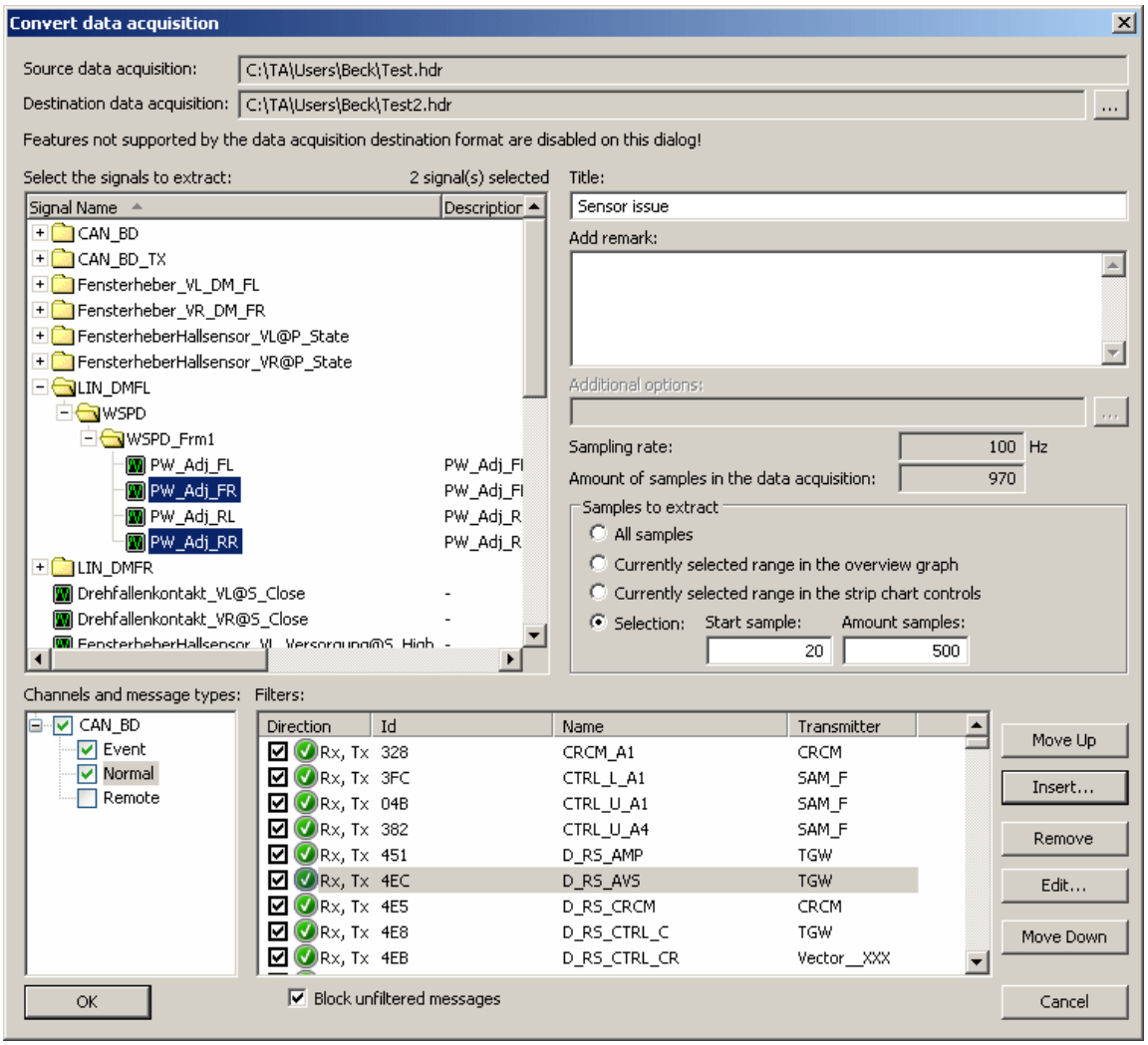


Figure 92 – Dialog for converting data acquisition files

After selecting the source data acquisition (when opening this dialog via the context menu of the overview graph automatically the currently opened data acquisition) the file selection dialog pops up (see chapter ***3.1 The file selection dialog***) to choose the destination path and format (via combo box `Save as type`).

Die dialog in ***Figure 92*** enters both information in the two topmost fields. Use the button ... to change the destination data acquisition afterwards.

In case the destination format does not support some options (e.g. only signals but no messages), the according fields of the dialog are disabled and these options are not available in the destination data acquisition.

You can change the following options in the dialog:

Use `Select the signals to extract` for selecting the signals you want to have in the destination data acquisition (see chapter ***4.2 The Signal Selection window***).

Attention: Not all file formats support signals. As an example the CANalyzer ASCII format only supports messages while exporting.

The lower part of the dialog is the same as the dialog for setting up the filters for the message window (see chapter ***4.4.14.1 The property dialog of the message window***). In case the source data acquisition has message channels, you can select if and which messages you want to see in the destination data acquisition. Attention: Not all file formats support messages. As an example, the MDF format only supports signals.

On the right side you can change the title of the data acquisition and add a comment in case the destination format supports this.

Use the `Samples To Extract` group to specify the time range to be contained in the destination data acquisition in samples:

`All Samples`:
All samples of the source data acquisition file will be transferred to the destination file.

`Currently selected range in the Overview Graph`:
This radio button is only available when you enter the dialog via the context menu of the overview graph. The range of samples shown in the edit boxes `Start` and `Duration` are used for the conversion.

`Currently selected range in the Strip Chart Controls`:
This radio button is only available when you enter the dialog via the context menu of the overview graph. The range of samples shown in the edit boxes `T1` and `T2` are used for the conversion (the values in these edit boxes correspond to the selected range in the strip chart controls).

`Selection`:
Use this radio button to select the samples manually by changing the values in the edit boxes `Start Sample` and `Amount Samples`.

Depending on the destination format, you can configure additional settings via the field `Additional options`. Use the button ... of the field to open a dialog for configuring the additional options. In case the destination format does not support additional options, these controls are deactivated.

After performing all these settings click on the `OK` button to start the conversion.



While converting data acquisitions to foreign formats information can get lost. Additionally some formats are not documented officially by the vendors, e.g. the CANalyzer ASCII log file format, so anomalies can happen.

The chapter ***8.13 Tips and tricks*** contains additional information about the data acquisition file formats.

3.5 The Macro menu

The *Macro* menu (see ***Figure 93***) is used to record and execute command sequences.

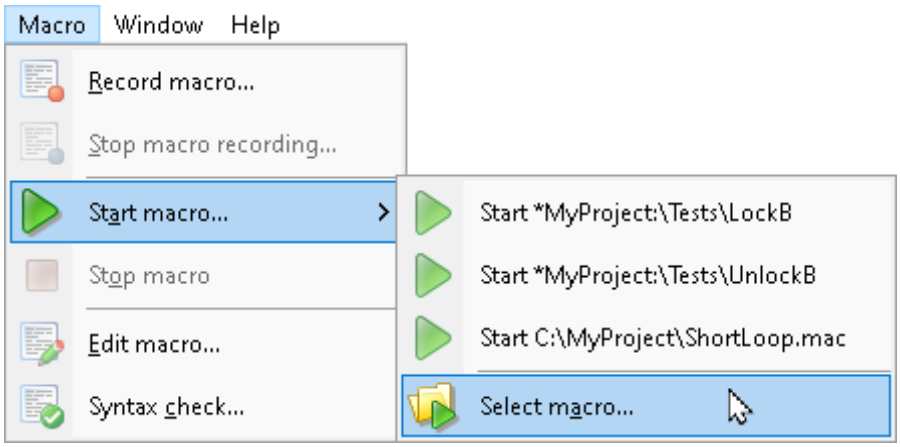


Figure 93 – The Macro menu

The following table contains a brief description of the menu items:

Record Macro…	Starts the recording of a macro. PROVEtech:TA displays the dialog shown in ***Figure 94***. You can also start the macro recording via the Test Manager (see chapter ***5.3.4 Recording a new test***).
Stop Macro Recording…	Stops the recording of a macro. In case you started it via *Record Macro…*, PROVEtech:TA displays the file selection dialog (see chapter ***3.1 The file selection dialog***) to select either a path to a .mac file or a Test Manager folder to store it. In case you started it via the Test Manager (see chapter ***5.3.4 Recording a new test***), it gets directly stored there. This menu item corresponds to the button  in the toolbar, if macro recording is active.
Start <Macro path>	The macro indicated by the path is executed. Meanwhile the color of the arrow gives information whether the macro is still running or the result value of the test result. This menu item corresponds to the button  in the toolbar.
Select Macro…	This displays the file selection dialog (see chapter ***3.1 The file selection dialog***) for selection of a .mac file or test object of the Test Manager tree that is subsequently started as a macro in the background. The path of the macro is added to the end of the recent macro list.

Stop Macro	Stops any macro and test running in the background premature. Caution: Stopped macros cannot be continued. You must restart the macro with *Start Macro…*. This menu item corresponds to the button  in the toolbar.
Edit Macro…	Opens a .mac file in an editor once you have selected the file in the file selection dialog (see chapter ***3.1 The file selection dialog***). The editor used for .mac files is the one you have associated with the extension .txt. In case the extension .txt is unassigned, Windows Notepad displays the .mac file.
Syntax check…	The file selection dialog (see chapter ***3.1 The file selection dialog***) appears for selecting a .mac file. A WinWrap Basic syntax check is then performed on the file content.

First, the following dialog is displayed when recording a macro:

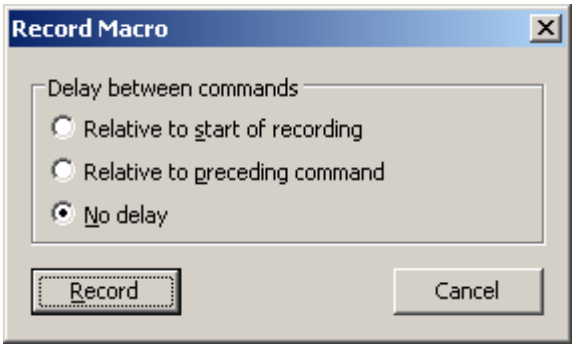


Figure 94 – Selection dialog for macro recording

Three recording types with different delays between the recorded commands are available:

Relative to start of recording	The duration of an execution at a later point in time corresponds to approximately the recording duration. Delays of individual commands are balanced across the entire macro.
Relative to preceding command	The later execution of a macro with this recording type usually takes longer than the original recording. This guarantees that the delay between the individual commands corresponds to at least the delay during recording.
No delay	The execution duration of this recording type is shorter than the recording. The time between the individual operations during the recording is irrelevant.

Click on the `Record` button to start recording. This is indicated by the animated mouse cursor:  . Your subsequent operating steps with the keyboard and the mouse are automatically turned into test language program code and buffered in working memory. Recording is only possible for operating sequences which can be converted to the test language. If a control is moved on the workpage, for example, this step is ignored while changing a signal value via a working control is recorded and converted to the test language.

You stop the recording via the menu item *Macro|Stop Macro Recording…*. In case you started it via *Record Macro…*, PROVEtech:TA displays the file selection dialog (see chapter ***3.1 The file selection dialog***) to select either a path to a .mac file or a Test Manager folder to store it. In case you started it via the Test Manager (see chapter ***5.3.4 Recording a new test***), it gets directly stored there.

You can import and export .mac files into/from the Test Manager. Hotkeys can be associated with .mac files and with tests in the database (see chapter ***3.2.7.1 Common***).

For information about the macro commands have a look in the test language manual.

The  button shows a yellow background if any macro (not test) is currently running. When the last macro terminates, the background shows the macro result: red for “failed”, green for “succeeded” and grey for “indifferent”. The test language command ***System.SetResult*** influences the macro result.

The spinning icon at the top right position of the first monitor indicates if any macro or test is currently running:



Figure 95 – Spinning icon

In case you are executing a test in the Test Manager (see chapter ***5.5 Executing tests***), you can click on this spinning icon to select the `Execute` tab of the Test Manager.



In opposite to the execution of tests, PROVEtech:TA creates no result in the database when executing a macro. This is also valid for the execution of event macros (see chapter ***3.2.7.2.1 Event macros***) and the execution of macros via the workpage. However, you can execute the same Basic program as a test or a macro. Chapter ***5.5 Executing tests*** describes how to execute tests and generate test results. The test language command ***TM.Execute*** also creates a test result for the test it executes, and you can use it within macros.



The macro recording is disabled while executing a macro or test.



Recording macros has limitations. It cannot cover all test language commands and not all PROVEtech:TA actions are covered by the test language commands. The created macros produce no real test results as they evaluate no data. But they give you hints how to use the test language commands.



You can execute several macros and tests in parallel, but you are responsible for a useful usage of this possibility. For example, it makes no sense to access signals from one macro while a 2nd one re-loads the model in parallel.

3.6 The Window menu

In general, the *Window* menu helps to manage some of the PROVEtech:TA windows.

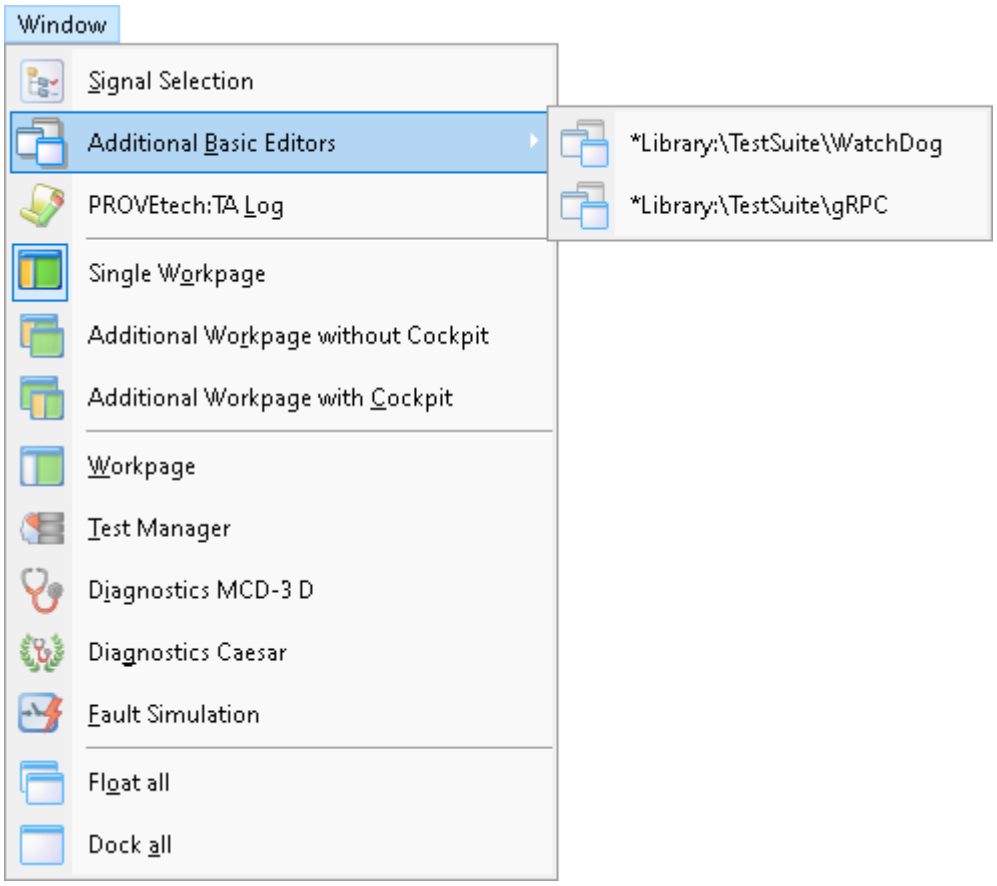


Figure 96 – The Window menu

The submenu item *Signal Selection* is used to display and hide the Signal Selection window (see chapter ***4.2 The Signal Selection window***). As an alternative, the button  on the toolbar provides the same functionality. When the Signal Selection window is visible, the icon of the menu item is marked and the button has the following appearance: 

With *Additional Basic Editors* you can change to one of the additional opened Basic editors in an external window.

In menu *Windows* you find three items to open a second window with a Workpage and with or without a cockpit. This second window may be useful to display it on a second screen. This feature replaces the former feature of displaying a Workpage on a window extended over two screen.

You can create non-modal windows from the tabs of the functional areas (Workpage, Test Manager, Diagnosis, Fault Simulation) via drag and drop (except the Workpage). You can dock them back to the other tabs by closing the windows. See also ***2.6 Placing the functional areas***.

In the menu *Window* you can bring these windows to the foreground or select them in the tabs.

You can float all tabs (except the Workpage) with `Float all` and dock them back with `Dock all`. You also have these functions in the context menu of the tab control.

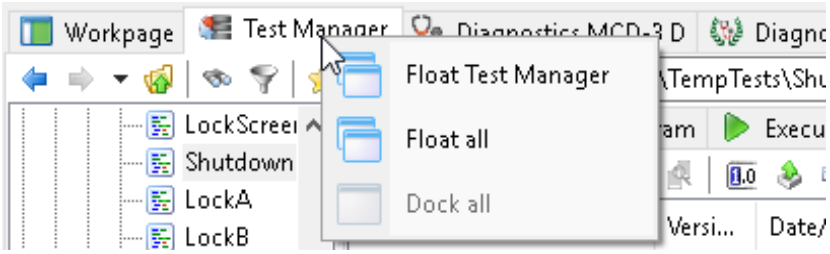


Figure 97 – Context menu for floating the working panels

3.6.1 PROVEtech:TA Log

Select *PROVEtech:TA Log* (corresponds to the button ) to bring the PROVEtech:TA Log window (see ***Figure 98***) to the foreground.

The PROVEtech:TA software is modular in design. It consists of several program libraries. The PROVEtech:TA Log is a log window in which messages that were generated by the individual modules are displayed in chronological order. The messages thus allow you to detect, which functions are running at the moment in which program modules.

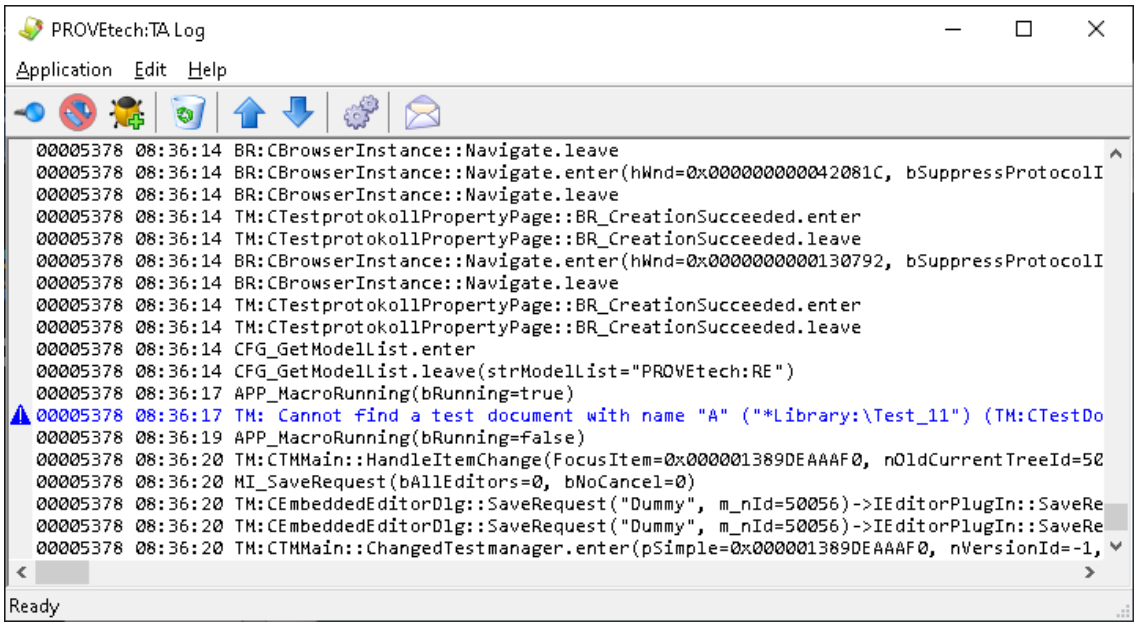


Figure 98 – PROVEtech:TA Log window

"Normal" status messages are displayed black and are primarily of interest to PROVEtech:TA developers. Messages of interest to the user are displayed in color. Two types of messages are distinguished: Warnings (identified by the symbol on the left side) and error messages (symbol). While warnings merely draw the user's attention to a status that may result in a problem later on, errors usually cause the current function to be aborted.

If the PROVEtech:TA Log outputs an error message, then the toolbar button starts to blink red. You first must click the button, or you have to wait for a certain time, then the blinking will disappear.

Contrary to the normal messages, the output of warnings and error messages cannot be disabled. Refer to the chapter ***3.6.1.2 Settings*** for information on setting the messages to be output.

The first column in the window shows the hexadecimal thread Id that generated the message, and the second column displays the local time when it happened.

This window can only hold a limited number of messages. When this limit is reached, the PROVEtech:TA Log deletes old messages. Furthermore, it may not display all messages if you disabled the output of some modules. In the background, PROVEtech:TA also outputs all messages into a log file without the filtering. The menu item *Edit|Open in external Editor* opens that file in your default editor application.

3.6.1.1 The menu bar and the toolbar

A description of the menu items and the toolbar buttons of the PROVEtech:TA Log follows:

Menu item	Icon	Description
Application|Settings…		See chapter ***3.6.1.2 Settings***.
Application|Always Top		When this function is activated (symbol  on the toolbar), the PROVEtech:TA Log window always remains in the foreground. If the function is inactive, other windows may cover up this window.
Application|No Scrolling		Switches automatic scrolling due to new messages off and on.
Application|Show Additional Debug Output		Switches the output of additional debug messages off and on. These additional messages help the developers to analyze issues.
Application|Reduce to Icon		The PROVEtech:TA Log disappears from the user interface and is displayed as an icon () on the task bar next to the time display. Click on the icon to redisplay the window.
Edit|Clear Window		Deletes all messages from the window.
Edit|Copy		Copies the selected line to the clipboard.
Edit|Open in external Editor		Opens the standard text editor of your system and displays the log file.
Edit|Previous Error or Warning		Goes to the next earlier warning or error message. If there is no earlier message, the first message is displayed.
Edit|Next Error or Warning		Goes to the next (more recent) warning or error message. If there is no later message, the first message is displayed.
Edit|Find…		Displays a dialog to specify the find options and then searches for an according occurrence in the PROVEtech:TA Log.
Edit|Find Previous Occurrence		Searches for the previous occurrence of the specified find options.

Edit|Find Next Occurrence		Searches for the next occurrence of the specified find options.
Help|Help		Opens the help for the PROVEtech:TA Log
Help|Technical Support…		Opens the dialog described in chapter ***2.13 Creating a problem report*** for contacting PROVEtech Support

3.6.1.2 Settings

The “PROVEtech:TA Log Settings” (***Figure 99***) dialog configures the output of the PROVEtech:TA Log.

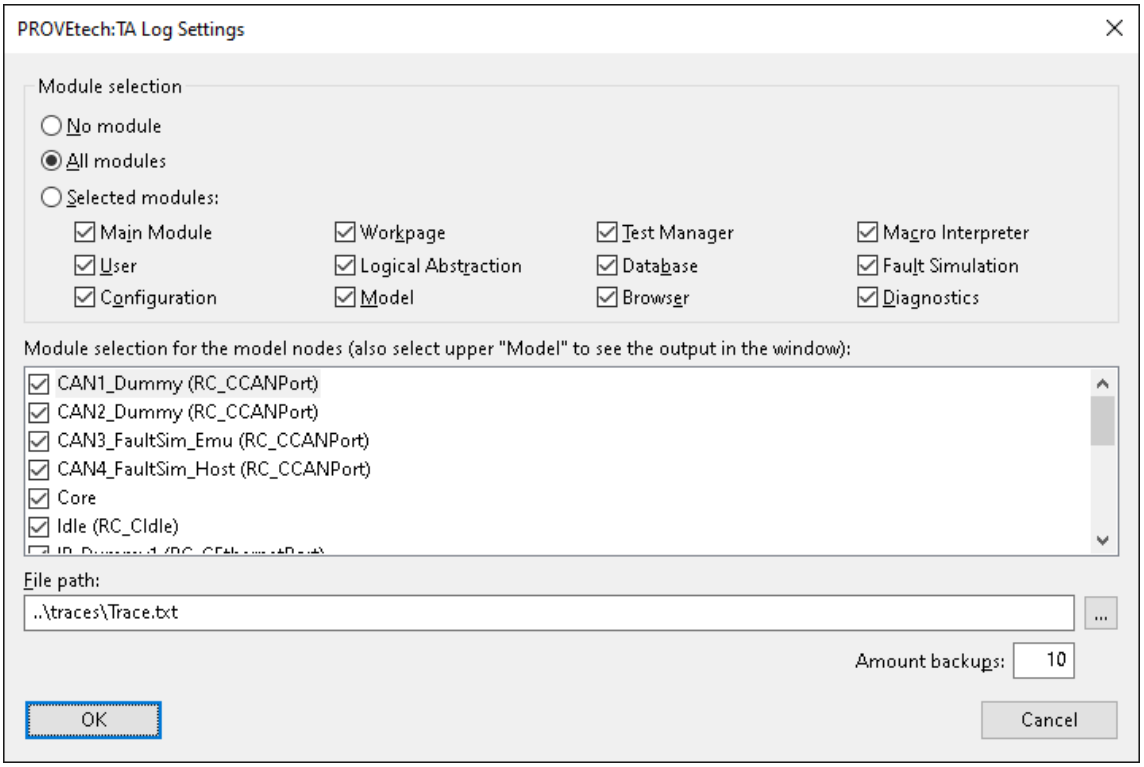


Figure 99 – PROVEtech:TA Log settings

In the `Module Selection` group you select the PROVEtech:TA modules that are allowed to output messages in the PROVEtech:TA Log window. You can disable or enable all modules or you can do an individual selection by using the check boxes:

To display messages related to…	**Activate…**
general functions of the main module	`Main Module`
accesses to user-relevant data	`User`
accesses to the test system configuration	`Configuration`

output from your model (if it supports that)	Model
the test language editor, the execution of tests and recording of macros	Macro Interpreter
graphics output (workpage, cockpit, offline mode)	Workpage
access to signals, data acquisitions and video/audio	Log. Abstraction
usage of the Test Manager	Test Manager
calls to the test database interface	Database
test protocol output or other web browser related things	Browser
activities relating to the simulation of errors in the cabling and sensors / actuators	Fault Simulation
processes in the diagnostics module for ECU diagnosis	Diagnostics

The Module selection for the model nodes list works similar to the Module selection group. However, it controls the model text output instead of the PROVEtech:TA internal text output. The list is only enabled if your currently loaded model supports this (mainly PROVEtech:RE models). PROVEtech:TA gets the module names and the check box selection from the model. And PROVEtech:TA transmits the selection back to the model if you click OK. In opposite to the Module selection group, it also enables/disables the output to the file, because messages are not even transmitted to PROVEtech:TA if you disable them. Furthermore, the upper Model check box controls the output to the PROVEtech:TA Log window, but not the output to the file.

Example for the output of the upper GenericApp:

GenericApp checked	Model checked	File output	Window output
X		X	
X	X	X	X
	X		

File Path:
This edit box defines the path of the output file. PROVEtech:TA creates the file each time it starts and renames the existing one for backup purposes. Use the edit box Amount Backups to set the amount of backup copies PROVEtech:TA should keep before deletion.

3.7 The Help menu

This menu (see ***Figure 100***) provides additional information of PROVEtech:TA and is used to access this manual. Click on *Help* to display the manual in the Windows help format. Select the menu item *Homepage* to open your Internet browser and load the PROVEtech homepage. *Change Request* displays the homepage to add and manage change requests. You can send a problem report together with the current PROVEtech:TA version status to the support by choosing the menu *Technical Support…* (see ***2.13 Creating a problem report*** or ***3.7.1 Technical Support***). The menu item *Update License…* opens the license dialog (see ***2.2.1 Computer-bound licensing***). Click on *Info…* to obtain information on the individual PROVEtech:TA program modules and an e-mail address for questions, suggestions and problems encountered (see ***3.7.2 The Info dialog***).

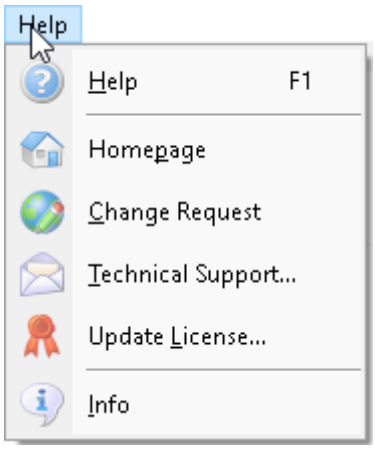


Figure 100 – The Help menu

3.7.1 Technical Support

This menu item of PROVEtech:TA or the PROVEtech:TA Log opens the dialog shown in ***Figure 16***. In case PROVEtech:TA cannot start anymore, you can also open this dialog by starting the application TALog.exe of the PROVEtech:TA directory.

For more information, please have a look at chapter ***2.13 Creating a problem report***.

3.7.2 The Info dialog

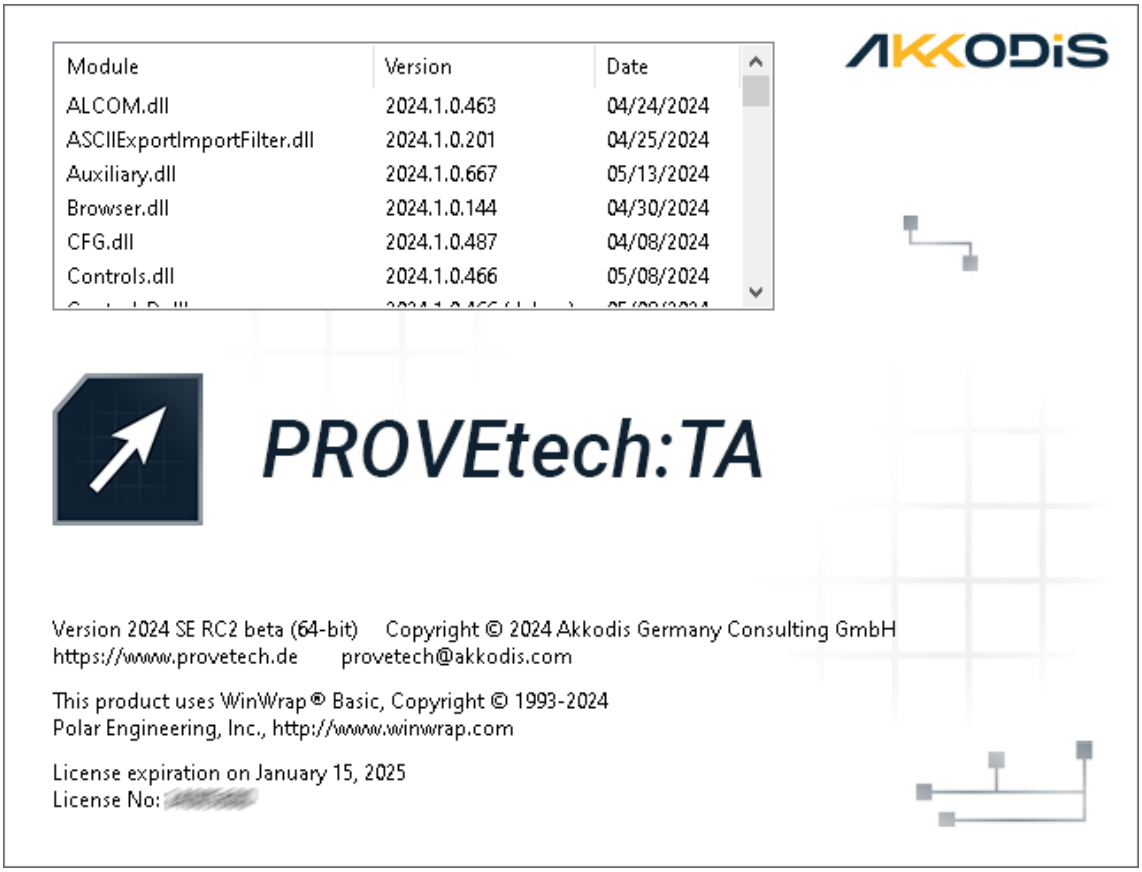


Figure 101 – The Info dialog

In the Info dialog you can see the following information:

- Version name of the current PROVEtech:TA installation (in the figure it is PROVEtech:TA version 2023 SE (64-bit), “SE” is Summer Edition, “RC” would mean Release Candidate, “SP” is Service Pack and “beta” means preliminary version)
- Product and company logos as well as legal information
- The contact address of the technical support provetech@akkodis.com and the homepage https://www.provetech.de regarding downloads, release notes, newsletter and more.
- License information (e.g. expiry date or if the license is bound to a specific PROVEtech:TA version, or if it is granted to start several PROVEtech:TA instances in parallel by different Windows users, see ***2.2.1 Computer-bound licensing***) and license number. If a license is version-bound, all service packs for this version are still included.
- Internal version information of the PROVEtech:TA modules

3.8 Hotkeys overview

This chapter contains an overview of helpful hotkeys for some modules.

3.8.1 Hotkeys for the workpage

In case the keyboard focus is on the cockpit or the workpage, you can use the following hotkeys. On the lower tabs of the workpage the meaning of some hotkeys differ.

Hotkey	Function
F2	Rename the selected page
F12	Open the signal selection window and select the signal of the selected control
Del	Delete the current page or the selected controls
Alt+Enter	Open the property dialog
Ins	Insert a new page
Ctrl+Down	Align the selected controls at the bottom
Shift+Down	Move the selected controls down pixel by pixel
Ctrl+Left	Align the selected controls at the left
Shift+Left	Move the selected controls left pixel by pixel
Ctrl+Right	Align the selected controls at the right
Shift+Right	Move the selected controls right pixel by pixel
Ctrl+Up	Align the selected controls at the top
Shift+Up	Move the selected controls up pixel by pixel
Ctrl+A	Select all controls
Ctrl+C / Shift+Del	Copy the selected controls to the clipboard
Ctrl+F	Open the find dialog
Ctrl+H / Ctrl+R	Open the replace dialog
Ctrl+L	Load a workpage
Ctrl+Shift+L	Load a cockpit

Ctrl+N	Create a new workpage
Ctrl+Shift+N	Create a new cockpit
Ctrl+S	Save the current workpage using its current path
Ctrl+Shift+S	Save the current cockpit using its current path
Ctrl+V / Shift+Ins	Paste the clipboard
Ctrl+X	Cut the selected controls
Ctrl+Y	Redo
Ctrl+Z / Alt+Backspace	Undo
Ctrl+Plus	Increase the view scaling by one step
Ctrl+Minus	Decrease the view scaling by one step
Shift+ Mouse wheel	Scroll horizontally

3.8.2 Hotkeys for the Test Manager (Tree, Explorer-tab, Info/Version-tab)

Hotkey	Function
Ctrl+F5	Refresh
Ctrl+A	Select all
Alt+Return	Display properties
F2	Rename the test object
Alt+F2	Rename the prefix of the test object's name
Ctrl+X / Shift+Del	Cut into the clipboard
Ctrl+C / Ctrl+Ins	Copy a test object and its Test Manager path into the clipboard
Ctrl+Shift+C / Ctrl+Shift+Ins	Same as Ctrl+C except that the Test Manager paths contain database identifiers (see also in chapter ***Test paths*** in the Test Language section)
Ctrl+Shift+V	Paste
Ctrl+V / Shift+Ins	Paste with reference to older versions

Ctrl+E	Paste with test results
Ctrl+L	Create link
Ctrl+B	Open the Branch Selection dialog (see ***5.8.7.2 The branch selection overview dialog***)
Ctrl+Shift+B	Branch Selection: Set/Reset as current version (see ***5.8.7.1 Define the branch selection by context menu***)
Ctrl+T	ToDo list dialog (see ***5.17 ToDo list dialog***)
Ctrl+F	Find (see ***5.18.1 Finding tests and test results***)
Ctrl+H / Ctrl+R	Find and replace (see ***5.18.3 Find and replace in test programs***)
Ctrl+Z	Undo
Ctrl+Y	Redo
Del	Deleting tests
Ctrl while drag and drop	Copying tests
Shift while drag and drop	Creating links
Ctrl+Shift+L	Select left side to compare
Ctrl+Shift+R	Select right software to compare / Compare to
Ctrl+Alt+0..9	Set selected module as a hotkey
Alt+0..9	Select module
Ctrl+Alt+Backspace	Navigate back
Shift+Backspace	Navigate forward
Shift+Up	Move up test object
Shift+Down	Move down test object

3.8.3 Hotkeys for the test parameters

Hotkey	Function
Ctrl+S	Save parameters
Ctrl+X	Cut parameters
Ctrl+C	Copy parameters (This hotkey also copies the selected parameters as a xml string into the clipboard. Then you can paste it into a text editor.)

Ctrl+V	Paste parameters
Del	Delete parameters
Shift+Del	Delete the parameter in the whole test object's subtree (Deep delete)
Ins	Add a new parameter into the selection
F2	Rename a parameter
Ctrl+F2	Rename a parameter in the whole test object's subtree (Deep rename)
Ctrl+A	Select all
F4	Edit parameter type
Ctrl+T	Open dialog "Edit choice types"
Ctrl+W	Edit parameter value
Ctrl+K	Add/edit hyperlink
Ctrl+U	Edit parameter unit (only possible if the parameter is of type Long or Double)
Ctrl+D	Edit parameter description
Ctrl+I	Show inheritance dialog

3.8.4 Hotkeys for the Basic editor

The following hotkeys / keyboard shortcuts are available in the editor:

Hotkey	Symbol	Function
F1		Displays help information for the currently marked command
Shift+F1		Calls up the online help for Basic
F2		Go to next bookmark, modus selectable, see below
Shift+F2		Go to previous bookmark, modus selectable, see below
Ctrl+F2		Set / remove bookmark
F3		Continue searching with the current settings (defined via Ctrl+F); direction: down
Shift+F3		Continue searching with the current settings (defined via Ctrl+F); direction: up
Ctrl+F3		Search for the currently marked text or for the word under the keyboard cursor; direction: down

Ctrl+Shift+F3		Search for the currently marked text or for the word under the keyboard cursor; direction: up
Alt+F3		Starts the incremental search, the mouse cursor changes and you can start typing the text you want to find; the editor directly marks the first match; the status bar informs about the typed text; direction: down
Alt+Shift+F3		Same as Alt+F3 but the direction is “up”
F4		Syntax check
Shift+F4		In case “ShiftF4Path” is specified in TM.ini: Saves the currently displayed module and starts the specified module using *System.GetGVar("_OriginPath_")* to get the displayed module’s path for a user defined extended syntax check
Ctrl+F4		Closes a floating Basic window
F5		Start / continue
Shift+F5 / Esc		Terminate
F7		Execute up to cursor position
Shift+F7		Shows the next statement
Ctrl+F7		Sets the next statement
F8		Jump in
Shift+F8		Skip
Ctrl+F8		Jump out
F9		Set / remove breakpoint
Shift+F9		Calculate expression (view in the output window)
Ctrl+F9		Add expression to the watch window
Ctrl+Shift+F9		Remove all breakpoints
F12		Tries to find the definition of the item whose name the keyboard cursor currently resides on, e.g. a function. In case the keyboard cursor is on a '#uses line, the editor opens the corresponding module. In case of a signal name the signal selection window shows up and selects that signal.
Ctrl+Space		Complete word
Ctrl+Tab		Alternates between the Basic windows in case several ones are open

Ctrl+Shift+Tab		Alternates between the Basic windows in reverse order in case several ones are open
Shift+Enter		Auto completion of pre-defined text modules (see ***5.4.5 The edit window***)
Ctrl+0..9		Memorize currently selected module in one of 10 entries
Alt+0..9		Select the module previously memorized with Ctrl+0..9
Ctrl+A		Mark all
Ctrl+C		Copy marked area to the clipboard
Ctrl+D		Empty the output window
Ctrl+E		Sets the focus to the output window
Ctrl+F		Search (see ***5.4.6 The find and replace dialog***)
Ctrl+G		Opens a dialog for entering the line number you want to display
Ctrl+H		Replace (see ***5.4.6 The find and replace dialog***)
Alt+H		Replace the symbol name at the caret position within the whole module (but not across several modules)
Ctrl+I		Display parameter info
Ctrl+J		Comment out the marked area
Ctrl+K		Remove comment characters in the marked area
Ctrl+M		Automatically formats the selected lines (*)
Ctrl+O		Shows / hides the output window. While executing a test it is always visible.
Ctrl+P		Print
Ctrl+Q		Sets the focus to the test tree
Ctrl+R		Replace (see ***5.4.6 The find and replace dialog***)
Ctrl+S		Save changes to the database
Ctrl+Shift+ S		Completes an inchoate signal name, find a sub string
AltGr+S		Same as above but search for identical beginning
Ctrl+T		Set the focus to the stack window
Ctrl+V		Paste clipboard

Ctrl+Shift+V		Paste clipboard embedded in single quotation marks
AltGr+V		Paste clipboard embedded in double quotation marks
Ctrl+W		Set the focus to the watch window
Ctrl+X		Cut out marked area and copy to clipboard
Ctrl+Y		Redo
Ctrl+Z		Undo
Shift+Backspace		Forward to the module edited before
Ctrl+Backspace		Back to the latest edited module
AltGr+Backspace		Deletes the word before the keyboard cursor
Alt+Up		Move the current line or the marked lines up
Alt+Shift+Up		Mark previous symbol usage
Alt+Down		Move the current line or the marked lines down
Alt+Shift+Down		Mark next symbol usage
Alt+Shift+Home		Mark symbol definition
Ctrl+'+' / Ctrl+'-'		Expands/collapses the current function/type body (toggles the outlining)
Ctrl+Shift+'+'		Expands all function and type bodies (outlining)
Ctrl+Shift+'-'		Collapses all function and type bodies (outlining)



(*) The automatic format of the selected lines via Ctrl+M first searches the line before the selection to detect the correct indention. In case there is none (selection starts at the beginning of the text), the algorithm starts without indention. After that it indents the selected lines automatically. It only uses the tab character for the indention. It takes care of connected lines. The characters Space+_ at the end of the line mark that the next line is part of the multi-line statement. These lines get an additional tab for the indention.

The algorithm considers not only the normal Basic statements like "If" / "End If", "For" / "Next", "Function" / "End Function", but also the Automation Library statements like "RT.BeginComputation" / "RT.EndComputation".



Move the mouse cursor across the toolbar buttons slowly to display a brief explanation of each button function. Keyboard shortcuts exist for some functions; these are shown in brackets. The context menu also displays the according hotkeys.



You can use bookmarks to mark lines in the module. Press F2 or Shift+F2 to jump between these lines. PROVEtech:TA saves the set bookmarks when you change the module so that you can use them again when you go back. PROVEtech:TA looses this information when you close it.

You can change the jump mode for the bookmarks via the context menu item *Jump To Bookmarks Module Spanning*:
- Module internal for jumping to the bookmarks within modules in the ascending or descending order.
- Module spanning for jumping to the bookmarks in the order you have set them across modules.

Chapter

4

Workpage

The workpage is used to display and stimulate signals. You select some signals from the quantity of all signals and adjust the desired signal values with various controls (for example, by means of slide controls or buttons). The chronological progression of signals is represented by strip chart controls. Additionally you can display message traces (see chapter ***4.4.14 The message window***. The complete workpage can be operated conveniently by means of drag and drop or copy and paste and with the mouse via the context menu. Numeric values can also be entered via the keyboard.

In this chapter you will find out

- how to work with several pages of a workpage
- how to work with your stock of signals
- how to represent your signals as display and control elements
- how to display messages
- how to manage these controls
- what configurable properties these controls have
- how to display recorded signal data and messages in the so called offline mode

4.1 Managing the workpage

To access the workpage in PROVEtech:TA, click on the tab marked in ***Figure 102***; it is located in the upper quarter of the PROVEtech:TA window.

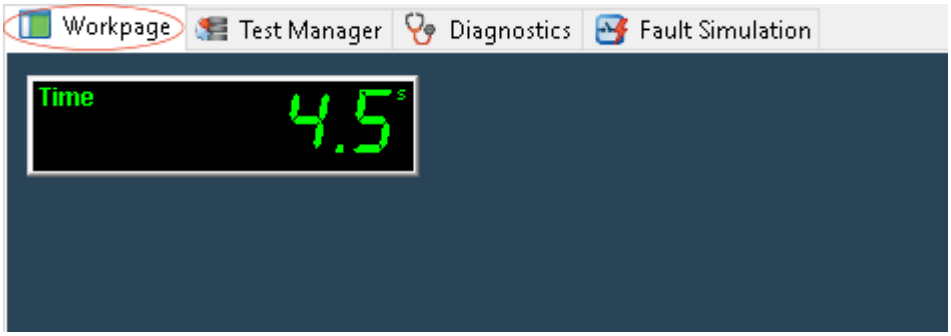


Figure 102 – Workpage tab

The workpage has several pages. In the lower quarter of the PROVEtech:TA window you see additional tabs (***Figure 103***). If you work with a second workpage, these tabs are displayed on both windows. Each tab corresponds to one page of the workpage. The selected tab, `Page2` in this case, identifies the page that is currently visible. Click on a different tab to change pages. If you work with two windows and you select the page on one window that is being displayed on the other window, the page is automatically changed on the other window, i.e. the pages displayed are exchanged on the two windows.

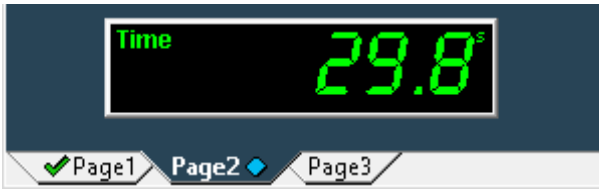


Figure 103 – Tabs for the pages of a workpage

You can use the drag and drop technique to change the order of the pages: click on a tab and pull it to the desired position. The place where the page will be inserted is identified by a triangle (see ***Figure 104***). The position changes when you release the mouse button.



Figure 104 – Changing the page sequence using drag and drop

As you can see in the previous figure, the display also supports colored characters (Emojis) in page names. Windows supports the global hotkey Windows+. to enter Emojis.

When you click on the page tab with the right mouse button, the following menu appears:

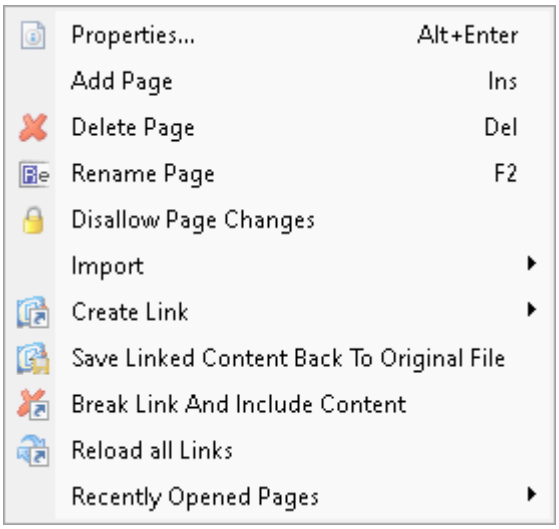


Figure 105 – Page management menu

This menu is used to add, delete or rename pages. Adding pages also works with this + button:



Figure 106 – Add a page

The menu item *Disallow Page Changes* protects the page from unwanted changes. As an example it prevents creating new controls on the page or deleting or moving them or changing their properties. A lock symbol appears on the tab:



Figure 107 – Locked page

Right-click on the tab again to display and select the menu item *Allow Page Changes*. This removes the page lock.

Select the menu item *Import* to add one or several pages from a previously saved workpage. In the pop up sub menu you can select one of the recently used workpage files or you choose *Select File…* to select your file via the file selection dialog (see chapter ***3.1 The file selection dialog***). Then you select the pages to be imported in the dialog shown in ***Figure 108*** and click on `Import`.

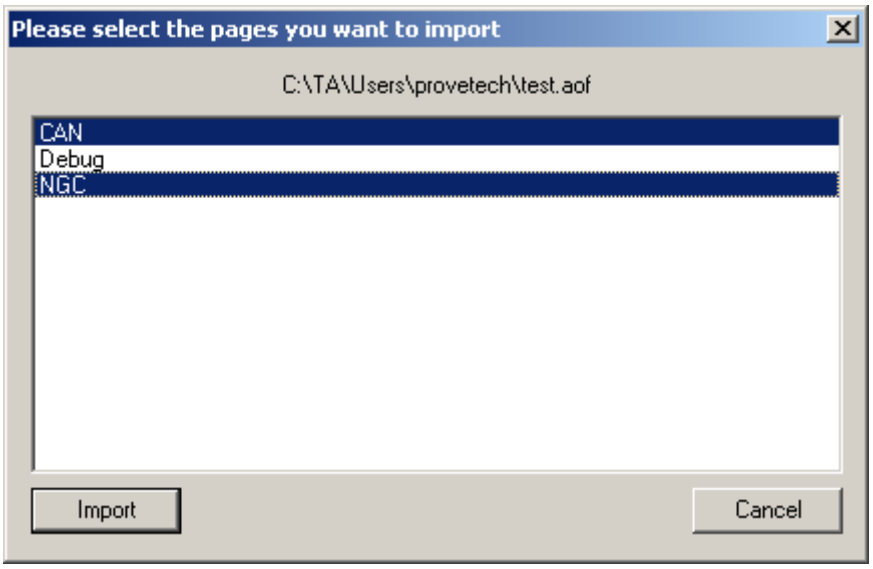


Figure 108 – Importing pages of a workpage

After this the new pages are available on your workpage.

The *Create Link* menu item (***Figure 105***) works similar as *Import*. The difference is, that the inserted pages are links that have a special symbol on the according tabs:



Figure 109 – Tab with link symbol

While loading a workpage file, linked pages get their content from the linked original workpage file. Changes on the controls of a linked page will not be saved when saving your workpage file. But you can explicitly save your changes back to the original file by selecting the menu item *Save Linked Content Back To Original File* (***Figure 105***). The menu item *Break Link and Include Content* dissolves the link and imports the content.

Select *Reload all Links* after clicking on a linked page to reload all linked pages and group controls (see ***4.4.16 The group control***) from their source files. Any changes within linked pages and group controls are lost and it is not possible to undo this operation.

The menu item *Recently Opened Pages* shows a sub menu with the recently opened pages. Select one to open it.

Refer to chapter ***3.3.2 New workpage, load workpage and save workpage*** for information on how to load and save workpages.

The cockpit area on the left side of the PROVEtech:TA main window can display the same controls as the workpage and you can control it the same way. It always stays visible, so it is

suited for displaying very important signals. The chapter ***3.3.1 New cockpit, load cockpit and save cockpit*** describes the file management for the cockpit.

4.1.1 View scaling

The workpage and the cockpit support scaling the view (zoom) independent from the Windows screen scaling setting. It is a user dependent setting between 100 % and 225 % in 12.5 % steps. The lower right corner of the workpage has a combo box to select it:

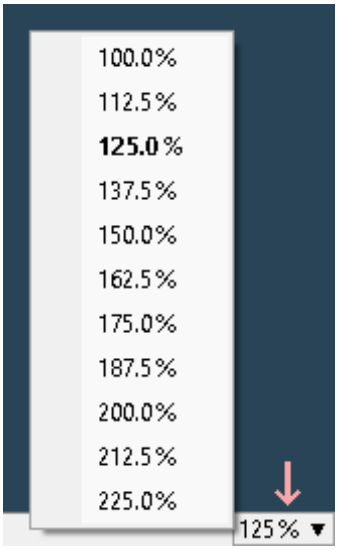


Figure 110 – View scaling

It is a common setting for both workpage windows and the cockpit.

Please be aware that the possibility to scale the view has some drawbacks. In the end, your monitor displays pixels. All output is rounded to these pixels. In some cases like scaling bitmaps, interpolation helps to hide issues with that. In other cases where you expect sharp edges (e.g. frames around controls) interpolation would look bad and rounding to one of the pixels is necessary. Furthermore, up-scaling bitmaps tries to view more information than there actually is. Scaling fonts is even harder as you need a balance for good readability vs. exact scaling. This means scaling the view is always a compromise and never exact.

We chose a step width of 12.5 % for the scaling setting (half the default Windows step width of 25 % for the screen scaling). This means, scaling numbers that are a multiple of 8 works without rounding issues (1/8 = 0.125). Therefore, the default raster is 16 pixels at 100 % scaling (see chapter ***4.3.2 The property dialog for pages and group controls***). This places and sizes controls at a multiple of 16 pixels for 100 % scaling. We recommend sticking to that.

The property dialogs of all controls display any position/size number after applying the view factor. This means, they display different values after changing the view scaling. However, the workpage and cockpit files store all values for the 100 % view (with decimal places if necessary). This means, PROVEtech:TA versions prior to version 2023, which added this feature, can load these files, but they ignore the decimal places. Furthermore, these older versions had a default raster setting of 49x25 pixels, which scales badly. Switching to 16x16 at 100 % scaling is recommended.

We recommend to always use the same view scaling when changing workpage and cockpit files. This ensures a consistent result.

4.2 The Signal Selection window

The Signal Selection window (***Figure 111***) is activated via the menu *Window|Signal Selection*, the button  (see chapter ***3.6 The Window menu***) or via the context menu of the workpage (click with right mouse button on a free location of the workpage). The activated window always remains in the foreground until you close it by clicking on the button ×.

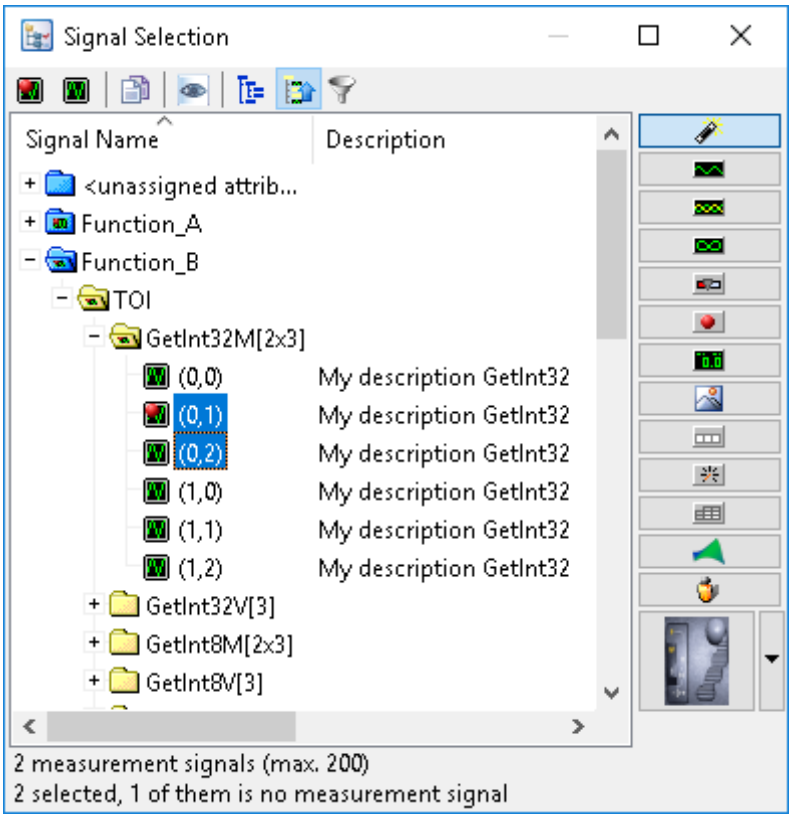


Figure 111 – The Signal Selection window

The following chapters describe the various function areas of the window.

4.2.1 Signal area describes the center area of the window.

4.2.2 Column headers describes the column headers of the signal area.

4.2.3 Measurement signal selection, clipboard and info field describes the toolbar and the lower information text display of the window.

4.2.4 Selecting control types, adding controls describes the right buttons of the window.

4.2.5 Changing CCP/XCP event channel numbers describes how to change CCP/XCP event channel numbers for CCP/XCP signals.

Please be aware that the function of the signal selection window slightly changes in the offline mode. See chapters ***4.5 The offline mode***, ***4.5.3 Extract signals from messages*** and ***4.5.4 Calculated signals***.

4.2.1 Signal area

This area provides a mixture of a tree and a list view of your signals. It can contain XCP signals as well. The tree is located in the “Signal Name” column. It has three different element

types: attributes (,), nodes (,) and signals (,). The other columns contain additional information on your signals. The symbol identifies a measurement signal. A measurement signal is a signal acquired during a data acquisition. Please refer to chapter ***3.4 The Data Acquisition menu*** for information on how to start data acquisition. The symbol indicates that the signal is not a measurement signal, i.e. is of no importance during a data acquisition. Click on this symbol to change the measuring signal status of this individual signal. The attribute and node symbols changes to or respective or if the node includes measurement signals.

You must select one or several signals before any operation can be performed on signals. Proceed as in the Windows Explorer to select signals. For example, you click on a signal name to select a signal. Hold the Ctrl key down and click on several signal names to select these signals. Several contiguous signals can be selected by dragging with the mouse to draw a rectangle around the signal names. If you select a node, this automatically means that all signals contained in it are selected.

When you click with the right mouse button (context menu), the following menu appears:

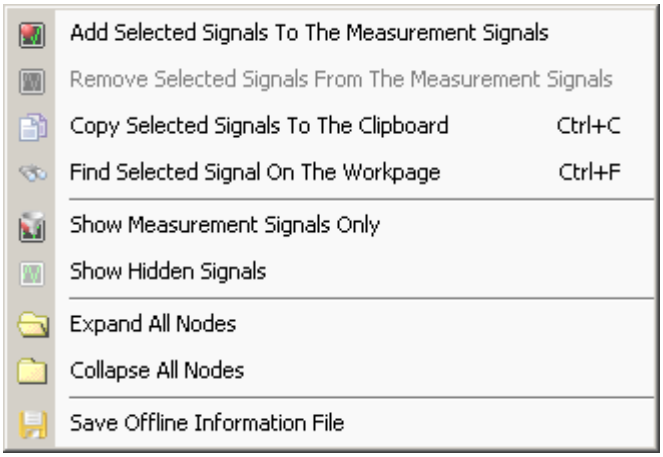


Figure 112 – Signal area context menu

The first three menu items correspond to the three toolbar buttons , and in chapter ***4.2.3 Measurement signal selection, clipboard and info field***.

Find Selected Signal On The Workpage:
In case the signal is somewhere on the workpage or the cockpit area, PROVEtech:TA marks the according control and opens the find dialog (see ***4.3.8 Find and replace***) for finding additional controls using the signal. Otherwise a message box tells you, that PROVEtech:TA did not find the signal.

Show Measurement Signals Only:
Sets / clears the filter which shows only measurement signals.

Show Hidden Signals:
Activate this switch to show also the hidden signals. It is only useful if your signal configuration has any hidden signal. Hidden signals and nodes have brighter symbols.

Expand All Nodes:
Opens all nodes in the tree to render all signals visible. Please choose this option only if there are only a few signals in the tree. Otherwise opening all nodes can take a long time.

Collapse All Nodes:
Closes all nodes in the tree.

Save Offline Information File:
This opens the file selection dialog (see chapter ***3.1 The file selection dialog***) to save all signal information in a so called Offline Information File with the extension ".oif". You can select this to use the signal information offline without a real HiL system in PROVEtech:TA, e.g. for creating test programs. For that PROVEtech:TA supports a separate platform type in the PROVEtech:TA model configuration. See chapter ***3.2.7.2.3.2 Akkodis Offline Information File***.



Drag the mouse cursor over a signal line and then do not move it for a while to see a tooltip displaying the current signal value with the signal unit. This does not work for CCP/XCP signals. The tooltip uses the last selected settings for the displayed signal in a number control (see ***4.4.6.1 The property dialog of the number control***). Among other things this contains the unit, the amount of displayed decimal places and the selected bits.

Each signal has a unique signal name that contains the names of all parent nodes and the actual name of the signal. The attributes are not part of the signal name, PROVEtech:TA only uses them for sorting the signals. Click  to select the attributes for the tree structure (see chapter ***4.2.1.1 Attribute selection***).

4.2.1.1 Attribute selection

PROVEtech:TA uses the attribute concept to display several thousand signals well arranged in a hierarchical structure. You can use the attribute selection dialog to adapt this structure. So an attribute is a sorting criteria for the signal tree structure. When you select the attribute "ECU" as an example, the signal tree displays a folder ( , ) for each existing ECU. Each ECU name is an attribute value of the attribute "ECU". When PROVEtech:TA builds up the signal tree it puts each signal into its corresponding ECU folder. Normally you can assign each signal exactly to one attribute value of an attribute (e.g. a signal can only have one physical unit). Sometimes this is not possible. As an example you can assign the functions "Exterior Light" and "Closing" to a signal that controls a direction indicator. In this case PROVEtech:TA shows the signal in the folder "Exterior Light" and in the folder "Closing" when you select the attribute "Function". When a signal has no attribute value for an attribute, PROVEtech:TA inserts it in an additional folder with the name "<unassigned attribute value>".

The easiest way to define attributes for signals is using the TOI library (see directory "<PROVEtechTA_SetupDirectory>\Goodies\TOI Library"). To open the attribute selection dialog (***Figure 113***) press the button of the signal selection window toolbar. It is disabled if your signals have no attributes.

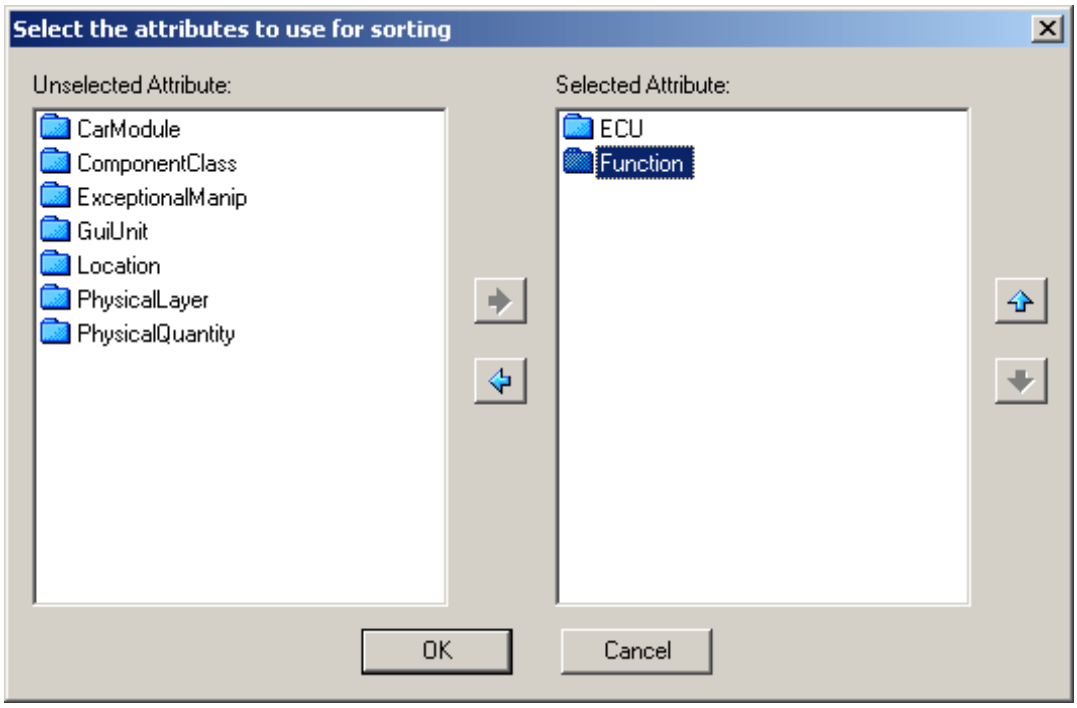


Figure 113 – Attribute selection dialog

The left list of the attribute selection dialog contains all attributes, that are not used for the sorting of the signal tree and the right list the ones, that PROVEtech:TA uses to build the lower tree levels. The order of the attributes in the right list determines which attribute is used to group the signals in which tree level. In the lower figure, the attribute "ECU" is the main sorting criteria (level 1 in the tree), after that "Function" is in level 2.

Figure 111 shows the result of this selection. In level 1 you can see the values of the attribute "ECU": "SAM" and "TSG". In the 2nd level there are the values of the attribute "Function": "Exterior Light" and "Powermanagement".

To move one or several attributes from one list of the dialog to the other one mark them and press or . Use drag and drop from one list to the other one as an alternative. Change the sorting order by moving the attributes in the right list up or down with drag and drop or select them and press or .

4.2.2 Column headers

The column header area in the signal selection window corresponds to the normal list view in Windows with a few extensions. The blue arrow designates the column and sequence (ascending or descending) by which the signals are sorted. Click on a column header to sort the signals by this column. Click on it again to change the sorting sequence. Use drag and drop to interchange columns. Change column widths by moving the separating lines.

Click with the right mouse button on a column to obtain the following context menu:

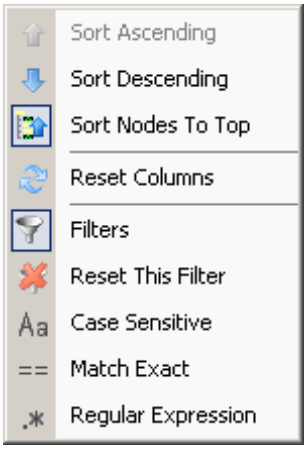


Figure 114 – Column header context menu

Sort Ascending:
Sorts by the column the mouse is clicked on in ascending order (a before z, 0 before 9).

Sort Descending:
Sorts by the column the mouse is clicked on in descending order (z before a, 9 before 0).

Sort Nodes To Top:
When activating this menu item, PROVEtech:TA sorts all nodes of the signal tree above the signals of the same level. Otherwise PROVEtech:TA does not differentiate nodes and signals when sorting. This menu item only has an effect when you sort by the column “Signal Name”.

Reset Rows:
Resets the column widths and the column sequence to the initial state.

Filters:
Activates / deactivates the filter mode described below:

The appearance of the column headers changes when the filter mode is activated:

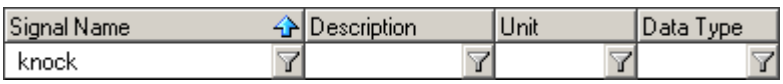


Figure 115 – Column headers with filter mode activated

Enter the desired filtering criteria in the edit boxes beneath the column headers. Only those signals are displayed which meet all criteria (AND operation between column filters). When the filter shown in ***Figure 115*** is entered, only those signals are available for selection whose name contains the text “knock”.

You can choose additional menu items when the filter mode is activated:

Reset This Filter:
Resets (clears) the filter of the column the mouse has been clicked on.

Case Sensitive:
Determines whether or not upper and lower case characters are distinguished.

Match Exact:
Activates/deactivates the checking mechanism for exact matches (every character must be identical, substrings are not allowed).

Regular Expression:
Activates/deactivates the consideration of the filter entry as a regular expression. An example of a regular expression: ^A for the signal names filter means that the filter searches for a capital A at the beginning of the signal name (see chapter ***5.18.4 Regular expressions***).

When the keyboard focus is in one of the filter text fields, the upper toolbar gets 4 additional buttons corresponding to these 4 context menu items (same icons).

4.2.3 Measurement signal selection, clipboard and info field

The info field at the lower margin of the window displays the following information:

1. The number of signals currently selected for data acquisitions.
2. The maximum number of measurement signals.
3. The number of signals currently selected.
4. The number of signals selected that are no measurement signals.

The buttons give you the following options:

 : Adds all selected signals to the measurement signals. If the maximum number of measurement signals would be exceeded as a result of this operation, the button remains deactivated and the info field is partially displayed in red (see ***8.2.4 The maximum amount of measurement signals is not sufficient***):

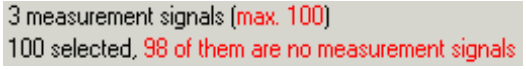


Figure 116 – Red info field

The workpage file saves the selection of measurement signals. The sampling rate selected in the configuration dialog (see chapter ***3.2.7.4 Data acquisition***) is the default for each measurement signal. It is possible to select a separate sampling rate per signal though. The column `Rate` of the signal selection window displays the sampling rate per signal and lets you change it. Doubleclick on the value or press F2 after selecting a signal to edit it using the unit Hz. However, while displaying the value PROVEtech:TA converts it to the next possible value. Data acquisitions happen on the model nodes where each model node can have an own sampling rate. That is the base rate (fastest possible rate) of available signal acquisition rates for the signals of a node. It is possible to sample signals slower by using down-sampling (acquire only in every 2nd, 3rd, 4th, etc. model step) to reduce the necessary bandwidth and memory. As an example, if a node has a base rate of 100 Hz, it is possible to select 50 Hz, 33.333 Hz, 25 Hz, 20 Hz, etc., but not 55 H. PROVEtech:TA would convert 55 Hz to 50 Hz. Please be aware that old dSPACE PHS bus systems only support one sampling rate for all signals per node. PROVEtech:TA uses the fastest one across all signals per node.

 : Removes all selected signals from the measurement signals.

 : Copies all selected signals to the Windows clipboard. These signals can then be inserted as controls on the workpage (see ***4.3.1 Context menus***) or as a signal name list in the test editor (see ***5.4 The Basic editor***).

 : Transmits all changed CCP/XCP event channel numbers to the according CCP/XCP instances (see ***4.2.5 Changing CCP/XCP event channel numbers***). This button is disabled in case you have no according signals.

 : Opens the attribute selection dialog (see ***4.2.1.1 Attribute selection***). The button is disabled in case your signals have no attributes.

 : Activates / deactivates sorting the nodes to the top (see ***4.2.2 Column headers***)

 : Activates / deactivates the filter mode (see ***4.2.2 Column headers***)

These operations are performed by selecting signals and clicking on the buttons or by dragging the selected signals to the buttons using drag and drop.

4.2.4 Selecting control types, adding controls

Several control types are available for the workpage. Some are used exclusively for display purposes, others additionally for stimulation. Select a control type with the buttons at the right (radio buttons in ***Figure 111***), select one or several signals and drag these to a free position on the workpage (drag and drop). The selected signals will then appear as controls of the previously selected type at this location.

As an alternative, you can drag signal names directly to one of the buttons to display these signals as controls at the very bottom of the currently visible page of the workpage. If you work with two Workpage windows and reach one of the buttons with the mouse cursor, the

following symbols are displayed:  . Release the mouse button as soon as the mouse cursor is on top of one of the two monitor symbols to add the controls to the corresponding page. The left monitor symbol represents the 1st Workpage in the main window, the right one – the second Workpage window.

Another alternative is dragging signals to the workpage with the right mouse button. After that PROVEtech:TA displays a context menu where you can choose the control type to use for insertion.



There is a maximum of controls that each page of the workpage can display (editable via the settings dialog, see ***3.2.7.3 Workpage***). If you select too many signals, they cannot be added to the workpage as controls.



You can use drag and drop to insert signals as a signal name list in the test editor (see ***5.4 The Basic editor***).

A brief description of the individual buttons follows:


Automatic type selection; when using this button PROVEtech:TA automatically selects the control type. It uses the following rules for that:
In case you select a minimum of 2 signals of the same array, PROVEtech:TA creates either a grid control (***4.4.10 The grid control***) or a custom control for a text signal array (***4.4.12.11 Text Signal Array***) if the model defines a code page for the signal array.
In case it is a signal of the data type integer (signed or unsigned) and a minimum value of 0 and a maximum values of 1, PROVEtech:TA creates a switch / LED control (***4.4.5 The switch / LED control***).
For read-only Signals PROVEtech:TA creates a number control (***4.4.6 The number control***).
In case the model defines more than 3 text table entries for the signal, PROVEtech:TA creates a custom control for a combo box (***4.4.12.8 Combo Box***).
For all other cases it creates a slider / bar control (***4.4.4 The slider / bar display control***).
In case you have specified a Basic script in the settings dialog for the workpage, you can manipulate the control creation with this Basic script (see chapter ***3.2.7.3 Workpage***).
Attention: In case you select more than 800 signals, PROVEtech:TA does not examine them and deactivates all these mechanisms because of self-protection.


Strip chart control; displays measurement signals during a data acquisition as a function of time. See ***4.4.2 The strip chart and multi-strip chart controls***. When creating controls of this type, PROVEtech:TA automatically adds the currently selected signals to the

measurement signals (up to the maximal possible amount, see chapter ***4.2.3 Measurement signal selection, clipboard and info field***).

 Multi-strip chart control; displays several measurement signals during a data acquisition as a function of time. A multi-strip chart control can hold up to 16 signals. If you select this control type, the selected signals are grouped into blocks of 16 each when added to the workpage. See ***4.4.2 The strip chart and multi-strip chart controls***. When creating controls of this type, PROVEtech:TA automatically adds the currently selected signals to the measurement signals (up to the maximal possible amount, see chapter ***4.2.3 Measurement signal selection, clipboard and info field***).

 XY strip chart control; displays two measurement signals as a chart during a measurement. One represents the X axis, the other one the Y axis. See ***4.4.3 The XY strip chart control***. When creating a controls of this type, PROVEtech:TA automatically adds the currently selected signals to the measurement signals (up to the maximal possible amount, see chapter ***4.2.3 Measurement signal selection, clipboard and info field***).

 Slider or bar display control; serves as a display and control element. Only changeable signals can be stimulated with this control. See ***4.4.4 The slider / bar display control***.

 Switch or LED control; serves as a display and control element. Only changeable signals can be stimulated with this control. See ***4.4.5 The switch / LED control***.

 Number control; represents signals as decimal, hexadecimal or binary numbers. See ***4.4.6 The number control***.

 Bitmap control; this control allows you to assign value ranges to images or texts. This control always shows the text or image of the value range the signal is currently in. See ***4.4.7 The bitmap control***.

 Bitswitch control; serves as a display and control element and represents signal values in binary form. See ***4.4.8 The bitswitch control***.

 Gauge; serves as a display and control element and represents signal values as a dial-type instrument. See ***4.4.9 The gauge***.

 Grid control; serves as a display and control element and displays vectors and matrices in a table. See ***4.4.10 The grid control***.

 Surface graph control: serves as a display element to display matrices in a 3-dimensional surface. See ***4.4.11 The surface graph control***.

 Custom control; you can configure the control with different embedded controls in any way. See ***4.4.12 The custom control***.

Use the last button to manage the special controls like the cruise-control or different shifting gates. Click on the arrow beside the button to open a list with all special controls:

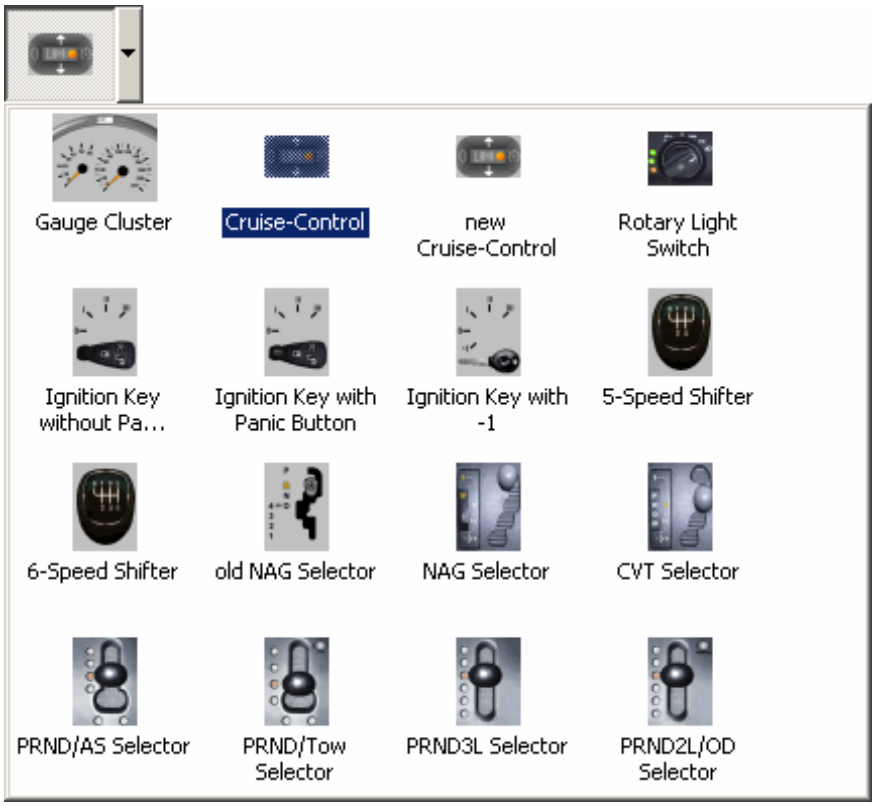


Figure 117 – Special control selection

Select a special control type and drag a signal to the workpage to create the control. This is handled equal to the other control types. When a control uses several signals use its configuration dialog to select the other signals (see chapter ***4.4.13 The special controls***).

4.2.5 Changing CCP/XCP event channel numbers

In case you loaded a CCP/XCP configuration using the so-called DAQ mode (CCP/XCP slave transmits signal values cyclically via events), you can change the used event channel numbers of the according signals at runtime. The event channel number typically determines the cycle time in which the slave transmits the signal values.

In case you have such CCP/XCP signals where the event channel number matters, the signal selection window displays the additional column “Event No.” displaying the current event channel number per signal. For other signals, the column stays empty.

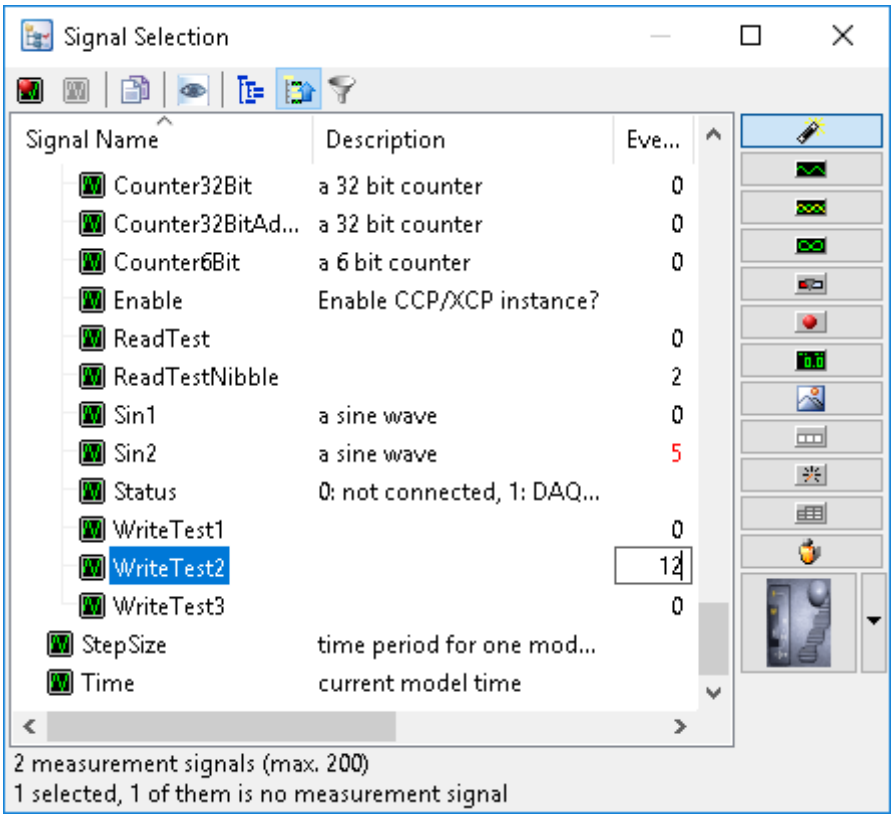


Figure 118 – Changing an event channel number

Double-click on an event channel number to edit it (see figure above). For CCP the theoretical valid range is from 0 to 255 and for XCP it is from 0 to 65535. However, typically the upper limit accepted by the slave is much smaller. All in all, the CCP/XCP master has no clue about what a specific event channel number means. Only the slave knows that. In some (not all) cases the according .a2l file has information about that. Therefore, if you want to know what the event channel numbers mean exactly, you can either have a look in the .a2l file or ask the slave developer (or who configured it).

After you change a value and accept it via Enter or clicking on a different position, the new number is displayed red. PROVEtech:TA does not perform your changes immediately, but waits until you click on the toolbar. This transmits the changes to the CCP/XCP master instances on the model computer(s). In case there is currently a connection to the slave, the master disconnects it briefly, re-connects, and transmits the DAQ lists with the new event channel numbers.

Please keep in mind that slave resources are limited and changing the event channel numbers might mean that it cannot transmit the signals anymore due to missing resources.

There is also the test language command **System.ModifyXcpConfig** to change the event channel numbers at runtime.

If you want to set your currently entered event channel numbers directly while loading the CCP/XCP configuration, you can save that after changing the numbers in the signal selection

window via the main menu item *Application|CCP/XCP Configuration|Save (As)*. See also chapter ***3.2.5 CCP/XCP configuration***.

4.3 Managing controls

This chapter describes how to handle and arrange the workpage controls.

4.3.1 Context menus

Controls are managed almost exclusively with the mouse. Click on a control with the right mouse button to display the following context menu:

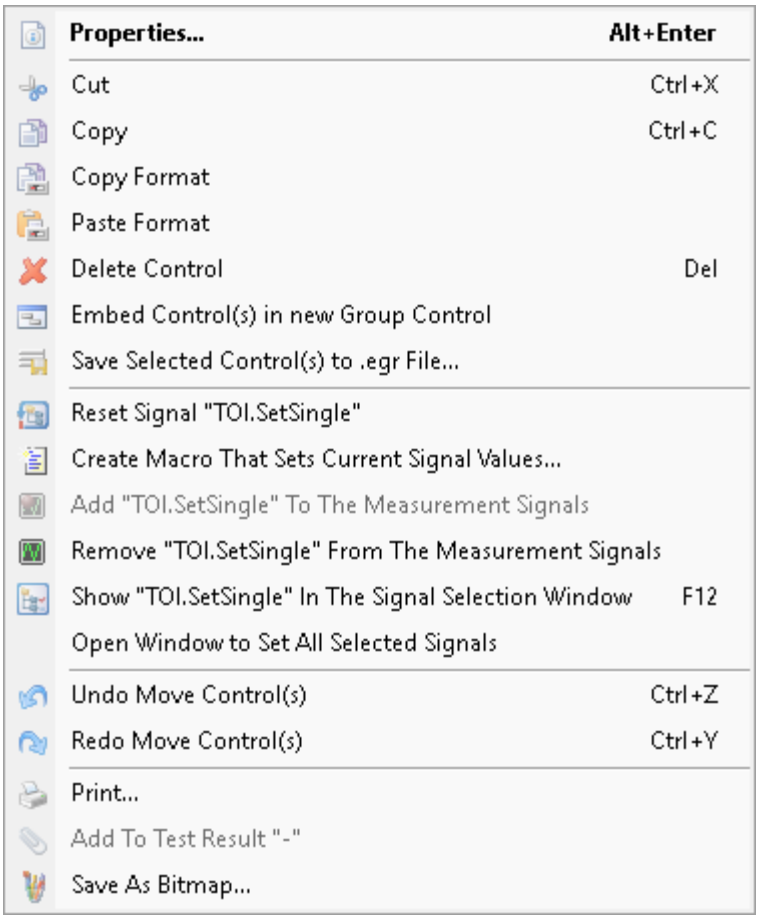


Figure 119 – Context menu of a control

Properties…:
Displays the properties of the control. Select this menu item to make changes to the way the control is displayed. See chapter ***4.4.1 General properties.***

Cut:
Removes the marked control from the workpage and copies it to the clipboard. Further on in this chapter you will find out how to paste controls from the clipboard.

Copy:
Copies the marked control to the clipboard.

Copy Format:
Copies the format (colors, type, size and all the other settings but not the assigned signal) of the marked control to the clipboard.

Paste Format:
Transfers all settings previously copied with *Copy Format* from the clipboard to the marked control. After that the control looks equal to the one you have copied, but the original signal keeps the same.

Delete Control:
Deletes the marked control.

Embed Control(s) in new Group Control:
Creates a new group control around the selected control(s) and embeds them within the group control (see chapter ***4.4.16 The group control***).

Save Selected Control(s) to .egr File:
This menu item is used to save the marked control or controls as an .egr file via the file selection dialog (see chapter ***3.1 The file selection dialog***), so that you can load it into another workpage later on. This option is primarily of interest when you work with several controls at the same time. Refer to chapter ***4.3.3 Selection of controls*** for further information. See below in this chapter for information on how to load an .egr file.

Reset Signal "x":
This menu item is only displayed for signals with a reset value (see chapter ***3.2.3 Reset signals***). It sets the signal to its reset value.

Create Macro That Sets Current Signal Values…:
Creates a macro (see ***3.1 The file selection dialog***) that resets the signal values of the selected controls (see ***4.3.3 Selection of controls***) to the current values. Use the file selection dialog to choose the destination of the macro file.

Add "x" To The Measurement Signals:
The signal must be no measurement signal that this menu item is active. Select the menu item to add the signal to the measurement signals (see chapter ***4.2 The Signal Selection window*** and ***3.4 The Data Acquisition menu***).

Remove "x" From The Measurement Signals:
The signal must be a measurement signal that this menu item is active. Select the menu item to remove the signal from the measurement signals (see chapter ***4.2 The Signal Selection window*** and ***3.4 The Data Acquisition menu***).

Show "x" In The Signal Selection Window:
Displays the signal selection window (see chapter ***4.2 The Signal Selection window***) and selects the signal.

Open Window to Set All Selected Signals:
Opens an additional window with a list of all writable signals which are contained in the selected controls. There you can enter a common signal value and set all signals to that by pressing the Enter key. This only affects the signals with activated check box. While writing PROVEtech:TA casts the entered value to the type of each signal by using the C casting rules. Setting the signals does not happen at the same time but one by one.

Undo…:
Undoes the last action.

Redo…:
Redoes the last undone action.

Print…:
Prints the selected control after displaying the dialog described in chapter ***4.3.7 Printing or adding controls to the test result***.

Add To Test Result…:
When you select a test result in the Test Manager, you can add a screenshot of the control as an attachment to it (see chapter ***5.6.9 Attachments***). When you have selected several controls (see chapter ***4.3.3 Selection of controls***) or when you are in the offline mode you can configure the result with a configuration dialog (see chapter ***4.3.7 Printing or adding controls to the test result***).

Save As Bitmap…:
Saves a screenshot of the control as a bitmap file. When you have selected several controls (see chapter ***4.3.3 Selection of controls***) or when you are in the offline mode you can configure the result with a configuration dialog (see chapter ***4.3.7 Printing or adding controls to the test result***).

When you click on a (multi) strip chart control, PROVEtech:TA behaves somewhat differently. The menu shown in ***Figure 120*** with additional menu items is displayed.

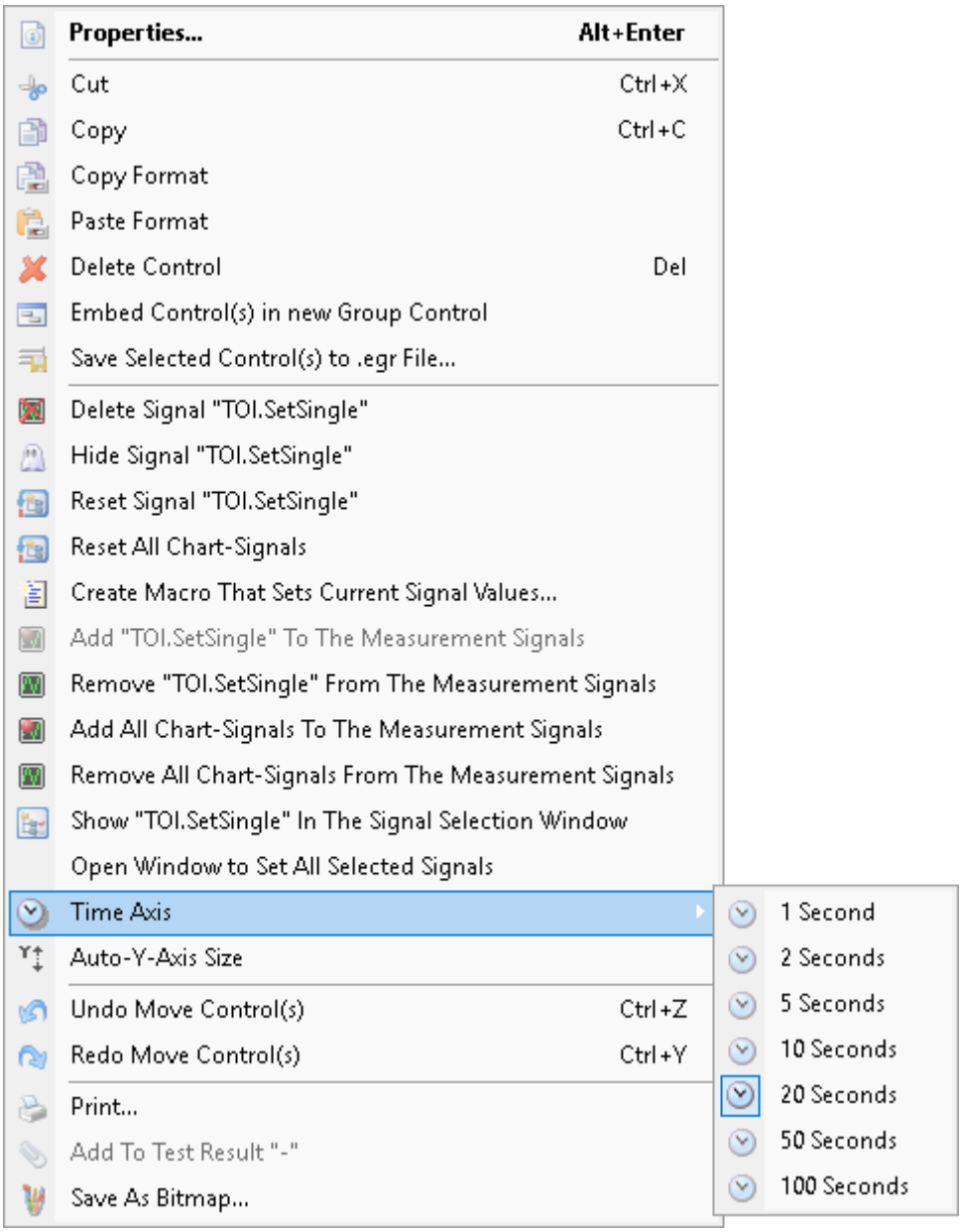


Figure 120 – Context menu of a strip chart control

Delete Signal "x":
Deletes signal "x" from the multi strip chart control.

Hide Signal "x":
Hides the signal "x" so that its plot is not visible anymore or displays it again. In case of a hidden plot the legend displays a ghost icon for the signal ( ). You can also directly click on the legend to hide or show a plot.

Reset All Chart-Signals:
Resets all signals of the multi strip chart control to their reset values (see chapter ***3.2.3 Reset signals***).

Add All Chart-Signals To The Measurement Signals:
At Least one of the multi strip chart control signals must be no measurement signal that this menu item is active. Select the menu item to add all signals of the multi strip chart control to the measurement signals (see chapter ***4.2 The Signal Selection window*** and ***3.4 The Data Acquisition menu***).

Remove All Chart-Signals From The Measurement Signals:
At Least one of the multi strip chart control signals must be a measurement signal that this menu item is active. Select the menu item to remove all signals of the multi strip chart control from the measurement signals (see chapter ***4.2 The Signal Selection window*** and ***3.4 The Data Acquisition menu***).

Time Axis:
This menu item opens a sub menu to adjust the length of the time axis for all strip chart controls using the common time axis. This way you can only select fixed values like “20 Seconds”. For a more exact setting, please use the property dialog (see chapter ***4.4.2.1 The property dialog of the strip chart control***).

Auto-Y-Axis Size:
Choose this menu item while or after running a data acquisition to adjust the range of the Y axis automatically to the measured values.

For XY strip chart controls, the according menu item is *Auto-Size Axes* and it adjusts both axes.

In case the strip chart control has several Y axes, the menu item is a popup menu and you can select one of the axes or the menu item *All but explicit excluded axes*. That one affects all axes not excluded via the property dialog (see ***4.4.2.1 The property dialog of the strip chart control***).

Another menu appears when you click with the right mouse button on a free space of the workpage:

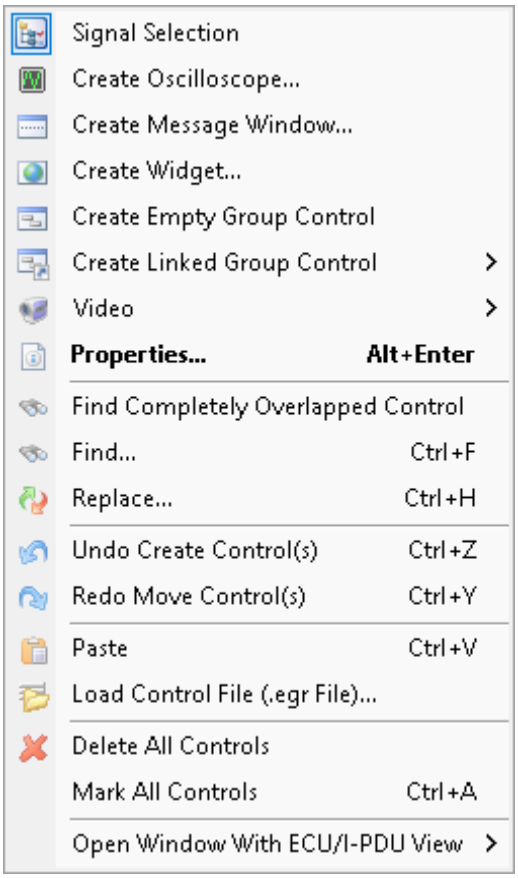


Figure 121 – Context menu of workpage

Signal Selection:

Displays or hides the Signal Selection window (corresponds to the button  on the toolbar). See chapter ***4.2 The Signal Selection window***.

Create Oscilloscope…:
Creates a new oscilloscope display at the clicked position. See chapter ***4.4.15 The oscilloscope display***. This menu item is only available when your model has a generic message channel with oscilloscope data.

Create Message Window…:
Creates a new message window at the clicked position. See chapter ***4.4.14 The message window***. This menu item is only available when your model platform supports the handling of messages in combination with PROVEtech:TA.

Create Widget…:
Creates a new widget at the clicked position. See chapter ***4.4.17 The widget***.

Create Empty Group Control:
Creates a new group control at the clicked position. See chapter ***4.4.16 The group control***.

Create Linked Group Control:
After selecting an existing page of a workpage file, a cockpit file or an .egr file this menu item

displays its content within a new group control at the clicked position. This is only a link to the original file, so changes within the original file get displayed the next time you load the workpage. Changes within the group control have no impact on the original file and get lost when closing the workpage. See chapter ***4.4.16 The group control***.

Video:
This menu item only appears if you have configured a video source via the settings dialog (see chapter ***3.2.7.3 Workpage***). Within *Video* PROVEtech:TA shows a sub menu item *Create Control for "x"* for all available video sources. Select one of them to create an according video window at the clicked position (see chapter ***4.4.18 The video control***). If a video source stops working at runtime, the sub menu item *Reinitialize "x"* might restore it. That menu item is also available when right-clicking on a video control.

Properties…:
Opens the dialog described in chapter ***4.3.2 The property dialog for pages and group controls***.

Find Completely Overlapped Control:
Marks and shows a control that was completely overlapped so far in case such a control exists.

Find…:
Opens the find and replace dialog on the page “Find” (see chapter ***4.3.8 Find and replace***).

Replace…:
Opens the find and replace dialog on the page “Replace” (see chapter ***4.3.8 Find and replace***).

Undo…:
Undoes the last action.

Redo…:
Redoes the last undone action.

Paste:
Insert the controls currently contained in the clipboard at the location you have clicked on. Controls are put in the clipboard by copying or cutting in the context menu described above (see ***Figure 119***) or by copying in the Signal Selection window (see ***4.2.3 Measurement signal selection, clipboard and info field***).

Load Control File (.egr File)…:
The file selection dialog (see chapter ***3.1 The file selection dialog***) is used to load an .egr file and insert the contained controls the location you have clicked on.

Delete All Controls:
Deletes all controls on the page of the workpage you have clicked on.

Mark All Controls:
Marks all the controls located on the page. Refer to the next chapter ***4.3.3 Selection of controls*** for further information.

Open Window With ECU/I-PDU View:
Opens a separate window to display which ECU sends and receives which I-PDUs. PROVEtech:TA gets this information from a selectable bus description file. Within *Open Window With ECU/I-PDU View* PROVEtech:TA displays a list of all currently loaded bus description files (e.g. AUTOSAR XML) assigned to the available message channels. Additionally you can choose *Select File…* to directly select a file. The I-PDU window displays detected SOME/IP PDUs red.

4.3.2 The property dialog for pages and group controls

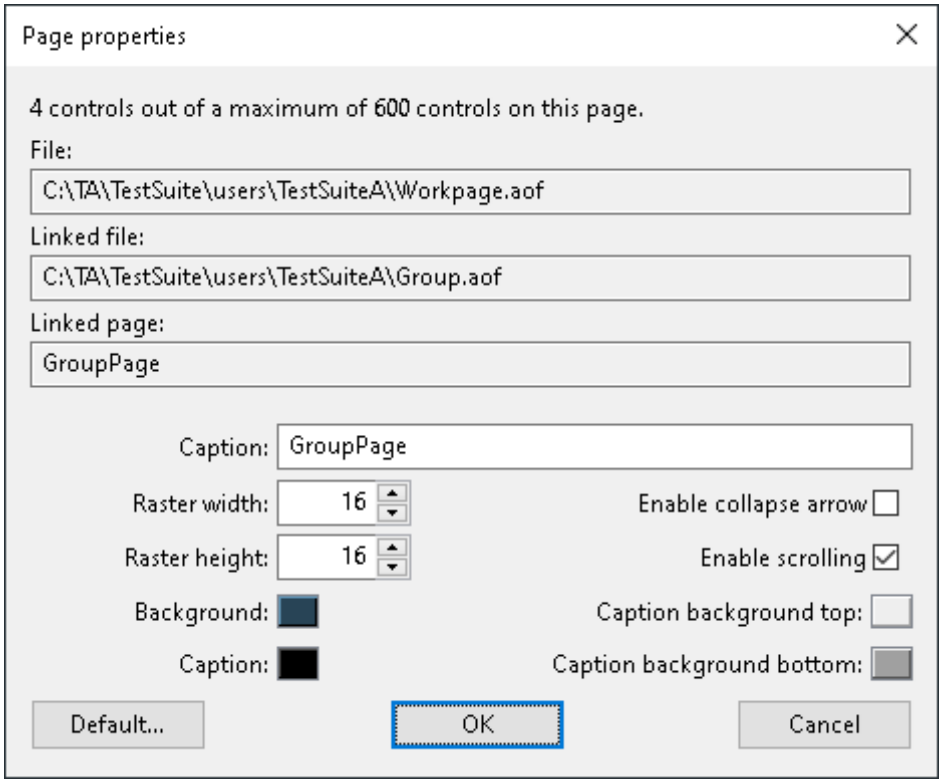


Figure 122 – Group control properties

The top of the dialog displays the following information:

- The current and the maximum number of controls on the workpage page or the group control (see chapter ***4.4.16 The group control***)
- The path of the workpage file
- For linked pages:

- The path to the file used as the source
- The name of the linked page in the source file

The edit box `Caption` specifies the caption of the page or group control. Use the text boxes `Raster width` and `Raster height` to specify the raster in pixels for the workpage control alignment. PROVEtech:TA uses this raster when doing drag and drop / copy and paste operations and when resizing controls. So, changing the values has no immediate effect. Instead, it is used for the next drag and drop / copy and paste / resizing operation. We recommend using raster vales which are a multiple of 8; see chapter ***4.1.1 View scaling***.

You can temporarily disable the raster on a page by pressing the shift key while doing an operation.

Use the `Background` button to select the background color of the page / group control. The other color buttons only exist for group controls. `Caption` is the font color for the caption. `Caption background top` and `Caption background bottom` specify the gradient of the title bar from top to bottom.

For pages the lower part looks like this:



Figure 123 – Page properties

The `Tab` button exists for selecting a separate color for the page's tab. It is only active in combination with the activated check box `Tab has own` color. In case the check box is deactivated, the tab color corresponds to the page background color when selected. When unselected the tab color is the standard gray.

Activating `Enable collapse arrow` is only possible for group controls. After the activation the group control has an arrow button in the upper left corner to expand and collapse the whole group control by clicking it with the mouse.

The check box `Enable scrolling` only exists for group controls. In case it is activated, the control behaves like a page of the workpage: When the contained controls do not fit into the view, scrollbars appear to select the visible area. In some cases, this might be annoying, e.g., when using drag and drop over the group control and it starts scrolling. Without the scroll bars it can happen that embedded covered controls are not reachable anymore. Therefore, the menu item *Resize Group Control to fit to Content* exists.

Click on `Default…` (***Figure 122***) to either set the current colors and the raster as default for new pages / group controls or to reset the current values to the default values.

4.3.3 Selection of controls

Before controls can be arranged, changed or deleted, it is necessary to select one or several controls. To mark a control, move the mouse cursor over its upper left corner until the mouse cursor changes:  . Then click with the left mouse button, and the appearance of the conteol frame changes as shown in the following example for a working control:

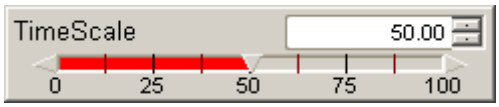


Figure 124 – Working control before selection

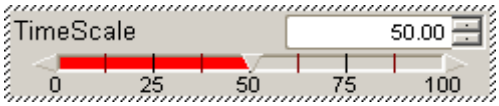


Figure 125 – Working control after selection

Select additional controls by pressing and holding the Ctrl key and clicking on the controls in the same manner.

To mark controls, which are contiguous, click on a free area of the workpage and move the mouse to draw a frame around the controls:

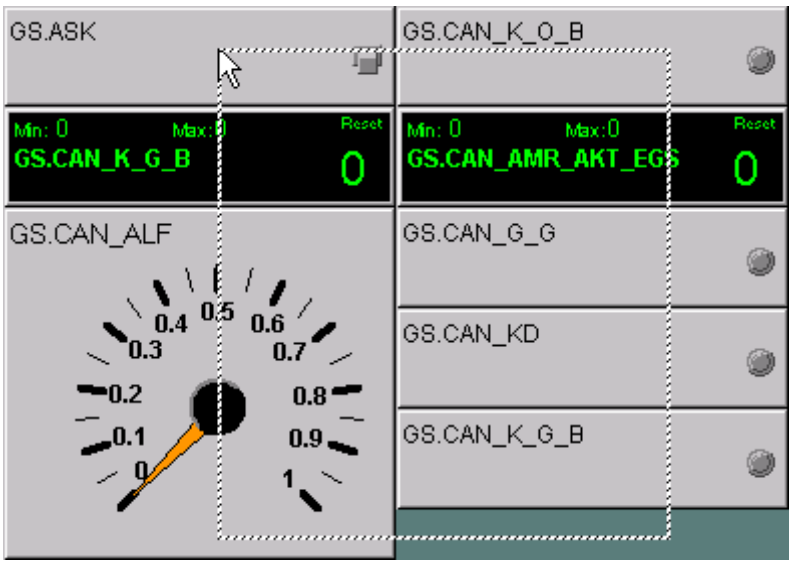


Figure 126 – Selection of several controls

For example, the operations described in chapter ***4.3.1 Context menus*** can be applied to all marked controls with these control groups.

4.3.4 Alignment of several controls

When you have marked several controls (see chapter ***4.3.3 Selection of controls***), these can be aligned at a border of the control group. To do this, click with the right mouse button on the group and select the *Align* submenu:

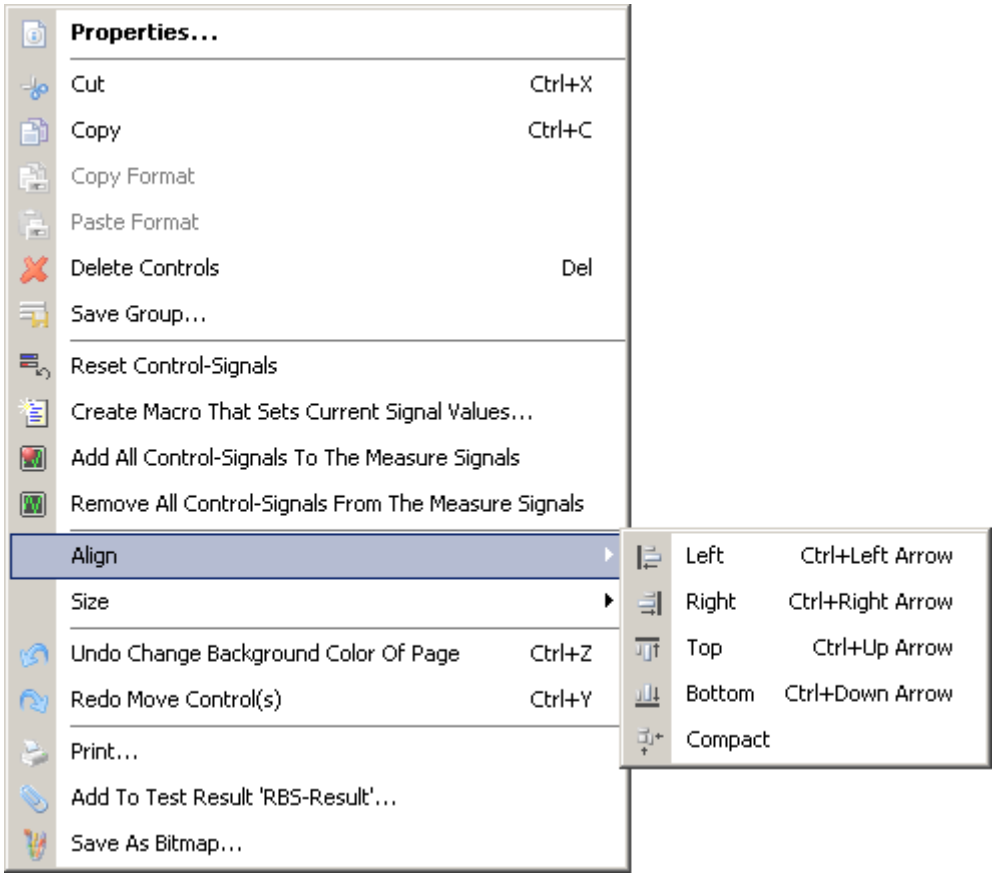


Figure 127 – The Align submenu

Then click on one of the menu items *Left*, *Right*, *Top* or *Bottom* to select the desired common border. Click on *Compact* to align the controls to the upper left corner of the selection. PROVEtech:TA tries to put the controls as close together as possible but keeps the controls in their original order.

4.3.5 Moving and duplicating controls

One or several controls are moved using the drag and drop technique. To move a control, move the mouse over the upper left corner until the mouse cursor changes:  . Then start the drag and drop procedure and drop the control at the new position. Move to the margins of the workpage during the drag and drop procedure to scroll in the corresponding direction. If you wish to move several controls at the same time, select these according to the description in

chapter ***4.3.3 Selection of controls***. As ***Figure 128*** shows, a frame is displayed during the movement which represents the size of the control(s) moved.

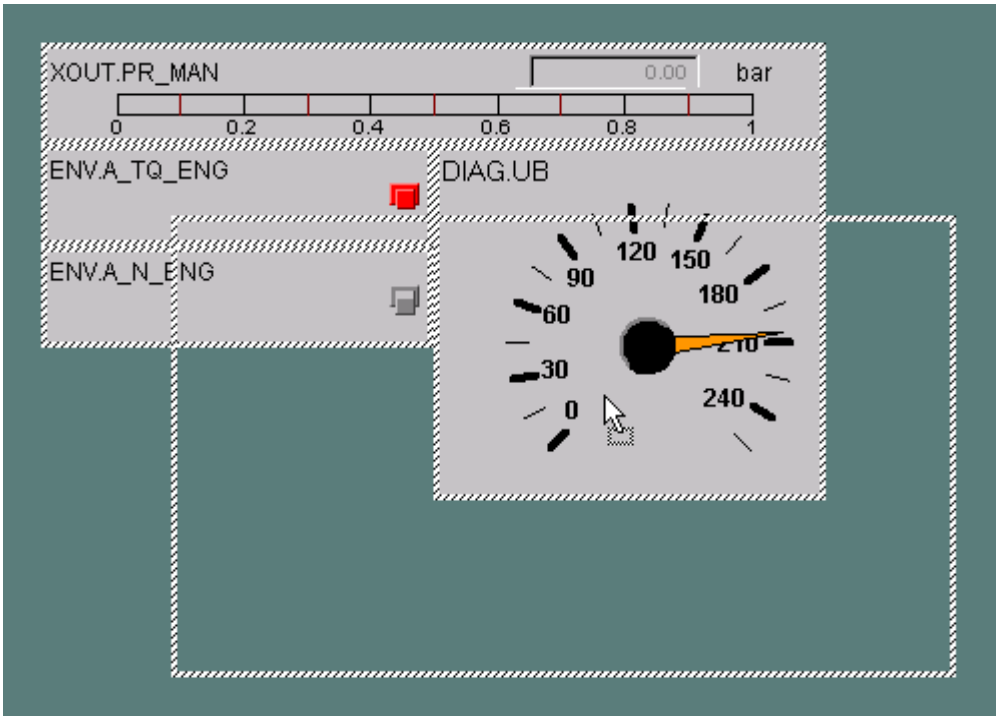


Figure 128 – Moving controls

Press the Ctrl key as you release the mouse button to duplicate the controls. The mouse cursor changes as follows:



Figure 129 – Mouse cursor during duplication

The procedure is the same when you work with two Workpage windows. Drag the controls with the mouse from one workpage to the other.

If you wish to insert the controls in a different page of the workpage, move to the tab of the desired page during the drag and drop operation (see ***Figure 130***). The pages are changed automatically, and you can insert the controls at the desired position.

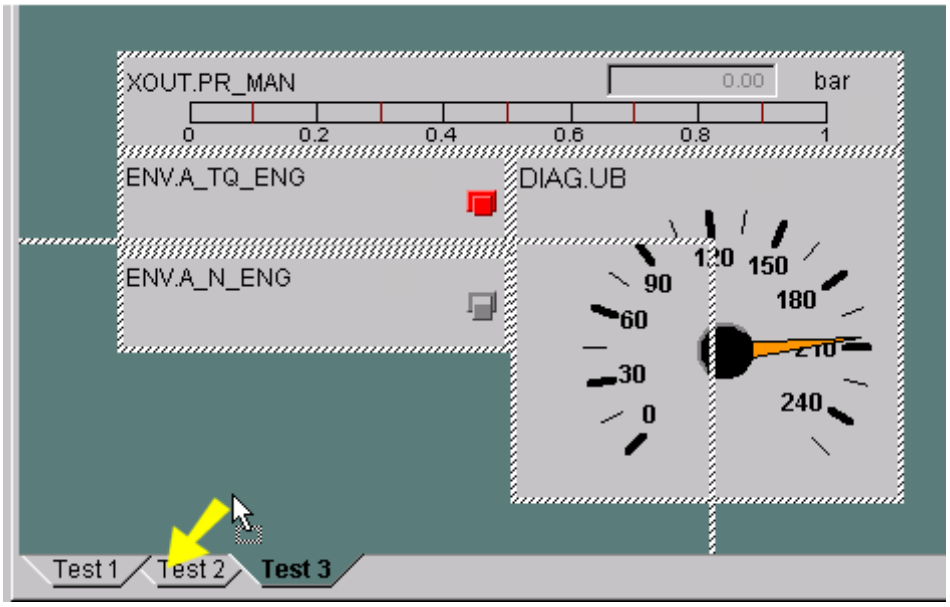


Figure 130 – Moving controls to a different page

If you wish to change the control type of one or several signals during duplication, drag the control to one of the right buttons of the Signal Selection window (see ***4.2.4 Selecting control types, adding controls***). The new controls appear at the bottom of the workpage.

For the alignment of the controls PROVEtech:TA normally uses the raster of the page (see ***4.3.2 The property dialog for pages and group controls***). You can disable it temporarily by holding the Shift key while moving the controls.

To move the selected controls pixel by pixel, hold the Shift key and press one of the arrow keys to select the direction.

4.3.6 Sizing controls

To change the size of a control, move the mouse to the bottom margin, right margin or lower right corner and drag the control to obtain the desired size while holding down the mouse button. This is shown in the following figures:

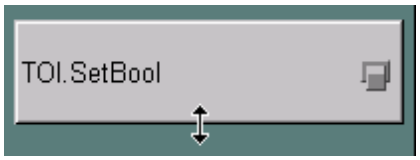


Figure 131 – Changing the height of a control



Figure 132 – Changing the width of a control

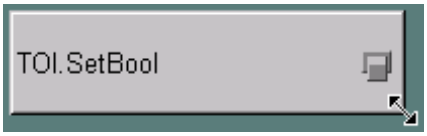


Figure 133 – Changing the height and width of a control

As an alternative to this method, the height and width of controls can be changed using the property dialogs. Refer to the chapter ***4.4.1 General properties*** for further information.

If you wish to change the size of several controls at a time, mark these first (see chapter ***4.3.3 Selection of controls***) and proceed as for one control.

For the alignment of the controls PROVEtech:TA normally uses the raster of the page (see ***4.3.2 The property dialog for pages and group controls***). You can disable it temporarily by holding the Shift key while resizing the controls.

To size several controls to the same size, click with the right mouse button on the selected controls and select *Size*:

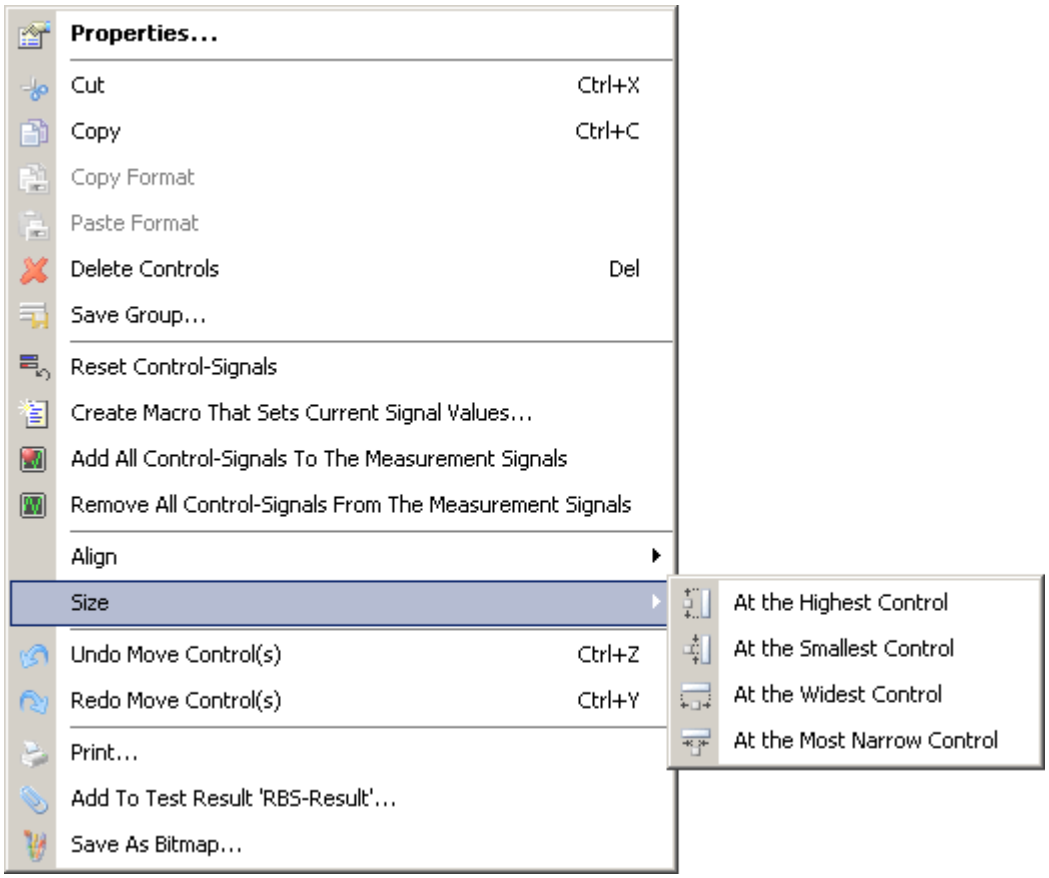


Figure 134 – The Size submenu

Then select one of the menu items *At the Highest Control*, *At the Smallest Control*, *At the Widest Control* or *At the Most Narrow Control* to adapt the width or height of all controls to that of one control.

4.3.7 Printing or adding controls to the test result

Use the context menu (see chapter ***4.3.1 Context menus***) to print controls, add them to test results or save them as bitmap files. PROVEtech:TA displays the dialog in ***Figure 135*** to configure the result:

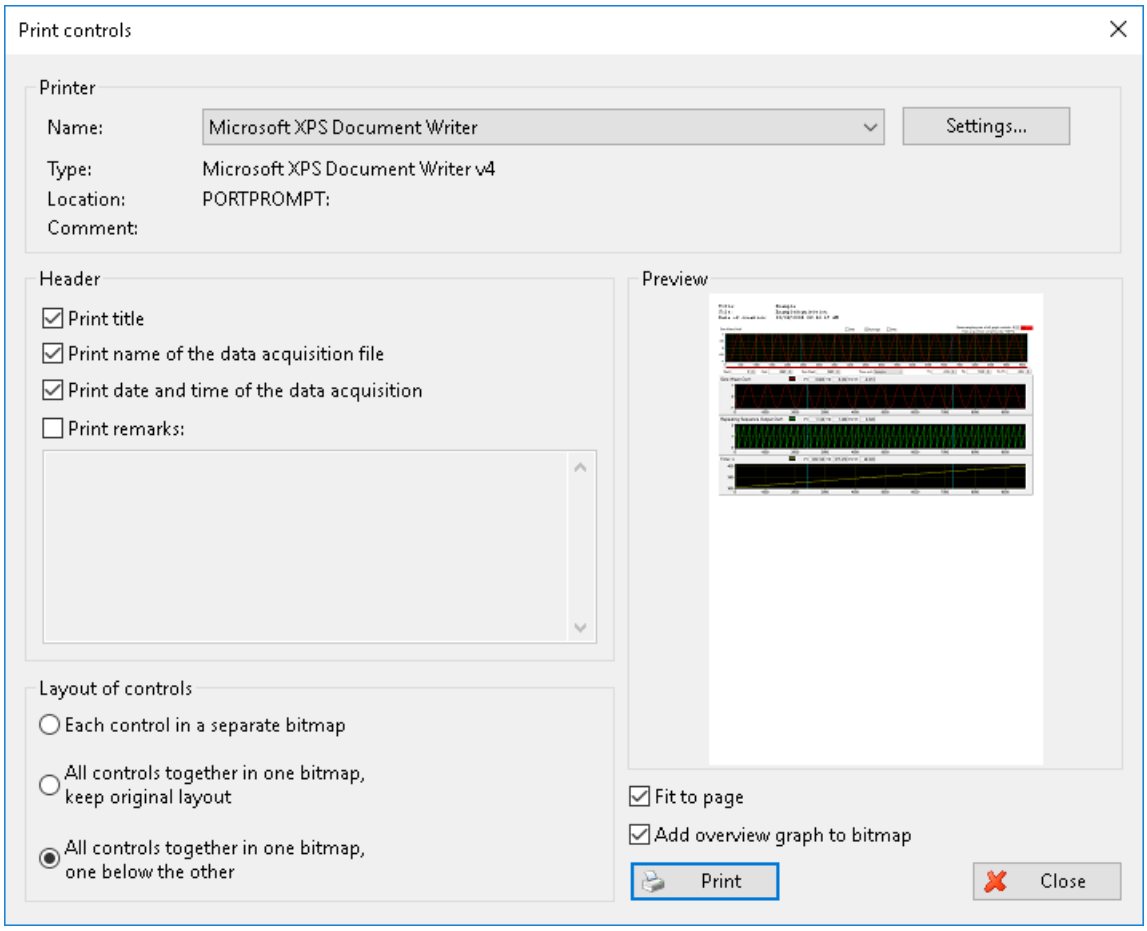


Figure 135 – Configuration dialog

The settings `Printer`, `Header` and `Fit To Page` are only enabled when printing controls. You can use most header check boxes and the `Add Overview Graph To Bitmap` check box only when you select a measurement in the offline mode (see chapter ***4.5 The offline mode***). Otherwise you can only add a remark for printing.

Use the three radio buttons in `Layout Of Controls` when you have selected several controls:

The option `Each Control In A Separate Bitmap` creates one page or one bitmap file for each control.

`All Controls Together In One Bitmap, Keep Original Layout` results in one page or one bitmap file with all selected controls in the original workpage layout.

`All Controls Together In One Bitmap, One Below The Other` also results in a single page or a single bitmap file with all selected controls. PROVEtech:TA left-aligns all controls and puts one below the other.

The `Preview` frame displays the result of your settings.

4.3.8 Find and replace

To open the find and replace dialog (***Figure 136***) use the context menu of a page on the workpage (see chapter ***4.3.1 Context menus***) or use the hotkeys Ctrl+F (Find) and Ctrl+H (replace) when the keyboard focus is on the workpage.

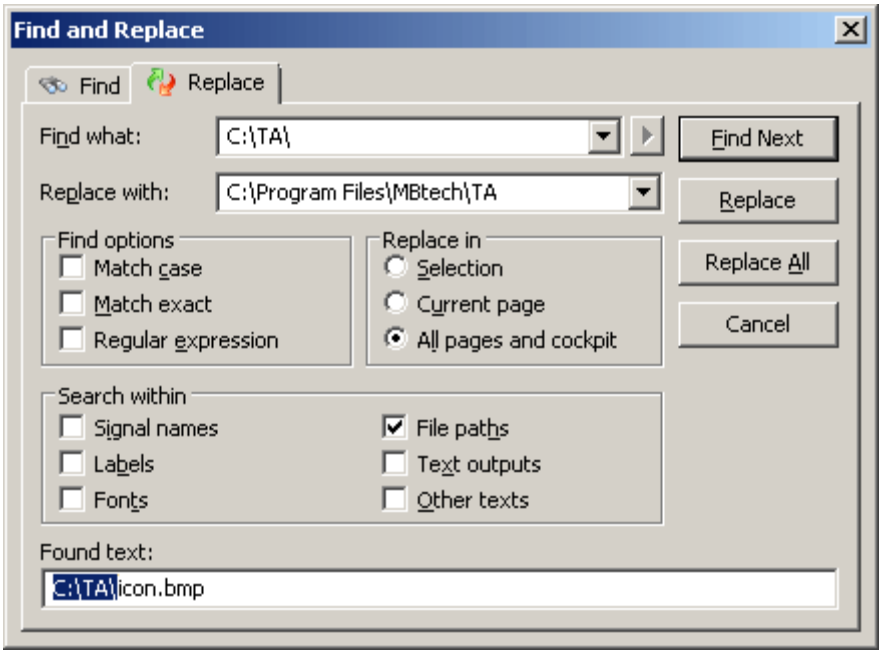


Figure 136 – Find and replace dialog

The page “Find” differs from “Replace” only in lesser options. This is why this chapter only describes the page “Replace”.

This dialog only searches and replaces texts within workpage controls. As an example you cannot search for colors.

Enter the text to search for in the field `Find what` and the text to replace it with in the field `Replace with`.

Check `Match case` to search case sensitive. Activate the check box `Match exact` to compare the whole strings and use no sub strings. Activating the `Regular expression` check box lets you search for regular expressions (see chapter ***5.18.4 Regular expressions***). This also activates the right arrow button beside the `Find what` combo box. Pressing the button opens a menu that helps you assembling regular expressions. The last menu item opens the homepage of the Boost Library containing extended information about the syntax.

The radio buttons in `Replace in` determine, which workpage controls the dialog browses. Choose `Selection` to browse only the currently selected workpage controls (see chapter ***4.3.3 Selection of controls***). `Current page` only browses the currently open page or the cockpit, in case you have opened the dialog via the cockpit. Select `All pages and cockpit` to browse everything.

The group Search within determines the text types within the controls for searching. Use Signal names to replace signals that became invalid due to a model change. Labels are user defined control captions. Fonts refers to names of used fonts. Select File paths in case you have moved used bitmaps on your hard disk to a different location. Text outputs refers to signal value output in textual form. Everything else is covered by the check box Other texts.

After this click on Find Next to begin or continue searching. Click on Replace to replace the currently found text (or to begin searching in case this did not happen before). Use the button Replace All to replace all occurrences of the text in one step. Click on Cancel to close the dialog.

The position of the workpage controls determines the search order: from top left to bottom right.

In case the dialog finds a text occurrence, it displays the whole text in the field Found text and marks the occurrence. Additionally the border of the according workpage control starts flashing in red color. If the dialog overlaps the found control, it displays a hint after Found text. Instead of clicking on Replace, you can also directly edit the displayed text. After two seconds your changes get assigned. Press Enter to immediately assign the changes. In case you do not alter the text and press Enter, it has the same effect as clicking the Replace button.

4.3.9 The hotkeys

In case the keyboard focus is on the cockpit or the workpage, you can use the following hotkeys. On the lower tabs of the workpage the meaning of some hotkeys differ.

Hotkey	Function
F2	Rename the selected page
F12	Open the signal selection window and select the signal of the selected control
Del	Delete the current page or the selected controls
Alt+Enter	Open the property dialog
Ins	Insert a new page
Ctrl+Down	Align the selected controls at the bottom
Shift+Down	Move the selected controls down pixel by pixel
Ctrl+Left	Align the selected controls at the left
Shift+Left	Move the selected controls left pixel by pixel

Ctrl+Right	Align the selected controls at the right
Shift+Right	Move the selected controls right pixel by pixel
Ctrl+Up	Align the selected controls at the top
Shift+Up	Move the selected controls up pixel by pixel
Ctrl+A	Select all controls
Ctrl+C / Shift+Del	Copy the selected controls to the clipboard
Ctrl+F	Open the find dialog
Ctrl+H / Ctrl+R	Open the replace dialog
Ctrl+L	Load a workpage
Ctrl+Shift+L	Load a cockpit
Ctrl+N	Create a new workpage
Ctrl+Shift+N	Create a new cockpit
Ctrl+S	Save the current workpage using its current path
Ctrl+Shift+S	Save the current cockpit using its current path
Ctrl+V / Shift+Ins	Paste the clipboard
Ctrl+X	Cut the selected controls
Ctrl+Y	Redo
Ctrl+Z / Alt+Backspace	Undo
Ctrl+Plus	Increase the view scaling by one step
Ctrl+Minus	Decrease the view scaling by one step
Shift+ Mouse wheel	Scroll horizontally

4.3.10 Managing bitmaps

Several controls support the display of bitmaps. As the bitmaps are typically external files you should think about how to manage these bitmaps. PROVEtech:TA supports three different variants to reference bitmaps:

a) Paths referencing the file system, e.g. C:\TA\MyBitmap.jpg.

b) Paths referencing test documents in the Test Manager tree, e.g. *MyProject:\TA\MyTest*doc*MyBitmap.jpg. This variant is only available in combination with a Test Manager Professional license. See also chapter ***5.3.9.4 The tab "Documents"***.

c) Names of bitmaps embedded in the workpage file, e.g. *emb*MyBitmap.jpg.

For a) and b) relative paths are relative to the workpage file containing the reference. As an example, if the workpage file is saved as "C:\TA\MyWorkpage.aof" and a bitmap reference is "test\MyBitmap.jpg", PROVEtech:TA would try to load it via "C:\TA\test\MyBitmap.jpg". For c) there are no relative paths or hierarchies.

All three variants have advantages and disadvantages concerning consistency, passing workpage files to others, multi-user environment and storage usage.

In case a control supports bitmaps, its property dialog has the possibility to open the file selection dialog (see chapter ***3.1 The file selection dialog***). The three tabs `File System`, `Test Manager` and `Embedded` distinguish the upper three variants:

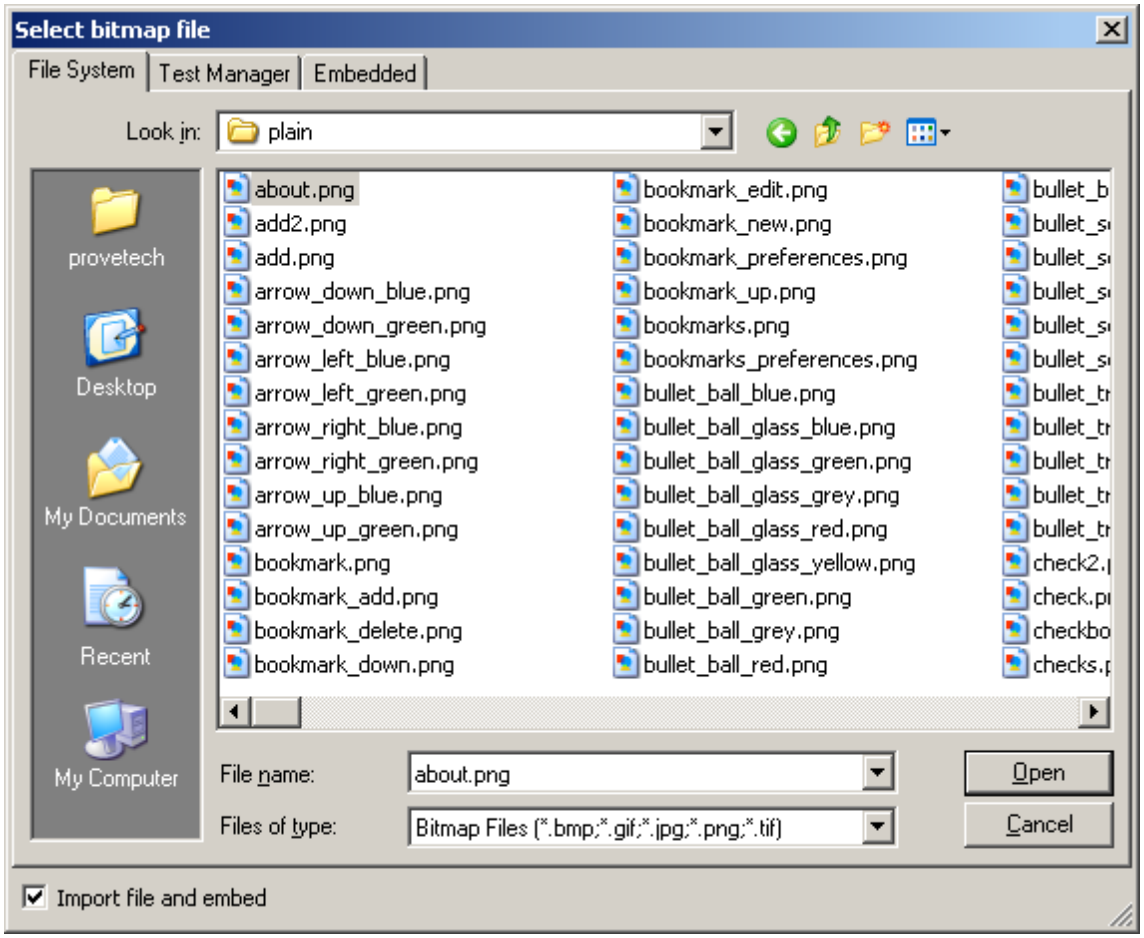


Figure 137 – File selection dialog for bitmaps

To embed a bitmap file into the workpage file, select it via the tabs `File System` or `Test Manager` and activate the check box `Import file and embed`. When clicking on `Open` PROVEtech:TA automatically embed the bitmap and renames the path accordingly.

To export an embedded bitmap just open the context menu of a bitmap entry:

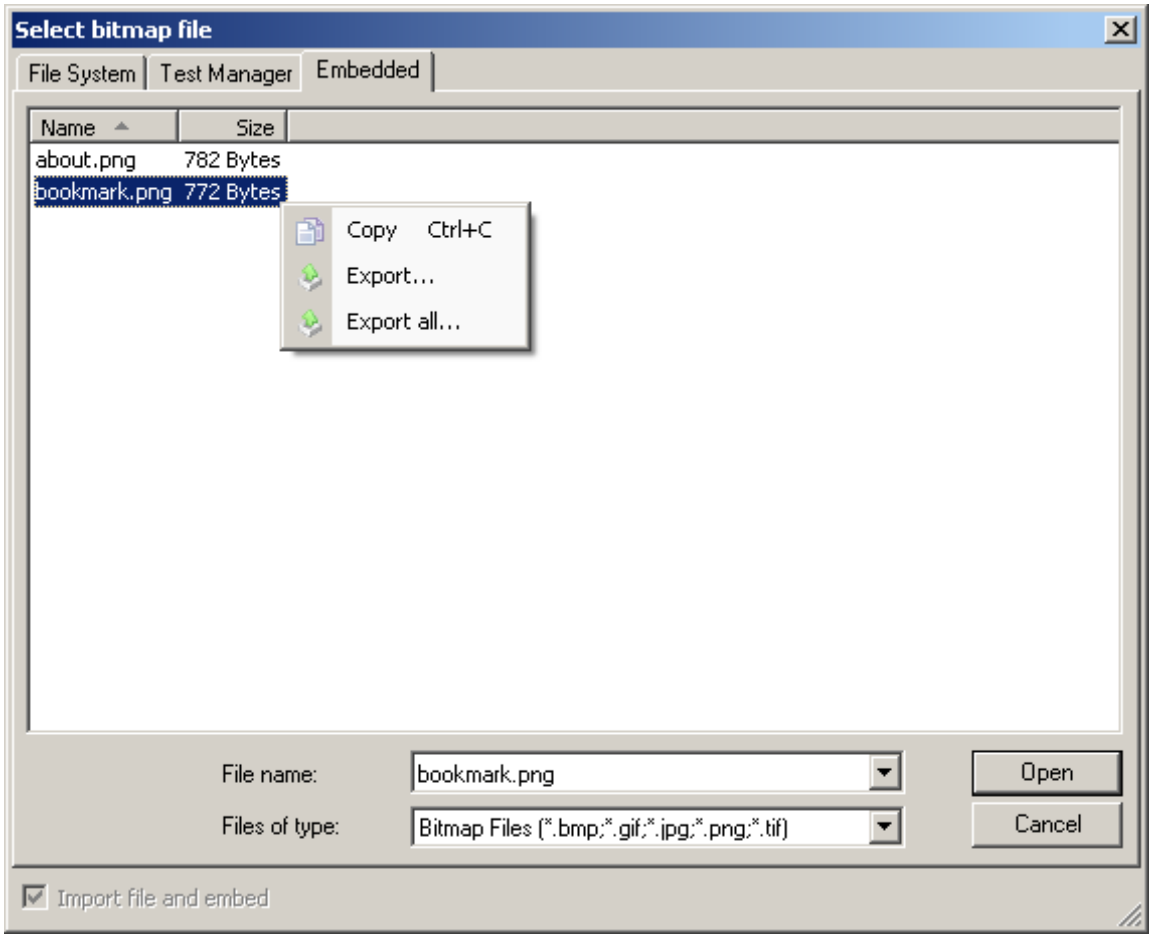


Figure 138 – Context menu of a bitmap

Select *Copy* to copy the bitmap into the clipboard. Use the Windows Explorer to paste it somewhere. Select *Export…* to export the bitmap or *Export all…* to export all embedded bitmaps.

PROVEtech:TA manages the names of the embedded bitmaps completely automatically. This means it also resolves naming conflicts by automatically renaming bitmaps. Additionally there is an automatic detection, if an identical embedded bitmap already exists with a different name. This adjustment also works for drag and drop or copy and paste of controls referencing embedded bitmaps between different workpage or cockpit files (also for linked workpage files).

As an example when you drag a control with an embedded bitmap from the workpage to the cockpit and PROVEtech:TA detects that the cockpit file already contains the same bitmap with a different name, it renames the bitmap reference within the moved control.

PROVEtech:TA manages the list of embedded bitmaps separately for each workpage or cockpit file. It is not possible to remove embedded bitmaps via the file selection dialog. Instead of that PROVEtech:TA automatically checks which embedded bitmaps still have references when saving a workpage file. It only stores these bitmaps within the workpage file. The unreferenced bitmaps are missing when loading the workpage file the next time.

4.3.11 Hexadecimal representation of signal values

Some controls support displaying or entering hexadecimal values with the prefix 0x. The number control (***4.4.6 The number control***) even supports binary output. However, this needs some additional explanation for some cases:

Any floating-point value must be type casted to an integer value (C type cast) which removes the decimal places. Any value smaller than -9223372036854775808 or bigger or equal to 18446744073709551616 exceeds the 64-bit integer limit and results in the output of the text *Overflow*. When using a unit conversion (see chapter ***4.4.1.1 The units converter***), the signal value is converted to a floating-point value for the calculation and the upper explanations also work for these values.

Negative values have a different hexadecimal representation dependent on the number of bits used to represent the values. As an example, the value -1 has the hexadecimal representation of 0xFFFF in case it has 16-bits or 0xFFFFFFFF in case it has 32-bits. PROVEtech:TA uses the signal data type to select this. Floating-point values are always converted to 64-bit integer values for the display – also after calculating a unit conversion.

Some fields of the controls' property dialogs support entering hexadecimal values with 0x prefix as well. However, these are always positive numbers.

4.4 The display and control elements

This subchapter describes the individual display and control elements and their properties. Some of these properties are common to all controls. For this reason, the next subchapter provides a description of these common properties. These are also available when several controls have been selected to change their properties.

In case a control displays a signal value numerically, there are four special values for floating-point signals: `NaN` (not a number), `-NaN` (negative not a number), `Inf` (infinity), `-Inf` (negative infinity). Please read the IEEE 754 standard for more information about that. In case the control provides the possibility to enter a signal value, you can also enter these four special values and the signal value is set accordingly. For `NaN` it always sets the "quiet" not a number.

4.4.1 General properties

The properties of a control are accessed via its context menu and the menu item `Properties...`. Select several controls and then `Properties...` to set properties common to these controls. The appearance of the property dialog depends on the control(s) selected. ***Figure 139*** shows a property dialog for a selection of several different controls.

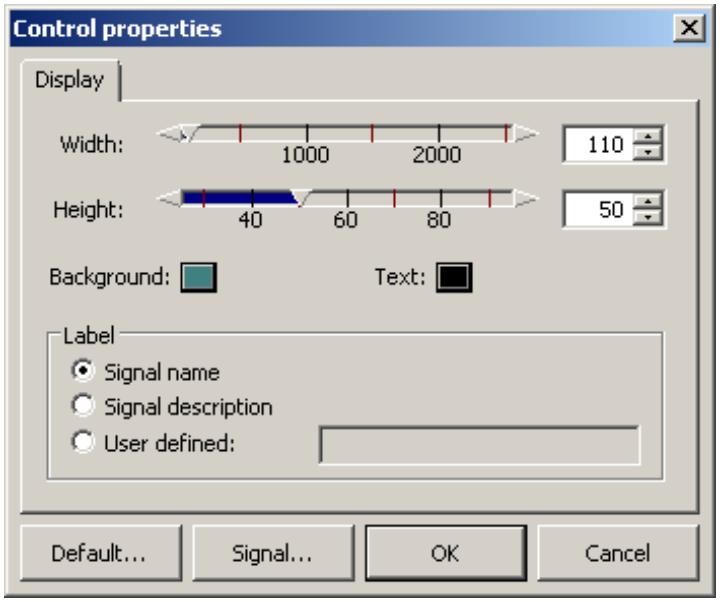


Figure 139 – General property dialog

The slide controls `Width` and `Height` determine the common size of the selected controls. The unit corresponds to the workpage grid. Use the `Background` and `Text` buttons to select the background and the text color. The radio buttons in the group `Label` determine whether the controls show the signal name (the whole path in the signal tree, the dot is the hierarchy separator), the signal description or a user defined string. If a single control has been selected, you also have the option of hiding the label completely.

Click on the `Default...` button to open the following dialog:

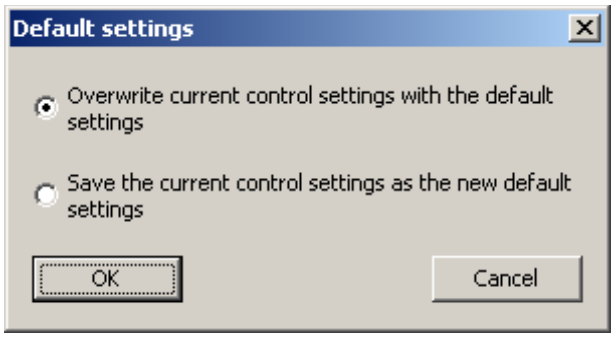


Figure 140 – Default Settings dialog

Choose the upper option to reset all properties of the selected control to their defaults. The lower option is only available when you only select one control. Use it to define the default settings for the type of the selected control.

Press the `Signal...` button in ***Figure 139*** to change the signal assigned to a control. Select a signal in the displayed Signal Selection window (see chapter ***4.2 The Signal Selection window***) and click on `OK`.

If you open the property dialog of an individual control, the `Info` tab is always a part of the dialog. This tab provides all the information on the control position and size and the signal assigned to the control (see ***Figure 141***). There you can also see if a signal has a text table and which values the text table stores.

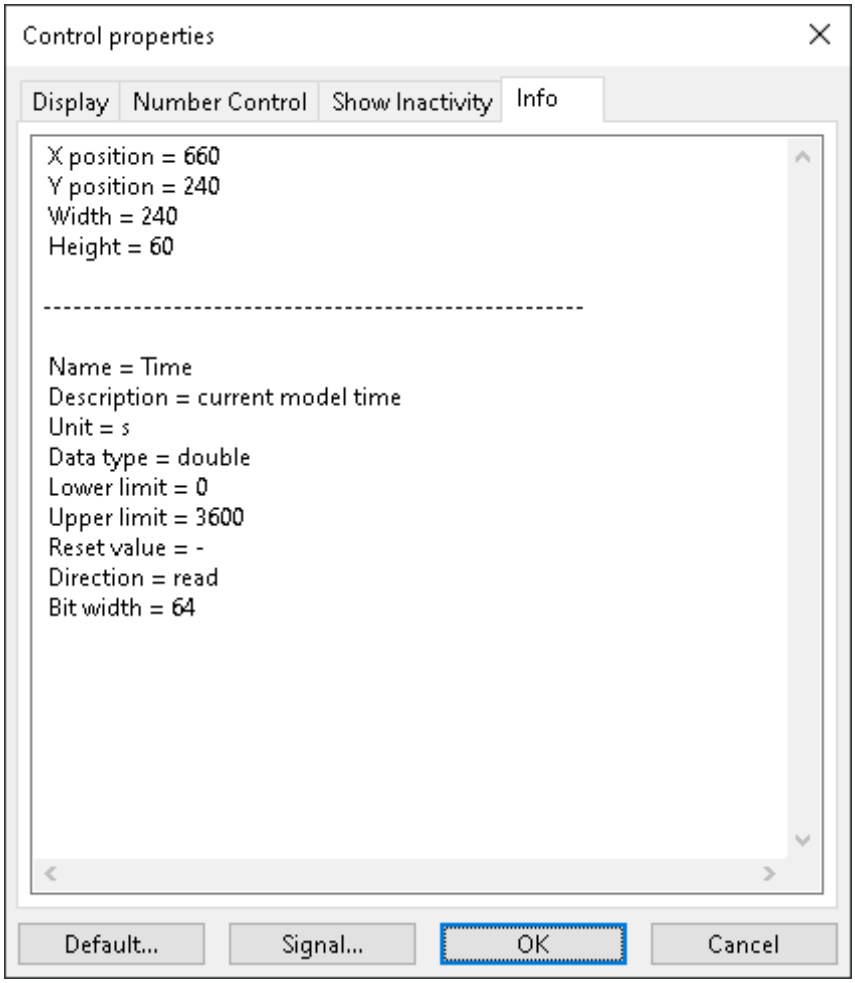


Figure 141 – Info tab of a property dialog

Most controls also support changing the background color to show inactivity. The `Show Inactivity` tab configures that:

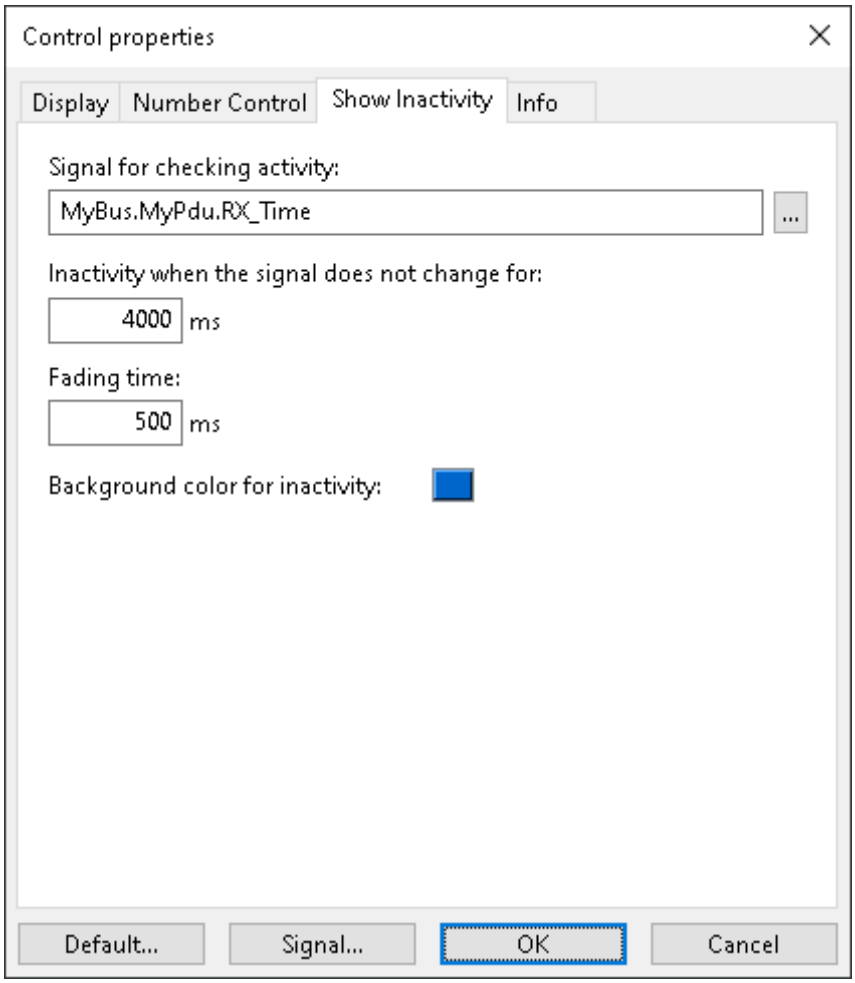


Figure 142 – Show Inactivity tab of a property dialog

If you do not specify `Signal for checking activity` or the specified signal does not exist, the feature is disabled. Otherwise you define a signal here which PROVEtech:TA checks cyclically for changes (about every 100ms – it does not check every model cycle). If the signal value changes, the control's background color stays on the specified default; typically set on the `Display` tab. If the signal does not change for the duration in milliseconds entered in the field `Inactivity when the signal does not change for`, the background color fades to the color defined by `Background color for inactivity`. The `Fading time` defines how long this takes in milliseconds. Please be aware that updating the background color means re-painting the whole control, which reduces the overall performance. So, the `Fading time` should be short. This feature shall mainly show if a signal is still transported on a data bus regularly. Typically, the rest bus simulation provides a signal containing the time for the last transmission of a PDU/signal on the data bus. You can use that signal to check the (in)activity. When the feature is active, the initial background color is the inactive one up to the first detection of a signal change.

Exit the property dialog by clicking on `OK` (changes to settings are applied) or `Cancel` (changed settings are discarded).

4.4.1.1 The units converter

Some control types allow you to convert the signal values from one unit to a different unit, for example, from millimeters to centimeters. An example of this is shown in ***Figure 143*** for a number control. Select the unit to be shown in the `Unit` combo box. Subsequently the control displays all signal values in the new unit.

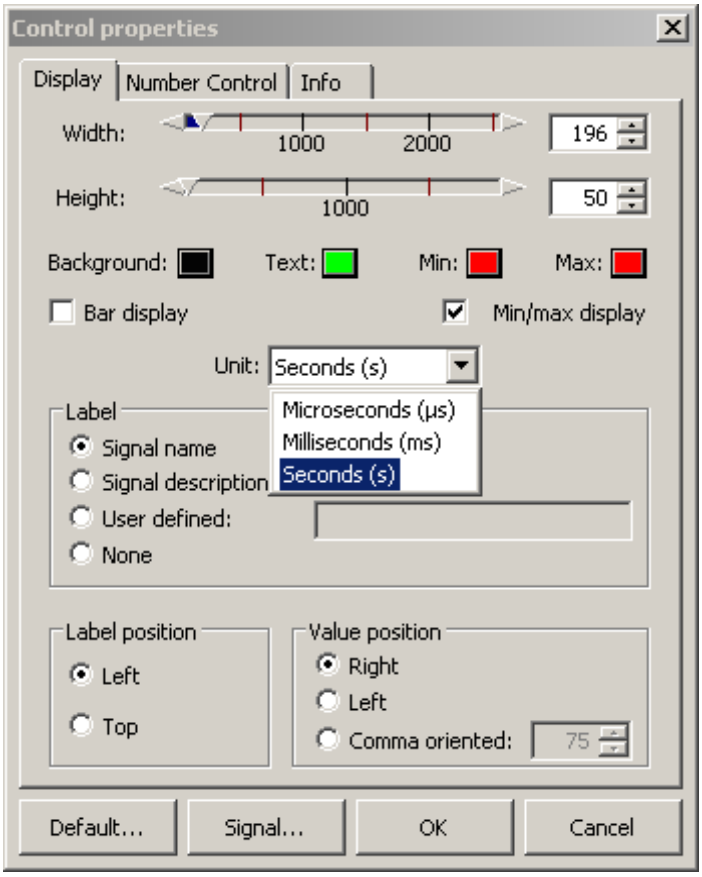


Figure 143 – Example of a units converter

The conversion options that are available depend on your settings in the settings dialog (chapter ***3.2.7.3 Workpage***).



Using a unit conversion also means, that the calculation uses the 64-bit floating-point data type (52-bit mantissa). This might have an impact on the precision of the displayed value. As an example, if your signal data type is UInt64 and the current signal value is very big like 72057594037927933, converting that value to floating-point for the unit conversion calculation reduces the precision. The next possible value the 64-bit floating-point data type can represent is 72057594037927936. See also chapter ***2.12 Working with floating-point numbers***.

4.4.1.2 Import of text tables

Some controls display text representatives for values, e.g. “Off” for the signal value 0. Very often these assignments for CAN, LIN, FlexRay or CCP/XCP signals are defined in .dbc, .ldf, AUTOSAR-XML, FIBEX or .a2l files. So, the according control property pages have the button `Import`…. After clicking it you can select a file via the file selection dialog (see chapter ***3.1 The file selection dialog***). PROVEtech:TA then displays a dialog containing all signals of the selected file that have text tables. Depending on the file type there are several hierarchies in the displayed tree. As an example for .dbc files there are nodes/ECUs, messages, signals and text tables. Select a signal and press `OK` to assume the according text table.

4.4.2 The strip chart and multi-strip chart controls

Contrary to the multi-strip chart control, the strip chart control represents a single signal over time. The associated buttons in the Signal Selection window (chapter ***4.2 The Signal Selection window***) look like this:  and  . Other than this there is no difference between these two controls. You can turn a strip chart control into a multi-strip chart control or add additional signals by dragging signals from the Signal Selection window or other controls into the strip chart control using the drag and drop technique.

Strip chart controls are only of interest during an ongoing measurement and for evaluation of measurements in the offline mode. ***Figure 144*** shows a multi-strip chart control during an ongoing measurement.

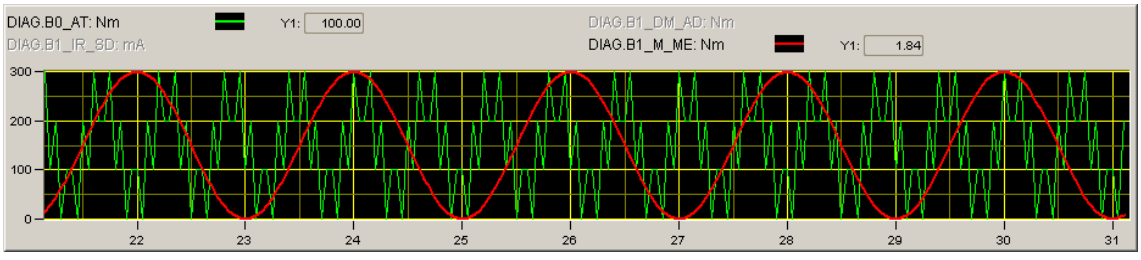


Figure 144 – Multi-strip chart control during measurement

Arranged in two columns, the upper part of the control shows the signals included in the strip chart, their units, the colors assigned to them and their current signal values (`Y1`):



Figure 145 – Description of a signal in the strip chart control

Only measurement signals appear in the strip chart control in this form. The names of the signals that have not been selected for measurement turn gray when a measurement is started:



Figure 146 – Grayed name of a signal that is not measured

The lower part of the strip chart control shows the signal curves during measurement. The unit used for the time range displayed is seconds; this corresponds a setting of the property dialog (see chapter ***4.4.2.1 The property dialog of the strip chart control***).

When a measurement is stopped, the multi-strip chart control looks like this:

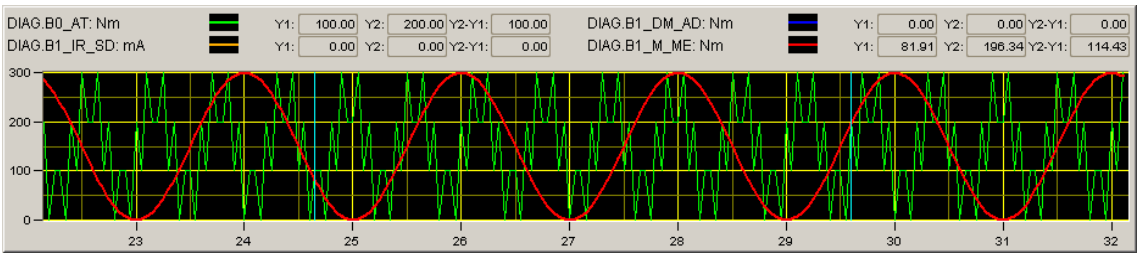


Figure 147 – Multi-strip chart control after a measurement

Two vertical cyan cursors are displayed (their color is adjustable). ***Figure 148*** shows one of them in an enlarged section. The signal descriptions contain two additional edit boxes `Y2` and `Y2-Y1` (see ***Figure 149***).

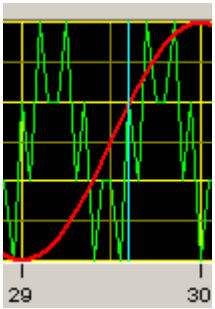


Figure 148 – Cursor in the strip chart control

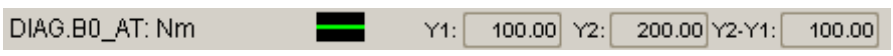


Figure 149 – Description of a signal after measurement

`Y1` contains the signal value at the left, `Y2` the signal value at the right cursor position. `Y2-Y1` shows the difference between the two values. In the offline mode (see ***4.5 The offline mode***), you have additional options for signal curve evaluation.

You can show or hide a signal plot by clicking on its line display in the legend. A ghost icon appears for a hidden plot in the legend: 

Press the Ctrl key and the left mouse button while moving the mouse cursor over the strip chart control to move the displayed value range (Y axis). In this case the mouse cursor changes to a hand symbol (see ***Figure 150***). In the offline mode you can additionally move the time range (X axis) when you have activated the update of the display controls in the overview graph (see ***4.5.1 The overview graph, evaluation of measure data***).

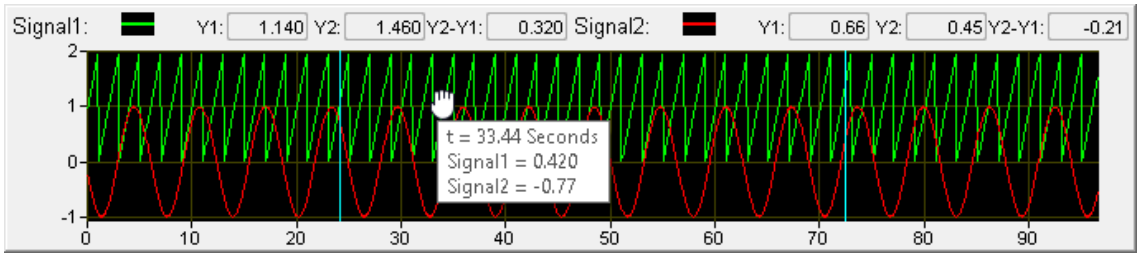


Figure 150 – Moving displayed ranges

When you press the Shift key and the left mouse button, the mouse cursor changes to a loupe. Now move the mouse cursor over the strip chart control to select a new display area (see ***Figure 151***). After releasing the mouse button, the strip chart control displays the selected ranges. This process always affects the value range (Y axis). It only affects the time range when you have activated the update of the display controls in the overview graph (see ***4.5.1 The overview graph, evaluation of measure data***).

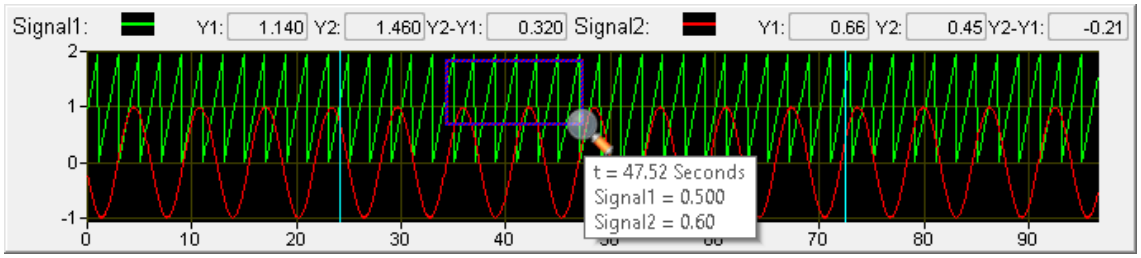


Figure 151 – Selecting displayed ranges

Hold the Ctrl key while moving the mouse wheel and while the mouse cursor is on the strip chart control to adjust the value range (Y axis zoom in/out).

4.4.2.1 The property dialog of the strip chart control

The property page `Display` of the property dialog of a strip chart control has the following appearance:

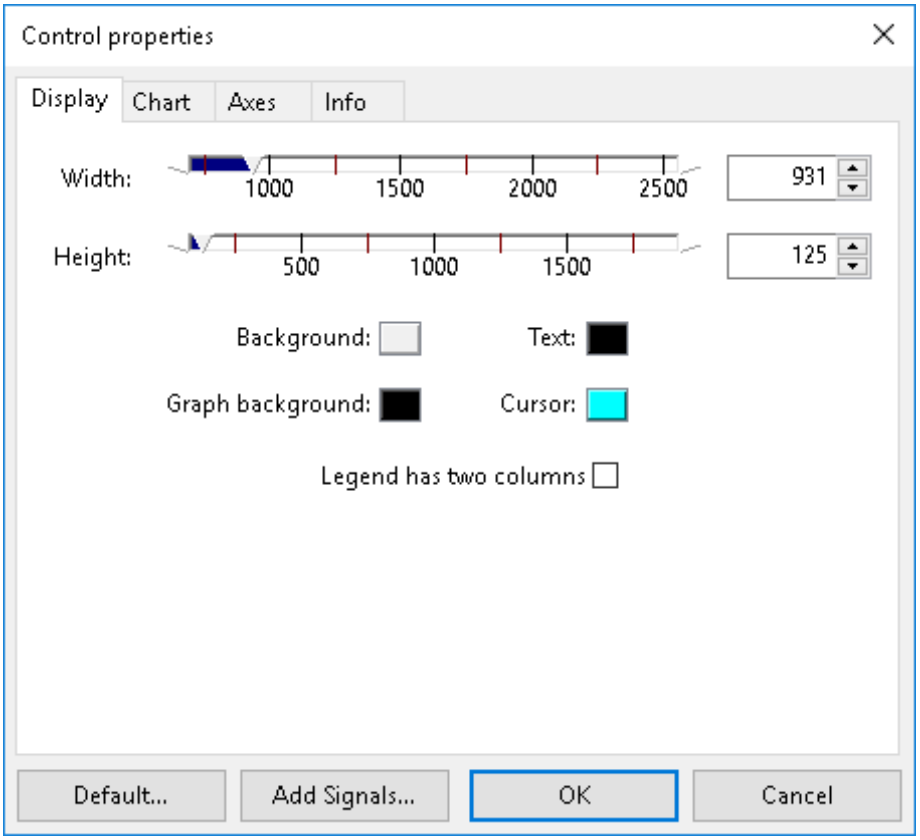


Figure 152 – Property page Display

Here you can set the `Width` and `Height` of the control, as well as the `Background` color (border), the `Graph background` color (the background inside the graph), `Text` color (signal name and value) and `Cursor` color.

Activate `Legend has two columns` in case your strip chart control has several signals and you want a more compact legend.

With the `Chart` tab you can adjust the representation features of individual signals.

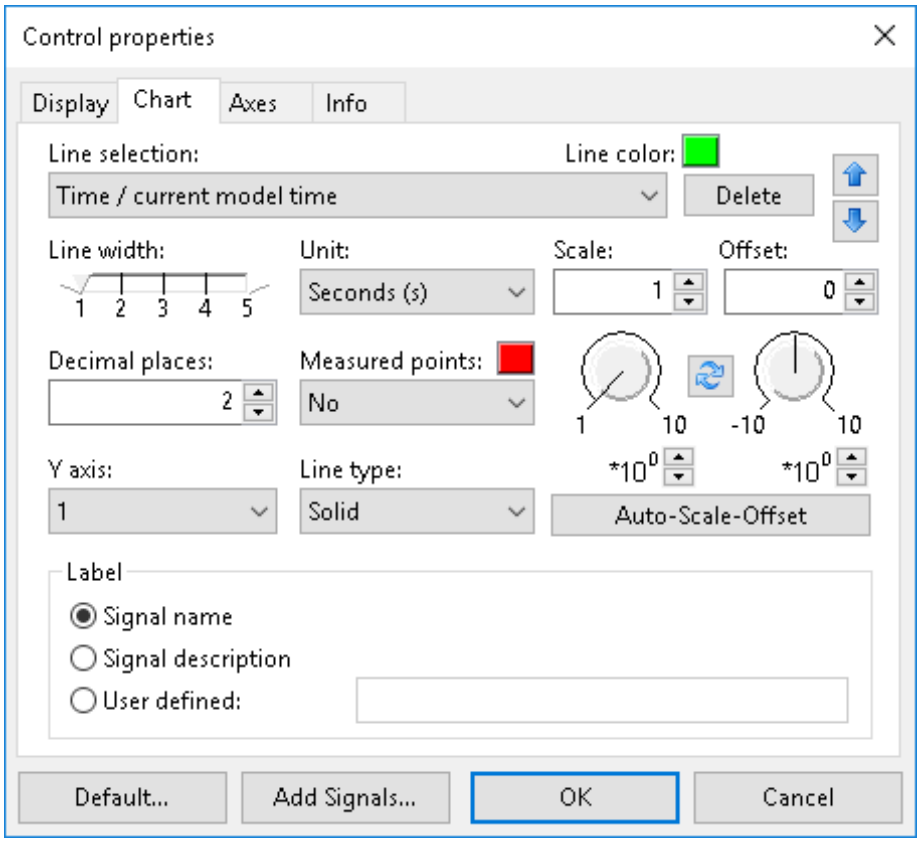


Figure 153 – Property page Chart

Select the signal you wish to edit in the combo box `Line Selection`. Click on `Delete` to remove the selected signal from the strip chart control. Use the arrow buttons right beside `Delete` to move the signal up or down in the legend of the strip chart control. The `Add Signals...` button is used to add signals to the strip chart control. You can also define the appearance of the plotted line for this signal: color, line width, line type (e.g. dotted) and select whether or not the measured points should be displayed. The amount of decimal places to show is set from the `Decimal Places` field.

For strip chart controls, you can define the scale and offset for each signal. PROVEtech:TA calculates the values for representation in the strip chart control according to the following formula: value=(signal value*scale)+offset.

Either enter the values directly in the edit boxes `Scale` and `Offset` or use the rotary controls below the boxes. The $*10^x$ value specifies the power of ten; this can be adjusted with the arrow buttons next to the values. Click on  to reset the scale to 1 and the offset to 0.

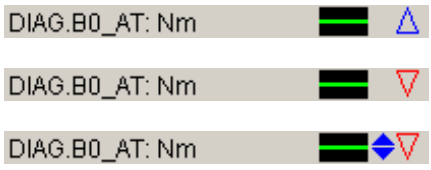
The following symbols in the strip chart control indicate for which signals a scale and/or offset has been configured:



scale>1, offset=0



scale<1, offset=0





The scale and offset settings can be changed during an ongoing measurement and then affect only new values (values already displayed are not affected).

Press `Auto-Scale-Offset` to adjust scale and offset for the signal automatically. After that the plotted line fits into the range of the Y-axis.

The `Axes` tab is used to define the appearance of the time-axis and the Y-axes:

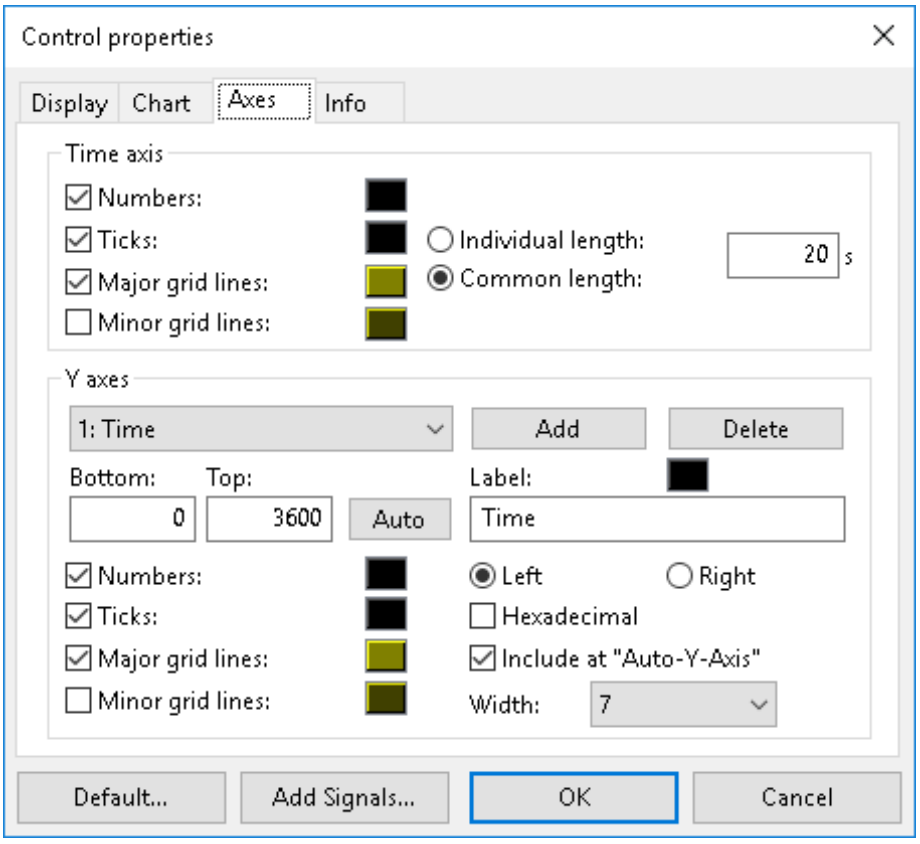


Figure 154 – Property page Axes

The settings at the upper part of the `Axis` tab correspond to the time-axis and those below correspond to the Y-axes. The check boxes determine if the `Numbers`, `Ticks`, `Major grid lines` and `Minor grid lines` are visible or not. Use the buttons right of them to

select the used colors. The numbers and ticks are outside the actual plot area while the grid lines are inside. The major grid lines correspond to the displayed numbers and ticks while the minor grid lines are between them. In case the minor grid lines are enabled and the major grid lines are disabled, the minor grid line color is used to paint the major lines.

More than one Y-axis can be displayed on a graph: Add another Y-axis to a currently displayed graph by clicking on the `Add` button in the `Y axes` section. Select one of the Y-axes via the combo box to edit its properties. You can also remove the currently selected Y-axis by clicking on the `Delete` button. However, a graph must have at least one Y-axis, that is to say, if only one Y-axis exists in the graph you will not be able to delete it. With `Bottom` and `Top` you can enter the value range for a Y-axis. Click on `Auto` to automatically calculate `Bottom` and `Top` from the currently displayed values.

The `Label` field defines an additional string displayed vertically beside the Y-axis. The button above the edit field for the label selects the color for the label.

The radio buttons `Left` and `Right` determine the position of the axis. The `Width` combo box can be used to set the width for the numbers of the axis (amount of characters). This is only relevant in case `Numbers` or `Ticks` are enabled.

Activate the check box `Hexadecimal` to display the axis numbers and signal values hexadecimal. Please read the chapter ***4.3.11 Hexadecimal representation of signal values*** for more information about that and also consider that any scale/offset calculation converts the signal value to a floating-point value.

`Include at "Auto-Y-Axis "` specifies, if this axis should perform an automatic range update in case you select the context menu item *Auto-Y-Axis Size|All but explicit excluded Axes* of the control. This check box is only available in case there are several Y axes.

Use the radio buttons `Individual length` and `Common length` to select if you want to use an individual time axis in the online mode or the same one together with other strip chart controls. An individual time axis is not changeable during a data acquisition but can have a much longer range than a common one. PROVEtech:TA calculates an individual downsampling rate for the view to only display every n^{th} sample. In the right text field you edit the length of the time axis in seconds.

In case you select `Common length`, the right text field edits the time axis length for all strip chart controls on the workpage, which do not use an individual time axis. The length can vary between 0.5 and 100 seconds in this case.

In the offline mode (see chapter ***4.5 The offline mode***) the selection in the overview graph determines the time axis length of all strip chart controls.

4.4.3 The XY strip chart control

The XY strip chart control displays two signals. One signal represents the X axis, the other one the Y axis. The corresponding button in the Signal Selection window (chapter ***4.2 The Signal Selection window***) has the following appearance: 

XY strip chart controls are only of interest during an ongoing measurement and for evaluation of measurements in the offline mode. ***Figure 155*** shows a XY strip chart control during an ongoing measurement. The upper signal name corresponds to the Y axis, the other one corresponds to the X axis. `Y1` and `X1` display the latest signal values. The pink dot (color and shape selectable) is called head cursor marks the latest measured point. You can also display a tail cursor.

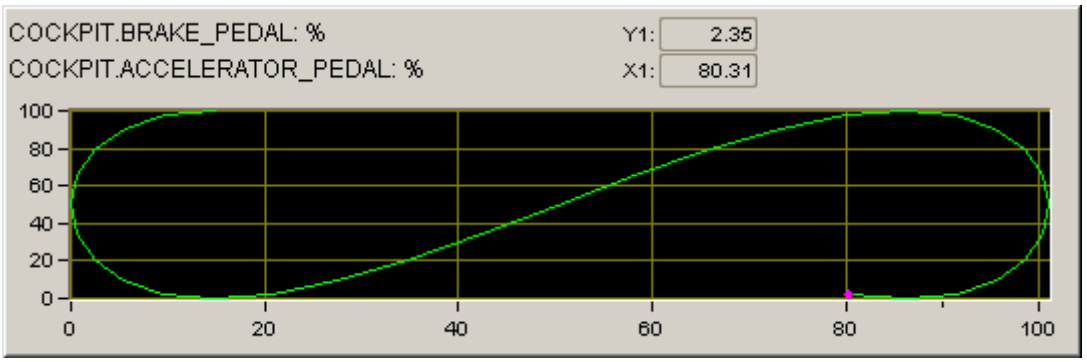


Figure 155 – The XY strip chart control during a measurement

Both signals must be measurement signals, otherwise PROVEtech:TA displays the signal name of a not measured signal grayed during the measurement and it does not draw the chart.

When you stop the measurement, the XY strip chart control changes its appearance:

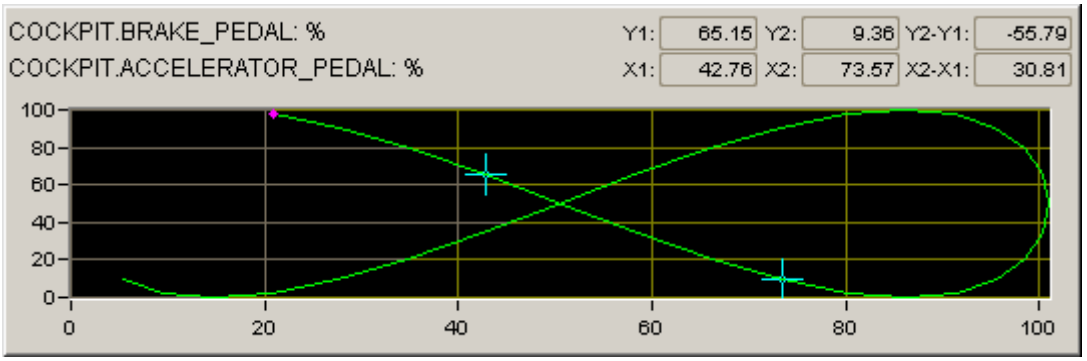


Figure 156 – The XY strip chart control after a measurement

Now there are two cursors you can move by dragging them while holding down the left mouse button. `Y1` and `X1` show the signal values of the left cursor, `Y2` and `X2` show the values of the right cursor and `Y2-Y1` and `X2-X1` show the differences.

After creating a XY strip chart control with drag and drop from the Signal Selection window both, the X and the Y axis are assigned to one signal. To assign a different signal to the Y axis,

drag it from the Signal Selection window and drop it on the Y axis or the upper signal name of the control. To assign a different signal to the X axis, drag it from the Signal Selection window and drop it on the X axis or the lower signal name of the control. Or you can use the property dialog (***4.4.3.1 The property dialog of the XY strip chart control***) to change the signals.

Press the Ctrl key and the left mouse button while moving the mouse cursor over the XY strip chart control to move the displayed ranges (X and Y axis). In this case the mouse cursor changes to a hand symbol (see ***Figure 157***).

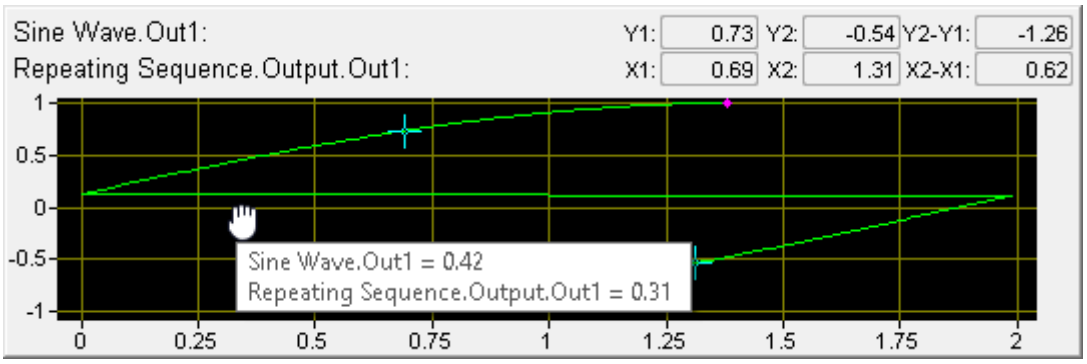


Figure 157 – Moving displayed ranges

When you press the Shift key and the left mouse button, the mouse cursor changes to a loupe. Now move the mouse cursor over the XY strip chart control to select a new display area (see ***Figure 158***). After releasing the mouse button, the XY strip chart control displays the selected ranges.

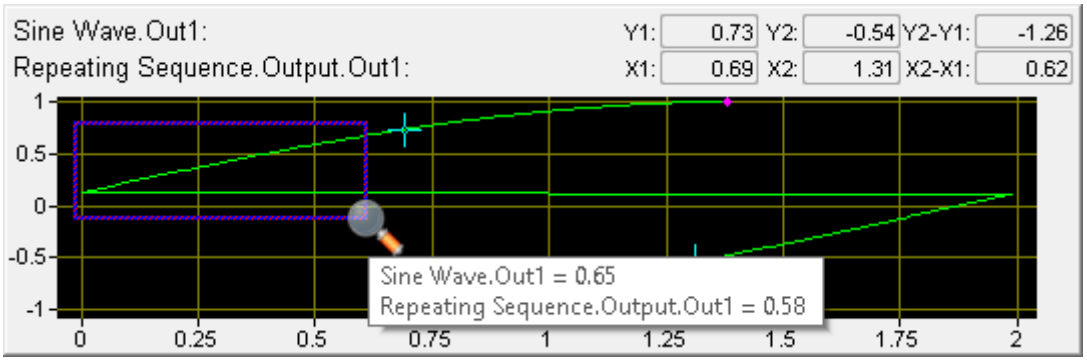


Figure 158 – Selecting displayed ranges

Hold the Ctrl key while moving the mouse wheel and while the mouse cursor is on the XY strip chart control to adjust both axes very fast (zoom in/out).

4.4.3.1 The property dialog of the XY strip chart control

The property page `Display` contains the settings for the size and the inner area (not the axes) of the XY strip chart control:

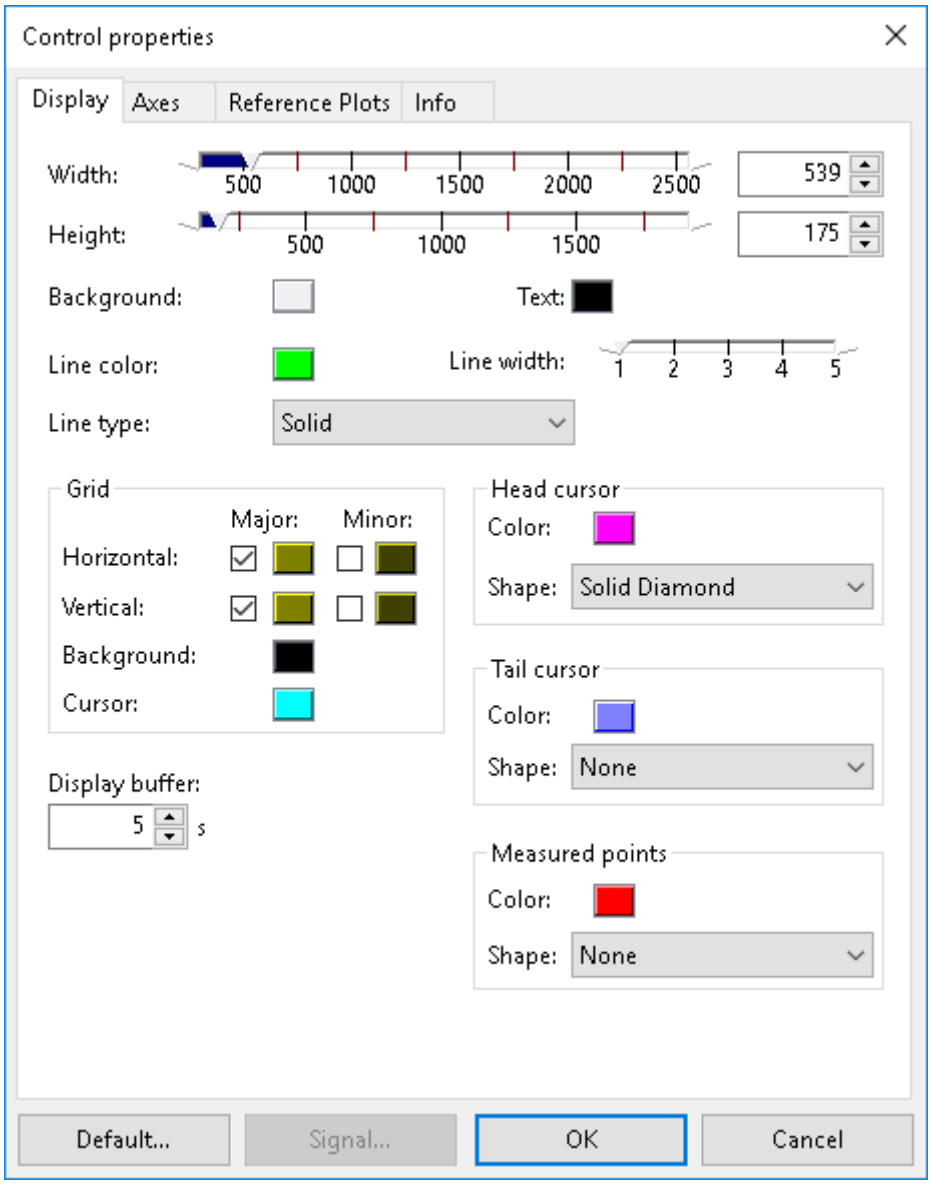


Figure 159 – Property page Display

Here you determine whether or not and which grid lines are to be displayed in which color. Select the colors for the grid's background, the cursors and the chart line. You can also change the color and the shape of the head and the tail cursor, the chart line width, color and type and the size and colors of the whole control. The `Display buffer` is the memory size that stores the signal values while measuring. After the selected time the control “loses” old values. The maximum is 100s. The `Display buffer` does not affect the offline mode.

The `Measured points` group determines if and how PROVEtech:TA marks the actual sample values. Please keep in mind that PROVEtech:TA typically performs a downsampling before the samples reach the control.

Use the property page `Axes` to adjust the both axes / signals:

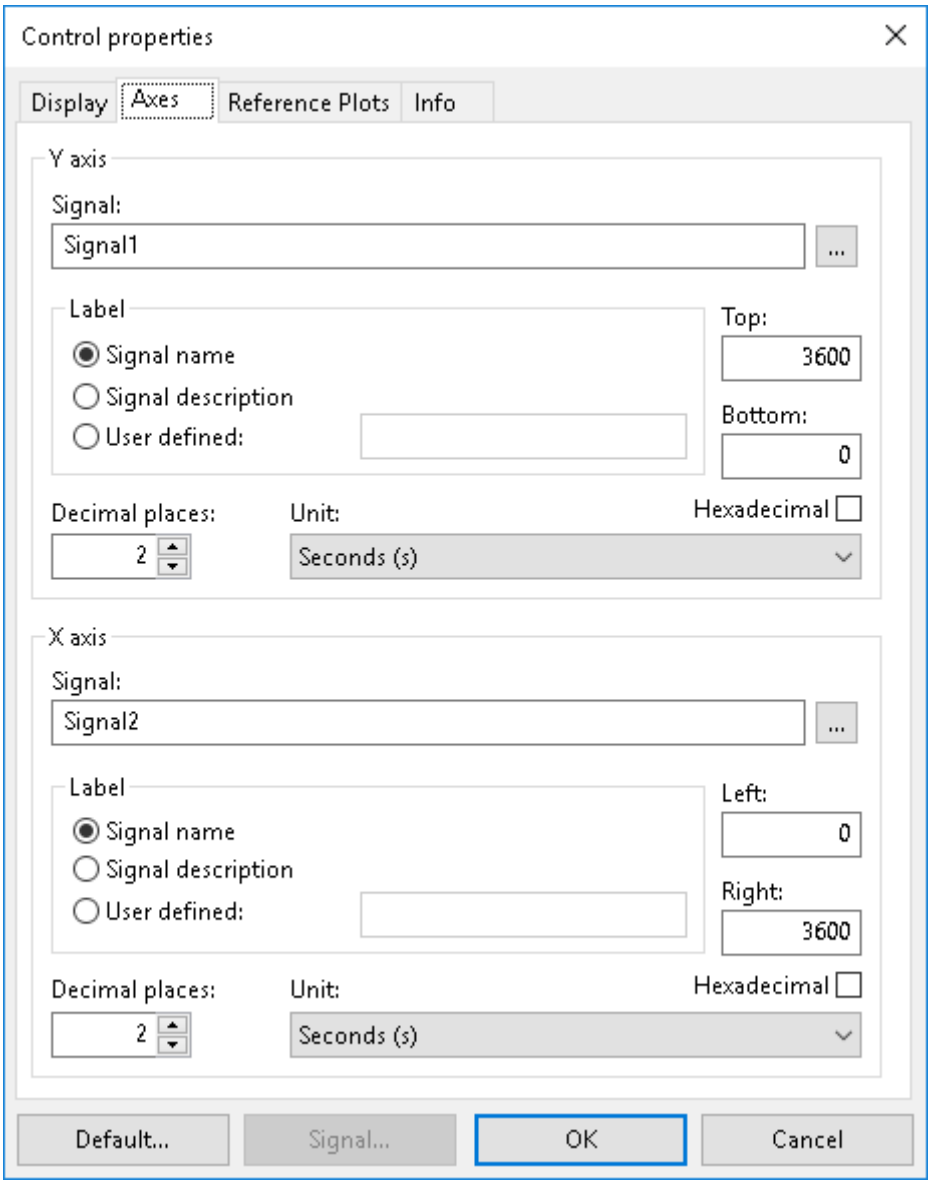


Figure 160 – Property page Axes

The Y axis (top) and the X axis (bottom) have the same settings: the according `Signal`, the `Label` type, the displayed value range (`Top`/`Bottom` or `Left`/`Right`), the amount of `Decimal places` in the signal value displays and the `Unit`.

Activate the check box `Hexadecimal` to display the axis numbers and signal values hexadecimal. Please read the chapter ***4.3.11 Hexadecimal representation of signal values*** for more information about that.

Change the signal by entering its signal name in the according edit box, by pressing the … button or by dragging a signal from the Signal Selection window and dropping it in the signal edit box.

Use the property page `Reference Plots` (***Figure 161***) to display additional plots in the XY strip chart control, which are defined via XY value pairs.

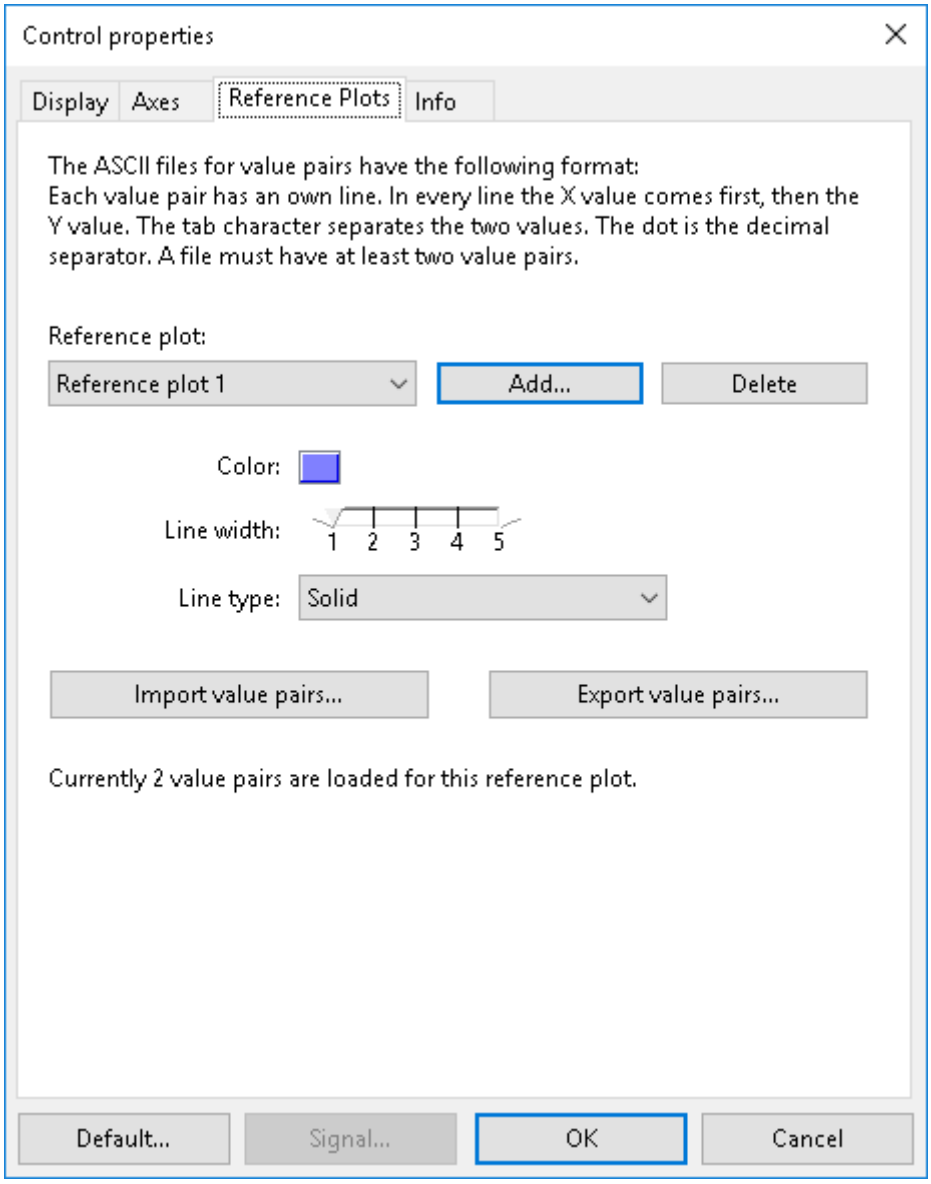


Figure 161 – Property page Reference Plots

As an example, the reference plots show limits that the displayed signal should not cross. The control can display up to 16 reference plots. Click on `Add...` to create a new reference plot. This opens the file selection dialog (see chapter ***3.1 The file selection dialog***) to select a file

containing the XY value pairs. The upper text of the property page describes the according file format.

You can select the reference plot to edit via the combo box `Reference plot`. Click on `Delete` to remove the currently selected reference plot. The properties that you can change are `Color`, `Line width` and `Line type`.

Click on `Import value pairs...` to replace the currently loaded value pairs of the reference plot by others and `Export value pairs...` to store the currently loaded ones in an ASCII file. PROVEtech:TA does not link value pair files in workpage files, but directly stores the value pairs.

4.4.4 The slider / bar display control

The slider / bar display control serves as a display and control element. The corresponding button in the Signal Selection window (chapter ***4.2 The Signal Selection window***) has the following appearance: 

There are three different versions of the slider / bar display control:

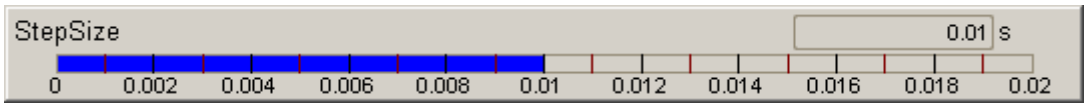


Figure 162 – Bar display for a non-changeable signal

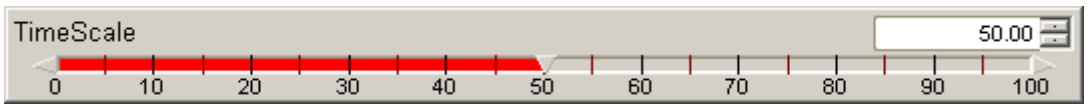


Figure 163 – Slide control for a changeable signal

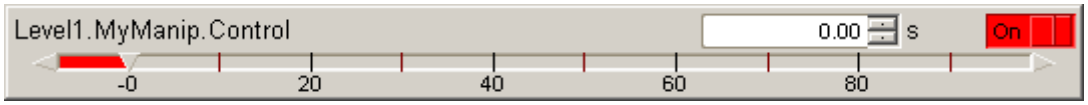


Figure 164 – Slide control and switch for a control signal

You can change the signal value of a slide control by dragging the center triangle, by entering a value in the edit box or by clicking on the arrows on the left and right sides of the slide control.

The slider control constitutes a special case for a so-called control signal. Activate the switch on the right side to break up the model at a specific position and inject the value defined via the slide control at this position. When the switch is deactivated, the model determines the value itself and ignores the setting defined here.

The appearance of these controls can be configured by many different properties as described in the following chapter.

4.4.4.1 The property dialog of the slider / bar display control

In addition to entering the usual specifications for height, width, colors and unit, by means of radio buttons, the `Display` property page is used to determine whether or not and in which positions the display components label, value, unit (see chapter ***4.4.1.1 The units converter***) and the slide control/bar display are to be displayed. The `Horizontal`, `Vertical` and `Pedal` setting influences the slider orientation. `Pedal` is a vertical slider with a pedal picture instead of an arrow for manipulating the value. The amount of decimal places to display is only editable for floating-point signals. For control signals you can also set the position of the activation switch.

Due to compatibility, the default background color for the number display is white for writable signals and the control's background color for read-only signals. Beginning with PROVEtech:TA 2024 SE, you can select the color by using the `Number display background color` button.

In case you activate the check box `Use text table of the signal` and your signal has an assigned text table (displayed on the `Info` tab, a table with value range to text assignments), the control displays the according text of the table in case the signal values matches an entry. Otherwise, it displays the number. For writable signals you can also enter the text of a table entry and it will set the signal value to the lower boundary of the according entry. In case you deactivate the check box, it always displays the signal value as a number, and you can only enter numbers.

Activate the check box `Hexadecimal` to display signal values in the hexadecimal format. Please read the chapter ***4.3.11 Hexadecimal representation of signal values*** for more information about that.

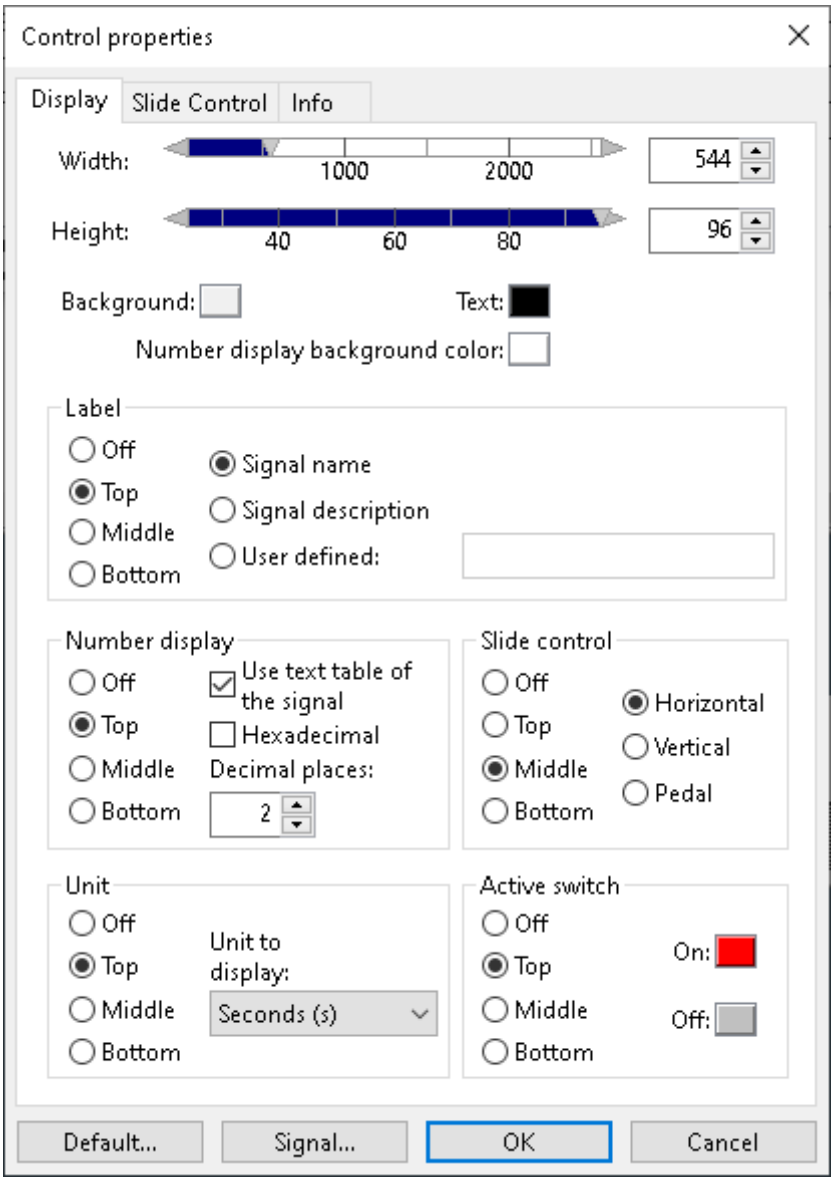


Figure 165 – Property page Display

The `Slide Control` property page is used to configure the representation range (`Left`, `Right`) of the slide control / bar display and the position of the axis display. The field names change slightly for a vertical orientation.

The tag sets a reference value for the signal value with a position you define. The display colors the area between the current signal value and tag with definable colors. PROVEtech:TA distinguishes by means of different colors whether the signal value is above or below the tag position.

You can adjust the step size when clicking on the arrow buttons beside the slide control (`Short activation`) and when holding them down (`Long activation`). Use `No spin buttons` to hide the arrow buttons. The group `Step size of the spin buttons` is not available when using a read only signal as it makes no sense there.

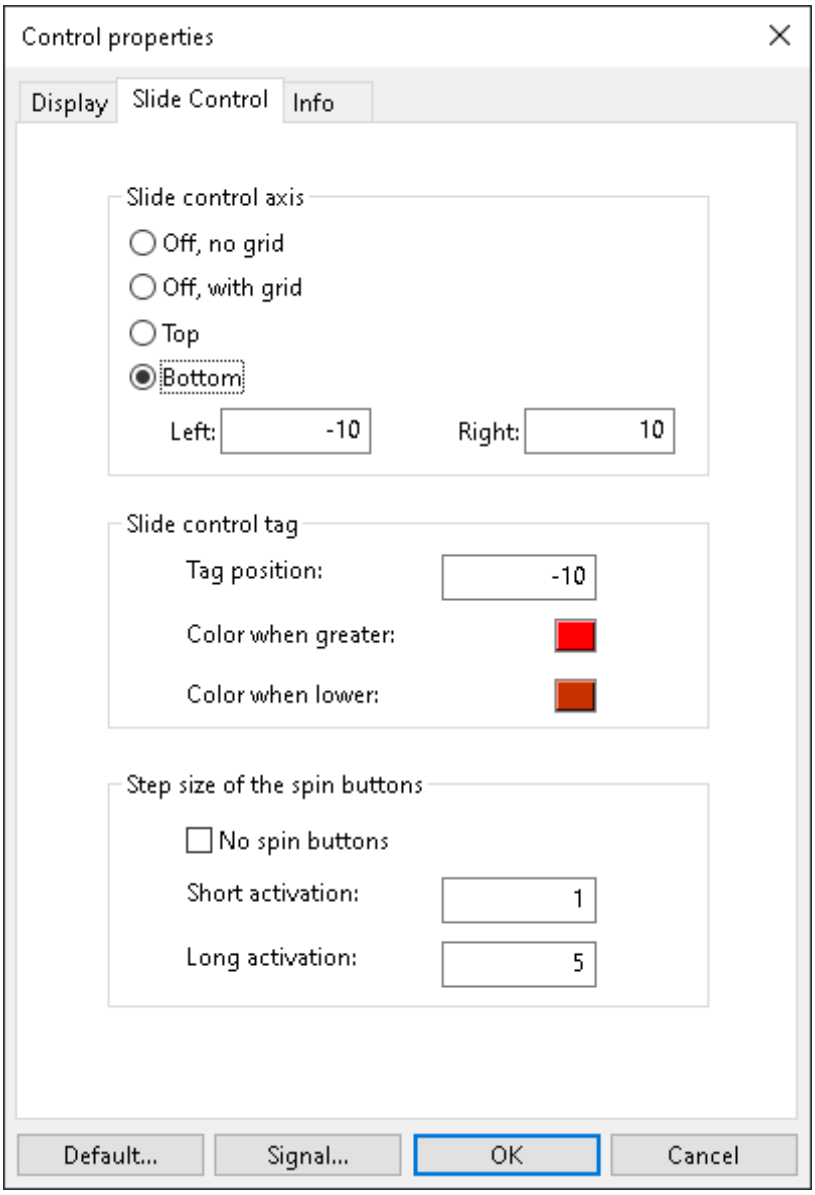


Figure 166 – Property page Slide Control

4.4.5 The switch / LED control

The switch / LED control serves as a display and control element. The corresponding button in the Signal Selection window (chapter ***4.2 The Signal Selection window***) has the following appearance: 

There are three different versions of the switch / LED control:



Figure 167 – LED for a non-changeable signal



Figure 168 – Switch for a changeable signal



Figure 169 – Two switches for a control signal

Change the signal by clicking on the switch with the mouse.

The switch control constitutes a special case for a so-called control signal. Activate the upper switch to break up the model at a specific position and inject the value defined via the lower switch at this position. When the upper switch is deactivated, the model determines the value itself and ignores the setting defined here.

The appearance of these controls can be configured by many different properties as described in the following chapter.

4.4.5.1 The property dialog of the switch / LED control

In addition to entering the usual specifications for size and colors, the `Display` property page is used to define the label position and the signal position (switch / LED) within the control.

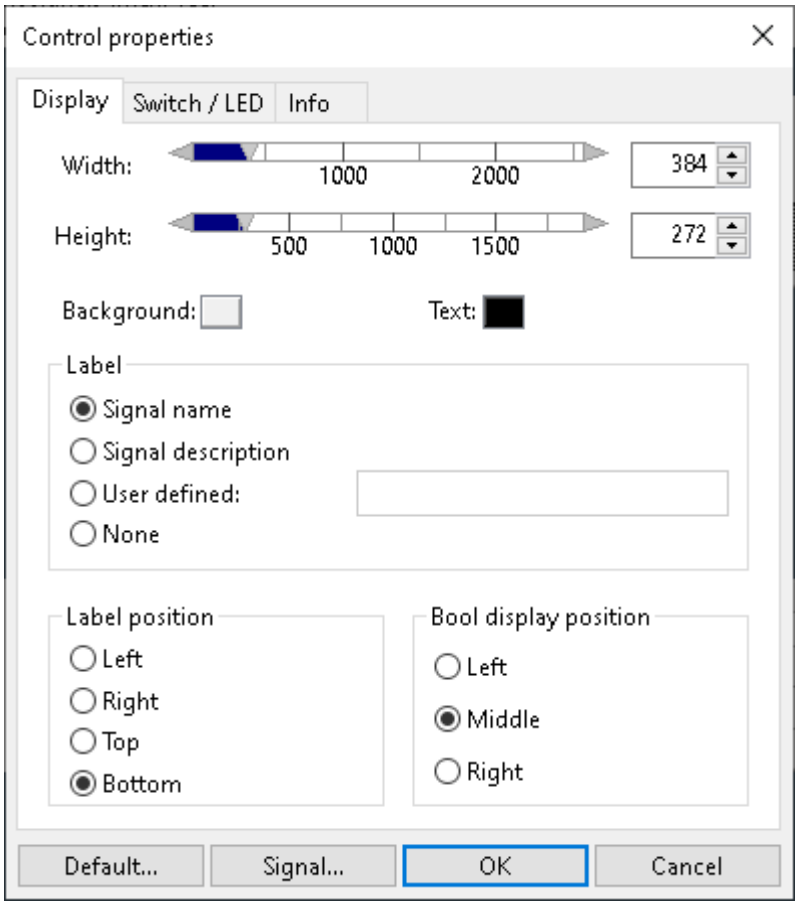


Figure 170 – Property page Display

In the case of a switch, i.e. a changeable signal, the following property page appears when you click on the `Switch / LED` tab:

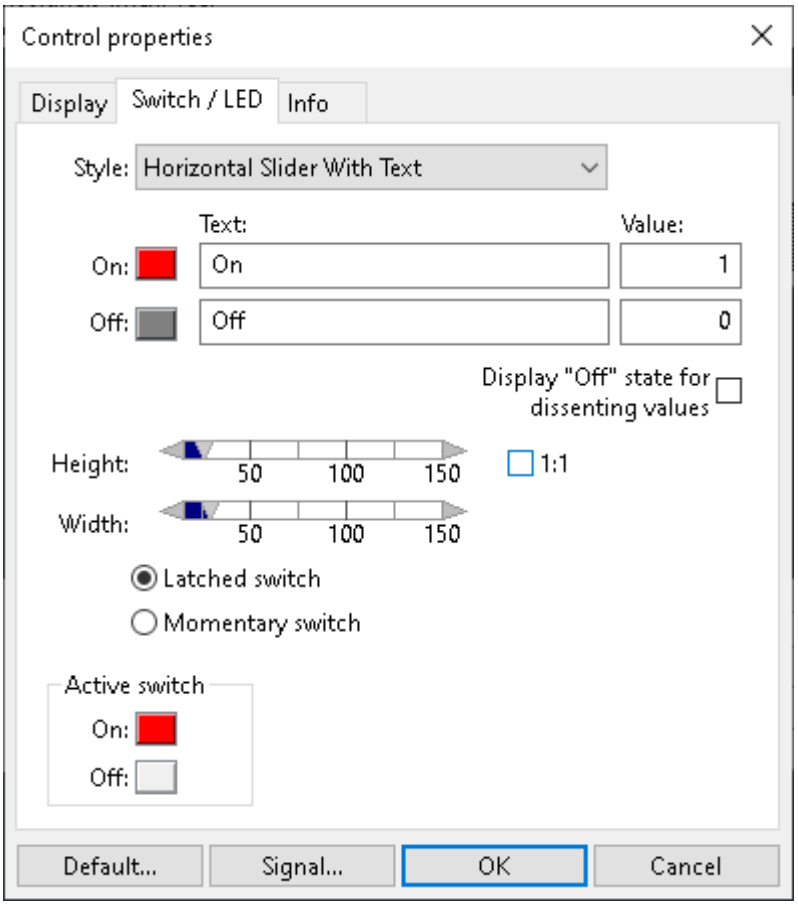
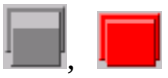


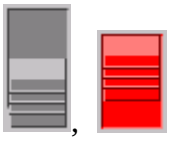
Figure 171 – Property page Bool Control

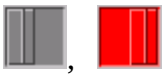
The `Style` combo box is used to select one of the various styles available. Some examples:

Off, On

 Square Push Button

 Round Push Button

 Vertical 3D Slider

 Horizontal Slider

Use the `On` button to define the color for the pressed (active) state and the `Off` button to define the color for the released (inactive) state. In case the selected style allows it, you can additionally define the `Text` to display.

The edit boxes `Value` specify the signal values in the “on” and in the “off” state for writable signals. Normally the control displays the “on” state when the signal value differs from both

values. To display the “off” state instead, activate the check box `Display "Off" state for dissenting values`. For read only signals, you can only edit the one value that is used for calculating the displayed state.

The size of the switch can be adjusted via the `height` slide control. Deactivate the check box `1:1` to define the height and width independently.

The radio buttons `Latched switch` and `Momentary switch` determine the behavior of the switch when clicking on it. A latched switch remains pressed until clicked on again. A momentary switch only remains activated until the mouse button is released.

In the case of an LED control, which represents a non-changeable signal, the radio buttons `Latched switch` and `Momentary Switch` are not available. Moreover, the entries in the `Style` combo box are different, e.g.:

Off, On

 ,  Round

 ,  Round With Star

 ,  Rectangular

 ,  Rectangular With Star

Even more settings are available for the button on the custom control, see chapter ***4.4.12.4 Button***.

In case the displayed signal is a control signal, a second switch displays the according active signal. Change its colors via the `Active switch` buttons.

4.4.6 The number control

The number control (***Figure 172***) indicates signal values as decimal, hexadecimal or binary values. The corresponding button in the Signal Selection window (chapter ***4.2 The Signal Selection window***) has the following appearance:  .

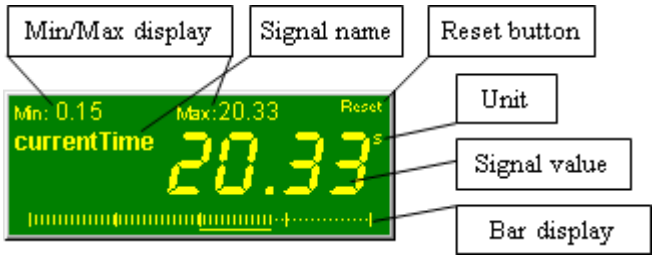


Figure 172 – The number control

The Min/Max display represents the minimum and maximum signal value since the creation of the control or since the last activation of the reset button. The bar display consists of three elements:

- A scale from the minimum to the maximum signal value allowed according to the signal database.
- A signal value display in the form of a bar display. In the example: 20.33.
- Another representation of the Min/Max display in the form of a line underneath the scale. In the example: from 0.15 to 20.33.

Press the `Reset` button to reset the Min/Max display and the bar display.



The Min/Max display gets updated with the same rate as the whole workpage (approximately with a rate of 100ms). Normally the model rate is much faster. This means, that it does not detect any spikes in the signal. Additionally the Min/Max display gets only updated when the page of the workpage is visible. For having an absolutely exact min/max detection, start a data acquisition with the model rate and examine the signal in the offline mode (see chapter ***4.5 The offline mode***).

4.4.6.1 The property dialog of the number control

Additional to the height, the width and the unit (see chapter ***4.4.1.1 The units converter***) you can use the radio buttons in the groups `Label position` and `Value position` on the `Display` property page to adjust the control layout. The check boxes `Min/Max display` and `Bar Display` are used to switch these display components on and off. The buttons `Background`, `Text`, `Min` and `Max` are used to define the colors of the control. The colors `Min` and `Max` are used when the signal value exceeds the maximum value or drops below the minimum value specified in the signal database.

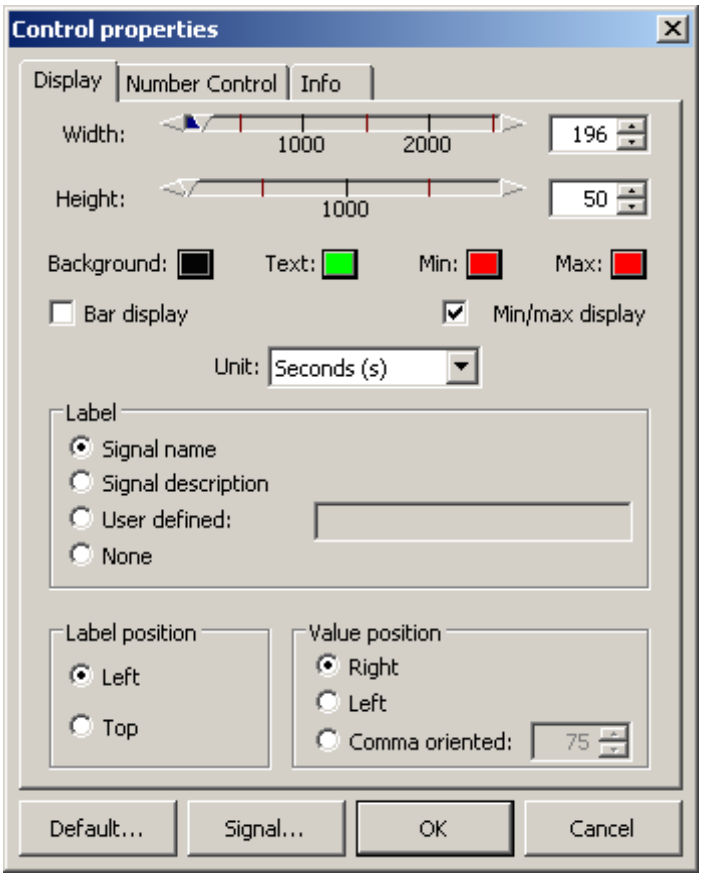


Figure 173 – Property page Display

The `Number Control` property page is used to configure the way the signal value is indicated: `Binary`, `Decimal`, `Decimal (scientific notation)` or `Hexadecimal`. When selecting `Binary` or `Hexadecimal` you can choose the minimal amount of digits to display. When selecting `Decimal` you can change the amount of decimal places to display only for floating-point signals. In case of selecting `Decimal (scientific notation)`, big or small signal values get displayed using the ‘e’ notation and you can select the amount of significant digits to display. Please be aware that a conversion to a floating-point value happens for that case.

Please read the chapter ***4.3.11 Hexadecimal representation of signal values*** for more information about using `Binary` or `Hexadecimal`.

In case you activate the check box `Use text table of the signal` and your signal has an assigned text table (displayed on the `Info` tab, a table with value range to text assignments), the control displays the according text of the table in case the signal values matches an entry. Otherwise it displays the number. In case you deactivate the check box, it always displays the signal value as a number.

Use the `Font` combo box to select the font used to paint the value.

Activate the check box `Select the bits to display` to restrict the displayed bits otherwise all bits are displayed. This check box is not available for floating-point signals. The bit extraction happens before the unit conversion if that is selected.

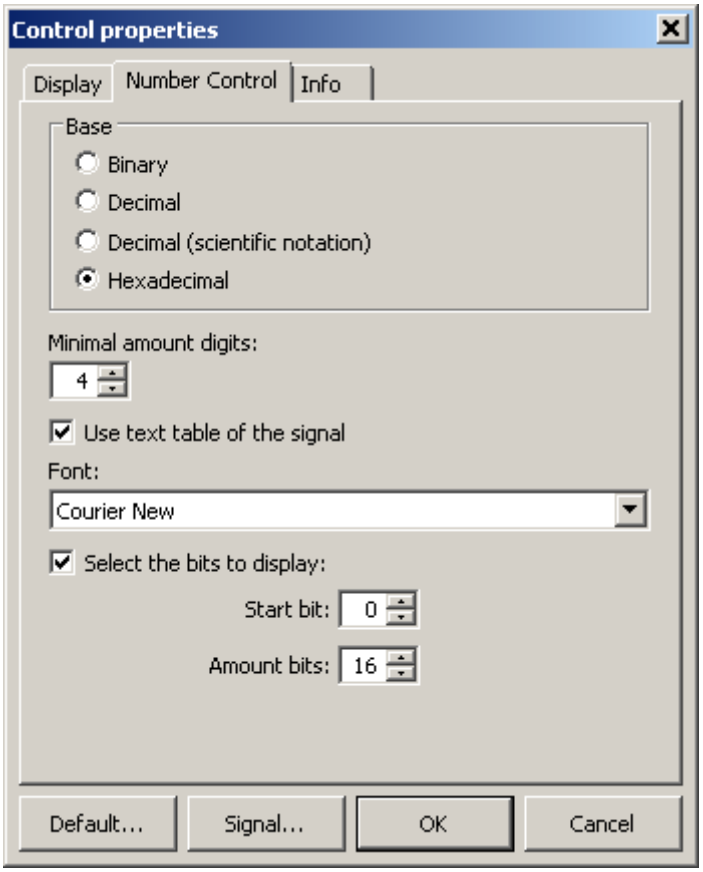


Figure 174 – Property page Number Control

4.4.7 The bitmap control

The bitmap control (***Figure 175***) displays any images or texts associated with signal values. The corresponding button in the Signal Selection window (chapter ***4.2 The Signal Selection window***) has the following appearance:  .

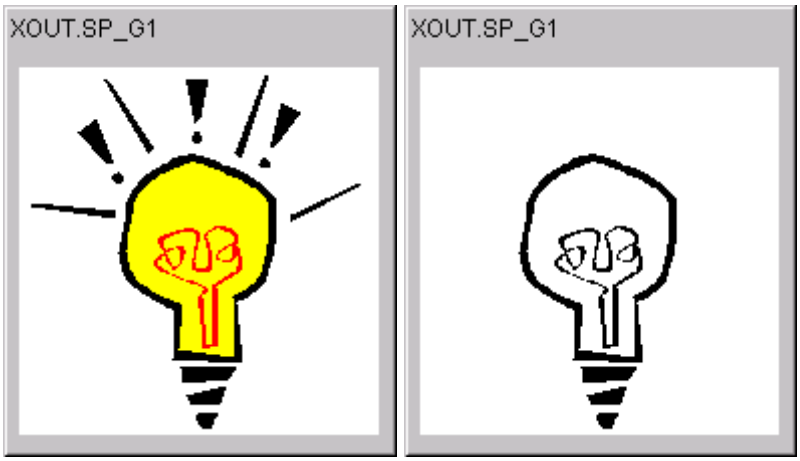


Figure 175 – The bitmap control

The images / texts to be displayed are assigned to the signal values via the property dialog (chapter ***4.4.7.1 The property dialog of the bitmap control***).

4.4.7.1 The property dialog of the bitmap control

The `Display` property page is used to define the usual (size, colors and label) parameters. The `Control is always in the back` check box determines whether or not the control is supposed to appear underneath other controls in the case of an overlap. Deactivate the `Frame` check box to hide the frame around the control. These settings may be helpful if you wish to combine several bitmap controls to form a larger picture.

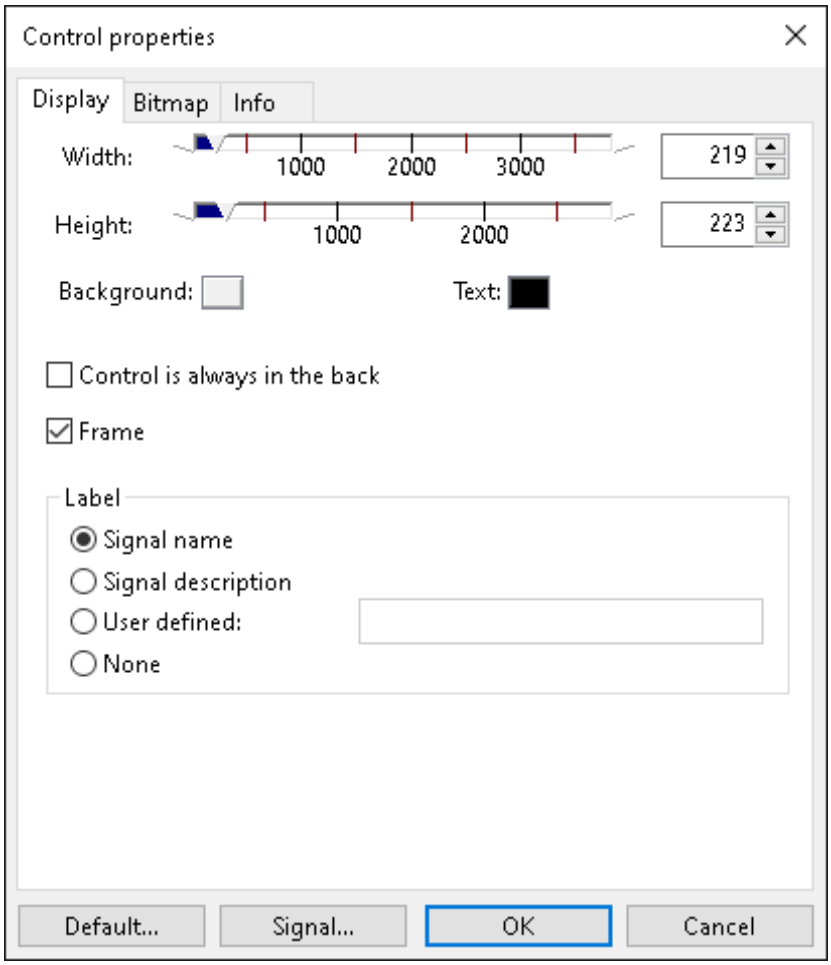


Figure 176 – Property page Display

On the `Bitmap` property page you can define different images or text strings to display for different signal values. Activate the `Centered` radio button to have the images or texts centered or select `User defined` to edit the coordinates manually.

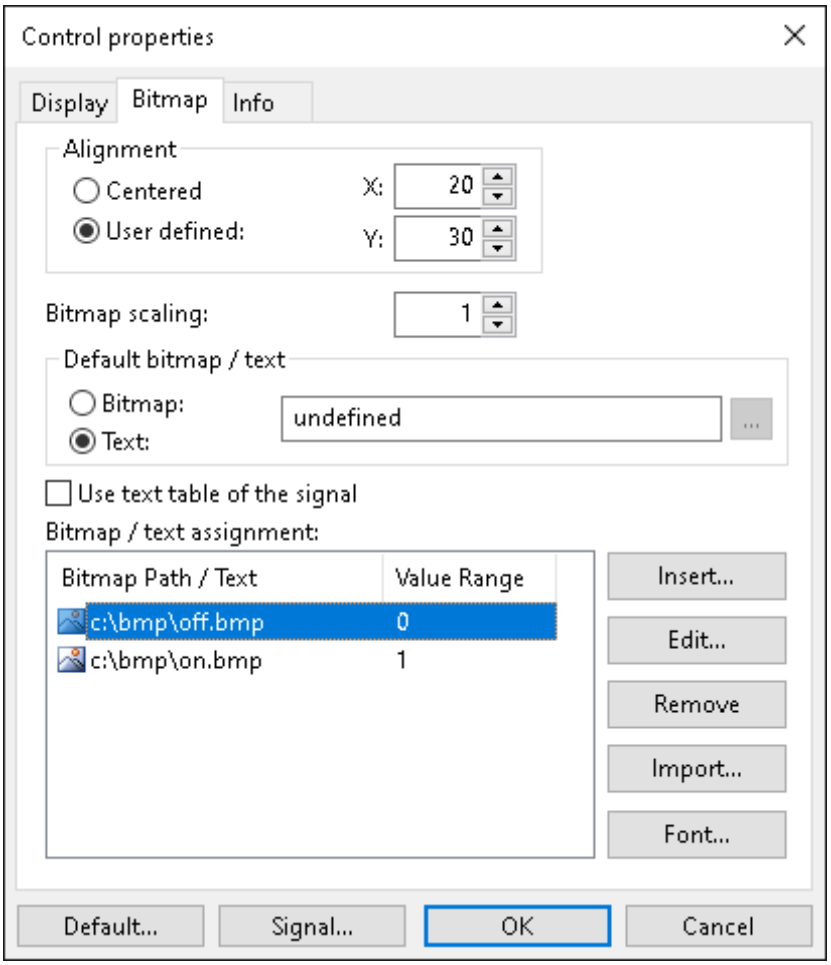


Figure 177 – Property page Bitmap

The `Bitmap scaling` field determines the current scaling of bitmaps while also taking the view scaling into account (see chapter ***4.1.1 View scaling***). A value of 1 means to display in native bitmap resolution (no scaling at all) for the currently selected view scaling.

Use the edit box `Default bitmap / text` to define what to display when none of the rules below matches. Press the radio button `Bitmap` and enter a path to a bitmap file with the extension .bmp, .gif, .jpg, .png or .tif to display a bitmap. Press the ... button to select a bitmap file via the file selection dialog (see chapter ***4.3.10 Managing bitmaps***). Choose the radio button `Text` to display the text entered in the edit box as default.

The left column of the `Bitmap / text assignment` list box shows the paths to the bitmaps (icon) or the texts (icon) to be displayed. The right column shows the associated value ranges. When the signal value is inside the range, the control shows the according bitmap or text. Click on `Insert...`, `Edit...` or `Remove` to insert, edit or delete an entry. The chapter ***4.4.1.2 Import of text tables*** describes the `Import...` button.

In case the model assigned a text table to the used signal, you can activate the check box `Use text table of the signal` as an alternative. In this case the list fills automatically using the according table values and you cannot edit it.

The `Font…` button is used to select the font for the text to be output. The following dialog is displayed when you click on `Insert…` or `Edit…`:

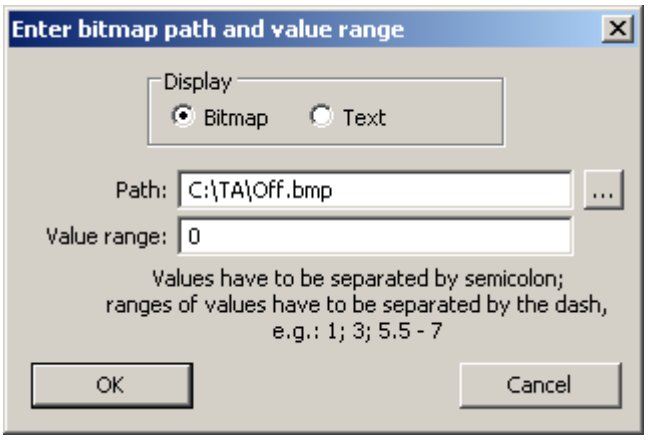


Figure 178 – Editing of a range assignment for an image

Select the path to the file to be displayed by pressing the … button. Enter the range assignments in the `Value Range` edit box. Some examples follow:

From -50 to –25.3:	-50 - -25.3
From 0 to 13 and from 18 to infinite:	0 - 13; 18 - inf
Prime numbers from 1 to 10:	1; 2; 3; 5; 7
From -infinite to 5 and 10.5:	-inf - 5; 10.5

If you wish to display a text instead of an image, click on the `Text` radio button, and the appearance of the dialog changes (***Figure 179***). Enter the text to be displayed in the `Text` edit box in place of an image path.

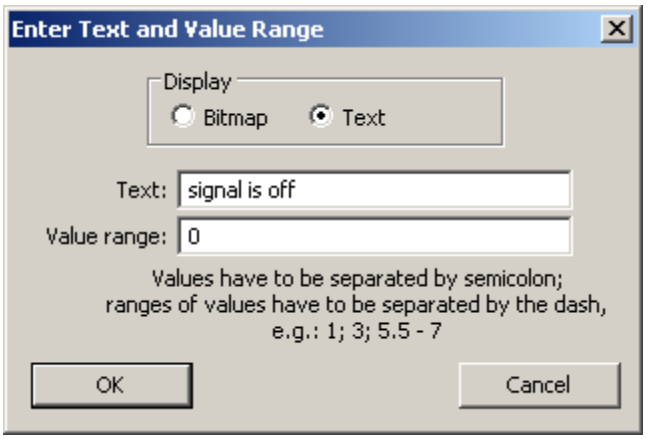


Figure 179 – Editing of a range assignment for a text

PROVEtech:TA supports bitmaps with alpha channel, which means, that the bitmap contains a transparency value for each pixel beside the color value. Every semi transparent pixel gets mixed with the background color. Completely transparent pixels are not drawn.

4.4.8 The bitswitch control

The bitswitch control (***Figure 180***, ***Figure 181***) displays the individual bits of a signal. The colored fields of the figures show the bit values, the numbers above them the bit position within the signal value. The bitswitch control is represented by the symbol  in the Signal Selection window (see chapter ***4.2 The Signal Selection window***). The bitswitch control behaves differently for changeable and non-changeable signals. In the case of changeable signals you can click on the individual bits to switch them.



Figure 180 – The bitswitch control for a read only signal

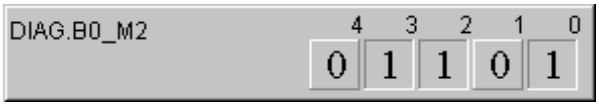


Figure 181 – The bitswitch control for a changeable signal

4.4.8.1 The property dialog of the bitswitch control

In addition to setting the standard properties, the radio buttons in the groups `Label position` and `Bitfield position` on the `Display` property page are used to define the positions of the label and the bitfield display in the control.

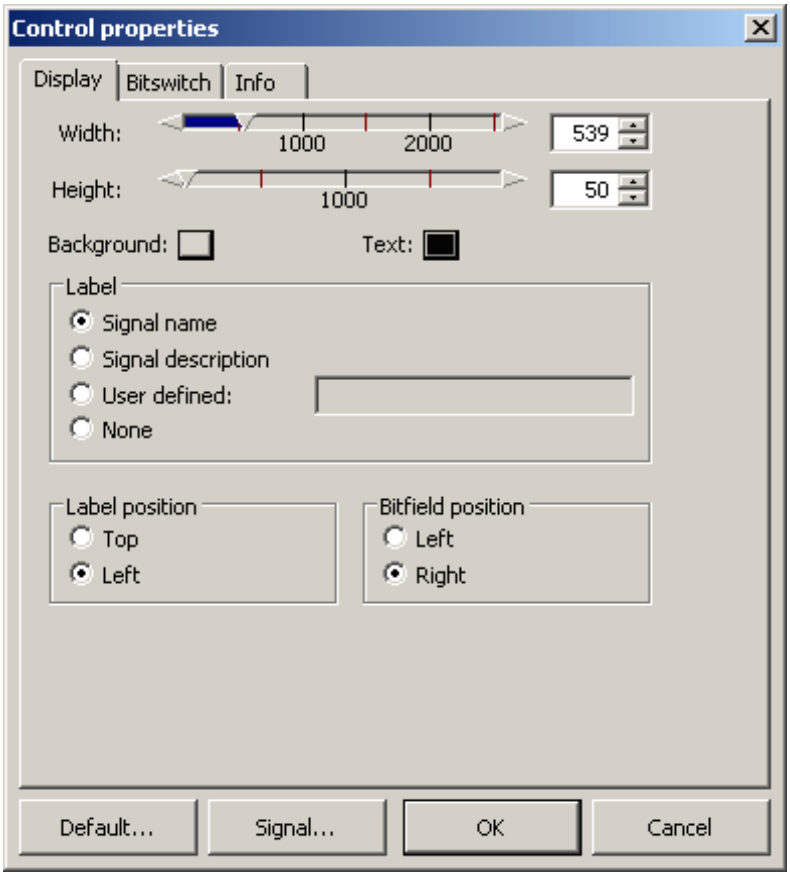


Figure 182 – Property page Display

The property page `Bitswitch` is used to select the bits to be displayed and their appearance:

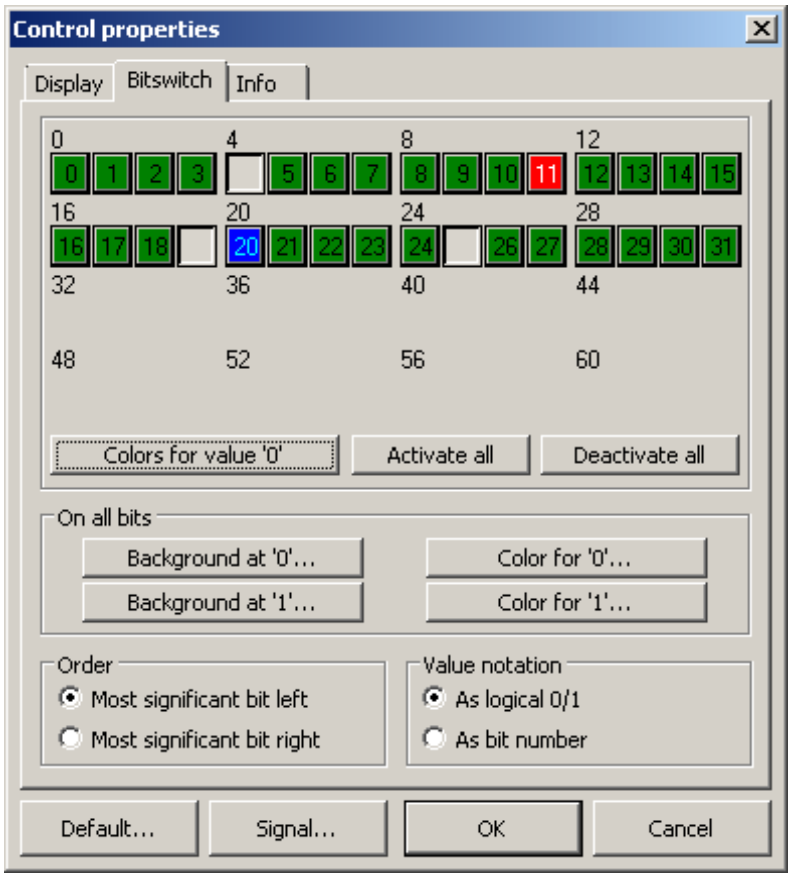


Figure 183 – Property page Bitswitch

Click on the buttons in the upper section to activate and deactivate individual bits for display in the bitswitch control. Activate several bits at the same time by drawing a rectangle around the buttons in question. The number of bits specified for the signal in the signal database determines the number of buttons displayed (a maximum of 64). The numbers on and above the buttons correspond to the bit numbers. The buttons `Activate all` and `Deactivate all` select / deselect all bits. The button `Colors for value '0'` is used to switch the representation of all the bit buttons above. These show a preview of the colors defined.

Four settings for the color selection are available:

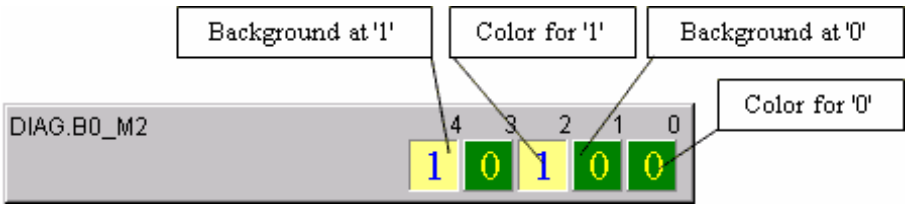


Figure 184 – Color settings for the bitswitch control

Click on a bit button with the right mouse button to choose individual color settings for the bits.

The radio buttons in the group `Value notation` determine the labels of the bit fields in the control. The setting `As logical 0/1` is shown in ***Figure 184***. If set to `As bit number`, the bit numbers are displayed in the fields in place of ‚0' or ‚1':



Figure 185 – Effect of the "As Bit-Number" setting

The `Order` group is used to determine whether the most significant bit in the bitswitch control is to be on the left or right side.

4.4.9 The gauge

The gauge (***Figure 186***) corresponds to a speedometer display. The corresponding button in the Signal Selection window (chapter ***4.2 The Signal Selection window***) has the following appearance: 



Figure 186 – The gauge

If you work with a changeable signal, you can move the pointer to change the signal value.

4.4.9.1 The property dialog of the gauge

The `Display` property page includes only standard settings. Refer to the chapter ***4.4.1 General properties*** for further information. Select the `Gauge` property page to configure the gauge display:

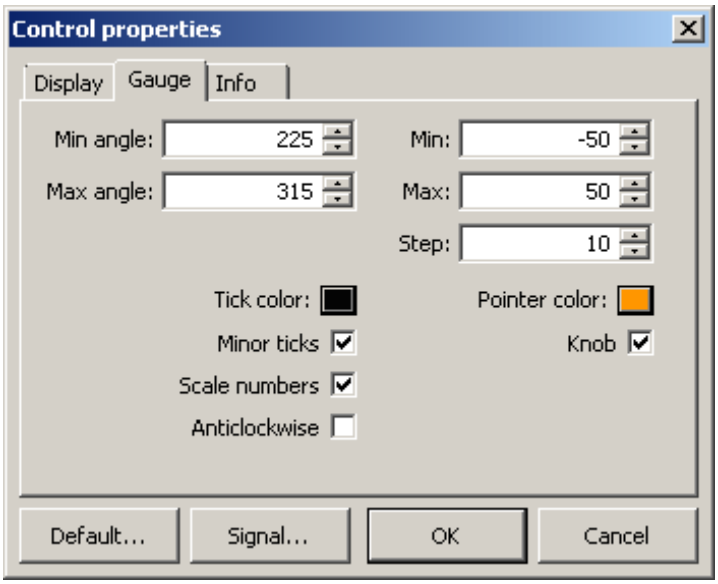


Figure 187 – Property page Gauge

The edit boxes `Min angle` and `Max angle` are used to define the beginning and the end of the scale in degrees. The usual angle definitions apply:

Right: 0° and 360°
Top: 90°
Left: 180°
Bottom: 270°

The edit boxes `Min` and `Max` are used to define the display range of your signal values. The step size of the axis label is defined in the `Step` edit box. Change the colors via `Tick color` and `Pointer color`. The three check boxes `Minor ticks`, `Scale numbers` and `Knob` are used to define which display elements are to be shown. To change the scaling direction, activate the `Anticlockwise` check box.

4.4.10 The grid control

The grid control (***Figure 188***) displays vectors and matrices in table form. The associated button in the signal selection window (chapter ***4.2 The Signal Selection window***) looks like this:



4x4 Matrix

	0	1	2	3
0	1.00	0.00	6.20	0.00
1	0.00	2.00	0.00	5.00
2	8.50	0.00	3.00	0.00
3	0.00	44.20	0.00	4.00

Figure 188 – The grid control

In the following text the term “matrix” also includes vectors.

You can create a grid control on the workpage by using drag and drop or copy and paste on an element (a single signal) of the matrix. The grid control then automatically displays the whole matrix. You can also select the complete matrix node in the signal selection window and insert it as a grid control.

The grid control provides a scroll-bar when all the items of the matrix cannot be displayed simultaneously. In ***Figure 188*** the gray row displays the column indexes and the gray column displays the row indexes of the matrix. For vectors the column index display does not apply, just as the row index display does not apply for transposed matrices (see chapter ***4.4.10.1 The property dialog of the grid control***). The grid control can also display a single signal, but in this case a different control type would probably be better suited.

Any writeable signals that appear in the matrix can be edited. To do so simply click on the signal and enter the value in the text field that appears. The following keyboard keys can be used to navigate within the matrix:

Tab	Move an element to the right
Shift+Tab	Move an element to the left
Up-Arrow key	Move an element up
Down-Arrow key	Move an element down
Shift+Home	Go to the first element in the row
Ctrl+Home	Go to the first element in the column
Shift+End	Go to the last element in the row
Ctrl+End	Go to the last element in the column
Page Up	Go one page up
Page Down	Go one page down
Enter	Accept new signal value and leave edit mode
Esc	Leave edit mode without accepting new signal value

4.4.10.1 The property dialog of the grid control

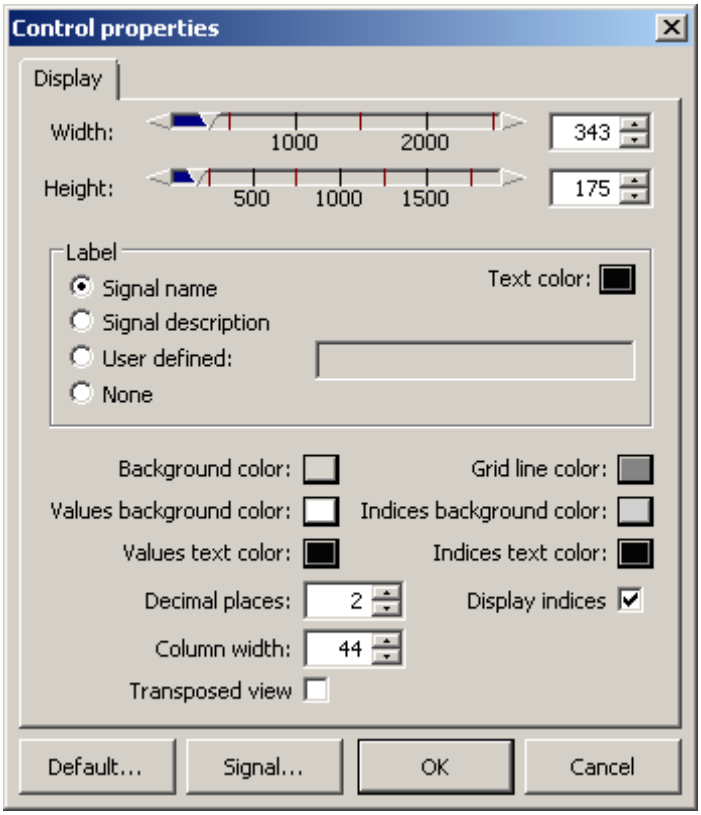


Figure 189 – Property dialog of the grid control

In the property dialog of the grid control, apart from the standard `Width`, `Height` and `Label` settings, the following additional settings have been provided:

`Text color`: Color of the label
`Background color`: Background color of elements outside the table
`Values background color`: Background color of the table for signal values
`Values text color`: Color of the grid for signal values
`Grid line color`: Color of the table lines
`Indices background color`: Background color of the index row and column
`Indices text color`: Color of the text in the index row and column
`Display indices`: Enables/disables the display of the index row and column (always disabled for scalar signals)
`Decimal places`: Number of digits to display after the decimal point for a signal values (only available for floating-point values)
`Column width`: Width of the signal value columns (in Pixels). To adjust a column width directly on the control, place the mouse cursor between two columns, click the mouse button, drag to the desired width and release the mouse button.
`Transposed view`: Activate this check box to swap the position of the rows and columns. For a deactivates check box the element (1,0) is below (0,0) and (0,1) is right of (0,0). For an activated check box, it is the other way around.

4.4.11 The surface graph control

The surface graph control can only display matrices in 3-dimensional form. The associated button in the signal selection window (chapter ***4.2 The Signal Selection window***) looks like this: 

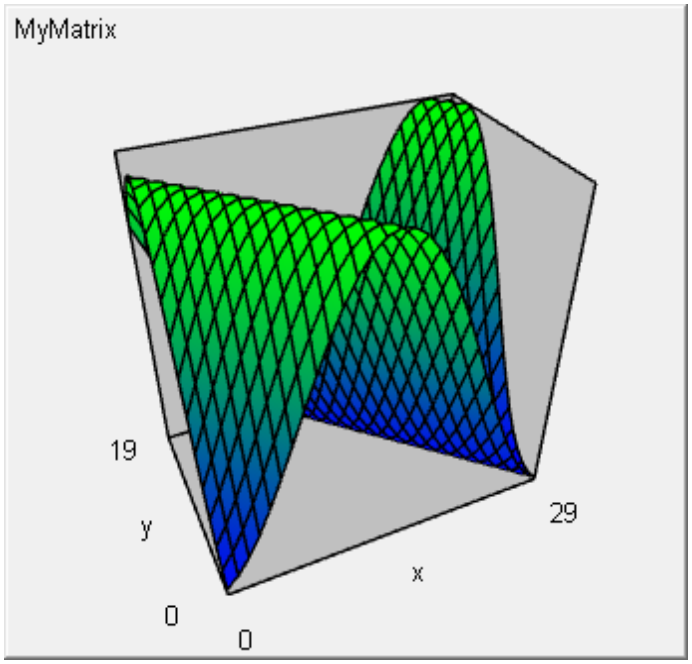


Figure 190 – Example for a surface graph control

You can create a surface graph control on the workpage by using drag and drop or copy and paste on an element (a single signal) of the matrix. The surface graph control then automatically displays the whole matrix. You can also select the complete matrix node in the signal selection window and insert it as a surface graph control.

If you try to create a surface graph control on anything else then a matrix, the display is grayed.

The first dimension of your signal matrix is the x coordinate and the second one the y coordinate. The z coordinate represents the signal values. All axes are always linear. Only the x and y axes have optional displayed labels. Place the mouse cursor on a grid point of the surface graph to display the current according signal value in a tooltip. Every grid point represents an element of your signal matrix.

This display uses a right-handed coordinate system. To change the view position, click the right or middle mouse button on the control and hold it. Then move your mouse left and right to rotate the view and up and down to tilt it. Rotate the mouse wheel to control the zoom.

As an alternative you can hold the Ctrl key and then rotate the mouse wheel for zooming.

You can change writable Signals in a surface graph control by clicking on a grid point (=signal value) with the left mouse button and then moving it in direction of the z axis while holding the mouse button. Meanwhile, the tooltip always shows the current signal value. The control limits the signal changes to the selected range of the z axis.

4.4.11.1 The property dialog of the surface graph control

In addition to setting the standard properties, this property dialog lets you choose the axes ranges. For the X and the Y axis the range specifies which part of the signal matrix to display (`X/Y axis from … to …`). For the Z axis the range corresponds to the signal value range to display (`Z axis from … to …`). You can use the two Z axis color buttons to specify the color to display when the signal value is exactly on one of the range boundaries or outside. In between the control mixes these two colors.

The color buttons for the axes labels, the grid lines, the back planes and the back lines have additional check boxes. Use them to show/hide the according elements in the control.

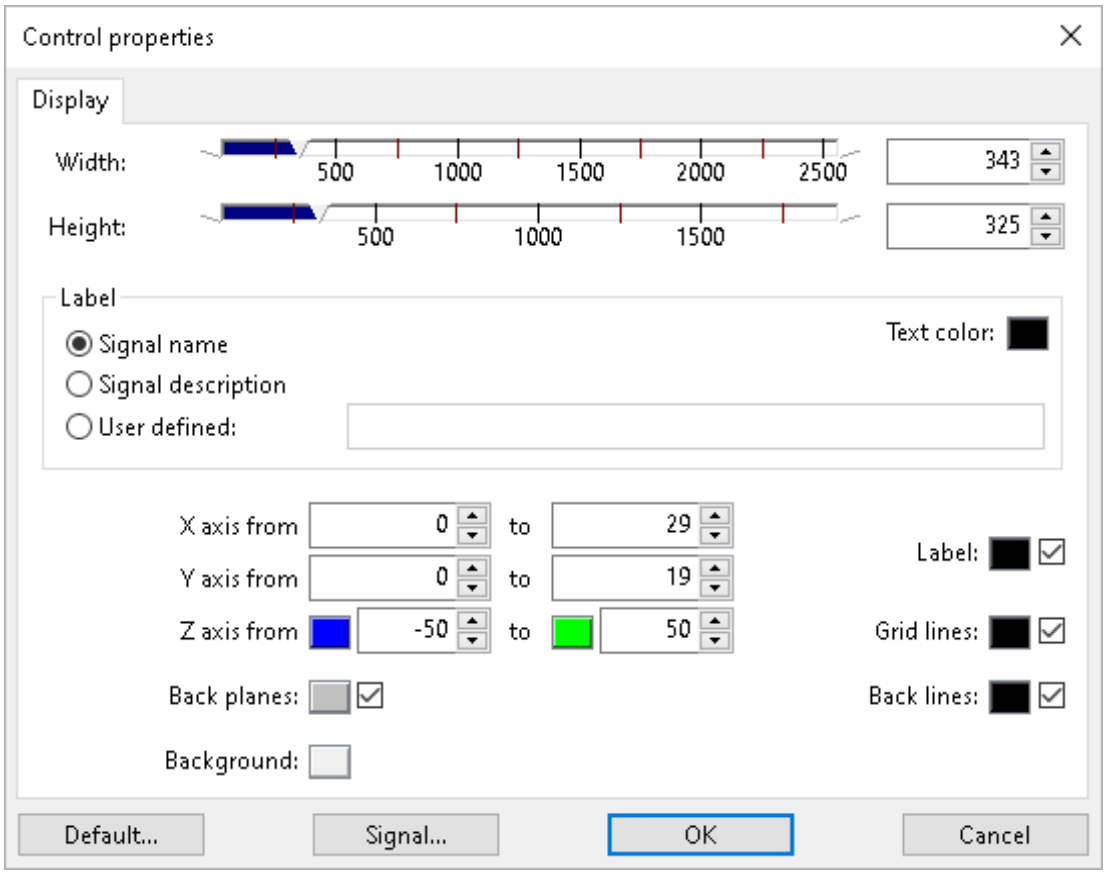


Figure 191 – Property dialog of the surface graph control

4.4.12 The custom control

You can configure your own controls with the custom control (button in the signal selection window, chapter ***4.2 The Signal Selection window***: ). It consists of any amount of embedded controls. ***Figure 192*** shows an example of a custom control for the network management.



Figure 192 – Example for a custom control

4.4.12.1 The property dialog of the custom control

The property dialog of a custom control contains the property pages `Display`, `Info` (see ***4.4.1 General properties***) and `Embedded Controls`. Additionally to the normal settings the page `Display` has the check box `Signal tooltip active`. When your mouse cursor stays on a control for some time, a tooltip displays the according signals. Deactivate this check box to suppress the tooltip.

The property page `Embedded Controls` contains all settings of the embedded controls (see ***Figure 193***). The tree view on the left shows a leaf per embedded control. The leaf's icon represents the control type (see ***Figure 194***). The optional folders () are useful for grouping and structuring the controls, but they are invisible in the custom control. The tree entry names are just for finding them easier, but they have no meaning. Press F2 or double-click slowly with the mouse to rename them. Empty leaf names get replaced by the control type name when closing and re-opening the dialog. Empty folders get removed when saving. The 2nd column shows signal names, macro paths, etc. for the controls.

Place the mouse cursor directly right of the tree view to resize the tree view via drag and drop (splitter). You can also enlarge the whole dialog with the mouse.

The order of the tree entries corresponds to the Z order of the controls. A button whose entry is above the entry of another button is displayed in front of that other button. Active controls (e.g. sliders) are always in front of inactive controls (e.g. bitmaps and texts). Active controls have blue text in the tree to distinguish them. Red text marks invalid controls where the specified signal is missing as an example. Use drag and drop to change the order of the tree entries:

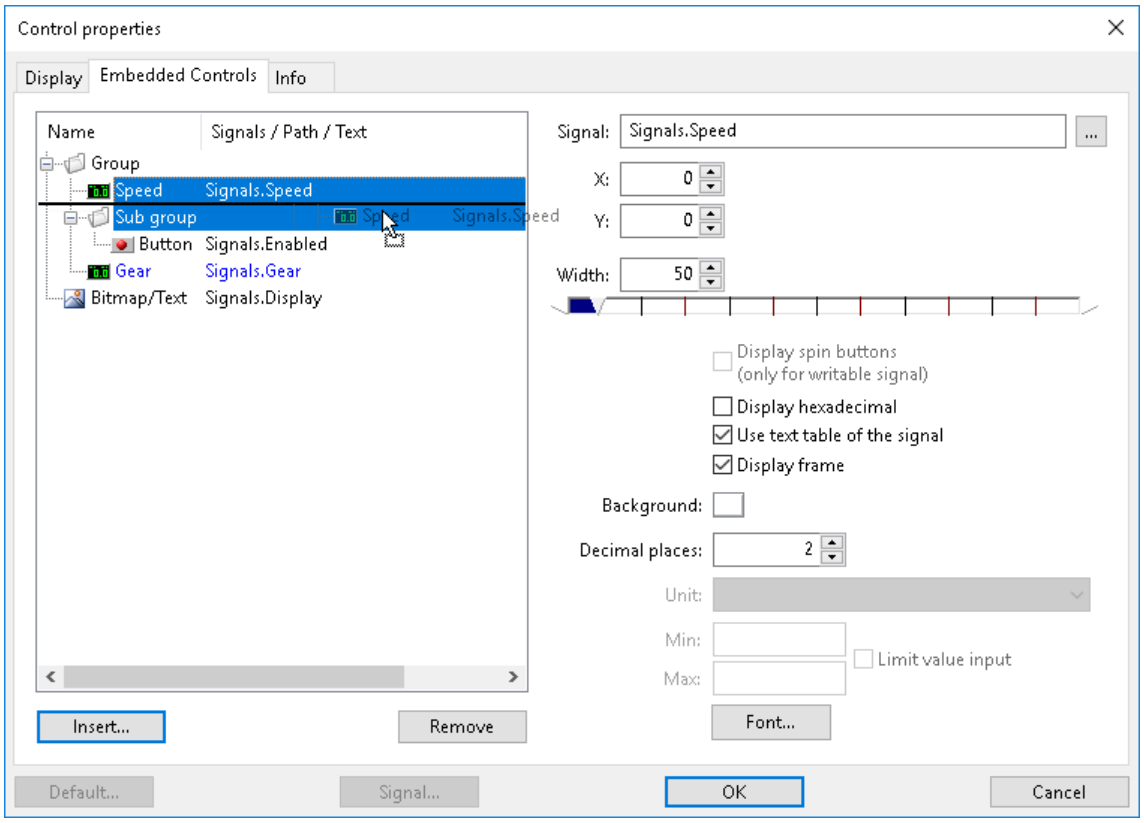


Figure 193 – Drag and drop of list entries

Hold the Ctrl key while releasing the mouse button to copy the entry instead of moving it.

Select the tree entry of a control and click on `Remove` to remove the selected entry. Click on `Insert...` to insert a new control before the selected list entry. In case it is a folder, the new entry appears within the folder. PROVEtech:TA then displays a dialog to select the new control type:

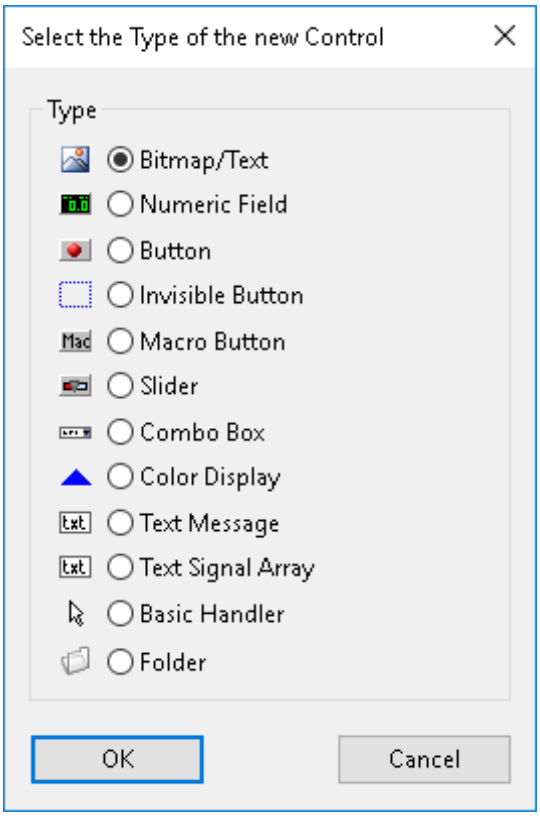


Figure 194 – Choosing the type of a new control

You can also use the context menu to add or remove entries.

When you select a tree item, PROVEtech:TA marks the according sub control in the custom control with gray blue rectangles:

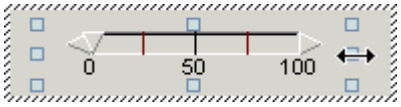


Figure 195 – Marked control in the custom control

Click on a square with the mouse and drag it while holding the mouse button to resize the control. Some control types just support horizontal resizing. The Bitmap / Text control does not support resizing.

When the mouse cursor is over the control (within the marked area), you can move the control via drag and drop. Hold down the Ctrl key while doing drag and drop for cloning a control.

You can also click on a sub control to select it. After that use the keyboard arrow keys to move the sub control pixel by pixel. This allows an exact positioning.



Use the context menu of a sub control or its according tree entry for copying it to the clipboard. After that you can paste it into the same custom control or a different one. This does not work for folders.

4.4.12.2 Bitmap / Text

The bitmap / text control is similar to the bitmap control (see ***4.4.7 The bitmap control***). However, it has much more functionality. You can specify seven optional signal names. If a signal name is specified but not existing, PROVEtech:TA displays a exclamation mark icon beside it (). Depending on which signal it is, this embedded control might be not drawn at all.

In case you specify the `X position` and/or the `Y position` signal, you can let the currently displayed bitmap or text move within the custom control. As an example, this way you can let the bitmap of a car move along a street map bitmap. The unit of this movement is screen pixels. Use `Scale` and `Offset` to convert the signal value to the screen pixels.

If you do not specify the X and/or Y position signal, you can only change the according `Offset` field(s) for the absolute positioning of the current bitmap or text. The marker square displayed on the custom control while editing shows this absolute offset position.

Activate the `Centered output` check box to use the calculated X and Y position as the center for the text or bitmap output. Otherwise it is the upper left corner of the bitmap or text.

The groups `X scaling` and `Y scaling` scale the bitmap dynamically in X and/or Y direction. Additional to the signal there are the fields `Scale` and `Offset`. The formula signal value*`Scale+Offset` calculates the actual scaling value for the bitmap. A list of interesting value ranges and their results follows:
> 1: Enlarges the bitmap
1: No change
> 0 and < 1: Narrows the bitmap
0: Bitmap invisible
> -1 and < 0: Narrows and mirrors the bitmap
-1: Mirrors the bitmap
< -1: Enlarges and mirrors the bitmap

The `Offset` values for the X and Y scaling are available even without specifying scaling signals. This way you can specify a base scaling of the bitmaps. As an example, your bitmap shall use its native resolution at 150 % view scaling (see chapter ***4.1.1 View scaling***). To do that, specify 0.66667 for the two scaling offsets.

The signal of the `Rotation` group specifies a rotation angle for bitmaps, e.g. a hand of an instrument cluster. The rotation does not work on texts. Beside the signal name you can specify a `Scale` and an `Offset` for the angle. The result is an angle with the unit degrees. The

rotation works counter clockwise (invertible via negative scale) and 360° is a full rotation. The Rotation point X and Y determine the rotation center point in pixels correlating to the bitmap (deactivated in case Centered output is active). You can use decimal places here and the point can be outside the bitmap.

The scaling of the bitmap (X scaling and Y scaling) uses the specified Rotation point X and Y as origin point as well (or uses the Centered output). Scaling happens before the rotation.

In case you configure the group Alpha, the field Signal specifies the name of a signal for manipulating the transparency of the whole bitmap. The fields Value for 0% opacity and 100% define its value range from complete transparency to complete opacity. Values lower or greater are rounded to these boundaries. Values between the boundaries set a semi transparency where every pixel gets mixed with the according background pixel.

The group Default bitmap / text specifies the text or bitmap to display in case the Bitmap / text assignment via Signal field is empty or the according signal does not exist or the value is not covered by the lower list. In that list you can specify signal value ranges (column Value Range) and assign them to different bitmaps and/or texts (column Bitmap Path / Text). Click Insert... to add a new entry, Edit... to modify an existing one or Remove to delete an entry. The chapter ***4.3.10 Managing bitmaps*** provides additional information about managing bitmaps on the workpage. The chapter ***4.4.1.2 Import of text tables*** describes the Import... button.

In case the model assigned a text table to the used signal, you can activate the check box Use text table of the signal as an alternative. In this case the list fills automatically using the according table values and you cannot edit it.

Use the Font... button to select a new font for the text output.

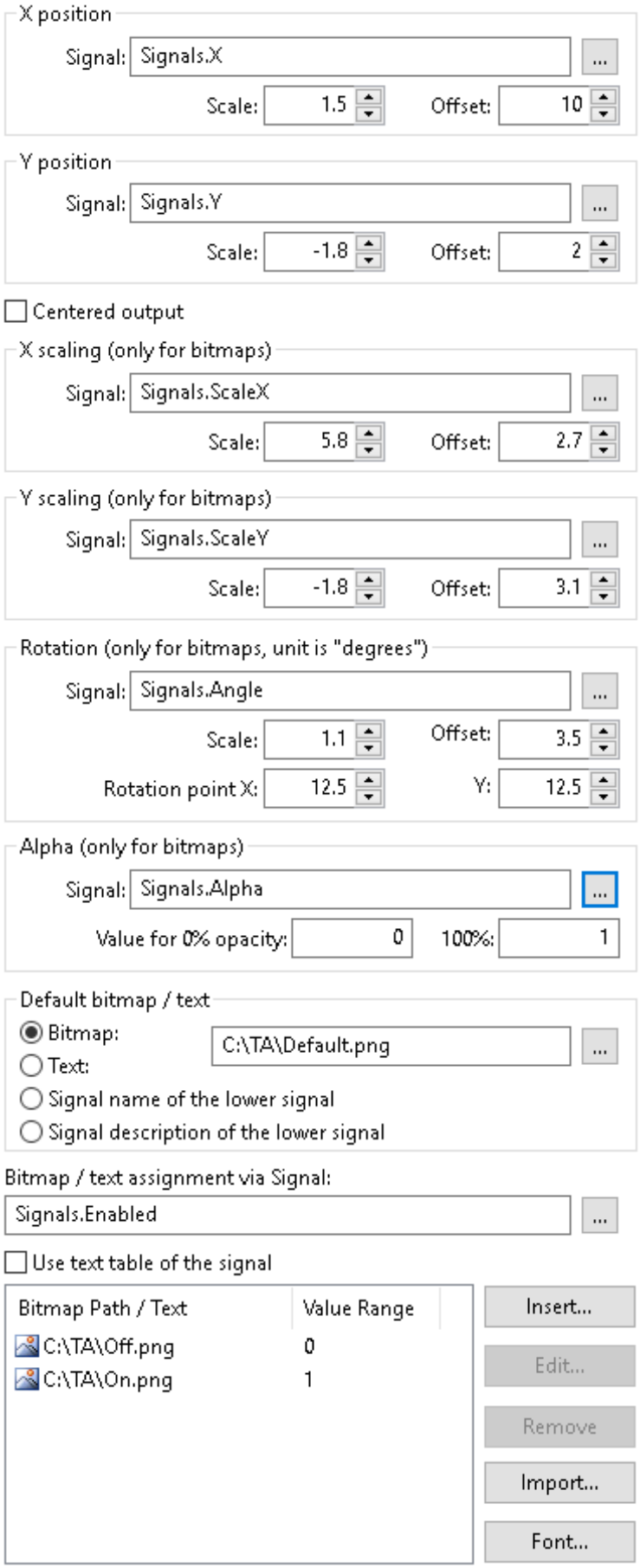

X position
Signal: Signals.X
Scale: 1.5
Offset: 10
Y position
Signal: Signals.Y
Scale: -1.8
Offset: 2
Centered output
X scaling (only for bitmaps)
Signal: Signals.ScaleX
Scale: 5.8
Offset: 2.7
Y scaling (only for bitmaps)
Signal: Signals.ScaleY
Scale: -1.8
Offset: 3.1
Rotation (only for bitmaps, unit is "degrees")
Signal: Signals.Angle
Scale: 1.1
Offset: 3.5
Rotation point X: 12.5
Y: 12.5
Alpha (only for bitmaps)
Signal: Signals.Alpha
Value for 0% opacity: 0
100%: 1
Default bitmap / text
Bitmap:
Text:
C:\TA\Default.png
Signal name of the lower signal
Signal description of the lower signal
Bitmap / text assignment via Signal:
Signals.Enabled
Use text table of the signal
Bitmap Path / Text
Value Range
C:\TA\Off.png
0
C:\TA\On.png
1
Insert...
Edit...
Remove
Import...
Font...

Figure 196 – Page of a bitmap / text control



PROVEtech:TA supports bitmaps with alpha channel, which means, that the bitmap contains a transparency value for each pixel beside the color value. Every semi transparent pixel gets mixed with the according background pixel. Completely transparent pixels are not drawn. This also works in combination with the `Alpha signal` setting.



The `Alpha signal` group only works in combination with bitmaps that have an alpha channel.

4.4.12.3 Numeric Field

A signal in- / output consists of a frame that contains the numerical output of a signal value:



Figure 197 – Signal value in- / output

On the corresponding page (***Figure 198***) you first have to enter the name of the linked signal in the edit box `Signal`. Choose the upper left corner of the control inside the custom control via `X` and `Y` and also change its `Width`. The height results from the font you select via the `Font...` button. Additionally you can change the amount of decimal places, the unit (see chapter ***4.4.1.1 The units converter***) and the background color.

For writable signals you can activate the check box `Display spin buttons`. This activates the buttons  for incrementing/decrementing the signal value step by step.

Activate the check box `Display hexadecimal` to display signal values in the hexadecimal format. Please read the chapter ***4.3.11 Hexadecimal representation of signal values*** for more information about that.

In case you activate the check box `Use text table of the signal` and your signal has an assigned text table (displayed on the `Info` tab, a table with value range to text assignments), the control displays the according text of the table in case the signal values matches an entry. Otherwise it displays the number. For writable signals you can also enter the text of a table entry and it will set the signal value to the lower boundary of the according entry. In case you deactivate the check box, it always displays the signal value as a number and you can only enter numbers.

In case the signal is writable, you can limit the range of entered values by activating `Limit value input` and entering the range in `Min` and `Max`.

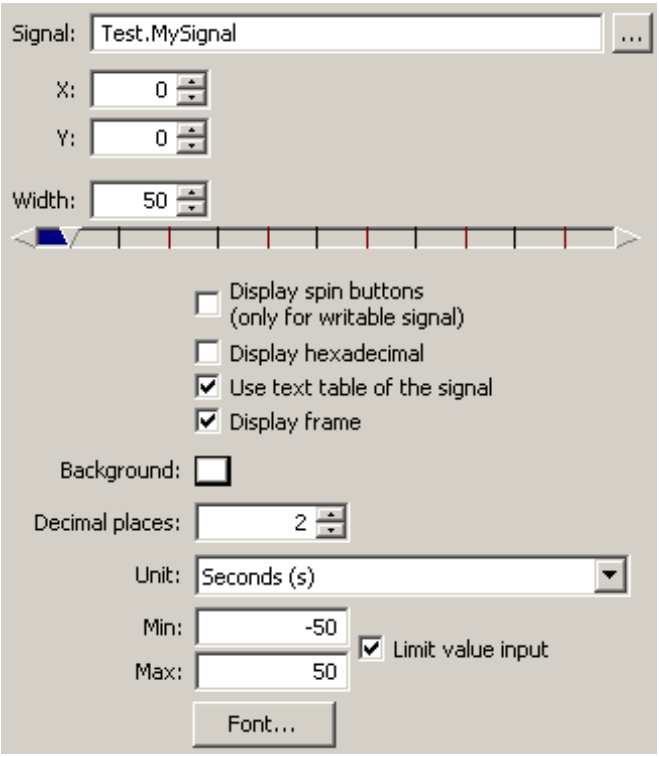


Figure 198 – Page of a signal value in- / output with a frame

In case you deactivate `Display frame` the layout of the page changes a little bit:

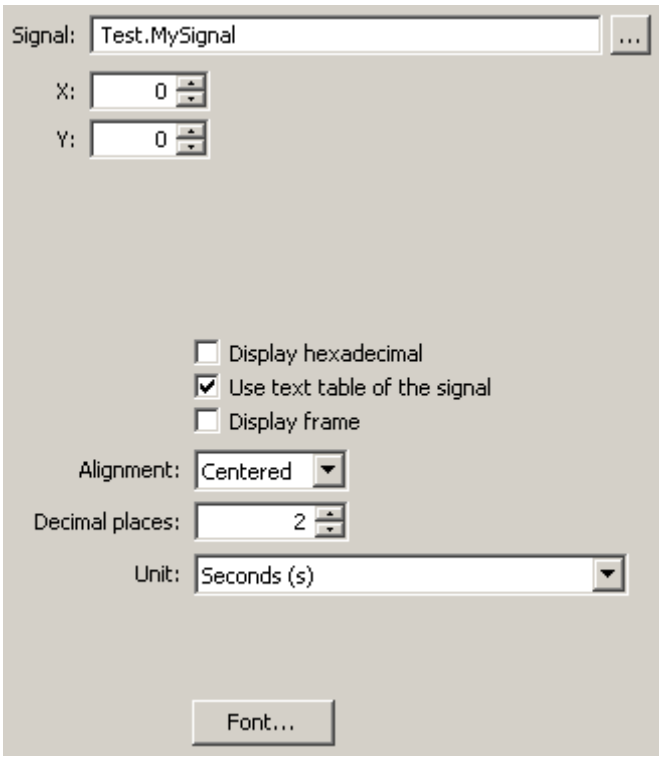


Figure 199 – Page of a signal value output without a frame

Now the text gets directly drawn without a background frame. Use the `Alignment` combo box to select if the horizontal output is `Centered`, `Left` or `Right` aligned to the position specified via `X` and `Y`. In this mode you cannot edit the value of writable signals.

See chapter ***4.4.1.1 The units converter*** for information about the `Unit` combo box.

4.4.12.4 Button

At first enter the name of the signal linked to the button in the `Signal` edit box of the page of a button control (***Figure 200***). Choose the upper left corner of the button inside the custom control via `X` and `Y` and also change its `Width` and `Height`. The content of the `Style` combo box changes depending on whether the signal is read only or not. Use it to change the look of button (see ***4.4.5.1 The property dialog of the switch / LED control***).

You can change the color and the text displayed on the button for both button states via the properties `On` and `Off`. Text output is only possible when selecting the style “Horizontal Slider With Text” or “Button With Text”. In this case you can also use the button `Font…` to change the text color and font. The edit boxes `Value` specify the signal values in the “on” and in the “off” state for writable signals. Normally the control displays the “on” state when the signal value differs from both values. To display the “off” state instead, activate the check box `Display "Off" state for dissenting values`. For read only signals, you can only edit the one value that is used for calculating the displayed state.

When it is a writable signal, you can use the radio buttons `Latched switch` and `Momentary switch` to define if the button keeps the “on” state after clicking on it or if it changes back to the “off” state after releasing the mouse button.

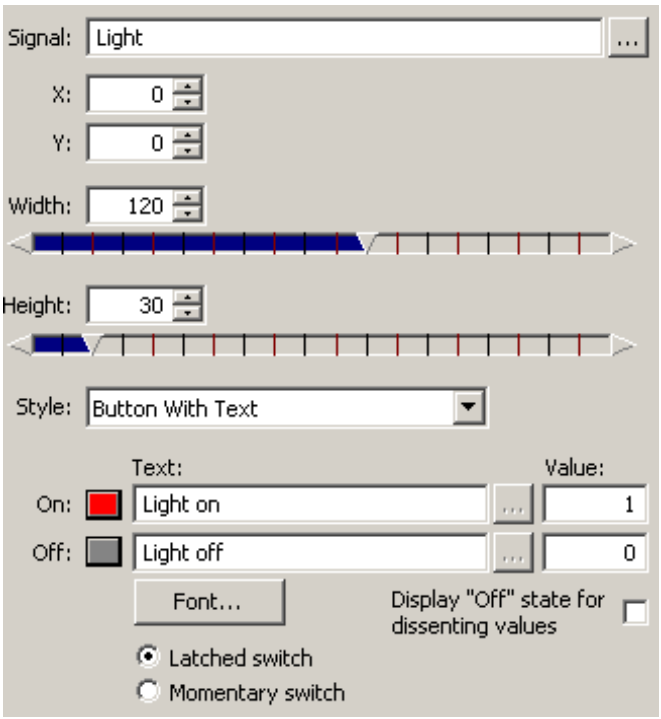


Figure 200 – Page of a button

A special case of the button is the setting “Bitmaps” in the `Style` combo box. In this case the page changes the way displayed in ***Figure 201***. Use the edit boxes `On` and `Off` to select two bitmaps for displaying the “on” and the “off” state. The chapter ***4.3.10 Managing bitmaps*** provides additional information about managing bitmaps on the workpage.

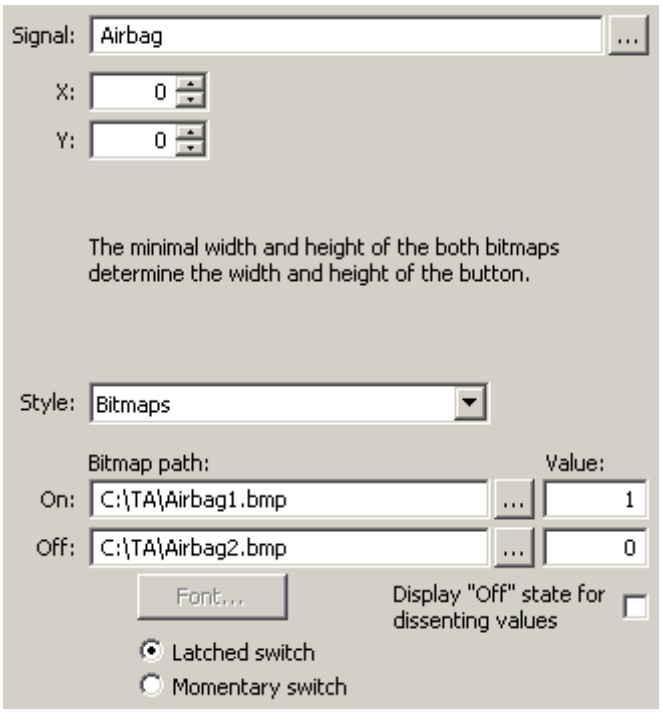


Figure 201 – Page of a button with bitmaps

4.4.12.5 Invisible Button

In opposite to the normal button, the invisible button has no own window. It only defines a rectangle area that changes a signal value when clicking on it with the mouse. In most cases you position it on top of a bitmap /text control to simulate photo realistic controls. In this case the signal of the bitmap / text control reacts on signal value changes of the invisible button. The Z sorting order of the invisible button works the same way as for the other control types (see above). It is an inactive control type.

Displaying the page of the invisible button (***Figure 202***) also activates a blue / red striped frame on the according custom control, which represents the area of the invisible button. You manipulate this area via `X`, `Y`, `Width` and `Height`. The radio buttons of the group `Type` define the behavior of the button when clicking on it:

Latches switch: The button toggles the signal value between `On Value` and `Off Value`.

Momentary switch: The signal switches to the `On Value` while clicking the left mouse button and switches back to the `Off Value` when releasing it.

Radio button: When clicking the left mouse button, the signal switches to the `On value`. The `Off value` is irrelevant in this case. As an example, this type is interesting for simulation a gear shift: You define an invisible button for each lever position that switches the same signal to different

values. Move the mouse over the different areas while holding the left mouse button to change the signal value.

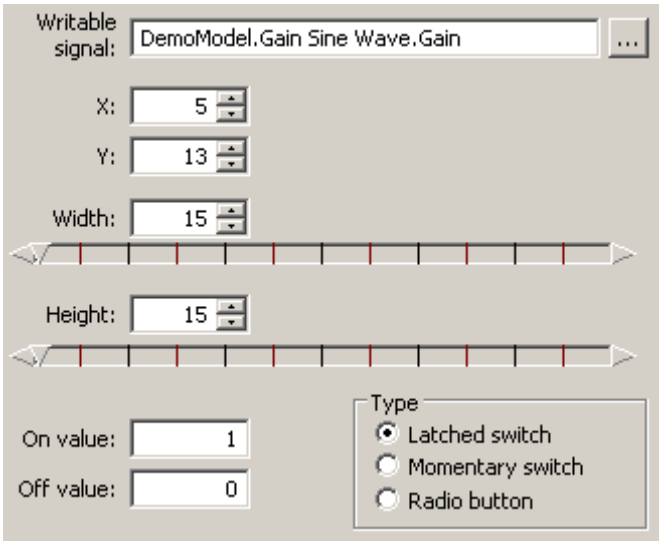


Figure 202 – Page of an invisible button

4.4.12.6 Macro Button

The macro button is completely different from all the other controls as it has nothing to do with signals, but starts a macro or test when pressing it. The execution is identical to the normal macro execution (see ***3.5 The Macro menu***) and you can abort the execution and detect the execution result via the toolbar. To visually distinguish the macro button from the other buttons on the workpage, it uses the normal Windows button control instead of an individually drawn button:



Figure 203 – Macro button

On its setting page (***Figure 204***) you can use the field `Macro` to enter the path to the Basic macro or test to start when pressing the macro button. Paths beginning with * are Test Manager paths (described in the test language manual) referencing tests in the database, others are file paths.

You can manipulate the position and size of the macro button within the custom control via `X`, `Y`, `Width` and `Height`.

The `Text` field defines the text displayed on the button and the `Icon` field the path to the bitmap file containing the icon to display left of the text (leave the path empty to display no icon). Very often icons are not rectangular, so you can use the `Transparent icon color` button to specify the color within the bitmap to be the transparent color. In case the

icon should not be transparent anywhere, specify a color that does not exist within the bitmap. The chapter ***4.3.10 Managing bitmaps*** provides additional information about managing bitmaps on the workpage.

Finally you can use the `Font...` button to specify the font for the text on the button.

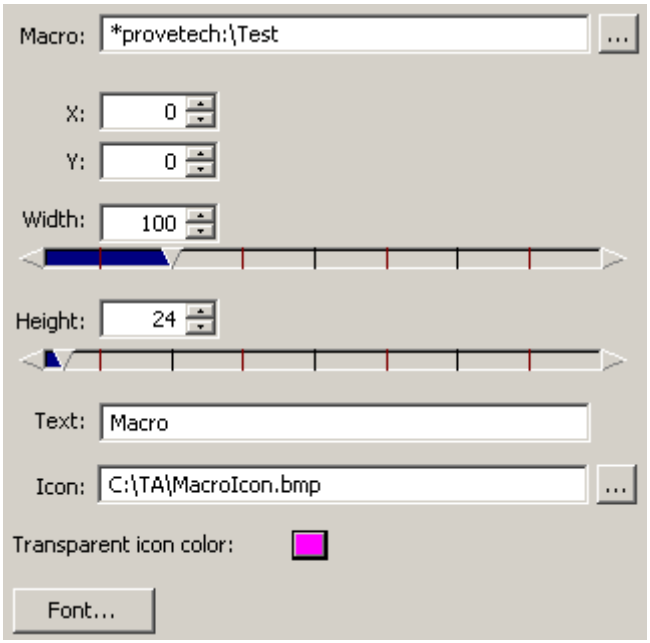


Figure 204 – Page of a macro button



The easiest way to create a macro button for a test in the Test Manager is to drag and drop or copy and paste it from the Test Manager tree to the workpage. It also works for pure text of an editor (e.g. the Basic editor) containing a Test Manager path. This creates a custom control containing a macro button referencing the test.



You can execute several macros and tests in parallel, but you are responsible for a useful usage of this possibility. For example it makes no sense to access signals from one macro while a 2nd one re-loads the model in parallel. Additionally the Automation Library cannot be used by several macros and tests in parallel.

4.4.12.7 Slider

Like the other elements the page of a slider (***Figure 205***) allows you to enter the name of the corresponding signal, the position (`X` and `Y`) and the size (`Width` and `Height`) of the slider inside the custom control. The slider is similar to a part of the working control. Therefore the other settings are similar (see ***4.4.4.1 The property dialog of the slider / bar display control***).

The group `Step size of the spin buttons` is disabled when using a read only signal as it makes no sense there.

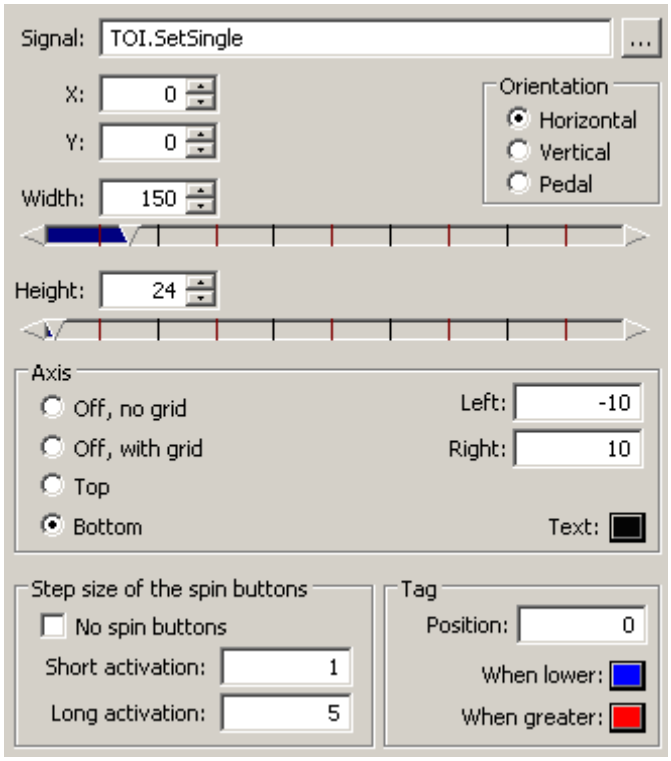


Figure 205 – Page of a slider

4.4.12.8 Combo Box

First of all choose the corresponding `Signal` on the page of the combo box (***Figure 206***), the position (`X` and `Y`) and the `Width` of the control inside the custom control. Then you can specify the mapping of the signal values to the displayed text via the `Combo box entries` list. Use the button on the right side to manipulate the list entries. As an alternative you can open the list's context menu with the right mouse button. The context menu also shows all the hotkeys to edit the list via the keyboard. The order of the entries in the list corresponds to their order in the combo box; you can manipulate it via the buttons `Move Up` and `Move Down`. The buttons `Font…` and `Background` define the font and color style of the combo box. Depending on the Windows version and in case Windows Visual Styles are activated the background color might be unchangeable.

The chapter ***4.4.1.2 Import of text tables*** describes the `Import…` button.

In case the model assigned a text table to the used signal, you can activate the check box `Use text table of the signal` as an alternative. In this case the list fills automatically using the according table values and you cannot edit it. As these text tables can contain value

ranges for specific text entries, selecting such an entry in the combo box uses the minimum value when setting the according signal.

Read only signals are displayed in an edit box instead of a combo box, because the drop down button would be useless in this case.

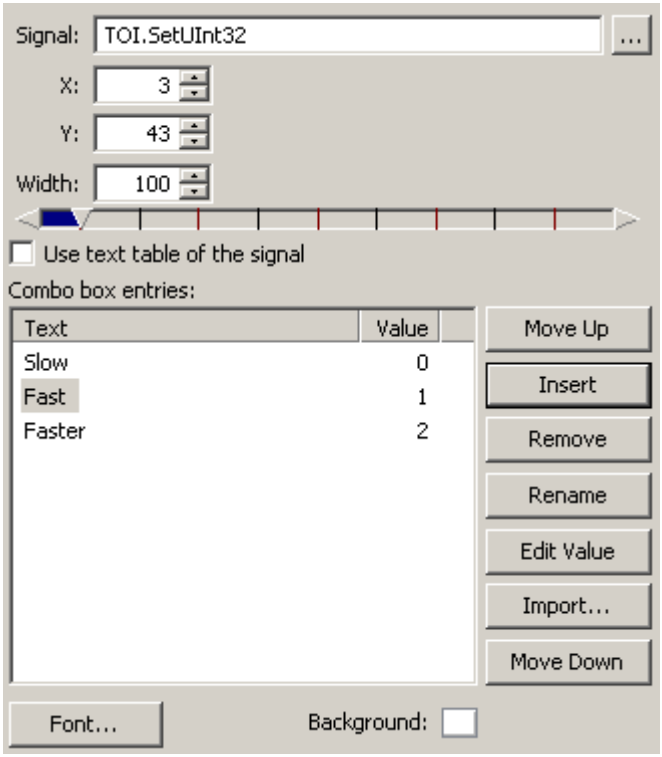


Figure 206 – Page of a combo box

4.4.12.9 Color Display

The color display draws a filled shape using a color depending on a signal value. The signal should have 32-bit and the type unsigned integer. The color display uses this 32-bit value the following way:

Bit 0..7: blue part
Bit 8..15: green part
Bit 16..23: red part
Bit 24..31: alpha channel (transparency)

The four parts can have the values 0 to 255. An alpha channel value of 0 means “invisible” and 255 means “opaque”. For values between, the color gets mixed with the background – a lower value means that the background is more visible.

Like the other elements the page of color display (***Figure 207***) allows you to enter the name of the corresponding signal, the position (`X` and `Y`) and the size (`Width` and `Height`) of the color display inside the custom control.

Additionally you can select a geometrical `Shape` via a combo box. In case you want to have a different shape than the supported ones, you can choose the rectangle shape and overlay it with a bitmap that has a transparent hole with your favored shape.

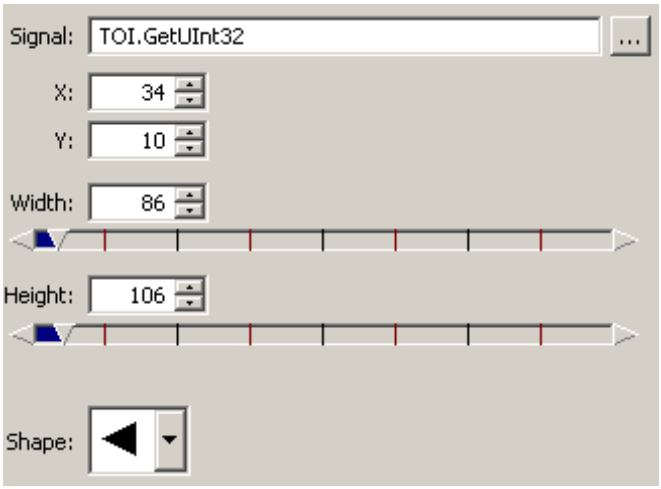


Figure 207 – Page of a color display

4.4.12.10 Text Message

The text message uses a message channel for receiving and displaying or sending text (in opposite to the ***4.4.12.11 Text Signal Array*** where signals transport the text). Use the combo box `Channel` to select the channel (see ***Figure 208***). It only displays fitting channels of the type "Generic" (no channels of the type CAN, FlexRay, etc. – see chapter ***4.4.14 The message window***). After that the combo box `Id` contains a list of available Ids for text on the selected channel. On a generic channel you can send messages with different Ids. The model determines which Id contains text messages and the code page used for the text encoding (e.g. Windows 1252 or UTF-8). So you can only use the text message for models supporting it.

Use the fields `X`, `Y` and `Width` to determine the position and size if the control. PROVEtech:TA automatically calculates the height from the selected font which you select via the `Font…` button. There you can also select the font color. Use the `Background` button to choose a background color for the control.

The model additionally chooses if you can also send a text message. In this case you can edit the text and send it by pressing the "Enter" key or by leaving the text message field. Press "Esc" to cancel editing the test message.

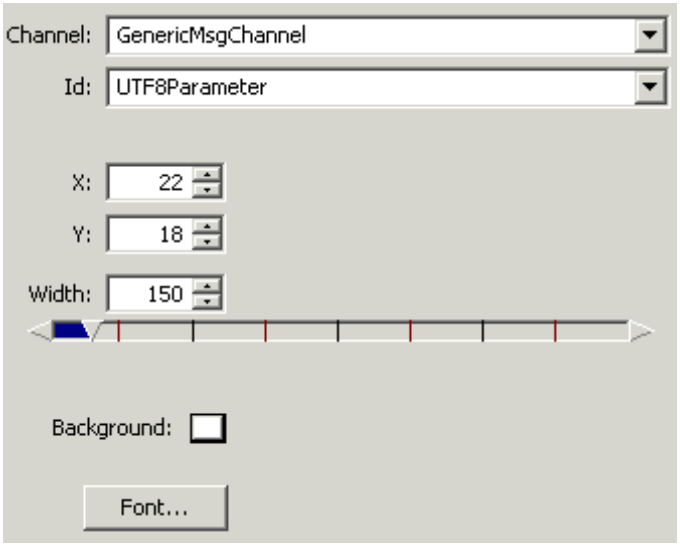


Figure 208 – Page of a text message

4.4.12.11 Text Signal Array

The text signal array control works on a signal array where the signal elements represent characters of a text (in opposite to the ***4.4.12.10 Text Message*** where messages transport the text). So PROVEtech:TA interprets the signal array elements as a byte array. The termination of the text is either the last signal array element or a signal with a value of 0. On the property page (***Figure 209***) enter the name of the signal array in the text field `Signal` or use the … button to select it via a dialog. As an example if the 1st element of the array has the name "Test.MyArray(0)", please enter "Test.MyArray".

Use the fields `X`, `Y` and `Width` to determine the position and size if the control. PROVEtech:TA automatically calculates the height from the selected font which you select via the `Font…` button. There you can also select the font color. Use the `Background` button to choose a background color for the control.

The single elements of the byte array for representing the text can only hold values between 0 and 255. This is not enough to distinguish all text characters of the world. So in different parts of the world people use different so called code pages that connect text characters to the values between 0 and 255. Please select the according code page via the `Code page` combo box. There you can also choose UTF-8 which is a Unicode encoding where multiple bytes can describe a single character and this way handle all defined Unicode characters. In case the model already defined a code page for the signal array, there is no need for a manual selection and this field is disabled.

In case the chosen signal array is writable, you can use the control to enter/modify the text. In that case the text gets written signal element by signal element where the last signal value is 0 or up to the last possible element. Writing the signals happens when pressing "Enter" or when leaving the text signal array field. You can cancel changing the text by pressing "Esc".

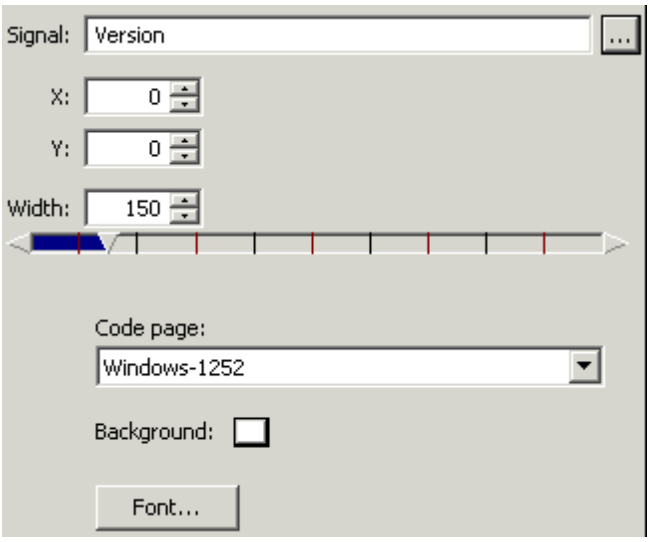


Figure 209 – Page of a text signal array

The maximum supported signal array length is 600.

4.4.12.12 Basic Handler

For the Basic handler control a Basic script programmed by you controls the behavior.

Displaying the page of the Basic handler (***Figure 210***) also activates a blue / red striped frame on the according custom control, which represents the active area. You manipulate this area via `X`, `Y`, `Width` and `Height`.

`Module path` is the path to your Basic script which gets triggered by the mouse in the upper defined area. The `Id` field is for specifying a user-defined string passed to your script by PROVEtech:TA. You can leave the `Id` field empty if you do not need it. Please keep in mind that your Basic script is only instantiated once if you use the same path in several Basic handler controls. You can use the `Id` field to differentiate them. Use a Basic class to have separate instances.

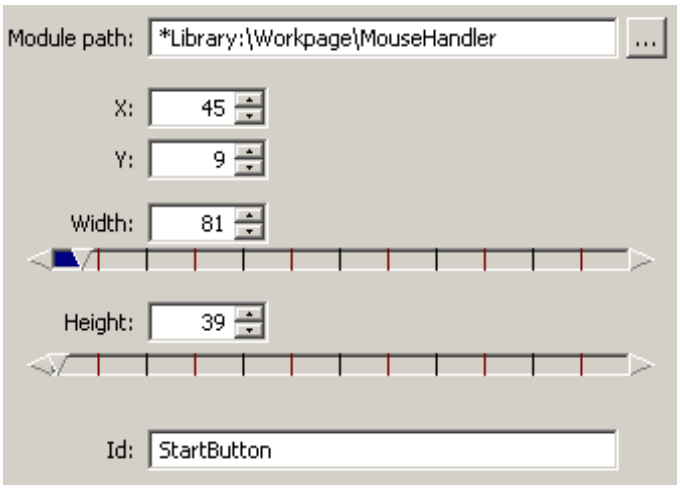


Figure 210 – Page of a Basic handler

PROVEtech:TA defines three functions as an interface. In your Basic script you must copy the names and parameter types exactly like this. Here is a small example:

```
'#Language "WWB-COM"

Option UsesAmbiguousError On
Option Explicit

Function MouseDown(ByVal dMousePosX As Double, _
        ByVal dMousePosY As Double, _
        ByVal strId As String, _
        ByVal dAreaPosX As Double, _
        ByVal dAreaPosY As Double, _
        ByVal dAreaWidth As Double, _
        ByVal dAreaHeight As Double) _
        As Boolean

    System.SetSignal(strId, 1)
    Return True
End Function

Sub MouseMove(ByVal dMousePosX As Double, _
        ByVal dMousePosY As Double, _
        ByVal strId As String)
End Sub

Sub MouseRelease(ByVal dMousePosX As Double, _
        ByVal dMousePosY As Double, _
        ByVal strId As String)

    System.SetSignal(strId, 0)
End Sub
```

Please be aware that all coordinates and the area are Double values (floating point). This might be astonishing for you as mouse coordinates cannot be between pixels. However, all these values are converted values for a 100 % view scaling (see chapter ***4.1.1 View scaling***). This way your Basic handler does not need to care about the current view scaling setting. This

means, if the view scaling is at 125 % and the user clicks on pixel 7, the value the handler gets is 7 / 1.25 = 5.6. Write your handler for a scaling of 100 % and it should also work without issues for 150 % as an example.

PROVEtech:TA calls the function `MouseDown` when the user clicks the left mouse button in the defined area. The function has the following parameters:

`dMousePosX`: X position of the mouse while clicking; 0 is directly right of the left border of the custom control.

`dMousePosY`: Y position of the mouse while clicking; 0 is directly below the upper border of the custom control.

`strId`: Corresponds to the `Id` field of the property page

`dAreaPosX`: Corresponds to the `X` field of the property page for the clickable area within the custom control

`dAreaPosY`: Corresponds to the `Y` field of the property page for the clickable area within the custom control

`dAreaPosWidth`: Corresponds to the `Width` field of the property page for the clickable area within the custom control

`dAreaPosHeight`: Corresponds to the `Height` field of the property page for the clickable area within the custom control

The example uses `strId` to pass a signal name and sets the corresponding signal to 1. The return value of the function `MouseDown` has the type Boolean. The return value `True` tells PROVEtech:TA to capture the mouse cursor for the control. This means the script gets `MouseMove` calls as long as the left mouse button is pressed even when the mouse cursor is outside the control and in the end a `MouseRelease` call when releasing the mouse button or losing the focus (other window gets into the foreground). In case the function `MouseDown` returns `False`, PROVEtech:TA will not call `MouseMove` and `MouseRelease`. The functions `MouseMove` and `MouseRelease` are optional and you need not implement them. However, `MouseDown` is not optional. `MouseMove` and `MouseRelease` have the following pre-defined parameters:

`dMousePosX`: Current X position of the mouse; 0 is directly right of the left border of the custom control.

`dMousePosY`: Current Y position of the mouse; 0 is directly below the upper border of the custom control.

`strId`: Corresponds to the `Id` field of the property page

In the upper example, the script sets the signal back to 0 when releasing the mouse button.

When creating the control PROVEtech:TA loads the specified script. Script changes only get active when loading it the next time. PROVEtech:TA does not load a script again if it was loaded already for another Basic handler control before. As long as no other custom control references the same script it is sufficient to open and close the property dialog to re-load the script.



PROVEtech:TA outputs any error message happening when executing your script in the PROVEtech:TA Log (see chapter ***3.6.1 PROVEtech:TA Log***).



The PROVEtech:TA main thread is blocked while calling your Basic functions. Thus, these Basic functions should return as fast as possible and never block. As an example, displaying message boxes within the functions is a bad idea.

4.4.13 The special controls

PROVEtech:TA has several special controls that represent different indicating and actuating elements of cars:

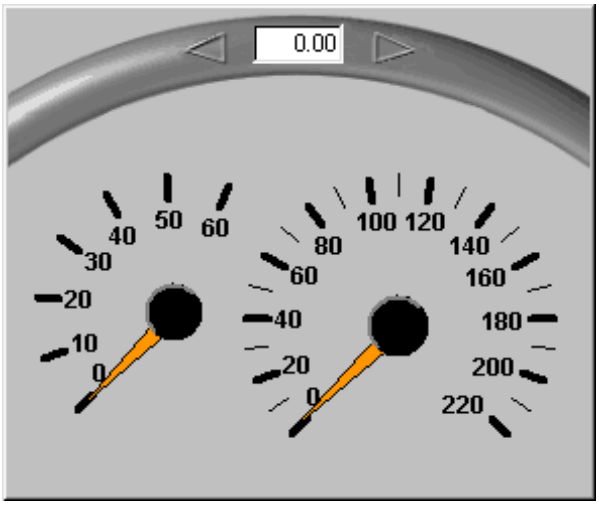


Figure 211 – The gauge cluster

The gauge cluster displays three signals: the steering angle, the revolution and the speed. You can use the keyboard in the upper edit control or the mouse with the arrow buttons left and right beside it to set the steering angle. The left arrow button decreases the steering angle and the right one increases it. The greater the distance between the mouse cursor and the middle of the control, the greater is the modification of the steering angle. The revolution display

divides the revolution signal by 100, so it displays 30 for a revolution of 3000 per minute as an example.

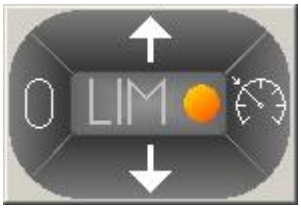


Figure 212 – The cruise-control and limiter

The cruise-control and limiter control displays a view of the lever in direction to the steering column. Clicking the middle of the control corresponds to pushing the lever to change the mode from “cruise-control” to “limiter” and vice versa. When the orange LIM lamp is on, the “limiter” mode is active. Move the lever up, down, to the back and to the front by clicking the four fields around the center. Clicking the left field corresponds to pushing back the lever, clicking the right field corresponds to pulling the lever to the front. The control sets the signal to the following values:

Signal value	*Lever position*	*Mode*
0	Middle	Cruise-control
1	Back (left)	Cruise-control
2	Up	Cruise-control
3	Front (right)	Cruise-control
4	Down	Cruise-control
5	Middle	Limiter
6	Back (left)	Limiter
7	Up	Limiter
8	Front (right)	Limiter
9	Down	Limiter

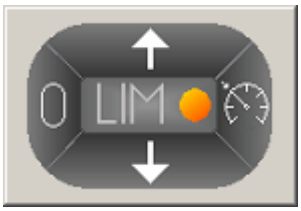


Figure 213 – The new cruise-control and limiter

The new version of the cruise-control differs from the former version in a two-step lever movement upwards and downwards. Additionally the LED does not shine automatically when you switch to the limiter mode. Instead of that PROVEtech:TA displays the lever smaller and a 2nd signal, the acknowledge signal of the ECU, controls the LED. The lever position signal has the following values:

Signal value	*Lever position*	*Mode*
0	Middle	Cruise-control
1	Up 1st step	Cruise-control
1+16=17	Up 2nd step	Cruise-control
2	Back (left)	Cruise-control
4	Down 1st step	Cruise-control
4+16=20	Down 2nd step	Cruise-control
8	Front (right)	Cruise-control
32	Middle	Limiter
1+32=33	Up 1st step	Limiter
1+16+32=49	Up 2nd step	Limiter
2+32=34	Back (left)	Limiter
4+32=36	Down 1st step	Limiter
4+16+32=52	Down 2nd step	Limiter
8+32=40	Front (right)	Limiter

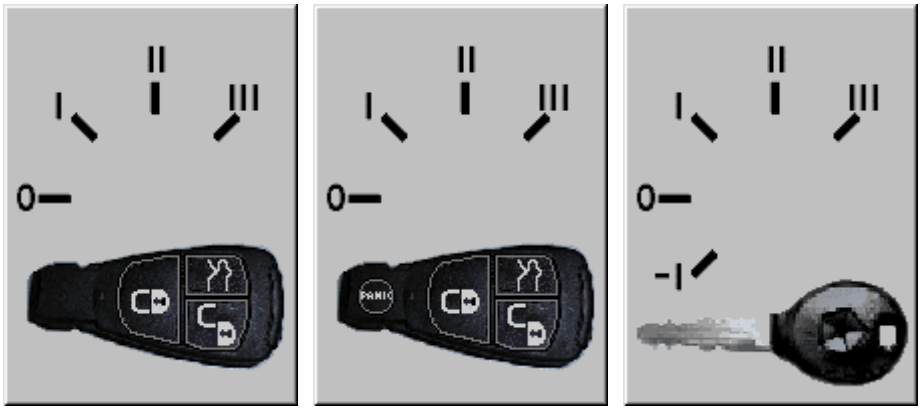


Figure 214 – Different ignition keys

PROVEtech:TA has three ignition keys: one for 0 to 3 with buttons for opening/closing the car and rear opening hood release, one with an additional panic button and one for -1 to 3

without buttons. To control the ignition keys click on one of the key position or buttons. This sets the signal to the following values:

Signal value	*Action*	*Variant 1*	*Variant 2*	*Variant 3*
-1	Key position -1			√
0	Key out	√	√	√
1	Key position 0 (key in)	√	√	√
2	Key position 1	√	√	√
3	Key position 2	√	√	√
4	Key position 3	√	√	√
9	Panic button		√	
10	Close car	√	√	
11	Rear opening hood release	√	√	
12	Open car	√	√	



Figure 215 – 5-and 6-speed shifter

Click on the different numbers to control the shifter, click in the middle to shift to neutral. The corresponding signal is set to the following values:

Signal value	*Position*	*5-speed*	*6-speed*
-1	Reverse gear		√
0	Neutral	√	√
1	1st gear	√	√
2	2nd gear	√	√
3	3rd gear	√	√

4	4th gear	√	√
5	5th gear	√	√
6	6th gear		√
6	Reverse gear	√	

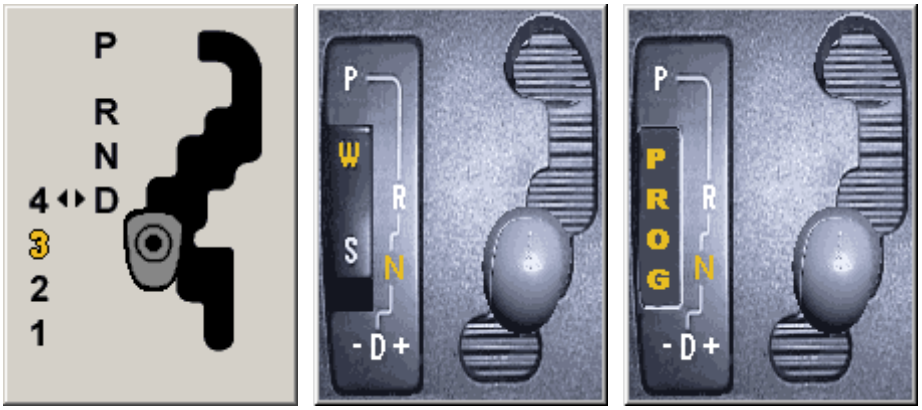


Figure 216 – Old and new NAG selector and CVT selector

Figure 216 shows the three Mercedes selectors. Click on a selector position to set the corresponding signal. The new NAG selector controls a second signal for the winter / summer switch, the CVT selector also controls a second signal with the PROG button. The selector signal has the following values:

Signal value	*Selector position*	*old NAG*	*new NAG*	*CVT*
0	1	√		
1	2	√		
2	3	√		
2	-		√	√
3	4	√		
3	+		√	√
4	D	√	√	√
5	N	√	√	√
6	R	√	√	√
7	P	√	√	√

The second signal has the value 0 for the S position of the new NAG selector control and 1 for the W position. The signal value 1 corresponds to a pressed PROG button, otherwise it is 0 for the CVT selector control.

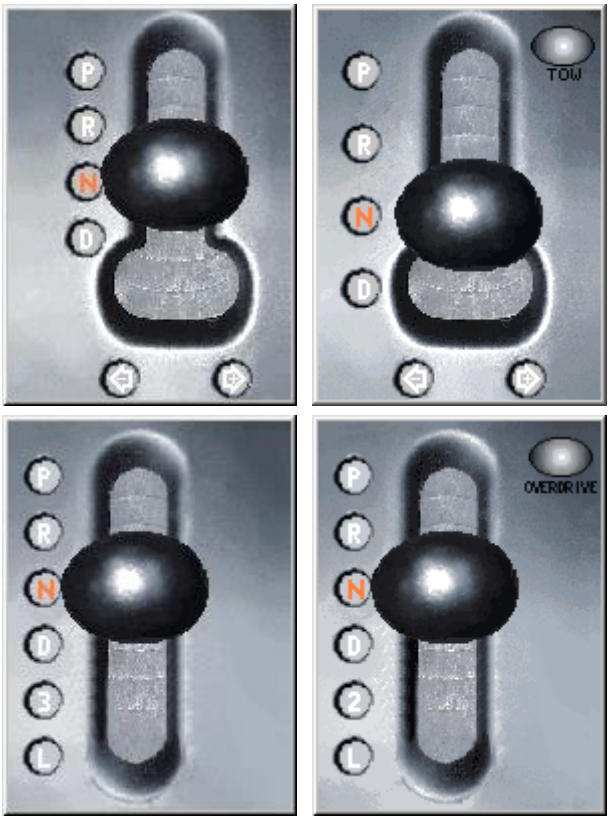


Figure 217 – PRND/AS, PRND/Tow, PRND3L and PRND2L/OD selector

You can see the four Chrysler selectors in ***Figure 217***. Use them like the other controls. The following table shows the signal values for the PRND/AS selector (which controls three signals):

Signal value 1	*Signal value 2*	*Signal value 3*	*Selector position*
0	0	0	P
1	0	0	R
2	0	0	N
3	0	0	D
4	0	0	Autostick middle
4	1	0	Autostick left
4	0	1	Autostick right

The following table contains the signal values for the PRND/Tow selector:

Signal value 1	*Signal value 2*	*Signal value 3*	*Selector position*
0	0	0	P

1	0	0	R
2	0	0	N
3	0	0	D middle
3	1	0	D left
3	0	1	D right

The Tow button controls a fourth signal, which is set to 1 when you press the button and 0 when you release it.

The next table contains the signal values for the PRND3L and the PRND2L/OD selector:

Signal value	*Selector position*	*PRND3L*	*PRND2L/OD*
0	P	√	√
1	R	√	√
2	N	√	√
3	D	√	√
4	3	√	
4	2		√
5	L	√	√

The PRND2L/OD selector additionally controls a second signal with the overdrive button. It has the value 1 when the button is pressed, otherwise it is 0.



Figure 218 – Rotary light switch

To rotate the rotary light switch click on the upper symbols for the left / right parking light, 0, Auto, side marker light and passing light. To pull out the switch for turning on the fog light and the rear fog light click on the according left symbols after selecting the side marker light or the passing light position. As an alternative pull out the switch by moving down the mouse from the symbol “side marker light” or “passing light” while pressing the left mouse button. The signal values for the rotary light switch are:

Signal value	*Rotary light switch position*

-2	Left parking light
-1	Right parking light
0	0 (off)
1	Auto
2	Side marker light
3	Passing light
4	Side marker light + fog light
5	Passing light + fog light
6	Side marker light + fog light + rear fog light
7	Passing light + fog light + rear fog light

A second signal controls the three LEDs on the left side. It corresponds to the response of the ECU to the selected rotary light switch position:

Signal value	*Rear fog light LED*	*Fog light LED*	*Driving light LED*
0	off	off	off
1	off	off	on
2	off	on	off
3	off	on	on
4	on	off	off
5	on	off	on
6	on	on	off
7	on	on	on

4.4.13.1 The property dialog of the special controls

In opposite to all the other controls the property dialog of the special controls only contains the selection of the signals that are linked to the control. The amount of linked signals differs for the different special controls. For some of the controls you can also change the background color via the button `Background`. In this example it is the property dialog of an ignition key with only one signal:

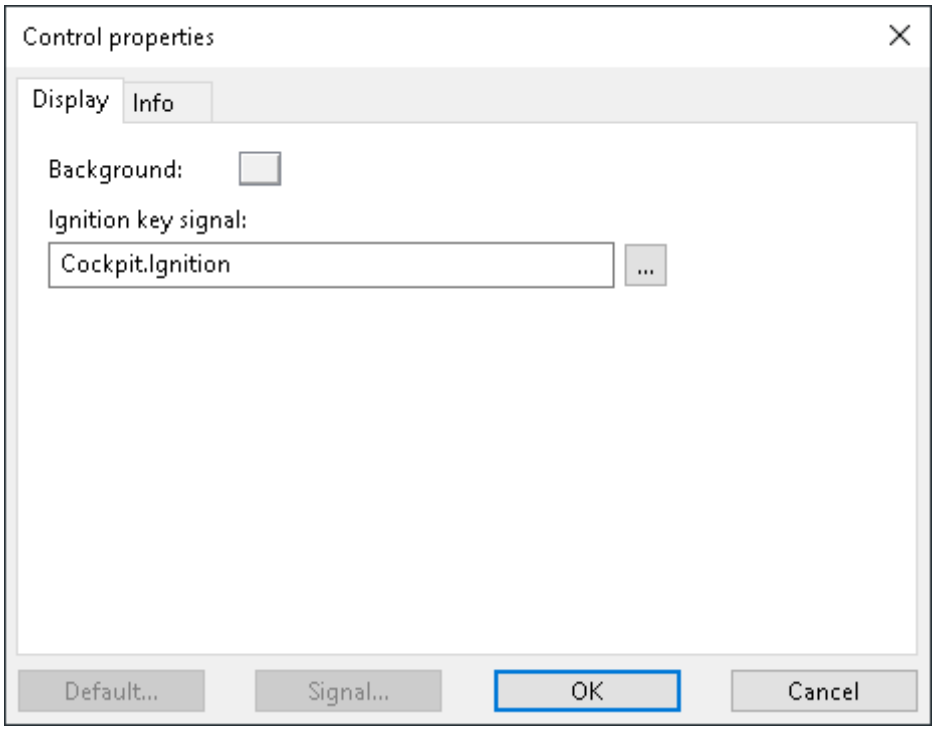


Figure 219 – Property dialog of the special controls

The gauge cluster control is an exception. You can additionally influence the appearance of the speed display with its property dialog:

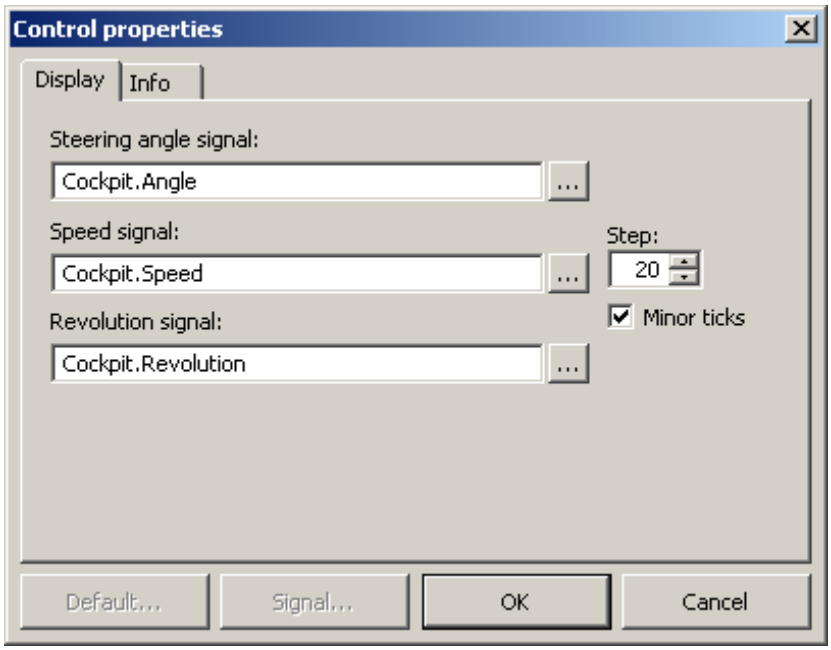


Figure 220 – Property dialog of the gauge cluster control

4.4.14 The message window

The message processing feature of PROVEtech:TA depends on its support by the platform and the model. The ModelCom module linked to the model provides an interface for adding message channels. Additionally, the model needs a direct access to the message hardware for getting the messages. Without this, you cannot use the message window and the according menu item is not available. The platform configuration chapters within chapter ***3.2.7.2.3 Models*** contain information about this separately for each platform.

Beside signals PROVEtech:TA can display messages of different message channels on the workpage in message windows (see ***Figure 221***).

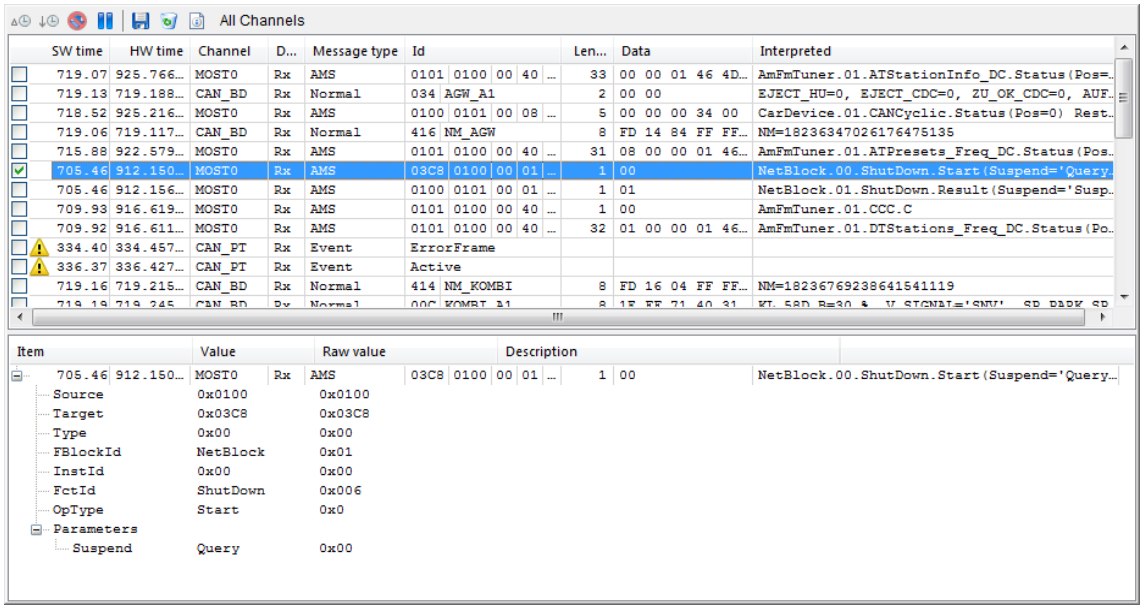


Figure 221 – The message window

To create a message window right click on a free part of the workpage and select *Create MessageWindow…*. This creates a new window at the clicked position and opens the property dialog for configuring the channels and filters (see chapter ***4.4.14.1 The property dialog of the message window***).

The window consists of three parts: the upper one is the toolbar and a user defined label. In the middle is a list of incoming messages and the lower part is a detailed view previously selected messages. Activate the check box at the front of a message line in the middle list to display it in the detailed view. As an example you can compare several messages this way. The message window automatically hides the detailed view in case it has no entries.

The detailed view displays a tree structure with columns. The topmost level of the tree replicates the line of the list view. The columns of this line also derive from the list view, so for

this line the column headings and widths of the list view are applied. Expand such a tree entry in the detailed view to see the details of a message.

For these sub entries the column headings and widths of the detailed view are applied. There are four columns: `Item`, `Value`, `Raw value` and `Description`. `Item` contains the name of a message part (e.g. the name of a CAN message signal)., `Value` interprets the according value (textual or numerical) and `Raw value` shows the value before a possible conversion in hexadecimal representation. The column `Description` displays additional informational text in case it is available. The displayed items depend on the message type and the message itself.

The list view has a check box in front of each line to add messages to the detailed view. Right beside it there is space for an icon that marks special messages. The list view has several columns. Apart from the first one you can change their order via drag and drop.

`SW time`:
Each message has two time stamps with a software time and a hardware time. The software time (SW time) derives from the model tick. So its resolution corresponds to the model cycle, the output is always in seconds. This time indicates, at which model step the message arrived in the model or when it was sent. PROVEtech:TA synchronizes the messages of different channels and signal data via the SW time.

`HW time`:
The message channel hardware (e.g. CAN card) generates the hardware time stamp. The hardware sets the resolution, the output is always in seconds. Thereby the precision normally exceeds the SW time. You can compare HW times only within a channel. SW time and HW time do not run synchronous and can differ completely. There can be one exception to this: In cases where the used hardware does not produce a time stamp, the SW time and HW time are identical.

In case of DLT (Diagnostic Log and Trace) the HW time shows the reception time in seconds beginning from 1/1/1970 (UTC). The system time of the receiving model node is the base for that.

`Channel`:
This column displays the unique channel name defined in the model.

`Direction`:
“Rx” for received and “Tx” for sent messages.

`Message type`:
Currently PROVEtech:TA supports CAN, FlexRay, Generic, LIN and MOST channels. This column displays the message sub type for these two channel types:

Channel type	**Message type**	**Meaning**
CAN / CAN FD	Normal	All messages that transport data and have a CAN Id

CAN FD	FD	All messages that transport data and have a CAN Id and where the EDL (Extended data length) bit is set
CAN / CAN FD	Event	Event messages, see column `Id`
CAN / CAN FD / LIN / FlexRay / IP	I-PDU	I-PDU extracted from a “normal” frame according to the assigned bus database file. A “normal” frame can contain one or several I-PDUs with either own header Ids or a static layout with optional update bits.
CAN / CAN FD	ISO-TP	Assembled CAN messages that use the ISO 15765-2:2016 transport protocol. You can only see these messages in case you have configured ISO-TP, see chapter ***3.2.7.2.3.12 Configuration of CAN channels***.
CAN / CAN FD	J1939-TP	Assembled CAN messages that use the SAE J1939 transport protocol. You can only see these messages in case you have activated the J1939 support for the channel, see chapter ***3.2.7.2.3.12 Configuration of CAN channels***.
CAN	Remote	CAN remote frames which do not transport any data, but only the CAN Id (not supported by all platforms)
DLT	Control	Diagnostic Log and Trace messages with “Message Type” 3 = Control
DLT	Log	Diagnostic Log and Trace messages with “Message Type” 0 = Log
DLT	Network	Diagnostic Log and Trace messages with “Message Type” 2 = Network
DLT	Other	Diagnostic Log and Trace messages with no “Message Type” or unknown “Message Type”
DLT	Trace	Diagnostic Log and Trace messages with “Message Type” 1 = Trace
FlexRay	Normal	All messages that transport data even if they have errors
Generic	Normal	All generic messages; generic messages are user defined. As an example you can use them to configure your hardware. PROVEtech:TA does not interpret these messages.
IP	Raw	Raw IP messages than only contain a payload beside the IP addresses and ports. (e.g. transported via Ethernet, APIX or BroadR-Reach)

IP	SOME/IP	SOME/IP (Scalable service-Oriented middleware over IP) messages with SOME/IP header and payload (e.g. transported via Ethernet, APIX or BroadR-Reach)
LIN	Normal	All messages that transport data and have a LIN Id (also messages with CRC error or without slave answer data)
LIN	Event	Event messages, see column `Id`
LIN	ISO-TP	Assembled LIN messages that use the ISO 17987-2 transport protocol. This is done automatically on the LIN Ids 60 and 61.
MOST	AMS	Control messages of the type “normal message” and assembled control messages
MOST	CMS	Control messages
MOST	Event	Event messages, see column `Id`

`Id`:
This column contains different information depending on the channel and message type:

Channel type	**Message type**	**Meaning**
CAN / CAN FD	Normal or Remote (not for FD)	Hexadecimal CAN Id plus message name in case it can be determined via the description file. In case you activate the J1939 support in the channel configuration dialog (see chapter ***3.2.7.2.3.12 Configuration of CAN channels***), extended Ids get displayed in the form Priority | Parameter Group Number | Destination Address (-- in case not available) | Source Address (all hexadecimal). After that the message name follows in case the description file defines it.

CAN / CAN FD	Event	Event (Icon ⚠): ErrorFrame: An ErrorFrame occurred on the bus or was sent; if the used platform provides additional information about the ErrorFrame type like bit error or form error, PROVEtech:TA displays that in parenthesis. Active: The CAN card switched to the Active state Warning: The error counter has reached the level Warning Passive: The error counter has reached the level for going Passive BusOff: The CAN card has entered the BusOff state WakeUp: WakeUp event Overrun: The CAN card had a buffer overrun
CAN / CAN FD / LIN / FlexRay / IP	I-PDU	Hexadecimal I-PDU header Id (if available) and the I-PDU name. In case it has a long (4 byte) and short (3 byte) header Id, the long one is preferred.
CAN / CAN FD	ISO-TP	Hexadecimal CAN Id plus message name in case it can be determined via the description file. In case of Extended or Mixed Mode ISO-TP messages you can additionally see the hexadecimal Target Address or Address Extension.
CAN / CAN FD	J1939-TP	Same as for the type “Normal” in case of extended identifiers for J1939
DLT	<all>	Application Id, Context Id and hexadecimal Session Id if available
FlexRay	Normal	Hexadecimal slot Id and cycle number plus frame name in case it can be determined via the XML data base
Generic	Normal	Hexadecimal Id plus message name in case it is specified in the configuration
IP	Raw	Source IP address, source port, target IP address and target port (everything decimal)
IP	SOME/IP	Hexadecimal Service Id, Method Id, Interface Version and Method Type
LIN	Normal	Hexadecimal LIN Id plus message name in case it can be determined via the .ldf file

LIN	Event	Event (Icon ⚠): SyncError: A synchronization error occurred on the bus Error: A generic error occurred (e.g. Id parity error) WakeUp: WakeUp event Sleep: Sleep event
LIN	ISO-TP	Hexadecimal LIN Id, hexadecimal Node Address and the message name
MOST	AMS	In hexadecimal representation: Source address, target address, message type (do not mix it up with the other column), FBlockId, InstId, FctId, OpType
MOST	CMS	In hexadecimal representation: Source address, target address, message type (do not mix it up with the other column)
MOST	Event	Event and the according icon: Rx Light Off: Light off at optical input terminal Rx Light On: Light on at optical input terminal Rx Light Modulated: Light modulated at optical input terminal Tx Light Off: Light off at optical output terminal Tx Light On: Light on at optical output terminal Tx Light Modulated: Light modulated at optical output terminal Ring Locked: Ring locked Ring unlocked: Ring unlocked

`Transmitter`:
Bus node sending the message according to the bus description file assigned to the channel. It can be several ones, the message window displays them as a comma separated list in that case. Currently the column outputs anything only for CAN, CAN FD, LIN and FlexRay channels. For activated J1939 on CAN (see chapter ***4.4.14.1 The property dialog of the message window***) there is a special behavior: The column determines the transmitter for extended identifiers via the Source Address.

For DLT (Diagnostic Log and Trace) messages the column shows the ECU Id if available.

`Length`:
Amount of displayed bytes in the column `Data` (decimal representation).

`Data`:
This column contains different information depending on the channel and message type:

Channel type	Message type	Meaning
CAN / CAN FD	Normal	0-8 data bytes of the CAN message (hexadecimal)
CAN / CAN FD	ISO-TP	Data bytes of the assembled ISO-TP message (hexadecimal)
CAN / CAN FD	J1939-TP	Data bytes of the assembled J1939-TP message (hexadecimal)
CAN FD	FD	0-64 data bytes of the CAN FD message (hexadecimal)
CAN / CAN FD / LIN / FlexRay / IP	I-PDU	Data bytes of the I-PDU
DLT	<all>	Complete DLT message (hexadecimal)
FlexRay	Normal	0-254 data bytes of the FlexRay message (hexadecimal)
Generic	Normal	0-x data bytes of the Generic message (hexadecimal)
IP	Raw	0-x data bytes (payload) of the message (hexadecimal)
IP	SOME/IP	0-x data bytes (payload) of the SOME/IP message (hexadecimal)
LIN	Normal	0-8 data bytes of the LIN message (hexadecimal)
LIN	ISO-TP	Data bytes of the assembled ISO-TP message (hexadecimal)
MOST	AMS	Data bytes of the parameters (hexadecimal)
MOST	CMS	17 data bytes of the control message, one TXStatus byte and two additional status bytes (hexadecimal)

`Status`:
This column contains different information depending on the channel and message type:

Channel type	Message type	Meaning
CAN / CAN FD	ISO-TP or J1939-TP	The following messages appear in case of an error: Invalid Length: One of the associated CAN messages has an invalid length parameter Unexpected Sequence Number: A Consecutive Frame has an invalid sequence number Aborted: The message was not completed Unknown: Unknown error

CAN FD	FD	BRS: Bit rate switch ESI: Error status indicator
DLT	<all>	DLT time stamp in seconds if available
FlexRay	Normal	This column contains several sub column. The first one is the hexadecimal header CRC value, the second one the hexadecimal frame CRC. After that the bits of the header and status bits follow in case they are set: RS: Reserved flag of the header PP: Payload Preamble Indicator of the header NF: Null Frame Indicator of the header (attention: this gets displayed in case it is a Null Frame, which means the bit is 0) SY: Sync Frame Indicator of the header SU: Startup Frame Indicator of the header VF: Valid Frame status bit SE: Syntax Error status bit CE: Content Error status bit BV: Boundary Violation status bit
LIN	Normal	In case the slave did not send answer data, this column displays “NoAnswer”. Otherwise it displays the CRC byte followed by “CRC”. In case of a CRC error, the displayed text is “CRC Error”.
LIN	ISO-TP	The following messages appear in case of an error: Invalid Length: One of the associated LIN messages has an invalid length parameter Unexpected Sequence Number: A Consecutive Frame has an invalid sequence number Aborted: The message was not completed Unknown: Unknown error

MOST	CMS	Displays the textual representation of the status bytes; a combination of several statements is possible: NoAck: Transmission failed. No response from the target node (no error in case the receiving device is between source and target in the ring). CRC: CRC error Unsupported: The target node does not support the message type Invalid: TxStatus has an invalid value; valid values are 0x00, 0x10, 0x11, 0x20, 0x21, 0x30 and 0x31 The following error status values come directly from the MOST hardware. You can get more information about them in the according manual: NoLight, NoLock, Biphase, MsgLen, Parity, FrameLen, PreambleType, CRC, TxUnlocked, TxTimeout.
other	other	Nothing

`Interpreted`:
For CAN, FlexRay and LIN messages of the type “Normal” or “I-PDU” this column displays the signal values in case the .dbc/.ldf/AUTOSAR-XML/FIBEX file contains the according information.

For CAN messages of the type “Normal” that belong to an ISO-TP message the column contains ISO-TP specific information.

For CAN messages of the type “Normal” that belong to a J1939-TP message the column contains J1939-TP specific information.

For DLT (Diagnostic Log and Trace) messages it displays a textual output of all parameters when the verbose flag is active. Otherwise, it displays the Service Id (known ones in text form, hexadecimal otherwise) only for “Control” messages followed by the parameters as hexadecimal Byte output. For “Control Response” messages there is an interpretation for the 1st byte after the Service Id as OK, ERROR or NOT_SUPPORTED if possible.

For LIN messages of the type “Normal” that belong to an ISO-TP message the column contains ISO-TP specific information.

For MOST AMS and CMS messages this column displays FBlockId, InstId, FctId, OpType and the parameters. Therefore, it uses the textual representation as long as the MOST function catalog contains this information.

For SOME/IP PDUs using Protocol Buffers it displays a textual output of all parameters.

The following paragraph describes the toolbar buttons:

: Switch between relative () and absolute time ( ) output (SW time and HW time). The relative time output shows the time difference to the previous message with the same Id. In case there is no previous message, the columns display 0. When changing this setting, the window rebuilds itself and uses all messages available in the global buffer for that.

: Switch between continuous () and overwrite ( ) mode. In the continuous mode the window always attaches new messages at the end of the list. In the overwrite mode, the window overwrites previous messages with the same Id. In case there is no previous message, it attaches the message at the end of the list. When overwriting a message that is also visible in the detailed view, the window also overwrites this one. When changing this setting, the window rebuilds itself and uses all messages available in the global buffer for that.

: Disables () and enables ( ) the automatic scrolling. In case the scrolling is enabled, the view scrolls to the end of the list when it attaches a new message. When it is disabled, scrolling only occurs in case the displayed messages disappear from the buffer.

: Pauses the message reception for that window and disables the other buttons. While paused (), the current messages stay visible in the window even if the global ring buffer removes them for new messages. Press the button again ( ) to restart the message reception for the window. This removes all messages from the window first. This button does not exist in the offline mode (see ***4.5 The offline mode***).

: Saves the messages currently in the global buffer to a data acquisition file without the need to start a data acquisition before. After specifying the destination path and format via the file selection dialog (see chapter ***3.1 The file selection dialog***) and optional format dependent settings, you can choose the message filters to use on the messages via a message box:

- `Use the settings of the message window` (Uses the filters of the message window for the lowest protocol layers. It is not possible to store upper protocol layers like I-PDU and ISO-TP directly in data acquisitions as they are extracted from the lowest layers.)
- `Use the current data acquisition settings` (see ***3.2.7.4 Data acquisition***)
- `Save all messages (no filtering)`

The duration of the saved data acquisition depends on the amount and size of messages on all channels currently residing in the global buffer. It is not predictable. To have predictable results, please run a normal data acquisition (see ***3.4 The Data Acquisition menu***). This save button does not exist in the offline mode (see ***4.5 The offline mode***). In pause mode ( ), it saves only the currently visible messages of the window, not the global buffer. This also happens when choosing one of the two CSV output formats which copy the window content to a text table file. You can also copy the window content to a text table in the clipboard via the context menu.

: Clears the list view and the detailed view. In case you only want to remove a message from the detailed view or you want to clear only the detailed view, right click on it and select the according context menu item.

: Opens the property dialog, see chapter ***4.4.14.1 The property dialog of the message window***. As an alternative you can double click the array right beside the toolbar.



Each message window owns a buffer for the displayed messages. This buffer depends only on the amount of displayed messages and not their sizes. In case the buffer overflows the oldest messages disappear from the window.

Additionally, a global buffer for all messages exists. This one has a fixed size and covers the message contents. In case it overflows, the oldest messages disappear in all windows displaying them.

In the offline mode (see ***4.5 The offline mode***) the same rules are valid. The global buffer corresponds to the selected time range in this case. When it is too large, a warning appears in the PROVEtech:TA Log (see ***3.6.1 PROVEtech:TA Log***) and only a part of the range is displayed.



In case you use a model that runs on several nodes (e.g. several computers or several CPU cores), PROVEtech:TA can fetch the messages of all nodes in parallel. However, while displaying messages of different nodes in the same message window, it might happen, that they are not ordered chronologically. So, the SW time is not monotonic increasing. This depends on when and in which order PROVEtech:TA gets the messages from the nodes. However, when you do data acquisitions, PROVEtech:TA automatically sorts the messages using the SW time and in the offline mode they get displayed chronologically.



In order to reduce the necessary resources, PROVEtech:TA disables the reception of a message channel in case there is no receiver for its messages: data acquisition, message window or running test program. However, if you refill a message window with the messages of the global buffer, this can lead to misinterpretation of the display in case the message reception was off somewhere between the beginning and the end of the global buffer (e.g. delta time display way too high for the first message after the gap).



Unfortunately, the AUTOSAR documents are poorly written in a lot of cases and let a lot of interpretation freedom: how to understand and implement a lot of details. This means, there are many different variants in ECUs of different customers. There is no “right” or “wrong” as the AUTOSAR documents do not clarify it. Therefore, many parts in PROVEtech:TA concerning AUTOSAR are simply “best effort”. This also means, the interpretation of message data might be different from what you expect. Or we simply did not implement a part of AUROSAR no other customer used so far. This is especially important for SOME/IP. Please contact us in that case to clarify it.



For DLT (Diagnostic Log an Trace) there is only support for version 1 based on the AUTOSAR standard R20-11. There is no support for AR-XML or Fibex descriptions.



The message window handles PDU/Signal/SignalGroup update bits. It does not display signals where any update bit is 0 in the column “Interpreted”. In the detailed view it displays these signals and PDUs with gray text instead of black. The displayed values are no valid signal values in that case.



Use the context menu to copy single cells or the whole content of the message window or detailed view to the clipboard. The context menu of the detailed view also provides an item for deleting single messages from the view.

4.4.14.1 The property dialog of the message window

You can use the property dialog of the message window (***Figure 222***) to change the displayed label and the used filters.

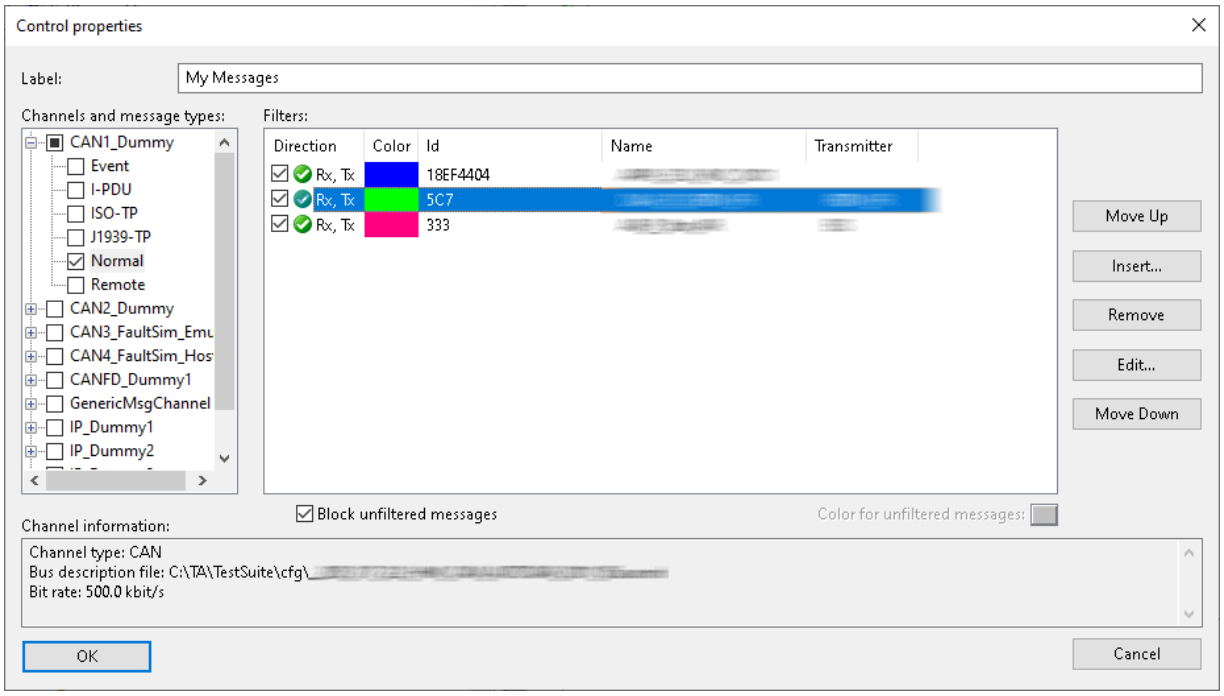


Figure 222 – Property dialog of the message window

Use the text field `Label` to change the label of the message window. You can see your changes immediately.

The tree `Channels and message types` displays all available message channels and the according sub types depending on the channel type (see column `Message type` in the chapter ***4.4.14 The message window***). Activate the check box of a sub type to display it in the message window in principle. After that, you can use the list `Filters` to add and remove filters for this sub type.

The field `Channel information` displays the type of the currently selected channel and additional information like the linked bus description file and the bit rate for the channel specified in that file.

PROVEtech:TA filters each message using the following scheme: It detects the channel and sub type for a new message. After that, it processes the according filter list from the top to the bottom. Every filter entry can produce one of the following three results:

1. The filter does not match and it passes the message to the next entry

2. The filter matches and it is a block filter, then it discards the message and cancels the filter processing

3. The filter matches and it is a pass filter, then it adds the message to the message window and cancels the filter processing

In case no filter entry matches, the setting of the check box `Block unfiltered messages` determines if PROVEtech:TA discards the message (check box activated) or adds it to the message window.

In case the list only contains pass filters (see below) and the check box is deactivated or the list only contains block filters or no filters and the check box is activates, the icon  appears left of the check box. This informs you that the settings are not useful and let all messages pass or block all messages.

The filter entries let you choose the colors of the output lines in the message window. The `Color` column shows the current color per filter. Click on an entry of the column to change the color. As an alternative open the context menu via right-click and select *Change the color…*. You can also select several lines and set a common color. The color is only relevant for pass filters  . If no filter matches and the check box `Block unfiltered messages` is inactive, the output uses the `Color for unfiltered messages`. Please be aware that color changes only apply for upcoming lines in the message window and not already existing lines.



In the dark mode (see chapter ***3.2.7.1 Common***) the message window background color is dark. PROVEtech:TA then automatically changes the color to white if the selected color is black for a message filter. PROVEtech:TA does not change other colors though.

You can use the buttons on the right side to edit the filter list. Use `Move Up` and `Move Down` to change the order. Click on `Insert…` to add new entries, `Remove` to remove the selected entries and `Edit…` to change the selected entries.

To deactivate a filter entry temporarily, deactivate the check box of the entry. Click on the icon aside it to change the filter type (pass filter  , block filter ). The asterisk * in a column output symbolizes a filter setting that covers the whole range of the according field (e.g. all CAN Ids). The columns of the list change according to the selected sub type. Just like the dialog for inserting and adding filter entries:

CAN, Generic and LIN messages, sub type “Normal”:
The dialogs to edit filters for the sub type “Normal” are very similar for CAN, LIN and MOST messages. This is an example for CAN messages:

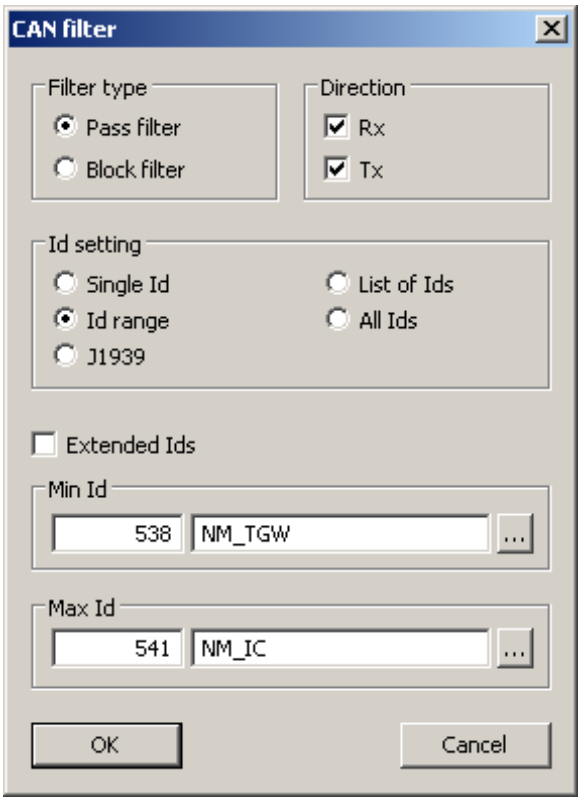


Figure 223 – Filter dialog for normal CAN messages

The upper part is similar for all filter types. A `Pass filter` passes messages to the message window, a `Block filter` discards messages.

Activate `Rx` to use the filter on received messages and `Tx` to use it (also) on transmitted messages.

The group `Id setting` determines how the lower part looks like. A single filter entry can cover CAN/Generic/LIN Id ranges, but no Id lists. So, when you select `List of Ids` and enter several Ids, the dialog creates several filter entries.

You can always enter CAN/Generic/LIN Ids hexadecimal or in the textual form. There are different edit boxes for this. The configuration (e.g. .dbc file) resolves the textual entry, but internally the storage is always numerical. Click on … to open a dialog for selecting CAN/Generic/LIN Ids in a comfortable way. For CAN Ids the dialog displays a tree using the following hierarchy as an example via a .dbc file: ECU → Rx/Tx → message. As an example you can select all messages an ECU transmits this way. For Generic and LIN there is just a list for the Id selection.

In case of CAN there the additional `J1939` radio button. When you select it, the dialog looks the following way:

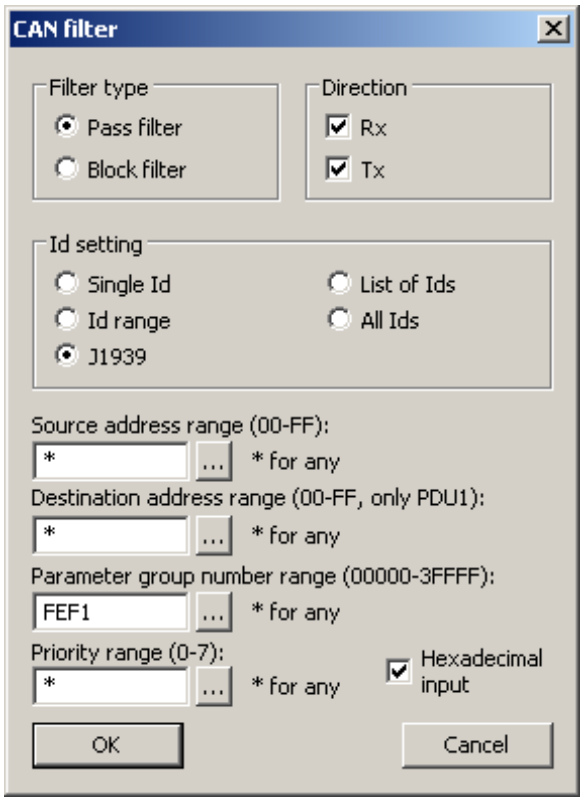


Figure 224 – Filter dialog for J1939 messages

For J1939-TP messages the filter dialog is identical, but you do not need the `Id setting` radio buttons there. Now you can specify source address, destination address, parameter group number and priority for matching. Each field can be a single value, a range (like FEF1-FEFF) or * which means “any”. The destination address matching only occurs in case of PDU1 format messages (parameter group number format 00-EF). The parameter group number field contains the reserved bit and the data page bit.

Use the `Hexadecimal input` check box to toggle between hexadecimal and decimal value input. The parent filter dialog (see ***Figure 222***) always displays hexadecimal numbers. For J1939 filters assigned to the sub type “Normal”, the “Name” column displays “J1939” and the “Id” column contains all the information about the filter where SA=source address, DA=destination address, PGN=parameter group number, Prio=priority.

CAN, LIN and MOST messages, sub type “Event”:
The dialogs for the sub type “Event” are very similar for CAN, LIN and MOST messages. This is an example for CAN messages:

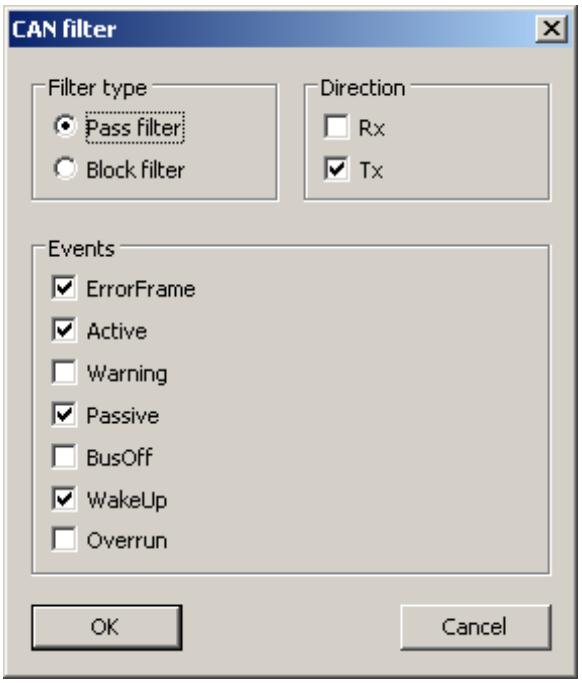


Figure 225 – Filter dialog for CAN event messages

The upper part is identical to the dialog for normal CAN messages. The group `Events` specifies, which events the filter covers. So a single filter can cover several events.

CAN messages, sub type "ISO-TP":
The dialog for the sub type "ISO-TP" looks like this:

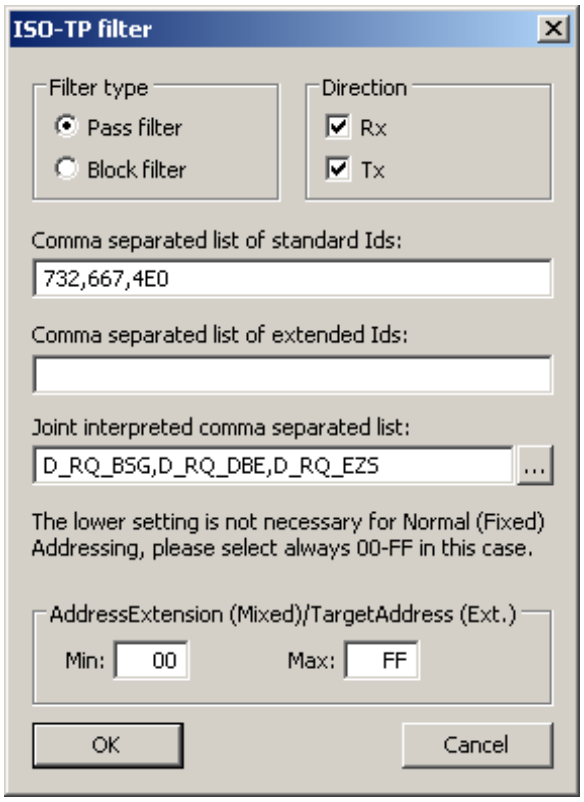


Figure 226 – Filter dialog for CAN ISO-TP messages

Use the field `Comma separated list of standard Ids` to enter hexadecimal standard Ids separated by commas and the field `Comma separated list of standard Ids` for hexadecimal extended Ids. The field `Joint interpreted comma separated list` shows the according message names in case the according description file defines them. When you enter message names the fields with hexadecimal representation update automatically and vice versa. Click on ... to open a dialog for an easy message selection

In case Extended or Mixed Mode is used, you can enter a range for the Address Extension or the Target Address in the lower fields. Otherwise the range should stay 00-FF.

DLT messages:
The dialog for DLT messages looks like this:

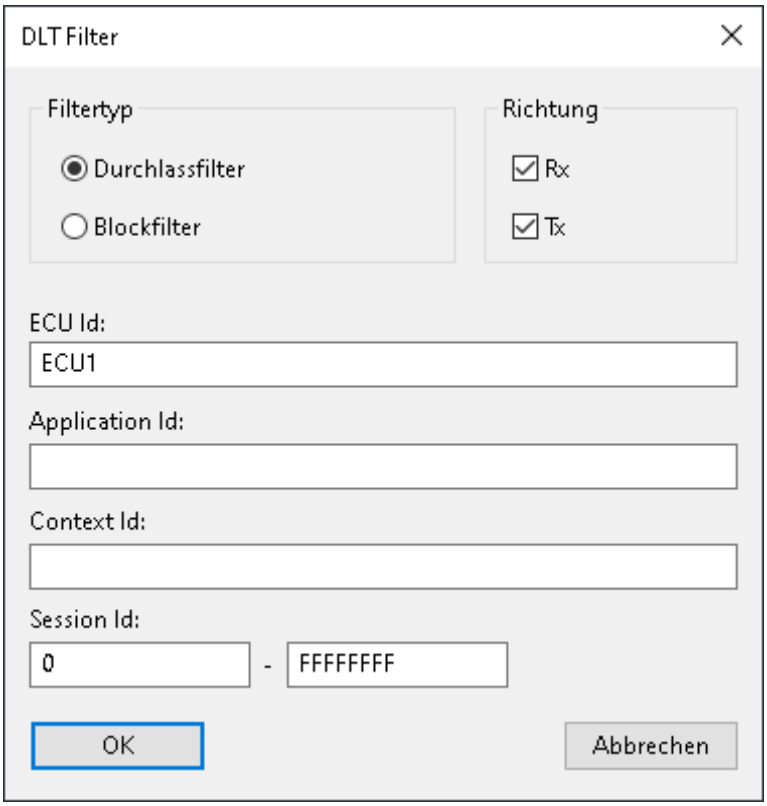


Figure 227 – Filter dialog for DLT messages

The upper part is similar for all filter types. A `Pass filter` passes messages to the message window, a `Block filter` discards messages.

Activate `Rx` to use the filter on received messages and `Tx` to use it (also) on transmitted messages.

If you specify `ECU Id`, `Application Id` and/or `Context Id`, the filter only matches if the string(s) match. It is case sensitive. An empty field means, that it matches anything, even if the message does not have the according Id. Please be aware that the AUTOSAR standard only allows ASCII characters. However, Byte values >=128 might exist and PROVEtech:TA uses the Latin 1 character set for them. It does not accept values outside Latin 1 or values <32.

For the `Session Id` it allows setting a range (hexadecimal). In case the range is from 0 to FFFFFFFF, it matches anything, even if the message does not have a Session Id.

FlexRay messages, sub type “Normal”:
The dialog for FlexRay messages looks like this:

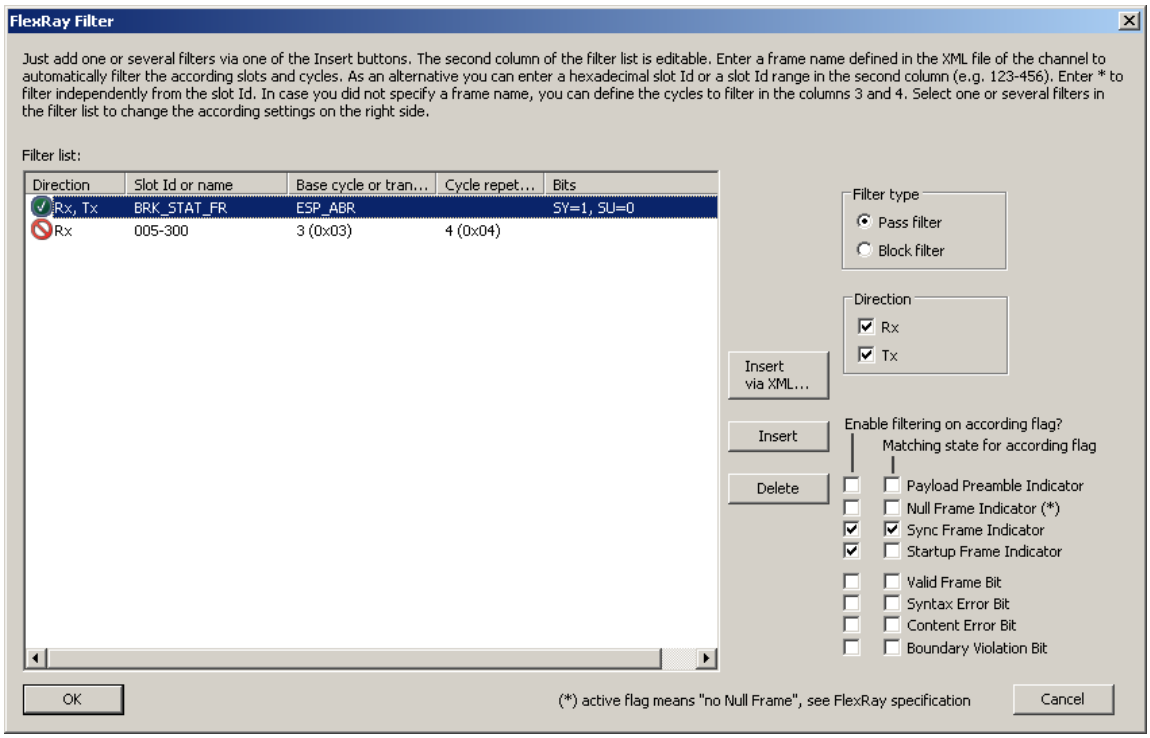


Figure 228 – Filter dialog for FlexRay messages

`Filter list` displays the filters you currently edit. The columns are identical to the ones of the main property dialog (***Figure 222***) for FlexRay filters. The icon of every entry shows if it is a pass or a block filter. Click on the `Slot Id or name` column of an entry to edit it. There are two different filter types:

- The first one filters on a frame name specified in the according XML data base. So click on the `Slot Id or name` column of the entry and after that on the drop down button of the appearing combo box to select the frame name. In this case the 3rd column displays the ECU(s) transmitting frame. You can also click on `Insert via XML...`, to open an additional dialog for selecting frames.
- The second one filters on a slot Id range and cycle numbers. After clicking on the `Slot Id or name` of the entry you can enter the hexadecimal slot Id range, e.g. 5-200. You can also enter a single number for filtering only on this slot Id or the * to filter on all slot Ids. Now you can edit the 3rd and 4th column showing the base cycle and cycle repetition. Examples:

base cycle	cycle repetition	matches on the cycle numbers
0	1	0..63
1	8	1, 9, 17, 25, 33, 41, 49, 57
19	64	19

Click on `Insert` to add a new entry to the filter list or `Delete` to delete the selected filters.

After selecting one or several filter entries in the list, the right fields of the dialog get enabled and you can edit additional options for the selected filters. The `Filter type` and `Direction` are identical to those of all the other message filters.

In the lower right check box area there are two check box columns. They represent all the header and status bits of a FlexRay message. The first column determines, which bits you want to use for filtering. So when the first column check box is unchecked, the second column check box is unused. When the first column check box is checked, the second column check box determines, if the filter matches a set bit (checked) or an unset bit (unchecked). This way you can exactly specify, which bits are relevant for your filter and which state the bits should have for a matching filter.

The column `Bits` of the filter list shows the expected bit states for each filter with abbreviations for the bits. After the abbreviation a =0 follows when the filter matches for an unset bit; =1 follows when the filter matches for a set bit. The following abbreviations exist:

PP: Payload Preamble Indicator of the header
NF: Null Frame Indicator of the header (attention: bit is 0 in case it is a Null Frame)
SY: Sync Frame Indicator of the header
SU: Startup Frame Indicator of the header
VF: Valid Frame status bit
SE: Syntax Error status bit
CE: Content Error status bit
BV: Boundary Violation status bit

To understand the bits, you need to have FlexRay expert knowledge.

CAN, LIN, FlexRay and IP messages, sub type “I-PDU”:

The filter dialog for I-PDUs has the same fields `Filter type` and `Direction` as the other filter dialogs. Additionally, there is the `Comma separated list of I-PDU names` text field to enter one or several I-PDU names separated by comma for filtering.

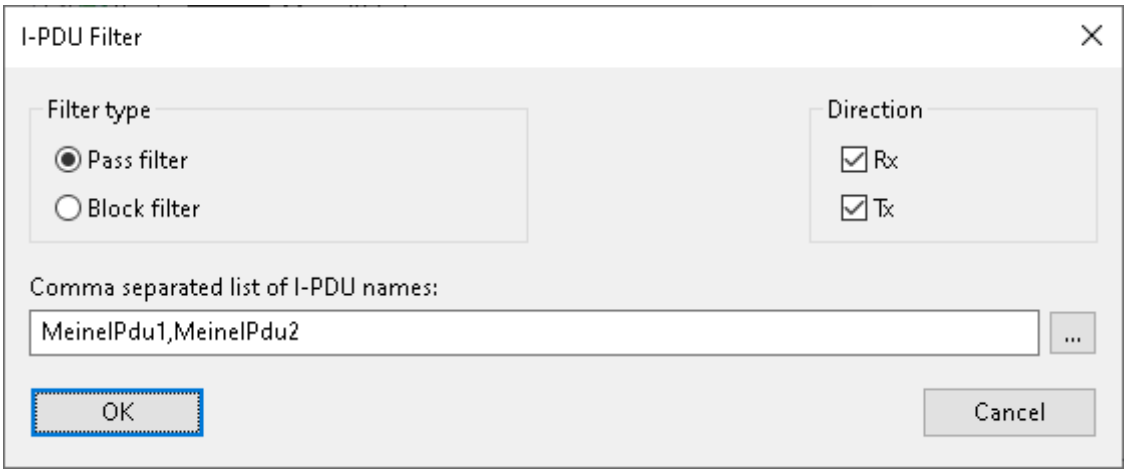


Figure 229 – Filter dialog for I-PDUs

Click on the ... button to open a dialog displaying all I-PDUs of the bus description file sorted by ECU for a comfortable selection.

For I-PDUs on IP/Ethernet channels you can additionally filter on IP addresses, port numbers and the transport protocol. The dialog has more fields accordingly:

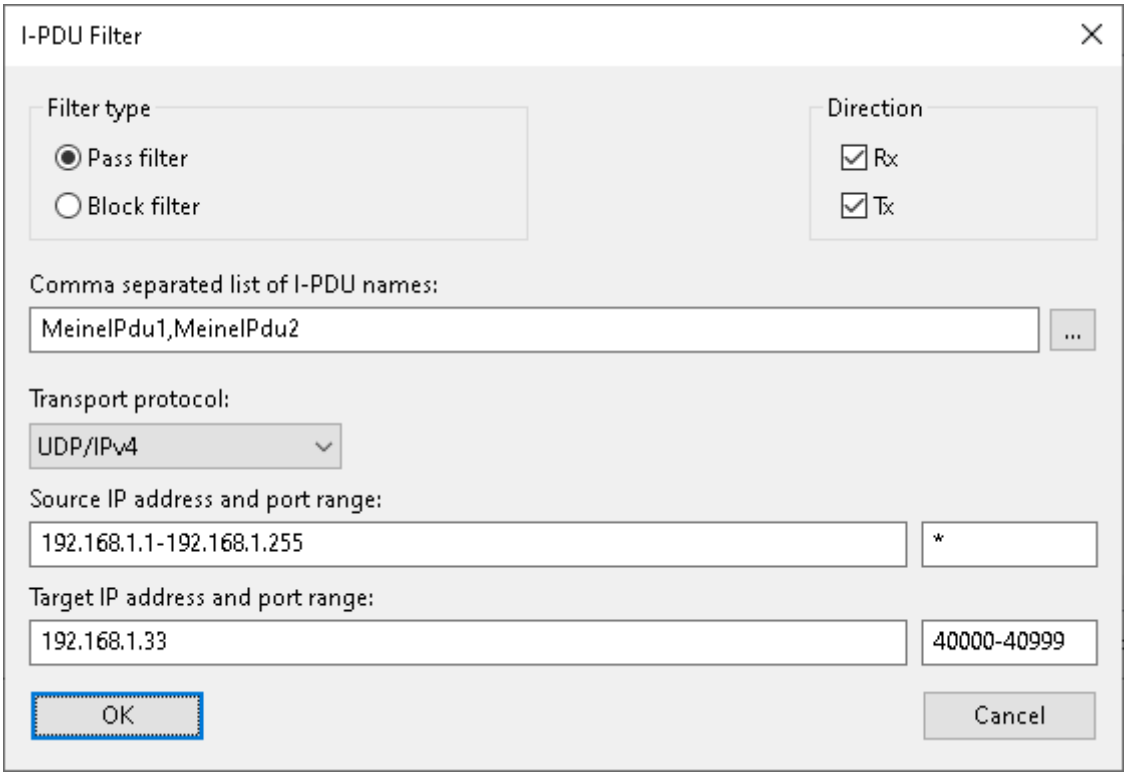


Figure 230 – Filter dialog for I-PDUs on IP/Ethernet channels

The lower fields allow single values or ranges. * or an empty fields means „all“ (so, no filtering of the according information). The IP address fields are not available when setting * for the transport protocol as then it is for any IPv4 or IPv6 address.

MOST messages, sub type “AMS” and “CMS”:
The filter dialogs for MOST AMS and CMS messages are identical in principle. However, for CMS messages the fields from FBlock are only relevant for “normal messages”.

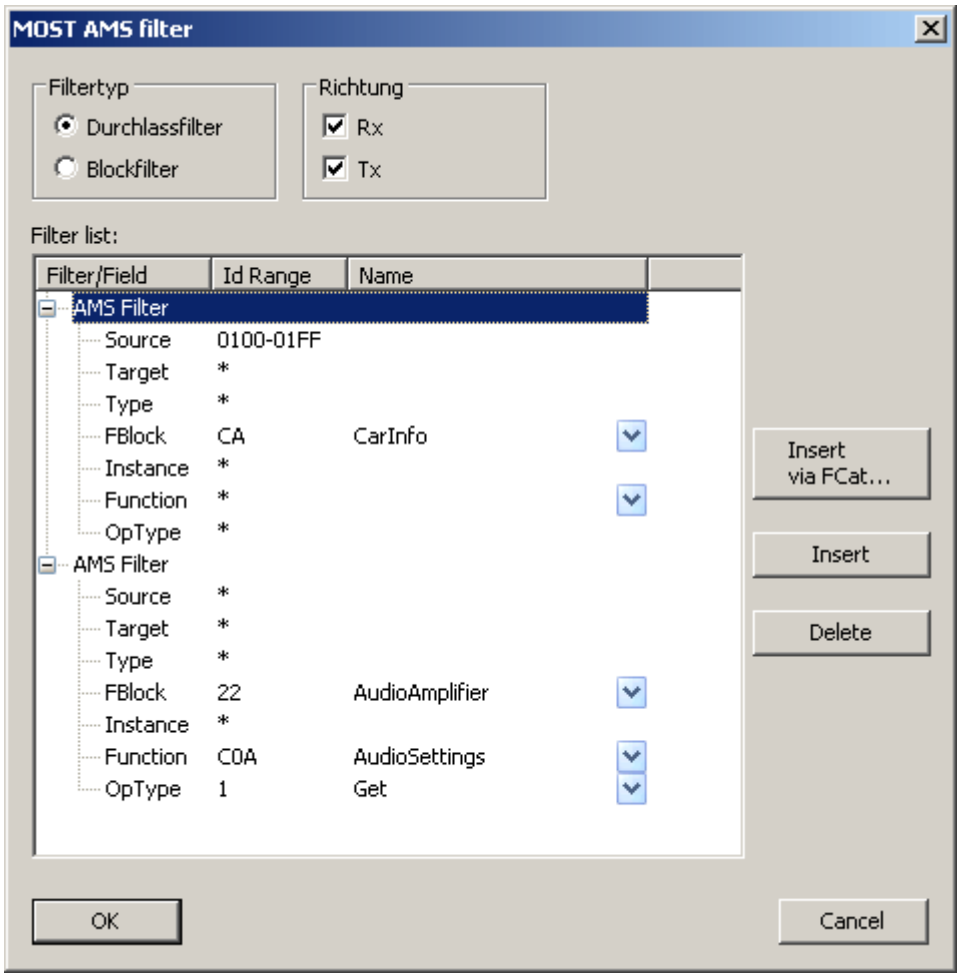


Figure 231 – Filter dialog for MOST AMS and CMS messages

`Filter list` displays a tree with two hierarchy levels. For every edited filter entry it displays an entry on the top level (AMS Filter or CMS Filter). Below this level you can see the editable fields for this filter: Source, Target, Type, FBlock, Inst, Fct and OpType. You can enter a single hexadecimal Id, an Id range (separated by -) or * for “all Ids” in the column `Id Range`. A click on a value field or F2 changes to the edit mode. You can set the fields FBlock, Fct and OpType by name via the MOST function catalog. To do this you can use the according combo boxes of the column `Name` (by using the mouse or F4 on the keyboard).

To create new entries click the button `Insert via FCat...` or `Insert` (or by using the key “Ins”). The later one immediately creates a new entry in the `Filter list`. The other one opens a dialog to create new entries via the MOST function catalog. The dialog displays the following hierarchy: FBlock → Fct → OpType.

To delete a filter entry, click on `Delete` or use the key “Del”.

IP messages, sub type “Raw”:
The dialog for Raw IP messages looks like this:

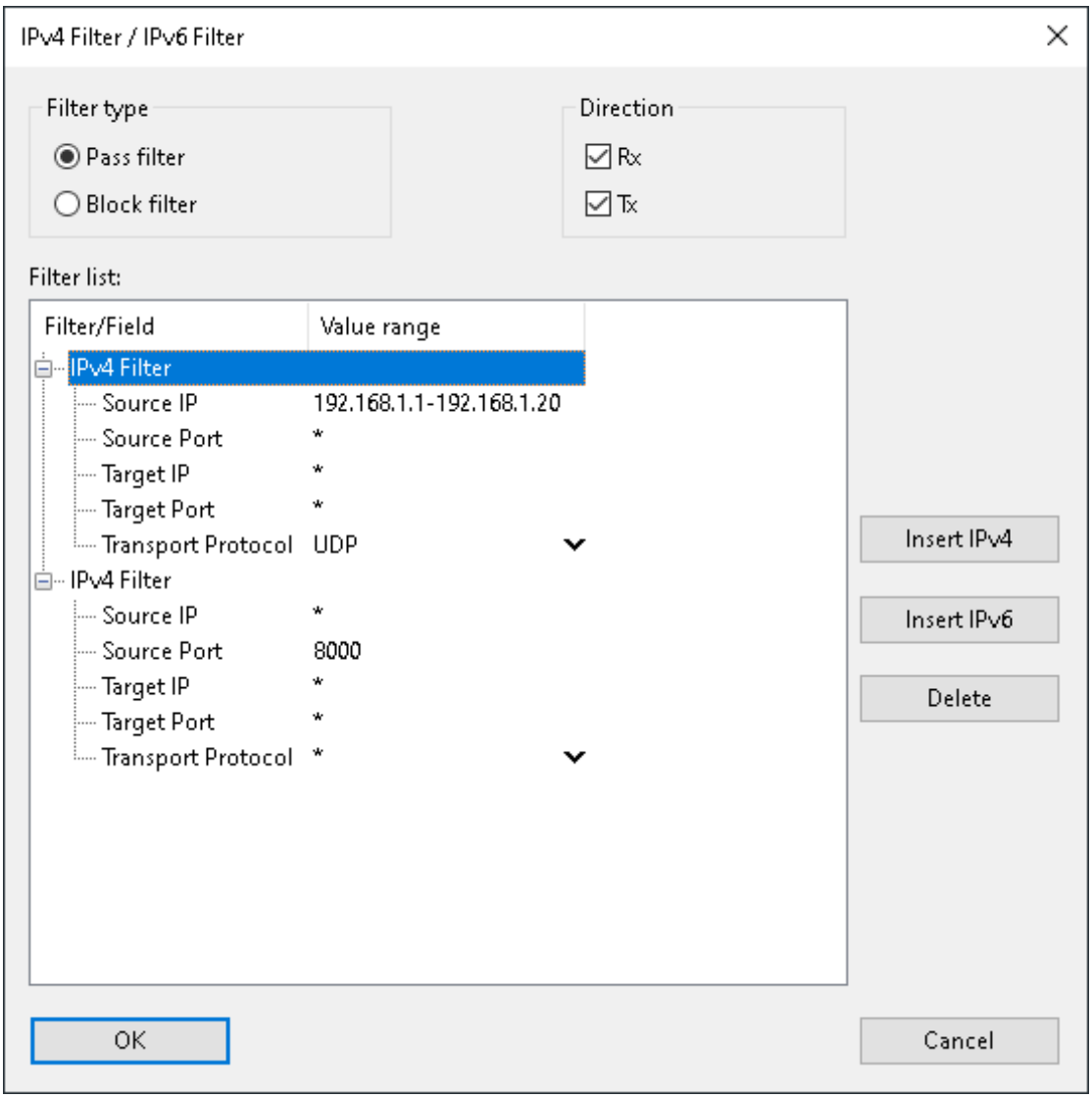


Figure 232 – Filter dialog for Raw IP messages

`Filter list` displays a tree with two hierarchy levels. For every edited filter entry, it displays an entry on the top level (IP Filter). Below this level, you can see the editable fields for this filter: Source IP (IP address of sending node), Source Port (port on which the message was sent), Target IP (IP address of receiving node), Target Port (port on which the message was received) and Transport Protocol (used protocol for transporting the message). You can enter a single value, a value range (separated by -) or * for “all possible values” in the column `Value range`. A click on the value or F2 changes to the edit mode. IPv4 addresses are 4 decimal values from 0 to 255 separated by the dot, IPv6 addresses are 8 hexadecimal 16-bit

numbers separated by the colon. Port numbers are decimal values from 0 to 65535. For Transport Protocol there is a combo box: UDP, TCP or *. * ignores the used transport protocol.

To create new entries, click the button `Insert IPv4` (or by using the key “Ins”) for IPv4 filters and `Insert IPv6` for IPv6 filters. To delete a filter entry, click on `Delete` or use the key “Del”.

IP messages, sub type “SOME/IP”:
The dialog for SOME/IP messages looks like this:

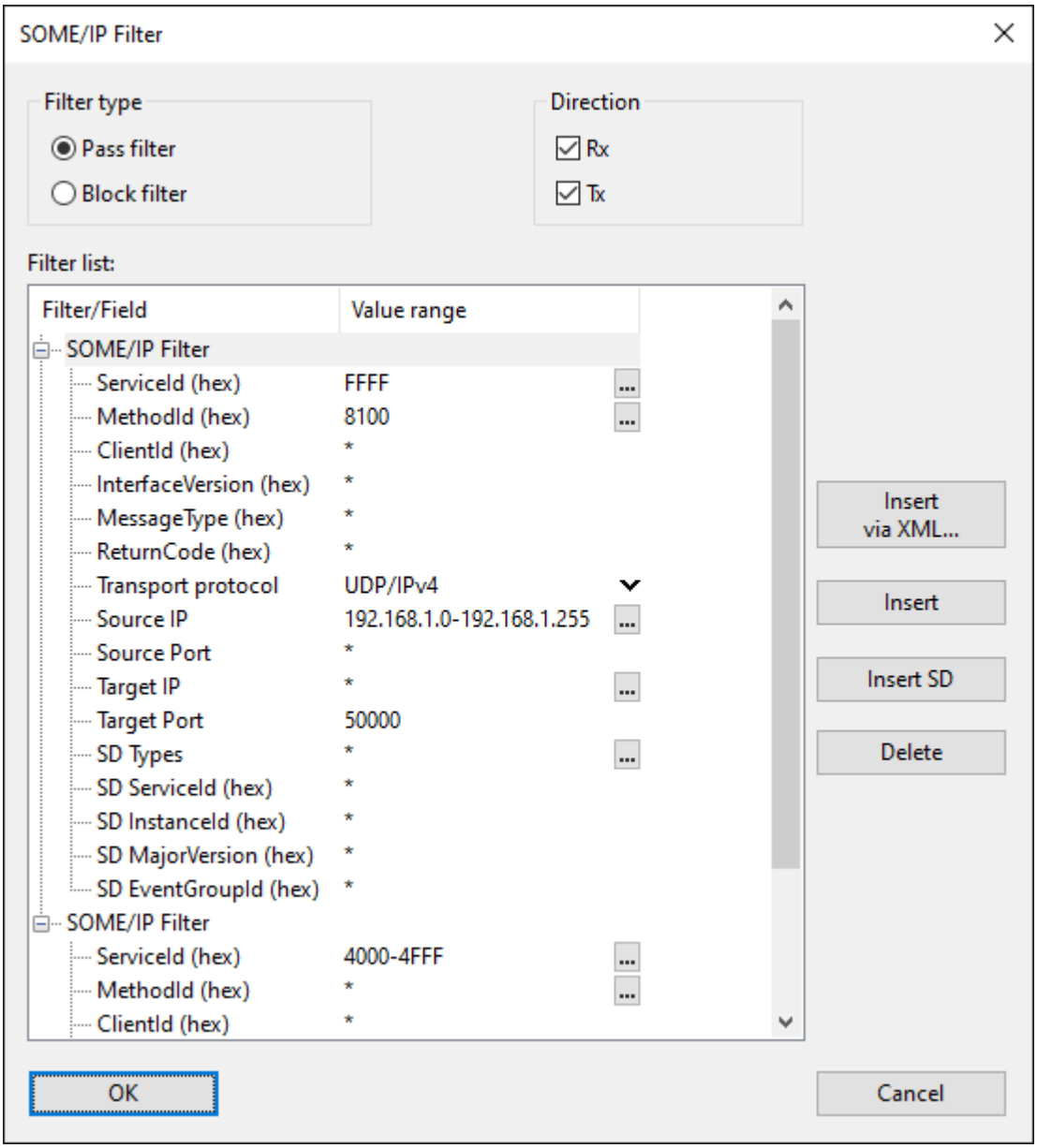


Figure 233 – Filter dialog for SOME/IP messages

`Filter list` displays a tree with two hierarchy levels. For every edited filter entry, it displays an entry on the top level (SOME/IP Filter). Below this level, you can see the editable fields for this filter: ServiceId, MethodId, ClientId, etc.. You can enter a single value, a value range (separated by -) or * for “all possible values” in the column `Value range`. A click on the value or F2 changes to the edit mode. Fields with (hex) in their name need a hexadecimal value input. The IP address fields are not available when setting * for the transport protocol as then it is for IPv4 as well as IPv6.

The `SD` (service discovery) entries only appear when you enter FFFF for `ServiceId` and 8100 for `MethodId` as then it filters service discovery messages. This way, you can filter the content of service discovery messages. The filter matches, if at least one of the service entries within the service discovery message matches your entered criteria(s).

To create new entries, click the button `Insert` (or by using the key “Ins”). To delete a filter entry, click on `Delete` or use the key “Del”. Click on `Insert SD` to insert a filter especially for service discovery messages with the service Id FFFF and the method Id 8100. `Insert via XML...` is only available when an AUTOSAR XML is attached to the message channel. It opens a dialog for adding one or several SOME/IP PDUs via a tree list view.

4.4.14.2 Handling of Ethernet

Ethernet is a special challenge for the message handling as it uses more layers with protocols for the communication compared to CAN as an example. For CAN an Id defines the content of a frame. It does not matter which CAN participant sends or receives the frame. For Ethernet such an Id per frame does not exist. Instead of that, MAC addresses, VLAN Ids, IP addresses and transport protocols like TCP and UDP are important. This happens on the layers 1-4 of the classic OSI layer model. The transport of the typical automotive messages like I-PDUs and SOME/IP or UDS for diagnostics or XCP happens on top of that. TCP has an additional obstacle as it transports a data stream after creating a connection with a special mechanism. If the data stream transports classic messages or an audio stream as an example is not detectable by viewing the data. It is only possible to extract messages like I-PDUs correctly from such a data stream if additional information is available and if the complete stream beginning with the TCP connection establishment is available. Single Ethernet frames only carry parts of the stream. It is not possible to determine where each PDU starts and ends without analyzing the whole data stream since the connection establishment as each PDU has an own length field, which is part of the data stream. That is why PROVEtech:TA actually needs information from an AUTOSAR-XML file to extract single messages.

PROVEtech:TA stores Ethernet messages not on Ethernet frame level, but it stores IP addresses, the transport protocol (UDP or TCP on IPv4 or IPv6) and the payload. This is the information necessary to send/receive the data via so-called sockets (Berkeley sockets). The layers below like MAC addresses, IP header data, VLAN Ids, etc. are not important for that.

What does this mean for the whole Ethernet handling with PROVEtech:TA? First of all, there are several sources for messages on Ethernet. The data acquisition formats MF4, ASC and PCAPNG always store raw Ethernet frames. On the model platforms the available data is either raw Ethernet frames as well (e.g. monitoring via PCAP like for ETAS LabCar or dSPACE SCALEXIO) or the data is from Berkeley sockets where the lower layers (MAC addresses, VLAN Id, etc.) are not available. This happens when using the PROVEtch:RE rest bus simulation as an example.

To be able to handle Ethernet frames, a preprocessing is necessary to extract I-PDUs or SOME/IP PDUs from the raw data (concatenating and breaking the raw Ethernet frames). Therefore, PROVEtech:TA uses a special module for the import of data acquisitions as well as monitoring of raw Ethernet frames. This module distinguishes IPv4, IPv6, UDP and TCP. UDP already transports messages with a specific length. PROVEtech:TA does not need an AUTOSAR-XML file to concatenate these (IP fragmentation). For TCP it is necessary to have an AUTOSAR-XML file to detect which data streams transport I-PDUs with own header Ids and lengths. PROVEtech:TA ignores other TCP streams. The differentiation happens on base of IP addresses and TCP port numbers.

For UDP as well as TCP the amount of parallel data streams PROVEtech:TA can handle in parallel is limited. As an example, for UDP it handles a maximum of 20 fragmented data streams in parallel. For TCP a reception/import is only possible with AUTOSAR-XML file and with I-PDUs that have a header Id. It ignores too large I-PDUs and it does not support IP fragmentation for TCP. It also does not support out-of-order reception for UDP or TCP.

For the export of Ethernet messages to MF4, ASC and PCAPNG, PROVEtech:TA must change the stored messages to raw Ethernet frames. As PROVEtech:TA does not store raw Ethernet frames, it must simulate missing information like MAC addresses or IP headers. For TCP there is also a simulation of the connection establishment and ACK packets. This means, the generated files do not represent the original Ethernet communication. Only the transported payload and IP addresses and ports correspond to the original data.

4.4.15 The oscilloscope display



The oscilloscope display is only available in case the model supplies the according data. A generic message channel with information about the transported signals transports the data.

To create an oscilloscope display right click on a free part of the workpage and select *Create Oscilloscope…*. This creates a new display at the clicked position and opens the property dialog for configuring the signals (see chapter ***4.4.15.1 The property dialog of the oscilloscope display***).

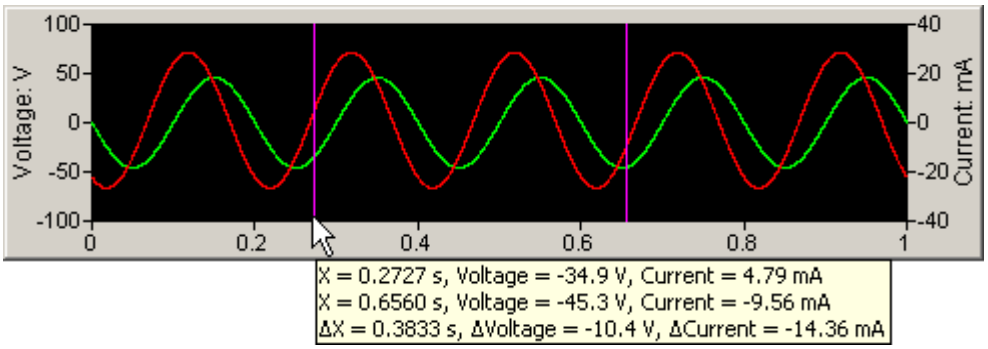


Figure 234 – The oscilloscope display

The oscilloscope display can show up to two signals with two separate Y-axes and a common X-axis simultaneously. In opposite to the strip chart control it typically displays triggered high frequency data. The model takes care of the triggering and it typically provides signals to determine the trigger threshold, display range, etc..

In the offline mode (see chapter ***4.5 The offline mode***) you additionally have the possibility to set up to two cursors by clicking and dragging in the plot area. A tool tip shows the X and Y values at the cursor positions. In case of two cursors it also displays the delta values (right cursor value minus left cursor value). This is visible in the upper figure.

4.4.15.1 The property dialog of the oscilloscope display

The property page `Display` of the property dialog of a strip chart control has the following appearance:

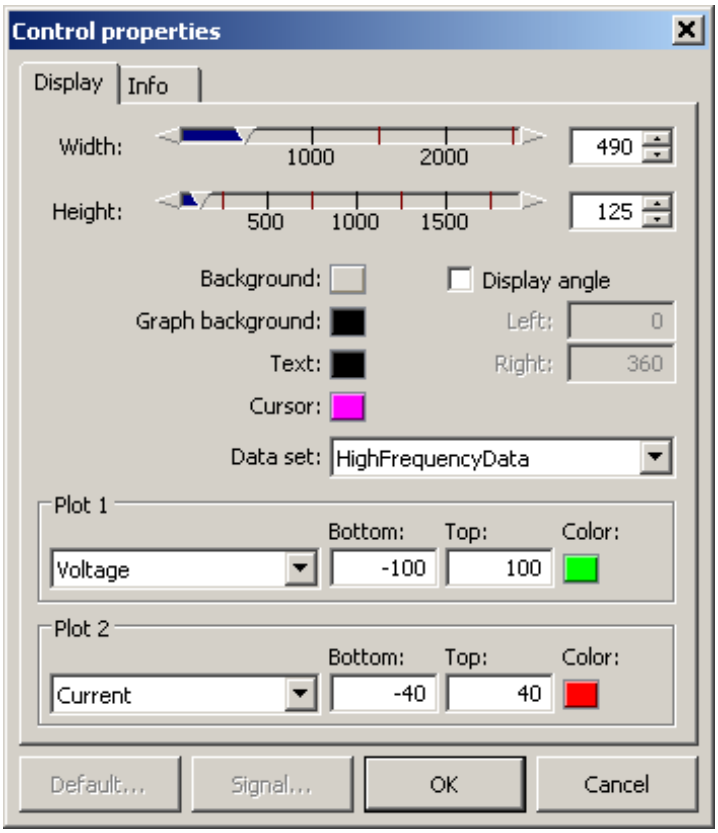


Figure 235 – Property dialog of the oscilloscope display

Here you can set the `Width` and `Height` of the control, as well as the `Background` color (border), the `Graph background` color (the background inside the graph), the `Text` color (axes labels) and the `Cursor` color (only visible in the offline mode).

The check box `Display angle` can only be activated in case the provided signal data contains angle information (special case crank angle). In case the check box is activated the X-axis displays an angle range instead of a time range. This setting is global for all oscilloscope displays. Use `Left` and `Right` to select the angle range.

When showing a time range the minimum of the X-axis is always 0 and the maximum depends on the transmitted signal data.

Use the combo box `Data set` to select a set of signal data to display. All signals of a data set have common X-values.

Use the combo box of `Plot 1` to select the first displayed signal, its Y-axis range (`Bottom` and `Top` for the left Y-axis) and the line color. Select "<no assignment>" in the combo box to disable the left Y-axis and the according signal display.

Use the combo box of `Plot 2` to select the second displayed signal, its Y-axis range (`Bottom` and `Top` for the right Y-axis) and the line color. Select "<no assignment>" in the combo box to disable the right Y-axis and the according signal display.

4.4.16 The group control

You can create group controls using three different context menu items (see chapter ***4.3.1 Context menus***):
1. Right click on selected controls: *Embed Control(s) in new Group Control*
2. Right click on a free area: *Create Empty Group Control*
3. Right click on a free area: *Create Linked Group Control*

A group control is a frame where you can embed other controls:

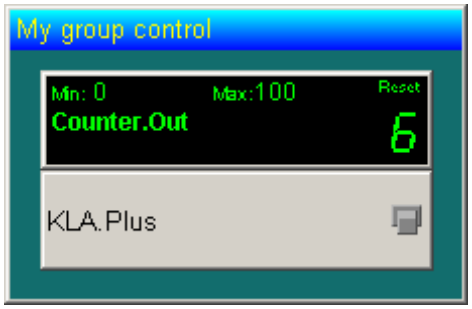


Figure 236 – The group control

Group controls can also contain other group controls. Beside this they behave identical to pages of the workpage.

Linked group controls get their content from other files and display the icon in the upper right corner. Changes within such group controls get lost when closing the workpage. The content is managed in the original file.

The chapter ***4.3.2 The property dialog for pages and group controls*** describes the according property dialog.

Group controls have the additional context menu item *Resize Group Control to fit to Content*.

4.4.17 The widget

The widget displays user defined content created via Basic scripts. This way you can display any information and input masks independent from model signals and messages. You create widgets by right-clicking on a free place of the workpage or cockpit. This shows the according context menu item *Create Widget…*. At the beginning, the widget is deactivated. The according properties dialog appears:

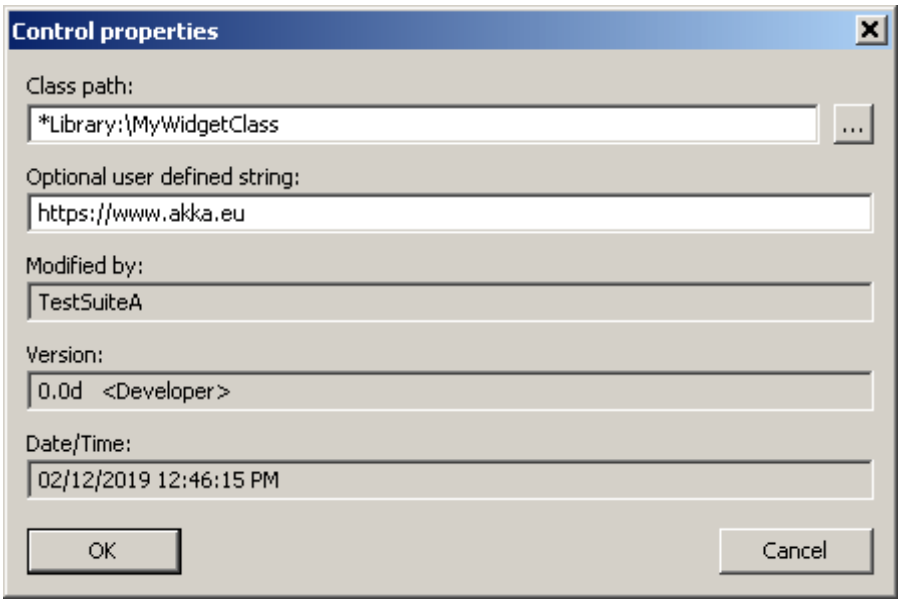


Figure 237 – Property dialog of the widget

Select the path to a Basic class in the Test Manager database or on the file system via the text field `Class path`. The class name has no meaning. Each widget instantiates a new object of the class when it is created. In case you reference the same class from different widgets, it creates an own object for each widget with own variables. However, if the class uses a code module or Basic object, the instances share their variables. All widgets work with the same Basic instance running in an own thread.

The fields `Modified by`, `Version` and `Date/Time` display additional information about the specified Basic module if available.

Use the field `Optional user defined string` to enter any optional string which PROVEtech:TA passes to the Basic interface method `Create2` (see below). This way it is possible to parametrize the Basic class per widget.

As an example, create a Basic class with the following content:

```
'#Language "WWB.NET"

Option UsesAmbiguousError On
Option Explicit

Imports System.Windows

'entry function called by the workpage to create the widget
Sub Create2(ByVal hParentWnd As Integer, ByVal iWidth As Integer, _
        ByVal iHeight As Integer, ByVal strUserString As String, _
        ByVal dViewScaling As Double)
    'create intermediate window to use parent window handle
    '&h5000000=WS_VISIBLE Or WS_CHILD
    '4=WS_EX_NOPARENTNOTIFY
    Dim wnd As New Interop.HwndSource(0, &h50000000, 4, 0, 0, iWidth, _
        iHeight, "", New System.IntPtr(hParentWnd))
    'create text block showing strUserString and attach it to the window
```

```
    Dim txt As New Controls.TextBox
    txt.Text=strUserString
    txt.Foreground=New Media.SolidColorBrush(Media.Colors.Aqua)
    txt.Background=New Media.SolidColorBrush(Media.Colors.Blue)
    txt.BorderThickness=New Thickness(0)
    txt.VerticalContentAlignment=VerticalAlignment.Center
    txt.HorizontalContentAlignment=HorizontalAlignment.Center
    txt.IsReadOnly=True
    wnd.RootVisual=txt
End Sub
```

In case you reference this class from a widget, it simply shows the text of the field `Optional user defined string`.

The signature of the method `Create2` in the Basic class must be exactly as in the upper example. It receives the window handle of the widget control itself as a parameter used as parent window. Additionally, `Create2` receives the current width and height of the widget client area and the user defined string mentioned above. The last parameter gets the currently selected view scaling (see chapter ***4.1.1 View scaling***). As an example, the value 1.25 means 125%.

Furthermore, PROVEtech:TA supports an optional method with the name `ChangeViewScaling` and the following signature:

```
Public Sub ChangeViewScaling(ByVal dViewScaling As Double)
```

In case of changing the view scaling, the widget gets the new view scaling value via this method shortly before resizing the window.

The upper example does not support switching the view scaling. Below is an example with switching the view scaling. The scaling feature exists since PROVEtech:TA version 2023. Therefore, the widget interface changed beginning with version 2023. Older versions only knew the method `Create` instead of `Create2` without the last parameter `dViewScaling`. For compatibility to old widgets, PROVEtech:TA additionally searches for `Create` in case `Create2` does not exist.

The widget expects that `Create2/Create` creates exactly one child window for “hParentWnd”. As an example, the widget forwards size changes to that child window via the standard Windows mechanisms. Additionally, the child window is destroyed automatically when the widget is destroyed.



The widget couples the workpage with Basic scripts. As an example, it exchanges window messages while resizing the widget or destroying it. The workpage is blocked until the Basic thread finishes the message processing. This means if the Basic thread is blocked due to some calculations in the script, the workpage does not respond anymore.



The workpage has no control concerning the content of widgets. Please think about any impact on security/safety when using them.



Some operations do not work as expected while using widgets as PROVEtech:TA has no influence on them. As an example, you can only move widgets when placing the mouse cursor on the upper or left border of the control. Additionally scrolling and moving other controls over widgets does not work as expected as the normal drag and drop is deactivated over widgets.



In general, the mechanism for creating widgets via the `Create2` method works with any window mechanism of Microsoft Windows. However, we only tested it with Microsoft Windows Presentation Foundation (WPF) windows based on .NET like the upper example.



The functions for printing controls or creating images might not work with widgets as the Microsoft Windows Presentation Foundation does not support the used mechanism.



Many actions in PROVEtech:TA lead to destroying and re-creating the workpage controls like switching to the offline mode or loading a model. This also leads to destroying the Basic class instance and re-creation for each widget. Attention: In case the class is still in the Basic cache, Basic uses the version from the cache when re-creating an instance even if you changed the Basic code in the meanwhile. In case of doubt you should restart PROVEtech:TA to make sure that it uses the changed Basic code.



In case your Basic code raises an error, you can see the error information in the PROVEtech:TA Log (***3.6.1 PROVEtech:TA Log***).

A typical use case for the widget is a user interaction where you code the according action in the Basic program. To be able to react on actions in the Basic program, you typically create event handlers like in the following example.



Figure 238 – A widget

Here is the according Basic code, which displays a message when clicking the button. Please be aware that the “MacroData” command exists only for PROVEtech:TA 2023 SE or later.

```
'#Language "WWB.NET"

Option UsesAmbiguousError On
Option Explicit

Imports System.Windows
Imports System.Windows.Media

'the button control reference
Private WithEvents m_button As Controls.Button
'the root element
Private m_root As FrameworkElement

'the control layout as XAML
#Data "XAML"
<Grid xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
      xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
      Background="DarkGray">
    <Button x:Name="MyButton" Height="21" Width="90">Click Me</Button>
</Grid>
#End Data

'entry function called by the workpage to create the widget
Public Sub Create2(ByVal hParentWnd As Integer, ByVal iWidth As Integer, _
        ByVal iHeight As Integer, ByVal strUserString As String, _
        ByVal dViewScaling As Double)
    'create intermediate window to use parent window handle
    '&h5000000=WS_VISIBLE Or WS_CHILD
    '4=WS_EX_NOPARENTNOTIFY
    Dim wnd As New Interop.HwndSource(0, &h50000000, 4, 0, 0, iWidth, _
        iHeight, "", New System.IntPtr(hParentWnd))
    'load XAML from m_strXaml
    Dim xaml As Object=Markup.XamlReader.Parse(MacroData("XAML"))

    wnd.RootVisual=xaml
    m_root=wnd.RootVisual
    ChangeViewScaling(dViewScaling)

    'get button control reference
    m_button=xaml.FindName("MyButton")
End Sub

'called by the workpage in case the view scaling changes shortly before
'the window resizing
Public Sub ChangeViewScaling(ByVal dViewScaling As Double)
    Dim st As ScaleTransform
```

```
    Dim ps As PresentationSource=PresentationSource.FromVisual(m_root)
    If ps Is Nothing Then
        st=New ScaleTransform(dViewScaling, dViewScaling)
    Else
        'replace the default scaling by our own
        Dim mx As Matrix=ps.CompositionTarget.TransformToDevice
        st=New ScaleTransform(dViewScaling/mx.M11, dViewScaling/mx.M22)
    End If
    m_root.LayoutTransform=st
End Sub

'handler function called when clicking the button
Private Sub m_button_Click(ByVal sender As Object, _
        ByVal e As RoutedEventArgs) Handles m_button.Click
    MsgBox("Button clicked.", vbInformation)
End Sub
```

4.4.18 The video control

The video control displays video data from a source like a frame grabber or camera in the online mode and in the offline mode (see chapter ***4.5 The offline mode***) it displays single frames from a video file which is part of a data acquisition. For the online mode you configure your video devices in the settings dialog (see chapter ***3.2.7.3 Workpage***).



Figure 239 – Video control displaying a cell phone recorded with a video camera

To create a new video control, just right-click on an empty spot of the workpage or cockpit and select the menu item *Create Video Control*, which displays a sub menu with all available video sources. In the offline mode, you can create several controls with the same video source. In the online mode, you can only create one control per source and the sub menu only displays sources not used yet.

In the online mode, it should display the current live view of the source. The next chapter describes the property dialog for the online mode.

In the offline mode, it displays the frame that covers the time `T1` or `T2` (see chapter ***4.5.1 The overview graph, evaluation of measure data***). The video files in the MP4 file format have own time stamps, which do not match the time base of the data acquisition. There is only loose time synchronization when recording videos during a data acquisition. Please read chapter ***3.4 The Data Acquisition menu*** for more information about that. In the end, data acquisitions store a time offset value per video file that describes which time in the video file corresponds to the time 0 of the data acquisition. You can manipulate that value by opening the settings dialog of the video control, e.g. by double-clicking it. You can save these changes when closing the offline mode.



Make sure to allow the camera and microphone access in the Windows privacy settings. The according global switch is sufficient. There is no application specific switch. Otherwise, the PROVEtech:TA Log shows a generic “access denied” error when initializing the video/audio sources. The installed virus scanner can suppress the camera or microphone access as well. In that case you need an exception rule for PROVEtech:TA.



Video/audio processing in PROVEtech:TA uses the Microsoft Media Foundation technology. This includes hardware acceleration via the graphics card, if the hardware and drivers support that. Please use an up-to-date driver from the manufacturer. Without hardware acceleration recording videos might fail as PROVEtech:TA uses the H.264 encoding standard which needs according performance.



Video/audio recording and display might be disturbed by power saving of Windows. Please disable the power saving options.

4.4.18.1 The property dialog of the video control

The property dialog of the video control in the online mode provides the possibility to change the settings provided by the video source and its driver. The amount of properties displayed depends on the video source and its driver and some do not support any settings at all.

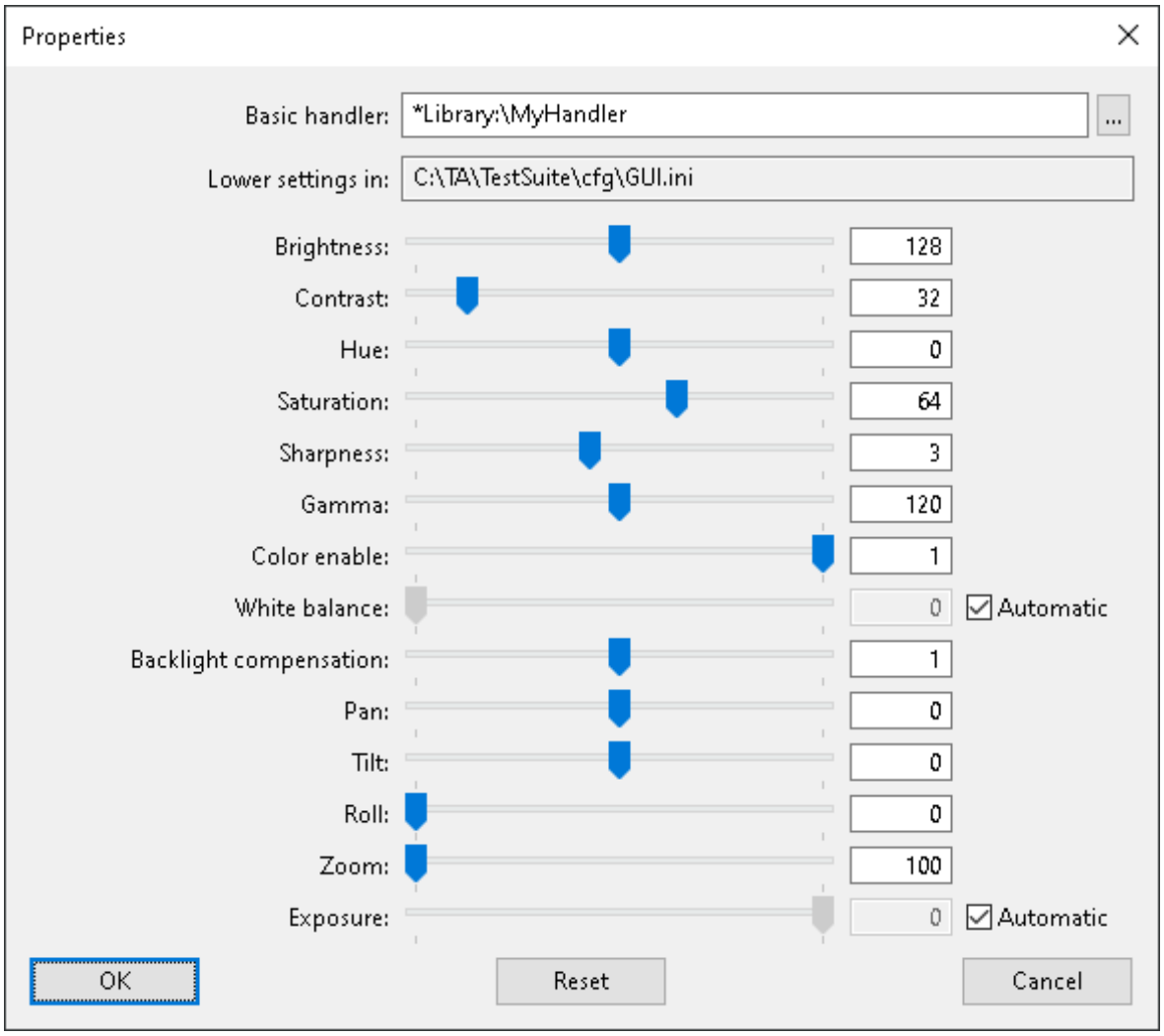


Figure 240 – Property dialog of the video control

The global GUI.ini file stores these settings and clicking `OK` directly writes the values in there. The `Lower settings in` text box shows the path to that file. This means, these settings are not part of the workpage or cockpit file. All the settings provided by the video source have a slider control and an edit box showing the selected number. PROVEtech:TA is unaware of the meaning of these numbers and the minimum/maximum and default values come from the video source. Also, the possibility to select `Automatic` comes from the video source. Clicking `Reset` sets the values back to the reported default values.

The upper field `Basic handler` is optional and directly stored in the workpage or cockpit file. Without it, you can move the mouse over the video window and move it via drag and drop. When specifying a handler, the mouse messages are forwarded to that Basic script instead. Moving the window then only works when dragging the upper or left window border. The Basic script can have these three functions (all optional) with exactly these parameters:

Called when the user presses the left mouse button:

```
Function MouseDown(ByVal dMousePosX As Double, ByVal dMousePosY As Double, ByVal
strVideoSource As String) As Boolean
```

Called when the user releases the left mouse button:

```
Sub MouseRelease(ByVal dMousePosX As Double, ByVal dMousePosY As Double, ByVal
strVideoSource As String, ByVal bCapture As Boolean)
```

Called when the user moves the mouse pointer:

```
Sub MouseMove(ByVal dMousePosX As Double, ByVal dMousePosY As Double, ByVal
strVideoSource As String, ByVal bCapture As Boolean)
```

`dMousePosX` and `dMousePosY` are the relative mouse pointer position within the video from 0 (upper left) to 1 (lower right). All values >=0 and <1 are within the video. Other values are outside (e.g. on black bars around the video). `strVideoSource` is the name of the video source. The return value of `MouseDown` determines, if clicking the left mouse button shall “capture” the mouse (return value `True`). Thus, all further mouse movement is forwarded to the video window even if the mouse pointer leaves the window. And it always gets the `MouseRelease` call even if the mouse button release happens outside the window. If captured, the call to `MouseRelease` also happens if the window loses the capture (e.g. when a message box pops up). The `bCapture` parameter tells if the window had the capture when the event happened.

Please keep in mind that your Basic script is only instantiated once if you use the same path in several Basic handler controls. You can use the `strVideoSource` field to differentiate them. Use a Basic class to have separate instances.

When creating the control PROVEtech:TA loads the specified script. Script changes only get active when loading it the next time. PROVEtech:TA does not load a script again if it was loaded already for another control before. As long as no other control references the same script it is sufficient to open and close the property dialog to re-load the script.



The PROVEtech:TA main thread is blocked while calling your Basic functions. Thus, these Basic functions should return as fast as possible and never block. As an example, displaying message boxes within the functions is a bad idea.

4.5 The offline mode

The offline mode allows you to view measurements in detail, i.e. to look at individual signal curves down to each value sampled.

To view an already saved data acquisition use the menu item *Data Acquisition|Display Data Acquisition from file…* (see chapter ***3.4 The Data Acquisition menu***). PROVEtech:TA automatically switches to the offline mode after selecting a data acquisition.

To create a new data acquisition, select your measurement signals first (***4.2 The Signal Selection window***) and then select the menu item *Data Acquisition|Start Data Acquisition with saving to file* (***3.4 The Data Acquisition menu***). Terminate the measurement via *Data Acquisition|Stop Data Acquisition*. As long as you do not save the data acquisition, it is accessible as "unsaved data acquisition" which PROVEtech:TA automatically displays when you switch to the offline mode via *Data Acquisition|Offline* or by pressing the button  on the toolbar.

In the offline mode, PROVEtech:TA hides the cockpit and in the upper part of the workpage a new window appears that contains the overview graph (see ***4.5.1 The overview graph, evaluation of measure data***).

You can leave the offline mode by selecting the menu item *Data Acquisition|Offline* again or via the menu item *Data Acquisition|Stop Data Acquisition* or the button  .

4.5.1 The overview graph, evaluation of measure data

The overview graph (***Figure 241***) shows one of the signals included in the data acquisition for the entire duration of the measurement.

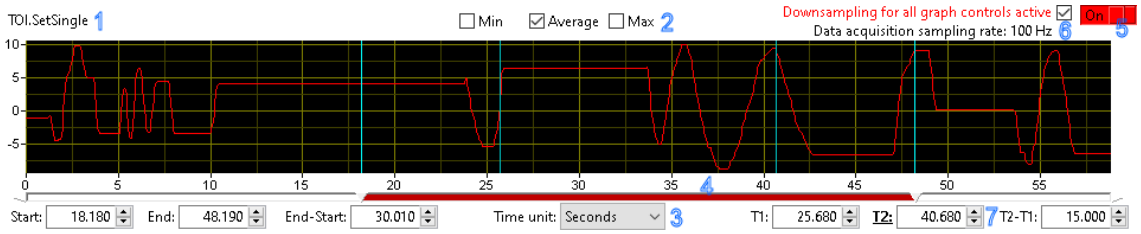


Figure 241 – The overview graph

Change the displayed signal via drag and drop of different signal on the overview graph. As an alternative click on the icon of a different signal in the signal selection window (***4.2 The Signal Selection window***). The signal selection window displays the signal currently displayed in the overview graph with the  icon. All other signals have the  icon.

The individual elements of the overview graph are described below according to the numbers in the ***Figure 241***:

1. Name of the signal currently displayed in the overview graph.
2. Check boxes for selection of the display mode. Normally the resolution of a monitor does not permit each sample of a signal to be shown. For this reason, PROVEtech:TA combines several samples per pixel in the overview graph. To do this, PROVEtech:TA performs three calculations: minimal value, average value and maximum value. The options `Min`, `Average` and `Max` are available for selection to display one or several of these curves.
3. Combo box for selection of the time unit. This time unit applies to the overview graph and all (multi-)strip chart controls of the workpage. The following units are available: microseconds, milliseconds, seconds, minutes, hours, days and weeks.

4. Use the double slide control or the vertical cyan (color adjustable) cursors above for a coarse selection of the range of the measurement, which is of particular interest to you. More accurate entries are possible via the edit boxes `Start`, `End` and `End-Start` (=Difference between `Start` and `End`). The time unit of these entries corresponds to the unit set under 3.

5. This button is used to switch the updating of the display controls on the workpage on and off. When updating is enabled, the controls continually show the range selected under 4. and are automatically updated by any change to the range. Controls that cannot display a range (e.g. the number control or the bitmap control or the video display) show the according signal value / video frame at the time `T1` or `T2` (see 7.) – the one which is currently bold and underlined. Click on the text `T1` or `T2` to select which one to use.

6. Just like the overview graph, the (multi-)strip chart controls cannot always show every sample in the selected range due to the limited resolution of your monitor. Additionally, loading all the data of the selected range could take a lot of time and memory. To avoid this, PROVEtech:TA automatically calculates a downsampling rate depending on the range and does not load every sample of the selected range. If the downsampling rate is 3, for example, this means that only every third sample is displayed. However, in case the signal changes its value just for a very short time, it might happen, that you do not see that. The check box activates and deactivates the automatic calculation of the downsampling rate. PROVEtech:TA passes all samples in the selected range to the (multi-)strip chart controls in case the downsampling is deactivated (downsampling rate=1). However, when you exceed a certain range, you cannot disable the downsampling anymore as this limits the main memory usage and loading time. The text left of the check box shows if downsampling is currently active or not. In case downsampling is active, the text is red.
   The displayed sampling frequency below corresponds to the fastest sampling rate in case the data acquisition contains several different sampling rates.

7. `T1` and `T2` define two points in time within the range selected under 4. Two additional cyan (color adjustable) vertical cursors in the overview graph and in all (multi-)strip chart controls display them. You can use the cursors to adjust them. `T2-T1` is the difference, i.e. the duration of the range. `T1` and `T2` can only assume values available after the downsampling described under 6. Please refer to the sections below for further information.

Figure 242 shows a multi-strip chart control with the range set under 4.

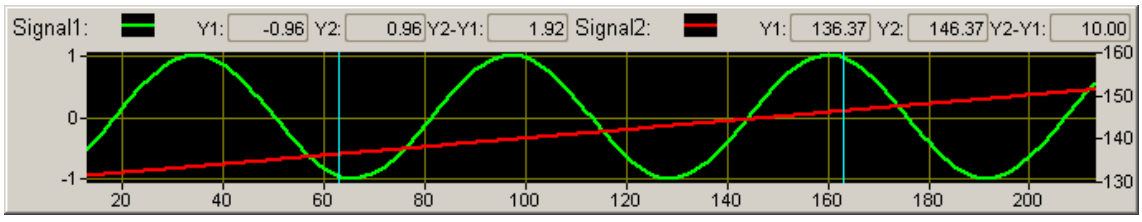


Figure 242 – Multi-strip chart control in the offline mode

Two vertical cyan (color adjustable) cursors are available to indicate the measured values at the positions defined with these cursors and to define new ranges. `Y1` shows the value of the corresponding signal at the left cursor position. `Y2` shows the value at the right cursor position. The edit box `Y2-Y1` shows the difference between these two values.

When clicking on a strip chart control with the right mouse button compared to the online mode the additional menu item *Zoom Range (T1..T2)* appears. It takes over the current cursor positions at T1 and T2 as the new display range (see 4.).

When you right click on the overview graph, the following menu pops up:

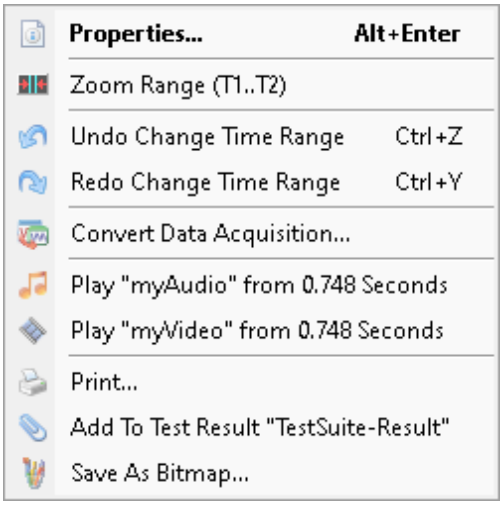


Figure 243 – Context menu of the overview graph

Use the item *Properties…* to open the dialog described in chapter ***4.5.2 The property dialog of the overview graph***. The menu item *Zoom Range (T1..T2)* corresponds to the one of the (multi-)strip chart controls and takes over the current cursor positions at T1 and T2 as the new display range (see 4.).

The item *Convert Data Acquisition* of the context menu opens the dialog described in chapter ***3.4.1 Converting data acquisitions***.

The menu items *Play "…" from x Seconds* only appear if the loaded data acquisition has video/audio recordings (icon for pure video file, for pure audio file, for video and audio file). The item opens the according media file in a separate window and starts playing it at the displayed time. The position of the right-click determines that time. In case it is within the graph, it is the position on the time axis. If it is close to one of the 4 vertical markers it uses the marker position. A right-click on one of the words `Start`, `End`, *`T1`* or `T2` uses the according time value. The current time displayed in the opened window in the lower right corresponds to the time in the data acquisition, which means that it takes a possible time offset between the media file and the data acquisition into account. However, this depends on the quality of the synchronization and the setting in the configuration (see setting `Delay` in the chapter ***3.2.7.3 Workpage***).

The same menu item exists in the context menu of the video control (see chapter ***4.4.18 The video control***) to play the video beginning with the displayed image.

Read the chapter ***4.3.1 Context menus*** for a description of the other menu items.

In case your data acquisition contains messages, you can use the message window (see chapter ***4.4.14 The message window***) to display the messages in the range set under 4. In the overwrite mode messages with the same Id are overwritten, so you will see the last one in the range. For messages the downsampling rate of 6. has no effect, so changing the range while the button of 5. is active can take much time depending on the amount of message data the range covers. In case the message data in the range exceeds 512 MB, PROVEtech:TA aborts loading it. It then displays an exclamation mark top right beside the overview graph.

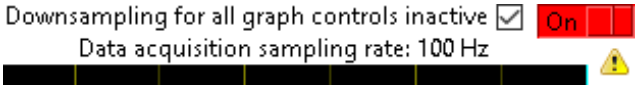


Figure 244 – Exclamation mark if message data is too large

Right click on a message in the upper list of the message window to open the following context menu:

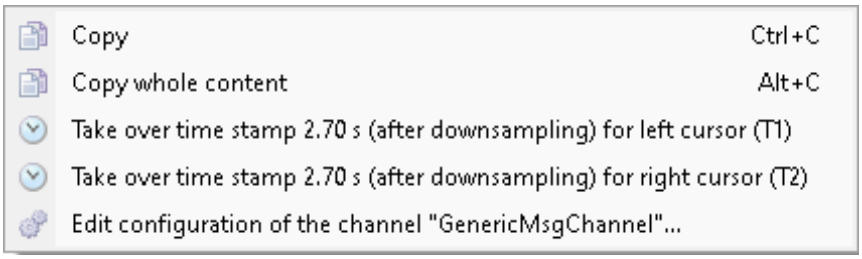


Figure 245 – Context menu of the message window

The third menu item takes over the time stamp of the message for T1, the fourth one for T2 (column `SW time`). In case after this action the time T1 would be after T2, the values are exchanged as T1 is always before T2 per definition. Please keep in mind that T1 and T2 can only take over time stamps after the downsampling described under 6., so the context menu displays the time stamp after the downsampling calculation.

Use the last menu item to change the message channel configuration for the channel of the clicked message. This is the same as changing the configuration in the property dialog (see chapter ***4.5.2 The property dialog of the overview graph***).



A data acquisition can contain data with different sampling rates (multi-rate). Furthermore, the message time stamps can also have an own resolution. Additionally, the start and end time of the different data streams might vary. This means, not all displayed data must necessarily start at time 0 and end at the same time. The display in the offline mode depends on the quality of the recorded data. This also means, that it might display data (signals/messages/video frames) to be at the same time, but it was originally at different points in time. This depends on the synchronization methods used while recording the data and this mainly depends on the possibilities/quality of the used hardware/software platform. As an example, real-time violations that happened during the data acquisition might influence the displayed time and missing samples might not be visible in the offline mode.



It is possible that data acquisition samples do not start at time 0 of the data acquisition or that they do not reach to the end of the data acquisition. In case of selecting a time before the start of the samples the controls show the first available sample value. For a time after the last sample they show the last sample value.



The number fields below the overview graph for entering times have up/down buttons (spin buttons) to increment/decrement. As a default, PROVEtech:TA uses the selected time unit for that. In case of “seconds”, it increments/decrements in “second” steps. If the unit is smaller than the fastest sample rate of the data acquisition, it increments/decrements with this fastest sample rate though. Tip: hold the Shift key while incrementing/decrementing to always use the fastest sample rate for that – independent from the selected time unit. When using the keyboard, press the Up or Down key while optionally holding the Shift key.

4.5.2 The property dialog of the overview graph

Figure 246 shows the property dialog of the overview graph:

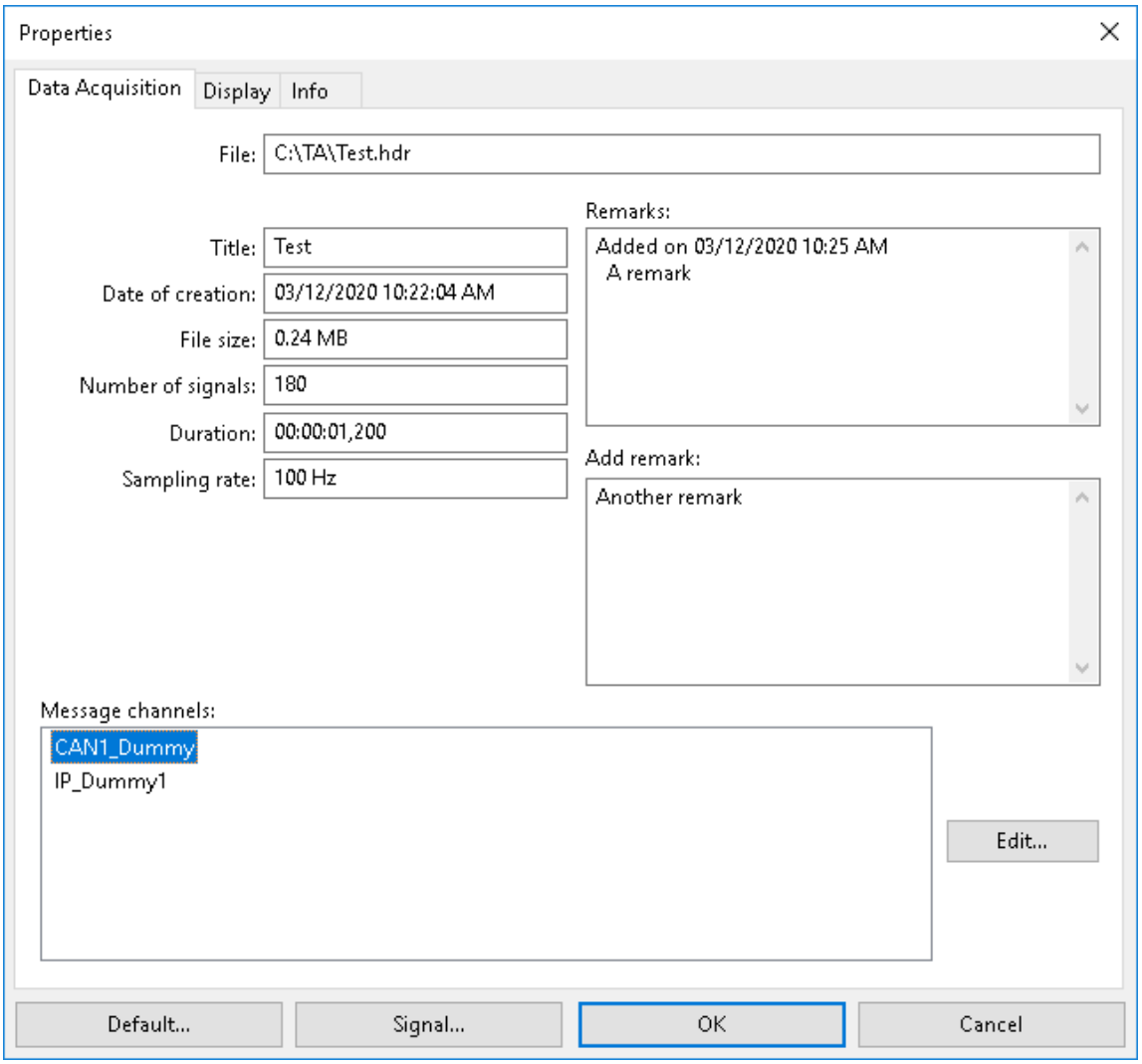


Figure 246 – Property page Data Acquisition

The property page `Data Acquisition` displays information about the data acquisition in the upper left area. The text field `Remarks` shows which user has already added which remark. Use the edit box `Add remark` to add a new one to the data acquisition.

The list `Message channels` displays the message channels of the data acquisition. Select an entry and click on `Edit...` to edit the channel settings. See also ***3.2.7.2.3 Models*** and ***3.2.7.2.3.12 Configuration of CAN channels***. You can reach the same message channel settings dialog via the context menu of a message window.

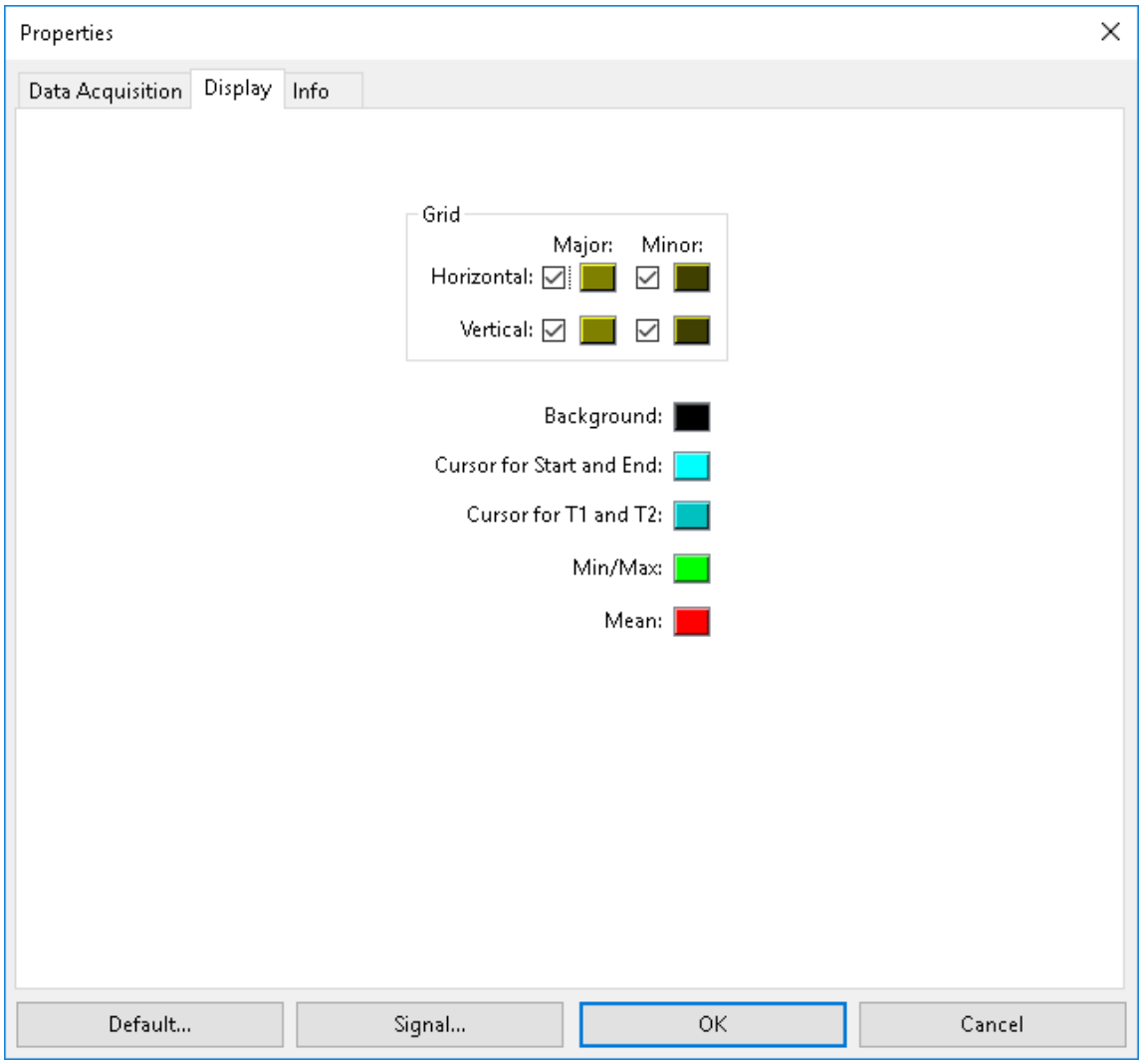


Figure 247 – Property page Display

On the property page `Display` you can change the display settings of the overview graph. Use the group `Grid` to select which grid lines are visible and which colors to use. You can change the colors of the other display elements and the plots via the other buttons.

The property page `Info` displays information about the currently selected signal in the overview graph (see chapter ***4.4.1 General properties***).

4.5.3 Extract signals from messages

In the offline mode, you can add signals dynamically extracted from messages to the signal tree. There must exist a bus description file (e.g. AUTOSAR-XML) for the according message channel. You can reference one via the property dialog of the overview graph (see chapter ***4.5.2 The property dialog of the overview graph***) if that reference is missing. Additional

buttons in the toolbar and context menu of the signal selection window add or remove such signals (see chapter ***4.2 The Signal Selection window***):

: Add extracted signals.

: Remove the selected extracted signals.

For adding signals this dialog appears:

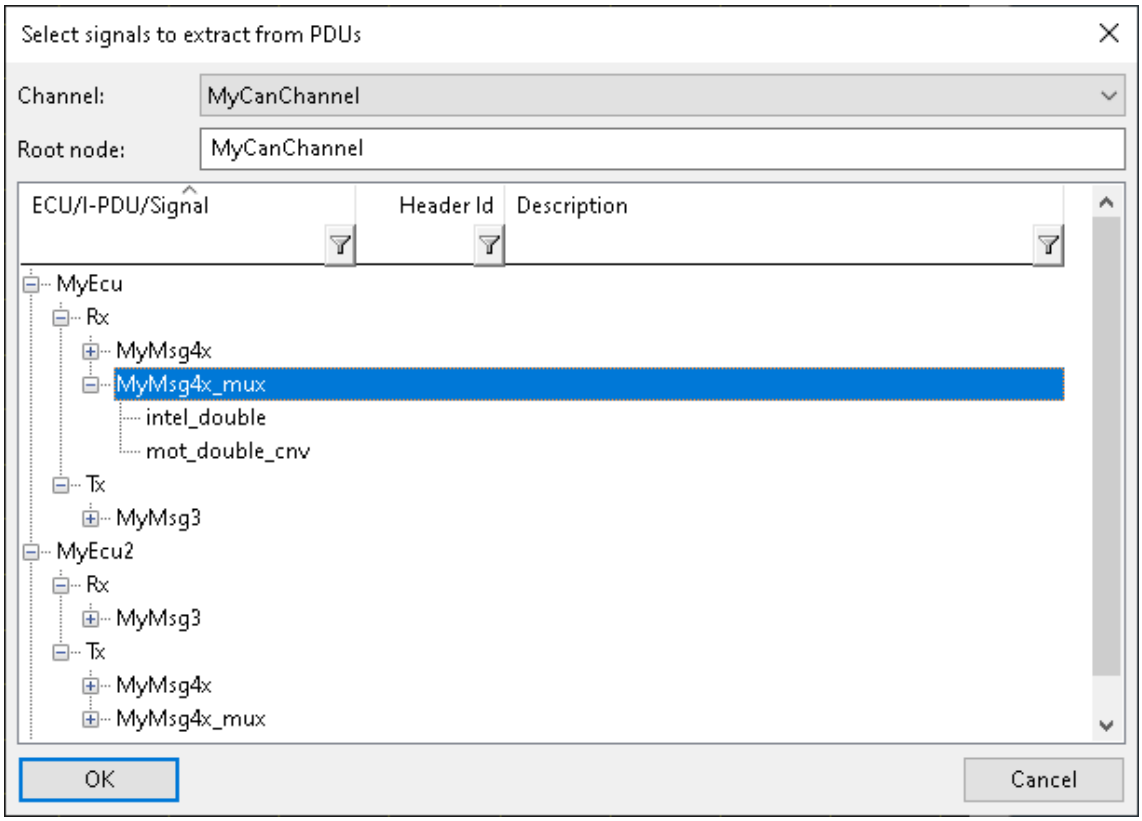


Figure 248 – Adding signals extracted from messages

Use the `Channel` combo box to select the message channel to extract messages from. The `Root node` edit field specifies the node in the signal tree to add the signals. The dot character is the hierarchy separator. The tree-list control shows the available I-PDUs and their signals within the according sending (Tx) / receiving (Rx) ECU. For Automotive Ethernet there is the additional channel (VLAN Id) level. You can select signals or whole I-PDUs to add (multi-select). Please be aware that there is currently no support to extract signals >64-bit or arrays or parameters of SOME/IP PDUs. Clicking `OK` shows the amount of selected signals and confirming that adds the signals. Please be aware that these signals are extracted while loading messages from the data acquisition. This costs memory and performance. So, you should only select the signals which are necessary.

The newly added signals have the path <RootNode>.<PduName>.<SignalName>. The information is also stored in the .hdr file of the data acquisition. So, it is not lost when you close the data acquisition and choose to save the changes.

In the signal tree these signals have the icon . Invalid signals have the icon , e.g. when they are missing in the bus description file.



It is possible that the selected time range has not a single message to extract a displayed signal. In that case the controls of the workpage display NaN (not-a-number) for floating point signal values and the value 0 for integer signals.

4.5.4 Calculated signals

In the offline mode, you can add artificial calculated signals to the signal tree. These use mathematical formulas to calculate the signal wave based on other signals. Therefore, additional buttons in the toolbar and context menu of the signal selection window exist (see chapter ***4.2 The Signal Selection window***):

: Adds a calculated signal. See the following dialog.

: Deletes the selected calculated signal. Before the deletion, a warning message shows all dependent signals, as they will be deleted as well.

: Edits the selected signal (also possible via double-click on the signal name in the signal selection window). See the following dialog.

Adding/editing shows the following dialog:

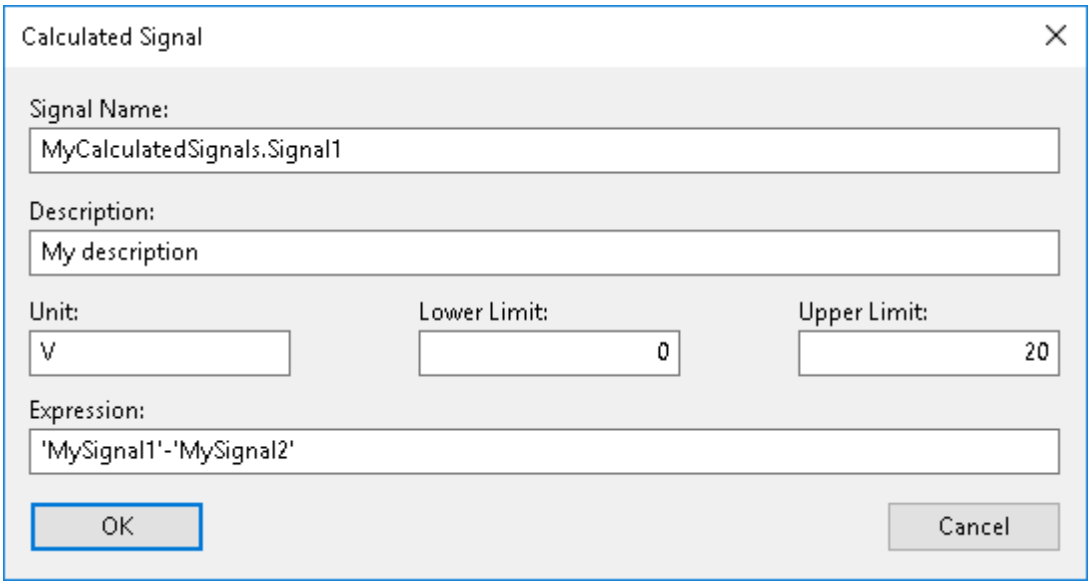


Figure 249 – Dialog for calculated signals

Enter the complete name of the signal in the field `Signal Name`. The dot is the hierarchy separator. The | character is not allowed. Edit additional attributes via `Description`, `Unit`,

`Lower Limit` and `Upper Limit`. The limits just determine the standard display range of the workpage controls and not the actual signal value range.

In the field `Expression` you enter a mathematical formula to calculate the signal. The formula can reference other signals including calculated signals by using their names embedded in single quotation marks. PROVEtech:TA inserts the according signal value of the data acquisition over the time. Strip chart controls and XY strip chart controls display the calculated value depending on the data acquisition time stamp this way.

In case a signal name contains a single quotation mark, replace it by two single quotation marks in the expression, e.g. 'My''Signal'. Also for other parts, the syntax is comparable to the Automation Library. However, the single quotation marks around signal names are always mandatory in opposite to the Automation Library.

The expressions support the following operators with descending precedence. Operators in one cell have the same precedence – in that case, the execution order is determined by the associativity:

operator	type	description	associativity
- !	unary	negation logical NOT	right-to-left
* /	binary	multiplication division	left-to-right
+ -	binary	addition subtraction	left-to-right
&	binary	bitwise AND	left-to-right
^	binary	bitwise XOR	left-to-right
|	binary	bitwise OR	left-to-right
< > <= >=	binary	“less than” comparison “greater than” comparison “less than or equal” comparison “greater than or equal” comparison	left-to-right
== !=	binary	“equal” comparison “not equal” comparison	left-to-right
&&	binary	logical AND	left-to-right
||	binary	logical OR	left-to-right

The parentheses can also control the exact calculation order, e.g. (1+8)*5

Calculations internally always use 64-bit floating-point numbers, which is also always the data type for calculated signals themselves. For bitwise operators the operands are converted

to 64-bit integers before the operation and the result to a 64-bit floating-point number afterwards. Logical operators treat 0 as “false” and every other number as “true”.

PROVEtech:TA optimizes simple constant sub expressions like 5*8-3 internally to a constant – in this case 37. Therefore, they do not negatively influence the calculation speed.

The dot is the decimal separation character for numbers (e.g. 5.8). The ‘e’ notation works as well (e.g. 1.5e8). You can enter hexadecimal numbers with the “0x” prefix (e.g. 0xa7).

Calculated signals can reference other calculated signals. This is useful for a multiple usage of the same intermediate number. Cyclic references are not possible though. And you cannot use signals extracted from messages (see chapter ***4.5.3 Extract signals from messages***).

The .hdr file of the data acquisition automatically stores the entered calculated signals when closing the data acquisition. The signal selection window uses the icon  for them. It is not possible to display calculated signal waves in the overview graph.



The signal selection window is accessible while the dialog for calculated signals is open. You can drag and drop signals from the signal selection window into the `Expression` field. It automatically adds the single quotation marks.

Chapter

5

Test Manager

With PROVEtech:TA you can test the reaction of single or multiple ECUs (electronic control units) to various signal values in the vehicle. Signals are supplied by sensors in the vehicle or by other ECUs and processed in the ECU. In the Workpage function area, you can manually stimulate signals and observe how the signal behavior affects the test system. In the **Test Manager**, you can use test programs to automate the stimulation process, which considerably expands the options for testing ECU functions compared to manual testing and recording of measurement results.

The Test Manager of PROVEtech:TA includes the environment for programming the test scripts as well as the complete management of all existing and executed test cases. It is therefore the key component of this operating software for automated testing. The test cases are implemented as macros in the embedded programming environment **WinWrap Basic**. The test scripts can be executed individually or in groups and are stored with the test results and protocols. All this together is the basis for a high-test coverage and allows easy execution of regression tests. For more details on the implementation of test programs and the operation of the Basic Editor, please refer to chapter ***5.4 The Basic editor***.

In the following, some special features of the Test Manager are briefly discussed:

Integrated user- and project management

With the help of integrated user- and project administration can be implemented the access rights. This allows several test systems to be operated independently of each other but using the same database. For example, libraries can be shared in order to create tests uniformly and efficiently. The defined user roles can be assigned the respective user rights, for example to develop tests, to execute the tests and to analyze the test results. More details can be found in chapter ***5.3.8 The tab "Users"*** and chapter ***5.2.2.1 The project***.

Version Management

In the Test Manager you can "version" your test objects, i.e. store, "freeze" and archive them. This allows you to revert to older test versions to reproduce test results. In the Test Manager tree, you can create a new release version on individual elementary tests or even entire subtrees. These are read-only ("frozen") and can later be restored and edited via Branch Selection or by creating a copy. In principle, you can only edit the latest version or a developer version of a version branch. A developer version is generated automatically when you save the current release version or when you create a new test. You can read more about this in chapter ***5.8 Versioning***.

User-defined test objects

User-defined test objects are similar to WinWrap Basic test objects. However, unlike the elementary tests, the user-defined object does not contain any Basic code, but any binary file (e.g. Excel file, text file or a file format of a proprietary software). This makes it possible for a customer to integrate already existing test structures into the Test Manager of PROVEtech:TA.

In the PROVEtech:TA settings you can define a special Test Manager path. With the structure of the contained objects, you can extend the context menu of the test objects and define new menu entries. In addition, you can execute these user-defined test objects (see ***5.9 User-defined test objects and extending the context menu***).

Testing in a real-time environment

If time measurements (e.g. with an accuracy in the millisecond range) are required, then tests will run with the Real-Time Automation Engine (RTAE) on a real-time computer. The Automation Library helps to define the sequences and parallelisms. You can read more about this in the Automation Library manual.

Supported database systems

The Test Manager in PROVEtech:TA supports the database systems Oracle, Microsoft SQL Server and PostgreSQL for shared data storage of test objects and test results. The freely available PostgreSQL can be used for standalone systems or in small departments,

for example, whereas Microsoft SQL Server and Oracle are often used in large-scale test operations or worldwide operation (see also in chapter ***3.2.7.6 Database***).

Database replication

The replication of PROVEtech:TA databases enables distributed teams around the world to collaborate easily. PROVEtech:TA users located in Germany, France, India, China and the USA, for example, can now easily share the same code base and libraries without speed problems due to slow database connections. Multiple database servers are connected with each other over long distances. The users in each particular distributed location or team are connected to a local database server and thus experience a smooth workflow.

All changed data sets are replicated between the remote locations asynchronously in the background. The time delay ranges from hundred milliseconds to a few seconds. As a result, data changes do not block the application and there is no need to wait until the changes have also been replicated on the distant servers.

The database replication engine runs independently of PROVEtech:TA. That is, there is no direct communication with the PROVEtech:TA application. The replication engine is only connected to two or more database server nodes. If you are interested in replicating your PROVEtech:TA database or if you want to purchase additional support for database replication, you can find more information in the setup package in *Goodies\Databases\Replication\PROVEtechTA_SymmetricDS_howto.pdf* or please contact our PROVEtech Support via provetech@akkodis.com.



Accessing the data of the database: Please access the database data exclusively via the PROVEtech:TA graphical user interface or via the PROVEtech:TA test language. Only in this way can PROVEtech support provide assistance in case of data loss or data inconsistency (see also chapter ***8.3.8 Data security and avoiding data loss in the Test Manager***).



When using **database replication**, we strongly recommend that you use the SymmetricDS software provided for this purpose in close consultation with PROVEtech support. With database replication, the data in the database is changed and synchronized regardless of the PROVEtech:TA client. Only in this way can PROVEtech support help in case of data loss or data inconsistency.

Regarding date and time values, please read in chapter ***8.3.5 Different time zones and daylight saving time***.

5.1 The structure of Test Manager

All test objects in Test Manager (projects, system tests, elementary tests,...) are displayed in a tree structure similar to the Windows Explorer. The test tree with the various hierarchy levels is located on the left side. If you mark an element in the test tree, you can edit it in the right area using the tabs `Explorer`, `Info/Version`, `Program`, `Execute`, `Results` and `Overview`. ***Figure 250*** shows the structure of the Test Manager.

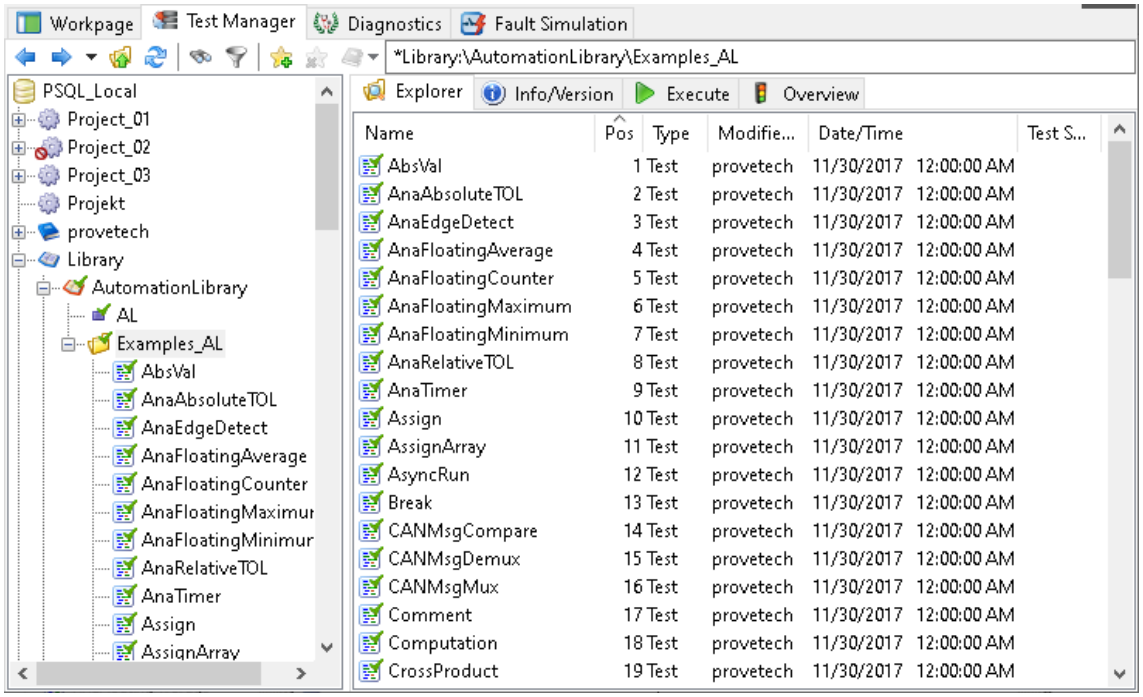


Figure 250 – Test Manager

In addition to the navigation in the Test Manager tree (selection, double-click, symbol ⊞ and ⊟) you can use the buttons ⬅ ➡ ▾ 🏠 as well as editing the path field in the navigation bar.

You can refresh the data using the button 🔄. This means that all data in the working memory is cleared. The displayed data is retrieved again from the database. This function is only required if you are working on the same data with several PROVEtech:TA clients in multi-user mode.

The buttons  control find and filter functionality. Read more about this in chapter ***5.18.1 Finding tests and test results*** and chapter ***5.18.2 Filter tests and test results***.

Favorites menu allows to save, delete and select the reference to preferred test objects. For saving, deleting and list the Favorites bookmarks, use the buttons  ,  and  .

If a given test object is not in Favorites bookmarked, the  button is enabled. By clicking this button, you can include the current selected test object in Test Manager tree at the bottom of the favorites list.

If a given test object is already in Favorites bookmarked, the  button is enabled. By clicking this button, the current selected test object in Test Manager tree will be deleted from the favorites list.

To check which favorite items are stored in Favorites list, click on the button  . This action will open a dropdown menu containing all favorite items added using the  button, as well as all module hotkeys memorized in the basic editor (see ***5.4.2 The hotkeys***). The button  is activated only when there is at least one bookmarked test object in the Favorites list.



When you terminate PROVEtech:TA the last selected test object in the Test Manager tree is saved. This object then appears as the selected object the next time you call up the Test Manager.



The Test Manager tabs are dynamically shown and hidden depending on the context.



The tabs `Explorer`, `Info/Version`, `Program`, `Execute`, `Results` and `Overview` display the information of the selected test object in the TM tree. If you selected an older version in the tab `Info/Version`, then the tabs on the right side `Program`, `Execute`, `Results` and `Overview` display the information of the **older** test version (see chapter ***5.8 Versioning***).

The complete hierarchical structure of the Test Manager tree and its test objects is stored in a database. Tests are always created within the subtree of a project or a library. Therefore, to access tests you must have the appropriate access rights to the relevant parent objects (see ***5.2.6 Access rights***).

In the Test Manager, you can see from the icons where you can create and edit tests. Using the example of a project object, the icon  shows you that you can create and edit tests. This icon  indicates that you are not allowed to create and edit the elements in the subtree.

If the PROVEtech:TA logo in the top right corner (blue mouse cursor) is rotating, then a test is currently being executed. Clicking on the logo automatically takes you to the `Execute` tab of the Test Manager.

5.2 The objects of the Test Manager

The following chapters explain the different kinds of objects in the Test Manager tree and their mechanisms of access.

5.2.1 Different types of tests

The objects in the Test Manager tree can basically be divided into four different categories. There are the objects for hierarchical arrangement with access rights management, then the objects for hierarchical arrangement with "inherited" access rights, there are the elementary tests, that are the leaves of the tree, and finally there are the remaining test objects (Database object and Trash folder).

A distinction is also made between executable objects, which can generate test results, and non-executable test objects, which can be used into other test objects or serve other administrative purposes. The executable test objects would be the Test, the System Test, the Project, and the User-defined test object.

Read also in chapter ***5.2.7 Object types in comparison***.

- The objects for hierarchical arrangement with access rights management (see chapter ***5.2.6 Access rights***) are the **Project** (see chapter ***5.2.2.1 The project***) and the **Library** (see chapter ***5.2.2.2.1 The Library in the Test Manager tree***). Projects are executable, but libraries are not executable.
- The objects for hierarchical arrangement with "inherited" access rights are the **System Test** and the **User Library** (see ***5.2.1.1 The system test*** and ***5.2.2.2.2 The User Library in the Test Manager tree***). System Tests are executable, but User Libraries are not.
- Elementary tests include the actual test (see ***5.2.1.2 The elementary test***), an object (see ***5.2.1.3 The object***) or a class (see ***5.2.1.4 The class***). Only these test objects contain WinWrap Basic code. Objects and classes are not directly executable. They can be included in other test scripts.

The **Text object** (not to be confused with the test object) is also an element type test, which contains pure unformatted text (e.g. documentation or parametrization), which can be edited with the functionalities of the WinWrap Basic editor (see ***5.2.1.5 The text object***). Text objects are not executable.
The **User-defined test object** can contain any file (e.g. an Excel file, a text file or a file format of a proprietary software) (see ***5.2.1.6 The user-defined test object***). Special mechanisms can also be used to make these objects run.

- The **Database object** or also called Root object represents a special database connection. Access rights can also be set here as well, for example to control whether projects or users may be created or deleted (see ***5.2.3 The database object***).
  The **Trash folder** collects the deleted test objects before they are finally removed from the database (see ***5.2.4 The trash folder***).

The different object types are identified by different icons in the Test Manager tree.

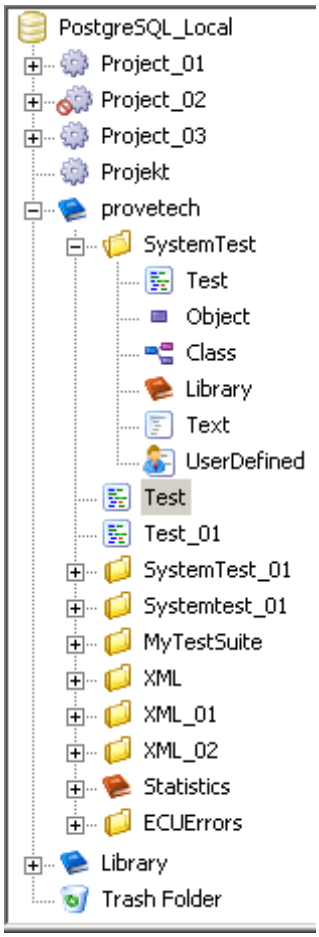


Figure 251 – System tests, tests, classes, and objects

The tests can be thought of as program files that process very tightly defined tasks. These files are put together in a file folder, the system test, and can be worked through in succession. This allows you to examine how the individual tests affect the internal states and influence the subsequent tests.

This means that you can investigate what behavior the virtual vehicle shows if several tests are run consecutively. You can compile complex system tests made up of clear individual tests and thereby investigate the function of the ECUs in interaction. For example, you can program all tests with which you want to test an ECU as elementary tests, compile them in a system test and run them consecutively. Comprehensive test tasks containing many individual tests can thus be run automatically.

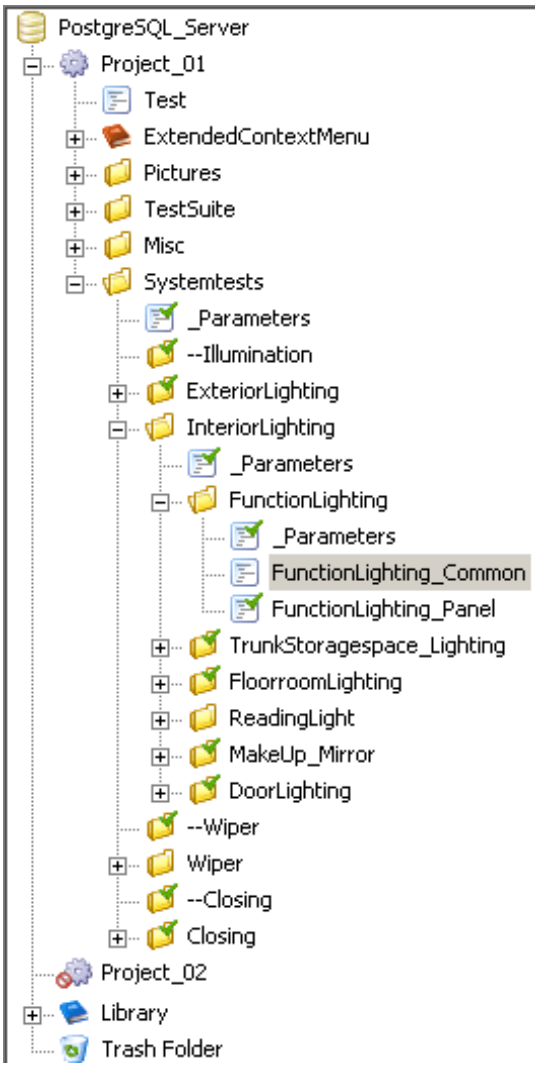


Figure 252 – Hierarchical structure of system tests and tests

You can arrange system test hierarchically. Elementary tests and user-defined test objects are the leaves of the Test Manager tree. Tests that were created under the same system test have the same hierarchy level and together represent a test suite.

This manual also uses the term **test** instead of elementary test. If there are different functions for elementary and system tests, these are referred to separately.

The term **module** is used in the manual in order to summarize the different types of test source codes. These may include elementary tests (see ***5.2.1.2 The elementary test***) as well as objects (see ***5.2.1.3 The object***) and classes (see ***5.2.1.4 The class***).

User objects are no longer managed as objects in the Test Manager tree as in previous PROVEtech:TA versions, but in the Users tab of the root object. For more information, see ***5.3.8 The tab "Users"***.

When assigning the names of test objects, special characters should be avoided if possible (such as punctuation marks, question marks, &-signs or %-signs). Since the test object names later become part of the navigation URLs, the underlying web browsers for protocol display could misinterpret the URL and thus prevent correct navigation.

5.2.1.1 The system test

System tests are represented in the Test Manager tree by this symbol (closed) or (open). They group together several individual elementary tests or system tests.

System tests are similar to a directory that can contain other directories and files. In this case they contain tests. When you run the system test, the individual tests run consecutively.

Write-protected system tests are represented in the test tree by the symbol (read-only). These system tests cannot be changed, but they may be copied.

5.2.1.2 The elementary test

Elementary tests are the actual test programs that influence the signals on the virtual vehicle and trigger certain processes in the test system. They are shown in the Test Manager tree by this symbol .

Stimulation of the signals is automated by the elementary tests you have programmed. It is possible to program very exact chronological processes. The test system reacts to the

stimulation and thus supplies test results. These are recorded and saved and can then be analyzed in detail.

Tests are written in the **programming language WinWrap basic** which is very similar to Visual Basic. For example, a “High beam test” could influence the signal for activating the high beam switch, the control element ignition on/off and the display of the ECU output for the lamp control (see ***5.4 The Basic editor***).

Write-protected elementary tests are represented in the test tree by the symbol  . These elementary tests cannot be changed, but they may be copied.

5.2.1.3 The object

The symbol  describes an object in the test tree. An object module constitutes a data type that provides methods to access its internal data to other modules (object-oriented). Contrary to the class module, a '#uses statement which refers to an object module directly creates an object of the data type.

Write-protected object modules are represented in the test tree by the symbol  . These objects cannot be changed, but they may be copied.

5.2.1.4 The class

A class module constitutes a data type that provides methods to access its internal data to other modules (object-oriented). A class is represented by the symbol  in the test tree.

Write-protected class modules are represented in the test tree by the symbol  . These classes cannot be changed, but they may be copied.

5.2.1.5 The text object

Sometimes it is useful to have plain text instead of Basic code in the Test Manager tree. This could be documentation or text that is used for other Basic scripts. To provide the functionality of the Basic editor as a plain text editor, you can create text objects with symbol  . This text is unformatted, and you cannot execute it as a test script.

Write-protected text objects are represented in the test tree by the symbol  . These classes cannot be changed, but they may be copied.

5.2.1.6 The user-defined test object

User-defined test objects do not contain Basic code. However, you can save any files at the test object in the database (e.g. an Excel file, a text file or any file format of proprietary software). E.g., you can read out and work on this file via Basic script of a different Elementary test (see ***5.9 User-defined test objects and extending the context menu***). They are shown in the Test Manager tree by this symbol  .

Even though there is no Basic code in a user-defined test object, you can execute them (see ***5.9.4 Executing user-defined test objects***).

Write-protected user-defined test objects are represented in the test tree by the symbol  . These test objects cannot be changed, but they may be copied.

5.2.2 Projects and libraries

Projects and libraries help to structure your Test Manager tree and control the access rights (see chapter ***5.2.6 Access rights***). Each of the object types has its own symbol. You can arrange projects and libraries hierarchically and cascade them anyway you like. However, placing them inside a system test or user library is not possible.



You can arrange projects and libraries hierarchically in the TM tree. When denying the *arBrowse* access right on this object or a parent object, it is no more displayed in the TM tree.

The following chapters describe these test objects in a more detailed way.

5.2.2.1 The project

To create a common workspace for a group of users, the PROVEtech:TA administrator or a user with *arManage* access rights can create a project. Read-only projects are marked with  , full-access projects with  . The project manager and the project team, consisting of several users, are controlled by access rights. In contrast to library objects, you can execute project objects and create test results.

You can create new projects inside the root object, a different project or inside a library, if the parent test object has the *arManage* access right set. The deepest project level decides about the access rights of the current user (see chapter ***5.2.6 Access rights***).

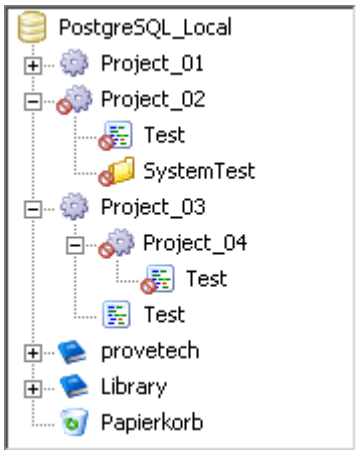


Figure 253 – Hierarchical order of projects

The following figure shows how to create a new project as PROVEtech:TA administrator (or as a manager of the parent project).

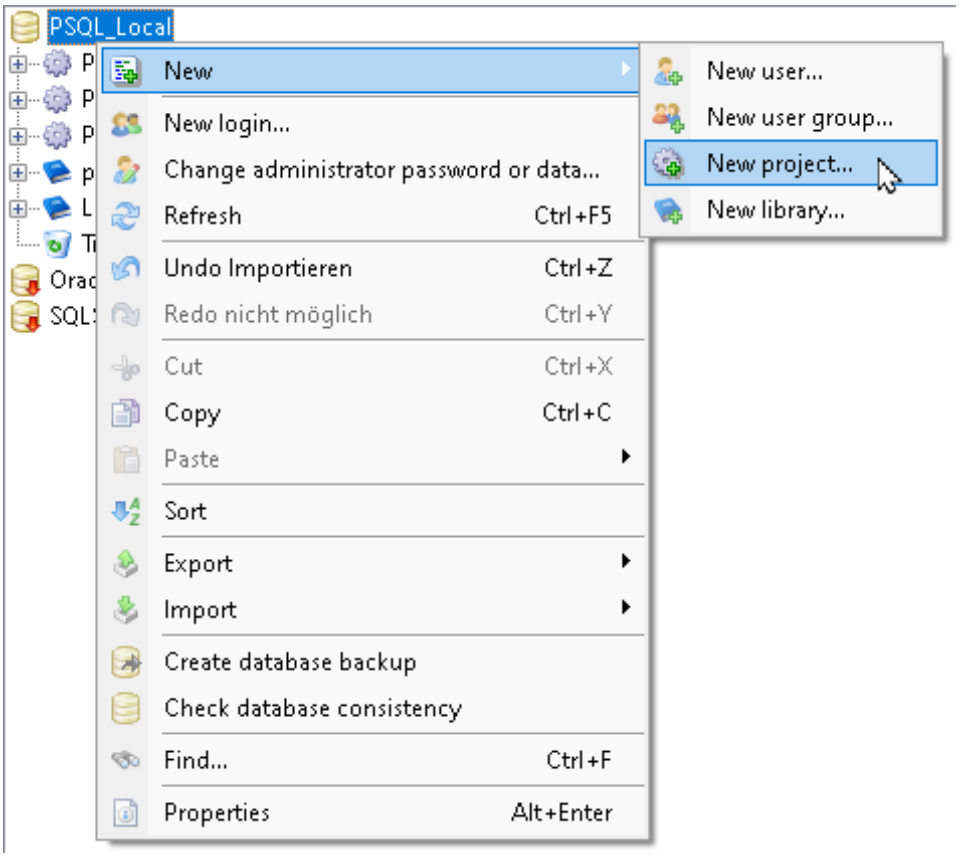


Figure 254 – Context menu "New project..."

Then the following dialog appears.

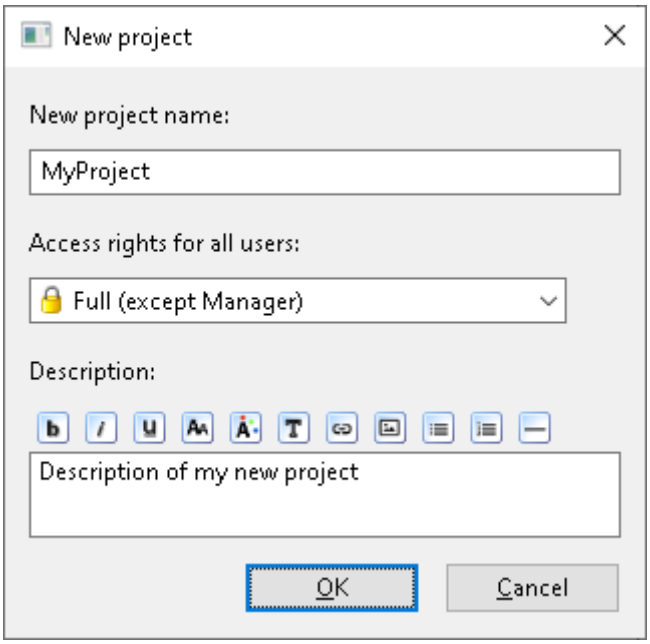


Figure 255 – The dialog "New project"

When creating a new project or a new library, you can specify a unique name, a descriptive text and the start setting for the access rights. You can adjust the access rights more precisely later.

Copying projects is by using copy and paste, drag and drop or the hotkeys is possible, too.



Projects have nothing to do with configurations. You use projects to control the access to Test Manager subtrees and you use configurations to choose your settings for the hardware connection.

5.2.2.2 The library

There are two types of libraries:

- the **Library** (formerly named as "Global library", see ***5.2.2.2.1 The Library in the Test Manager tree***), which is very similar to projects, with its own access rights too; but you cannot execute a Library; the Test Manager tree displays libraries with the symbol (closed) or (opened)
- the **User library** (formerly named as "Local library", see ***5.2.2.2.2 The User Library in the Test Manager tree***), which is similar to system tests, without its own access rights, but non-executable; the Test Manager tree displays libraries with the symbol (closed) or (opened)

5.2.2.2.1 The Library in the Test Manager tree

Libraries (in opposite to User Libraries) are displayed by the symbol and . They correspond to the projects in the Test Manager tree. Beside the icon, the only difference is that you cannot start a test run of a library. So, you can visualize of the content of this subtree are test objects rather than commonly used library functions. An example of this is the **Automation Library** that supports the real-time simulation. You can convert projects to libraries and in reverse if you have the according privileges.

In contrast to the user libraries, you can set the access rights on libraries. Therefore, you can control how different users can access its content, similar to the projects. See also in chapter ***5.2.6 Access rights***.

You can create a new library analog to the creation of a new project (see ***5.2.2.1 The project***).

5.2.2.2.2 The User Library in the Test Manager tree

Each user can create itself user libraries (or) or convert system tests to user libraries and in reverse, in case of write access. User libraries do not differ from system tests, except the fact that you cannot execute user libraries and you cannot create test results from user libraries. It simply constitutes a suitable location where the user can deposit those tests that are needed for and incorporated in their own tests repeatedly.

5.2.3 The database object

The database object is located at the first level in the Test Manager tree (root object) and represents the database connection. It is shown as if connected, otherwise as if the database is not connected. You can use the context menu (right-click on the object in the Test Manager tree) to start the following operations, among others:

- Log in or change user (see chapter ***2.3 Login***)
- Change the password of the logged-in user (see chapter ***5.3.8.1 The user***)
- Export all data of the database or import parts of the data from an export archive (.dbi.7z file) (see chapter ***5.3.10 Exporting and importing tests***)
- Create a database backup (see chapter ***5.11 Create and restore a database backup***)
- Check the consistency of the data in the database (see chapter ***5.12 Database consistency checks***)
- Search across the entire database (see chapter ***5.18.1 Finding tests and test results***)

- Display and change properties of the database (see chapter ***5.3.11.2 The tab "Database settings" in the property dialog***)

5.2.4 The trash folder

The trash folder is the bottom element in the test tree and is displayed by the symbol . It is similar in function to the trash folder in the Windows Explorer. If a user deletes or overwrites specific tests or system tests, these are first moved to the trash folder. This is very fast and has the advantage that the old status can be reconstructed in case of unintentional deletion. In such a case, the test is simply copied from the trash folder back to the original folder or you can use the "Undo"-functionality via context menu.

The trash folder object is a private trash folder. That means, you can only see the test objects deleted by yourself. If you are logged in as PROVEtech:TA administrator or as a user with arManage access rights on the root object, then you can see all test objects in the trash folder.

Occasionally it is advisable to empty the trash folder completely (context menu, right mouse button on trash folder, by the PROVEtech:TA administrator). This improves the system speed slightly and saves storage space in the database and for the database backups.

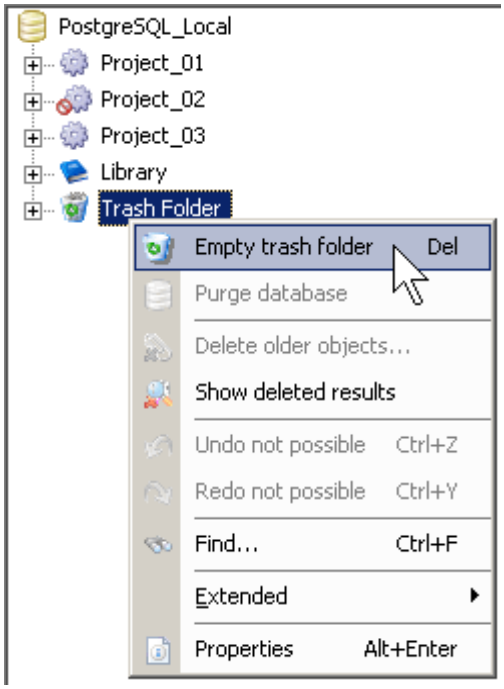


Figure 256 – Empty trash folder

Purge database

Even when you delete test results, attachments or test documents, they are only marked for deletion, but not shown in the trash folder. To delete them permanently from the database the PROVEtech:TA administrator can choose the context menu entry `Purge database`. This operation happens in a background process. A blue warning message is displayed in the PROVEtech:TA Log when this process has finished.

If records are not deleted after the purge operation, either they are just a few hours old or dependencies still exist. Therefore, we recommend that you perform the purge regularly once a week or once a month.



The purge process is very time-consuming and generates a high database load. All tables are successively searched for data records to which there are no more references from other data records or tables (primary/foreign key relationship). These are then permanently deleted. Depending on the amount of data, this process can take several minutes or even hours and should therefore only be carried out very rarely (e.g. once a week or once a month). The night or the weekend are suitable times for this.

Delete older objects

The context menu item *Delete older objects…* provides the PROVEtech:TA administrator the possibility to delete invalid test results and storage-extensive data acquisitions, that are older than a specific date, globally in the entire database. This means, you can provide new free database space and reduce the size of your backup files. On click the following message box appears:

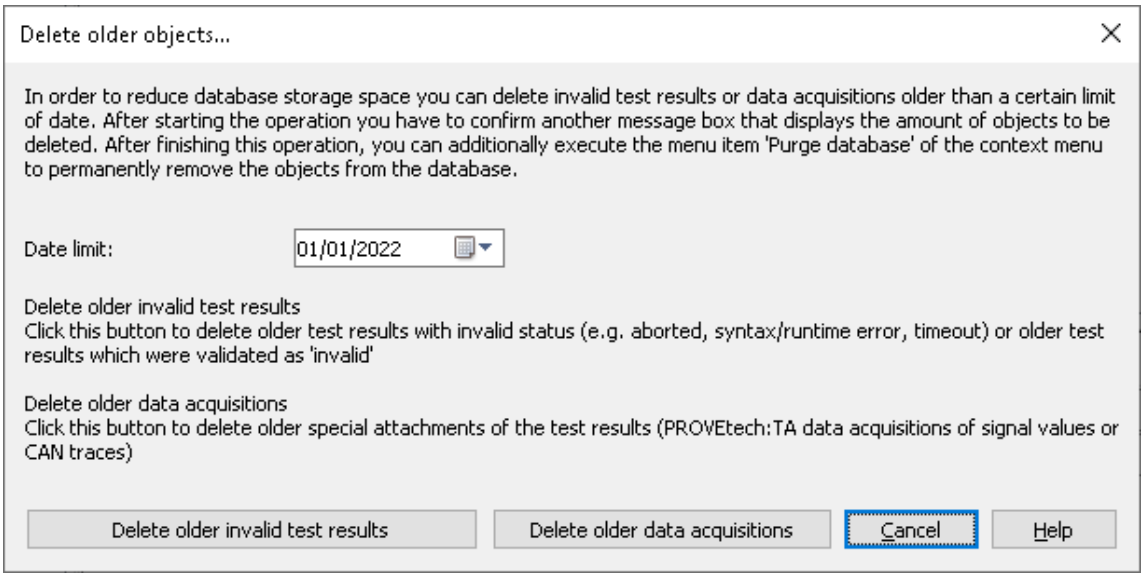


Figure 257 – Delete older objects…



If there is not much free space on the database server left anymore, you can try to empty the trash folder and purge the database to create more space for new PROVEtech:TA data. This does not automatically mean that you have created space on the disk, as the table space files in the database do not usually become smaller. This must be done in a database defragmentation step by your database administrator.

Please read also chapter ***5.10 The Quota mechanism*** and chapter ***8.4.9 There is not enough storage space in the file system left for the database***.



You can display and restore deleted test results, too. Referring to this, please read chapter ***5.7.4 Restoring deleted test results***.



When using **attachments that are linked into the file system**, the consistency of data between the database and the file system may get lost. So, for example, you can move or delete the linked files in the file system. Or, when you empty the Trash Folder, the linked files of the attachments stay alive in the file system. If you kill the PROVEtech:TA process or turn off your computer in a bad moment, there could also be consistency problems left.

Please read also chapter ***5.6.9 Attachments***.

5.2.5 Read-only test objects

You can access test objects inside read-only subtrees in the Test Manager tree, but you cannot edit them. Read-only test objects are marked with a corresponding symbol (e.g.  in case of a library, or  in case of a project.

This applies for both elementary tests and system tests. When you click on read-only test objects, almost all functions are deactivated in the context menu (click with the right mouse button), the input windows are disabled and grayed (see ***Figure 258***).

In detail this means:

- **Programming of elementary tests** is not possible. You cannot make any entries in the work folder for programming the test.

- **Renaming tests (or modifying other attributes)** is also not possible. The name cannot be edited in the Test Manager tree or in the context menu. The sequence of tests in the system test cannot be changed.
- **Deleting tests** is not possible.
- **Overwriting tests** is not possible.
- **Executing tests** is only possible if you have the appropriate user rights.

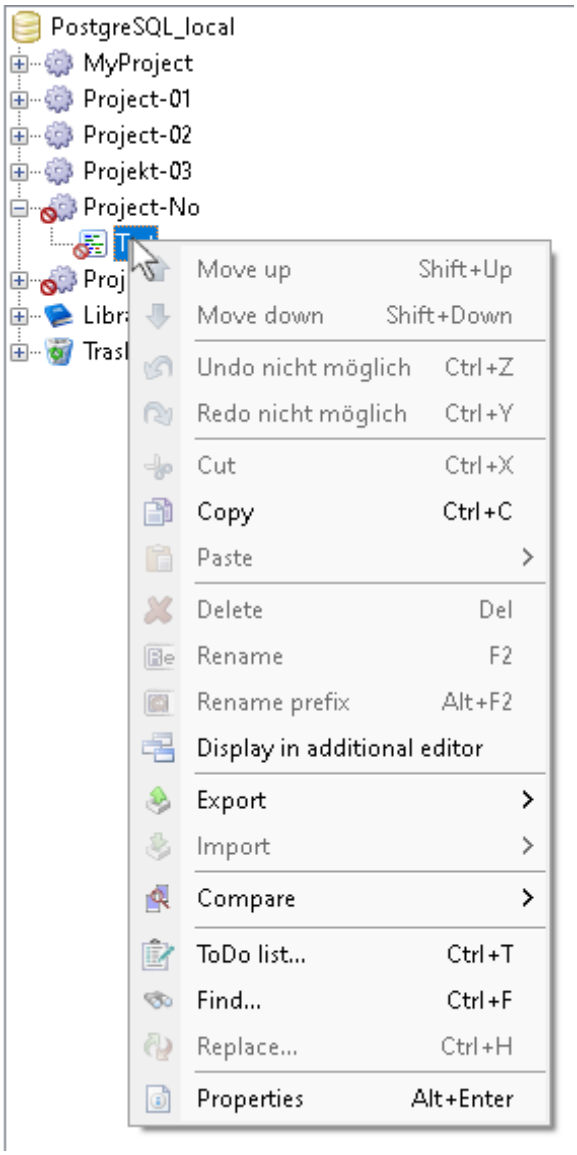


Figure 258 – Disabled functions for tests that are read-only

To be able to process read-only test, you must make these available inside a subtree with write permissions. You have the following ways of doing so:

- Set the according access rights (see also ***5.2.6 Access rights***)

- You must **copy the tests** inside a subtree with write permissions. Then you can run the tests and modify the content of the test program (see also ***5.3.5 Copying and moving tests, generating links***).
- It is equally possible to **export** read-only test objects. They can optionally export the tests with or without results. Then you can import the tests with or without results, depending on how they were exported (see also ***5.3.10 Exporting and importing tests***).

5.2.6 Access rights

Via the properties dialog of a test object (see chapter ***5.3.11.4 The tab "Security" in the property dialog***), via the “Users” tab (see chapter ***5.3.8 The tab “Users”***) or via the “User groups” tab (see chapter ***5.3.7 The tab “User groups”***) you can set the access rights to projects, libraries and the root object for each PROVEtech:TA user or a user group. You can either select from predefined combinations of access rights to all test objects and test results of a project or library located in the subtree or set each access right individually.

The *Manage* right grants modifying the access rights and creation and deletion of users, user groups, projects and libraries, whereas all further rights allow or forbid you to display, read, write, delete and execute the objects inside the subtree.

The *Manage* right to the root object of the Test Manager tree is a special access right with which the user has some configuration options that are otherwise only available to the PROVEtech:TA administrator. It is best to log in as PROVEtech:TA administrator to assign this right to a user.

The table below shows the different types of user rights as well as the different groups of access right combinations like *Manager*, *Full (except Manager)*, *Execute and read*, *Read-only* and *No access*:

Access rights	Description	Manager	Full (except Manager)	Execute and read	Read-only	No access
Manager (Manage user/user groups/project/ library/root)	This permission allows you to set the access rights on a project or library that contains the test object; furthermore, you can control the creation, renaming and deletion of users, user groups, projects and libraries; additionally, you can manage special settings in the configuration	**x**				
Access rights for test objects						
Display in Test Manager tree	Without this permission the project or library subtree is invisible to the user	**x**	**x**	**x**	**x**	
Read test objects	Without this permission the user cannot see the `Info, Documents, Parameters` and `Program` tabs; so the test code is hidden from the user	**x**	**x**	**x**	**x**	
Modify test objects	The user needs this permission for editing, inserting and deleting tests and test code	**x**	**x**			
Execute test objects	The user needs this permission for executing the tests in the `Execute` tab; if other modules are included by '#uses-call, either Read access or Execute access is sufficient for the included modules	**x**	**x**	**x**		

Delete test objects	Revoking this right prevents the deletion (i.e. moving to the trash folder) of test objects or individual release versions in the version chain (e.g. by the tab `Versions`)	**x**	**x**			
Access rights for test results						
Display test result list	Without this permission the `Results` tab is hidden; then, you cannot navigate in the result structure, and you cannot view the test results	**x**	**x**	**x**	**x**	
Read test results	With this right, the user can read and export test protocols and all other parts of the test result; if the user has only the “Display test result list” right and not the “Read test results” right, then only the property page `Comment` of the test result is enabled, and the user can read the “meta data” of the test result	**x**	**x**	**x**	**x**	
Modify test results	With this permission a user can edit the name and the description of a test result, set the “Validated” state, add or delete attachments and test parameters to/from a result and delete the test result; this right has nothing to do with the execution of a test object	**x**	**x**	**x**		

Delete validated test results	You can explicitly protect special test results from deletion. Therefore, deny this access right and modify the *Validated* attribute of the test result to any value	**x**	**x**	**x**		

The columns *Manager*, *Full (except Manager)*, *Execute and read*, *Read-only* and *No access* describe predefined patterns of access rights. Any other combination of access rights is marked as *User-defined*.

You can access this access rights dialog in ***Figure 259*** via the “User groups” tab when selecting the root object of the Test Manager tree (see chapter ***5.3.7 The tab “User groups”***).

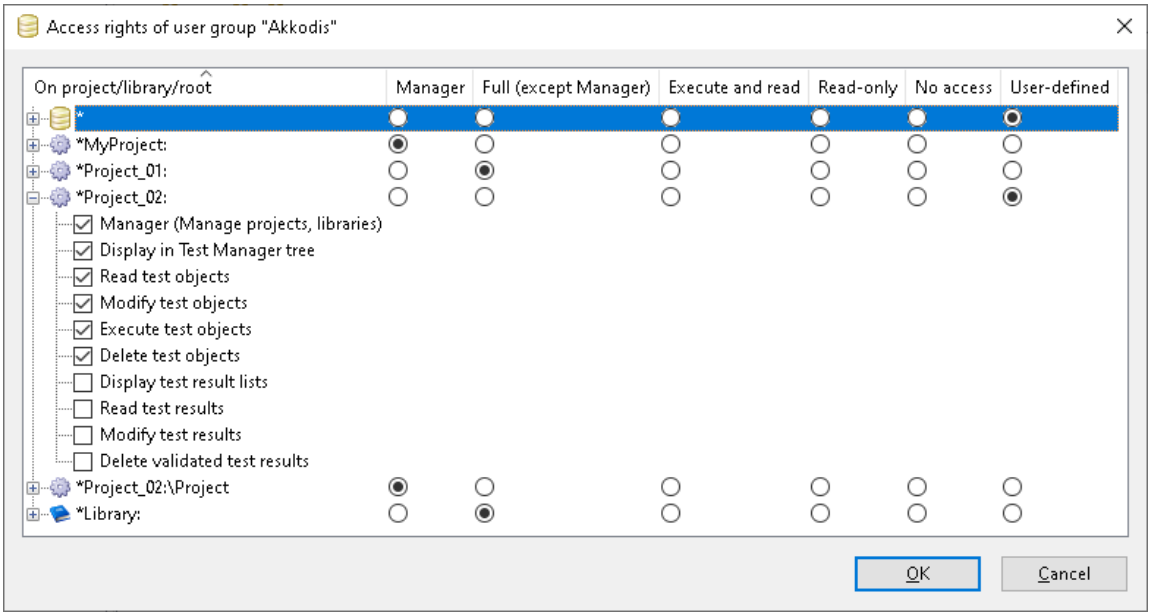


Figure 259 – The access rights of a user group

You can access this access rights dialog in ***Figure 260*** via the “Users” tab when selecting the root object of the Test Manager tree (see chapter ***5.3.8 The tab “Users”***).

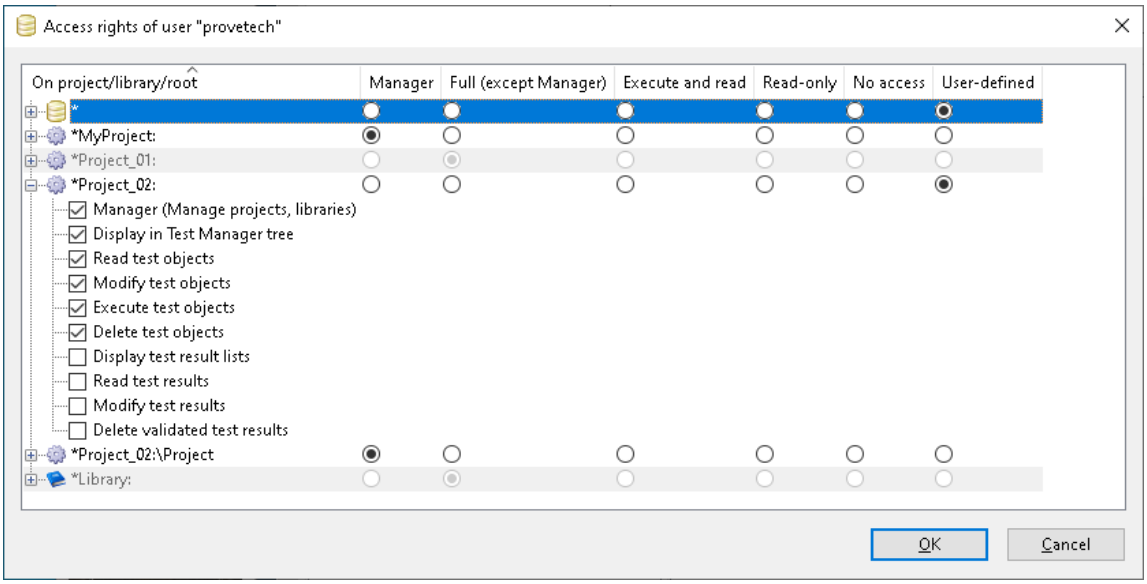


Figure 260 – *The access rights of a user*

You can see the access rights of the "provetech" user to the root object and all visible projects and libraries. Projects or libraries without the access right "Display in Test Manager tree" (arBrowse) are not listed. The access rights that you are not allowed to set because of the missing Manager right (arManage) are marked with a gray background. You can set the access rights in detail via the checkboxes on the left side or use the predefined compilations via the radio buttons on the right side.

The administrator has the capability to see and set all rights.

The access rights dialog to a project, library or root object shown in ***Figure 261*** can be accessed via the properties dialog of the selected test object in the Test Manager tree (see chapter ***5.3.11.4 The tab "Security" in the property dialog***).

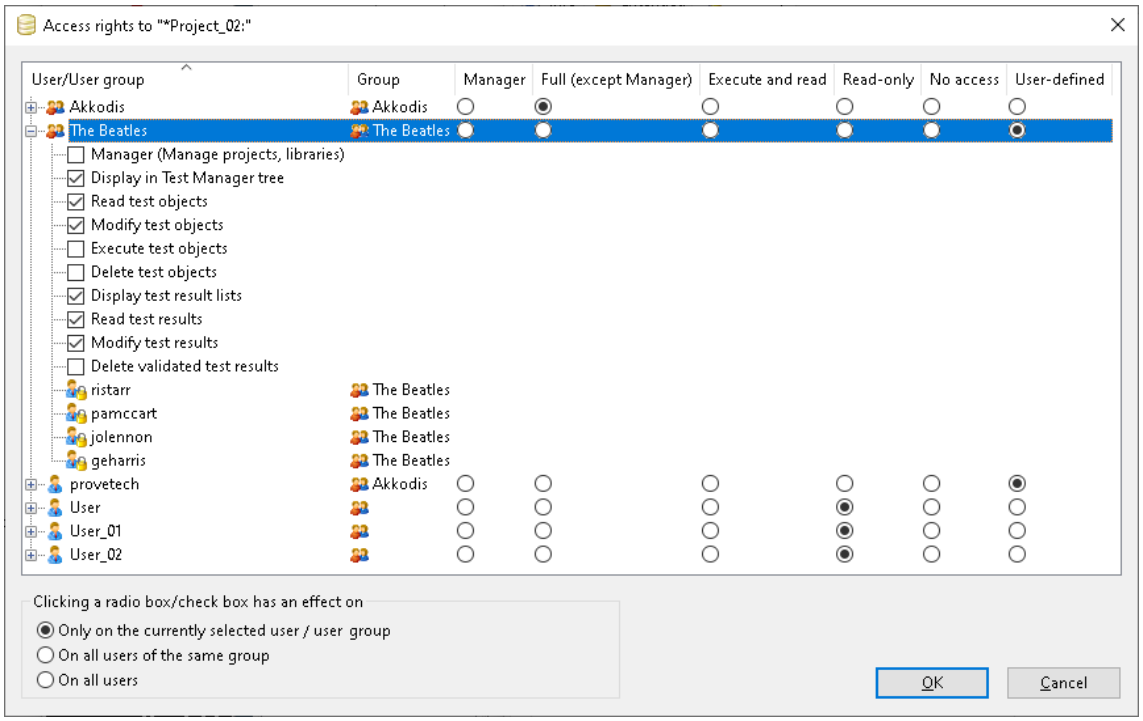


Figure 261 – The access rights of all users to a project, a library or the root object

The dialog shows the access rights of all users/user groups to the selected subtree. If the logged-in user does not have the Manager right on the selected path, the dialog is not accessible, and no access rights can be changed.

If a user is assigned to a user group, this does not automatically mean that the access rights of the user group apply to this user. It must also be set that the access rights for this user are managed by the user group. Users whose rights are managed by the user group are listed below the user group and marked with the icon . Changes to the access rights of the user group also affect these users (see chapter ***5.3.7 The tab "User groups"***).

In the lower part of the dialog, you can choose which objects are affected of the modification:

- `Only on the currently selected user / group` is the default and means that only the selected object is modified
- `On all users of the same group` means that the modification is performed for each user with the same group (regardless of whether the access rights are managed via the user group or not) and write permissions (the empty string is ignored)
- If `On all users` is chosen then your changes are applied to all users in the dialog for which you have write permissions.

That means that there are three views of user rights:

- **Access rights of user …** Here you can set, for a particular user, his access rights to projects, libraries and the root object (see ***Figure 260***).
- **Access rights of user group …** Here you can set, for a particular user group (and all users who are assigned to this user group and whose access rights are managed by the user group), its access rights to projects, libraries and the root object (see ***Figure 259***)
- **Access rights to …** Where you can set the access rights of all users and user groups to a special project, library or root object (see ***Figure 261***).

Regarding the access rights, you must be aware of the following constraints:

- You can deny the access rights of your own user too (except for the 'admin' user).
- If the *Execute* right is denied, you can anyway execute the test in the Basic debugger. Only the `Execute` tab is hidden and you cannot create test results.
- If the projects and libraries are hierarchically structured, then the access rights of child objects overwrite access rights of parent objects. Therefore, there is no inheritance of access rights.
  In contrast, no access rights can be set for system tests, user libraries and elementary tests. These take over the access rights of their parent test object.
- If projects and libraries are hierarchically structured, then you can delete parent objects with write rights even if the *Write* rights of child objects are denied.
- If projects and libraries are hierarchically structured and the access right *Display in Test Manager tree* is denied, then the whole subtree will be hidden, although this access right is set inside the subtree.
- If the access right to delete test objects has been withdrawn, new test objects that have just been created cannot be deleted directly. However, a developer version that has just been created can be deleted again, since this is an exception.
- If the access right to delete validated test results has been withdrawn, then it is also not possible to delete their associated test objects or test versions.
- To protect the deletion of test documents you must deny the *Write* right on test objects, but not the *Delete* right.
- To protect the deletion of test result attachments you must deny the *Write* right on test results, but not the *Delete* right.

- The access right *Delete validated test results* also refers to results that are reset to "Unprocessed" afterwards. So once this attribute is changed the test results stay validated. You can find more about this in chapter ***5.7.5 Validating test results***.
- If a user has *Manage* rights on any project or library, then this user is allowed to set the access rights for all users to this subtree.
- A user name (logon name) is displayed with a test object or test result, provided that the access rights *arRead* or *arTrRead* are available for the corresponding object.
- The name and type of a test object is displayed even if only the *Display* right is granted, but not the read right. The similar behavior is valid for test results.

Regarding the user and user group management and their attributes, please note the following:

- Each user can access the login names of all users as well as the names of all user groups. However, a user needs the manager access right (*arManage*) on the root object to access the additional attributes (full name/email/phone at the user, description and associated users at the user group) in the database.
- If the *arManage* access right to a project or library object or the root object of the Test Manager tree is granted, then the access rights of all users to this object can be changed.
- Creating a new user or user group from the context menu of a project or library object is possible if the *arManage* access right to this object has been granted.



Access rights refer to the access of PROVEtech:TA users to the Test Manager data, but not to the access to the test system hardware.

It is also not possible to control the database user's access rights to the database via the PROVEtech:TA Test Manager. In order to grant or deny the database user the right to change the database structure during a PROVEtech:TA update, please contact your database administrator. The name of the database user is visible in the settings dialog or the TM.ini. See also chapter ***5.3.11.2 The tab "Database settings" in the property dialog*** at the checkbox `Display message box to decide whether to keep a release version`.

Likewise, you cannot manage the access rights of Windows users to the file system via the PROVEtech:TA Test Manager.



If multiple PROVEtech:TA users have arWrite access on a test object then there might be conflicts when accessing the object at the same time. You can read this in chapter ***5.4.10 Locking the Basic code***.



The PROVEtech:TA administrator "admin" is even more powerful than a user with *arManage* access rights. For example, he can reset passwords (see also chapter ***5.3.8 The tab "Users"***).



If you use your computer as a remote test sequencer then the access rights have additional impact on that. Referring to this, please read chapter ***5.14.3 The remote test sequencer and access rights***.



If you create links (see ***5.3.14 Linked test objects***) between different projects or libraries then you must take care of the access rights. For example, if you want to modify the content of a link it is sufficient to have the corresponding user rights only for the selected test link.

You should consider this if you modify the access rights afterwards. However, it is not possible to obtain access rights by fraud in creating a link inside the own project.

5.2.7 Object types in comparison

The following table shows the common features and differences of the diverse test object types.

Test object type	Can contain children	Is executable	Contains Basic code	Can have links	Possibility to set access rights	Quota mechanism
Root (Database)	x				x	
Trash Folder	x					
Project	x	x			x	x
Library	x				x	x
System test	x	x				
User library	x					
Test		x	x	x		
Object			x	x		
Class			x	x		
Text			(x)	x		
User-defined test object		x		x		

(x): Contains a text instead of Basic code

The corresponding chapters are:

- ***5.5 Executing tests***
- ***5.4 The Basic editor***
- ***5.3.14 Linked test objects***
- ***5.2.6 Access rights***
- ***5.10 The Quota mechanism***

5.3 Managing tests

The following chapters show how to manage the test objects in the Test Manager tree.

In this overview you can see the possibilities to store information in the test object:

- Setting the test object's name and type (see ***5.3.1 Creating new tests*** and ***5.3.2 Renaming tests***)
- Setting a description and a test specification number (see ***5.3.9.1 The tab "Description"***)
- Creating WinWrap Basic source code including comments (see ***5.4 The Basic editor***)
- Creating a parameter set (see ***5.3.9.2 The tab "Parameters"***)
- Adding files as test documents (see ***5.3.9.4 The tab "Documents"***)

In chapter ***5.6 Test results*** you can find possibilities to store information at the test result.

5.3.1 Creating new tests

New tests (elementary tests, classes, objects,…) are generated in two steps:

1. You must first create a new test object. Here you define general data such as name and description of the test object's content.
2. You implement the test. That means, you design the actual test program using the **Basic programming language**.

You can create a new test (elementary test, class, object,…) via context menu of the parent test object (see ***5.2 The objects of the Test Manager***).

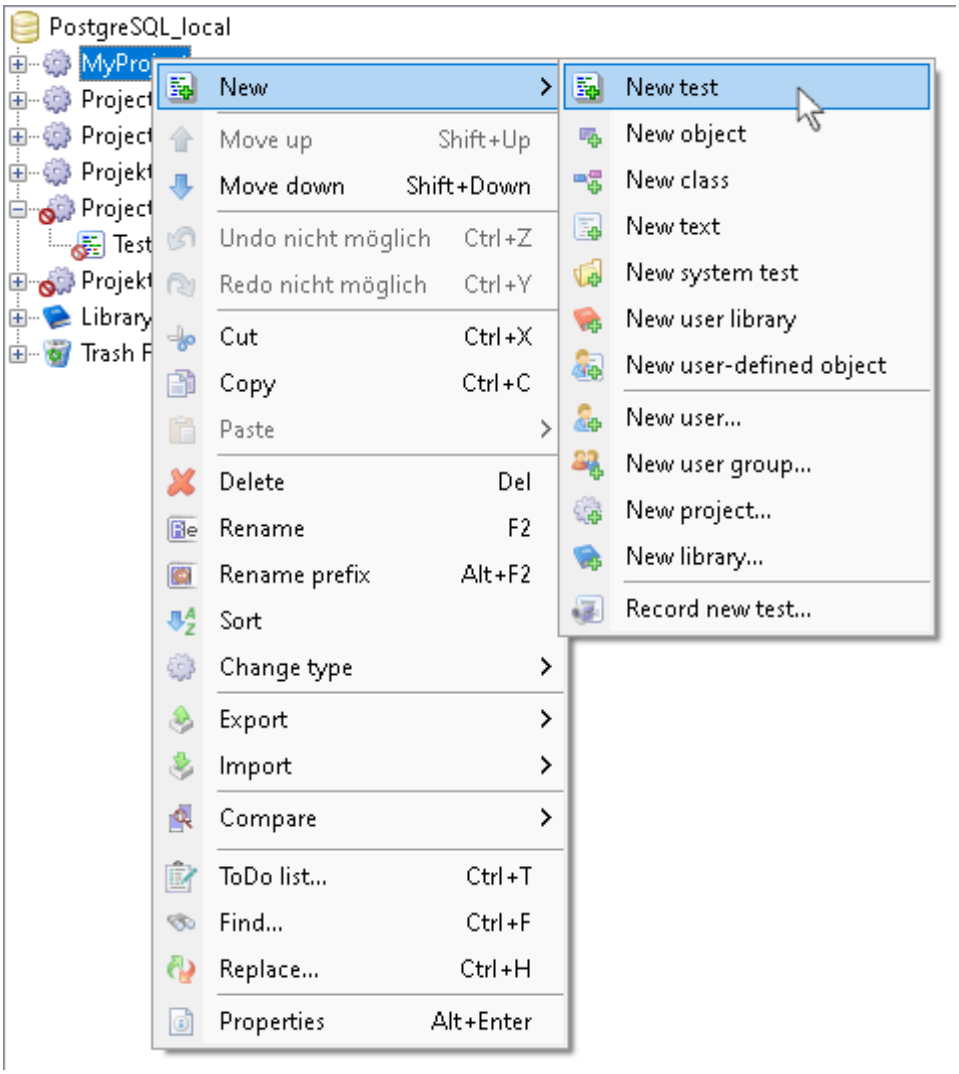


Figure 262 – Creating a new test

Choose *New System Test* or *New User Library* to create a system test or a new user library. Inside the new test object you can then define further system tests and individual elementary tests whose interaction you want to investigate.

You can also create a new test via the menu item *Record new test*. Please refer to chapter ***5.3.4 Recording a new test*** for further information.



To create new test objects you need write access in the according subtree (see ***5.2.6 Access rights***).



Test objects can also be created as a link. More details about this can be found in the chapter ***5.3.14 Linked test objects***.

If you want to create newprojects or libraries, please read chapter ***5.2.2.1 The project***.

If you want to create new users, please read chapter ***5.3.8 The tab "Users"***.

5.3.2 Renaming tests

You can change the name of a test object by clicking with the left mouse button on a marked element in the Test Manager tree or by pressing the F2 key after selecting the element. You can also use the *Rename* function in the context menu of the test object.



Figure 263 – Renaming a test

A test object name consists in the broadest sense of an icon, prefix and name. The name is part of the test object path, but not the icon and the prefix. The icon is predefined by the Test Manager depending on the test object type and access rights. For example, an additional user-defined icon in the form of an emoji from the Unicode character table can be set as a prefix (e.g. 👍 or ✔).

The following figure shows how to set an icon as a prefix for the test object's name (context menu *Rename prefix* or short key Alt+F2, then Windows+Point).

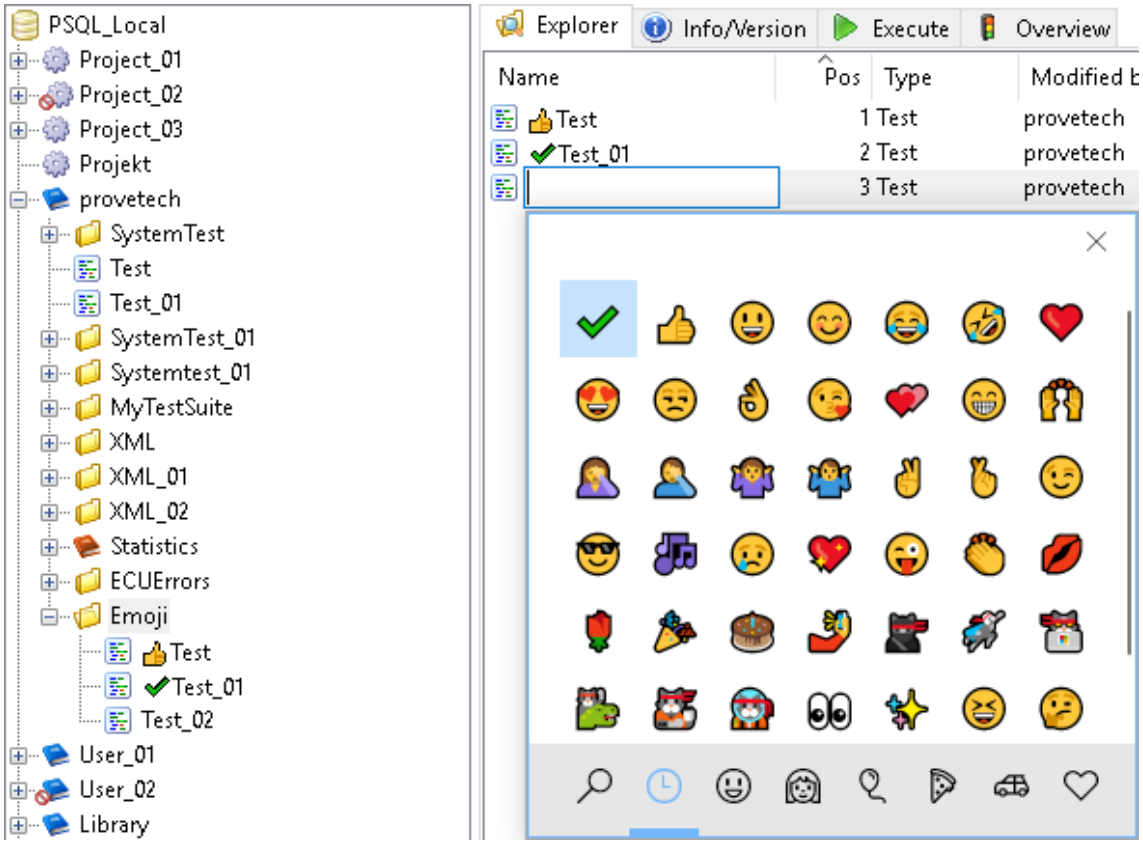


Figure 264 – Set an emoji as additional icon for the prefix of a test object's name

---

After renaming test objects it is possible that you have to adjust '#uses-statements in your WinWrap Basic scripts, too.

In contrast, renaming the prefix of a test object has no impact on the '#uses-statement.

5.3.3 Changing the type

By context menu in the Test Manager tree you can change the type of a test object.

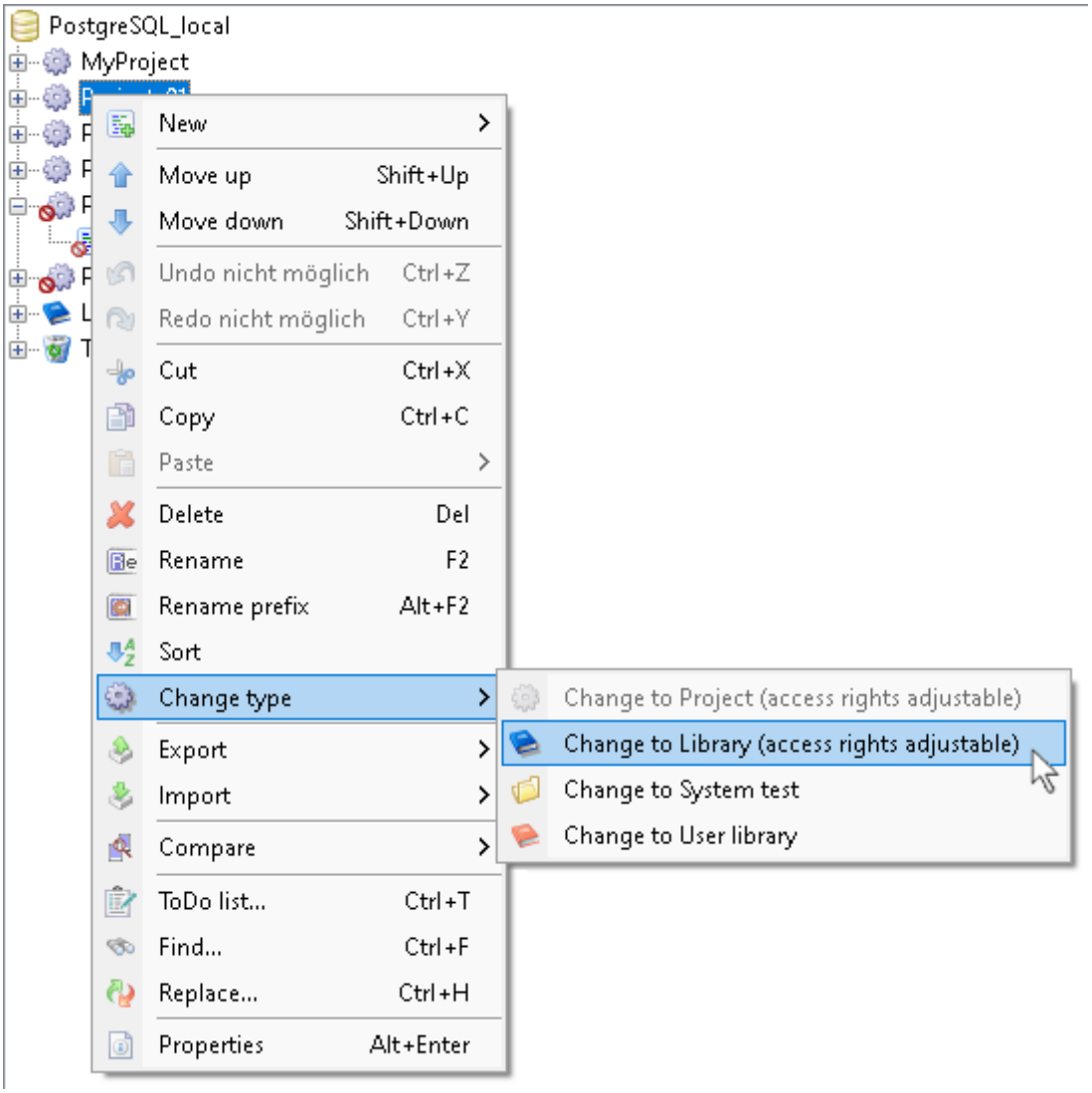


Figure 265 – Changing the type

However, you must pay attention to certain constraints:

- This context menu entry `Change type` is only available for projects, libraries, system tests and user libraries
- The appropriate access rights **arWrite** and respectively **arManage** (when acting with projects or libraries) must be granted
- The parent-child-structure must be kept valid, e.g. a system test cannot be the parent object of a project

- When converting a project or a library into a system test or a user library the access rights and the quota get lost for this test object and the data of the parent object is taken over
- When converting a system test or a user library into a project or a library, the access rights of the parent object are first taken over; then you can overwrite this data

You can change the type of a WinWrap Basic object between **Test**, **Class** and **Object** as well by changing its module properties. Please read also chapter ***5.4.1 The toolbar***.

5.3.4 Recording a new test

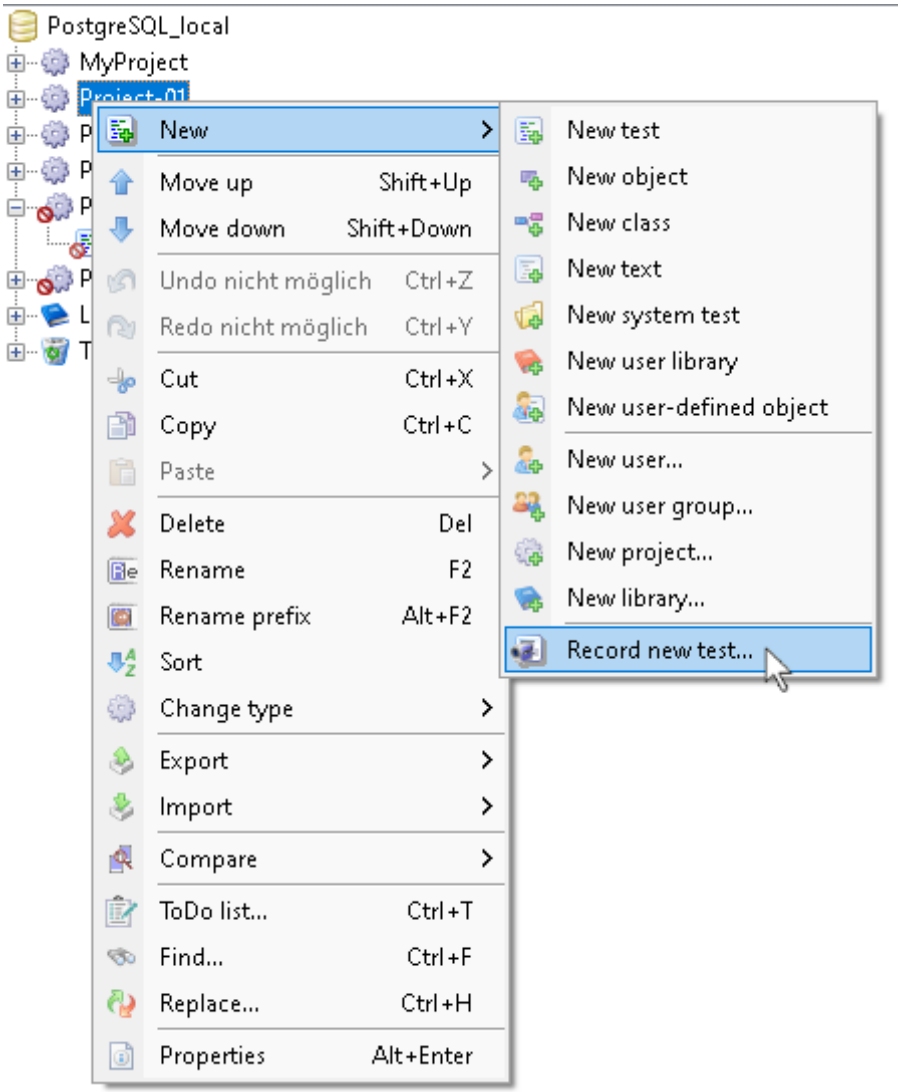


Figure 266 – Record new Test

Another alternative for creating a new elementary test is provided by the menu function *Record new test*. To create a new elementary test in this manner, mark the element in the test tree under which the new test is to be created. Then activate the command via the context menu (see ***Figure 266***).

Any subsequent operations are recorded in the test program (see chapter ***3.5 The Macro menu*** for further information). To stop recording use the menu item *Macro|Stop Macro Recording*…. or the button  in the toolbar.

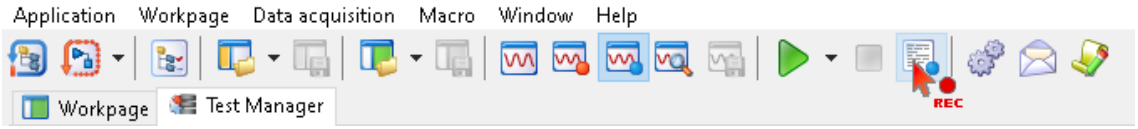


Figure 267 – Stop Macro Recording

5.3.5 Copying and moving tests, generating links

You can copy test objects in the Test Manager tree just like files and directories in the Windows Explorer.

Copying tests saves creating elementary tests or system tests twice. If you have created a test for a vehicle series, for example, and want to test the same signals in another test, you can just copy the appropriate test. When you copy a system test, all the tests included in it are copied as well.



If you copy a test you must pay attention whether you also want to copy the test results. Further you can decide whether the reference to previous versions is to be taken over (see also ***5.8 Versioning***). Appropriate menu options make these functions available.



You can control nearly all operations which manipulate the Test Manager tree by Undo/Redo. Use the context menu or hotkey Ctrl+Z/Ctrl+Y for that.

The versioning feature is an exception to this operation. It is not possible to undo the creation of a release or a developer version.

Also, changes within the test object such as changing the parameter set is not included. Changes in the Basic code can be undone directly within WinWrap Basic.

5.3.5.1 Copying and moving with the context menu

Click on the test you want to copy with the right mouse button and choose *Copy* (or *Cut* in case of a move operation) in the context menu. Click with the right mouse button on the system test or library to which you want to copy the test and choose *Paste* in the context menu.

There are three ways of inserting the test at the desired location:

- **Move here**: To move a test, you must cut it out. The test is then moved together with the results and previous versions.
- **Copy here**: Here you create a copy of the test object for later modification. Changes in the original are not transferred to your copy afterwards. You can select whether to copy the test results too and whether to take over the reference to the previous versions.

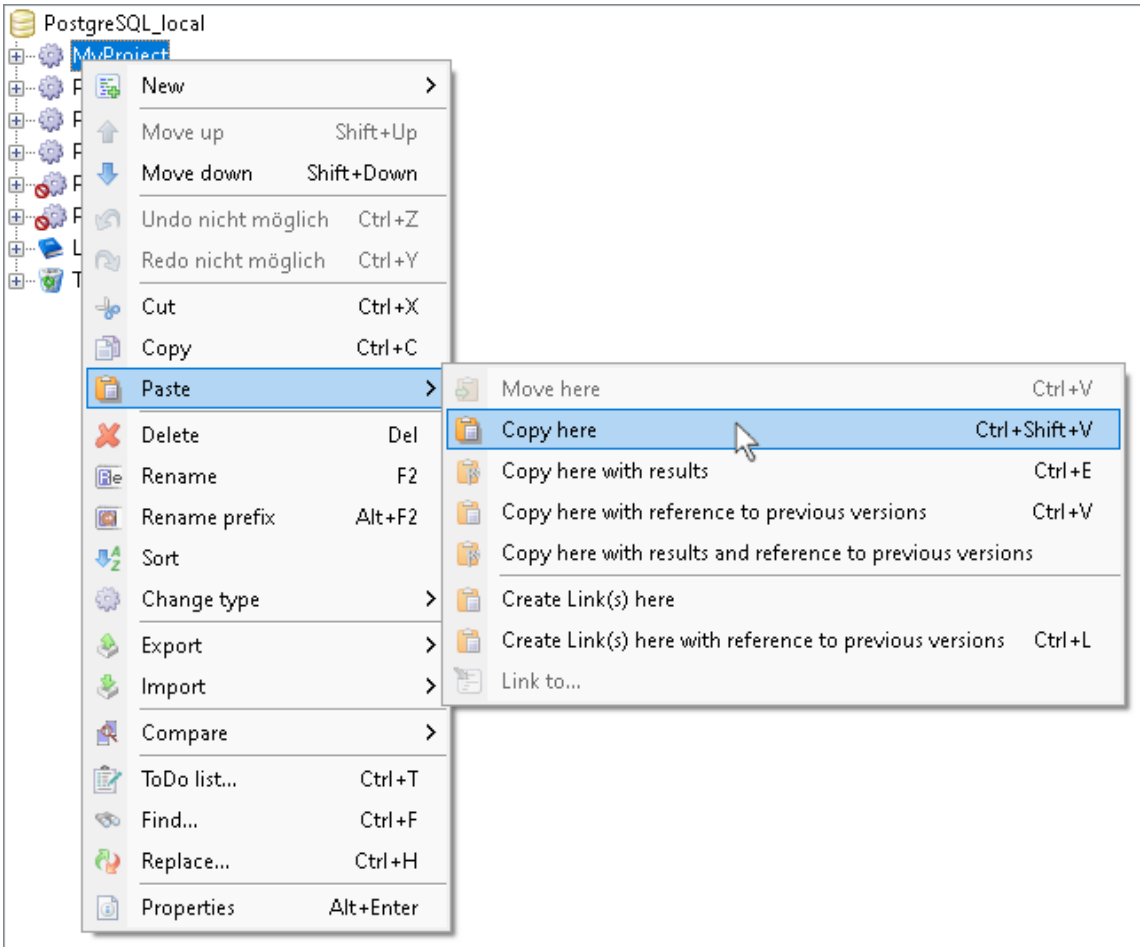


Figure 268 – Pasting of copies or links

- **Create Link(s) here**: In the Test Manager tree links are represented by the symbols (link to an elementary test), (link to an object), (link to a class), (link to a text object) and (link to a user-defined test object). When you insert a test

object as a link, PROVEtech:TA joins this test object with the original. The original becomes a link too, so that two equal objects are created. If you make changes in a link, then you can see these in all associated links (see also ***5.3.14 Linked test objects***). You can select whether to take over the reference to the previous versions. You can have all associated links of an elementary test displayed in the *Properties* of the context menu (see also ***5.3.11 Displaying the properties of tests***).



Links can only refer to elementary tests (tests, objects, classes, text objects or user-defined test objects), but not system tests or user libraries.



It is also possible to insert the path of one or several test objects into a test program. Copy the test objects from the Test Manager tree or `Explorer` tab into the clipboard, and then paste them into the `Program` tab to the appropriate position.

If you copy test objects with reference to previous versions but without test results, it may anyway happen that you can see results at the new copy. These are the results of older versions. In fact, they were not copied. They still exist only once. So, if you delete them then they are also deleted at the original test item. A warning message box informs you about that.

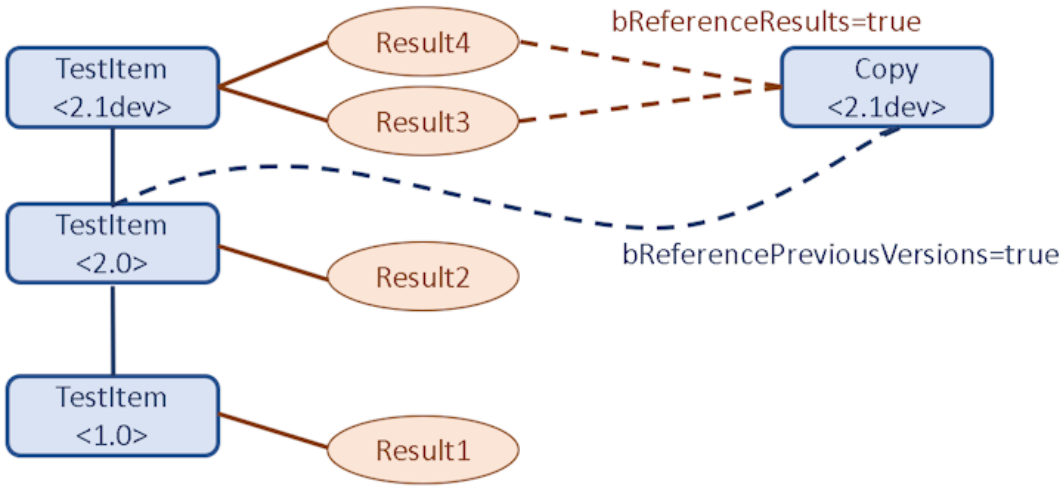


Figure 269 – Copy test objects with test results and reference to previous versions

Moving tests:

The order of the tests in the system test determines the sequence in which they will be executed (also see ***5.5 Executing tests***). The sequence can be changed as desired.

To change the position of a test in a system test, click on the test with the right mouse button. The context menu of the test will open up.

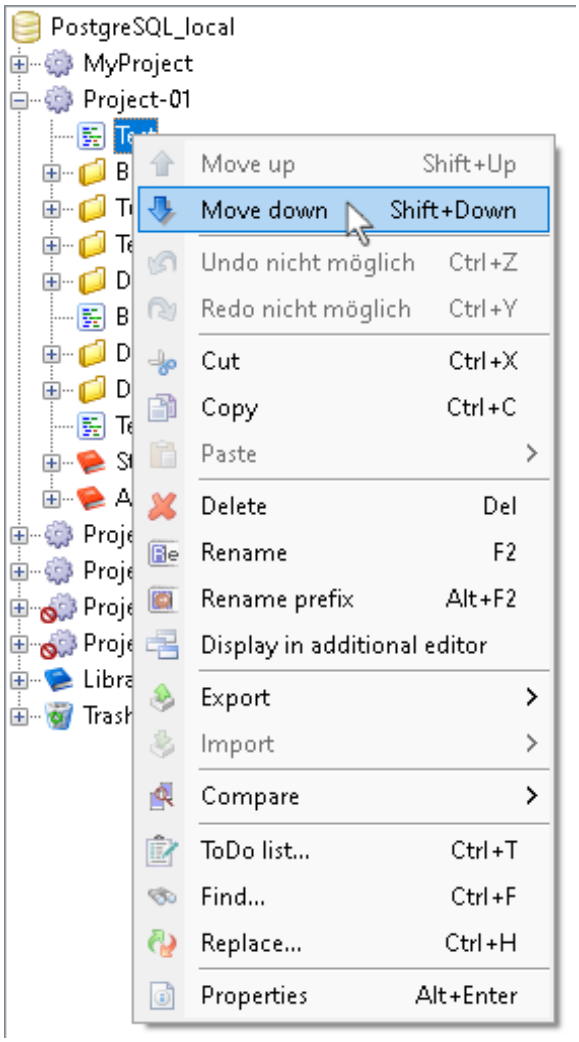


Figure 270 – Moving a test

Click on *Move Up* or *Move Down*. The test is relocated in the Test Manager tree. Keep moving the test until it is in the desired position.



You can also **move tests with the aid of the keyboard**. To do this, mark the test. Then press and hold down the Shift key and use the up and down arrow keys to move the test.



If you want to move or copy a test or create a link, you need write access in the target path. If you want to move tests, you also need write access in the source path (see ***5.2.6 Access rights***).

5.3.5.2 Copying and moving with drag and drop

Moving with drag and drop:

Click on the test you want to move with the left mouse button. Hold the button and drag the test in the tree to the destination position. Release the button to move the test.

Copying with drag and drop:

Click on the test you want to copy with the left mouse button and press the 'Ctrl' key.

The mouse cursor changes to  . Drag the test to the destination position and keep the mouse button and the key pressed. A gray separation bar shows the insertion position of the test. The bar disappears when you drag the test to a position where it would be copied into a system test. This system test is highlighted blue. Release the mouse button to copy the test.

Creating a link with drag and drop:

Click on the test you want to insert as a link in the tree and press the 'Shift' key. The mouse cursor changes to  . Now you can drag the link to the destination position in the tree the same way as copying it. Please read chapter ***5.3.14 Linked test objects*** for more information about links.

Figure 271 shows how to move an elementary test below a system test, this means it resides in the same level as the system test. The elementary test of ***Figure 272*** will be moved into the system test, it will be an element of and one level below the system test.

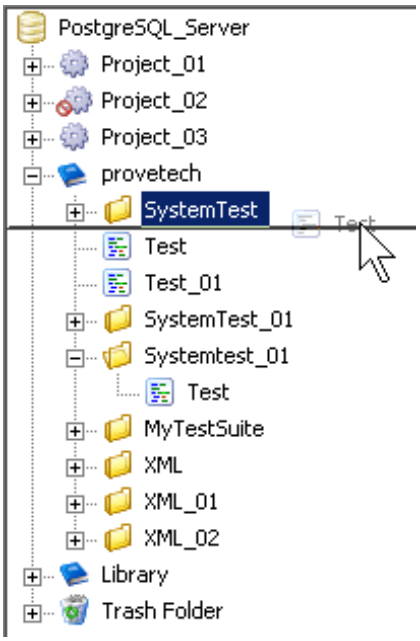


Figure 271 – Moving <u>below</u> the system test

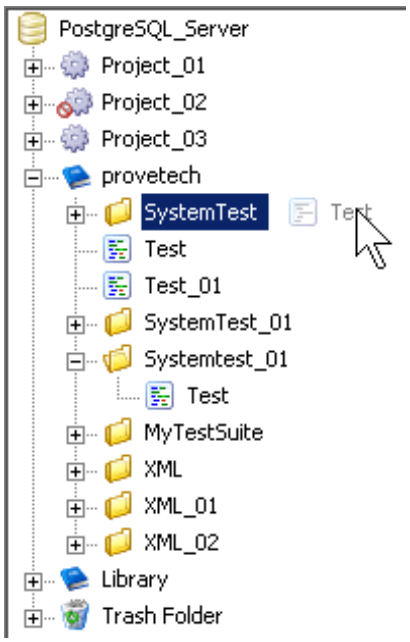


Figure 272 – Moving <u>into</u> the system test

This way you can take over elementary or system tests of other projects or copy them into other system tests very fast. If you do not have write access rights of the source object, then you can only create a copy of it.

You can also do a drag and drop operation with the right mouse button. Then by context menu, you can again confirm the action. Bold font deposited is here the standard action, which you would process if you took the left mouse button for drag and drop.

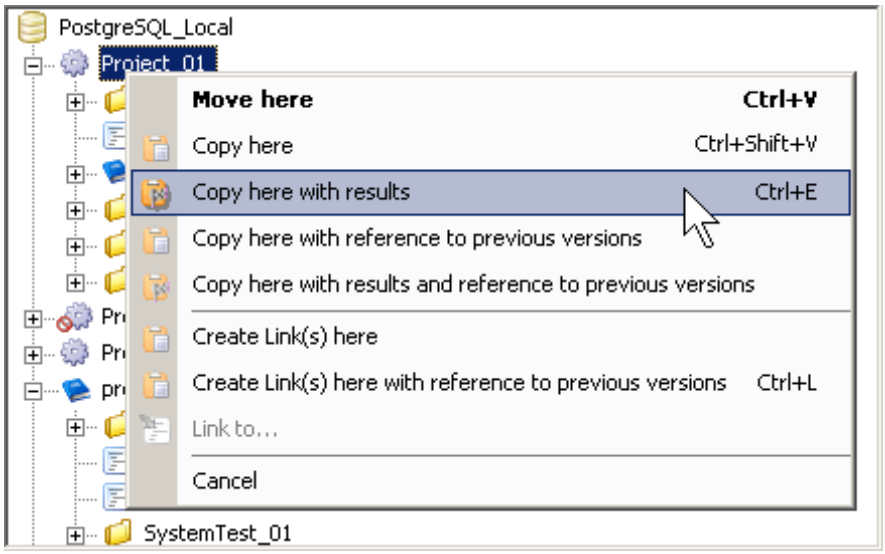


Figure 273 – Drag and drop using the right mouse button



When you move or copy a test below another one, the gray separation bar shows the insertion position. When you want to copy the test into a system test, the system test must be highlighted blue.



When you move a test object by drag and drop from the Test Manager tree into the program code of a test, then its path is inserted as text.



If you want to move or copy a test or create a link you need write access in the target path. If you want to move tests you also need write access in the source path (see ***5.2.6 Access rights***).

5.3.6 The tab “Explorer”

The tab `Explorer` is, together with the test tree, comparable with the Windows-Explorer. If a test object is selected in the Test Manager tree, then its child objects appear in the Explorer list.

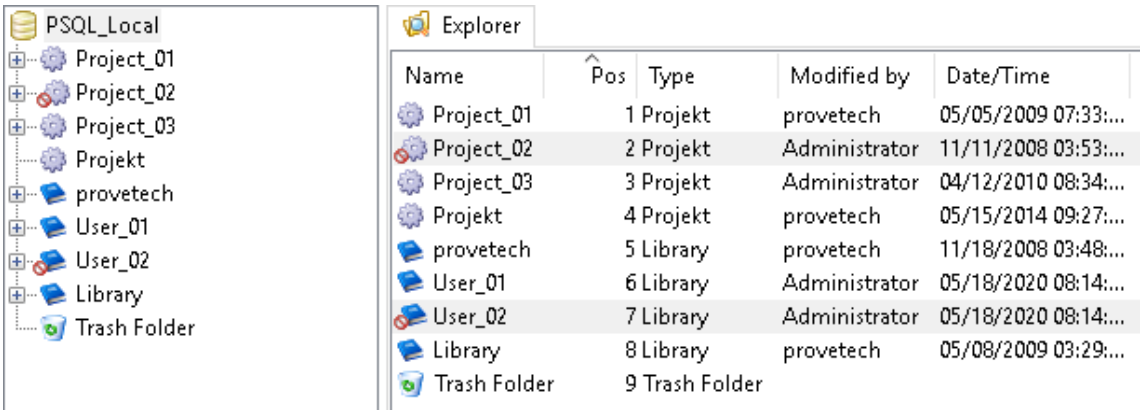


Figure 274 – The tab "Explorer"

The first column `Name` consists of the test object icon, prefix and name. The name is part of the test object path, but not the icon and prefix. For example, an additional user-defined icon in the form of an emoji from the Unicode character table can be set as a prefix (e.g. 👍 or ✔). Please read also in chapter ***5.3.2 Renaming tests***.

The second column `Pos` in the Explorer tab displays the order of the test objects. In contrast to Windows Explorer, the test objects inside its parent object are ordered.

The tab `Explorer` makes things easier for the user, in order to be able to execute certain functions more intuitively. Especially via multi-selection you can process a particular operation on many test objects with one click (e.g. deleting, copying, exporting, …).

Hotkeys

The following hotkeys are implemented:

Hotkey	Function
Ctrl+F5	Refresh
Ctrl+A	Select all
Alt+Return	Display properties
F2	Rename the test object
Alt+F2	Rename the prefix of the test object's name
Ctrl+X / Shift+Del	Cut into the clipboard
Ctrl+C / Ctrl+Ins	Copy a test object and its Test Manager path into the clipboard
Ctrl+Shift+C / Ctrl+Shift+Ins	Same as Ctrl+C except that the Test Manager paths contain database identifiers (see also in chapter ***Test paths*** in the Test Language section)
Ctrl+Shift+V	Paste

Ctrl+V / Shift+Ins	Paste with reference to older versions
Ctrl+E	Paste with test results
Ctrl+L	Create link
Ctrl+B	Open the Branch Selection dialog (see ***5.8.7.2 The branch selection overview dialog***)
Ctrl+Shift+B	Branch Selection: Set/Reset as current version (see ***5.8.7.1 Define the branch selection by context menu***)
Ctrl+T	ToDo list dialog (see ***5.17 ToDo list dialog***)
Ctrl+F	Find (see ***5.18.1 Finding tests and test results***)
Ctrl+H / Ctrl+R	Find and replace (see ***5.18.3 Find and replace in test programs***)
Ctrl+Z	Undo
Ctrl+Y	Redo
Del	Deleting tests
Ctrl while drag and drop	Copying tests
Shift while drag and drop	Creating links
Ctrl+Shift+L	Select left side to compare
Ctrl+Shift+R	Select right software to compare / Compare to
Ctrl+Alt+0..9	Set selected module as a hotkey
Alt+0..9	Select module
Ctrl+Alt+Backspace	Navigate back
Shift+Backspace	Navigate forward
Shift+Up	Move up test object
Shift+Down	Move down test object

5.3.7 The tab “User groups”

If the PROVEtech:TA administrator is logged on or the current user has the *arManage* access right to the root object, you can manage the users groups in PROVEtech:TA by selecting the root object and by choosing the `User groups` tab.

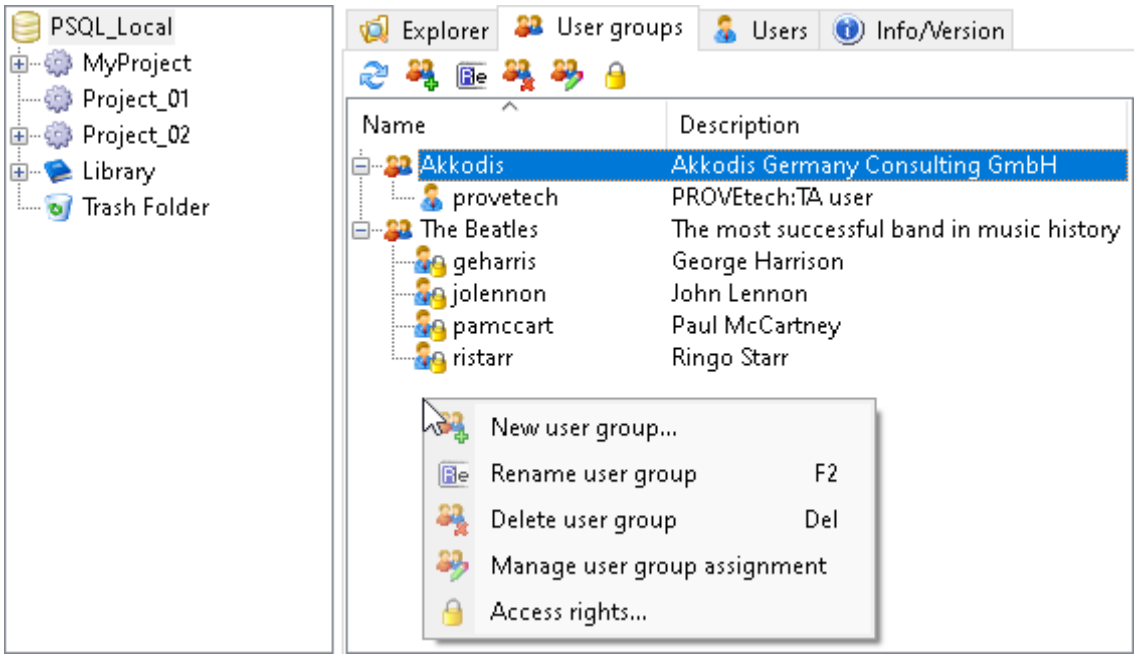


Figure 275 – The tab „User groups“

You can find a short explanation of the different attributes of a user group in chapter ***5.3.7.1 The user group***.

A click on the icon (Refresh) in the toolbar or the F5 key will cause the tab to be re-loaded from the database.

By icons , and you can create a new user group, rename the user group (also F2 key) and delete the user group.

The button with icon opens the user group dialog to administer the users assigned to this user group.

You can control the access rights of a user group to all projects, libraries and the root object via the icon . Read also in chapter ***5.2.6 Access rights***.

5.3.7.1 The user group

New user groups can be created either via the "User groups" tab (see chapter ***5.3.7 The tab “User groups”***) or via context menu on the root object. Either the PROVEtech:TA administrator can create new user groups or the currently logged in user must have the *arManage* access right on the root object or via context menu on the selected project or library.

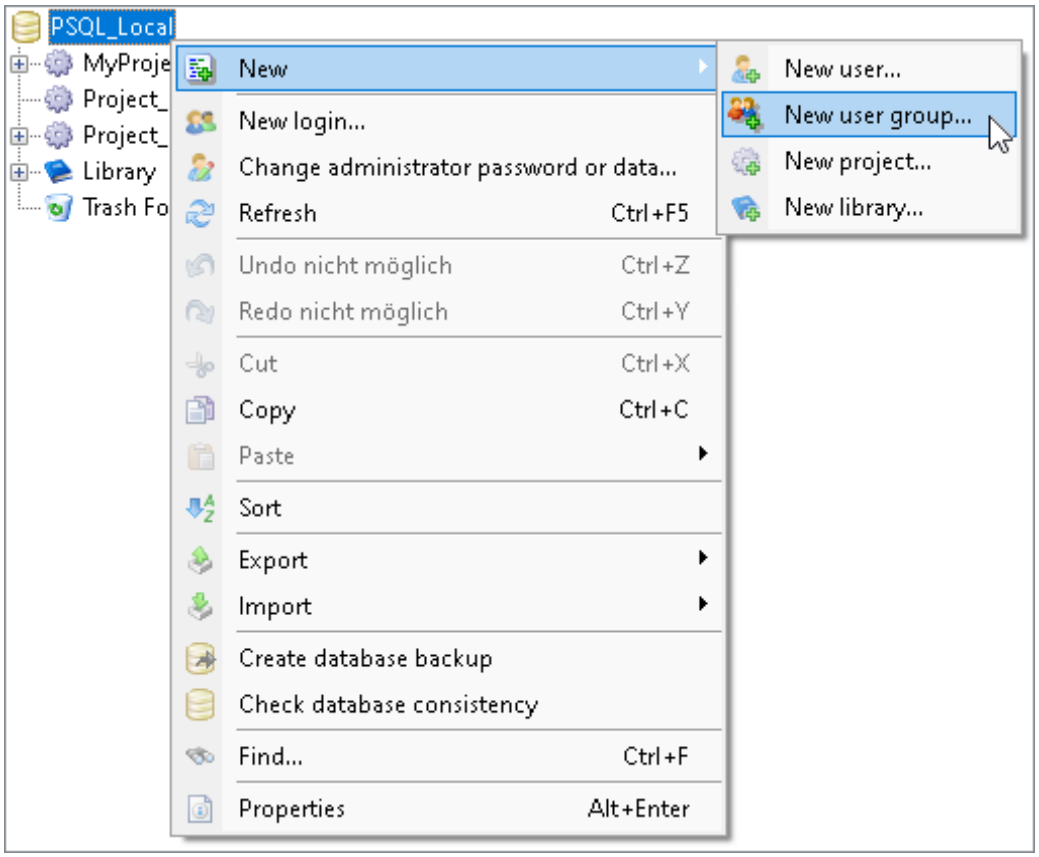


Figure 276 – Context menu „New user group“

Then the following dialog appears.

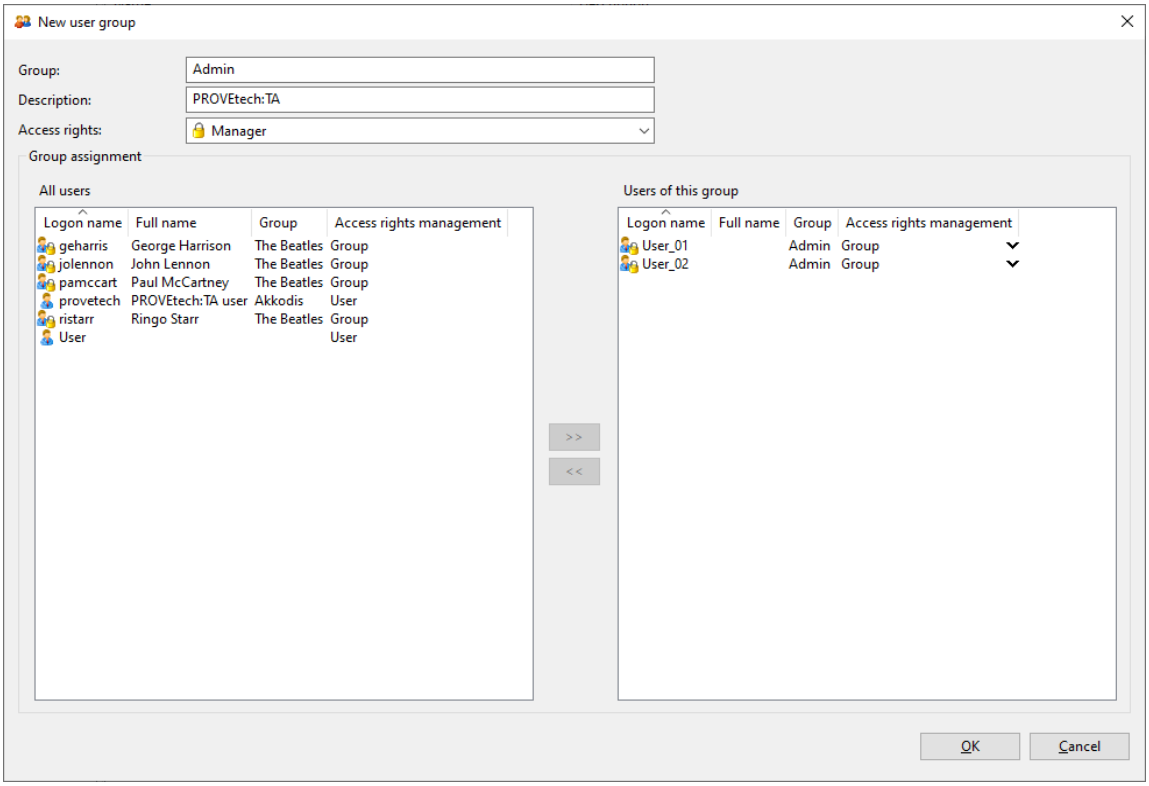


Figure 277 – The Dialog „New user group“

Please enter a new unique user group name and any description.

In the combo box `Access rights` you can set the initial settings for the user group access rights (see chapter ***5.2.6 Access rights***). Later, you can adjust the access rights more exactly.

You can assign any user to the user group. To do this, select users (multiselect possible) in the "All users" list and add them to the "Users of this group" list using the 

button. To remove users from this user group, select users in the "Users in this group" list and add them to the "All users" list using the 

button.

The access rights of the user group only become valid for the users of this user group when "Access rights management" is set to "Group". Users whose access rights are managed via the user group are marked with the icon for easier identification (see chapter ***5.2.6 Access rights***).

5.3.8 The tab “Users”

If the PROVEtech:TA administrator is logged on or the current user has the *arManage* access right to the root object, you can manage the registered users in PROVEtech:TA by selecting the root object and by choosing the `Users` tab.

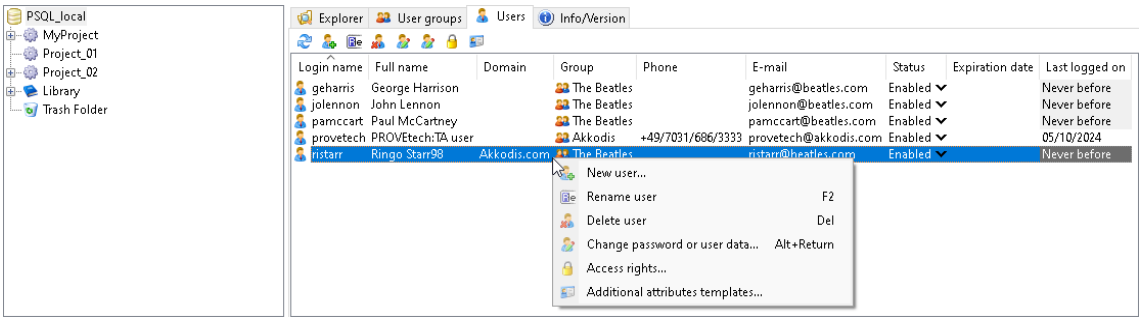


Figure 278 – The tab „Users“

You can find a short explanation of the different attributes of a user in chapter ***5.3.8.1 The user***. Certain attributes may also be grayed out, for example, if the status is set to "Disabled", if the expiration date has passed, or if the user has never been logged in (with a current PROVEtech:TA version).

The `Domain` column shows which login method is stored for the respective user in the PROVEtech:TA database:
a) Empty field means PROVEtech:TA user with password stored in the PROVEtech:TA database
b) Real domain name (e.g. akkodis.com or mycompany.com) means that the Windows password is used for the login
c) `OpenID Connect (OAuth)` means that the Microsoft mechanisms configured for OpenID Connect (OAuth) are used for the login (see chapter ***2.3 Login***).

A click on the icon (Refresh) in the toolbar or the F5 key will cause the tab to be re-loaded from the database.

By icons , and you can create a new user, rename the user (also F2 key) and delete the user. Only the PROVEtech:TA administrator can rename a user.

The two buttons with icon open the user dialog to change the data of the selected user or the administrator. Only the administrator itself can change its data.

You can control the access rights of a user to all projects, libraries and the root object via the icon . If several users are selected, the settings of the focused user are displayed in the access rights dialog. After confirming the access rights dialog, the changes will be applied to all selected users. Read also in chapter ***5.2.6 Access rights***.

Clicking on the icon opens the dialog for managing the additional user attribute templates. Read also in chapter ***5.3.8.2 The dialog “Additional attributes templates”***.

5.3.8.1 The user

New users can be created either via the "Users" tab (see chapter ***5.3.8 The tab "Users"***) or via context menu on the root object. Either the PROVEtech:TA administrator can create new users or the currently logged in user must have the *arManage* access right on the root object or via context menu on the selected project or library.

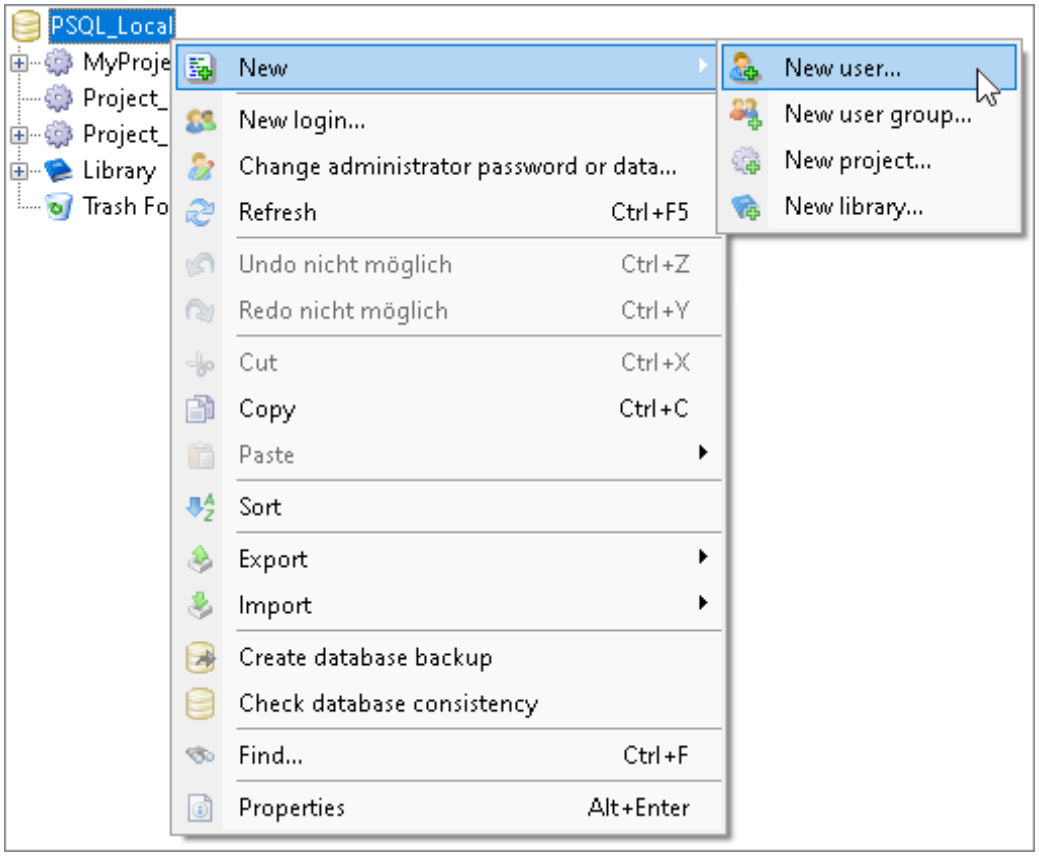


Figure 279 – Context menu "New user..."

Then the following dialog appears.

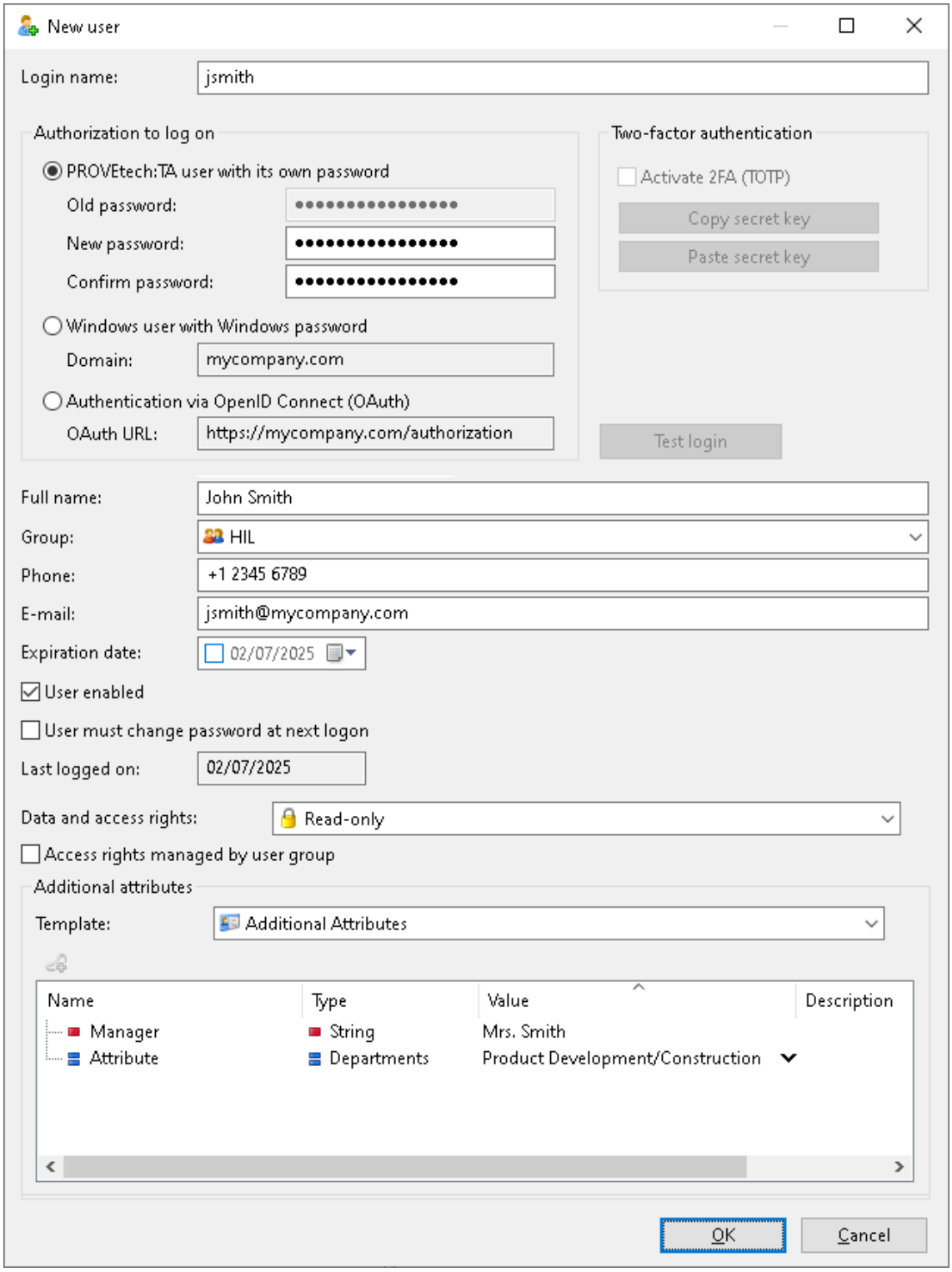


Figure 280 – The dialog "New user"

Please enter a new unique logon name. In the group box `Authorization to log on` you can decide if the user gets an own password that is stored in the PROVEtech:TA

database or if the user can login with its Windows password. This is only possible if the login name exists as a Windows user of the specified domain. Then PROVEtech:TA uses the Windows API function *LogonUser* with the login name, the configured domain, and the parameters LOGON32_PROVIDER_DEFAULT and LOGON32_LOGON_NETWORK to check the password (refer as well to the Microsoft documentation MSDN on the internet). As a third option, the user can be registered for login via OpenID Connect (OAuth), also abbreviated as OIDC. For more details, please refer to chapter ***2.3 Login*** and chapter ***8.3.2 Logging in to the Test Manager does not work***.

Please use the button `Test login` to test the login with the Windows password or login via OpenID Connect (OAuth).

The user settings `Full name`, `Phone` and `E-mail` are used to better document your users and can also be retrieved via the test language. Please read also in chapter ***2.14 Processing and protection of personal data***.

In the `Group` property, you can assign user to a user group. The list of groups results from all created user groups from the "User groups" tab (see chapter ***5.3.7 The tab "User groups"***). To manage the access rights of the user via the user group select the `Access rights managed by group` checkbox (see chapter ***5.2.6 Access rights***).

The `Expiration date` specifies until when the user is allowed to log on to PROVEtech:TA. After that the logon will be blocked.

The display of `Last logged on` helps to see if a user can be deleted. If the last logon date is far in the past, the `User enabled` checkbox can be cleared in the first step. Later, in the second step, the user can be permanently deleted.

If for example a start password was assigned when the user was created, or due to other security policies, the user can be prompted to change the password using the `User must change password at next login` checkbox. However, the choice of password strength is in the user's responsibility. Windows users with a Windows password, on the other hand, must control the password via the Windows operating system.

The combo box `Data and access rights` is only enabled if the checkbox `Access rights managed by group` is not set. You can set the initial settings for the user data and access rights (see chapter ***5.2.6 Access rights***). Later, you can adjust the access rights more exactly.

In the `Template` property, users with arManage access rights to the root object can assign a template with additional attributes to the user. The list of templates results from all created templates via the "Additional attributes templates" dialog (see chapter ***5.3.8.2 The dialog "Additional attributes templates"***). The attributes present in the selected template can then be filled with the respective user specific data.

You can activate two-factor authentication (2FA) for your own user (that is, for the logged-in user) by selecting the `Activate 2FA (TOTP)` checkbox and entering the old password. The following dialog then opens.

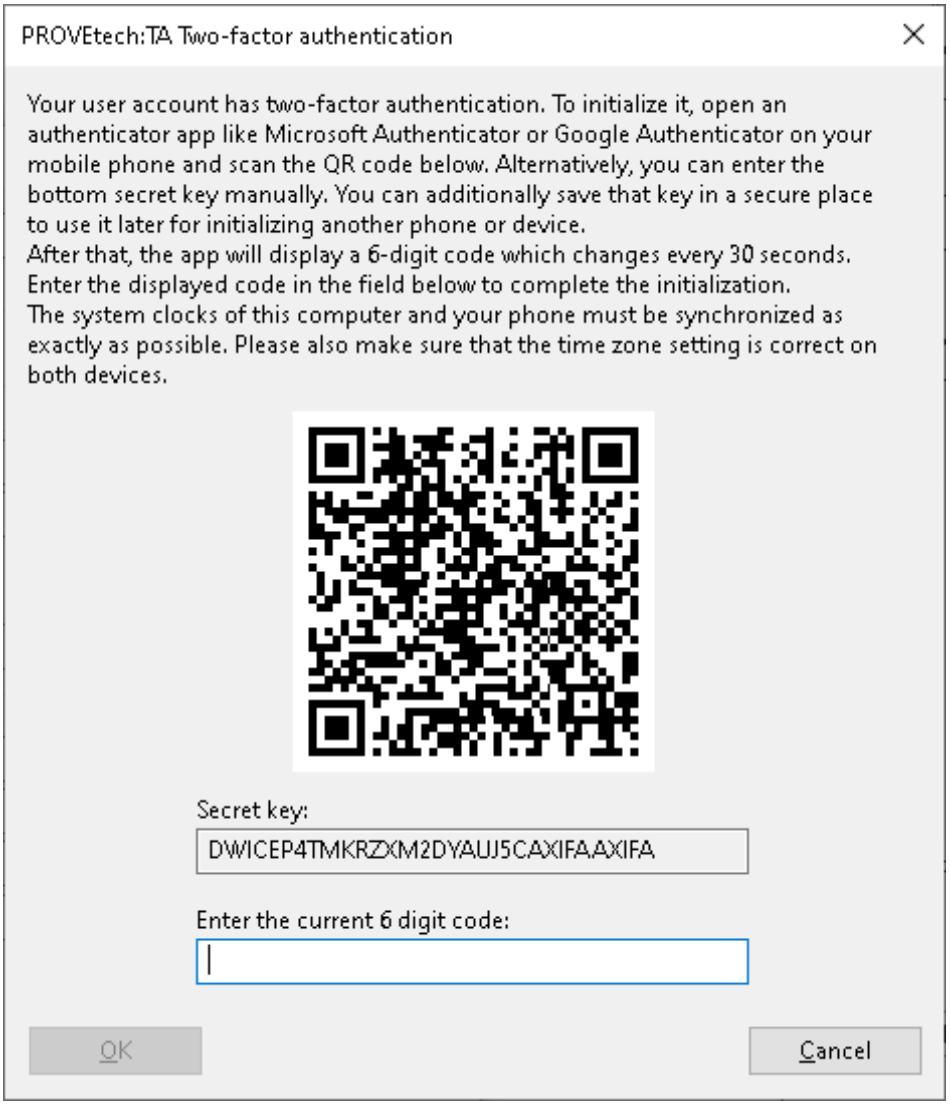


Figure 281 – Two-factor authentication

Follow the instructions in the dialog to activate it.

Two-factor authentication is also known as 2FA, multi-factor authentication or MFA.

You can also transfer the second factor to another user or another database by using `Copy secret key` and `Paste secret key` and specifying the old password and the TOTP code. The secret key is additionally encrypted and stored unrecognizably in the clipboard. This way, you do not have to handle the second factor for different users, and it allows automatic logon to an additional database if all the credentials match.

The PROVEtech:TA administrator can also specify that every user must mandatorily set up the second factor via TOTP. Read about this in the chapter ***5.3.11.2 The tab "Database settings" in the property dialog***.

You can use the context menu of the root object to change the password or edit the attributes of the currently logged-on user. This is also valid for the PROVEtech:TA administrator user 'admin'.

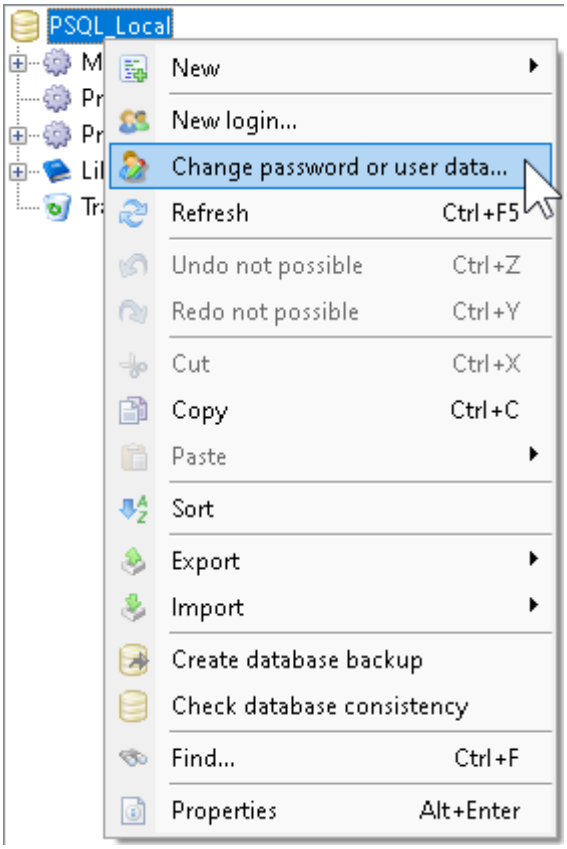


Figure 282 – Change password or user data...

PROVEtech:TA attaches great importance to the protection of personal data. Read more in chapter ***2.14 Processing and protection of personal data***.

User objects are no longer managed as objects in the Test Manager tree as in previous PROVEtech:TA versions, but in the Users tab of the root object. For more information, see ***5.3.8 The tab "Users"***.

Two-factor authentication via TOTP

Currently there are two ways to enable the second factor:

1) Every user **can** activate the second factor for themselves.
2) The PROVEtech:TA administrator can control by checkbox `Two-factor authentication using TOTP is mandatory` that every user **must** set up the second factor for himself (including PROVEtech:TA-admin user themself, see as well in chapter ***5.3.11.2 The tab "Database settings" in the property dialog***).

This ensures that the PROVEtech:TA administrator does not see the secret key of another user.

5.3.8.2 The dialog "Additional attributes templates"

In this dialog you can manage templates for additional user attributes. Attributes have a name, a type and a value. When creating a template, only the name and type can be saved, the value is set individually for each user. The following applies to existing attributes: they cannot be renamed, nor their type changed. The templates can only be managed via the graphical user interface.

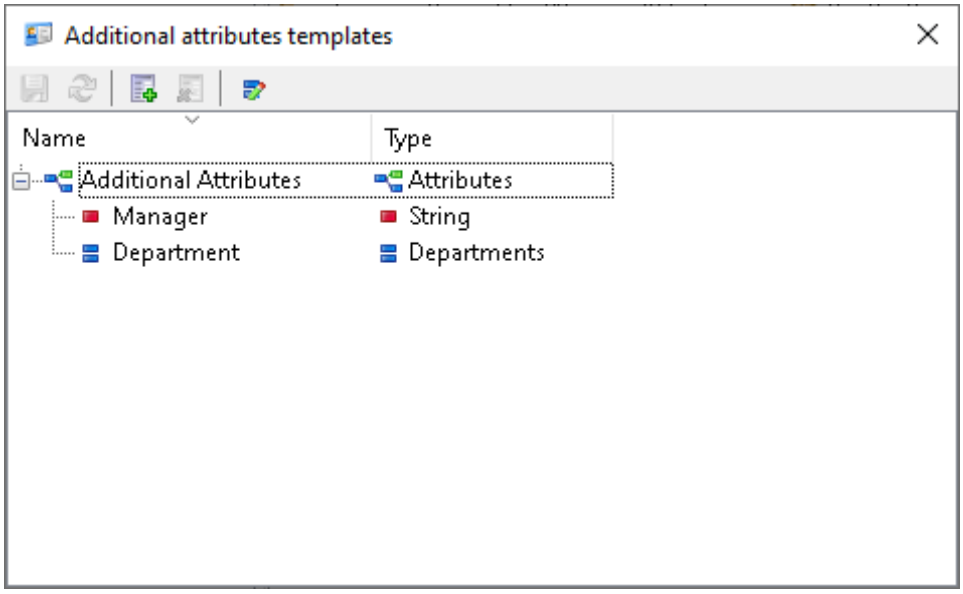


Figure 283 – The dialog "Additional attributes templates"

The following table shows the columns.

Column	Value
Name	Unique name of the attribute

Type	Attribute type (*String*, *Boolean*, *Long*, *Double*, *Date* or *CustomAttributes*); see as well in chapter ***5.3.8.2.1 The attribute type "Choice"***

The sorting criteria of the display is stored for future PROVEtech:TA sessions.

You can group and structure attributes hierarchically by using the type *CustomAttributes*. Click to save your changes in the database, or to reset the content. You can edit your attributes directly in the list control, via context menu or you can use hotkeys.

Choose *Edit choice types…* to define your own enumeration types (***5.3.8.2.1 The attribute type "Choice"***).

Hotkey	Function
Ctrl+S	Save attributes
Del	Delete attributes
Ins	Add a new attribute into the selection
F2	Rename a attribute
Ctrl+A	Select all
F4	Edit attribute type
Ctrl+T	Open dialog "Edit choice types"



- Attribute name and attribute type can only be changed as long as the attribute is a new creation.
- If an attribute is deleted, it is also removed for every user who used the template.
- In future PROVEtech:TA versions it is possible that there are additional attribute types.

5.3.8.2.1 The attribute type "Choice"

Use the context menu *Edit Choice-Types...* or the button to open a dialog in which you can define your own choice types. A choice type is like an enumeration type and can be set as an attribute type. Per choice type, you can specify which values an attribute can accept. This avoids typing mistakes and helps to define your processes.

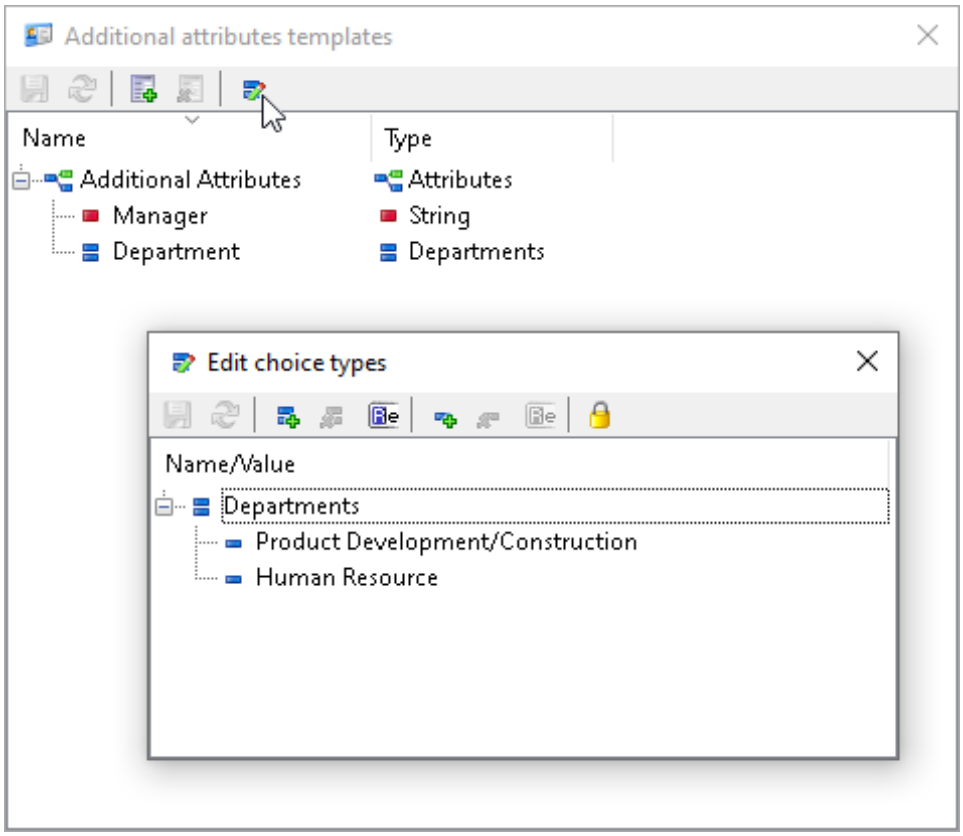


Figure 284 – The attribute type "Choice"

Every choice type is related to a database connection and stored globally in the database for all users.

The choice values of a certain choice type are sorted and must be unique. The empty choice value is also possible.

Users with *arManage* access rights to the root object can configure whether only "managers" or all users are allowed to edit the choice types. If a choice type of a attribute has been deleted, the attribute type automatically changes to *String*.

5.3.9 The tab “Info/Version”

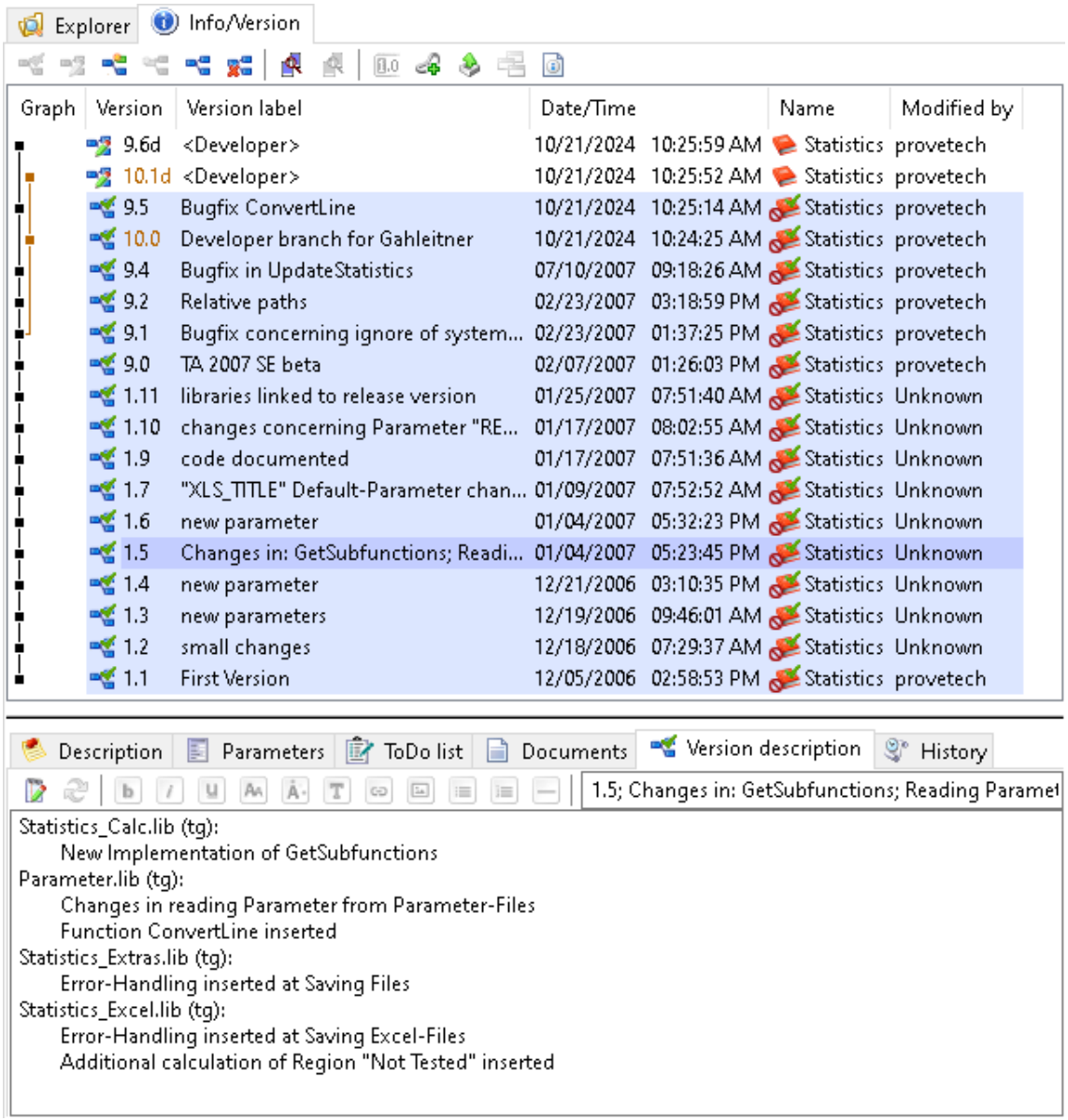


Figure 285 – The tab “Info/Version”

The tab `Info/Version` is separated into two parts. The upper list displays information referring to the versioning of test objects. The lower part shows the tabs `Description`, `Parameters`, `ToDo list`, `Documents`, `Version description` and `History` that provide additional information and allow modifications to the selected test version. If the selected test object is a link, then an additional tab `Link` is displayed. These tabs in the lower section are described in the following chapters.

Depending on the upper selection, the tabs below display additional information to the test version. The same behavior is valid for the tabs `Program`, `Execute`, `Results` and `Overview`, on the right-hand side of `Info/Version`. The background color

“white” means writeable, light blue marks older release versions and gray stands for “read-only”. Older release versions are read-only. You can only modify a copy or create a new developer version from this one.

The Version of a test object is a result of the versioning feature (see ***5.8 Versioning***). The light-brown line in the version graph indicates a version branch. `Date/Time` and `Modified by` are updated when an attribute of the test version is changed. If the time zone is relevant, you will find information in chapter ***8.3.5 Different time zones and daylight saving time***.



Regarding a released version, the Date/Time field is not the point of creating the release version, but the timestamp of the last modification (e.g. Basic code).

5.3.9.1 The tab “Description”

The tab `Description` is located inside the tab `Info/Version`. All test object types in the Test Manager (except the Test Manager root, that means the database item, and the Trash Folder) can contain a description. Here you can see all information of a test object.

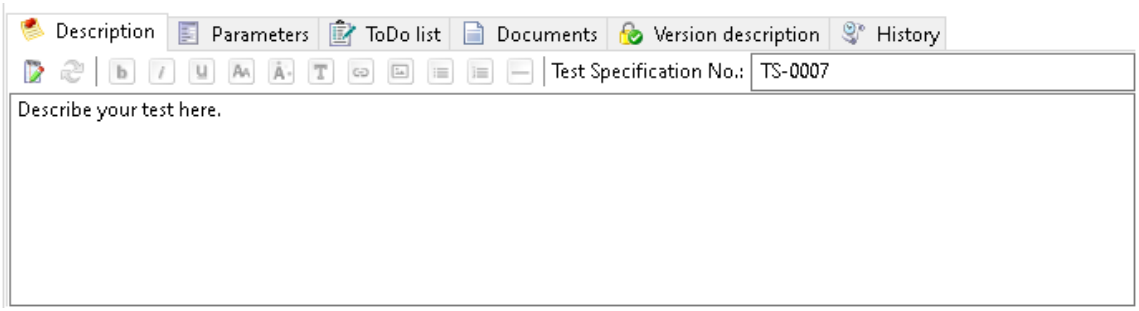


Figure 286 – The tab “Description” of the test object

You can set the `Test Specification No.` in all test objects except the root object and trash folder as well. This information can be retrieved from the Basic language, but it has no further impact.

In the `Description` field you can enter remarks on the test object which will help you and other users when working with the test.
The description text can be stored as formatted text based on HTML. For this purpose, some buttons for text formatting have been added to the user interface.
More detailed information on the use of the HTML formatted description text can be found in chapter ***5.3.9.1.1 The HTML Text Edit control***.

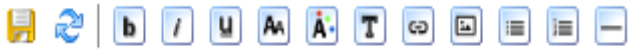


Figure 287 – Text formatting buttons

If no changes have been made in the tab yet, the text formatting buttons for the description are still deactivated and the button on the far left is this one: (`Start Editing`). As soon as is clicked, the HTML editing mode starts, and the text formatting buttons are activated (this also occurs when a change is made in one of the other two text fields `Test Specification No.` or `Type Name`).

The changes in a test object are only saved in the database with the (`Save`) button. If the changes have not yet been saved, the initial state can be restored with the (`Reset`) button.

If a user-defined test object is selected in the Test Manager tree, you have the following view on the `Description` tab.

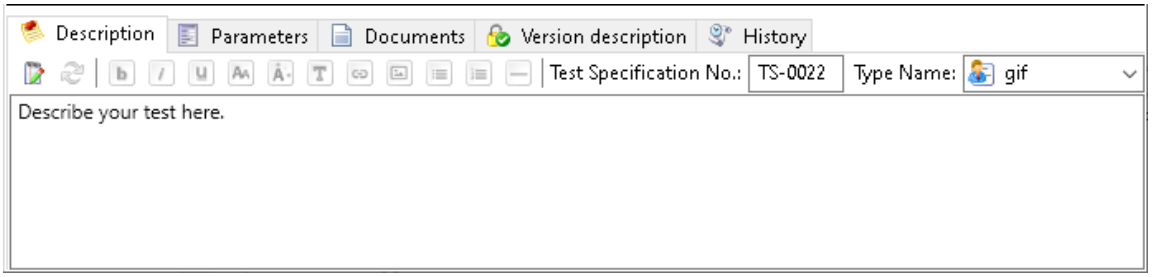


Figure 288 – The tab "Info/Version" of a user-defined test object

You can choose any string for the `Type Name`. This plays a role in the Extended Context Menu. User-defined test objects of the same kind (e.g. Excel files with a special format) should have the same Type Name. You can find more about user-defined test objects in chapter ***5.9 User-defined test objects and extending the context menu***.

5.3.9.1.1 The HTML Text Edit control

In addition to a number of text formatting options, the HTML text editing field also offers the possibility to access PROVEtech:TA test objects and their attached files, such as documents and attachments.
For example, hyperlinks to other test objects or even images can be embedded in the text. For this purpose, you can use the Test Automation references that PROVEtech:TA also supports for creating protocols (see ***5.6.3.4 Test Automation references***).

The HTML Text Edit control (for example, for the `Description`) is only activated after clicking the button (`Start Editing`). After that, the description text can be entered as usual. Since various HTML elements such as hyperlinks are only active when

the HTML content is in a non-editable state, it is necessary to switch between the editable and non-editable states.

The changes made are not saved in the database until the button (`Save`) is pressed. If the changes have not yet been saved, the initial state can be restored via the (`Reset`) button.

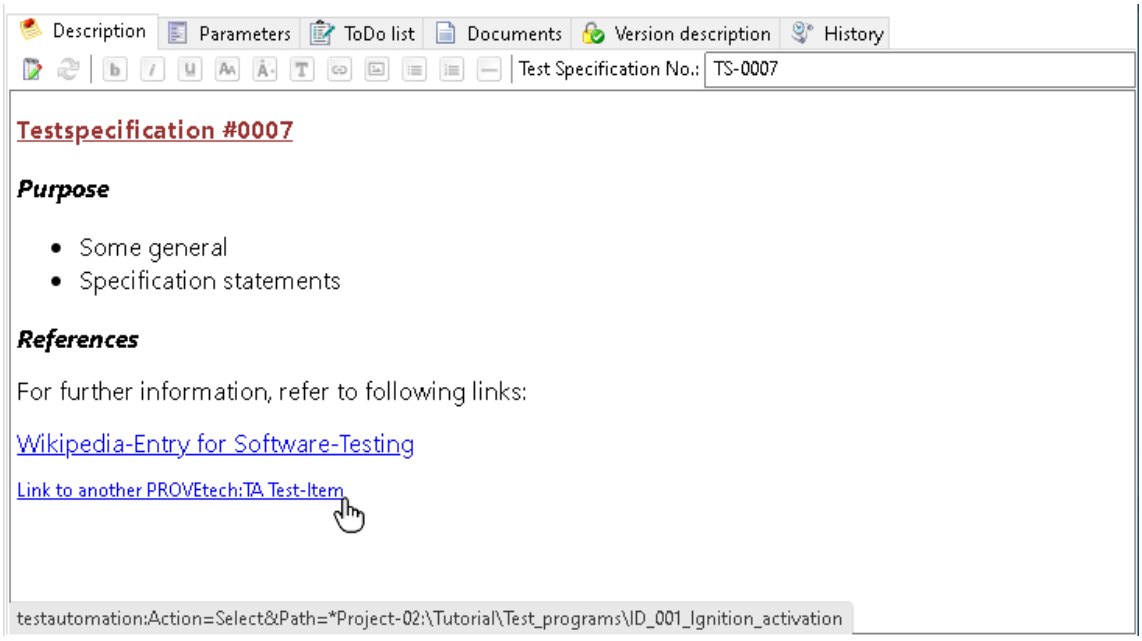


Figure 289 – Example of HTML formatted text

The following buttons are available for text formatting:

- Bold font
- Italic font
- Underlined font
- Choice of font size
- Choice of text color
- Choice of font family
- Inserting a hyperlink
- Inserting an image
- Unordered list
- Ordered list

Horizontal line

All buttons affect the existing, currently selected text. That is, they are not buttons that activate or deactivate a certain mode.

If you want to write bold text, for example, first enter the text, then select it (either with the mouse or by holding down the control key and using the arrow keys), and then click on the `Bold` button.
The same applies if you want to undo a formatting. For example, select a bold text and click the `Bold` button again to return the text to its normal format.

This concept applies to the buttons `Bold`, `Italic`, `Underline`, `Font size`, `Text color`, `Font family`, `Unordered List`, `Ordered List`, and `Insert Horizontal Line`.

For some buttons there are further settings dialogs, which are described here.

- `Font size`

To change the font size, select the desired text and click on the `Font size` button. The following dialog appears:

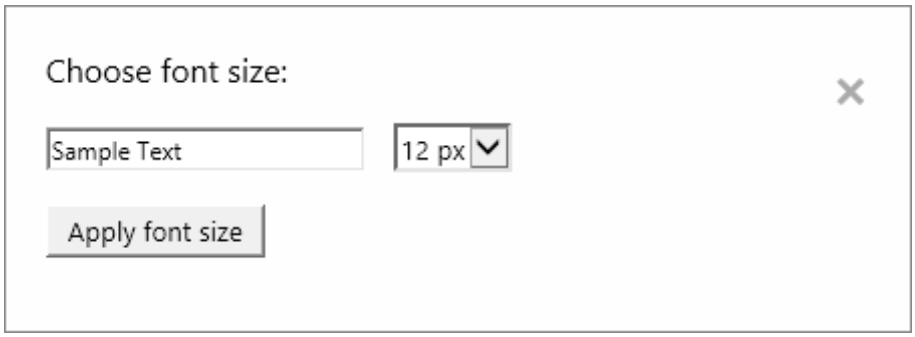


Figure 290 – Settings dialog „Choose font size”

With the combo box you can set the desired font size. A sample text on the left shows a preview of the currently set font size. If you want to apply the change, click on the `Apply font size` button. If you do not want to apply the change, click the `X button` at the top right, or outside the settings dialog.

- `Text color`

To change the font color, select the desired text passage and click on the button `Text color`. The following dialog appears:

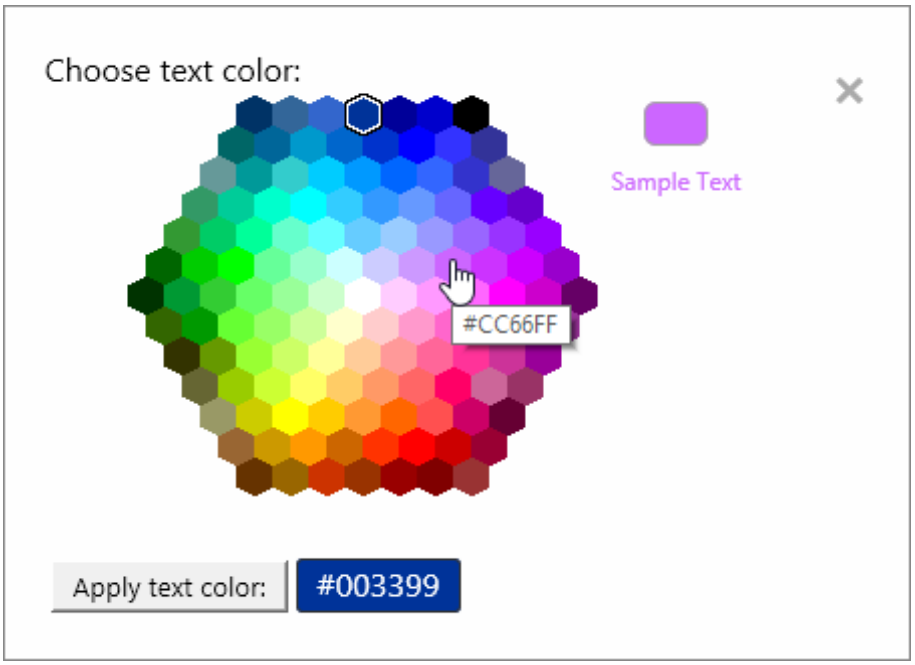


Figure 291 – Settings dialog „Choose text color”

Here you can select the desired color from a selection of predefined colors. If the mouse pointer is moved over the hexagonal view without clicking, a preview of the color over which the mouse pointer is currently located is shown on the right-hand side. The preview color is available on the one hand as a colored rectangle and on the other hand as colored sample text on a white background.
When a color is clicked, it is marked with a small hexagon and the slightly larger rectangle at the bottom of the dialog is colored accordingly. To apply this marked color, click on the `Apply text color` button.

- `Font family`

To change the font family, select the desired text passage and click on the `Font family` button. The following dialog appears:

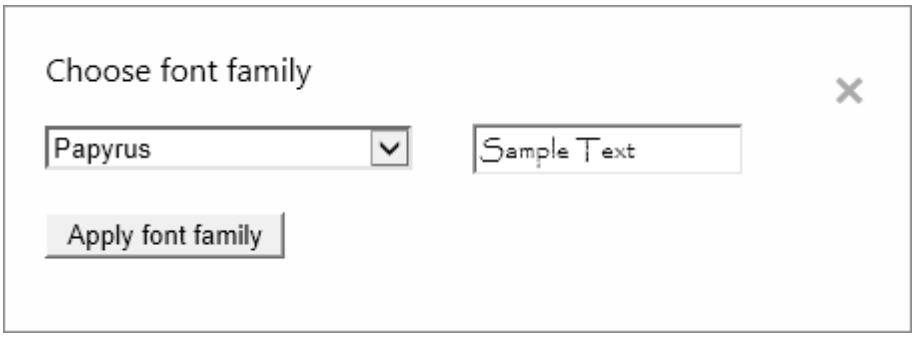


Figure 292 – Settings dialog „Choose font family”

Use the combo box to select the desired font family. A sample text on the right side shows a preview of the selected font. If you want to apply the change, click on the `Apply font family` button. If you do not want to apply the change, click on the `X button` at the top right or outside the settings dialog.

- `Insert Hyperlink`

To insert a hyperlink in the text, position the text cursor at the desired text position and click the `Insert Add Hyperlink` button. In the settings dialog you can enter the URL of the hyperlink and the displayed text of the hyperlink. Click on the `Add Hyperlink` button to apply the settings. Here, too, the process can be cancelled by clicking on the `X button` or by clicking outside the dialogue.

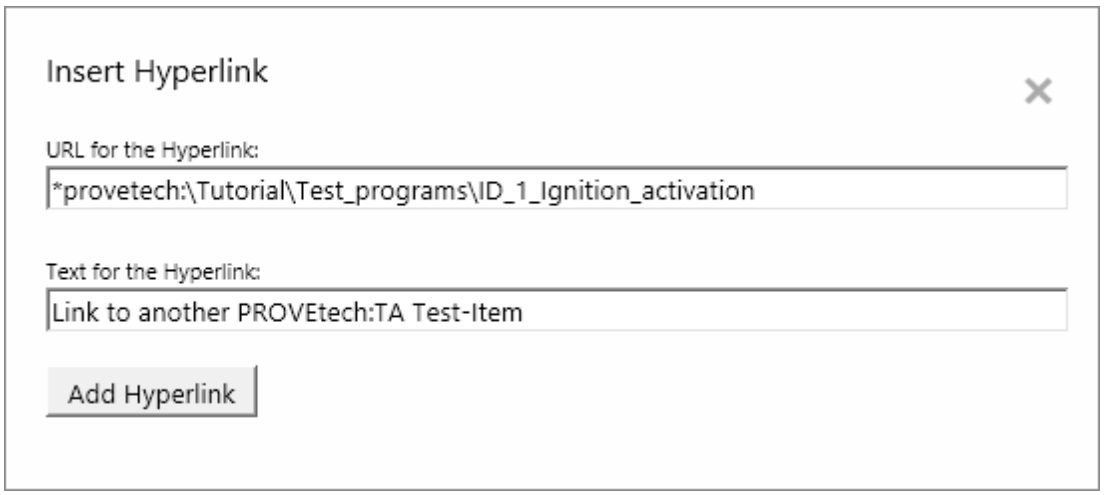


Figure 293 – Settings dialog „Insert Hyperlink“

This settings dialog also offers the possibility to edit an already existing hyperlink within the text. To do this, the entire text of the existing hyperlink must be selected and then the `Insert Hyperlink` button must be pressed. In the setting dialog then the two input fields for URL and text are already pre-filled. You can edit the contents of the two input fields as you wish and apply them by clicking on the `Add Hyperlink` button.

The HTML Text Edit control supports three different URL types for the hyperlinks:

1. Hyperlinks to external websites
    - If the URL begins with the prefix http:// or https://, clicking on the hyperlink will open the system default web browser and display the URL there.
    - Example:
      `https://www.provetech.de/`

2. Hyperlinks to files

    - If the URL refers to a file, the file is opened and displayed with the corresponding system standard app for the file extension.
    - The URI type file:// does not have to be placed in front of the file path. This is done automatically internally.
    - Example:
      `C:\TEMP\ExampleFile.txt`

3. Hyperlinks to PROVEtech:TA Test Automation References

    - If the URL starts with a * or with the prefix TestAutomation:, and the path refers to a test object of the PROVEtech:TA database, the URL is interpreted as a Test Automation reference. Thus, the referenced test object can either be selected, displayed, or executed. See also chapter ***5.16 Supported URL formats***.
    - The prefix TestAutomation: is automatically added internally before the path if it begins with a *.
    - Examples:

        i. `TestAutomation:Path=*MyProject:\Test1*res*20180531T165213Z*att*Measure.hdr&Action=Open`

        ii. `TestAutomation:Action=Select&Path=*MyProject:\Test1*res*20180531T165213Z`

        iii. `*MyProject:\Test1*res*20180531T165213Z`

Restrictions:

Test-Automation references beginning with the prefix taref: cannot be used as URLs for a hyperlink. For the HTML hyperlinks, only PROVEtech:TA paths with the prefix TestAutomation: are allowed.

-  `Insert Image`

To insert an image into the text, position the text cursor at the desired text position and click the `Insert Image` button. In the settings dialog you can enter the URL of the image. Click on the `Add Image` button to apply the settings. Here, too, you can cancel the process by clicking on the `X button` or by clicking outside the dialog.



Figure 294 – Settings dialog „Enter image URL“

The HTML Text Edit control supports three different URL types for images:

1. Images from external websites
    - If the URL begins with the prefix http:// or https://, the image is always loaded from the specified network path. URLs from the internet as well as from the intranet can be used.
    - Example:
      `https://www.wikipedia.org/portal/wikipedia.org/assets/img/Wikipedia-logo-v2.png`
2. Images from files
    - If the URL refers to a local file or a mapped network drive path, the image is always loaded from the specified file path.
    - Example:
      `C:\TEMP\ExampleImage.png`
3. Images from PROVEtech:TA Test Automation References
    - If the URL begins with a * or the prefix taref:, and the path refers to a test object of the PROVEtech:TA database, the image is loaded and displayed from the specified test object. See also chapter ***5.6.3.4 Test Automation references***.
    - The prefix taref: is automatically added internally before the path if it begins with a *.
    - Examples:
        i. `taref:*MyProject:\Tutorial\Test_programs\ID_1_Ignition_activation*doc*HW_Setup.bmp`
        ii. `*MyProject:\Tutorial\Test_programs\ID_1_Ignition_activation*doc*HW_Setup.bmp`

General notes on the HTML Text Edit control

You can also paste content from numerous sources into the Text Edit control using the Windows Copy & Paste functions. Please note that the default font in the Text Edit control is "Segoe UI". If the inserted content does not have an explicit HTML <font> tag, it will also be displayed with the font "Segoe UI". The Text Edit control currently does not have a button to change the font.

It is possible to embed images in the text that have previously been copied to the Windows clipboard. In this case, the embedded image does not require an URL, but is inserted directly into the text as a base64 encoded string.

It is also possible to change the size of inserted images. To do this, the editable mode of the HTML Text Edit control must be active. By clicking on the image, corresponding anchor points are displayed around the image. By dragging and dropping the anchor points, you can drag the image to the desired size.



Before saving the contents of the HTML Text Edit control in the PROVEtech:TA database, the contents are always checked first to see whether HTML formatting is actually contained. If this is not the case, the content is stored in the database as plain text. This means that for some special characters the form stored in the database differs from the displayed form. Users must take this into account during manual search/replace operations.
Affected special characters are:

<	<
>	>
&	&
‘	'
“	"
Linebreaks \r\n	
Blank spaces	



The HTML Text Edit control is an embedded web browser. Therefore, you can use the `ResultBrowserType` entry in the TM.ini file to decide whether to use Microsoft Internet Explorer or Microsoft Edge (see ***Administrator's Guide***).



Some of the common web browsers do not support certain image formats. Therefore, embedded images may not be displayed correctly. For example, Microsoft Internet Explorer is not able to interpret the image format "WebP".



The above-mentioned function of changing the image size by dragging and dropping the anchor points is currently only available if Microsoft Internet Explorer is used. The Microsoft Edge browser does not yet support this feature.

5.3.9.2 The tab “Parameters”

The tab `Parameters` is located inside the tab `Info/Version`. In the tab `Parameters` you can add parameters to a test object. A parameter has a name, a type, a value and other attributes. You can process parameters manually via the graphical user interface or automatically via the test language.

Furthermore, parameter inheritance is supported. This means that a parameter of the parent test object is as well displayed at all child objects. The child object can also override inherited parameters. Attention: Inherited parameters have no influence on the versioning of child test objects. If, therefore, a new parameter is created for the parent test object, the child objects will not turn from release versions to developer versions.

In addition, you can store one parameter set in the current test result.

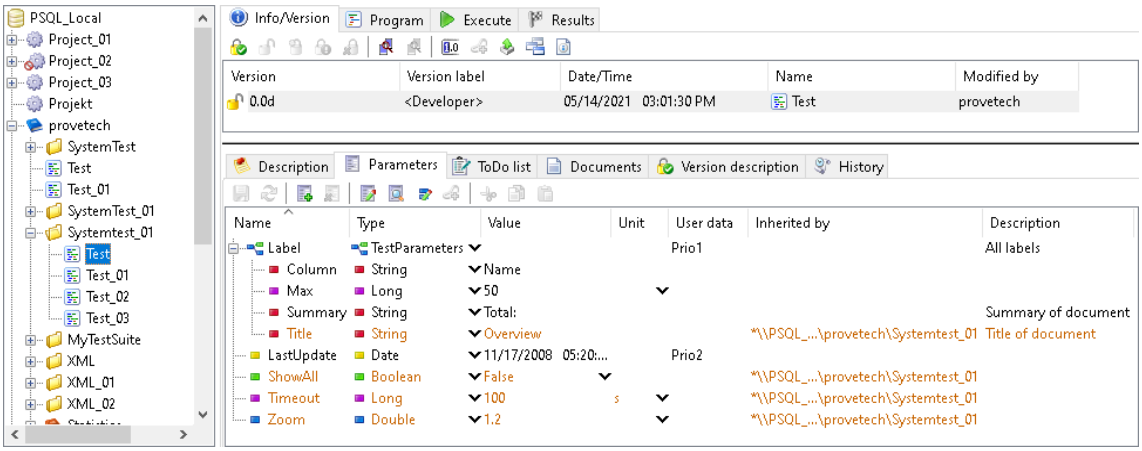


Figure 295 – The tab "Parameters"

All test object types in the Test Manager (except the Test Manager root, that means the database item, and the Trash Folder) can contain parameters. The following table shows the columns.

Column	**Value**
Name	Unique name of the parameter
Type	Parameter type (*String*, *Boolean*, *Long*, *Double*, *Date* or *TestParameters*); see as well in chapter ***5.3.9.2.1 The parameter type "Choice"***
Value	Parameter value that matches the type; the type *TestParameters* has no value
Unit	You can set the unit at the parameter type *Long* or *Double*; the unit has no further functionality
User data	Additional information that can be used e.g. as a sorting criteria
Inherited from	Origin path of an inherited parameter
Description	Parameter description

The sorting criteria of the display is stored for future PROVEtech:TA sessions.

You can group and structure parameters hierarchically by using the type *TestParameters*. Click to save your changes in the database, or to reset the content. You can edit your parameters directly in the list control, via context menu or you can use hotkeys.

You can rename or delete a parameter in the whole subtree of a test object. Therefore, you can use *Deep rename* and *Deep delete* in the context menu.

Choose *Edit choice types…* to define your own enumeration types (***5.3.9.2.1 The parameter type "Choice"***).

Use *Show inheritance…* to display the whole inheritance overview of a parameter (see ***5.3.9.2.2 The dialog "Show inheritance"***).

If you have a gray background, then the parameters are read-only. Parameters of an older test version are displayed with a light blue background.

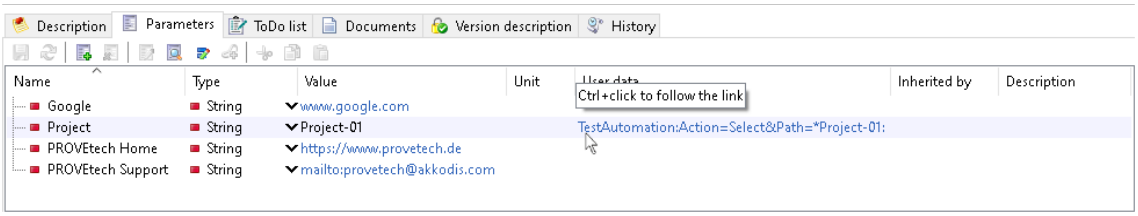


Figure 296 – Hyperlinks as parameter value, user data or description

You can set a URI (hyperlink) as a value, user data or description of test parameters. Clicking on such a URI while holding the Ctrl key will execute it with the application set in the Windows Registry. In case of **https:** or **www** this could be the default browser, for **mailto:** your default mail program, for **TestAutomation:** it is PROVEtech:TA itself and for **file:** any file on the file system.
It is also possible to attach a URL directly to a parameter of the type *String*, which then uses the display text in the 'Value' field. This means that the hyperlink URL can be defined independently of the text in the 'Value' field. If a URL is specified, the text of 'Value' turns a shade of blue and the hyperlink can also be jumped to via Ctrl+mouse click.
To do this, use the `Add hyperlink` or `Edit hyperlink` button.
For a detailed description of all possible hyperlink URLs, see chapter ***5.16 Supported URL formats***.

While executing a test you can set a parameter set to the current test result via test language. For example, you can save the current parameter set in the test result. After that you can see the parameters at the test result (see ***5.6.8 Parameters***).

The following table shows all hotkeys of the parameter dialog.

Hotkey	Function
Ctrl+S	Save parameters
Ctrl+X	Cut parameters
Ctrl+C	Copy parameters (This hotkey also copies the selected parameters as a xml string into the clipboard. Then you can paste it into a text editor.)
Ctrl+V	Paste parameters
Del	Delete parameters
Shift+Del	Delete the parameter in the whole test object's subtree (Deep delete)
Ins	Add a new parameter into the selection

F2	Rename a parameter
Ctrl+F2	Rename a parameter in the whole test object's subtree (Deep rename)
Ctrl+A	Select all
F4	Edit parameter type
Ctrl+T	Open dialog "Edit choice types"
Ctrl+W	Edit parameter value
Ctrl+K	Add/edit hyperlink
Ctrl+U	Edit parameter unit (only possible if the parameter is of type Long or Double)
Ctrl+D	Edit parameter description
Ctrl+I	Show inheritance dialog



- The parameters are assigned to the Test Manager tree entries. That means, each link of a test object can have its own parameters.
- You can use Copy and Paste to copy parameters between test objects. If you copy parameters, they are also put in the clipboard in an xml formatted string. In this format the parameters are stored (compressed) in the database, too.
- You can Copy and Paste the parameters into a text editor, edit them and then Copy and Paste them back to the test item.
- Ever parameter contains information about the timestamp of the last modification. This is displayed by tooltip.
- In future PROVEtech:TA versions it is possible that there are additional parameter types.



The test parameters are not intended to store larger amounts of data. For this, it is better to use test documents or attachments. Test parameters are optimized concerning caching, database access, hierarchy, inheritance, extensibility in future PROVEtech:TA versions, and versioning.

5.3.9.2.1 The parameter type "Choice"

Use the context menu *Edit Choice-Types...* or the button to open a dialog in which you can define your own choice types. A choice type is similar to an enumeration type and can be set as a parameter type. Per choice type, you can specify which values a test parameter can accept. This avoids typing mistakes and helps to define your processes.

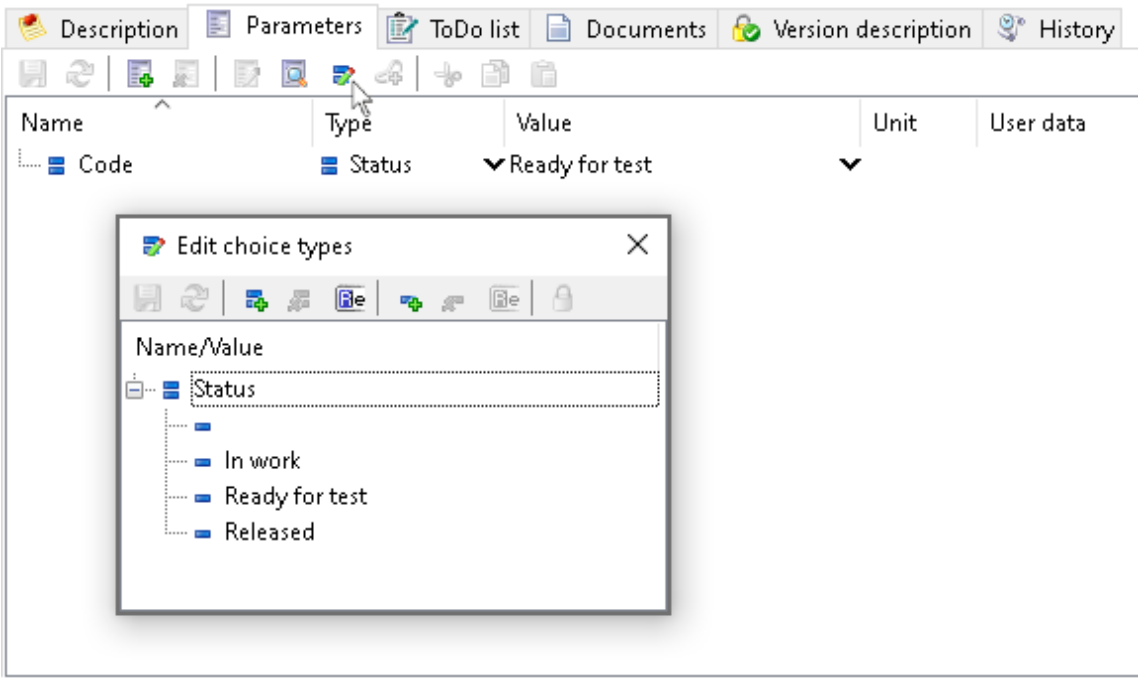


Figure 297 – The parameter type "Choice"

Every choice type is related to a database connection and stored globally in the database for all users.

The choice values of a certain choice type are sorted and must be unique. The empty choice value is also possible.

Users with *arManage* access rights to the root object can configure whether only "managers" or all users are allowed to edit the choice types. If a choice type of a parameter has been deleted, the parameter type automatically changes to *String*.

5.3.9.2.2 The dialog “Show inheritance”

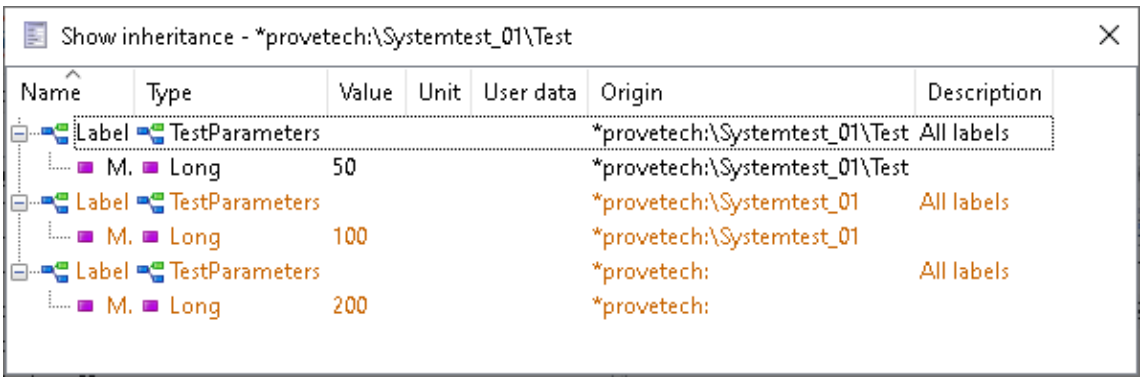


Figure 298 – The dialog “Show inheritance”

You can open the “Show inheritance” dialog via the parameter context menu or the hot-key Ctrl+I to display the complete inheritance structure of a parameter. When you double-click one parameter you go to the definition of the parameter (see also ***5.3.9.2 The tab “Parameters”*** and ***5.6.8 Parameters*** of the test result).

5.3.9.3 The tab “ToDo list”

The tab `ToDo list` is located inside the tab `Info/Version`. Here you can see a list of comments from the program code that contain one of the following keywords:

#TODO

#INFO

#NOTE

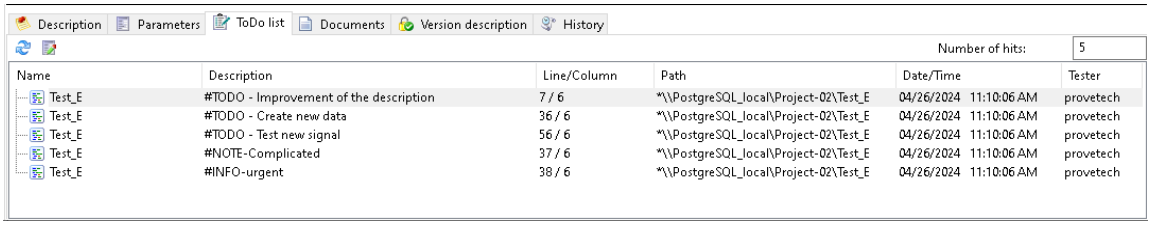


Figure 299 – The tab “ToDo list”

The program code is analyzed for keywords both when saving the program code and when selecting a version in the version list (only if the `ToDo list` tab is also open).

You can also update the listing manually using .

By double-clicking on an entry or using , you can jump directly to the location in the program code.

If you would like a list of the ToDo list entries of a system test, library or project across all sub-elements, use the ToDo list dialog (see ***5.17 ToDo list dialog***)

If you edit keywords in the program code with older versions before PROVEtech:TA 2024 SE, there will be no automatic analysis for keywords when saving and therefore the `ToDo list` entries in the database will not be updated.



However, you can also trigger the analysis manually for all test object types in the Test Manager (except the Test Manager root, that means the database item, and the Trash Folder). To do this, after selecting the tree entry, click on the in the `ToDo list` tab while holding down the CTRL + SHIFT key combination.

Attention: Depending on the number/complexity of the substructures to be analyzed, this may well be a database intensive operation.

5.3.9.4 The tab “Documents”

The tab `Documents` is located inside the tab `Info/Version`. Here you can see the files that are attached to the test object. These files are also stored in the database and changes are considered regarding the versioning of test objects.

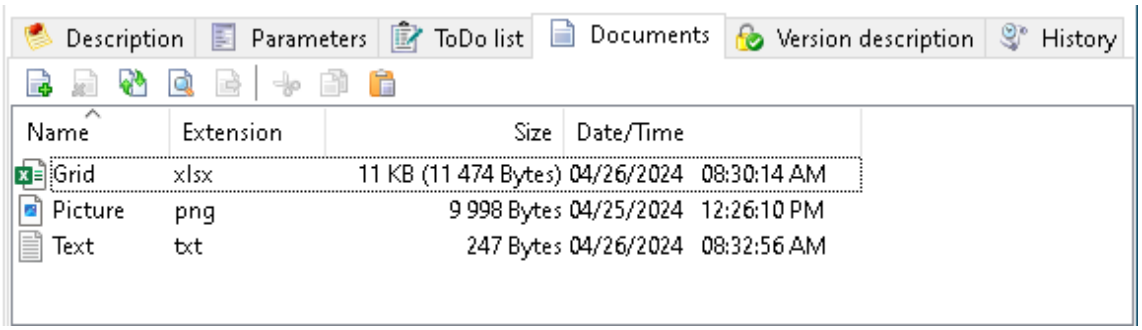


Figure 300 – The tab “Documents”

All test object types in the Test Manager (except the Test Manager root, that means the database item, and the Trash Folder) can contain one or more documents. You can process them manually via the graphical user interface or automatically via the test language. For manually working you can use the buttons, the context menu or the default short keys. If you double-click a document, then Windows tries to show this document. You can also process a document automatically in a test script, e.g. export the document to a temporary directory and readout the content.

If you have a gray background, then the documents are read-only. Documents of an older test version are displayed with a light blue background.

The column `Name` is editable. By default, it is the name of the file. You can use a document name multiple time, too. The `Extension` of the document corresponds to the file extension. The document `Size` is the original size of the file. It is stored in the database in a compressed format. You can display both sizes when moving the mouse pointer

over the entry. The column `Date/Time` is the timestamp of the last update. The timestamp is updated when the binary data of the test document changes, but not when the test document is renamed.

You can use copy and paste to insert, move or copy a test document. Drag and drop of a file from Windows Explorer and drag and drop of a test document path/file path from an editor is possible, too.



If you want to integrate the test document in the test protocol, we strongly recommend not using the '#' character or other special characters in the attachment name that have a special meaning in URLs. Otherwise, it is possible that the subsequent navigation to this attachment or its display will not work.

5.3.9.5 The tab “Link”

The tab `Link` is located inside the tab `Info/Version`. If a link object was selected then you can see the path names of all linked objects in the tab `Link`. You can perform operations on these links via toolbar or context menu. Please read chapter ***5.3.14 Linked test objects*** for more information about links.

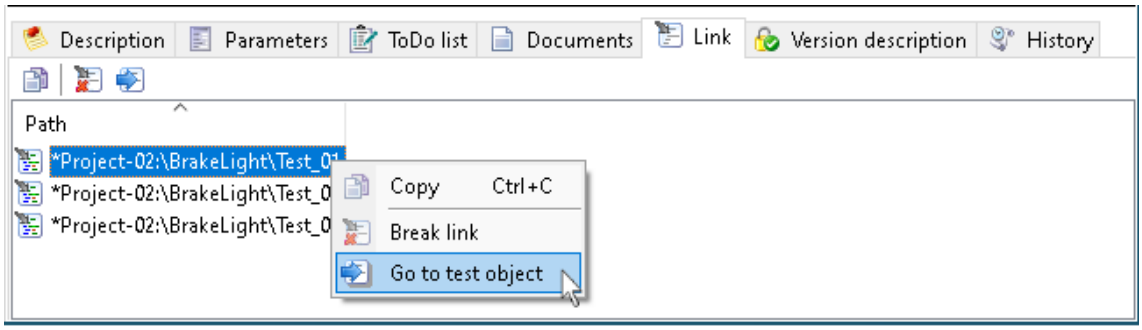


Figure 301 – The tab “Link” in the property dialog

5.3.9.6 The tab “Version description”

The tab `Version description` is located inside the tab `Info/Version`. You can edit the `Version description` for the marked release version. The description of the version is the same for all objects, which were released together. You cannot change the version description of a developer version (see also ***5.8 Versioning***).

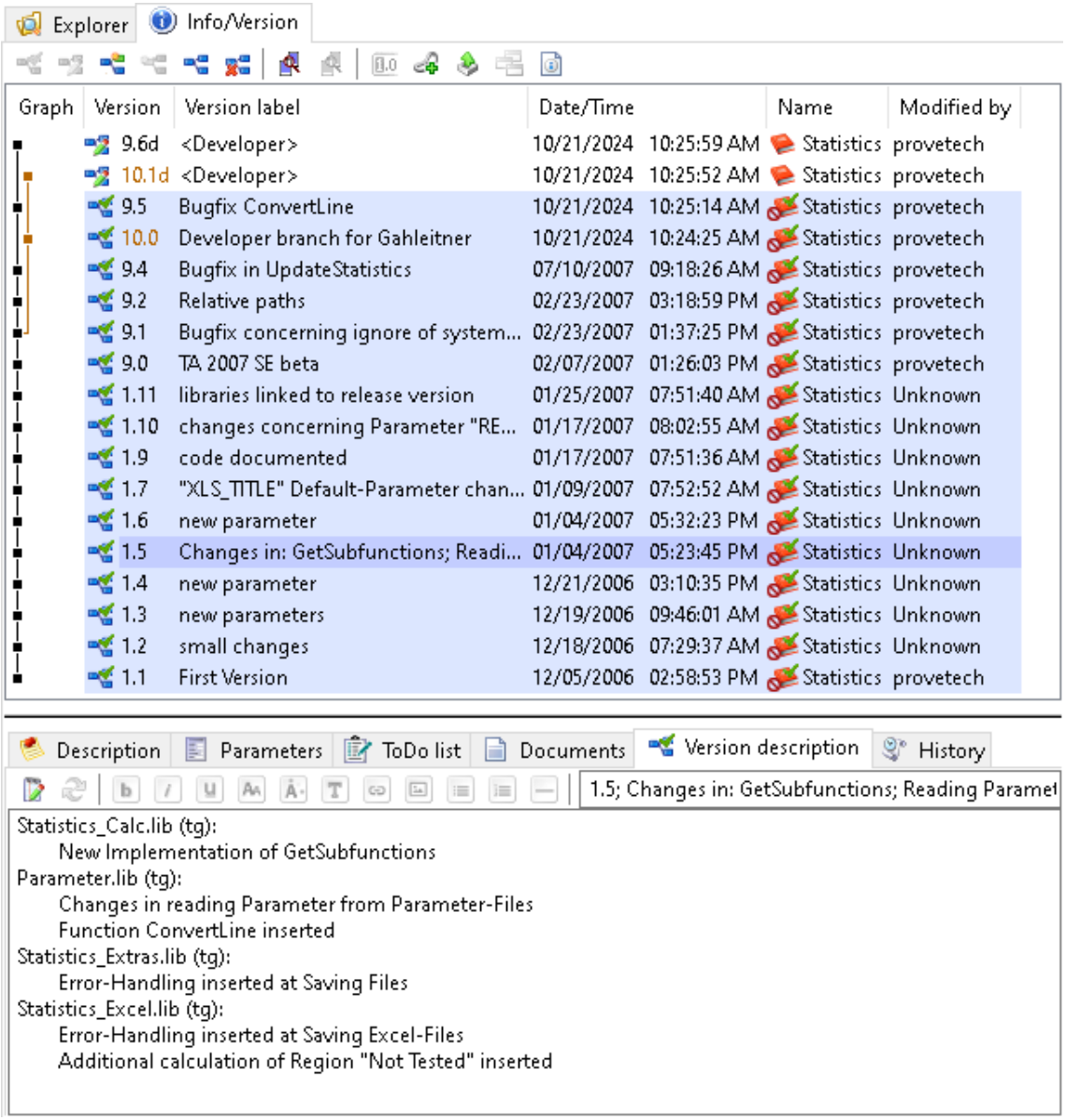


Figure 302 – The tab "Version description"

5.3.9.7 The tab "History"

The tab `History` is located inside the tab `Info/Version`. Some of the actions that modify the test object can be logged in the database for documentation and traceability (see also ***5.6.10 The history of a test result***).

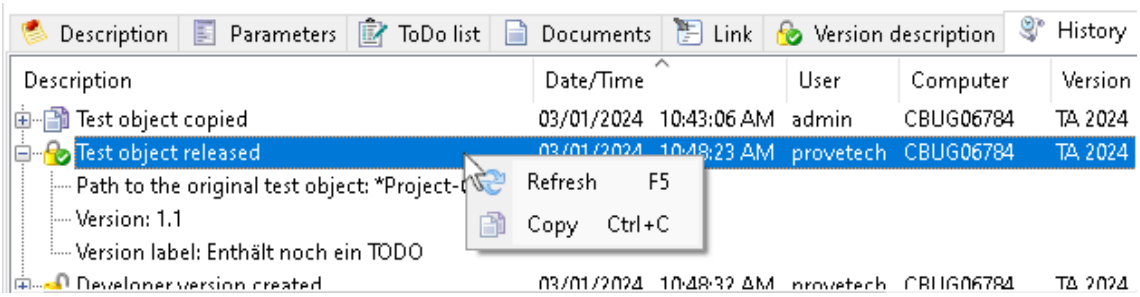


Figure 303 – History of the test object



To avoid overflowing, not all the actions can be logged that modify the test object (e.g., saving the Basic code, saving the description, modifying test documents, modifying test parameters).

Moving and deleting a test object is only logged at the parent test object if the object contained release versions.



If older PROVEtech:TA versions connect to the same Test Manager database, then it is possible that these versions produce no history entries yet.

The PROVEtech:TA administrator user can globally disable the logging. Therefore, open the dialog `Properties` of a test item and select the tab `Security`. There you can find the checkbox `Enable logging for history of test result` (see also chapter ***5.3.11.2 The tab "Database settings" in the property dialog***).

5.3.10 Exporting and importing tests

You can export and transfer any Test Manager subtrees in files to use them at other test systems.

Open the context menu of the test (click with the right mouse button). Click on *Export|To PROVEtech:TA Archive File…* in the context menu.

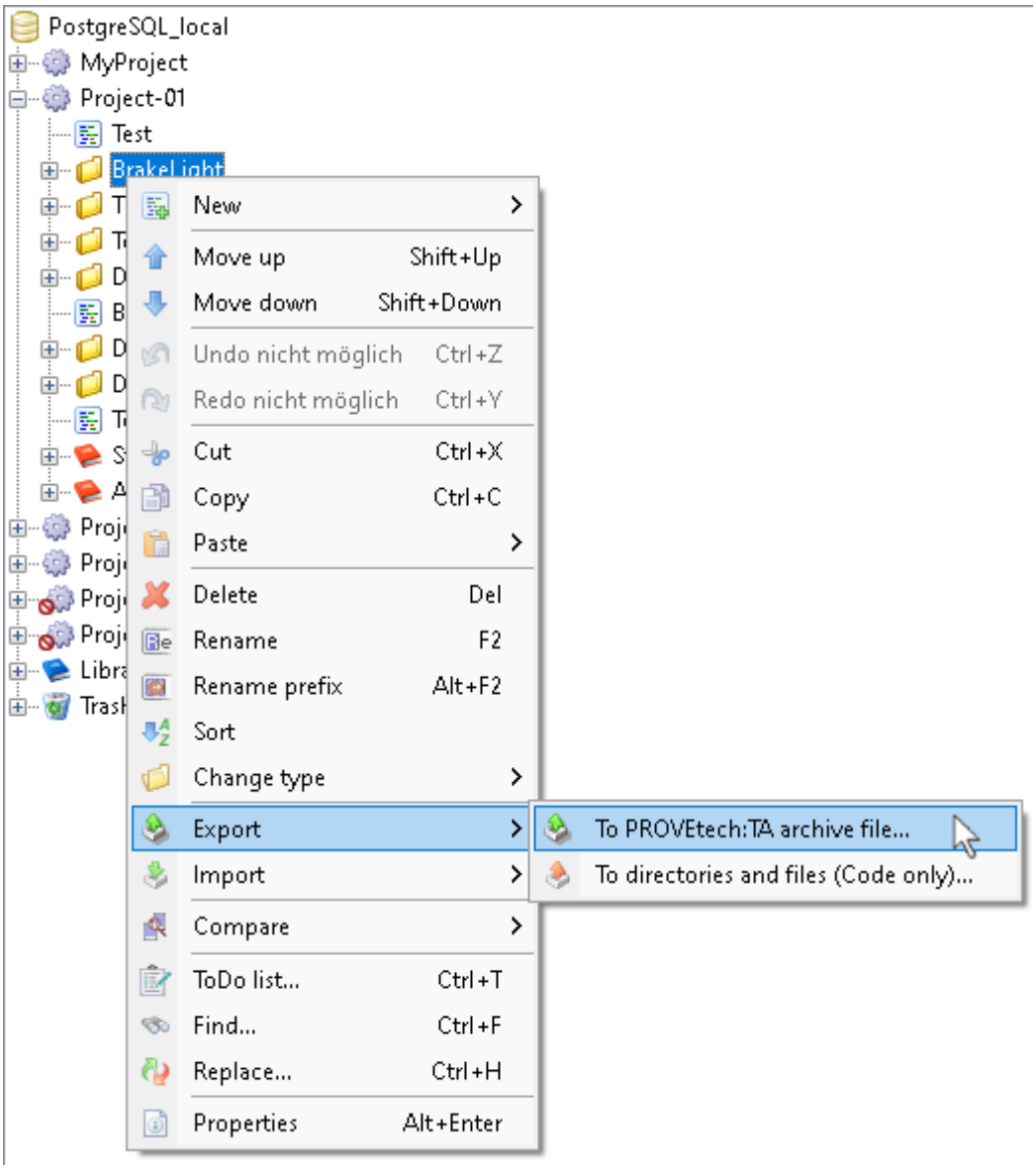


Figure 304 – Exporting a system test

A dialog opens in which you can determine the name for the export file.

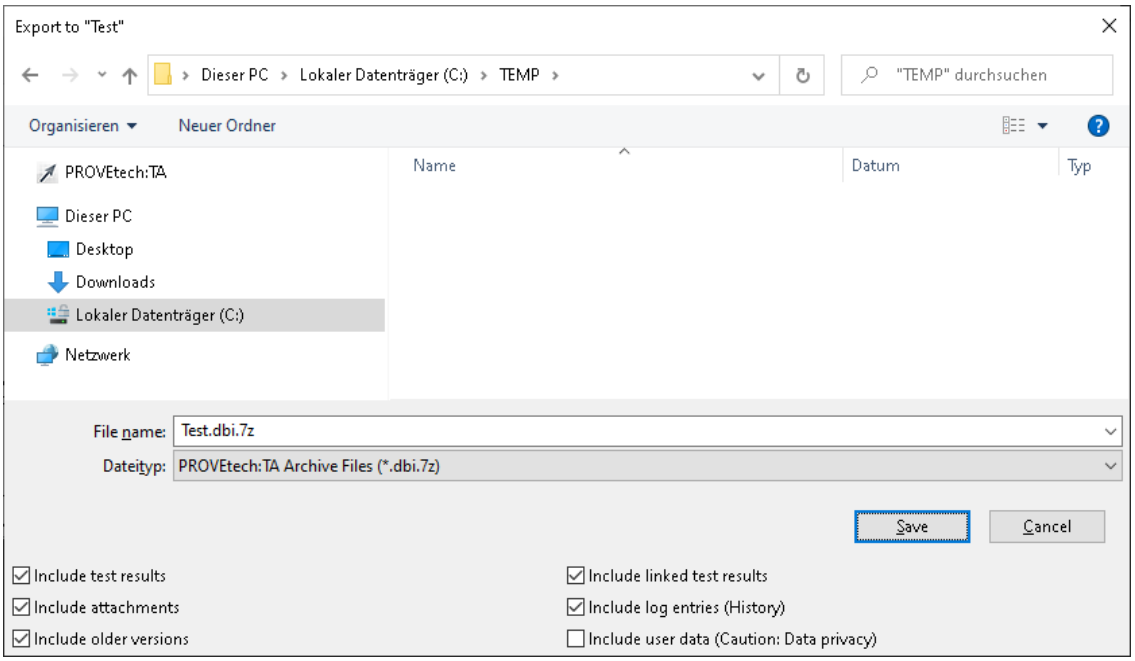


Figure 305 – Export dialog of a test

PROVEtech:TA creates a packed export file where database tables are saved in csv-format, program code and descriptions in txt-format, protocols in html and measures/attachments in compressed format. In user-defined test objects the file extension of the binary file is used in the packed file. To reduce the export and import times and to keep the export files small you can think about exporting without certain objects:

- Test results
- Attachments -- only available when Test results are selected
- Older test versions (and their test results)
- Linked test results (that means test results of linked test objects that are not part of the subtree) -- only available when Test results are selected
- Log entries of the history
- Include user data (Caution: Data privacy) -- This data is used to correctly assign the author to the test object and the executor and validator to the test result when importing it into another database later. See also chapter ***2.14 Processing and protection of personal data***.

Therefore, you uncheck the appropriate options.

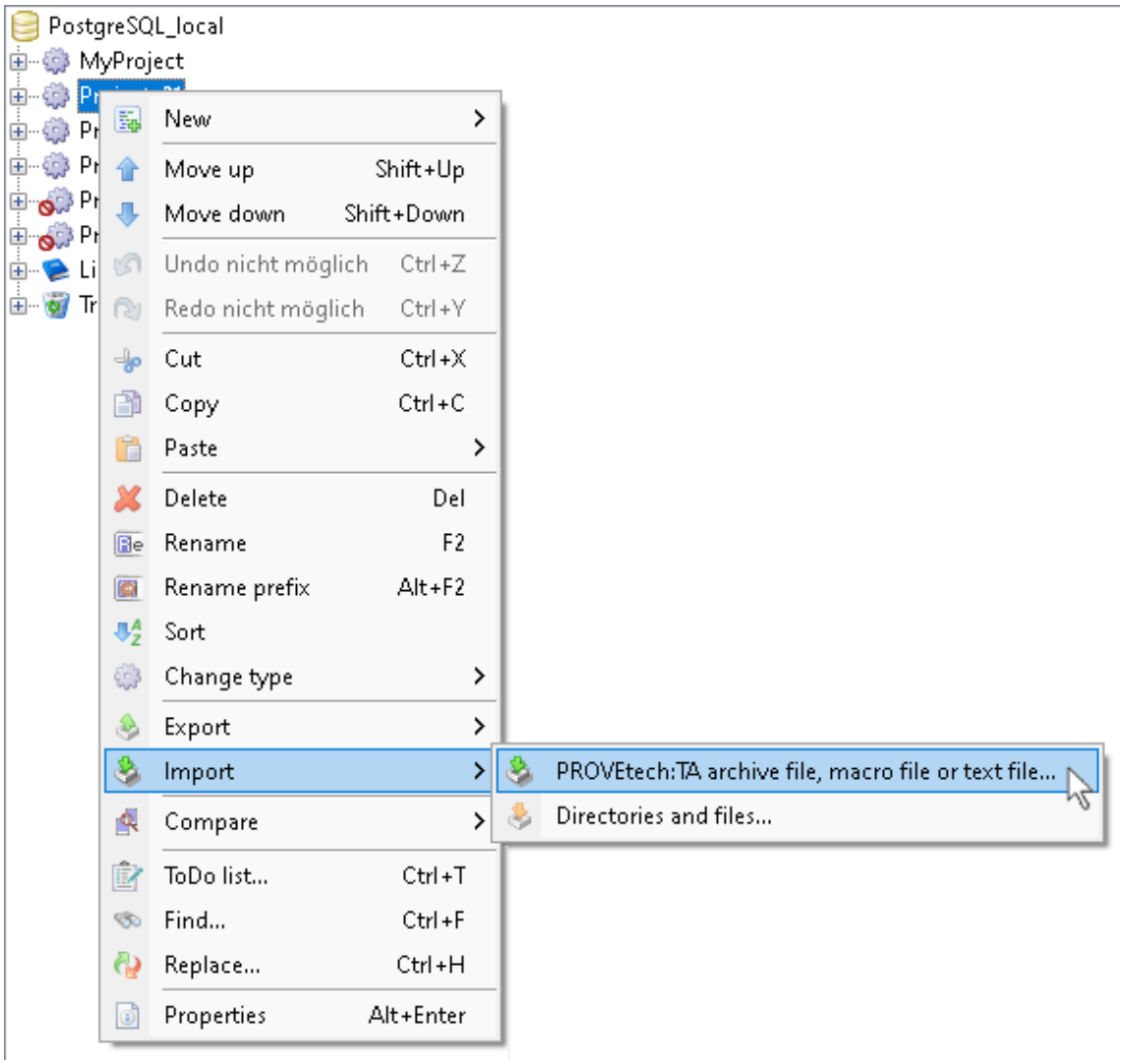


Figure 306 – Importing a test

To import the files into a system test, project or library, open the context menu of the object which you want to import the file (click with the right mouse button). In the Open dialog, choose the file containing the desired test data (see ***Figure 306***).

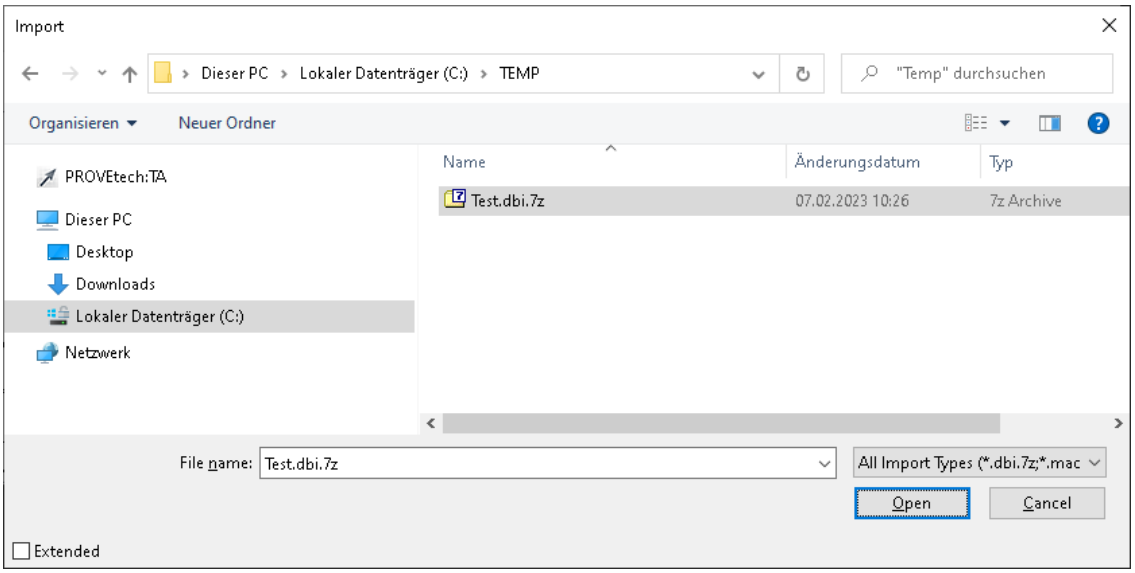


Figure 307 – File Open dialog at import

If you only want to import parts of the export file, then you check `Extended` and select the import file. Then a further import dialog appears (see ***Figure 308***).

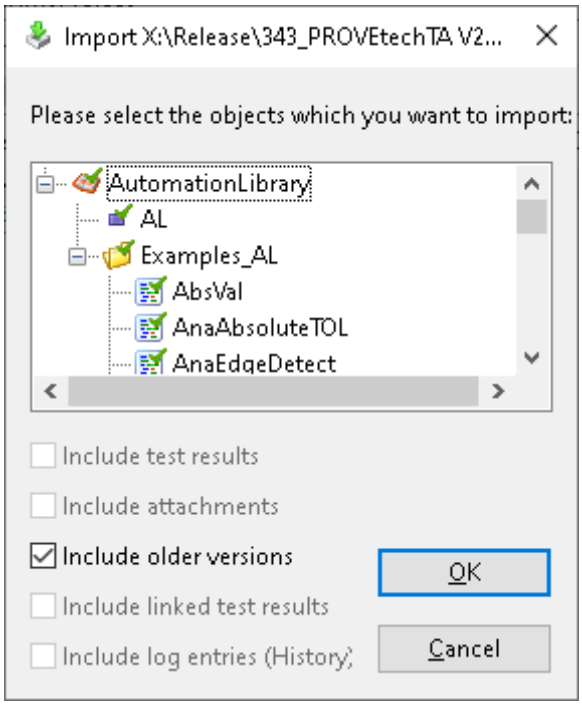


Figure 308 – Extended import dialog

In the extended import dialog, you can select the subtrees that you want to import. If you do not need the test results, attachments or older test versions, you can remove the check marks of the appropriate check boxes. This way you can reduce the import time and you save database storage space. The check boxes are shown grayed out if the archive file does not contain the according objects.



If you export tests with the current PROVEtech:TA version and import these tests with an older version, then it is possible that the import is not complete.



If you export projects or libraries, then you must consider the following points when importing these objects again:

- The access rights of the parent objects are taken over
- The parent test object must be the root, a project or a library. In addition, the *arManage* access right must be set for the parent test object. Otherwise, projects are converted to system tests and libraries are converted to user libraries (without access rights).

If user objects from older PROVEtech:TA versions were exported, they will be converted to library objects during import.



When exporting and importing Test Manager subtrees to PROVEtech:TA archive file (extension .dbi.7z) the files of linked attachments are not included in the export (see also ***5.6.9 Attachments***).



Export and import of older test versions

In the PROVEtech:TA database, the reference to the parent object, the reference to the neighbor object and the reference to the predecessor object are stored for each test object (or better to say, each test version). When a release version is created, the references are frozen for this respective test version. But the structures can change during further processing in the current test objects of this subtree.

If now, for example, an older version of a system test is exported, the frozen references (meaning, at the time the release was created) are decisive. Only these dependencies are exported. As a result, new changes in the subtree structure may not be reflected in the export archive of the older test version.



If test results of older test versions are to be exported or imported as well, both checkboxes `Include test results` and `Include older versions` must be set.



If a database inconsistency is displayed during export or import, please also refer to the AdministratorsGuide, chapter TM.ini in section `[ExportImport]`.



Regarding **data privacy**, please read in chapter ***2.14 Processing and protection of personal data***.

The menu command *Export|To Macro File (Code only)…* or *Export|To directories and files (Code only)…* in the context menu is used to map the test structure at the file level. This means that system tests become directories, elementary tests become individual files and user-defined test objects become binary files. The command *Import |Directories and Files…* can be used to read this structure back into the Test Manager.

Export and import of very large amounts of data

If you want to export very big subtrees or a large amount of data into a PROVEtech:TA archive (.dbi.7z file) then the working memory of the PROVEtech:TA process may exceed and PROVEtech:TA is terminating. Now you can try the following possibilities:

- dividing the export in several smaller subtrees
- using a database backup instead of an export file, if possible
- economical usage of the working memory (disabling unused modules, using an empty workpage, small model or avoiding to load the model,...)
- updating to PROVEtech:TA 64-bit to have no 4 gigabyte limitation (see chapter ***2.10 32-bit and 64-bit***)

Please also note that the data can be exported in blocks, as the export format is an image of the data in the database. In contrast, the import must be carried out individually for each data record, which takes much longer. Here, the new data must be merged with the existing data, for example by generating new database IDs.

Because the export of large parts of the tree and its files can be time consuming, the export process is non-blocking. This means that you can continue to work in PROVEtech:TA while an export is executed. However, you should avoid working with objects that are currently exported since this could corrupt the export file.

When exporting and importing, please also note the settings in TM.ini (see Administrator's Guide) in order to control the handling of error messages before the export/import operation is started. Otherwise, an error message may appear after a long period of time and the entire process may have to be repeated.

If the amount of data in the export archive is extremely large, it is possible that contained .csv files that are larger than decompressed 2 GByte may lead to an error message during import, even when using the 64-bit version of PROVEtech:TA. You sometimes only notice this some time later, possibly when the original data is no longer available. You should therefore consider whether there is a possibility of working with a database backup or restore instead.

Please also bear in mind that during the import, a large number of INSERT SQL queries are sent to the database. These are logged in the files of the transaction log, which causes these files to grow considerably, especially with large PROVEtech:TA archives. If there is no more memory available for the transaction logs, this can lead to the entire database being blocked for all connected PROVEtech:TA clients. Please also refer to chapter ***8.4.15 SQL Server error message about Transaction log***.

5.3.11 Displaying the properties of tests

Click the menu item *Properties…* to see additional information to the objects (see ***Figure 309***).

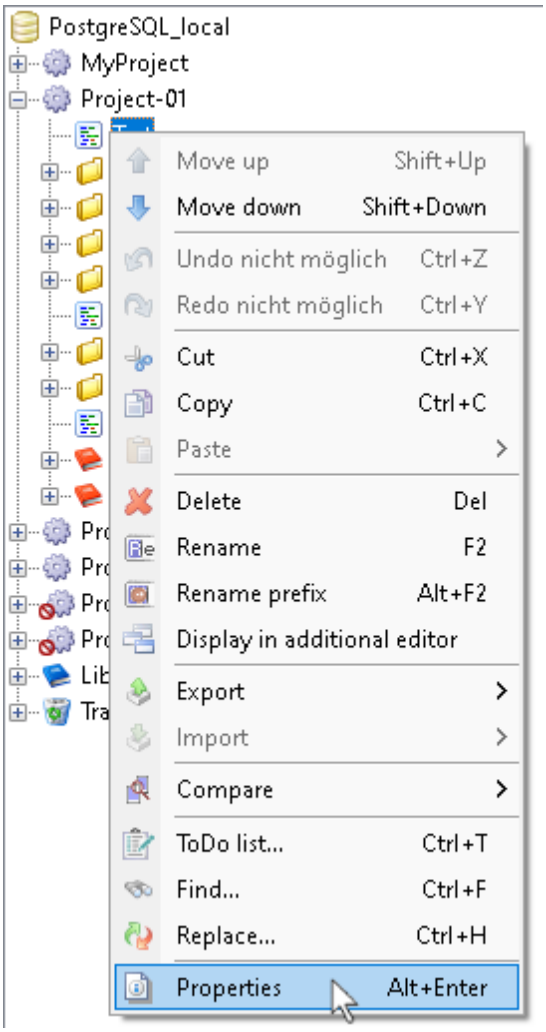


Figure 309 – Properties in the context menu

5.3.11.1 The tab “Info” in the property dialog

The property page `Info` displays general information about the test object. The `Database-ID` is only for PROVEtech:TA-internal usage.

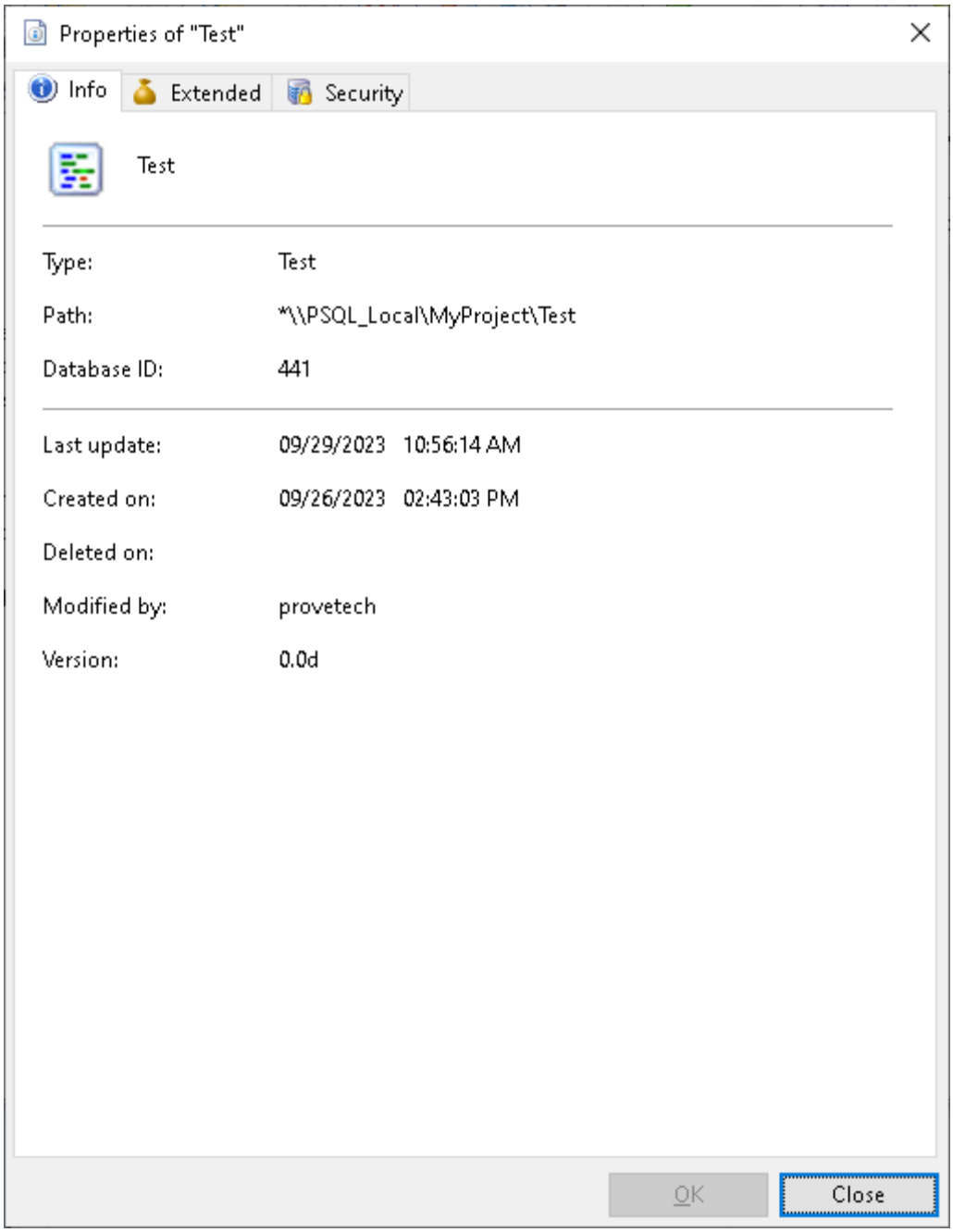


Figure 310 – The tab "General" in the property dialog

5.3.11.2 The tab "Database settings" in the property dialog

The `Database settings` tab is visible when the settings of the database object (root object in the Test Manager tree) are displayed with the database connected. These settings are stored in the database and affect all data managed in the selected database. The settings are visible to all PROVEtech:TA users. To change the settings, the *arManage* access right to the database object is required, or you must be logged in as

PROVEtech:TA administrator for certain settings. For more information see chapter ***3.2.7.6 Database***.

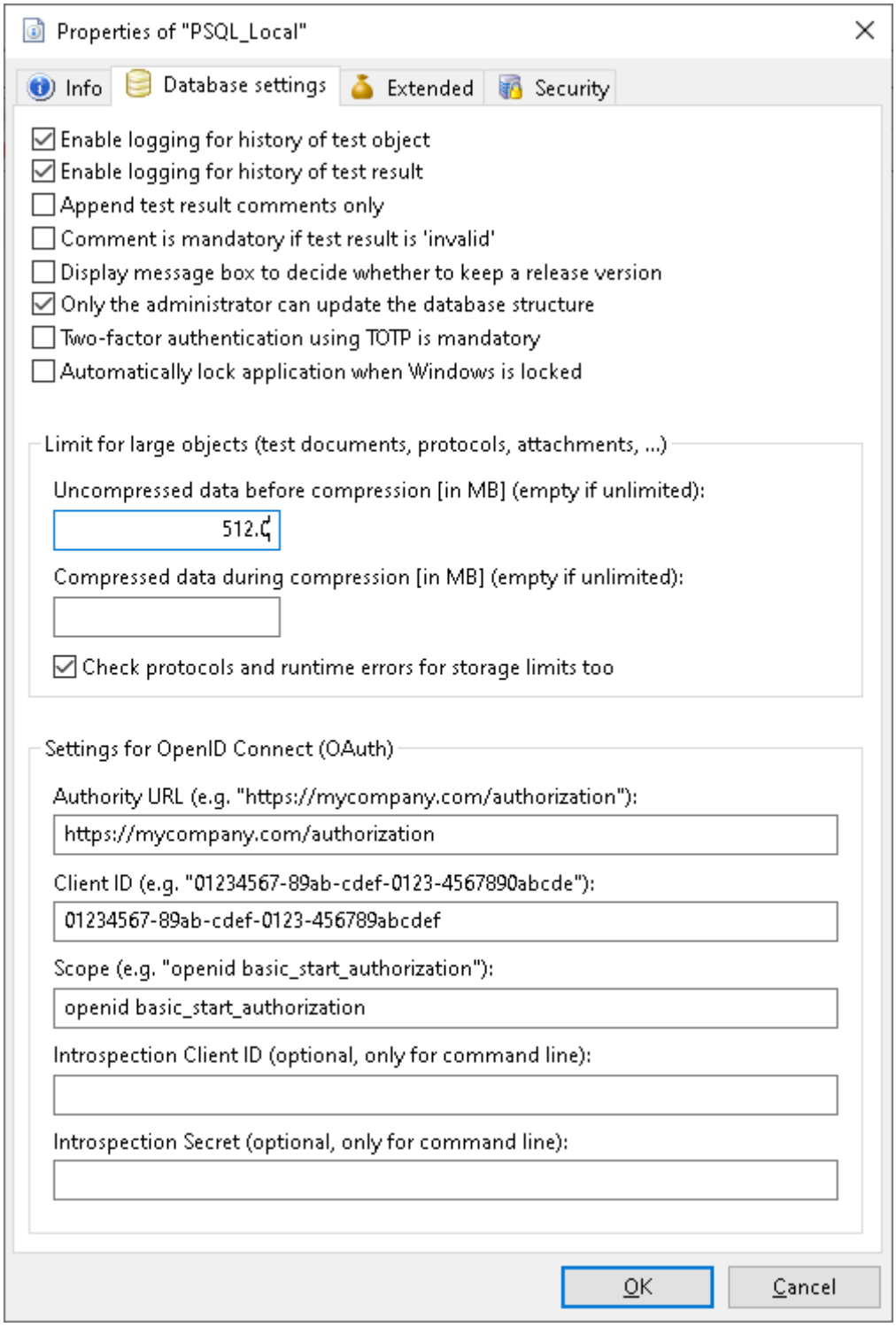


Figure 311 – The tab "Database settings" in the property dialog

The administrator can enable and disable logging by checking `Enable logging for history of test object` and `Enable logging for history of`

`test result` (see chapter ***5.3.9.7 The tab "History"*** and chapter ***5.6.10 The history of a test result***).

The administrator can control the behavior of the test result's comment by setting the checkboxes `Append test result comment only` and `Comment is mandatory if test result is 'invalid'`. You can find more about this in chapter ***5.6.2 The Comment*** and ***5.6.3.1 The test protocol of an elementary test***.

If the check box `Display message box to decide whether to keep a release` version is selected, you can control the behavior when changing and saving a release version. Either a new developer version is created, or the release version is kept and an entry is logged in the history (see chapter ***5.8 Versioning***).



Changing and keeping release versions is not recommended for process reasons regarding functional safety (keyword ISO26262).

The PROVEtech:TA administrator can set the checkbox `Only the administrator can update the database structure` to control when a database structure change should be performed with a new PROVEtech:TA version.

If the `Two-factor authentication using TOTP is mandatory` checkbox is set, each user must additionally set up the 2FA procedure using TOTP to log in to PROVEtech:TA. Please read also in chapter ***5.3.8.1 The user***.

With `Automatically lock application when Windows is locked` the PROVEtech:TA administrator (or a user with *arManage* access rights to the root object) can force an automatic locking of PROVEtech:TA to ensure that it's in fact the logged-in user who is working with PROVEtech:TA. Please also refer to chapter ***3.2.8 Lock User Interface***.



In certain cases, PROVEtech:TA is not automatically locked when Windows is locked. This can happen, for example, if a modal dialog (e.g. the settings dialog) or a message box is currently open in PROVEtech:TA.

In the group box `Limit for large objects`, you can specify a size limit for large database objects (test documents, test protocols, attachments, but also Basic code, description and comments). In this way, you can prevent accidental growth of memory in the database or problems occurring from very large data sets when using database replication.

Large binary data is often kept uncompressed in the working memory or on the local disk. The data is usually stored compressed in the database to save storage space and increase transfer speed. Here, the limit for uncompressed data can be checked before the storage and compression operation. The limit for compressed data, on the other hand, is checked continuously during the storage and compression operation. If the limit is exceeded, the operation is rolled back, and an error message is returned.

If you want to make sure that test protocols and runtime errors are written regardless of their data size, you can remove the checkmark for `Check protocols and runtime errors for storage limits too`. The default option is checked here and becomes active as soon as a limit is specified.

If authentication via OpenID Connect (also abbreviated as OIDC) is to be supported, as a PROVEtech:TA administrator (or as a user with *arManage* access rights to the root object), please consult your company IT department regarding the `Settings for OpenID Connect (OAuth)` and also refer to chapter ***2.3 Login*** and chapter ***5.3.8.1 The user***.



To set up authentication via OpenID Connect (OAuth), your company IT department must register the PROVEtech:TA application for OIDC. You often must request the “OIDC Service Configuration” and/or “ALICE” services for this. It is important that your IT enters the following as “Redirect URL”:

```
http://127.0.0.1:*/*
```

All in all, you need the settings for the OIDC Authority URL, the OIDC Client ID and the OIDC Scope from your company IT department, which you as PROVEtech:TA administrator can enter here in the settings (see figure above) and which are then saved in the database.

If the login is to take place automatically via the command line, you also need the OIDC Introspection Client ID, the OIDC Introspection Secret and the OIDC token from your company IT, which must be specified as a command line parameter (see chapter ***2.7 Command line parameters***).

5.3.11.3 The tab “Extended” in the property dialog

The tab `Extended` in the property dialog displays information about number and size of objects. In the upper group box only the selected test objects and their test results are considered. In the lower group box, the complete subtrees are regarded, inclusive older test versions and test result children.

Sizes refer to the compressed data in the database. So you can use these numbers to analyze the database storage size (see also in chapter ***8.4.10 How can I determine the actual storage space of my database?***).

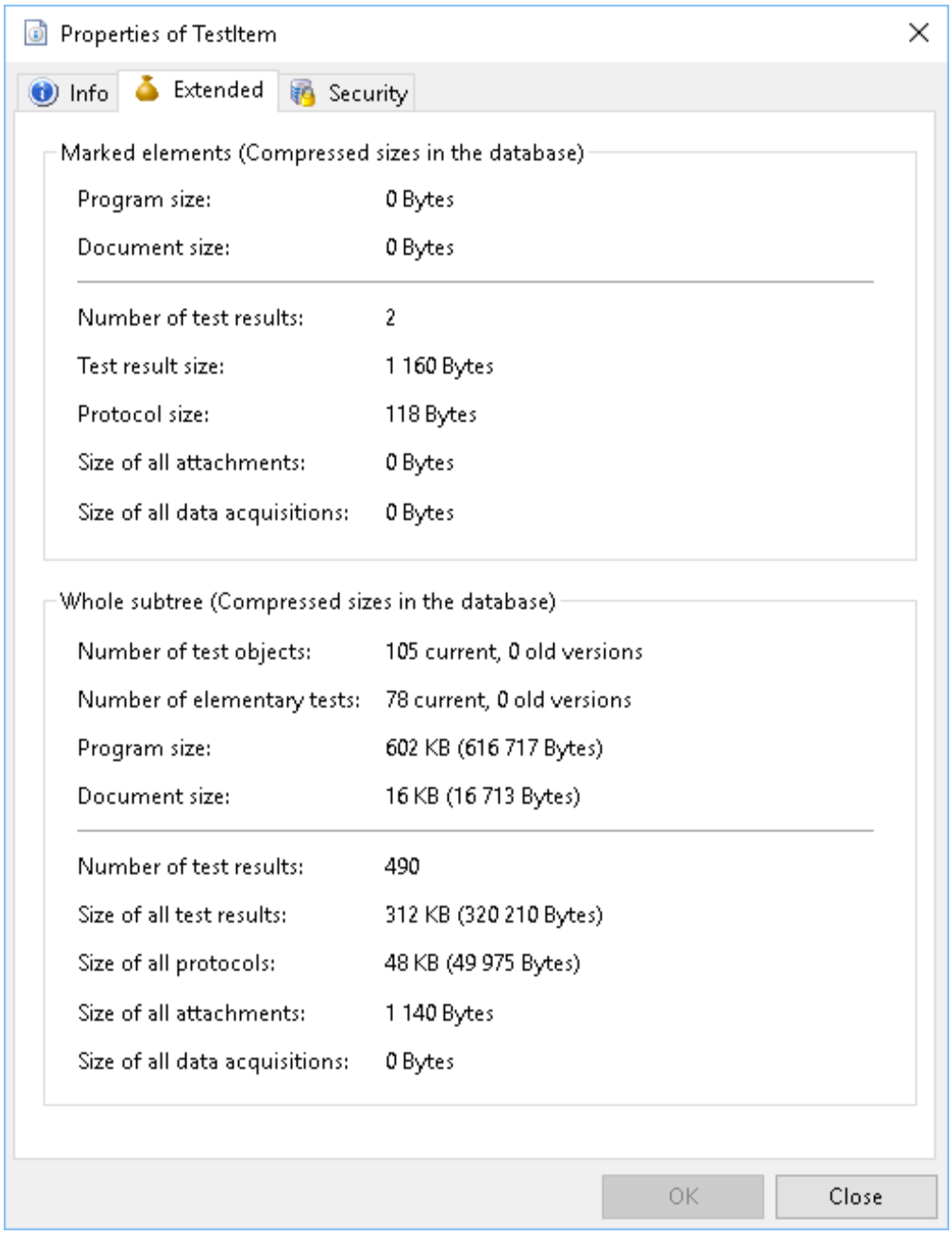


Figure 312 – The tab "Extended" in the property dialog

Please consider the following issues:

- Basic code, descriptions and comments are stored in the database uncompressed as plain text. But test documents, test result protocols, attachments and data acquisitions are compressed in the database.
- The size of the protocols is the sum of all sub protocol sizes plus the size of the main protocol.
- Data acquisitions are special attachments. So, they are a subset of the attachments.
- The size of a test results is the sum of all protocols and attachments, the comment of the result and meta data. Additionally, information of the runtime errors, ECU errors, keyword values and log entries of the history is taken into account.
- You often get more significant information by using the Test Language command ***TestItem.GetStorageSpace***. This avoids running recursively through the whole subtree and through version chains. The command *TestItem.GetStorageSpace* calculates the result directly inside the database.

5.3.11.4 The tab "Security" in the property dialog

In the tab `Security` you can see the access rights of the currently logged-on user (in this case the user “provetech”) to the project or library in which this test object is located (here it is the project “MyProject”, see ***Figure 313***).

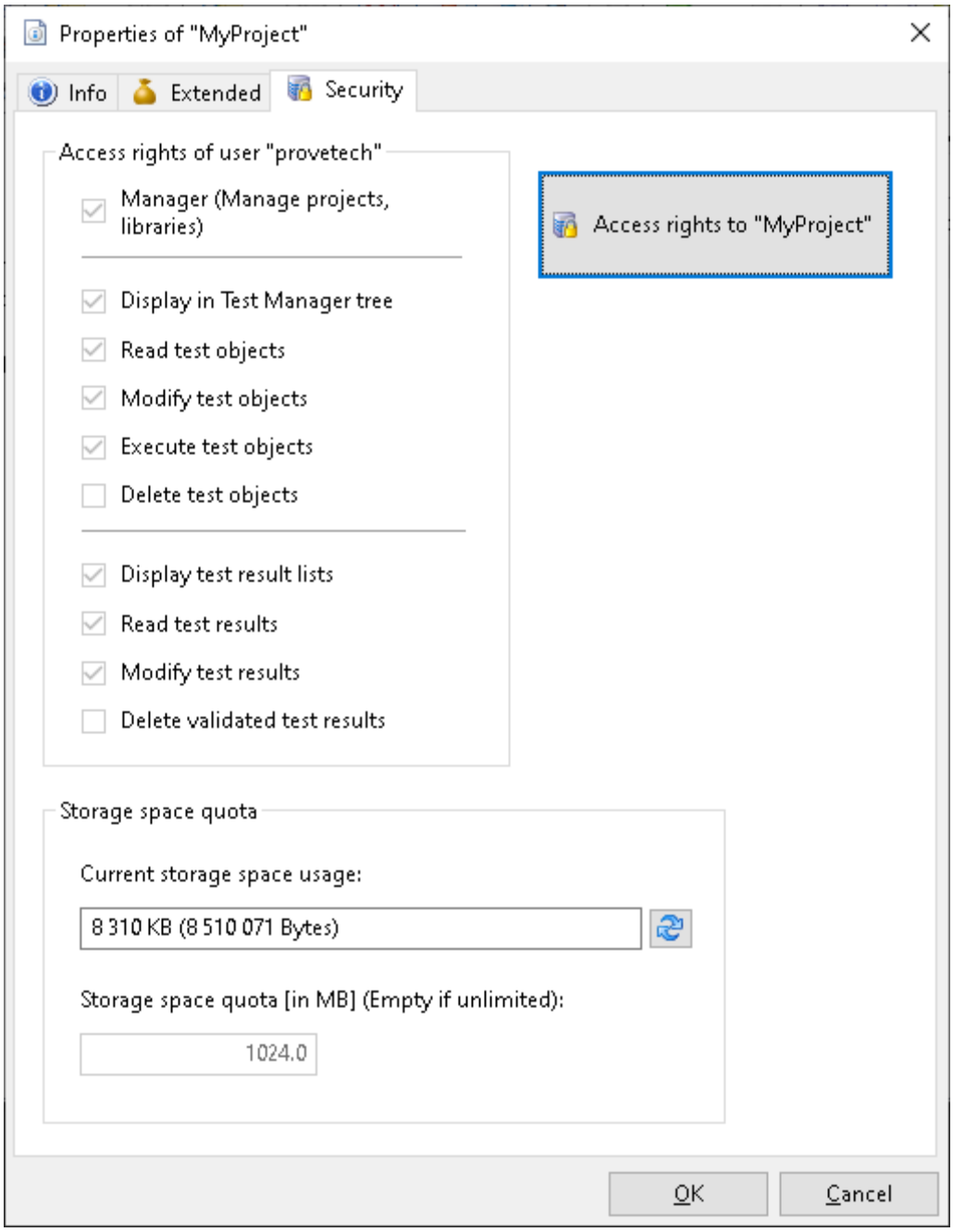


Figure 313 – The tab "Security" in the property dialog

User access rights on the selected test object can be set for different users via the button `Access rights to "…"` (see ***5.2.6 Access rights***). This button is only visible if the selected test object is a project, a library (but no user library) or the root object. Otherwise, the selected test objects take the access rights of its parent object. If there are no *arManage* access rights, then the buttons are disabled.

The field `Current storage quota` displays the current database storage usage for the project or library in which this test object is located. This value can be set with

administrator rights. Further information on this subject can be found in chapter ***5.10 The Quota mechanism***.

5.3.12 Deleting tests

If you wish to delete test objects, select the menu item *Delete* in the context menu. The test will disappear from its current location and move into the trash folder. You need write access on the test object to perform this operation.

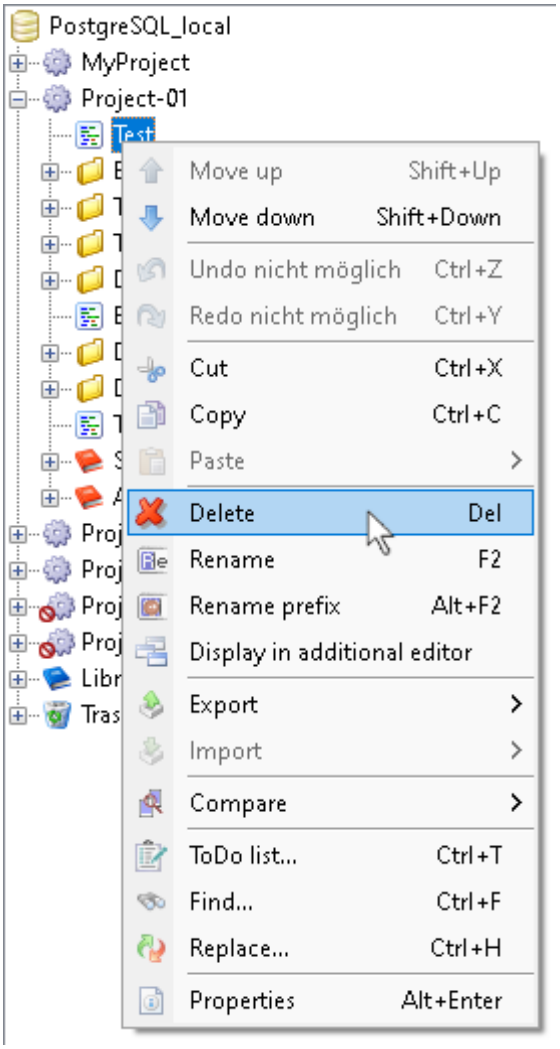


Figure 314 – Deleting a test object

You can also delete test objects using the keyboard. Select the test object and press the 'Del' key.

Deleted tests are moved into the Trash Folder. So, they are not immediately discarded from the database (see ***5.2.4 The trash folder***). The Trash Folder works just as it does

under Windows. Click on it to display the contents of your private Trash Folder. There are several possible ways of reactivating deleted tests:

- **Moving the test:** Click with the right mouse button on the deleted test and select *Move*. Click with the right mouse button on the system test or library into which you want to move the test and select *Move here* in the menu. The test will be removed from the trash folder and made available again.
- **Copying the test**: Click with the right mouse button on the deleted test and select *Copy*. Click with the right mouse button on the system test or library into which you want to insert the test and select *Copy here* in the menu. A copy of the test will be inserted and made available again.
- **Drag and drop**: Click with the left mouse button on the test in the Trash Folder. Keep the button depressed and drag the test to the position where you want to drop it in the Test Manager tree. The test will be moved from the trash folder and will be available again.
- **Undo**: If the delete action has just happened, then click with the right mouse button in the Test Manager tree and choose *Undo*. The last action is thereby undone.

The hierarchy of the elementary tests and system tests is kept in the Trash Folder. If you retrieve a system test from the Trash Folder, all the child elementary tests will also be restored in their original sequence.

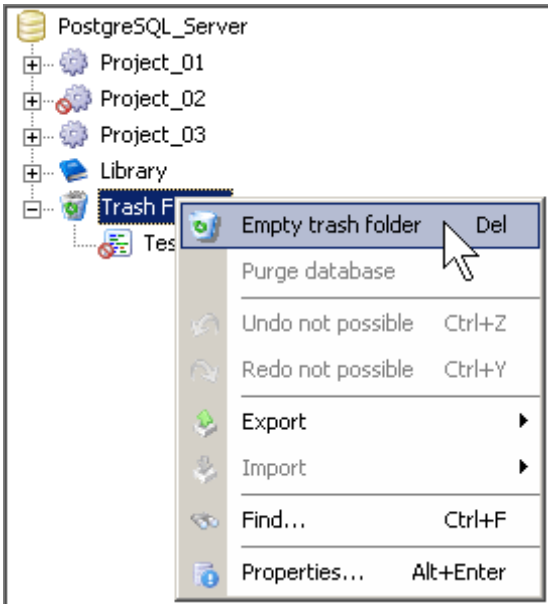


Figure 315 – Emptying the trash folder

You delete the tests permanently by emptying the Trash Folder. To do this, click with the right mouse button on the Trash Folder symbol and select `Empty trash folder`. Your entire private content of the Trash Folder will then be removed from the database.

You can remove individual tests from the Trash Folder by calling the appropriate Delete function from its context menu.



The Trash Folder of the Test Manager is private, that means, only the test objects are displayed, which were deleted by the current user. Only the PROVEtech:TA Administrator can see the whole content of the Trash Folder. So, the Administrator can find test objects even a longer time after they were accidentally deleted.



If you delete a link from the trash folder, then you only remove the tree entry from the Test Manager tree. The associated test object is kept as long as further links refer to this object (including test objects from the Trash Folder). See also ***5.3.14 Linked test objects***.

5.3.13 Comparing and merging test code

You can compare and merge the source code of two tests with the help of an external program like the tool “WinMerge.exe”. By default, the PROVEtech:TA bin directory is searched for WinMerge.exe, which is freely available at (https://winmerge.org), but is not automatically installed for legal reasons.

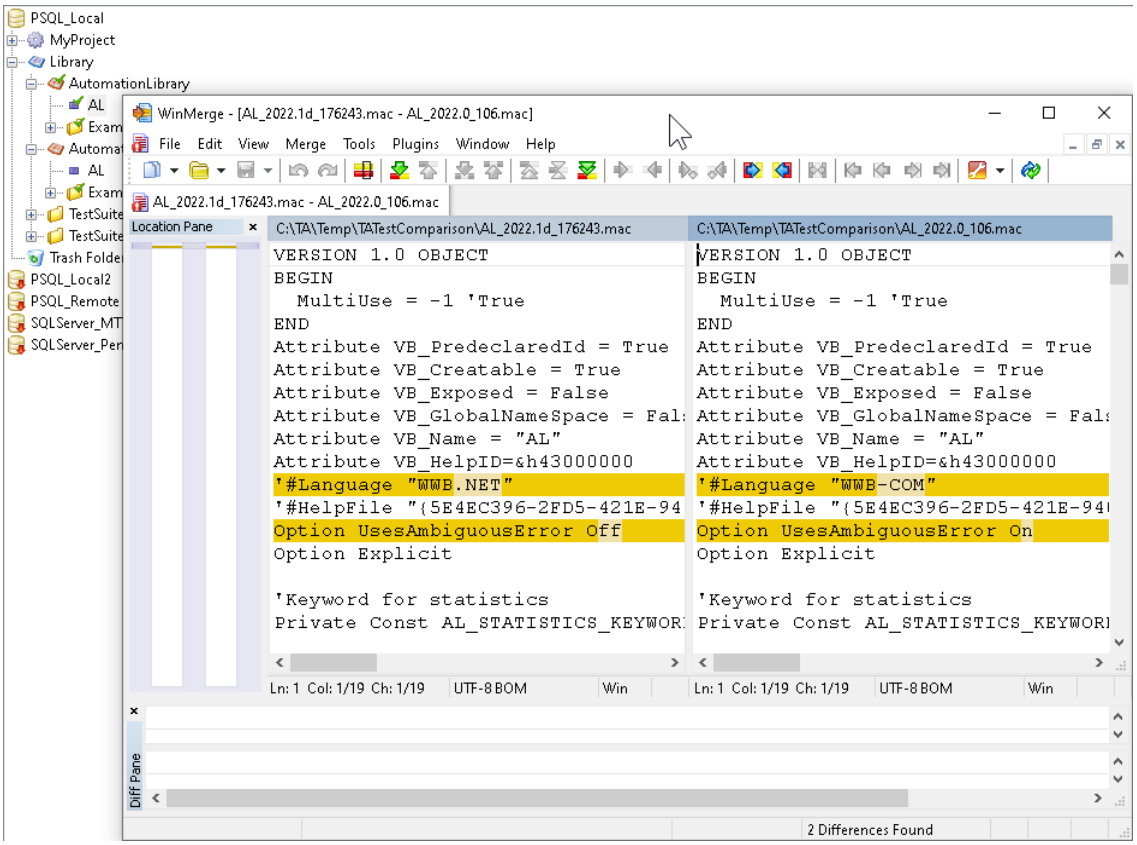


Figure 316 – Code comparison of two tests using WinMerge.exe

You can also compare the structure of two system tests or libraries.

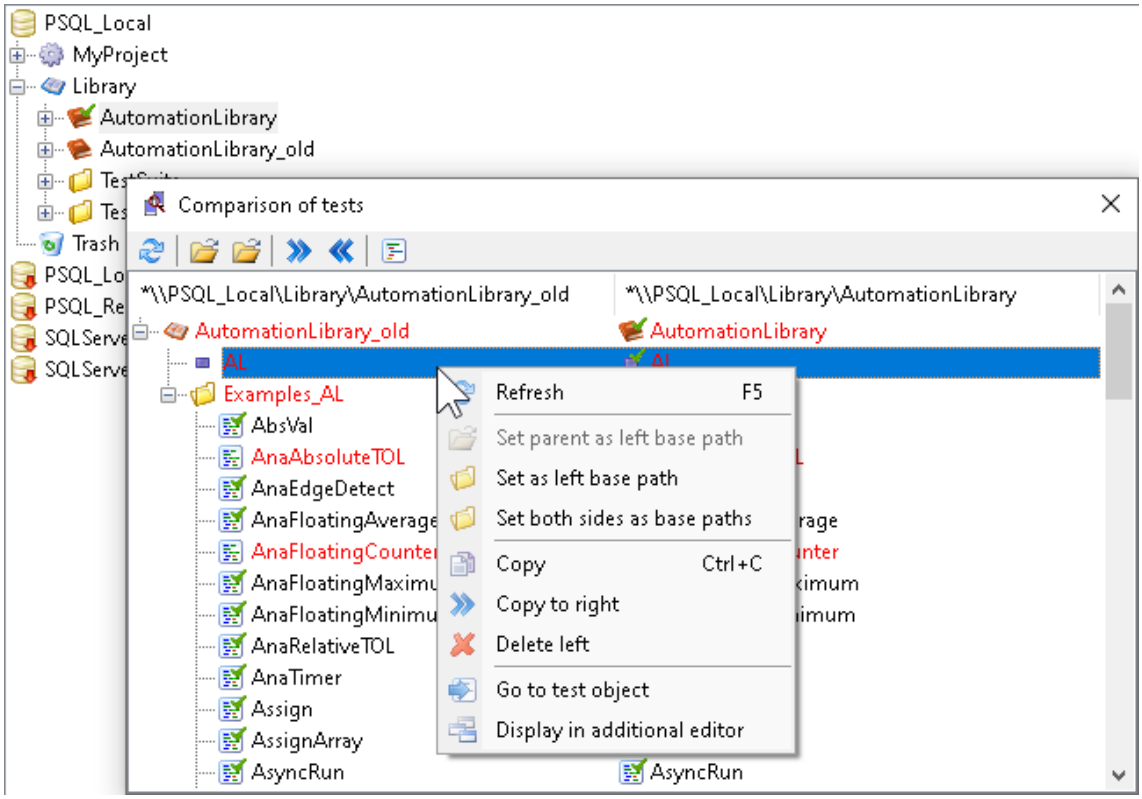


Figure 317 – Comparison of two libraries

Two test objects are compared (that means, they match and are contrasted in the dialog) if the name and object type are identical. If the test objects differ in other attributes such as the WinWrap Basic code, these are marked with red text color.

Two test objects are identical if the name, the object type (elementary test, object, class, system test, ...), the description and test specification number, for user-defined test objects the type name, the program code, the test parameters and the test documents are identical. For system tests and user libraries, the child objects must also be identical and in the correct order.

The toolbar and the context menu (right-click on the entry) contain the following functions:

	The function `Refresh` rebuilds the tree and compares all entries from new.
	The function `Set parent as left base path` is processed for the selected column.
	The function `Set as left/right base path` is enabled for the root item of the left/right subtree and processed for the selected column.
	The function `Set both sides as base paths` sets the selection as subtree roots for both sides and refreshes all entries.
	The `Copy` function copies the left/right selected test object or its Test Manager path to the clipboard.
	The function `Copy to right` copies the test object on the left side to the location of the right subtree, including all attributes. If necessary, the old test object is overwritten. (Note: Not to be confused with copying to the clipboard, Ctrl+C)
	The function `Copy to left` copies the test object of the right side to the location of the left subtree, including all attributes. If necessary, the old test object is overwritten. (Note: Not to be confused with copying to the clipboard, Ctrl+C)
	The function `Delete` deletes the left/right test object at the chosen side.
	The function `Go to test object` selects the chosen left/right test object in the Test Manager tree.
	The function `Display in additional editor` opens the WinWrap Basic code of the left/right test object in an additional window.
	The `Only compare Basic code` function controls whether only the Basic code or also other attributes such as description, parameters and test documents are to be taken into account when comparing two objects.

When you move your mouse pointer over one of the lines in the comparison tree then you can see the difference via tooltip.

With a double click on an elementary test you can get the code comparison again.

The modifications in the Basic code saved in the merge tool are automatically saved in the Test Manager database.

You can also choose your own compare tool or merge tool in *Application|Settings…* (see chapter ***3.2.7.5 Test Manager*** in the Settings dialog). The file paths to the temporarily created files are added automatically as the two last options in the command line.

The modifications of read-only test objects (e.g. for older test versions) will not be saved in the Test Manager database and will display an error in the PROVEtech:TA Log.

You can choose the two objects that should be compared via the context menu of the Test Manager tree or the short keys Ctrl+Shift+L and Ctrl+Shift+R.

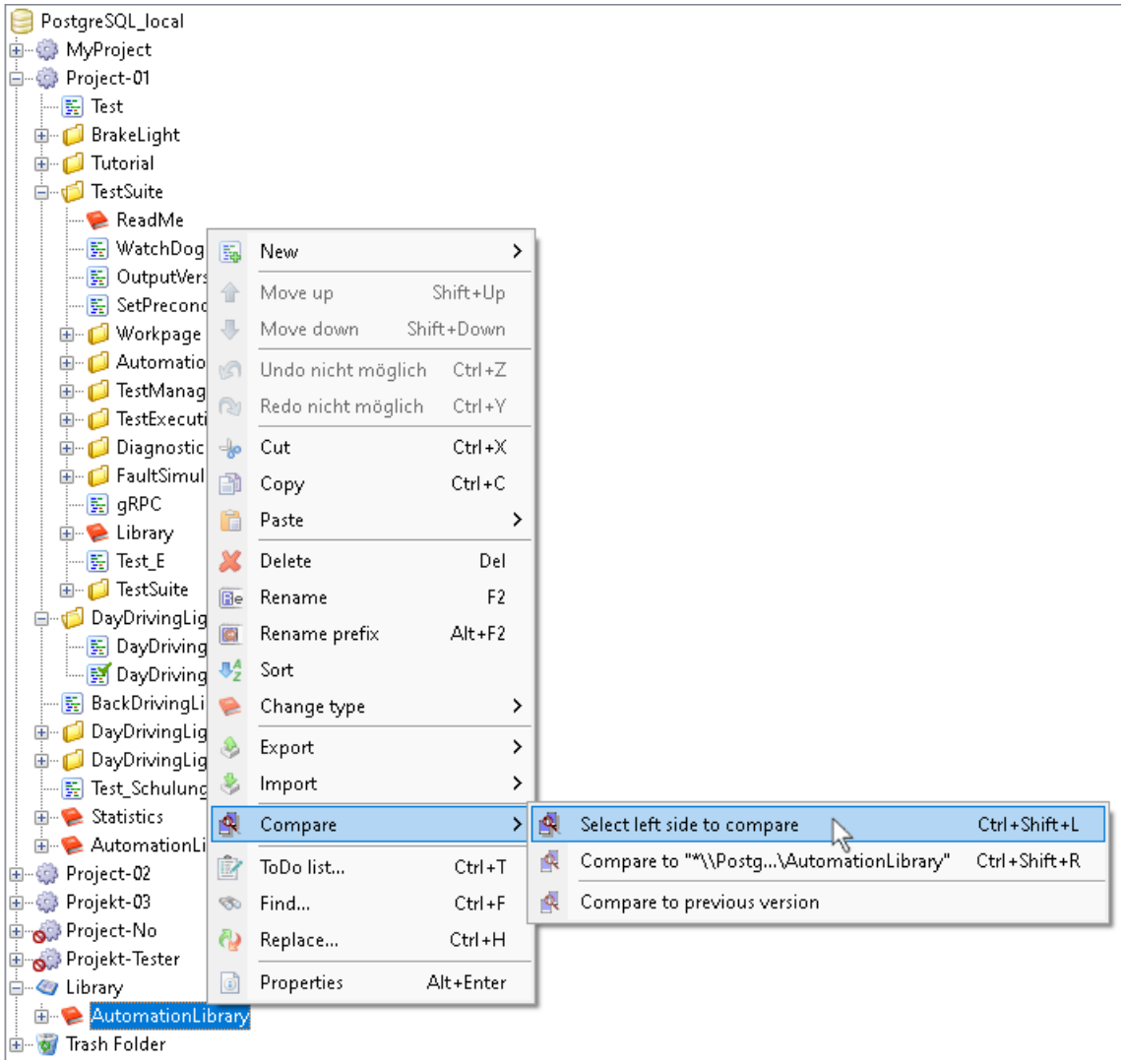


Figure 318 – First test for code comparison

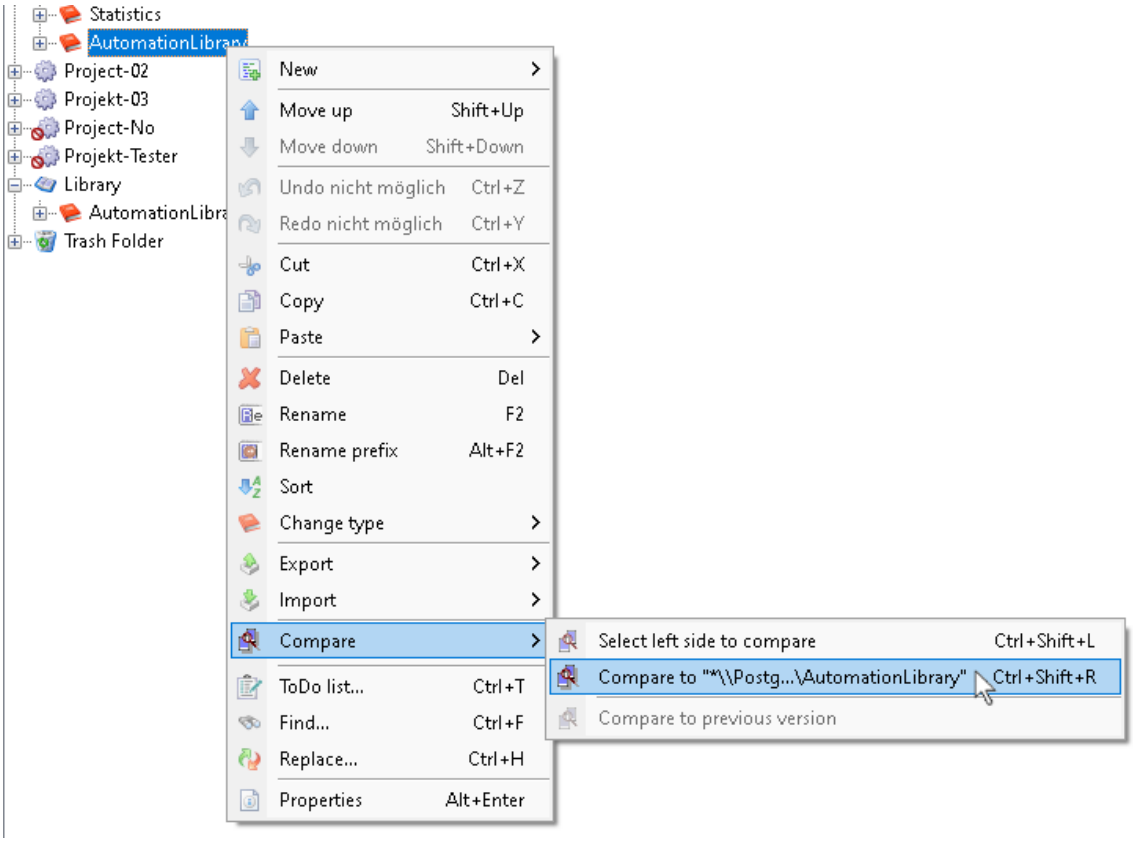


Figure 319 – Second test for code comparison

You can also compare older object versions. For this you change to the tab `Versions` and select the version that should be compared.

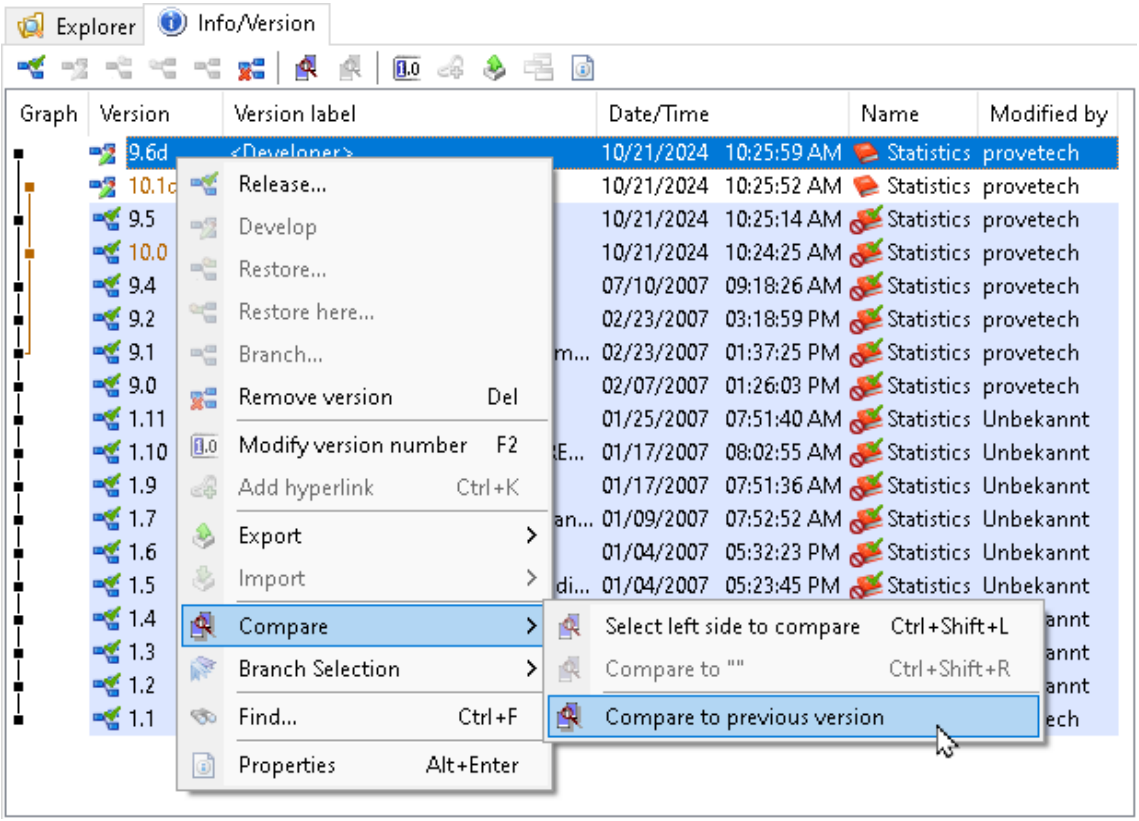


Figure 320 – Code comparison with older test versions

5.3.14 Linked test objects

A Link is a test object that is referenced from different entries of the Test Manager tree. These tree entries are marked with a black arrow in the icon , , , and . After you have copied an elementary test into the clipboard, you can create links as follows:

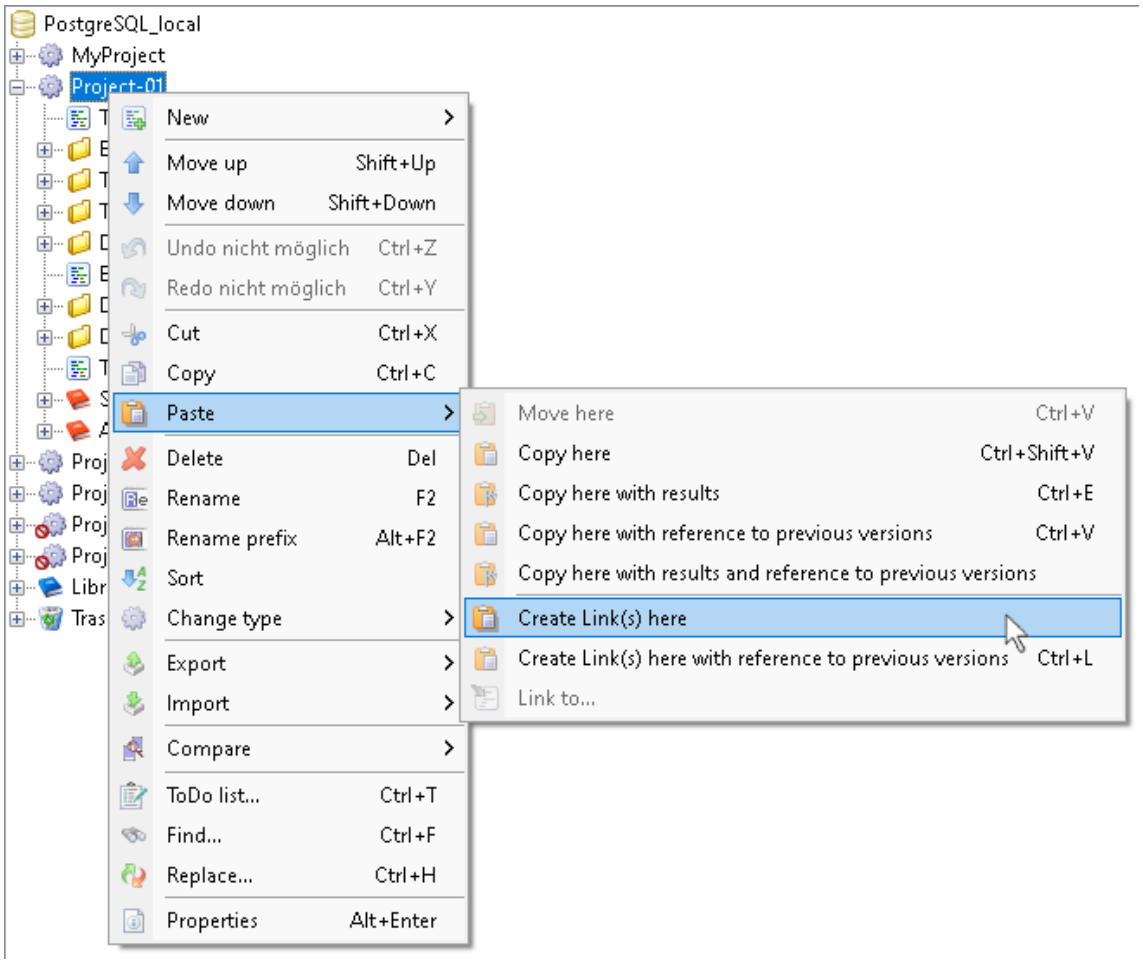


Figure 321 – Creating a link

You can only parameterize a link by using the test parameters (see ***5.3.9.2 The tab "Parameters"***). In opposition to all other attributes, the test parameters of matching links can be different.

Links to tests only

You can only create a link to an elementary test, a class, an object, a text object or a used-defined test object but not to a system test or a user library. The created test object and the original object are equal links to an object then.

Displaying linked test objects

You can display the path names of all matching links to an object in the tab `Link` of a test object (see ***5.3.9.5 The tab "Link"***).

Copying links

When you copy a link, the copy becomes an independent test object that is no longer linked with the original. The copy of a link also forms an independent test object and no longer depends on the source.

If you copy a Test Manager subtree that contains links, you will be asked to choose whether to create links in the target subtree for each link, or to create real copies without links. Alternatively, you can also export and re-import the subtree if links are only to be retained within the target subtree.



Security hint:

If you create links between different projects or libraries, then you must be careful of the access rights. For example, if you want to modify the content of a link it is sufficient to have the corresponding user rights only for the selected test link.

Link existing tests

You can link existing test objects afterwards, e.g. to keep their already generated test results. Thereby, one of the test objects loses its content (except for the test parameters). After you have copied an elementary test into the clipboard, you can link existing test objects as follows:

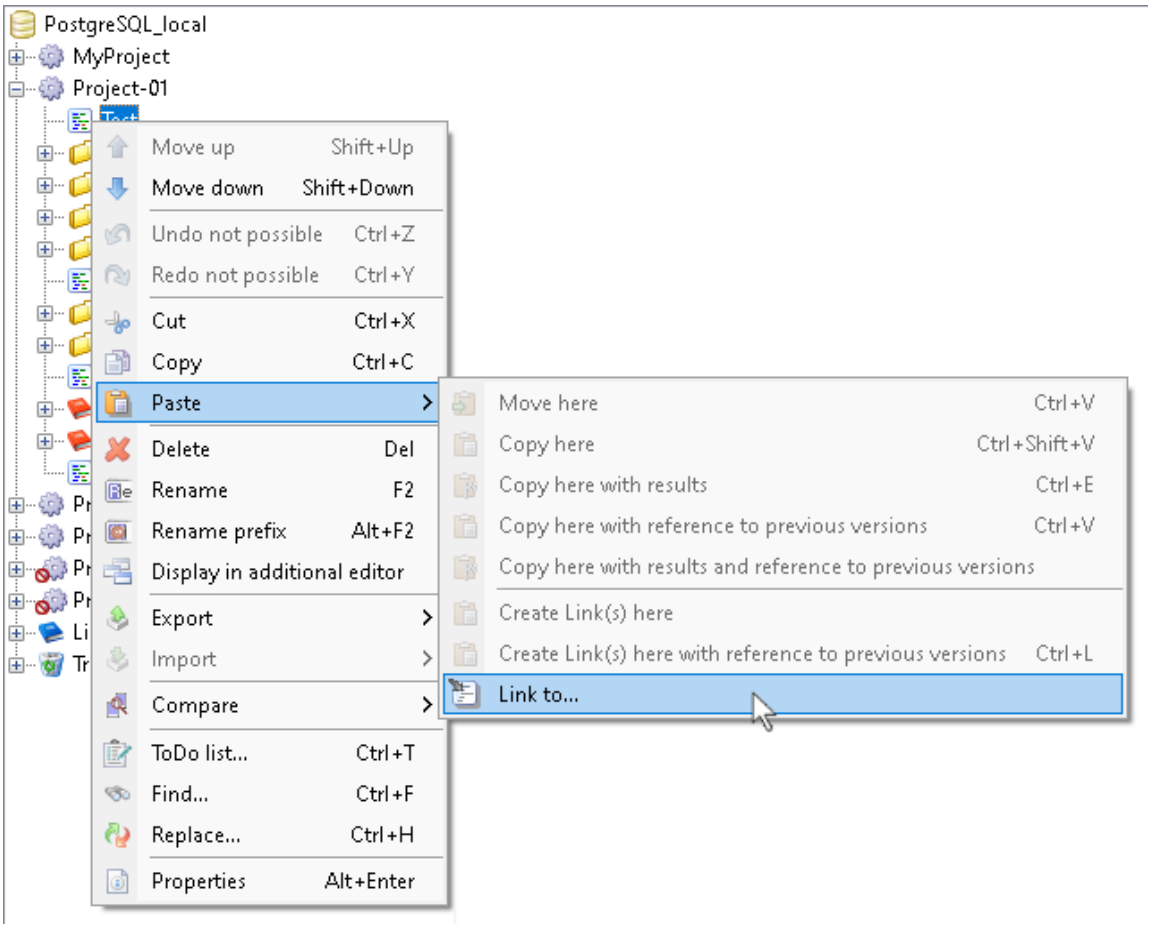


Figure 322 – Link existing tests

Deleting links

When you delete a link, the associated object is only deleted when no further link to the object exists, even in the Trash Folder.

Breaking links

Via context menu you can break links. Then you have got two different test objects with the same content.

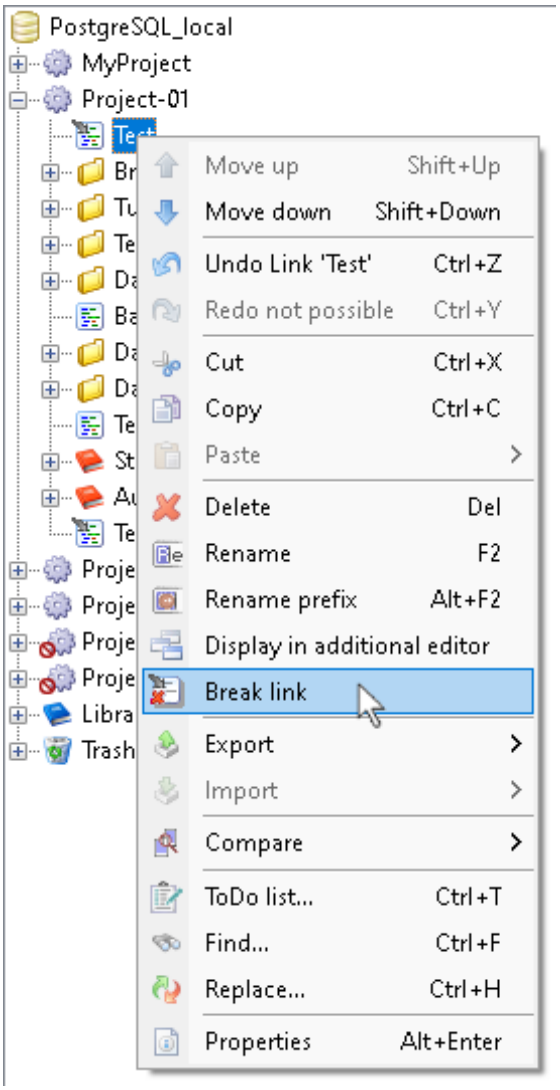


Figure 323 – Breaking links

5.4 The Basic editor

To program an elementary test, click on the `Program` tab. The editor shown in ***Figure 324*** is available for programming tests.

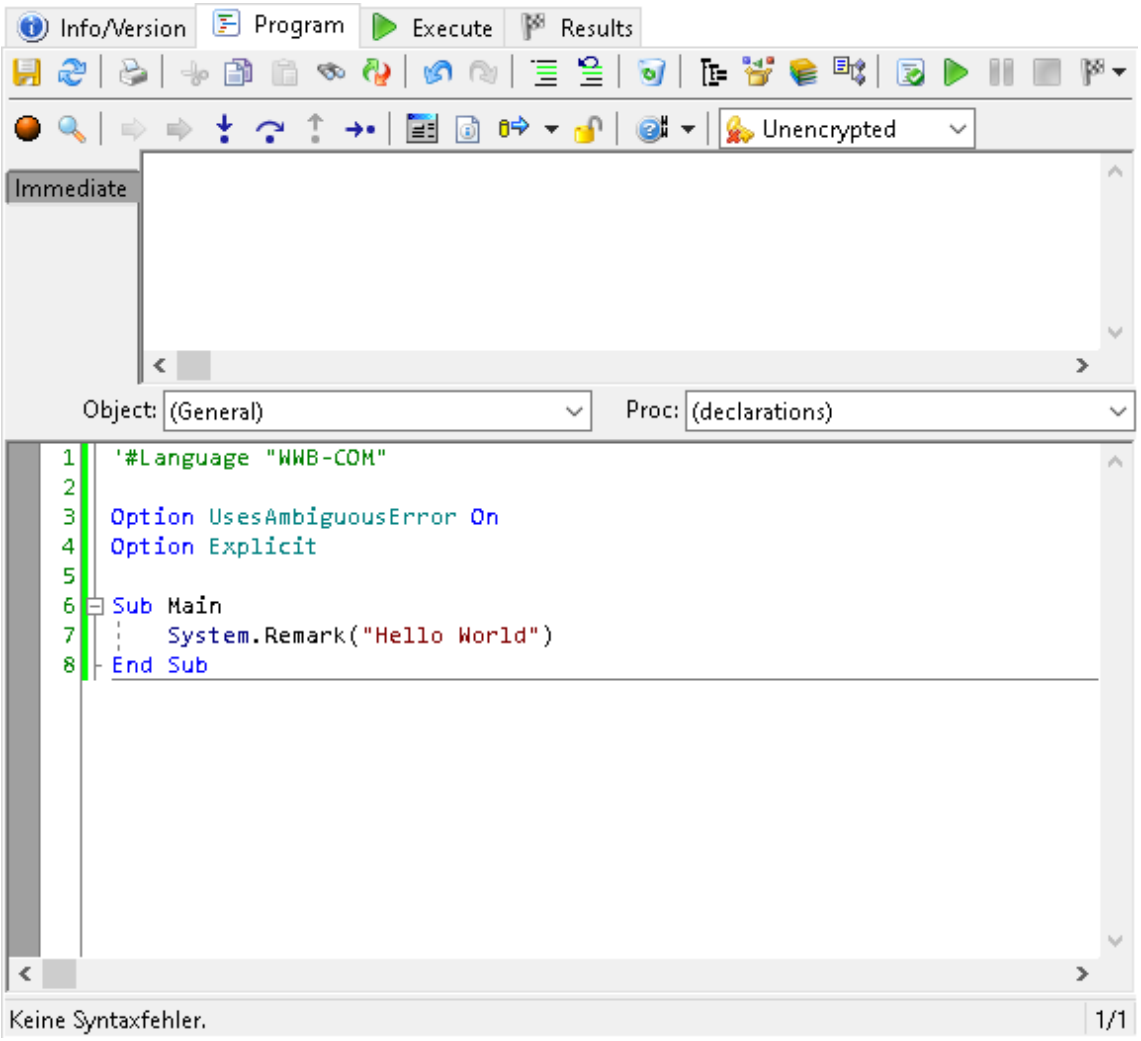


Figure 324 – The test editor

The editor is divided into 5 areas: the upper toolbar, the output window (Immediate, Stack, Watch) below it, the object and procedure selection in the middle, the edit window below that and the lower status bar.



Executing tests directly in the Basic editor is for debugging purposes. You can single step your test program and watch your variable values. The execution in the editor creates a temporary test result in the database which is available during the debug session.

5.4.1 The toolbar

The toolbar consists of the following buttons:


Transfers the changes made to the test to the database.


After a security query, gets the version of the test saved in the database and displays it on the screen.


Prints out the marked area of the test; if no area is marked, the entire test will be printed out.


Cuts out what is currently marked and puts it on the clipboard.


Copies the area currently marked to the clipboard.


Inserts the contents of the clipboard from the cursor position or overwrites the marked area with the contents of the clipboard.


Opens the Dialog for finding a character string within a module (see ***5.4.6 The find and replace dialog***).


Opens the dialog for replacing a character string within the module by a different string (see ***5.4.6 The find and replace dialog***).


Undoes the last change.


Repeats the last undone change.


Comments out all lines that are marked (inserts a ' character at the beginning of the line).


Deletes the comment character at the beginning of each marked line (if present).


Empties the output window.


Opens / closes the method selection window (see ***5.4.7 The method selection window***).


Opens the dialog for analyzing objects (see Help on Basic, or file WW10_000.chm in the PROVEtech:TA bin directory).


Opens the dialog for referencing of COM servers present in the system or .NET assemblies (see Help on Basic, or file WW10_000.chm in the PROVEtech:TA bin directory).


Opens the “dependencies” window (see ***5.4.8 The dependencies window***).

Saves the test and performs a syntax check of the test; this is where input errors such as “System.Rumark” instead of “System.Remark” are detected.

Saves the test and executes it in the debug mode. Debug mode means that the test result (see ***5.6 Test results***) is marked as deleted during the execution and finally removed from the database after the test execution. Outputs appear additionally in the output window.

Stops the current test and marks the next line to be executed (pause). Please read the lower notice.

Aborts the current test. Please read the lower notice.

This button manages three options related to the test result of a debug session. Click on the arrow beside it to open a menu to select these options:

- Protocol output in the Immediate Window.
- Show Test Manager result automatically when debugger starts.
- Show Test Manager result.

Sets a breakpoint at the cursor position. The test stops before any such marked line is executed (see). Breakpoints are only stored for the duration of the PROVEtech:TA session.

On a paused test, shows the content of a marked expression in the output window, so that you can, for example, check the contents of a variable at the run time. Pressing the key combination Ctrl+F9 has a similar effect, but the output is then directed to the watch window (see output window) and is recalculated after every execution step (see , , etc.).

On a paused test, sets the next line to be executed.

On a paused test, shows the next line to be executed.

Executes the next line in single step mode; if it contains a function from another module, this method is entered in.

Executes the next line in single step mode, functions contained therein are executed without entering in.

Jumps from the current method and stops at the next line of the calling method. If this is a main method (Sub Main), the execution terminates.

As , but stops as soon as the current cursor position is reached during execution.

Opens the dialog for editing a user-defined dialog (see Help on Basic or file WW10_000.chm in the PROVEtech:TA bin directory). The program lines for the user-defined dialog are inserted at the current cursor position.

Opens the dialog for editing the module/test properties. You can choose whether the test is to be a macro/code module, an object module or a class module. In the case of an object or class module you can specify the object/class name. More information can be found in the help for Basic or under .

Tries to complete an inchoate signal name or shows a list of all signal names that contain the already entered sub string. Press *Ctrl+Space*, *Tab* or *Enter* to select one list entry. For example enter *"t15* and press to get a list of all signal names that contain *t15*. When only one signal name exists that contains *t15*, PROVEtech:TA automatically exchanges *t15* by the complete signal name. Click on the arrow aside to get a menu to select if you want to search for a sub string in the signal names or for an identical beginning.

Locks the Basic code permanently or unlocks it; see chapter ***5.4.10 Locking the Basic code***.

This button manages up to four different help files. Click on the arrow beside it to open a menu for selecting the file to display:

- opens the Basic help (WW10_000.chm)
- opens the PROVEtech:TA help for the test language extensions
- opens the Automation Library help (in case you have this license)
- opens the user defined help file in the case it is specified via ***3.2.7.5 Test Manager*** in the Settings dialog.

The toolbar also has a combo box for code encryption. See chapter ***5.4.11 Encryption of Basic code***.

Stopping a test program does not stop its execution immediately. In opposite to a compiled program the Windows process (PROVEtech:TA) continues after stopping a script. So, stopping a script starts cleaning up, which means executing the destructors of all created objects. This can take a while – an endless loop in a destructor results in never stopping.

The test object programmer must take into account that a destructor might get executed earlier than it typically happens when a test program terminates regularly.



The behavior is as follows:

- Pressing the Stop button while a script executes normally stops the execution at the current position and starts executing the destructors.
- Pressing the Stop button while a destructor is currently executing completely terminates the execution without further destructor processing – so pressing the Stop button twice immediately terminates the execution; as a consequence needed clean-up might not happen.
- Pressing the Pause button while executing a destructor stops executing the destructor and pauses on the next command if there are additional commands to execute.

5.4.2 The hotkeys

The following hotkeys / keyboard shortcuts are available in the editor:

Hotkey	Symbol	Function
F1		Displays help information for the currently marked command
Shift+F1		Calls up the online help for Basic
F2		Go to next bookmark, modus selectable, see below
Shift+F2		Go to previous bookmark, modus selectable, see below
Ctrl+F2		Set / remove bookmark
F3		Continue searching with the current settings (defined via Ctrl+F); direction: down
Shift+F3		Continue searching with the current settings (defined via Ctrl+F); direction: up
Ctrl+F3		Search for the currently marked text or for the word under the keyboard cursor; direction: down

Ctrl+Shift+F3		Search for the currently marked text or for the word under the keyboard cursor; direction: up
Alt+F3		Starts the incremental search, the mouse cursor changes and you can start typing the text you want to find; the editor directly marks the first match; the status bar informs about the typed text; direction: down
Alt+Shift+F3		Same as Alt+F3 but the direction is “up”
F4		Syntax check
Shift+F4		In case “ShiftF4Path” is specified in TM.ini: Saves the currently displayed module and starts the specified module using *System.GetGVar("_OriginPath_")* to get the displayed module’s path for a user defined extended syntax check
Ctrl+F4		Closes a floating Basic window
F5		Start / continue
Shift+F5 / Esc		Terminate
F7		Execute up to cursor position
Shift+F7		Shows the next statement
Ctrl+F7		Sets the next statement
F8		Jump in
Shift+F8		Skip
Ctrl+F8		Jump out
F9		Set / remove breakpoint
Shift+F9		Calculate expression (view in the output window)
Ctrl+F9		Add expression to the watch window
Ctrl+Shift+F9		Remove all breakpoints
F12		Tries to find the definition of the item whose name the keyboard cursor currently resides on, e.g. a function. In case the keyboard cursor is on a '#uses line, the editor opens the corresponding module. In case of a signal name the signal selection window shows up and selects that signal.
Ctrl+Space		Complete word
Ctrl+Tab		Alternates between the Basic windows in case several ones are open

Ctrl+Shift+Tab		Alternates between the Basic windows in reverse order in case several ones are open
Shift+Enter		Auto completion of pre-defined text modules (see ***5.4.5 The edit window***)
Ctrl+0..9		Memorize currently selected module in one of 10 entries
Alt+0..9		Select the module previously memorized with Ctrl+0..9
Ctrl+A		Mark all
Ctrl+C		Copy marked area to the clipboard
Ctrl+D		Empty the output window
Ctrl+E		Sets the focus to the output window
Ctrl+F		Search (see ***5.4.6 The find and replace dialog***)
Ctrl+G		Opens a dialog for entering the line number you want to display
Ctrl+H		Replace (see ***5.4.6 The find and replace dialog***)
Alt+H		Replace the symbol name at the caret position within the whole module (but not across several modules)
Ctrl+I		Display parameter info
Ctrl+J		Comment out the marked area
Ctrl+K		Remove comment characters in the marked area
Ctrl+M		Automatically formats the selected lines (*)
Ctrl+O		Shows / hides the output window. While executing a test it is always visible.
Ctrl+P		Print
Ctrl+Q		Sets the focus to the test tree
Ctrl+R		Replace (see ***5.4.6 The find and replace dialog***)
Ctrl+S		Save changes to the database
Ctrl+Shift+ S		Completes an inchoate signal name, find a sub string
AltGr+S		Same as above but search for identical beginning
Ctrl+T		Set the focus to the stack window
Ctrl+V		Paste clipboard

Ctrl+Shift+V		Paste clipboard embedded in single quotation marks
AltGr+V		Paste clipboard embedded in double quotation marks
Ctrl+W		Set the focus to the watch window
Ctrl+X		Cut out marked area and copy to clipboard
Ctrl+Y		Redo
Ctrl+Z		Undo
Shift+Backspace		Forward to the module edited before
Ctrl+Backspace		Back to the latest edited module
AltGr+Backspace		Deletes the word before the keyboard cursor
Alt+Up		Move the current line or the marked lines up
Alt+Shift+Up		Mark previous symbol usage
Alt+Down		Move the current line or the marked lines down
Alt+Shift+Down		Mark next symbol usage
Alt+Shift+Home		Mark symbol definition
Ctrl+'+' / Ctrl+'-'		Expands/collapses the current function/type body (toggles the outlining)
Ctrl+Shift+'+'		Expands all function and type bodies (outlining)
Ctrl+Shift+'-'		Collapses all function and type bodies (outlining)



(*) The automatic format of the selected lines via Ctrl+M first searches the line before the selection to detect the correct indention. In case there is none (selection starts at the beginning of the text), the algorithm starts without indention. After that it indents the selected lines automatically. It only uses the tab character for the indention. It takes care of connected lines. The characters Space+_ at the end of the line mark that the next line is part of the multi-line statement. These lines get an additional tab for the indention.

The algorithm considers not only the normal Basic statements like "If" / "End If", "For" / "Next", "Function" / "End Function", but also the Automation Library statements like "RT.BeginComputation" / "RT.EndComputation".

Move the mouse cursor across the toolbar buttons slowly to display a brief explanation of each button function. Keyboard shortcuts exist for some functions; these are shown in brackets. The context menu also displays the according hotkeys.

You can use bookmarks to mark lines in the module. Press F2 or Shift+F2 to jump between these lines. PROVEtech:TA saves the set bookmarks when you change the module so that you can use them again when you go back. PROVEtech:TA looses this information when you close it.

You can change the jump mode for the bookmarks via the context menu item *Jump To Bookmarks Module Spanning*:
- Module internal for jumping to the bookmarks within modules in the ascending or descending order.
- Module spanning for jumping to the bookmarks in the order you have set them across modules.

5.4.3 The output window

The `Immediate` tab of the output window is used during execution within the editor to represent the outputs by the commands Debug.Print and System.Remark (if the Option "Protocol output in the Immediate Window" is checked, see chapter ***5.4.1 The toolbar***). You can also use  on the toolbar to calculate expressions and to display these in this tab.

Unlike outputs through Debug.Print, outputs through the method System.Remark appear in the test protocol.

Right click in the output window and select the menu item *Clear* () to delete the content of the window or select the menu item *Display In Browser* () to save the content in a temporary .html file and display it in a browser window.



Figure 325 – The tabs of the output window during execution

While a test is being run, two further tabs – `Watch` and `Stack` (see ***Figure 325***) – appear. Mark an expression and press Ctrl+F9 to display it in the `Watch` tab. Unlike , the value displayed is updated every time the test is stopped. The `Stack` tab shows the current call hierarchy of the methods. Click on an entry in the list to view the call line in a test.



You can hide the output window while editing (but not while debugging). You can activate/deactivate this variant via the hotkey Ctrl+O.

5.4.4 Object and procedure selection

The combo box `Proc` shows the procedure currently being processed or "(Declarations)" for the global declaration area for variables and module integrations ('#uses instructions) at the start of the test. Open the complete method list by clicking on the button and select an entry to jump directly to a procedure in the test.

With the `Object` combo box you can select an object that supports results. "(General)" represents the current test. You can use the procedure selection to jump to the selected event handling procedure. If it does not yet exist, it is automatically added.

5.4.5 The edit window

In the edit window you can enter your module. Click with the right mouse button to display a context menu containing a lot of useful functions. Click with the left mouse button on the gray area on the left to set or remove a breakpoint () at the respective line. An arrow () marks the next line to be executed in single step mode or after a test has been paused. A flag ( ) is a bookmark.

The hotkey Shift+Enter triggers the auto completion for pre-defined text modules. As an Example enter “RT.BeginComputation” in the editor and press Shift+Enter. This automatically reformats the text to:

RT.BeginComputation

RT.EndComputation

And it places the keyboard cursor between “If” and “End If”. Here is a list of texts that trigger the auto completion: Do, Enum, For, Function, If, Property, Select, Sub, Type, While

When the Automation Library (library to program real-time tests) is activated, there are additional pre-defined auto completions for all RT.Begin… commands and AL.Init.

You can also define your own auto completions in the file “TM.ini”. For example you can define, that the editor should insert all your default '#uses commands when entering “std” and pressing Shift+Enter. Read the ***Administrator’s Guide*** for more information about that.

You can find more information about programming tests in the test language manual.

The context menu item *Settings|Select Colors…* opens the following dialog:

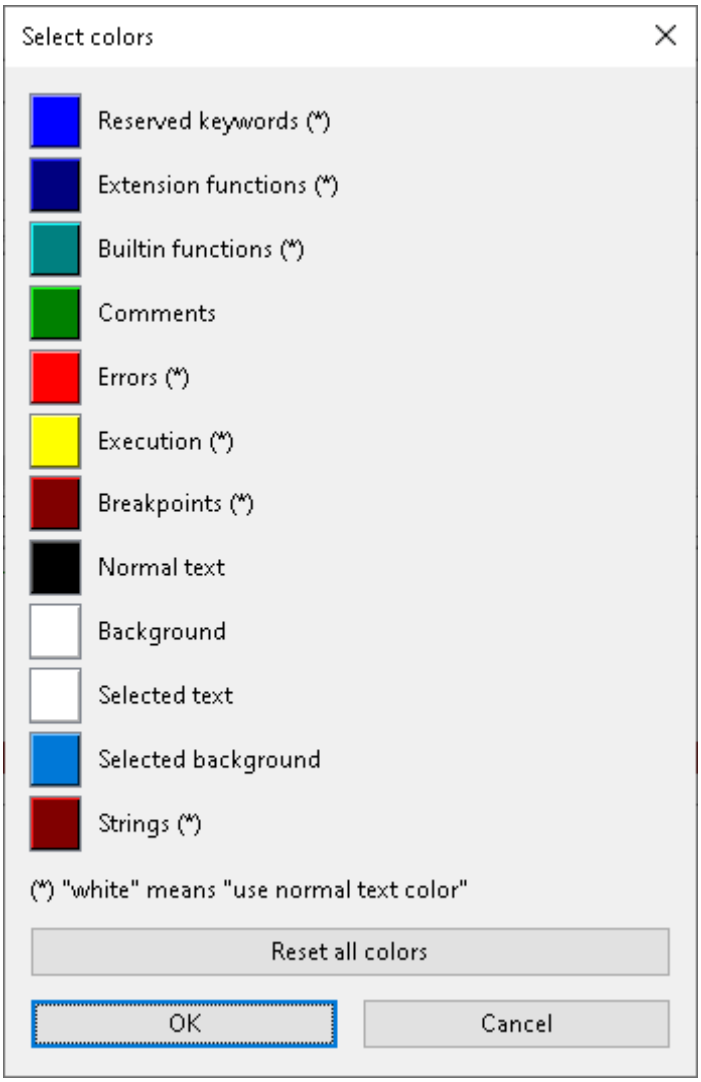


Figure 326 – Select colors dialog

Click on the color buttons to select the colors for the specific elements. This immediately updates the according color in the editor window. To reset the colors to their default values click on `Reset all colors`. `OK` acknowledges all changes and `Cancel` undoes them. Each use can select the colors individually.

The *Settings* sub menu of the context menu has more user specific settings for the editor like the font or to view/hide certain elements.

Drag signal names from the Signal Selection window (see ***4.2 The Signal Selection window***) to this window using the drag and drop technique to obtain a signal name list (names separated by |). Hold the Alt key while dropping one or several signal names to embed the characters in double quotation marks. Hold the Shift key for single quotation marks.

Drag a test entry from the Test Manager tree to this window in order to obtain the path to this test. Hold the Alt key while doing this to embed it in double quotation marks.

Click with the right mouse button to display the context menu. Beside a lot of the toolbar commands it contains the menu item *Select Module "..."* when clicking on a **'#uses** command. Use this menu item to straightly select the referenced module.

Click on the name of a function, method, property, constant, type, class, object, enum or enum value and choose the menu item *Go To Definition Of "..."* to search and display the definition of the clicked name.

Beside the main edit window on the `Program` tab, you can open additional floating edit windows to work on several test items simultaneously. These windows are for editing purposes only, not for executing and debugging tests. Hold down the Ctrl key while selecting the context menu items *Select Module "..."*, *Go To Definition Of "..."* or *Select Hotkey-Module* to display the result in a new edit window. As an alternative select the context menu item *Display In Additional Editor* to open the already displayed test item in an additional editor.

Drag and drop between different Basic editors works only when the editors display different test items.

5.4.6 The find and replace dialog

Open the find and replace dialog via the button  or  of the toolbar or the hotkeys Ctrl+F, Ctrl+H or Ctrl+R:

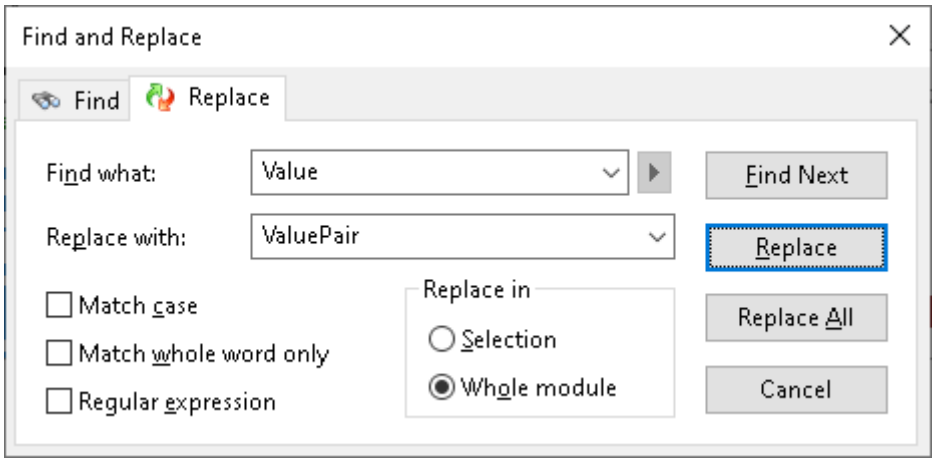


Figure 327 – The find and replace dialog

You can switch between find and replace via the tab control. The following text only describes the replace tab, which is mainly a superset of the find tab.

The `Find what` combo box initially shows the current word under the keyboard cursor or the current selection. This combo box specifies the string to be replaced. The `Replace with` combo box contains the new text that replaces the found strings.

Activate the check box `Match whole word only` to replace only whole strings consisting of characters, numbers and underscores. Check `Match case` to search case sensitive. Activating the `Regular expression` check box disables the `Match whole word only` check box. This option lets you search for regular expressions (see chapter ***5.18.4 Regular expressions***). It also activates the right arrow button beside the `Find what` combo box. Pressing the button opens a menu that helps you assembling regular expressions. The last menu item opens the homepage of the Boost Library containing extended information about the syntax.

When you select a part of the text before entering the replace dialog, you can use the `Selection` radio button to replace only inside the selected text. In this case the buttons `Find Next` and `Replace` are disabled. Use the `Whole module` radio button to ignore the selection.

Press `Find Next` to select the next text part that matches your search criteria. Press `Replace` to replace the current matching selection and jump to the next matching text part. Use the `Replace All` button to replace all matching text parts. Finally press `Cancel` to exit the dialog.



To replace not only within the current module but a whole subtree of the Test Manager, use the replace dialog described in chapter ***5.18.3 Find and replace in test programs***.

There are also hotkeys for finding the next occurrence of the word under the cursor or for an incremental search; see chapter ***5.4.2 The hotkeys***.

If the check box *Regular expression* is checked, then \r, \n, and \t can also be used in the *Replace with* text.

5.4.7 The method selection window

The method selection window (see ***Figure 328***) can be placed anywhere on the screen. You can display or hide the window via the button of the toolbar. It contains all objects () and methods () of the Basic extensions of PROVEtech:TA in a tree view. The window makes for easy programming via dialogs.

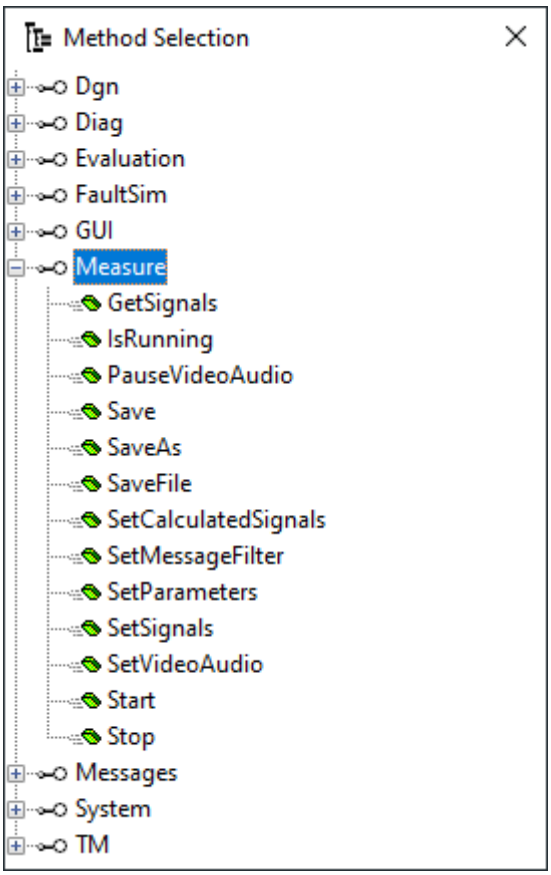


Figure 328 – The method selection window

Click on a method with the left mouse button to insert it immediately at the cursor position if it has no call parameters. Otherwise, a dialog opens up for convenient selection of the parameters to be passed. This programming technique is particularly recommended for beginners and methods that are rarely used.

Click with the right mouse button on an object or method to open the relevant online help information.

You can extend the method selection tree with own entries by creating a Basic script. Specify the path to the script in the configuration of the Test Manager within the Basic handler selection for `User-defined method selection window` (see chapter ***3.2.7.5 Test Manager***). The script must implement two predefined methods:

```
Function GetEntries(callback As Object) As String()

Sub GenerateText(hOwnerWnd As System.IntPtr, strMethod As String)
```

The `GetEntries` method returns the tree entries to create as string array. The | character is the hierarchy separator. As an example, return `"User defined|Color selection"` as an array element to create the entry `Color selection` in the folder `User defined`. User entries have a blue icon in the tree (). The parameter `callback` is an object to insert text in the Basic editor. Call its method to insert text:

```
Sub InsertText(Optional strText As String = "", Optional bFocusToEditor As
Boolean = True)
```

The 1st parameter is the text to insert and the 2nd parameter specifies if the keyboard focus shall be in the editor window afterwards – typically useful when the user defined dialog closes afterwards. The string `strText` can contain special characters:

vbLf for a line break (Enter key)
ChrW(1)+ChrW(35) to move the insertion mark to the end of the line (End key)
ChrW(1)+ChrW(36) to move the insertion mark to the beginning of the line (Pos1 key)
ChrW(1)+ChrW(37) to move the insertion mark one character left (Left key)
ChrW(1)+ChrW(38) to move the insertion mark one line up (Up key)
ChrW(1)+ChrW(39) to move the insertion mark one character right (Right key)
ChrW(1)+ChrW(40) to move the insertion mark one line down (Down key)

While inserting the text PROVEtech:TA removes `"Set "` at the beginning of lines automatically if the edited script's language is WWB.NET (WWB.NET does not need the `Set` command for assigning objects in opposite to WWB-COM).

PROVEtech:TA calls the method `GenerateText` when clicking on a user defined tree entry. The Basic script should then display a user defined window or immediately insert text by calling `InsertText`. The parameter `hOwnerWnd` is the window handle of the tree window. It shall be the owner of the newly displayed window to position the new window closely to the tree window and on to of it in the Z order. When the user finishes

editing the settings in the user defined windows the script calls `InsertText` to insert the according text in the editor window.

An example script follows which creates two entries in the tree:

```
'#Language "WWB.NET"

Option UsesAmbiguousError On
Option Explicit

Imports System.Windows
Imports System.ComponentModel
Imports System.IO
Imports System.Text

'PROVEtech:TA object to insert text and move the keyboard focus into the editor window
Private m_callback As Object

'the RECT structure of Windows
Private Structure RECT
    Dim iLeft As Integer
    Dim iTop As Integer
    Dim iRight As Integer
    Dim iBottom As Integer
End Structure

'Windows function to get window rectangle
Private Declare Function GetWindowRect Lib "User32.dll" (hWnd As System.IntPtr, _
    ByRef sRect As RECT) As Integer

'the window layout as XAML
Private Const m_strXaml As String="<Window "+ _
    "xmlns=""http://schemas.microsoft.com/winfx/2006/xaml/presentation"" "+ _
    "xmlns:x=""http://schemas.microsoft.com/winfx/2006/xaml"" "+ _
    "Width=""300"" Height=""130"">"+ _
    "<Grid Margin=""10,10,10,10"" FocusManager.FocusedElement="""+ _
        "{Binding ElementName=Insert}"">"+ _
    "<TextBlock Text=""Text to insert:"" HorizontalAlignment=""Left"" "+ _
        "VerticalAlignment=""Top""/>"+ _
    "<TextBox x:Name=""Insert"" Height=""21"" Margin=""0,21,0,0"" "+ _
        "VerticalAlignment=""Top""/>"+ _
    "<Button x:Name=""Assume"" Content=""Assume"" IsDefault=""true"" "+ _
        "Height=""21"" Width=""90"" HorizontalAlignment=""Left"" "+ _
        "VerticalAlignment=""Bottom""/>"+ _
    "<Button x:Name=""Cancel"" Content=""Cancel"" IsCancel=""true"" "+ _
        "Height=""21"" Width=""90"" HorizontalAlignment=""Right"" "+ _
        "VerticalAlignment=""Bottom""/>"+ _
    "</Grid></Window>"

'handler function to get the method selection entries
Public Function GetEntries(callback As Object) As String()
    m_callback=callback
    Return { "Example|Test", "Example|EnterText" }
End Function

'handler function called when clicking on items in the method selection
Public Sub GenerateText(hOwnerWnd As System.IntPtr, strMethod As String)
    Select strMethod
    Case "Example|Test"
        'after inserting "Test()" hit the keys Enter, Up, End, Left
        'to place the caret between ()
        m_callback.InsertText("Test()"+vbLf+ _
            ChrW(1)+ChrW(38)+ChrW(1)+ChrW(35)+ChrW(1)+ChrW(37))
    Case "Example|EnterText"
```

```
        OpenXamlWindow(hOwnerWnd, strMethod)
    End Select
End Sub

'sub to open a WPF/XAML window
Private Sub OpenXamlWindow(hOwnerWnd As System.IntPtr, strMethod As String)
    Dim xamlWindow As Window

    'load XAML from m_strXaml
    xamlWindow=Markup.XamlReader.Load(New MemoryStream( _
        Encoding.UTF8.GetBytes(m_strXaml)))

    'get upper left corner of method selection window and place our own window on top
    Dim sRect As RECT
    Dim helper As New Interop.WindowInteropHelper(xamlWindow)
    Dim button As Controls.Button

    helper.Owner=hOwnerWnd
    GetWindowRect(hOwnerWnd, sRect)
    xamlWindow.Left=sRect.iLeft
    xamlWindow.Top=sRect.iTop
    xamlWindow.Background=SystemColors.ControlBrush
    xamlWindow.Title=Replace(strMethod, "|", ".")  'use dots in the title
    'fill text control
    xamlWindow.FindName("Insert").Text="InsertMe"

   'add handlers
    AddHandler xamlWindow.Closing, AddressOf Window_Closing
    button=xamlWindow.FindName("Assume")
    AddHandler button.Click, AddressOf AssumeButton_Click
    button=xamlWindow.FindName("Cancel")
    AddHandler button.Click, AddressOf CancelButton_Click

    xamlWindow.Show()
End Sub

'Assume button handler
Private Sub AssumeButton_Click(sender As Object, e As RoutedEventArgs)
    'get the text box control
    Dim tb As Controls.TextBox=sender.FindName("Insert")
    'notify PROVEtech:TA to insert text with line break
    m_callback.InsertText(tb.Text+vbLf, False)
    sender.Parent.Parent.Close()  'close the window
End Sub

'Close button handler
Private Sub CancelButton_Click(sender As Object, e As RoutedEventArgs)
    sender.Parent.Parent.Close()  'close the window
End Sub

'handler called when closing window
Private Sub Window_Closing(sender As Object, e As CancelEventArgs)
    m_callback.InsertText()  'move keyboard focus to editor
End Sub
```

5.4.8 The dependencies window

The button  of the toolbar opens the “dependencies” window (Figure 329).

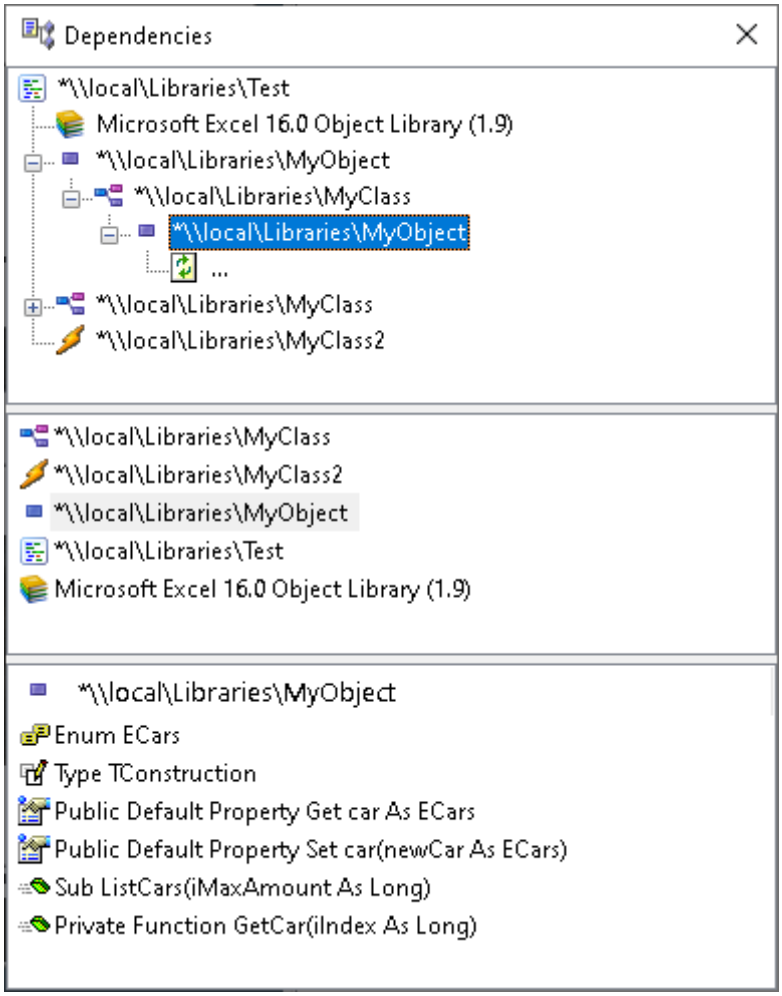


Figure 329 – The dependencies window

The window consists of three parts:

- The upper part contains all module dependencies in a tree view. The current module is the root entry. The entries below it are the references to components of the registry or .NET assemblies (icon) followed by the '#uses command dependencies to other modules (icon for elementary tests and code modules, for classes and for objects). The icon shows invalid dependencies. Expand the nodes of the tree view to display their dependencies to other modules. The icon followed by “...” marks recursive dependencies. Use the right mouse button to open the following menu:

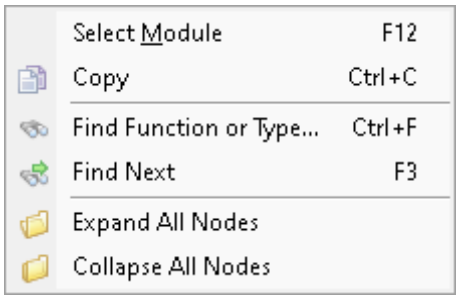


Figure 330 – Context menu of the tree view

Choose *Select Module* to open the clicked module in the Basic editor. Select *Copy* to copy the module path to the clipboard and *Expand All Nodes or Collapse All Nodes* to expand or collapse all nodes of the tree view. The find function is described below.

- The middle part of the “dependencies” window shows an alphabetically sorted list of all entries of the upper tree view, whereby each entry exists only once. Double click on an entry or use the context menu to open a module in the Basic editor.
  Choose *Select module in upper tree* of the context menu to find the first occurrence of the module in the dependency tree and select it.

- The lower part displays all **Enum**, **Type**, **Const**, **Sub**, **Function**, **Property** and **Class** definitions of the module selected in the upper tree view or in the middle list (the list header shows the name of the module). The icons have the following meaning:

Icon	Description
	Sub or Function
	Default Sub or Function
	Property
	Default Property
	Enum
	Type
	Const
	Embedded class

  The list is sorted in the order of appearance in the module. Click on the list header to sort it alphabetically. Double-click on an entry or use the context menu to jump to the according Basic code of the module in the Basic editor.

Press *F5* to refresh the view that currently has the keyboard focus. Use drag and drop to copy the dragged item's text to the Basic editor.

Use the context menu of one of the three parts or press *Ctrl+F* to open the find dialog:



Figure 331 – Dialog to find functions and types

PROVEtech:TA uses the word under the keyboard cursor or the currently marked text in the Basic editor as the default for the search item. With this dialog you can search through all modules listed in the middle part of the dependencies window for a **Function**, a **Sub**, a **Property**, a **Const** value, a **Type** or an **Enum** containing the search item text in its name. Select the check box `Match whole word only` to find an exact match. When PROVEtech:TA finds a matching code position, it marks the found **Function**, **Sub**, **Property**, **Const**, **Type** or **Enum** in the lower list and the corresponding module in the middle list. Press *F3* or use the context menu to continue the search process.

5.4.9 Edit references as text

The toolbar has the button for editing references to foreign components (DLLs) via check boxes in a list view. In the end, this adds lines in front of your Basic code which the editor does not display (hidden lines, which also include a header for classes and objects where WinWrap Basic stores the class or object name).

To edit the hidden reference lines directly, open the context menu of the edit window and select *Edit References As Text…* which has the same icon . It displays this dialog:

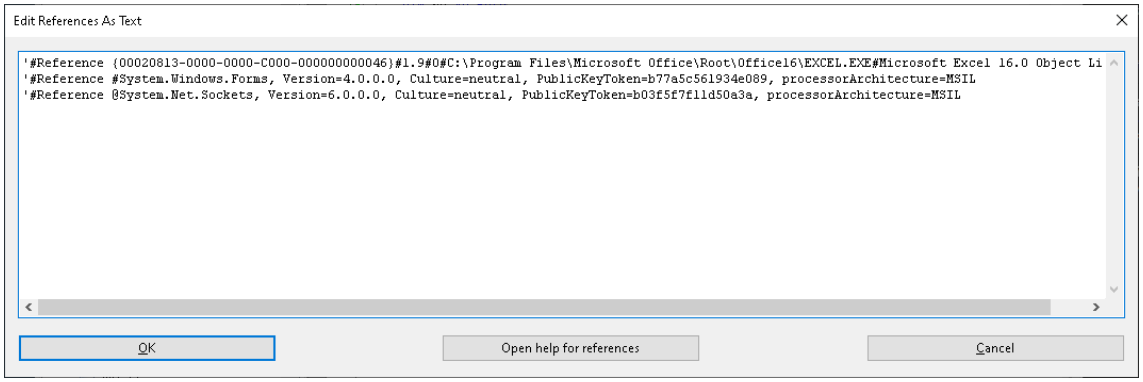


Figure 332 – Dialog to edit references as text

This is only for experts. The upper example shows three references. The first one is for a COM component, the 2nd one is for a .NET 4 assembly and the third one for a .NET 5+ assembly. References for .NET assemblies can either use the global storage via the

“PublicKeyToken” like in the upper example or they use the DLL path. In case of a path, you can replace parts of it with Windows environment variables embedded in %. Example:

%ALLUSERSPROFILE%\MyApp\MyAssembly.dll

Please be aware that COM references might show paths, but the paths are not relevant. Only the GUID in curly brackets followed by the version number and locale Id are relevant parts for looking up the component in the Windows registry (type library).

All lines must start with '#Reference, anything else is not allowed like empty lines or spaces in front of lines. The lines have a specific syntax. Click on `Open help for references` to open the according WinWrap Basic help page.

Clicking OK replaces the hidden reference lines of your module by the content of the edit field. This does not check if the lines are valid.

5.4.10 Locking the Basic code

Locking the Basic code prevents simultaneous changes on the code by several users. PROVEtech:TA has two mechanisms for that:

1. Automatic lock/unlock: When a user starts changing the code, PROVEtech:TA sets the lock automatically. If a different user now tries to change the code at the same time, he gets a warning message (see below). When the first user saves or dismisses his changes, PROVEtech:TA automatically unlocks the code and other users can change it.

2. Manual permanent lock/unlock: A click on the open lock icon () of the toolbar (see ***Figure 324***) locks the Basic code permanently. This means that saving or dismissing changes of the code does not unlock the code. A different user who tries to change the code gets a warning message (see below). This variant is useful for changing code over several days. The closed lock icon () and the pressed toolbar button indicates that you currently have the permanent lock on the displayed code. Click on it to save any code changes and unlock the code (icon changes to open lock, button is not pressed anymore). In case the toolbar button displays a red lock icon (), the code is currently locked permanently by a different user. Click on it to take over the lock permanently. You get a warning message in this case (see below).

In case you try to modify locked Basic code or in case you click on the red lock icon, you get the following warning message:

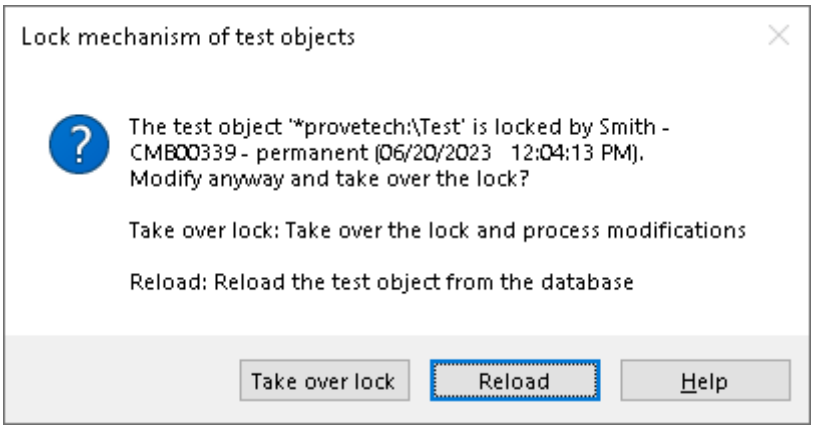


Figure 333 – Warning when trying to edit locked code

The warning shows which user (here: Smith) on which computer (here: CMB00339) at which time (here: 06/20/2023 12:04:13 PM) has locked the code. In case of a permanent lock, there is the additional text “permanent”. Click on `Take over lock` to take over the lock to your user (information about this below). Click on `Reload` to dismiss your code changes and reload the code from the database. The `Cancel` button is only available if you have clicked on the lock icon. Your code changes stay in the editor and you can copy them to the clipboard as an example. The `Help` button opens this manual page.

In case you save code changes and a different user has taken over the lock from you while you have edited the code (see button `Take over lock` in ***Figure 333***), you get the following warning message:

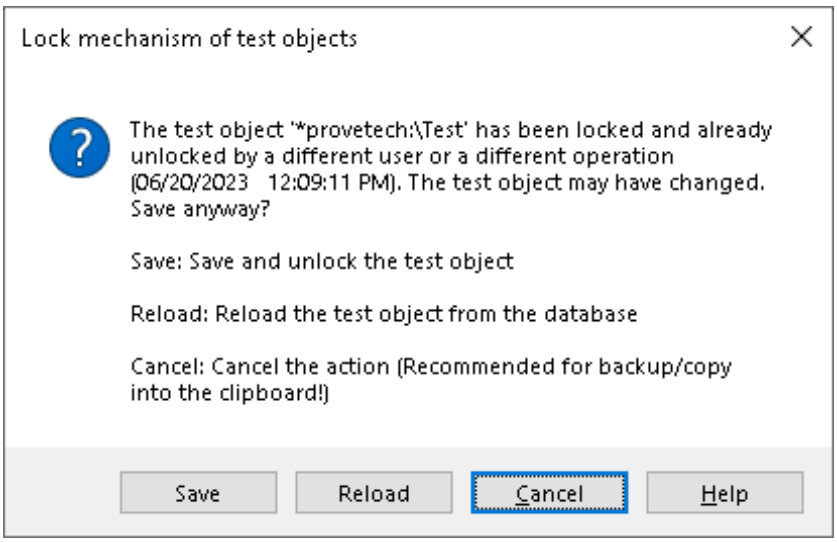


Figure 334 – Warning when trying to save

Again the warning contains the information, who has taken the lock on which computer and when it happened. The word “permanent” marks a permanent lock. Click on `Save` to save your changes anyway and unlock the code. Click on `Reload` to dismiss your

code changes and reload the code from the database. `Cancel` does not save your changes and you have the possibility to copy your changes to the clipboard as an example. The `Help` button opens this manual page.

You can find more information in chapter ***5.2.6 Access rights***.



Except for the Basic source code, no other fields are protected by a lock mechanism. So it is possible to overwrite changes e.g. in the field `Description` or in the parameters of a test object. Or there may be conflicts in the `Name` or `Comment` field of a test result.



PROVEtech:TA supports overriding locks set by other users on purpose. As an example this prevents being unable to work in case the user who set the lock is ill or on vacation. However, this possibility presumes some discipline while working with it.

5.4.11 Encryption of Basic code

PROVEtech:TA provides the possibility to encrypt Basic code transparently for the user and store it in the database in encrypted form. Exporting and importing encrypted modules works seamlessly.

There are two different mechanisms: Either the PROVEtech:TA license controls it or you specify a password.

In case the PROVEtech:TA license controls it (see chapter *2.2 Licensing*):

The possible options are a matrix as you can control it individually for different packages and each license/computer. A package typically contains several Basic modules licensed together, e.g. a whole library for special diagnostics functionality. For such a package it is possible to select if it shall be executable or additionally editable for each PROVEtech:TA license.

In case the right for execution is missing, you get an error message when executing an encrypted module. This can also happen when you link such a module via the '#uses command.

In case you select an encrypted module in the Test Manager tree, open the Program tab and have no editing right for the according package, you get an information text instead of the module code (see ***Figure 335***). Find and replace via the Test Manager tree also

does not work in this case. In case you have editing rights, you automatically also have the right for executing it.

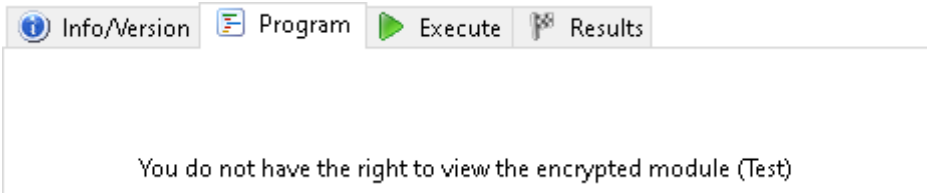


Figure 335 – Missing editing right

If you link an encrypted module via the '#uses command and you have no editing right, you cannot step into its code or set breakpoints in it while debugging.

The toolbar of the editor (see chapter ***5.4.1 The toolbar***) has a combo box. If your license contains the editing right for any package, it has some entries beside “Unencrypted” and ”<Add…>”:

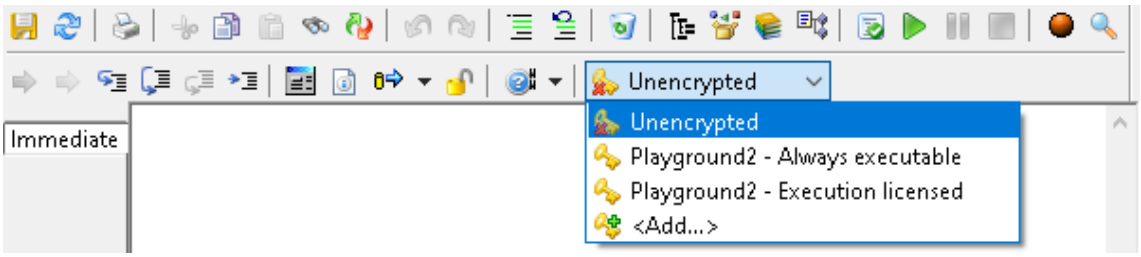


Figure 336 – Toolbar with combo box for encryption

Use this combo box to choose if and for which package you want to encrypt the module. The package names are part of the license information. Select “Unencrypted” for no encryption. When creating new modules this is the default.

The following overview explains the different kinds of encryption variants:

Type of encryption	**Requirement of desired action**	
	Encoding, decoding, viewing and editing the Basic code	Executing the Basic code
Unencrypted (plain) Basic code	No special licensing necessary for this computer	No special licensing necessary for this computer
Basic code with encryption “Variant1” including control to edit and execute	Licensing of “Variant1” requires edit and execution right for this computer	Licensing of “Variant1” requires execution right for this computer
Basic code with encryption “Variant2” including control only to edit	Licensing of “Variant2” requires edit right for this computer	No special licensing necessary for this computer, all computers can execute

If you want a new encryption package, please contact the PROVEtech Support (provetech@akkodis.com).

In case you use a password:

This works independent from the PROVEtech:TA license. Basic modules encrypted this way are always executable but viewing and editing them only works after entering the correct password. Thus, she same rules apply as described above for “Variant 2”. Just like the license-controlled way, you use “packages” to bundle several modules where you use the same password and package name. To create a new package, open the combo box (see ***Figure 336***) and select “<Add…>” to display the following dialog.

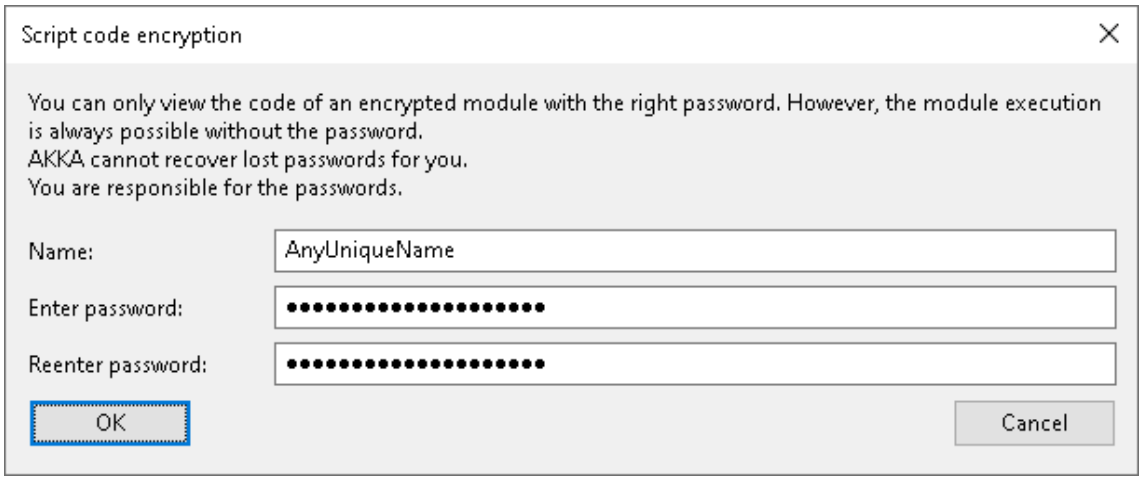


Figure 337 – Creating a new package

There you specify the new package name and enter the new password twice (to ensure it is entered correctly). You are responsible for the password. The mechanism does not store your password in clear text. The package name should be unique. PROVEtech:TA cannot ensure that. If you or your colleagues have several packages with the same name but different password, it will be hard for you to know which password is the correct one. It is also possible to use the same password for several packages. To change a password, simply create a new package with a new name and re-encrypt the code with the new one.

After closing the dialog with `OK`, it creates a new entry in the combo box with the specified name and additionally selects that entry. When saving your code, PROVEtech:TA automatically encrypts it. To change the encryption, simply choose a different combo box entry or choose “Unencrypted” to safe it unencrypted.

The new entry remains in the combo box as long as you are logged in. If you select a module encrypted with a yet unknown password and want to view the code (tab `Program`), PROVEtech:TA automatically opens the password dialog for entering the password to view the code:

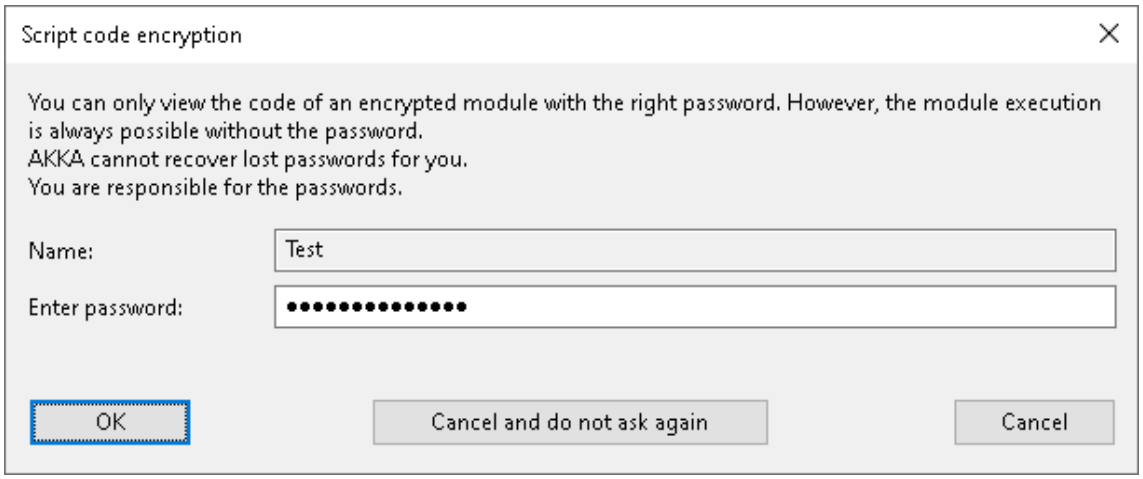


Figure 338 – Entering password for viewing code

If you enter the wrong password and/or cancel the dialog, PROVEtech:TA displays the message shown in ***Figure 335***. If you enter the correct password, it shows the code and adds the according package to the combo box. If you click on `Cancel and do not ask again`, PROVEtech:TA will not ask you again for the password for that package until you re-login.

In case encryption packages are available (in the combo box), you can encrypt an entire subtree by context menu, too. That means, encrypt the WinWrap Basic code of each single test object.

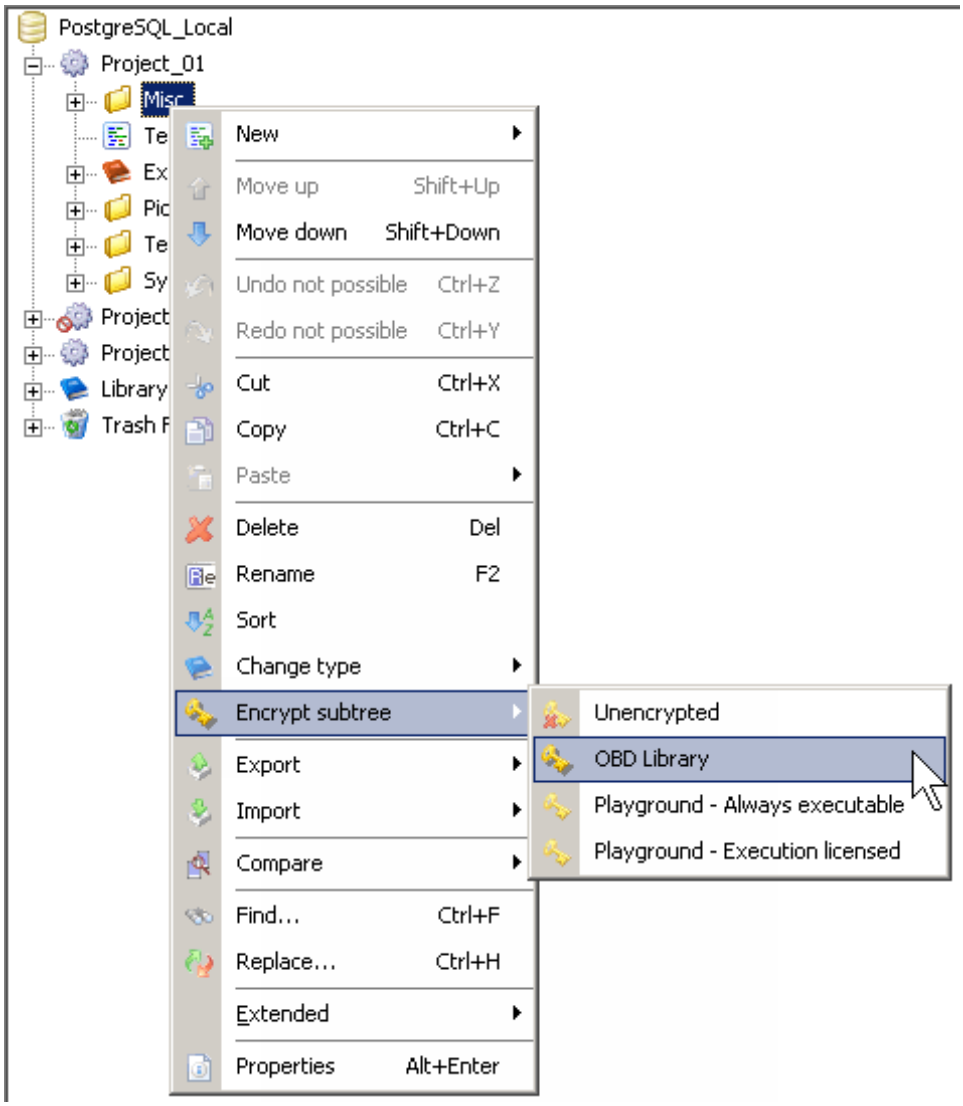


Figure 339 – Encrypt a subtree



To execute an encrypted module PROVEtech:TA must decrypt the basic code internally. The encryption is done only on the basic code and nothing else. Please note that hackers/crackers may be able to get hold of the unencrypted basic code despite compliance with the current encryption standards in PROVEtech:TA. This is especially security critical in the upper "Variant2" or the password variant (all computers can execute, see above).

The encryption and decryption happens within PROVEtech:TA using AES-256. This means while transporting the modules to/from the database, they are in the encrypted form. The database stores them in the encrypted form. When exporting them to the file system, they are in the encrypted form. This means that a database administrator and also the PROVEtech:TA administrator only see the encrypted form if the computer does not have a corresponding PROVEtech:TA license or the password is unknown.

5.5 Executing tests

You can execute individual elementary tests, which means the WinWrap Basic test scripts, or system tests and project objects that may include several individual tests.

Click on test you wish to execute. Then click on the `Execute` tab.

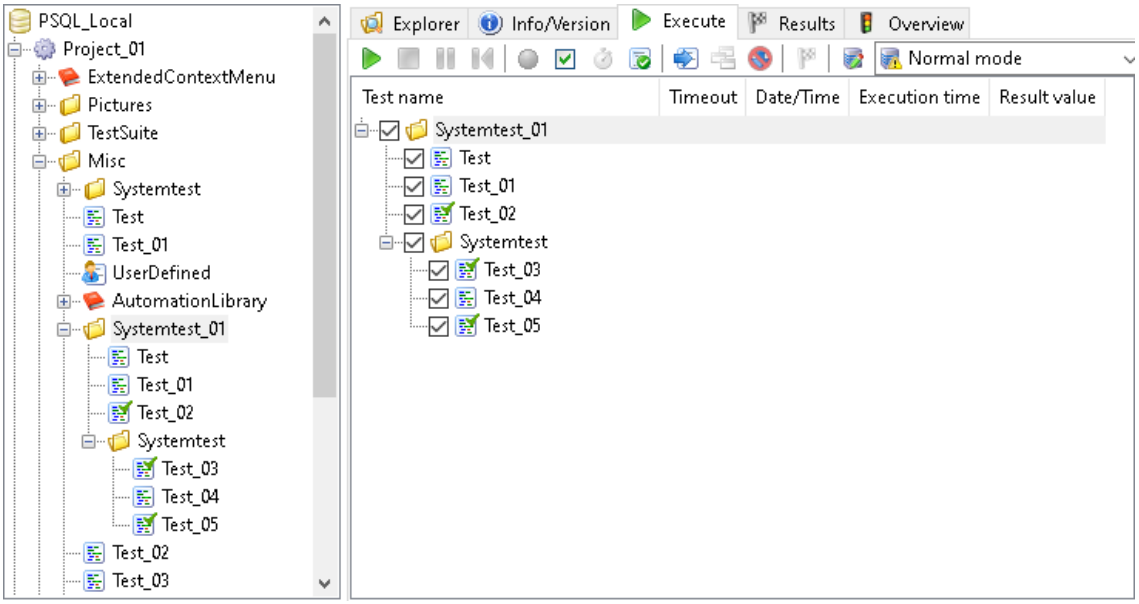


Figure 340 – The "Execute" property page

In the `Execute` property page you can see all elementary tests, user-defined test objects (if executable, see in the Info box below), system tests and project objects in hierarchical order to the selected test object in the Test Manager tree. Regarding the column `Timeout` please read in chapter ***5.5.4 The timeout value***.

When or after executing a test, the execution timestamp (`Date/Time`), the `Execution time` (duration) and the `Result value` of the test result is shown in the corresponding columns. The execution time corresponds to the end time of the test execution. If the test is still running, the start time is displayed.

The `Estimated execution time` is displayed as tooltip on the `Execution time` and results from the execution time of the latest results of this test. If you have selected an elementary test in the Test Manager tree then only this is displayed in the execution list.

In the top of the `Execute` property page there is a toolbar to control the test execution and to display the results. There are corresponding items in the context menu. In the following table all operations are explained:

(In Sequencer-Mode the toolbar and functionality is different. Please read chapter ***5.14 The remote test sequencer***.)

 Play	Starts the execution of the selected test object; if the button is disabled you probably have to click `Stop` and `Reset` first; after the test run you can go to the test result by double-click
 Stop	Interrupts the current execution of the test object; all subsequent test objects are skipped; the test result is written to the database; please read the lower notice in chapter ***5.4.1 The toolbar*** concerning the stop behavior
 Pause	Interrupts the current execution of the test object and pauses test execution of the whole execution list. Click on PLAY again to continue execution with the next elementary test.
 Reset	Resets a test execution or a syntax check; then the test results are not in direct access anymore and you can start a new test run or a new syntax check
 Breakpoint (Ctrl+F9)	Sets a breakpoint at which the test run will be stopped; this function allows you to interrupt test runs after individual test objects; all breakpoints are marked with this icon
 Skip (Ctrl+F8)	The execution of tests or the syntax check skips the selected test objects; skipped test objects are shown unchecked
	Please read more in chapter ***5.5.4 The timeout value***.
 Syntax check (Ctrl+F4)	Starts the syntax check; after that you can go to syntax error via double-click
 Go to test	Selects the corresponding test object in the Test Manager tree
 Display in additional editor	Displays the test code in an additional WinWrap Basic editor

	No scrolling	If the button is not pressed, test objects are selected for execution and scrolled to the visible area. Terminated system test runs with status “passed” or “indiferent” will be collapsed. If the button is pressed, no automatic scrolling and collapsing operations take place.
	Test result… (double-click)	Displays the test result in a separate dialog after executing a test; after a syntax check you can go to the syntax error via this button
	Configure remote test sequencer...	Opens the remote test sequencer configuration dialog (see ***5.14 The remote test sequencer***)
	(Ctrl+A)	Selects all entries
	(Ctrl+F)	Opens the Find dialog

If older versions of test objects are displayed then they appear with light blue background. Read-only tests have a gray background.

While executing a test the PROVEtech:TA icon is rotating on the right top corner of the application. If you change the selection in the Test Manager tree while executing a test then the `Execute` tab is not updated until the test execution has finished.

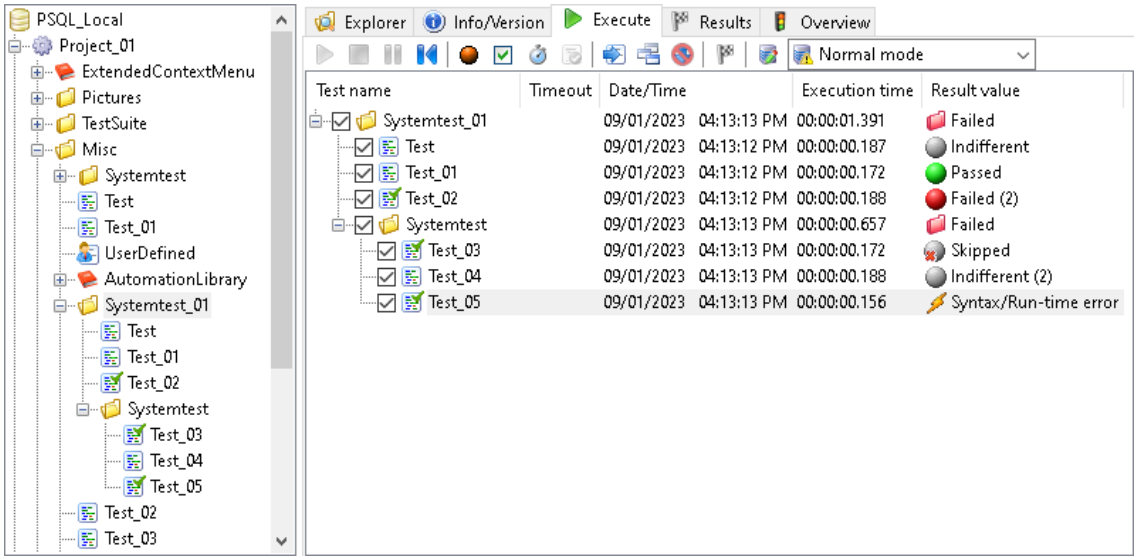


Figure 341 – The property page “Execute” after executing a system test



While executing tests you can also navigate in the test tree and even edit tests. While and after the test execution the tab "Execute" is not rebuilt. If you click on the PROVEtech:TA logo at the top right corner then the last executed test or system test is selected again and the tab "Execute" is shown again.



In addition to elementary tests, system tests and project objects you can also execute user-defined test objects. You can read more about this in chapter ***5.9 User-defined test objects and extending the context menu***.



You can execute older versions of a test too. See ***5.8.6 Executing older test versions*** on this.

A system test or a project object can contain several elementary tests. The test sequence in a system test determines the order in which the tests are running. However, you can also change the order of the test objects in the Test Manager tree later on.

You can start test execution with any elementary test from the list. Once the elementary test has been started, all subsequent elementary tests will be processed consecutively.

After the test run this property page `Execute` provides the possibility to validate your test results, too. Please read more in chapter ***5.7.5 Validating test results***.



You can control the execution order with the Test Language command ***TM.Execute*** or ***TestItem.Execute***, too. Please read the Test Language manual for that.



You can control the execution list by remote, e.g. from any computer at your desk. Please read chapter ***5.14 The remote test sequencer*** for that.

5.5.1 The status display

Behind every test object there is a status display in the form of an LED icon. It provides information on the status and result of the respective elementary test and is controlled by

the ***System.SetResult*** command in the test program and by the Test Manager. Meaning of LED colors:

	(no LED)	The test object has not yet been run.
	(yellow LED)	The test object is being run now.
	(green LED)	The test result is successful.
	(red LED)	The test result is not successful.
	Stop watch	The execution time of the test was longer than the specified time limit.
	(gray LED)	The test result has the result value “indifferent”
	(brown LED)	The test result has the result value “not implemented”, because the Basic script is not finished yet
	(lightning symbol)	Syntax or runtime error in the program code.
	(green check)	The syntax check was okay.
	(gray LED with red square)	The execution of the test was stopped.
	(gray LED with red cross)	The test object was skipped or ignored. (In fact, the test result was generated and after the test run immediately removed again.)

If an additional result value was set not equal 0 then this is indicated in parentheses behind the LED icon.



A test result is automatically set from "running" to "stopped" if it is at least one day old and is no longer recognized as "running" on the test execution computer. For example, if the PROVEtech:TA process was terminated by the Windows operating system during execution and PROVEtech:TA is restarted after one day, the status is set to "stopped" when the test result data is fetched.

5.5.2 Breakpoints

Breakpoints  allow you to run system tests in steps. You can stop the execution before an elementary test in the list by setting a breakpoint. You can set as many breakpoints as you like and thereby run a system test in steps, elementary test by elementary test.

Click on the elementary test before which execution is to be interrupted. The elementary test is shown in the list on a blue background.

Then click on or on the icon of the test object. The elementary test is marked as a breakpoint, and when the test is being run execution will be stopped before this point.

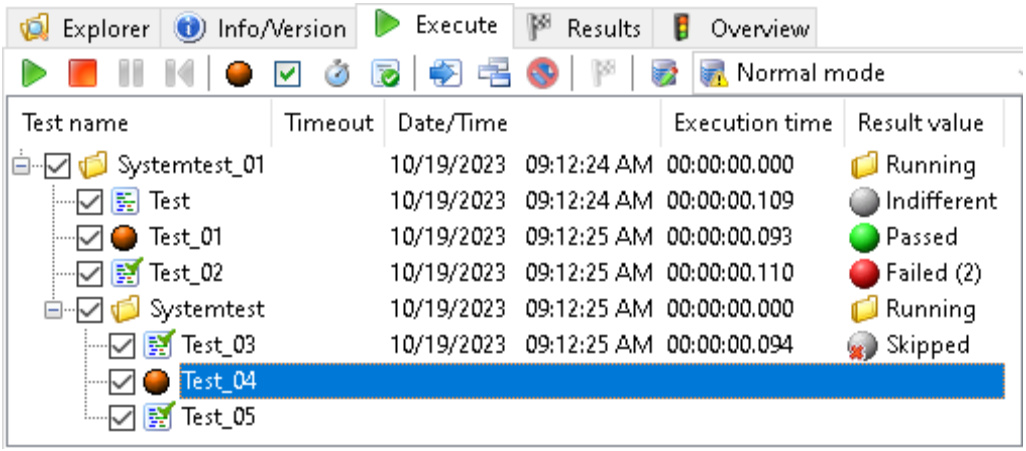


Figure 342 – Setting breakpoints

You can continue the test run by clicking on . Remove the breakpoint by clicking on the elementary test with the breakpoint and then clicking on again.

Breakpoints (and skipping elementary tests and timeout values) are stored during the current PROVEtech:TA session. If you work in Sequencer mode, then you can remotely control breakpoints via the Sequencer Client window (see chapter ***5.14.2 Working with the remote test sequencer***).



When execution stops at a breakpoint, it is not possible to debug further execution in the Basic Editor. Breakpoints in the tab `Execute` only control the test sequencing.

5.5.3 Skipping test objects

You can mark tests which should be skipped while execution. Therefore select the test and click at in the toolbar. You can also uncheck the test object.

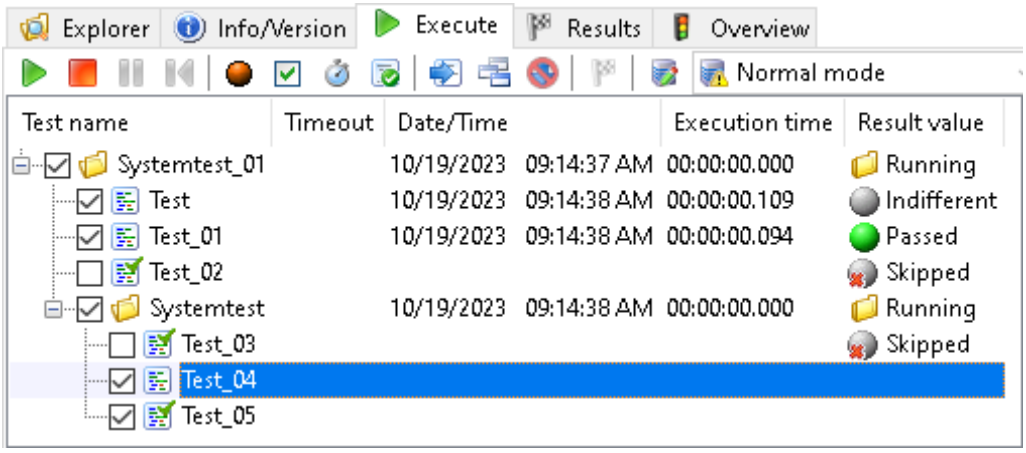


Figure 343 – Skipping elementary tests

During execution you can also skip one or more test objects with selection and breakpoint. To do this, click on the elementary test with which you want to start execution. Then click on ▶. All elementary tests before the selected one will be skipped.

Proceed as follows if you wish to skip an elementary test that is in the middle of the list:

1. Set a breakpoint in front of this elementary test using ●.
2. Click on the elementary test with which you want to begin the execution.
3. Start running it by clicking on ▶. Execution will stop at the breakpoint.
4. Select the next elementary test to be executed. Start to run it by clicking on ▶. The elementary tests from the breakpoint to the selected elementary test will be skipped.

The currently skipped elementary tests (and also breakpoints and timeout values) are stored during the current PROVEtech:TA session. If you work in Sequencer mode, then you can remotely control skipping the elementary tests via the Sequencer Client window (see chapter ***5.14.2 Working with the remote test sequencer***).

5.5.4 The timeout value

If you have unintentionally programmed infinite loops in your Basic script, or the call of a command hangs, or if the expected execution time is exceeded by far for some other reason, the entire execution plan may be blocked. This can be very annoying when executing tests unsupervised over night or over the weekend. For this purpose, you can set timeout limits for each of the test objects. If the timeout value is exceeded during execution, the test run is aborted before processing the next command. In case of the Remote Sequencer or when executing a system test or project, the next elementary test is then executed automatically.

If a Basic command is blocked (i.e. does not return) during the exceeding of a time limit or if a dialog or a message box is currently open, the test cannot be aborted during this time.

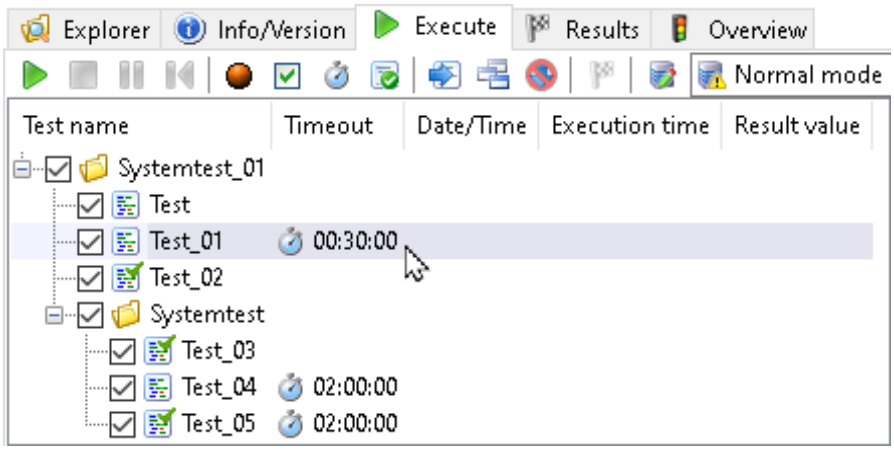


Figure 344 – Test objects with timeout values for execution

If the test has not yet been executed, you can enter timeout values either directly in the control element, or you select the test objects that you want to change and open the following dialog by context menu or the toolbar button .

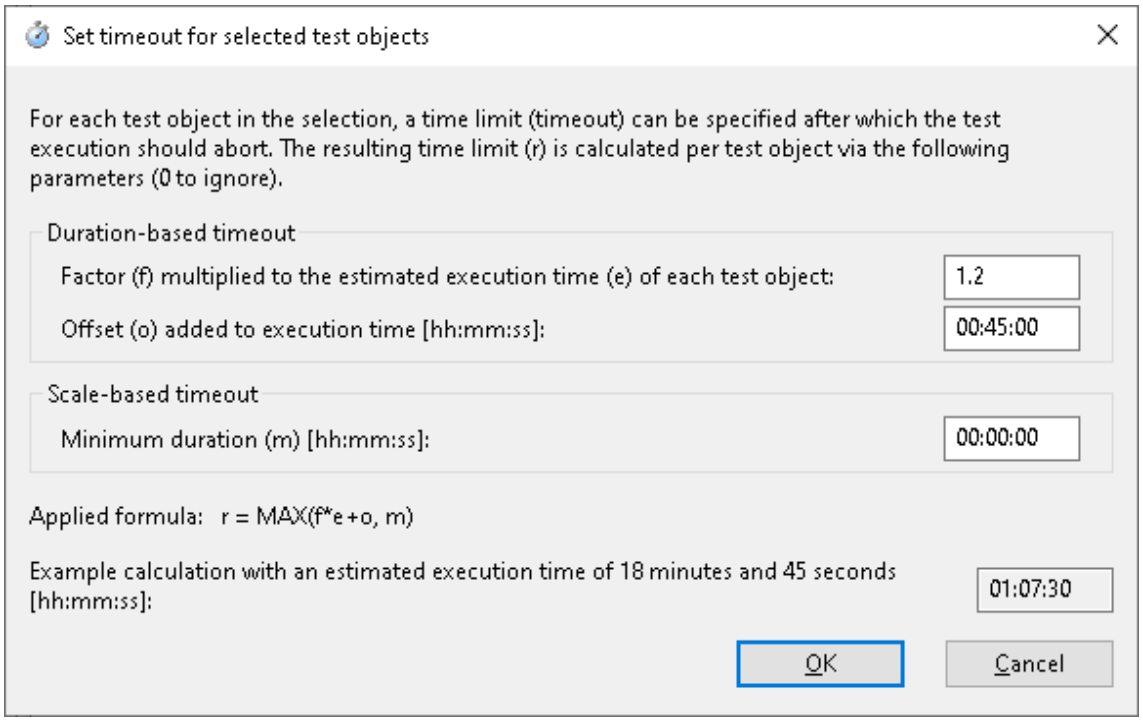


Figure 345 – Dialog for setting the timeout values

The timeout values (and also skipping elementary tests and breakpoints) are stored during the current PROVEtech:TA session. If you work in Sequencer mode, then you can remotely control the timeout values via the Sequencer Client window (see chapter ***5.14.2 Working with the remote test sequencer***).

5.5.5 Pre-process the next test object for execution

In the Settings dialog, you can specify a Test Manager path to a Basic handler that is called before each execution of the next test object (see chapter ***3.2.7.5 Test Manager***). Here you can modify the test object, or you can decide whether to skip the execution of the test.

This Basic handler is called for each test object in the execution list even if the check mark for execution in the Execute tab has already been removed. So, you can always overwrite the settings in the dialog.

This Basic handler is not called when the test is executed in the Basic debugger.

The following is an example source code. The interface is predefined. The content of the function can be freely designed.

```
'#Language "WWB.NET"
Option UsesAmbiguousError On
Option Explicit

'Callback function which is called before each execution of the next
test object
'[in] ti: test object to be executed
'[out] bSkip: True if execution is to be skipped
Sub ControlExecution(ByVal ti As TestItem, ByRef bSkip As Boolean)

    'MsgBox ti.Path, "ControlExecution" 'For debugging purpose

    'Skip execution if there are already test results
    bSkip=(ti.Type=TestItemType.titTest And Not ti.GetResults.IsEmpty)
End Sub
```

Also read chapter ***3.2.7.2.1 Event macros*** on how to react automatically to certain events.

5.5.6 Post-process the test result after execution

In the Settings dialog, you can specify a Test Manager path to a Basic handler that is called after each execution of an elementary test object (see chapter ***3.2.7.5 Test Manager***). This means that the Basic handler is called after the result status is set for the test result. Here you can still post-process the test result or perform another action.

If no test result is generated, this handler is not called.

This Basic handler is not called when the test is executed in the Basic debugger.

The following is an example source code. The interface is predefined. The content of the function can be freely designed.

```
'#Language "WWB.NET"
Option UsesAmbiguousError On
Option Explicit

'Callback function which is called after each execution of a test ob-
ject
'[in] tr: Generated test result
Sub AfterExecution(ByVal tr As TestResult)

    'Example for debugging purpose: Display message box
    MsgBox(tr.Path(True), VbMsgBoxStyle.vbOkOnly, "AfterExecution")
End Sub
```

Also read chapter ***3.2.7.2.1 Event macros*** on how to react automatically to certain events.

5.6 Test results

Once you have executed elementary tests, system tests or projects, you can view, analyze, validate and comment the test results. You can, of course, also display and manage the test results of older test runs.

During the test execution you can add and store many kinds of information in the test result. Once a test result is finished after the test run, you can only modify certain properties of the test result. These are renaming the rest result, modify comments, validating the test result, overwriting the parameters and add/remove attachments afterwards.

Below you can read how to display the results of an elementary test or system test immediately after it has been run and how to note information on the test in the test protocol.

Which are the possibilities to store information at the test result?

- Setting a result value and an additional result value during the test execution (see ***5.6.1 Displaying the test result in a dialog***)
- Setting a name and a comment (see ***5.6.2 The Comment***)
- Validation of the test result (see ***5.6.3.1 The test protocol of an elementary test***)
- Creating a test protocol including sub protocols (in .html format or .xml format) during the test execution (see ***5.6.3 The test protocol***)
- Adding a file as an attachment (see ***5.6.9 Attachments***)
- Adding a data acquisition (see ***5.6.9 Attachments*** and ***3.4 The Data Acquisition menu***)
- Logging of ECU diagnostic trouble codes during the test execution (see ***5.6.6 ECU errors***)
- Adding keyword values during the test execution (see ***5.6.7 Keyword values***)
- Adding a parameter set during and after the test execution (see ***5.6.8 Parameters***)

In chapter ***5.3 Managing tests*** you can find possibilities to store information at the test object.

If you want to control the entire list of test result history, please read in chapter ***5.7.1 Displaying the list of test results***.

5.6.1 Displaying the test result in a dialog

After executing a system test, a project, or an elementary test, you can view the test result in two different ways.
Either in the `Execute` tab by double-clicking on a test object or via the context menu `Test result` (see ***Figure 346*** and ***Figure 347***).
Or in the `Results` tab in the `Test Protocol` sub-tab (see ***Figure 348*** and chapter ***5.7.1 Displaying the list of test results***).

Test results in the `Execute` tab:

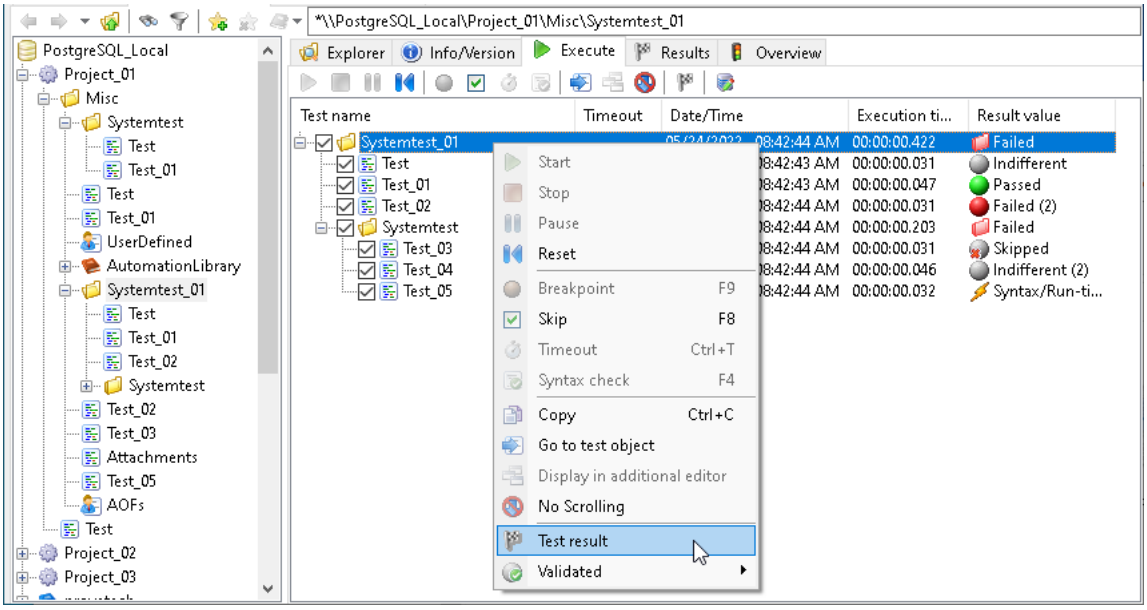


Figure 346 – Display the test result by context menu after executing a system test

The context menu item `Test result` opens the following dialog.

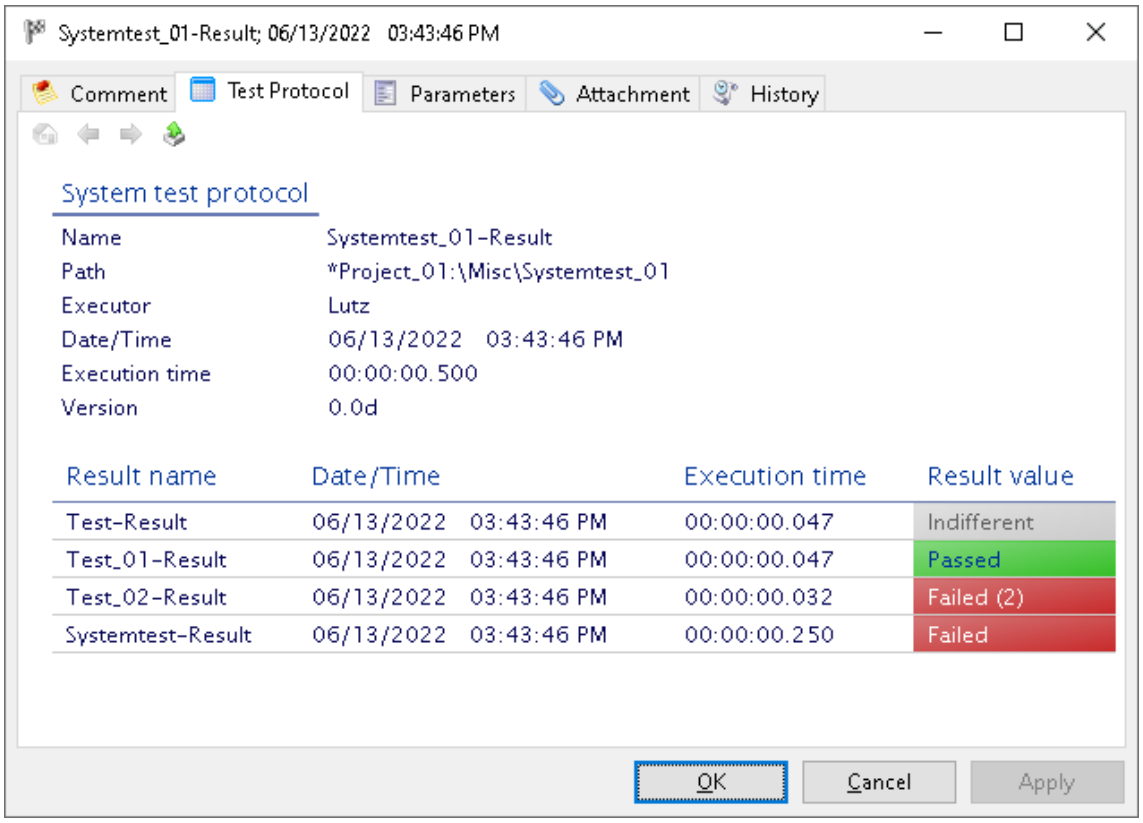


Figure 347 – The system test result

Test results in the `Results` tab:

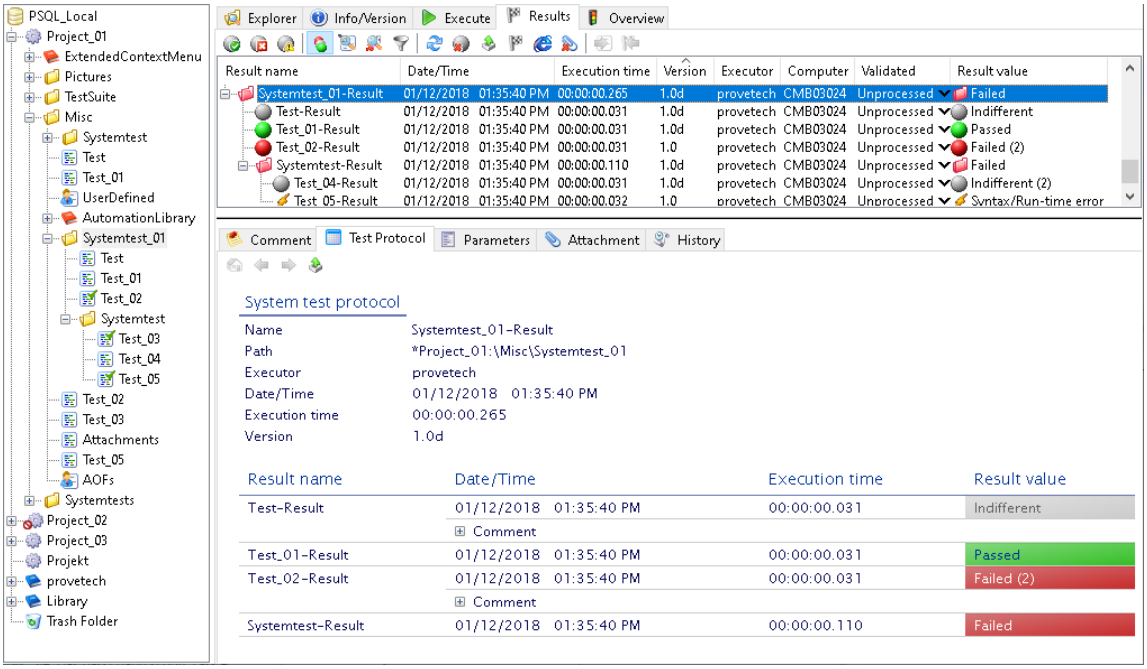


Figure 348 – Display the test results in the "Results" tab

If you want to view the results of an individual elementary test after executing a system test, mark it with the left mouse button in the list of elementary tests. Then click on `Test result` or double-click the elementary test. The following dialog is opened:

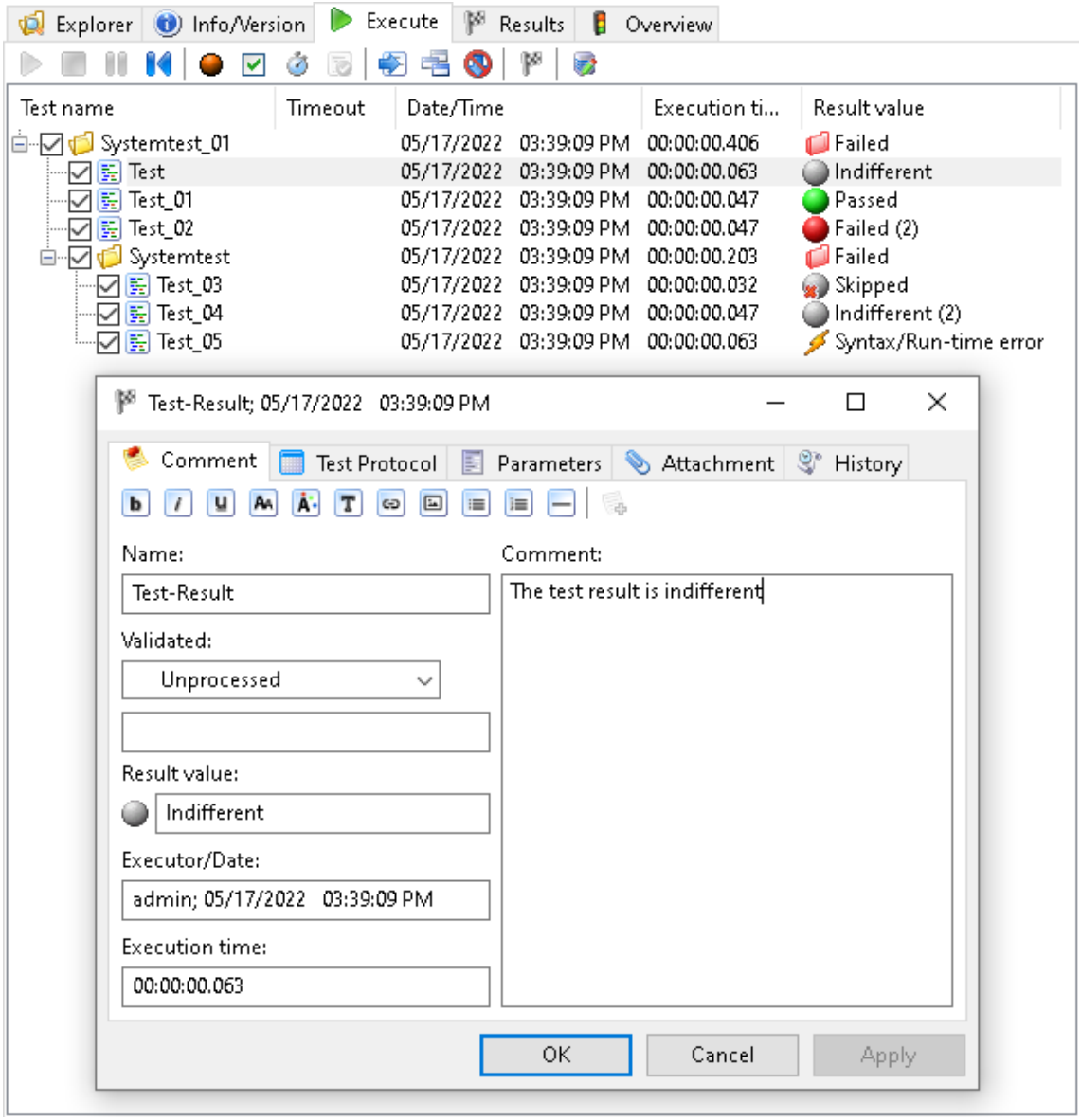


Figure 349 – The elementary test result

5.6.2 The Comment

In this property page you can modify the name, the “Validated” property and the comment of the test result if you are authorized to make modifications. Further on, this page displays the result value that is set by the test program while execution. The executor is

the user who was logged on while execution, and the date corresponds to the timestamp when the execution was finished (timestamp of the database server). If the test is still running, the start time is displayed. The execution time is displayed exact on milliseconds. The computer indicates on which computer PROVEtech:TA was running and the test script was executed. The default name of the test result corresponds to the test object's name.

In the Comment field you can enter peculiarities and details about the results. In this field you can note everything that is characteristic or unusual for this test result or mention any observations while it was running.
The comment text can be stored as formatted text based on HTML. For this purpose, some buttons for text formatting have been added to the user interface.
More detailed information on the use of the HTML-enabled description text can be found in chapter ***5.3.9.1.1 The HTML Text Edit control***.

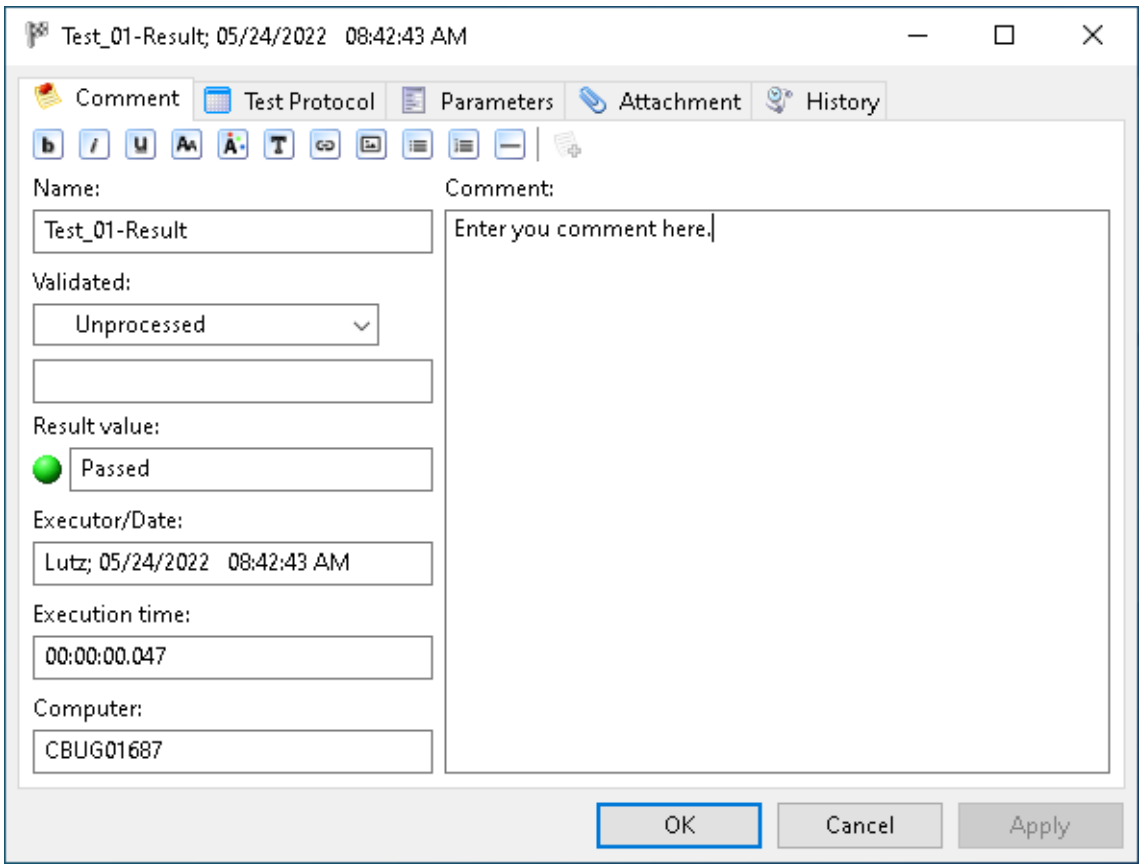


Figure 350 – The comment of a test result

In the combo box inside the toolbar you can set the "Validated" status of the test result (see chapter ***5.7.5 Validating test results***).

If the database-global setting `Append test result comments only` is set (see chapter ***5.3.11.2 The tab "Database settings" in the property dialog***), you can only add a comment to the list of already existing test result comments. However, the

administrator or a user with the 'arManage' access right to the root object can always edit the complete comment, at least via the user interface.

For users without the 'arManage' access right, there is only one button on the user interface to add a new comment instead of the text formatting buttons.

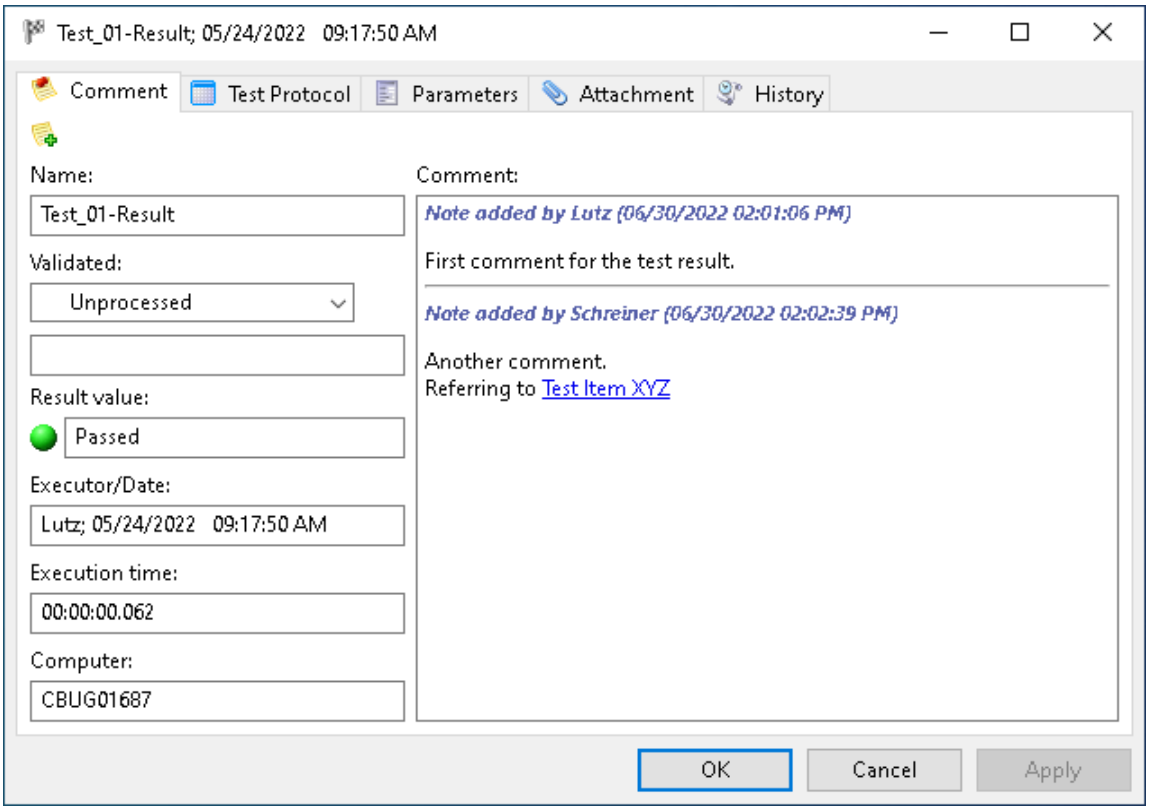


Figure 351 – Add a new comment

By clicking on the `Add comment` button, a separate editing dialog appears in which all text formatting options are available.

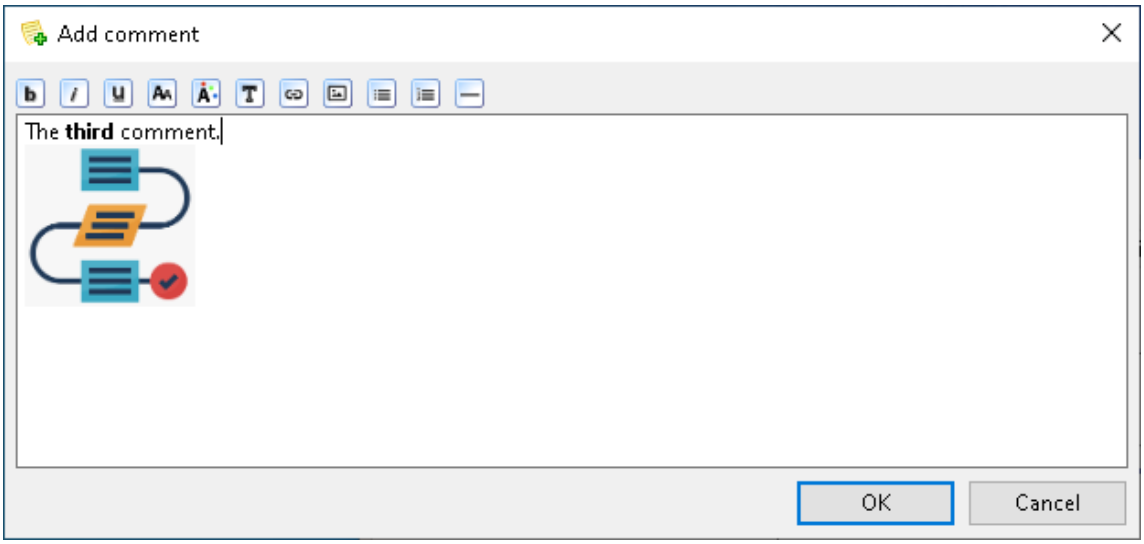


Figure 352 – Comment editing dialog

After confirmation, the newly added comment appears in the Comment field. As long as this is not also confirmed with the `Ok` or `Apply` button, the new comment can still be edited with the upper button `Edit current comment` (this now has a slightly different symbol than before). The newly added comment is only transferred to the database after confirmation in the Comment field.

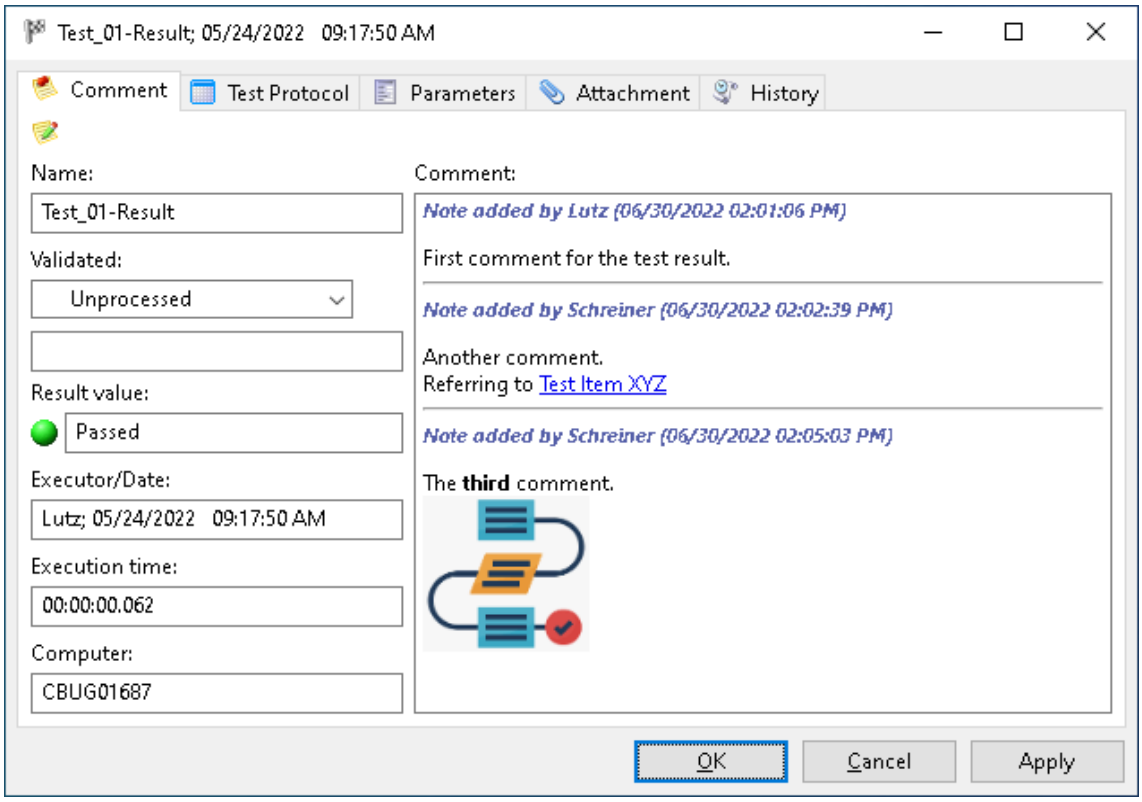


Figure 353 – Newly added comment

If the global setting `Append test result comments only` is active for the test result, all comments entered are saved in the database as HTML-formatted text. This means that some characters are stored in the database in modified form. If the user wants to search for these characters, e.g. in the search dialog or in a test script, this modified form must be taken into account.

Affected characters are:



<	<
>	>
&	&
‘	'
“	"
Linebreaks \n\r	
Blank spaces	

5.6.3 The test protocol

Click on the `Test Protocol` tab to display the protocol of a test result. In fact, a test protocol is some kind of report of a test run. The test protocols are stored in the database in a compressed format. When displaying they are uncompressed and temporarily stored in the local file system. The following chapters explain the elementary test protocol, system test protocol and project test protocol and the navigation between the test results and sub protocols.



You cannot manipulate the test protocol of an elementary test after the test execution has finished.

PROVEtech:TA creates a system test protocol or project test protocol dynamically when displaying it in the tab.

5.6.3.1 The test protocol of an elementary test

The test script is responsible for the content of the test protocol. The protocol represents the progress of the test program execution. Each **System.Remark** statement in the test program generates an entry in the test protocol.

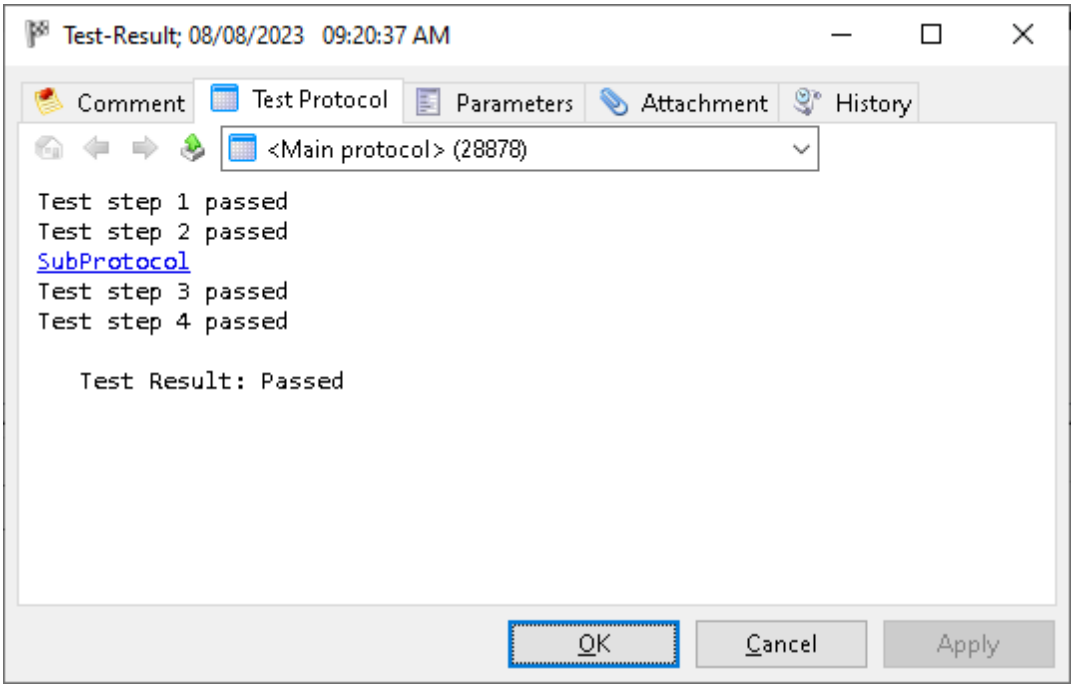


Figure 354 – The test protocol of an elementary test

You can design your test protocols as sophisticated as you want. Please read more about that in chapter ***5.6.3.3 Writing test protocols in HTML format***, chapter ***5.6.3.5 Writing test protocols in XML and formatting via XSL-T*** and chapter ***5.6.3.6 Test protocols formatting using post-processing***.

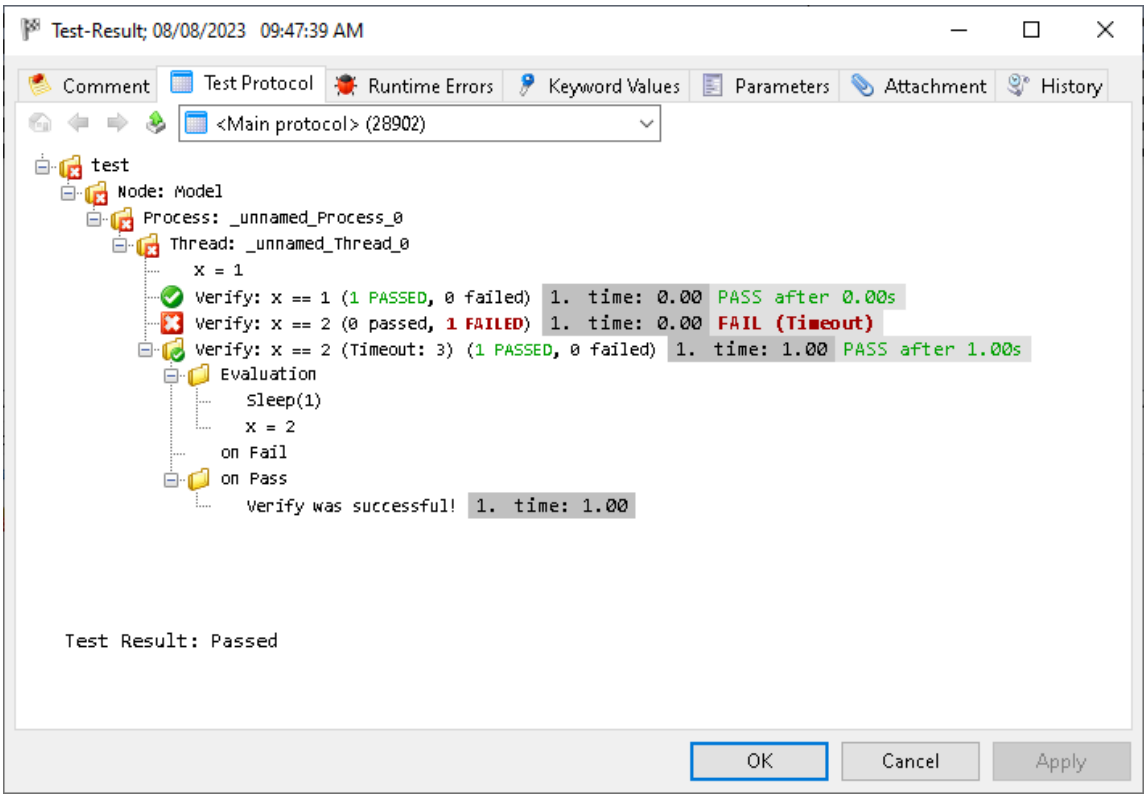


Figure 355 – The RTAE test protocol (real-time protocol) of an elementary test in HTML format including Java scripts

With the Basic command **System.ChangeProtocol** you can create a test protocol with included sub protocols. Then, in the tab `Test Protocol`, you can navigate between the different protocols like you navigate in a web browser (see ***Figure 356***). Additionally for the navigation, you have got the buttons  ,  ,  and the combo box containing the list of sub protocols.

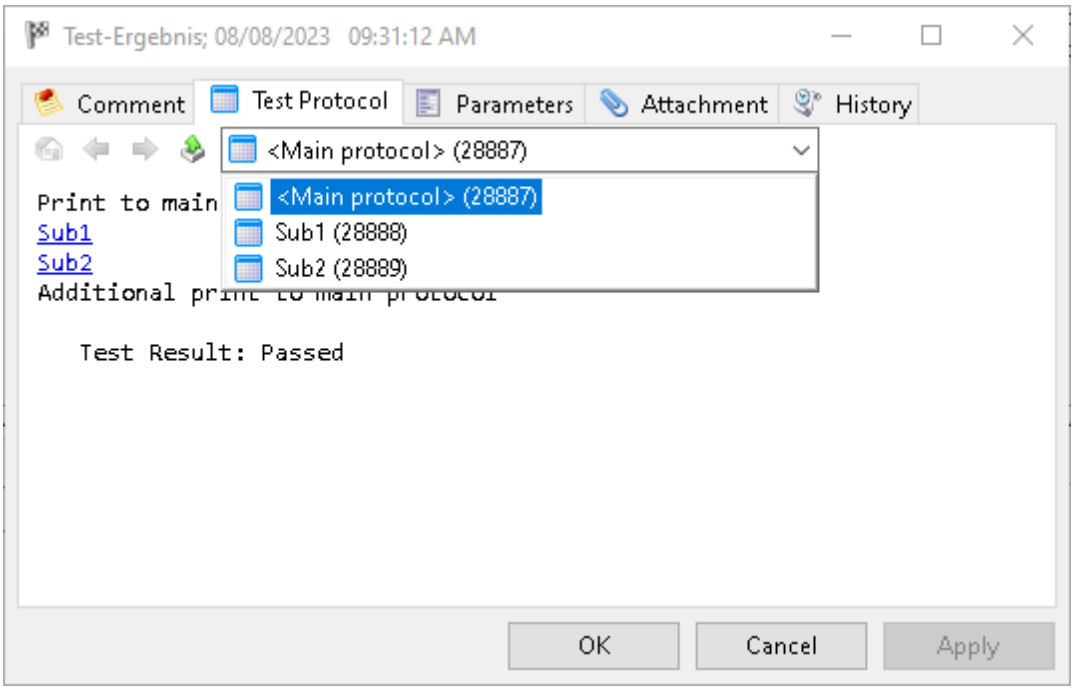


Figure 356 – Navigation in the test protocol

When you click on the button you can export the main protocol and the associated sub protocols and sub results to HTML, MHTML or XML file format, depending on the format in which the test protocols are stored in the database. A conversion between different formats is not possible.

The sub protocols are a flat set of objects without hierarchical structure. The main protocol is the entry point to this set.

5.6.3.2 The test protocol of a system test or a project

In the protocol of a system test result or a project test result you see the table of elementary test results and their result value.

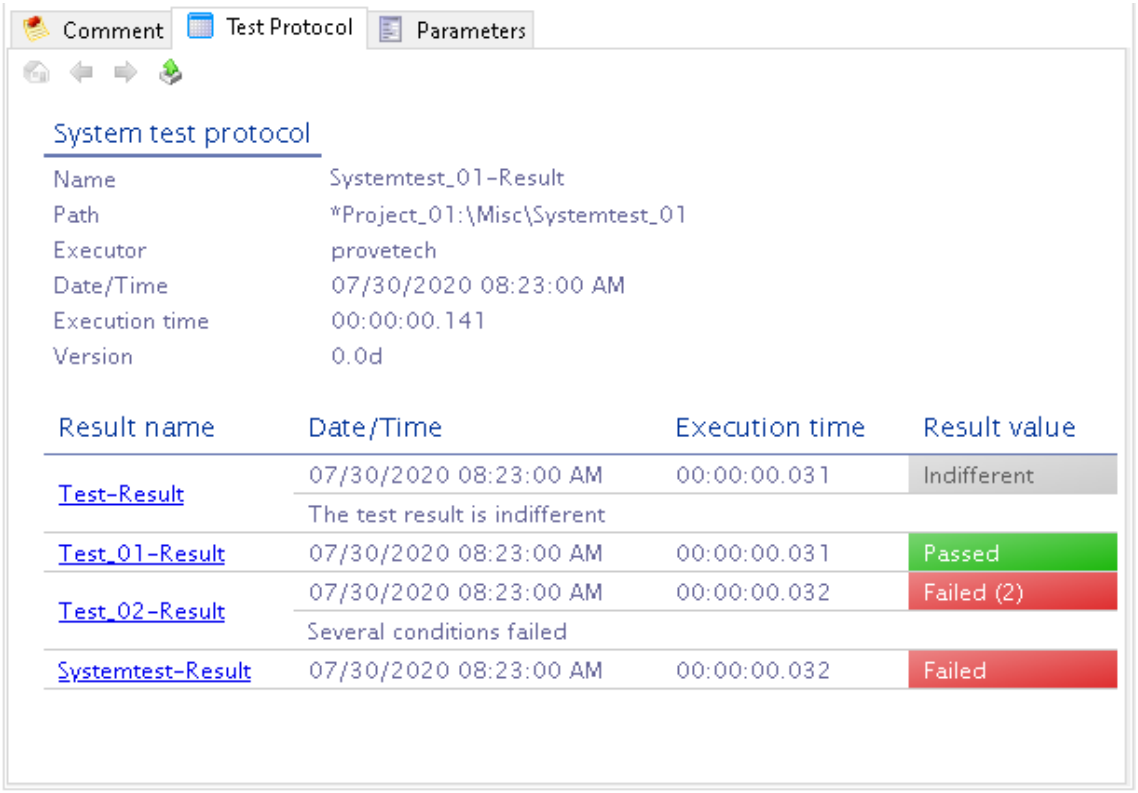


Figure 357 – The test protocol of a system test

This property page shows you on which day and time an elementary test was executed within the system test (timestamp when the test run was finished) and whether the execution was successful. You can also see test result comments if not empty. If you have complex tests to execute, it allows you to see quickly whether a certain elementary test was successful or not.

You can navigate through the single elementary tests like in a web browser by clicking the hyperlinks. For navigation you can also click the buttons , and . If you want to export the test protocols associated to the test result in HTML or MHTML format (depending on the format in which the test protocols are stored in the database), then click on the button (see ***Figure 357***). A conversion between different formats is not possible.

The upper table shows general information about the selected system test protocol. The name of the system test, the Test Manager path, the name of the executor, date and time of the end of the system test, the total execution time and the version of the system test.

The table below contains information about the results of each elementary test. The name of the elementary test is given as a link to navigate directly to the test protocol of the elementary test (see ***5.6.3.1 The test protocol of an elementary test***). The second column shows the date and time of the elementary test end. Any comments of the elementary test result are also displayed here (see ***5.6.2 The Comment***). The third column shows the execution time of the elementary tests. The fourth column shows the color-

coded test result of the individual elementary tests and the value of the specified additional optional result value (see ***System.SetResult*** in the Test Language manual).



You cannot post-process a system test protocol by a WinWrap Basic handler in PROVEtech:TA (in contrast to an elementary test protocol) (see ***5.6.3.6 Test protocols formatting using post-processing***).



System test protocols are generated dynamically according to the sub ordinary elementary test results. So if you delete an elementary test result, this line is also removed from the list inside the system test protocol.



When exporting system test protocols or elementary test protocols to the MHTML file format, please note that when viewing the MHTML files in external web browsers, navigation will probably be restricted or not possible at all. The reason for this is that according to modern standards the MHTML format is classified as insecure, and modern web browsers prevent the navigation. Internet Explorer does not implement this restriction, but it is increasingly being removed from common operating systems. If a modern web browser supports the so-called "Internet Explorer Mode", MHTML navigation can possibly be restored.

5.6.3.3 Writing test protocols in HTML format

Internally, the test protocols are stored HTML-formatted and they all have the html-tag <pre> at the beginning. But you also can directly create test protocols in HTML format. Only set the HTML tag </pre> first. The internal handling is that the <pre>-Tag is deleted at the beginning of the protocol.

An example follows which you can copy to the Basic editor:

```
'#Language "WWB-COM"
Option Explicit

Sub Main
    System.Remark "</pre>", False
    System.Remark "<FONT COLOR=""#FF0000"">Hello</FONT><BR>"
    System.Remark "<FONT COLOR=""#00FF00"">Hello</FONT><BR>"
    System.Remark "<FONT COLOR=""#0000FF"">Hello</FONT><BR>"
End Sub
```

Attention: If you want to design your test protocol in html, you should make sure that the protocols are still readable in Dark Mode operation on different PROVEtech:TA installations.

Adding images to HTML protocols

In HTML an image is typically referenced as an external file by its paths. This is either a local file path or a path on the HTTP server. PROVEtech:TA stores the HTML protocols in the database and exports them temporarily to the file system for displaying them in the embedded browser. The browser cannot access the database directly. There are two possibilities for embedding. The first solution is using the Test Automation references (see ***5.6.3.4 Test Automation references***). The other alternative is embedding the images directly in the HTML string. The following example shows how to read an image from the file system and embed it in the HTML string:

```
'#Language "WWB-COM"
Option Explicit

Sub Main
    Dim uaFile() As Byte

    'copy file "C:\TA\test.png" to byte array
    Open "C:\TA\test.png" For Binary Access Read As #1
    ReDim uaFile(LOF(1)-1)
    Get #1, , uaFile
    Close #1

    'output beginning of HTML protocol with embedded image reference
    System.Remark("</pre><html><body><img src=""", _
        False)
    'output image Base64 encoded
    System.Remark(Encode64B(uaFile), False)
    'output end of HTML protocol
    System.Remark("""></body>", False)
    'set result and prevent output of result to protocol
    System.SetResult(trvPass, , False)
End Sub
```

5.6.3.4 Test Automation references

The Test Automation reference is a URI schema ("**taref:**") which allows to embed the following items inside a given test protocol:

- Attachments associated to a test result
- Test documents associated to a test object
- Other test protocols
- Files located within a .hdr measurement file

HTML Tags

Any reference to an attachment, a test document, a test object or a test protocol must be preceded by the prefix taref:*. The Test Automation references can be used in all common HTML elements. Here are some examples:

HTML Element	HTML tag with Test Automation Reference
Stylesheets (CSS)	`<link rel="stylesheet" type="text/css" href="taref:**doc*someS`
JavaScripts	`<script type="text/javascript" src="taref:**att*someJavaScript`

Anchor links	`<a href="taref:**doc*someBitmap.bmp">Link text</a>`
	
	`<a href="TestAutomation:Path=*MyProject:\Test_01&Action=Se`
	The above `&` represents an HTML-encoded `&` character – so the path is
	`TestAutomation:Path=*MyProject:\Test_01&Action=Select`
	
	`<a href="TestAutomation:Path=*MyProject:\Test_01%26Action=Sele`
	The above `%26` represents an HTML-encoded `&` character – so the path is
	`TestAutomation:Path=*MyProject:\Test_01&Action=Select`
	See below: **Special characters in HTML references**

Images	`<img src="taref:**doc*Image.jpg" />`
Videos	`<video controls><source src="taref:**att*Video.mp4" type=""vid`

Embedded files of an .hdr measurement file

The Test Automation references can also be used to directly access embedded files within an .hdr measurement file. For this purpose, the following must be taken into account when specifying the path:
The embedded file is identified with a prefix string which corresponds to the filename of the .hdr file.

Example:
In an .hdr file with the name **MyMeasurement.hdr**, the embedded file **Video_1.mp4** is to be referenced.
The path to be used for a Test Automation reference must therefore be:
MyMeasurement.Video_1.mp4

This follows the same scheme that an .hdr file uses to handle its sub-files (see ***Administrator's Guide***).

Attention:
This naming scheme can lead to ambiguities under certain circumstances!
If, for example, there is a measurement file on the one hand and a second file on the other hand in the test object attachments or documents, where the measurement file

contains an embedded file of the same name, these files may possibly have the same Test Automation reference path. A clear distinction is then not possible. This is reported by PROVEtech:TA with a warning.
The following figure shows this case.

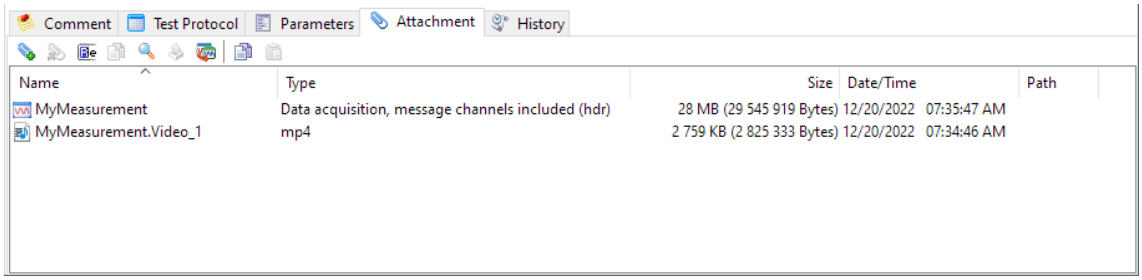


Figure 358 – Possible identical names for measurement files

Relative and absolute references to Test Manager objects

Test Automation Reference	Description
`taref:**att*Image.jpg`	Attachment associated to the
`taref:**doc*Image.gif`	Test document associated to
`taref:**res*20180601T120102Z`	Test protocol of a sibling test
`taref:*..\*doc*someJavaScript.js`	Test document associated to
`taref:*..\Sibling*doc*someStyleSheet.css`	Test document associated to
`taref:*..\Sibling*res*20180101T121212Z`	Test protocol of the sibling te
`taref:*..\Sibling*res*20180101T121212Z*att*Image.jpg`	Attachment associated to the test object
`taref:*..\..\*doc*Image.png`	Test document associated to
`taref:*...\*doc*Image.png`	Equivalent to `taref:*..\..`
`taref:*..\..\..\*doc*Image2.bmp`	Test document associated to object
`taref:*....\*doc*Image2.bmp`	Equivalent to `taref:*..\..`
`taref:*...\..\*doc*Image2.bmp`	Equivalent to `taref:*..\..` and `taref:*....\*doc*log`
`taref:*user:\Path\Test*doc*Image.jpg`	Absolute reference to a test d
`taref:*user:\Path\Test*res*20180101T121212Z`	Absolute reference to the ma
`taref:*user:\Path\Test*res*20180101T121212Z*att*Image.gif`	Absolute reference to an atta

`taref:**att*MeasurementFile.someVideo.mp4`	Embedded file “someVideo.mp4”, the current test result.

For using absolute references, the user can use the copy feature on the corresponding object (test protocol, test document or attachment) and paste the reference in the test script.

Special characters in HTML references

If the test protocol references contain certain special characters that are interpreted in a special way by the used browser, these special characters must be entered in the HTML protocol in the so-called **percentage encoding**. Otherwise, the browser will for example try to access a wrong path during navigation. PROVEtech:TA cannot convert these characters automatically, as the special characters that appear depend on the context that is currently being used.

For example, a path of this form

```
<a href="TestAutomation:Path=*MyProject:\Test01_(100%)">Test object</a>
```

must be converted to a percentage encoded path of this form:

```
<a href="TestAutomation:Path=*MyProject:\Test01_%28100%25%29">Test
object</a>
```

The following table lists all special characters that must be transferred to the percentage encoding by the user.

Special character	Percentage encoding
<space character>	%20
!	%21
#	%23
$	%24
%	%25
&	%26
‘	%27
(	%28
)	%29
*	%2A
+	%2B
,	%2C
/	%2F
:	%3A
;	%3B

=	%3D
?	%3F
@	%40
[	%5B
]	%5D

Alternatively, the .NET function `System.Net.WebUtility.UrlEncode()` can be used for automatic percentage coding. However, the colon after “taref” must stay. Example:

```
'#Language "WWB.NET"

Imports System.Net

Sub Main
    System.Remark "<img src=""taref:"+ _
        WebUtility.UrlEncode("*Project3:\HilTests\FullTest100%*doc*"+ _
        "ImageNameWithSpecialChars[100%]@&.png")+""">"
End Sub
```



Please avoid special characters in Test Manager paths, especially the # symbol, because they may have a special meaning in the web browser. For example, the # symbol is used as an “anchor”.

The & character could also be critical, as this is used as a separator for TestAutomation URIs.

For other special characters, the percentage encoding can be used instead, as described in the chapter above.

The used browser for displaying the protocols (Internet Explorer or Edge based on Chromium) might have issues detecting the correct character encoding for referenced text documents like JavaScript or cascaded style sheets and the different browsers might behave in a different way. Edge based on Chromium assumes that the referenced text uses the same encoding as the document referencing it. The PROVEtech:TA result protocols typically use the UTF-16 character encoding (Little Endian). To avoid any issues, the best way of handling this is specifying the character encoding of the referenced document. So, as an example if the referenced document uses UTF-8 encoding, you can specify it the following way for cascaded style sheets in HTML:

```
<link rel="stylesheet" type="text/css" charset="UTF-8"
href="taref:**att*someStyleSheet.css">
```

JavaScript in HTML:

```
<script type="text/javascript" charset="UTF-8"
src="taref:**att*someJavaScript.js"></script>
```

XSL in XML:

```
<?xml-stylesheet type="text/xsl" charset="UTF-8"
href="taref:**att*someXsl.xsl"?>
```

Example

```
Sub Main
    'Enables the test result output in HTML format
    System.Remark "</pre><!DOCTYPE html>"
    System.Remark "<html><head>"
    'Stylesheet
    'Add the "my.css" stylesheet test document associated to the cur-
rent test object
    System.Remark "<link rel=""stylesheet"" type=""text/css""
href=""taref:**doc*my.css"">"
    'JavaScript
    'Add the "my.js" JavaScript attachment associated to the current
test result
    System.Remark "<script type=""text/javascript""
src=""taref:**att*my.js""></script>"
    System.Remark "</head><body>"

    'JavaScript execution
    'Executes a JavaScript function defined in "my.js"
    System.Remark "<script>MyJavaScriptFunction(20,17);</script>"


    'Links
    'Add an anchor to an attachment of the current test result
    System.Remark "<a href=""taref:**att*myimage.jpg"">Link to myimage
in attachments</a>"
    'Add an anchor to a test document of the current test object
    System.Remark "<a href=""taref:**doc*mytext.txt"">Link to mytext in
test documents</a>"
    'Add an anchor to a test object
```

```
    System.Remark "<a
href=""TestAutomation:Path=*MyProject:\Test_01&amp;Action=Select&amp;Li
ne=1"">Link to a test object</a>"

    'Images
    'Add a relative attachment reference to the current test result
    System.Remark "<img src=""taref:**att*myimage.jpg""/>"
    'Add an absolute attachment reference (equivalent to the previous
taref)
    System.Remark "<img
src=""taref:*user:\Path\To\Test*res*20180101T120000Z*att*myimage.jpg""/
>"
    'Add a relative test document reference to the current test object
    System.Remark "<img src=""taref:**doc*mylogo.jpg""/>"
    'Add an absolute document reference (equivalent to the previous
taref)
    System.Remark "<img
src=""taref:*user:\Path\To\Test*doc*mylogo.jpg""/>"
    'Add a relative test document reference to the parent test object
of the current test object
    System.Remark "<img src=""taref:*..\*doc*myotherlogo.jpg""/>"
    'Add a relative test document reference to the sibling test object
of the current test object
    System.Remark "<img src=""taref:*..\Sibling*doc*mygraphic.png""/>"
    'Add a relative document reference to the grandparent test object
of the current test object
    'Note: The following two references are equivalent
    System.Remark "<img src=""taref:*..\..\*doc*otherimg.jpg""/>"
    System.Remark "<img src=""taref:*...\*doc*otherimg.jpg""/>"

    System.Remark "</body></html>"
End Sub
```

5.6.3.5 Writing test protocols in XML and formatting via XSL-T

Internally, the test protocols are stored HTML-formatted and they all have the html-tag <pre> at the beginning. But you also can create test protocols in XML format. Only set the HTML tag </pre> first, followed by <?xml. The internal handling is that the <pre>-Tag is deleted at the beginning of the protocol.

An example follows which you can copy to the Basic editor:

```
'#Language "WWB-COM"

Option Explicit

Sub Main

    System.Remark "</pre>", False
    System.Remark "<?xml version=""1.0"" encoding=""UTF-16""?>"
    System.Remark "<?xml-stylesheet type=""text/xsl"" charset=""UTF-16"" ",
False
    System.Remark "href=""signal_table.xsl"" ?>"

    System.Remark "<data_acquisition>"
    System.Remark "<signal><name>a</name><value>2</value></signal>"
```

```
    System.Remark "<signal><name>b</name><value>3</value></signal>"
    System.Remark "<signal><name>c</name><value>4</value></signal>"
    System.Remark "</data_acquisition>"

    System.SetResult trvPass, 0, False
End Sub
```

For the graphical formatting of your XML protocol you can now use XSL-Transformation (XSL-T, also named XSLT). The corresponding .xsl file (and image files if needed) are registered in the `Additional protocol files` box of the `Settings` dialog (see also ***3.2.7.5 Test Manager*** in the Settings dialog).

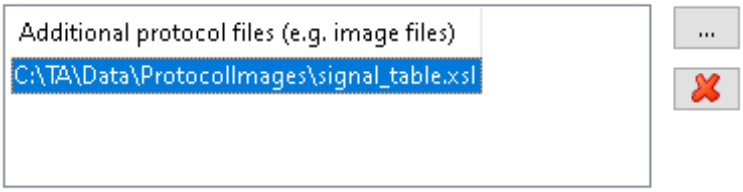


Figure 359 – Additional protocol files

Here you can see the content of the signal_table.xsl file (in UTF-16) that is responsible for the formatting of the XML protocol.

```
<?xml version="1.0" encoding="utf-16"?>
<xsl:stylesheet version="1.0"
xmlns:xsl="http://www.w3.org/1999/XSL/Transform">

<xsl:template match="/">
  <html>
  <body>
  <h2>Data Acquisition</h2>
    <table border="1">
      <tr bgcolor="#9acd32">
        <th>Signal</th>
        <th>Value</th>
      </tr>
      <xsl:for-each select="data_acquisition/signal">
      <tr>
        <td><xsl:value-of select="name"/></td>
        <td><xsl:value-of select="value"/></td>
      </tr>
      </xsl:for-each>
    </table>
  </body>
  </html>
</xsl:template>
</xsl:stylesheet>
```

The fact that this file is stored centrally provides the opportunity for all XML-defined protocols to change the formatting in one file. As normal, the XML protocol is displayed in the property page `Test Protocol` of the test results.

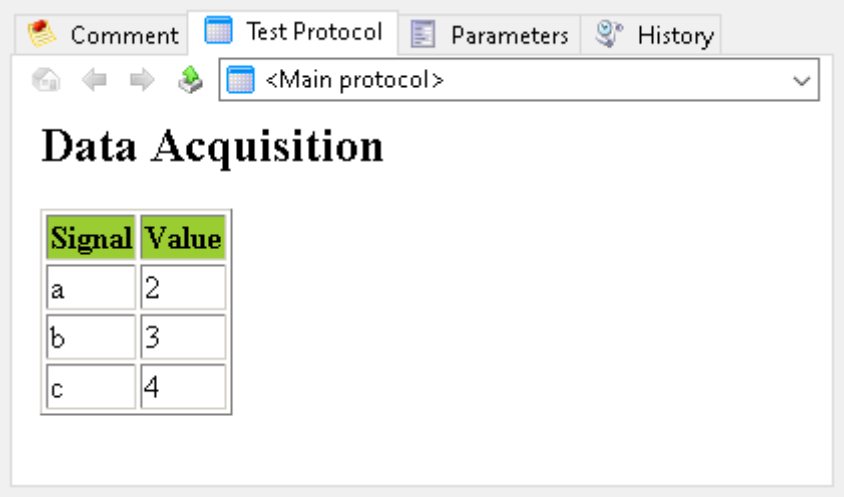


Figure 360 – XML protocol, formatted via XSL-T

In the XML header you specify the “encoding”. In order to create a correct XML protocol, the characters passed to “System.Remark” must use this encoding. The test protocols are stored in UTF-16 format.

The correct encoding must also be specified in the XSL-T stylesheet definition. The Microsoft Edge Chromium Browser could interpret the stylesheet with incorrect encoding if this definition is missing. To do this, specify the parameter `charset="UTF-16"` within the `<?xml-stylesheet >` tag.

If there are syntax errors in the XSL-T (e.g., unclosed <tags> or misspelled keywords), the Microsoft Edge Chromium Browser is currently unable to report these correctly to the user. In this case, the displayed result is a completely empty, white page. If you see an empty page as a result, please check the XSL-T file for errors.

5.6.3.6 Test protocols formatting using post-processing

Test protocol post-processing allows test protocols to be analyzed and formatted for the desired output. To use post-processing, a separate test script must be created, which implements the following function, with this signature:

```
' Callback function which performs a post-processing before the protocol is
displayed
' [in] tp   Test protocol that should be processed
' [ret]     Protocol string that is stored as a .html file;
'           empty string if there should not be a special processing
Function ProcessTestProtocol(tp As TestProtocol) As String
```

The post-processing path to the Test Manager object should be defined in the `Test Manager` tab of the *Settings...* dialog (see ***3.2.7.5 Test Manager***).

If the path for post-processing of test protocols is defined in the Test Manager settings, post-processing is applied to all test protocols. This also applies to the export of test protocols (see ***5.6.3.1 The test protocol of an elementary test***).

If post-processing is activated, the output of the `ProcessTestProtocol` function completely replaces the contents of the initial test protocol. This means in particular that if parts of the initial test protocol are to be displayed or used for data processing, then these parts must first be read from the initial test protocol.
This is done using the commands

`tp.Data` (for .html protocols), or

```
Dim xmlDoc As New XmlDocument
xmlDoc.LoadXML(tp.Data)
```
(for .xml protocols).

Post-processing is applied to both HTML– and XML–protocols. If you need more information about creating HTML– or XML–protocols, please follow the instructions in the chapters ***5.6.3.3Writing test protocols in HTML format*** respectively ***5.6.3.5Writing test protocols in XML and formatting via XSL-T***.

Please note that applying protocol post-processing can change the type of the initial test protocol. For example, the XML–data of an initial XML–protocol can be transferred to an HTML–table of a post-processed HTML–protocol. Of course, the post-processing script can also be designed so that the type of the post-processed protocol does not change.

Example: Post-process the initial XML–protocol into a HTML–protocol

For example, if you want to capture data stored in key/value pairs in the initial test, you can create this test code:

```
'#Language "WWB-COM"
Option Explicit

Sub Main
    System.Remark "</pre>", False
    System.Remark "<?xml version=""1.0"" encoding=""UTF-16""?>"
    System.Remark "<data_acquisition>"
    System.Remark "<signal name=""a"">2</signal>"
    System.Remark "<signal name=""b"">3</signal>"
    System.Remark "<signal name=""c"">4</signal>"
    System.Remark "</data_acquisition>"

    System.SetResult trvPass, 0, False
End Sub
```

The corresponding (initial) result will be the following:

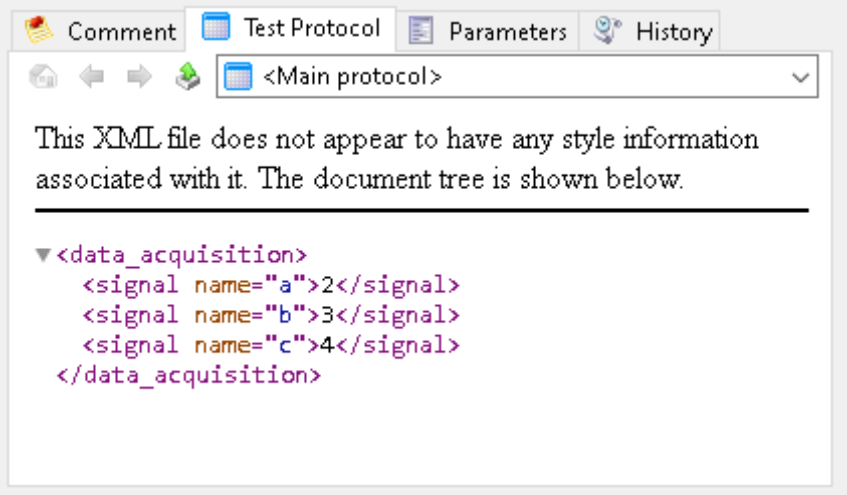


Figure 361 – XML protocol without post-processing

If you want to post-process the initial data and transform the output into a table of keyword/value-pairs, you can create a test script and set its path as the post-processing path in the *Settings...* dialog of the `Test Manager` (see also ***3.2.7.5 Test Manager***).

The following example shows how the `ProcessTestProtocol` function could be implemented. In this post-processing test script, the Microsoft .NET XML parser `XmlDocument` is included to read the XML structure of the initial test script (this requires the use of a .NET test script).

```
'#Language "WWB.NET"
Option Explicit
Import System.Xml

'Post-processing callback function
Function ProcessTestProtocol(tp As TestProtocol) As String

    Dim sOutput As String
    sOutput="<html><body><h2>Data acquisition</h2>"
    sOutput+="<table border=""1"">"+vbCrLf
```

```
    sOutput+="<tr bgcolor=""#9acd32""><th>Signal</th>"
    sOutput+="<th>Value</th></tr>"+vbCrLf

    Dim sName As String
    Dim sValue As String
    Dim xmlDoc As New XmlDocument
    Dim nodes As XmlNodeList

    xmlDoc.LoadXml(tp.Data)
    nodes=xmlDoc.DocumentElement.SelectNodes("/data_acquisition/signal")

    For Each node As XmlNode In nodes
        sName=node.Attributes.GetNamedItem("name").InnerText
        sValue=node.InnerText
        sOutput+="<tr><td>"+sName+"</td><td>"+sValue+"</td></tr>"+vbCrLf
    Next

    sOutput+="</table></body></html>"
    ProcessTestProtocol=sOutput
End Function
```

The result after postprocessing is the following

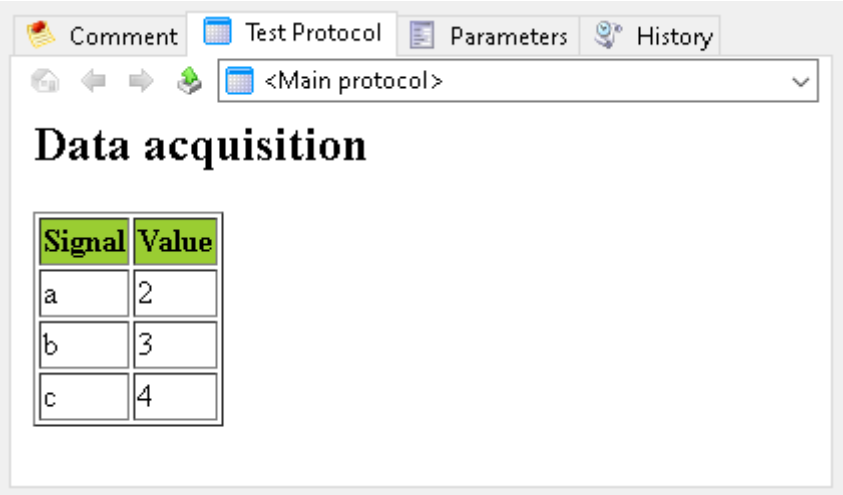


Figure 362 – XML protocol, formatted via post-processing

The function used in the above example to select the XML nodes to be read `xmlDoc.DocumentElement.SelectNodes()` uses the so-called query language **XPath** to define the nodes. If you need more information about this, please have a look at https://www.w3schools.com/xml/xpath_syntax.asp.

If the user does not want to apply the post-processing to all test objects but only to a self-defined selection, he can filter according to special conditions in order to either have the post-processing executed or deactivated. For example, a certain parameter can

be defined in the test objects, which is queried by the post-processing test script (e.g. `"EnablePostprocessing"=True`).
An example for checking whether such a parameter is defined and set to `True` is shown in the following example (the changes from the previous example are formatted in bold).

Initial test object with condition parameter:

```
'#Language "WWB-COM"
Option Explicit

Sub Main
    System.Remark "</pre>", False
    System.Remark "<?xml version=""1.0"" encoding=""UTF-16""?>"
    System.Remark "<data_acquisition>"
    System.Remark "<signal name=""a"">2</signal>"
    System.Remark "<signal name=""b"">3</signal>"
    System.Remark "<signal name=""c"">4</signal>"
    System.Remark "</data_acquisition>"

    Dim tps As New TestParameters
    tps.Add("EnablePostprocessing", True)
    Dim tr As New TestResult
    tr.TestParameters=tps

    System.SetResult trvPass, 0, False
End Sub
```

Post-Processing test script with condition parameter:

```
'#Language "WWB.NET"
Option Explicit
Import System.Xml

'Post-processing callback function
Function ProcessTestProtocol(tp As TestProtocol) As String

    If IsPostProcessingActive(tp.Result.TestParameters) Then
        Dim sOutput As String
        sOutput="<html><body><h2>Data acquisition</h2>"
        sOutput+="<table border=""1"">"+vbCrLf
        sOutput+="<tr bgcolor=""#9acd32""><th>Signal</th>"
        sOutput+="<th>Value</th></tr>"+vbCrLf

        Dim sName As String
        Dim sValue As String
        Dim xmlDoc As New XmlDocument
        Dim nodes As XmlNodeList

        xmlDoc.LoadXml(tp.Data)
        nodes=xmlDoc.DocumentElement.SelectNodes("/data_acquisition/signal")

        For Each node As XmlNode In nodes
            sName=node.Attributes.GetNamedItem("name").InnerText
            sValue=node.InnerText
            sOutput+="<tr><td>"+sName+"</td><td>"+sValue+"</td></tr>"+vbCrLf
        Next
```

```
        sOutput+="</table></body></html>"
        ProcessTestProtocol=sOutput
    End If
End Function

Function IsPostProcessingActive(tps As TestParameters) As Boolean
    Dim tp As TestParameter
    Dim bEnable As Boolean

    On Error Goto NotFound
    'Check the TestParameter "EnablePostProcessing"
    tp=tps("EnablePostprocessing")
    If VarType(tp.Value)=vbBoolean Then
        bEnable=tp.Value
        'When the parameter is True, set return value to True
        If bEnable Then
            IsPostProcessingActive=True
            Exit Function
        End If
    End If
    On Error Goto 0
    IsPostProcessingActive=False
    Exit Function
NotFound:
    On Error Goto 0
    IsPostProcessingActive=False
End Function
```



For using the Microsoft .NET `XmlDocument` parser as in the previous example, make sure that your post-processing test script includes the .NET library (via `'#Language "WWB.NET"`).



In the XML header you specify the `"encoding"`. In order to create a correct XML protocol, the characters passed to `"System.Remark"` must use this encoding. The test protocols are stored in UTF-16 format.



You cannot post-process a system test protocol by a WinWrap Basic handler in PROVEtech:TA (in contrast to an elementary test protocol).

The WinWrap Basic script for post-processing the protocols is kept in a separate Basic instance in the working memory of the sequencer server. That means, global variables are still valid during several function calls.

If you modify the WinWrap Basic code, then you must recreate the Basic instance. You can do this by a click on the button `OK` in the Settings dialog.

If there are syntax errors in the XSL-T (e.g., unclosed <tags> or misspelled keywords), the Microsoft Edge Chromium Browser is currently unable to report these correctly to the user. In this case, the displayed result is a completely empty, white page. If you see an empty page as a result, please check the XSL-T file for errors.

5.6.4 Links

When you copy test objects with test results (see ***5.3.5.1 Copying and moving with the context menu*** or Test Language command ***TestItem.Copy***) then you will get different links to the same test result object. These links are displayed in the tab “Links” after selecting the test result.

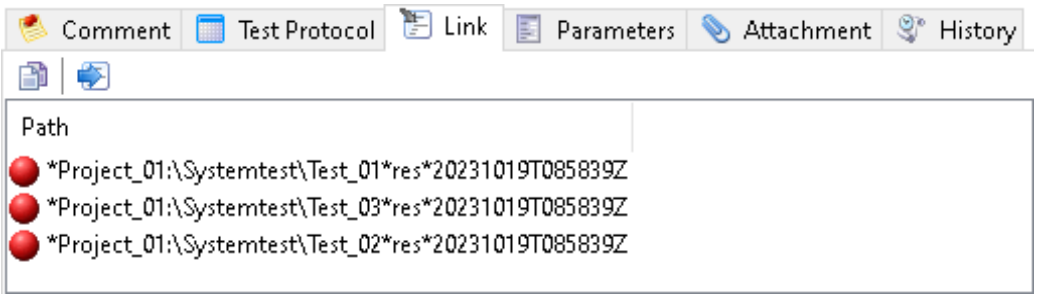


Figure 363 – Linked test results

The different result links can get different names and you can delete these links separately. All further changes on the test result affect all result links.

You can find more at the test language command ***TestResult.GetLinks***.

5.6.5 Runtime errors

If runtime errors or warnings occur while executing a test then they are displayed in the tab `Runtime Errors` of the test result. The warnings are displayed with a blue icon and the errors with a red icon.

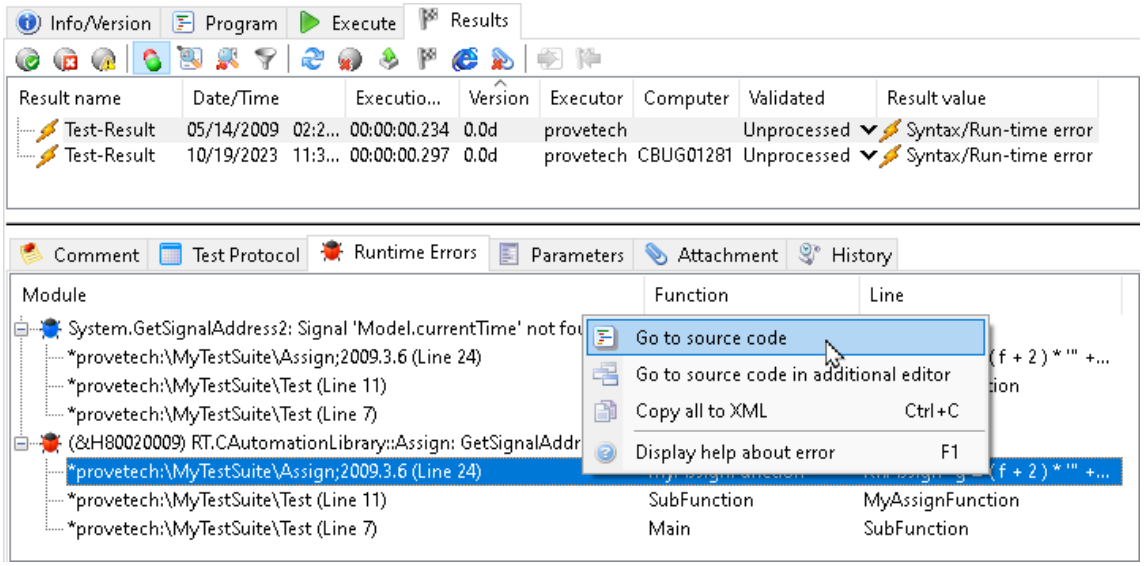


Figure 364 – Runtime errors and warnings

In addition to the error or warning message and its timestamp, the subtree contains the whole calling stack, one line for each call. If you double-click the entry or choose *Go to source code* in the context menu you can go to the code position in the Basic editor. By pressing F1 or via context menu *Display help about error* a context sensitive help page is shown, provided that the corresponding help information is available. If you press Ctrl+C or choose *Copy all to XML* in the context menu then you can copy the complete XML string of all error messages with additional information into the clipboard as it is stored in the database.

The blue icon means, that it is a warning, and the test execution continues. For example, if you call ***System.SetSignal*** and it fails, it just outputs a warning (if the compatibility mode is True). On the other hand, a red icon means a runtime error where the test execution stops. For example, if you have an integer division by 0, it stops the execution if you do not catch the error in your code. Nevertheless, warnings should also be avoided if possible. The symbol only appears if PROVEtech:TA is able to get any stack information (call stack when the error/warning happens). Sometimes for example, the error or warning happens in some destructors (executed to clean up the objects like `Object_Terminate` or `Class_Terminate`) and there's no stack information is available.

5.6.6 ECU errors

In Caesar diagnostics (not in MCD-3 D diagnostics), if you program the command to read out the error memory contents of the tested ECU in the test program, these data will be added to the results of the test. You can view the data read from the test program and generated by the diagnostics software.

The error number and environment data are read from the error memory of the ECU.

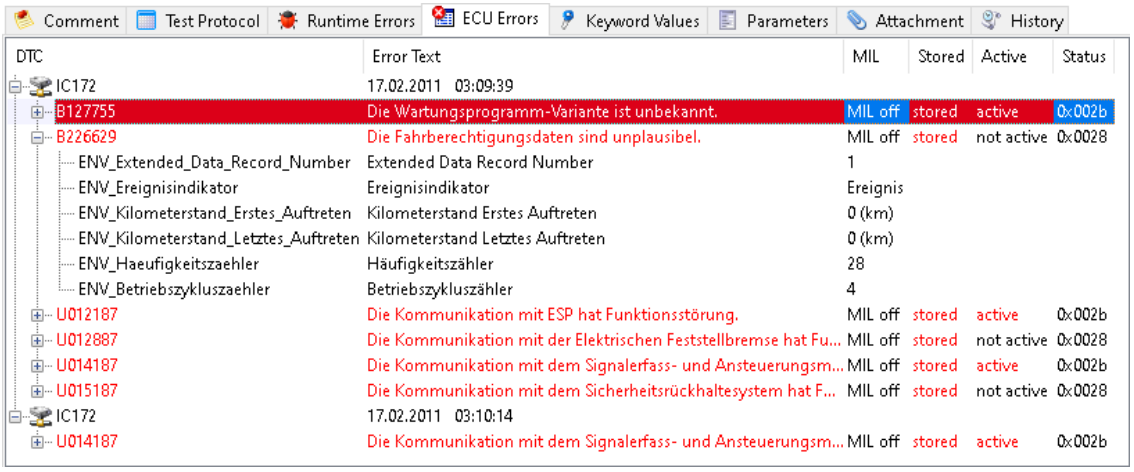


Figure 365 – ECU errors

The diagnostics software then supplies for the error codes from the ECU an error text that is likewise displayed in the property page. These data are represented in the property page in the form of a hierarchical table. Each ECU error of the ECU error memory has red text color.

5.6.7 Keyword values

The keyword values are a legacy of older PROVEtech:TA versions to process statistical evaluations on the test results. Keyword values provide the possibility to store numerical and string values at the test result.

If you have assigned values to the keywords during the test run (Basic command ***TM.AddKeywordValue***), then these can be viewed on the `Keyword Values` property page. Later on, you can use these values to find test results and to perform statistical evaluations.

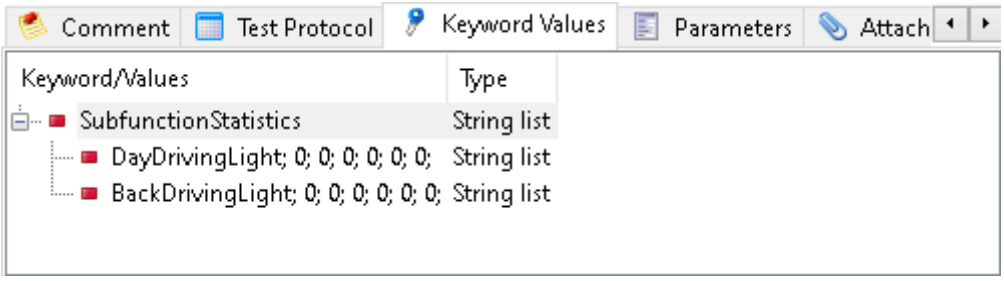


Figure 366 – The Keyword Values property page

In the first column you see the name of the keyword at the top level. The associated values follow in the rows below.

5.6.8 Parameters

During the test run you can assign a parameter set by test language to the current test result. Parameters that are displayed in brown font were originally inherited at the test object and later written to the test result.

It is still possible to modify the test parameters after execution. Then the modified parameters are displayed in red font to indicate that they are not part of the test run. You can display the timestamp of the last modification by tooltip.

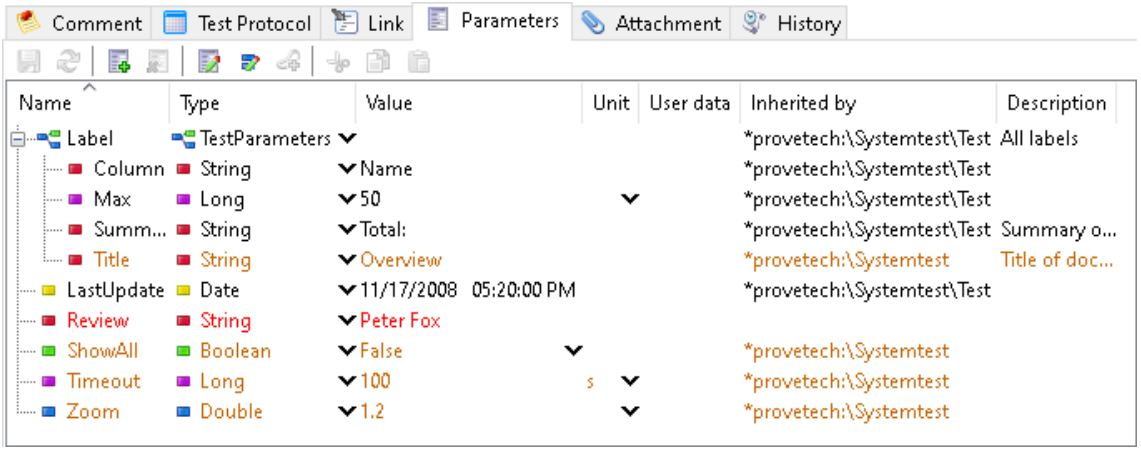


Figure 367 – Parameters of the test result

The columns and the entries of the context menu correspond to the parameter dialog of the test object (see ***5.3.9.2 The tab "Parameters"***).



In contrast to the test object, the parameters of the test result and the keyword values are completely independent. You can read more about parameters, types, inheritance and URIs (hyperlinks) in ***5.3.9.2 The tab "Parameters"***.

5.6.9 Attachments

Every time you execute a test, you can save any files or signal curves (data acquisition) along with the test result. The saved files and the recorded signal curves are called an attachment.

Even after the execution you can add (or remove) attachments to the test result. If the last modification date of the attached file is newer than the date of the test result, the date of the attachment is then displayed in red font color.

Before you can save signal curves while a test is running, you must start and stop a data acquisition in the test program by the commands ***Measure.Start*** and ***Measure.Stop***.

Then you can save the generated signal curves via the command ***Measure.Save***. Signal curves are automatically assigned the type ‘hdr’. You can also see in the attachment type of a data acquisition contains messages (like CAN-, MOST- or LIN-messages).

With the command ***TestResult.AddAttachment*** (or ***Measure.SaveFile*** from prior PROVEtech:TA versions) in the test program you can save any file with the test result.

The Attachment property page displays a list of all attachments that were saved during or after execution of the test.

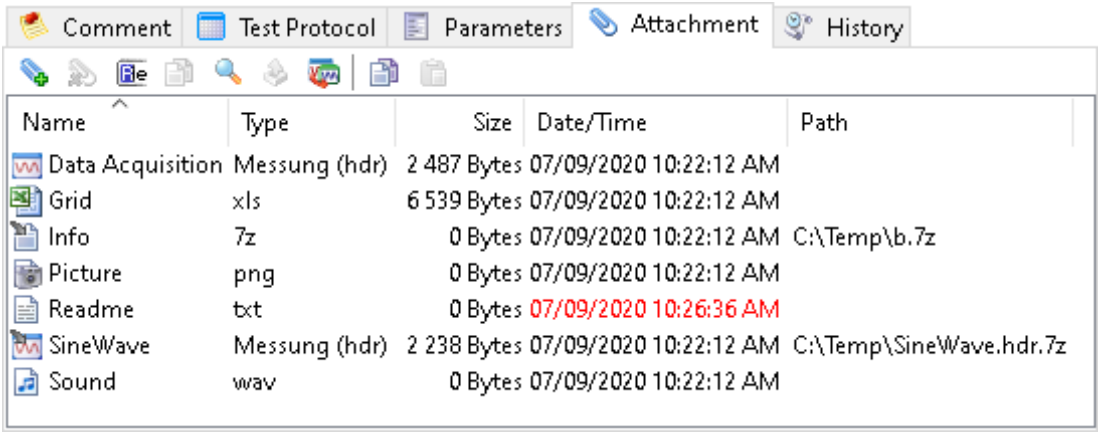


Figure 368 – Attachment property page

Column	**Description**
Name	You can choose a name for the attachment (Default is the file name without extension)
Type	Type of attachment corresponding to the file extension; ,”hdr” is the type of a data acquisition file. You can display and modify a data acquisition via toolbar button *Offline*. If you have an attachment linked into the file system with file extension .7z (compressed file in 7zip format) then the original file extension is shown in this column.
Size	Original size of the attachment file (not the compressed size in the database). If it is an attachment linked into the file system, then the file size is shown here. You can display the compressed and uncompressed size when moving the mouse pointer over the entry.
Date/Time	Date and time of the last change (here you can also see by means of the red text color whether the last write date of the attached file is newer than the date of the test result and thus was added subsequently).

Path	In case of an attachment linked to the file system the file path is displayed here. If it is an attachment that is stored in the database this column stays empty.



If you want to integrate the attachment in the test protocol, we strongly recommend not using the '#' character or other special characters in the attachment name that have a special meaning in URLs. Otherwise, it is possible that the subsequent navigation to this attachment or its display will not work.

Attachment linked into the file system

You also have the possibility to create attachments that are linked to a file in the file system. Therefore, only the file path is stored in the database. This file path is displayed in the column `Path`. It helps you to distinguish the linked and non-linked attachments.

You can create an attachment that is linked into the file system by using the Test Language command ***TestResult.AddAttachment*** with the corresponding parameters. The linked file is not explicitly compressed anymore. You have to do this first in your test script if you want. There is some special handling when opening and saving data acquisition files. In the Test Language command ***Measure.SaveAs*** you can use the file extension .hdr for saving the data acquisition as a non-compressed or .hdr.7z as a compressed file (see also in the Test Language manual).

There are the following advantages and disadvantages of linked attachments in contrast to attachments that are stored in the database:

- **Advantage:** The database requires less storage space, since the binary data of the attachments are stored in the file system instead of in the database.
- **Advantage:** Creating a database backup is easier and faster because the attachment data is not included in the backup.
- **Advantage or disadvantage:** Manual access and intervention in the linked files inside the file system is possible.
- **Disadvantage:** The consistency of data between database and file system gets lost. So, for example, you can move or delete the linked files in the file system. Or, when you empty the Trash Folder, the linked files of the attachments stay alive in the file system. If you kill the PROVEtech:TA process or turn off your computer in a bad moment, there could also be consistency problems left.
- **Disadvantage:** Paths may not be found if e.g. network drives are not mounted.

- **Disadvantage:** You need a synchronous backup of the database and the file system if you want to restore your system after a data loss.
- **Disadvantage:** You must control the access rights in PROVEtech:TA on the one hand and in the file system on the other hand.
- **Disadvantage:** You must think of a clever hierarchical file structure to avoid overflowing of files in one single directory.
- **Disadvantage:** In case you use database replication the file share must be replicated too, or be reachable from all PROVEtech:TA clients.



When exporting and importing Test Manager subtrees to PROVEtech:TA archive file (extension .dbi.7z) the files of linked attachments are not included in the export (see also ***5.3.10 Exporting and importing tests***).

Delete attachments

Mark an attachment and click  to delete an attachment from the actual test result.



You can also delete all attachments from one or several test results in one step. Choose `Delete all attachments…` in the context menu of the test result.

If you want to delete attachments that are linked into the file system then a message box appears. Now you can decide if you want to delete or keep the linked files in the file system.

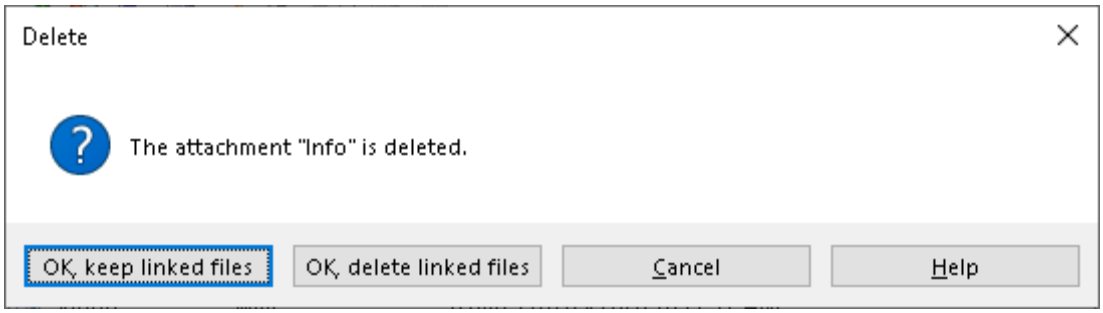


Figure 369 – Deleting attachments that are linked into the file system

When using attachments that are linked into the file system, the consistency of data between database and file system may get lost. So, for example, you can move or delete the linked files in the file system. Or, when you empty the Trash Folder, the linked files of the attachments stay alive in the file system. If you kill the PROVEtech:TA process or turn off your computer in a bad moment, there could also be consistency problems left.

Display an attachment

Mark an attachment in the list and click on  . If it is an attachment of the type “hdr” (or also with file extension .hdr.7z in case of a linked attachment), the program changes to offline visualization and the chronological curves of the signals are shown in the form of line writer strip charts. Attachments of a different type are opened with the program intended for that type.

If you want to display a linked attachment with file extension .7z then PROVEtech:TA first examines how many files are inside the archive. In case of exactly one file this file is temporarily decompressed and then displayed.

You can display attachments by double-clicking them, too.

Export (and convert) an attachment

1. Click on the attachment that you would like to export.
2. Click on  . The program opens the file selection dialog (see chapter ***3.1 The file selection dialog***).
3. Enter a file name and click on *Save*. The file is saved.

When you copy the export files for data acquisitions, you must transfer all files created in the relevant export. If you do not know which they are, the best thing would be to create a new directory and export the files to this directory.

If you want to export a linked attachment with file extension .hdr.7z or .7z then there is a special handling analog to the display of linked attachments (see **Display an attachment**).

It is also possible to export and convert a data acquisition in one step. Just select the data acquisition and click on .

5.6.10 The history of a test result

Nearly all actions that modify the test result can be logged in the database for documentation and traceability.

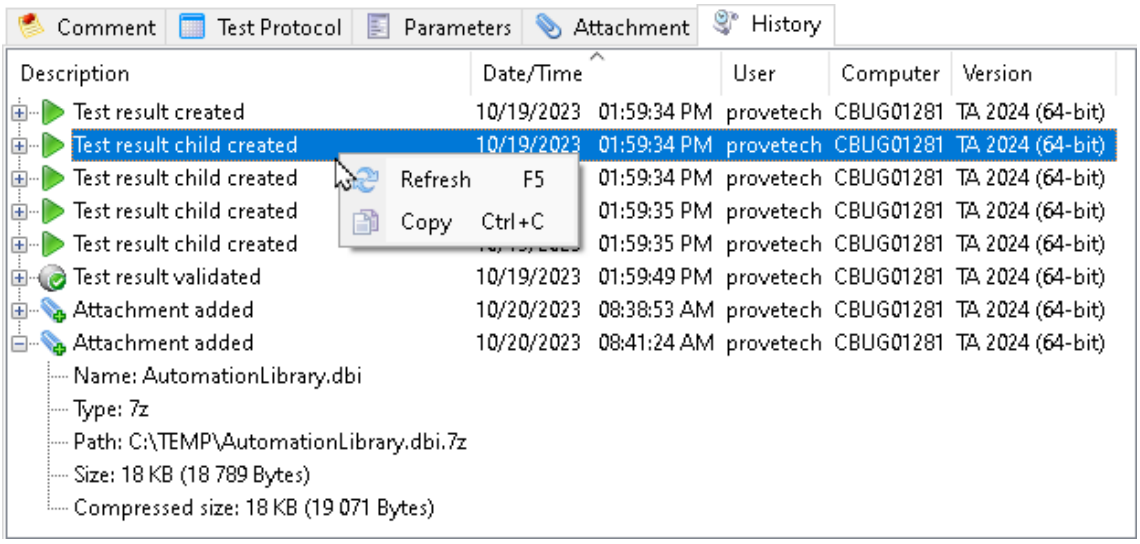


Figure 370 – History of a test result

To avoid overflowing, the history list not all the actions can be logged that modify the test result. So, some of the operations are not logged while executing the test (e.g. adding a text to the test protocol), but almost all modifications are logged after generation of the test result.

The “Delete” operation of a test result is only logged at the corresponding test object and the corresponding parent test result, if the test result was validated (see ***5.3.9.7 The tab “History”*** and ***5.7.5 Validating test results***). This should avoid an overflow of the history list.



If older PROVEtech:TA versions connect to the same Test Manager database, then it is possible that these versions produce no history entries yet.

The PROVEtech:TA administrator can globally disable the logging. Therefore, open the dialog `Properties` of a test item and select the tab `Security`. There you can find the checkbox `Enable logging for history of test result` (see also chapter ***5.3.11.2 The tab "Database settings" in the property dialog***).

5.7 Managing test results

Once you run a test, you can access the test result data in the Results tab. This tab contains the execution history of the test structured in a hierarchical tree containing all sub results, the associated execution and validation result data, and the generated test protocols and attachments. The test result cannot be manipulated, but the user can validate the test result, display and export the test result data and associated protocols, and add additional comments, attachments and data if required.

5.7.1 Displaying the list of test results

In the `Results` property page in the top area, you can see a list of test results corresponding to the selected test object in the Test Manager tree. If you click on one of the test results, then the result data is displayed in the bottom area. The test results are sorted hierarchically according to the end timestamp of their execution.

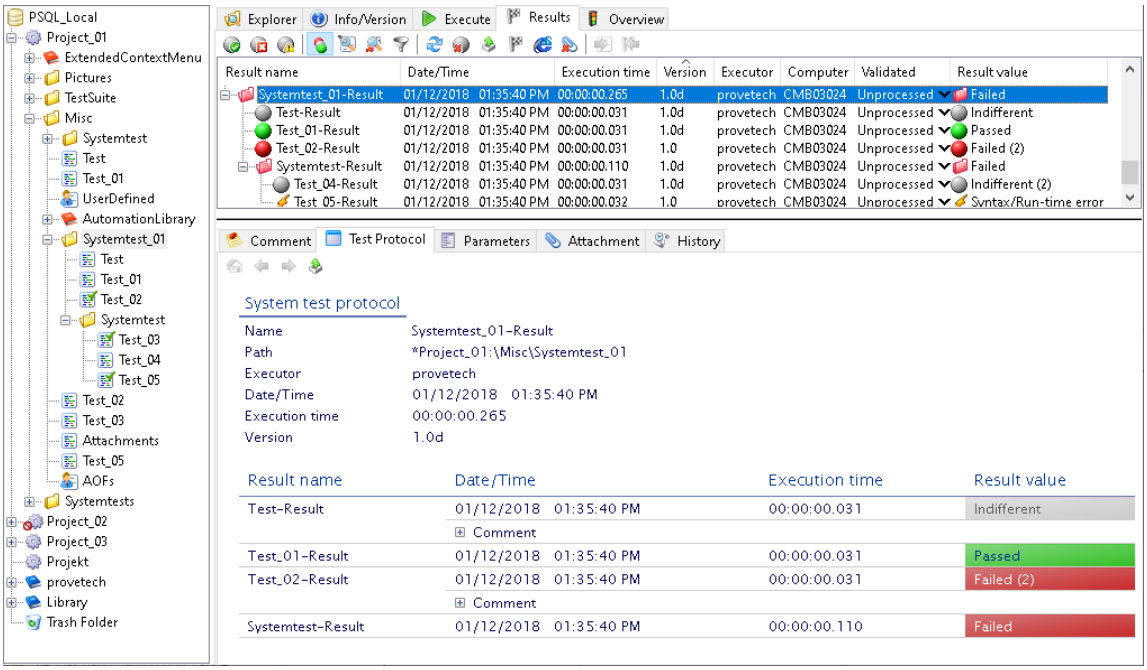


Figure 371 – Results of system test

The test results are displayed with the `Date` and `Time` of their creation (timestamp when the test run was finished) and `Execution time`, also `Executor`, `Computer` (or other host information) and `Result value`. A tooltip on the executor shows additional information, such as further attributes of the user or the computer name on which the test was executed. Test results with result value “Passed” are displayed with a green icon; the value “Indifferent” are displayed in gray, “Running” are displayed in yellow, “Not Implemented” is displayed in brown, the others with red icon. The number in parentheses behind the result value indicates the additional result value, if this was set not equal 0. The columns `Version` and `Version Label` show the version of the test program at execution time.

In the column `Validated` you can set the “Validated” status of the test result (see chapter ***5.7.5 Validating test results***).

If a system test result (or a project test result) is marked and you want to display the individual elementary test results, then select the results of a system test execution. Click on the symbol ⊞ in front of the system test results or double click on the system test. This expands the list and displays the elementary tests of the system test results.

The `Test Protocol` property page of the system test result (lower half of the `Results` property page, see ***Figure 371***) displays the test protocols generated during the test execution. You can export these in HTML, MHTML or XML format. The latest is only possible if the protocol is generated in XML-format.

In the top of the `Results` property page there is a toolbar to control the test results. There are corresponding items in the context menu. In the following table all operations are explained:

	Valid	Sets the Validated state to “Valid”
	Invalid	Sets the Validated state to “Invalid”
	In process	Sets the Validated state to “In process”
	Display icons in first column	Switching on and off the icons in the first column; alternatively, an emoji can also be used in the test result name instead.
	Show results of links	This is a global flag. If the selected test object has got links then you can control if the test results of the linked test objects should also be shown in the result list. Please read chapter ***5.3.14 Linked test objects*** for more information about links.
	Show deleted results	Displays test results as well, which are marked as “deleted”
	Filter	Shows/hides the filter bar in the test results list to filter for the displayed attributes of the test results.
	Refresh selected test result	Refreshes the display of the property pages at the bottom (e.g. test protocol)
	Delete	Deletes the selected test results after confirmation
	Export test result	Exports a test result with all attributes, protocols and further data; so not only test protocols are exported
	View test result in additional dialog	Displays the test result in an additional window
	View test protocol in browser	Temporarily exports the test protocols and displays them in the default browser
	Delete all attachments	Deletes all attachments of the selected test results

	Go to test	Selects the referring test of a test result in the Test Manager tree
	Go to parent test result	Selects the parent test result if existent

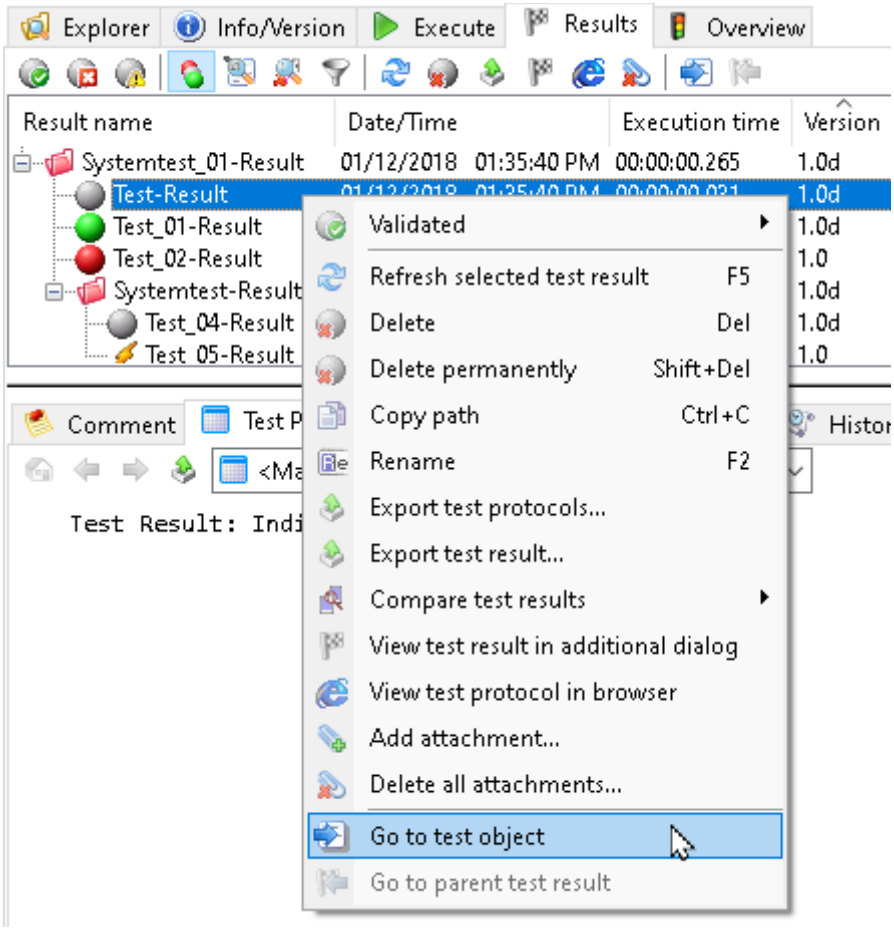


Figure 372 – Context menu of a test result

By clicking on the filter icon in the toolbar, you can switch the filter bar on and off above the test result list under the column names. In the filter bar you can then filter for substrings in the individual result attributes. In the columns "Date/Time" as well as "Execution time" it is also possible to specify ranges, e.g. "01/20/2023-07/31/2023" (between), "-12/31/2022" (older than), "01/01/2023-" (newer than) or "01:00-02:00" (between one and two hours).

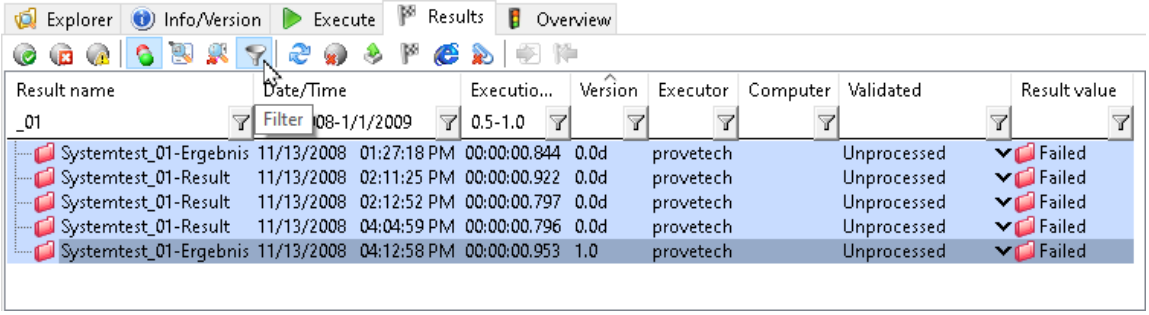


Figure 373 – Filter for test results

Please read also in chapter ***5.6.1 Displaying the test result in a dialog***.



A test result is automatically set from "running" to "stopped" if it is at least one day old and is no longer recognized as "running" on the test execution computer. For example, if the PROVEtech:TA process was terminated by the Windows operating system during execution and PROVEtech:TA is restarted after one day, the status is set to "stopped" when the test result data is fetched.

5.7.2 Test results of older test versions

Test results of older test versions are displayed in the result list with light blue background (see ***5.8 Versioning***).

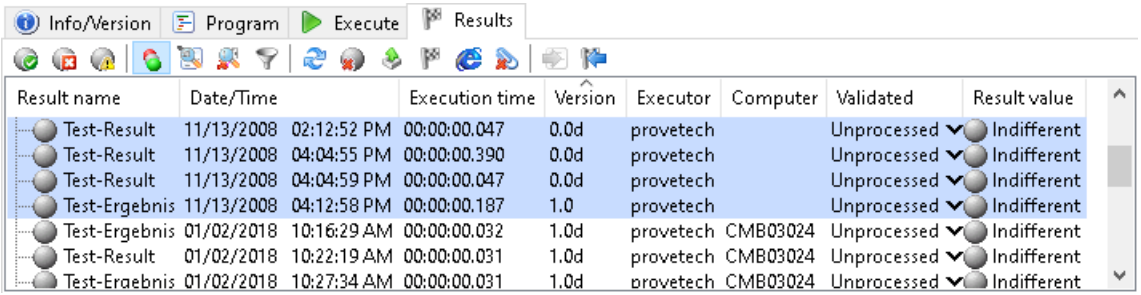


Figure 374 – Test results of older test versions



When you **copy** and **paste test results with reference to previous versions**, but without results, then there might be test results listed anyway. These test results were not copied. They are only linked with the previous versions of the test object.

5.7.3 Deleting test results

The saved test results can take up a lot of space on the hard disk, often several megabytes, depending on the number of signals recorded and the recording duration. You should therefore delete test results that you no longer require.

1. Select all test results in the list that you want to delete.
2. Then press the  button.
3. Confirm the following security dialog. When you delete system test results or project results you probably are asked if you want to delete child results too (see ***Figure 375***).

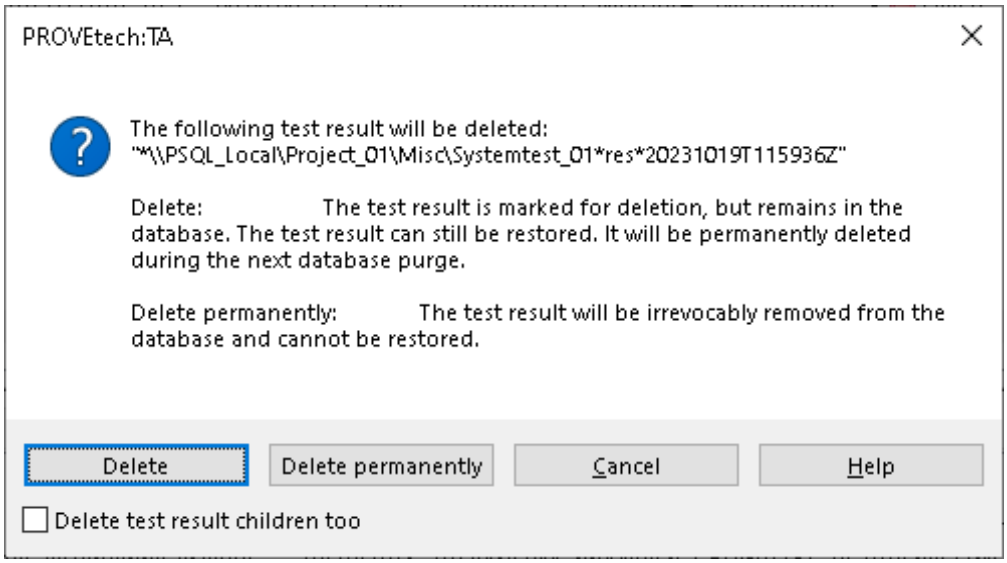


Figure 375 – Security dialog when deleting

If you delete test results of older test versions then they are no more accessible from other version branches, too.

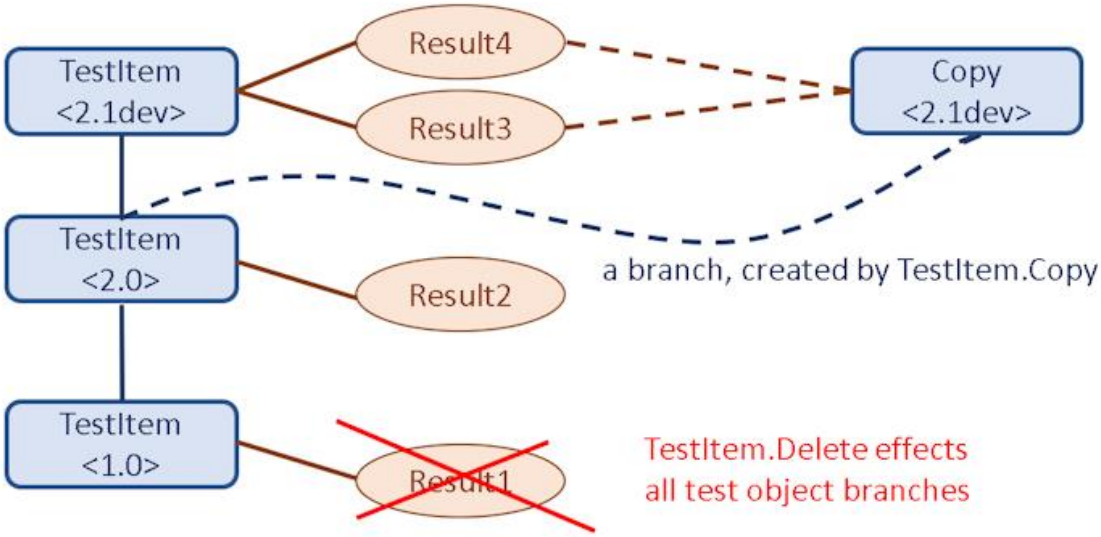


Figure 376 – Deleting test results of older test versions

When you delete test results, they are not finally removed from the database. They are only marked as deleted. You can restore these test results again (see chapter ***5.7.4 Restoring deleted test results***). If you purge the database (see chapter ***5.2.4 The trash folder***) then the test results are finally removed.

The hotkey *Shift+Del* leads to a permanent deletion of the selected test results, too (see Test Language command ***TestResult.Delete***).

5.7.4 Restoring deleted test results

If the button is activated in the toolbar, then the deleted test results are displayed again (marked with the icons or). It is a precondition that the deleted test results are still kept in the database and not deleted by the last purge of database. In addition, the column `Deleted on` is displayed, which indicates the time of the deletion action. Please see also chapter ***5.7.3 Deleting test results***.

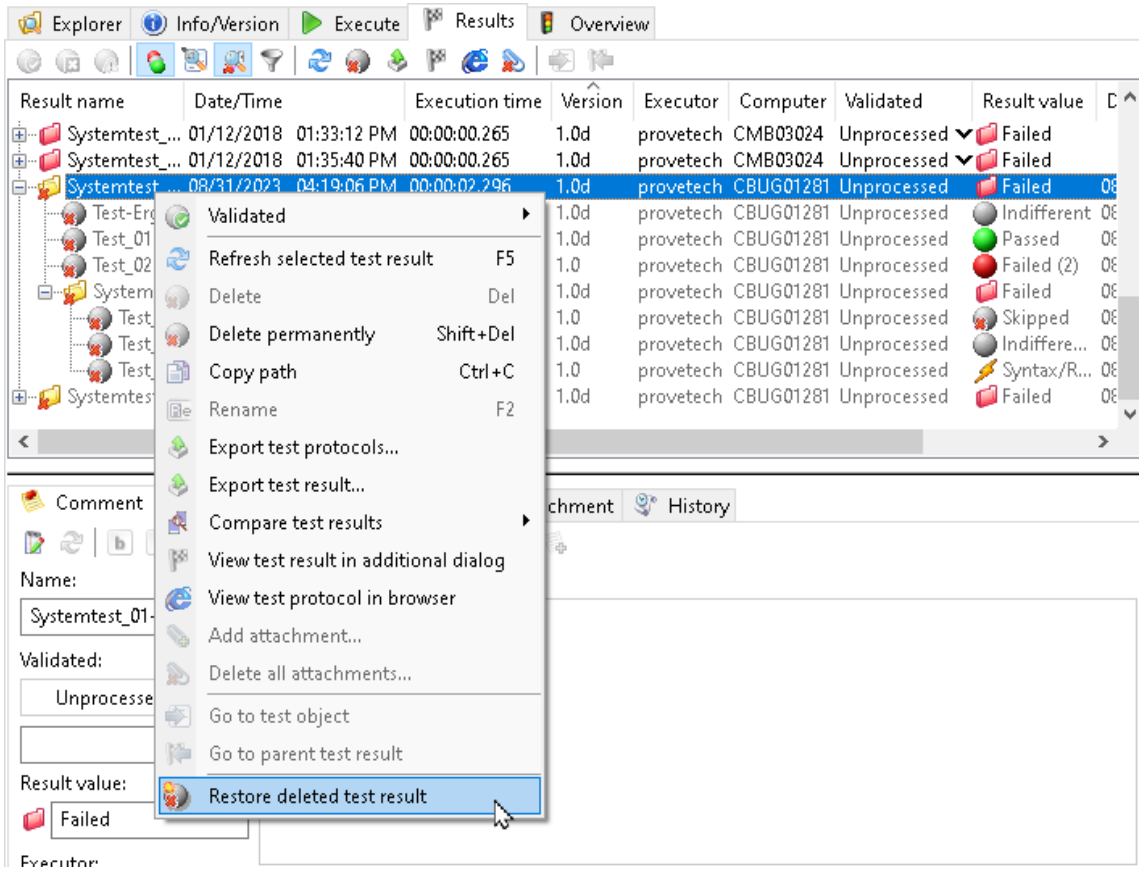


Figure 377 – Restoring deleted test results

To restore the deleted test results click on the appropriate entry in the context menu.

By context menu on the trash folder object in the TM tree you get a list of all deleted test results, assumed that you own the access rights to show the results (see also chapter ***5.2.4 The trash folder***).

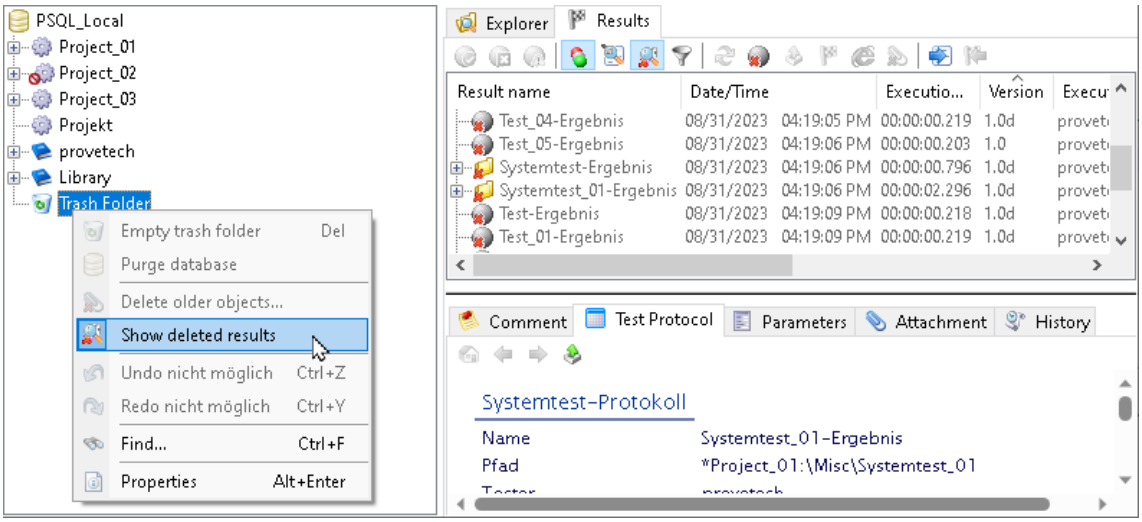


Figure 378 – Displaying all deleted test results by trash folder

5.7.5 Validating test results

For certain reasons, test results can be valid or invalid. For example regarding a test result with result value “Failed” (red icon), you could set the result to “Invalid” if the test script contained an error, the ECU had an outdated software version or there was a cabling fault in the test system. But if you actually find an incident in the ECU then you can set the test result to “Valid”, which means, you must do regression testing later.

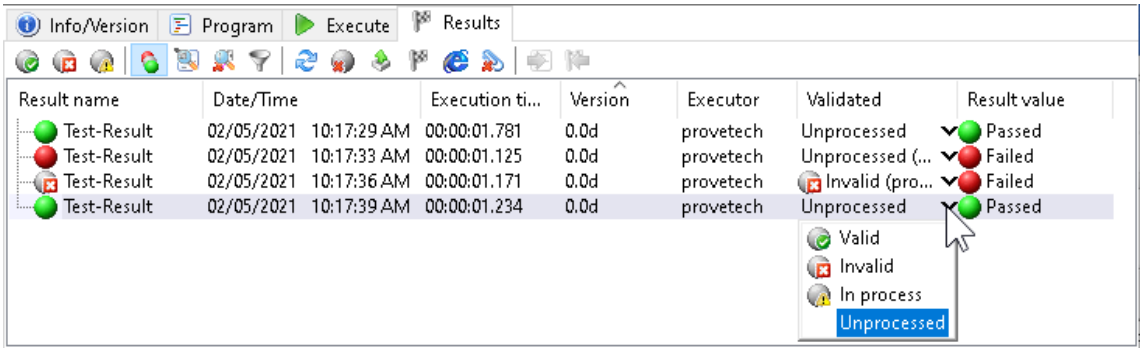


Figure 379 – Validating test results

In the tab `Results`, `Overview` and `Comment` you can set the “Validated” status of the test result in the combo box inside the toolbar. 3 states are possible:

- `Unprocessed` with no icon is the initial status
- `Valid` with icon means that the test result was validated and set to valid (e.g. this is the expected behavior, or an ECU error was found)
- `Invalid` with icon means that the test result was validated and set to invalid because of any reason (e.g. scripting error in the Basic code)
- `In process` with icon means that the tester has already started to analyze the test result, but the validated status is not yet defined

All state transitions are possible. Then, the user and the timestamp of the last change is stored in addition.

If the database-global setting `Comment is mandatory if test result is 'invalid'` is set (see chapter ***5.3.11.2 The tab “Database settings” in the property dialog***), you must comment your test result when validating it to ‘Invalid’.

5.7.6 Comparing test results

For comparing two test results, the user can use the `Compare tool` defined in Test Manager settings dialog (see chapter ***3.2.7.5 Test Manager***). The user can either select

two test results of the same test item and click the context-menu option `Compare selected test results`, or compare test results by selecting one by one using the context-menu option `Compare test results`. The later allows comparing test results of different test items. The action will export selected test results in a temporary folder which contains the following items:

- `TestResultData.xml`: The data described in this file contains all stored information of the test result, excluding the test protocols (see chapter ***5.6 Test results*** for further details). For comparison purposes, the test result data containing timestamp references are excluded.
- Exported test protocols. The exported test protocols will be stored using the same hierarchical structure as the one depicted in the test result tab, being the main protocol the file `MAIN.html` (if the test protocol is a normal text or a HTML-protocol) or `MAIN.xml` (if the test protocol is an Automation Library test saved in XML-format or a test protocol remarked in XML-format).

The external compare tool will compare the full exported folder structure of the selected test results.

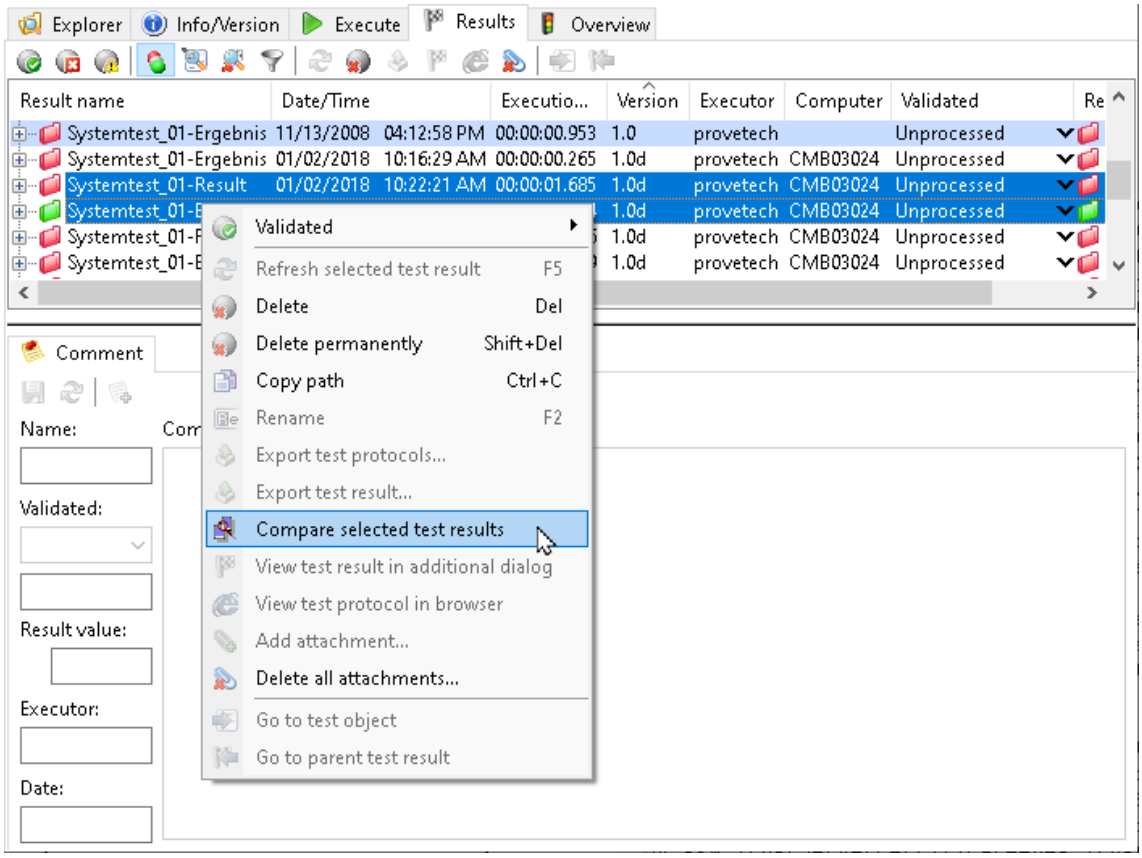


Figure 380 – Compare two test results

`TestResultData.xml` contains the execution time, which is likely to differ among two executions. The user should decide if this difference can be considered as acceptable for considering whether two test results are the same or different.

The comparison will export all sub results recursively. Exporting all sub results can be a lengthy operation, depending on the number of sub results attached to the root test result. All sub results attached to the current test result contain their own `TestResultData.xml` stored in the corresponding sub folder.

The modifications saved using the merge-tool in the exported test result data are not stored into the Test Manager database.

5.7.7 The tab "Overview"

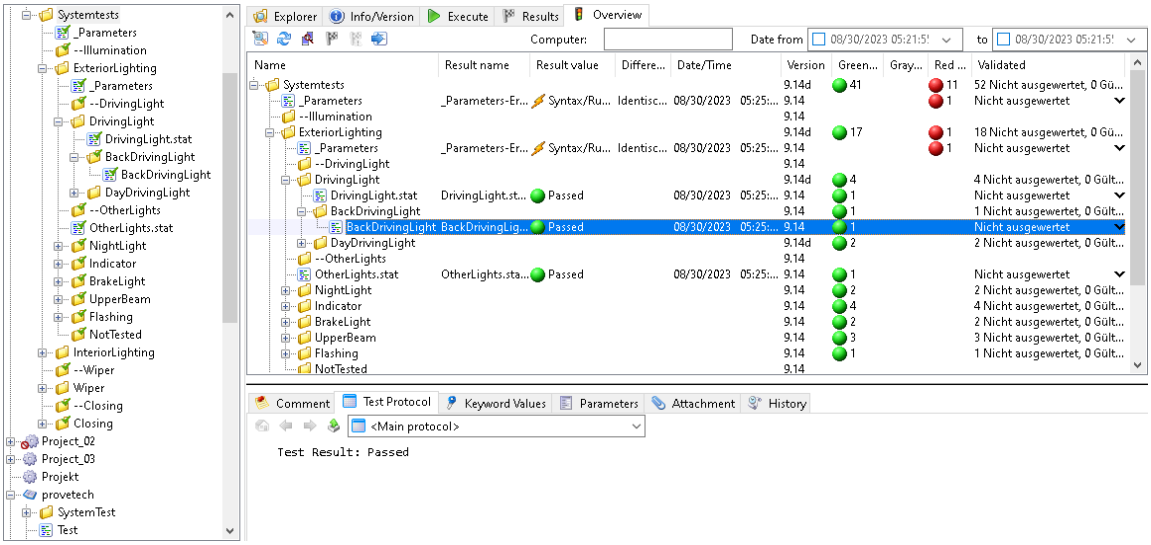


Figure 381 – The tab "Overview"

The tab `Overview` helps you to get a quick summary of the test result status of a complete subtree in the Test Manager. Furthermore, you can compare the current result value with the previous test results. The tab is especially useful to see, how many of the tests runs within a subtree were classified as “red”, “gray” or “green”, or how many test results were validated with “Valid” (see chapter ***5.7.5 Validating test results***). With this one can quickly find the test cases that need a closer inspection. Using a filter `Computer` by (exact) computer name, you can consider only test results that were executed

on this computer. Additionally, a date filter `Date from to` helps to view the test results within a specific time frame. So you can, for example check if all test cases within a sub tree have been executed after a certain date, or if any test case within a sub tree indicates an outdated result (for example, after an ECU software update).

The tab is displayed if the following conditions are valid:

- The selected test object is a system test or a project.
- The subtree of the selected test object is not empty.
- You have got the access rights to display test result lists.

Similar to the tab `Results` (see ***5.7.1 Displaying the list of test results***) the result data of the selected test result is shown in the lower tabs.

The toolbar consists of the following buttons:


Controls the inclusion of test results that are executed by linked tests. Please read chapter ***5.3.14 Linked test objects*** for more information about links.


Refreshes the view.


Controls if the corresponding current test result should be compared with the last validated or any last test result.


Displays the current test result of the marked test in a separate window.


Displays the second last test result of the marked test in a separate window.


Selects the marked test object in the Test Manager tree.

The `Date filter` helps you to determine which test results are displayed. Only the test results with creation date inside the date interval are shown.


You can use the filter functionality of the find dialog to include only special tests and test results in the overview (see chapter ***5.18.2 Filter tests and test results***).

Some of the columns are calculated in an additional background thread so that your work is not blocked. The following table explains the columns in the upper area.

Column name	Row is a system test	Row is an elementary test
Name	Name of the test object	Name of the test object
Result name		Name of the current test result
Result value		Result value of the tests result
Difference to second last test result	Number of differences in the subtree	Difference to the second last test result (Result value, runtime errors, keyword values)
Date/Time		Timestamp of the current test result when it was finished
Version	Version of the test object	Version of the test object that is related to the current test result
Green results	Number of green test results inside the subtree	1 if the test result is green (Passed)
Gray results	Number of gray test results inside the subtree	1 if the test result is gray (Indifferent)
Red results	Number of red test results inside the subtree	1 if the test result is red (Failed)
Validated	Sums of the “Validated”-States of all test results inside the subtree	“Validated” state of the test result (see below)

You can get additional information via tooltip on the fields.

In the column `Validated` you can set the “Validated” status of the test result (see chapter ***5.7.5 Validating test results***).

5.8 Versioning

In PROVEtech:TA Test Manager you can control versions of your test objects, that means, freeze the state of a test object and archive it inside the database. Thus, you can fall back to older test versions, in order to be able to reproduce a test result. Versioning contains the following functionalities:

Developer Versions

After you have created a new test object, the chain of test versions first consists of a single developer version. When you save changes to a test object, the saved version is typically a developer version.

Create a release version

In the Test Manager tree, you can make a release version on individual test objects or also on entire subtrees. These release versions are normally write-protected, but can later be executed, read, referenced or copied. If you make changes on the current release version and save this, then a new developer version is automatically created. So the original release version is preserved unmodified. If you want to modify an older release version then you explicitly have to create a version branch first (see also ***5.8.1 Release a test version***).

If certain attributes of the test object (test parameters, test documents, description, test specification number or name prefix) are changed and corresponding settings are activated (see chapter ***5.3.11.2 The tab "Database settings" in the property dialog*** and ***TestItem.KeepRelease*** in the Test Language Manual), then you get a message box when saving the changes. There you can decide whether the release version should be kept instead of creating a new developer version and an entry in the history (see chapter ***5.3.9.7 The tab "History"***) should be made instead.

Create a branch

If you want to continue developing a library in a test team with several people while the main version is currently in productive use, you can create a version branch for this purpose. Please read also in chapter ***5.8.2 Create a branch***. The features of the Branch Selection are also helpful here, as described in chapter ***5.8.7 The Branch Selection***.

Restore a release version

You can restore a released version by creating a copy of it. Then you can do your changes on this copy. Thereby the reference to previous versions is preserved. Furthermore, it is possible to copy an older release version to the top of the release chain (see also ***5.8.3 Restore a test version***).

Delete a version

You can delete a test version, which means, removing it from the versioning chain. This is only possible if the following conditions are met: The test object is not part of a completely released subtree and there are still versions left in the versioning chain. You can remove release versions as well as developer version from the chain of test versions (see also ***5.8.4 Delete a test version***).

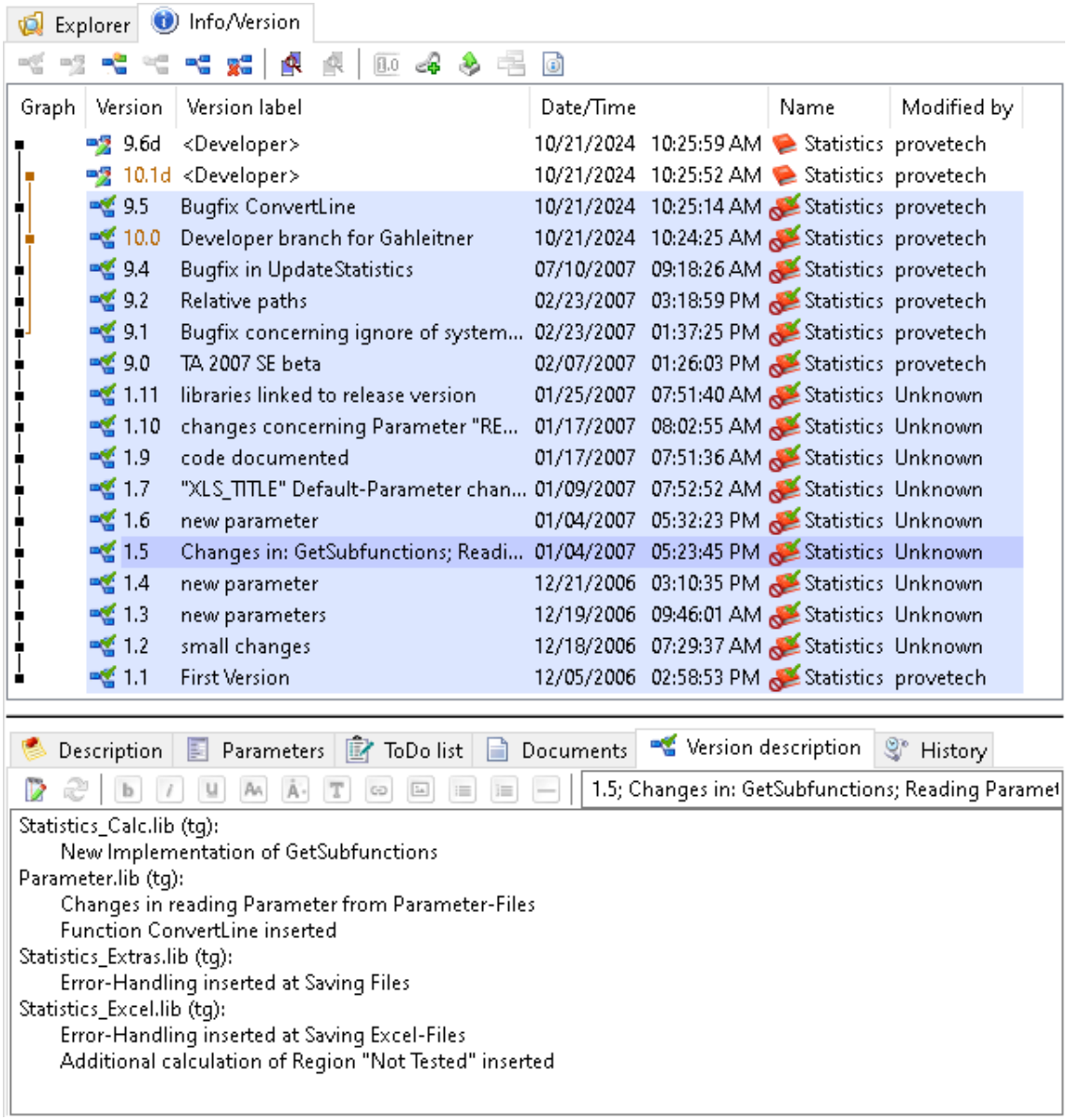


Figure 382 – The tab "Info/Version"

In the tab `Info/Version` you can see all versions of a test in a list control, ordered by the `Date/Time` column. In this case there are two developer versions which have white background color. Only these two versions can be modified if no special properties are set. All released versions are typically read-only. You can only create a copy or a new version branch to make your changes on them.

The edit field `Version description` is an exception. You can edit this for the marked release version. The description of the version is the same for all objects, which were released together. You cannot change the version description of a developer version.

The version description can be stored as formatted text based on HTML. For this purpose, some buttons for text formatting have been added to the user interface.

More detailed information on the use of the HTML-enabled description text can be found in chapter ***5.3.9.1.1 The HTML Text Edit control***.

The release versions are marked in the Test Manager tree by a green checkmark in the icon (e.g.  or ).

The test results get the version number and label of the test program at the time of execution (see also ***5.7.2 Test results of older test versions***).



The tabs `Explorer`, `Info/Version`, `Program`, `Execute`, `Results` and `Overview` display the information of the selected test object in the TM tree. If you selected an older version in the tab `Info/Version`, then the tabs on the right side `Program`, `Execute`, `Results` and `Overview` display the information of the **older** test version.



- Similar to “check in” and “check out” of a test there is a separate mechanism to lock and unlock test code (see chapter ***5.4.10 Locking the Basic code***).
- The versioning is an internal feature of PROVEtech:TA and acts on the Test Manager database. This versioning feature has nothing to do with the file system. There are no external version programs included, which e.g. manage different versions of files in the file system.



- It is not possible to overwrite (e.g.) the Basic code of a release version accidentally. But this may happen on a developer version.
- Test results executed by developer versions do not have a demand on reproducibility.
- You do not make versions from test results. They only get the version number and label from the test program at the time of execution.
- Deleting a test version means deleting its corresponding test results, too.
- The versioning does not support *Undo* and *Redo* .
- An older test version can also be referenced via several different paths if, for example, a kind of branch was created with “Copy with reference to previous version” or “Restore older release version” (see also the Test Language manual, chapter ***Test paths***).



The versioning does not support the following functions:

- '#uses-dependencies in Basic on old releases cannot be restored; therefore, you always have to release the entire subtree and the #uses-dependencies have to include relative paths
- Hardware configurations and initialization files cannot be restored

5.8.1 Release a test version

You can create a new release version from a developer version when you click on the button . This is possible for all test object types (except for the root object and the trash folder).

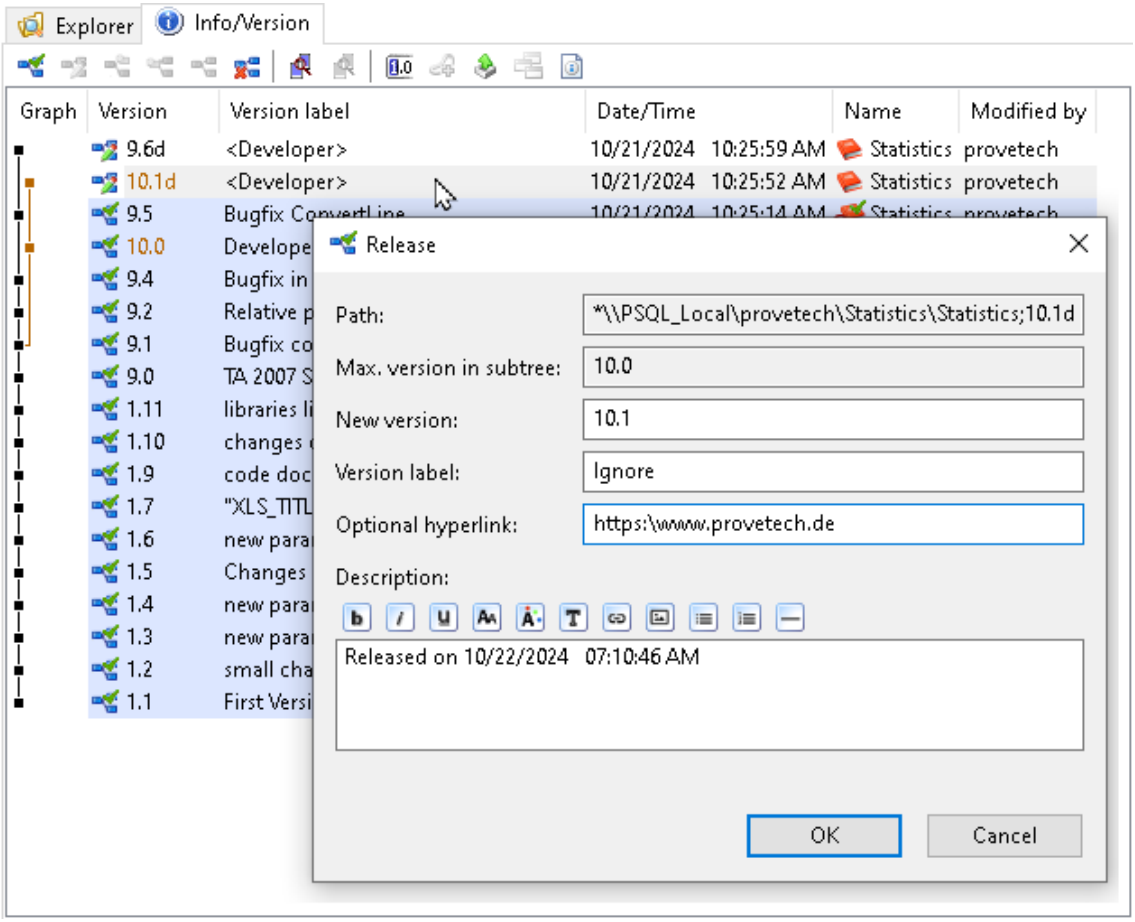


Figure 383 – Creating a release version

When you create a release version, a new version number and a standard description are set in the Release dialog. You can still change the version number manually and must assign a version label. This could, for example, describe the project or vehicle for which this version was created (SAF 09, BR 211, ...). The specification of a version description is optional and can still be changed later.
The version description can be stored as formatted text based on HTML. For this purpose, some buttons for text formatting have been added to the user interface.
More detailed information on the use of the HTML-enabled description text can be found in chapter ***5.3.9.1.1 The HTML Text Edit control***.

It is possible to specify a corresponding hyperlink for the version label. This can either link to

- an internet or intranet page (prefix **html://**)
- a document in the file system (prefix **file://**)
- an attachment in the same or another test (prefix **TestAutomation:**)

This hyperlink can only be attached to a release version, not to a developer version. However, it is possible to edit this hyperlink even for older release versions. For this purpose, the button `Add hyperlink` or `Edit hyperlink` can be used.
See also chapter ***5.16 Supported URL formats***.

On the `Info/Version` tab, you can then jump to the linked document by clicking on the version label (while holding down the control key).

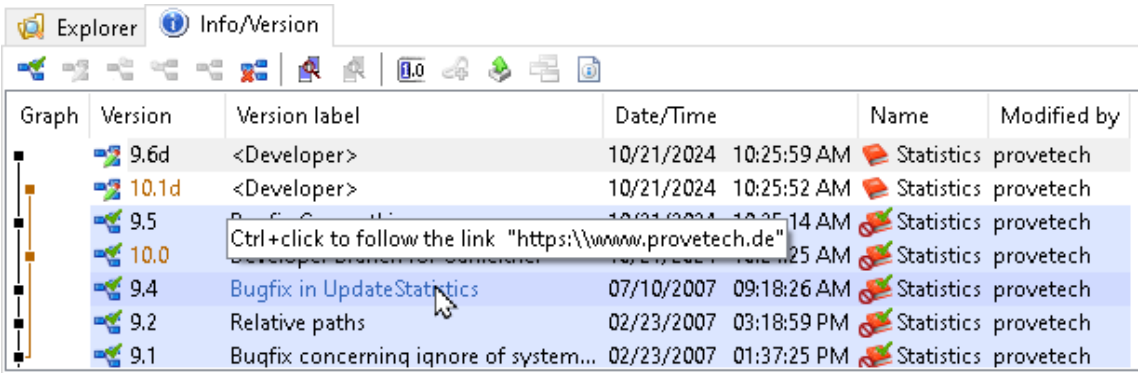


Figure 384 – Version label with hyperlink

If the current version is a release version and you modify it, then typically, a new developer version is automatically created when saving the test object. The version number is incremented and a “d” like “developer” is added at the end (see ***5.8.5 The version number***). A new copy of the release version is created, which becomes the new current version. You can see this in the tab “Versions”.

When a test version changes to a developer version (after saving the modifications on the current release version, e.g. modify Basic code or description or test parameters) then all parent system tests become (or stay) a developer version.

When you create a release version from a test object with child objects (system test, user library, project, or library), then a release version is always created for all tests in the subtree, with the same version number and version label. This leads to the following: If the root of a subtree is a release version, all child objects are also release versions related to the same version. Conversely, if a test object is a developer version, its father is also a developer version.

If you are in a version branch of a subtree, you can only create a new release on its root.

Older release versions are typically read-only. They can only be modified on a branch or a copy. Please read chapter ***5.8.2 Create a branch*** how to create a branch. You can create a copy of a release version if you restore a test version (see ***5.8.3 Restore a test version***). The only exception is when the release version is the current version. Then the developer version is created automatically when saving your changes, as long as there are no special properties set.

In the dialog for creating a release version you can define the version description text on your own. Therefore, set a reference to a text file in the TM.ini, section `[General]`,

key `ReleaseDialogTextPath` (see ***Administrator's Guide***). The text in this file can also contain variables with the following meaning:

%d	Current date
%u	Current PROVEtech:TA user
%w	Current windows user
%%	Percent sign



If you perform a release of a large subtree, this operation can take some time. During the operation, all affected test versions in the subtree are locked in the database and are therefore only available for changes from other PROVEtech:TA clients afterwards.

5.8.1.1 Pre-process before creating a release version

In the Settings dialog, you can specify a Test Manager path to a Basic handler that is called before creating a release version (see chapter ***3.2.7.5 Test Manager***). Here you can still edit the test object or check whether the release operation must be canceled.

The following is an example source code. The interface is predefined. The content of the function can be freely designed.

```
'#Language "WWB.NET"
Option UsesAmbiguousError On
Option Explicit

'Callback function which is called after modifying the test item
'[in] ti:         Test object to be released
'[in] nV1:        New major version number of release version
'[in] nV2:        New minor version number of release version
'[in] strLabel:   New version label of release version
'[out] bAbort:    Set to false to return an error and abort the release
operation
Sub BeforeReleaseTestItem(ByVal ti As TestItem, ByVal nV1 As Long,
ByVal nV2 As Long, _
        ByVal strLabel As String, ByRef bAbort As Boolean)
    'For debugging purpose: Uncomment the following lines
    bAbort = False
    Dim str As String = "Path: " & ti.Path & vbCrLf & "Version: " &
CStr(nV1) & "." & CStr(nV2) _
        & vbCrLf & "Label: " & strLabel
    Dim b As VbMsgBoxResult = MsgBox(str, VbMsgBoxStyle.vbOkCancel,
"BeforeReleaseTestItem")
    If b = vbCancel Then bAbort = True
End Sub
```

5.8.2 Create a branch

You can create a version branch using the toolbar or context menu with the release version selected. This is useful if, for example, you want to further develop a library together with several people while the main branch is in productive use.

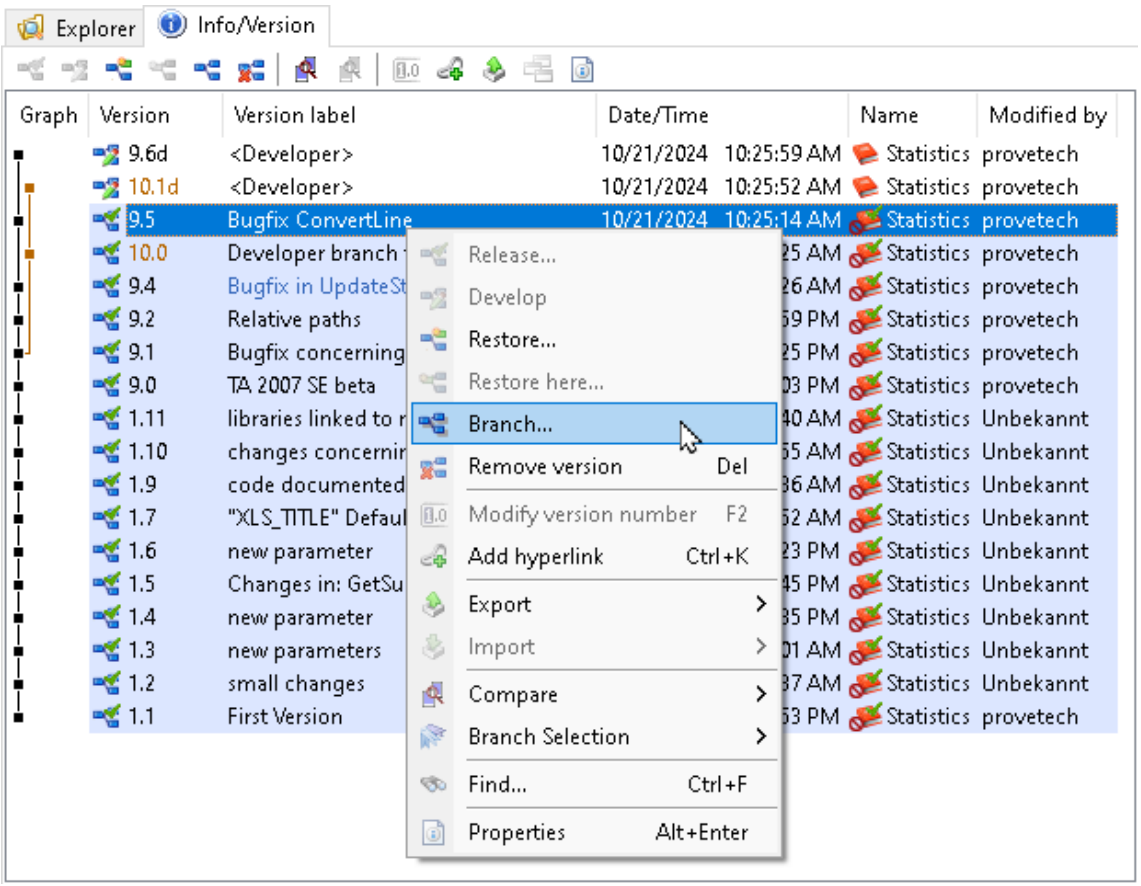


Figure 385 – Creating a version branch

The branch then appears as a fork with a new color in the version graph. You can create further release and developer versions as well as further branches on this branch. You can change individual attributes in the developer version of the branch or the entire tree structure if the branch is a subtree.

If you later want to merge the changes in your branch back into the main branch, you can use the comparison dialog (see chapter ***5.3.13 Comparing and merging test code***).

You can use the branch selection features to make changes and test your version branch (see chapter ***5.8.7 The Branch Selection***).

5.8.3 Restore a test version

To restore old release versions, you can choose the location in the Test Manager tree, where the release version should be included. In case of a released subtree, the restore operation copies the entire subtree.

To create a copy of an old release version, click on the button 

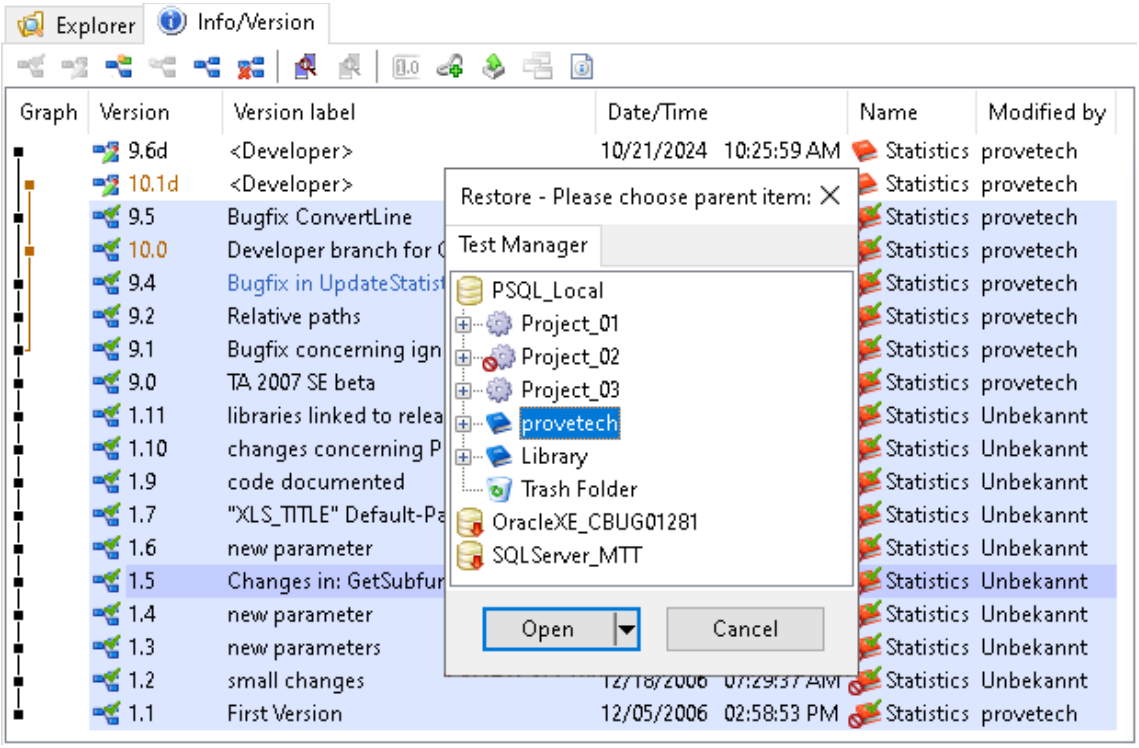


Figure 386 – Restore a test version

When a release version is restored, a kind of “branch” is created. The new copy gets the same previous test version as the original version.



Restoring a release version means creating a kind of branch.

Another way, to restore an older version, but only for elementary tests, is offered by the context menu entry "`Restore here…`" (see ***Figure 387***). Then the whole content of the older test version is copied into the current version and after that the current version is released. So, you do not create a version branch. This operation is deactivated for system tests, libraries and projects.

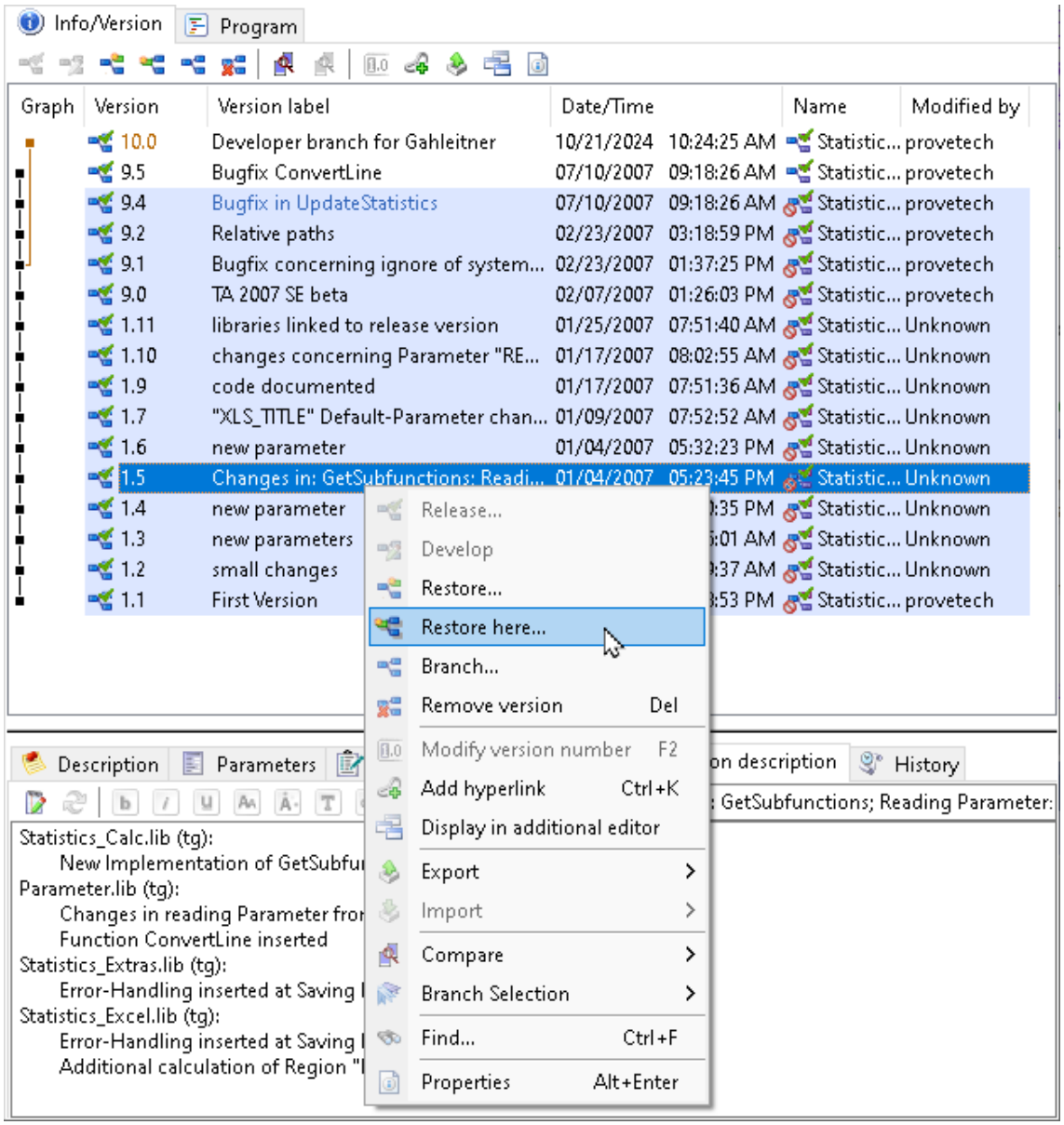


Figure 387 – The context menu entry "Restore here…"

5.8.4 Delete a test version

You can delete a selected version of a test object by clicking on the button .

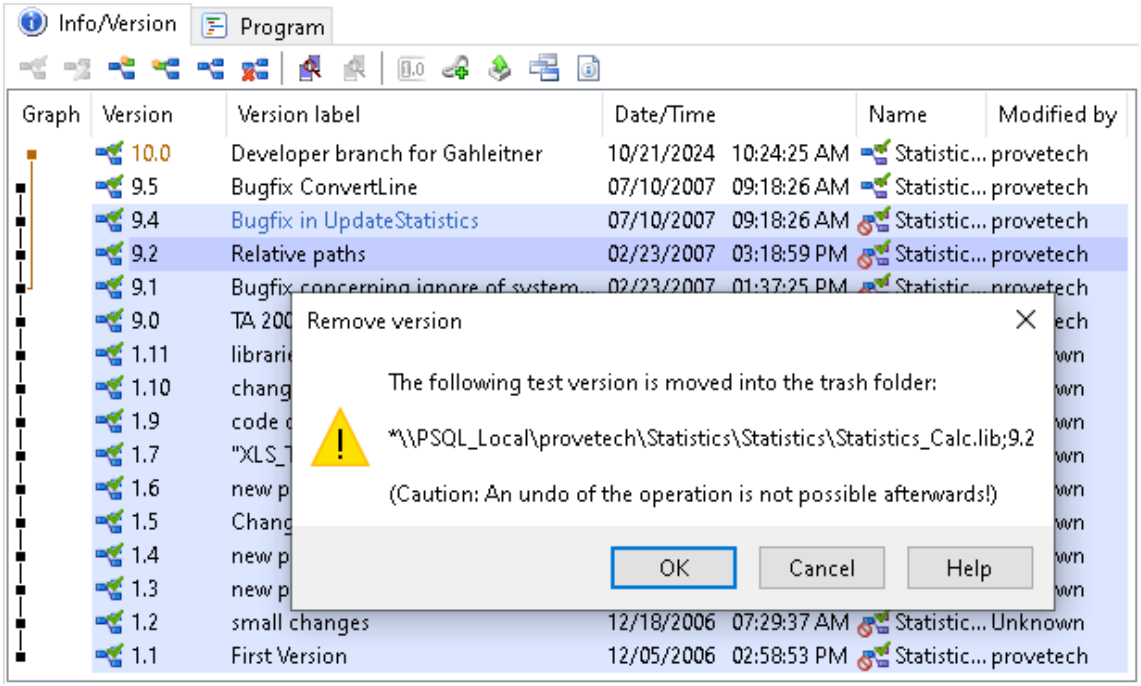


Figure 388 – Deleting a developer version

There are some requirements to be able to delete a single test version:

- The test object must contain further test versions.
- A test version cannot be a current version and an older version at the same time (e.g. in a version branch)
- The test version you want to delete must not be part of a released subtree. That means, the parent object must not be a release version.
- When deleting versions through subtrees, all contained links must be either release versions or developer versions

Deleted test versions are first moved to the trash folder. When emptying the trash folder, older versions get permanently deleted from the database if there are no more dependencies on this version (see chapter ***5.2.4 The trash folder***). Exceptions are older project and library versions, which are deleted directly without any intermediate steps.

If you use linked test objects then each link is handled the same (see chapter ***5.3.14 Linked test objects***). That means, it is not possible that one link is a release version and the other link is a developer version.



Deleting a test version means deleting its corresponding test results, too.

Deleting a current test version across a versioned subtree (e.g. system test) also results in the deletion of the containing test objects.

5.8.5 The version number

Each test object gets a version number that consists of 2 parts (e.g. 2.13 or 1.0). If the test version is a developer version then a “d” is added at the end (e.g. 2.13d). For example, the first number could be incremented for new functions and the second number for bug fixes. Or, the first number is incremented when a version branch is created and the second number is incremented as long as you are on the same version line.

The test result also gets a version number. It is the version of the test program at the time of execution.

You can release complete subtrees (when it is a system test, user library, project or library). Then the maximum of all including tests is considered. All subtree objects get the same version number.

If you create a new developer version, the second part of the version number is automatically incremented. You can adjust this version number after that. If you release this version later then this version number can be kept.

5.8.6 Executing older test versions

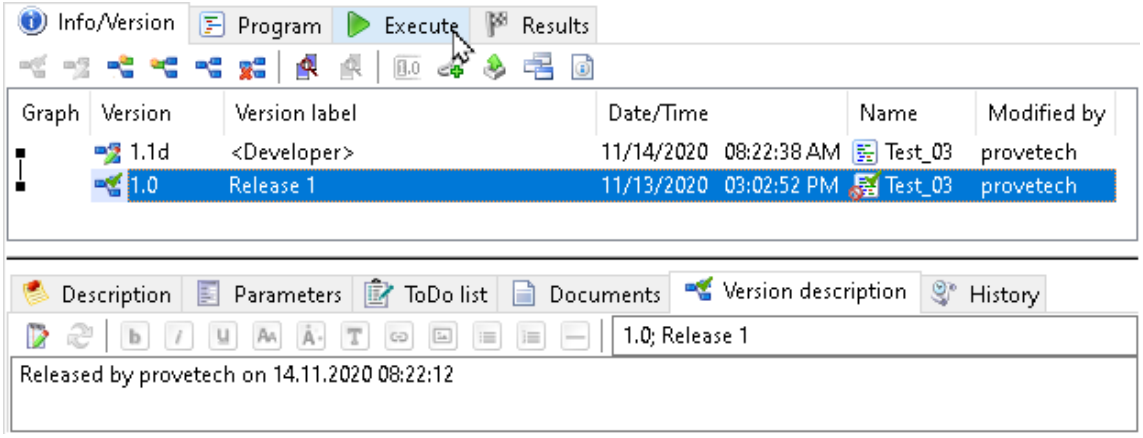


Figure 389 – Selecting older test version

It is possible to execute an older version of a test object and create a test result. To do this, simply select the test version in the `Versions` tab and change to the `Execute` tab (see ***Figure 389***).

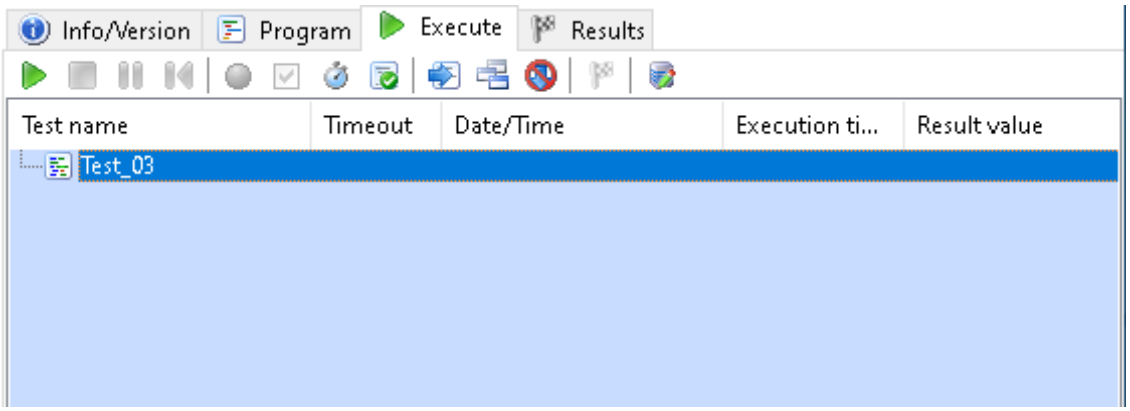


Figure 390 – Executing an older test version

In the Execute tab you can see the version number of the test object in the path displayed at top of the window (see ***Figure 390***). The test can then be executed as usual.

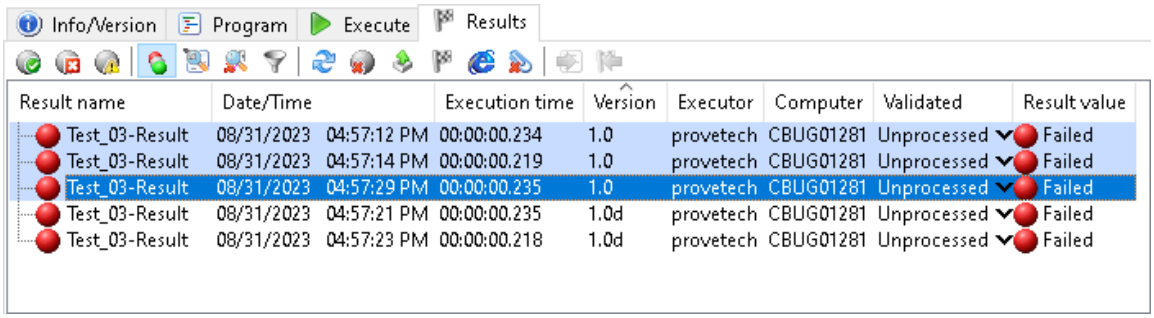


Figure 391 – Result list after executing an older test version

In the `Results` tab (see ***Figure 391***) the new test result is not listed at the bottom because of the sorting by `Version`. You can also see this when you look at the `Date/Time` of execution.



When executing older test versions attention must be paid to using the correct library paths in the '#uses commands.

Please read also in chapter ***5.8.7 The Branch Selection*** and in the section ***Test Language*** in chapter ***Test paths***.

5.8.7 The Branch Selection

In the PROVEtech:TA Test Manager, a branch selection is a collection of test versions that replaces the current test version. The replacements can be spread across the entire

Test Manager tree and even across several connected PROVEtech:TA databases. A branch selection entry can be an older release or developer test version, or a test version from a branch. This can either consist of many individual elementary tests or a version of one or more sub-trees. The requirement for subtrees is that the creation of a release has also taken place via the complete subtree.



Version branches do not necessarily have to exist for this branch selection feature to be used. Even in a linear version chain of the current and older versions, you can go back to an older version using branch selection.

The branch selection applies to the PROVEtech:TA session and is stored in the working memory. However, it can also be saved in a file or a test document, and loaded from there, and automatically reloaded when you start PROVEtech:TA. The data of the database is not affected by the branch selection and remains untouched. The different storage locations (data in the database, branch selection at file level) make it possible for each PROVEtech:TA user to configure their own branch selection.

Once the branch selection has been loaded, the current test object is replaced by the older test version or branch at the corresponding positions in the Test Manager tree, and the version number is also displayed. If this is read-only, it is highlighted in light blue. Test paths without version information then refer to the loaded branch selection.

The test versions belonging to the Branch Selection can be executed again. Included '#Uses statements without a concrete version specification now take the current Branch Selection into account..

The current Branch Selection can be modified via the context menu of the Test Manager tree (see chapter ***5.8.7.1 Define the branch selection by context menu***).

The currently active Branch Selection is visible in an overview dialog. You can also manage the Branch Selection here. Read chapter ***5.8.7.2 The branch selection overview dialog***.

Please read also in chapter ***5.8.6 Executing older test versions*** and in the section ***Test Language*** in chapter ***Test paths***.

5.8.7.1 Define the branch selection by context menu

You can reach the menu items for Branch Selection via context menu on a test object or a test version.

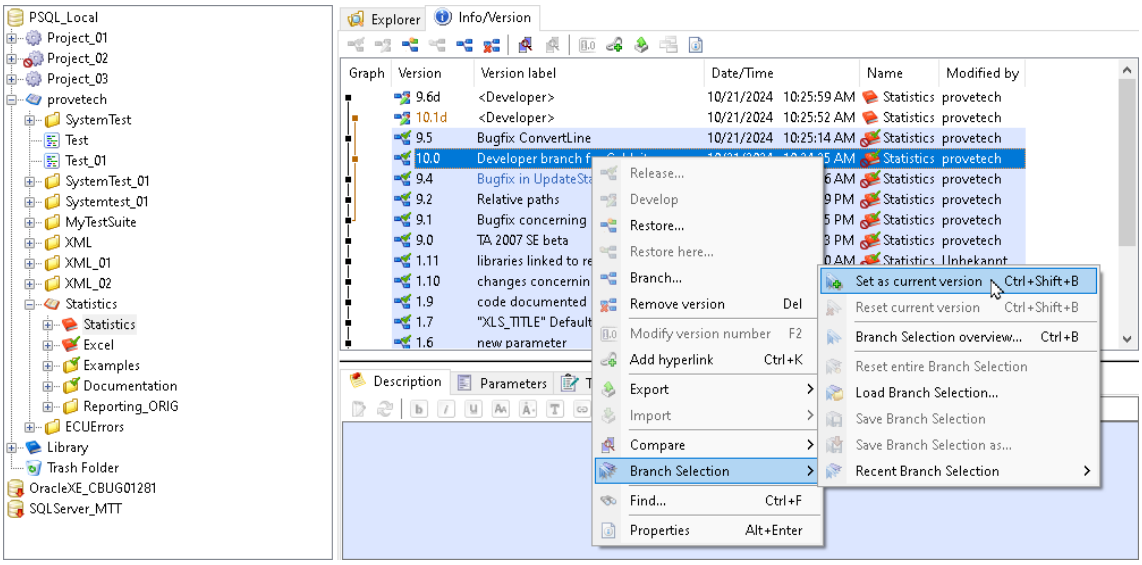


Figure 392 – Context menu for Branch Selection

The Branch Selection feature of the PROVEtech:TA Test Manager has a global replacement table, stored internally in the working memory, in which pairs of current test versions and older release versions or branch versions are stored. By default, this table is empty, which means that no replacements take place.

Via *Set as current version* and *Reset current version* you can now fill this replacement table or empty it again. This is also possible with the keyboard shortcut Ctrl+Shift+B.

Via *Branch Selection overview...* you can open the overview dialog, which is described in more detail in chapter ***5.8.7.2 The branch selection overview dialog***.

Use the *Reset entire Branch Selection* menu item to clear the entire replacement table so that the current entry is displayed again at each position in the Test Manager tree.

Load and save a predefined Branch Selection using *Load Branch Selection...*, *Save Branch Selection*, *Save Branch Selection As...* and *Recent Branch Selection*.

The following should be considered when using Branch Selection:

- If subtrees (in contrast to element tests) are included, you can control them exclusively via the root entry of the subtree, but you cannot make additional replacements within the subtree.
- If you have made replacements in parent and child structures, parent replacements will overwrite child replacements. That is, subordinate replacements are then ignored.
- If tests are being executed while modifying the current Branch Selection, unforeseen effects may occur.

- Deleting a test version from the current or even another PROVEtech:TA client (i.e. from another computer) can lead to unwanted effects if this test version belonged to a Branch Selection replacement table.
- If an older test version is set in the Branch Selection, its newer test versions and associated test results are hidden.

5.8.7.2 The branch selection overview dialog

The Branch Selection overview dialog opens via the context menu in the Test Manager tree or via the hotkey Ctrl+B.

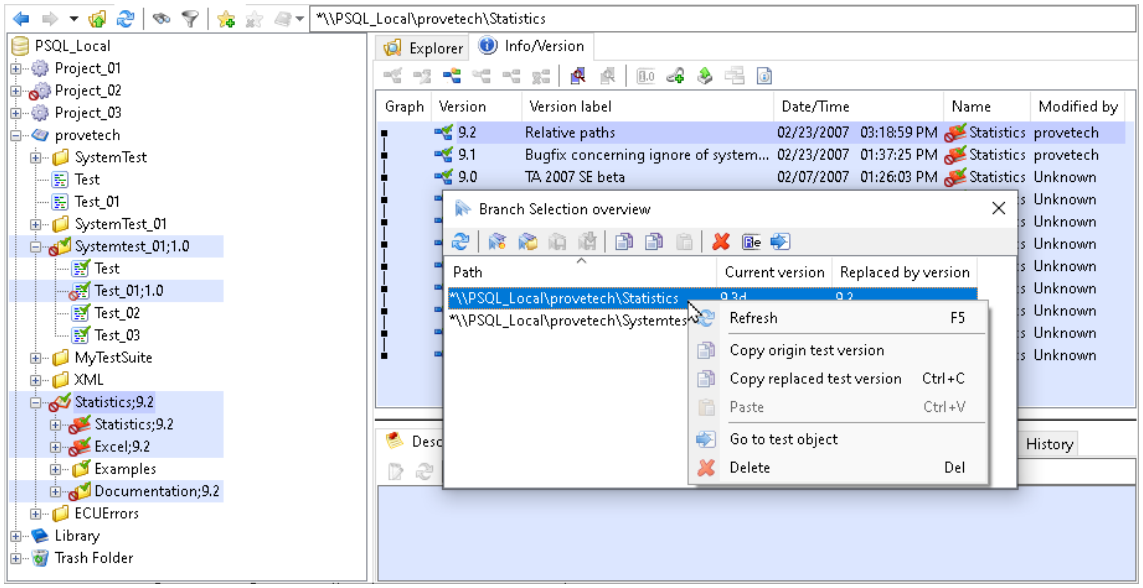


Figure 393 – The Branch Selection Overview dialog

All currently active replacements are displayed here in a list. In addition to the functions mentioned in chapter ***5.8.7.1 Define the branch selection by context menu***, the toolbar and the context menu also provide functions for refreshing, copying to the clipboard and pasting from the clipboard.

Go to test object selects the test object in the Test Manager tree.

5.9 User-defined test objects and extending the context menu

User-defined test objects are similar to elementary tests. But they contain no Basic code as elementary tests, they contain one or several documents (e.g. Excel file, text file or a file format of a proprietary software, also see chapter ***5.2.1.6 The user-defined test***

object and ***5.3.9.4 The tab "Documents"***). The extended context menu helps you to manage and access the documents of the user-defined test objects by additional WinWrap Basic scripts.

If you want to extend the context menu in the Test Manager tree or the context menu of the test results, you can choose a special Test Manager path via main menu *Application|Settings...* in the tab "Test Manager". The structure of the including test objects determines whether the popup menu *Extended* appears in the context menu and which names the menu entries get (see ***3.2.7.5 Test Manager*** in the Settings dialog).



If no `Path for extended context menu` is defined in *Application|Settings...* in the tab "Test Manager" then the popup menu *Extended* does not appear in the context menu of the Test Manager tree.

This chapter contains the following subchapters:

- ***5.9.1 The structure in the path for the extended context menu***
- ***5.9.2 Creating a new user-defined test object***
- ***5.9.3 Extending the context menu***
- ***5.9.4 Executing user-defined test objects***
- ***5.9.5 Embedded editors***

5.9.1 The structure in the path for the extended context menu

The structure inside your special Test Manager path for the extended context menu could look like this (see ***Figure 394***).

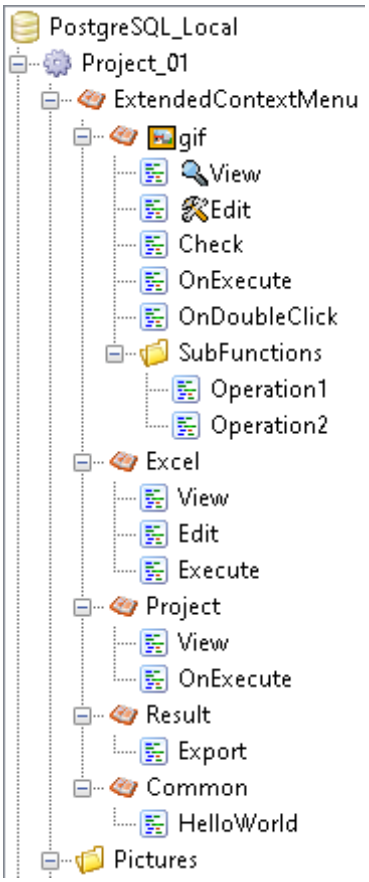


Figure 394 – Structure for the extended context menu

In this case the path for the extended context menu would be `*Project_01:\ExtendedContextMenu`.

The libraries `gif` and `Excel` handle the user-defined test objects with the type name `gif` and `Excel`. The library `Project` handles the project object, `Result` refers to test results and the library `Common` corresponds to all test objects and test results.

You can also set an additional user-defined emoji from the Unicode character table as a prefix to the test object name (e.g. 🖼 or 🔍). This will then appear as an icon in the extended context menu (see chapter ***5.3.2 Renaming tests***).

In the following table you can see the name for each type of test object that you must take for the name of the library or system test.

Type of test object	**Name for the library**
Test Manager (Root)	`TestManager`
Trash Folder	`TrashFolder`
Project	`Project`

System Test	`SystemTest`
Library	`Library`
Elementary test	`Test`
Object module	`Object`
Class module	`Class`
Text object	`Text`
User Library	`UserLibrary`
User-defined test object	Type name of the user-defined test object (e.g. gif, Excel)
Test results	`Result`
All types of test object	`Common`

All the names of the elementary tests that are defined inside the library should appear again in the context menu *Extended* as a menu entry for the corresponding test object or test result. They also could be hierarchically structured as you can see at the system test `SubFunctions`. If you choose one of these menu entries then the corresponding elementary test in the special path is automatically executed.

The elementary tests with name `OnCreate`, `OnExecute`, `OnSave` and `OnDoubleClick` have got special meanings, as shown in the following table. These key names do not appear as a menu entry in the popup menu *Extended*.

Test object name in the extended context menu	**Time of execution**
`OnCreate`	Is executed when creating a new test object (not valid for test results)
`OnDoubleClick`	Is executed after a double-click on an elementary test (not valid for other test objects or test results)
`OnExecute`	Is used for the Test Manager tab "Execute" of user-defined test objects (not valid for other test objects or test results)
`OnSave`	Is executed when saving Basic code or a test object's description (not valid for test results)

5.9.2 Creating a new user-defined test object

If you have already defined special type names for user-defined test objects in your path for extended context menu then the popup menu for creating a new user-defined test object looks like the following (see ***Figure 395***).

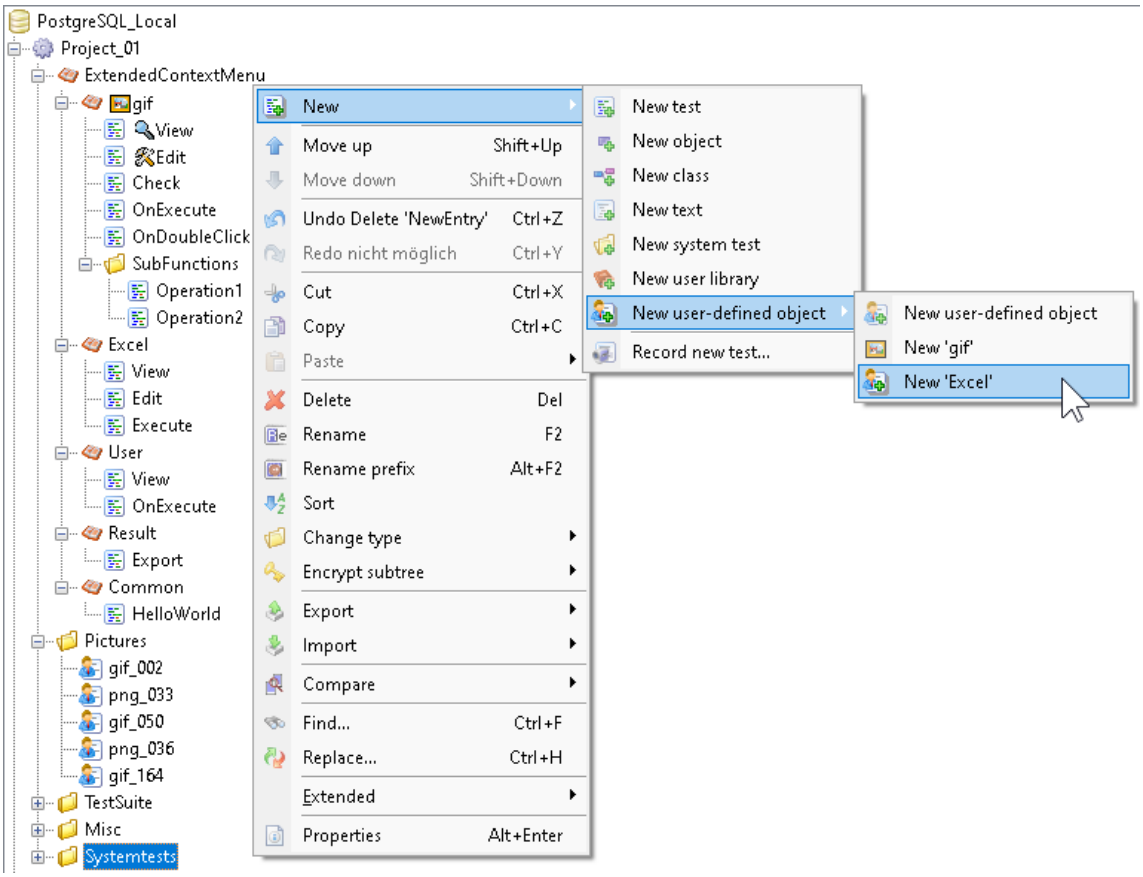


Figure 395 – Creating a new user-defined test object

If you choose *New 'gif'* or *New 'Excel'* in this example, then a new user-defined test object with type name `gif` or `Excel` is immediately created. In the second step you have to import one or several files for the user-defined test object (see ***5.3.9.4 The tab "Documents"***). The libraries `User` and `Common` do not appear in the popup menu because of their special meaning.

5.9.3 Extending the context menu

Please take a look at the following example (see ***Figure 396***).

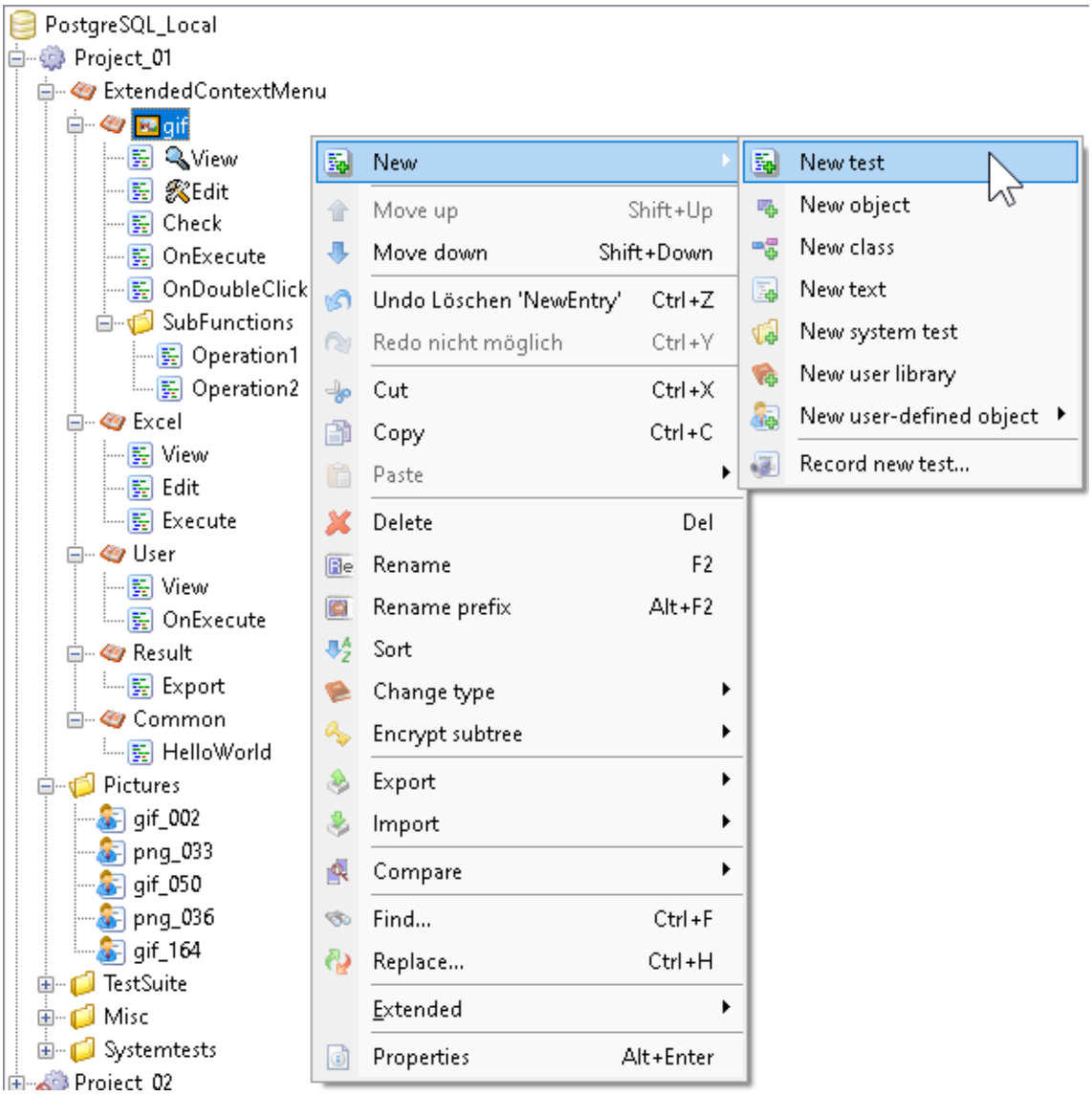


Figure 396 – New menu entry for the context menu "Extended"

If you want to insert a new menu entry for the user-defined test object with type name "gif" in the popup menu *Extended* of the context menu in the Test Manager tree, then insert a new elementary test inside the library `gif`. After that you set the name of the elementary test. This name should also appear in the menu now.

You can also set an additional user-defined emoji from the Unicode character table as a prefix to the test object name (e.g. 🖼 or 🔍). This will then appear as an icon in the extended context menu (see chapter ***5.3.2 Renaming tests***).

When you open the popup menu *Extended* of a user-defined test object with type name "gif", you can see the new menu entry (see ***Figure 397***).

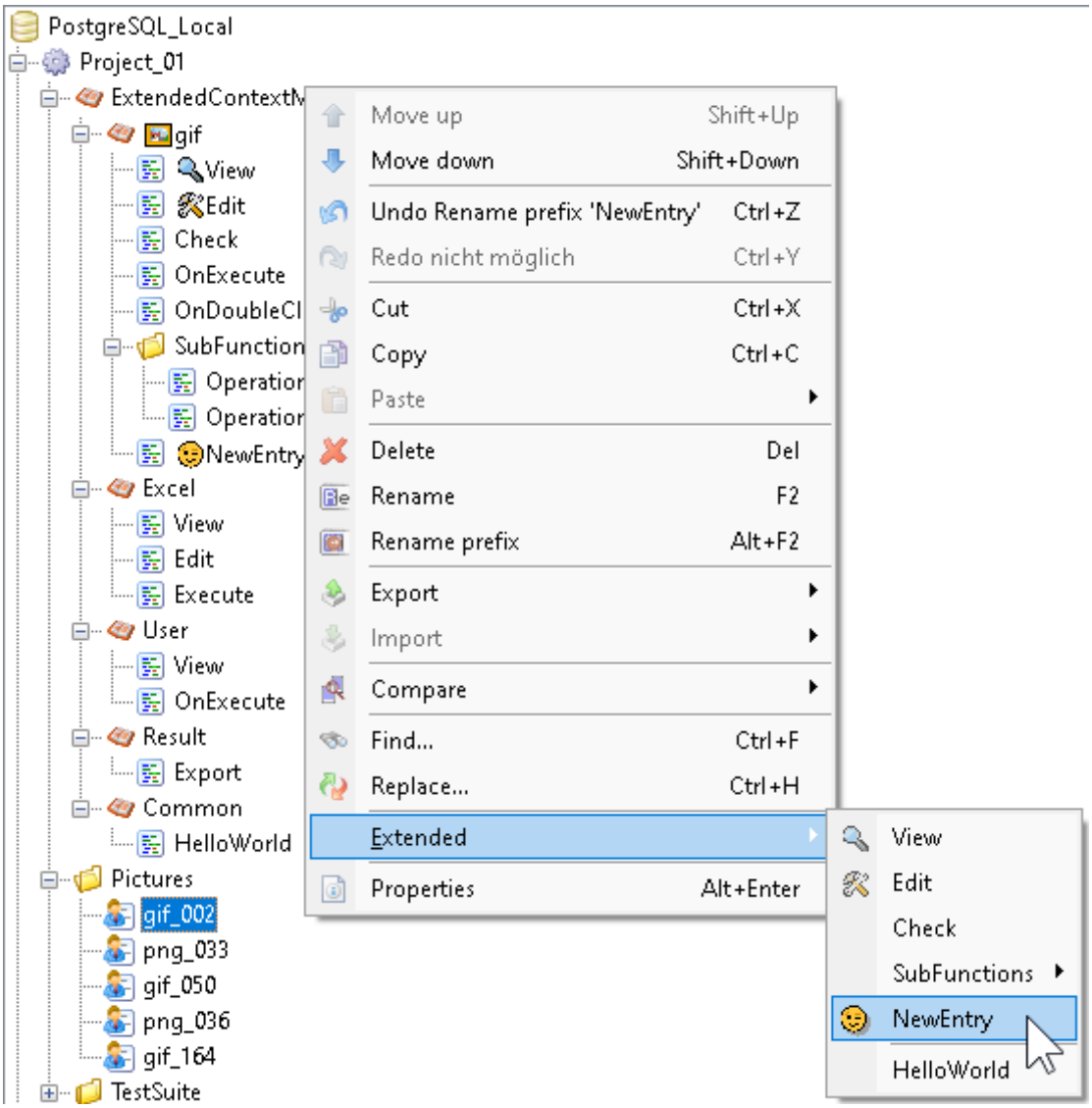


Figure 397 – New menu entry of the context menu "Extended"

If you choose the new menu entry *NewEntry* then the corresponding elementary test with path `*Project_01:\ExtendedContextMenu\gif\NewEntry` is executed.



After executing a menu entry, you can see the result value when you look at the color of the button  in the toolbar.

For example, the Basic code for opening a user-defined test object that includes an Excel file, could look like the following (see ***Figure 398***). The global variable

OriginPath (accessible via the command **System.GetGVar**) contains the Test Manager path to the executed user-defined test object.

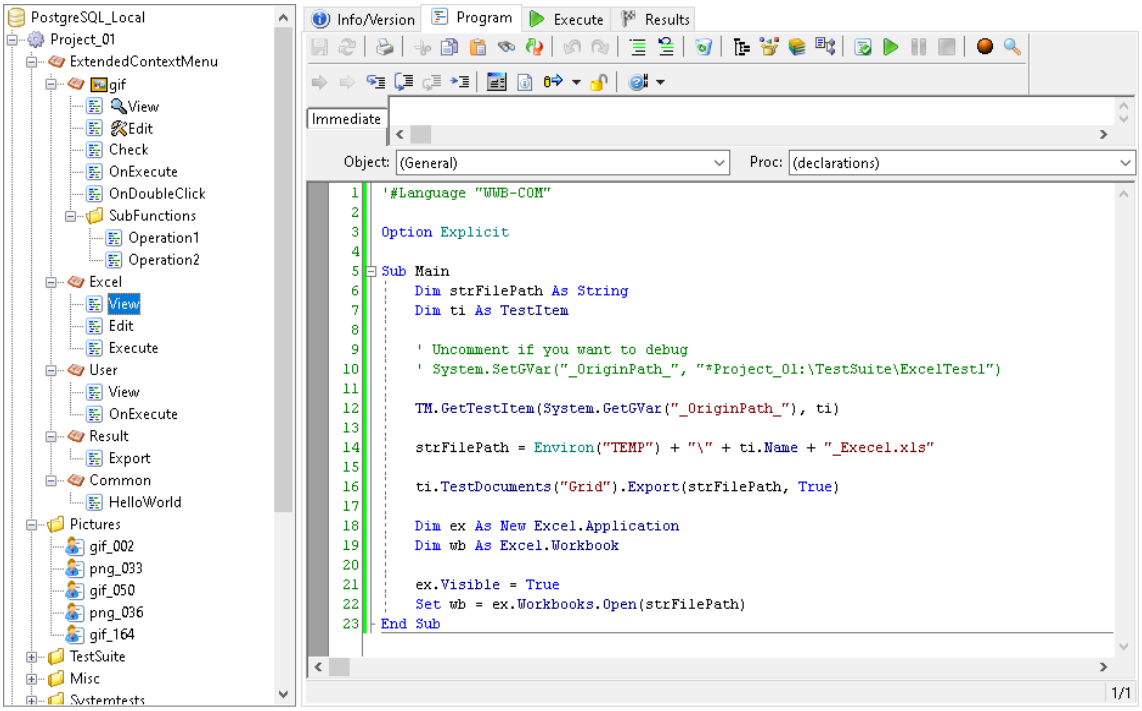


Figure 398 – Basic code of the menu entry

Here you can see the corresponding WinWrap Basic code again. Please do not forget to add the reference to the Excel Type Library by WinWrap Basic toolbar.

```
'#Language "WWB-COM"
Option Explicit

Sub Main
    Dim strFilePath As String
    Dim ti As TestItem

    'uncomment if you want to debug
    'System.SetGVar("_OriginPath_", "*Project_01:\TestSuite\ExcelTest1")

    TM.GetTestItem(System.GetGVar("_OriginPath_"), ti)

    strFilePath=Environ("TEMP")+"\"+ti.Name+"_Execel.xls"
    ti.TestDocuments("Grid").Export(strFilePath, True)

    Dim ex As New Excel.Application
    Dim wb As Excel.Workbook

    ex.Visible=True
    Set wb=ex.Workbooks.Open(strFilePath)
End Sub
```

There are three reserved global variables (accessible via the command **System.GetGVar**) that could be helpful when using the extended context menu.

- `_ExecutionMode_` contains information about how a test was started
- `_OriginPath_` contains the Test Manager path of the original test object when executed via the extended context menu
- `_OriginTestResults_` contains a `VariantList` of the selected test results of type `TestResult` when executed via the extended context menu

You can read more about this in the Test Language manual.

The following is an example of accessing test results:

```
'#Language "WWB.NET"
Sub Main
    Dim vl As VariantList = System.GetGVar("_OriginTestResults_")
    Dim tr As TestResult
    If Not IsNothing(vl) Then
        'loop through all selected results
        For Each tr In vl
            MsgBox tr.Path(True)
        Next
    End If
End Sub
```

You can also set keyboard shortcuts in the menu items of the extended context menu to activate the menu item by pressing "Context menu -> E -> Shortcut". To do this, use the "&" character within the corresponding test object name.

5.9.4 Executing user-defined test objects

If you have defined an elementary test with name `OnExecute` for a special user-defined test object in the path for the extended context menu, then all the user-defined test objects with this type name can be executed in the tab "Execute". The corresponding elementary test `OnExecute` of the user-defined test object is thereby started. If there is no elementary test `OnExecute` defined for a special type name of user-defined test objects, then these objects do not appear in the tab "Execute" (see ***Figure 399***).

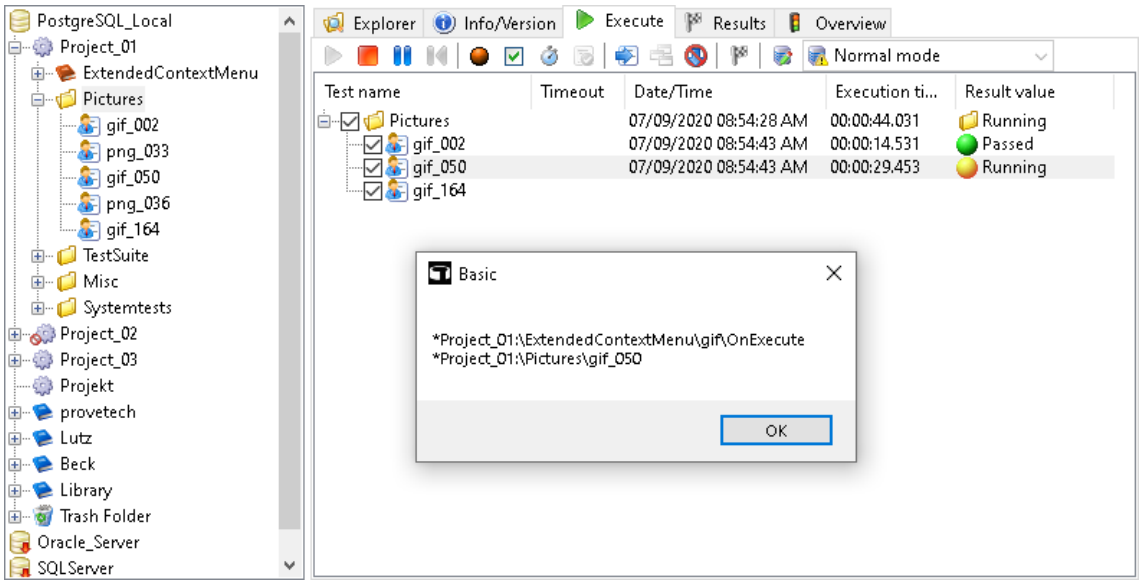


Figure 399 – Executing user-defined test objects

In ***Figure 400*** you can see the corresponding Basic code.

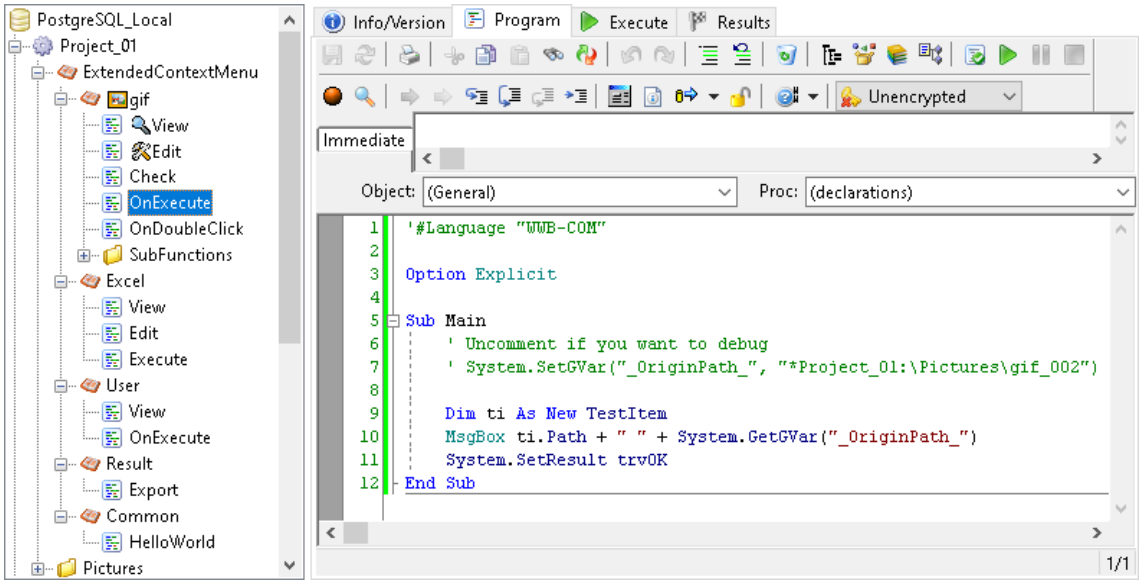


Figure 400 – Basic code of OnExecute

Here you can see the corresponding WinWrap Basic code again:

```
'#Language "WWB-COM"
Option Explicit

Sub Main
    'uncomment if you want to debug
    'System.SetGVar("_OriginPath_", "*Project_01:\Pictures\gif_002")

    Dim ti As New TestItem
    MsgBox ti.Path+" "+System.GetGVar("_OriginPath_")
    System.SetResult trvPass
End Sub
```

The global variable `_OriginPath_` is automatically set to the Test Manager path of the user-defined test object when starting the object. You can use this to work on the user-defined test object in the Basic script.



For debugging the Basic code in ***Figure 400***, just set the global variable `_OriginPath_` to the Test Manager path of the user-defined test object. This shows the comment line in the Basic script. The same applies to the global WinWrap Basic variable `_OriginTestResults_` and the currently selected test results. You can find more in the ***Test Language manual*** and in the chapter ***5.9.3 Extending the context menu***.

5.9.5 Embedded editors

In PROVEtech:TA it is possible to embed external editors or third-party software via Plug-In interface. So, you can for example display and edit a special document of a test object inside the Test Manager. To be able to do this it is necessary to implement the functions of a COM interface in an intermediate layer and register it. This means that the Plug-In is not part of the PROVEtech:TA installation package, but the graphical user interface, the tab in the settings dialog and extensions in the context menu of the Test Manager tree are usually programmed by the PROVEtech:TA users themselves.

The Plug-Ins must be activated via licensing and are registered inside the configuration file TM.ini (see ***Administrator's Guide***). If you want to know more about the embedded editor interface, please contact our PROVEtech support (provetech@akkodis.com).

Regarding the integration of third-party software, the following chapters could also be of interest:

- Chapter ***5.15 Access from foreign software via URI***
- Chapter ***2.7 Command line parameters***
- Chapter ***5.9.3 Extending the context menu***
- Chapters ***Accessing foreign software*** and ***Access from foreign software*** in the Test Language Manual

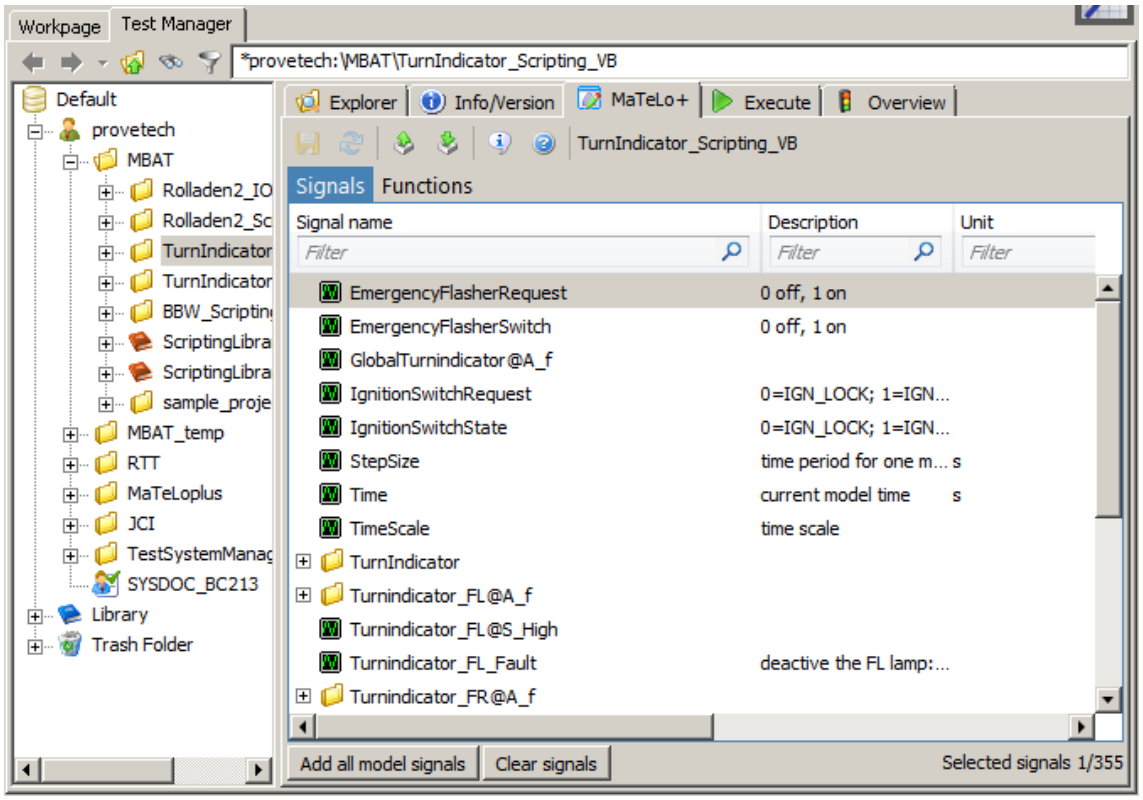


Figure 401 – PROVEtech:TA MaTeLo+ (model-based testing tool) as an embedded editor

5.10 The Quota mechanism

The PROVEtech:TA administrator can specify the database quota for each project or (global) library via the properties dialog of a test object, “Security” tab (see ***5.3.11.4 The tab "Security" in the property dialog***). Here you can also see the current database space consumption for the project or library. You can click on the button  to refresh this value.

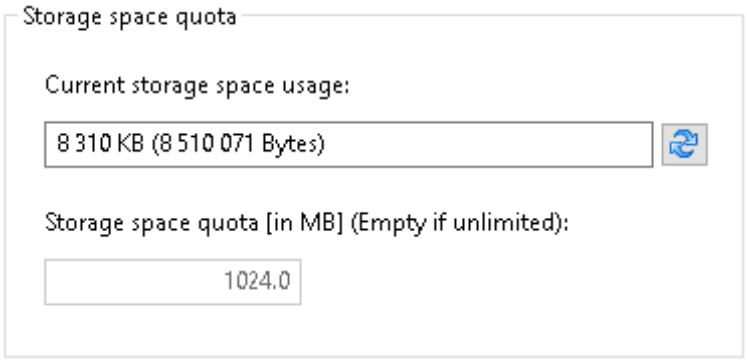


Figure 402 – Storage space quota

Each time a document, parameter, runtime error, ECU error, keyword value, attachment or protocol is added to the database the following quota algorithm is executed:

1. Get the current quota set for the corresponding project or library (this value is kept in the RAM).
2. If no quota is set, then the operation is executed and completed.
3. Get the value for the current database memory usage (this value is kept in the RAM and recalculated every hour).
4. If the used memory is reduced by overwriting the data then the operation can be executed and completed.
5. In the case of a test protocol the operation is executed and completed without consulting the quota (protocols are accumulated, but never blocked).
6. If the quota is not surpassed then the operation is executed and completed (when 95% of the quota is consumed the PROVEtech:TA Log gives a warning).

(From this point on assume that the quota has been surpassed)

7. Delete all data records that were only marked for deletion from the database.
8. Retrieve once more the quota and the current database memory consumption directly from the database.
9. If the quota is no longer surpassed, execute and complete the action.
10. If the quota is still surpassed, reverse the operation and generate an error message.

For more on this topic, see chapter ***5.3.8.1 The user***.

The project reference of the database objects (test documents, test protocols, attachments, …) is determined only once when creating the object. If you move the object into a different project, it nevertheless keeps the project reference to the original project.

In order to reduce the current storage space you have to empty the trash folder or purge the database. Therefore, please read chapter ***5.2.4 The trash folder***.

5.11 Create and restore a database backup

All Test Manager data in PROVEtech:TA is stored in a database. As a safety measure, you should regularly backup this database. The database backup mechanism in PROVEtech:TA aids in this purpose by generating periodical database backup reminder messages. In the event of a disk crash or accidental deletion of data, the lost data would be irretrievable.

When you terminate PROVEtech:TA a database backup reminder appears.

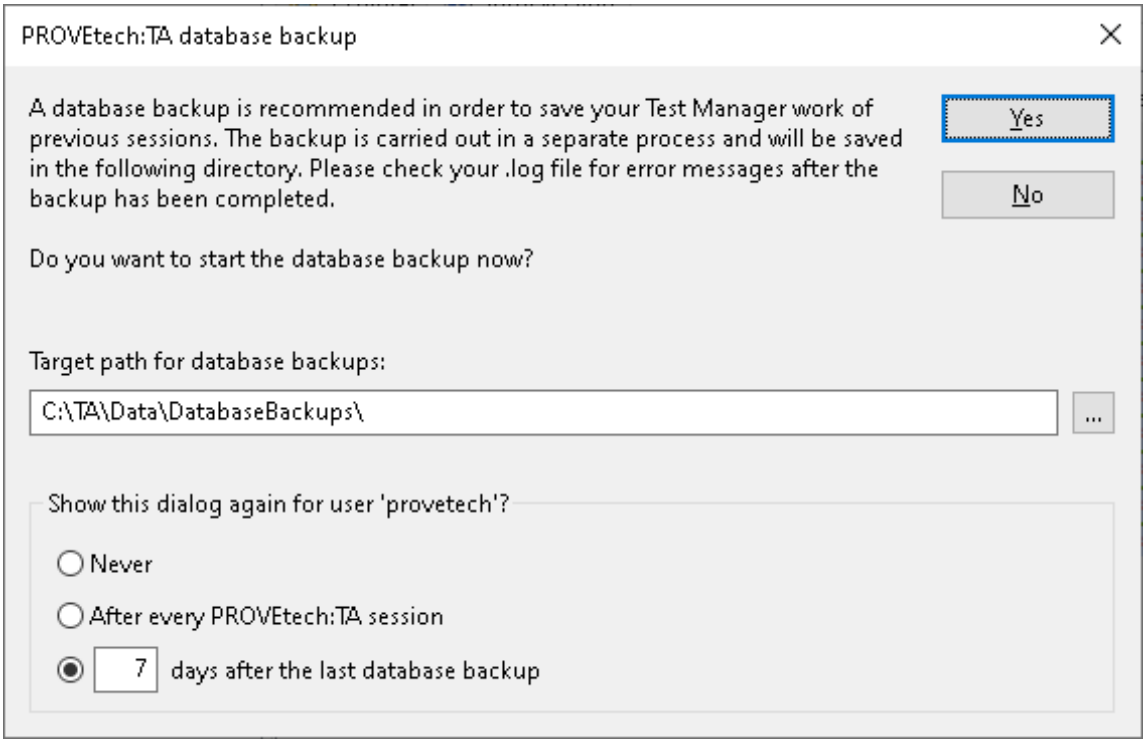


Figure 403 – Database backup

The dump file from the database backup and a .log file will be stored in the target path. In the .log file you can see whether the database backup was successfully completed. The dump file may be re-imported if necessary.

After you have acknowledged the dialog with `Yes`, the database backup is started in a separate batch process and PROVEtech:TA is terminated. Despite a loss in speed during the database backup, you may continue to work on the computer as usual. Please en sure that the database backup is finished before shutting down the computer.

In the dialog of ***Figure 403***, you can choose, for a given user, how often PROVEtech:TA should remind you to create a backup of the database. Therefore, a protocol entry of the last backup was made in the database.

You can configure all options of the database backup in the `Database` tab of *Application|Settings…* in the main menu (see ***3.2.7.6.5 Configure the database backup***). Here you can also start the database backup manually.

A database backup can also be initiated from the context menu of the root entry in the Test Manager tree (see ***Figure 404***).

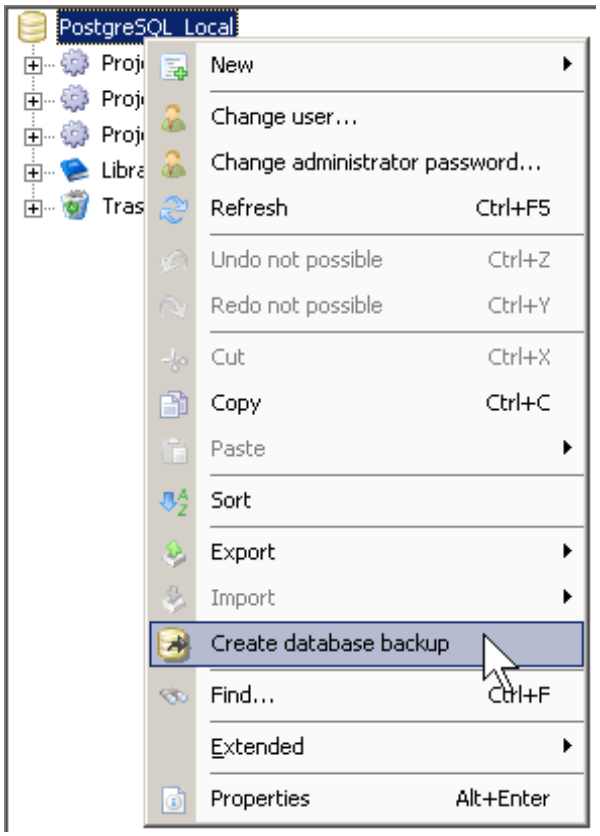


Figure 404 – Database backup in the context menu



Caution: It is strongly recommended to create daily backups of your PROVEtech:TA database and quarterly long-term archival of your data. The PROVEtech Support is not obligated to restore data after a database crash. Standard operating procedure in this case is to reinstall the database server and restore a current database backup.

Please read also in chapter ***8.3.8 Data security and avoiding data loss in the Test Manager*** and chapter ***8.4.11 Failed to create a database backup***.



In any case, after the database backup has been performed in the background, please check the enclosed .log file for error messages. If you want to be on the safe side, you can import and verify the database backup on a temporary database.



If the data storage of your free or local database exceeds the limit of 1 GBytes, then it is strongly recommended to move your database to a server which is hosted by your IT. So you can avoid problems with database backups.



For security reasons there is no automatism in PROVEtech:TA to restore a database backup or to import data into the database. However, all corresponding batch files needed for an import/export can be found in amongst the PROVEtech:TA installation files (e.g. see in Goodies\Databases\PostgreSQL\ExportImportScripts\Import.bat). Should you require help or information concerning this topic, please contact PROVEtech Support (provetech@akkodis.com).



If you have installed a Microsoft SQL Server then the backup path refers to the server computer. In PostgreSQL and Oracle, this path refers to the local computer.



To further secure your data, you should copy the dump file of the database backup to a server computer that is backed-up daily, e.g. to a streamer and the server computer should have a RAID system. After the copy you can delete the dump file. So you avoid that your hard disk eventually gets full.



The backup automatism in PROVEtech:TA can be disabled via the settings. If this is done, then it is advisable to employ an alternative backup process, e.g. a scheduled Windows task. Please contact PROVEtech support (provetech@akkodis.com) for further information regarding database backups.



PROVEtech:TA provides a setting to automatically create backup files in the temporary directory of PROVEtech:TA when saving Basic code, description text and test parameters. You can find more about this in chapter ***3.2.7.6.5 Configure the database backup***.



Please differ between a backup of the file system and a database backup. A backup of the file system does not guarantee that you can restore a consistent state of the database. Always use the mechanisms of the database to back up your Test Manager data.

5.12 Database consistency checks

You can open the database consistency test dialog via the settings dialog (see chapter ***3.2.7.6 Database***) or via the context menu on the database object in the Test Manager tree (root object, see chapter ***5.2.3 The database object***) in order to subsequently test the database for data consistency.

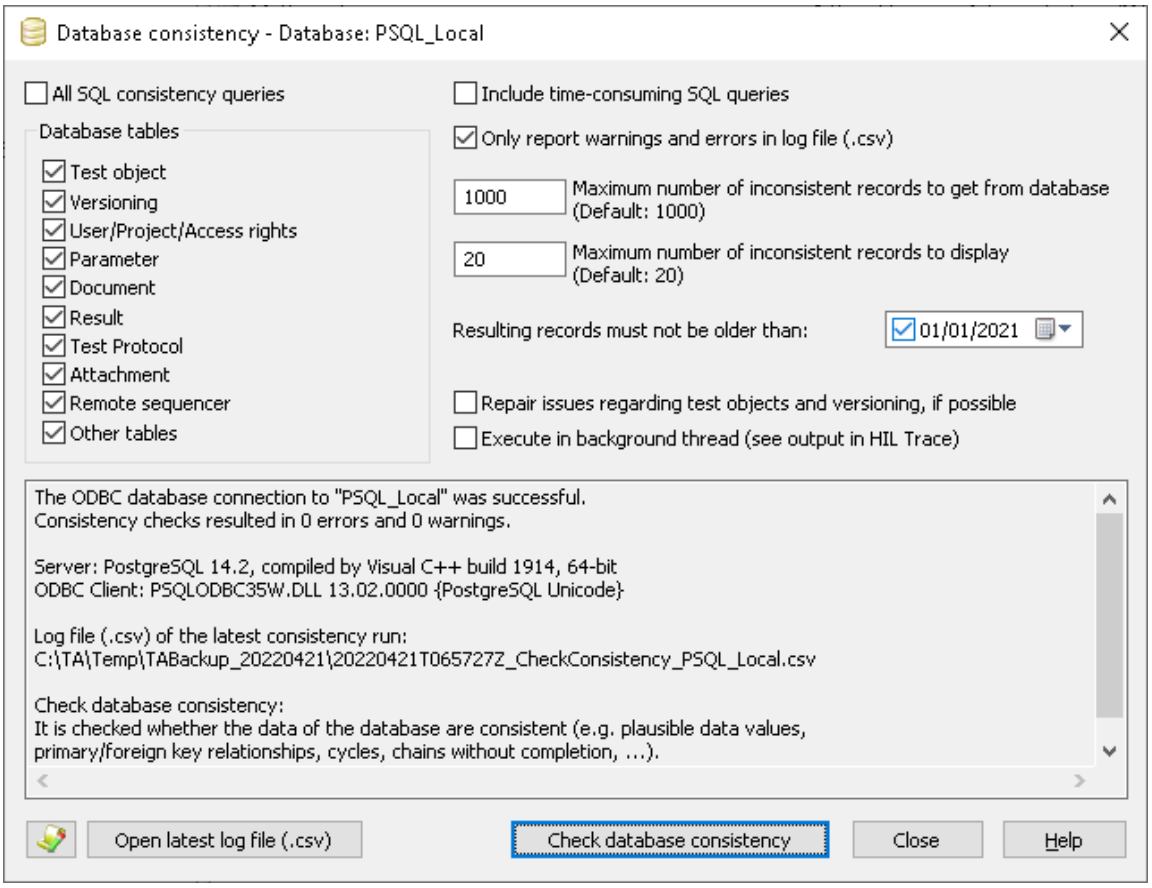


Figure 405 – Check database consistency

In the dialog options, you can specify the areas from which consistency should be checked and how the output should be generated. The following applies here: The smaller the search space, the shorter is the execution time of the consistency checks. The execution time can also vary depending on the amount of data in the database.

If the output in the PROVEtech:TA Log (see chapter ***3.6.1 PROVEtech:TA Log***) as well as in the .log file is sufficient for you and you want to continue working in PROVEtech:TA meanwhile, you can specify that the operation is executed in a background thread.

Start the operation by clicking the `Check database consistency` button. After the operation is finished, you can switch to the PROVEtech:TA Log by the button  or you can view the log using the `Open latest log file (.csv)` button. Further below in this chapter you will also find how inconsistency can happen in the database and how you can try to repair them yourself.

As PROVEtech:TA administrator or if the 'arManage' access right to the root object is available, you can activate the `Repair...` checkbox. Please consider then the following warning.

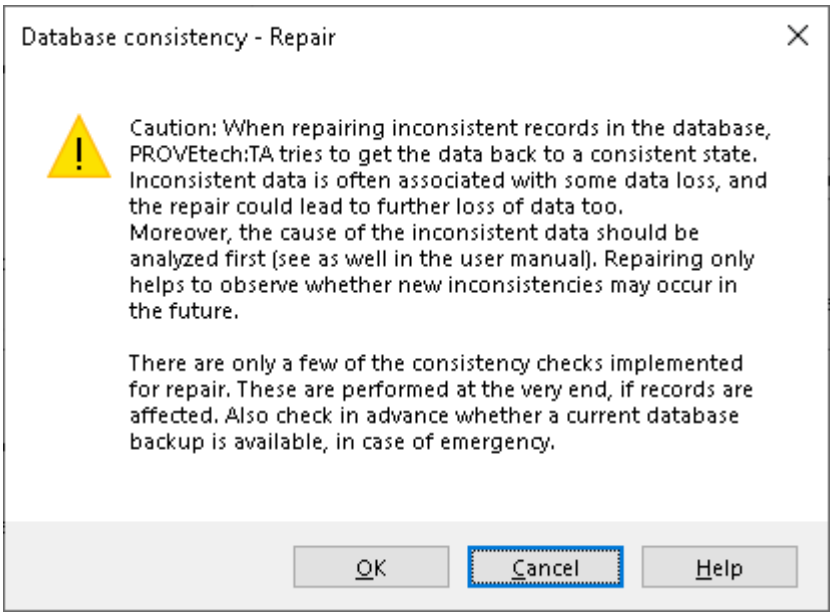


Figure 406 - Warning for repairing database inconsistencies

How can inconsistent data happen in the database?

The following are reasons that can lead to inconsistency in the database:

- When importing PROVEtech:TA archive files (.dbi.7z files), if the archive file is from an outdated PROVEtech:TA version or has been manually edited. Here, the archive file could already contain inconsistencies.
- When importing PROVEtech:TA archive files (.dbi.7z files), if the import was aborted due to an error. These inconsistencies are automatically deleted after emptying the trash folder and database purge (see chapter ***5.2.4 The trash folder***).
- Incomplete import of a database backup or error messages when restoring a database backup
- New database inconsistencies can occur as a result of other inconsistencies, for example in versioning.
- If database replication is used, this always involves certain risks, for example in the event of a network failure over a longer period. In addition, there may be problems in complying with the database transaction concept.
- Always use PROVEtech:TA to access the data of the database. If third-party software is used, the data should also be accessed via the PROVEtech:TA Test Language. If you directly access the database by SQL, there is a risk of data becoming inconsistent or the loss of data.

- Last but not least, software bugs in older PROVEtech:TA versions can also cause data to become inconsistent. Therefore, update to current versions and service packs as early as possible and inform PROVEtech Support about strange behavior in PROVEtech:TA.
- If you have the feeling that new database inconsistencies keep appearing over time, then run the consistency check regularly, e.g. weekly, and compare the resulting .csv file.

However, the following cases should never lead to database inconsistency:

- The transaction concept of a database (ACID, Atomicity, Consistency, Isolation, Durability) prevents that an operation is only partially executed, for example by a software crash, process kill by hand, computer switch off, server crash, ... Either the operation could still be executed completely, or it will be entirely reset.
- Also, with multi-user operation, no database inconsistency can arise. The database serializes all parallel transactions. If several PROVEtech:TA clients process the same data, the one that writes last "wins".

Database inconsistency is most apparent when exporting and importing larger Test Manager subtrees. Therefore, always test whether warning or error messages occurred during export and whether an exported PROVEtech:TA archive (.dbi.7z file) can also be imported again without errors.

How to repair inconsistent data by yourself?

Below you will find possible solutions to remove the inconsistent data from the database on your own. Make sure that a database backup has been performed before.

- Empty the Trash Folder by PROVEtech:TA administrator and purge the database (see chapter ***5.2.4 The trash folder***)
- Check if you can delete and recreate the objects of the displayed paths in the PROVEtech:TA Log or in the log file (.csv file). Maybe export and import can help. Please also refer to the AdministratorsGuide, chapter TM.ini in section `[ExportImport]`.
- You can ignore consistency problems if the data is no longer being processed or if the consistency problems are not serious (see Severity level).
- As PROVEtech:TA administrator, activate the `Repair...` checkbox in the consistency dialog, considering the displayed warning note (see above).
- For inconsistencies with current date, you can try to find out on which computer the inconsistency was created. To do this, select the corresponding path in the Test Manager tree. Using the `History` tab (see chapter ***5.3.9.7 The tab***

"History"), you may be able to identify the computer (or at least the user) that last worked on the record. Then create a problem report from this computer and send it to PROVEtech support (provetech@akkodis.com).

- You can request assistance from PROVEtech Support if the above options did not help. To do so, please generate a problem report (see chapter ***2.13 Creating a problem report***) and send it together with the .csv file from above to provetech@akkodis.com. Afterwards a database administrator of the customer's IT department can access the database directly via SQL statement with the help of PROVEtech support.

You can also run a consistency check for a single test object or a single test version using the following basic script:

```
'#Language "WWB.NET"
Option UsesAmbiguousError On
Option Explicit
' Check the database consistency of one test item (PROVEtech:TA 64bit re-
quired)
' Output of consistency warnings and errors in the PROVEtech:TA Log in red or
blue color
' [in] ti              Test item to be checked (COM-Objekt)
' [in] bAbortOnError   If true, return false on error; otherwise, log error
and continue
' [in] bRepair         If true, repairing is included if this is possible;
otherwise, repairing is skipped
' [ret]                VARIANT_FALSE in case of an internal error (besides
consistency problems), VARIANT_TRUE otherwise
Private Declare Function TM_Internal_CheckConsistency Lib "TM" Alias
"?TM_Internal_CheckConsistency@@YAFPEAUITestItem@@FF@Z" (ByVal ti As TestItem,
ByVal bAbortOnError As Boolean, ByVal bRepair As Boolean) As Short

Sub Main
    Dim tiCorrupt As TestItem =
TM.GetTestItem2("*MyProject:\Systemtest\Test;3.0") ' Please adjust path
    TM_Internal_CheckConsistency(tiCorrupt, False, True)
End Sub
```

5.13 Multiple databases in the Test Manager tree

You have got the possibility to connect simultaneously to multiple databases in the Test Manager and operate with their test trees.

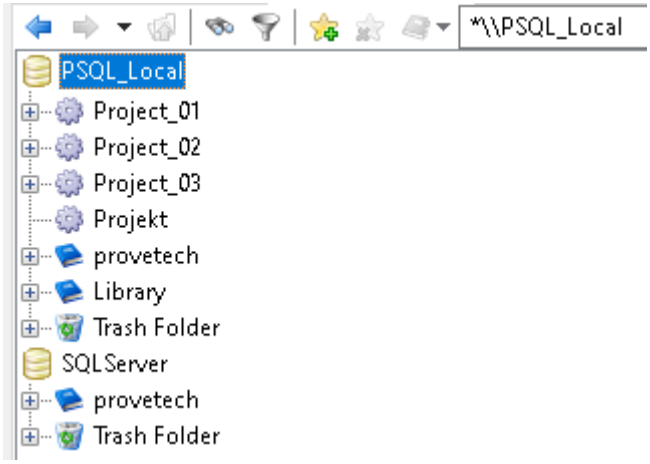


Figure 407 – Multiple databases in the Test Manager tree

For example, the following use cases are possible:

- Testing in the vehicle (Onboard tests): Inside the vehicle you can do your tests with a locally installed database without network connectivity; later you can synchronize your data with a central database server via WinWrap Basic script
- Data partitioning: For each product line, store your data in a separate database; you can also store your shared libraries in a common database
- Virtual test center: Partitioning of data with frequent and infrequent access; you could store the frequently accessed data on a local database and the infrequently accessed data on a remote database server
- Synchronization of central WinWrap Basic libraries between various databases (e.g. departments of a company)

This would result in the following benefits:

- Smaller amounts of data per database (important for storage management and backup strategy)
- No new logon required anymore if you want to access another database
- Synchronization of databases based on WinWrap Basic scripts is much simpler
- More speed when using remote databases

You can use the checkbox `Show ODBC alias always in TM tree (in case of multiple connections)` to configure this (see in chapter ***3.2.7.6 Database***). If you set the check then the database is always visible in the Test Manager tree, no matter which database connection you have chosen in the login dialog. PROVEtech:TA takes the connection name as the item name in the tree.

One could say that the database connection from the login dialog is the primary database. You can find it always on top root of the Test Manager tree. If you select a further

database tree item then PROVEtech:TA tries to connect automatically to the new database with the current user and password if necessary. If this fails then the login dialog for the further database connection appears.

If you operate with multiple databases simultaneously then an additional syntax for the paths in the Test Manager tree is necessary:

Current path syntax:
```
*Project_03:\Systemtest\VerifyFunction
```

New path syntax (in addition to the current one):
```
*\\PSQL_Local\Project_03\Systemtest\VerifyFunction
```

So in the new path syntax the name of the database connection is used and the colon is dropped. If you still use the older current path syntax then PROVEtech:TA tries to find the correct reference to the context. So there is no compatibility problem.

You can use the new path syntax inter several databases in the test language, e.g. in the '#uses command, ***TM.Execute***, ***TM.GetTestItem*** or ***TestItem.Copy***. Furthermore, the new path syntax is displayed in many places in PROVEtech:TA when you work on multiple databases, for example in the Dependency dialog in the WinWrap Basic editor. On demand, PROVEtech:TA always tries to connect automatically to the additional database.

You also can copy or move tests inter different databases via context menu, drag and drop or short keys. PROVEtech:TA internally uses the export and import commands for that.



Links on test objects between multiple databases are not possible and lead to an error message.



If you change the name of a database connection, then you might have to change paths e.g. in '#uses commands, too.

It is possible that you must adjust your database connection names if you use the new path syntax on several PROVEtech:TA client installations.

5.14 The remote test sequencer

The remote test sequencer enables you to display and control the execution list of the test objects on the test system by remote from a different computer (see also chapter ***5.5 Executing tests***). So, in addition to test development and test result analysis you can

sequence your test cases for execution on the test system by remote from your computer at your desk. A client-server-architecture is used for that. The computer on the test system is the sequencer server and the computer at your desk the sequencer client. This feature is especially interesting in the context of test development teams distributed geographically, accessing and running their tests on the test systems in central test labs. This however is also interesting for a developer trying to remotely access the test system in the lab from his desk.

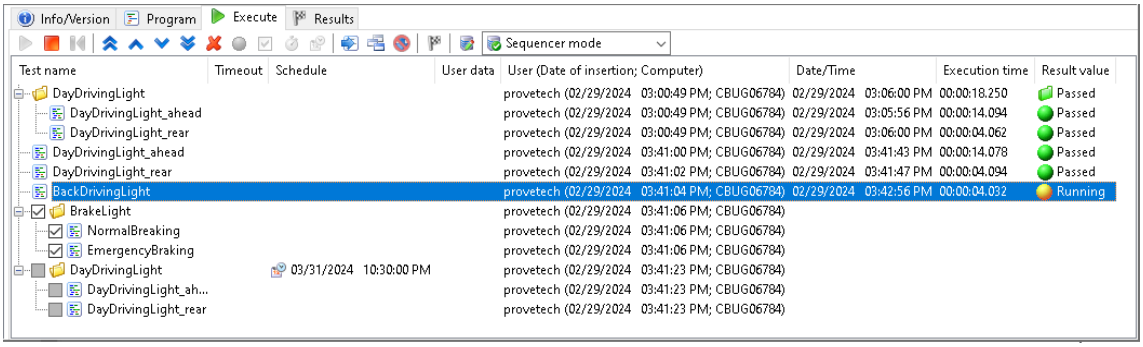


Figure 408 – The remote test sequencer server on the computer CBUG06784

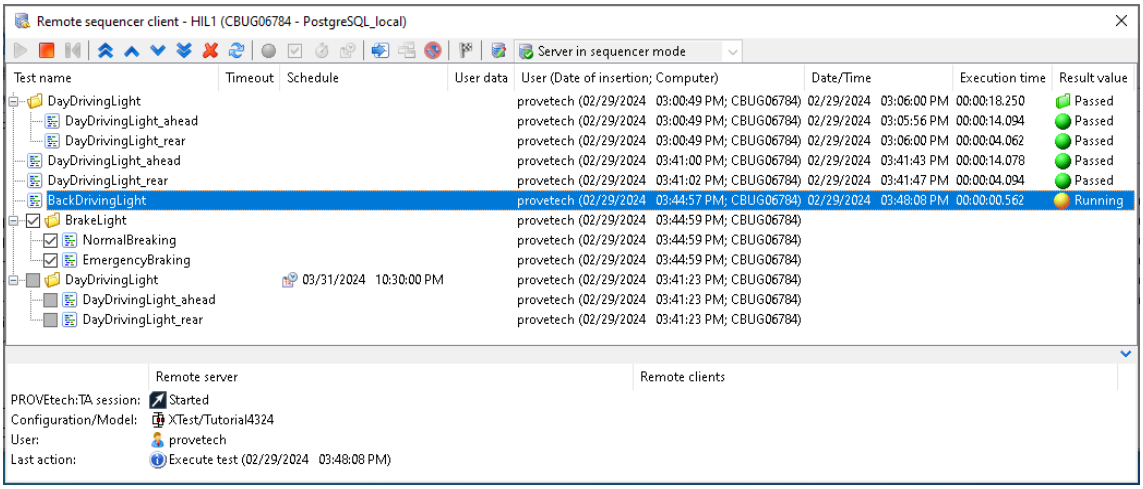


Figure 409 – The corresponding remote test sequencer client on the computer CMB01739

The following requirements must be met:

- The license file License.key must have the remote sequencer module enabled (see ***2.2.1 Computer-bound licensing***)
- The sequencer server and the sequencer client computers must be connected to the same database; the network speed should be sufficient
- PROVEtech:TA must be started on the sequencer server computer and switched to sequencer mode

The following advantages are obtained when using the Remote test sequencer, e.g. compared with a remote control via Windows remote access to the distant computer:

- The existing database connection is used, there is no additional TCP/IP connection required
- An execution flow control is included
- The Sequencer Client window displays the relevant information in a clearly-arranged way
- Only the sequencer data is transmitted over the network, but no graphical contents
- It is an integrated solution in the PROVEtech:TA Test Manager; so the familiar PROVEtech:TA operations are provided
- Any number of users can simultaneously monitor and control the test execution on the test system
- The remote sequencer also controls the availability of the test system
- Efficient 24/7 use, which means around-the-clock operation tests including night, weekend and holidays



Please also note the information in chapter ***8.4.8 The database is very slow*** regarding remote test sequencers.



Caution: When using the Remote Sequencer, the client computer can retrieve information from the server computer via Winwrap Basic script, even from the user who is currently logged on to the remote server.

5.14.1 Register the remote test sequencer

First of all, it is necessary to register the test system computer as a remote test sequencer server. To do this, select the property page `Execute`, open the remote test sequencer configuration dialog by clicking the button  in the toolbar and press  `Add current computer as a sequencer server`.

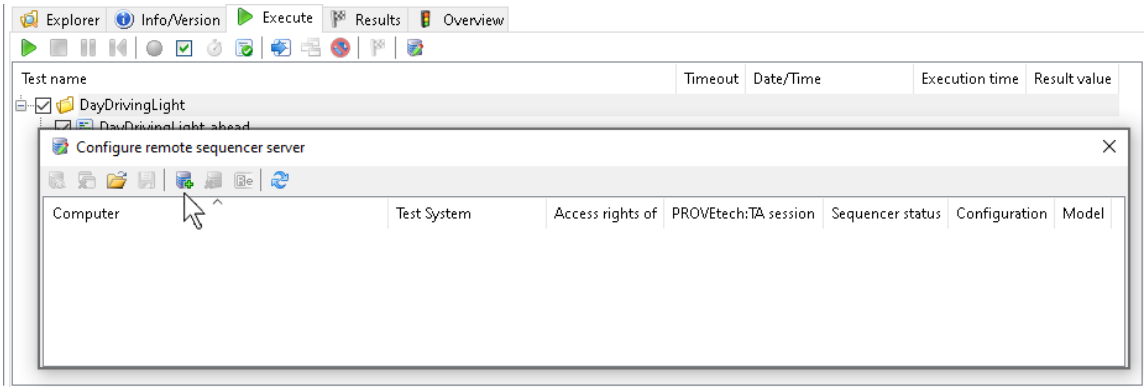


Figure 410 – Add current computer as a sequencer server

After that you can set the name of the test system and, if needed, the access rights in the dialog (see chapter ***5.14.3 The remote test sequencer and access rights***).

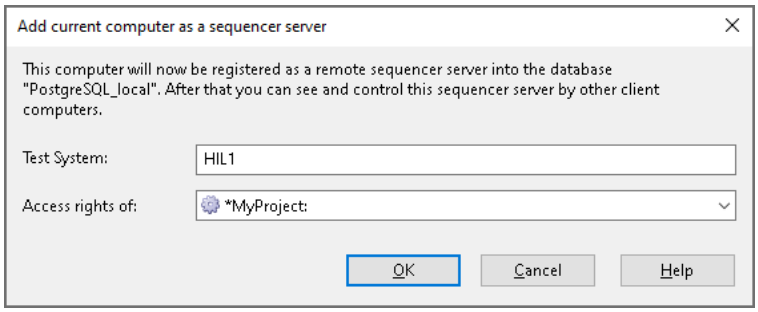


Figure 411 – Add current computer as a sequencer server

Now the computer is displayed in the list of available remote test sequencer servers. Further on, you can set a name in the column `Test System` and assign the access rights of a project, e.g. to hide the sequencer server for non-project members.

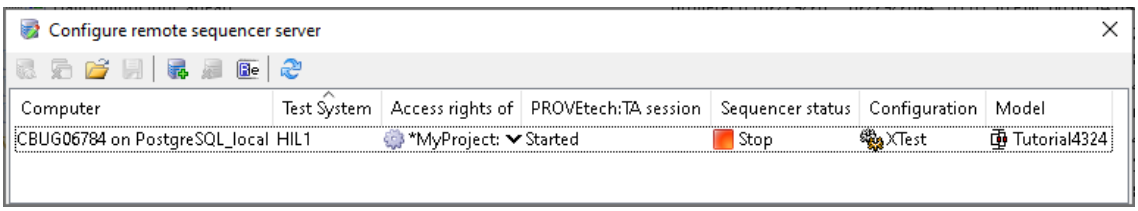


Figure 412 – Configure remote test sequencer

Now in the `Execute` tab an additional combo box is displayed to switch between `Sequencer mode` and `Default mode`. Accordingly, the buttons in the toolbar change, too.

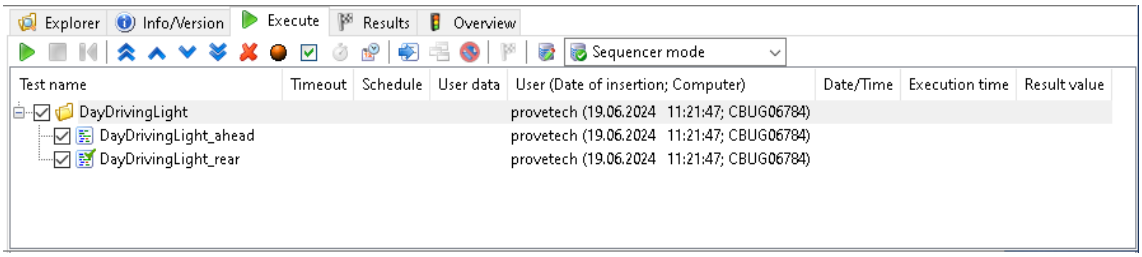


Figure 413 – The sequencer mode

Go to the sequencer client computer now, for example your computer at your desk that is connected to the same database, select the `Execute` tab and open the remote test sequencer configuration dialog via button `Configure remote test sequencer`.... Here you should also see the sequencer server in the list. You can open a sequencer client window by double-click, toolbar or context menu.

5.14.2 Working with the remote test sequencer

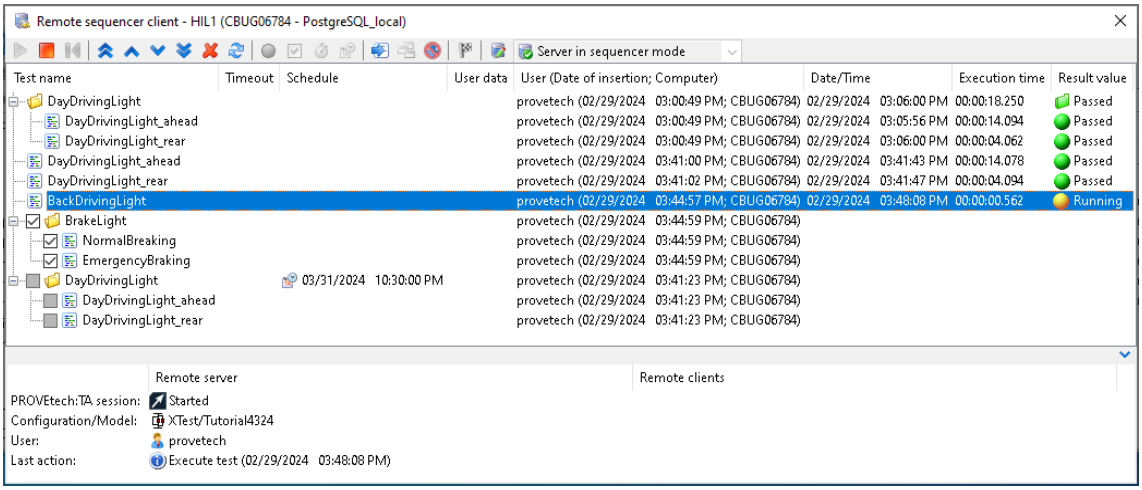


Figure 414 – Remote test sequencer client with running tests in the execution list

In the **upper area** of the sequencer client window you can add test objects to the execution list per copy and paste or drag and drop, remove test objects (), change the execution order of the tests that are not executed yet (), jump to the corresponding test object () or display the test result of an executed test (). You can also add complete system tests to the execution list (see ***Figure 414***).

Press the button to disable scrolling and automatically collapsing system tests that are successfully executed..

If `Server in sequencer mode` is displayed in the combo box and the button `Start` is pressed () then the test objects are automatically executed according to

the execution list. In almost the same manner the buttons and in the `Execute` tab behave differently if the computer is in sequencer mode (see also chapter ***5.5 Executing tests***). On the sequencer client, the buttons and also allow you to stop and continue the automatic test execution. If all tests in the execution list are already processed then the sequencer server does not switch to “Stop”, but keeps waiting in the “Start” state. If further tests objects are added in this state, the execution continues immediately. It is therefore not possible to set a timeout or schedule before starting the test, for example. New tests can therefore be added in the skip status by holding down the shift key. Tests objects can also be added to the sequencer run list when the sequencer server is in the "stop" state or in default mode. These will then be executed the next time the system starts, unless they are set to skip status.

With you can reset a still executed test object or system test; then the test results are not in direct access anymore and you can start a new test run.

Every couple of seconds the sequencer client windows are refreshed. You can also enforce this by clicking the “Refresh” button ( ) or pressing the F5 key. When forcing explicitly “Refresh” a refresh on the sequencer server inclusive clearing the cache is done, too.

You can also set breakpoints or skip non-executed tests by the remote sequencer client. Please refer to chapter ***5.5.2 Breakpoints*** and chapter ***5.5.3 Skipping test objects***.

You can directly set any string inside the column *user data*. This data can be used to just display information, or you can read out the string by Test Language to control the list of sequencer objects.

Regarding the column `Timeout` and the toolbar button and controlling the values by remote please read more in chapter ***5.5.4 The timeout value***.

Regarding the column `Schedule` and the toolbar button please read more in chapter ***5.14.5 The schedule***.

In the **lower area** of the sequencer client window you can see additional information about the current sequencer state. This area can be expanded or collapsed by click on the upper right corner.

You can open several different sequencer client windows by the same time and so control several test system computers at a glance. The window positions and column sizes can be saved using the buttons and in the sequencer configuration dialog.



The remote test sequencer performs by **communication via the existing database connection**. So the sequencer server and sequencer client computer must be connected to the same database and there is no additional connection necessary (e.g. special TCP/IP connection).

The client and server windows are refreshed by “polling”. That means, every couple of seconds a SQL statement is executed on every sequencer server computer and sequencer client computer. So, the speed of the database connection and the network can affect the performance and the sequencer client is updated with a corresponding delay.



If you move the mouse pointer on a sequencer object then you get a tooltip containing the corresponding Test Manager path.

5.14.3 The remote test sequencer and access rights

If you want to revoke special access rights when you use the remote test sequencer (e.g. to hide the sequencer server for non-project members) then you can assign the access rights of a project to the sequencer server (see also chapter ***5.2.6 Access rights***). The following table shows which operations are disabled if the currently logged-on user has not the access rights of the assigned project:

Missing access right of the assigned project	**Operations at the sequencer server and sequencer client that are thereby disabled**
accManage is missing	• the sequencer server cannot be renamed • the access rights of the sequencer server cannot be modified • the sequencer server cannot be removed • When modifying the access rights of the sequencer server, only projects are listed for which the currently logged-on user has accBrowse and accManage access rights

accBrowse is missing (Display in Test Manager tree)	• the sequencer server is not displayed in the configuration dialog • it is not possible to get status information of the sequencer server • the corresponding sequencer client window cannot be opened • there is no further operation available
accRead is missing (Display test objects)	• it is not possible to get status information of the sequencer server
accExecute is missing (Execute test objects)	• the status of the sequencer server cannot be changed (Start/Stop/Default) • test objects cannot be added to the execution list • test object cannot be moved or deleted in the execution list • you cannot set a timeout value or user data • you cannot set skip or breakpoints
accTrRead is missing (Read test results)	• you cannot go to the test result
accTrWrite is missing (Modify test results)	• you cannot validate test results

5.14.4 Automatic control of the sequencer order

When using the remote test sequencer you have got the possibility to control the execution sequence automatically. By WinWrap Basic script you can program which test object and which logic should be used to continue the execution.

First of all, you program the function `GetNextTestItemToExecute` in an elementary test which has fix defined parameters. The following code example shows how you could implement this function.

```
'#Language "WWB.NET"
Option Explicit

'global map that stores the time accounts of the users
Dim g_mapUser2TimeAccount As New MapString2Variant

'"GetNextTestItemToExecute" is a common sequencer handler function
'with fix defined Interface
'Param(In): aTestSequencerItem is the Array of non-executed test objects
'           in the sequencer list
'Return value: Test item that is moved to the top of the sequencer list
'              before execution
'              If "Nothing" is returned and the sequencer list is not empty,
'              then the execution is stopped
'              If the return value is not in the sequencer list then it is
'              added automatically

Function GetNextTestItemToExecute(aTestSequencerItem() As TestSequencerItem) _
    As TestItem

    'find next element for execution with lowest time account
    Dim nNextTimeAccount As Long=&HFFFFFFFFFFFFFFFF
    Dim tsiNext As TestSequencerItem=Nothing

    Dim i As Long
    For i=LBound(aTestSequencerItem) To UBound(aTestSequencerItem)
        Dim nTimeAccount As Long=GetTimeAccount(aTestSequencerItem(i).User)
        If nTimeAccount<nNextTimeAccount Then
            nNextTimeAccount=nTimeAccount
            tsiNext=aTestSequencerItem(i)
        End If
    Next
    If IsNothing(tsiNext) Then Return Nothing

    'add estimated execution time to global map
    g_mapUser2TimeAccount(tsiNext.User)=GetTimeAccount(tsiNext.User) _
        +tsiNext.TestItem.EstimatedExecutionTime
    Return tsiNext.TestItem
End Function

'this function gets the current user's time account from the global map
Function GetTimeAccount(strUser As String) As Long
    Dim vTimeAccount As Object
    If Not g_mapUser2TimeAccount.Lookup(strUser, vTimeAccount) Then Return 0
    Return CLng(vTimeAccount)
End Function
```

For the automatic sequence control you can register the Test Manager path to this elementary test on the remote test sequencer server (but not on the sequencer client) in the property page `Test Manager` of the `Settings` dialog (see chapter ***3.2.7.5 Test Manager*** in the Settings dialog), which includes the fix defined interface function `GetNextTestItemToExecute`.

Now each time the next test item is about to be executed, the function `GetNextTestItemToExecute` on the sequencer server computer is called. That

means, if the sequencer server computer is in sequencer mode, then the function `GetNextTestItemToExecute` is called after the following two actions:

1. When clicking the `Play` button
2. When a test execution in the `Execute` tab is terminated

The returned test object is first included (if not already existent) in the sequencer list. Then it is moved to the top of the list. If the return value is `nothing` and there are still non-executed test objects in the sequencer list, then the execution stops.

You can test the execution of your sequencer handler by the following script:

```
'#Language "WWB.NET"
Option Explicit

Function GetNextTestItemToExecute(aTestSequencerItem() As TestSequencerItem) _
    As TestItem

    Dim ti As TestItem

    TM.GetTestItem("*MyProject:\Test", ti) 'any valid test path
    MsgBox ti.Path
    Return ti
End Function
```



You can also use the extended test language to control the remote sequencer, in particular the ***SequencerServer*** object and the ***TestSequencerItem*** object (see inside ***Test Language*** in the Manual).



The WinWrap Basic script that controls the execution sequence in the remote test sequencer is kept in a separate Basic instance in the working memory of the sequencer server. That means, global variables are still valid during several function calls.

If you modify the WinWrap Basic code then you must recreate the Basic instance. You can do this by a click on the button `OK` in the Settings dialog.

5.14.5 The schedule

With the remote test sequencer, you have the option of planning the start of the process and, if desired, also planning for the test process to be repeated. Scheduling can be carried out from both the sequencer server and the sequencer client.

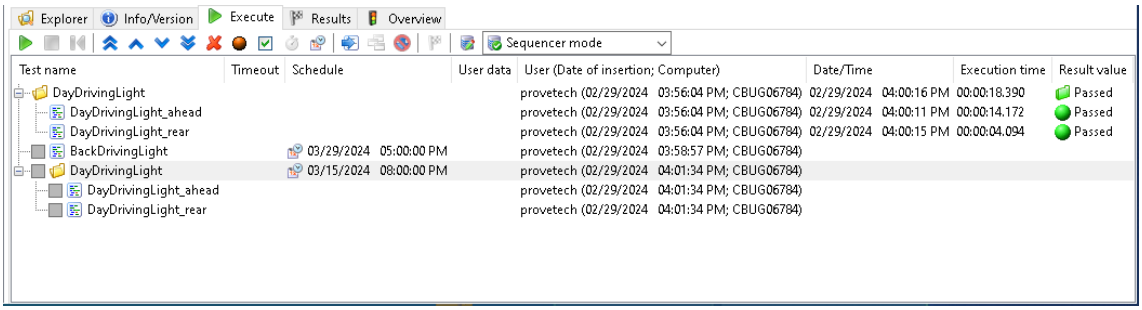


Figure 415 – Test object/Test system with scheduling

The schedule for test objects/test systems can only be set at the main level and applies to the entire test process. If the schedule is active, the selection boxes ☐ cannot be activated by checking them manually. Activation takes place at the scheduled time by the sequencer server.

If the test has not yet been executed, you can configure a schedule by selecting the test objects/test systems that you want to change and using the context menu or the button on the toolbar and opening the following dialog.

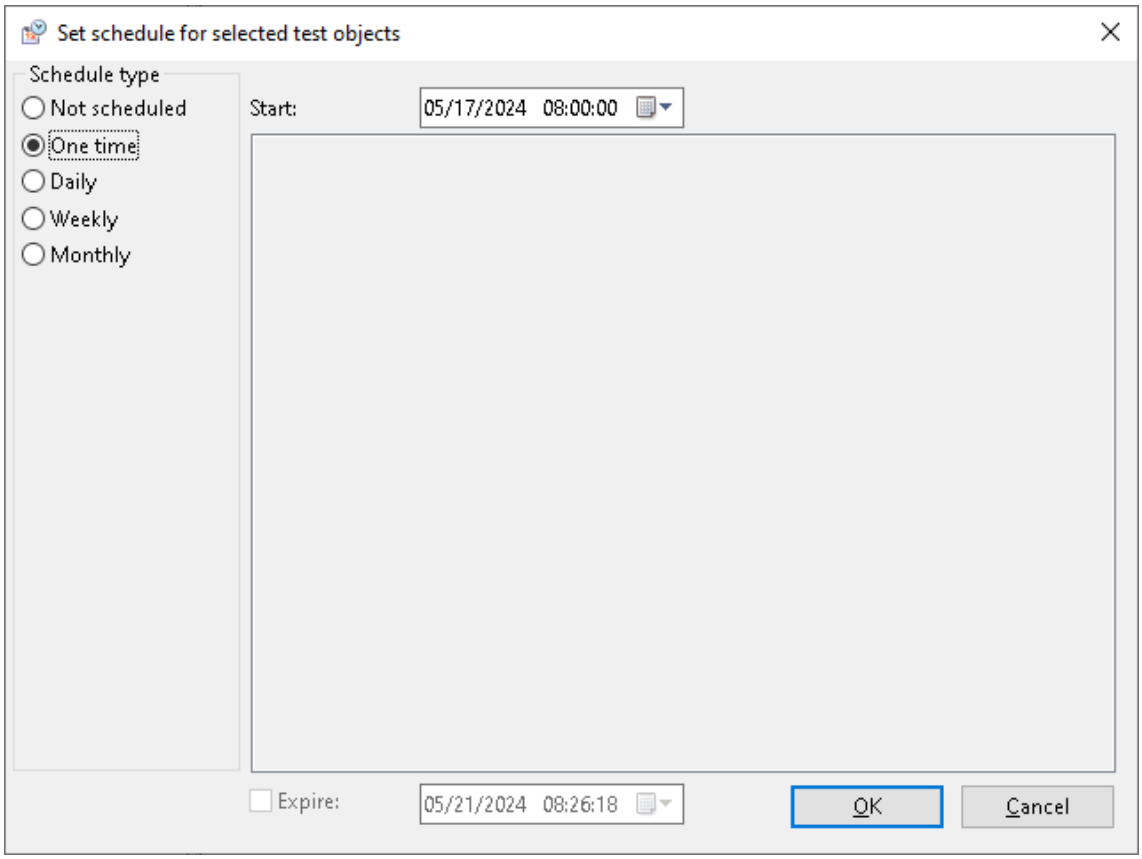


Figure 416 – Schedule for one time start

You can schedule a “One-time start” by selecting the “One-time” schedule type and specifying a start date/time.

The following schedule types are available for repeating the test process:

- Daily
- Weekly
- Monthly

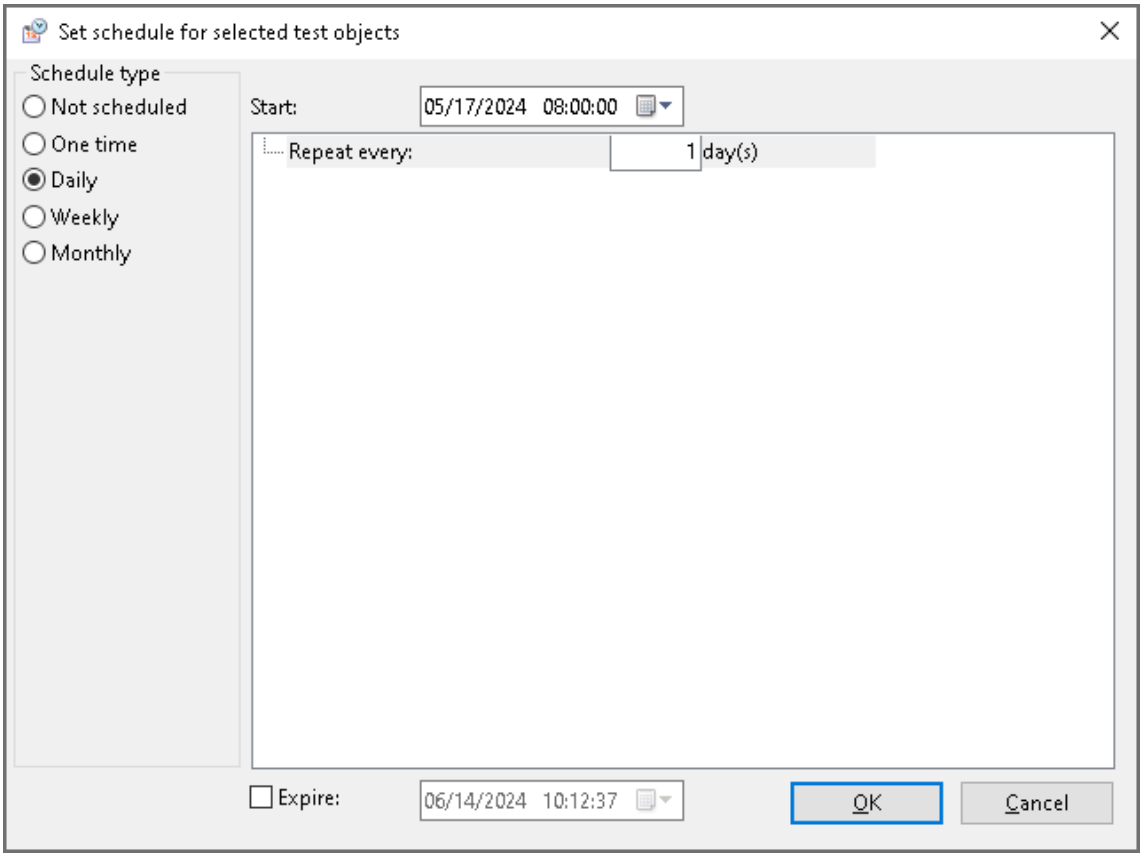


Figure 417 – Schedule for a daily start

Daily start: After selecting the schedule type “Daily” and specifying a start date/time, the repeat cycle in days can be entered directly in the list.

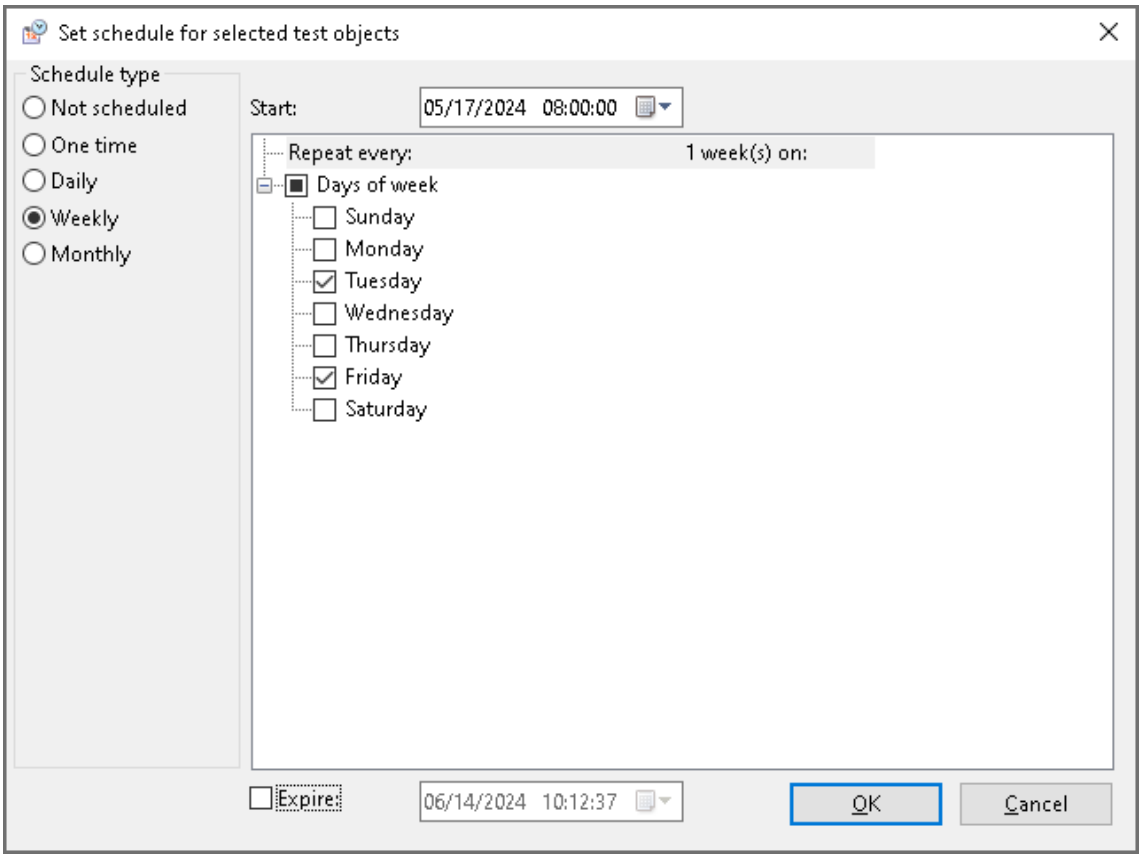


Figure 418 – Schedule for a weekly start

Weekly start: After selecting the schedule type “Weekly” and specifying a start date/time, the repeat cycle in weeks can be entered directly in the list.

You must select at least one day of the week to run the run. If you select several days of the week, when rescheduling, another start may be scheduled in the same week before the next start is planned for the first day of the week in the specified weekly rhythm.

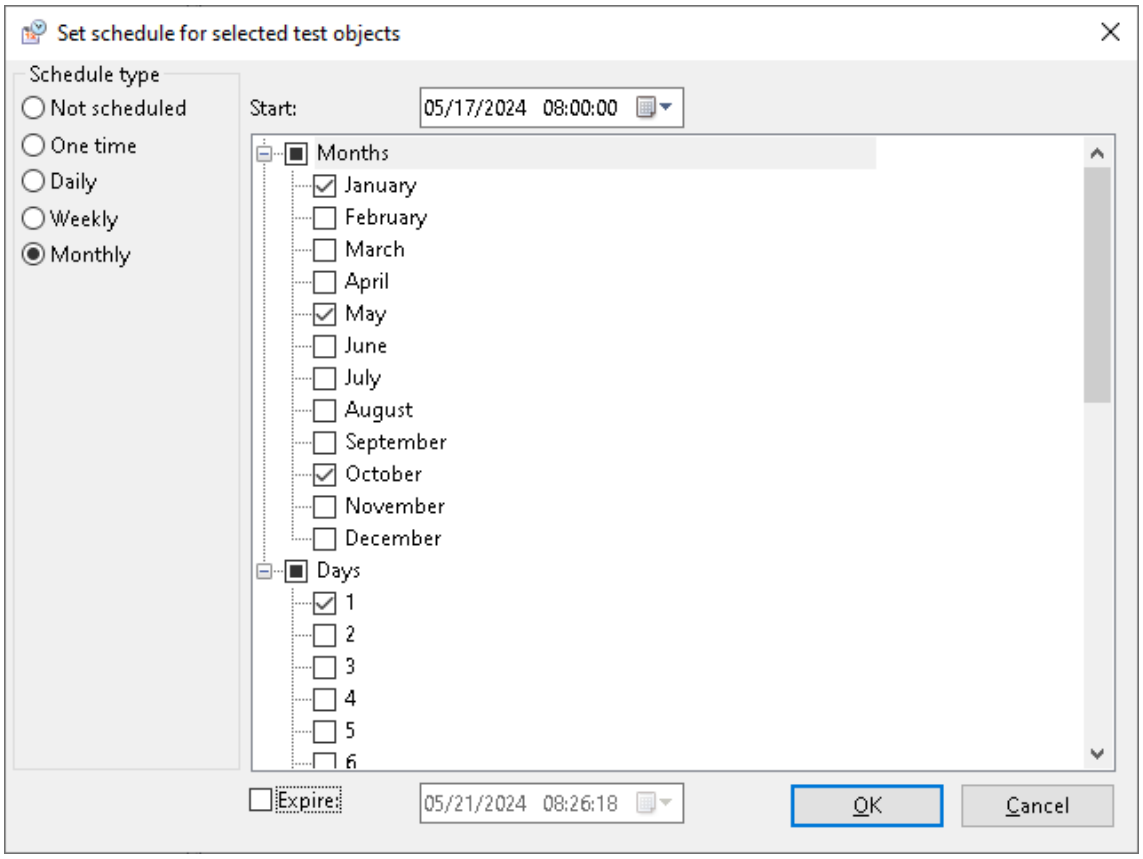


Figure 419 – Schedule for a monthly start

Monthly start: After selecting the schedule type “Monthly” and specifying a start date/time, you can select in which months (at least one month) a start should take place.

You must select at least one day of the month to run the run. If you select several days, the rescheduling may initially schedule another start in the same month before the next start is scheduled for the first day of the next specified month.

A special case is the selection “Last day”. This is not a fixed day but always the last day of the month.

For repeat planning, an expiry date/time can be specified after which the repeat ends.

If a start will be made up because, for example, the sequencer server was not running at the actually planned start, the repeat scheduling is based on the original start but always at a point in time after the current execution.

5.15 Access from foreign software via URI

Beside test objects you can reference test documents, test results and attachments by using a Test Manager path as well. Please look at the following examples:

```
*MyProject:\Rotarylightswitch\Test1
*MyProject:\Rotarylightswitch\Test1*doc*config.csv
*MyProject:\Rotarylightswitch\Test1*res*20180531T165213Z
*MyProject:\Rotarylightswitch\Test1*res*20180531T165213Z*att*Meas-
ure.hdr
```

`20180531T165213Z` is a timestamp exact to the second in the format yyyymmddThhmmssZ that represents the test result's date and time. The "T" corresponds to the ISO8601 standard, and the "Z" stands for the time in UTC. You can use these paths to access PROVEtech:TA from foreign software and process certain actions. Therefore, PROVEtech:TA registers the URI prefix `TestAutomation` in the Windows registry which you can use in combination with a path and an action. URI stands for Uniform Resource Identifier e.g., for web pages (URL), web services or any other files. The following examples illustrate this:

```
TestAutomation:Action=Execute&Path=*MyProject:\Rotarylightswitch\Test1

TestAutomation:Action=Open&Path=*MyProject:\Rotarylightswitch\Test1*doc
*config.csv

TestAutomation:Action=Select&Path=*MyProject:\Rotarylightswitch\Test1*r
es*20180531T165213Z

TestAutomation:Path=*MyProject:\Rotarylightswitch\Test1*res*20180531T16
5213Z*att*Measure.hdr&Action=Open

TestAutomation:Action=Select&Path=*MyProject:\Rotarylightswitch\Test1&L
ine=117
```

Special characters in Test Manager paths can be escaped by %25 for the % character and %26 for the & character. You can also protect the character & inside the path by using &&. The table below gives you an overview about the possible actions:

Action	Description
`Select`	Selects the specified object of the TM path in the PROVEtech:TA Test Manager. This is the default action. If a test, a class or an object is referenced, you can specify the line in the WinWrap Basic code by setting `&Line=117`.
`Execute`	Executes the specified test object of the TM path in the PROVEtech:TA Test Manager. A test result is also generated, which is stored in the database.
`Open`	Opens the specified test document or attachment of the TM path in an external tool; in case of a data acquisition (.hdr file) the attachment is displayed in the offline mode of PROVEtech:TA (see chapter ***4.5 The offline mode***).

You can insert these URIs per Copy and Paste from PROVEtech:TA into the foreign software. Please copy the object via toolbar, context menu or hotkey into the clipboard. If the external application knows the clipboard format "HTML Format" then you can paste the URI as a link directly from the clipboard. After that, you need to adjust the action inside the URI string if required.

A click on this link inside the external application starts PROVEtech:TA with the appropriate command line parameter `TestAutomation:` (see chapter ***2.7 Command line parameters***). If PROVEtech:TA is already started then you switch automatically to the existing PROVEtech:TA session. After that the Test Manager is selected and the action is processed, provided that the Test Manager path exists. In case of a data acquisition file (.hdr file) PROVEtech:TA switches to the offline mode. The command line call could look like the following:

```
TA.exe
TestAutomation:Action=Execute&Path=*MyProject:\Rotarylightswitch\Test1
```

Please read also in chapter ***Access from foreign software*** in the Test Language manual.



If the Login dialog of PROVEtech:TA is currently displayed or if an older version of PROVEtech:TA is currently running, then clicking on the link inside the external application may lead to an error message.

5.16 Supported URL formats

PROVEtech:TA supports the use of hyperlinks and other URLs in some user elements. In addition to the conventional URL formats, other formats are supported, both application-internal and Windows-specific.

The application-internal URL formats are **TestAutomation:** and **taref:**.

- **TestAutomation:** can be used in all hyperlinks to specify the target. It is possible to simply define a path to a test object as well as to execute an actual action. A detailed description of the URL format TestAutomation: can be found in chapter ***5.15 Access from foreign software via URI***.
- **taref:** is limited to use within the HTML Text Edit control (see ***5.3.9.1.1 The HTML Text Edit control***) and is used there as a reference to test object-specific documents or attachments. A detailed description of the URL format taref: can be found in chapter ***5.6.3.4 Test Automation references***.

Windows-specific URL formats depend on installed programs and services and generally differ on different computers. The number of formats varies from a few dozen to several hundred. PROVEtech:TA reads all available formats when it is started and processes them accordingly during its runtime. An overview of the available Windows URL formats can be found in the corresponding Windows settings.

Overview of all URL formats supported by PROVEtech:TA

URL format	**Description**	**Usage in PROVEtech:TA ***
Standard hyperlink formats		
www.[path]	The standard web browser is opened and the [path] is navigated to as the destination	HTML, Ver, Param
http://[path]		
https://[path]		
Standard other formats		
file://[filepath]	The file is opened with the default application associated with the file extension	HTML, Ver, Param
mailto:[email-address]	The default email application is opened, and a new email is created with the specified [mail-address]	HTML, Ver, Param
PROVEtech:TA internal formats		
TestAutomation:[path/action]	An internal test object of PROVEtech:TA can be referenced, and a corresponding action executed	HTML, Ver, Param

taref:[path]	Referencing an attached document or attachment of a PROVEtech:TA internal test object	HTML
Windows-specific formats (excerpt)		
acrobat:[path]	The corresponding standard application is opened and the [path] is transferred for further processing	HTML, Ver, Param
callto:[path]		
feed:[path]		
ms-cortana:[path]		
search:[path]		
telnet:[path]		
onenote:[path]		

* Abbreviations:
HTML: The HTML Text Edit control
Ver: The hyperlink attached to the version label
Param: The hyperlink attached to the parameter attribute 'Value'

5.17 ToDo list dialog

In the ToDo list dialog you have got the possibility to look for saved Todo list entries over a whole subtree. You open this dialog by the context menu of any tree object in the Test Manager tree (with the exception of root database entry and the trash folder). The shown ToDo list entries are restricted to the subtree of the selected test object.

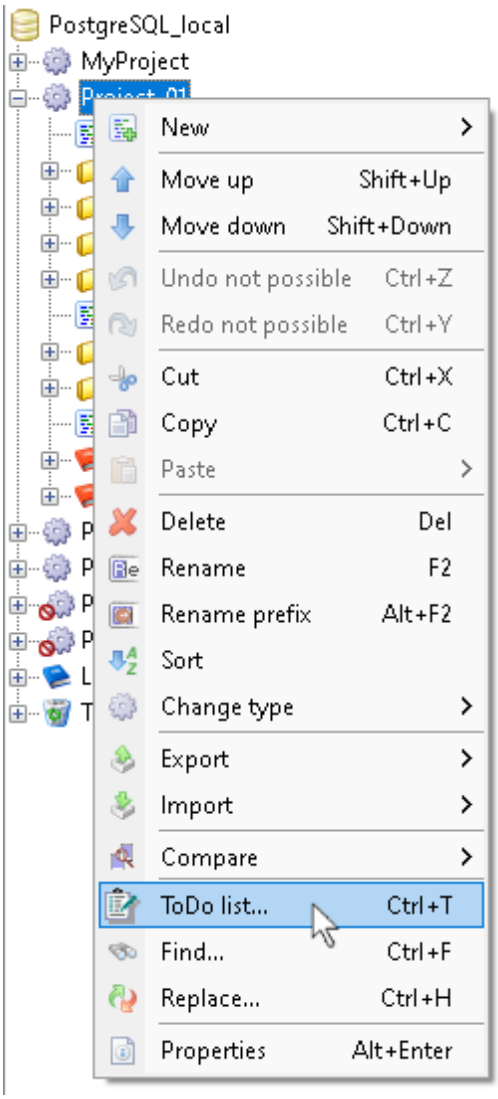


Figure 420 – Menu item „ToDo list“

Open the ToDo list dialog by activating the context menu (right mouse button) and choose the item *ToDo list…*

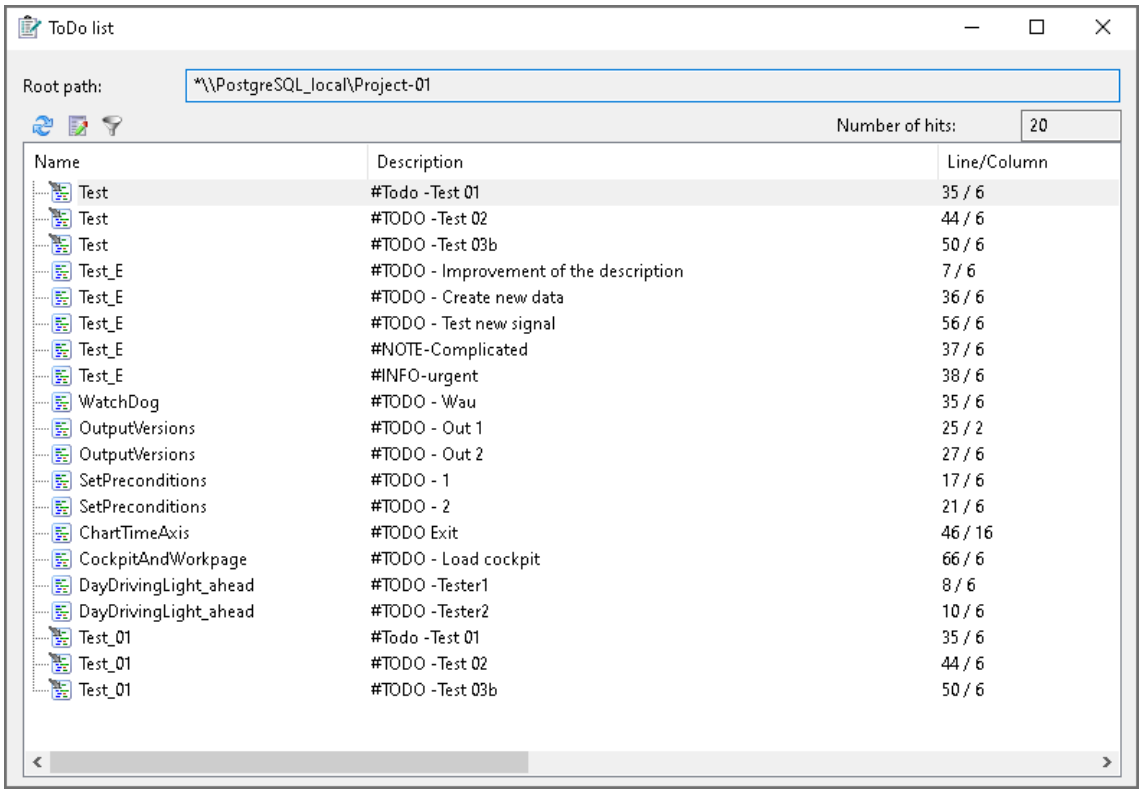


Figure 421 – The ToDo list dialog

You can also update the listing using .

By double-clicking on an entry or using , you can jump directly to the location in the program code.

For further information see ***5.3.9.3 The tab "ToDo list"***

If you want to filter the shown ToDo list entries, you can do this using .

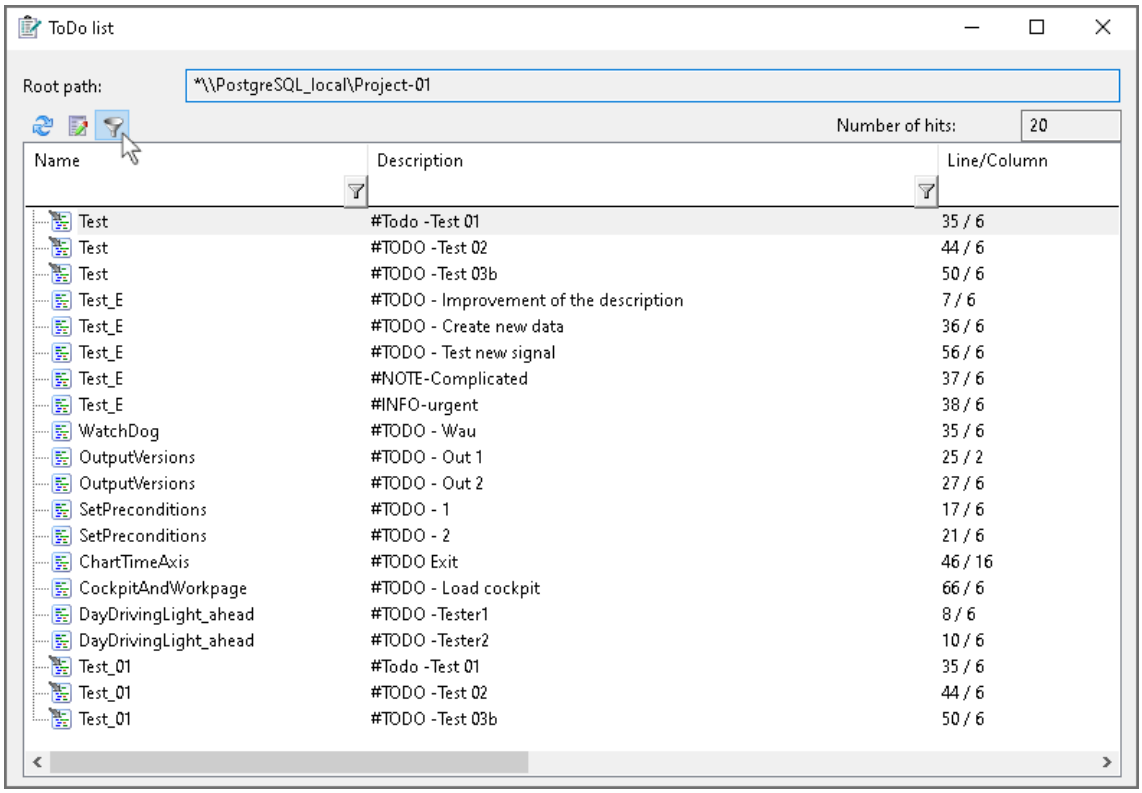


Figure 422 – The ToDo list dialog with list filter header

You can use the list header fields to set a column filter.

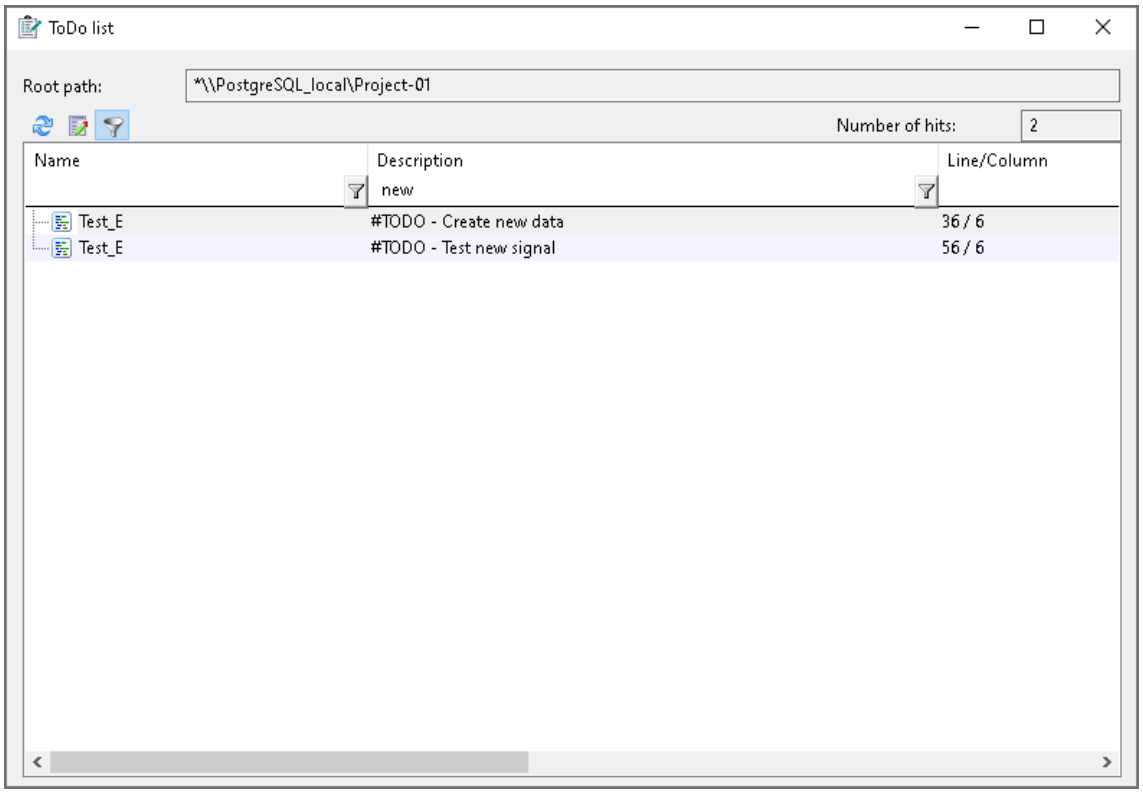


Figure 423 – The ToDo list dialog with column filter

You can reset filter settings and go back to all entries using again.

5.18 Find, filter and replace

5.18.1 Finding tests and test results

In the find dialog you have got the possibility to find and filter tests and test results over a whole subtree. You open this dialog by the context menu of any tree object in the Test Manager tree. Then, the find or filter operation is restricted to the subtree of the selected test object.

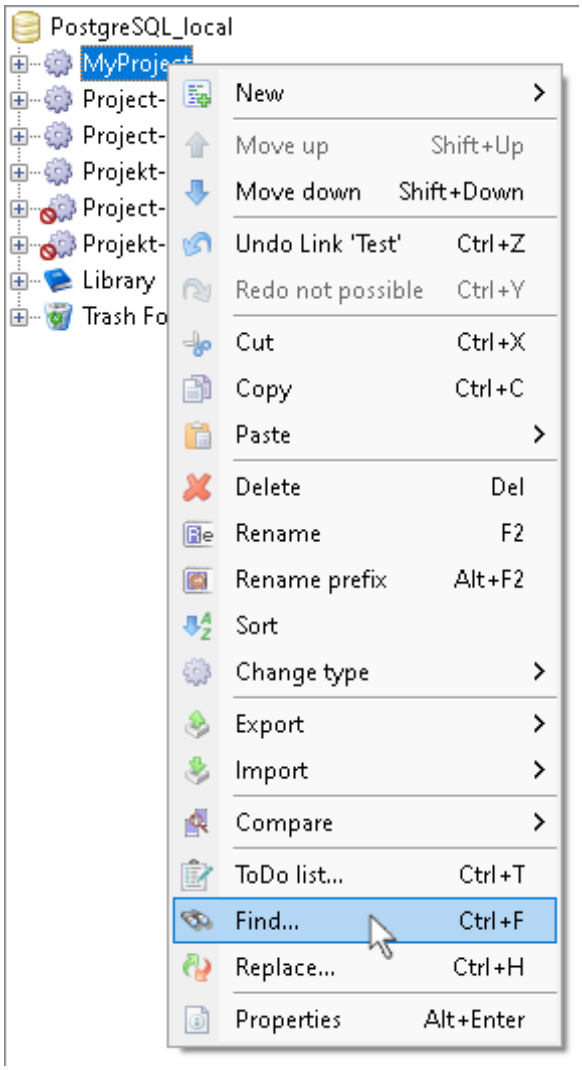


Figure 424 – Menu item "Find"

Open the find dialog by activating the context menu (right mouse button) and choose the item *Find…* (see

Figure 424 – Menu item "Find")

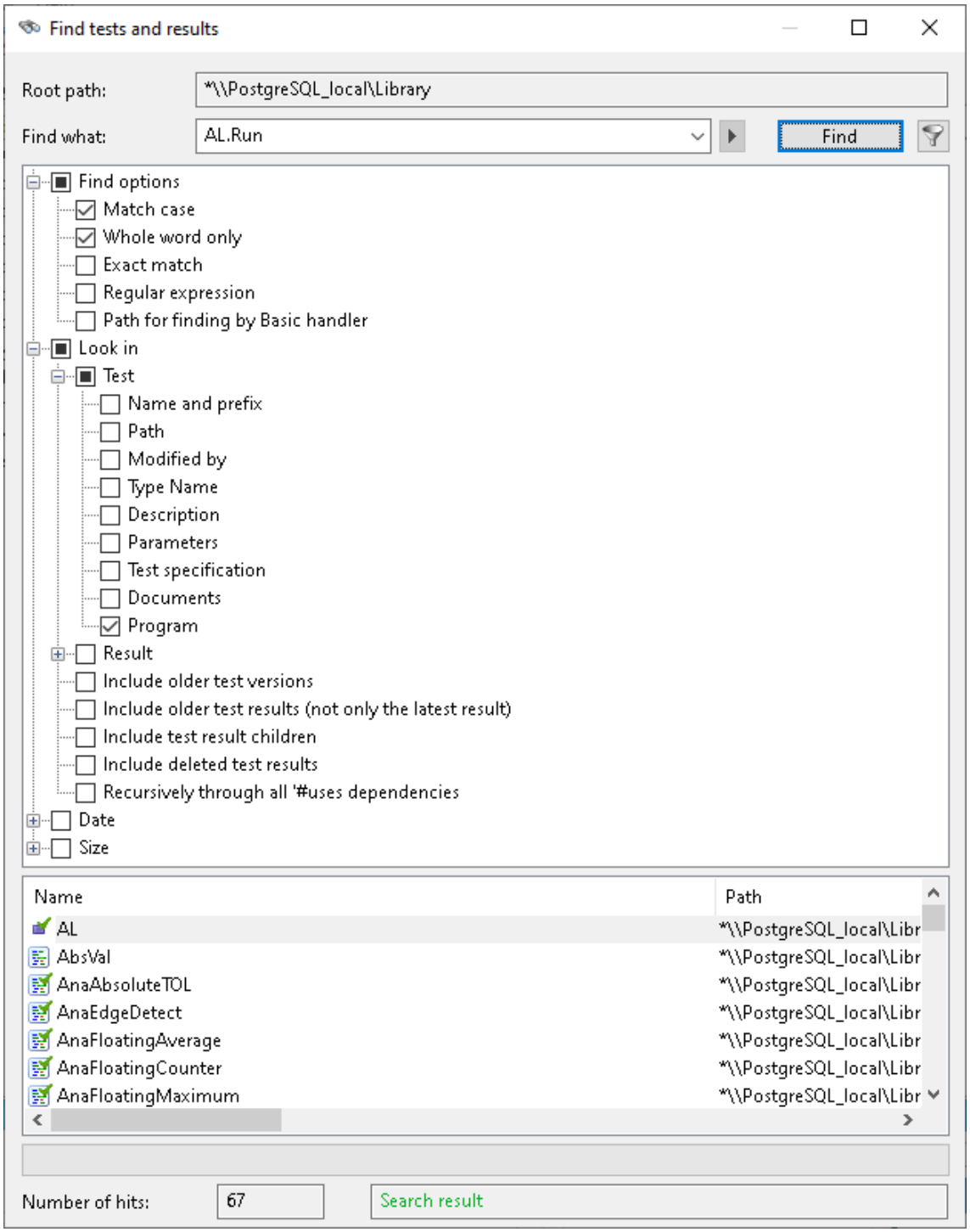


Figure 425 – The find dialog

The `Root path` indicates the subtree's root object to find in. In `Find what` you can set the text you are searching for. You can set the find conditions in the areas *Find options*, *Look in*, *Date* and *Size*.

The area *Find options*

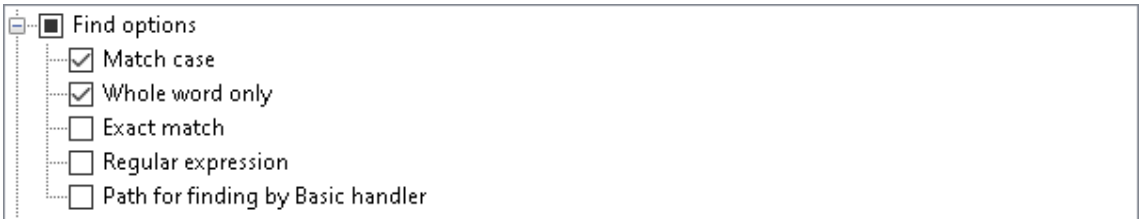


Figure 426 – The area „Find options“

Match case: Differences of case in search string and target string are considered/ignored.

Whole word only: A character except of a letter, a digit or an underscore limits the search text.

Exact match: Activates/deactivates the verification of exact matches (every character must be identical; substrings are not allowed).

Regular expression: The text in `Find what` is interpreted as a regular expression (see chapter ***5.18.4 Regular expressions***). If not checked, the text in `Find what` is an ordinary substring.

Path for finding by Basic handler: You can define your own complex find conditions by using a Basic script. Read more about this in chapter ***5.18.1.2 Finding by Basic handler***.

The area *Look in*



Figure 427 – The area "Look in"

In the upper area you can choose in which fields and attributes of the tests and test results the search string must appear. In the lower area there are the following additional options possible:

Include older test versions: You can search in older test versions, too.

Include older test results: All test results of a test object are considered. Otherwise, only the latest test result of each test object is evaluated.

Include test result children: If you already deleted or moved test objects then it may happen that several test results are only accessible by their parent results.

Include deleted test results: In addition to the existing test results you can also search in the already deleted test results. Of course, these results must only be marked as deleted, but still be stored in the database.

Recursively through all '#uses dependencies: If this checkbox is checked then you scan through all referenced test objects in addition to the whole subtree. This may lead to matches outside the subtree.

The area *Date*



Figure 428 – The area "Date"

If you are searching for tests or test results with specific date criteria, then you need not define a search string. Otherwise, all searching criteria must match.

If the time zone is relevant, you will find information in chapter ***8.3.5 Different time zones and daylight saving time***.

The area *Size*

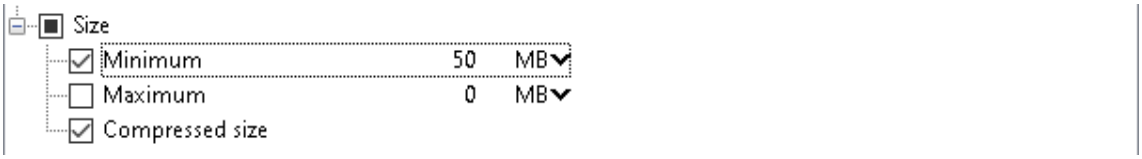


Figure 429 – The area "Size"

If you are looking for test objects in `Look in` then the sum of all corresponding objects and attributes (Basic scripts, description, test documents, …) must match the size criteria. Regarding test results, it is the sum of all result attributes and objects (protocols, attachments, comment, …). Since test documents or attachments are stored in the database in a compressed format, you can decide if you want to handle original sizes or storage space that is used in the database.

The bottom list control shows the search result while the find operation is running or already finished. If you double-click one entry, then PROVEtech:TA navigates to the hit in the background. You can process additional functions by context menu of the search result (see ***Figure 430***).

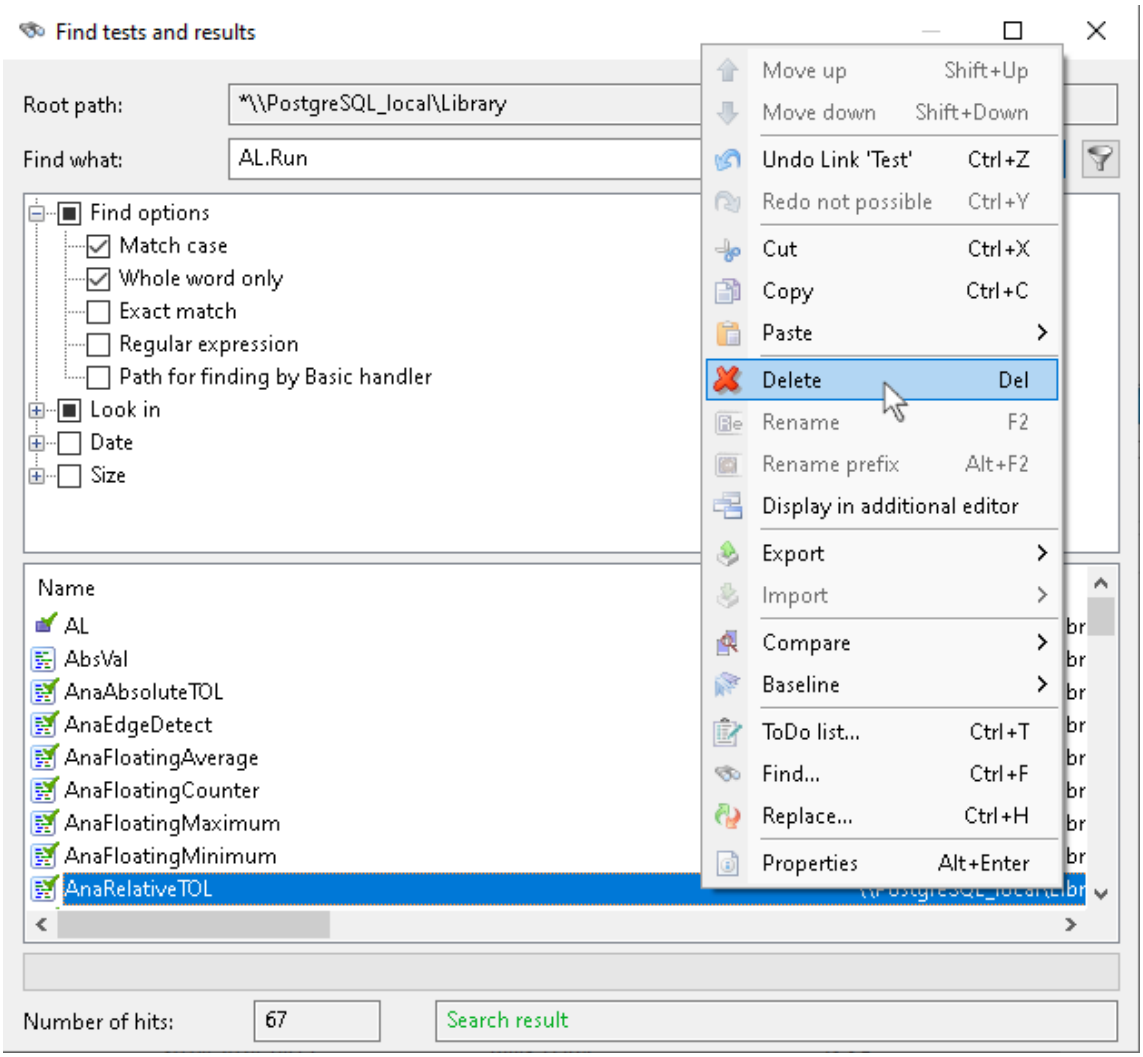


Figure 430 – Context menu of search result



The more you restrict the search area (Look in) the faster you get the result of your find operation.

5.18.1.1 Examples of find options

1. In a subtree, you want to get all matches of the sub string `System` in the Basic code of the test programs. The search is case-sensitive. The search result consists only of test objects, but no test results.

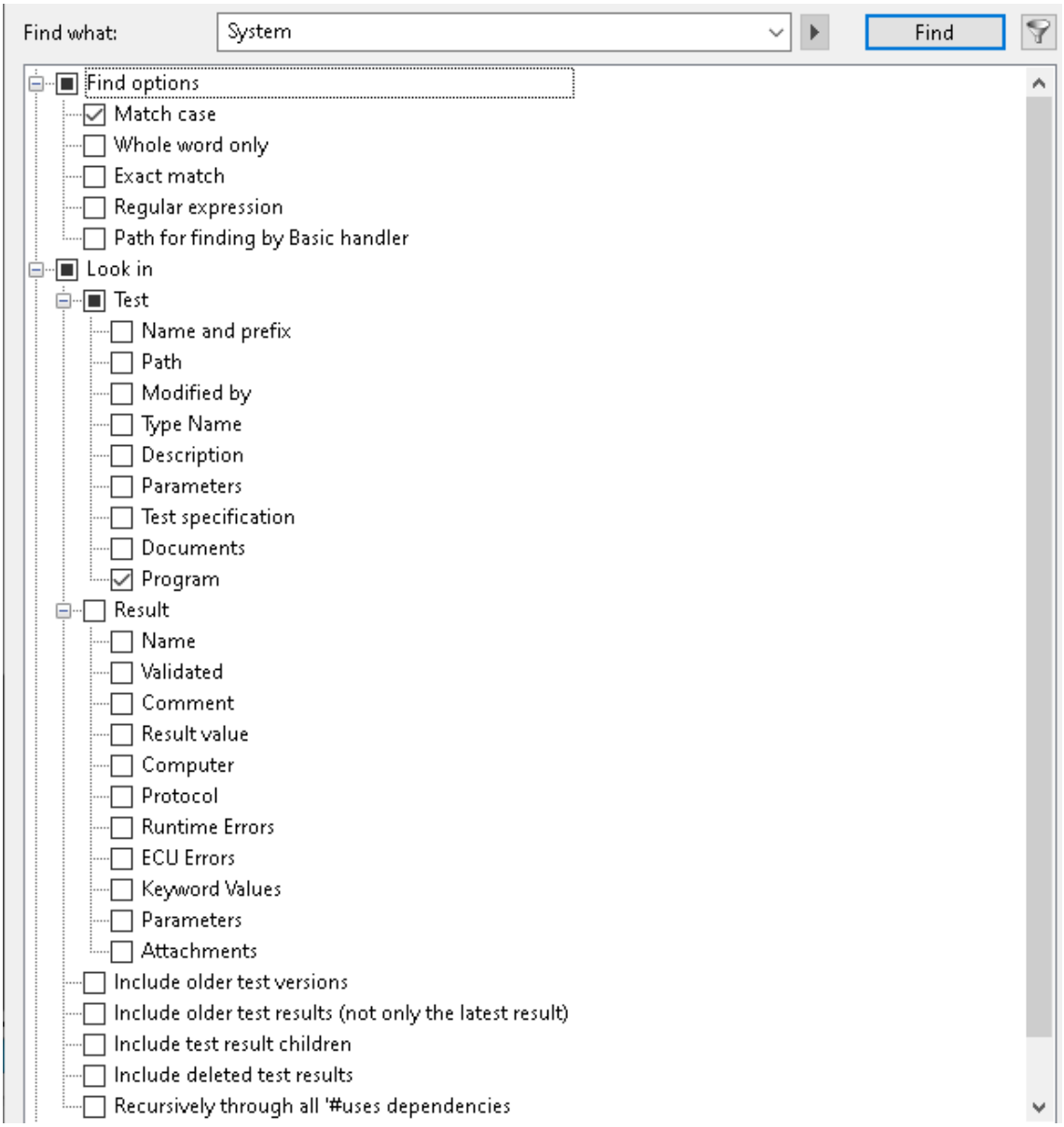


Figure 431 – Find dialog Example 1

2. You search for the whole word of `Signal` in the subtree's test results. That means, you search case-sensitive and the letter before and after the word must not be an alpha-numeric character or underscore. The match can be in the main protocol or in one of its sub protocols (caution because of HTML format). All non-deleted test results are examined. The test result must not be older than 7 days.

Here the search result consists of tests and test results.

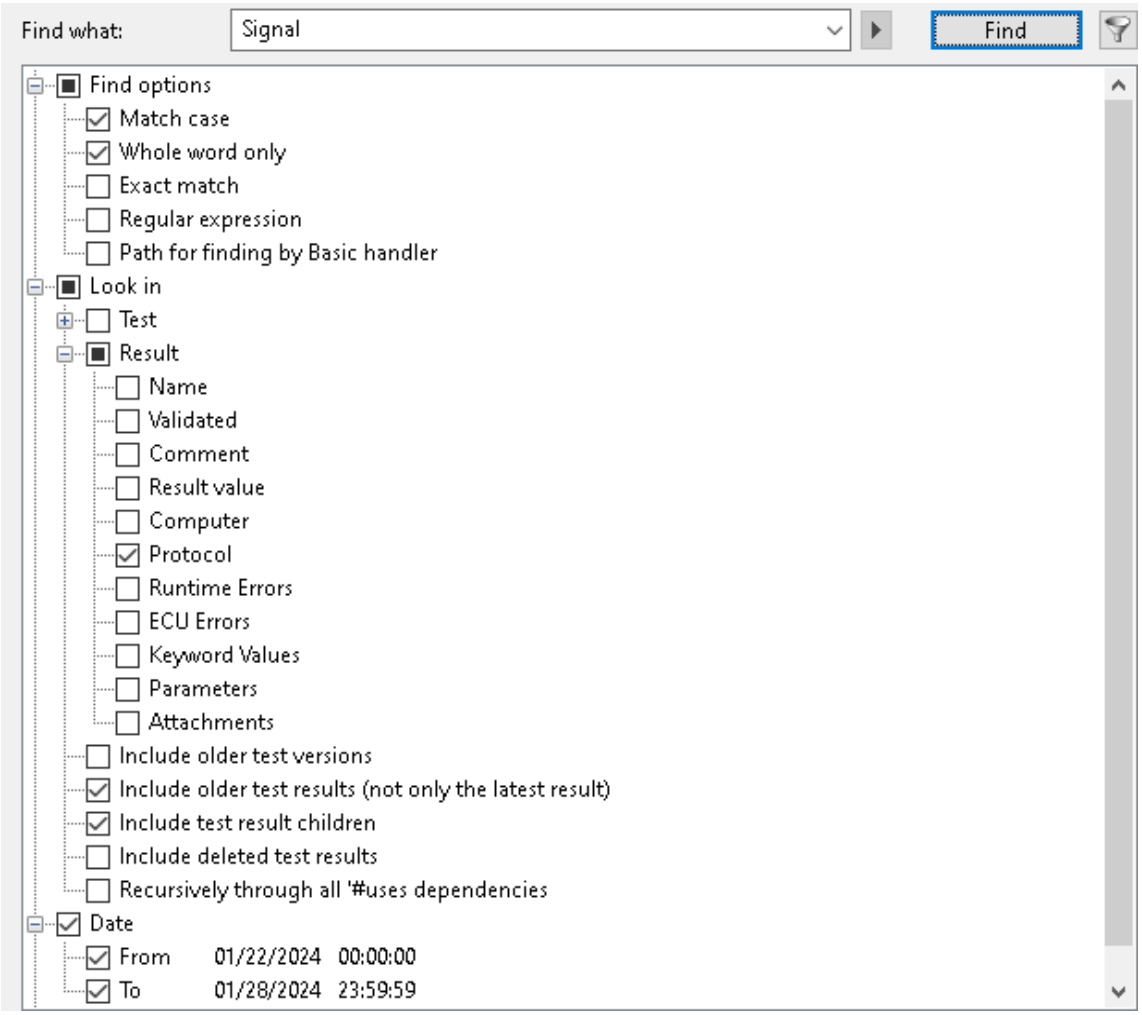


Figure 432 – Find dialog Example 2

3. You try to find test results with big protocols or attachments in the whole database. Therefore, you select the root object in the TM tree and open the find dialog by context menu. The test results with sum of space (compressed) bigger than 250 Megabytes will be listed in the control below.

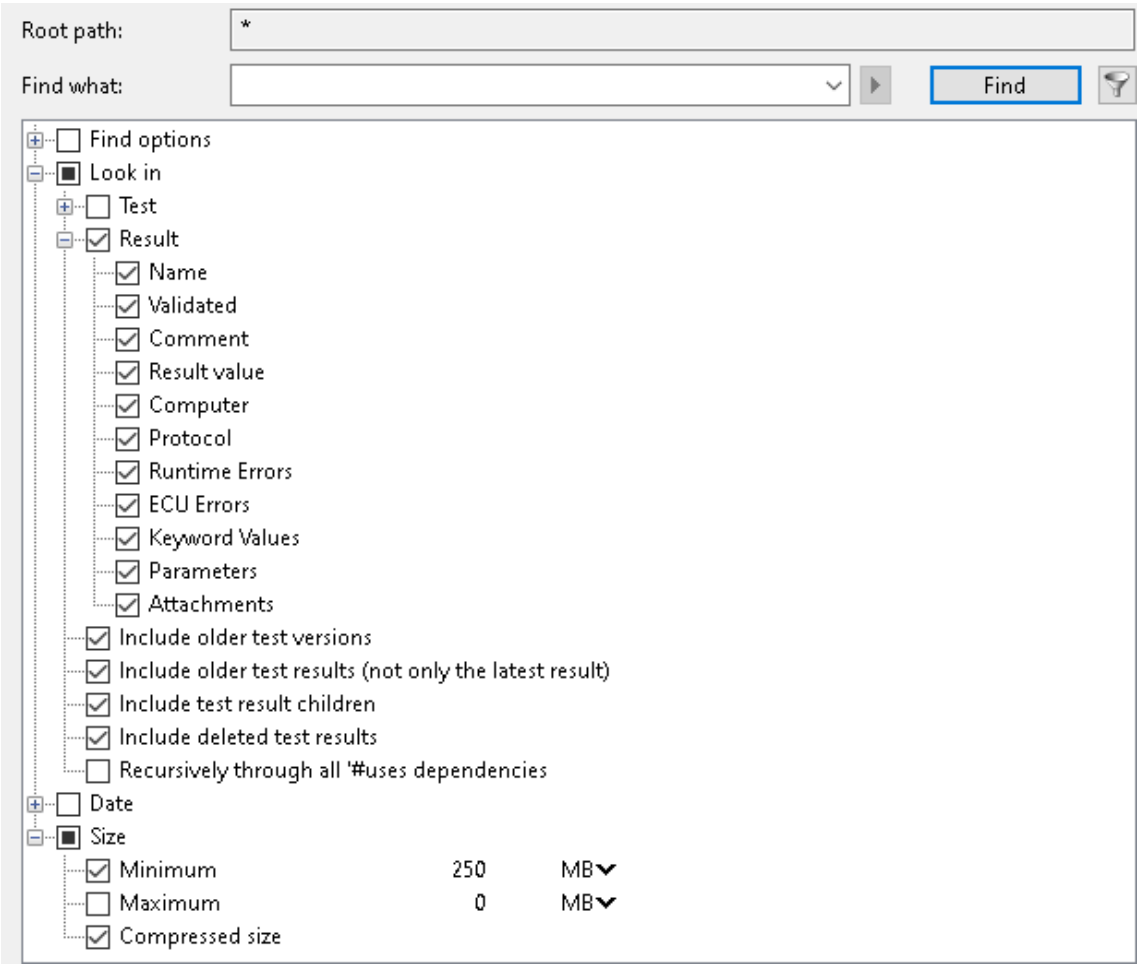


Figure 433 – Find dialog Example 3

5.18.1.2 Finding by Basic handler

In case of more complicated find requests you have the possibility to decide by Basic handler if a test object or a test result is part of the result list. Depending on whether you search for test objects, test results or both, you can define a function in your Basic handler with predetermined interface. When the search process is running through the Test Manager tree as usual, this function is called once for each test object or test result.

To use this, please activate `Path for finding by Basic handler` and enter the Test Manager path to the handler.

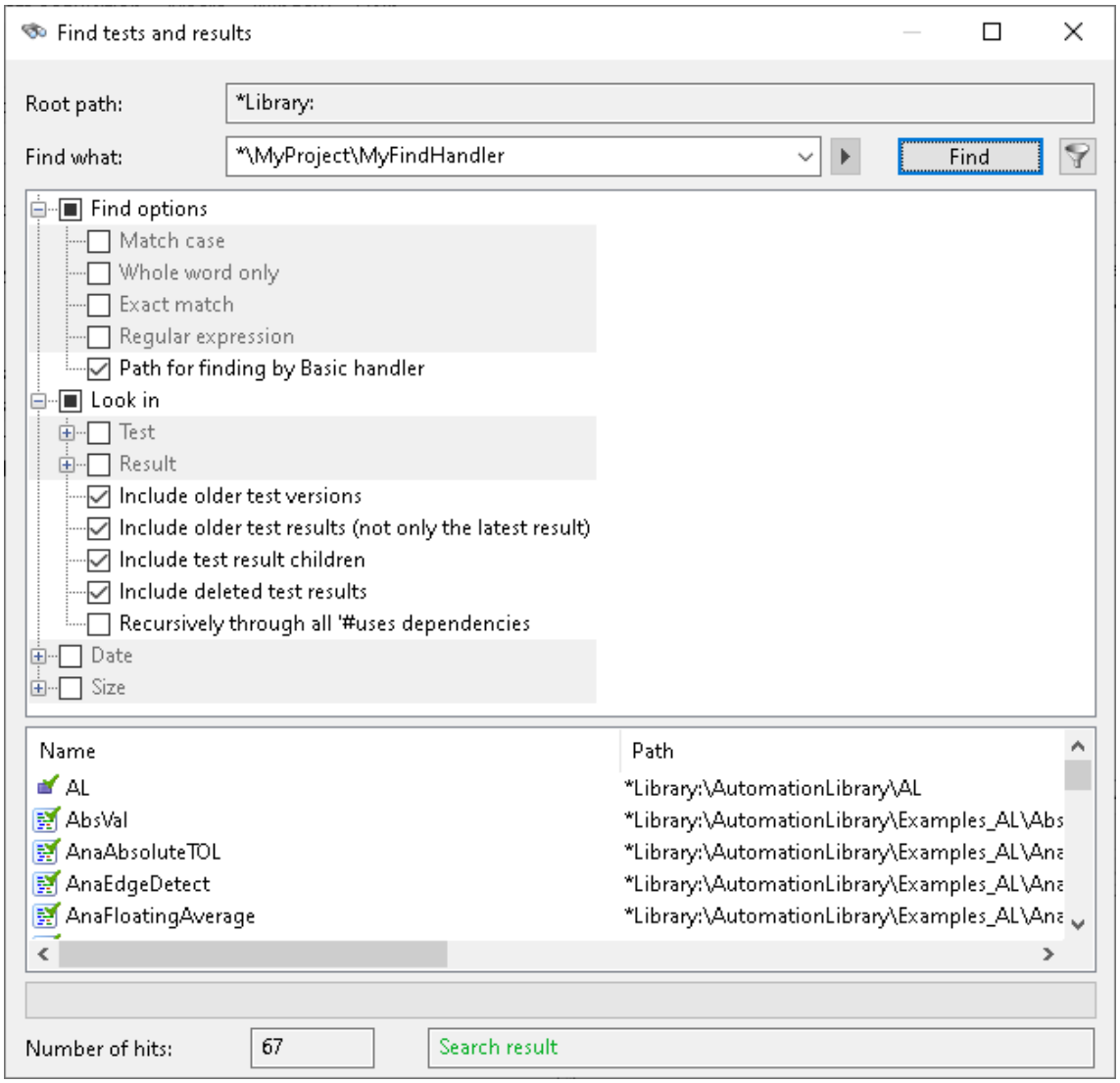


Figure 434 – Find by Basic handler

The Basic handler, for instance, could be implemented as follows:

```
'#Language "WWB.NET"
Option UsesAmbiguousError On
Option Explicit

'This function is called each time a test item is analyzed;
'Return True if the test item is part of the search result;
'Remove this function if you are not looking for test items;
Function FindTestItem(ti As TestItem) As Boolean
    'Verify if "fog" is part of the name
    Return 0<InStr(ti.Name, "fog")
End Function

'This function is called each time a test result is analyzed;
'Return True if the test result is part of the search result;
'Remove this function if you are not looking for test results;
Function FindTestResult(tr As TestResult) As Boolean
    'Verify date
    If tr.Date<#15/02/2018# Then Return False

    'Verify result parameter
    Dim tp As TestParameter = Nothing
    On Error Resume Next 'in case of non-existent parameter
    tp = tr.TestParameters("Par")
    On Error GoTo 0
    If IsNothing(tp) Then Return False
    Return CStr(tp.Value)="V3"
End Function
```

5.18.2 Filter tests and test results

You can use the search result as a filter for all Test Manager windows (e.g. Test Manager tree, Execute tab, overview tab). When the search operation has finished, a click on the button `Apply as filter` effects that only the objects of the result list are displayed.

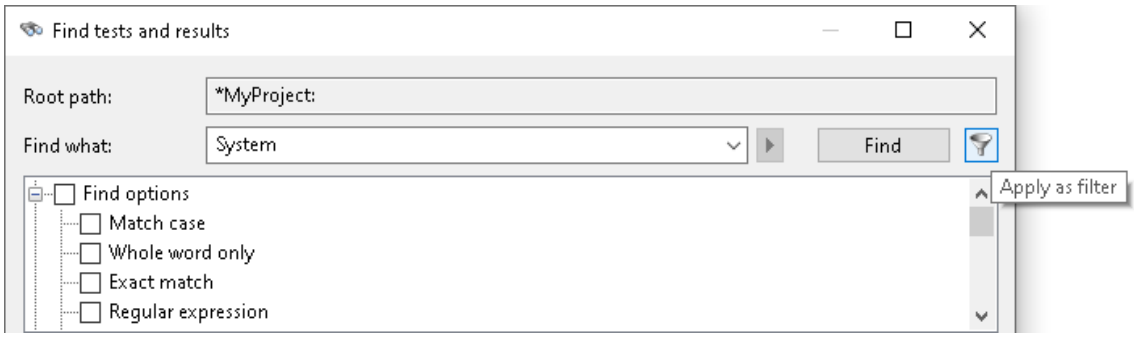


Figure 435 – Apply as filter

After that, the filter button in the Test Manager toolbar is displayed in green color.



Figure 436 – Test Manager toolbar after activated filter

By click on this button you can toggle between activated  and deactivated  filter. If the displayed objects do not match the search result anymore (e.g. a second find operation was started), then the button icon shows a red filter  .

So, a green or red filter icon indicates that not all of the existing objects may currently be visible.



The button `Apply as filter` is only activated if the set of found objects is a subset of all objects.



The Test Language is not affected by the filtered objects.



When executing via the `Execute` tab (also in the Remote Test Sequencer mode), the filter is also considered. In Remote Test Sequencer mode, however, it is important that the filter is set on the Sequencer server. On the Sequencer client of another computer, on the other hand, the filter has no influence on the test objects to be executed.

5.18.3 Find and replace in test programs

If you want to replace a search string by a new string through all test programs of a subtree then you have to activate the context menu of the subtree's root item. Then click on *Replace…* (see ***Figure 437***).

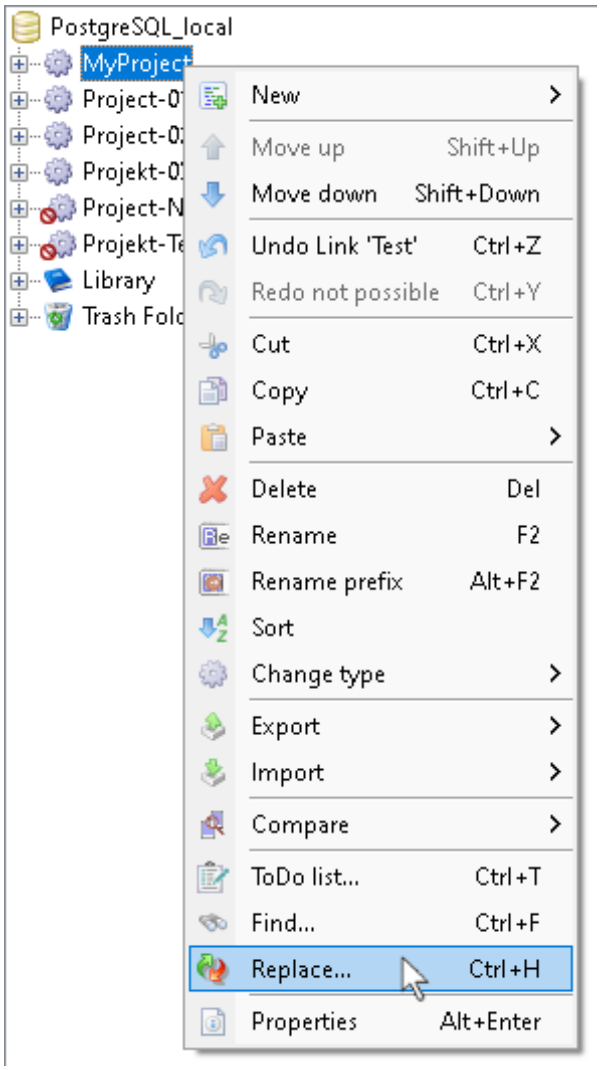


Figure 437 – Menu item "Replace…"

The Replace dialog is like the Windows standard dialog for replacing text (see ***Figure 438***).

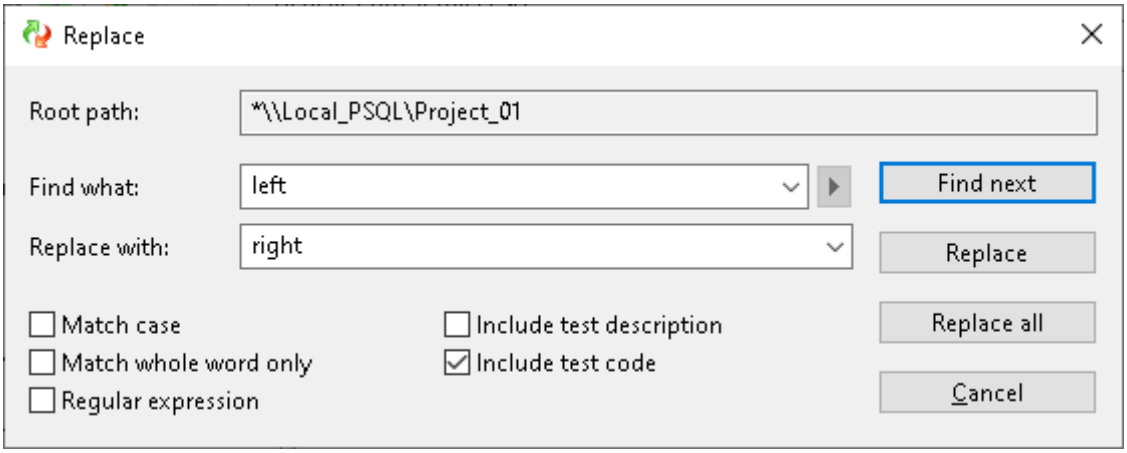


Figure 438 – The Replace dialog

Root path: Contains the root item of the search subtree

Find what: the search string that has to be replaced

Replace with: the new text that replaces the string that was found

Match case: Differences in case between search string and found string will not be ignored.

Match whole word only: a character except of a letter, a digit or an underscore must limit the found string.

Regular expression: The search string is interpreted as a regular expression (see chapter ***5.18.4 Regular expressions***).

Include Test Description: Find and replace in the description of the tab `Info`.

Include Test Code: Find and replace in the source code of the test in the tab `Program`.

If you do not want to replace the found string and continue at the next match, you can click on the button `Find next`. To replace the text, you click on `Replace`. If you do not want to replace incrementally, but you want to replace all at once, then click the button `Replace All`. With the button `Cancel` you finish the replacement.



The function `Replace` only can be used on tests with writing grants.



If the check box *Regular expression* is checked, then \r, \n, and \t can also be used in the *Replace with* text.

5.18.4 Regular expressions

Regular expressions are text patterns that are used for string matching. Regular expressions are strings that contain a mix of plain text and special characters to indicate what kind of matching to do. PROVEtech:TA uses the parser of the Boost Library (www.boost.org) for regular expressions. The following text only describes a small part of the possibilities. Please open the Boost Library homepage for more information about the syntax.

Suppose we are looking for a numeric digit then the regular expression we would search for is “[0-9]”. The brackets indicate that the character being compared should match any

one of the characters enclosed within the bracket. The dash (-) between 0 and 9 indicates that it is a range from 0 to 9. Therefore, this regular expression will match any character between 0 and 9, that is, any digit. If we want to search for a special character literally, we must use a backslash before the special character. For example, the single character regular expression "*" matches a single asterisk. In the table below the special characters are briefly described.

Character	Description
^	Beginning of the string. The expression "^A" will match an 'A' only at the beginning of the string.
^	The caret (^) immediately following the left-bracket ([) has a different meaning. It is used to exclude the remaining characters within brackets from matching the target string. The expression "[^0-9]" indicates that the target character should not be a digit.
$	The dollar sign ($) will match the end of the string. The expression "abc$" will match the sub-string "abc" only if it is at the end of the string.
|	The alternation character (|) allows either expression on its side to match the target string. The expression "a|b" will match 'a' as well as 'b'.
.	The dot (.) will match any character.
*	The asterisk (*) indicates that the character to the left of the asterisk in the expression should match 0 or more times.
+	The plus (+) is similar to asterisk but there should be at least one match of the character to the left of the + sign in the expression.
?	The question mark (?) matches the character to its left 0 or 1 times.
()	The parenthesis affects the order of pattern evaluation and also serves as a tagged expression that can be used when replacing the matched sub-string with another expression.

[]	Brackets ([and]) enclosing a set of characters indicates that any of the enclosed characters may match the target character.
\r \n \t	Special characters for page break and tabulators. Please notice that line breaks in Windows always consist of two characters represented by the \r\n string.

The parenthesis, besides affecting the evaluation order of the regular expression, also serves as tagged expression that is something like a temporary memory. This memory can then be used when we want to replace the found expression with a new expression. The replace expression can specify $& which means that the $& represents the sub-string that was found. So, if the sub-string that matched the regular expression is “abcd”, then a replace expression of “xyz$&xyz” will change it to “xyzabcdxyz”. The replace expression can also be expressed as “xyz$0xyz”. The “$0” indicates a tagged expression representing the entire sub-string that was matched. Similarly we can have other tagged expression represented by “$1”, “$2” etc. Note that although the tagged expression $0 is always defined, the tagged expressions $1, $2 etc. are only defined if the regular expression used in the search had enough sets of parenthesis. Here are few examples.

String	Search	Replace	Result
Mr.	(Mr)(\.)	$1s$2	Mrs.
abc	(a)b(c)	$&-$1-$2	abc-a-c
bcd	(a|b)c*d	$&-$1	bcd-b
abcde	(.*)c(.*)	$&-$1-$2	abcde-ab-de
cde	(ab|cd)e	$&-$1	cde-cd

Chapter

6

Diagnostics

PROVEtech:TA supports two different diagnostic variants. You typically only install one of them. The following chapters describe them.

- The ***CAESAR diagnostics*** (chapter ***6.1 CAESAR diagnostics***) is based on the CAESAR software library and is the standard diagnostics package for the Daimler AG.

- The ***MCD-3 D diagnostics*** (chapter ***6.2 MCD-3 D diagnostics***) is based on the usage of the MCD-3 D API to execute the diagnostic services and currently uses the DSA or the Softing implementation of the MCD-3 D API.

6.1 CAESAR diagnostics

In this chapter you will find out:

- how the diagnostics dialog works
- how you can read the ID Block of an ECU
- how ECU faults are read and cleared
- how you can execute a diagnostic service
- how you can code an ECU with the aid of “code strings” and/or “fragments”
- how to flash an ECU

6.1.1 Basics

For a better understanding of its functionality, the internal structure of the diagnostics is briefly described here.

Database and communication subsystem

The diagnostics essentially consist of two subsystems, a database and a communication component, where the communication component internally uses information from the database part.

The communication component is responsible for the data exchange between your computer/test system and an existing ECU via a data line or a data bus (CAN). Before a data exchange can take place, this component sets up a communication channel to the corresponding ECU.

For each ECU used, the database component manages a detailed description of the specific properties of the ECU. This description must be available in the form of a .cbf file (CAESAR Binary Format). The software uses this information to address an ECU correctly via a communication channel.

Moreover, the diagnostic software uses the data of the .cbf file to provide you with “offline information” on the relevant ECU. This information is available even when the relevant ECU is not connected.

Structure of diagnostics in PROVEtech:TA

PROVEtech:TA manages all of the diagnostic objects in a polymorphic tree structure whose root is the so called “ECU Manager”. Each diagnostic object has different

attributes and functions, which appear in the diagnostics dialog as soon as the object is selected in the tree control of the diagnostics dialog.

6.1.2 The diagnostics dialog

With the diagnostics dialog, you can perform all diagnostics functions supported by PROVEtech:TA.

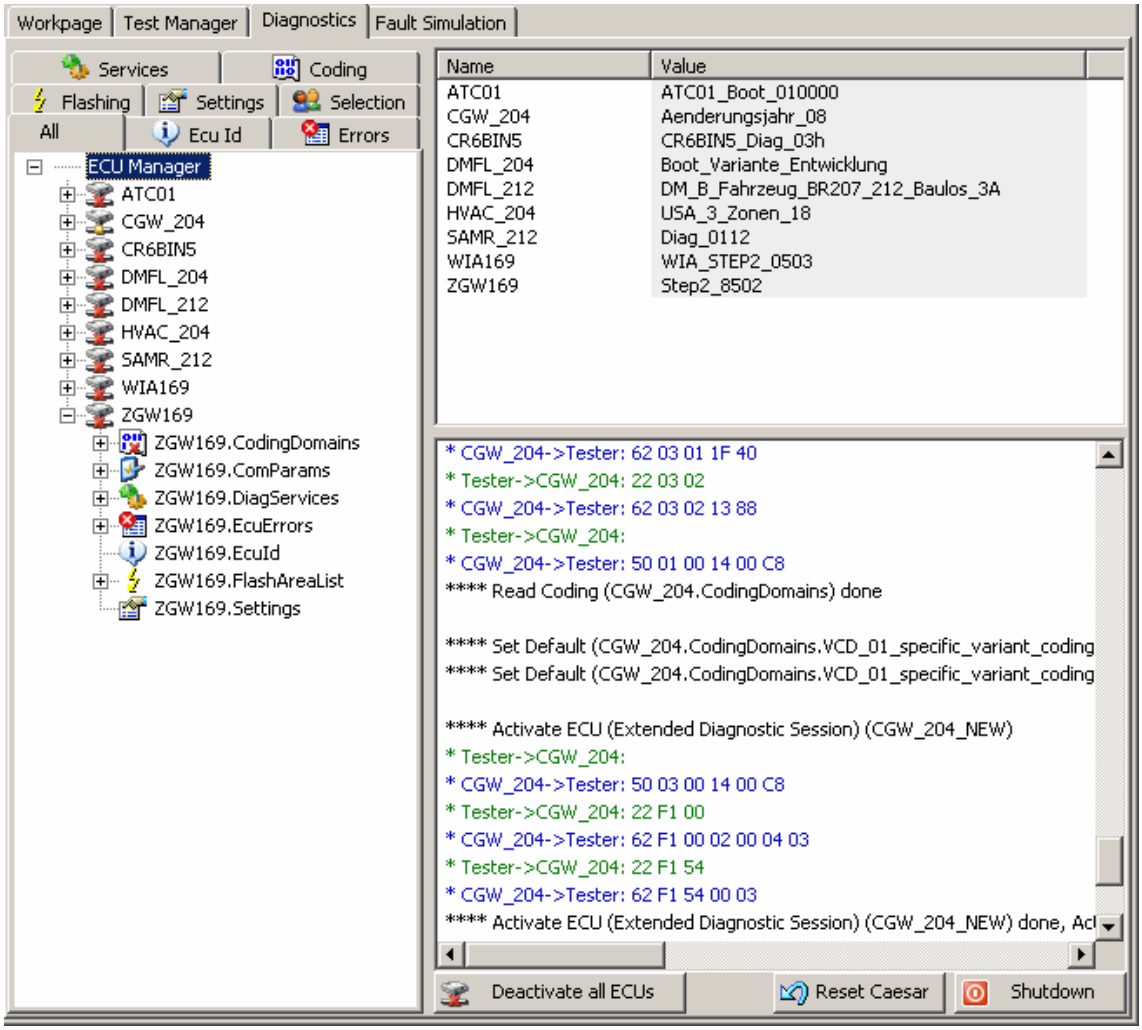


Figure 439 – Diagnostics dialog

The different parts of the diagnostics dialog are:

- Left, the diagnostics tree, which consists of eight independent views, that can be selected with the tabs (left top) `All`, `Ecu Id`, `Errors`, `Services`, `Coding`, `Flashing`, `Selection` and `Settings`.
- Top right of the diagnostics tree is the data window. Here, similar to the windows explorer, a tabular representation of the children nodes of the currently

selected tree object are displayed. The number and types of the displayed columns depends on the type of the currently selected diagnostic object in the tree. Editable data fields are marked by a slightly different background color (see column “Value” in above figure).

- Bottom right of the diagnostics tree, the trace window for displaying user actions and messages between PROVEtech:TA and the ECUs.
- Below the trace window is the command area. Here a varying number of buttons, whose number and functionality depend on the selected diagnostic object in the tree or data window, are displayed. Along with the context dependant buttons, on the right side of the command area you will find the static buttons `Reset Caesar` and `Shutdown`.

Execution of a diagnostics command

If you select an object in the diagnostics tree on the left, then an object dependant number of buttons will appear in the dynamic command area in the bottom right part of the dialog. These buttons represent the available commands of the selected diagnostic object.

Alternatively, you can select a diagnostic object in the data window of the diagnostics dialog. Once more, the available commands of the selected object will be shown as buttons in the dynamic command area.

The following sections describe the different parts of the diagnostics dialog, followed by a number of chapters containing brief descriptions of the different diagnostic objects an their available commands.

6.1.2.1 Static commands of the diagnostics dialog



Figure 440 – Static commands of the diagnostics dialog

Unlike the context dependant buttons, the static buttons of the diagnostics dialog are always present.

The button `Reset Caesar`

The button `Reset Caesar` performs a total reset of the diagnostics module and the underlying CAESAR library.

The button `Shutdown/Restart`

With the button `Shutdown`, you can temporarily deactivate the diagnostics module. After the deactivation, the button text changes to `Restart`. By another click on this button, the diagnostics module is newly initialized.

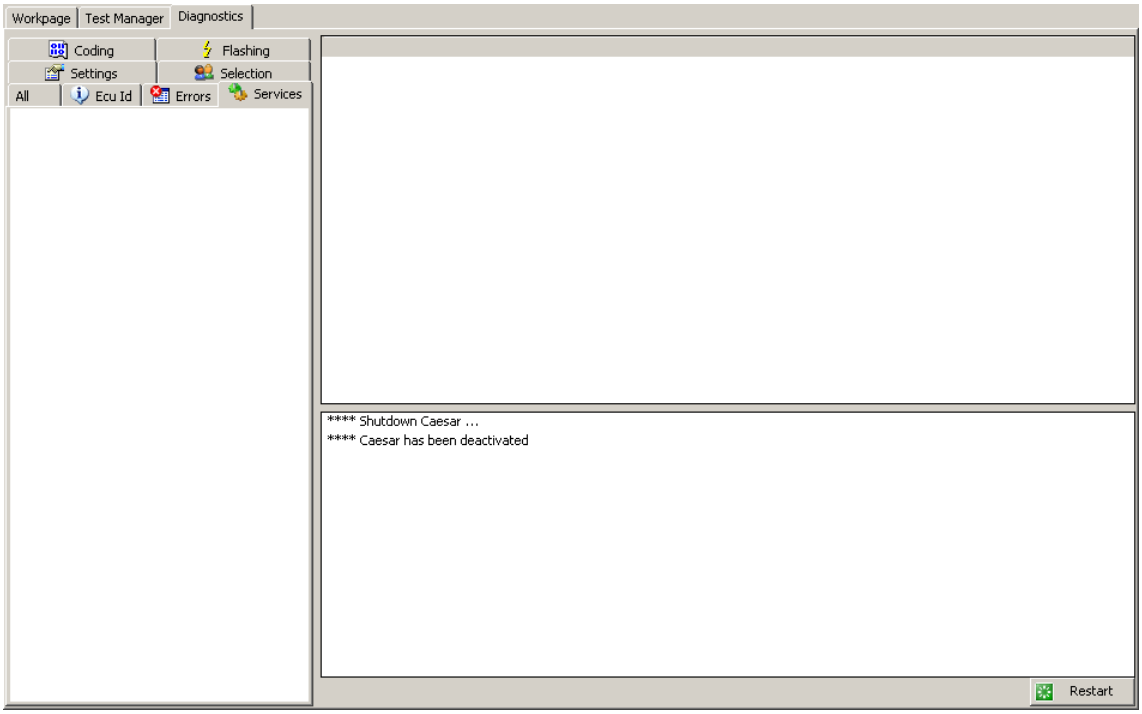


Figure 441 – Diagnostics module in shutdown state



When the diagnostics module has been shutdown, you can change the diagnostics configuration file or you can exchange .cbf files without having to shutdown PROVEtech:TA.

6.1.2.2 The diagnostics tree

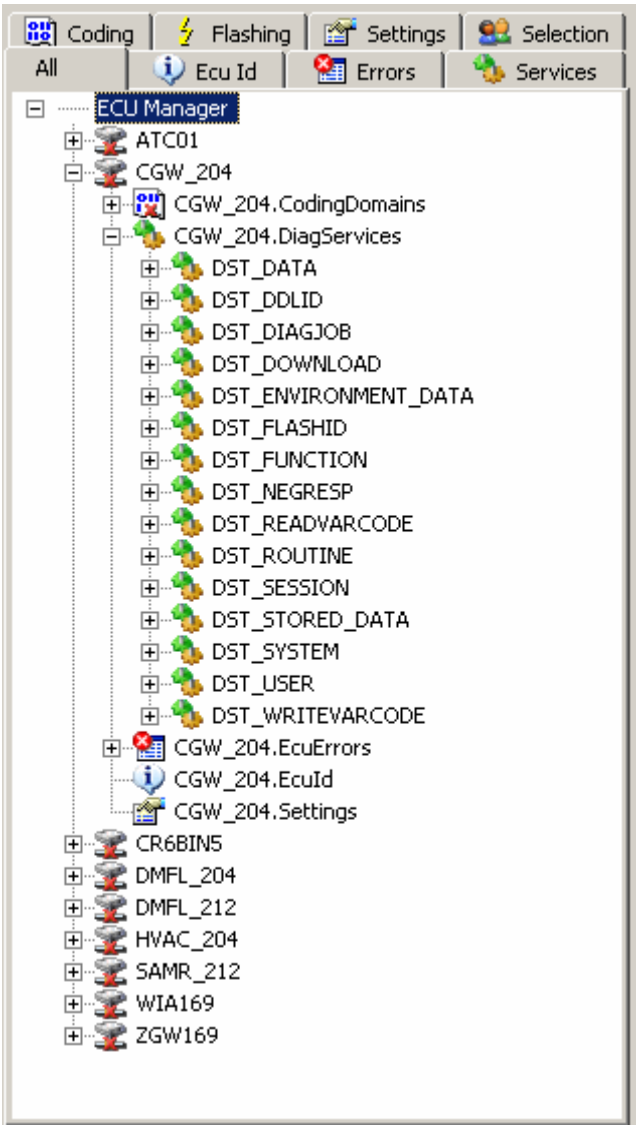


Figure 442 – The diagnostics tree

In the diagnostics tree the different diagnostic objects are displayed in a hierarchical order. The root object of the diagnostics tree is the “ECU Manager”, whose sub nodes represent the different ECUs. Depending on the activation state, the ECUs will be shown with two different symbols. The symbol represents an ECU that is not activated while symbolizes an activated ECU.

6.1.2.2.1 The context menu of a diagnostic object

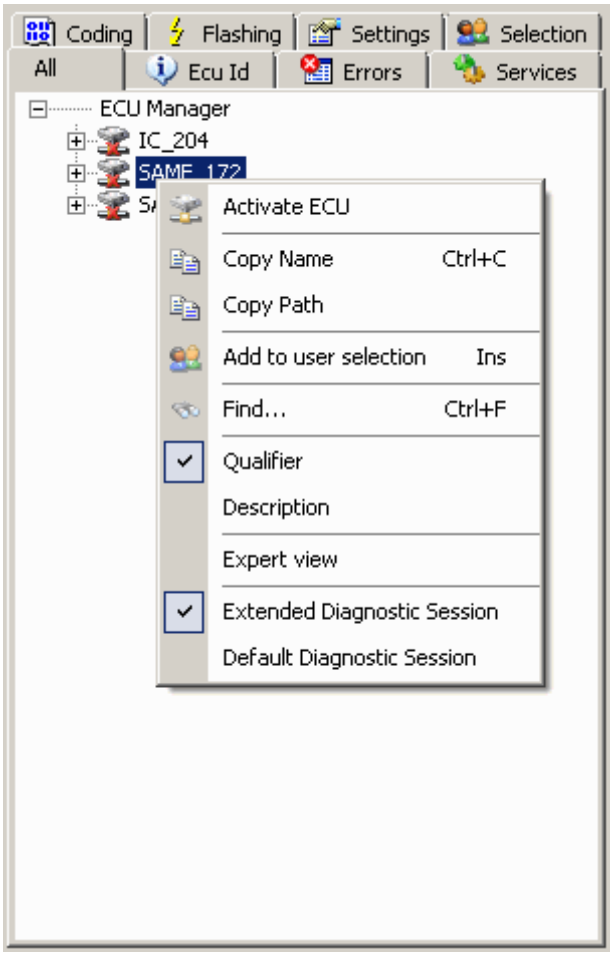


Figure 443 – Context menu of a tree object

Each object in the diagnostics tree has a context menu that can be activated by clicking the object with the right mouse button.

The context menu offers, along with search/copy functions and functions to control the view, object specific functions. The same actions can be performed using these object specific functions as with the object specific buttons in the dynamic command area. You will find a description of the possible object specific actions in the following chapters.

Copy Name

You can use the command *Copy Name* to copy the name of a diagnostic object into the clipboard. Depending on the viewing mode (*Qualifier* or *Description*), the so called "Qualifier" or the "Description" of the selected object will be copied.

Copy Path

The command *Copy Path* copies the path name of the selected object into the clipboard. Depending on the viewing mode, the path name consists of the “Qualifiers” or the “Descriptions” of the parent nodes and the selected node, separated by a period.

Qualifier and Description

With the menu commands *Qualifier* and *Description* you can display two different object names in the diagnostics tree. The “Qualifier“ is a short and unique object name which comes from the corresponding .cbf file. The “Description” is a more detailed object name which can, in the case of an ECU, also be user defined.

Expert view

In expert view additional diagnostic objects appear in the diagnostics tree that are not needed for normal usage of the diagnostics module. You can activate or deactivate the expert view for each of the eight tree views separately.

Activation / deactivation of an ECU

The commands *Activate ECU* and *Deactivate ECU* are available in all the nodes of the diagnostic tree except the ECU Manager object. Use them to activate or deactivate the corresponding ECU. Depending on the activation state of the corresponding ECU, one of the two menu commands is grayed.

The actual activation state of an ECU in the diagnostics tree is indicated by the symbols shown below:

	Deactivated ECU
	Activated ECU

For a description of the menu commands *Extended Diagnostic Session* and *Default Diagnostic Session* see chapter ***6.1.10 Extended and default diagnostics session***.

6.1.2.2.2 Searching in diagnostics tree

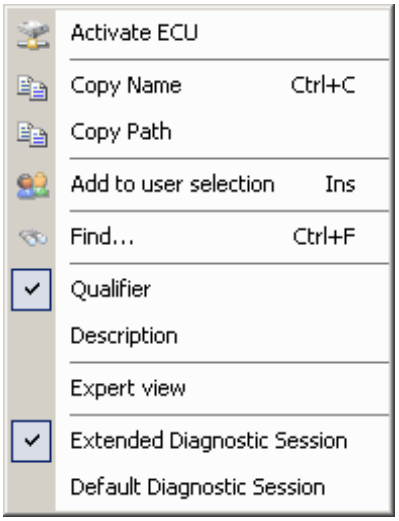


Figure 444 – Searching via context menu

You can search the diagnostic tree completely or partially for character strings that occur in the descriptions or qualifiers of the tree items. Searching is always limited to the sub-tree of the item on which it has been initiated. Items that match the search criteria are selected one after the other in the tree. When a search task is finished, the item on which the search was started is selected once more.

A click with the right mouse button to a tree item opens a context menu that contains the item *Find…*. The hotkey Ctrl+F is an alternative to open the following dialog:

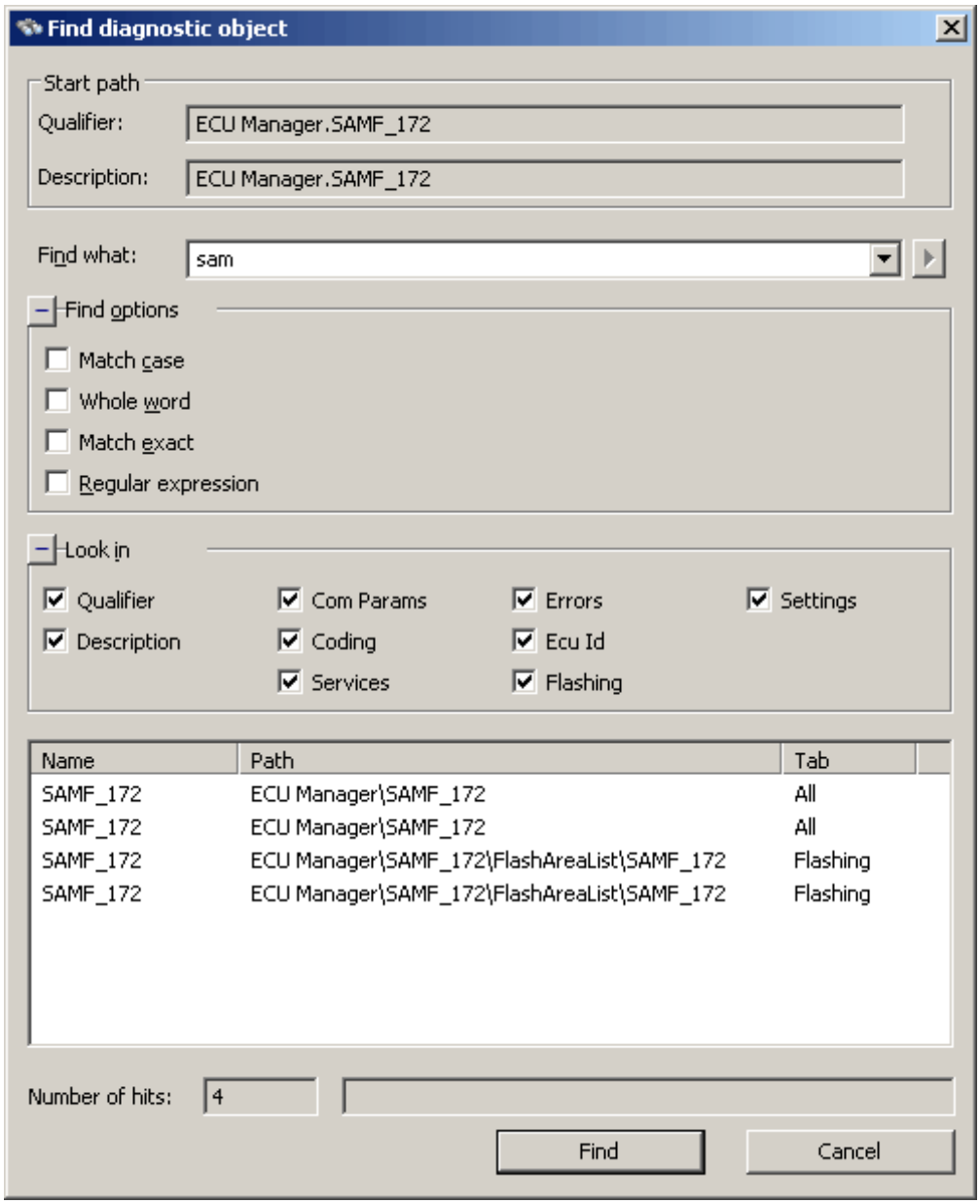


Figure 445 – The find dialog

Within `Start path`, the search dialog shows you the path of the subtree, which will be searched in "Qualifier" and/or "Description" notation. In the combo box `Find what`, you can enter a search expression or select an already used search expression. With the button `Find`, you can start searching. The button text switches to `Stop` during the search procedure. Click it or close the dialog to abort the procedure.

Additionally, the search dialog offers the following settings to control the searching behavior:

- `Find options`
  - `Match case`: Controls, if string comparisons are case sensitive or not.

- `Whole word`: Controls if matching only works for complete words
- `Match exact`: Controls, if the search expression can be a substring of a tree entry or if the search expression must match exactly.
- `Regular expression`: Controls, if the search expression is a regular expression or a common search expression. The check box also activates the right arrow button beside the `Find what` combo box. Pressing the button opens a menu that helps you assembling regular expressions. The last menu item opens the homepage of the Boost Library containing extended information about the syntax.

- `Look in`
  - `Qualifier`: The qualifiers of the diagnostic objects are searched.
  - `Description`: The descriptions of the diagnostic objects are searched.
  - The other check boxes define in which sub nodes to search in case of searching in one or several complete ECUs.

The find operation outputs the hits in the lower list. Double-click on a list entry to select it in the diagnostics tree.

6.1.2.3 The data window of diagnostics dialog

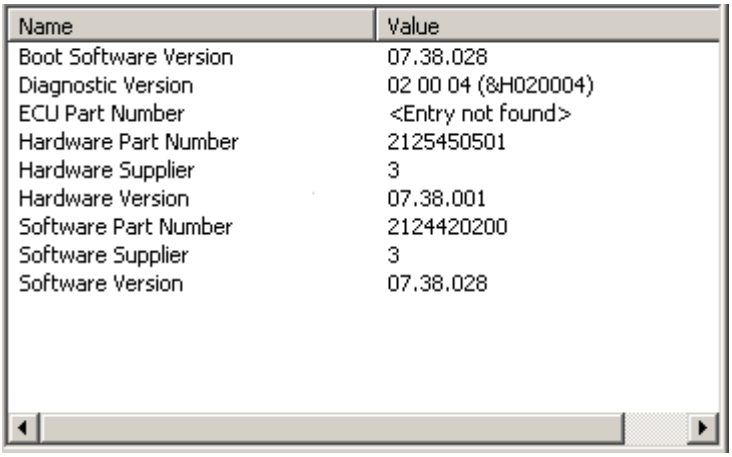


Figure 446 – Diagnostic data window

The diagnostic data window displays in tabular form, similar to the windows explorer, the children nodes and attributes of the currently selected object in the diagnostics tree.

Depending on the type of the currently selected object in the diagnostics tree, the data window displays different columns and contents. These are discussed in the following chapters.

Sorting the data window

If you click on a column header in the data window then the displayed data will be sorted by this column or the sorting sequence will be toggled. Changes to the sorting sequence affect all objects of the same type in the diagnostics tree and will be stored for the duration of the current PROVEtech:TA session.

Copying the data window contents into the clipboard

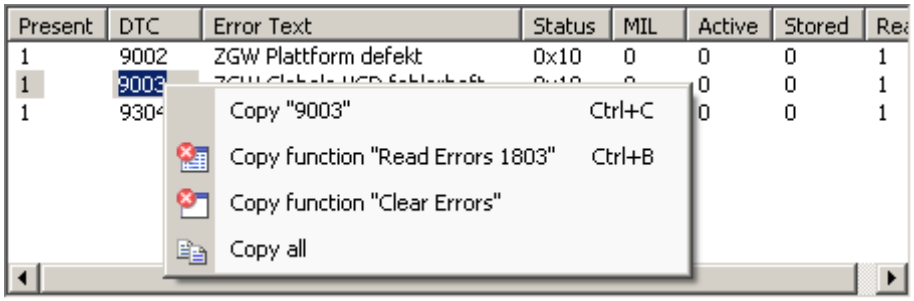


Figure 447 – Copy data window content to clipboard

You can copy the content of a single data element to the clipboard by using Ctrl+C. Alternatively you can just right click on the data element, and select the first option in the context menu.

You can copy the complete data window content to the clipboard by using the “Copy all” option in the context menu. The output format is a semicolon separated text.

Copying testscript functions into the clipboard

Ctrl+B copies the testscript call which fits to the content of the data window, into the clipboard. If there exist more than one function and testscript call, one function is offered via Ctrl+B, the others are offered via the context menu.

6.1.2.3.1 Data types used in data window

The diagnostics data window may have editable columns whose data fields store values of different data types. A brief description of the currently used data types in the diagnostics module follows below:

Numeric types	
UBYTE	Integer value between 0 and 255
UWORD	Integer value between 0 and 65 535

ULONG	Integer value between 0 and 4 294 967 296
SBYTE	Integer value between –128 and 127
SWORD	Integer value between –32 768 and +32 767
SLONG	Integer value between –2 147 483 648 and +2 147 483 647
FLOAT	Floating-point number
Other types	
BOOL	Boolean data type: values 0 = false, 1 = true
DUMP	Memory dump: representation as a variable-length string consisting of hexadecimal numbers
STRING	Plain text
UTF8 STRING	Plain text in UTF8 format
CHOICE	Combo box: a single value can be selected from a list of values determined by the software (only exists as a pass parameter)
Special types	
SPECIAL	Special type: pass parameters only, not editable
UNKNOWN	Unknown data type: the correct type might be determined after communicating with the corresponding ECU

6.1.2.4 The trace window of the diagnostics dialog

The trace window logs the actions performed by user interaction with the buttons of the diagnostics dialog or with the context menu of the objects of the diagnostics tree.

Additionally, the trace window displays the messages between PROVEtech:TA and the ECUs in hexadecimal format. This logging is asynchronous and is independent of direct user action. For example, if you run a test program that uses the diagnostics interface there can be logged messages at any time.

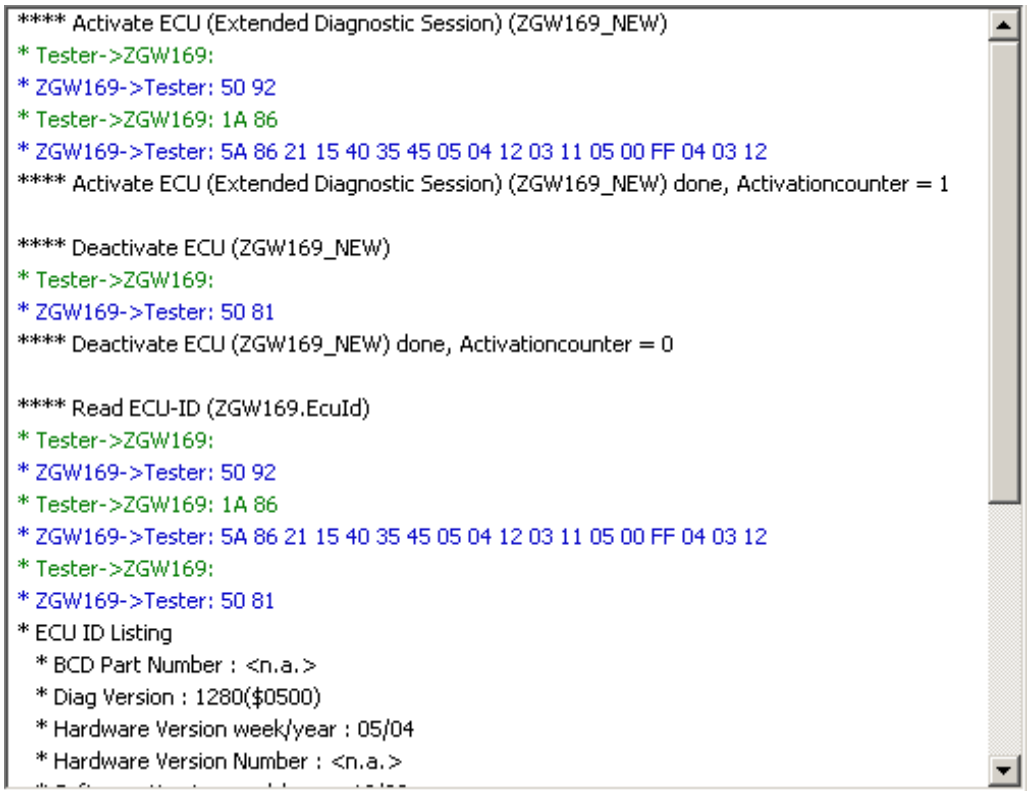


Figure 448 – Trace window

The trace window uses different colors to make it easier to distinguish between different types of log entries.

- Common messages produced by user interactions in the diagnostics dialog are displayed as black text. Different user actions are separated by a blank line.
- Red colored text, signals the occurrence of an error
- Messages from an ECU addressed to PROVEtech:TA are displayed in blue.
- Messages from PROVEtech:TA to an ECU appear in green.



You can copy the current trace window contents onto the clipboard by using the right mouse button (context menu) or by using the keys Ctrl+C or Ctrl+Ins.



The trace window can be cleared by using the right mouse button (context menu).

The displayed messages do not contain the session management and the lower protocol layers (e.g. raw CAN messages). In case the message processing is configured accordingly in your model, you can also analyze the messages with the message window on the workpage (see chapter ***4.4.14 The message window***). You can even see the ISO-TP protocol on CAN (see chapter ***3.2.7.2.3.12 Configuration of CAN channels***).

6.1.2.5 User defined selection of diagnostic objects

The tab `Selection` is provided for a user defined collection of often used diagnostic objects. Upon PROVEtech:TA termination, the selected objects are automatically stored under you user account and will be restored the next time you start the program.

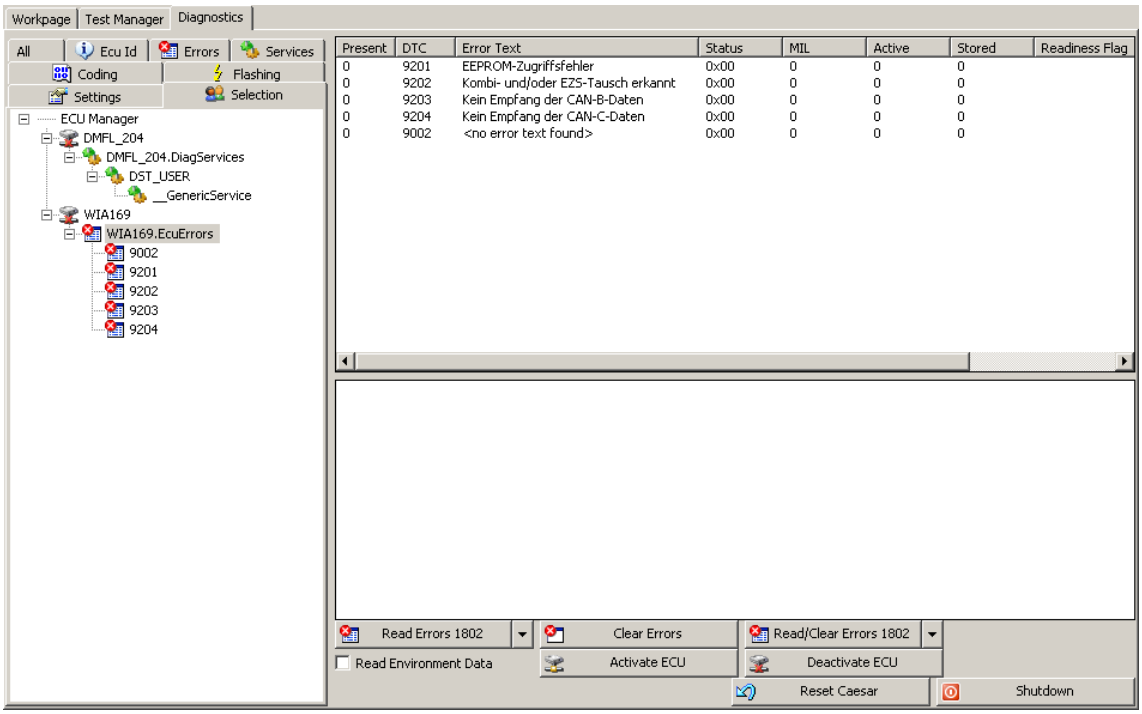


Figure 449 – User defined selection of diagnostic objects

6.1.2.5.1 Adding a diagnostic object to the user selection

To add a diagnostic object to the user selection, just select the object in one of the views (except the view `Selection`) of the diagnostics tree and press the key Ins. Alternately, this may be done via the context menu of a diagnostic object:

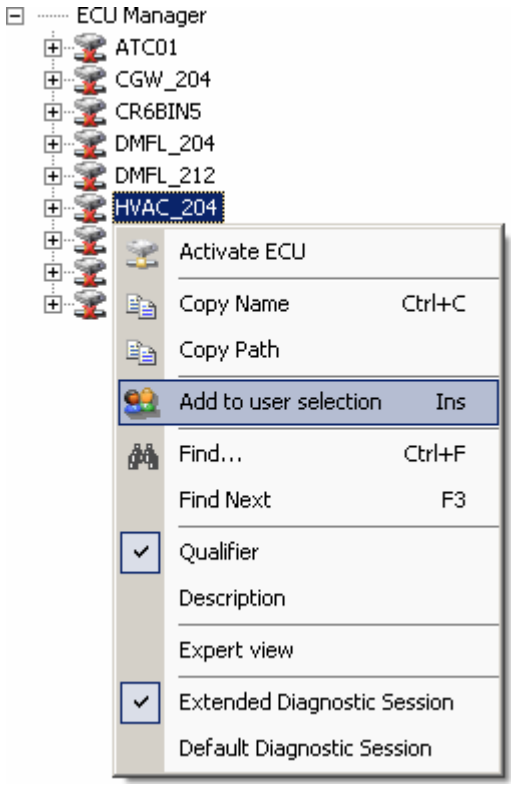


Figure 450 – Add a diagnostic object to the user selection

If the selected object has already been inserted into in the user selection tab, the corresponding menu command is not shown. In this case pressing the Ins key results in no action.

6.1.2.5.2 Removing a diagnostic object from the user selection

To remove an object from the user selection tab, just select the object in any of the tree views and press the Del key. Alternately, you can remove the object by using the context menu:

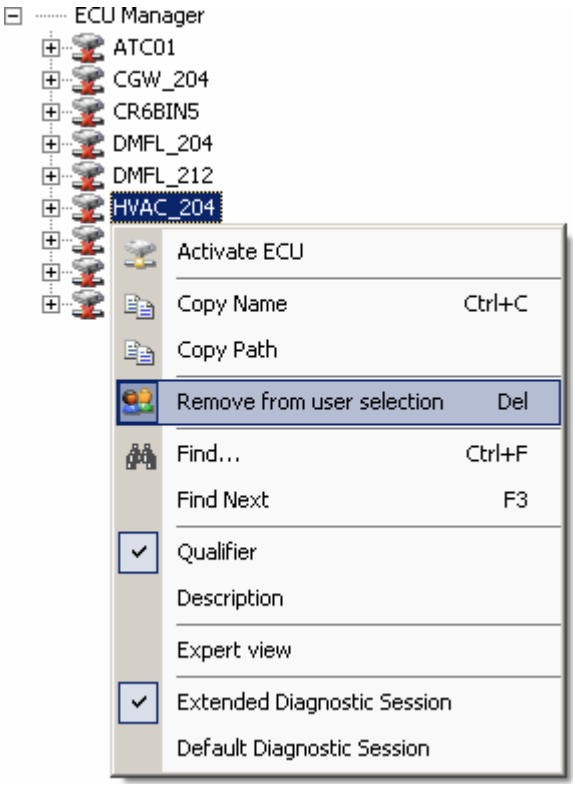


Figure 451 – Remove a diagnostic object from the user selection

6.1.2.5.3 Apply or reset the current user selection

To store the current user selection before PROVEtech:TA terminates or to reset the user selection, switch to the user selection tab.

With the context menu, you can apply (store) the current user selection before PROVEtech:TA terminates or reset the content of the user selection to the last stored version.

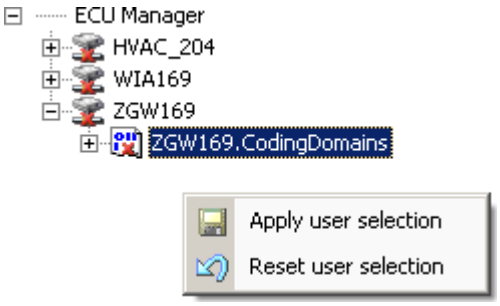


Figure 452 – Apply or reset the current user selection

If you insert a diagnostic object into the user selection, all child nodes of the object will also be inserted into to user selection.

6.1.3 ECU ID

Reading the ID block of an ECU is supported in the tree views `All` and `Ecu Id`.

In the view `Ecu Id` you only need to select an ECU object. Whereas, in the view `All` you must select additionally the child node “EcuId” of the corresponding ECU.

When the selection is performed, the dynamical command area (below the trace window) displays the button `Read ECU-ID` which can be used to read the ID block.

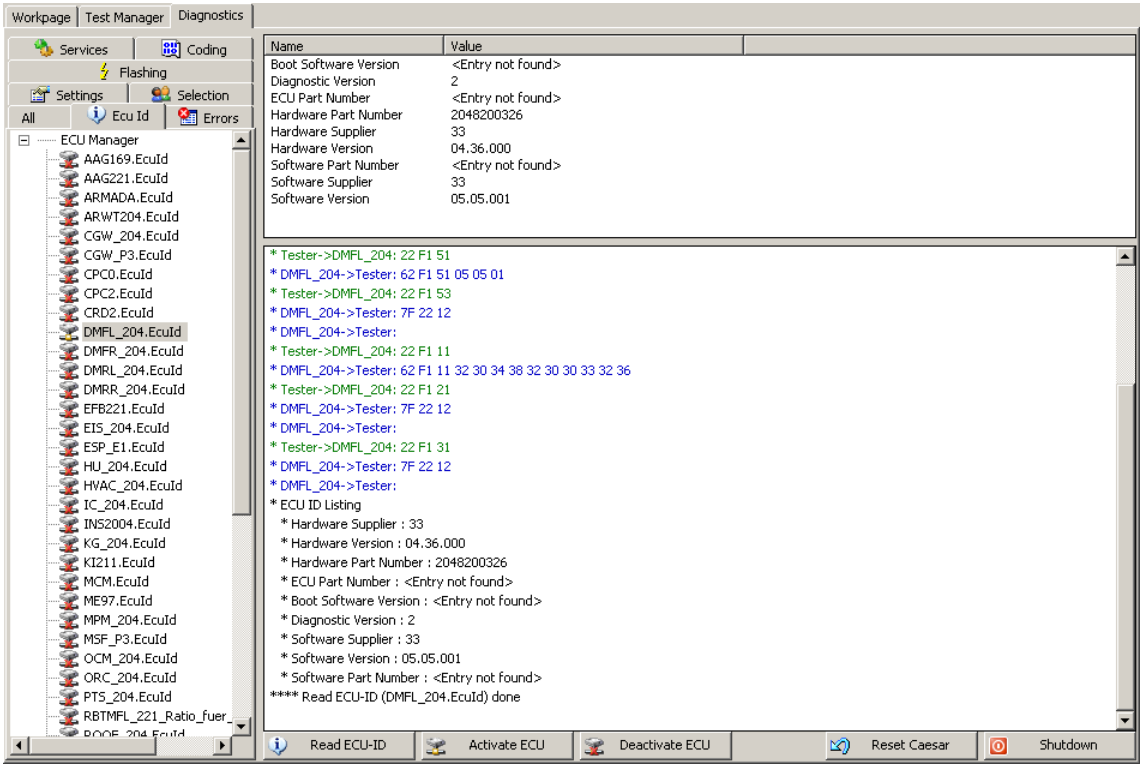


Figure 453 – ECU ID

The different data fields of the ID block are shown in the data windows in tabular form. Additionally, the data is written into the trace window of the diagnostics dialog.

The ID block is used internally to detect and set the correct variant of the ECU.

Depending on the ECU and the diagnostics protocol used, some fields of the ID data can be empty. In this case, you will see the output <n.a.> in the trace window.

6.1.4 ECU errors

To read or clear the error memory of an ECU, you can use the views `All` and `Errors` of the diagnostics tree.

In the view `All`, you have to select the ECU and then its child node “EcuErrors”. In the data window of diagnostics a tabular list of all possible errors of the ECU appears. In the `Errors` view of the tree, you only need to select the ECU.

For certain protocols PROVEtech:TA offers extended command options for reading or clearing the error memory of an ECU. You can recognize the existence of extended command options by the presence of an option button next to the command button. A description of the protocol specific extensions is contained in the subsequent chapters.

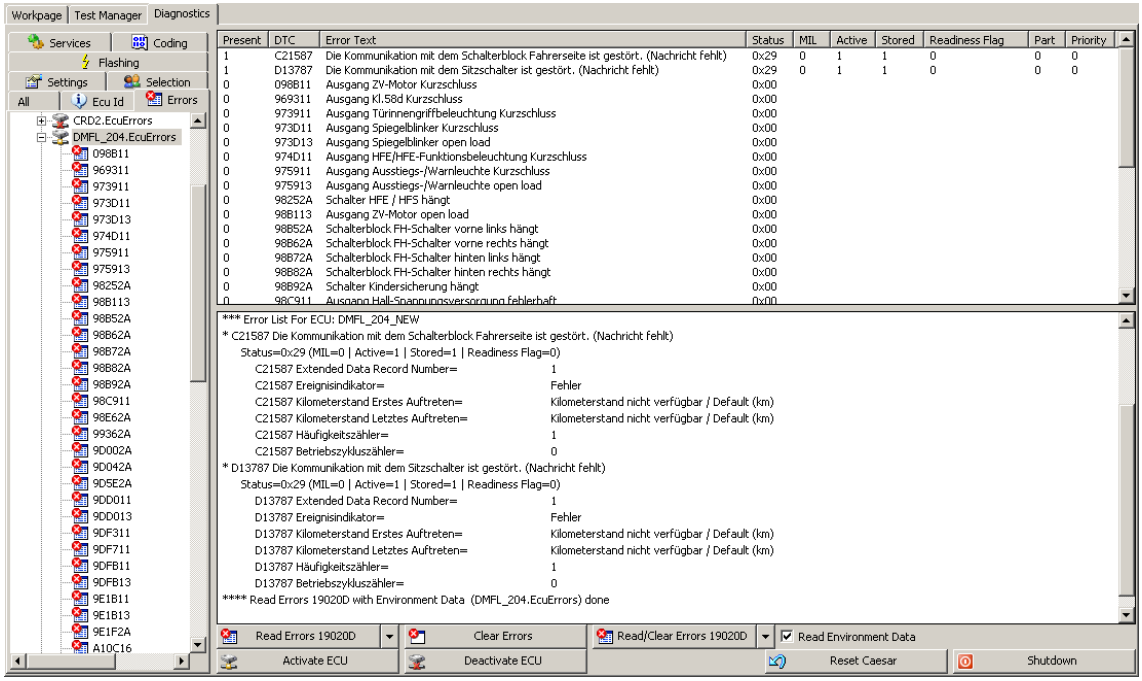


Figure 454 – ECU errors

A “1” in the “Present” column of an error entry in the data window of the diagnostics dialog signals that the corresponding error was reported at the time of the last read errors operation.

- The Read Errors button gives you a detailed report on the current fault conditions of the selected ECU in the data window of the diagnostics dialog.

- The 

button clears the fault memory of the selected ECU and resets the relevant ECU to a fault-free condition.
- The 

button automatically performs the two previously described operations in one step. For some ECUs only this method may be used since the errors must be read and cleared in a single communication cycle (please refer to the ECU specifications for details).
- If the check box 

is checked then the sub errors or environment data will also be read.

If these functions fail, it may be necessary to increase the “ConnectRetryCount”.

6.1.4.1 Environment data (sub errors)

For some ECU errors additional information (environment data and sub errors) is available.

When present, you can show the environment data list by selecting an ECU error in the diagnostics tree.

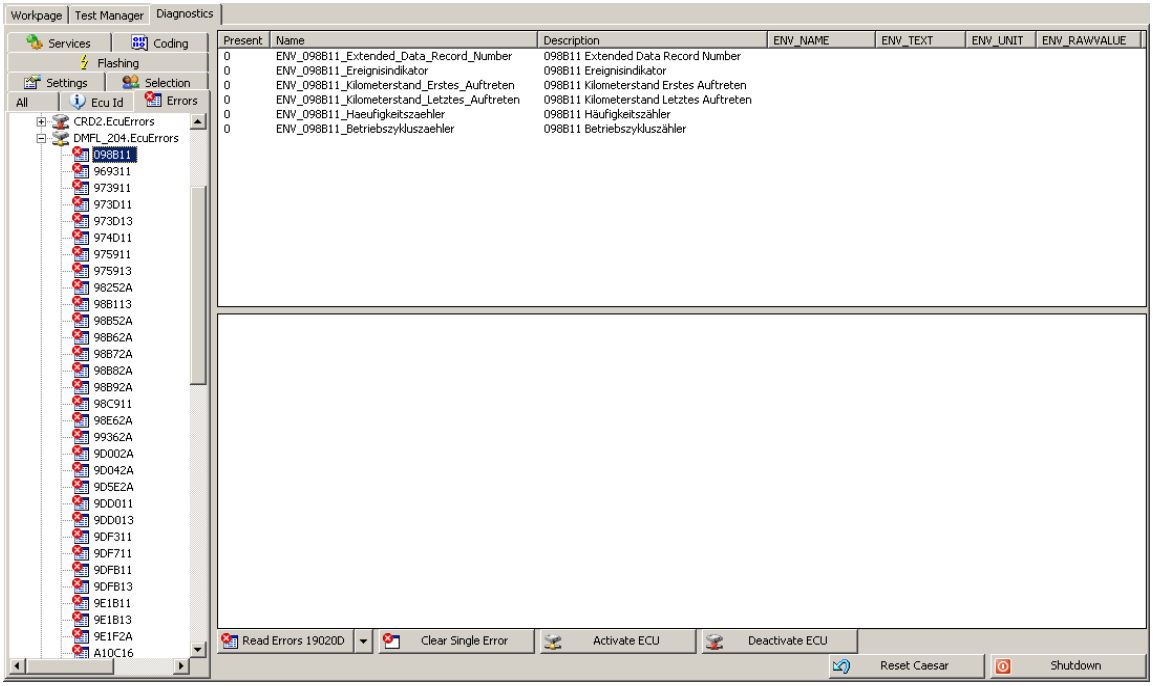


Figure 455 – Environment data of an ECU error

A “1” in the column “Present” of a sub error entry in the data window of the diagnostics dialog signals that the corresponding sub error was reported at the time of the last read errors operation.

With the button 

, you can read the error memory of an ECU. The displayed environment data will be updated according to the new information from the ECU.

The button 

will clear only the currently selected ECU Error.



Some ECUs do not support the clear single error feature. For details, refer to corresponding ECU specification.

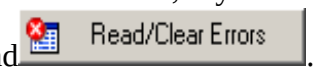
6.1.4.2 Extensions for the keyword protocol 2000

With keyword protocol 2000, the diagnostic trouble codes are read by executing service 0x18 (“Read Diagnostic Trouble Codes By Status”, see the protocol specification).

The diagnostics dialog of PROVEtech:TA supports the sub functions 0x00 (“Request Identified 2 Byte Hex DTC and Status” SAE J2012/ISO 15031-6 format), 0x01 (“Request Supported 2 Byte Hex DTC and Status” SAE J2012/ISO 15031-6 format), 0x02 (“Request Identified 2 Byte Hex DTC and Status”) and 0x03 (“Request Supported 2 Byte Hex DTC and Status”) of the service 0x18.

Remark: The sub functions 0x00 and 0x01 are mainly used with American ECUs.

By default, service 0x1802 is executed, if you click on the buttons 

and 

. If you click on the option button  next to the corresponding command button, a menu opens which can be used to execute service 0x1803.

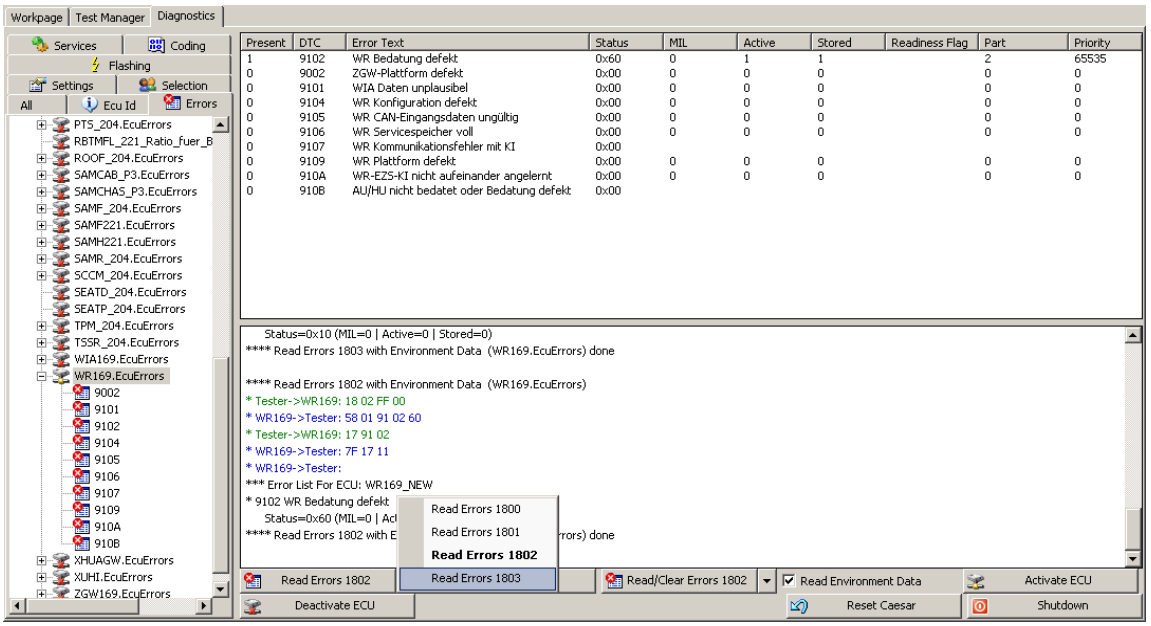


Figure 456 – Read Errors with option menu (keyword protocol 2000)

After a command execution via the option menu, the caption of the command button changes. The new default command of the corresponding command button is now the command executed via option menu.

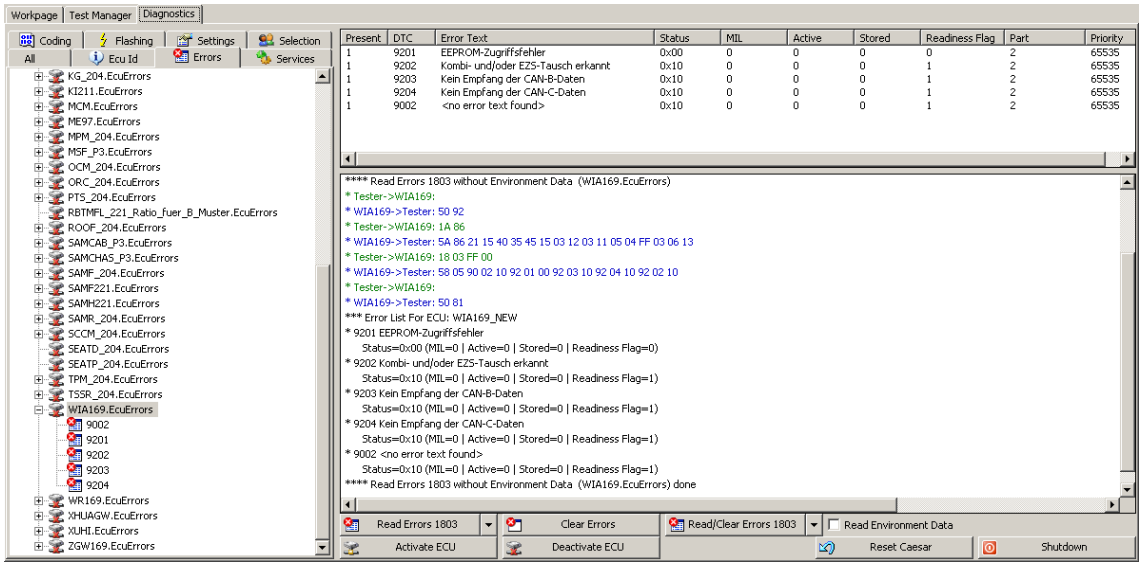


Figure 457 – Changed meaning of command buttons after command execution via option menu

6.1.4.3 Extensions for the UDS protocol

In the UDS protocol, reading diagnostics trouble codes is performed with the service 0x19 and its sub functions . The diagnostics dialog supports the sub functions 0x02

(report trouble codes by status mask), 0x0F (report mirror memory trouble codes by status mask) and 0x0A (List of all DTCs). More options are available by using the test language commands or by using the dialog for user defined items.

As is the case of the keyword 2000 protocol (see ***6.1.4.2 Extensions for the keyword protocol 2000***) the different services are executed by using an option button.

There is a possibility to create own buttons additionally to the preconfigured ones for reading the DTCs via a dialog. The dialog is opened by clicking the option `Configure user defined items…` in the options menu. The option is available for both the `Read Errors` option menu and the `Read/Clear Errors` option menu.

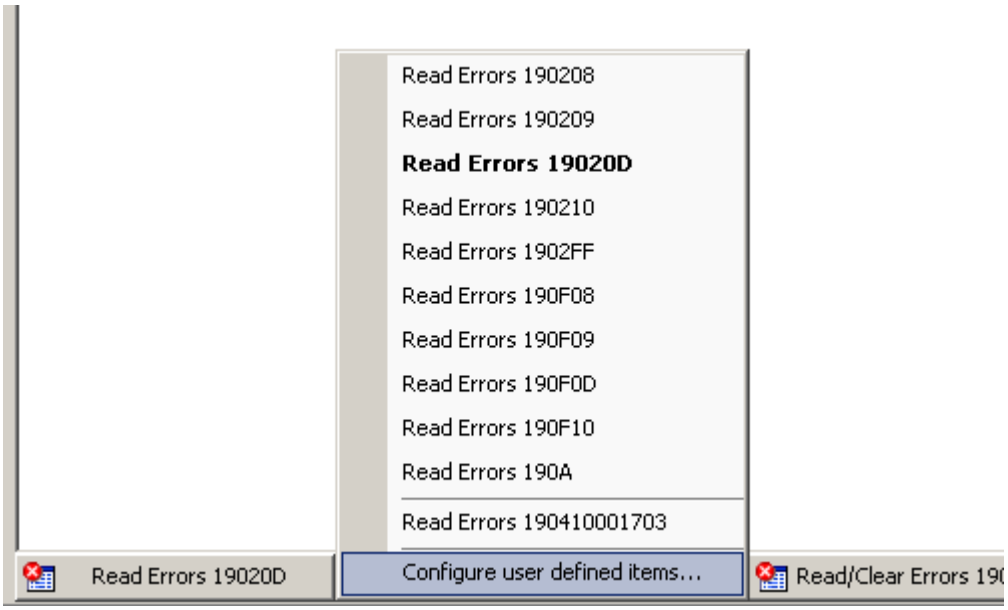


Figure 458 – Adding own options

There are three segments in the options menu. The most top segment consists of the preconfigured options for reading the DTCs, which are provided by PROVEtech:TA. The middle segment contains the user defined options. In the last segment there is only one option (`Configure user defined items…`), that opens the dialog for creating an own option.

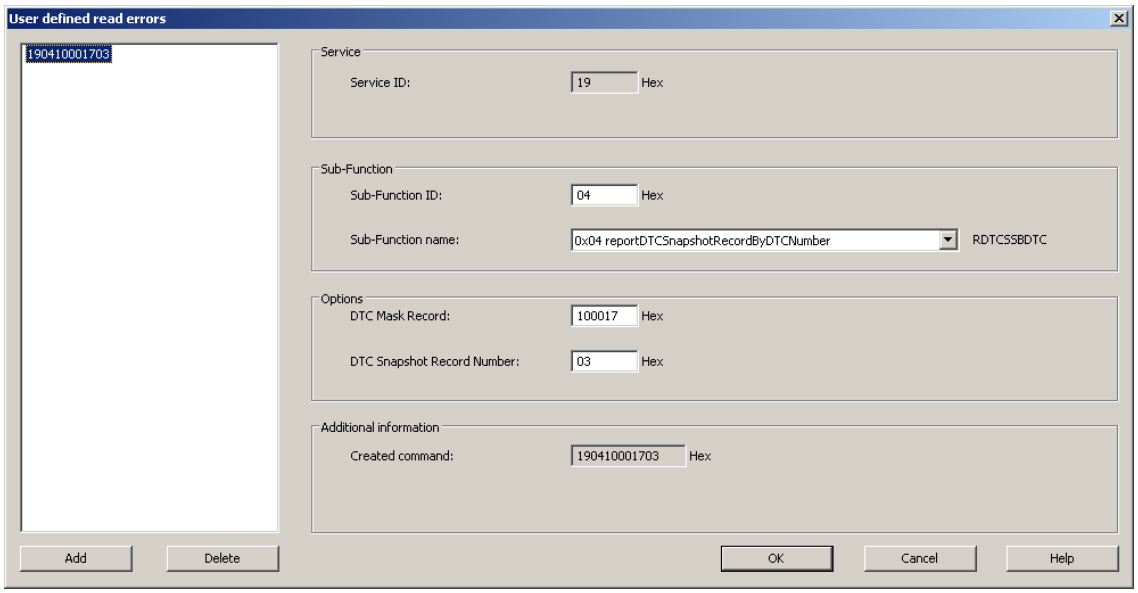


Figure 459 – Dialog for administrating own options

The dialog for creating user defined options is divided into two main sections. On the left side there is a list with the already created options. The buttons `Add` and `Delete` offer the possibility to extend or to reduce the list. In case of an empty list the button `Add` has to be pushed first. This inserts a new element into the list, that can be subsequently edited. The editing is done via the second main section on the right. In this area of the dialog the parameters are arranged, which are specified by the UDS protocol. A more detailed overview of the parameters can be found in the UDS specification. The service ID is in the most top group box of the dialog. Since the only correct value is 0x19 the field is read only. The only goal of the field is to offer a better and complete overview of the created option. The sub function can be adjusted in two different ways in the group box `Sub-Function`. Either via the sub function ID or via the sub function name. Both are equivalent and always synchronized to each other. The group box `Options` offers the possibility to set further parameters for the selected sub function. The available parameters are depending on the selected sub function. The amount of zeros in each parameter field corresponds to the amount of nibbles that the parameter requires. As a consequence of wrong inputs like a too big value or a forbidden sign (only digits and the letters from A to F are allowed) PROVEtech:TA will not allow the addition of the list or any further options until a correct value is inserted. A preview of the created command is shown in the group box Additional information. There are limitations for the parameters. Combinations that are defined in the UDS protocol do not have always an effect. In that case a message is shown in the group box Additional information. Such an option can be inserted but will have no effect when executed. Another limitation is caused by the ECU itself. In that case the ECU will answer with a negative response to the request. The list of the options is assigned with a click on `OK` and the new buttons are available in segment two of the options menu. Clicking `Cancel` restores the prior list. It should be minded that the list is stored user specific, especially when working on systems with multiple users.

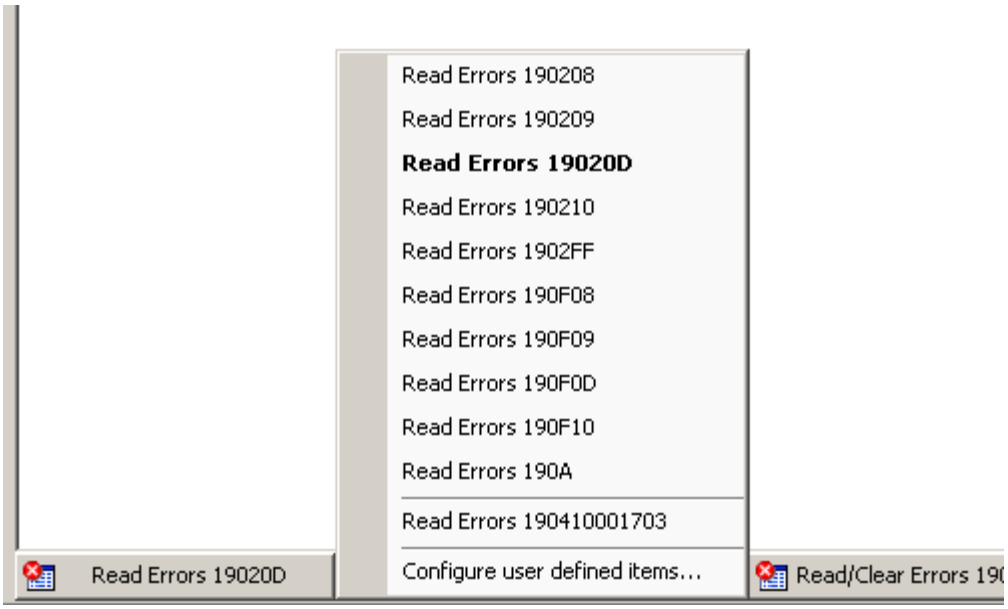


Figure 460 – Read errors with option menu (UDS protocol)

Below is a brief tabular description of the available options:

Service	**Function**	**State mask**
19020D	Read trouble codes with state	0D ➔ Test failed | Pending | Confirmed (= default value)
190210	Read trouble codes with state	10 ➔ Readiness Flag
190209	Read trouble codes with state	09 ➔Confirmed | Test failed
190208	Read trouble codes with state	08 ➔ Confirmed
1902FF	Read trouble codes with state	FF ➔ Get all errors
190F0D	Read mirror memory with state	0D ➔ Test failed | Pending | Confirmed
190F10	Read mirror memory with state	10 ➔ Readiness Flag
190F09	Read mirror memory with state	09 ➔Confirmed | Test failed
190F08	Read mirror memory with state	08 ➔ Confirmed
190FFF	Read mirror memory with state	FF ➔ Get all errors

The 8 bits of the DTC status bye have the following meaning:

- Bit0: TestFailed (corresponds to former Active flag)
- Bit1: TestFailedThisOperationCycle
- Bit2: PendingDTC

- Bit3: ConfirmedDTC (corresponds to former Stored flag)
- Bit4: TestNotCompletedSinceLastClear (former Readiness flag)
- Bit5: TestFailedSinceLastClear
- Bit6: TestNotCompletedThisOperationCycle
- Bit7: WarningIndicatorRequested

These eight bits are displayed on the GUI in two ways. On the one hand there is the hexadecimal value of all bits in the column “Status” on the other hand the single bits are displayed in their respective columns. In case you are using a J1939 ECU or the setting UseJ1939 is set to Yes there will be two more columns displayed with the SPN and FMI.

6.1.5 Diagnostic services

Before you can setup and execute a diagnostic service, you have to select a service in the diagnostics tree. For this purpose you can use the tree views `All` or `Service`.

In the view `All` you must first select an ECU in the diagnostics tree, then the child node “DiagServices” and finally the corresponding service class.

In the `Services` view, you only need to select the ECU followed by the service class.

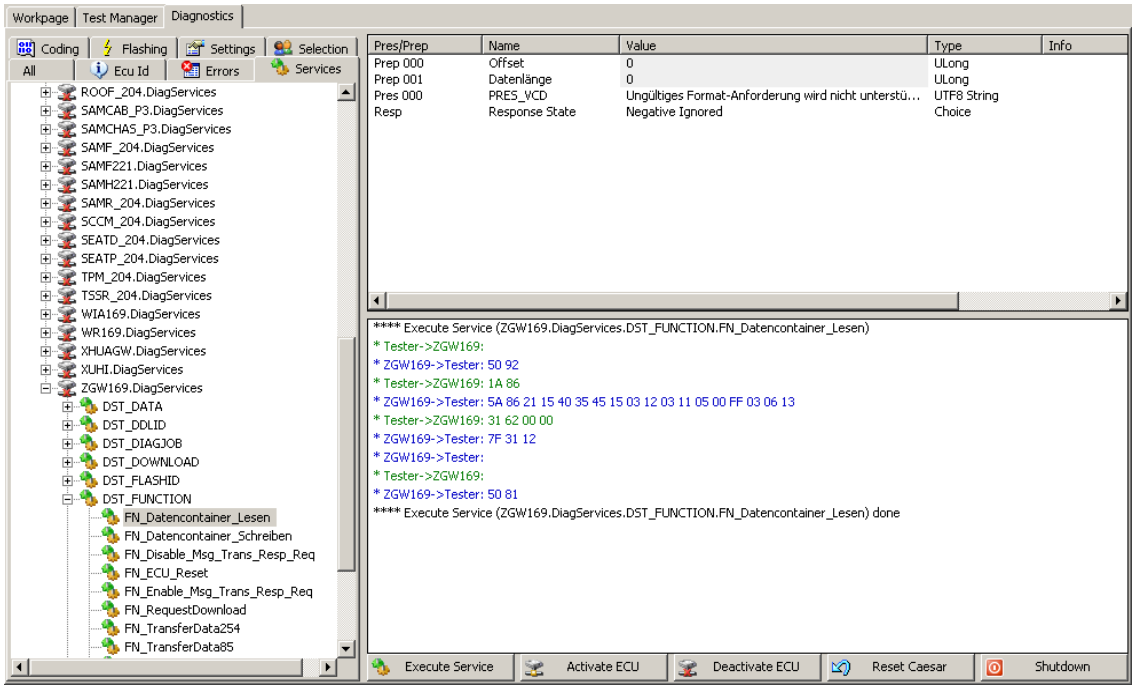


Figure 461 – Parameters of a diagnostic service

After selecting a diagnostic service the data window displays a tabular list of the service parameters. Furthermore, the button `Execute Service` appears in the dynamic command area of the diagnostics dialog. You can use this button to execute the service after you have entered pass parameters where necessary.

Below is a brief description of the columns of the data window.

Pres/Prep	Shows whether the object is a preparation or pass parameter (Prep), a presentation or return parameter (Pres), or a response state (Resp)
Name	Name of the service parameter
Value	Data value of the service parameter
Type	Data type of the service parameter
Info	Additional information, for example a default value

Preparation parameters (Prep) are values, which will be sent to the ECU when a service is executed. These values must be entered by the user. Each service can have no or any number of preparation parameters (pass parameters). The value column in the data window of parameters of this type is editable.

Presentation parameters (Pres) are values that the ECU returns to the tester after the service has been executed. Each service can have no or any number of presentation parameters (return parameters).

The response state (Resp) is a special type of presentation parameter. Its value field indicates whether the service execution was successful. Possible return values are “Positive”, “Negative” and “Negative Ignored”. Each service has exactly one parameter of this type.



You can edit a preparation parameter’s value by clicking on its value column in the data window of the diagnostics dialog. Depending on the data type, an entry field or a combo box will appear (CHOICE data type).

The background color of the preparation parameters value field is slightly different, this indicates that the field is editable.



Since PROVEtech:TA is designed for the use on HiL systems all services are visible. Therefore it can happen that services that PROVEtech:TA shows are not visible in other tools. For instance this is the case with the E-Tester. The E-Tester hides services that start with WVC_, RVC_ or DNU_.

6.1.5.1 Sending a user defined request message

In the diagnostics tree, you will find underneath each ECU node in the branch “DiagServices” a service class named “DST_USER”. This service class contains a diagnostics service called “__GenericService”. With this service, you can send user defined messages to the corresponding ECU.

The “__GenericService” has two parameters called “RequestMessage” and “ResponseMessage”.

- The preparation parameter “RequestMessage” is used to enter a user defined message, which will be sent to the ECU. The entry has to be in hex format.
- After execution of the service, the presentation parameter “ResponseMessage” contains the response of the ECU. The output format is also hex.

Example:

With a KW2000 ECU, you read the diagnostic trouble codes (DTC) by using the service identifier “0x18”. If you send the message “1803FF00”, the ECU responses with a list of all supported diagnostic trouble codes including the status byte of each DTC (see ***Figure 462***).

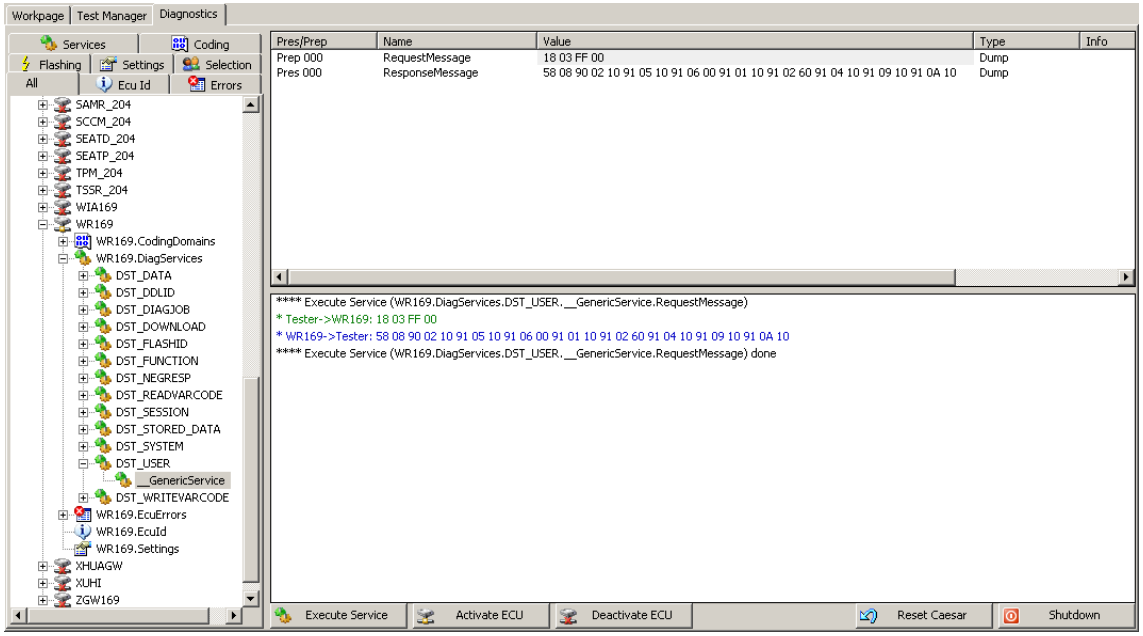


Figure 462 – Sending a user defined message

The diagnostics service “__GenericService” is not defined in the ECU’s .cbf file, but is provided by PROVEtech:TA.

The meaning of the sent messages and the ECU responses is protocol specific. For details look at the protocol specification or the specification of your ECU.

Only experienced users who are familiar with the protocol and ECU specification should use the “__GenericService”. PROVEtech:TA does not examine any sent messages or interpret the ECU responses.

An improper usage of the “__GenericService” can threaten the stability of CAESAR (and PROVETech:TA) or even damage your ECU.

6.1.6 Coding

6.1.6.1 General features

To be able to code an ECU, you must select a coding domain in the diagnostics tree. Coding domains are supported in the tree views `All` and `Coding`. Both views offer the same features. They only differ in the length of the selection path.

A “CodingDomain”, which shows the coding from ECU description file, is displayed with the symbol  . The Symbol  means that the coding displayed is synchronized to the coding in the ECU.

If you select “Read coding”, the used symbol changes to the “synchronized” symbol. The “syncflag” is also displayed directly in expert view inside of the coding.

When you use the view `All`, first select the ECU, then its child node “CodingDomains”. In the view `Coding`, the available coding domains are direct children nodes of the corresponding ECU object.

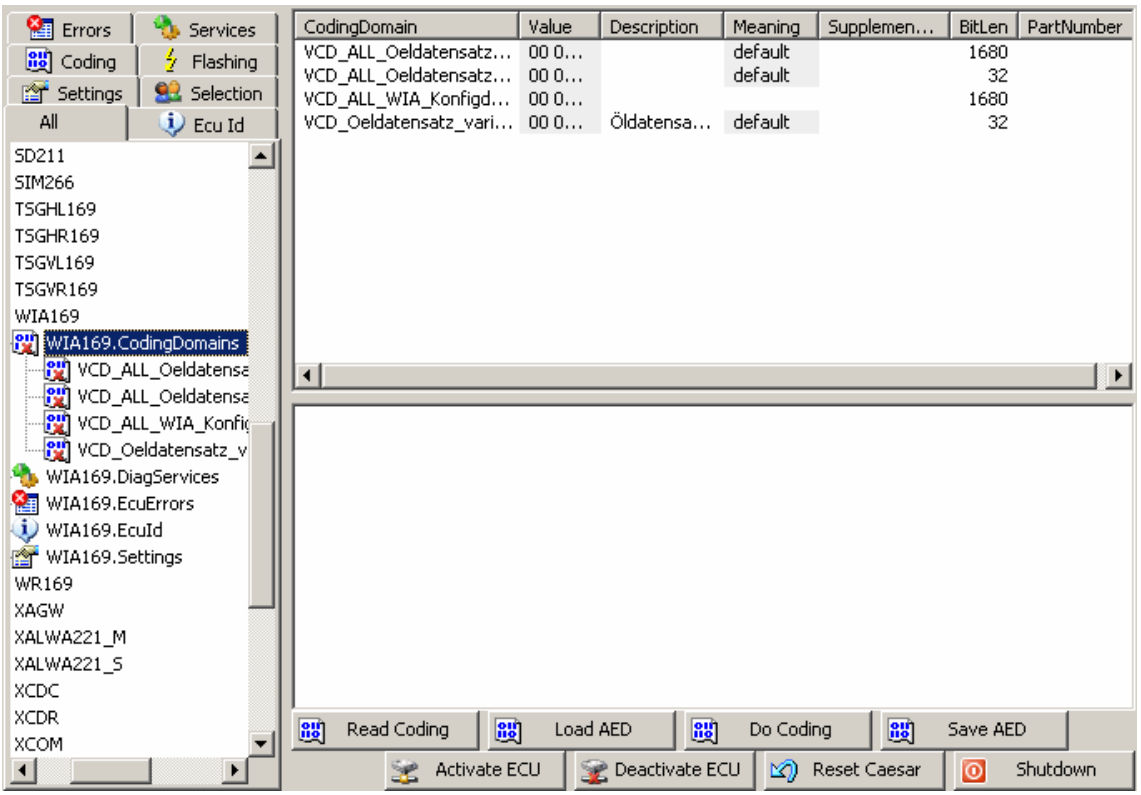


Figure 463 – Coding domains of an ECU

In the above figure, you can see in the data window an overview of the available coding domains of an ECU. Also visible is the corresponding code string , “Meaning” (optional), “PartNumber” (optional), “SupplementKey” (optional), “Description” (optional) and the length of the code string (BitLen) in Bit.

“Meaning”, “PartNumber” and “SupplementKey” are optional unique identifiers for predefined code string values. If available, you can set a special code string by selecting a “Meaning”, a “PartNumber” or a “SupplementKey” in the data window.

Alternatively you can edit a code string explicitly in the column “Value”. If you do so and enter one of the predefined code strings, the corresponding identifiers (“Meaning”, “PartNumber” and “SupplementKey”) are shown automatically.

Predefined code strings and their identifiers “Meaning” and “PartNumber” often come from an optional .ccf file (CAESAR Coding File).

Below is a brief tabular overview of the columns in the data window:

CodingDomain	Name of the corresponding coding domain
Value	Editable value of the code string
Meaning	Optional unique name for the value of a code string
PartNumber	Optional unique name for the value of a code string

SupplementKey	Optional unique name for the value of a code string
Description	Optional description of the coding domain
BitLen	Length of the code string in bit

With the button `Do Coding` in the command area, all changed code strings are sent to the ECU.

With the button `Read Coding`, you can read all coding domains of the selected ECU.

With the button `Load Aed`, you can read all coding domains of the selected ECU from the .aed file. See also chapter ***6.1.6.2 Loading and Saving .AED files***.

The button `Save Aed` saves all coding strings of the selected ECU to the .aed file. See also chapter ***6.1.6.2 Loading and Saving .AED files***. The path to save the files can be modified in the configuration dialog.

By selecting a coding domain in the diagnostics tree, the code string and its corresponding fragments will be shown in the data window. The fragments are predefined named parts of a code string. The location of a fragment in its code string is given by a bit position and a bit length (see ***Figure 464***).

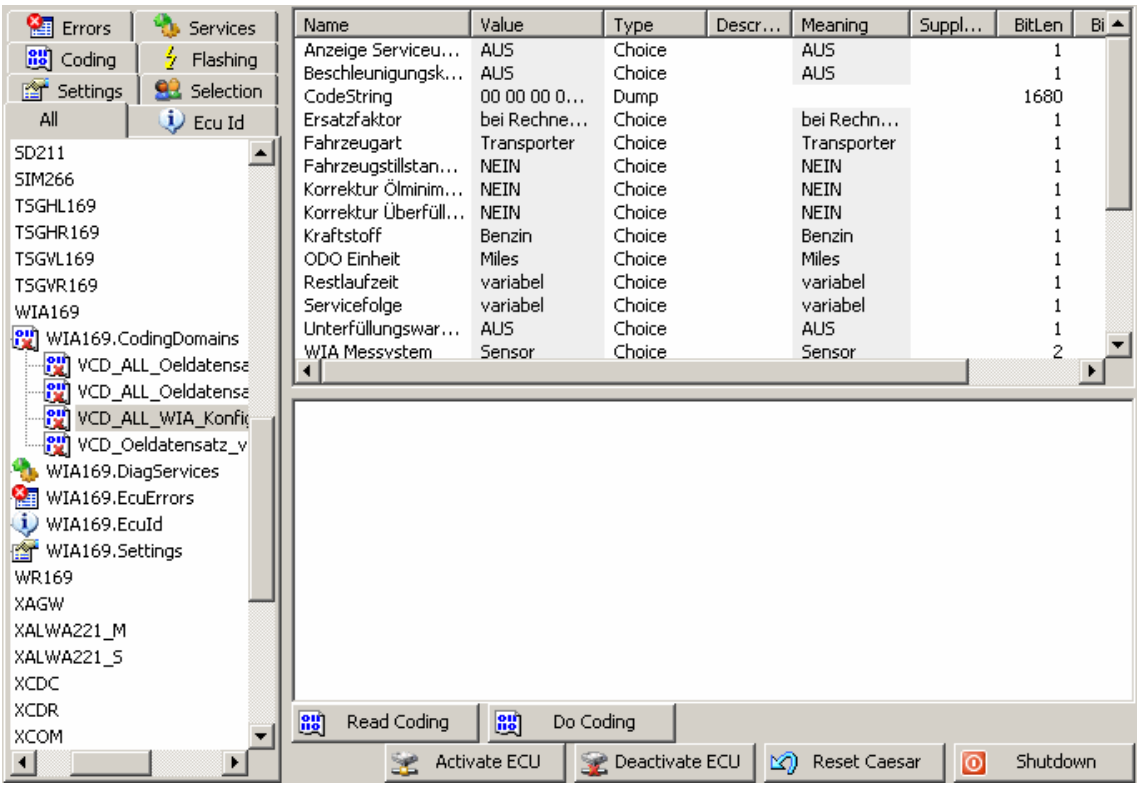


Figure 464 – Code string and fragments of a coding domain

With the exception of the columns “Type” and “BitPos” the function and meaning of the columns in the above window corresponds to those of ***Figure 463***.

The column “Type” shows the data type of the corresponding fragment. “BitPos” contains the start position of the fragment (in bit) inside its code string.

If “CodeString” is selected in diagnostics tree, an additional “SyncFlag” is displayed in the data window. When “Read Coding” took place already, codestring details in the data window are synchronized to the coding of the current ECU and the “SyncFlag” value is 1. Otherwise the “SyncFlag” value is 0.

Below is a brief tabular description of the data window columns:

Fragment/Codestring	Name of the corresponding fragment or code string as name of the complete code string of the selected coding domain
Fragmentvalue	Editable value of the fragment or code string
Type	Data type of the fragment value (see data types in the data window)
Meaning	Optional unique name for the value of a fragment or code string
PartNumber	Optional unique name for the value of a fragment or code string
SupplementKey	Optional unique name for the value of a fragment or code string
Description	Optional description of a fragment or code string
BitLen	Length of the fragment or code string in bit
BitPos	Start position of a fragment inside the corresponding code string



You can edit a fragment value by clicking on its value column in the data window of the diagnostics dialog. Depending on the data type of the fragment, an entry field or a combo box will appear (CHOICE data type).

With the button `Do Coding` in the command area, you can code the current coding domain. Changes you have made on the fragments or code string in the data window will be sent to the ECU.

With the button `Read Coding`, the code string of the currently selected coding domain will be read from the ECU and displayed in the data window. All fragment values are also updated.

With the button `Set Default` you can set the code string to the default value.

Changes made to a fragment value results in an update of the shown code string immediately after finishing the edit operation. Vice versa, a manipulation of the code string will immediately cause the change of one or more fragment values.

The changed code string must have the same length as the code string originally displayed. Please ensure that you only enter valid hexadecimal characters (the numbers 0..9 and the letters A, a, B, b, C, c, D, d, E, e, F, f).

6.1.6.2 Loading and Saving .AED files

When button Load AED is pressed PROVEtech:TA opens a dialog:

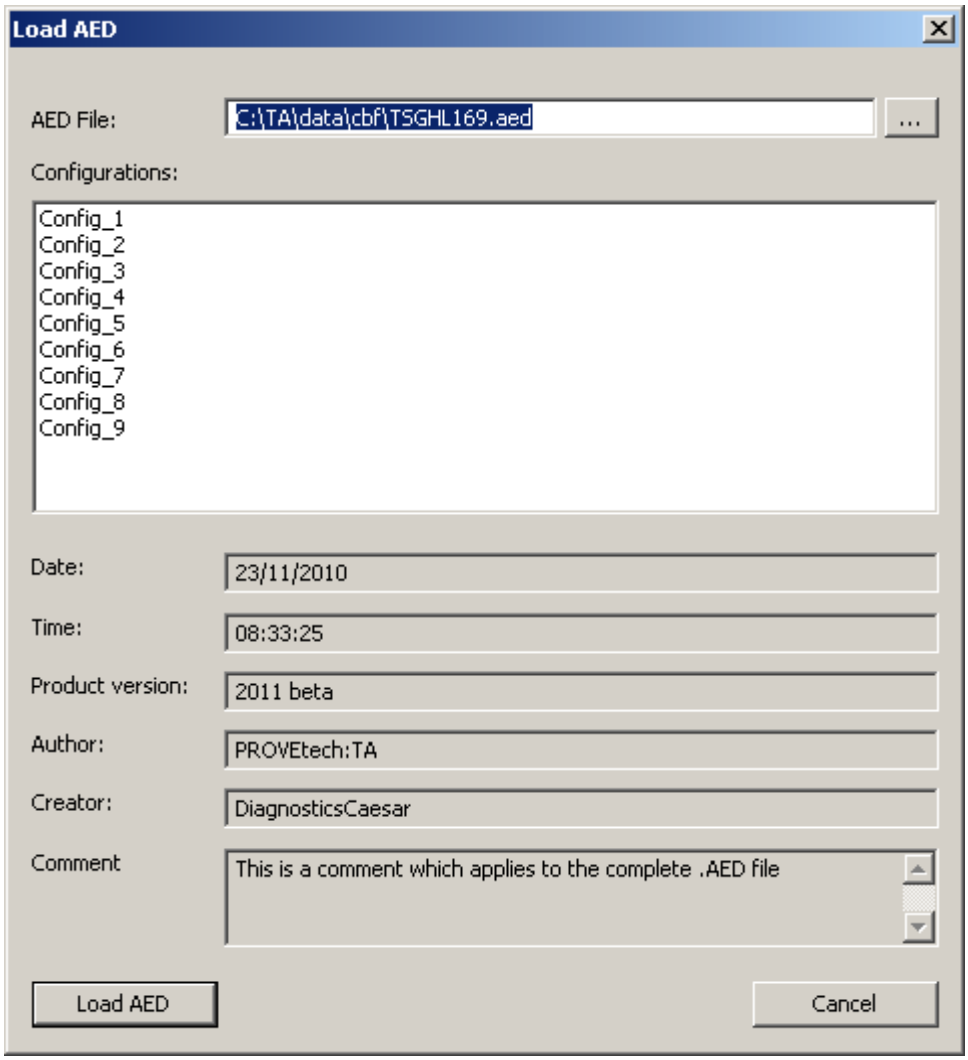


Figure 465 – Load AED dialog

In the upper area, the path to the .AED file and all stored configurations in the .AED file are displayed. When choosing an .AED file it is to mind that the .AED file corresponds with the selected ECU. It is also to mind that the configuration, which is chosen to be loaded, corresponds to both the ECU itself and the currently active variant on the ECU. In the lower area, information like the date created, time created, creator, author and the comment are shown. The lower area is valid for the complete file. It is not possible to store this information on a configuration basis.

Button Save AED opens a similar dialog to the Load AED dialog. In contrast, this dialog is capable of setting a new configuration name and of editing the comment field.

It should be kept in mind that the ecx format (successor format of the aed format) is not supported for Caesar Diagnositcs.

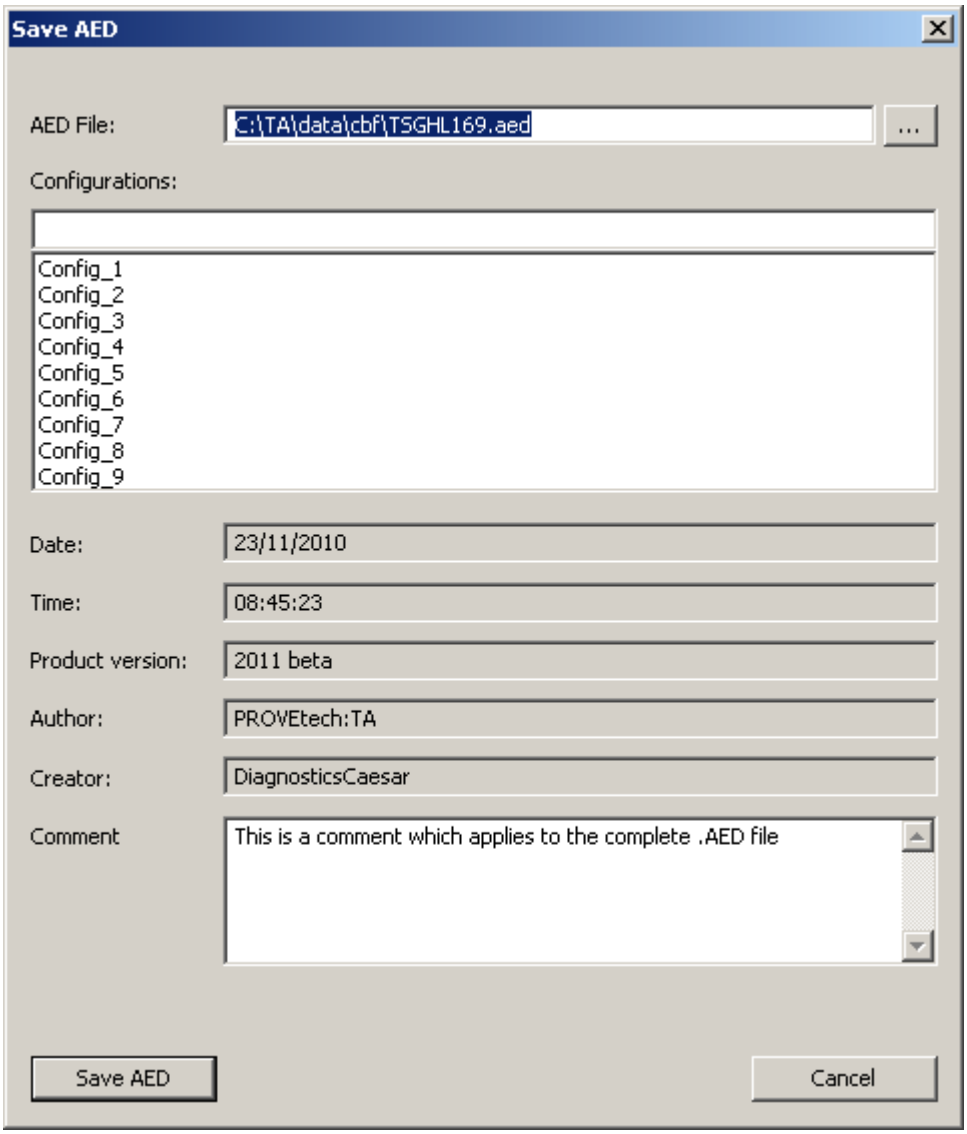


Figure 466 – Save AED dialog

6.1.7 Flashing an ECU

To flash an ECU, you can use the views `All` and `Flashing` of the diagnostics tree.

FlashAreas are logical groups of memory segments in the ECU.

Meanings are different options for one flash area. With those, you can choose different flashware versions.

The flash key is the part number which is unique to one meaning.

Using the tree view `All`, you must first select an ECU then the child node “FlashAreaList”.

Using the view `Flashing`, you only need to select an ECU.

After performing the selection, the data window of the diagnostics dialog displays a tabular list of so called flash areas. Each flash area contains one or more flash tables (also called “meaning”).

For each flash area, you can select one flash table (meaning) to be prepared for flashing. To do so, click on a field in the column “Flashtable (Meaning)” of the data window.

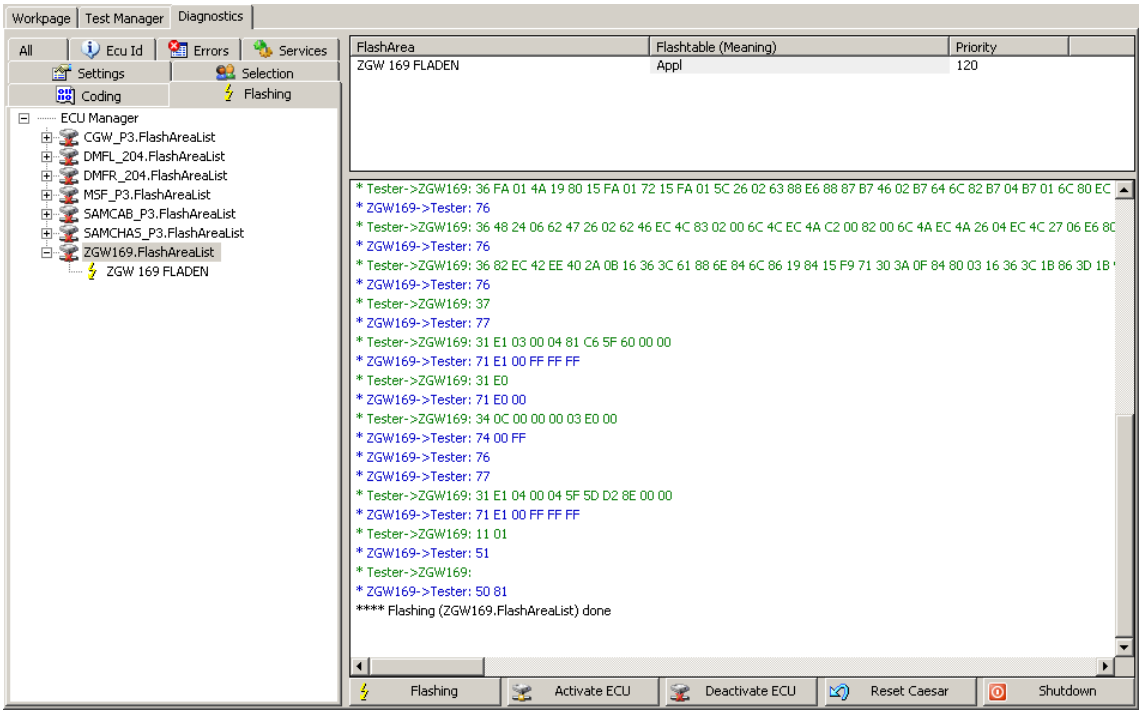


Figure 467 – Flash areas of an ECU

The flash operation is started via the button `Flashing` in the dynamic command area of the diagnostics dialog.

During the flash process, all of the selected flash tables (meanings) are transferred to the ECU. The sequence of transfer depends on the value in the “Priority” column of the data window.

Flash tables with a lower priority value will be flashed first.

6.1.7.1 Changing the priority of a flash table

To change the priority of a flash table (meaning), you must select the corresponding flash area (child node of the “FlashAreaList”) in the diagnostics tree.

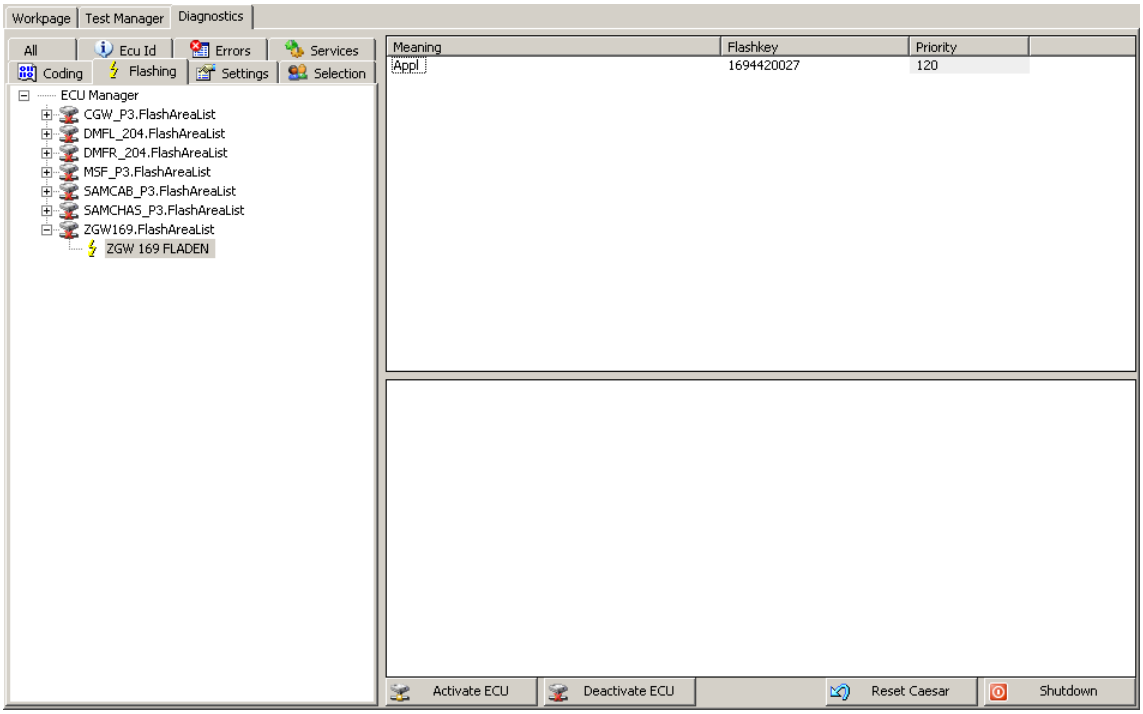


Figure 468 – Changing the Priority of a Flashtable

In the data window of the diagnostics dialog there is a list of the flash tables (meanings) belonging to the selected flash area.

You can edit the “Priority” value of a flash table by clicking on the corresponding field in the data window.

6.1.7.2 Flashing a single flash table

In addition to flashing an ECU you can also flash a single flash table (meaning).

To do this, simply select the meaning you want to flash in the data window.

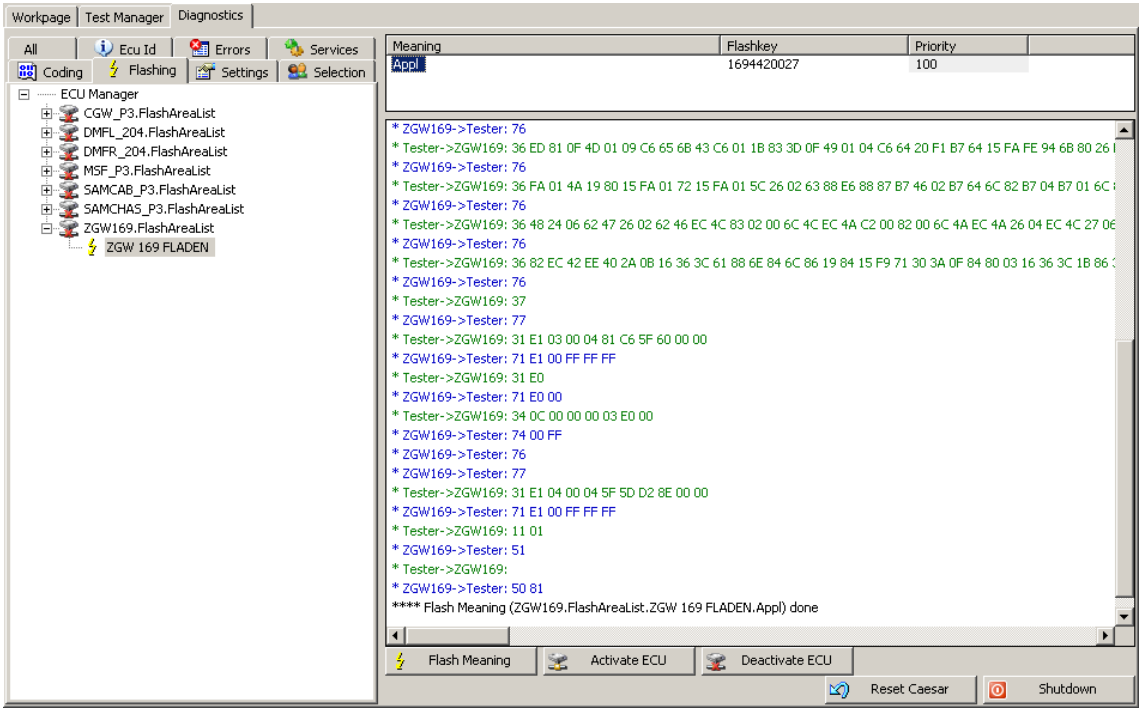


Figure 469 – Flashing a single flash table

In the command line the button `Flash Meaning` appears. By pressing this button, the selected flash table (meaning) will be sent to the ECU.

6.1.8 Variants of an ECU

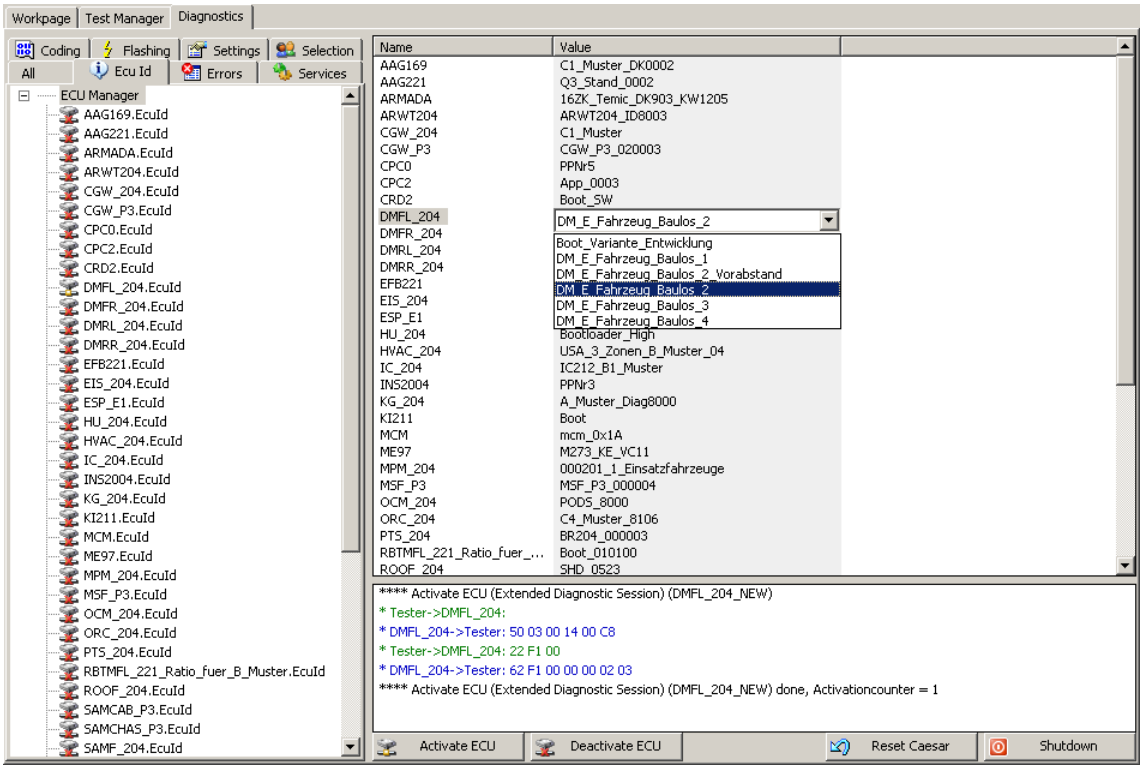


Figure 470 – Viewing or changing the variant of an ECU

To check or change the current variant setting of an ECU, select the “ECU Manager” object in the tree view `All` of the diagnostics dialog.

In the data window on the right side appears a list of the (registered) ECUs. The left column contains the CAESAR names (Qualifier) of the ECUs, the right column shows the name of the currently selected variants.

To change the variant settings, just click on a variant name in the ECU list of the data window. A combo box appears, containing all variant names available in the .cbf file of the corresponding ECU.

If you change the variant setting of an ECU, the complete subtree of this ECU will be rebuilt. This is because normally the number, types and names of the diagnostic objects contained in different variants are not identical.

You also can change the variant with the “Settings” node of the ECU. The way in which this is done is described in chapter ***6.1.11 ECU settings***. In the same chapter, you will find a description of how to activate an ECU even if the correct variant is not available in the .cbf file.

The change of an ECU variant only remains in effect as long as the ECU stays in a deactivated state. As soon as the ECU is activated, via the diagnostics dialog or in a running test program, PROVEtech:TA automatically switches to the variant requested by the ECU.

On some commands, the automatic variant switching cannot be completed in the execution time of the command. In this case, PROVEtech:TA displays an error message and completes the variant switching during the next command, which activates the ECU.

If necessary, you can disable the automatic variant switching completely. Details for this are described in chapter ***6.1.11 ECU settings.***

6.1.9 Communication parameters

Each ECU has communication parameters, as an example containing timeout values. So also CAESAR needs to know communication parameters as a counterpart. To show these parameters in the diagnostics tree, select the tree view `All`.

The communication parameters are only visible in expert mode. With the context menu (right mouse click onto a diagnostic object in the tree) of a diagnostic object in the tree, you can toggle the expert mode on or off.

If you are in expert mode, you will find the “ComParams” node in the tree directly under the corresponding ECU.

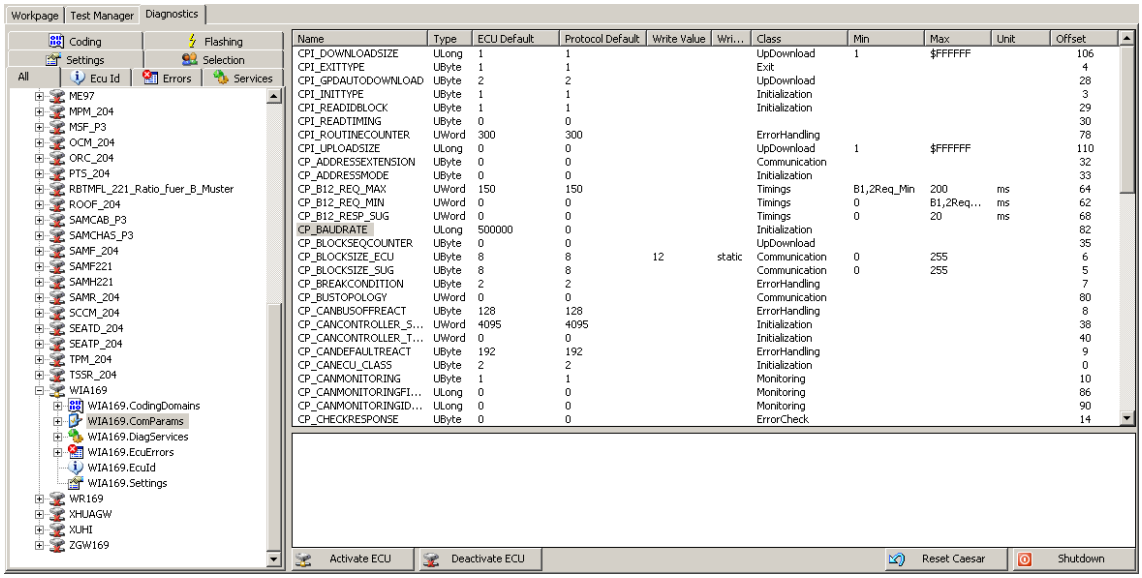


Figure 471 – Communication parameters in diagnostics dialog

By selecting the node “ComParams” in the diagnostics tree, you get a list of the available communication parameters in the data window on the right. Each row represents a

single communication parameter, which is characterized by a set of attributes in the different columns. Below is a brief description of the different columns:

Name	Parameter name
Type	Data type of the parameter
ECU Default	ECU specific default value
Protocol Default	Protocol specific default value
Write Value	Empty or contains a value which is written when the ECU will be activated
Write Option	Buffer to which the above value will be written
Class	Parameter class name
Min	Minimum value of the parameter
Max	Maximum value of the parameter
Unit	Parameter unit
Offset	Parameter offset in CAESAR buffer

The values of the communication parameters are held by CAESAR in two memory regions with different life-spans. The static memory region holds the values during the activation period of an ECU, while the temporary memory region is reset after the next action.

Edit a communication parameter value (ECU not activated)

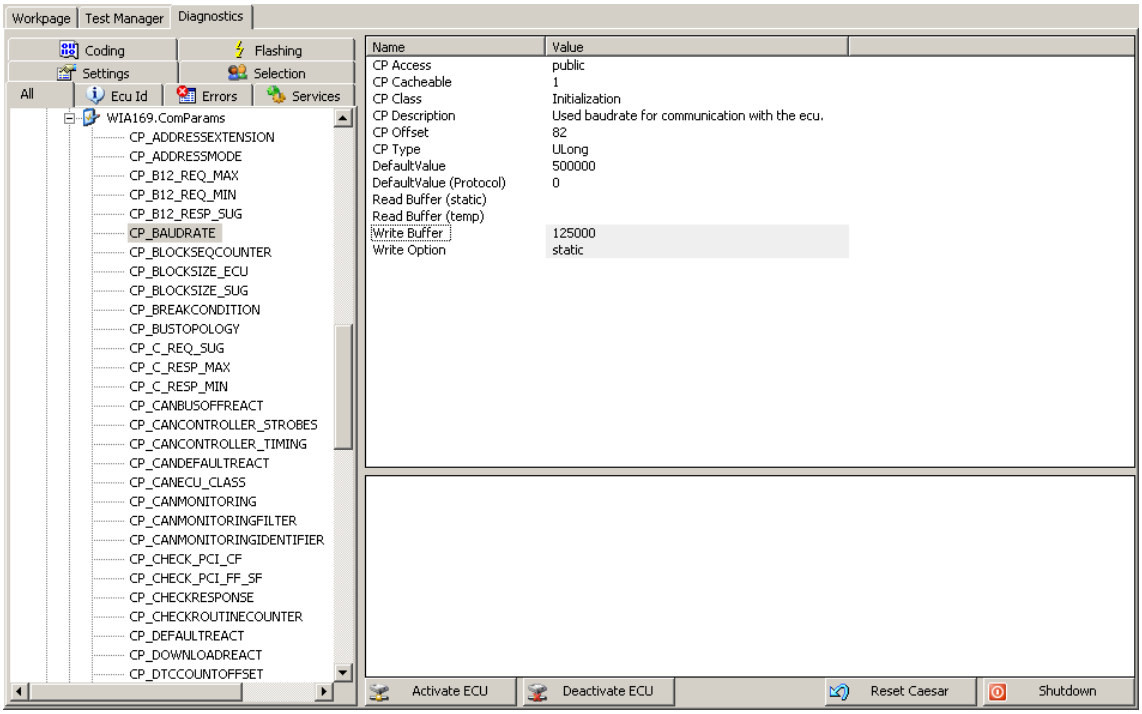


Figure 472 – Edit the value of a communication parameter

To edit the value of a single communication parameter, simply select it in the diagnostics tree. The data window on the right will show a number of attributes, two of which are the fields “Write Buffer” and “Write Option”.

In the field “Write Buffer” you can enter a new parameter value, which will be set automatically the next time the ECU is activated. After the parameter is set, the field “Work Buffer” will be cleared again.

The field “Write Option” allows you to select the internal buffer, to which the value will be written. Possible selections are “static” or “temp”.

Directly read or write a communication parameter when ECU is activated

When the ECU is activated, two additional buttons appear in the dynamic command area. They can be used to write a new parameter value directly or to read the current values of the two internal buffers (fields “Read Buffer (static)” and “Read Buffer (temp)”).

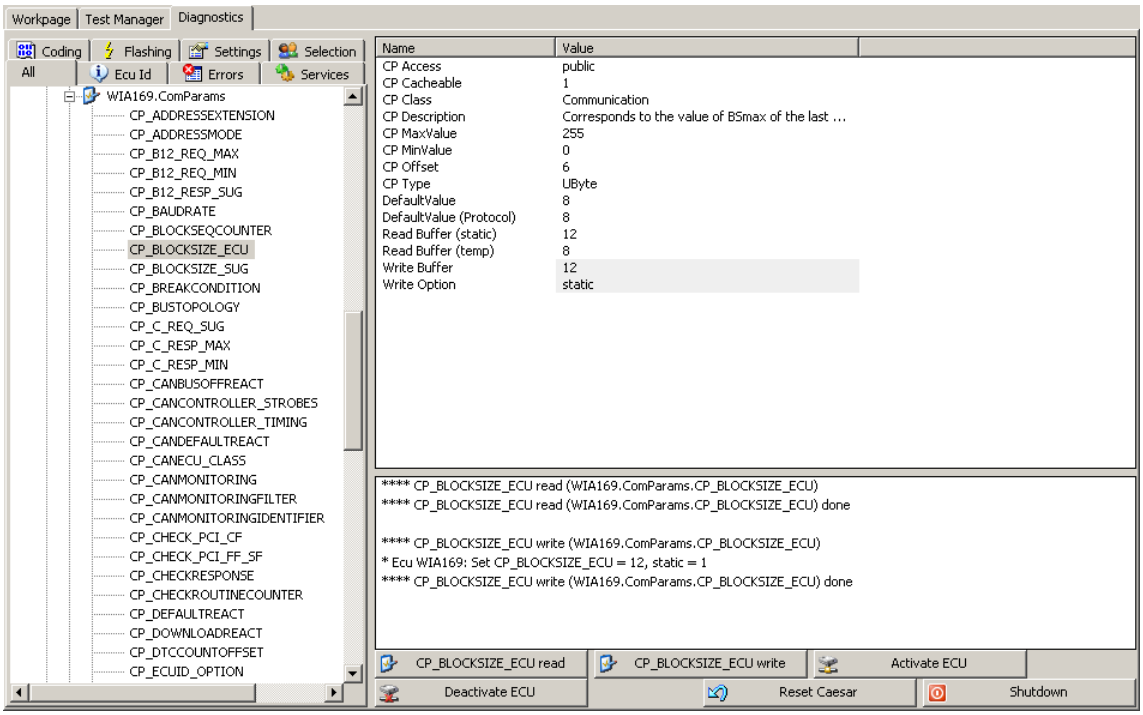


Figure 473 – Directly read/write parameter value

The type and number of communication parameters of an ECU is protocol dependant. You can find a description of the communication parameters and their values in the corresponding protocol specification.

Changes to communication parameter values should be done only by experts. Wrong settings can threaten the stability of CAESAR and PROVEtech:TA respectively and lead to a program crash.

6.1.10 Extended and default diagnostics session

By default, PROVEtech:TA uses a so called extended diagnostics session for communication with an ECU. In such a session, the complete diagnostics functionality of an ECU is available, especially you can execute write operations like variant coding.

PROVEtech:TA also supports the so called default diagnostics session, in which only a reduced set of diagnostics functionality is available.

PROVEtech:TA has no knowledge, which diagnostics functions are available in a default session and which are not. If you plan to use the default diagnostics session, you should consult the protocol and ECU specification, to learn which functionality is available in a default session.

The default diagnostic session should only be used by advanced users. If you have any doubts, use an extended diagnostic session.

6.1.10.1 Switching the diagnostic session type

With the context menu of a diagnostic object, you can switch the session type of the corresponding ECU. The context menu also shows the currently selected session type.

The diagnostic session type can be switched before an ECU is activated or while it is activated. With an activated ECU, the session type is switched immediately, in the other case the selected session type is used for the next activation of the ECU.

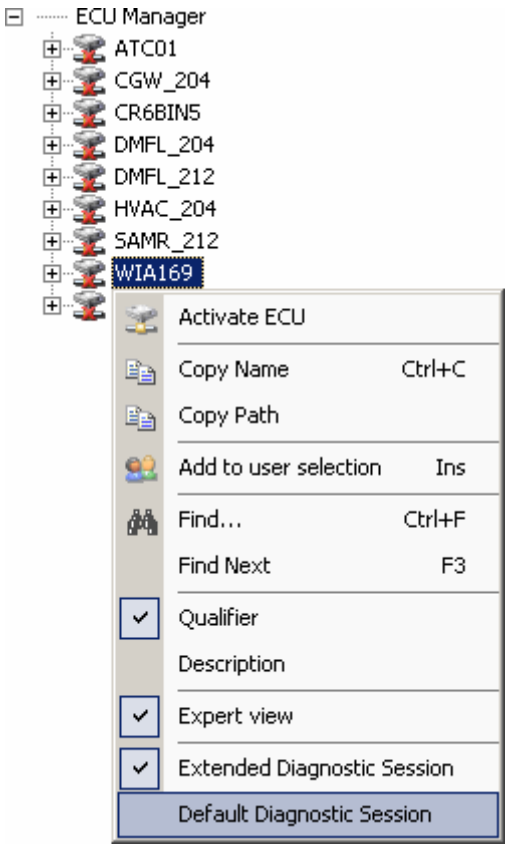


Figure 474 – switching the diagnostics session

6.1.11 ECU settings

For each ECU there are several settings, which will be stored persistently in an initialization file. These settings may be directly edited from the diagnostics dialog.

To edit these settings, select an ECU in the tree view `All` and then the sub node “Settings”. Alternatively, you can go to the tree view `Settings` and just select the desired ECU.

In the data window on the right, appears a list of parameters, which can be adjusted for the selected ECU.

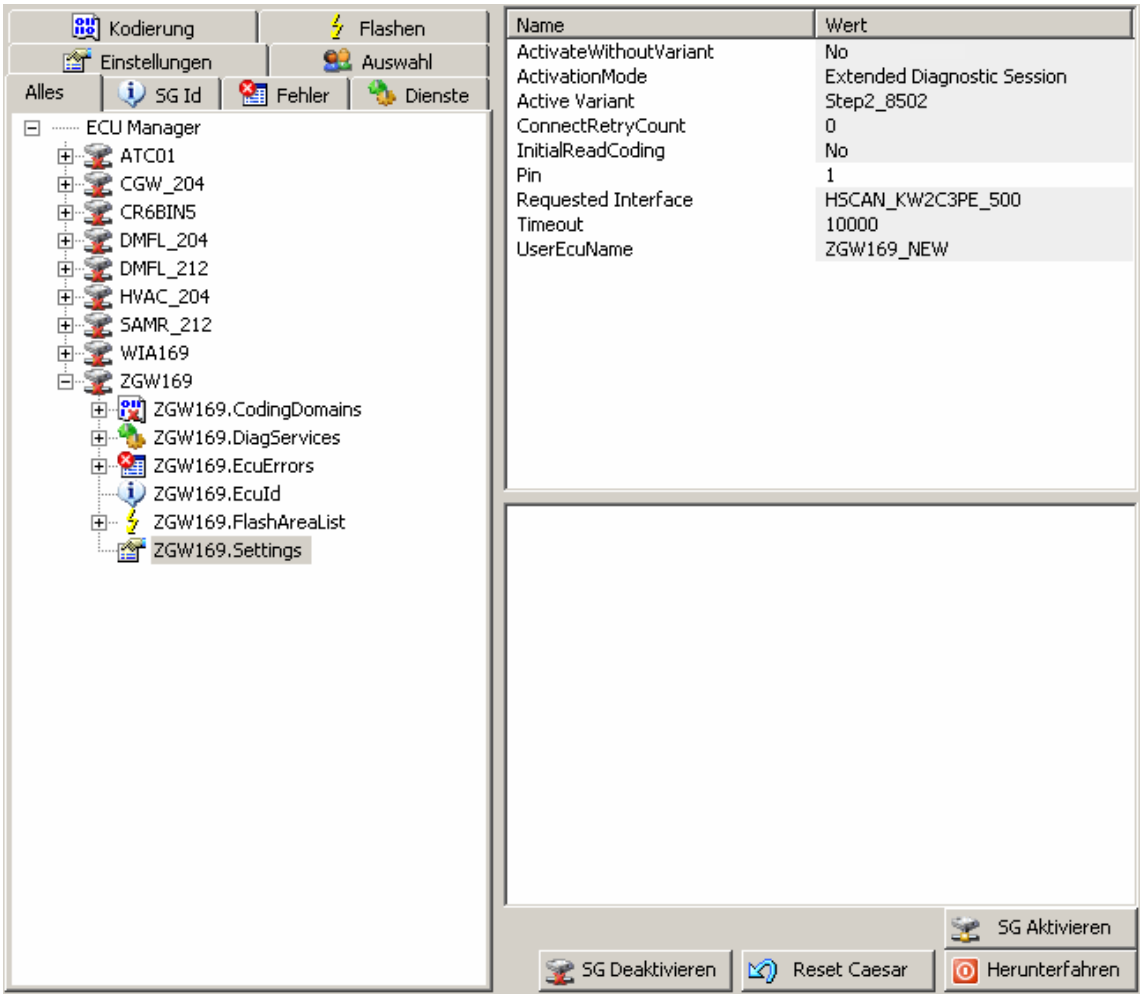


Figure 475 – ECU settings

A brief description of the currently available parameters and their possible values follows:

ActivationMode

The chapter ***6.1.10 Extended and default diagnostics session*** describes this setting.

ActivateWithoutVariant and Active Variant

These settings define which variant (see chapter ***6.1.8 Variants of an ECU***) of the according .cbf file to use for communicating with the ECU. The variants can differ a lot and when using the wrong variant, the communication with the ECU is invalid.

When connecting an ECU, CAESAR gets information about the active ECU variant. The following matrix shows how the two settings influence the ongoing behavior:

	ActivateWithoutVariant		
	No	**Yes, in case automatic detection fails**	**Yes, override automatic detection**
ECU variant available in .cbf file	ECU variant overwrites “Active Variant” and the communication works with this variant	ECU variant overwrites “Active Variant” and the communication works with this variant	The communication works with the setting “Active Variant”
ECU variant not available in .cbf file	Communication cancelled	The communication works with the setting “Active Variant”	The communication works with the setting “Active Variant”

By default, “ActivateWithoutVariant” should stay on “No” and “Active Variant” gets detected automatically. Before selecting a different setting please try to get a correct .cbf file.



The settings “Yes, in case automatic detection fails” and “Yes, override automatic detection” can threaten the stability of PROVEtech:TA and lead to a program crash.

You should use this setting only with greatest care and in situations where a PROVEtech:TA crash is a tolerable.

ConnectRetryCount

This item defines how often an ECU activation can be retried after the first attempt fails. Before each retry attempt there is a wait time of 500ms. The default is “0” (no retry).

A value > 0 is practical in cases where activation often fails, for example, in the event of hardware problems. Values that are too large can negatively influence speed (of your test program for example).

InitialReadCoding

With this switch you can establish whether the variant coding of the ECU is automatically read on the first attempt to access a coding domain. Possible selections are:

- Yes
- No

If you want to read coding defaults from CBF file you need to set this switch to “Do not read VarCoding automatically”. After restarting PROVEtech:TA the default variant coding as defined in CBF is displayed.



The default setting “Yes” causes an automatic activation of the ECU on the first attempt to access a coding domain. This can happen indirectly by a test program or by selection of a coding domain in the diagnostics tree.

If you want to prevent the automatic activation, select the option “No”.

Pin

The meaning of the value which can be set here depends on your hardware.

- If you use CAN hardware then the correct value is set automatically depending on the selected interface (see next setting “Requested Interface”). In this case, you cannot edit this parameter and the value is “1” for HSCAN and “2” for LSCAN.
- If you use KLINE hardware (for example with Part C) then you must manually set the correct value. The value corresponds to the physical number of the used KLINE connector.
- When using the eCOM-Box it is to mind, that there are two HSCAN ports. Depending on whether the ECU is connected to CAN 1 or CAN 2 the value “1” or the value “2” is correct for the pin setting.

Requested Interface

This item defines the diagnostics hardware, the protocol, and the baud rate. The available “interfaces” are defined in the corresponding .cbf file and will be displayed in the column “Value” in a combo box.

Timeout

This is the timeout value in milliseconds for the communication between the CAESAR master and slave components. Default value: 10000ms.



CAUTION: A too small timeout value can cause a crash of CAESAR and PROVEtech:TA. The recommended value is 10000ms.

UserEcuName

With this item you can specify a user defined name for the ECU. This name will be used in the test language commands in order to access the ECU. By default, the “UserEcuName” is set to the CAESAR name.

UseJ1939

This setting allows to interpret UDS protocol data as J1939 protocol data. It works solely for ECUs that according to their CBF use the UDS protocol. After adjusting this setting CASAR should be restarted. Possible selections are:

- Yes
- No

The default value is No.

6.2 MCD-3 D diagnostics

The MCD-3 D diagnostic within PROVEtech:TA is the module, that is based on the ASAM standard of the same name. It realizes the diagnostics with MCD-3 D servers in PROVEtech:TA. At this moment the MCD-3 D servers of DSA (including the special version VW kernel) and Softing are supported. The MCD-3 D module implements the application layer and abstracts the used diagnostic server. This means, that the GUI and the test language in PROVEtech:TA are always identical for the MCD-3 D diagnostic independent of the chosen server.

In this chapter you will find out:

- how the diagnostic window is composed
- which elements in the diagnostics exist
- how to parameterize services and jobs
- how to execute services and jobs
- which automations exist
- where the responses are find
- how to code and flash your ECUs
- how to read and clean the DTCs



A precondition for using the PROVEtech:TA MCD-3 D diagnostics module is a license and installation of at least one of the supported MCD-3 D servers. Even when using a PROVEtech:TA desk license without a real ECU PROVEtech:TA needs a MCD-3 D server to access the ECU data.

6.2.1 Basics

To use the MCD-3 D module in PROVEtech:TA it is necessary to have one of the supported MCD-3 D servers installed on your system. The functionality of MCD-3 D diagnostics is based on the data of the control unit description file.

Please see chapter ***8.10.1 Configuring MCD-3 D servers*** about installing and configuring MCD-3 D servers.

Functionalities, which rely on reading the database of the control unit description file, are available for all installations of PROVEtech:TA.

Functionalities, which rely on interaction with the ECU, are available for installations which include hardware access.

Database and communication component

The diagnostics consists of two components, a database and a communication component.

The **database component** reflects the data of the control unit description file as far as this data is available via the MCD-3 D API. PROVEtech:TA never directly accesses the data but always uses the MCD-3 D server API to get this information.

For MCD-3 D diagnostics, control unit description files are usually ODX files, i.e. description files according to ASAM Data Description Interface MCD-2 D. The ODX files are either packed in a PDX or are stored as separate files (ODX-D, ODX-C, etc.). These files have to be converted into a runtime format of the chosen MCD-3 D server with the transformer. The specific transformers are provided by the publishers of the MCD-3 D server.

These data include information about all available services and jobs, the service parameters and the job parameter as well as the possible responses and the possible response parameters.

The data for the available jobs are also included. There are JAR files in the PDX (see chapters ***6.2.2.8 Packaged ODX (PDX)*** and ***6.2.2.9 Java Archive (JAR)***), which are included in the runtime format by the transformer. These jobs are later found in the PROVEtech:TA diagnostic module beneath the tree element `Jobs` (see chapter ***6.2.3.12 Services and jobs***).

The control unit description files also include a list of the error codes which the ECU can send, and a list of the communication parameters. PROVEtech:TA only shows the data from the ODX files and has no influence on the content. For the content of the files the supplier of the data is responsible. PROVEtech:TA has some settings to facilitate the handling for the user (see chapter ***6.2.3.8 The ECU element***).

All these information can be stored for different ECU variants. More detailed information you will find in the ASAM MCD-2 D standard, which describes the construction and content of the ODX files.

These data can be browsed on PROVEtech:TA diagnostics MCD-3 D panel.

The **communication component** reflects interaction with the ECU via a data bus (e.g. CAN). Generally this means actual communication with the ECU. One exception is the simulation mode of the MCD-3 D server (if offered), that can be enabled in the configuration manager of the MCD-3 D server, to simulate actual hardware and to guarantee a minimum of functionality. It should be minded that generally there are counterparts in the database and the runtime components. Unfortunately they do not exist in every case and that is why we need the simulation mode. To get to data that is not offered by the database objects. Otherwise not all functionality is guaranteed in PROVEtech:TA (e.g. the coding).

All MCD-3 D interaction with an ECU takes place via service or job execution. This means, that for instance the actions read or clear ECU errors are done by executing the respective service. With proper ODX files PROVEtech:TA offers a special view for the work with coding and DTCs (see chapters ***6.2.3.9 The coding*** and ***6.2.3.10 The DTCs***).

The following diagram shows the simplified diagnostics setup:

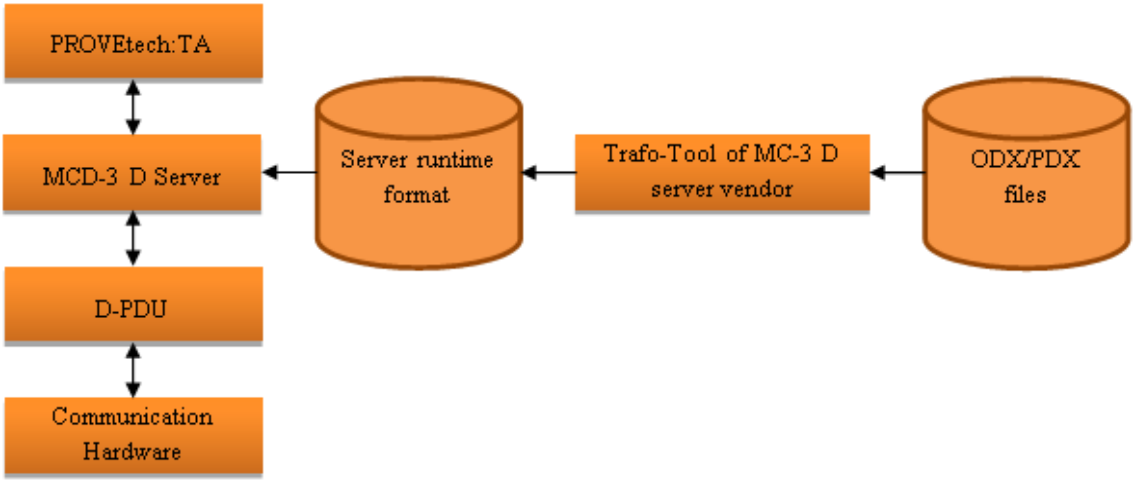


Figure 476 – Diagram of simplified diagnostics setup

6.2.2 Terms in the MCD-3 D diagnostics

To avoid misunderstandings and to make clarity about the terms used in this chapter here the most important terms are explained.

6.2.2.1 Measurement Calibration Diagnostics (MCD)

The often used abbreviation MCD stands for Measurement Calibration Diagnostics. These are the three kinds to work with an ECU. Measurement is used for measuring the signals of an ECU. Calibration means to write data into the ECU. This is also used to manipulate the ECU. Diagnostics is the part that describes the diagnostic functionalities. Although these terms are always used together the focus of this chapter is on the Diagnostics.

6.2.2.2 MCD-3 D

The ASAM standard MCD-3 D describes a diagnostic server, its functionalities and its API. The diagnostic module described in this chapter is named after this in PROVEtech:TA, because it is based on the general interface and composes the application layer above the MCD-3 D server.

6.2.2.3 D-PDU API

The D-PDU API is the communication layer between the MCD-3 D server and the MVCI. When sending a diagnostic message the D-PDU prepares it for the corresponding field bus (e.g. CAN or FlexRay, Ethernet). In receiving direction the diagnostic message is prepared for the MCD-3 D server. Communication errors like time outs are recognized in this layer.

The D-PDU API defines a diagnostic interface (sometimes two, e.g. a real CAN interface and a VirtualCAN interface). An interface has several interface resources, representing a specific access to the ECU using a defined protocol and channel, e.g. UDS_on_CAN2.

6.2.2.4 Modular Vehicle Communication Interface (MVCI)

MVCI stands for Modular Vehicle Communication Interface and represents the hardware for the diagnostic communication. Depending on the MVCI device there are different modules for the diagnostic communication, for example K-Line, CAN, Ethernet (for Diagnostics over IP – DoIP).

6.2.2.5 Simulating diagnostic communication

For programming of tests it might be helpful to execute services and jobs to get real responses.

For this purpose, the Softing MCD-3 D server DTS has the possibility to record and simulate real communication.

To do this, first you need a PC with hardware and a connection to the ECU. Use SystemConfiguration, a Softing program delivered together with DTS8, to activate the recording of simulation data. Then start a diagnostic session and execute all relevant jobs and services. The recorded simulation file must then be included into the same MCD-3D project on the PC without hardware (also using SystemConfiguration). On this PC also a simulation interface must be included and activated.

If you have a PROVEtech:TA desk license (without hardware usage), you can use the simulation interface of DTS anyway, if one more condition is met. The system property **com.softing.ShowPreparedAsConnectedInterfaces** must be set to **TRUE.** Use the SystemConfiguration in “Expert Mode” or add the following line to the DtsConfig.xml file in the SystemProperties part:

```
<SystemProperty ShortName="com.softing.ShowPreparedAsConnectedInterfaces"
Value="TRUE"/>
```

More information is available in the DTS documentation or from the Softing support.

Hint: take a look at the simulation file. The format is a quite clear XML structure. You can edit the request/response pairs to simulate responses which could not be recorded. By using the “repetitioncount” with values >1 you can simulate different answers to the same requests. Especially it might be useful to check the reaction of the system to negative responses (7F messages)

6.2.2.6 DoIP – Diagnosis over IP

ECUs can be contacted via different media. Years ago, the K-line has been replaced by CAN bus. Now the number of ECUs with an Ethernet port is increasing.

The MCD-3 D server needs an additional software layer for every used kind of communication. This so-called D-PDU API software provides one (or more) interface used by the MCD-3 D server to exchange standard messages (e.g. UDS) with the ECU. You find more details about the software architecture in chapter ***8.10.1 Configuring MCD-3 D servers***.

For DoIP a D-PDU API is needed which provides a Ethernet interface. PROVEtech:TA has been successfully tested with software by Berner&Mattner (GDPM 5.2) , I+ME Actia (GDPM 5.3) and Bosch (Ver. 5.22-5.70). The interface has to be listed in the RDF file used by your MCD-3 D server. The server must be configured appropriately. The D-PDU API software has also its own configuration files – please consider the original documentation.

Typically the Ethernet chip of the ECU must be switched on by a separate signal. The ActivationLine (a.k.a. Wakeup) must be connected to 6-15V. If a eCOM CAN hardware is used and properly wired, it can be used for automatically switching the ActivationLine. It can also be switched manually or by a special signal controlling the HiL hardware. It can also be permanently switched on, but in this case the ECU might not enter the sleep mode.

The following chapters apply to DTS8 MCD-3 D server. DoIP has not been used with DSA server yet.

6.2.2.6.1 Standard DoIP configuration

The Ethernet interface can be activated and configured with the Softing SystemConfigurator, which is used for MCD-3 D server setup.

Please keep in mind that any configuration fits only a special combination of DTS and D-PDU API version. Some ECUs can be handled only with special combinations of software versions.

When PROVEtech:TA loads a configuration and the attached MCD-3 D project gets loaded, the Ethernet interface gets activated. After selecting a Ethernet LogicalLink and clicking `Activate`, a network communication will be established.

6.2.2.6.2 Manually configuring DoIP

Some MBC and Daimler ECUs do not follow the ISO 13400 standard for DoIP. To handle them, a special version of D-PDU API is needed (Bosch 5.28 and newer).

The behavior of DTS differs in different versions. Also the D-PDU API act differently. Therefore it might be necessary to follow another approach and get more influence on the communication parameters. You can manually configure the DoIP communication (see chapter ***3.2.7.8 MCD-3 D diagnostics settings***) and set up the following items:

- OptionString – this string starts a D-PDU API procedure for ECU detection. The format of the string is described in the MCD-3 D documentation, but it is often not followed by the D-PDU API software. The single parameters vary depending on software version, ECU and topology.
- Name fragment for interface identification – the D-PDU API follow very inhomogenous naming conventions for the LongName. If the system contains more than one Ethernet interface, enter here a substring the proper interface might be identified by.

In this case the interface will not be activated at loading the MCD-3 D project (after loading PROVEtech:TA configuration), but directly before the 1st activation of an ECU via an Ethernet LogicalLink. The program performs the following steps:

- Getting the list of all known interfaces. You can see the list of interfaces in the trace window (and in the trace file). Some D-PDU API (Bosch 5.26 and newer) provide at this step one or more Ethernet interfaces, although they are not active yet.
- Switching the Wakeup signal (ActivationLine) via eCOM, if available.
- Starting to detect ECUs connected to the network (=”DoIP-Entities”) by calling DetectInterfaces with the OptionString as argument. From that moment the software starts sending messages over Ethernet and analyzes the ECU answers.
- Several loops to get the interface list and check if the wanted Ethernet interface is activated. This can last several seconds.
- Establishing the ECU contact using the activated interface

For every following activation of an ECU using a Ethernet LogicalLink the same interface will be used. The initialization procedure will not be repeated any more, except the ECU has been attached or switched on after the initialization. In this case a reinitialization with the Basic function *Dgn.ReinitDoIP* may be necessary.

If contact to the ECU gets lost and cannot be established any more, only a restart of the module can help (see ***8.10.3.1.6 Loosing contact with ECU***).

6.2.2.6.3 Hints for DoIP configuration

- The D-PDU API by Berner&Matner and I+ME Actia often have problems with MBC/Daimler ECUs
- We recommend using a RDF file containing only one interface for CAN and one for Ethernet. Several alternative D-PDU API at a time may interfere and it is not always clear which one is used.
- The special procedure followed by those ECUs (not following the ISO 13400 standard) can be fulfilled only by the DTS version 8.13 (or newer) and the D-PDU API by Bosch (5.26 or newer). Please ask the ECU developer which procedure - ISO or Daimler – has to be followed, and configure the system as needed. Consider also the file D_PDU_API_CFG.xml, a configuration file of the D-PDU API.
- During the initialization you find a lot of useful information in the trace file, e.g. you can see if the OptionString has been accepted by the interface and if the interface is properly identified.
- It is recommended to check the data traffic on the Ethernet. This can be done by Wireshark® or similar network analysis tools. The least you should see is an increased traffic in the initialization phase.
- For the latter reason it is recommended to attach the ECU to a network port without any other traffic.
- Using a USB2Lan adapter should be possible, but until now many problems have been observed (no contact or even system crash), not occurring with a direct Ethernet connection.
- The OptionString must be individually composed and tested for every combination of ECU, DTS and D-PDU API version.

This is an example for a OptionString, successfully tested with DTS 8.12 and Bosch 5.28 (all params in one line):

PDU_IOCTL='PDU_IOCTL_VEHICLE_ID_REQUEST'
CombinationMode='DoIP-Collection'
PreselectionMode=0
PreselectionValue=''
VehicleDiscoveryTime=2000
NumDestAddrs=0

If not successful, first the following values could be changed:

PDU_IOCTL='PDU_IOCTL_MS_VIR_V1'

and:

CombinationMode='DoIP-Entity'

Attention! Especially a wrong valule for CombinationMode might cause problems. The value needed by Bosch 5.28: **'DoIP-Collection'** causes Bosch 5.24 to crash. **'DoIP-Entity'**, working with Bosch 5.24, leads to an error message when used with Bosch 5.28.

6.2.2.7 Open Diagnostic Data Exchange (ODX)

The ODX format is specified in the ASAM standard MCD-2 D. In the ODX files all relevant ECU data is stored to offer a human readable possibility to interpret diagnostic messages. Within the files the linking between the bytes sent by the ECU and the texts are established. You can think as if the ECU is speaking its own language, i.e. bytes. We humans can hardly understand this language. Therefore we need a dictionary. In the diagnostic world the dictionary is the database and in this special case the dictionary is written in the ODX format.

6.2.2.8 Packaged ODX (PDX)

The PDX is a packaged bundle of all ODX files. Generally it is possible to open such files with 7-zip.

6.2.2.9 Java Archive (JAR)

JAR files are used in the MCD-3 D diagnostics to pack jobs that are programmed in Java and are referenced from the ODX file. These jobs are also part of the PDX. In PROVEtech:TA these jobs can be executed.

6.2.2.10 Jobs In Diagnostics

A job is a diagnostic procedure programmed in Java. It is completely free programmable and it is packed into a JAR file, which is part of the PDX. A job can be executed just like a service and it also provides a structured and interpreted result. In contrast to a service (see ***6.2.2.11 Service In Diagnostics***), during a job execution there might be more than one request and response message to and from the ECU involved.

A job can have parameters which change the byte values of the request message or influence the interpreting of the response message(s).

6.2.2.11 Service In Diagnostics

A service contains a request message to be sent to the ECU and a procedure how to interpret the response message. The result contains a structure with the interpretation of the received message. A service can have parameters which are translated to special values of the request message.

6.2.2.12 Semantic As Attribute

In diagnostics a semantic is an attribute of a service or a job. These attribute categorizes the services and jobs according to their functionality. In PROVEtech:TA semantics are used to group all services or jobs with the same semantic.

6.2.2.13 Vehicle Information

The vehicle information describes the vehicle topology thru which the MCD-3 D diagnostic test system accesses the vehicle and the ECUs. The vehicle information is selected within PROVEtech:TA within configuration. To select different vehicle information, you need to change your configuration.

6.2.2.14 Logical Link

A logical link is the representation of one access path to a protocol which is used for communication with one ECU, for communication with a functional group of ECUs or for communication with an ECU base variant. Thus this also reflects how the ECU is built into the vehicle topology. For ECUs capable of communication via different ways (e.g. CAN and Ethernet), the logical link is to be selected in the request view of the ECU (see ***6.2.3.4.1.1 ECU***).

6.2.2.14.1 Special Case Diag Over IP (DoIP)

If a LogicalLink of type Ethernet is selected, the communication with the ECU runs via Ethernet. If the Ethernet interface is manually configured (see also ***6.2.2.6 DoIP – Diagnosis over IP***), the Ethernet chip of the ECU must be switched on (ActivationLine, a.k.a. wakeup signal) and the interface must be activated first. This can take some time (up to 15-20 seconds). The ActivationLine can be switched automatically by the CAN hardware (eCOM), manually or by a signal and the HiL hardware. It can also

permanently be switched on, but this can have some influence on the behavior of the ECU (no sleep mode).

When using standard configuration of interfaces, the Ethernet interface is being activated at the beginning (at loading the configuration) and therefore the ActivationLine must be on.

After the 1st activation via a Ethernet LogicalLink every next activation happens much faster.

After losing Ethernet contact the MCD-3 D server is often not able to re-establish the contact again. In this case a restart of the MCD-3 D module might help. If this also does not work, the ECU is in an instable state. Please switch off the ActivationLine (or the whole ECU), shutdown the MCD-3 D module, switch on the ActivationLine and restart MCD-3 D after a few seconds.

For more details please see chapter ***3.2.7.8 MCD-3 D diagnostics*** and chapter ***8.10.1.4 Configuring DTS8 and DTS9***.

6.2.2.15 Variant

ECUs usually have several variants differing in the data. Services and jobs are typically valid for a special variant. It means, in another variant there can be different services and jobs, or they have another algorithm and deliver different results. Therefore it is essential to set the proper variant before executing services and jobs.

PROVEtech:TA can check the ECU variant after activating (establishing contact) and correct the used variant, if a wrong one was set. This variant checking can be deactivated.

Functional groups are treated the same way as ECUs, but they do not have variants.

6.2.2.16 Session

The ECU has different modes, so-called sessions. The normal mode (when no contact between tester and ECU is established) is the default session. For diagnostics purposes typically the extended session is activated. Sometimes there is also a programming session.

The session is set by establishing the contact (activation) in a special way. The session type to be used can be selected in the ECU's settings and it can be changed at any time at runtime.

Since new security mechanisms have been established, the new state of Security Session is introduced. This mode is not activated by a simple message, but with a complex routine, typically programmed in a diagnostics job. As this method includes certificates, it

is an effective measure to allow critical actions like flashing or coding to authorized persons only.

6.2.2.17 Activation

Activation of the ECU means establishing the communication. After the ECU is activated, services and jobs can be executed.

The activation sequence depends on the chosen session and on the optional variant check.

When contact is established, there can be some messages exchanged automatically (so-called TP = TesterPresent Message) to keep the contact alive. This is managed in the background by the MCD-3 D layer, depending on the chosen session).

PrePostActivationHandler

You can define special actions to be automatically executed when an ECU is being activated or deactivated. For that you need a test or object in the Test Manager – or a MAC file in the file system. In this you need to define four routines with following signatures:

```
Function PreActivation(sECU As String) As Boolean
Sub PreDeactivation(sECU As String)
Sub PostActivation(sECU As String)
Sub PostDeactivation(sECU As String)
```

This test or object must be entered in the MCD-3 D configuration under “Pre/post (de)activation actions“. See chapter ***3.2.7.8 MCD-3 D diagnostics settings*** for details.

The PreActivation action differs from the others. As a function, it returns a Boolean value. If it returns False, the activation will not be continued. The other actions are subroutines and have no return value.

All four actions get the name (the displayed alias name) of the ECU as argument. Using this, the code can execute different actions depending on the ECU. As an example the following use case is possible: When you want to activate a DoIP ECU via Ethernet, the routine can automatically switch the activation line (if such a signal is defined in your model), e.g.:

```
Function PreActivation(sECU As String) As Boolean
    Dim ecu As DgnEcu
    Set ecu=Dgn.Ecus(sECU)
    ' Special ECU - switch AL on
    If ecu.LogicalLink="UDS_Ethernet_DoIP_HU9" Then
        System.SetSignal("ActivationLine", 1)
        Wait 5
    End If
```

```
    Return True
End Function
```



The actions are executed in own threads. They can interact with or block other threads. It is recommended to keep the code as simple as possible to avoid negative impact.

6.2.2.18 The Fault Memory (DTC)

ECUs have a fault memory, to be read and cleared with predefined UDS commands.These fault information is usually named DTC (diagnostic trouble codes). You find the special services in the semantics FAULTREAD, FAULTCLEAR, ENVREAD. For a more comfortable handling there are typically also jobs, making dealing with the complex structure easier.

When the ODX data comply with specific guidelines, the DTCs can be also handled in a special table view – see chapter ***6.2.3.10 The DTCs*** for details.

6.2.2.19 Variant coding

Coding (variant coding) permanently sets some variant specific parameters. The set values remain unchanged even during a reset of the ECU. The ***Variant*** will not be changed. You find the specific services in the semantics STOREDDATA_READ, STOREDDATA_WRITE, VARCODEREAD, VARCODEWRITE or similar. For data compliant with specific guidelines a special coding domain view is available (see chapter ***6.2.3.9 The coding*** and ff).

6.2.2.20 Flashing

You can flash the ECU's EEPROM with new program and/or data. During this transmission of big data amount special care must be taken – interrupting the flash process might make the ECU unusable or even damage it permanently.

After flashing the ECU must be regarded as a new one with another Variant than before. See chapter ***6.2.3.12.2 Special case: the FlashJob*** and the following for details.

6.2.2.21 Communication parameters

The communication is influenced by a set of parameters. These parameters are defined and set in the ODX data for any ECU. They can differ also for special services or jobs. Normally there is no need to control or change them, but sometimes some adjustments must be made. This is done using the special service named “ProtocolParameterSet” (or similar) in the service group with no semantic.

6.2.2.22 QuickTest

This test, also known as ShortTest or FastTest, reads pre-configured informations (trouble codes, variant coding, other services…) out of all attached ECUs and writes the results in a text, HTML or XML protocol file. The advantage of this is getting a lot of information without any user interaction.

PROVEtech:TA gives you the possibility to program complex routines. Therefore it was not necessary to implement a QuickTest directly into PROVEtech:TA, but you find a ready-to-use script in the folder “Goodies\QuickTest”. You can import this script into the Test Manager and use it as-is or make any individual changes or extensions.

Please read the file “ReadMe.txt” in the same folder for details about the usage and examples.

6.2.2.23 Monitoring

To monitor diagnostic messages please use the message window and a model which receives CAN messages. In the model configuration an ISO-TP configuration must be added to a message channel. For that you need the request and response Id of the ECU. See also chapter ***3.2.7.2.3.12 Configuration of CAN channels***.

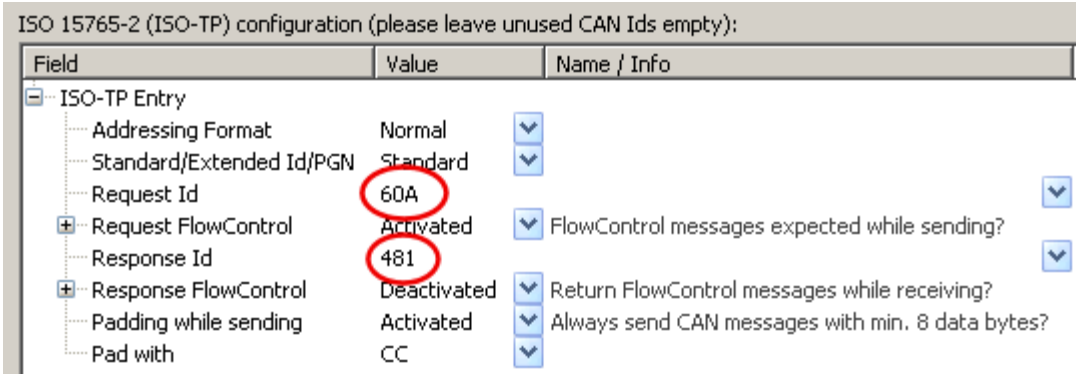


Figure 477 – ISO-TP configuration

To find out the values of the request and response Id, read the ComParams. In the service ProtocolParameterSet (for DTS) resp. #RtGen_ProtocolParameterSet (for DSA

and VW) you find the values CP_CanPhysReqId and CP_CanRespUSDTId.
Attention: these values are sometimes coded as decimal, in other cases hexadecimal.

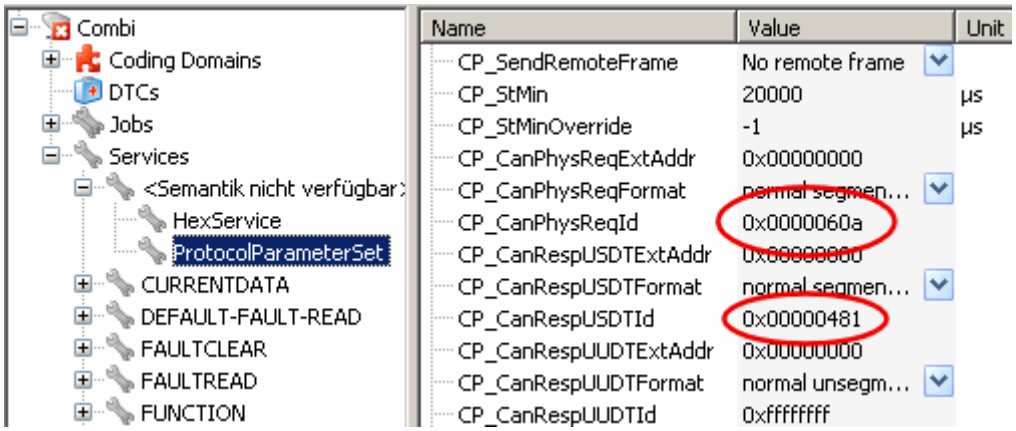


Figure 478 – ComParams defining Req-Id and Res-Id

On the workpage you can open a message window and select for “Channels and message types” only ISO-TP:

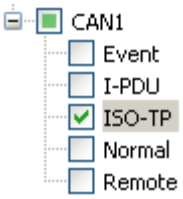


Figure 479 – message type selection

With this configuration you get all diagnostic messages exchanged with the ECU:

SW time	Id	Data	Channel	D...	Message ...
5.93	60A	22 F1 00	CAN1	Rx	ISO-TP
6.08	60A	22 F1 00	CAN1	Rx	ISO-TP
6.10	481	62 F1 00 00 14 07 01	CAN1	Rx	ISO-TP
6.11	60A	22 F1 54	CAN1	Rx	ISO-TP
6.13	481	7F 22 78	CAN1	Rx	ISO-TP
6.16	481	62 F1 54 00 11	CAN1	Rx	ISO-TP
6.17	60A	10 03	CAN1	Rx	ISO-TP
6.19	481	50 03 00 14 00 C8	CAN1	Rx	ISO-TP
8.19	60A	3E 00	CAN1	Rx	ISO-TP
8.20	481	7E 00	CAN1	Rx	ISO-TP
10.20	60A	3E 00	CAN1	Rx	ISO-TP
10.20	481	7E 00	CAN1	Rx	ISO-TP
12.20	60A	3E 00	CAN1	Rx	ISO-TP
12.20	481	7E 00	CAN1	Rx	ISO-TP
12.29	60A	10 01	CAN1	Rx	ISO-TP
12.29	481	7F 10 78	CAN1	Rx	ISO-TP
12.32	481	50 01 00 14 00 C8	CAN1	Rx	ISO-TP

Figure 480 – ISO-TP messages

6.2.2.24 DoIP monitoring

For monitoring Ethernet communication, usually Wireshark will be used. The diagnostic server DTS9 has a built-in capability to record DoIP messages in a pcap file. This file contains only filtered diagnostics relevant messages and can be opened in Wireshark for analysis.

To start and stop recording, use the Basic functions *Dgn.StartMonitoringDoIP* und *Dgn.StopMonitoringDoIP*.

6.2.3 Diagnostics MCD-3 D window

The diagnostic window consists of two toolbars and four windows:

- The action toolbar
- The control toolbar
- The navigation area (left-most)
- The request area (upper right)
- The response area (middle-right)
- The communication view (lower right)

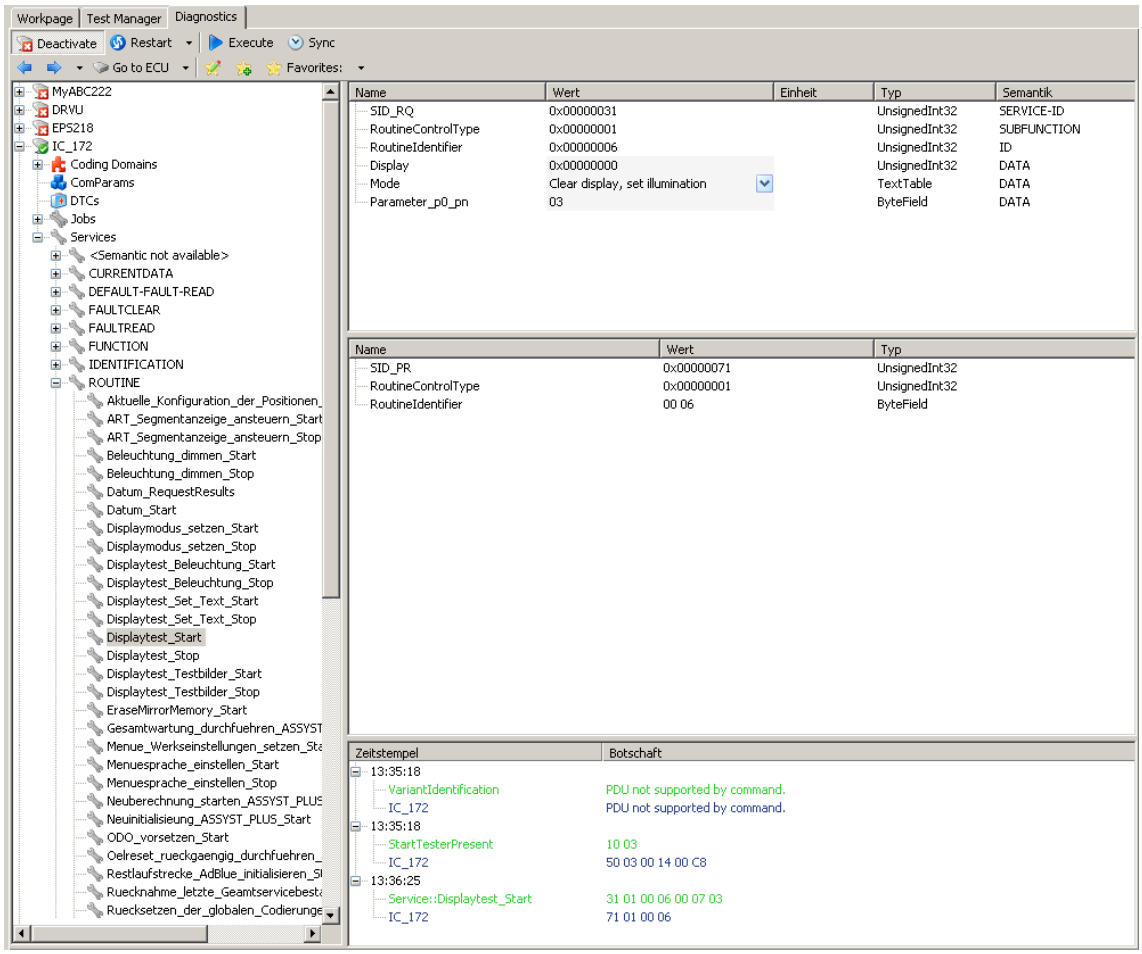


Figure 481 – Diagnostics MCD-3 D window

Communication with the ECU is available via context menu or toolbar (if PROVEtech:TA license contains access to the hardware – test system license).

In most windows multiple selection is possible. The selected lines can be copied into the clipboard with the standard copy hotkeys: Ctrl-C, Ctrl-Ins, or by the context menu.

6.2.3.1 Action toolbar

The action toolbar contains depending on the selected element type in the navigation area the buttons that execute the allowed actions in this context. This toolbar distinguishes actions that are available on all elements and actions that are only available in specific contexts. There are two actions that are always available. On the one hand there is the button `Activate` or `Deactivate` (depending on the activation state of the selected ECU), that activates of deactivates the ECU. On the other hand there is the button to shutdown, start or restart the MCD-3 D module. These two buttons are left of the delimiter. Right of the delimiter there are the context-dependent buttons.

The button for activating the ECU changes dependent on the current activation state of the ECU. Either it is the button for activating the ECU or for deactivating the ECU. The button for shutdown, start and restart behaves the same way. Depending on the state of the module the button offers the action to change the state. If the module is halted the button offers the action `Start` and if it is started it offers the action `Shutdown`. Furthermore it depends on the last user action if the specific action is offered or the more general action `Restart`.

The levels "ECU", "Jobs", "Job-Semantic", "Service" and "Service-Semantic" (see chapter ***6.2.2.12 Semantic As Attribute***) of the navigation area (see chapter ***6.2.3.4 Navigation area***) do not have any context-dependent buttons.

The levels "CodingDomains" and "CodingDomain" contain additional buttons for `Read coding` and `Write coding`. The level "CodingDomains" contains additionally buttons to invoke a context menu for loading or saving a SGDE, ECX or AED. The level “CodingDomain” contains an additional button to switch the display of the parametrization window between *short name* and *long name*.

The level of DTCs expands the toolbar with the buttons `Read DTC(s)` and `Clear DTC(s)`.

The level of the specific services and jobs contain the button `Execute`, that executes the selected element with the currently set parameters.

Symbol	Action	Tree selection	Description
	Activate	All	Activates the ECU, that is selected in the navigation area if it is inactive.
	Deactivate	All	Deactivates the ECU, that is selected in the navigation area if it is active.
	Restart	All	Restarts the MCD-3 D module.
	Shutdown	All	Shutdowns the MCD-3 D module.
	Start	All	Starts the MCD-3 D module.
	Execute	Service/ Job	Executes the selected service or job.
	Sync / Async	Service, Job, CodingDomain, CodingDomains	Determines if the execution happens synchronously (imme-diately) or asynchronously (background)

	Read DTC(s)	DTCs	Reads the DTCs from the selected ECU.
	Clean DTC(s)	DTCs	Cleans the DTCs in the selected ECU.
	Read coding	CodingDomains/ CodingDomain	Reads the current coding from the ECU.
	Write coding	CodingDomains/ CodingDomain	Writes the current coding to the selected ECU.
	Load SGDE…	CodingDomains	Opens the SGDE/ECX/AED dialog in loading mode.
	Save SGDE…	CodingDomains	Opens the SGDE/ECX/AED dialog in saving mode.
	Long Name Short Name	Service, Job, CodingDomain	Switch the parametrization window between *long name* and *short name* of the parameters or fragments
	Load flash file	ECU, FlashJob	Loads additional flash files into the project
	Open flash window	All	Opens a window for flashing several ECUs in one step

6.2.3.2 Control toolbar

This toolbar contains buttons to navigate in the tree of the navigation area. It contains the buttons for the history `Backward` and `Forward` and the button `Go to ECU`. The MCD-3 D module stores the latest ten selected elements in a history. You can step thru the list with the buttons `Backward` and `Forward`. The button `Forward` offers additionally a context menu that offers the possibility to jump directly to one of the recent elements. The button Go to ECU offers a context menu with a list of all the ECU elements in the project. It is possible to jump directly to one of the ECU elements by clicking on the respective ECU. A click on the button itself executes a jump to the last selected ECU. After a restart this is the first ECU in the list.

The control toolbar contains the following elements:

Symbol	**Action**	**Tree selection**	**Description**
	Backward	All	Jumps one element back in the history of selected elements.

	Forward	All	Jumps one element forward in the history of selected elements.
	Go to ECU	All	Jumps to the last selected ECU.
	Switch filter header on/off	All	Switches the filter header in the navigation, the request and response views
	Configure favorites	All	Opens a dialog window for configuring the favorite list
	Add to favorites	Services, jobs, coding, DTCs	Adds currently selected object with current parameter values into favorite list
	Favorites	All	Selects last used favorite (or opens a list with favorites, if none was used yet)

6.2.3.3 Filtering the views

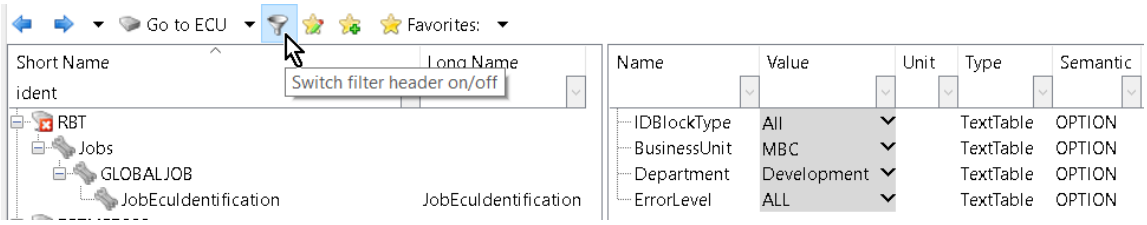


Figure 482 – The filtering header lines

When filter headers are active (activated by the filter icon in the control toolbar), the navigation, request and response views get an additional header line where you can type in any text to filter the displayed elements in each column. The elements will be displayed only when the entry in the column contains the entered text – or if a child element of the item fits the filter.

The filtering is not case-sensitive.

For tree structures the following applies:

- Elements containing positively filtered sub-elements will be also included, regardless if their own name fits the filter
- In the navigation area all elements of the 1st level (ECUs and functional groups) ale always visible, even if they have no visible sub-elements
- After a change in the filters all nodes will be first displayed in a collapsed state – except the selected element which stays visible. To make all elements visible,

select the chosen node (e.g. ECU) and press the * key. This will expand all branches.

Note that the find window (chapter ***6.2.3.4.4 Find in navigation area***) and the favorites (chapter ***6.2.3.4.2 Favorites***) will be affected when filtering is on: items suppressed by the filter will not be found neither by the find function nor by favorites.

6.2.3.4 Navigation area

Here you can navigate thru all database information available from control unit description file of your currently selected MCD-3 D project and MCD-3 D vehicle information.

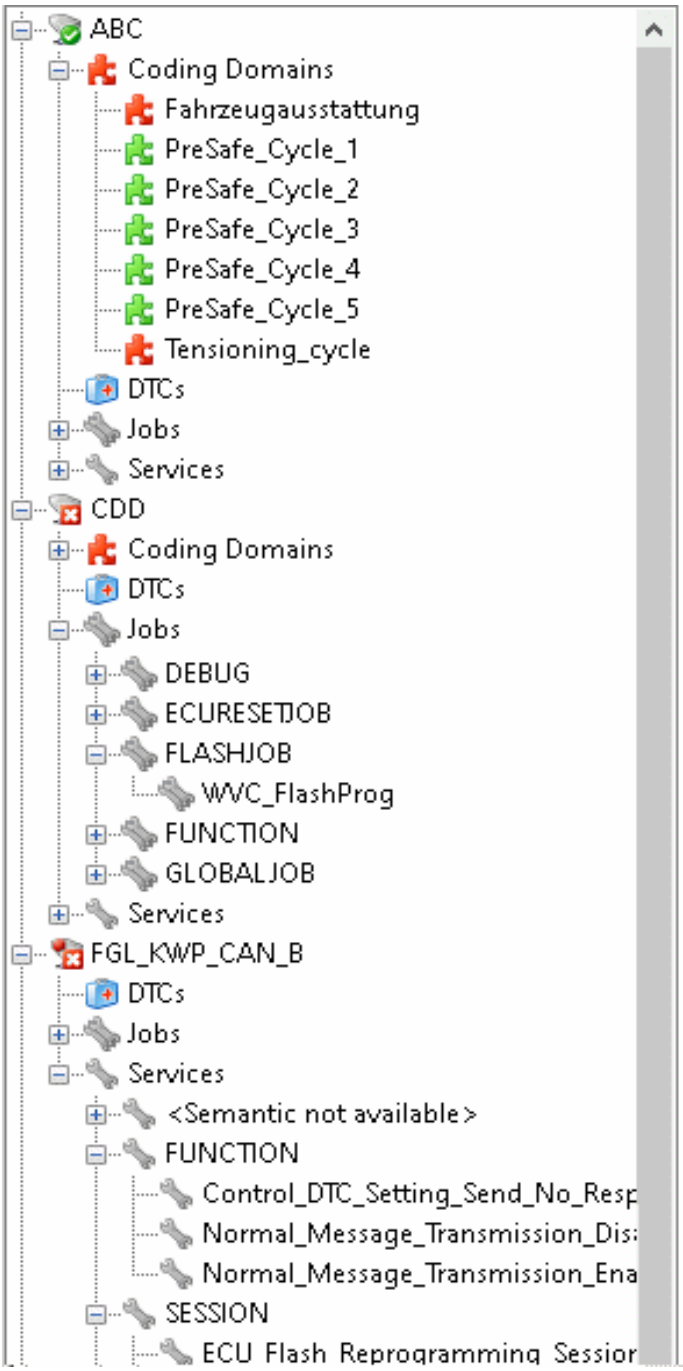


Figure 483 – Diagnostics MCD-3 D navigation area

Navigation area reflects the hierarchies in the control unit description file. Furthermore offers this area some more elements to assure a better overview for the user.

Selection in the navigation area causes the data area (request view and response view) to be updated accordingly. Depending on the selection there is no response view at all. For example there is no response view on CodingDomains level because the responses are set directly to the parameters in the request view.

In the navigation area, ENTER or a double click causes the default action of the selected element to be performed. In case of an ECU it is activation or deactivation, depending on the current state of the contact. For coding domains it is reading the coding. For DTCs reading of the trouble codes is called. For jobs and services the default action is executing them with the currently set parameter values. In this case an error message can happen if some parameters have no set value.

In case of folders containing services, jobs or semantics, there is no default action defined.

There is a set of ECUs for each project that is worked with. To change the set of ECUs of the navigation tree, you need to change the MCD-3 D project and MCD-3 D vehicle information. This is done by loading a different configuration (see chapter ***3.2.7.2 Configurations***).

To edit the MCD-3 D diagnostic part of the configuration or add a configuration with a different MCD-3 D project or MCD-3 D vehicle information, go to settings dialog in the *Application* menu or have a look at the ***Administrator's Guide***.

Exactly one MCD-3 D project and exactly one MCD-3 D vehicle information can be active at a time.

If you are interested in parts of control unit description file which are not displayed within PROVEtech:TA please use the ODX viewer of your choice.

6.2.3.4.1 Navigation hierarchies

Symbol	Level	Content	Corresponds to ODX
	0	List of ECUs List of Functional Groups	<LOGICAL-LINK>
	2	Services	<DIAG-SERVICE>
	3	Service semantics	Attribute SEMANTIC
	4	Services within a semantic	<DIAG-SERVICE>
	2	Jobs	<SINGLE-ECU-JOBS>
	3	Job semantics	Attribute SEMANTIC
	4	Jobs within a semantic	<SINGLE-ECU-JOB>

Further symbols used:

Symbol	Level	Content
	1&2	Coding data of a coding domain or the coding domains element above, that are asynchronous to the data stored in the ECU.
	1&2	Coding data of a coding domain or the coding domains element above, that are synchronous to the data stored in the ECU.
	1	View for DTCs. If this element is available and selected, the module changes to the DTCs view. Here it is possible to set the parameters for reading the DTCs, read the DTCs and clean the DTCs.

This elements are only available with proper ODX files.

6.2.3.4.1.1 ECU

The name of the ECU within PROVEtech:TA MCD-3 D diagnostics is the short name of the base variant of the ECU in the control unit description file. Furthermore it is possible to set an own name for the ECU via the attribute `Displayed name (Alias)`. If there is no alias set then the alias corresponds with the short name of the base variant of the ECU. This name is set by the ODX supplier in the ODX-D file.

In the ECU request view some parameter can be set: the alias name (displayed name), the ECU variant, variant checking at activation, the current session (used for ECU activation) and the initial session (with which to start) and the logical link.

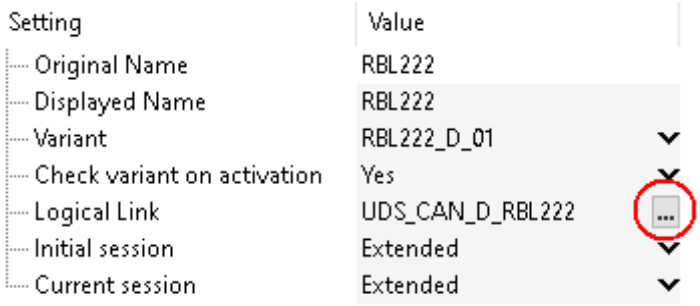


Figure 484 – parametrization area of the ECU

When the option `Manually configure interfaces` has been activated in the configuration window (see chapter ***3.2.7.8 MCD-3 D diagnostics settings***) together with the logical link also the interface and interface resource (see chapter ***6.2.2.3 D-PDU API***) can be selected. In this case the button "…" (marked with red circle) opens a selection window. Using this, you can assign different channels (e.g. CAN1 or CAN2) or even different communication hardware (e.g. eCOM instead of Vector box) to any separate ECU.

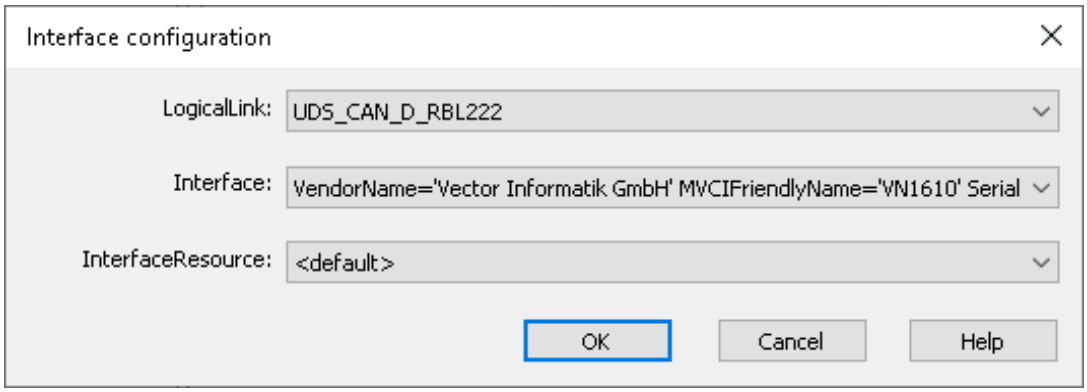


Figure 485 – configuring the communication channel

After selecting a logical link (this might be one for CAN and one for Ethernet) the 2nd line contains all available interfaces. The 3rd line contains a list of all interface resources available for the currently selected interface. Selecting <default> lets the diagnostic server manage the channel. The last valid selection is written to the configuration settings in the diagnostics INI file and will be restored the next time the configuration is loaded.

Important. The name of the interface contains some information about the used hardware (e.g. serial # or IP address). When the test stand is reconfigured, the name may change. In this case the former interface will not be recognized at startup and reset to <default>.

The currently used symbol of the ECU distinguishes between activated ( )and deactivated ECUs ( ).

There are two kinds of activation for an ECU in PROVEtech:TA. On the one hand the explicit activation via the button Activate. On the other hand the implicit activation via the execution of a service or job. In the latter case the ECU is always activated before the execution of the service or job and deactivated afterwards. When executing many services or jobs in a row than the complete execution time is increased by this. Therefore it is recommended to activate the ECU before executing several services or jobs.

When PROVEtech:TA is closed or the diagnosis module is shut down, all ECUs are deactivated.

6.2.3.4.1.2 Functional group

Functional groups lie on the same level as ECUs and are treated in a similar way. They do not have variants and cannot be flashed, but have ComParams, DTCs, jobs and services depending on the diagnostic data. They can be initialized as an ECU. The jobs and services generally are not bound to only one special ECU (functional addressing).

Including functional groups in the navigation area is optional. It can be switched off by the configuration.

6.2.3.4.1.3 Services

A diagnostic service defines a communication process between a tester and an ECU in order to inquire diagnostic information from the ECU or/and to modify the behavior for diagnostic purposes.

Possible service responses that are expected from the ECU are defined in the control unit description file.

In the best case at least three service responses are defined in the ODX file:

- Positive response
  Defines a positive response of the ECU for the specific service. This is how the MCD-3 D system is able to decode the response.
  Example:
  The ECU replies on the service 0x22 0xF1 0x03 with 0x64 0xF1 0x03 0xNM 0xNM 0xNM

- Negative response
  Defines negative responses of an ECU for a service. So the MCD-3 D system is able to decode the responses.
  Example:
  The ECU replies on the service 0x22 0xF1 0x03 with 0x7F 0x22 0x22.

- Global negative response
  Defines a negative response that is valid for all services.
  Example:
  The ECU replies on the service 0x22 0xF1 0x03 with 0x7F 0x22 0x33. 0x7F 0x22 0x33 is not defined as a negative response of the service. A global negative response 0x7F 0xNM 0xNM (NM = all possible hex values) is defined. So this pattern matches and the negative response is decoded.

Example:

- Select the service in the navigation area.
- If there are editable request parameters without a value, set the value in the data area.
- Select *Execute* in the context menu, press “Execute” in the toolbar or press the hotkey F5 when the service is selected in the navigation area.

6.2.3.4.1.4 Request parameter

Like already mentioned in chapter ***6.2.2.11 Service In Diagnostics*** a service always corresponds to a sequence of bytes sent on the bus, that instructs the ECU to execute an

action. It is not very wise to create a separate service for each possible user input. Just to adjust the temperature from 0°C to 30°C 31 services would be needed. This effort is too much. For this case there are request parameters for services, that can be freely used. In case a temperature has to be adjusted there is the service 0x2E for writing bytes to the ECU memory. Additionally a sub parameter is defined for adjusting the temperature 0x42. Last but not least a request parameter is defined, that contains the temperature in degree Celsius. In case the ECU knows the sequence of the message 0x2E 0x42 0x20 the ECU would change the value of its temperature to 32°C.

The request parameters are shown in the request view in PROVEtech:TA (see chapter ***6.2.3.5 The request view***), in case a service or job is selected in the navigation area (see chapter ***6.2.3.4 Navigation area***). Writable parameters have a different background color.

6.2.3.4.1.5 Response parameter

The counterpart of the request parameter is the response parameter. The difference is that a response parameter is returned by the ECU and the task of the MCD-3 D system is to estimate the correct value according to the ODX files. An example is to read the temperature value from the ECU. For this task the UDS protocol defines the service 0x22. The service 0x22 0x42 (again 0x42 the code that we have defined for the temperature) is send to the ECU. As a positive response we will receive 0x62 0x42 0x20. Our response parameter is 0x20 now and tells us that the temperature is at 32°C. This is also what we were expecting, since we set this value for the temperature in chapter ***6.2.3.4.1.4 Request parameter***.

PROVEtech:TA shows the results and therefore the response parameters in the response view (see chapter ***6.2.3.6 The response view***).

6.2.3.4.1.6 Service semantic

Every service can have only one semantic.

The following list contains the predefined service semantic values as proposed in the ASAM MCD-2 D (ODX 2.0.1) standard.

Pre-defined value	**Description**
SESSION	Switch a diagnostic session
SECURITY	Enable security access
IDENTIFICATION	Read ECU or flash identification
FAULTREAD	Read diagnostic trouble codes
FAULTCLEAR	Clear diagnostic trouble codes

ENVREAD	Read environment data of diagnostic trouble codes
VARCODING	Job or service for variant coding
STOREDDATA	Read/write stored data
CURRENTDATA	Read current data
MEMORY	Read / write ECU memory
CONTROL	Start/stop routing, request routine results
CALIBRATION	Calibration
ROUTINE	Start/stop a routine and/or request routine results
FUNCTION	Execute function like enable normal message transmission, reset ECU
DOWNLOAD	Request download transfer data
UPLOAD	Request upload transfer data
FLASHJOB	Special job used for flashing
TESTERPRESENT	Handshake of tester
DEFINE-DYN-MESSAGE	The belonging service defines a DYN-ID record
READ-DYN-DEFINED-MESSAGE	The belonging service reads data using a DYN-ID defined before.

6.2.3.4.1.7 Job and job response

A diagnostic job defines a usually complex communication process between a tester and an ECU. It is specified by JAVA program code and may contain an arbitrary number of service executions, for instance.

Typical use cases for jobs are seed-key functionalities.

A job can have parameters, but the request as well as the response parameter set can be empty.

Each job returns one final result. Always at least one response exists, even if no response parameters exist. The result of a job is freely designed by the job writer.

Only single ECU jobs are supported. These are jobs which are assigned solely to one DIAG-LAYER and thus only communicate with one ECU.

For these jobs, only physical addressing is possible, functional addressing is not allowed.

6.2.3.4.1.8 Job semantic

Every job can have only one semantic.

The following list contains the predefined job semantic values as proposed in the ASAM MCD-2 D (ODX 2.0.1) standard

Pre-defined value	**Description**
VARCODING	Job or service for variant coding
FLASHJOB	Job used for flashing

6.2.3.4.2 Favorites

Executable objects – services, jobs, coding domains, DTCs – can be collected in a list with up to 200 elements. They can be accessed from the list without the need to search in the navigation area. The parameters of the objects can be pre-set with values.

The favorite list consists of two parts. The users own favorites can be individually configured by any user. These favorites are stored in the user's configuration (file User.ini). The common favorites is valid for all users and it is stored in the file DiagnosticsMCD3D.ini. This file can be changed only by users with admin privileges.

The favorites are project specific and the depend on the used language. Some labels in the navigation area differ in different language settings. A favorite list might not work in an other language than it was created.

6.2.3.4.2.1 Properties

A favorite is a reference to an executable element with the following data:

- Name – can be changed by user
- Path – information about the position of the element in the navigation area (not changeable)
- Action – activating a favorite causes the referenced element to be selected and sets parameters to stored values (setting "Select"). Optionally the element can be immediately executed (setting "Execute")
- Parameters – changeable parameters of the element can be optionally pre-set with values

6.2.3.4.2.2 Add to favorites

When an executable element is selected in the navigation area, there appears in the toolbar and in the context menu the item *Add to favorites*. This command adds the selected element to the user specific favorite list.

The newly added favorite gets the name of the referenced element, preceded by the ECU's name. It can be renamed in the favorite configuration window. This might be useful when the same element is to be used with different parameter values.

The parameter values, as set in the request view, are automatically stored in the favorite list. These values can be changed. Parameters can be also excluded from setting by favorites.

New favorites by default get the setting "select". This can be changed in the favorite configuration window to "execute".

6.2.3.4.2.3 Using favorites

Clicking the button `Favorites` in the opens a list of all favorites:

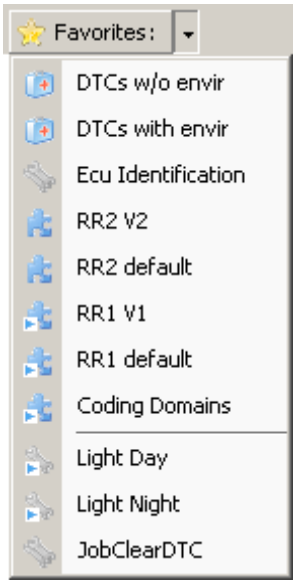


Figure 486 – List of favorites

The type of elements can be identified by the icons – it is the same icons as used in the navigation area (for example for services). Clicking on an item in the list causes the referenced element to be selected. If parameter values are stored for this element, these values are copied into the request view.

If the icon has an additional small blue triangle symbol (for example for services), then it will be immediately executed after selection.

The last used favorite is put on the `Favorites` button:



Figure 487 – Favorites button after selecting an item

This favorite can be therefore activated again by a single click. To use another favorite, click on the small triangle symbol next to the button to open the favorite list again.

6.2.3.4.2.4 Configuring favorites

A click on the button opens a window for configuring favorites. In the left list all favorites of the current project are contained. When one of them is selected, the right window contains all parameters of this element.

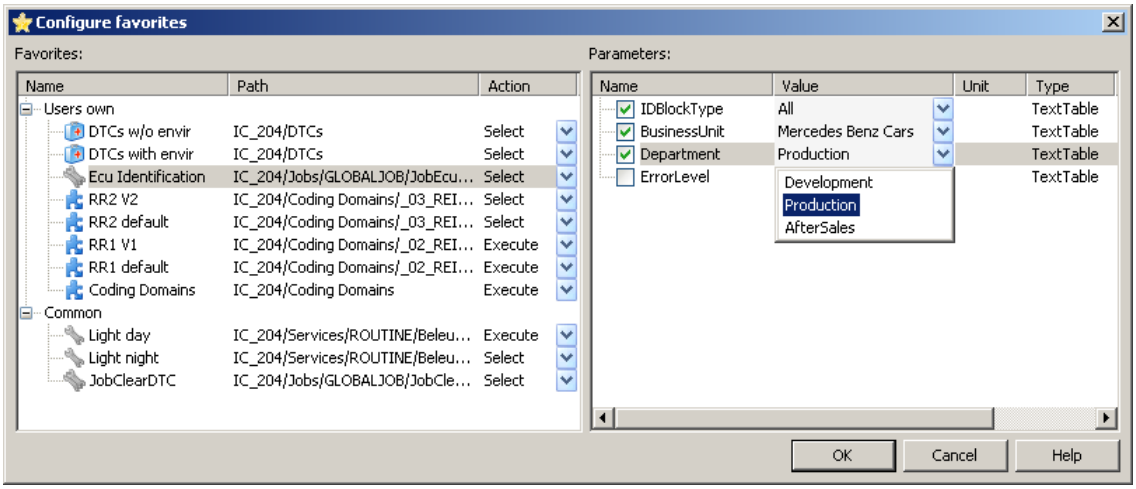


Figure 488 – Favorites dialog

All possible changes can be accessed by a context menu – right mouse button click on the element to be changed. In the left window the menu contains the following commands:

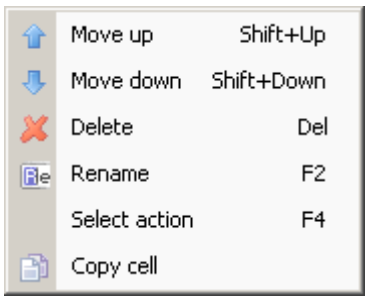


Figure 489 – Favorite context menu

A right click on a parameter in the right window opens the following menu:

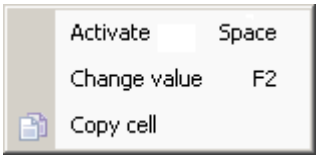


Figure 490 – Parameter context menu

All functions can be also accessed by keyboard, as specified in the menu. The following changes are possible:

- Re-sort favorites. Shift + arrow moves a single line up or down. This is also the way to move a favorite from the user specific list to the common list (or the other way), if the user has admin privileges.
- Delete a favorite with the key “delete”.
- Rename a favorite. F2 or a (double) click on the label opens an edit control in which the name can be changed.
- Setting the action. F4 or a click on the combo box opens a list with the options “Select” and “Execute”.
- Setting parameters. The right list displays all parameters of the element selected in the left window. Parameters with grey background can be changed by editing or by selecting one of several possible values (F2, F4 or click on the value). The check box left of the name determines if the parameter should be set when activating the favorite. Unmarked check box means, the value of this parameter will not be changed by the favorite.
- Copy cell (only by menu item). The content of the clicked table cell is copied into the clipboard. It can be used e.g. to be inserted in a text document.

Close window. The button `OK` confirms the changes. `Cancel` closes the window discarding all changes.

Attention. Favorites are stored in two INI files: in the user.ini (user’s own favorites) and in the DiagnosticsMCD3D.ini (common favorites). The saving into files happens during closing the application or changing configuration (loading of a new project).

6.2.3.4.2.5 Possible problems

Errors in the name, path or parameters of a favorite can have different effects: the application might not find the referenced object, or the object cannot be executed due to erroneous parameters. Please note the displayed error message. This can be caused by:

- Using a favorite created with a different language setting

- Using a different version of ECU data. The named element might not exist or might have different parameters
- Using an ECU with a different variant. Some elements are defined not for all variants
- Some parameters of the favorite are not pre-set. If these parameters have no default value and are not set explicitly, execution will lead to an error message or to unpredictable results.

Another possible problem: if the filtering is active (chapter ***6.2.3.3 Filtering the views***) and the object of interest or its parameter is filtered out, it can happen that the referenced object will not be found or its parameters will not be set completely according to the preferences stored in favorites.

6.2.3.4.3 Navigation area context menu

The content of this context menu depends on the type of the selected element.

Symbol	Action	Type of element	Description
	Copy	All	Copies the name of the currently selected element.
	Find…	All	Opens the find dialog for a search beneath the selected element.
	Activate ECU	ECU	Activates the selected ECU.
	Deactivate ECU	ECU	Deactivates the selected ECU.
	Switch to default session	ECU	Switches to default session.
	Switch to extended session	ECU	Switches to extended session.
	Read coding	CodingDomains	Reads all CodingDomains from the ECU.

	Write coding	CodingDomains	Writes the currently in PROVEtech:TA set values of all CodingDomains to the ECU.
	Reset Sync Flag	CodingDomains	Resets the sync flag of all CodingDomains to asynchronous.
	Read coding	CodingDomain	Reads the value of the currently selected CodingDomain from the ECU.
	Write coding	CodingDomain	Writes the value of the currently selected CodingDomain to the ECU.
	Execute	Service/ Job	Executes the currently selected service or job.
	Add to favorites	Service, Job, Coding Domain(s), Dtc	The selected object and its currently set parameter values is added to the favorite list.

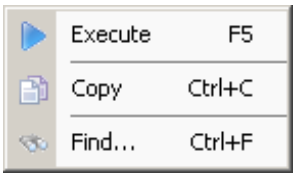


Figure 491 – MCD-3 D diagnostic context menu

For PROVEtech:TA installations which are hardware-enabled, the context menu additionally allows to activate an ECU, deactivate an ECU and to execute a service or job.

6.2.3.4.4 Find in navigation area

To enable search for a service or job, a find dialog is available in navigation area thru its context menu or the Hot Key Ctrl+F.

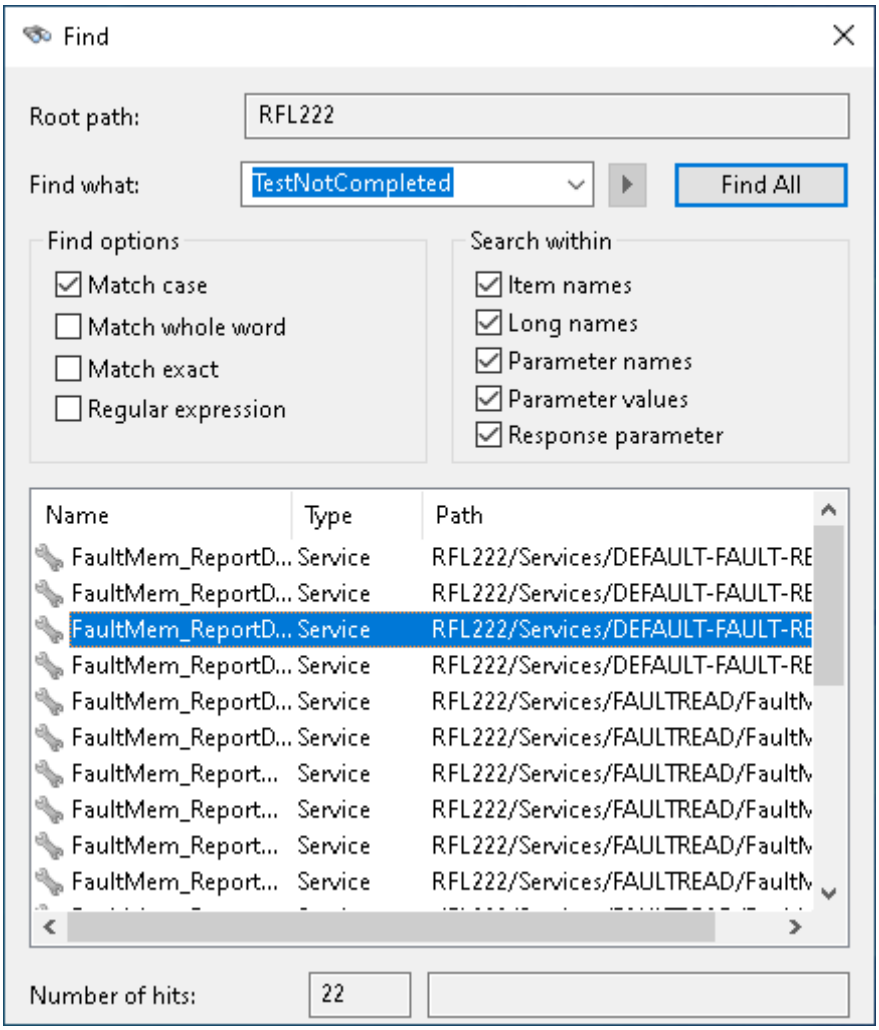


Figure 492 – Find dialog of the navigation area

The find dialog like it is shown in ***Figure 492*** consists of several parts. The combination field `Find what` contains the term to search for. The latest six terms are stored and they are shown in the combination field in chronological order. The search is refined with the search criteria in the `find options`. If the check box `Regular expression` is active the button with the triangle next to the combination field `Find what` becomes enabled. Regular expressions can be constructed via this button (see ***Figure 493***). After the execution of a search the results are shown in the list.
The criteria `Search within` determine what the search key should be compared with: the item name, long name, the parameter name and/or the parameter value.

Important: the search of parameter's names depends on the setting *long name* or *short name*:
You can go to the found element in the navigation area by double clicking on the result element in the list. If the found element is a request parameter, a double click will select the parent element in the navigation area (service or job) and also select the parameter in the request view. In case of a response parameter, the property window will be opened

for the selected element in the navigation area and the found DbResponseParam (this is the term in MCD-3 D) will be marked there as well.

If the service is available for a different ECU variant only, it can be found only if the other variant is selected in the ECU settings.

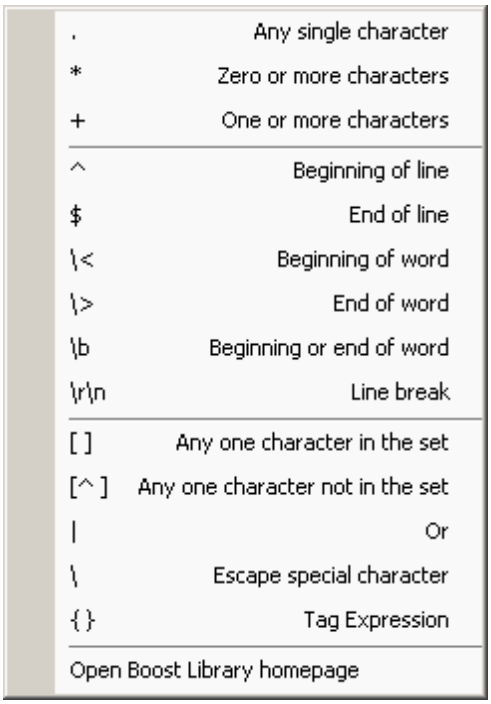


Figure 493 – Menu for regular expressions

6.2.3.4.5 The properties dialog

The properties dialog shows details for the selected element. There are two possibilities to open the dialog. On the one hand the dialog can be opened via the context menu of the particular element. In case the dialog is available for the respective element PROVEtech:TA shows the entry `Properties` in the context menu. On the other hand the dialog is opened via the hotkey `Alt+Enter`. The properties dialog is available for the elements `ECU`, `coding domain`, `DTCs`, `Service` and `Job`.

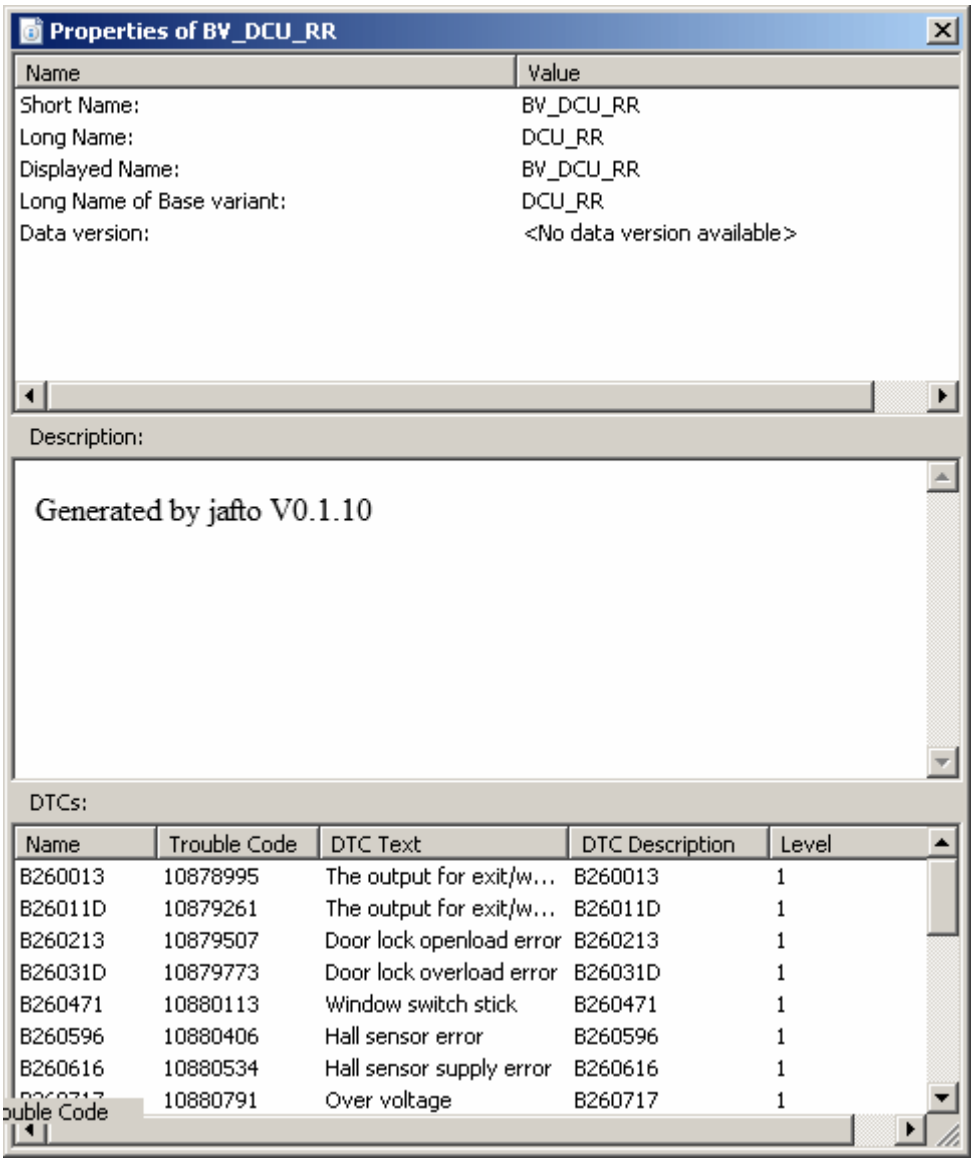


Figure 494 – The properties dialog for an ECU with DTCs

Depending on the selected element the properties dialog shows different information. Figure 494– The properties dialog for an ECU with DTCs shows the properties of an ECU. The distinctiveness of the properties of an ECU is the DTC field. Only the properties of an ECU element or the DTCs element itself show the DTCs. These are all known DTCs from the data of the ECU.

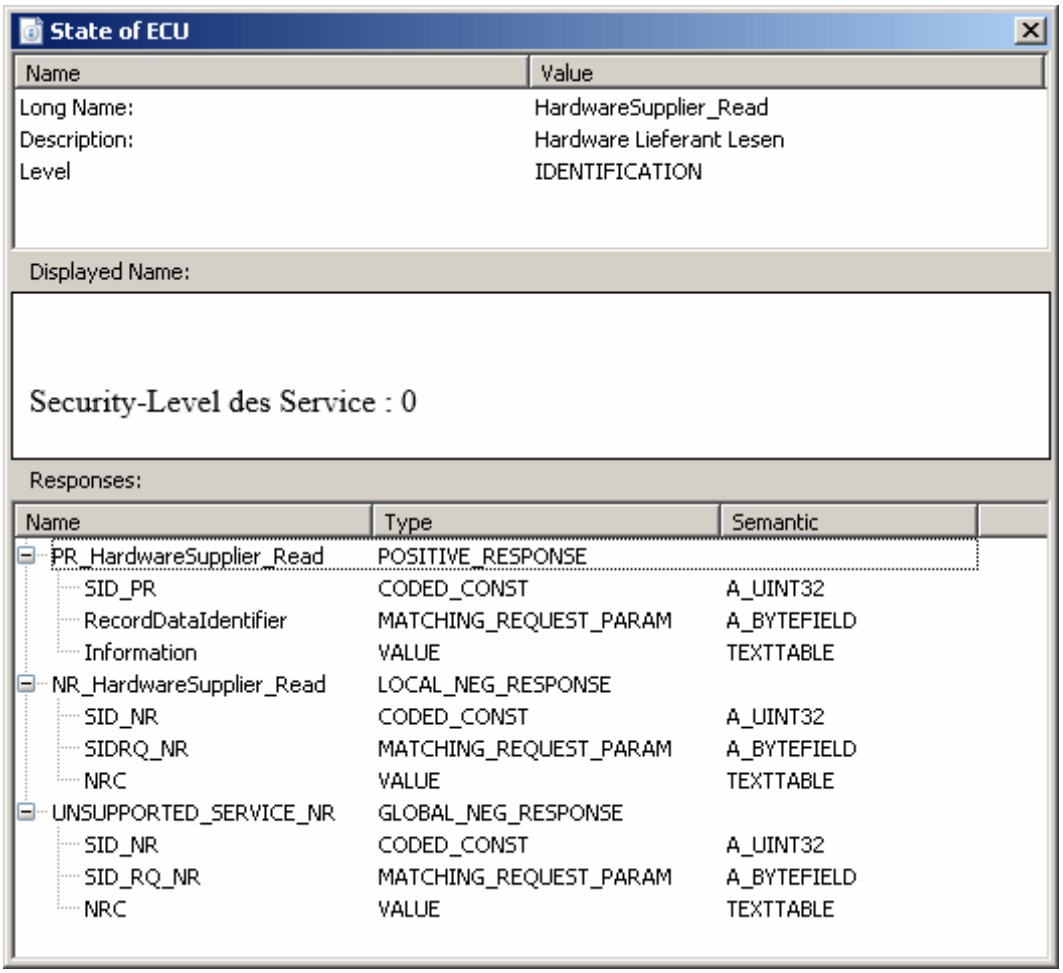


Figure 495 – Properties of a service

The properties dialog for the services and jobs is different (see Figure 495– Properties of a service). Other properties appear than for an ECU for example (see Figure 494– The properties dialog for an ECU with DTCs). The description is extracted from the data of the respective ECU. In particular explanations of the data authors for services and jobs are found in the descriptions. Typically there are hints for the purpose of a service or job, how the parameters have to be set or which prerequisites exist to execute the service or job. Which information is actually contained in the description depends exclusively on the data author.

In the lower part of the window you will see the structure of all possible positive and negative responses (PR and NR). This may be helpful for programming tests to automatically analyze the responses.

6.2.3.5 The request view

The request view is located on the upper right side of the diagnostic window. All parameters of the particular elements are shown here. Depending on the selected element in

the navigation area the content of the request view is changed. The purpose of this area is to configure the service or job before executing it. If a parameter is writable the background color is changed.

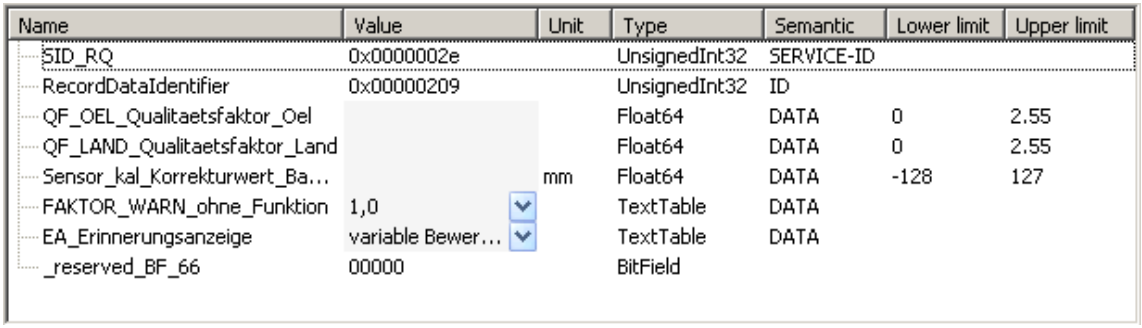


Figure 496 – The request view

Display filters

When there are very many parameters it might help to reduce the displayed items by a filter. You can activate the filter header in the request view by clicking the  button. After entering some characters in the column's headers, only items will be displayed that have the entered string as a substring in the column (not case sensitive). When in a tree-like hierarchy an item is being displayed then its parent item is being displayed also.

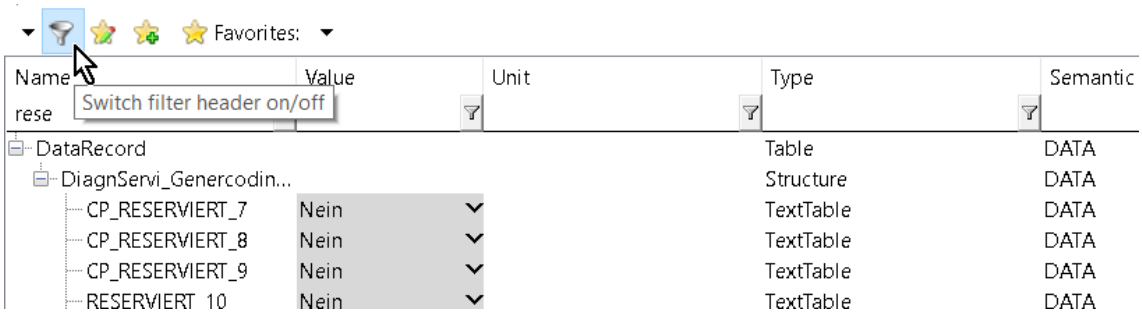


Figure 497 – The filter header

There are two different kinds of writable parameters. On the one hand there are the combo boxes that allow the selection of a predefined value from the ODX file. On the other hand there are the edit boxes that are configured freely. It should be minded that the entries in the edit boxes must correspond with the respective data type. Otherwise the entry is lost because in case of an error the value is set to the last correct value.

Type	**Input**	**Example**
AsciiString	Any sign from the ASCII repertoire	Hello world!
UnicodeString	Any sign from the Unicode repertoire	Motörhead!

Bitfield	0 and 1 that are optionally delimited by a blank or a comma.	101 1 0 1 1,0,1
Bytefield	Hexadecimal values that are optionally delimited by a blank or a comma.	C0FF EE,260B 17 7E
UnsignedInt32	Digits 0 to 9. Depending on the ODX data the value in the edit box is shown in hexadecimal form.	1754
Float64	Digits 0 to 9 and dot as delimiter. Depending on the ODX data (precision) the entered value is rounded.	7.8
Int32	Digits 0 to 9 and the minus symbol	-123
DTC	Digits 0 to 9	9598805

Easy parameter input

It is not very efficient to move the mouse and type values via keyboard for every single value. Therefore we extended the behavior of the input elements:

- After confirming the typed value by ENTER, the next editable parameter will be automatically opened for editing. Without using the mouse, you can type a series of values and simple confirm them with ENTER
- You can set several parameters to the same value in one step. Select all params to be set and press F2 resp. F4 to open the input control. The typed value will be set after ENTER to all selected parameters.

6.2.3.5.1 Name in request view

The name column is available for all navigation area selections except the vehicle information. It is the *short name* of the parameter listed in that row. In case of coding domains' parameters (aka fragments) you can switch the display between *short name* and *long name*.

6.2.3.5.2 Value in request view

The value column displays the value of the parameter in this line. Gray background color indicates that the value is editable.

If the value is editable, you can set the cursor on that field. An edit box or a combo box will pop up and allow the input.

There are following gimmicks for entering values:

- You can use the keyboard instead of the mouse. Just use the arrow keys to move the selection and press F2 or F4 to open the edit/combo box.
- After confirming the value by pressing *ENTER*, the focus is moved to the next editable parameter and the input box is opened. You can directly enter the next value.
- You can set several parameters to the same value in one step. Therefore you need to select multiple lines first (Move the mouse with pressed left button, click on several items while pressing *Shift* or *Ctrl* key or move the selection by arrow keys while pressing *Shift*) and press F2 or F4. The input box for the last selected line will be opened and the typed/selected value will be passed to all selected parameters.
  In case you accidentally select parameters which cannot be set (or not set to the entered value), you will get an error message and the other parameters will be processed.

Special case: sometimes the possible choice values cannot be displayed due to restrictions of MCD-3 D API. Then you need to check in the ODX files for the possible values.

6.2.3.5.3 Unit in request view

This column contains the unit of the parameter value.

The unit column is available if service/job is selected in the navigation area and a parameter has a unit value.

6.2.3.5.4 Type in request view

This column displays the type of the parameter value.

The following enumerated types exist:

ddtAsciiString	ASCII-coded zero-terminated character field
ddtBitfield	Bitfield

ddtBoolean	Boolean value
ddtBytefield	Byte field
ddtFloat32	32-bit float
ddtFloat64	64-bit float
ddtInt16	16-bit signed integer
ddtInt32	32-bit signed integer
ddtInt64	64-bit signed integer
ddtInt8	8-bit signed integer
ddtUInt16	16-bit unsigned integer
ddtUInt32	32-bit unsigned integer
ddtUInt64	64-bit unsigned integer
ddtUInt8	8-bit unsigned integer
ddtUnicode2String	UNICODE2-coded zero-terminated character field (UCS-2)
ddtField	Number of entries; parameter can have children parameters
ddtMultiplexer	Branch index; Parameter can have children parameter
ddtNoType	
ddtStructure	Number of entries; parameter can have children parameters
ddtDTC	Number is a DTC; number is displayed as hex value.
ddtEndOfPDU	Number of items; Parameter can have children parameter, only for request parameters
ddtEnvData	Environment data (value of DTC); can have children parameter
ddtKey	Key number
ddtLengthKey	Length of a variable length parameter in bits
ddtTable	Table data type
ddtTextTable	Text table available in ODX with choices for this parameter
ddtUnknown	Type cannot be specified

The type is available for ComParams, service, job and response selection in the navigation area.

6.2.3.5.5 Semantic in request view

This column displays the parameter semantic.

The following list contains the predefined parameter semantic values as proposed in the ASAM MCD-2 D (ODX 2.0.1) standard.

Predefined value	Description
ID	Identifier
SERVICE-ID	Service identifier
SUBFUNCTION	Subfunction of service identifier
CONTROL	Control parameter
DATA	Data
LOCAL-ID	Local identifier
COMMON-ID	Common identifier
CONTROLPARAMETER-ID	Control parameter identifier
DATA-ID	Data identifier
DATAPACKAGE-ID	Data package identifier
PARAMETER-ID	Parameter identifier
DYN-ID	Dynamically defined identifier within a request and a response

The semantic column is available if service/job or response is selected in the navigation area.

6.2.3.5.6 Upper and lower limit in request view

These two columns contain the maximal resp. minimal values of the parameter – if they are defined in the ODX.

6.2.3.5.7 Special case: variable parameters

Parameters of type END_OF_PDU have a sub-parameter defined in the ECU data. As soon as the runtime objects are initialized (after activating a LogicalLink), the count of sub-parameters may increase. All of the sub-parameters have the same type and the same name. Their values are inserted into the request message one after another.

The values of the parameters can be set as usually in the request view. If you intend to do this by scripting, you can reference the parameters only by their index (starting with 0), as their names are all identical. Referencing by name returns only the 1st parameter in this set.

There is another kind of variable parameters. A parameter of type ddtTable has a structure of sub-parameters depending on another parameter (of type ddtKey) of the same service. When the value of the key parameter is set to a new value, the structure of the table parameter changes.

6.2.3.6 The response view

The response view shows the responses of diagnostic services and jobs. This area is only shown if a service, a job or the DTC element is selected. It contains the result of the last execution of the selected job or service, regardless if it was called by the button in the GUI or by a function call in the test language. Nested sub parameters are shown hierarchically. The response cannot be manipulated.

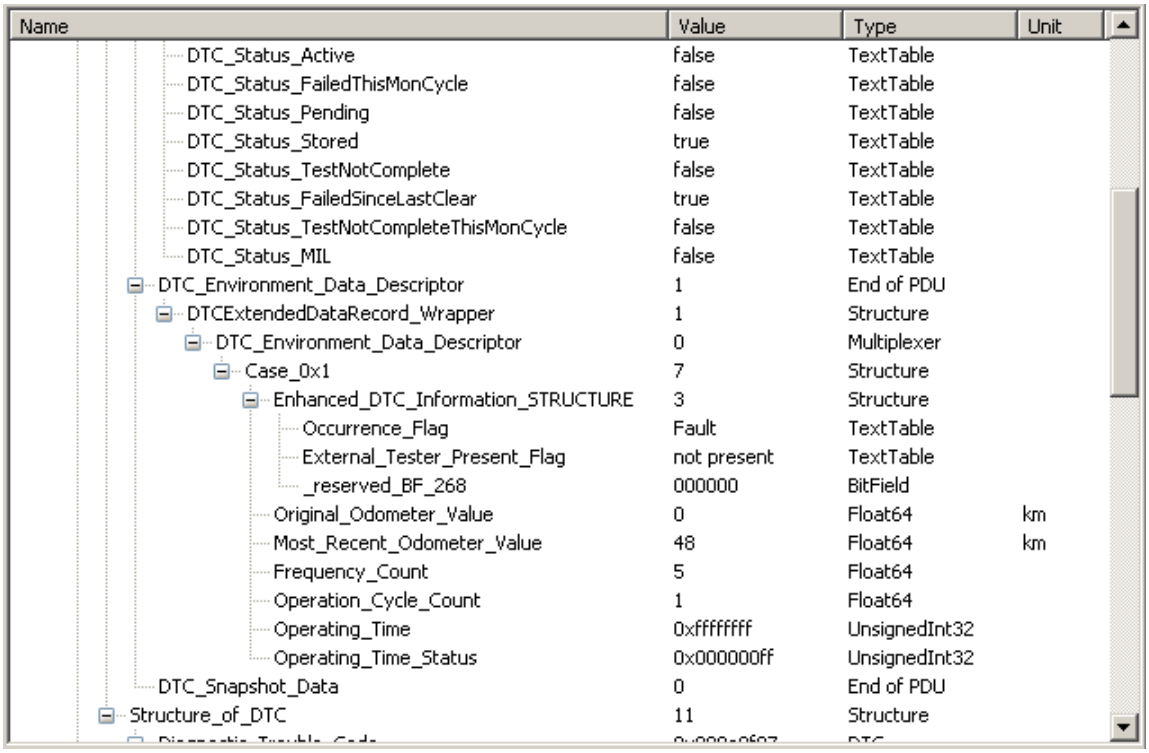


Figure 498 – The response view

Display filters

The response parameters can be filtered the same way as parameters in the request view (see chapter ***6.2.3.5 The request view***)

In case the ECU delivers no valid value, instead of the value of the response parameter the cause of the problem (ValueRangeInfo in the MCD-3 D standard) will be displayed in red:

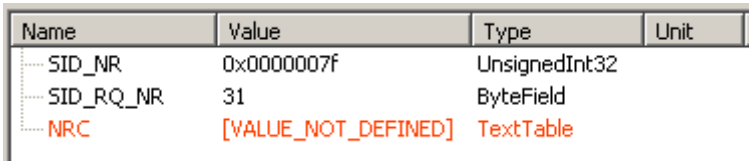


Figure 499 – display of an invalid response value

The following enum values may occur for ValueRangeInfo:

Predefined value	**Description**
vriValueNotDefined	Value not defined
vriValueNotAvailable	Value not available in range
vriValueNotValid	Value is not invalid
vriValueNotInitialized	Value was not set and there is no default value defined
vriCodedToPhysicalFailed	The value is not valid. This range info can only occur in the PDU value or the request parameter value in case of coding the PDU from the request parameters or encoding the PDU value to the request parameters failed
vriValueValid	Value is valid

The response view of the DTCs element differs from this structure: it contains texts and status information in an extended table.

The context menu of the response view contains commands for copying one cell, one line or several selected lines into the clipboard.

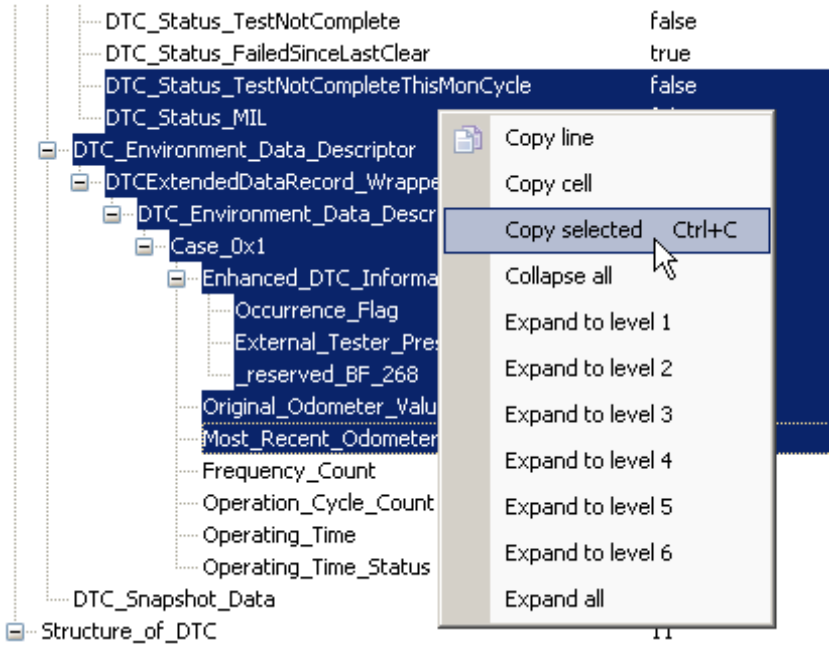


Figure 500 – The context menu of the response view

When the view contains tree-like structured data, the whole tree can be expanded to a specific level. Expanding and collapsing can be done also with hotkey Ctrl-PLUS and Ctrl-MINUS (the + and – keys in the numeric key pad).

6.2.3.6.1 The result of an execution

Service execution result is organized hierarchically.

- A service execution always returns one result.
- One result contains several responses, one response for each ECU which responded. All responses are displayed in the data area.
- One response contains a list of response parameters. The parameters of one response are listed as one block in the data area. Parameters of different responses are separated by an empty line.
- One response parameter of type ddtField or ddtStructure can contain a list of child response parameters by itself.
- Index in data area shows the hierarchical construction of the response parameters.
- The result may contain an error. If so, the error is displayed in the communication view (see chapter ***6.2.3.7 Communication view***).

The service result has the properties service state and service type. For details about the state and type (see chapter ***6.2.3.6.1.1 The result status and result type***)

6.2.3.6.1.1 The result status and result type

Service result state enumerated values:

Predefined value	Description
esAllFailed	Result is empty; none of the addressed ECUs did respond.
esAllInvalidResponse	Result has a response with the error eRT_INVALID_RESPONSE with an empty collection of response parameters.
esAllNegative	All addressed ECUs returned a negative response.
esAllPositive	Result contains only positive answers, one from each addressed ECU.
esCanceledDuringExecution	Execution has started, canceled and removed from queue.
esCanceledFromQueue	At least one of the addressed ECUs did not respond, there is an eCOM_BUS_ERROR in the result.
esFailed	At least one ECU send a strange response, the result contains an empty response for each of those.
esNegative	All ECUs did respond, at least one ECU returned a negative response.
esUnknown	The state cannot be defined due to an unexpected error. Service result type enumerated values.

Service result type enumerated values:

Predefined value	Description
srtResponse	Result contains response information.
srtRequest	Result contains request information.
srtRequestAndResponse	Result contains request and response information.
srtUnknown	Type is undefined.

6.2.3.7 Communication view

The communication view is located in the lower right area of the diagnostic tab. The communication between PROVEtech:TA and the ECU is shown here. The view shows the latest 100 messages. One message consists of a request and a response. A complete message is shown hierarchically. The request is tinted green and the response blue. In

case the ECU responds with a negative response the message is tinted orange. If PROVEtech:TA is not able to establish a communication with the ECU at all then the message is tinted completely red.

During communication with the ECU, status- and error messages are logged in this window.

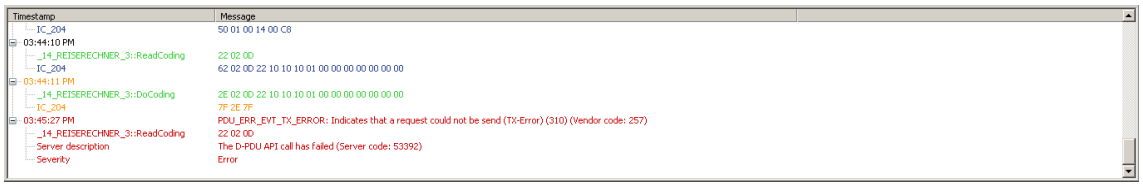


Figure 501 – The communication view

Interaction from context menu, toolbar as well as interaction issued by a test script is logged in the output area.

The byte stream sent to the ECU or received from ECU is logged in hexadecimal format. This information can be displayed only for executed services, but not for jobs. Also temporary messages (like a BUSY message) will not be shown here. To get a complete trace of the CAN messages, please use a message window in the workpage.

You can copy the currently marked line of the current output area contents to the clipboard by using Ctrl+C or Ctrl+Insert.

It is possible to perform different copy actions via the context menu of the communication view or to delete all current entries in it. It is possible to copy one line, one particular cell, a complete command or the complete current entries.

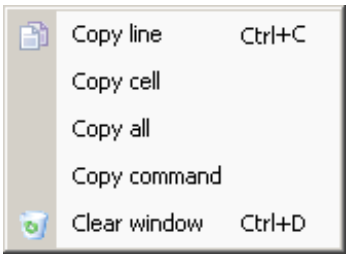


Figure 502 – The context menu of the communication view

6.2.3.8 The ECU element

If an ECU is selected in the navigation area the request view shows the settings for that ECU that can be adjusted by the user.

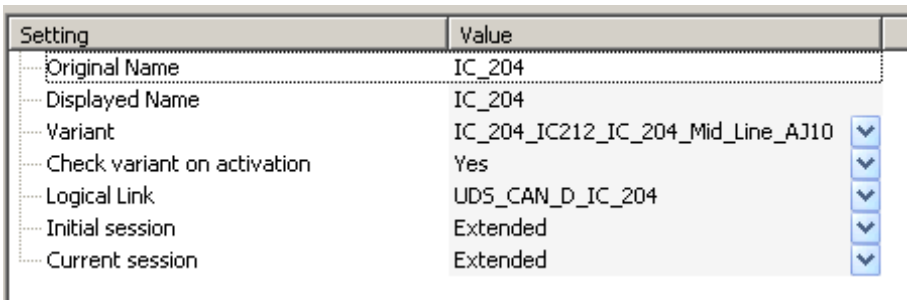


Figure 503 – ECU settings

6.2.3.8.1 The setting Alias

The setting `Alias` is a user defined name for the ECU. The test language uses the `Alias` to determine the correct ECU. Therefore the all aliases must be unique in one project.

6.2.3.8.2 Variant identification and selection

The ODX file contains at least the basic variant (<BASE-VARIANT>) and it can contain additionally several ECU variants (<ECU-VARIANTS>). Via the setting `Variant` the user can select any ECU variant that is defined in the ODX file. Generally the selected variant should match with the active variant of the ECU because only in this case it can be guaranteed that the ECU is capable of all the services and jobs from the ODX data. In case the selected variant and the active variant on the ECU do not match and the ECU does not know the service it will return an error 0x7F. To make sure that the variants match there is the setting `Check variant on activation`.

6.2.3.8.3 Check variant on activation

This setting possesses three options. The first one is `Yes`. In this case PROVEtech:TA checks before executing each action if the variant is correct. If this is not the case the variant is set automatically to the active variant of the ECU. Furthermore a message box will pop up and ask to retry the latest action due to the change of the variant. If no adequate variant is found among all the variants in the ODX data an error message will show up and the action will not be executed. The second option is `No`. If this option is set then no check is done and the action is executed with the variant that is currently set in the setting `Variant`. The third option is `Yes, but ignore if detection fails`. This means that the variant is always checked. But in case that it is not possible for PROVEtech:TA to set the correct variant it will execute the action non the less.

If the variant changes in PROVEtech:TA then the elements beneath the ECU are reloaded in the navigation area. An active ECU is automatically deactivated.

6.2.3.8.4 The attribute Logical Link

The logical link describes the communication path to the ECU. If there is more than one way to communicate with the ECU it is selected with this setting. For example an ECU can communicate both via CAN and Ethernet (DoIP). You can select this by setting the value of `LogicalLink`.

For the selected logical link also the interface (D-PDU API) can be configured, e.g. if CAN1 or CAN2 is to be used. Find a full description of this in the chapter about navigation areas ***6.2.3.4.1.1 ECU***.

6.2.3.8.5 Initial session and current session

In diagnostics there are several kinds of session. Depending on the needs of the user the correct one must be selected. For working with PROVEtech:TA the sessions *Default* and *Extended* are relevant.. Typically the extended session is used because it is the most common session in development. The `Initial Session` value is stored in the file **DiagnosticsMCD3D.ini** and it determines the session type at start of the program or after loading a configuration, i.e. it is copied into the `Current session`.

The `Current session` setting determines the actual way of activating the ECU. It can be changed also for an activated ECU. This setting is not stored in the configuration file.

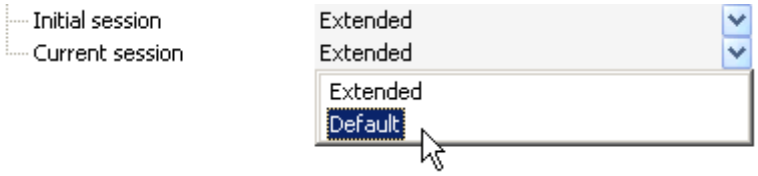


Figure 504 – Changing the diagnostic session at runtime

Further sessions can be selected via the execution of specific services.

It is not possible to change this setting via the test language. Via the test language it is possible to adjust the current (runtime) session, but not the initial session.

6.2.3.8.6 The context menu on ECU level

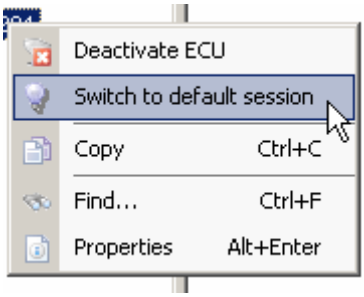


Figure 505 – Context menu on ECU level

The context allows switching the activation state of an ECU. Furthermore the runtime session is set here to `Default` or to `Extended` (see chapter ***6.2.3.8.5 Initial session***). It is also possible to copy the text of the ECU element (which is the alias of the ECU) and read the properties – names, version, description and the complete list of all defined DTCs:

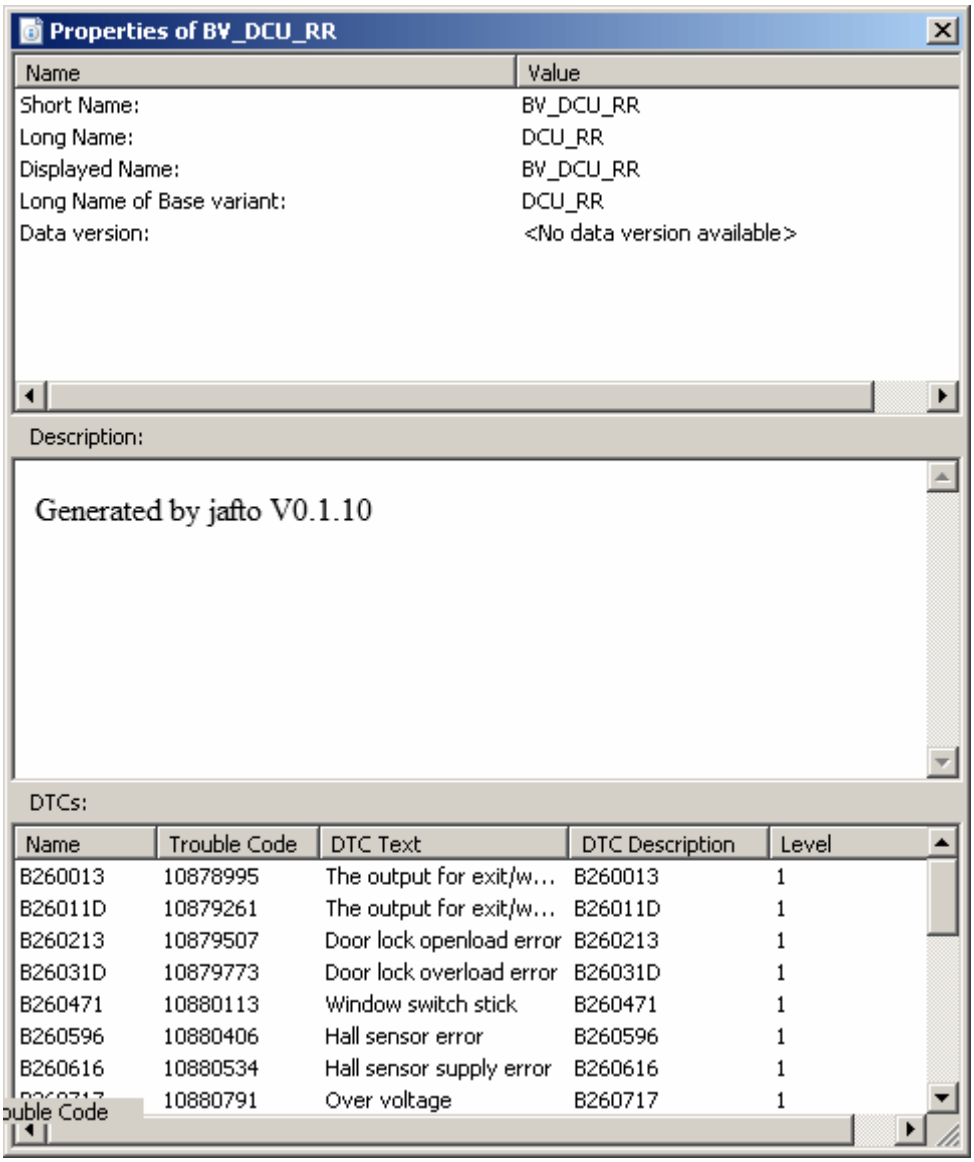


Figure 506 – The ECU property window

6.2.3.9 The coding

When the ODX data matches some specific criteria PROVEtech:TA enables a special element just for coding. This coding element simplifies the work for the user by abstracting and synchronizing three different services for each coding domain. Furthermore the color of the symbol in the navigation area signals if the coding data shown in PROVEtech:TA is matching the one in the ECU. The synchronization mark (sync flag) appears green ( ) in case the coding is synchronous and red ( ) in case the data is asynchronous. The synchronization of the coding data takes place via a reading or writing action. PROVEtech:TA stores the last state of each read or written coding domain

and sets the synchronization mark accordingly. In case a value is changed and later set back to the original value the red icon changes back to a green one.

Important: A green flag after writing indicates that the ECU accepted the message without any error message. It does not mean however that the ECU actually internally accepted the new value. This can be checked only by reading the coding after writing. This inconsistency is a relatively popular error in ECUs (or in the ECU data).

6.2.3.9.1 Prerequisites for the coding

There are prerequisites in the ECU data that have to be fulfilled to enable the coding in PROVEtech:TA. Overall, three concerted services must exist. There has to be one service for each semantic, VARCODEREADDUMP, VARCODEWRITEDUMP and VARIANTCODINGWRITE. Except for the suffix the names of the three services must match.

The subsequent table shows the necessary suffixes for the respective semantic.

Semantic	**Postfix**
VARCODEREADDUMP	_READ_DUMP
VARCODEWRITEDUMP	_WRITE_DUMP
VARIANTCODINGWRITE	_WRITE

The parameters of the WRITE services must comply with the following conventions:

- The name of the 1st parameter representing the 1st byte of the message must be “SID_RQ”.
- The name of the 2nd parameter must be “RecordDataIdentifier” or “LID”.
- The WRITE_DUMP service must have a bytefield parameter named “DataRecord”, representing the coding string (in TA this parameter is named “CodeString”). The WRITE service can have a variable number of parameters (representing the coding fragments)
- After those parameters there may be further optional parameters not interpreted as coding data. These parameters must have the following names: “Tester_ID”, “TesterID” or “SCNNumber”.
- The READ_DUMP service returns as a result a structure containing the coding string as a “DataRecord” parameter, corresponding to the “DataRecord” (“CodeString”) parameter of the WRITE_DUMP service

Only if these criteria are fulfilled PROVEtech:TA recognizes the togetherness of the services and creates a coding domain.

6.2.3.9.2 The VCD qualifier

It happens that for coding old legacy files must be used (AED files created for the usage with CAESAR). These files use the notion of “VCD qualifier”, which is the name of the domain with the prefix “VCD_”.

The ASAM standard does not define “VCD qualifiers”, but this can be included in the ECU data as so-called “SpecialDataGroup”. It becomes even more tricky when the VCD qualifier in the SpecialDataGroup does not correspond to the name of the domain.

These errors in the data are quite popular. Therefore PROVEtech:TA supports the usage of both: domain name with prefix “VCD_” or the VCD qualifier defined in the SpecialDataGroup. To define which should be used when reading and writing AED files, set the parameter CodingDomainsEnabled in the file DiagnosticsMCD3D.INI to 1 (names with prefix) or 2 (VCD qualifier from SpecialDataGroup). See also chapter ***6.2.3.9.4 The AED file dialog.***

6.2.3.9.3 The context menus of the coding

It is always possible to reset the synchronization mark to asynchronous. This is only possible for the complete ECU and not for one specific coding domain. Consequently the option `Reset sync flag` is only offered in the context menu of the coding domains level.

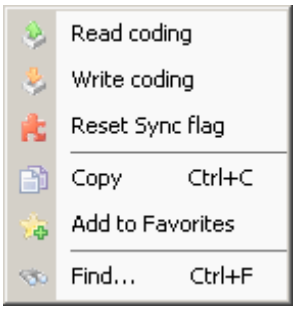


Figure 507 – Context menu on domains level

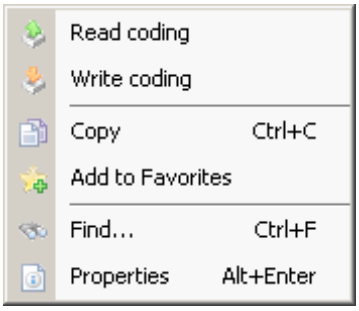


Figure 508 – Context menu on specific domain

A further attribute of the coding view is that it has no response view, even though services are executed in the background. This is because the values of a result are immediately written to the request parameters in the request view.

On the coding domains level, the request view contains all domains and their coding string. The buttons `Read coding`, `Write coding` and `Save AED / Save ECX` activate the reading, writing or storing in a AED/ECX file of all domains. If some of the domains in the request view are selected (blue background), the button cation applies to the selected domains only. In this case the labels of the first two buttons changes to `Read selected codings` and `Write selected codings`.

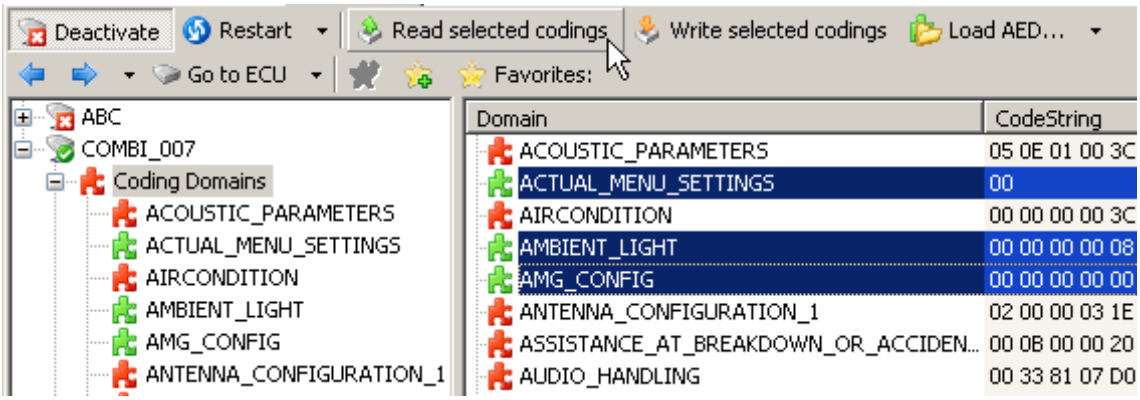


Figure 509 – Request view for coding domains with partially selected domains.

On the coding domain level (a single domain is selected in the navigation area), the request view contains the coding string and all parameters (in case of coding domains parameters are called “fragments”). As for domains, a red or green icon signals if the fragment is equal to the value of the last read or write operation. The domain is synchronized when all her fragments are synchronized too.



Figure 510 – Request view for a coding domain with partially changed fragments.

As the *short name* sometimes contains less information than the *long name*, you can switch between those two for fragments.

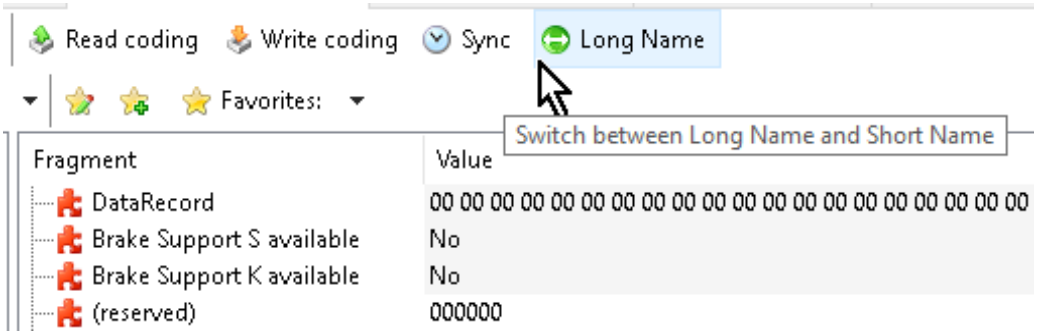


Figure 511 – switching between long and short name for fragments

6.2.3.9.4 The AED file dialog

The AED file dialog is accessed via the button `Load AED...` and `Save AED...` that is placed on the coding domains level. Via the AED file dialog it is possible either to save or to load coding data that correspond to the AED format. These files can be stored and exchanged between different installations of PROVEtech:TA. It is also possible to exchange the AED data with other programs if the other programs support the AED file format.

The AED file can be stored as usually in the file system, or attached as additional document to a test item in the Test Manager. For this, simply select the tab “File System” or “Test Manager” in the file open dialog.



It should be ensured, that in case of a coding data exchange the variant of the ECUs are all the same. Only in this case a successful data exchange is guaranteed.

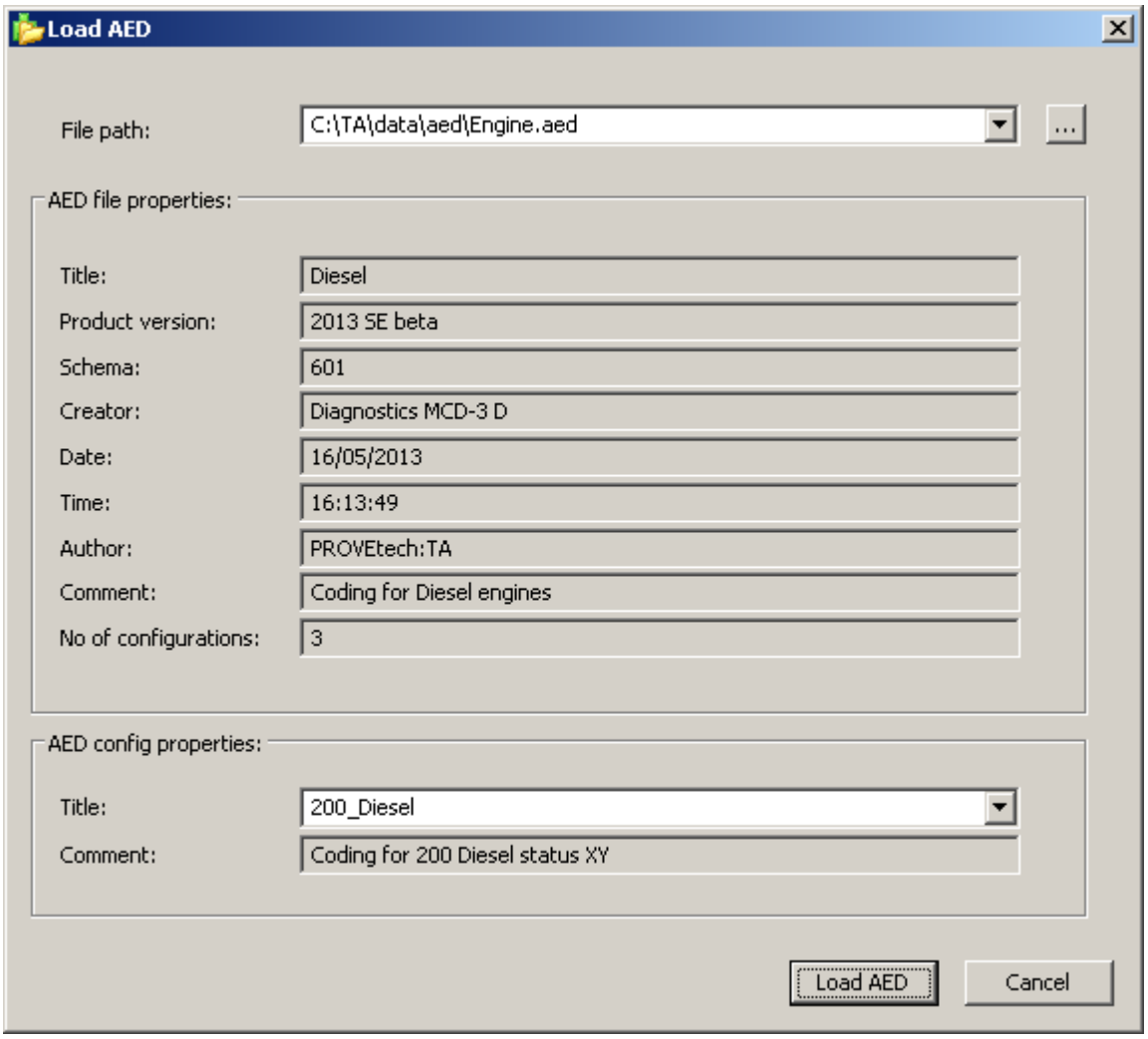


Figure 512 – The AED dialog for loading an AED file

The coding information from an AED file is immediately transferred to the writable fields of the request view after clicking on `Load AED`. During loading, the entries are associated with the proper domain by comparing with the name (preceded by "VCD_") or with the "VCD qualifier". See chapter ***6.2.3.9.2 The VCD qualifier*** for more details. If errors occur during this process entries will be made to the PROVEtech:TA Log (see chapter ***3.6.1 PROVEtech:TA Log***). After loading the information the synchronization marks are also updated.

After loading data from an AED file, the contained domains (it is possible that the AED contains only a sub-set of all domains) will be selected in the request view (see Figure 509). Clicking the button `Write selected codings` will cause writing the new values of the selected domains into the ECU. The non-selected domains stay unchanged.

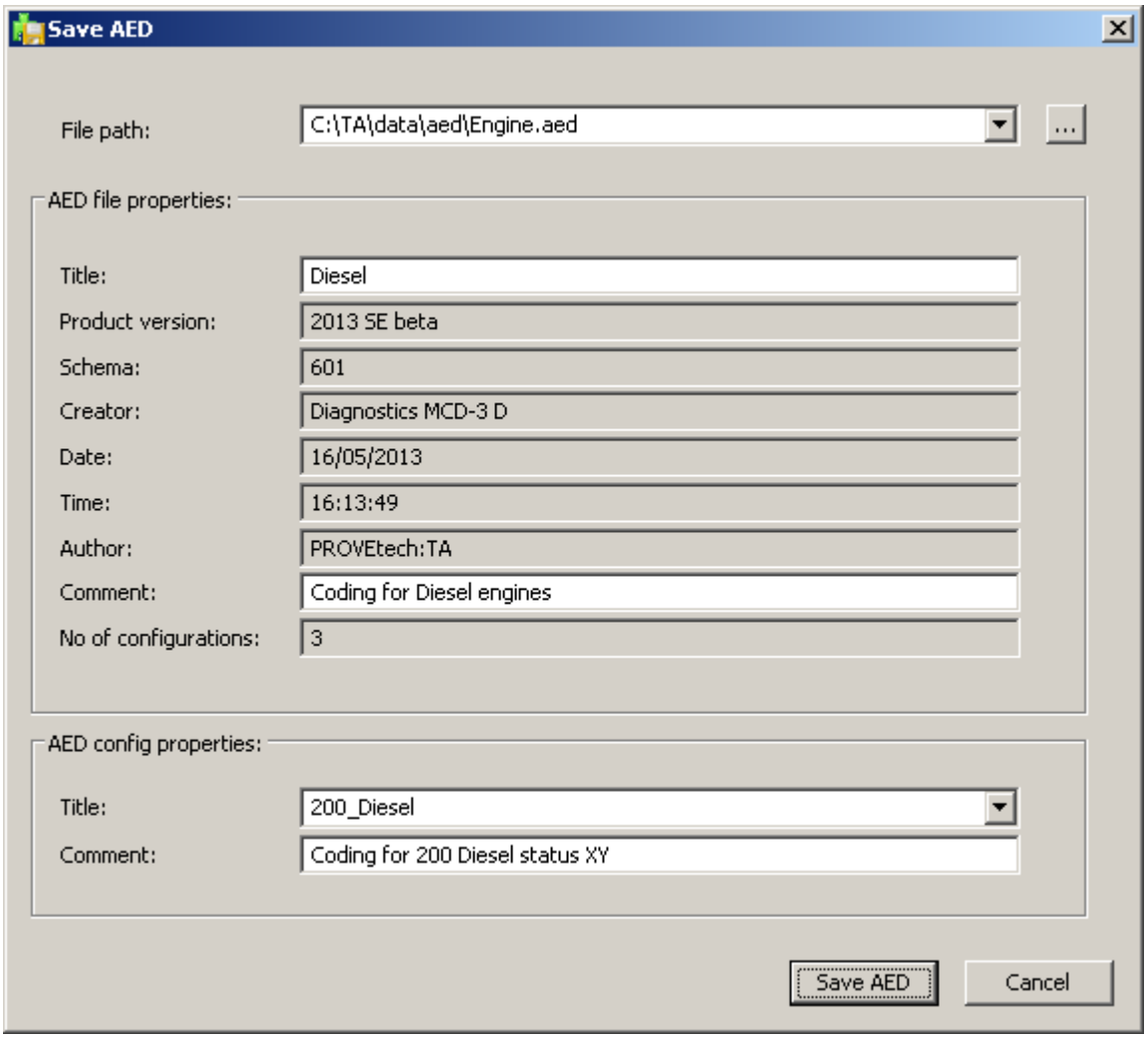


Figure 513 – The AED file dialog for saving an AED file

When saving an AED file (to the database or to the file system) all current information from the request view are written to the AED file. The entries are named either by the domain name (preceded with “VCD_”) or by the “VCD qualifier”, depending on the same settings as for reading.

Each AED file can have several configurations. Via the combo box Title in the AED config properties it is possible to choose which configuration shall be loaded or in case the dialog is opened for saving a name for the configuration is specified here.

A common mistake with aed files is that the variants do not match. This leads to errors in many cases because the coding string length within the variants may differ.

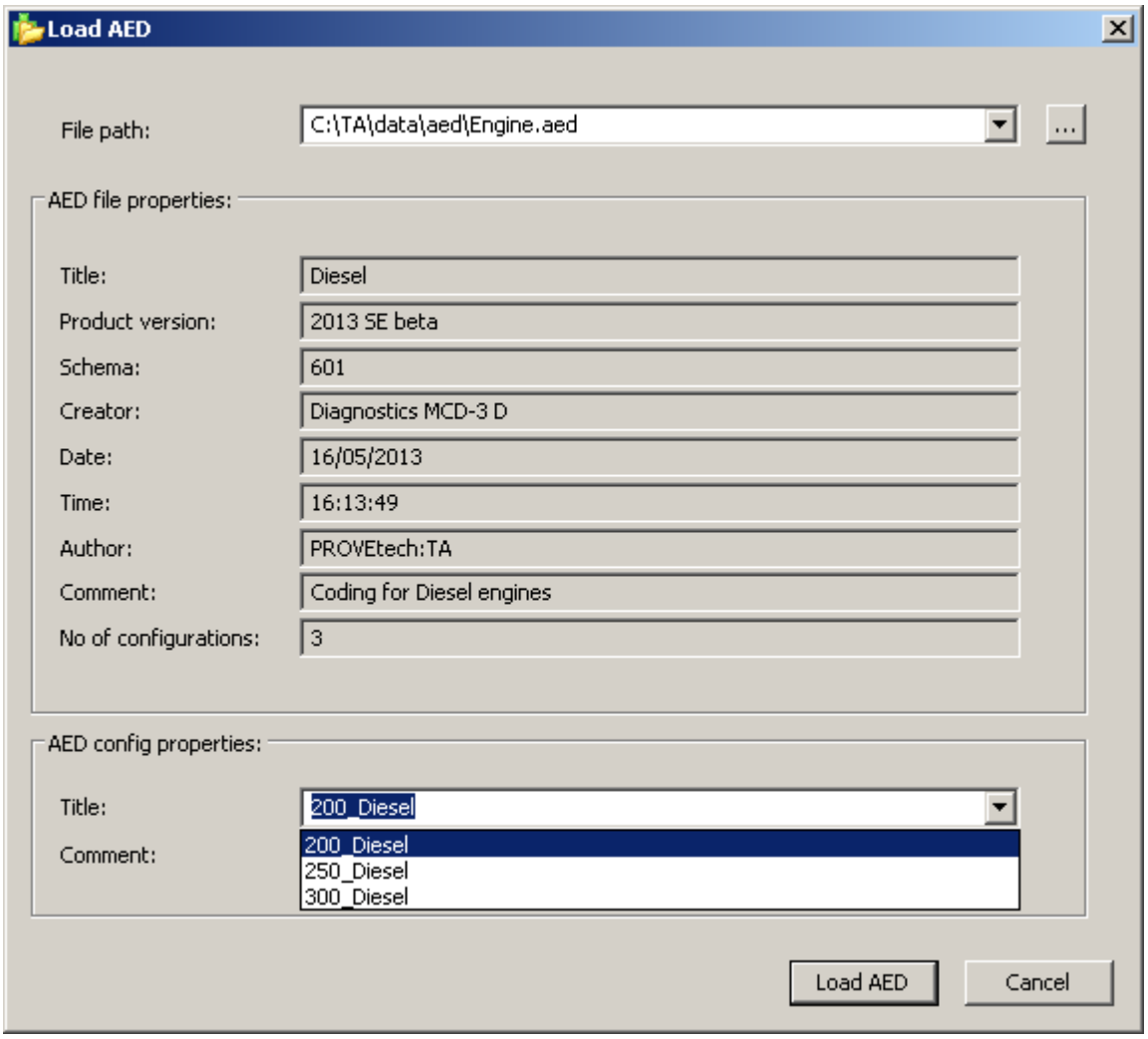


Figure 514 – An AED file with several configurations

6.2.3.9.5 The ECX file dialog

The ECX file dialog offers the possibility to load coding data from ECX files or to save the current coding data to a file in the ECX format.

The ECX format is based on XML format, and its definition is based on a well-defined XSD (XML Schema Definition). The ECX format allows storing several coding sets, containing different coding data information.

It could be possible that the user defines for an ECU several variants, containing similar coding domain sets. In order to increase the ECU development variability, PROVEtech:TA allows to load ECX coding domains set corresponding to different ECU Variants and even to different ECU. Nevertheless, in order to avoid incorrect coding set load/overwrite, PROVEtech:TA provides visual warnings and specific dialog message boxes to ensure that the user is aware about the load/overwrite process. In addition, the

PROVEtech:TA Log will display the incompatible coding sets in the target ECU. For further information, please check the documentation in the following subsections.

You find more details in the following chapters:
6.2.3.9.5.1 Loading ECX files and ***6.2.3.9.5.2 Saving ECX files***.

6.2.3.9.5.1 Loading ECX files

To load an ECX file first specify the path to the file either in the file system or in the database by freely entering the path or selecting it in the combo box `File path`. The

button labeled as ... that is located next to this combo box opens the according file selection dialog (see chapter ***3.1 The file selection dialog***).

The ECX file can be stored as usually in the file system, or attached as additional document to a test item in the Test Manager. For this, simply select the tab “File System” or “Test Manager” in the file open dialog.

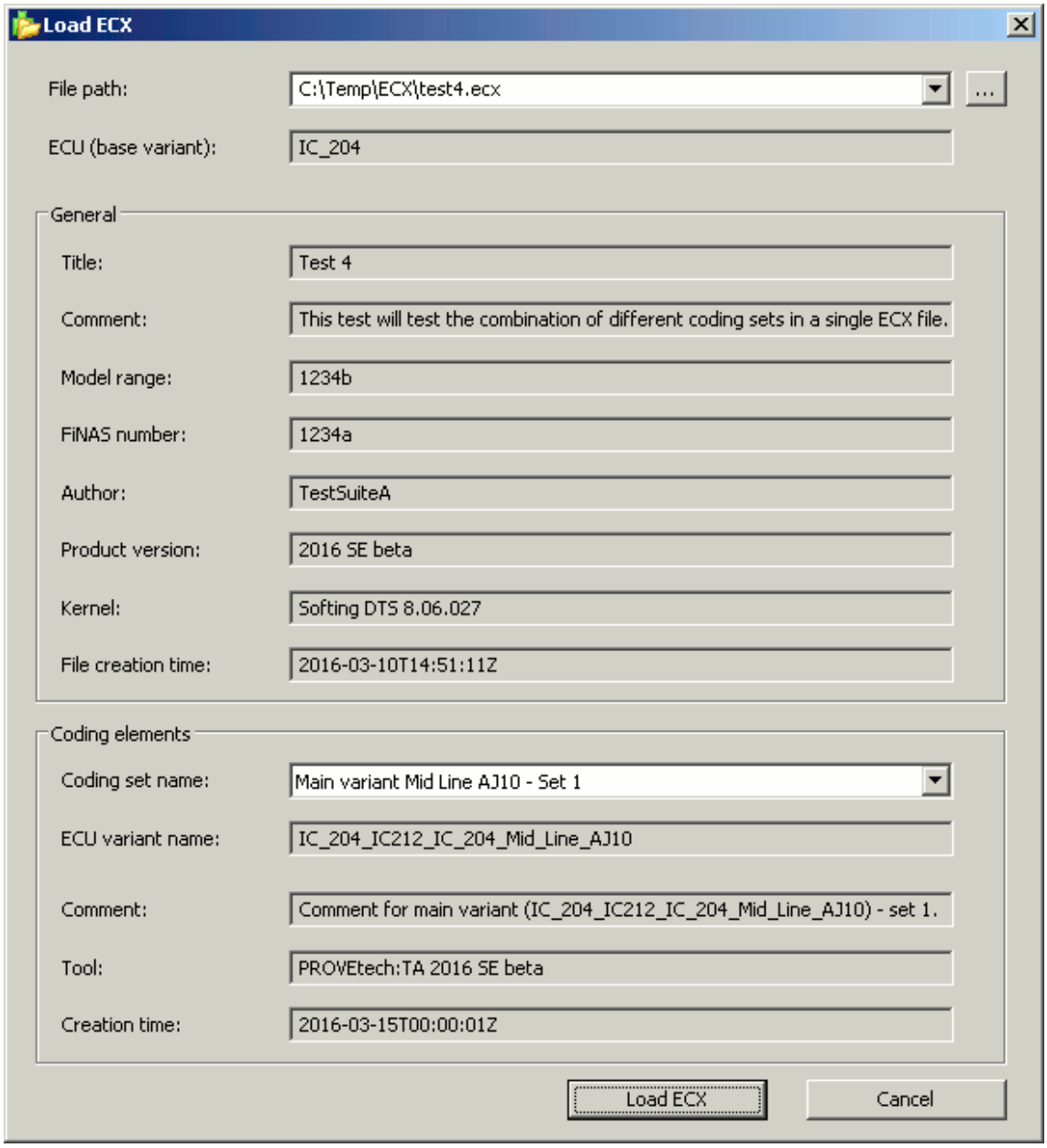


Figure 515 – The ECX dialog for loading an ECX file.

PROVEtech:TA stores the list of the latest ECX files used for load/store. The list can be directly accessed by opening the combo box of the File path.

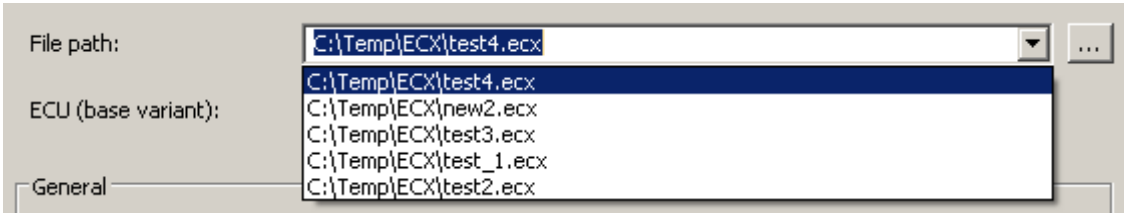


Figure 516 – The file path combo box in ECX dialog

One ECX file may contain several different coding sets. When loading these coding sets can be selected and loaded to the coding area of PROVEtech:TA.

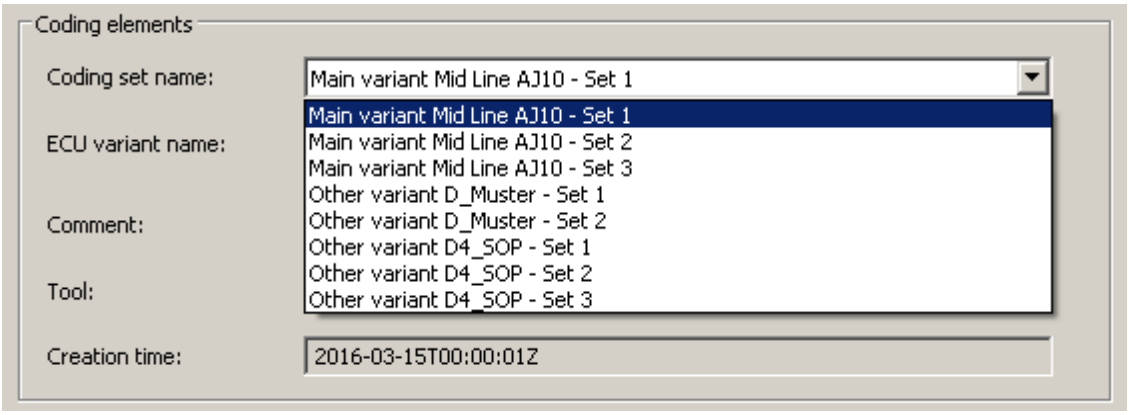


Figure 517 – The coding set name combo box in ECX dialog

In case, the selected ECX file refers to another ECU, this is displayed in the dialog displaying a warning symbol, and the name of the current selected ECU.

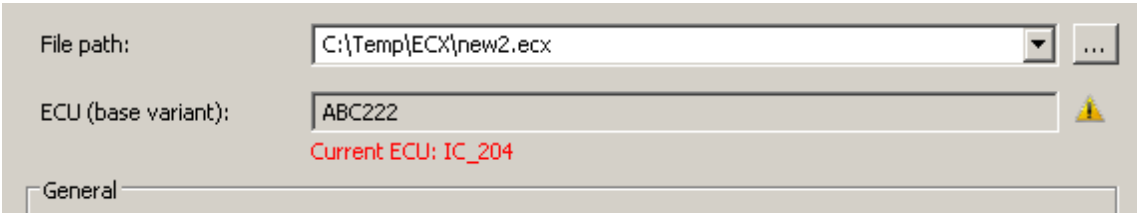


Figure 518 – Warning notification for a different ECU in selected ECX file

In case of trying to load an ECX file associated to a different ECU, a confirmation message box request for confirmation after clicking on `Load ECX` button.

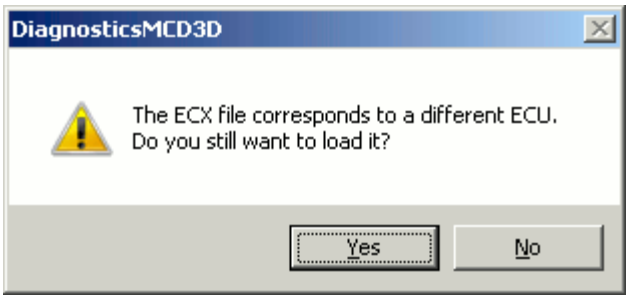


Figure 519 – Warning message to confirm the load of an ECX file associated to a different ECU

Similarly, the same warning and red text is displayed when the selected ECX coding set corresponds to another ECU Variant.

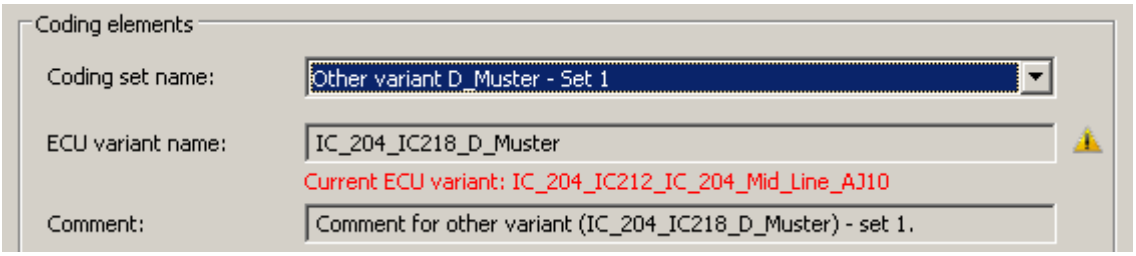


Figure 520 – Warning notification for a different ECU variant in selected coding set

The confirmation message box notifies the user after clicking on `Load ECX` button.

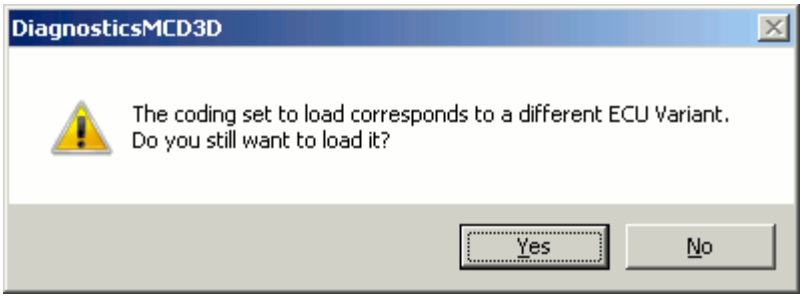


Figure 521 – Warning message to confirm the load of a different ECU variant

After loading data from an ECX file, the contained domains (it is possible that the ECX contains only a sub-set of all domains) will be selected in the request view (see Figure 509). Clicking the button `Write selected codings` will cause writing the new values of the selected domains into the ECU. The non-selected domains stay unchanged.

6.2.3.9.5.2 Saving ECX files

When saving an ECX file different settings can be made. The two obligatory fields are the file name and the coding set name. Without this information no ECX file can be stored.

The coding set contains data of coding domains and some other information (like name of the coding set, ECU, variant etc.) If just some of the domains in the request view are selected, the set includes only the selected domains.

The ECX file can be stored as usually in the file system, or attached as additional document to a test item in the Test Manager. For this, simply select the tab “File System” or “Test Manager” in the file open dialog.

To Save an ECX file, first specify the path to the file either in the file system or in the database by freely entering the path or selecting it in the combo box `File path`. The button labeled with ... that is located next to this combo box opens the according file selection dialog (see chapter ***3.1 The file selection dialog***).

The ECX file can correspond to either an existing or a new ECX file.

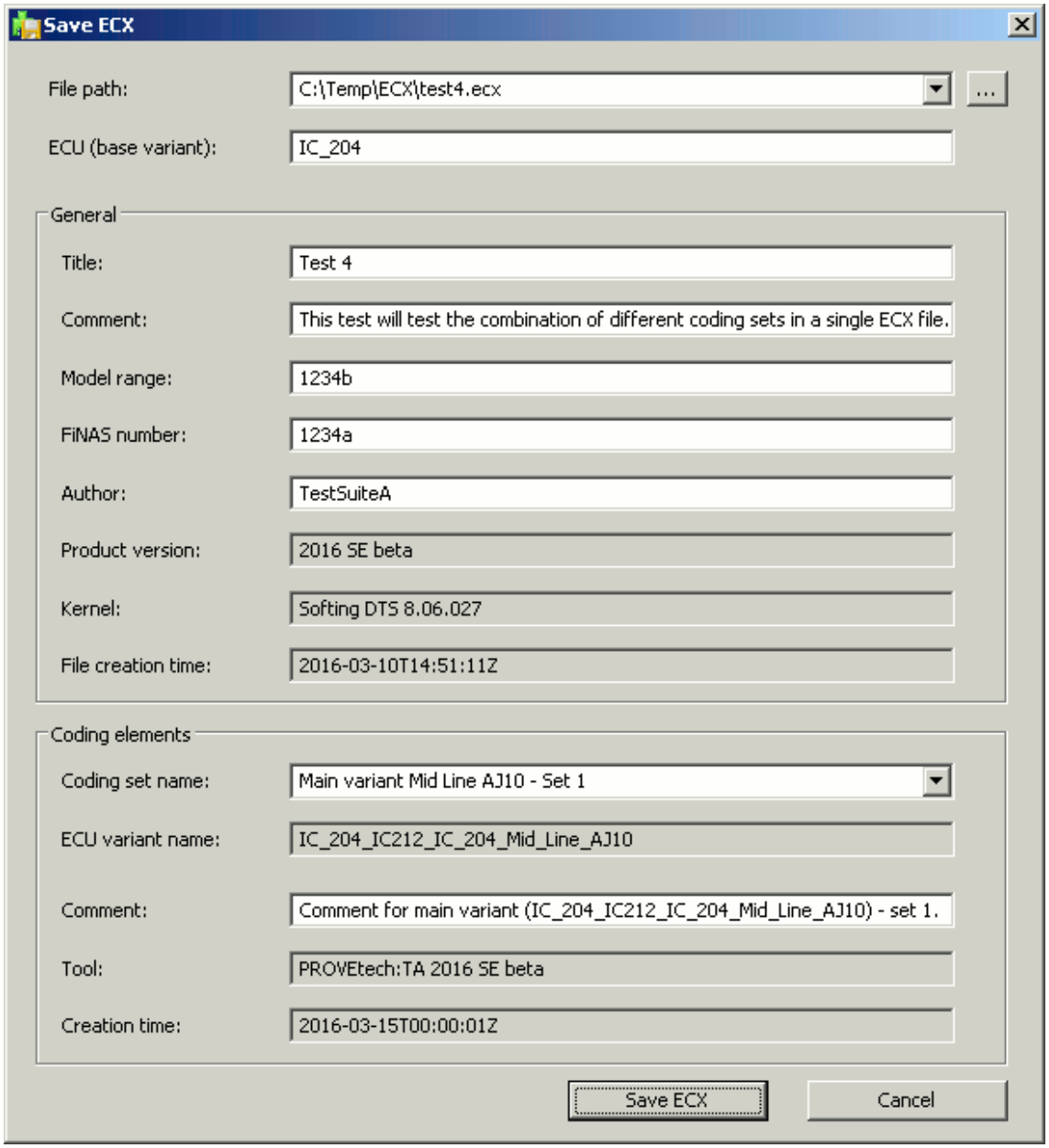


Figure 522 – The ECX dialog for modifying an existing ECX file

In case the ECX file corresponds to another ECU, this is displayed below the ECU text field.

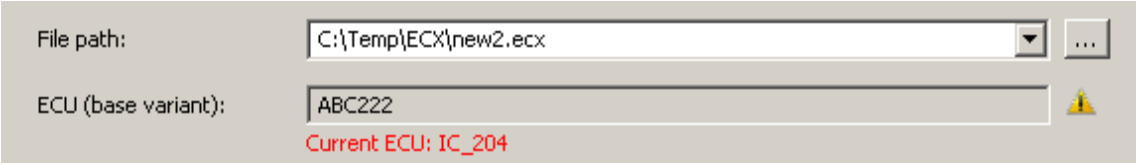


Figure 523 – Warning notification for a different ECU in selected ECX file

When the user selects in `File path` a new file, some fields are generated automatically.

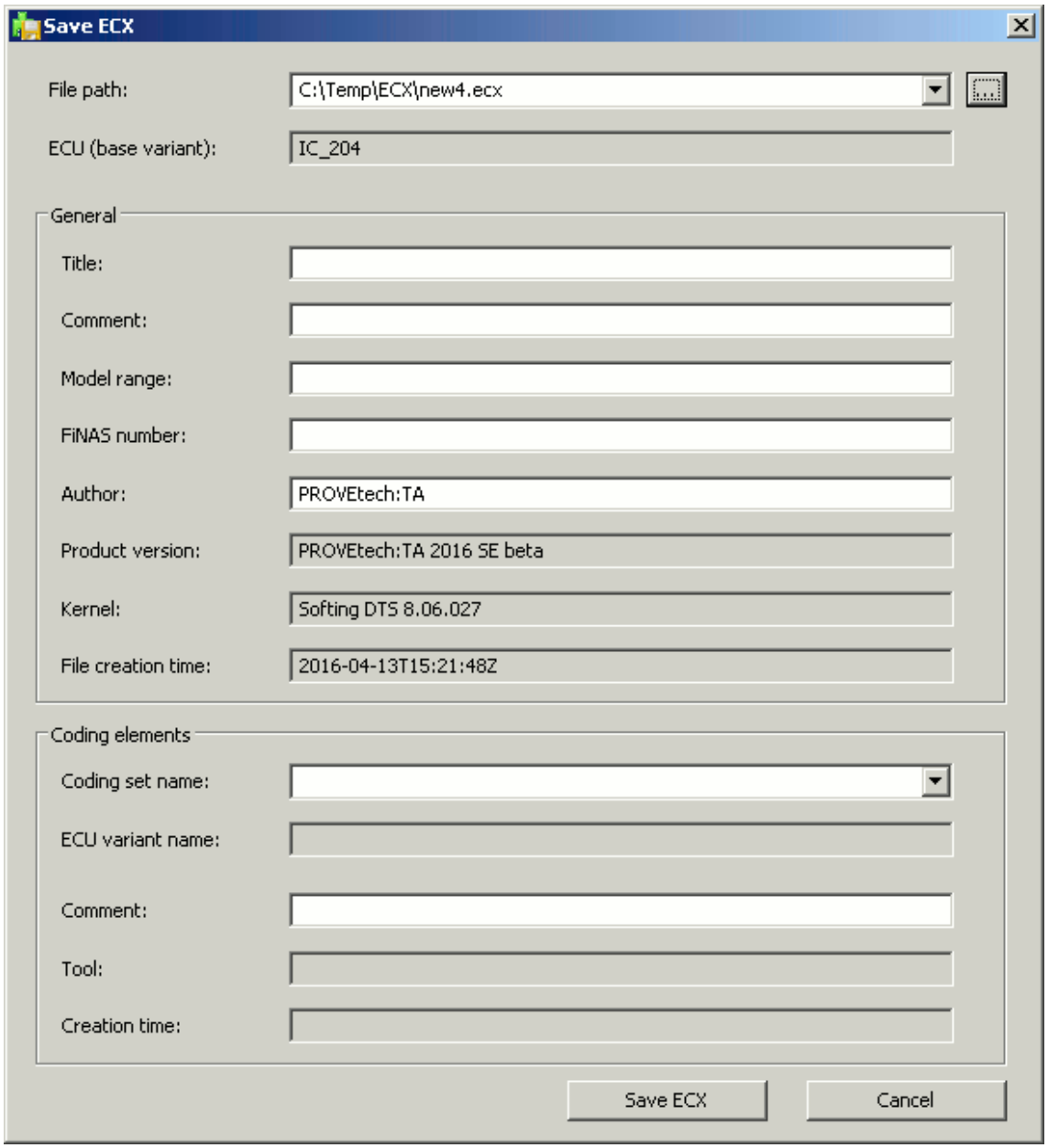


Figure 524 – The ECX dialog for saving a new ECX file

After filling/modifying the `General` section, the user can select either a new coding set, or specify an existing coding set, to overwrite it.

For creating a new coding set, type the coding set name in the combo box.

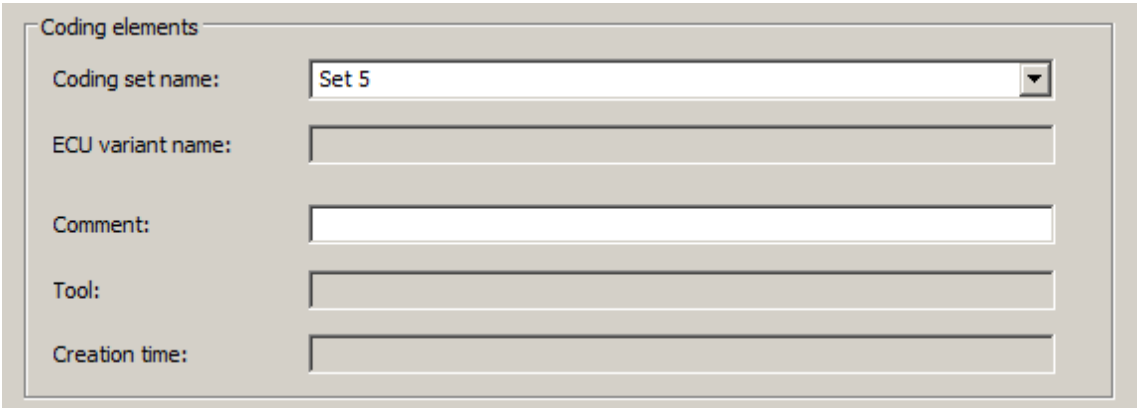


Figure 525 – Editing a coding set name

If there are other coding set with the same coding set name, the existing coding set data is loaded in the dialog while typing.

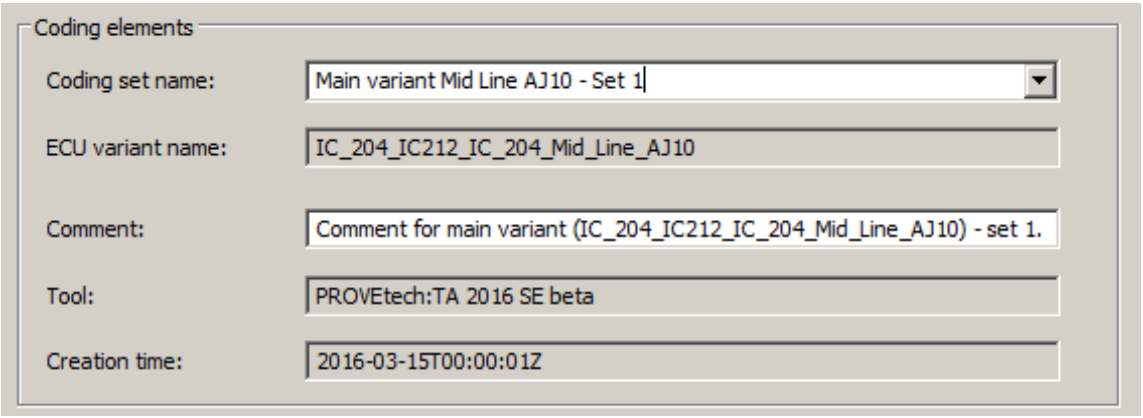


Figure 526 – Modification of an existing coding set

For new coding sets, after pressing Tab button, the `ECU variant name`, `Tool` and `Creation time` fields are automatically generated.

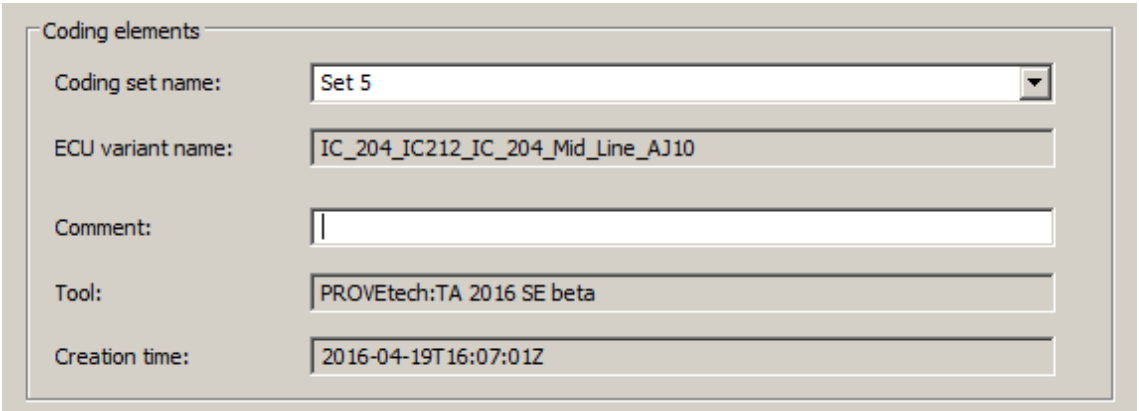


Figure 527 – Adding the comment for a new coding set

It is also possible to overwrite the coding element of an existing coding set. The ECU Variant of the coding set is checked, in order to ensure that the user is not overwriting data associated to a coding set corresponding to another ECU Variant. In such case, a visual warning is displayed.

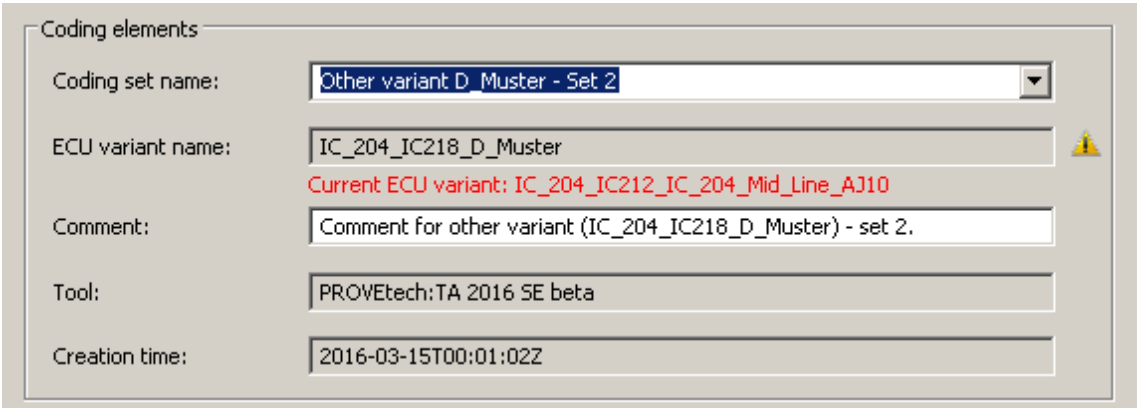


Figure 528 – Warning for overwriting of an existing coding set encoded for a different ECU variant

The edit box `ECU Variant name` shows the currently selected ECU variant of PROVEtech:TA. The coding strings of the selected ECU variant will be the base for overwriting coding strings matching the same coding service name in the target ECU variant. Of course, it is possible to overwrite coding sets this way. You should keep in mind that a later ECX load will only work with the new selected variant, because the coding strings can differ from variant to variant.

In case, the user want to overwrite an existing coding set, PROVEtech:TA requests for confirmation on overwriting just the coding set fields, or the coding set fields and coding strings.

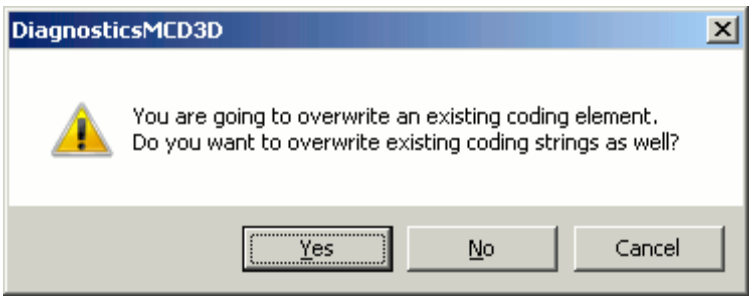


Figure 529 – Warning message to confirm the overwrite of an existing coding element

The operation of saving an ECU in an ECX file already containing coding sets of another ECU is not allowed, and an error will arise.

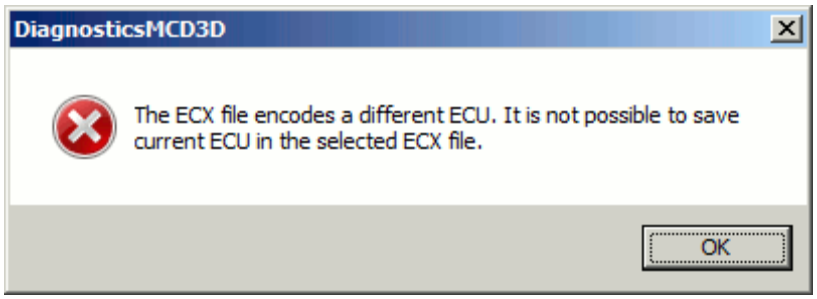


Figure 530 – Error message for trying to overwrite an ECX file encoded for another ECU

6.2.3.9.6 The SGDE file dialog

A more recent file format for storing coding data is SGDE. The file is organized hierarchically: it contains one or more definitions of **Vehicle**, associated with one or more elements of type **EcuRecord**, which is defined for a specific ECU variant and contains elements of type **CodingSet**. One EcuRecord can be associated with more than one Vehicle.

A CodingSet contains a selection (or all) coding domains of the ECU and defines their CodingStrings.

The Vehicle, EcuRecord and CodingSet elements have an unambiguous ID, a name and an optional comment (in case of EcuRecord there is the ECU's variant instead of the comment).

The SGDE dialog window allows to comfortably transfer the coding information of a chosen CodingSet into the ECU. The coding information can be edited manually and stored in a changed file.

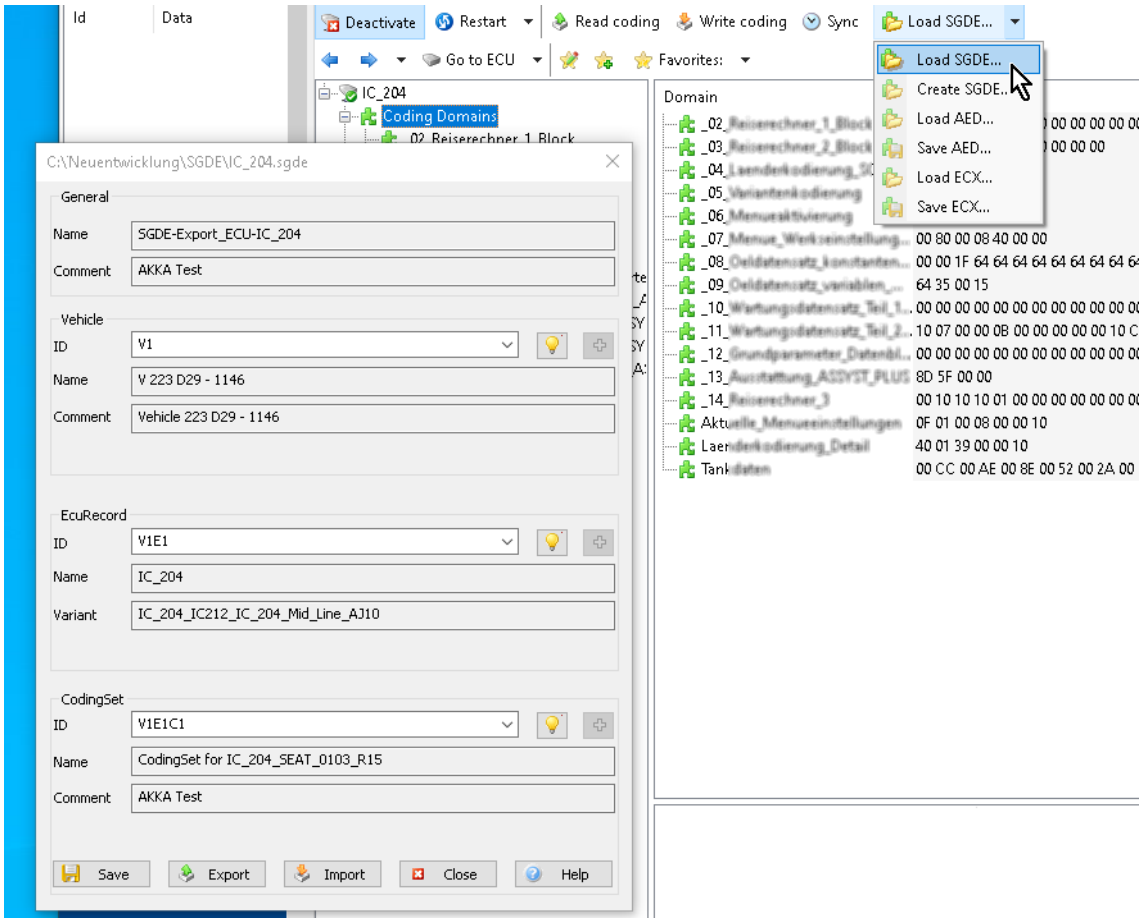


Figure 531 – opening the SGDE dialog window for a specific ECU

Transfering coding informationfrom SGDE into the ECU

To code an ECU, please do the following:

In the navigation area select the item **Coding Domains** of the ECU to be coded. In the toolbar click the item "`Load SGDE...`" to open the file. The SGDE dialog can be freely moved to have free access to the request view with coding domains.

In the window you see all data needed to select the chosen coding data. The SGDE file can contain much more information not relevant for coding. To read a full information of an element, use the lightbulb button beside it.

First select the Vehicle element in the combo box, then one of its EcuRecords. If the variant stored in the EcuRecord differs from the current ECU's variant, a warning will be displayed. In this case using the coding information will not be blocked by the program, but it can happen that the ECU refuses to accept the command.

In the end select the CodingSet of your choice.

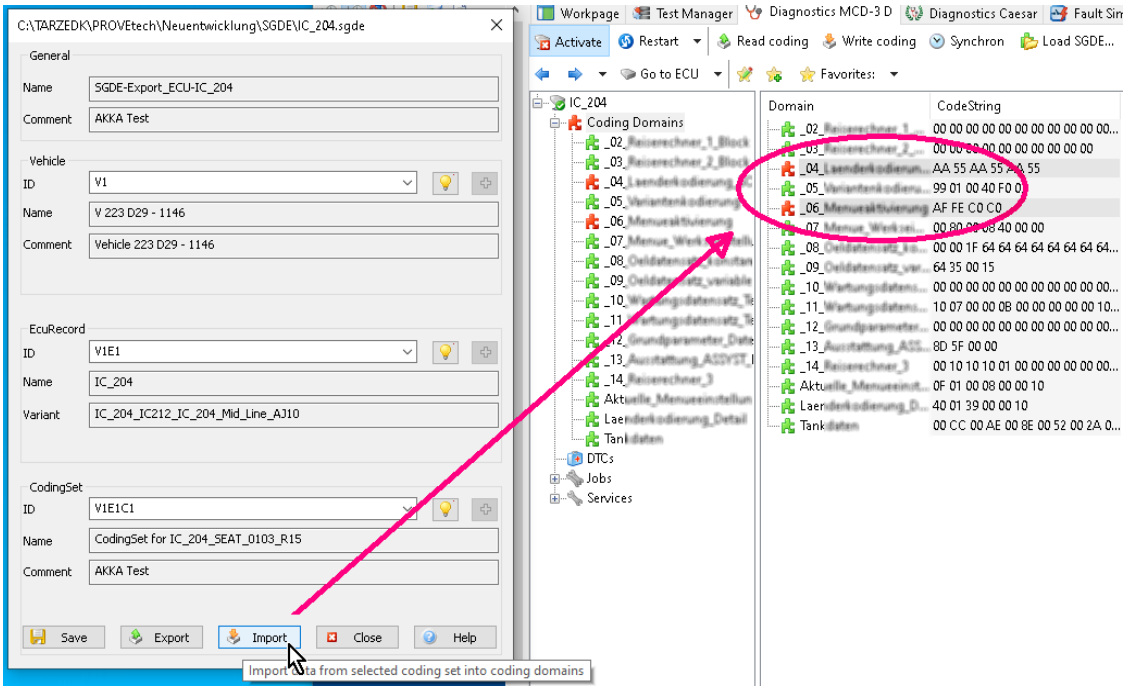


Figure 532 – importing data from file into request view

After clicking "`Import`" the content of the CodingSet will be transferred into the request view. The transferred domains will be selected (grey background in the image above). At this moment, no data has been sent to the ECU yet.

Without closing the SGDE window, you can now change to the main window and edit the coding strings if needed.

After clicking "`Write coding`" the data will be sent to the ECU.

Changing coding data in the SGDE file

If you need to change the coding data, first set all coding domains in the request view as required. In the single coding domain view you can also set the fragments separately.

Next select the Coding Domains item in the navigation area and select the domains of your choice in the request view. If no domains are selected, then all of them will be used.

In the SGDE window click the "`Export`" button. The coding information of the selected coding domains will be exported into the currently selected CodingSet and overwrite its former content.

Instead of overwriting an existing CodingSet, you can add new elements by entering a new (non-existing) ID into the combobox and clicking the "+" button. A new EcuRecord will automatically get the name and variant of the current ECU. A new CodingSet and Vehicle requires at least entering a name. The comment is optional.

To fill the newly created CodingSet, click “Export” to transfer the data from selected domains.

Creating a new SGDE file

A new SGDE file can be created by clicking “Create SGDE” in the pulldown menu. There you can create and fill all elements as mentioned above. Before saving the file, please enter also a name and a comment for the file. After saving it is not possible to change the texts in the SGDE window.

Clicking “Save” will write the content of the window into a SGDE file.

Restrictions

Due to a great complexity of SGDE and of the data structure in PROVEtech:TA it is not possible to adopt all possibilities of SGDE:

It is not possible to code several ECUs in one step. The SGDE window is assigned to one ECU, selected before opening the window. To code other ECUs, the window must be closed and reopened after selecting another ECU.

It is also not possible to delete parts of the SGDE structure using the SGDE window.

It is not possible to enter or edit additional data, not directly used for coding, as Vehicle-Code or PartNumber.

6.2.3.10 The DTCs

Each ECU has a memory, where it stores the malfunctions that occurred during operation. This memory is read via diagnostics with the service 0x19 (UDS protocol) and with the service 0x14 (UDS protocol) this memory is cleaned. PROVEtech:TA offers for the work with this DTC memory a special view. This view is activated with a click on the element "DTCs" in the navigation area. Premises for this view are special jobs for reading and cleaning DTCs.

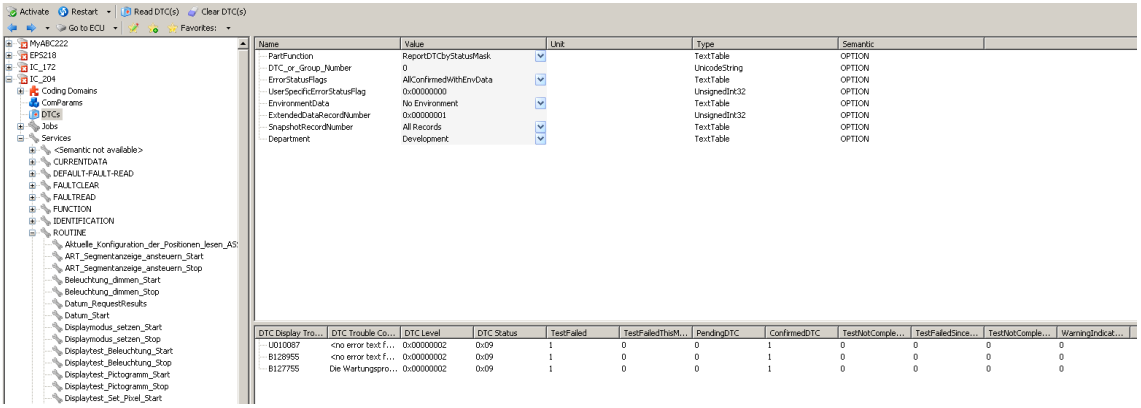


Figure 533 – The DTC view

The request view contains the parameters to set the sub functions for reading the DTC. When clicking on `Read DTC(s)` the service 0x19 is executed with the values that are set in the request view. Furthermore this view contains a button `Clear DTC(s).` Clicking on this button executes the job for clearing the DTCs with the values as they are set at the moment of the execution.

6.2.3.10.1 Reading and clearing DTCs

In case this view does not exist in your configuration reading and clearing DTCs are performed this way:

- Find the service which reads / clears the error codes. By Asam Standard MCD-2 D 2.0.1, these services have the service semantic FAULTREAD and FAULTCLEAR. For details about service semantic, see chapter ***6.2.3.4.1.6 Service semantic***.
- Execute the service (see example in chapter ***6.2.3.4.1.3 Services***).
- The service execution result lists the error codes. The data type ddtDTC is used for error codes.

No special handling is available within MCD-3 D diagnostics for reading/clearing error codes. The functionality is realized by a service or a job.

6.2.3.11 Artificial elements

An artificial element in the MCD-3 D module is an element that is not representing a hierarchical order from the ODX file but the MCD-3 D module itself. For example the semantic of a service or a job is an attribute of the particular service or job. In PROVEtech:TA this attribute is used to group services and jobs with the same semantic. The nodes of services and jobs are beneath another node named after the respective semantic. Consequently this node is an artificial one and has the purpose to create a more convenient overview of services and jobs for the user by grouping them by the semantic. The hierarchy for services and jobs in the navigation area consists of the most top nodes services and jobs. A differentiation is made at this point to make it for the user clear if a service or a job is executed. These nodes are also artificial nodes in PROVEtech:TA. Beneath these nodes there are several other nodes with names of the respective semantic. This is determined in the ODX data. Generally the naming for the semantic is according to the ASAM MCD-2 D standard. But this is not guaranteed, since the naming in the ODX file is up to the author. In case there is no semantic at all for a service or job PROVEtech:TA groups that service or job beneath the semantic name <Semantic not available>. For example the hex service is listed in this category. When selecting an artificial element there is no response view. The request view shows in this case all sub

elements. Double clicking the element in the request view executes a jump to the element in the navigation area an refreshes all views.

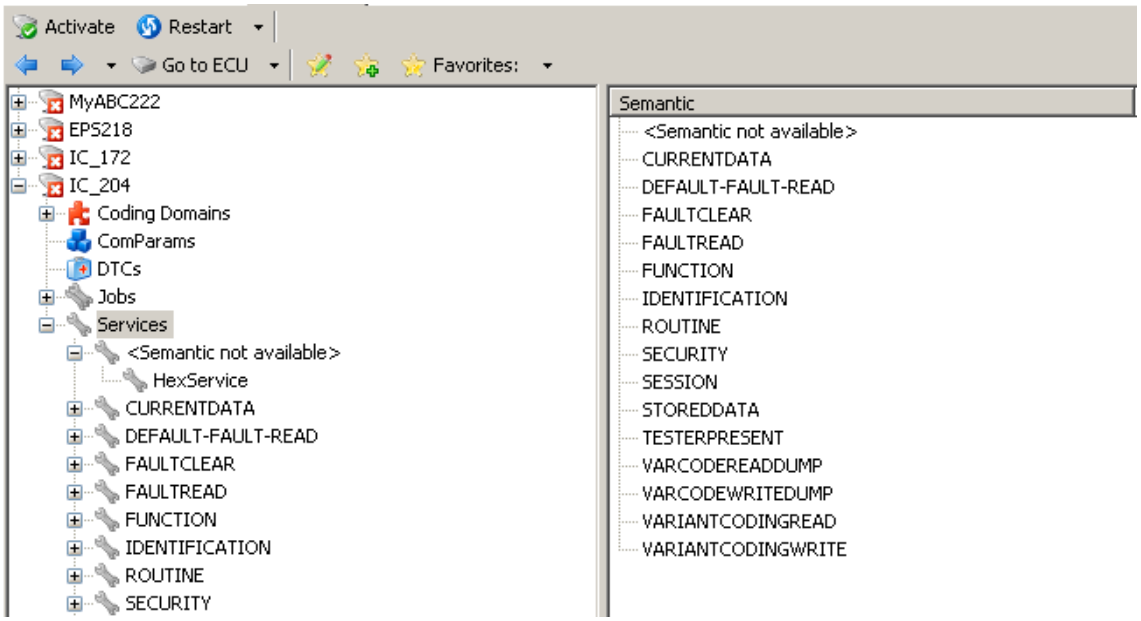


Figure 534 – The artificial element services

Figure 534 shows how services are selected in the navigation area and how the response view shows the particular semantics.

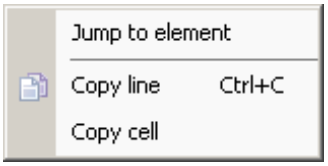


Figure 535 – The context menu in the request view

It is also possible to jump to an element via the context menu in the request view or to copy its text.

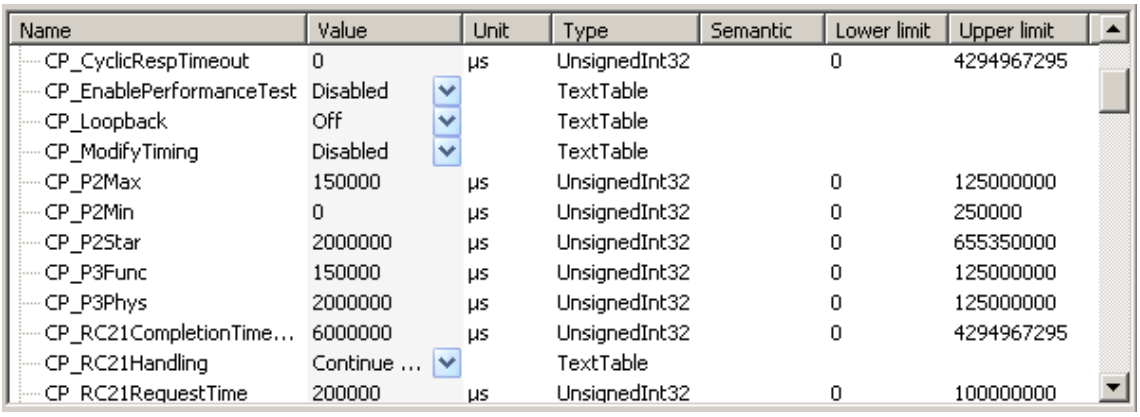
6.2.3.12 Services and jobs

When a service or job is selected in the navigation area PROVEtech:TA shows the parameters of the particular service or job in the request view. There is a differentiation between parameters that are read only and parameters that are writable (different background color). There are two kinds of writable parameters. There edit boxes that can take anything according to their data type. The data type should be considered because PROVEtech:TA does not accept entries with a wrong data type. In this case an error message pops up and the value is set back to the last correct value. The second kind is a combo box. It offers only options to choose. The values are taken from the ODX data and they can be changed by the supplier of the ODX data only.

After the execution of a service or job the response view is refreshed and the values of the result object of the MCD-3 D diagnostic are entered. Sub parameters are shown hierarchically.

6.2.3.12.1 Special case: ComParams (service ProtocolParameterSet)

The service ProtocolParameterSet (in the group with no defined semantic) is a special case. It does not communicate with the ECU, but sets the communication parameters of the diag server. Here the baudrate, the timeout P2 or the request ID (and many more) values can be read and changed. After changing one or more parameter values the service must be started by the `Execute` button to pass the new parameter values to the server. In some cases, the new parameter values take effect not immediately but after the next activation of the ECU.



Name	Value	Unit	Type	Semantic	Lower limit	Upper limit
CP_CyclicRespTimeout	0	µs	UnsignedInt32		0	4294967295
CP_EnablePerformanceTest	Disabled		TextTable			
CP_Loopback	Off		TextTable			
CP_ModifyTiming	Disabled		TextTable			
CP_P2Max	150000	µs	UnsignedInt32		0	125000000
CP_P2Min	0	µs	UnsignedInt32		0	250000
CP_P2Star	2000000	µs	UnsignedInt32		0	655350000
CP_P3Func	150000	µs	UnsignedInt32		0	125000000
CP_P3Phys	2000000	µs	UnsignedInt32		0	125000000
CP_RC21CompletionTime...	6000000	µs	UnsignedInt32		0	4294967295
CP_RC21Handling	Continue ...		TextTable			
CP_RC21RequestTime	200000	µs	UnsignedInt32		0	100000000

Figure 536 – fragment of ComParam list

ComParams are essential for establishing the contact to the ECU and must not be changed without an important reason and sufficient knowledge. Normally the default values guarantee an effective communication.

6.2.3.12.2 Special case: the FlashJob

To flash an ECU, you need:

- A FlashJob
- FlashSessions associated with the FlashJob.

In the navigation area you find FlashJobs in the branch `Jobs – FLASHJOB`. There can be several FlashJobs, e.g. one for boot sector, one for program and one for data sector. Other combinations are also possible.

A **FlashJob** writes binary data from an appropriate flashware file into the ECU. The binaries are stored in separate files and collected during runtime in FlashSessions. A

FlashSession is identified by an official and unique **FlashKey** and it is associated with one special FlashJob of the ECU. This helps to avoid flashing the data into a wrong ECU or a wrong memory segment.

The flash files found during startup in the MCD-3 D project data path determine which FlashKeys be available.

You can temporarily load additional flash files (by clicking the button `Load flash file...` when an ECU or a FlashJob is selected). The system will automatically assign the read flash sessions to the proper FlashJobs. The new FlashKeys might stay permanently selectable, or only until closing or re-initializing the system, depending on the version of diagnostic server.

Summary:

One ECU can have one or more FlashJobs, depending on the ECU data.

One FlashJob can have several FlashKeys (or none) depending on the available flash files. One FlashKey identifies one special flashware (FlashSession).

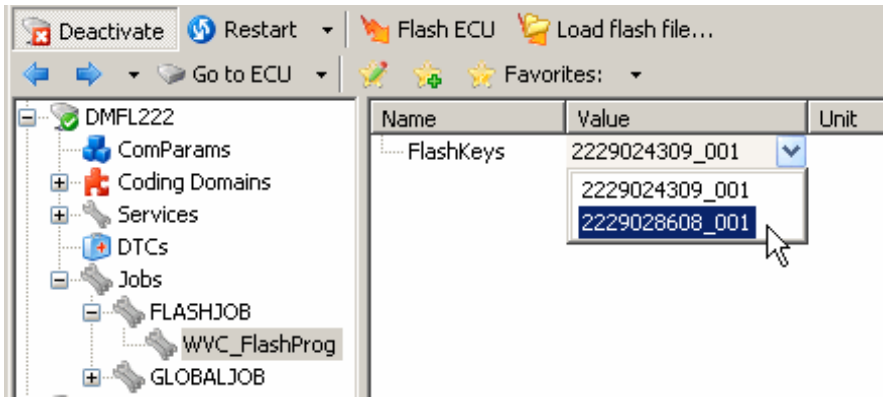


Figure 537 – FlashJob and its parameter for FlashKey selection

To write the flashware into the ECU, please select the proper FlashJob in the navigation area. Then choose the FlashKey to be flashed in the combo box of the parameter. Only valid and available FlashKeys are offered for selection. Then start the FlashJob as any other job (Button `Flash ECU`). As the flash process lasts quite long (up to several minutes), the process runs as a background task. In the meantime a progress window appears to inform about the task completion.

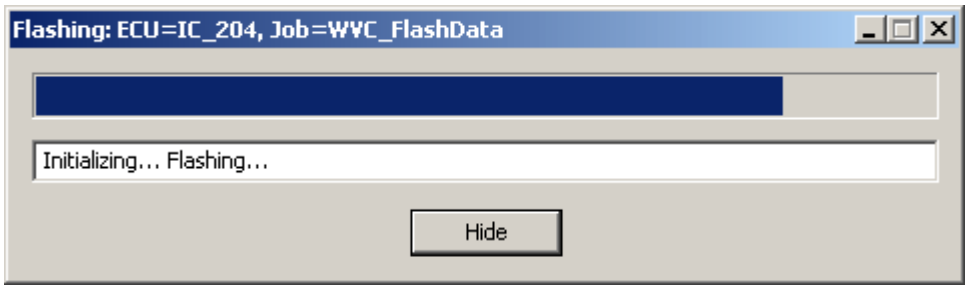


Figure 538 – Progress information about the flash process

You can move this window aside or hide it. The flashing continues in the background. You can do some tasks in the software, but any diagnosis functionality of the flashed ECU will be blocked. You should also avoid any activities producing data traffic on the CAN bus as this could interrupt the data transfer and even spoil the ECU.

In case of an Ethernet connection the data transfer rate is more than sufficient to communicate with several ECUs without disturbing the flashing at the same time.

After the flashing is finished, the result (“OK” or “Failed”) will be shown in the text line of the flash progress window (if the window was hidden, it will appear again now).



Flashing incomplete data (e.g. only the program segment without the fitting data segment), wrong data, or breaking the flash process by program shutdown or switching off the power supply will damage your ECU.

During the flash process cables must not be disconnected and PROVEtech:TA must not be shutdown.



Flashing might last several minutes. In this time no other actions are possible.

After finishing the ECU is a “new” one – with a different ID and different variant.

6.2.3.12.3 Flash window

When there are more than one flash keys to be flashed, you can open the flash window. You will see there a list of all flash jobs with all available flash keys. You can drag & drop one or several flash files (or folders containing such) into the flash window. The newly loaded keys will be marked. You can change these marks manually.

The button `Start flashing` causes all marked flash keys to be flashed sequentially in the proper order (starting with lower priority). The actions results will be shown in the lower part of the window.

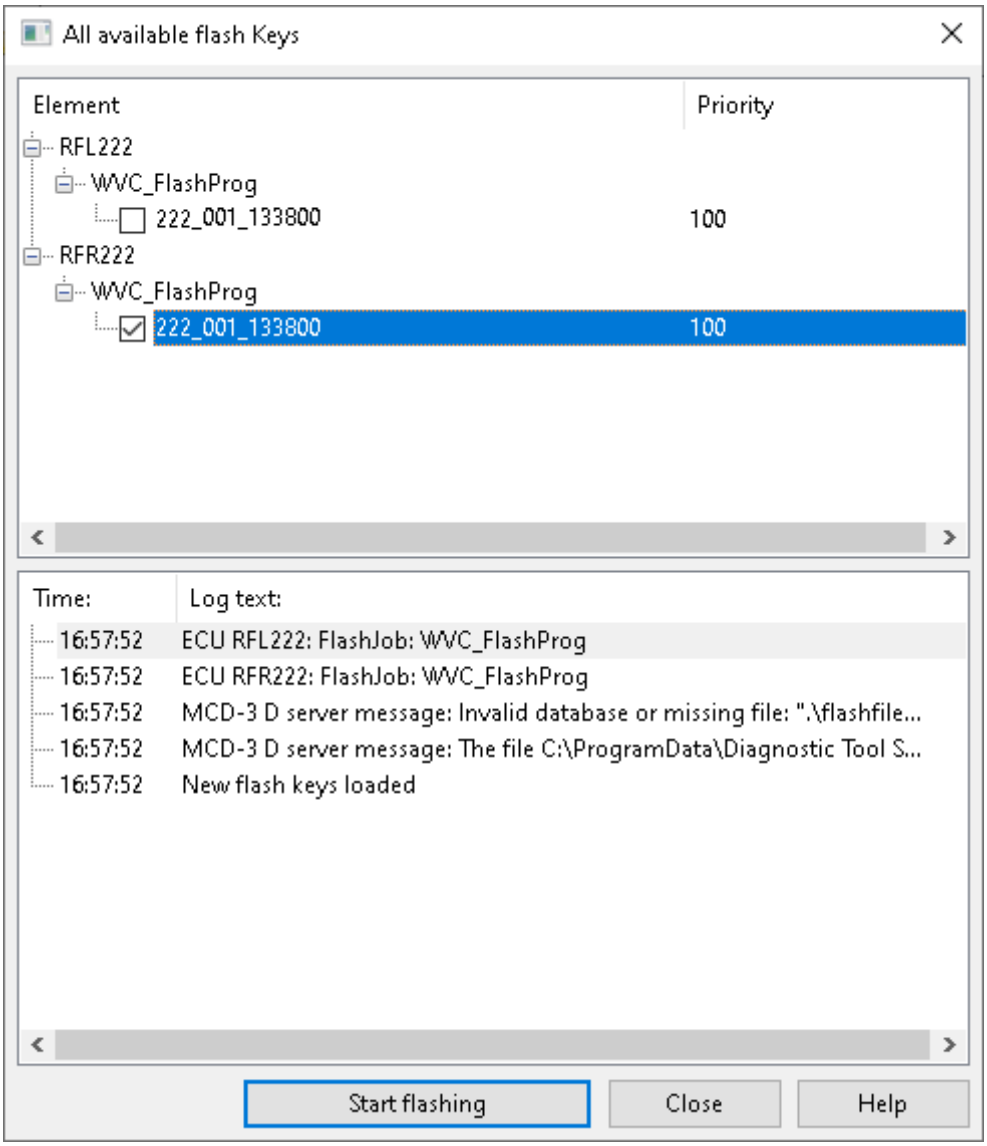


Figure 539 – The flash window

6.2.4 Button to terminate the MCD-3 D server

PROVEtech:TA accesses a so-called MCD-3 D server for this diagnostics variant. It is a foreign software supporting the ASAM MCD-3 D interface. Normally, the server runs within the same Windows process as the client (PROVEtech:TA). Additionally, the MCD-3 D server accesses the hardware via D-PDU API interfaces of other foreign software, to send and receive CAN messages as an example. Furthermore, the MCD-3 D server executes Java Code (via a Java Virtual Machine) or additional DLLs on demand, which are part of the diagnostic data. All these foreign components typically run in the same Windows process. If one component produces a memory access violation or is stuck in a deadlock or endless loop or waits for an ECU answer with a very long timeout, the whole Windows process hangs or crashes. This means that then working

normally with PROVEtech:TA is not possible anymore. To be able to continue working with PROVEtech:TA in such a scenario without completely restarting it, it has a new software architecture beginning with version 2025. The MCD-3 D server including all its dependencies runs in a separate Windows process. In case of an issue with the process, you can terminate and restart that process without restarting PROVEtech:TA.

Please be aware that this is just a workaround for issues in the foreign components. It would be better to fix the issues in the foreign components, but the suppliers did not do that in the last years. Unfortunately, using a separate process costs performance. On the other hand, it is possible to use 64-bit PROVEtech:TA with a 32-bit MCD-3 D server this way. The name of the separate process is “DiagnosticsMCD3DExt_???.exe”, where ??? varies with the used MCD-3 D server.

PROVEtech:TA differentiates MCD-3 D calls which should return very fast (e.g. get the name of a service) from calls which can take a long time (e.g. execute a service synchronously). Therefore, it uses 2 different timeout values, configurable in “DiagnoisticsMCD3D.ini” (see ***Administrator's Guide***): “RpcShortTimeout” and “RpcLongTimeout”. PROVEtech:TA uses “RpcLongTimeout” typically for calls via its test language. All commands which might take a long time have an additional timeout parameter with the default value 0. For 0 PROVEtech:TA uses “RpcLongTimeout”, otherwise the provided value. Commands that take only a short time always use “RpcShortTimeout”. Manual actions on the graphical user interface which take a long time do not use a timeout. So, they wait endless.

In case a call does not return after 10 seconds, this button appears in PROVEtech:TA left of the symbol which rotates while executing tests:

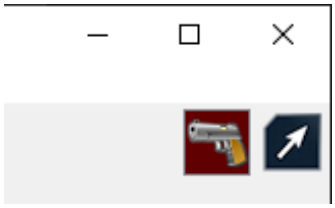


Figure 540 – Button to terminate the MCD-3 D server process.

When the call finally returns, the button disappears. Some calls can take a very long time naturally, which is no wrong behavior. Right-clicking on the Windows Taskbar icon of PROVEtech:TA shows the context menu item *Terminate MCD-3 D diagnostics server* additionally to the button. In case you click the button and acknowledge the displayed warning or select the context menu item, PROVEtech:TA forcefully terminates the MCD-3 D server process. The hanging call returns with an error message and all following calls return according error messages as the MCD-3 D server does not run anymore. Only restarting MCD-3 D helps to get it running again. To do this click on Restart in the diagnostics toolbar or call the test language command ***Dgn.Reset***. To check if the MCD-3 D server is currently running, call ***Dgn.IsStarted***. All error

messages resulting from an unplanned termination of the MCD-3 D server contain this language independent unique string to detect this state: {💀}

Even if the process terminates itself due to a crash of the MCD-3 D server, an error message with this string appears and you must restart the server.

An automatic restart is currently not planned. Such an automatic restart can hang again or lead to another crash and thus to an endless loop of restarts. And PROVEtech:TA cannot restore the exact same state which was active before terminating the server. There is no way that a test program just continues working after such an issue as if nothing has happened. Please be aware that this whole mechanism is just a workaround for issues with foreign software. It is no solution for the issues. Only the suppliers of the foreign software components could actually resolve the issues.

Chapter

7

Fault simulation

In the Fault Simulation function area you can simulate faults in the CAN bus and cabling of the vehicle's electrical system. The electronic faults may concern individual ECUs or several ECUs at the same time. PROVEtech:TA additionally supports the setting of multiple faults.

The electric fault simulation consists of the required physical modules and bus lines in the signal adaptations of the individual ECUs. PROVEtech:TA guarantees that these modules are controlled correctly and, by means of a plausibility check, minimizes the risk that the test system and test object may be destroyed.

PROVEtech:TA supports several different fault simulation hardware types:

- PROVEtech:µHiL (see chapter ***7.1 PROVEtech:µHiL fault simulation***)
- Softing Engineering & Solutions (see chapter ***7.2 Softing Engineering & Solutions fault simulation***)
- ETAS ES4440 (see chapter ***7.3 ETAS ES4440 fault simulation***)
- dSPACE FullSize controlled via CAN (see chapter ***7.4 dSPACE FullSize fault simulation via CAN***)
- dSPACE FullSize controlled via RS232 (see chapter ***7.5 dSPACE FullSize fault simulation via RS232***)
- dSPACE MidSize (see chapter ***7.6 dSPACE MidSize fault simulation***)
- dSPACE SCALEXIO (see chapter ***7.7 dSPACE SCALEXIO fault simulation***)
- dSPACE DS5394C1 (see chapter ***7.8 dSPACE DS5394C1 fault simulation***)



An improper use or wrong data files might demolish your hardware (e.g. your ECU or fault simulation box).

7.1 PROVEtech:µHiL fault simulation

Read this chapter in case you use the PROVEtech:µHiL for the fault simulation.

In this chapter you will find out:

- how the fault simulation function area is structured
- how you can configure wiring faults

7.1.1 Architecture

Select the tab `Fault Simulation` to display the following panel:

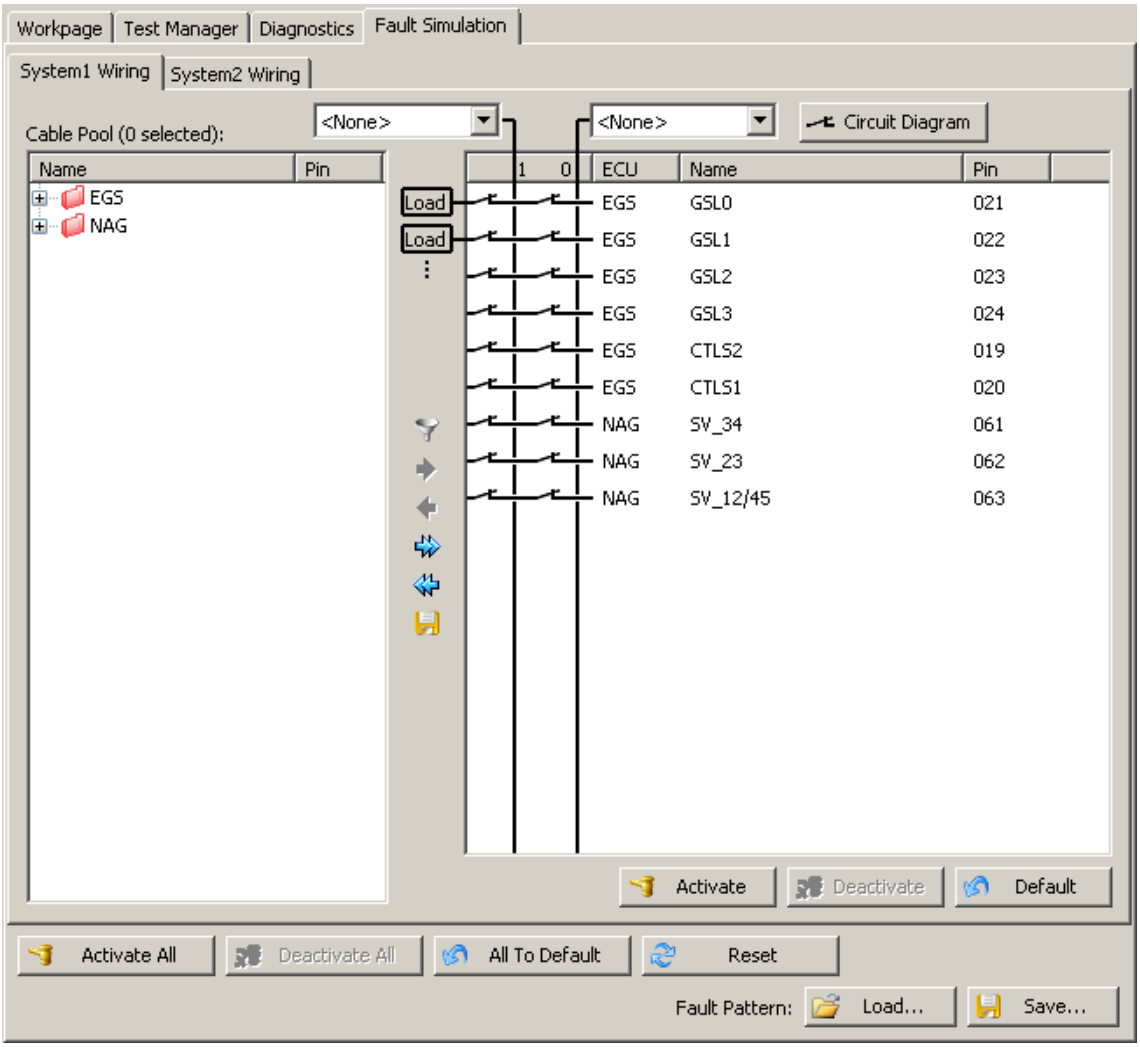


Figure 541 – Architecture of the µHiL fault simulation

In this example the fault simulation controls two systems: System1 and System2 (The names are configurable, see chapter ***3.2.7.9.1 PROVEtech:µHiL fault simulation***). In most cases you only use one system. For every system the fault simulation displays a tab to set wiring faults (here: `System1 Wiring` and `System2 Wiring`).

Chapter ***7.1.2 Wiring faults*** describes how to set wiring faults. Finally Chapter ***7.1.3 Fault patterns and reset*** treats fault patterns and the `Reset` button.

7.1.2 Wiring faults

This chapter describes how to simulate different faults concerning the ECU wiring.

7.1.2.1 Preselection of cables

Figure 542 shows the page for wiring faults of a system.

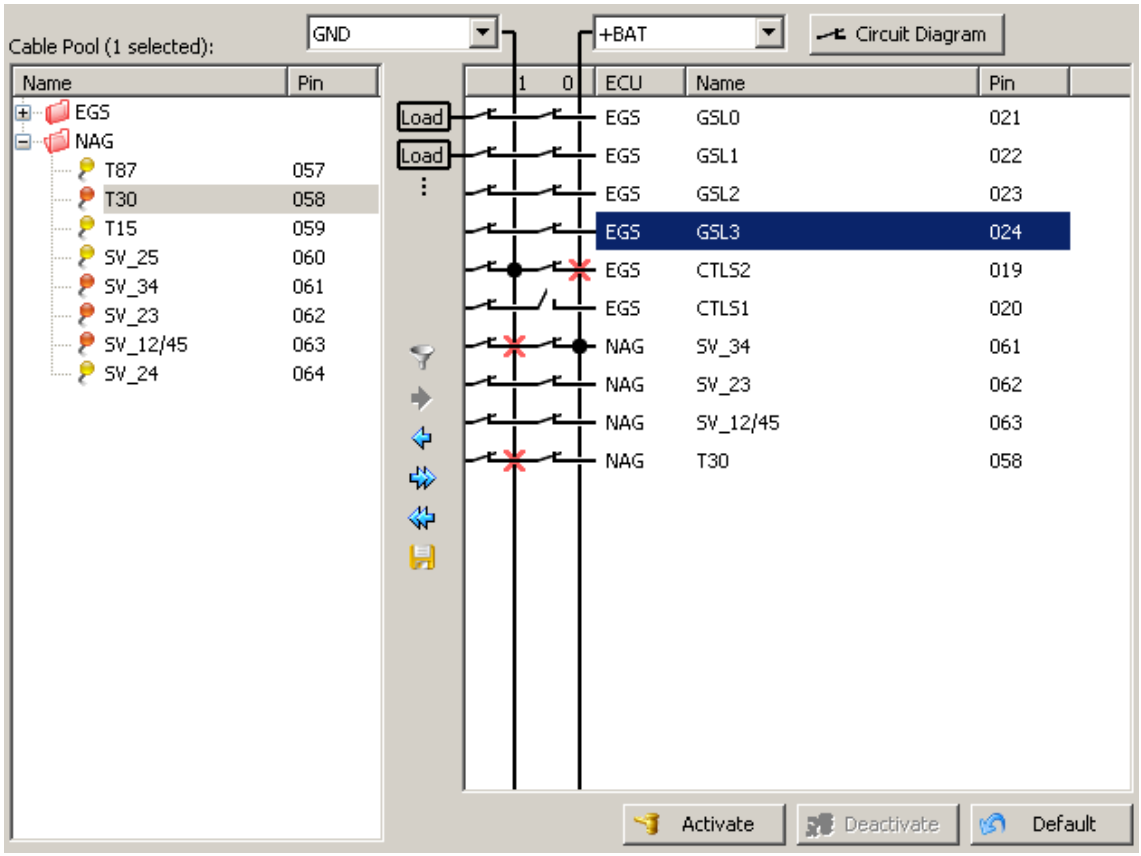


Figure 542 – Page for wiring faults

The `Cable Pool` tree on the left side contains all ECUs in the first level and in the second level all their cables you can manipulate with the fault simulation.

To set wiring faults you must preselect some cables of the cable pool first. The right cable list shows your current preselection. To preselect the cables you have to select them in the cable pool tree and press . As an alternative you can drag them from the tree and drop them in the preselection list. Press to preselect all existing cables. Select one or several cables in the preselection list and press to remove them from the list or press to remove all cables. Again you can drag them from the list and drop them in the cable pool as an alternative.

Red colored icons in the cable pool tree mark all cables that are already preselected.

You cannot remove cables from the preselection that still have fault settings.

Click on the central button to save the current preselection list of all wiring fault pages for your user name and the current configuration. When you click on `Reset` or re-load this configuration, you will get this list again.

To find cables in the cable pool more easily, you can click on or right click on the column headers to open a menu and select *Filters*. This activates a filter bar below the column headers. There you can enter a string for filtering the cable pool. As an example enter “t87” in the column `Name` to display only cables containing “t87”. The string matching is case insensitive. Click on the button again or select the menu item again to close the filter bar.

PROVEtech:TA displays the amount of currently selected cables after the text `Cable Pool` in parenthesis. In case you select an ECU entry, you can detect the amount of cables within that entry.

Double click on a cable name or select *Info…* in the context menu of the cable pool or the preselection list to display a dialog with additional cable information in case you have configured that (see chapter ***3.2.7.9.1 PROVEtech:µHiL fault simulation***):

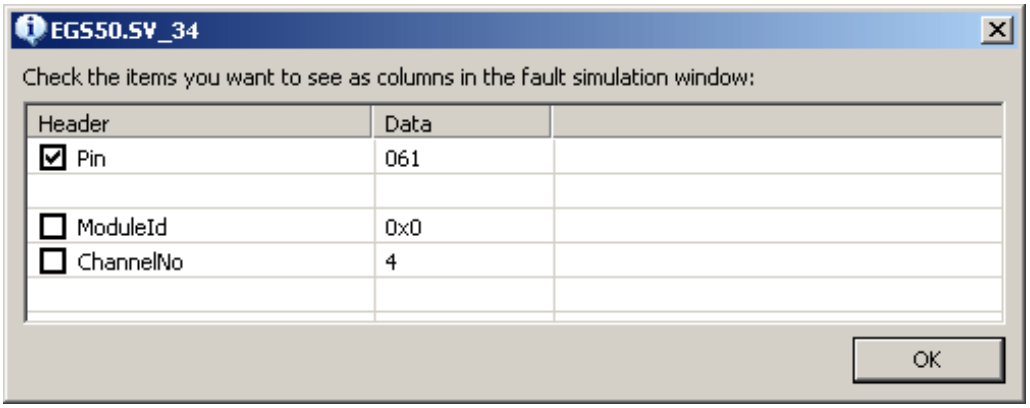


Figure 543 – Additional cable information

To see some of this information as additional columns in the preselection list or the cable pool (depends on which window you have used to open the info dialog), set check marks in the according lines.

7.1.2.2 Shorts and breaks

The hardware architecture of the fault simulation consists of two rails and four relays for each cable. Two relays control the connection to the rails and two relays can break the cable:

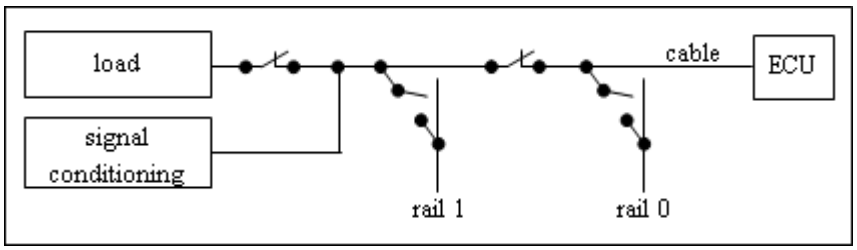


Figure 544 – Hardware architecture

The right break relay disconnects the possibly existing load and the signal conditioning and with it the connection to the real-time computer. The left one only disconnects the load.

PROVEtech:TA replicates the electrical design the following way:

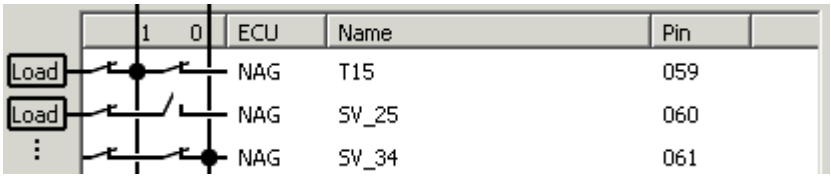


Figure 545 – Cable list

The first column of the list view shows the four relays per cable. The vertical lines are the two rails. The horizontal lines are the control unit cables with two relays (⊸) for breaking it. The left side symbolically displays the loads. The ECU pin is displayed symbolically via the ECU name, the pin name and the pin number. The signal conditioning is missing in this view.

You can click on the symbolic relays to change their states. PROVEtech:TA uses different symbols to show the relay states and the possibility to change the state.

For relays that connect to rails:

- The connection is not established and you are allowed to change it.
- The connection is established and you are allowed to change it.
- The connection is established and you are not allowed to change it (this only exists in the active state).
- The connection is not established and you are not allowed to change it.

For relays that break the cable:

- The connection is not broken and you are allowed to change it.
- The connection is broken and you are allowed to change it.
- The connection is broken and you are not allowed to change it (this only exists in the active state).

— The connection is not broken and you are not allowed to change it.

The wiring fault simulation differentiates two states: the active and the inactive state. After launching PROVEtech:TA the fault simulation is in the inactive state. This means that PROVEtech:TA does not transmit the faults you edit directly to the hardware. Press 

to enter the active state and transmit your fault settings to the hardware of one system. After that all changes you make are transmitted directly to the hardware. Press 

to return to the inactive state. This takes back all fault settings from the hardware. The buttons 

and 

behave identically, but they change the state of all systems simultaneously.



In case your configuration only has one system, the buttons `Activate`, `Deactivate` and `Default` are hidden, because in this case their functionality is the same as `Activate All`, `Deactivate All` and `All To Default`.

To break a cable set one of the break relays and select the active state:

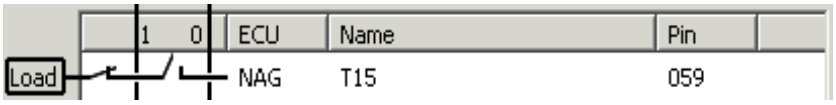


Figure 546 – Cable break

To short two cables connect both to the same rail and select the active state:

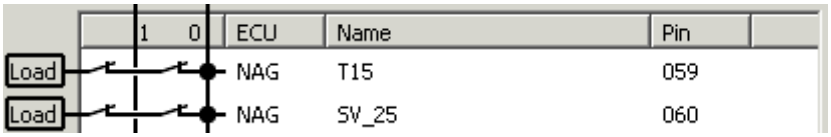


Figure 547 – Short of two cables

To short a cable to a potential (typically battery power or ground) connect the cable to rail 0 or 1 and then connect the rail to the potential:

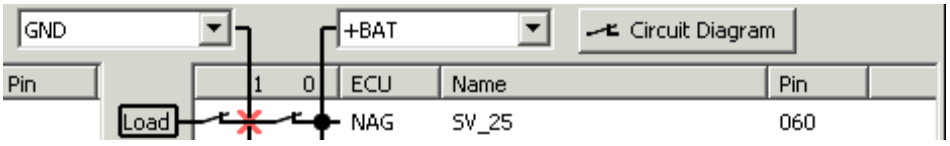
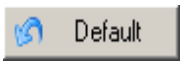


Figure 548 – Short to potential

To connect rail 0 or 1 to a potential select the potential in one of the two upper combo boxes. In this example rail 0 is connected to battery power and this means also “SV_25” is connected to it. Activate these settings by switching to the active state.

The µHiL has a fixed wiring for the two rails. So you can only connect the battery power to rail 0 (selection “+BAT” in the combo box) and ground to rail 1 (selection “GND” in the combo box). Additionally you can connect the rails to external connectors (selection “EXT” in the combo boxes) and you can connect them to the external connectors and the potentials at the same time (“GND/EXT” and “+BAT/EXT” in the combo boxes).

You can combine different faults by setting the according relays. For example you can connect one cable to ground via rail 1 and at the same time you can short two other cables via rail 0.

Press 

to take back all current settings of a system. Use 

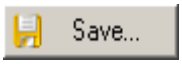
to take back all settings of all systems.

Click on 

to open this chapter of the manual.

7.1.3 Fault patterns and reset

Push the buttons 

and 

to load and save fault patterns (see ***Figure 541***). A fault pattern file contains all the settings you can manipulate in the fault simulation. With fault patterns you can quickly save and restore complex or extensive fault settings.

Click 

to completely reset the fault simulation hard- and software.

7.2 Softing Engineering & Solutions fault simulation

Read this chapter in case you use the Softing Engineering & Solutions fault simulation hardware.

In this chapter you will find out:

- how the fault simulation function area is structured
- how you can configure wiring faults
- how you set CAN faults

7.2.1 Architecture

Select the tab `Fault Simulation` to display the following panel:

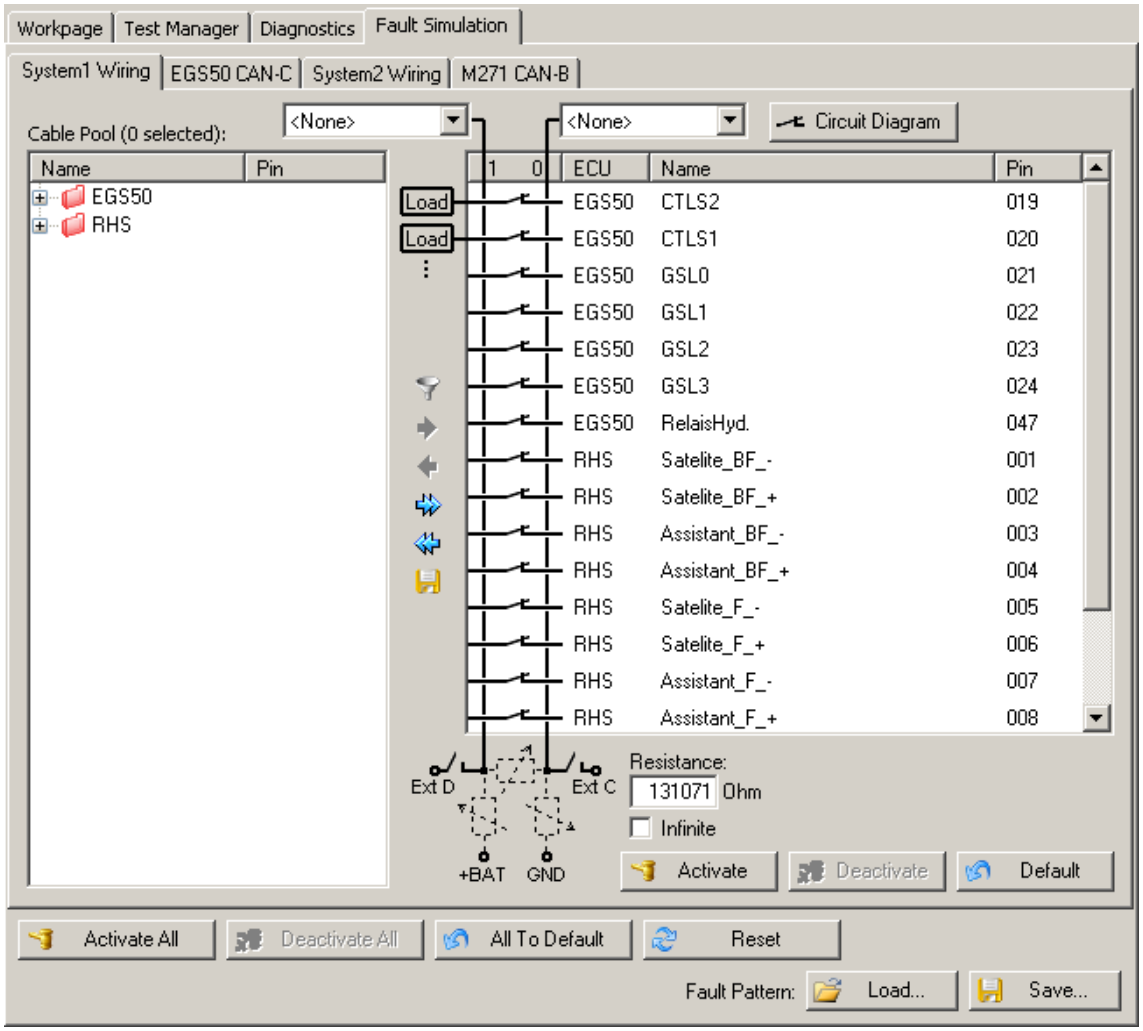


Figure 549 – Architecture of the Softing Engineering & Solutions fault simulation

In this example the fault simulation controls two systems: System1 and System2 (The names are configurable, see chapter ***3.2.7.9.2 Softing Engineering & Solutions fault simulation***). In most cases you only use one system. For every system the fault simulation displays a tab to set wiring faults (here: `System1 Wiring` and `System2 Wiring`). Depending from your configuration additional tabs for CAN fault settings exist per system; here: `EGS50 CAN-C` and `M271 CAN-B`. Again the names are configurable.

Chapter ***7.2.2 Wiring faults*** describes how to set wiring faults and Chapter ***7.2.3 CAN fault settings*** describes CAN fault settings. Finally Chapter ***7.2.4 Fault patterns and reset*** treats fault patterns and the `Reset` button.

7.2.2 Wiring faults

This chapter describes how to simulate different faults concerning the ECU wiring.

7.2.2.1 Preselection of cables

Figure 550 shows the page for wiring faults of a system.

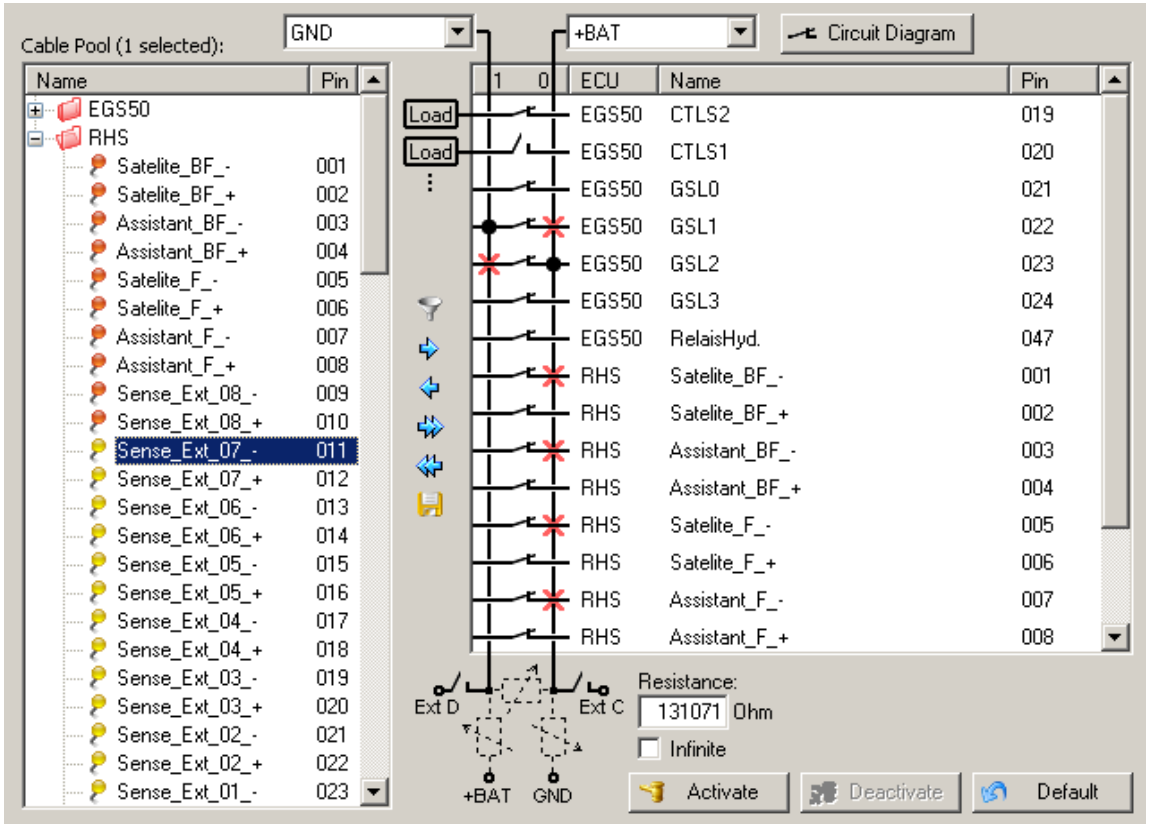


Figure 550 – Page for wiring faults

The `Cable Pool` tree on the left side contains all ECUs in the first level and in the second level all their cables you can manipulate with the fault simulation.

To set wiring faults you must preselect some cables of the cable pool first. The right cable list shows your current preselection. To preselect the cables you have to select them in the cable pool tree and press . As an alternative you can drag them from the tree and drop them in the preselection list. Press to preselect all existing cables. Select one or several cables in the preselection list and press to remove them from the list or press to remove all cables. Again you can drag them from the list and drop them in the cable pool as an alternative.

Red colored icons in the cable pool tree mark all cables that are already preselected.

You cannot remove cables from the preselection that still have fault settings.

Click on the central button to save the current preselection list of all wiring fault pages for your user name and the current configuration. When you click on `Reset` or re-load this configuration, you will get this list again.

To find cables in the cable pool more easily, you can click on or right click on the column headers to open a menu and select *Filters*. This activates a filter bar below the column headers. There you can enter a string for filtering the cable pool. As an example enter “t87” in the column `Name` to display only cables containing “t87”. The string matching is case insensitive. Click on the button again or select the menu item again to close the filter bar.

PROVEtech:TA displays the amount of currently selected cables after the text `Cable Pool` in parenthesis. In case you select an ECU entry, you can detect the amount of cables within that entry.

Double click on a cable name or select *Info…* in the context menu of the cable pool or the preselection list to display a dialog with additional cable information in case you have configured that (see chapter ***3.2.7.9.2 Softing Engineering & Solutions fault simulation***):

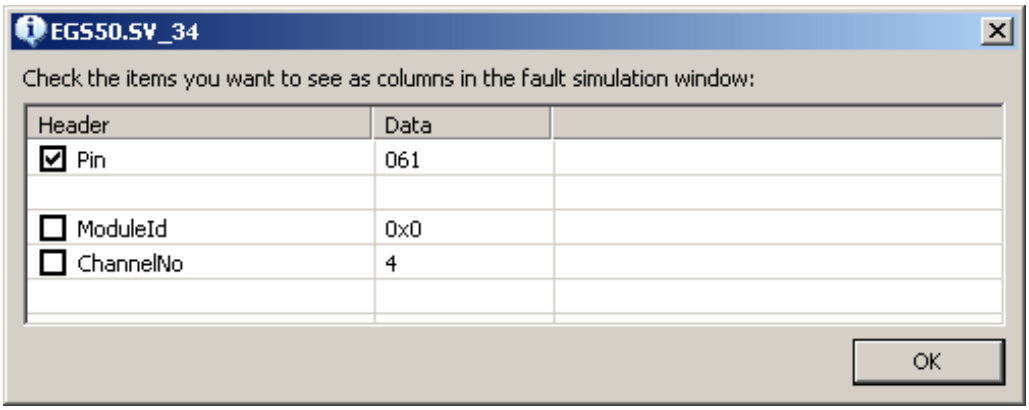


Figure 551 – Additional cable information

To see some of this information as additional columns in the preselection list or the cable pool (depends on which window you have used to open the info dialog), set check marks in the according lines.

7.2.2.2 Shorts and breaks

The hardware architecture of the fault simulation consists of two rails and three relays for each cable. Two relays control the connection to the rails and one relay can break the cable:

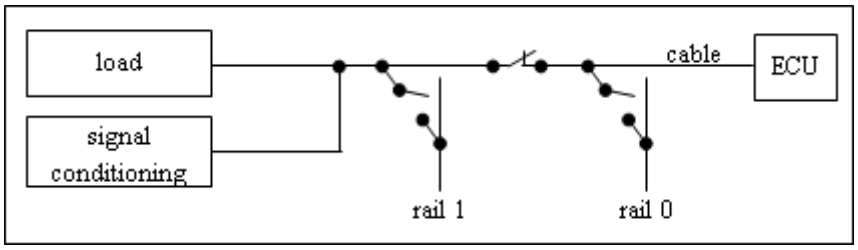


Figure 552 – Hardware architecture

The break relay disconnects the possibly existing load and the signal conditioning and with it the connection to the real-time computer.

PROVEtech:TA replicates the electrical design the following way:

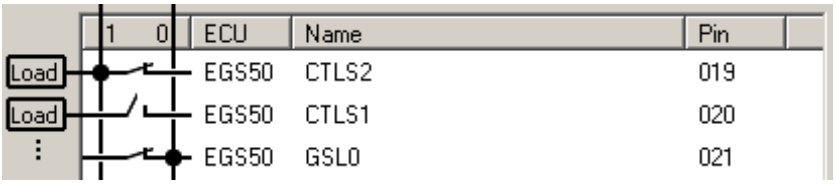


Figure 553 – Cable list

The first column of the list view shows the three relays per cable. The vertical lines are the two rails. The horizontal lines are the control unit cables with the relay (⎯⎯) for breaking it. The left side symbolically displays the loads. The ECU pin is displayed symbolically via the ECU name, the pin name and the pin number. The signal conditioning is missing in this view.

You can click on the symbolic relays to change their states. PROVEtech:TA uses different symbols to show the relay states and the possibility to change the state.

For relays that connect to rails:

- The connection is not established and you are allowed to change it.
- The connection is established and you are allowed to change it.
- The connection is established and you are not allowed to change it (this only exists in the active state).
- The connection is not established and you are not allowed to change it.

For relay that breaks the cable:

- The connection is not broken and you are allowed to change it.

The connection is broken and you are allowed to change it.

The connection is broken and you are not allowed to change it (this only exists in the active state).

The connection is not broken and you are not allowed to change it.

The wiring fault simulation differentiates two states: the active and the inactive state. After launching PROVEtech:TA the fault simulation is in the inactive state. This means that PROVEtech:TA does not transmit the faults you edit directly to the hardware. Press Activate to enter the active state and transmit your fault settings to the hardware of one system. After that all changes you make are transmitted directly to the hardware. Press Deactivate to return to the inactive state. This takes back all fault settings from the hardware. The buttons Activate All and Deactivate All behave identically, but they change the state of all systems simultaneously.



In case your configuration only has one system, the buttons `Activate`, `Deactivate` and `Default` are hidden, because in this case their functionality is the same as `Activate All`, `Deactivate All` and `All To Default`.

To break a cable set the break relay and select the active state:



Figure 554 – Cable break

To short two cables connect both to the same rail and select the active state:

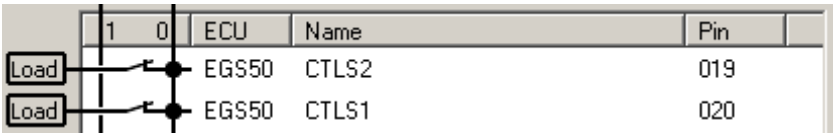


Figure 555 – Short of two cables

To short a cable to a potential (typically battery power or ground) connect the cable to rail 0 or 1 and then connect the rail to the potential:

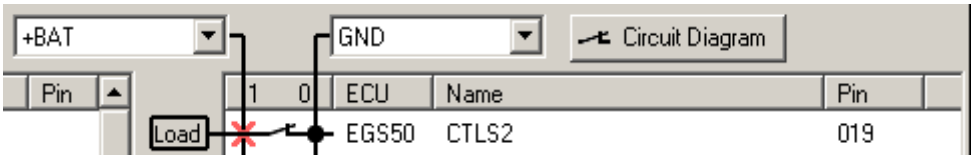


Figure 556 – Short to potential

To connect rail 0 or 1 to a potential select the potential in one of the two upper combo boxes. In this example rail 0 is connected to ground and this means also “CTLS2” is connected to it. Activate these settings by switching to the active state.

You can combine different faults by setting the according relays. For example you can connect one cable to ground via rail 0 and at the same time you can short two other cables via rail 1.

Press Default to take back all current settings of a system. Use All To Default to take back all settings of all systems.

Click on Circuit Diagram to open this chapter of the manual.

7.2.2.3 Contact resistances and external devices

The part below the cable list represents an adjustable resistor you can connect to the rails and two relays for connecting two external rails to the internal rails:

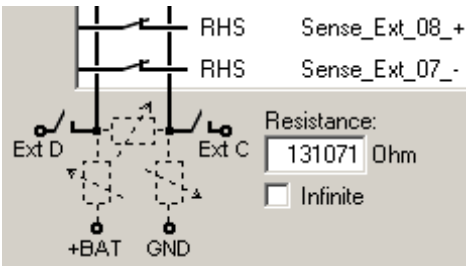


Figure 557 – Resistor and external rails

There are three positions for inserting the resistor:
- between rail 1 and potential 0 (normally battery power)
- between rail 0 and potential 1 (normally ground)
- between rail 0 and rail 1

These positions are marked by the dashed resistor symbols. In case your system does not have an adjustable resistor, you will not see this.

To insert the resistor into one of these positions, just click on the dashed symbol and the dashed lines will transform to thick lines. To remove it, just click on it again. To adjust

the resistance, use the according edit box or activate the check box `Infinite` to have an infinite resistance.

To insert a contact resistance of 100 Ohm in a cable use the following settings as an example and enter the active state:

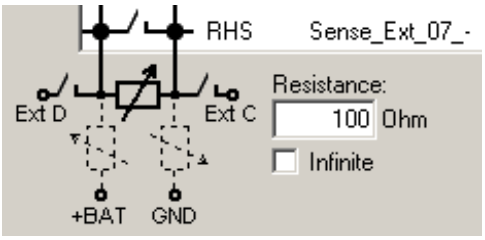


Figure 558 – Contact resistance

For a 100 Ohm leakage resistance to ground (a cable connected to ground via a resistor) use the following settings as an example and enter the active state:

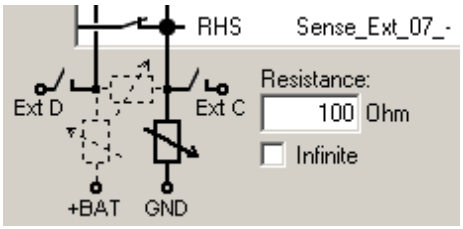


Figure 559 – Leakage resistance

The Softing Engineering & Solutions fault simulation hardware has two internal rails on every cartridge (a unit, normally one per ECU that contains a main control module) and four external rails (A-D) to connect external devices (like the adjustable resistor) or for interconnections (connecting two pins of different cartridges/ECUs). PROVEtech:TA uses the two external rails A and B exclusively for doing interconnections and for connecting the adjustable resistor. The external rails C and D are free to use them for any purpose you like, e.g. for connecting external devices. Just click on the relay symbols to toggle them. Here is an example that connects the rails C and D to the pin "Sense_Ext_07_-":

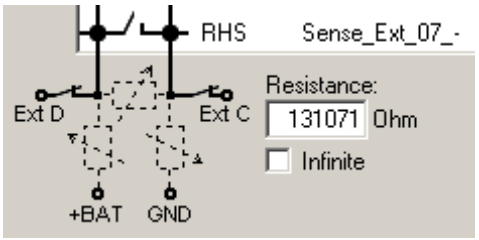


Figure 560 – External rails

7.2.3 CAN fault settings

Figure 561 displays the page for CAN fault settings:

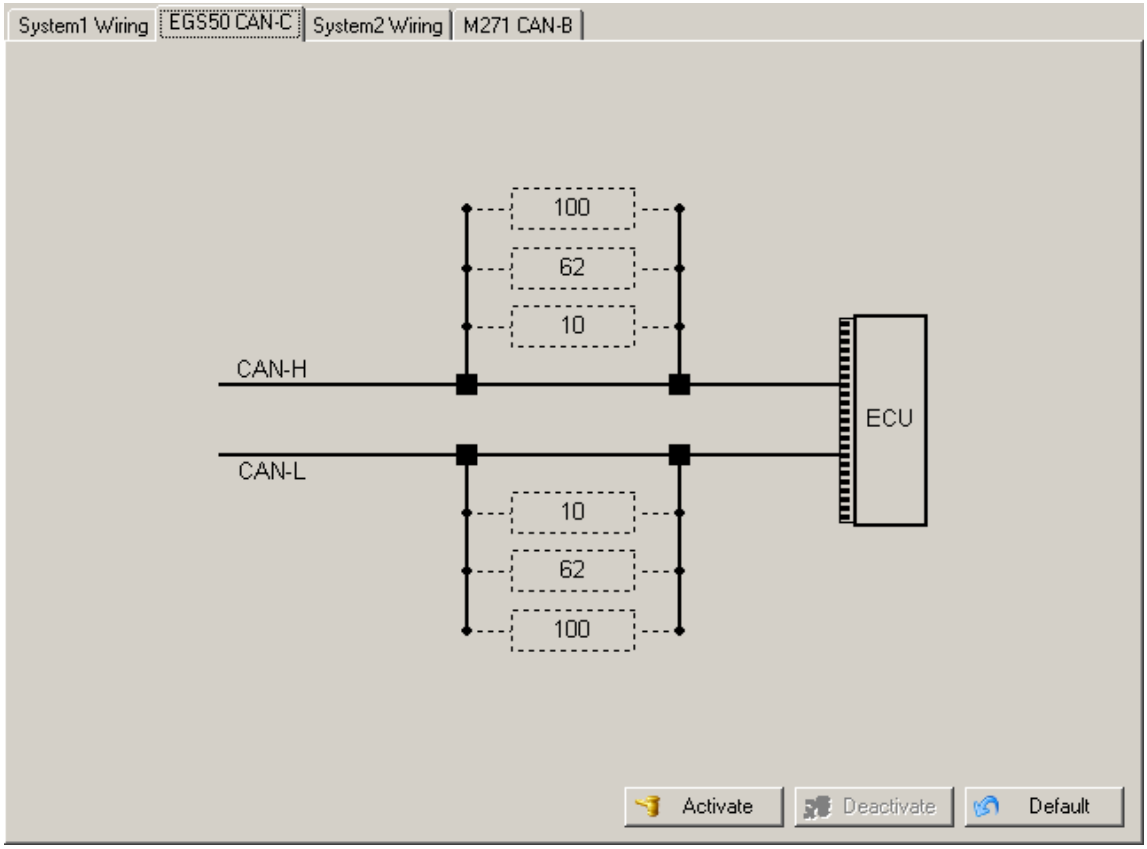


Figure 561 – Page for CAN fault settings

The amount of pages of this type can vary per system. This view represents the electrical configuration. On the left side you can see the CAN bus. It is system dependent what is connected to the bus. Normally at least the vehicle matrix simulation is connected to it. The right side displays the ECU. The two long horizontal lines represent the CAN high and the CAN low wiring. Click on the four filled rectangles to change the connections. First click on one rectangle (it changes to white) and after that a 2nd one to create or delete a connection:

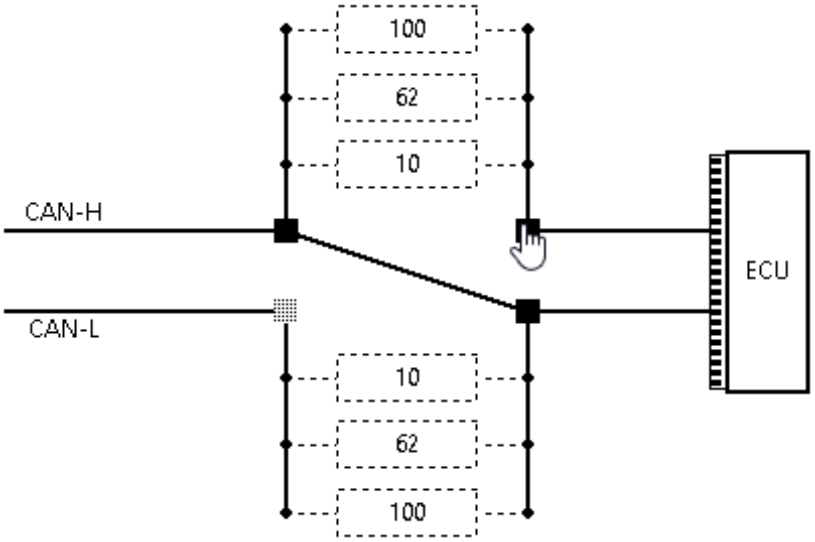


Figure 562 – Changing CAN connections

The six dashed rectangles represent six resistors with fixed values (values are system dependent). Click on them to connect or disconnect them.

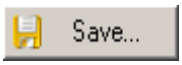
The configured CAN faults need to be activated just like the wiring faults by clicking `Activate` or `Activate All`. An activation always applies to a whole system (wiring and CAN fault settings). So it is not possible to only activate the CAN fault settings but not the current wiring fault settings.

The `Default` button also applies to the whole system.

In case your configuration only has one system, the buttons `Activate`, `Deactivate` and `Default` are hidden, because in this case their functionality is the same as `Activate All`, `Deactivate All` and `All To Default`.

7.2.4 Fault patterns and reset

Push the buttons 

and 

to load and save fault patterns (see ***Figure 549***). A fault pattern file contains all the settings you can manipulate in the fault simulation. With fault patterns you can quickly save and restore complex or extensive fault settings.

Click 

to completely reset the fault simulation hard- and software.

7.3 ETAS ES4440 fault simulation

Read this chapter in case you use the ETAS ES4440 for the fault simulation.

In this chapter you will find out:

- how the fault simulation function area is structured
- how you can configure wiring faults



The ETAS ES4440 fault simulation only lets you configure predefined error settings. The user interface of PROVEtech:TA directly displays these limitations. So in case the user interface prevents you from doing some settings, this is based on the ES4440 hardware limitations. Please read the ES4440 documentation for information about the possible fault settings.

7.3.1 Architecture

Select the tab `Fault Simulation` to display the following panel:

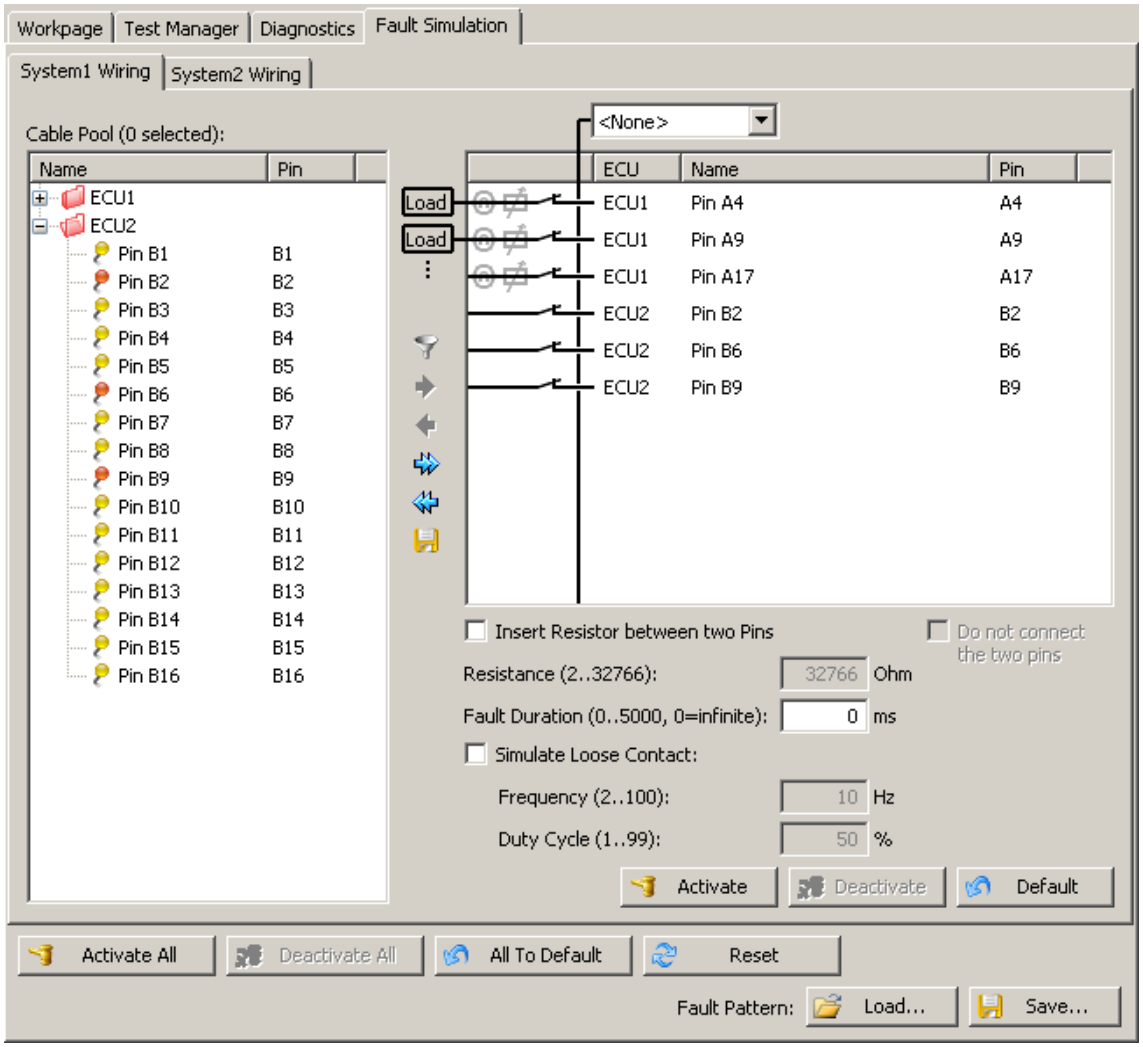


Figure 563 – Architecture of the ETAS ES4440 fault simulation

In this example the fault simulation controls two systems: System1 and System2 (The names are configurable, see chapter ***3.2.7.9.3 ETAS ES4440 fault simulation***). In most cases you only use one system. For every system the fault simulation displays a tab to set wiring faults (here: `System1 Wiring` and `System2 Wiring`).

Chapter ***7.3.2 Wiring faults*** describes how to set wiring faults. Finally Chapter ***7.3.4 Fault patterns and reset*** treats fault patterns and the `Reset` button.

7.3.2 Wiring faults

This chapter describes how to simulate different faults concerning the ECU wiring.

7.3.2.1 Preselection of cables

Figure 564 shows the page for wiring faults of a system.

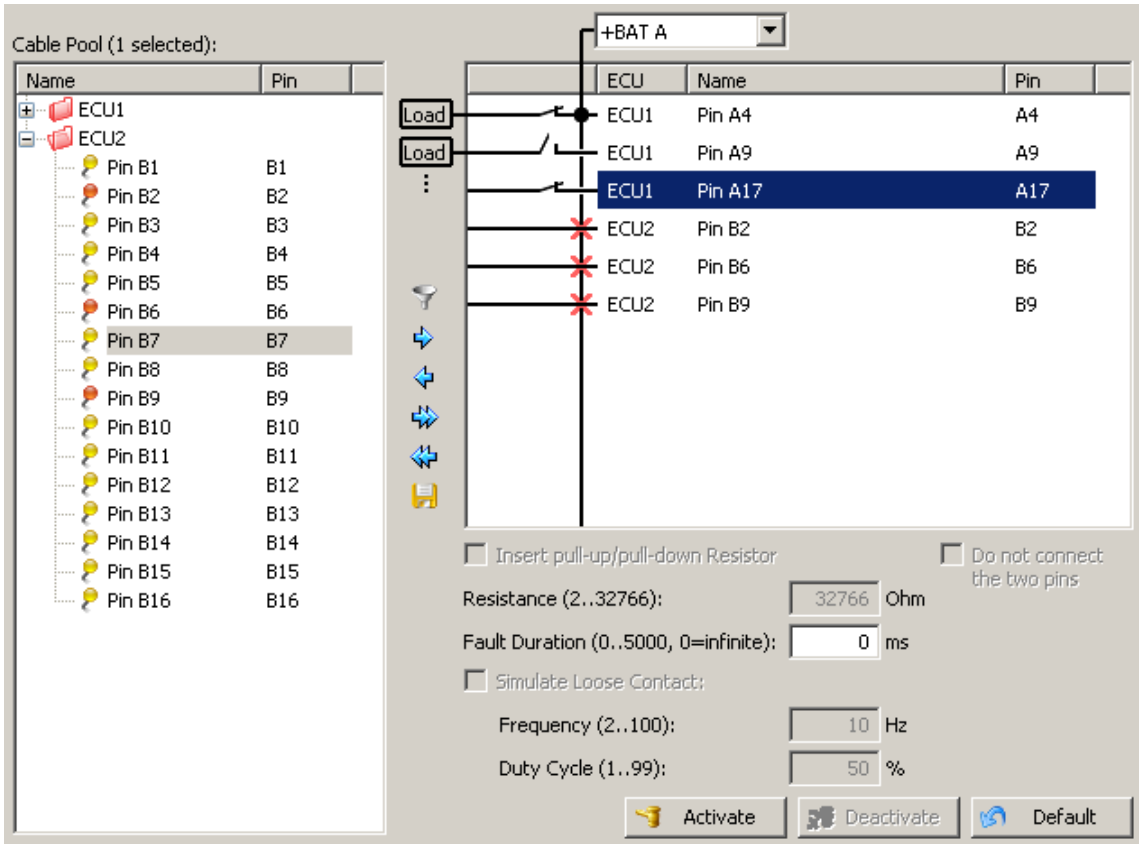


Figure 564 – Page for wiring faults

The `Cable Pool` tree on the left side contains all ECUs in the first level and in the second level all their cables you can manipulate with the fault simulation.

To set wiring faults you must preselect some cables of the cable pool first. The right cable list shows your current preselection. To preselect the cables you have to select them in the cable pool tree and press . As an alternative you can drag them from the tree and drop them in the preselection list. Press to preselect all existing cables. Select one or several cables in the preselection list and press to remove them from the list or press to remove all cables. Again you can drag them from the list and drop them in the cable pool as an alternative.

Red colored icons in the cable pool tree mark all cables that are already preselected.



You cannot remove cables from the preselection that still have fault settings.

Click on the central button to save the current preselection list of all wiring fault pages for your user name and the current configuration. When you click on `Reset` or re-load this configuration, you will get this list again.

To find cables in the cable pool more easily, you can click on or right click on the column headers to open a menu and select *Filters*. This activates a filter bar below the column headers. There you can enter a string for filtering the cable pool. As an example enter “t87” in the column `Name` to display only cables containing “t87”. The string matching is case insensitive. Click on the button again or select the menu item again to close the filter bar.

PROVEtech:TA displays the amount of currently selected cables after the text `Cable Pool` in parenthesis. In case you select an ECU entry, you can detect the amount of cables within that entry.

7.3.2.2 Shorts, breaks and current measurement

The hardware architecture of the ETAS ES4440 fault simulation is not public. So the user interface in PROVEtech:TA does not represent the hardware but just helps you in configuring the fault settings.

The cable list represents the whole path of an ECU pin to the load or sensor. On the right side is the ECU pin and on the left side is the load or sensor (represented by the “Load” rectangles on the user interface). Between them are the fault settings (1st column of the list) The signal conditioning is missing in this view. At the beginning there are no faults inserted and the ECU pin is connected to the load or sensor:

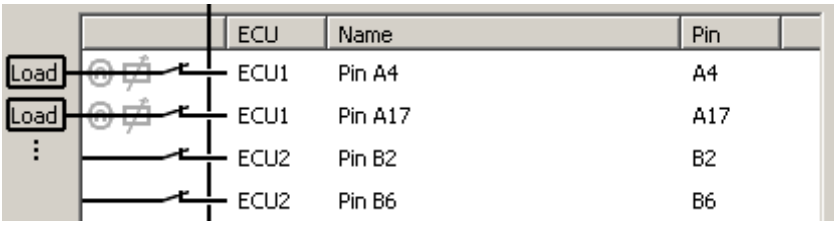


Figure 565 – Cable list

The ES4440 box distinguishes high current and high voltage channels where you have more possibilities when using high current channels. In the cable list you can see the difference. The cables associated with high current pins show additional icons. So for high current cables there are four positions where you can click with your mouse and for the high voltage cables there are only two. When you click on a position, the corresponding fault status toggles.

A description of the symbols from left to right follows. The leftmost one is for the current measurement:

 The current measurement is disabled.

 The current measurement is enabled.

 The current measurement is not possible.

The next one is for a contact resistance:

 The adjustable resistor is not inserted.

 The adjustable resistor is inserted.

 Inserting the adjustable resistor is not possible.

The 3rd one is for breaking the cable:

 The connection is not broken and you are allowed to change it.

 The connection is broken and you are allowed to change it.

 The connection is broken and you are not allowed to change it.

 The connection is not broken and you are not allowed to change it.

The last one is for the rail connection:

 The connection is not established and you are allowed to change it.

 The connection is established and you are allowed to change it.

 The connection is established and you are not allowed to change it.

 The connection is not established and you are not allowed to change it.

Please be aware that connecting a high voltage pin to the rail automatically also breaks the connection to the load or sensor. This is a hardware limitation of the ETAS ES4440 box.

The wiring fault simulation differentiates two states: the active and the inactive state. After launching PROVEtech:TA the fault simulation is in the inactive state. This means that PROVEtech:TA does not transmit the faults you edit directly to the hardware. Press 

to enter the active state and transmit your fault settings to the hardware of one system. After that all changes you make are transmitted directly to the hardware. Press 

to return to the inactive state. This takes back all fault settings from the hardware. The buttons 

and 

behave identically, but they change the state of all systems simultaneously.



In case your configuration only has one system, the buttons `Activate`, `Deactivate` and `Default` are hidden, because in this case their functionality is the same as `Activate All`, `Deactivate All` and `All To Default`.

To break a cable click on its break symbol and select the active state:

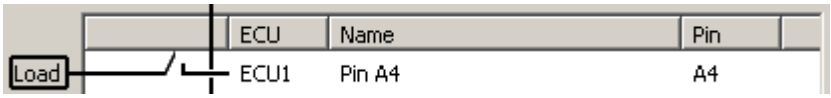


Figure 566 – Cable break

To short two cables connect both to the rail and select the active state:

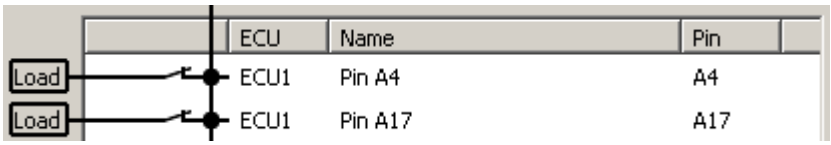


Figure 567 – Short of two cables

To short a cable to a potential (typically battery power or ground) connect the cable to the rail and then connect the rail to the potential:



Figure 568 – Short to potential

To connect the rail to a potential select the potential in the upper combo box. In this example the rail is connected to battery power A and this means also “Pin A4” is connected to it. Activate these settings by switching to the active state.

The current measurement is only available for a single cable and the ES4440 box must have a connection to an external measurement device which, as an example, provides the measurement value as a signal via the model. Click on the left symbol of the cable and then select the active state (not available for high voltage channels):

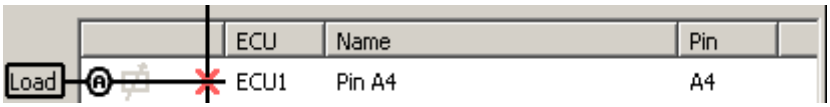


Figure 569 – Current measurement

You can combine different faults by toggling the according settings. For example you can connect one cable to ground via the rail and at the same time you can break a different cable.

Press Default to take back all current settings of a system. Use All To Default to take back all settings of all systems.

7.3.2.3 Contact resistances, duration and loose contacts

The fields below the cable list provide additional settings for extended faults.

The following figure shows how to insert an adjustable resistor between an ECU pin and its load or sensor (not available for high voltage channels):

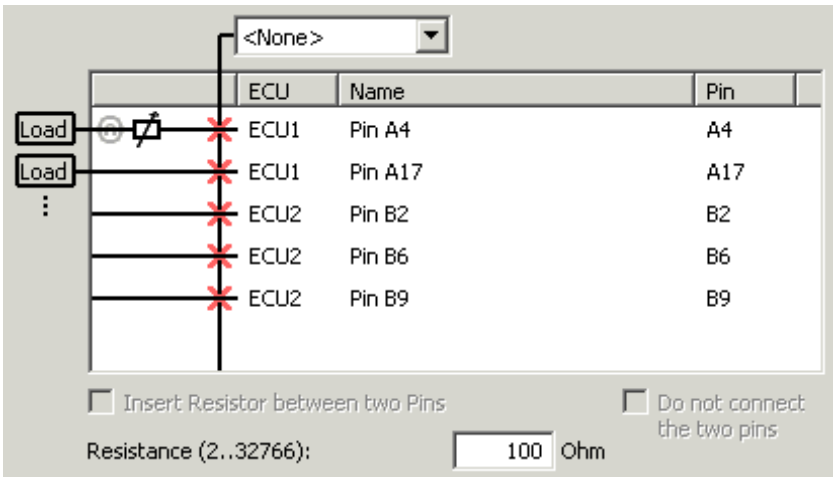


Figure 570 – Resistor in cable

The next setting connects two ECU cables via an adjustable resistor (not available for high voltage channels):

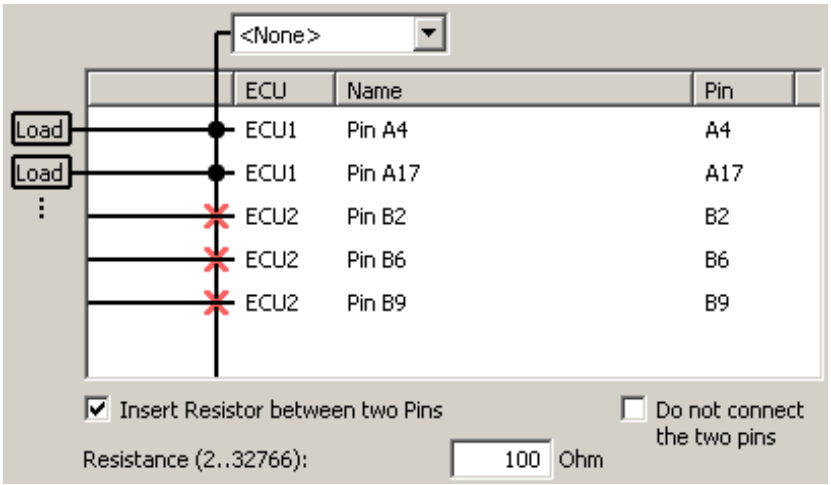


Figure 571 – Resistor between cables

The third variant is a connection of an ECU cable to a potential via an adjustable resistor (not available for high voltage channels):

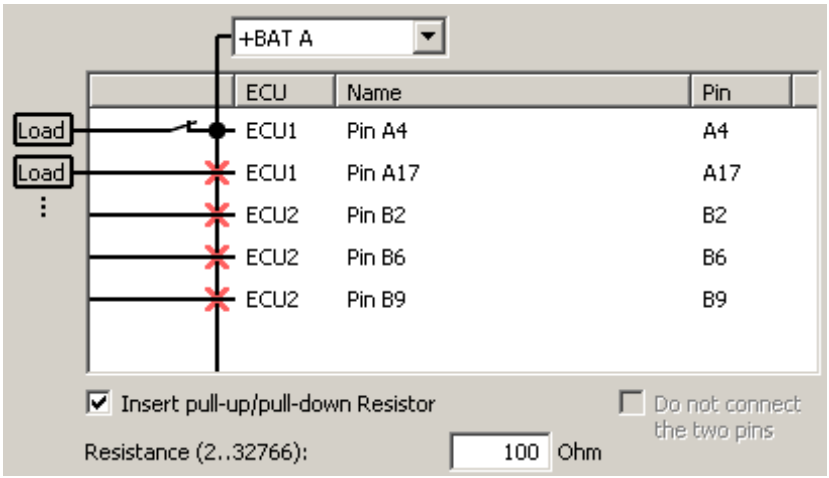


Figure 572 – Resistor to potential

When you activate the fault settings, these settings stay active as long as you do not deactivate them. However, in case you want to test timing critical fault settings, you can use the `Fault Duration` text box to enter a duration in milliseconds. In case you enter a value different from 0 in that box and click the `Activate` button, the fault settings only stay active for the entered amount of time and the system is in the system is in the deactivated state afterwards. PROVEtech:TA is blocked in that time and wait for the response of the ES4440 box. Please keep in mind that the timing of the ES4440 is not absolutely exact.

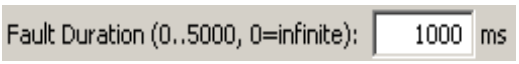


Figure 573 – Fault duration

For some fault settings you can additionally simulate a loose contact (not available for high voltage channels). Just enable the `Simulate Loose Contact` check box and fill the `Frequency` and `Duty Cycle` text boxes as the ES4440 generates the loose contact via a PWM. There is a limitation for 2Hz: In this case the duty cycle only has the range from 35 % to 65 %.

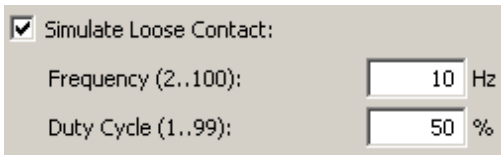


Figure 574 – Loose contact simulation

7.3.3 Special cases

You can use the ES4440 also for voltage measurement on pins via the internal rails. There are three cases:

To connect a pin just with the internal rail 1 of the ES4440, use the following setting:

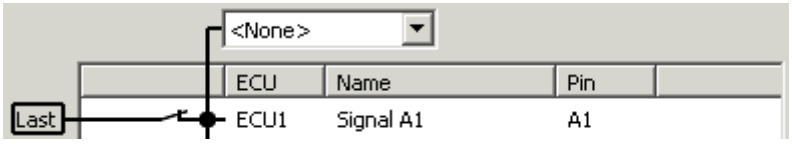


Figure 575 – Voltage measurement on ES4440 rail 1

To connect a pin with the internal rail 2 of the ES4440, use the following setting (setting the break additionally is mandatory for that):



Figure 576 – Voltage measurement on ES4440 rail 2

To connect two pins with the internal rails 1 and 2 for differential voltage measurement, activate the check box `Do not connect the two pins` after connecting two pins with the virtual rail:

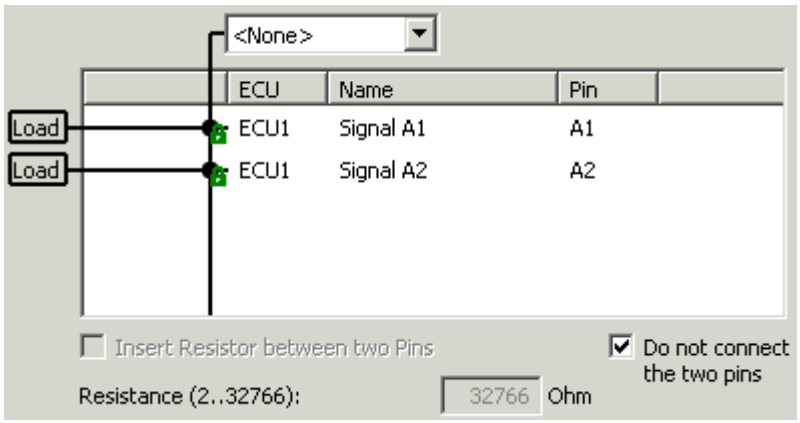


Figure 577 – Differential voltage measurement on ES4440

Please keep in mind that the ES4440 contains no voltage measurement device. Instead of that you must attach an external device to the internal rails of the ES4440.

7.3.4 Fault patterns and reset

Push the buttons 

and 

to load and save fault patterns (see ***Figure 563***). A fault pattern file contains all the settings you can manipulate in the fault simulation. With fault patterns you can quickly save and restore complex or extensive fault settings.

Click 

to completely reset the fault simulation hard- and software.

7.4 dSPACE FullSize fault simulation via CAN

Read this chapter in case you use the dSPACE FullSize fault simulation hardware controlled via CAN.

In this chapter you will find out:

- how the fault simulation function area is structured
- how you can configure wiring faults
- how you can change the CAN topology
- how you set CAN faults

7.4.1 Architecture

Select the tab `Fault Simulation` to display the following panel:

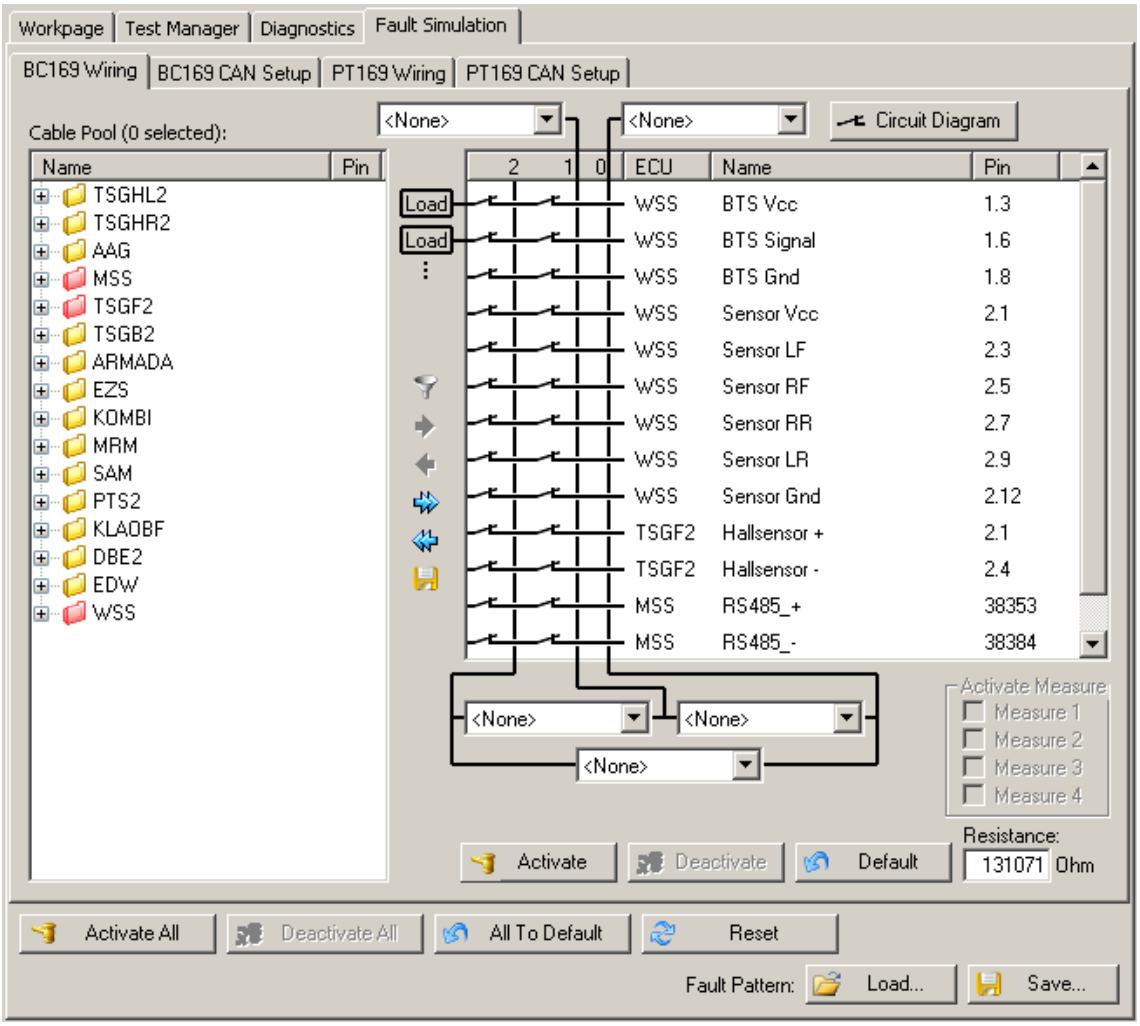


Figure 578 – Architecture of the dSPACE FullSize fault simulation via CAN

In this example the fault simulation controls two systems: BC169 and PT169. The fault simulation displays two tabs per system: one to set wiring faults (here: `BC169 Wiring` and `PT169 Wiring`) and one to manipulate the CAN settings (here: `BC169 CAN Setup` and `PT169 CAN Setup`).

Chapter ***7.4.2 Wiring faults*** describes how to set wiring faults, chapter ***7.4.3 CAN settings*** describes the CAN topology and CAN fault settings and chapter ***7.4.4 Bus faults*** describe how to insert bus faults. Finally chapter ***7.4.5 Fault patterns and reset*** treats fault patterns and the `Reset` button.

7.4.2 Wiring faults

This chapter describes how to simulate different faults concerning the ECU wiring.

7.4.2.1 Preselection of cables

Figure 579 shows the page for wiring faults of a system.

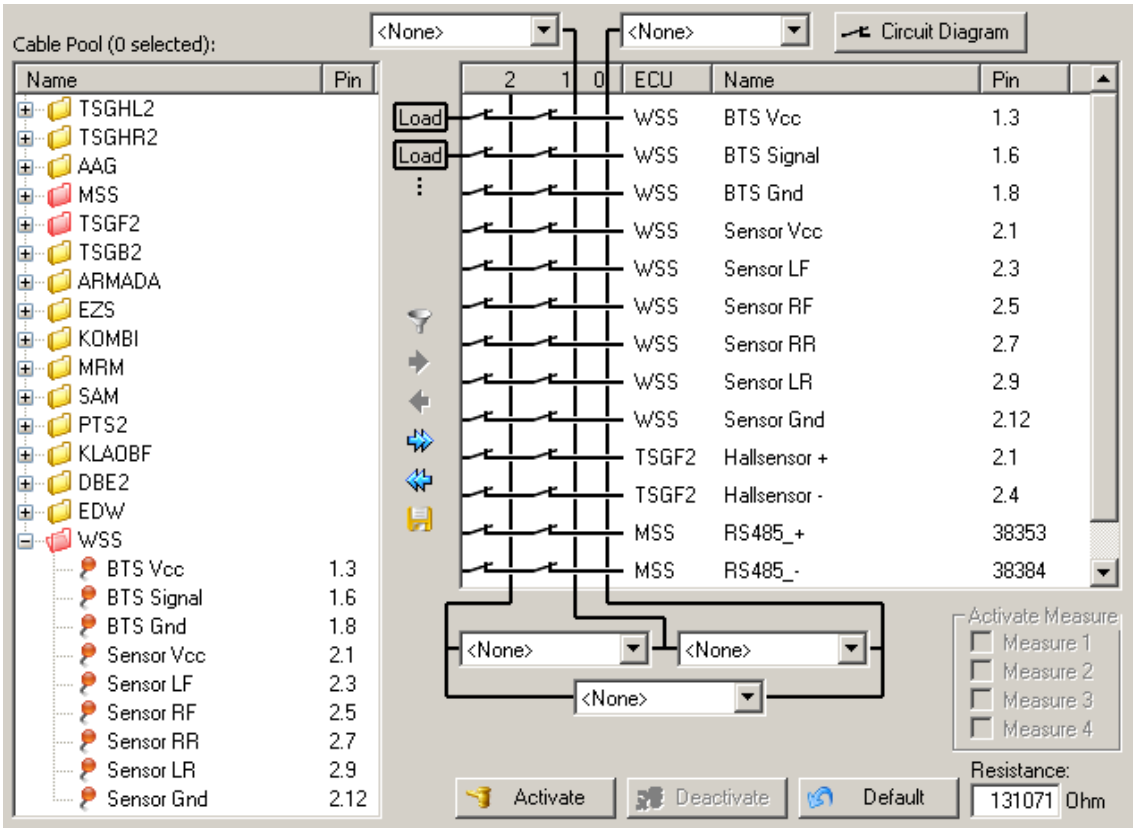


Figure 579 – Page for wiring faults

The `Cable Pool` tree on the left side contains all ECUs in the first level and in the second level all their cables you can manipulate with the fault simulation.

To set wiring faults you must preselect some cables of the cable pool first. The right cable list shows your current preselection. To preselect the cables you have to select them in the cable pool tree and press . As an alternative you can drag them from the tree and drop them in the preselection list. Press to preselect all existing cables. Select one or several cables in the preselection list and press to remove them from the list or press to remove all cables. Again you can drag them from the list and drop them in the cable pool as an alternative.

Red colored icons in the cable pool tree mark all cables that are already preselected.

You cannot remove cables from the preselection that still have fault settings.

Click on the central button to save the current preselection list of all wiring fault pages for your user name and the current configuration. When you click on `Reset` or re-load this configuration, you will get this list again.

To find cables in the cable pool more easily, you can click on or right click on the column headers to open a menu and select *Filters*. This activates a filter bar below the column headers. There you can enter a string for filtering the cable pool. As an example enter “t87” in the column `Name` to display only cables containing “t87”. The string matching is case insensitive. Click on the button again or select the menu item again to close the filter bar.

PROVEtech:TA displays the amount of currently selected cables after the text `Cable Pool` in parenthesis. In case you select an ECU entry, you can detect the amount of cables within that entry.

Double click on a cable name or select *Info…* in the context menu of the cable pool or the preselection list to display a dialog with additional cable information in case you have configured that (see chapter ***3.2.7.9.4 dSPACE FullSize fault simulation via CAN***):

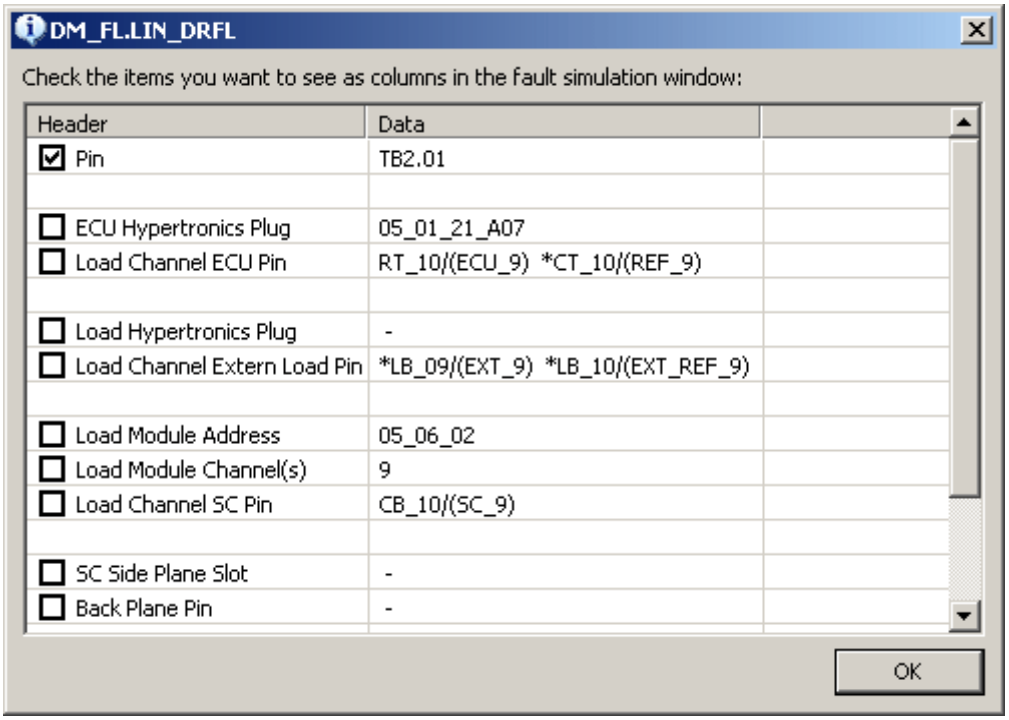


Figure 580 – Additional cable information

To see some of this information as additional columns in the preselection list or the cable pool (depends on which window you have used to open the info dialog), set check marks in the according lines.

7.4.2.2 Shorts and breaks

The hardware architecture of the fault simulation consists of three rails and five relays for each cable. Three relays control the connection to the rails and two relays can break the cable:

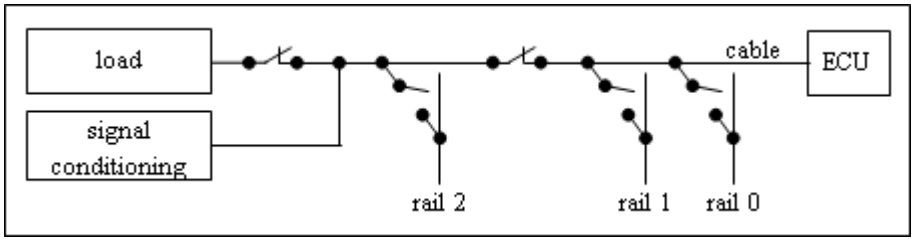


Figure 581 – Hardware architecture

The right break relay disconnects the possibly existing load and the signal conditioning and with it the connection to the real-time computer. The left one only disconnects the load.

PROVEtech:TA replicates the electrical design the following way:

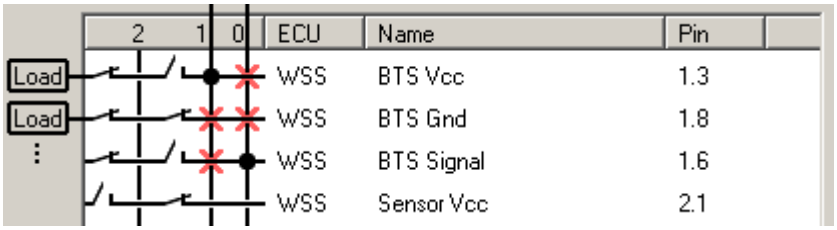


Figure 582 – Cable list

The first column of the list view shows the five relays per cable. The vertical lines are the three rails. The horizontal lines are the control unit cables with two relays (⌐) for breaking it. The left side symbolically displays the loads. The ECU pin is displayed symbolically via the ECU name, the pin name and the pin number. The signal conditioning is missing in this view.

You can click on the symbolic relays to change their states. PROVEtech:TA uses different symbols to show the relay states and the possibility to change the state.

For relays that connect to rails:

- The connection is not established and you are allowed to change it.
- The connection is established and you are allowed to change it.
- The connection is established and you are not allowed to change it (this only exists in the active state).

The connection is not established and you are not allowed to change it.

For relays that break the cable:

The connection is not broken and you are allowed to change it.

The connection is broken and you are allowed to change it.

The connection is broken and you are not allowed to change it (this only exists in the active state).

The connection is not broken and you are not allowed to change it.

The wiring fault simulation differentiates two states: the active and the inactive state. After launching PROVEtech:TA the fault simulation is in the inactive state. This means that PROVEtech:TA does not transmit the faults you edit directly to the hardware. Press Activate to enter the active state and transmit your fault settings to the hardware of one system. After that all changes you make are transmitted directly to the hardware. Press Deactivate to return to the inactive state. This takes back all fault settings from the hardware. The buttons Activate All and Deactivate All behave identically, but they change the state of all systems simultaneously.



In case your configuration only has one system, the buttons `Activate`, `Deactivate` and `Default` are hidden, because in this case their functionality is the same as `Activate All`, `Deactivate All` and `All To Default`.

To break a cable set one of the both break relays and select the active state:

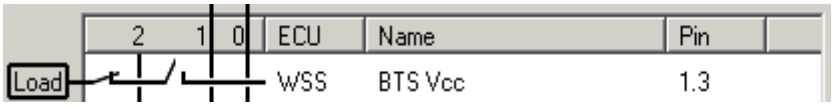


Figure 583 – Cable break

To short two cables connect both to the same rail and select the active state:

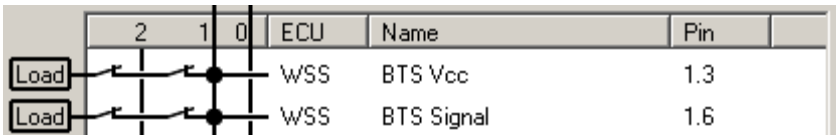


Figure 584 – Short of two cables

To short a cable to a potential (e.g. Terminal 30 or 31) connect the cable to rail 0 or 1 and then connect the rail to the potential:

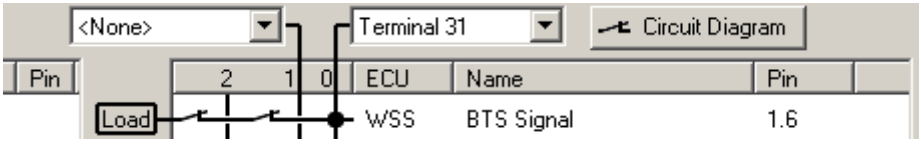


Figure 585 – Short to potential

To connect rail 0 or 1 to a potential select the potential in one of the two upper combo boxes. In this example rail 0 is connected to Terminal 31 and this means also “BTS Signal” is connected to it. Activate these settings by switching to the active state.

You can combine different faults by setting the according relays. For example you can connect one cable to Terminal 30 via rail 0 and at the same time you can short two other cables via rail 2.

Press Default to take back all current settings of a system. Use All To Default to take back all settings of all systems, this includes the CAN settings.

Click on Circuit Diagram to open this chapter of the manual.

7.4.2.3 Contact resistances and external devices

The combo boxes below the cable list represent external devices that you can insert between the rails:

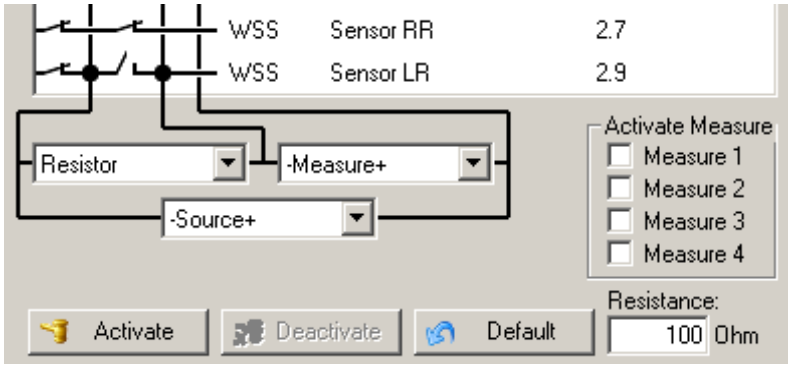


Figure 586 – Resistor and other external devices

The left combo box corresponds to the device between rail 1 and 2, the right one corresponds to the device between rail 0 and 1 and the middle combo box represents the device between rail 0 and 2. The fault simulation differentiates three devices: Resistor (with an adjustable resistance), Source (a signal source) and Measure (a measuring device). You can select six different entries in the combo boxes:

<No Connection>	Connect no device
Resistor	Connect the adjustable resistor
+Source-	Connect the signal source; left rail to +, right rail to -
-Source+	Connect the signal source; left rail to -, right rail to +
+Measure-	Connect the measuring device; left rail to +, right rail to -
-Measure+	Connect the measuring device; left rail to -, right rail to +

You have to connect the signal source and the measuring device manually to the test system before you can use them. When you connect the measuring device, you can additionally use the four check boxes `Measure 1-4`. Activate the check boxes to connect the according terminals MEAS 1-4 of the FIU parallel to the measuring device. The dSPACE document “Hardware documentation of DS293-01 board (FIU Module)” describes the hardware architecture of the FIU.

The resistor is a part of the test system, adjust it in the edit box `Resistance` from 0 to 131071 Ohm with an accuracy of 1 Ohm.

To insert a contact resistance of 100 Ohm in a cable use the following settings as an example and enter the active state:

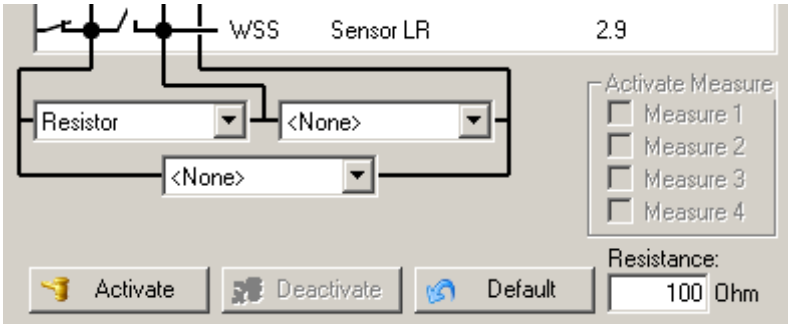


Figure 587 – Contact resistance

For a 100 Ohm leakage resistance to terminal 31 (a cable connected to terminal 31 via a resistor) use the following settings as an example and enter the active state:

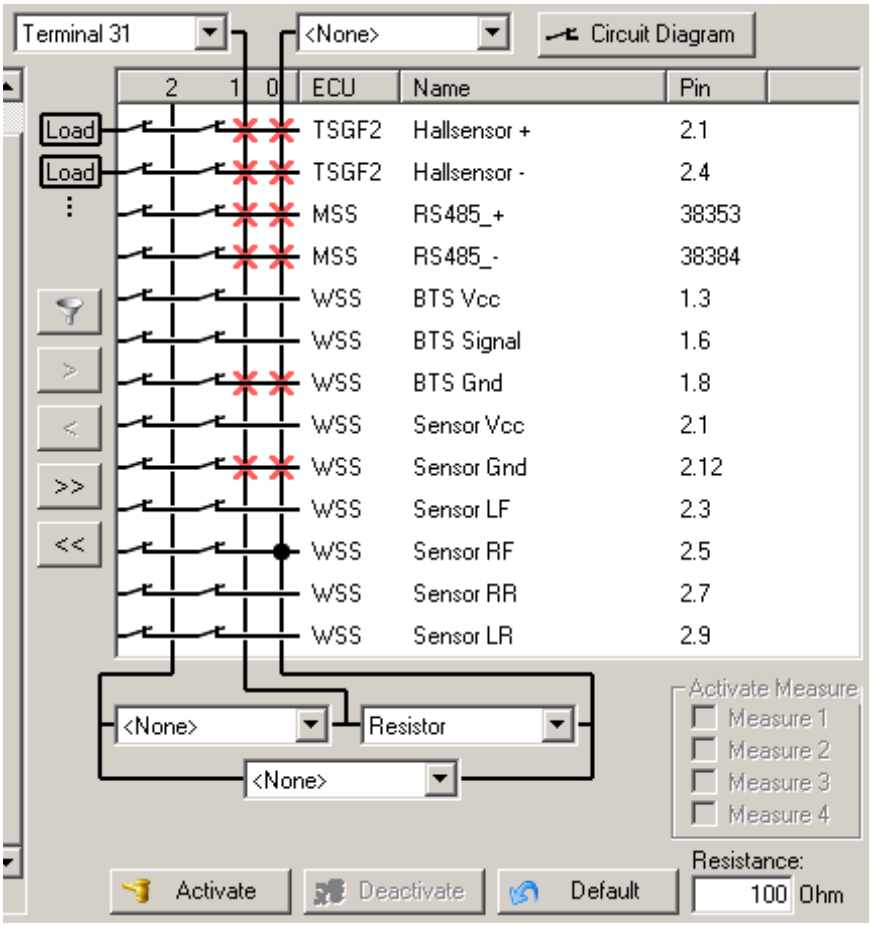


Figure 588 – Leakage resistance

7.4.3 CAN settings

This chapter describes how to manipulate the CAN settings of a system.

7.4.3.1 CAN topology

Figure 589 shows the page for the CAN settings of a system:

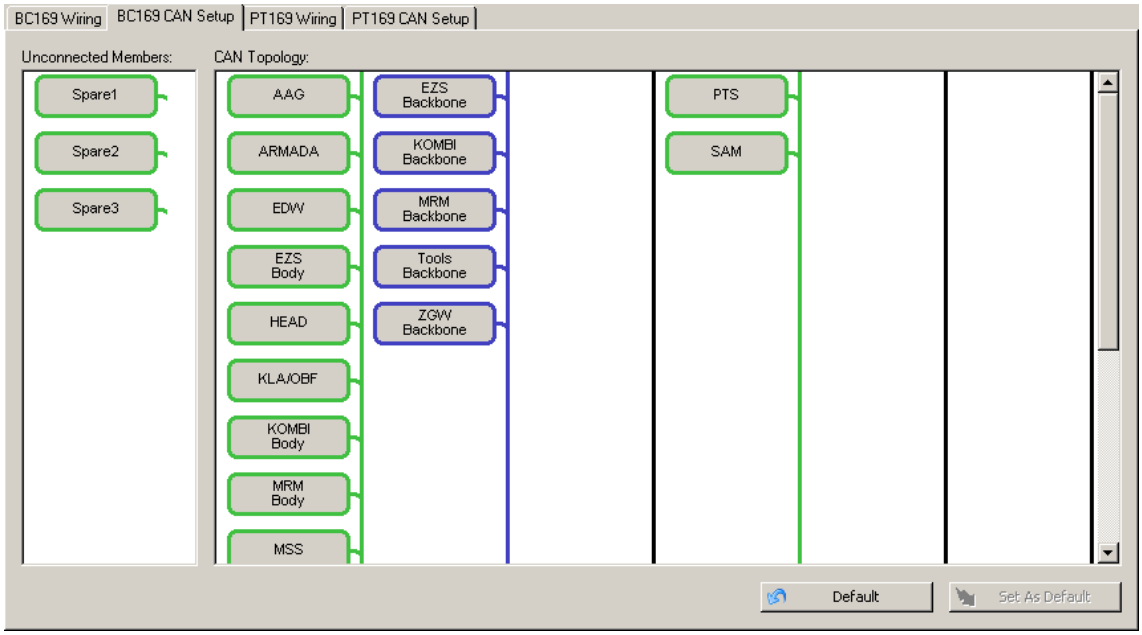


Figure 589 – Page for CAN Settings

You can manipulate the CAN terminals of all CAN members (ECUs, CANoe, HIL system, etc.) in this page.

The left list displays the unconnected CAN members, their CAN terminals are disconnected from the bus. The right list displays six segments (represented by the vertical lines) and the CAN members connected to them. All CAN terminals of the CAN members of one segment are connected to each other.

The color of a symbol of a CAN member represent its CAN type. In this example green means CAN-B and blue means CAN-C. When a segment only connects members of the same CAN type the according vertical line has the color of that CAN type. Otherwise the line is black.

When a CAN member has several different CAN terminals, one symbol for each CAN terminal pair exists. In the upper example the KOMBI ECU has CAN-C and CAN-B terminals. So there is a symbol “KOMBI Body” and a symbol “KOMBI Backbone”.

To manipulate the CAN topology mark one or several symbols and use drag and drop to move them to the destination segment or to the list of unconnected members (***Figure 590***). In opposite to the wiring settings PROVEtech:TA transfers these changes directly to the hardware. The fault simulation does not differentiate an active and an inactive state here.

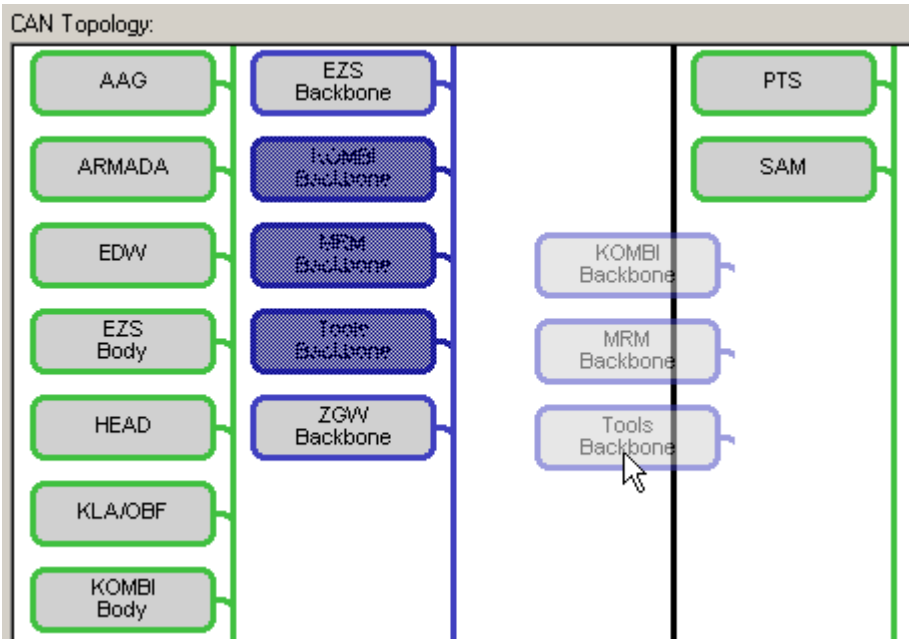


Figure 590 – Manipulating the CAN topology with drag and drop

Click the button 

to reset the CAN topology to the startup state of the system. Only the administrator or a user with *arManage* access rights on the Test Manager's root object can use the button 

. It programs the startup state of the CAN topology. This means when you switch on the system it reactivates the state the system had when the button was pressed.

7.4.3.2 CAN fault settings

To manipulate the fault settings of a CAN member double click on its symbol or select *Insert Fault* in the context menu. The symbol changes the color to red and the elements for setting the fault appear below the topology list (see ***Figure 591***). You can only set faults for one CAN member. When you double click on the symbol of a different CAN member the current faults are reset and PROVEtech:TA prepares the new member for CAN faults.

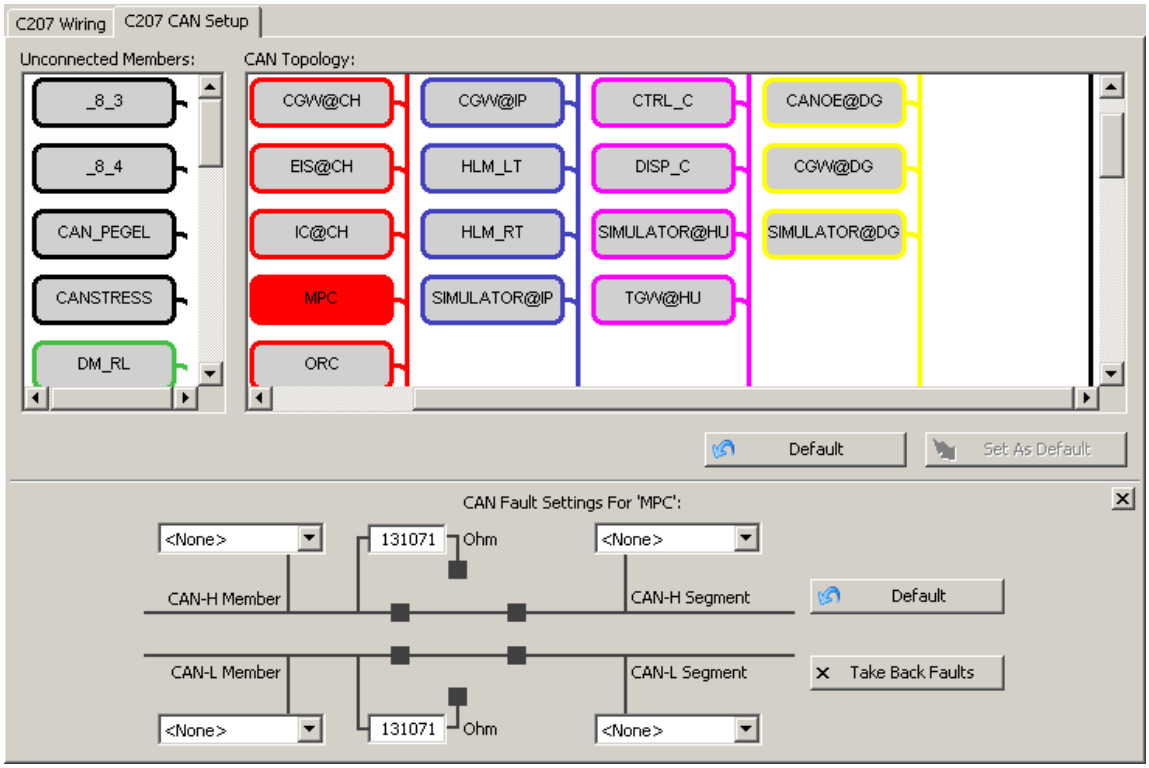


Figure 591 – CAN fault settings

Beside connecting the CAN member to the CAN fault injection unit, the CAN cables keep their normal connections at first. The elements for setting the faults represent a wiring diagram. The two CAN cables of the member run through the fault injection unit and end on the segment (right side of the diagram). You can connect the CAN cables to potentials with the four combo boxes as you can see in the following example:

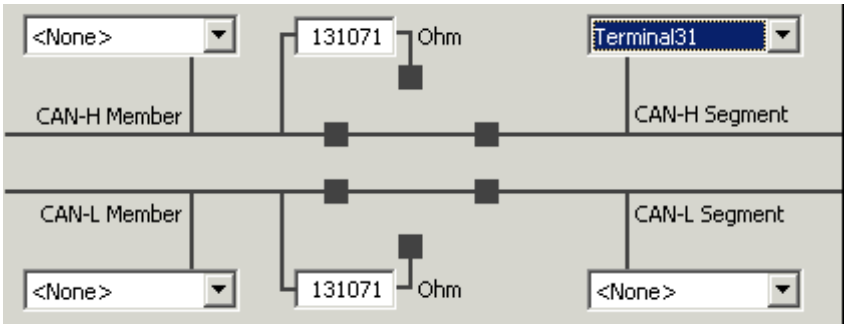


Figure 592 – Connecting CAN-H to Terminal 31

Like changing the CAN topology you do not need to activate your fault settings. The transmission to the hardware is directly linked to your actions.

Use the six boxes in the middle to manipulate the wiring of the CAN cables. Click on one box and it changes its color to white. After that click on an other box to set or reset the connection between the two boxes.

The following figure is an example for crossed CAN cables:

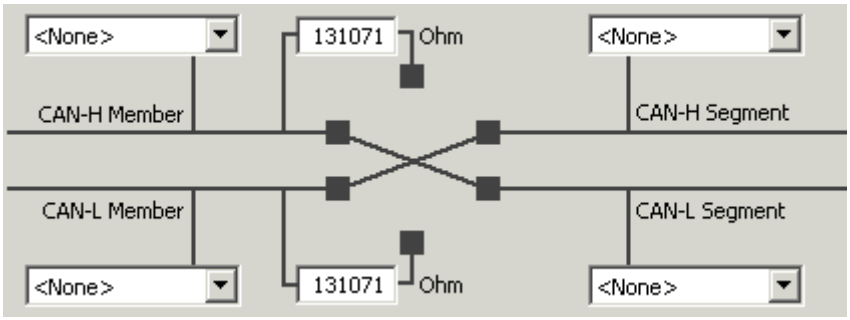


Figure 593 – Crossed CAN cables

Additionally to the normal CAN wiring faults there are two resistors that are adjustable between 0 and 131071 Ohm (with 1 Ohm precision). ***Figure 594*** shows an example how to insert a 100 Ohm contact resistance in the CAN-H cable.

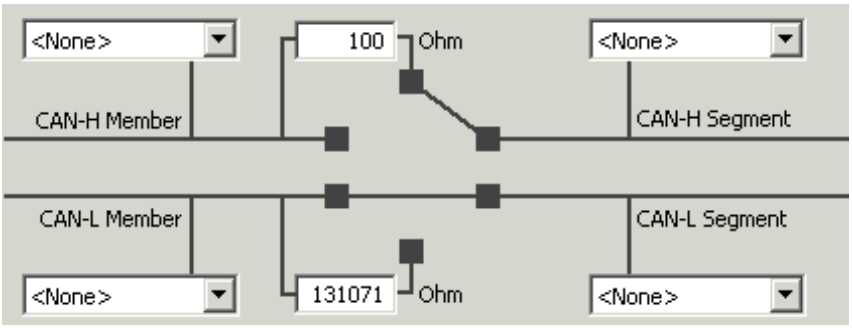


Figure 594 – Contact resistance for CAN-H cable

Click on 

beside the fault setting elements to restore the default state (CAN-H and CAN-L are connected normally). To reset all fault settings push the buttons 

or  or select *Take Back Faults* in the context menu when clicking on the symbol of the member that is connected to the fault injection unit.

7.4.4 Bus faults

This chapter describes how to insert bus faults in case your system has the according modules and you have configured them (see chapter ***3.2.7.9.4 dSPACE FullSize fault simulation via CAN***). In case everything is configured correctly, you can see the tab `Bus Faults` prefixed by the system name. The page looks like ***Figure 595***.

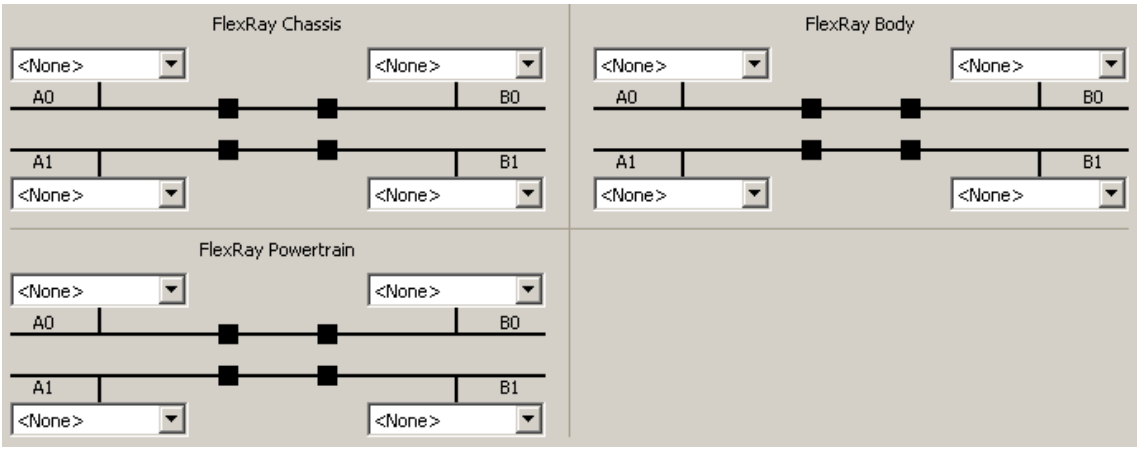


Figure 595 – Bus faults

The controlled bus fault modules are single dSPACE DS295 modules. Originally this module was designed for inserting faults in a connection between a CAN member and the according CAN bus (see ***7.4.3.2 CAN fault settings***). But you can use it also for inserting FlexRay, LIN or Ethernet faults as long as the electrical properties of the board are sufficient for that.

In ***Figure 595*** you can see three modules with their configured names “FlexRay Chassis”, “FlexRay Body” and “FlexRay Powertrain”.

Each module has four pins where two rails get in and out. We call these pins A0, A1, B0 and B1 (pins R1, R2, R3 and R4 of the connector P4 on the DS295). It is possible to connect any pin with any other pin. Additionally you can connect the five fault simulation potentials to each pin.

Click on one of the black squares and then on a different one to open or close the according connection. Use the combo boxes to short the according pin to one of the fault simulation potentials. ***Figure 596*** shows a fault setting for the bus “FlexRay Chassis”.

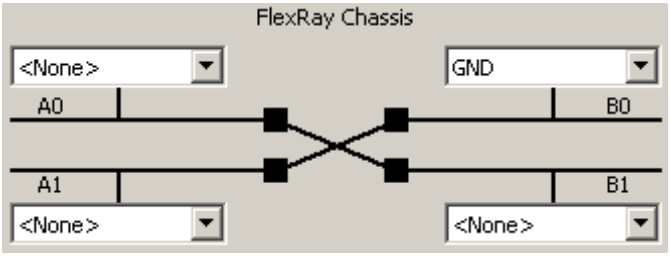
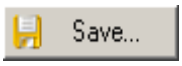


Figure 596 – Crossed cables and short to ground

All fault settings get transferred immediately to the according modules and are activated. You do not need to activate them explicitly.

7.4.5 Fault patterns and reset

Push the buttons 

and 

to load and save fault patterns (see ***Figure 578***). A fault pattern file contains all the settings you can manipulate in the fault simulation. With fault patterns you can quickly save and restore complex or extensive fault settings.

Click 

to completely reset the fault simulation hard- and software.

7.5 dSPACE FullSize fault simulation via RS232

Read this chapter in case you use the dSPACE FullSize fault simulation hardware controlled via RS232.

In this chapter you will find out:

- how the fault simulation function area is structured
- how you can configure wiring faults

7.5.1 Architecture

Select the tab `Fault Simulation` to display the following panel:

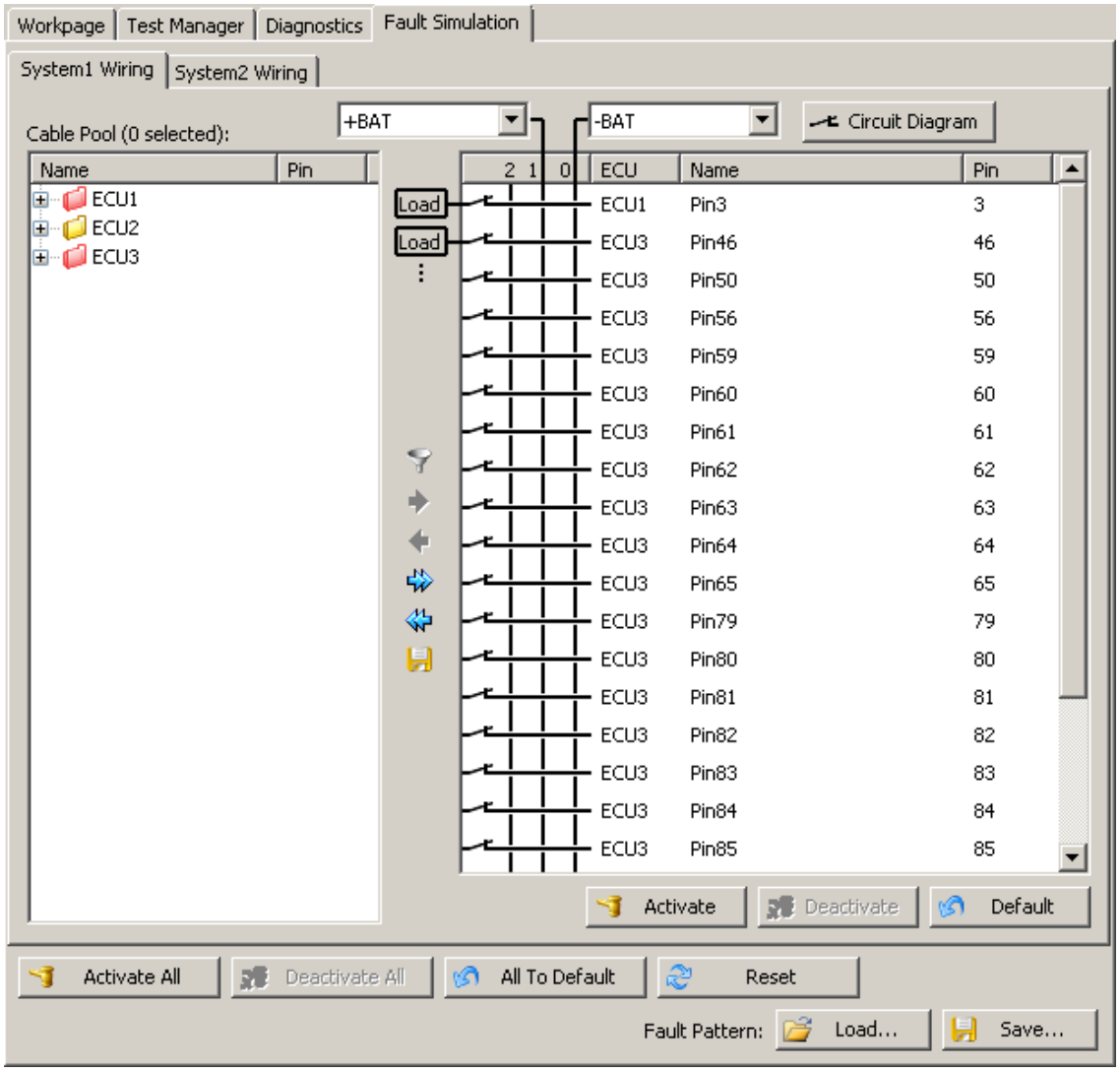


Figure 597 – Architecture of the dSPACE FullSize fault simulation via RS232

In this example the fault simulation controls two systems: System1 and System2 (The names are configurable, see chapter ***3.2.7.9.5 dSPACE MidSize or FullSize fault simulation via RS232***). In most cases you only use one system. For every system the fault simulation displays a tab to set wiring faults (here: `System1 Wiring` and `System2 Wiring`).

Chapter ***7.5.2 Wiring faults*** describes how to set wiring faults. Finally Chapter ***7.5.3 Fault patterns and reset*** treats fault patterns and the `Reset` button.

7.5.2 Wiring faults

This chapter describes how to simulate different faults concerning the ECU wiring.

7.5.2.1 Preselection of cables

Figure 598 shows the page for wiring faults of a system.

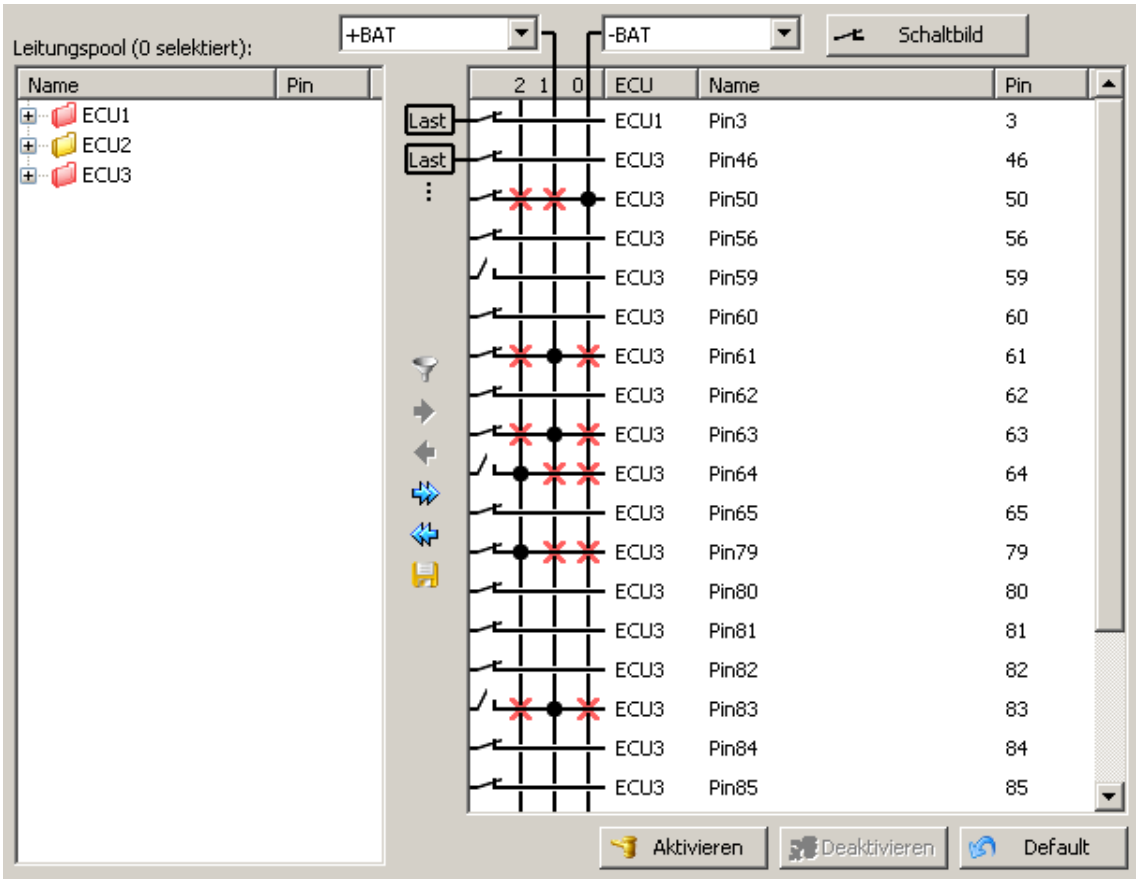


Figure 598 – Page for wiring faults

The `Cable Pool` tree on the left side contains all ECUs in the first level and in the second level all their cables you can manipulate with the fault simulation.

To set wiring faults you must preselect some cables of the cable pool first. The right cable list shows your current preselection. To preselect the cables you have to select them in the cable pool tree and press . As an alternative you can drag them from the tree and drop them in the preselection list. Press to preselect all existing cables. Select one or several cables in the preselection list and press to remove them from the list or press to remove all cables. Again you can drag them from the list and drop them in the cable pool as an alternative.

Red colored icons in the cable pool tree mark all cables that are already preselected.



You cannot remove cables from the preselection that still have fault settings.

Click on the central button to save the current preselection list of all wiring fault pages for your user name and the current configuration. When you click on `Reset` or re-load this configuration, you will get this list again.

To find cables in the cable pool more easily, you can click on or right click on the column headers to open a menu and select *Filters*. This activates a filter bar below the column headers. There you can enter a string for filtering the cable pool. As an example enter “t87” in the column `Name` to display only cables containing “t87”. The string matching is case insensitive. Click on the button again or select the menu item again to close the filter bar.

PROVEtech:TA displays the amount of currently selected cables after the text `Cable Pool` in parenthesis. In case you select an ECU entry, you can detect the amount of cables within that entry.

Double click on a cable name or select *Info…* in the context menu of the cable pool or the preselection list to display a dialog with additional cable information in case you have configured that (see chapter ***3.2.7.9.5 dSPACE MidSize or FullSize fault simulation via RS232***):

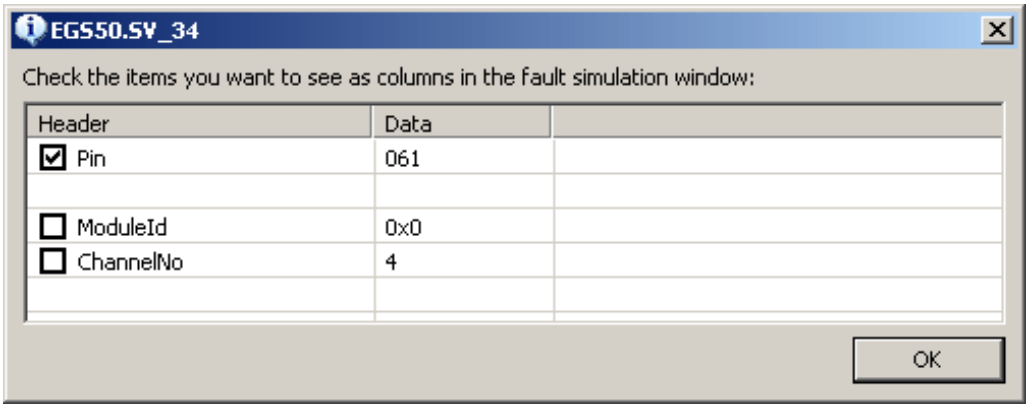


Figure 599 – Additional cable information

To see some of this information as additional columns in the preselection list or the cable pool (depends on which window you have used to open the info dialog), set check marks in the according lines.

7.5.2.2 Shorts and breaks

The hardware architecture of the fault simulation consists of three rails and four relays for each cable. Three relays control the connection to the rails and one relay can break the cable:

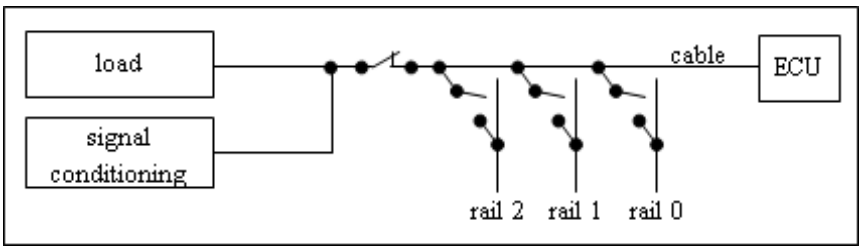


Figure 600 – Hardware architecture

The break relay disconnects the possibly existing load and the signal conditioning and with it the connection to the real-time computer.

PROVEtech:TA replicates the electrical design the following way:

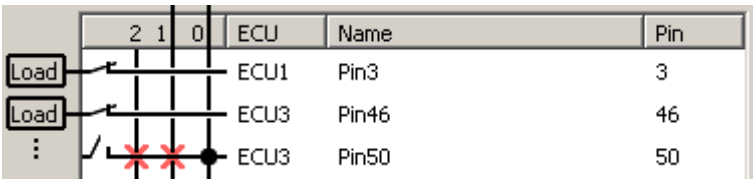


Figure 601 – Cable list

The first column of the list view shows the three relays per cable. The vertical lines are the two rails. The horizontal lines are the control unit cables with the relay (⊸) for breaking it. The left side symbolically displays the loads. The ECU pin is displayed symbolically via the ECU name, the pin name and the pin number. The signal conditioning is missing in this view.

You can click on the symbolic relays to change their states. PROVEtech:TA uses different symbols to show the relay states and the possibility to change the state.

For relays that connect to rails:

- The connection is not established and you are allowed to change it.
- The connection is established and you are allowed to change it.
- The connection is established and you are not allowed to change it (this only exists in the active state).
- The connection is not established and you are not allowed to change it.

For relay that breaks the cable:

- The connection is not broken and you are allowed to change it.
- The connection is broken and you are allowed to change it.
- The connection is broken and you are not allowed to change it (this only exists in the active state).
- The connection is not broken and you are not allowed to change it.

The wiring fault simulation differentiates two states: the active and the inactive state. After launching PROVEtech:TA the fault simulation is in the inactive state. This means that PROVEtech:TA does not transmit the faults you edit directly to the hardware. Press 

to enter the active state and transmit your fault settings to the hardware of one system. After that all changes you make are transmitted directly to the hardware. Press 

to return to the inactive state. This takes back all fault settings from the hardware. The buttons 

and 

behave identically, but they change the state of all systems simultaneously.



In case your configuration only has one system, the buttons `Activate`, `Deactivate` and `Default` are hidden, because in this case their functionality is the same as `Activate All`, `Deactivate All` and `All To Default`.

To break a cable set the break relay and select the active state:

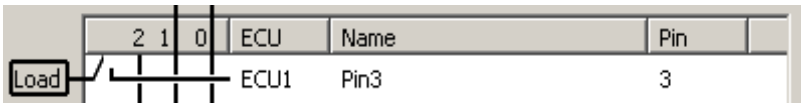


Figure 602 – Cable break

To short two cables connect both to rail 2 and select the active state:

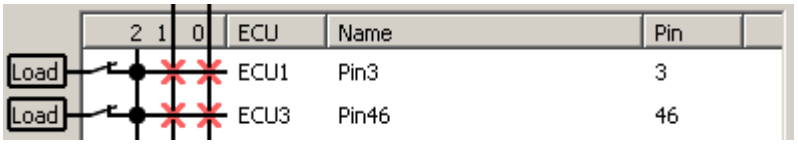


Figure 603 – Short of two cables

To short a cable to a potential connect the cable to rail 0 (fixed on ground) or 1 (fixed on battery power) and select the active state:

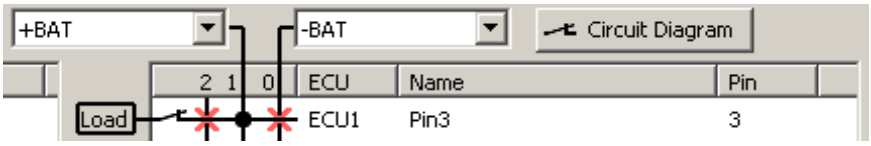


Figure 604 – Short to potential

In opposite to other fault simulation variants, rail 0 and 1 are fixed to the ground and battery power and it is not possible to change this, thus the combo boxes only contain one unchangeable entry.

You can combine different faults by setting the according relays. For example you can connect one cable to ground via rail 0 and at the same time you can short two other cables via rail 2.

Press 

to take back all current settings of a system. Use 

to take back all settings of all systems.

Click on 

to open this chapter of the manual.

7.5.3 Fault patterns and reset

Push the buttons 

and 

to load and save fault patterns (see ***Figure 597***). A fault pattern file contains all the settings you can manipulate in the fault simulation. With fault patterns you can quickly save and restore complex or extensive fault settings.

Click 

to completely reset the fault simulation hard- and software.

7.6 dSPACE MidSize fault simulation

Read this chapter in case you use the dSPACE MidSize fault simulation hardware.

In this chapter you will find out:

- how the fault simulation function area is structured
- how you can configure wiring faults

7.6.1 Architecture

Select the tab `Fault Simulation` to display the following panel:

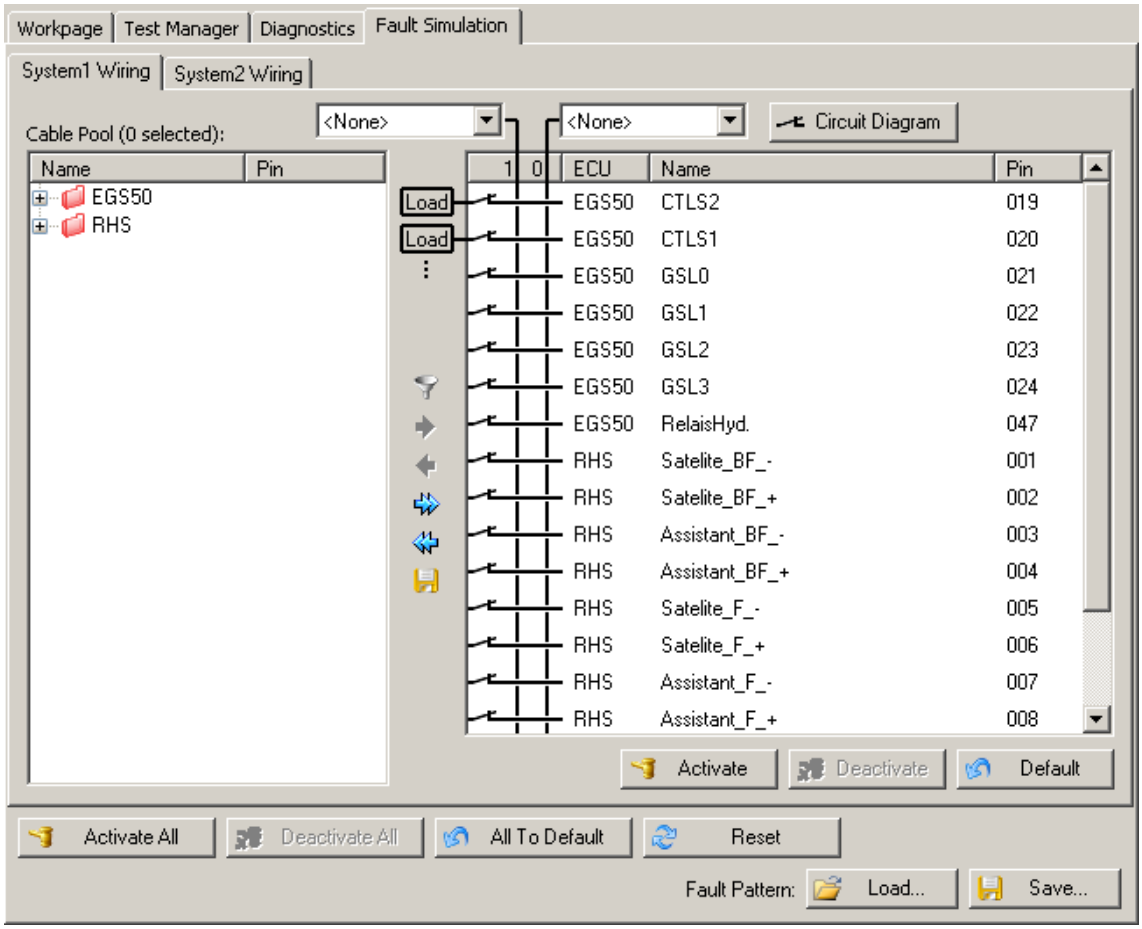


Figure 605 – Architecture of the dSPACE MidSize fault simulation

In this example the fault simulation controls two systems: System1 and System2 (The names are configurable, see chapter ***3.2.7.9.5 dSPACE MidSize or FullSize fault simulation via RS232***). In most cases you only use one system. For every system the fault simulation displays a tab to set wiring faults (here: `System1 Wiring` and `System2 Wiring`).

Chapter ***7.6.2 Wiring faults*** describes how to set wiring faults. Finally Chapter ***7.6.3 Fault patterns and reset*** treats fault patterns and the `Reset` button.

7.6.2 Wiring faults

This chapter describes how to simulate different faults concerning the ECU wiring.

7.6.2.1 Preselection of cables

Figure 606 shows the page for wiring faults of a system.

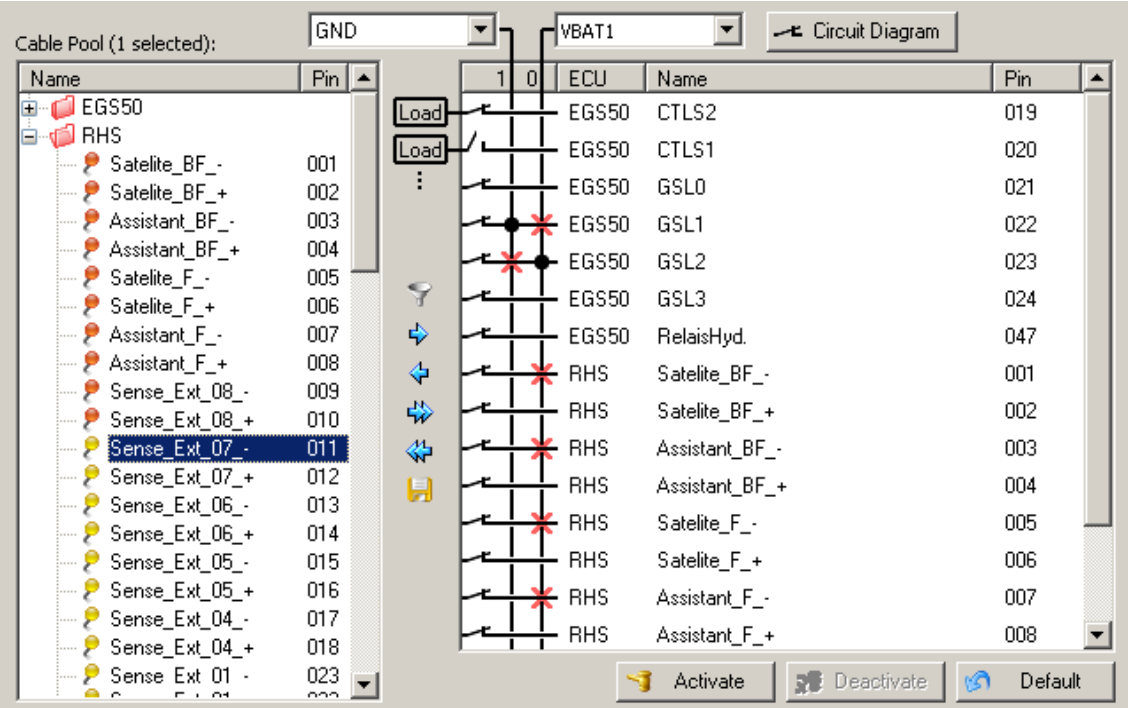


Figure 606 – Page for wiring faults

The `Cable Pool` tree on the left side contains all ECUs in the first level and in the second level all their cables you can manipulate with the fault simulation.

To set wiring faults you must preselect some cables of the cable pool first. The right cable list shows your current preselection. To preselect the cables you have to select them in the cable pool tree and press . As an alternative you can drag them from the tree and drop them in the preselection list. Press to preselect all existing cables. Select one or several cables in the preselection list and press to remove them from the list or press to remove all cables. Again you can drag them from the list and drop them in the cable pool as an alternative.

Red colored icons in the cable pool tree mark all cables that are already preselected.



You cannot remove cables from the preselection that still have fault settings.

Click on the central button to save the current preselection list of all wiring fault pages for your user name and the current configuration. When you click on `Reset` or re-load this configuration, you will get this list again.

To find cables in the cable pool more easily, you can click on  or right click on the column headers to open a menu and select *Filters*. This activates a filter bar below the column headers. There you can enter a string for filtering the cable pool. As an example enter “t87” in the column `Name` to display only cables containing “t87”. The string matching is case insensitive. Click on the button again or select the menu item again to close the filter bar.

PROVEtech:TA displays the amount of currently selected cables after the text `Cable Pool` in parenthesis. In case you select an ECU entry, you can detect the amount of cables within that entry.

Double click on a cable name or select *Info…* in the context menu of the cable pool or the preselection list to display a dialog with additional cable information in case you have configured that (see chapter ***3.2.7.9.5 dSPACE MidSize or FullSize fault simulation via RS232***):

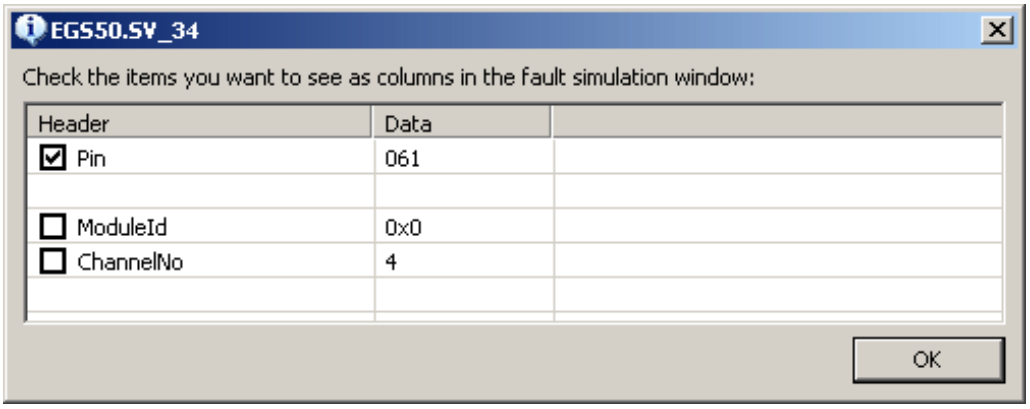


Figure 607 – Additional cable information

To see some of this information as additional columns in the preselection list or the cable pool (depends on which window you have used to open the info dialog), set check marks in the according lines.

7.6.2.2 Shorts and breaks

The hardware architecture of the fault simulation consists of two rails and three relays for each cable. Two relays control the connection to the rails and one relay can break the cable:

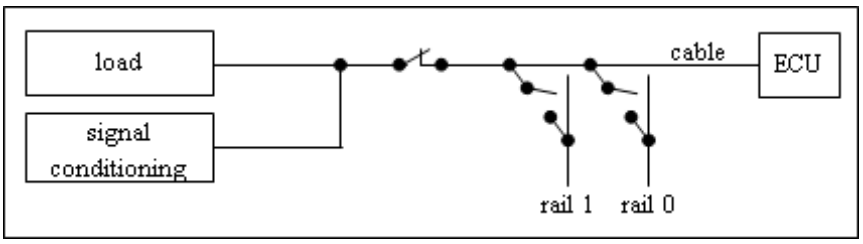


Figure 608 – Hardware architecture

The break relay disconnects the possibly existing load and the signal conditioning and with it the connection to the real-time computer.

PROVEtech:TA replicates the electrical design the following way:

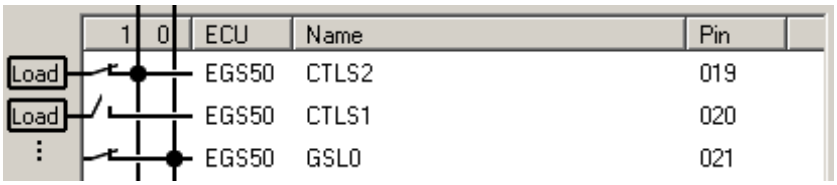


Figure 609 – Cable list

The first column of the list view shows the three relays per cable. The vertical lines are the two rails. The horizontal lines are the control unit cables with the relay (⊸) for breaking it. The left side symbolically displays the loads. The ECU pin is displayed symbolically via the ECU name, the pin name and the pin number. The signal conditioning is missing in this view.

You can click on the symbolic relays to change their states. PROVEtech:TA uses different symbols to show the relay states and the possibility to change the state.

For relays that connect to rails:

- The connection is not established and you are allowed to change it.
- The connection is established and you are allowed to change it.
- The connection is established and you are not allowed to change it (this only exists in the active state).
- The connection is not established and you are not allowed to change it.

For relay that breaks the cable:

- The connection is not broken and you are allowed to change it.
- The connection is broken and you are allowed to change it.
- The connection is broken and you are not allowed to change it (this only exists in the active state).
- The connection is not broken and you are not allowed to change it.

The wiring fault simulation differentiates two states: the active and the inactive state. After launching PROVEtech:TA the fault simulation is in the inactive state. This means that PROVEtech:TA does not transmit the faults you edit directly to the hardware. Press 

to enter the active state and transmit your fault settings to the hardware of one system. After that all changes you make are transmitted directly to the hardware. Press 

to return to the inactive state. This takes back all fault settings from the hardware. The buttons 

and 

behave identically, but they change the state of all systems simultaneously.



In case your configuration only has one system, the buttons `Activate`, `Deactivate` and `Default` are hidden, because in this case their functionality is the same as `Activate All`, `Deactivate All` and `All To Default`.

To break a cable set the break relay and select the active state:

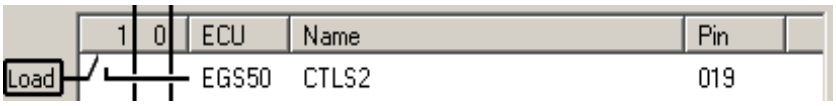


Figure 610 – Cable break

To short two cables connect both to the same rail and select the active state:

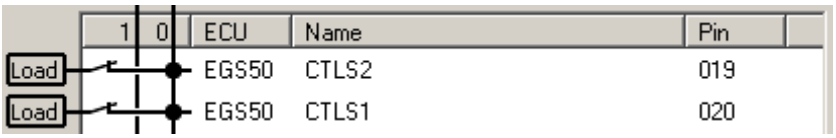


Figure 611 – Short of two cables

To short a cable to a potential (typically battery power or ground) connect the cable to rail 0 or 1 and then connect the rail to the potential:

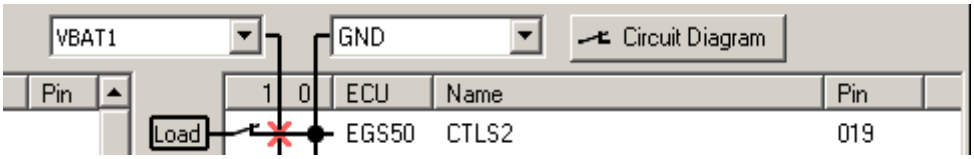


Figure 612 – Short to potential

To connect rail 0 or 1 to a potential select the potential in one of the two upper combo boxes. In this example rail 0 is connected to ground and this means also “CTLS2” is connected to it. Activate these settings by switching to the active state.

You can combine different faults by setting the according relays. For example you can connect one cable to ground via rail 0 and at the same time you can short two other cables via rail 1.

Press 

to take back all current settings of a system. Use 

to take back all settings of all systems.

Click on 

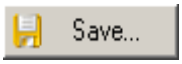
to open this chapter of the manual.



In case your fault simulation hardware has no backplane (DS686) and your configuration contains this information, the two combo boxes for selecting the potentials are not available.

7.6.3 Fault patterns and reset

Push the buttons 

and 

to load and save fault patterns (see ***Figure 605***). A fault pattern file contains all the settings you can manipulate in the fault simulation. With fault patterns you can quickly save and restore complex or extensive fault settings.

Click 

to completely reset the fault simulation hard- and software.

7.7 dSPACE SCALEXIO fault simulation

Read this chapter in case you use the dSPACE SCALEXIO for the fault simulation.

In this chapter you will find out:

- how the fault simulation function area is structured
- how you can configure wiring faults



PROVEtech:TA uses the dSPACE XiL-API to control the SCALEXIO fault simulation. This API uses the running model. Please ensure that the .portconfig file (see chapter ***3.2.7.9.6 dSPACE SCALEXIO fault simulation***) always fits to the currently running model to avoid any damage. Update the .portconfig file after every relevant model change and do not switch to a different model without ensuring that the fault simulation runs with the matching .portconfig file.



Using the SCALEXIO fault simulation does not work without an installation of the dSPACE XiL-API. The dSPACE XiL-API consists of several .NET assemblies. PROVEtech:TA loads these assemblies dynamically at runtime to be compatible to different versions of that API. However, it is not sure that PROVEtech:TA works with any other dSPACE XiL-API version than the one used while testing PROVEtech:TA.



To be able to access the SCALEXIO fault simulation hardware it is necessary to load the according SCALEXIO model via PROVEtech:TA as the access needs additional parameters of the PROVEtech:TA model configuration for that (e.g. path to the .sdf file).



At the moment PROVEtech:TA does not support the usage of dynamic fault settings with frequencies or timers.

7.7.1 Architecture

Select the tab `Fault Simulation` to display the following panel:

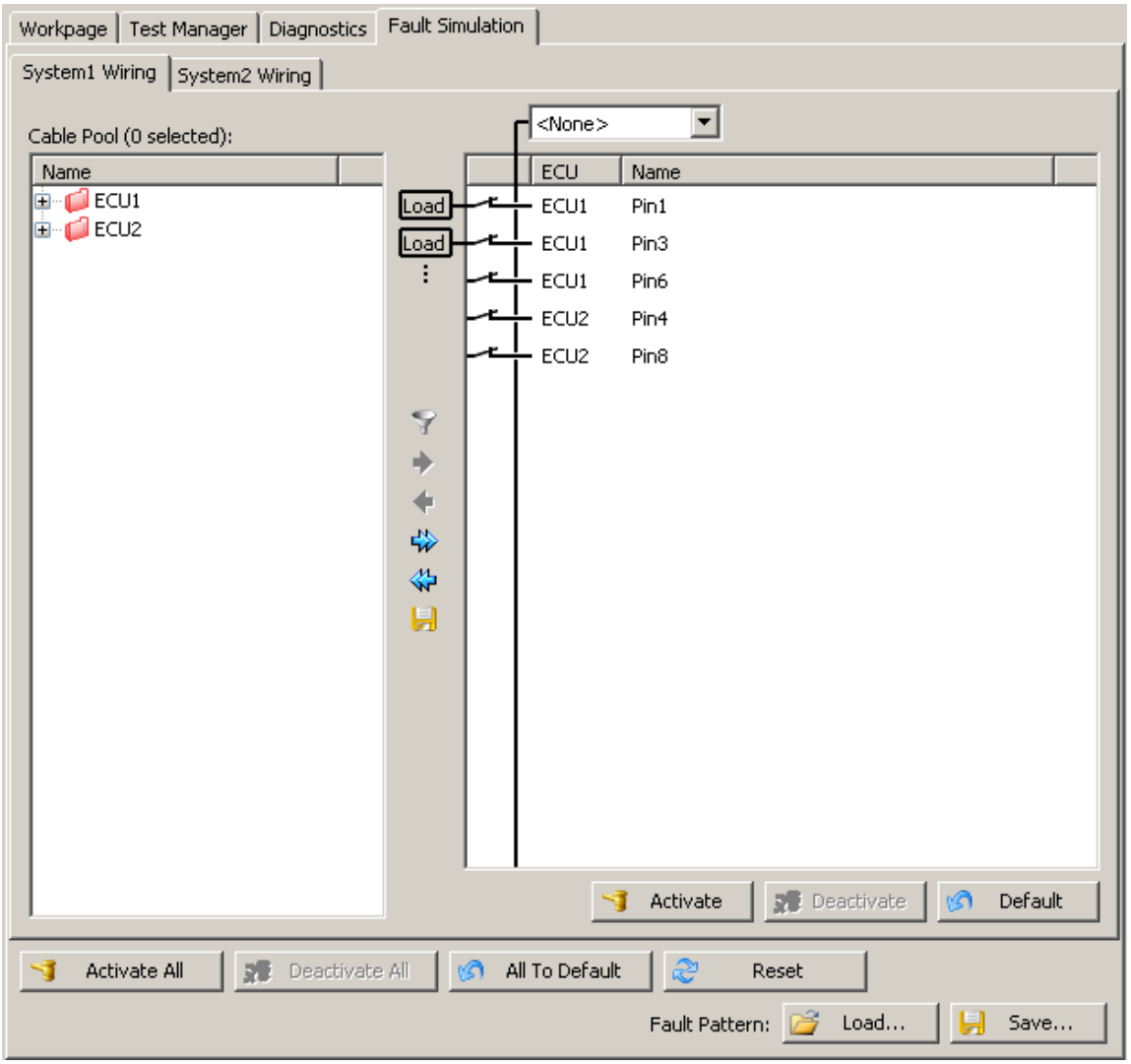


Figure 613 – Architecture of the dSPACE SCALEXIO fault simulation

In this example the fault simulation controls two systems: System1 and System2 (The names are configurable, see chapter ***3.2.7.9.6 dSPACE SCALEXIO fault simulation***). In most cases you only use one system. For every system the fault simulation displays a tab to set wiring faults (here: `System1 Wiring` and `System2 Wiring`).

Chapter ***7.7.2 Wiring faults*** describes how to set wiring faults. Finally Chapter ***7.7.3 Fault patterns and reset*** treats fault patterns and the `Reset` button.



At the moment it is not possible to control two SCALEXIO fault simulation systems at the same time. However, you can control a SCALEXIO fault simulation system and a different one at the same time.

7.7.2 Wiring faults

This chapter describes how to simulate different faults concerning the ECU wiring.

7.7.2.1 Preselection of cables

Figure 614 shows the page for wiring faults of a system.

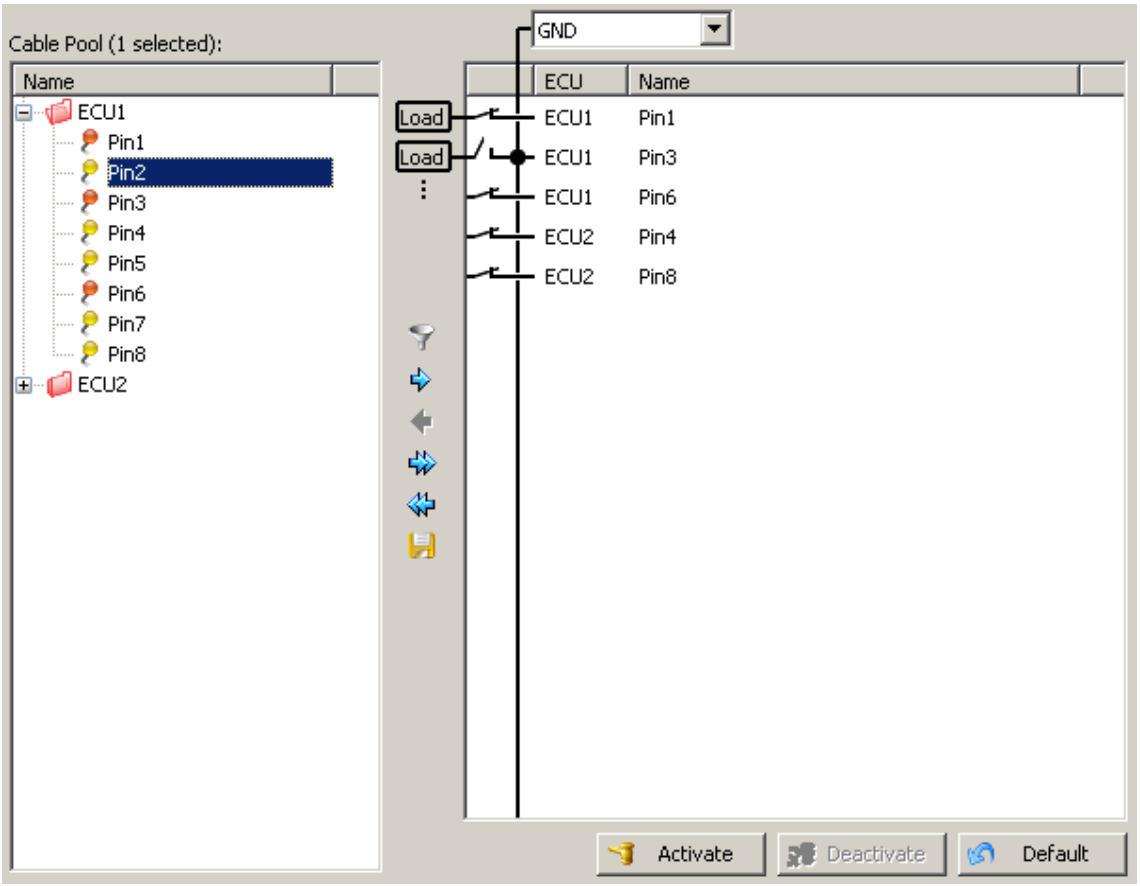


Figure 614 – Page for wiring faults

The `Cable Pool` tree on the left side contains all ECUs in the first level and in the second level all their cables you can manipulate with the fault simulation.

To set wiring faults you must preselect some cables of the cable pool first. The right cable list shows your current preselection. To preselect the cables you have to select them in the cable pool tree and press . As an alternative you can drag them from the tree and drop them in the preselection list. Press to preselect all existing cables. Select one or several cables in the preselection list and press to remove them from the list or press to remove all cables. Again you can drag them from the list and drop them in the cable pool as an alternative.

Red colored icons in the cable pool tree mark all cables that are already preselected.

You cannot remove cables from the preselection that still have fault settings.

Click on the central button to save the current preselection list of all wiring fault pages for your user name and the current configuration. When you click on `Reset` or re-load this configuration, you will get this list again.

To find cables in the cable pool more easily, you can click on or right click on the column headers to open a menu and select *Filters*. This activates a filter bar below the column headers. There you can enter a string for filtering the cable pool. As an example enter “t87” in the column `Name` to display only cables containing “t87”. The string matching is case insensitive. Click on the button again or select the menu item again to close the filter bar.

PROVEtech:TA displays the amount of currently selected cables after the text `Cable Pool` in parenthesis. In case you select an ECU entry, you can detect the amount of cables within that entry.

7.7.2.2 Shorts and breaks

In opposite to most other fault simulation types, PROVEtech:TA does not directly control the SCALEXIO fault simulation but uses the dSPACE XiL-API. So the user interface in PROVEtech:TA does not directly represent the hardware but just helps you in configuring the fault settings.

The cable list represents the whole path of an ECU pin to the load or sensor. On the right side is the ECU pin and on the left side is the load or sensor (represented by the “Load” rectangles on the user interface). Between them are the fault settings (1st column of the list) – at the beginning there are no faults inserted and the ECU pin is connected to the load or sensor:

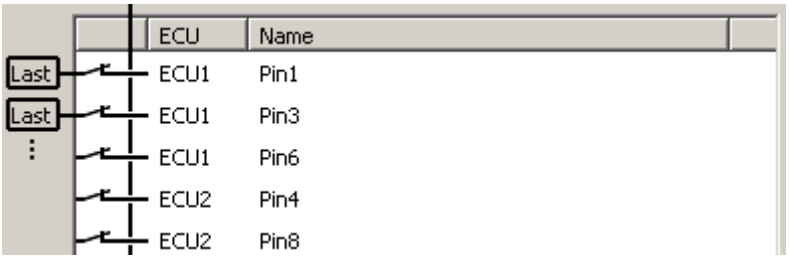


Figure 615 – Cable list

You can click on the symbols in the first column to change them.

A description of the symbols on the left side of the first column follows:

- The connection is not broken.
- The connection is broken.
- The connection is broken and you are not allowed to change it (activated).
- The connection is not broken and you are not allowed to change it (activated).

The right side is for connections to a rail:

- The connection is not established.
- The connection is established.
- The connection is established and you are not allowed to change it (activated).
- The connection is not established and you are not allowed to change it (activated).

The wiring fault simulation differentiates two states: the active and the inactive state. After launching PROVEtech:TA the fault simulation is in the inactive state. This means that PROVEtech:TA does not transmit the faults you edit directly to the hardware. Press Activate to enter the active state and transmit your fault settings to the hardware of one system. After that all changes you make are transmitted directly to the hardware. Press Deactivate to return to the inactive state. This takes back all fault settings from the hardware. The buttons Activate All and Deactivate All behave identically, but they change the state of all systems simultaneously.



In case your configuration only has one system, the buttons `Activate`, `Deactivate` and `Default` are hidden, because in this case their functionality is the same as `Activate All` and `Deactivate All`.

To break a cable click on its break symbol and select the active state:



Figure 616 – Cable break

To short two cables connect both to the rail and select the active state:



Figure 617 – Short of two cables

To short a cable to a potential (typically battery power or ground) connect the cable to the rail and then connect the rail to the potential:



Figure 618 – Short to potential

To connect the rail to a potential select the potential in the upper combo box. In this example the rail is connected to ground (GND) and this means also “Pin1” is connected to it. Activate these settings by switching to the active state.

You can combine different faults by toggling the according settings. For example you can connect one cable to ground via the rail and at the same time you can break it.

Press Default to take back all current settings of a system. Use All To Default to take back all settings of all systems.



As PROVEtech:TA only has the information of the .portconfig file to control a SCALEXIO fault simulation system, it does not know, which ECU pins support which fault settings. Only when activating with a real system the underlying dSPACE XiL-API returns an error message in case the chosen settings are not possible.



The dSPACE XiL-API does not support changing any fault settings in the activated state.

7.7.3 Fault patterns and reset

Push the buttons Load... and Save... to load and save fault patterns (see ***Figure 613***). A fault pattern file contains all the settings you can manipulate in the fault simulation. With fault patterns you can quickly save and restore complex or extensive fault settings.

Click 

to completely reset the fault simulation hard- and software.

7.8 dSPACE DS5394C1 fault simulation

Read this chapter in case you use the dSPACE DS5394C1 CAN gateway modules. In opposite to all other fault simulation types this one has no typical wiring faults but just changes the bus configuration. Additionally, controlling it works via signals of your model. Thus, your model must have the according dSPACE module to provide these signals. Here is an example of the view for 2 buses with several ECUs:

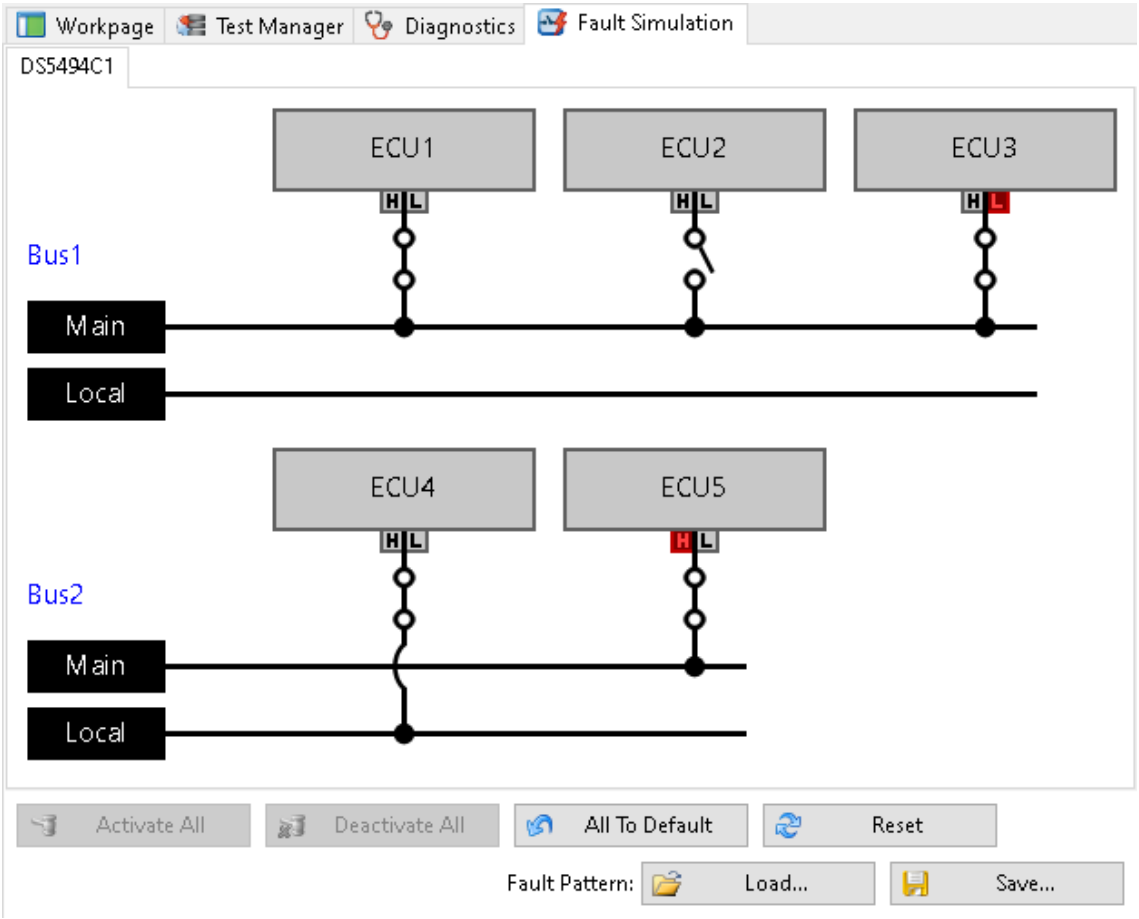


Figure 619 – DS5394C1 fault simulation

For each ECU there are 4 signals:
- Switch between Main and Local (see upper ECU1 vs. ECU4)
- Disconnect the whole ECU (see upper ECU2)
- Disconnect the CAN Low pin (see upper ECU3)
- Disconnect the CAN High pin (see upper ECU5)

This view does not cover the fault simulation rail of the DS5394C1 module. There are other signals for that which you can control separately. For more detailed information please read the dSPACE documentation.

Click on the according symbols to switch the signals. The button `All To Default` sets the signals to their default values. This only works if your dSPACE .trc file provides these default values and if you have not disabled storing them. The button `Reset` additionally reloads the whole fault simulation configuration.

The button Save… saves your current settings (signal values) and Load… later loads them again.

The display internally uses the workpage and its view scaling (***4.1.1 View scaling***). So, changing the view scaling there also changes the displayed size on the `Fault Simulation` tab.

See chapter for ***3.2.7.9.7 dSPACE DS5394C1 fault simulation*** configuring this fault simulation variant.



The signal values must be 0 or 1. The behavior for other signal values is undefined. In case a signal does not exist, the according part of the view will be missing.

Chapter

8

Troubleshooting

This chapter describes the following topics:

- ***8.1 Problems concerning the model***
- ***8.2 Problems concerning the workpage***
- ***8.3 Problems concerning the Test Manager***
- ***8.4 Problems concerning the database***
- ***8.5 Problems concerning test programming***
- ***8.8 Problems concerning the eCOM***
- ***8.9 Problems concerning the CAESAR diagnostics***
- ***8.10 Problems concerning the MCD-3 D diagnostics***
- ***8.11 Problems concerning the fault simulation***
- ***8.12 Other issues***
- ***8.13 Tips and tricks***

8.1 Problems concerning the model

PROVEtech:TA supports several different hardware types, so the problems that occur when accessing the model can vary very much. The communication with the model is different for the hardware types, but there are three basic variants:

a) Communication via shared memory in the real-time computer (for dSPACE PHS bus systems, but not SCALEXIO)

b) The model runs within the PROVEtech:TA process (DLL variant or offline simulation without real model connection)

c) Communication via TCP/IP (all other systems, such as PROVEtech:RE or dSPACE SCALEXIO)

For a) and c) there is a special software module called ModelCom that needs to be linked into the model. Please make sure that you always use a current version of ModelCom in your model.

For all variants there is a directory “Goodies\ModelComAndRTAE” on the PROVEtech:TA installation medium containing instructions for connecting the model to PROVEtech:TA. In most cases, it is sufficient to follow this instruction once more step by step to fix the problem. Please check, that the used PROVEtech:TA version supports all used 3rd party software versions.

For a more detailed problem analysis concerning model trouble please read the sub chapters:

- ***8.1.1 Problem analysis concerning dSPACE systems***
- ***8.1.2 Problem analysis concerning the communication via TCP/IP (includes dSPACE SCALEXIO)***
- ***8.1.3 Real-time violations***
- ***8.1.4 Problems concerning the TOI block-set***
- ***8.1.5 Which MATLAB/Simulink versions are supported***

8.1.1 Problem analysis concerning dSPACE systems

Loading the model works via the dSPACE command line tool “ScoutCmd.exe” or “CmdLoader.exe”. PROVEtech:TA redirects the output of this tool into the PROVEtech:TA Log (see chapter ***3.6.1 PROVEtech:TA Log***). So in case ScoutCmd or CmdLoader reports an error, please look at the according output in the PROVEtech:TA Log. These errors are independent from PROVEtech:TA and should be fixed together with the modeler in charge.

Please ensure that you have also checked the common information of chapter ***8.1 Problems concerning the model***.

Other problems and solutions for dSPACE PHS bus systems (not SCALEXIO – for SCALEXIO please read the next chapter):

1. Error message: Connecting the model communication module failed for the application '…

This means that the ComInit() call in the model does not occur. Possible reasons:

1a) The model does not contain the ModelCom module at all

1b) The call to ComInit() is missing in the model

1c) The model crashes before calling ComInit()

1d) The model initialization takes too much time before calling ComInit(). In this case you can increase the connection timeout in the configuration dialog (see chapter ***3.2.7.2.3.10 dSPACE System***).

2. Error message: The sampling period of the model thread that calls ComAcquisition in the application '…' is uninitialized

This means that the ComAcquisition() call in the model does not occur. Possible reasons:

2a) The call to ComAcquisition() is missing in the model

2b) The model initialization takes too much time to calculate the first model sample, see 1d)

3. Error message: The Idle Loop of the model seems not to run anymore

The idle loop in the model executes the communication between PROVEtech:TA and the model via the call to ComIdle(). The error message means, that ComIdle() is not called anymore. Normally this happens when the model stops running because of an unexpected state (e.g. real-time violation, crash, endless loop in the model loop, etc.).

The following actions may help avoiding this error (see also chapter ***8.1.3 Real-time violations***):

3a) The modeler should make sure that the model tick is long enough to calculate the entire model step

3b) The tester should make sure that the AL/RT scripts are not too big (number of parallel tasks)

3c) When having a multi node model, the tester should make sure that the test runs on the optimal node to avoid traffic via the Gigalink connections

4. Error message: Failed to load 'DSP32.DLL'

DSP32.dll is a dSPACE DLL. PROVEtech:TA needs the DSP32.dll to connect to dSPACE boards. It is only loaded if you have enabled the communication to the model in your configuration. Then, Windows uses the PATH environment variable to search this DLL.

Without a dSPACE board you should disable the communication to the model.

5. Trouble with the DS1006 Multi Core

The DS1006 Multi Core processor board externally acts like several DS1006 Single Core boards. So, every core has an own name. Unfortunately, it happens very often that all cores have the same name in the dSPACE configuration file %DSPACE_ROOT%\Config\dSPACE.ini. Please make sure that the cores have unique names so PROVEtech:TA can assign them correctly to the model.

6. Error No. 2 of the CmdLoader

If dSPACE CmdLoader returns error No. 2, this means that the model could be loaded, but the checking algorithm detected a problem after loading the model.

However, ControlDesk does not perform this check and therefore does not report a problem.

The exact problem can only be seen in the output of the CmdLoader, which is also reported as a black message in the PROVEtech:TA log. For a detailed explanation, please contact your model responsible or dSPACE directly.

You can also ignore the error in PROVEtech:TA via the parameter "IgnoreError2=1" in the .cfg file (see ***Administrator's Guide***). However, we do not recommend this.

7. Anything else

Sometimes a Windows reboot helps or a dSPACE driver reset via “dcont –r” (e.g. in case boards are locked). Additionally, sometimes the dSPACE.ini file is corrupt and you need to rebuild it via ControlDesk. ControlDesk sometimes also displays additional error information.

If there are still problems, please check all settings again:

- Are all processor boards configured correctly in dSPACE ControlDesk and do they all have the correct firmware?
- Can you load the model in ControlDesk (does not mean, that it is absolutely correct)?
- Does ControlDesk display an error message in case you start it after PROVEtech:TA has displayed an error message?
- Is the integration of ModelCom/RTAE in the model correct? On the PROVEtech:TA installation medium there is a directory “Goodies\ModelComAndRTAE” containing the according instructions.
- Is the assignment of nodes to the processor boards correct and the Gigalink topology corresponds to the reality?
- Did you update ModelCom after updating the dSPACE software?
- Is the file dSPACE.ini / RecentHardware.xml in the dSPACE configuration directory correct?
- Is the PROVEtech:TA model configuration correct? (see ***3.2.7.2.3.10 dSPACE System***, e.g. increase the timeout)
- Does ScoutCmd.exe/CmdLoader.exe display an error message?
- Are there any errors logged in the dSPACE.log file of your dSPACE installation?

PROVEtech:TA uses the following configuration and model files while loading the model, PROVEtech:TA does not modify them:

- .sdf file (one per model, entry point)
- .trc files (one per model node, signal information)
- .map files (one per model node, signal address information)
- _tp.m file (one per model, topology information for the RTAE)
- dSPACE.ini or RecentHardware.xml (global for the test system, board information)

PROVEtech:TA does not read the last three ones when working without a test system.

8.1.2 Problem analysis concerning the communication via TCP/IP (includes dSPACE SCALEXIO)

Please check if the ModelCom/RTAE module is correctly integrated in your model. On the PROVEtech:TA installation medium there is a directory “Goodies\ModelComAndRTAE” containing the according instructions.

Please check that no firewall disturbs the TCP connection to the model. At connection time, ModelCom in the model is the TCP server and listens on the port 7658. PROVEtech:TA is the client and does not use a specific port. Please configure your firewall to accept that PROVEtech:TA connects the remote port 7658.

Additional ports might be necessary to transfer the model; this depends on the supplier of the hardware platform.

For PROVEtech:RE the process ModelLauncher.exe uses the TCP port 7657. Please configure your firewall to accept that PROVEtech:TA connects the remote port 7657. On the PROVEtech:RE computer the firewall must accept the local TCP port 7658 for RE.exe and 7657 for ModelLauncher.exe.

In case several model nodes (several ModelCom instances) run on one computer, each instance gets an own port: 7658, 7659, 7660, etc.

For AICORE it is TCP port 11220 for ModelLauncher and 11221 for ModelCom. Additional instances get the TCP ports 11222, 11223, 11224, etc.

For dSPACE VEOS it is the TCP ports 23591, 23592, 23593, etc. for ModelCom.

If no trace output with prefix "HW:" is displayed in the PROVEtech:TA Log, it is possible that the model has already terminated, possibly even immediately after the model was started. In this case it is useful to check the respective .log files of the platform manufacturer’s software for corresponding messages.

The Windows error code 10060 "Connection timed out error" indicates that the network between your host PC and SCALEXIO seems to be unstable. Please check the network connections (switches, cables, settings, etc.) or ask dSPACE for further advice. Also, a firewall could disturb the network connection.

If the error message "No connection could be made because the target machine actively refused it. (network command "connect(7657)" returns error: 10061)" appears when loading the model, it may have the following cause. ModelLauncher.exe is not running on the computer where you want to run PROVEtech:RE. If it is the same computer where you're running PROVEtech:TA, ModelLauncher.exe must run on the same computer as well. If it is running on a different computer than PROVEtech:TA, you must ensure that the firewall accepts the connection. ModelLauncher.exe is part of PROVEtech:RE (in the "bin" directory of PROVEtech:RE) and the PROVEtech:RE installer adds it to the auto start of the Windows user. Please make sure that ModelLauncher.exe is running correctly.

The error message “An existing connection was forcibly closed by the remote host. (network command "recv" returns error: 10054, node "…")” means, that the model computer closed the TCP connection between PROVEtech:TA and ModelCom. This can be

caused by several things, e.g. the ECU power supply also powers the Ethernet switch or the model has a real-time violation and thus terminates. Typically, in case of dSpace platform, the best approach is to have a look at the SCALEXIO log when it happens. Open a web browser and enter the SCALEXIO IP address and have a look at the model's log message output there.

There can be several causes if the TCP/IP connection breaks after successfully loading the model (very often in combination with the Windows error code 10060 in the PROVEtech:TA Log):

- Physical issues, e.g. network, overheating of the model computer, issues with the power supply
- The model does not react anymore; as an example there was a real-time violation (see chapter ***8.1.3 Real-time violations***) or the model produced an access violation (crashed) or there was a different model issue. In that case very often analyzing the log files of the platform provider helps. For dSPACE SCALEXIO systems you can launch ControlDesk in parallel to PROVEtech:TA as an example to display possible system error messages there.

Below you can find error causes regarding dSPACE Scalexio:

- Model build and SCALEXIO firmware must match. In case of problems you may find a hint message of the following type in the PROVEtech:TA log: "HW: The model was build with dSPACE release 2022-A, but the SCALEXIO firmware you're using is from dSPACE release 2022-B."
- PROVEtech:TA must be newer than the dSPACE release.
- In case of problems you may find a message of the following type in the PROVEtech:TA log: "HW: HCN: The real-time application was built using an older RTLib version '5.1.2p10'. Currently installed is the backwards-compatible firmware '22.2.1'." While that line claims that the firmware is backward-compatible - it is not. Please downgrade the firmware on your SCALEXIO to a version that fits to the dSPACE Release used to build the model. Also make sure that you install ModelCom via ModelComAndRTAESetup with that dSPACE Release.
- If a model can be loaded via ControlDesk, this does not automatically mean that the model runs smoothly. As long as you do not actually work with the model in ControlDesk (that means, display and change signals with a proper license), you will not notice that something is wrong. You cannot rely only on the green arrow next to the board in the display.
- Is it possible to load older models?

- Is it possible to load the model with ControlDesk? If yes, please load the model with ControlDesk and then start PROVEtech:TA and select no reload of the model.
- Have you already switched off the Scalexio system (power plug disconnected for 30 seconds) and rebooted?
- Have you re-registered the SCALEXIO system in dSpace ControlDesk or dSpace ConfigurationDesk or dSpace Firmware Manager? (The error message “The platform SCALEXIO is not registered” can then be found in the PROVEtech:TA log in a black color). Sometimes it also helps to reboot the Windows PC and the SCALEXIO computer afterwards.
- Can you check please the log of all real-time computers in the web interface? Maybe the direct log of SCALEXIO provides more information. You can view that in a browser by entering the IP address for the selected SCALEXIO system. The web interface has a button for viewing the log.
- However, there's also the case that loading the model and the .trc files actually worked but the user seems to be too impatient and just kills PROVEtech:TA while it's still loading the .trc files.
- Your PROVEtech:TA version is very old and doesn't have the optimization for loading dSPACE models yet. Beginning with PROVEtech:TA 2022 SE we drastically improved loading time.
- From PROVEtech:TA 2023 SE, the timeout for the model connection is increased.

Please see also chapter ***8.1.5 Which MATLAB/Simulink versions are supported***.

8.1.3 Real-time violations

Typically, a timer controls an HiL model running with a pre-defined step size, e.g. 1ms. This means that the entry function of the model always gets called with a time distance of this cycle time. In each cycle a typical HiL model reads all input values from the hardware cards, calculates the environment model (e.g. the engine behavior) and outputs the calculated values to the hardware cards. Additionally, it has additional tasks like the residual bus simulation (rest bus simulation) and the synchronization and data exchange with other model nodes.

As the Real-Time Automation Engine (RTAE) executes tests in real-time, its calculations happen within the model cycle time. Model and RTAE share the available CPU time.

In case all these tasks cannot be calculated within the model cycle time, you have a real-time violation. For the most HiL platforms, you can adjust the behavior in case of a real-

time violation. The classic setting and concerning real-time best reaction is to stop the model immediately with an error message. On dSPACE PHS bus systems the error message is “Task Overrun: Program cannot be executed in real-time” and “Task Overrun: Occurred in task "Timer Task 1", execution step …”.

What can you do if you encounter real-time violations?

a) Optimize your model so that it can run faster, e.g. simplify residual bus simulation (rest bus simulation)

b) Remove unused model parts

c) Use a faster CPU for the model

d) Use unused CPU cores or spread the model across several nodes in a more balanced way without unnecessarily increasing the needed communication effort

e) Increase the model cycle time

f) Use less signals for data acquisitions as the model must copy them in each step to a buffer

g) Simplify your RTAE test programs; make sure that not too many instructions happen in the same model step. As an example, several ***RT.Assign*** commands without a blocking actor between them are executed in a single model cycle. This creates spikes in the time consumption per cycle.
   Another example is the use of the Basic *For*-loop instead of the RTAE actor ***RT.BeginFor***. So, all included actors in the body are created for each loop instead of only one time.
   It is possible to create an endless loop with the RTAE that has no blocking actor. As an example, you can create a loop without any ***RT.Sleep*** command. This will block the model execution and thus leads to a real-time violation.
   For a model across several nodes, signals and events are exchanged between the nodes, which takes a lot of execution time. Try to reduce the amount of transferred data between the nodes (e.g. by running the test on a different node; preferred are the ones that have much time per cycle left). The command ***RT.ViewCompiled*** shows the transferred amount of data.

h) Update to the latest RTAE version

i) Not recommended: Change the model behavior for real-time violations and accept them

8.1.4 Problems concerning the TOI block-set

TOI (Test and Operating Interface) is a Simulink block-set (TOI blocks, Get block, Set block, Manip block) with two purposes:

a) Create signal names that are independent from the model hierarchy.

b) Add more information to signals like a physical unit, min/max value or other attributes.

Please keep in mind that TOI signals are just copies of existing Simulink signals. So these signals are twice in the signal tree and only the copies with the new names have the additional information like the physical unit.

A Manip block has 4 signals. If one of the signals is not available, then the complete TOI block is not visible in the PROVEtech:TA signal selection tree.

In most cases the model settings in Simulink are incorrect when PROVEtech:TA cannot create the TOI signals as expected. Please use the settings described in the TOI block documentation (see directory “<PROVEtechTA_SetupDirectory>\Goodies\TOI Library” “, chapter "3 Model settings", settings "Block reduction optimization" and " Signal storage reuse" deactivated).

When using dSPACE RTI a setting limits the maximum length of block comments. To increase this, please edit the file
%DSPACE_ROOT%\MATLAB\RTI1006\TLC\rti_modeltrc_blk.tlc
or
%DSPACE_ROOT%\MATLAB\RTI1005\TLC\rti_modeltrc_blk.tlc
or
%DSPACE_ROOT%\MATLAB\RTI1401\TLC\rti_modeltrc_blk.tlc
or (for dSPACE release 6.3 and later)
%DSPACE_ROOT%\MATLAB\DSRTSU\TLC\rti_modeltrc_blk.tlc
The line “%assign TRCMaxDescLength” contains this setting.

8.1.5 Which MATLAB/Simulink versions are supported

The question, which MATLAB versions PROVEtech:TA supports cannot be answered across the board, because it depends on the used platform and the thereby used 3rd party software beside other things.

First, you need to check which MATLAB versions are supported by your used platform and the thereby used 3rd party software. Typically you find the information, which versions of the 3rd party software PROVEtech:TA supports, in the according “ReadMe.txt” file located in the folder “Goodies\ModelComAndRTAE” on the PROVEtech:TA installation medium for each platform.

For the supported dSPACE platforms there is a compatibility check shortly after(!) each dSPACE release. So in case the release is about ¼ year before the PROVEtech:TA release, PROVEtech:TA typically supports it. As dSPACE transfers the Simulink information into their .trc files, PROVEtech:TA is mostly independent from the used MATLAB version.

For the other platforms PROVEtech:TA parses the Simulink information via the generated C header files. The tool “MatlabPatch.exe” located on the PROVEtech:TA installation medium changes the comments in the generated header files by Simulink, so PROVEtech:TA can read them. “MatlabPatch.exe” depends on the used MATLAB version. The tool outputs all supported MATLAB versions at startup.

Generally you can say if the used software (MATLAB or platform) was released after a PROVEtech:TA version, there was no compatibility check with this combination. You can test this combination on your own. In some cases a PROVEtech:TA Service Pack helps to get the compatibility afterwards. So you should always use the latest PROVEtech:TA Service Pack for your checks. However, there is no guarantee that any combination beside the officially tested ones works.

8.2 Problems concerning the workpage

The following chapters describe problems and solutions concerning the workpage.

8.2.1 Trouble while parsing an .a2l file

Problem:

While starting the model there is a error message concerning a .a2l file or not all signals of the .a2l file appear in the signal tree.

Solution:

The .a2l format is an ASAM standard, which gets interpreted very individually by different suppliers. Writing a common parser for this format is very hard also regarding the fact that the suppliers mainly orient on one of the two standard tools for this file format: Vector CANape or ETAS INCA. These tools focus exactly on the .a2l functionality whereas PROVEtech:TA focuses on test automation and CCP/XCP is just a minor feature.

In case there is trouble with an .a2l file, please create a report (see chapter ***2.13 Creating a problem report***) and send it together with the .a2l file to the displayed e-mail address. PROVEtech Support can analyze it with this information and fix the issue.

8.2.2 CCP/XCP signals do not get updated

Problem:

When displaying CCP/XCP signals on the workpage the displayed values are not updated.

Solution:

First of all please always add the CCP/XCP “Status” signal to the workpage to see, if the ECU communication works. Only in case the status is 1 everything is okay. If it stays 0, please check if the ECU has battery power, if the communication parameters in PROVEtech:TA are correct (e.g. CAN channel and CAN Ids) and if the communication channel transports data at all (invalid transceiver, bus termination, bus setup in the model, rest bus simulation of the model, wrong or unplugged cables, etc.). If the status stays 2, please check if you have selected a correct default event number in the configuration dialog. See chapter ***3.2.7.2.3.13 Configuration of CCP/XCP***.

Please be aware that CCP and XCP cannot handle several masters at the same time on the same slave (ECU). So please do not try to connect a slave from PROVEtech:TA at the same time as with ETAS INCA or Vector CANape or any other tool of this kind.

Further possible sources of error are:

- The CCP/XCP Enabled signal must be set to 1, otherwise the communication is stopped (see also chapter ***3.2.7.2.3.13 Configuration of CCP/XCP***, checkbox `Instance initially enabled`).
- The .a2l file does not match the ECU.
- If CCP/XCP signals can be read but not written: There is an issue with seed&key (see also chapter ***3.2.7.2.3.13 Configuration of CCP/XCP***, text field `Seed&Key .dll file`)
- If CCP/XCP signals can be read but not written: The calibration page is not correctly set (see Settings dialog, chapter ***3.2.7.2.3.13 Configuration of CCP/XCP***, text field `Calibration page to select`).
- Typically CCP/XCP slaves have limits concerning the amount of measured signals in parallel (maybe also per event number). If you overrun this limit, the Status does not switch to 1. A workaround for that limit is switching to the polling mode (see also chapter ***3.2.7.2.3.13 Configuration of CCP/XCP***, checkbox `Use polling instead of DAQ`).
- The PROVEtech:TA version or the ModelCom version is outdated. Please update the ModelCom version in your model to the latest version. It depends on the used model platform how this works exactly. Please use the

ModelComAndRTAESetup.exe from the PROVEtech:TA installation package where applicable.

Possibilities for analyzing communication with the ECU:

- Check the PROVEtech:TA Log for XCP error messages, such as "HW: XCP / CCP instance 0: No connection established".
- Regarding XCP on CAN: If there is CAN communication, the CAN messages can be displayed in the message window.
- Regarding XCP on TCP/UDP/SxI: Set XCPDebug = 1 (see ***Administrator's Guide***) to create a generic channel for debugging. This can be displayed in a message window.
- Please be aware that recordings of signals do not help. Only recordings or data acquisitions of the XCP raw messages plus the special signals "Status" and "Enabled" are helpful.
- The communication protocol is documented in the ASAM MCD-1 (XCP) standard, Protocol Layer Specification. (This is not part of the PROVEtech: TA installation package).
- The communication can also be recorded in a .hdr data acquisition file and then visualized via a PROVEtech: TA widget (see XCPDescriptor on https://www.provetech.de/index.php/downloads/provetech-ta/provetech-ta-widgets).

If these hints do not help you to solve your problem, you can commission PROVEtech Support (provetech@akkodis.com) for further analysis. Please provide the following data for this:

1. PROVEtech:TA problem report (see ***2.13 Creating a problem report***) after reproducing the issue.
2. .a2l file
3. CAN/TCP/UDP/SxI trace while reproducing the behavior (see XCPDebug=1 above and ***Administrator's Guide***); you can create this by using the messaging feature of PROVEtech:TA (see ***3.2.7.4 Data acquisition*** and ***4.4.14 The message window***) or you use CANoe/CANalyzer from Vector. Save in the HDR, MF4 or ASC format though. Do **not** use the BLF format.

8.2.3 Button to configure CCP/XCP disabled

Problem:

The button to configure CCP/XCP in the settings dialog is disabled (see chapter ***3.2.7.2.3 Models***).

Solution:

1. You do not have a CCP or XCP license. Please contact PROVEtech Support to obtain a license if you want to use this feature.
2. The hardware type of the model is “DLL”.

8.2.4 The maximum amount of measurement signals is not sufficient

Problem:

PROVEtech:TA limits the maximum amount of possible signals for data acquisitions (see chapter ***4.2.3 Measurement signal selection, clipboard and info field***). This limitation exists for self-protection. An exorbitant amount of measurement signals increases the memory usage, reduces the performance of the whole system, stresses the model, the communication channel and possibly leads to forcefully stopping the data acquisition due to buffer overruns.

Solution:

You can increase the maximum amount of measurement signals on your own risk per .cfg file of the model configuration (entry “MaxMeasSignals”, see ***Administrator’s Guide***). Please keep in mind that the bandwidth of the physical communication channel between the model and PROVEtech:TA is limited. Especially for dSPACE PHS-bus systems the maximum is about 1.5 MB/s. Beside the measurement data it must transport other data in parallel. You can calculate the needed bandwidth using the following formula:

MeasureFrequency * BytesPerSample

BytesPerSample is influenced by the data types of the selected measurement signals. Example for 2000 measurement signals of the data type “Double” at 1000 Hz measure frequency:

1000 1/s * 2000 * 8 Bytes = 16000000 Bytes/s = 15,3 MB/s

This exceeds the possibilities of systems connected via Fast Ethernet. It is even worse for PHS-bus systems.

Instead of acquiring the signals it might be helpful to acquire the raw bus messages containing the signals (e.g. CAN/LIN/FlexRay). The bus messages are coded more efficiently and are only acquired when sending/receiving them.

8.2.5 Issues with the video capturing

In case of issues with the video capturing (see chapter ***4.4.18 The video control***) please make sure to install the latest video card driver of your video card manufacturer. Do not use the default Windows video card driver from Microsoft. Additionally, the Windows Task Manager displays information about the CPU and GPU usage. Capturing very high resolutions or too many video sources might overload them.

In case the video source selection (see chapter ***3.2.7.3 Workpage***) does not show your video source, it is probably not compatible to Microsoft Media Foundation. PROVEtech:TA has a plug-in interface to add custom camera sources. Please contact the PROVEtech:TA support for further advice (provetech@akkodis.com).

8.3 Problems concerning the Test Manager

The following chapters describe problems and solutions concerning the Test Manager.

8.3.1 The “Test Manager” tab is not displayed

Problem:

The “Test Manager” tab is not displayed.

Solution:

Please check out the following issues:

- You have **logged on** with an unknown user or wrong password. Please take care of case sensitivity. You can find a list of default users and passwords in the ***Administrator’s Guide***.
- In the login dialog the **password** field is not displayed; this is an indication that one of the problems listed here applies.

- The **database connection** was not successful. An accordant error message is displayed in the PROVEtech:TA Log. Please see also in chapter ***8.4.1 Problems when connecting to the database***.
- The **database server** or the database instance is down. Please check on the server computer whether the associated Windows service exists and has been started.
- The flag in **TA.ini**, section [Panels] is not set to TestManager=1 (see ***Administrator's Guide***)
- In **TA.ini** the reference to TM.ini is not correctly set (see section [Ini], see ***Administrator's Guide***)
- The reference to the **License.ini** is set in the TA.ini and the flag in the License.ini, section [LicenseFeatures] is not set to TestManager=1 (see ***Administrator's Guide***)
- PROVEtech:TA is running in **DAQViewer mode** (see ***2.9 PROVEtech:TA DAQViewer***). Only the Offline mode can be activated. Please check in Base.ini (see ***Administrator's Guide***) and licensing.
- There is a licensing problem (see ***2.2.1 Computer-bound licensing***). Therefore, please have a look into the licensing e-mail. Make sure if there are no error messages in the PROVEtech:TA Log which lead to other causes (user login and password, database connection problem, …).
- The file DBODBC.dll cannot be found in the TA\bin directory or loaded; maybe the file is out of date and the interface is no longer compatible

If the problem still exists or the reason is still not obvious please create a PROVEtech:TA problem report and send this to our PROVEtech Support provetech@akkodis.com (see ***2.13 Creating a problem report***).

8.3.2 Logging in to the Test Manager does not work

After the PROVEtech:TA client has established the ODBC connection to the database, the credentials of the PROVEtech:TA user are checked (see also chapter ***2.3 Login*** at the bottom). All PROVEtech:TA users are managed in the database, whereby certain data such as the password or a second factor are stored in encrypted form. If you receive an error message when checking the PROVEtech:TA user's credentials, please consider the following points:

- the user name; the user name is case-insensitive; if the Windows password is used or the login happens via OpenID Connect (OAuth), the user name must still be created separately as a PROVEtech:TA user in the database

- the password; the password is case-sensitive and should comply with your company's password guidelines; the password is typically stored in encrypted form in the database; if the Windows password is used instead (see the settings in the user dialog, chapter ***5.3.8.1 The user***), authentication takes place via a call to the Windows API (see below)
- if it is configured that the Windows password is used for this user, please also check the user's domain setting
- if the login is via OpenID Connect (OAuth), the PROVEtech:TA administrator should check the OpenID Connect configuration data (see in chapter ***5.3.11.2 The tab "Database settings" in the property dialog***); the user's login must also be set to OpenID Connect (OAuth) (see also chapter ***5.3.8.1 The user***)
- if two-factor authentication (2FA) via TOTP pin is set up, the 6-digit pin must be read from a pin generator (Microsoft Authenticator or Google Authenticator, typically via app on your smartphone) and entered (see also chapter ***5.3.8.1 The user***)
- the user must be activated and the expiration date must still be valid (see also chapter ***5.3.8.1 The user***)

If, on the other hand, an error is displayed which comes from the ODBC driver (Oracle, SQL Server or PostgreSQL), the PROVEtech:TA user's credentials have not yet been checked, but the database connection could not be established (see chapter ***8.4.1 Problems when connecting to the database***).

If the login via OpenID Connect (OAuth), also abbreviated as OIDC, does not work, please check the following points:

- Check the OpenID Connect configuration both globally in the database (see chapter ***5.3.11.2 The tab "Database settings" in the property dialog***) and for the corresponding user (see chapter ***5.3.8.1 The user***);
  for example, the error message "`Error in module "*OAuth*": Exception has been thrown by the target of an invocation.`" may indicate an incorrect OAuth URL.
- Is the OAuth ClientID for PROVEtech:TA configured correctly on the OpenID Connect server?
- Is the OpenID Connect server ready for use and is the network working?
- Are the software components used compatible with each other? (In the case of OpenID Connect, this would be the IdentityModel including dependencies, at least .NET 8, current Windows Edge version, OAuth module of the server)

- Are the versions of Microsoft Windows, Microsoft Edge, etc. and PROVEtech:TA itself up to date?
- Restart the computer and try again
- Are there any problems with the virus scanner or has your IT made any changes?

If the login was not successful for an unknown reason, the PROVEtech:TA administrator “admin” can reset the credentials of this user.

If you have forgotten the credentials of the “admin” user, the last option is to delete the database including the Test Manager data and create a new one. However, please contact PROVEtech support in advance at provetech@akkodis.com.

Default passwords can be found in the ***Default passwords*** chapter. Please always change default passwords and set a secret password.

General information on data security can be found in chapter ***8.3.8 Data security and avoiding data loss in the Test Manager***.

If the PROVEtech:TA user is configured to use the Windows password, PROVEtech:TA uses the Windows API function LogonUser with the login name, the configured domain and the parameters LOGON32_PROVIDER_DEFAULT and LOGON32_LOGON_NETWORK to check the password. For more details, please refer to the Microsoft documentation (MSDN). If error messages occur, the following basic script can be used to analyze the error.

```
'#Language "WWB.NET"
Private Declare Function LogonUser Lib "advapi32.dll" (ByVal
strUsername As String, ByVal strDomain As String, ByVal strPassword As
String, ByVal dwLogonType As UInteger, ByVal dwLogonProvider As UInte-
ger, ByRef phToken As System.IntPtr) As Integer
Private Declare Function CloseHandle Lib "kernel32.dll" (ByVal hHandle
As System.IntPtr) As Integer

Option UsesAmbiguousError On
Option Explicit

Sub Main
    Dim strUser As String = "myname"     ' Please replace by your Win-
dows user login
    Dim strDomain As String = "AKKODIS.COM"        ' Please replace by
your real domain
    Dim strPassword As String = "MySecretWindowsPwd" ' Please tempo-
rarily replace by your real password in plain text for testing purpose
    Dim uLogonType As UInteger = 3 'LOGON32_LOGON_NETWORK
    Dim uLogonProvider As UInteger = 0 'LOGON32_PROVIDER_DEFAULT
    Dim hToken As System.IntPtr = 0
```

```
    Dim nResult As Integer = LogonUser(strUser, strDomain, strPass-
word, uLogonType, uLogonProvider, hToken)

    If nResult <> 0 Then
        CloseHandle(hToken)
        System.Remark "LogonUser succeeded"
        System.SetResult TestResultValue.trvPass
    Else
        System.Remark "LogonUser failed with code " & Err.LastDLLError
        System.SetResult TestResultValue.trvFail
    End If
End Sub
```

8.3.3 The Test Manager cannot show or save the special characters correctly

Problem:

The Test Manager cannot show or save the special characters correctly in the Basic code and other control elements.

Possibilities for analyzing and solving:

- In many cases the regional coding and the used character set on the database server and client has to be adjusted.
- In case of Oracle, you might have an old database schema created by an old PROVEtech:TA version not supporting Unicode. Or you get the error message "ORA-24812: character set conversion to or from UCS2 failed". In that case you could contact the PROVEtech support (provetech@akkodis.com) to help you migrating to a Unicode schema (see also chapter ***3.2.7.6.1 Configure an Oracle ODBC connection*** and the topic NLS in the Oracle documentation as well as the environment variable NLS_LANG). The recommended settings at the database server are NLS_CHARACTERSET: AL32UTF8, NLS_NCHAR_CHARACTERSET: AL16UTF16. The tool "Oracle Database Migration Assistant for Unicode" may also help. Do not use the Oracle character set "UTF8", because this is a legacy from Oracle and refers to the format CESU-8, which is explicitly not recommended as a data exchange format.
- In case of Oracle, the database columns for strings should be of Unicode type NVARCHAR2 and NCLOB, instead of ANSI type VARCHAR2 and CLOB.
- In case of Microsoft SQL Server the database columns for strings should be of Unicode type NVARCHAR instead of ANSI type VARCHAR.
- In case of PostgreSQL you should always set UTF-8.

- Get the program code by using the test language command ***TestItem.Program***. If the characters are still incorrect then it seems to be rather a database problem than a WinWrap Basic problem.
- Compare the behavior to another computer that connects to the same database. In case the problem does not occur on this second computer, then it is not a problem of the database server. Now you can compare the PROVEtech:TA version and the ODBC driver version.
- Please check the country settings and code page of the Windows operating system.
- Please check if you are using the latest PROVEtech:TA version.

The PROVEtech Support can check problems related to special characters if you perform the following steps:

1. Start PROVEtech:TA and connect to the database.
2. Go to the `Database` tab in the settings dialog and run the database tests (press the button `Test ODBC connection` and then, button `Further database checks`, as explained in chapter ***3.2.7.6.3 Test ODBC connection and further database checks***).
3. Create a problem report file (see ***2.13 Creating a problem report***) and send it to provetech@akkodis.com

8.3.4 Start script

Problem:

What are the possibilities to immediately start a Basic script after launching PROVEtech:TA?

Solution:

- Command line (see chapter ***2.7 Command line parameters***)
- Event macro (see chapter ***3.2.7.2.1 Event macros***)
- Hyper link (registered URI, se chapter ***5.15 Access from foreign software via URI***)
- Via COM interface (see Test Language Manual, chapter ***Access from foreign software***)

8.3.5 Different time zones and daylight saving time

In PROVEtech:TA, the Test Manager considers different **time zones** as well as daylight saving time. The reference here is always the time on the database server, if for example the time on the PROVEtech:TA client is not set exactly. The following things should be noted:

- With Microsoft SQL Server and Oracle, clock times are always stored with time zone. Special case, PostgreSQL: With PostgreSQL all times are stored in UTC. In this case the time zone information is lost.
- When saving times, the time zone of the PROVEtech:TA client is always included. Special case, PostgreSQL: For the PostgreSQL server the time of the PROVEtech:TA client is converted to UTC and stored.
- When displaying times, the time zone is included if the time (with time zone info) does not correspond to the time zone of the PROVEtech:TA client. Special case, PostgreSQL: For the PostgreSQL server, therefore, either time zones are never displayed if the time zone of the server corresponds to the time zone of the PROVEtech:TA client, or the time zone of the PostgreSQL server is always displayed as well. This is especially true if the time was not fetched from the local cache, but freshly loaded from the database.
- In the Test Language, that means when programming in WinWrap Basic and executing Basic scripts, for example with the data type *Date*, the local time of the PROVEtech:TA client is always considered. If necessary, the time zone is converted internally, for example if the database server is located in a different time zone.
- Certain date and time values, such as in the test parameters or history, are not stored directly in the database, but as a compressed XML structure. Time zone information gets never lost here.
- The date and time values are output formatted according to your country setting in PROVEtech:TA.

8.3.6 Exporting including all dependencies

Problem:

How can I export a test script including all dependencies and transfer it to another computer?

Possibilities:

Moving test scripts including their dependencies is unfortunately no easy task without manual work. But there are possibilities which could be of help:

- Use relative Test Manager paths in ‘#uses commands
- Export well-chosen entire subtrees instead of single test objects (see chapter ***5.3.10 Exporting and importing tests***)
- The dependency window lists all referenced modules (see chapter ***5.4.8 The dependencies window***)
- Within the find dialog you can search only in the referenced modules (see chapter ***5.18.1 Finding tests and test results***)
- The Test Language commands ***TM.GetLoadedModules*** and ***TestItem.GetReferencedModules*** return the paths of all loaded or respectively referenced modules
- Use a central database instead of several single local databases

8.3.7 The test object got lost

Problem:

The test object "XYZ" or the system test "XYZ" together with the subtree underneath got lost!

In which states could the test object currently be?

- The test object is still in the right place. Possibly the access rights do not fit anymore, however, no longer, especially when working with hierarchical projects (see ***5.2.6 Access rights***).
- The test object is located elsewhere in the Test Manager tree. It may have been accidentally moved by another user.
- The test object is in the trash folder. It could have been accidentally deleted by another user. Since the user only sees those test objects in the trash folder which were deleted on their own, the test objects may be hidden in the trash folder (see ***5.2.4The trash folder***).
- The test object was deleted by mistake and the trash folder has already been emptied. Then the tree entry is finally deleted, but the dataset with the Basic code and the test object attributes may still be present in the database.

- The test object was deleted by mistake and the database has already been purged. Then there is no test object data left in the current database.
- In the case of database replication, the test object could still be present on other server nodes. Regarding database replication you can find more in chapter ***5 Test Manager***.

How can I analyze the current state and try to find the test object again?

- Is there possibly an error message in the PROVEtech:TA Log (see ***3.6.1 PROVEtech:TA Log***) that a test object can no longer be found, e.g. if you cannot jump from a hierarchical test result to the corresponding test object? If paths or database identifiers are mentioned, they might be useful for further analysis.
- For test objects with release versions, the delete/move is logged in the parent test object's history (see ***5.3.9.7 The tab "History"***). This could give an indication of when, by whom and where the test was removed.
- If a release version has been created by the parent test object, it may still contain the missed test object in older versions (***5.8.3 Restore a test version***).
- Log in as "admin" in PROVEtech: TA and look in the trash folder.
- When was the test object still there? You can restore a database backup (see ***5.11 Create and restore a database backup***) to a temporary database and copy the test object by export/import.
- By exporting the parent test object (or the parent-parent test object) including the older versions, you can analyze the data records of the database in the export archive (see ***5.3.10 Exporting and importing tests***).
- A search via the find dialog could help if the subtree or the search area is not too big (see ***5.18.1 Finding tests and test results***).
- In case of deleted test results, please refer to chapter ***5.7.4 Restoring deleted test results***.
- As a last resort (very time-consuming and only possible if a direct access to the database at the customer is possible), you could directly search the database by SQL query. You could e.g. search for a substring in all Basic scripts or return a list of all child test objects. However, expert knowledge is required for this.

How can it happen that test objects are "suddenly" no longer found?

- The test item was accidentally moved by another user to another location or to the trash. This may have happened recently or even a few weeks ago.

- Access rights have been changed by another user.

Rather unlikely are the following options:

- Can test objects get lost due to program crashes or deliberate kill of process of PROVEtech:TA? - Rather no because this is already intercepted by the transactional concept of the database.
- Can test objects get lost due to an error in the PROVEtech:TA source code? - Rather no, because this is a basic functionality, which is rarely modified, and intensively verified by automated self-tests before every release of a new PROVEtech:TA version. The decisive factor is always the reference to the parent test object in the database.
- Can test objects get lost when using database replication? - Experience has shown that even this is unlikely. Here database replication would have to accidentally change the parent reference of a test object. Database replication issues usually have a different impact.

Precautions:

- E.g. in the case of drag and drop, it can quickly happen that a test object is accidentally moved to an undesired location. Therefore, PROVEtech:TA displays warning message box as soon as a test object is to be assigned to another parent test object by move or delete operation.
- Create a release version of the complete subtree. Under certain circumstances, the test object can still be "rescued" by restoring older versions of the parent test object. In addition, a log entry is then created in the history if contained test objects are moved.
- Run regular database backups. By restoring old data to a temporary database, the test objects can then be retrieved.
- If the WinWrap Basic code or the description text is changed, an automatic file-level backup of one week can be configured (see chapter ***3.2.7.6.5 Configure the database backup***).
- Always update your PROVEtech:TA installation to the latest release and service pack. In case of database replication, update your replication software (SymmetricDS) to the latest release.
- Only access the data of PROVEtech:TA's database via the most current version of PROVEtech:TA or in consultation with the PROVEtech support. Do not use third party software!

Please read as well in chapter ***8.3.8 Data security and avoiding data loss in the Test Manager***.

8.3.8 Data security and avoiding data loss in the Test Manager

Absolute data security can never be guaranteed, and data loss can never be completely secured. Nevertheless, it is worthwhile to think about it in good time. This chapter is intended to assist you in this context.

What are the general risks regarding data security and data loss?

- Data loss because of **faulty operation** due to ignorance, training, complexity, lack of documentation, misunderstandings, miscommunication
- Data loss because of **accidental** deletion or modification (e.g. power switch off without shutdown, forgot to save, ...) due to distraction, carelessness, negligence, stress, pressure of time, pressure of cost
- Data breaches or **data leaks** in the company (access rights incorrectly set, accidental sending of data, ...)
- Deletion or modification due to **intentional damage** (consider high-security access, data manipulation, forgery)
- **Theft of data**, access without permission (consider high-security access, disclosure)
- **Identity theft**, an unauthorized person knows the login name and password
- Loss or **theft** of a laptop, USB flash drive, ...
- Electronic **sending of data** to an unauthorized recipient (e.g., by e-mail)
- Incorrect **access rights**
- The computer or application is not **locked** while leaving the computer
- **Programming error** caused by the software developer
- **Virus**, hacker attack, malware
- Blackmail by **encrypting** the data
- **Hardware failure** (fatigue of material, vibrations, heat, ...)
- **Force majeure** (fire, lightning strike, flooding, ...)
- Protection and circulation of **personal data**

What are the general risks in PROVEtech:TA regarding data security and data loss?

Where is data stored?

- Database
- Configuration files locally and on network drive
- Data files (e.g., measurements)
- Backup files

Risks at the user:

- Overwriting **program code**, descriptions, parameters, …
- Deleting test objects or **subtrees** in the Test Manager tree
- Deleting **test results**
- Accidental PROVEtech:TA update including **database structure update** and incompatibilities to older versions or their blocking when connecting to the database
- Wrong **database connection** was set or chosen
- **PROVEtech:TA passwords** are forgotten, handed over or even not set (PROVEtech:TA admin, PROVEtech:TA user)
- If **Two-factor authentication** (2FA, MFA) is active, the second factor, the 6-digit pin of the TOTP procedure, is no longer available (TOTP account on the Authenticator app deleted, app uninstalled, smartphone no longer available)

Risks at the database administrator:

- **Software update**, update of the operating system, update of foreign components
- **Database backup** cannot be created
- **Database backups** cannot be restored
- **Database passwords** are forgotten, handed over or the default setting is kept (database admin, database user)
- Data loss when **administrating** the PROVEtech:TA database (Modifying or deleting the database structure, other manual interventions)
- It is no more possible to start the database service, or the **database service** produces error messages (database crash)
- The data in the database is accessed by **third-party software**

Risks when developing the PROVEtech:TA software

- General **software errors** due to new features
- Programming errors when updating the **database structure** when installing a new PROVEtech:TA version
- Use of outdated **encryption methods**

General security actions

- Well thought-out **backup strategy**, double data storage (replication, fail-over), check your backup log files, per-form a practice alarm, restore a database backup on a spare database
- **3-2-1 backup rule**: There should be at least three copies of your data. Save the copies on two different media. Keep one backup copy in an external location.
- **online - offline - offsite**: one copy accessible via the network, one copy in an external medium that is not connected, one copy in another building or in a fireproof cabinet

- Responsible usage of safe and strong **passwords** (amount of characters, upper/lower case, digits, special characters, use of a password manager, no multiple use of passwords in different applications, regular password changes, ...)
- Usage of **safe encryption algorithms**, secrecy of the used algorithm and the keys
- Test modifications in the **source code** (test manually and automatically)
- The user first must test **new software versions**
- Avoiding **stress** and pressure of time
- **Documentation** of potential risks and how to avoid them
- Thorough **training** of new colleagues
- Clarify and document responsibilities, legal notices and questions of liability
- Prompt implementation of these security actions and **control** if these actions are really performed
- Analysis of the **reason of the problem** before fixing the problem so that the problem will not occur in the future



Accessing the data of the database: Please access the data of the database exclusively via PROVEtech:TA interface or via PROVEtech:TA test language. This is the only way PROVEtech support can assist in case of data loss or data inconsistency (see also chapter ***5.12 Database consistency checks***).

Security features in PROVEtech:TA

Actions by the user

- Use safe **passwords** for the PROVEtech:TA administrator and PROVEtech:TA user. Update your password at least every few months. Newer PROVEtech:TA versions also use more secure encryption methods, which are then used to secure the password.
  Attention: it is not checked if the password is "strong" (more than 10 characters, upper- and lower-case letters, digits, special characters, even not empty password. This is in the user's responsibility.
  If you have forgotten the password of the PROVEtech:TA administrator, only the database administrator can reset the password via direct modification in the database. Please contact the PROVEtech support at provetech@akkodis.com.
  Please read as well in chapter ***8.3.2 Logging in to the Test Manager does not work***.
- Log in your PROVEtech:TA user in the Test Manager using the **Windows password** or even more securely using **OpenID Connect (OAuth)**. Read more about this in chapter ***2.3 Login***.
- Use **Windows Authentication** for the database connection. You need not manage a separate database user and transfer their password in the ODBC connection string (see as well in chapter ***3.2.7.6.4 Windows Authentication***)
- Set **access rights** in PROVEtech:TA
- Use of **Two-factor authentication** (also known as 2FA, two-factor authentication, multi-factor authentication, MFA); read more about this in chapter ***5.3.8.1 The user***

- Control the **amount of data**; strategy how to archive and delete data, so that the database gets not too big (this influences the backup strategy)
- Test **new PROVEtech:TA versions** (installation of PROVEtech:TA beta versions and release candidates as soon as possible)
- When switching to a new PROVEtech:TA version **with database structure update**: Set the checkbox that only the PROVEtech:TA administrator is allowed to perform a database structure update to control the blocking of the database connection with very old PROVEtech:TA versions (see also chapter ***5.3.11.2 The tab "Database settings" in the property dialog***)
- Encryption feature to **encrypt** Basic code to protect against theft of data (Please consider: The integrated method of encryption is subject to secrecy of Akkodis and not published to any user or customer)
- • **Lost passwords** related to the encryption libraries cannot be restored by PROVEtech support. The encrypted scripts are then no longer recoverable (see chapter ***5.4.11 Encryption of Basic code*****Export** of smaller Test Manager subtrees could be a possibility to backup data and Basic code independent from database backups processed by your IT
- Create new **release versions** of whole Test Manager subtrees, also as fallback solution for accidentally deleted test objects

Actions by the database administrator

- Protect the password of the **database user and database administrator**; these are the most important passwords regarding data security; first set the database password inside the database and then adjust it via the PROVEtech:TA settings.
- **Please keep in mind**: By copying the database connection entry from the TM.ini file it is possible to connect to the database without knowing the password. To avoid this, you can consider database authentication by the Windows user.
- **Backup strategy** with weekly full-backup, daily incremental backup, transaction log, sufficient disk drive storage, double data storage by RAID system or fail-over, creating long-term data archives (3-2-1 backup rule, online – offline – offsite, see above)
- After creating a database backup, check of the corresponding backup **.log files**.
- Restore a database backup regularly on a spare database.
- Please differ between a **backup of the file system** and a database backup. A backup of the file system does not guarantee that you can restore a consistent state of the database. Always use the mechanisms of the database to back up your Test Manager data.
- To avoid unintentional updating of the database structure and the resulting incompatibilities with very old PROVEtech:TA versions, the database administrator can withdraw the **right to change the database structure** from the database user (default: 'provetech')
- When using **database replication**, we strongly recommend that you use the SymmetricDS software provided for this purpose in close consultation with PROVEtech support. With database replication, the data in the database is changed regardless of the PROVEtech:TA client. Only in this way can PROVEtech support help in case of data loss or data inconsistency (see also chapter ***5.12 Database consistency checks***).

- Only access the data of PROVEtech:TA's database via the most current version of PROVEtech:TA (only access by PROVEtech:TA graphical user interface or extended test language) or in consultation with the PROVEtech support. Do not use **third party software**!
- Lock PROVEtech:TA and the computer when you leave the computer. Read also in chapter ***3.2.8 Lock User Interface***.

Actions inside PROVEtech:TA

- Use of up-to-date and secure **encryption methods** (SHA-512, AES-256,...)
- Option of **Two-factor authentication** when logging in with the PROVEtech:TA user
- **Trash folder** in the Test Manager
- Two-stage deletion of **test results** by delete-flag and purge of the database
- When **WinWrap Basic code**, the description text or test parameters are modified, you can set an automatic backup into file system up to one week (see chapter ***3.2.7.6.5 Configure the database backup***)
- When creating a database backup via PROVEtech:TA, the application stores the last **backup paths** in the TM.ini and User.ini file
- Chapter **Troubleshooting** in the user manual
- This chapter in the **user manual** regarding data security and risks of data loss
- Modifications in the **PROVEtech:TA source code** are tested by the software development (manual testing, automated Test Suite)
- The **protection of personal data** is determined by the legislator; in PROVEtech:TA, this is regulated in the purchase contract, in particular the handling of the problem report for the correction of defects (see also chapter ***2.14 Processing and protection of personal data*** and chapter ***2.13 Creating a problem report***)

8.4 Problems concerning the database

The following chapters describe problems and solutions concerning the database which keeps the Test Manager’s data.

In general, please make sure first, that you installed a current database server and all Service Packs and patches are updated.

8.4.1 Problems when connecting to the database

In the following tables there are listed error messages and possible reasons if the database connection fails. It should then be clear from the error message that it was generated by the corresponding ODBC driver. This is separated in 4 categories:

- ***8.4.1.1 Oracle error messages***
- ***8.4.1.2 PostgreSQL error messages***

- ***8.4.1.3 Microsoft SQL Server error messages***
- ***8.4.1.4 Problems related to LoadLibraryEx***

Please use the Find function in the corresponding chapter and search for the error text or the error number to go to the specific information of your error message. The error message is also displayed in the PROVEtech:TA Log (see ***3.6.1 PROVEtech:TA Log***).

In case the login failed or the Test Manager tab is not visible after startup, please refer to chapter 8.3.1The “Test Manager” tab is not displayed or the chapter ***8.3.2 Logging in to the Test Manager does not work***.

8.4.1.1 Oracle error messages

(For further information about Oracle error messages, please read the Oracle documentation or search on the Internet for the ORA error code.)

ORA-12514: TNS: listener does not currently know of service requested in connect descriptor
ORA-12505: TNS: listener does not currently know of SID given in connect descriptor
ORA-01033: ORACLE initialization or shutdown in progress
ORA-12528: TNS: listener: all appropriate instances are blocking new connections
Cause: The database server or the database instance is down; check on the server computer whether the Windows service “OracleServiceProve”, “OracleServiceXE” or similar is running
Or: You must wait till the booting process is finished
Or: The SERVICE_NAME in your tnsnames.ora is unknown;
probably you have to use the database SID with full domain (global database name, e.g. prove.companyname.com)
Or: Try (SID = ...) instead of (SERVICE_NAME = ...) in your tnsnames.ora
Or: Try to connect via Oracle Easy Connect to avoid using the tnsnames.ora (see chapter ***3.2.7.6.1 Configure an Oracle ODBC connection***)
Or: Check for an event in the listener.log file on the database server
Or: Check which services are currently known by the listener by executing: lsnrctl services <listener name>

ORA-12535: TNS: operation timed out
Cause: The Windows firewall blocks the port (normally: Port 1521)
Or: The database server or the database is down
Or: The network is down

Or: The computer name or IP-address in your tnsnames.ora is wrong
Or: The port number in your tnsnames.ora is wrong; please check the port number in your tnsnames.ora on the host computer and check the port number in the listener.ora on the database server (if you change your listener.ora then you must restart your ORACLE services after that)

ORA-12154: TNS: could not resolve the connect identifier specified
Cause: There is no DB Alias with this name (e.g. TEST.DB) in your tnsnames.ora; please check your tnsnames.ora; check your database connection in the property page `Database` in *Application|Settings...* (see chapter ***3.2.7.6 Database***) or look into your TM.ini; check the entry DBQ= in your ODBC Connection string (Section [DB])
Or: Your computer cannot find the file tnsnames.ora; in the property page `Database` in *Application|Settings...* you can set the file path to your tnsnames.ora file
Or: There are several tnsnames.ora files on your computer and you are editing to wrong one; in the property page `Database` in *Application|Settings...* you can set the file path to your tnsnames.ora file
Or: You use Oracle Easy Connect and in your Oracle configuration file sqlnet.ora the entry `NAMES.DIRECTORY_PATH = (TNSNAMES, EZCONNECT)` is missing (see also Oracle documentation)

ORA-12545: Connect failed because target host or object does not exist
Cause: The name of the host computer in your tnsnames.ora is unknown (check (HOST=...)); probably you must use the IP address (without special characters like \\ or //) or hostname with domain instead

ORA-12546: Permission denied
ORA-28547: connection to server failed, probable Oracle Net admin error
Cause: The Windows firewall blocks the port (normally: Port 1521); please check this on server and client side

ORA-12541: TNS: no listener
Cause: The name of the server computer or the IP address points to the wrong computer
Or: There is no database listener installed on the database server
Or: The listener service on the database server is down or has crashed
Or: The specified connection data does not correspond to the data of the listener; please compare your tnsnames.ora of the client computer to the listener.ora on the server computer; maybe it is the wrong port

ORA-01017: invalid username/password; logon denied
ORA-01005: null password given; logon denied
Cause: Correct user and password in the property page `Database` in *Application|Settings…* (see chapter ***3.2.7.6 Database***)
Or: There may be a problem with Windows authentication (see chapter ***3.2.7.6.4 Windows Authentication***)

ORA-12638: Credential retrieval failed
ORA-12570: TNS: packet reader failure
Cause: Please check the entry SQLNET.AUTHENTICATION_SERVICES= (NONE) in your sqlnet.ora; probably this entry was set to (NTS)

ORA-27101: shared memory realm does not exist
Cause: Restart your ORACLE services
Or: Reboot your computer
Or: Check the computer name/IP address in the listener.ora and tnsnames.ora (if you change your listener.ora then you must restart your ORACLE services after that)

ORA-12170: Connect Timeout occurred
Cause: Check the computer name/IP address in the listener.ora and tnsnames.ora (if you change your listener.ora then you must restart your ORACLE services after that); this error message already occurred when the computer got a automatically generated IP-address via DHCP.
Or: The connection to the server failed because of firewalls blocking the connection and network routing errors; open the firewall settings and ensure that the listener port is not blocked; use tools like *ping*, *tnsping*, *netstat* and *traceroute* to see if the two systems are connected on the network

ORA-28000: The account is locked
Cause: The user has entered wrong password consequently for maximum number of times specified by the user's profile parameter FAILED_LOGIN_ATTEMPTS (normally 10 tries)
Or: The DBA has locked the account; you have to wait for PASSWORD_LOCK_TIME or contact your Database Administrator

ORA-28001: The password has expired
Cause: Oracle sets a default password expiration time by default. In "<PROVEtech:TA Setup Package>\Goodies\Databases\Oracle Server\Scripts\CreateDBFiles.sql" you will find SQL commands which you can execute as a database admin so that the password

never expires:

```
-- Set password to unlimited (password never expires)
ALTER PROFILE DEFAULT LIMIT PASSWORD_REUSE_TIME UNLIMITED;
ALTER PROFILE DEFAULT LIMIT PASSWORD_LIFE_TIME UNLIMITED;
```

ORA-12516: TNS: listener could not find available handler with matching protocol stack
Cause: The maximum number of database sessions on the server is reached; you can try to reboot the database server or clients
Or: Increase the corresponding initialization parameter on the database server (see Oracle documentation)

8.4.1.2 PostgreSQL error messages

(Note: Sometimes you can solve PostgreSQL problems only in stopping and restarting the PostgreSQL service on the database server via Windows Control Panel or via the Start-Menu.)

Errorcode 101: Could not connect to the server; Could not connect to remote socket.
Errorcode 101: Could not connect to the server; No connection could be made because the target machine actively refused it.
Errorcode 101: Could not connect to the server; Could not resolve hostname
Errorcode 101: Could not connect to the server; A connection attempt failed because the connected party did not properly respond after a period of time, or established connection failed because connected host has failed to respond
Errorcode 101: Could not connect to server: Connection refused (0x0000274D/10061) Is the server running on host "localhost" (127.0.0.1) and accepting TCP/IP connections on port 5432?

Cause: The database server or the database is down. Please check Windows Services on the server computer if there is a service named "postgresql-x64-16 - PostgreSQL Server 16" or similar and if it is started.
Or: You must wait till the booting process is finished
Or: There is no PostgreSQL database server installed at all, such as a local database server on the same computer. Please check under Windows "Add or remove programs" if there is an entry named "PostgreSQL" or similar. If no, then on your PROVEtech:TA installation package you will find the file Goodies\Databases\PostgreSQL\InstallPostgreSQL.exe, which you can launch as a local Windows administrator. Then select "Install Server" and set strong passwords.
Or: The PostgreSQL Service on the database server is not started. Please check

Windows Services if there is a service named "postgresql-x64-16 - PostgreSQL Server 16" or similar and if it is started.
Or: The PostgreSQL Service could not start because the Windows user 'Postgres' is deactivated or his password has to be changed; please also check the Windows event log in the Control Panel
Or: The PostgreSQL Service could not start because the lock file <PostgreSQL>\data\postmaster.pid could not be created due to missing permissions; please also check the Windows event log in the Control Panel
Or: There is no database server installed. Please check if a PostgreSQL service exists and is started
Or: The network is down
Or: The Windows firewall blocks the port (normally: Port 5432)
Or: The port number in your ODBC connection string is wrong; please correct the port in the property page `Database` in *Application|Settings…* (see chapter ***3.2.7.6 Database***) or check the corresponding entry PORT= in your TM.ini (Section [DB])
Or: Check the entry listen_addresses='*' in the postgresql.conf file on the server
Or: The IP address of the PostgreSQL Client is not registered on the server; please check the entries at the end of the pg_hba.conf file on the server
Or: Check the entry port=5432 in the postgresql.conf file on the server (Note: If the PostgreSQL Service on the database server is not started, try to start it manually; perhaps there is something wrong with the OS-user of the service; be careful when you change passwords for OS-users; services may not start anymore!)
Or: The entry in SERVER=... in your ODBC connection string is wrong; probably you have to use "localhost", the correct server name or the IP address of the server; please correct the port in the property page `Database` in *Application|Settings…* (see chapter ***3.2.7.6 Database***) or check the corresponding line in your TM.ini (Section [DB])

Errorcode 210: password authentication failed for user ... (SQLState 28P01)
Errorcode 112: A password is required for this connection (SQLState 28000)
Cause: Correct user and password in the property page `Database` in *Application|Settings…* (see chapter ***3.2.7.6 Database***)

No data found (SQLState 02000)
Cause: There is no SERVER=... or DATABASE=... in your ODBC connection string; please correct the server or database entry in the property page `Database` in *Application|Settings…* (see chapter ***3.2.7.6 Database***) or check the corresponding line in your TM.ini (Section [DB])

Errorcode 210: database "TEST.DB" does not exist
Cause: There is no database created on the server
Or: The entry in DATABASE=... in your ODBC connection string is wrong; please

correct the database entry in the property page `Database` in *Application|Settings...* (see chapter ***3.2.7.6 Database***) or check the corresponding line in your TM.ini (Section [DB])

Errorcode 210: no pg_hba.conf entry for host "127.0.0.1", user "provetech", database "TEST.DB", SSL off
Cause: Check the local connections in the pg_hba.conf on the server; it should have the form "host all all 127.0.0.1/32 md5"; probably you must insert additional lines for each client IP address; the exact syntax is also described in the pg_hba.conf file

Errorcode 210: missing or erroneous pg_hba.conf file
Cause: The pg_hba.conf file is missing
Or: Check the syntax of the pg_hba.conf file; the exact syntax is described inside this file

Errorcode 7: ERROR: type “lo” does not exist; Error while executing the query
Cause: The large object (lo) package was not included when the database was installed; please reinstall your PostgreSQL database server or import a workable database backup

Errorcode 210: connection limit exceeded for non-superusers
Errorcode 210: remaining connection slots are reserved for non-replication superuser connections
Cause: Currently there are too many clients connected with the database server; increase the initialization parameter `max_connections`
Or: Currently there are too many clients connected with the database server; restart the database service on the server computer
Or: Currently there are too many clients connected with the database server; reboot the server computer

Errorcode 101: fe_sendauth: no password supplied (lNativeError=101, wstrSqlState=08001)
Cause: There may be a problem with Windows authentication (see chapter ***3.2.7.6.4 Windows Authentication***)

8.4.1.3 Microsoft SQL Server error messages

(Note: Sometimes you can solve Microsoft SQL Server problems only in stopping and restarting the Microsoft SQL Server services on the database server via Windows Control Panel or via the Start-Menu.)

[DBNETLIB]SQL Server does not exist or access denied. [Microsoft][ODBC SQL Server Driver][DBNETLIB]ConnectionOpen (Connect()). (2) (1326)
A network-related or instance-specific error occurred while establishing a connection to SQL Server. The server was not found or was not accessible. Verify that the instance name is correct and that SQL Server is configured to allow remote connections.
Cause: The database server or the database instance is down. Please check on the server computer whether the associated Windows service “SQL Server (MSSQLSERVER)” or similar exists and has been started.
Or: You must wait till the booting process is finished
Or: The MSSQLSERVER service on the database server is not started
Or: The network is down
Or: The entry in SERVER=... in your ODBC connection string is wrong; probably you have to use "localhost", the correct server name or the IP address of the server; please correct the server in the property page `Database` in *Application*|*Settings*... (see chapter ***3.2.7.6 Database***) or check the corresponding entry PORT= in your TM.ini (Section [DB])
Or: The name of the computer hosting SQL Server is incorrect or not accessible.
Or: The instance doesn't resolve the correct IP address.
Or: The TCP port number isn't specified correctly.
Or: Please check whether TCP/IP is activated in the SQL Server Configuration Manager
Or: Please check whether the Windows service "SQL Server Browser Service" is started on server side

Timeout expired (0)
[Microsoft][SQL Native Client]Unable to complete login process due to delay in opening server connection (0)
Cause: The Windows firewall blocks the port (default: Port 1433); make sure SQL Server and SQL Server Browser are exempted by the firewall on the server machine; this is done by putting sqlservr.exe and sqlbrowser.exe as an exception in the windows firewall
Or: You must wait till the booting process is finished
Or: If you are using SQL Express Server, check the server name <server>\SQLEXPRESS in the settings (e.g. localhost\SQLEXPRESS or .\SQLEXPRESS) (see chapter ***3.2.7.6 Database***)
Or: Make sure that the TCP/IP protocol on the SQL Server is enabled (particularly on

SQL Express Server)
Or: Make sure that the Windows service “SQL Server Browser Service” is started on server side (particularly on SQL Express Server)
Or: In case of a DB connection via network, make sure that you use dynamic ports for all IP addresses in your TCP/IP settings on the server
Or: If the port for the database connection is not automatically detected, you can specify the port explicitly after the server name, separated by a comma

Cannot open database "provetech" requested by the login. The login failed. (4060)
Cause: Correct user and password in the property page "Database" in *Application|Settings…* (see chapter ***3.2.7.6 Database***)

Cannot open database "test.db" requested by the login. The login failed. (4060)
Cause: There is no database created on the server
Or: The entry in DATABASE=... in your ODBC connection string is wrong; please correct the database entry in the property page `Database` in *Application|Settings…* (see chapter ***3.2.7.6 Database***) or check the corresponding line in your TM.ini (Section [DB])

Neither DSN nor SERVER keyword supplied (0)
Cause: There is no SERVER=... in your ODBC connection string; please correct the server entry in the property page `Database` in *Application|Settings…* (see chapter ***3.2.7.6 Database***) or check the corresponding line in your TM.ini (Section [DB])

Invalid connection string attribute (0)
[DBNETLIB]ConnectionOpen (ParseConnectParams())
Cause: Please check the corresponding line in your TM.ini for syntax and invalid attributes (Section [DB])
Or: You must restart the MSSQLSERVER service

SSL Provider: The certificate received from the remote server was issued by an un-trusted certificate authority error when you connect to SQL Server (wstrSqlState=08001)
SSL Provider: The certificate chain was issued by an authority that is not trusted
SSL Provider: The target principal name is incorrect. (wstrSQLState=08001)
SSL Provider: The received certificate has expired.
Cause: Renew your database certificate on the server side; this is the proper, but not quick method and usually requires the assistance of the IT department responsible for database administration

Or: Set `TrustServerCertificate=Yes` in the ODBC connection string of each PROVEtech:TA client (via settings dialog, see chapter ***3.2.7.6.2 Configure a Microsoft SQL Server ODBC connection***, or directly in the TM.ini); this solution is not recommended for security reasons!
Or: Set `Encrypted=No` in the ODBC connection string of each PROVEtech:TA client (via settings dialog or directly in the TM.ini); this solution is not recommended for security reasons!

Attention: Starting with PROVEtech:TA 2024, the Microsoft SQL Server ODBC driver V18.3.2 and following was introduced. Microsoft has increased the security precautions here. You can also read about this in https://learn.microsoft.com/en-us/sql/connect/odbc/connection-troubleshooting

Login failed for user 'XXX'. (lNativeError=18456, wstrSqlState=28000)
Cause: There may be a problem with Windows authentication (see chapter ***3.2.7.6.4 Windows Authentication***)

ODBC Error: Cannot get error text because hHandle is 0!
Cause: Check whether the .rll file (e.g. msodbcsql18.rll) is also present in the TA\bin\ODBCMicrosoft directory in addition to the dll file (e.g. msodbcsqlr18.dll). Compare the directory with a neighbouring computer.

8.4.1.4 Problems related to LoadLibraryEx

Problem:

Loading the DLL (e.g, ODBC driver or other DLL) via LoadLibraryEx fails.

Error message:

Windows error 126: “The specified module could not be found.”

Analysis options:

1) Compare the contents of the directory with those of a neighboring computer with PROVEtech:TA installation

2) Check whether there are additional installations of database clients or ODBC drivers (e.g., with Oracle) on your computer (at file level or also via registry entries)

3) Check the PATH environment variable

4) Using the external tool Dependency Walker (https://www.dependency-walker.com/), you can check for incorrect dependencies

Error message:

Windows error 1114: “A dynamic link library (DLL) initialization routine failed.”

Meaning:

This means that the DLL could be loaded (no problem with static dependencies). But the call of the entry point *DllMain* failed. Since the DLL is usually a third-party component and the PROVEtech team does not know the implementation, the analysis possibilities are relatively limited.

Analysis possibilities:

1) Check whether there is another/newer version of this DLL.

2) Check whether the DLL is loadable with another/newer version of PROVEtech:TA.

3) Was the DLL running with this PROVEtech:TA installation before?

4) Is the DLL running with the PROVEtech:TA installation of another computer? If yes, is it really the same PROVEtech:TA version and is the directory of the DLL identical?

5) Check whether you are working with the latest possible version of PROVEtech:TA (current service pack?). For example, new database ODBC drivers are also installed with a PROVEtech:TA update.

6) With the external tool "Process Monitor" check for incorrect registry entries

7) We have also experienced problems with LoadLibrary when current Windows patches or driver patches (e.g. graphics card drivers) were not applied.

8.4.2 Problems caused by lost connection to the database

Problem:

What happens in PROVEtech:TA if the database connection is lost or interrupted (e.g. network connection is broken, database server is rebooted, laptop goes into sleep mode, ...)?

Related database error messages:

[Oracle] ORA-03113: end-of-file on communication channel

[Oracle] ORA-01013: user requested cancel of current operation

[Microsoft][ODBC SQL Server Driver]Communication link failure (SQLState: 08S01)

[Microsoft][ODBC SQL Server Driver]TCP Provider: Timeout error (SQLState: 08S01)

[PostgreSQL] ERROR 08006 connection failure

[PostgreSQL] ERROR 57014: canceling statement due to statement timeout; Error while executing the query

Solution:

1. When PROVEtech:TA tries to get or modify data of the database, PROVEtech:TA transfers an SQL query to the ODBC client for transmission and execution on the database server and thus starts a new database transaction, i.e. a list of several SQL queries.
2. The ODBC client sends the SQL query to the database server via TCP/IP.
3. A timeout is stored in the ODBC client (in some ODBC drivers even adjustable). If the sending did not work within the timeout, the ODBC client notices that the connection is broken. The ODBC client returns a corresponding error message to PROVEtech:TA.
4. PROVEtech:TA now tries to establish a new connection to the database server (via SQLDriverConnect interface function of the ODBC client), again with timeout. You can configure how often this reconnect should be attempted per SQL query. The default is 1 time per connection failure (see ***TM.ini*** in the Administrator's Guide, section [General], Key: DBReconnectTimeout). If configured more than once, the reconnect is first attempted every 5 seconds, then every 10 seconds after one minute.
5. If the new connection has worked, the original SQL query is sent again and PROVEtech:TA continues as usual (it can also happen that it only continues with

the next operation, if the database connection got broken at an unfavorable moment, e.g. in the middle of an ongoing database transaction).

6. If the new connection did not work, the original error ("No connection to the database") is returned to the caller (e.g. the test language command or an action on the user frontend) and the standard error handling takes effect. You can use the WinWrap Basic code to handle what should happen with the error (***On Error Goto ...*** or ***System.CompatibilityMode***, see manual chapter ***Error Handling***, or ***TM.AddRuntimeError***).
7. The next operation (action on the user frontend or new test language command) will attempt to re-establish the database connection. Since PROVEtech:TA does not know when a database connection is possible again, this procedure can be repeated as often as necessary.

In this respect, there have been problems with old PROVEtech:TA versions or with incorrect behavior of the ODBC driver in the past. Therefore, an update of PROVEtech:TA is always recommended. This also automatically updates the ODBC driver.

Caution: If the database connection is interrupted for too long or the connection is interrupted too frequently, the corresponding test result is not reliable and must be restarted. However, when testing overnight or over the weekend, the entire test series is not interrupted.

8.4.3 Problems when installing the database

Problem:

During or after the installation of the database software there are error messages displayed or the connection does not work.

Solution:

For all supported variants of database systems there is a directory “Goodies\Databases” on the PROVEtech:TA installation medium containing instructions for the proper uninstall and installation process of your database software. In most cases it is sufficient to follow this instruction once more step by step to fix the problem.

If errors occur even during manual installation, you will usually get a more detailed error description here. Now often the last chance is to research the error on the Internet, since this is third-party software.

Sometimes problems occur after installation if there was already database software installed on the computer. For example, registry keys might be overwritten. Often the only

solution is a complete clean-up of the computer from any database software inclusive Windows registry and a proper reinstallation.

Check that all software is up to date (PROVEtech:TA, PostgreSQL/Oracle/Microsoft SQL Server, Windows operating system). Please note that Windows firewall and antivirus scanners may affect the database installation process.

Problem:

When updating PROVEtech:TA to a newer version there are error messages from the PostgreSQL database.

Solution:

It could be that the PL/SQL package is not installed on the PostgreSQL server.

Problem:

During the installation of the PostgreSQL server the error message *PostgreSQL initialization failure: "Failed to load SQL modules into the database cluster"*.

Solution:

This error message indicates a problem with Windows access rights.

1. Check if there is a Windows user *postgres* If so, delete it and try to reinstall the PostgreSQL server.
2. Try to install PostgreSQL into a folder where you are sure that all Windows access is granted, e.g. "C:\PostgreSQL".
3. After the installation, check that the Windows service *postgresql-x64* is running If not, try to start it manually.

8.4.4 The locally installed PostgreSQL server no longer starts

Problem:

The locally installed PostgreSQL server no longer starts

Analysis/Solution:

- Check whether the PostgreSQL Server Windows service is started, should be started automatically, or can be started at all.
- Check whether recent database backups are available.
- Uninstall the local PostgreSQL database and remove associated directories. Re-install the PostgreSQL server using the file \Goodies\Databases\PostgreSQL\InstallPostgreSQL.exe from your PROVEtech:TA setup package. Restore the database backup at the end.

Problem:

The PostgreSQL Server service no longer starts with Windows Error Code 1067.

Possible reasons:

In the registry under Computer\HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\Session Manager\Environment the environment variable ComSpec with “%SysTemRoot%\system32\cmd.exe” is not set to type REG_EXPAND_SZ, but only to REG_SZ.

8.4.5 The database schema must be updated

In the PROVEtech:TA database a special version number is stored in the database table "db_version", which ensures the compatibility to the used PROVEtech:TA version. For example, for PROVEtech:TA 2023 this is the version number 81.

When PROVEtech:TA is now started and the login dialog is confirmed, it connects to the database and the version number of the database structure is checked. The following cases can occur:

1. **The actual version in the database matches the expected version of PROVEtech:TA.** This is the default case and PROVEtech:TA can continue the startup process without any additional actions.
2. **The actual version in the database is smaller than the expected version of PROVEtech:TA.** This means that PROVEtech:TA would like to update the database schema of the instance using corresponding SQL commands. But first it is

verified whether the checkbox `Only the administrator can update the database structure` is set (see chapter ***5.3.11.2 The tab "Database settings" in the property dialog***), and whether the PROVEtech:TA administrator "admin" logged on. After successful update the new version number is stored in the database table "db_version" and the PROVEtech:TA startup process continues as usual.
CAUTION: This has an impact on all other PROVEtech:TA clients that want to connect to the database (see point 3.)

3. **The actual version in the database is greater than the expected version of PROVEtech:TA.** This means that a newer PROVEtech:TA version has already been connected to the database from another computer and the database structure has been updated. Now additional compatibility data is read from the database table "db_version" to check whether the older PROVEtech:TA version still copes with the new database schema. If this is the case, the PROVEtech:TA startup process continues as usual.
   If not, first check your license data and then update your PROVEtech:TA installation (see chapter ***3.7.2 The Info dialog*** and chapter ***2.2 Licensing***)

4. **The version number cannot be found in the database.** In this case it is checked whether the table "db_version" exists. If it does not exist, PROVEtech:TA assumes that the database schema must be completely created. This happens automatically when installing PROVEtech:TA with a local PostgreSQL database.

Hints for updating the database schema

If the database schema is to be updated because of a new PROVEtech:TA version, a message will pop up that contains information about compatibility with older PROVEtech:TA versions. If the database is connected from multiple PROVEtech:TA clients on additional computers, please make sure that there are no compatibility issues on these computers. If in doubt, check the appropriate PROVEtech:TA version information on the other computers.

Before starting the database schema update, please make sure that a timely database backup is available. The only way to undo an update of the database schema is by restoring a database backup. For more information, see chapter ***5.11 Create and restore a database backup***.

If error messages or problems occur during the update of the database schema, please create a problem report using the additional application PROVEtech:TA Log and send it to PROVEtech Support (provetech@akkodis.com). Often it is possible to repeat the update attempt after the problem has been analyzed. See chapter ***2.13 Creating a problem report*** for how to create the problem report.

The database user from the database connection data (that means, not the PROVEtech:TA user) needs database rights to change the structure (e.g. create new

tables/columns/indexes) in order to update the database schema. If these are denied, a database error message will occur. In this case, contact your database administrator. They can set the database rights temporarily so that the update can be performed.

If the database connection is tested in the settings dialog, only a compatibility note appears here if the database schema does not match. The database schema is only updated if you connect to the database via the login dialog. However, after updating the database schema, you can use the check box `Further database checks` to verify whether the new database schema works without any problems. The checkbox can be found in the Settings dialog in the `Database` tab, see chapter ***3.2.7.6.3 Test ODBC connection and further database checks***).

If you want to know exactly which SQL commands are executed when updating the database schema, you can perform the update on a spare database after copying the data. During this process, activate the `Show additional debug output` check box in the additional application PROVEtech:TA Log (see in the Settings dialog of PROVEtech:TA Log, chapter ***3.6.1.2 Settings***). After execution you can then see the SQL commands in the trace.



- Only the database instance on the database server where the connection to PROVEtech:TA is established will be updated.
- Please refer to the release notes (ChangeLog) in https://www.provetech.de regarding database schema changes and compatibility.

8.4.6 The database schema is internal information

Question:

Can you help me to understand the database tables?

Answer:

The database schema and the relationships between the database tables are PROVEtech-internal information that we usually do not share with customers. We also recommend that you only access the database data via the PROVEtech:TA client, for example via its extended test language. From a contractual point of view, this is a requirement for support reasons alone.

Of course, you can still view the database schema on your own via a database administration tool. But changes to the database schema or data without using PROVEtech:TA are at your own risk.

8.4.7 The database behaves very strange

Problem:

The database behaves very strange. (E.g., the database produces error messages even the database is connected)

Solution:

If unusual database error messages occur or if you want to check if the installed database version and database configuration is compatible with PROVEtech:TA, you can perform the `Further database checks` operation. Here all database operations are tested, which are necessary for PROVEtech:TA (e.g., get/insert/modify/delete data, modifications in database structure/schema, indices, sequences, character set encoding, date and time, data volume, ...). This can take a few seconds or even minutes depending on the transfer speed and data volume.

Please proceed the following steps:

1. Execute the operation `Further database checks` as described in chapter ***3.2.7.6.3 Test ODBC connection and further database checks***. If possible, logon as PROVEtech:TA administrator first.
2. Check the PROVEtech:TA Log for error messages and warning messages (see ***3.6.1 PROVEtech:TA Log***)
3. Create a PROVEtech:TA problem report (see ***2.13 Creating a problem report***), describe your problem or your guess in a short sentence and send this to PROVEtech Support (provetech@akkodis.com).

8.4.8 The database is very slow

Problem:

The response times of the database are very slow.

Questions:

1. Is the speed from other computers connected to the same database slow, too?
2. Is the speed slow only during special actions/operations?
3. Are you sure that the performance problems are caused from the database connection? (see chapter ***8.12.3 PROVEtech:TA runs very slowly***)
4. Did the speed slow down suddenly (from a certain point of time) or gradually?

Cause:

There are many things which can play a role concerning the speed of the database connection or the database responsiveness:

1. The network connection is slow (for example because of firewalls , local distance, bad connection, hardware problem, problem of the network driver). Here it is helpful to measure the execution time of each SQL query. Therefore, check the setting `Show Additional Debug Output` in the Settings dialog of the PROVEtech:TA Log (see ***3.6.1.2 Settings***). After that you can see the execution times of each SQL query in the PROVEtech:TA Log messages. When using a nearby database server then the SQL execution times are about 20 milliseconds, in case of a distant database server the execution times should be not more than 200 milliseconds.
   You can set a special mode for slow database connections. Read chapter ***3.2.7.6 Database*** for that.

2. A database backup is currently running. Clarify this with your database administration in charge.

3. The data of the database is currently purged. This may for example be triggered when the trash folder is emptied, and the according option was activated.

4. The data volume of the database is very large. You can see the storage space in the properties dialog of a test object (see ***5.3.11.4 The tab "Security" in the property dialog***) or you can use the test language command ***TestItem.GetStorageSpace*** (see Test Language User Manual).

5. The number of rows in the database tables is very large. Click on the button `Further database checks` in the property page *Database* of the Settings dialog (see chapter ***3.2.7.6.3 Test ODBC connection and further database checks***) and check the messages in the PROVEtech:TA Log window.

6. You are working with very large binary data. This could be data acquisitions or additional attachments of test results or also big test protocols. Read in chapter ***5.18.1 Finding tests and test results*** how you can find these big attachments.

7. There are table indices missing in the database. Check the setting `Show Additional Debug Output` in the Settings dialog of the PROVEtech:TA Log (see ***3.6.1.2 Settings***), click on the button `Further database checks` in the property page *Database* of the Settings dialog (see chapter ***3.2.7.6.3 Test ODBC connection and further database checks***) and check the messages in the PROVEtech:TA Log window. If warning messages appear in this regard, you can consider re-indexing the database (see chapter ***3.2.7.6.6 Re-index the database***).

8. The database server has displayed warning messages. For example, several rows or complete tables are locked over a longer period, or tablespaces are currently extended. Clarify this with your database administration in charge.
9. The database is accessed by other computers with older PROVEtech:TA versions.
10. The database server currently performs very processor-consuming SQL queries that are executed by further PROVEtech:TA clients (maybe older PROVEtech:TA versions). The database administrator can check this by using database server tools.
11. If you import or restore large amount of data into a PostgreSQL server, then you should execute the SQL query “ANALYZE” to optimize data access.
12. For Oracle, set the server option CURSOR_SHARING = "FORCE" to avoid hard parsing. Accordingly set the server option "FORCED PARAMETRIZATION" in Microsoft SQL Server. This parses SQL statements faster and better caches the execution plans. This is already set by default in PostgreSQL.
13. You can disable logging for history of test objects and test results to increase speed. Read more about this in chapter ***5.3.11.2 The tab “Database settings” in the property dialog***.
14. In case you use the remote test sequencer, please make sure that the sequencer lists are not too long. Read more about this in chapter ***5.14 The remote test sequencer***.

How can I provide the PROVEtech Support (provetech@akkodis.com) with the necessary information to analyze the performance problem?

1. Update your PROVEtech:TA installation to a current version (including Service Packs)
2. Start PROVEtech:TA and check the setting `Show Additional Debug Output` in the Settings dialog of the PROVEtech:TA Log (see ***3.6.1.2 Settings***)
3. Choose your database connection in the login dialog and log on to PROVEtech:TA
4. Click on the button `Further database checks` in the property page *Database* of the Settings dialog (see chapter ***3.2.7.6.3 Test ODBC connection and further database checks***). This may take several seconds (or minutes)
5. Reprocess the operation that (in your opinion) takes longer than typically expected
6. Create a problem report (see ***2.13 Creating a problem report***). Describe the operation again in the problem report that causes the performance problem and send this to the PROVEtech Support (provetech@akkodis.com)

Which alternatives do exist when connecting to a distant database server?

1. Use **database replication** and synchronize your data automatically to an additional distant database server. If you are interested in replicating your PROVEtech:TA database or if you want to purchase additional support for database replication, you can find step-by-step installation guide in the setup package of PROVEtech:TA in the directory *Goodies\Databases\Replication* or please contact our PROVEtech Support via provetech@akkodis.com. You can find more about database replication in chapter ***5 Test Manager***.

2. Use **Windows Remote Desktop Services** (also known as **Windows Terminal Services**). Please install PROVEtech:TA on a Windows Server which is near to the database server. You can get information from PROVEtech Support, if you want to start several PROVEtech:TA sessions in parallel from your distant computers. (Several PROVEtech:TA sessions in parallel are only possible by using **Multi Instance License**, depending on your license file License.key, see chapter ***2.2.1 Computer-bound licensing***.)

3. **Synchronization by Basic script:** Install one database server on each location and synchronize the shared subtrees by Basic script. This variant is very time-consuming and error prone. Therefore, it is only intended as a "last option", if the other alternatives are out of the question. You can contact the PROVEtech Support too (provetech@akkodis.com).

Please consider: The **Remote Sequencer** has some disadvantages with a slow database connection, as the polling mechanism (regular database queries every 2 seconds) generates some network traffic and server load. If the Remote Sequencer is to be used in conjunction with database replication, a test operation should first take place as to whether this combination can even be handled due to the double asynchronous delay. Read more about this in chapter ***5.14 The remote test sequencer***.

8.4.9 There is not enough storage space in the file system left for the database

When you delete test objects, test documents, test results or attachments (data acquisitions, files, …) in PROVEtech:TA, there is no immediate impact on the storage space of the database. The corresponding datasets are not finally deleted from the database until you execute special actions like “Empty trash folder” or “Purge database” in PROVEtech:TA’s Test Manager (see ***5.2.4 The trash folder***).

Database systems store the data of their instances inside table spaces, respectively in the corresponding table space files in the file system. If there are “auto-expand” parameters activated and there is not enough free contiguous space left in the storage file anymore then the table spaces grow automatically larger. Deleting datasets leads only to free space inside the files, but it does not make them shrink.

Now it may unfortunately happen that the database files consume the whole storage space on your hard disk. This could also lead to error message when starting vacuum tools or defragmentation tools, because they need temporary table space, too.

In PROVEtech:TA you will then get error messages like:

- ERROR: [Microsoft][ODBC Driver 17 for SQL Server][SQL Server]Could not allocate space for object '…' in database 'test.db' because... (SQL State 42000, SQL Server)
- ORA-01653: unable to extend table in tablespace (Oracle)
- ORA-01691: unable to extend lob segment (Oracle)
- ERROR 53100: could not extend file "base/16381815/130778960.10": No space left on device (PostgreSQL)

Please read also in chapter ***8.4.10 How can I determine the actual storage space of my database?***.

Problem:

How can I create additional storage space in the file system of the database server?

Solution:

1. Please contact your IT administrator. E.g., in case of a virtual server it might be easy to extend the capacity of the hard disk partition.
2. Please check if there are big files on the database server's hard disk that are not required to run the server. Maybe there are backup files which you can simply move to another storage drive.

Problem:

How can a defragmentation of the database be performed?

Solution:

1. The safest way to defragment the database is to back up the database and restore it afterwards (see below).
2. In the case of PostgreSQL, execution of the SQL VACUUM ANALYZE statement causes defragmentation within the tablespace files. However, the tablespace files themselves will not be downsized.

3. In the case of PostgreSQL, the tool VacuumLo.exe can be used to release large unneeded binary data (e.g. test protocols, data acquisitions and attachments of test results that are no longer referenced, or test documents at the test object). This also reduces the memory usage on the disk.
4. In the case of a Microsoft SQL Server, the call DBCC SHRINKDATABASE reduces the size of the data and log files in the specified database.
5. In the case of Microsoft SQL Server or Oracle, please contact your database administrator.

Problem:

How can I reduce the storage space of a PROVEtech:TA database in the file system, so that I am able to continue working again?

Solution:

The safest way to do a defragmentation of the data inside the database is to re-install the database system and re-import a database backup. Therefore, you should execute the following steps:

1. Backup your database onto a second storage medium (e.g. external hard disk) and check the corresponding .log file for successful operation after that (see ***5.11 Create and restore a database backup***). Even more secure would be to import the database backup into an independent second database server and verify the data with PROVEtech:TA.
2. Delete the test objects, test documents, test results and attachments (like data acquisitions) which are no more needed. The Find dialog can help you finding them, especially when you have older versions of subtrees (see ***5.18.1 Finding tests and test results***)
3. Choose “Delete older objects…” in the context menu of the trash folder to delete invalid test results and storage-extensive data acquisitions (see ***5.2.4 The trash folder***).
4. Empty the trash folder and purge your database (see ***5.2.4 The trash folder***).
5. Create a further database backup that you can use later for re-import. Check this backup, too, as described above.
6. Uninstall the complete database system and remove all database files as described in the installation directory, e.g. in Goodies\Databases\PostgreSQL\UninstallReadme.txt.
7. Arrange sufficient hard disk storage space for the database server.

8. Install a current version of your database system, e.g. by executing Goodies\Databases\PostgreSQL\InstallPostgreSQL.exe
9. Re-import the currently created database backup (see Goodies\Databases\PostgreSQL\ExportImportScripts\Import.bat) and check the data after that.

8.4.10 How can I determine the actual storage space of my database?

Problem:

How can I determine the storage space in the file system that my PROVEtech:TA database currently needs at least?

Solution:

To get the minimum storage space in the file system of the database data (that means in defragmented format and without system files) there are several possibilities:

1. Create a database backup (see ***5.11 Create and restore a database backup***); this is the safest and most exact way to determine the current storage space of the data
2. Test language command ***TestItem.GetStorageSpace*** (see Test Language Manual and corresponding example); this command returns storage size information about test documents, test protocols and attachments; these are the fields that cover 99% of data volume
3. Button `Further database checks` in the Settings dialog (see chapter ***3.2.7.6.3 Test ODBC connection and further database checks***); together with much more information, the storage sizes are written into the PROVEtech:TA Log (see ***3.6.1 PROVEtech:TA Log***)
4. The `Extended` tab in the properties dialog of a test object provides you some statistical data about the subtree in the Test Manager by running recursively through its children and their version chains (see chapter ***5.3.11.3 The tab "Extended" in the property dialog***)

Please read also in chapter ***8.4.9 There is not enough storage space in the file system left for the database***.

8.4.11 Failed to create a database backup

In order to avoid the loss of data, it is essential to regularly backup your database (see ***3.2.7.6.5 Configure the database backup*** and ***5.11 Create and restore a database***

backup). The following checklist describes points to consider and possible reasons for errors regarding the database backup.

- First check when the last successful and available database backup was created. This should be a "real" database backup that was created with a database management tool. If you only have a backup of your file system, this does not guarantee that the data can be recovered in an emergency.
- Please verify the version of your database server and backup tool. Do not use outdated versions. Update to the latest available Service Pack and patch.
- Check the file path to your backup tool as well as the existence of the target directory.
- Verify the .log file for error messages after performing the database backup.
- After creating the database backup, please verify if the size of the backup file is plausible.
- Verify the duration for creating the database backup in respect of the storage space of the data.
- Please always execute the database backup locally on the database server, but not over network.
- Make sure that there is enough free disk space available.
- Make sure that there are as few as possible processes running on the database server to keep the processor load small while doing the backup.
- Make sure that there are as few as possible client connections opened.
- Do not backup data that is no more needed. Do not let the data grow too big. In case of problems, empty the trash folder of PROVEtech:TA and purge the database. Execute vacuum and defragmentation tools, if still possible.
- Verify PROVEtech:TA for extremely large data records (e.g. attachments or test documents that are larger than 1GB).
- In case of error messages concerning the memory management (“out of memory”) check the database server parameters regarding memory usage. E.g. in case of PostgreSQL you can try to double the parameter `effective_cache_size` or the physical memory. Please also verify if you use the Windows 64-bit version of the database server and backup tool. Even on a 32-bit database server it could be helpful to use the 64-bit version of the backup tool.
- In case of unknown error messages (e.g. in the .log file) an internet research might be helpful.

- If there are further issues please contact PROVEtech Support (provetech@akkodis.com).

8.4.12 Transfer the data to another database instance

The transfer of PROVEtech:TA data between different database instances should always be coordinated with the customer's IT department or the database administrator, unless it is a standalone solution. In principle, the following always applies: Back up your data via a database backup (see chapter ***5.11 Create and restore a database backup*** Chapter) and then remove superfluous data from the database (see chapter ***5.2.4 The trash folder***), as this saves time and storage capacity during migration.

To transfer the data of the entire database instance (all data of the Test Manager) from one database to another, two cases have to be distinguished with regard to the database system (PostgreSQL, Microsoft SQL Server, Oracle):

- Both databases are from the same database system: Here you can use the database backup and restore tools to transfer the entire database with relatively little effort (example: porting data from PostgreSQL version 10 to PostgreSQL version 12)
- The databases are from different database systems: Here we recommend the migration via a commercial additional tool or a database replication system. For example, a PostgreSQL database is coupled with the SQL Server database via database replication and thereby the data is migrated. Or the export/import tools of a replication system are used for this purpose. PROVEtech support can assist you with this procedure:
    - Possibly commissioning of our experts at PROVEtech Support
    - Create a PostgreSQL backup and send the dump to provetech@akkodis.com
    - PROVEtech Support migrates the data via database replication (e.g. via the export and import tools of the SymmetricDS replication software).
    - The PROVEtech support creates a SQL Server backup and sends it back
    - Installing the backup on your SQL server (e.g. by your database administration)
    - The whole process often takes a few days (for organizational reasons). In the end, the user receives the status of the data at the time of the backup, i.e. possibly several days old.

If the database is to be moved to another server computer using the same database system, the following procedure can be used:

1. Delete superfluous data in the Test Manager and empty the trash folder object as well as purging the database (see chapter ***5.2.4 The trash folder***)
2. Create a database backup with the database export tool or via PROVEtech:TA backup mechanism
3. Install the database server and an empty database instance on the new server computer according to the readme files in the installation directory
4. Import the backup data to the new database instance according to the readme files in the installation directory
5. Set up a new database connection in the Settings dialog of PROVEtech:TA and test the new server
6. Do not forget to test the backup strategy on the new server
7. Set the time for the new productive operation
8. If necessary, uninstall the database server on the old server computer according to the readme files in the installation directory

Another use case is the merging of parts of a database into another database. However, this is complicated and requires a lot of work by hand. Use the export/import functionality via PROVEtech:TA archive files (.dbi.7z files, see chapter ***5.3.10 Exporting and importing tests***). So, this is a database-independent mechanism and runs completely via the PROVEtech:TA client. The following must be taken into account:

- The more extensive the Test Manager subtree to be imported, the more time is needed for the export. On the other hand, the larger the subtree, the more links between the records are preserved (links, parent-child relationships, ...). It is best to start with small subtrees and work your way up to larger subtrees.
- The export process and especially the import process takes much more time than a database backup and database restore. It could even be that importing a complete backup with following deletion of unnecessary data is faster than importing a very large subtree.
- If there are inconsistencies in the data (e.g., a test version references a previous version that is not contained in the database), this can lead to error messages during the import and the import may even be aborted. Therefore, check the consistency of the data in advance. Please also read chapter ***5.12 Database consistency checks***.
- The less data is transferred over the network, the faster the single step is. (This applies to all variants).

- For reasons of traceability, the "reassignment" of test results to other test objects is not intended.

8.4.13 PostgreSQL error message “Out of shared memory”

Problem:

A laborious SQL query such as reading larger binary data (test protocols, attachments, or test documents) or database purge results in the "Out of shared memory" error message.

Reason:

Please increase the parameter `max_locks_per_transaction` parameter in the postgresql.conf file on the database server (e.g. from 64 to 512) and restart the database service.

8.4.14 New created database IDs violate unique index constraint

Problem:

When creating new data, I get an error message of the following type:

PostgreSQL: DB: ODBC Error: ERROR: duplicate key value violates unique constraint "info_pkey (lNativeError=1, wstrSqlState=23505)"

Oracle: ORA-00001: unique constraint (PROVETECH.SYS_C0064350) violated~ (lNativeError=1, wstrSqlState=23000)

SQL Server: [SQL Server]Violation of PRIMARY KEY constraint 'PK_GuildChars'. Cannot insert duplicate key in object 'dbo.GuildChars'., SQL STATE: 23000, NATIVE ERROR: 2627/2601

Solution:

In the PROVEtech:TA database, Sequence objects are used to create new unique database identifiers. Here it must be ensured that the new IDs are not already in use.

This means for the ***standard case*** (i.e., no replication system) that the newly generated ID of the Sequence object must always be larger than the previously used IDs from the data records of the tables. If error messages of the above kind occur here, all Sequence

objects can be deleted by the database administrator. The PROVEtech:TA client will then automatically create correct new Sequence objects.

In a ***replication system***, additional care must be taken to ensure that the range of new IDs does not overlap between the different replication nodes. This can be solved, for example, on a two-node replication system (two database instances) by creating only odd IDs (101, 103, 105, …) on the first replication node and only even IDs (102, 104, 106, …) on the second. The database administrator must fulfill this requirement here by manual intervention.

You can check the database sequences by running `Further database checks` (see chapter ***3.2.7.6.3 Test ODBC connection and further database checks***).

There is always a possibility that the error message occurs because of a software bug in PROVEtech:TA. In case of this suspicion, please proceed as described in chapter ***2.13 Creating a problem report***.

8.4.15 SQL Server error message about Transaction log

Problem:

From time to time your SQL Server returns the error message "ODBC Error: [Microsoft][ODBC Driver 17 for SQL Server][SQL Server]The transaction log for database '…' is full due to 'LOG_BACKUP'. (lNativeError=9002, wstrSqlState=42000)".

How can I fix this?

Solution:

Logging and recovery refer to a collection of measures and techniques to prevent data loss in database systems after an error (e.g., after a system crash or a hard disk failure) and to continue to guarantee that the database is error-free (consistent). The transaction log records all changes made to the database during operation, and the recovery function restores the database to its last current and consistent state after an error (e.g., after a reboot in case a system crash was responsible for the error). The system components needed for logging and recovery are temporary log files.

Transaction log contains 3 types of transactions.

1. Just running transactions
2. Transaction was confirmed but is it not written to the data files (.mdf files)

3. Transaction was confirmed and written to the data files (.mdf files); in this point data is written in 2 places: the .mdf file and the log file. This is performed occasionally at one minute or longer intervals.

There are 3 types of recovery models in the SQL Server (There are only mentioned the properties that are relevant to the problem with the transaction log file):

a) Simple: Transactions of type 3 do not exist

b) Full: It is possible restore to any time in history. A full backup deletes the type 3 in the transaction log

c) Bulked Log: Same as Full with only one exception – bulk operation is not written to the database as simple recovery model

Everyone has a full backup recovery model. Simple is just for testing and bulk full is for some database initialization.

From the description above there is more option how to shrink the SQL Server log file (https://www.mssqltips.com/sqlservertutorial/3311/how-to-shrink-the-transaction-log/):

Here are possible solutions how to shrink the log file:

1. Perform a regular full backup (If recovery model is full or bulked logged); it does not shrink the log file but marks the log space for reuse, so it can help to keep the transaction log file from growing too large.
2. Perform a log backup (If recovery model is full or bulked logged); it does not shrink the log file but marks the log space for reuse, so it can help to keep the transaction log file from growing too large.
3. Use DBCC function to shrink the file. It is not recommended because shrink all empty space of all virtual log and it cause performance issue of the database.

```
DBCC SHRINKFILE (MyDatabase_log, 1)
```

Summary:

Problems with database transaction log files are most likely the result of a missing database maintenance strategy that needs to be set up.

8.4.16 The database is busy

Problem:

The database returns one of the error messages:

ORA-30006: resource busy; acquire with WAIT timeout expired

ORA-00054: resource busy and acquire with NOWAIT specified or timeout expired

PSQL ERROR 53000: insufficient resources

SQL Server (lNativeError=8642, wstrSqlState=42000) The query processor could not start the necessary thread resources for parallel query execution.

SQL Server (lNativeError=8645, wstrSqlState=42000) A timeout occurred while waiting for memory resources to execute the query in resource pool '%ls' (%ld). Rerun the query.

SQL Server (lNativeError=701, wstrSqlState=42000) There's insufficient system memory to run this query.

SQL Server (lNativeError=1222, wstrSqlState=42000) Lock request time out period exceeded.

Reason:

The error message means that certain records are locked for write access because they are currently being written by another PROVEtech:TA client, or that the database is currently busy. The transaction concept of the database plays a role here. An operation should always be done within a single transaction, so that at the end it is either executed completely (by COMMIT) or not at all (by ROLLBACK), and in no case only half of it. If now on a PROVEtech:TA client a complex operation takes place (e.g., release creation of a large subtree or also copying of a large subtree including test results), then the transaction takes longer and locks out other PROVEtech:TA clients on these data sets. This results in the mentioned error messages. Therefore, please check if there are any heavy operations on neighboring PROVEtech:TA clients. Or maybe the database server is performing an exclusive database backup. As a last resort, it might also help to restart the database server.

Solution:

Try the operation again later.

8.5 Problems concerning test programming

The following chapters describe problems and solutions concerning test programming.

8.5.1 A strange WinWrap Basic license error dialog pops up

Probably the file “Application-8a0654e1.htm” of your PROVEtech:TA installation is damaged. The file should have a size of about 2 KB. If it is larger, you should check your computer for viruses modifying HTML files to spread themselves – especially if the problem still occurs after re-running the PROVEtech:TA setup.

8.5.2 How to access COM / .NET Assemblies / Word / Excel / CANoe / ActiveX / OLE / third-party components in your test program

Please read the chapter ***Accessing foreign software*** of the test language manual.

Additional hints:

- The external components are typically not developed by Akkodis. This means, we only have very little insight, and therefore our possibilities to support are very limited.
- A small, but completely running Basic script is often very helpful for analysis. It would be best to have it as an exported .mac file that includes all internal references.
- To determine if the problem is located in PROVEtech:TA or in the third-party component, you can try to run your script via VBA e.g. as a Microsoft Excel or Word macro.
- In case of a COM interface: It may happen that you are not able to add a .dll file using the `Browse...` button in the WinWrap Basic References window (“Bad reference: Error loading type library/DLL”). It fails while loading the type library (Windows function “LoadTypeLib”). Either the type library file is corrupt on this computer or the according registry entries are damaged. We suggest to re-install the third-party component on your computer and then try it again.
- Check whether there is a more recent version of the third-party component. Sometimes uninstalling and reinstalling the third-party component can help.
- The WinWrap Basic script refers to the wrong GUID or version number, for example after installing a more recent version of the third-party component; the GUID and the version are the crucial components for the reference, the other details (e.g. name) are additional information for display; the references are stored as hidden lines in the header of a WinWrap Basic script; you can see these lines when exporting to .mac file

- You tried to add a DLL as .NET assembly, but the DLL is no .NET assembly (it is just a regular DLL).
- DLL dependencies do not fit (e.g. the Microsoft C runtime is missing); you can see this with the programming tool “Dependencies” or "Dependency Walker"
- Check whether this is a syntax error or a runtime error. In the case of a runtime error, the third-party component could at least be loaded.
- It is a 32bit/64bit problem (actually only with DLL, not with EXE, because it runs outside the process); read more in chapter ***2.10 32-bit and 64-bit***
- You can try to delete the reference in the script and create a new one (Caution: This could be problematic, because on other computers the script might not work anymore)
- Does the problem only occur on this PROVEtech:TA installation?
- A problem in the Windows operating system cannot be ruled out. Check for patches and updates, please research on the Internet.

When WinWrap Basic resolves the COM reference, the following happens:

1. Read the 128-bit GUID and version number from the test program
2. Lookup the GUID and version number in the Windows registry (64-bit variant as PROVEtech:TA is 64-bit) to find the path to the DLL
3. Load the DLL

If one of these steps fails, it’s typically because of:

- The DLL is not registered in the Windows registry correctly
- The DLL is a 32-bit DLL, but PROVEtech:TA needs a 64-bit DLL
- Loading the DLL fails due to missing dependencies (DLL needs other DLLs which it doesn't find)
- The DLL has no COM interface at all. This means, the test program simply uses it in a wrong way.

The following must be considered when updating or auto-updating the third-party software:

- While a DLL is loaded (no matter if via PROVEtech:TA or any other software), you cannot replace it (e.g. via an auto-update mechanism). Windows prevents this, because the file is locked.

- If the interfaces of DLLs do not change when they are exchanged, this normally has no effect. If, on the other hand, the interfaces change, the user may have to adjust their WinWrap Basic scripts when calling the interfaces. The DLL must then be replaced on all PROVEtech:TA clients that use the new interface.
- For COM servers a registration in the Windows Registry is done. Again, it is the responsibility of the DLL to remain compatible. If it is, the caller usually does not have to change anything. If it becomes incompatible, the user must typically make appropriate changes in the WinWrap Basic script at the caller.
- There is also the possibility of having different versions of a DLL in parallel on the system. This is also controlled by the manufacturer of the third-party software, not by PROVEtech:TA.

8.5.3 Error message when accessing a Basic object or a Basic class

Problem:

I try to link an object or a class via `'#uses` command, but I get an error message when accessing it. My code looks like this:

```
'#uses "*MyClass"
Sub Main
    Dim cls As New MyClass   'this produces an error message
End Sub
```

Solution:

Be aware that Basic distinguishes a file name and a class / object name (just like C++). In the example above, the class has the file name "MyClass". It can have a totally different class name. You can view and edit the name of a class or object by clicking on `Edit Module Properties` in the Basic toolbar. This problem normally occurs after renaming a class or object in the test tree; this only changes the file name but not the class or object name.

8.5.4 Problems concerning “Go To Definition Of...”

Problem:

I select *Go To Definition Of ...* in the context menu, but the editor displays the wrong definition of the selected text.

Solution:

The menu item *Go To Definition Of ...* tries to detect to which module the marked method/variable/etc. belongs to (the context). This only works if WinWrap Basic knows the symbol. If not, PROVEtech:TA searches methods/variables/etc. with the same name and marks the first match. Why is it hard to create a better search function? Just have a look at the following example:

```
Sub DoSomething(obj As Object)
    obj.MySub
End Sub
```

When you have several "MySub" implementations - which one is the correct one to display in this case?

The best way to avoid this problem is to use unique names.

8.5.5 Alternative test languages instead of Basic

Problem:

I do not like Basic, I'd like to have <enter your preferred programming language here> (a small extract of what users already have demanded: Python, Perl, Java, C, C++, C#, Ruby, Kotlin, PHP, M-Script, Pascal, Lua, any object orientated language). Will you replace Basic by this or support it additionally?

Solution:

No. There are many different programming languages. Very often, the personal feeling stands before the actual technical requirements concerning this. Some people also think that it is possible to re-use existing test programs written in a different programming language from a different system. However, these typically use system specific libraries that look differently when exchanging the system.

Here is an incomplete list of WinWrap Basic advantages:

- WinWrap Basic has an editor with syntax highlighting, parameter tooltips and auto completion, which you can integrate and control within your application.
- There is an integrated debugger.
- You can extend WinWrap Basic with own commands and the user does not need to add lines for including them (PROVEtech:TA test language).
- WinWrap Basic can use COM and .NET interfaces without additional libraries and it can directly display type information, auto completion, etc. for them. For example,

you can directly access Microsoft Excel, Word, Vector CANape, CANoe and ETAS software without additional dependencies.

- WinWrap Basic can directly access the Microsoft .NET framework without additional libraries. This means there is a well-documented and reliable framework for a lot of typical tasks like XML parsing, displaying graphical user interfaces (XAML), communication (TCP/UDP/RS232) and common file access.
- WinWrap Basic can directly access the native Win32 API. For that, you just need Declare statements (easy to find in the Internet).
- In case of issues, we can contact a reliable and fast support and very often, we have a working solution the next day.
- It has a simple licensing model fitting to PROVEtech:TA.

By the way, it is also possible to access PROVEtech:TA's COM interface externally from different programming languages which support COM. The test language manual has a chapter ***Access from foreign software*** with more information.

8.5.6 Output parameters in Basic methods

Problem:

I am calling a test script method, which has an output parameter in the parameter list.

But the parameter is not filled as I expect after running the method.

Solution:

Please check the type of the parameter.

For example, if you use an Integer, but the method expects a Long, the output parameter will not be filled. No warning is issued!

Example from CAESAR diagnostics and parameter iStatus / lStatus:

```
Dim strOutputParams$
Dim iStatus As Integer
Dim lStatus As Long
Dim bOK As Boolean

'error!
bOK=Diag.SendService("DMFL_204_NEW", _
    "RT_LIN_Diagnose_Start_Response", _
    "00|00|00|00|00|00|00|00|00|00", _
    strOutputParams$, iStatus)
```

```
Debug.Print "iStatus="+CStr(iStatus)  'iStatus is still 0!

'the next call is fine
bOK=Diag.SendService("DMFL_204_NEW", _
    "RT_LIN_Diagnose_Start_Response", _
    "00|00|00|00|00|00|00|00|00|00", _
    strOutputParams$, lStatus)
```

8.5.7 How can I uniquely reference a test object?

Problem:

Can I use the database ID of the test object for unique referencing?

Solution:

We recommend that you use the path of the test object or test version to reference it. There are two variants: Paths consisting of the test object names and paths that use the database ID (see ***TestItem.Path*** in the **Test Language manual**). The different path specification options have advantages and disadvantages which have to be considered depending on the use case (e.g. validity after moving operations, readability of paths, validity after export and import, versioning has to be considered).

8.5.8 Resolving cyclic references

Problem:

How can I resolve cyclic references of '#uses calls in my WinWrap Basic code?

Solution:

The default process for resolving cyclic references is the following:

If a module A contains a function P and it uses functions of module B, and module B uses the function P of module A, then you simply source function P out in a third module C which is referenced by both modules A and B.

8.5.9 '#Language "WWB.NET" is not working

Problem:

I get an error message on all '#Language "WWB.NET" lines telling me that this is not available.

Solution:

In order to use "WWB.NET", WinWrap Basic needs to access Microsoft's .NET Framework. There might be several issues when trying to do this:

- Maybe your IT has installed an update of the .NET Framework and you do not even know about that. First try to reboot your computer to finish the installation process.
- To load the .NET components WinWrap Basic needs several Windows registry keys created by PROVEtech:TA at startup. Unfortunately, Microsoft's components access different keys when the current Windows user has administrator rights. This means PROVEtech:TA needs to create different keys in that case. From time to time Windows confuses all those registry keys. Please try:
  a) Reboot
  b) Run PROVEtech:TA with Windows administrator rights once
  c) However, never work with Windows administrator rights in normal cases
- Sometimes the .NET Framework itself gets damaged. Please try:
  a) Install the latest .NET Framework update
  b) There is a repair tool from Microsoft for fixing .NET issues:
  https://www.microsoft.com/en-us/download/details.aspx?id=30135
  If this link is not working anymore search for "Microsoft .NET Framework Repair Tool".
  c) There is a more radical tool:
  https://blogs.msdn.microsoft.com/astebner/2008/08/28/net-framework-cleanup-tool-users-guide/
  If this link is not working anymore search for ".NET Framework Cleanup Tool".
  d) Re-install Windows completely
- There are several .NET Common Language Runtime variants. WinWrap Basic only supports version 4. Useful information:
  https://en.wikipedia.org/wiki/Common_Language_Runtime

8.5.10 The execution time of the Basic script takes very long

The following are hints to help you analyze the execution time of the Basic script:

1. Use the Basic debugger to get a feeling of how or how often `If` statements, loops, and functions are being run through.
2. Use ***System.Remark*** `CStr(Now)` to measure execution times of program parts, as well as to see how often certain program parts are run through.
3. Are the same intermediate results calculated over and over again? (e.g. in loops)
4. Do time-consuming statements happen outside or inside loops?
5. Are many or sometimes even unnecessary libraries included per `'#uses` statement? Are several different versions of the same library included? Is there an incorrect use of the ***FixVer*** addition?
6. Do many accesses to the network take place? (E.g., loading of configuration files , workpages, models or diagnostic data)
7. Is ***System.GetValidSignalList*** called very often? Do you use complicated regular expressions? Are very long signal lists generated?
8. Are very large data acquisitions generated?
9. Avoid frequent ***TM.ClearCache*** calls
10. Is the same data fetched from the database over and over again? (e.g. in loops)
11. Are the same Test Manager paths constantly being parsed?
12. Avoid frequent ***System.Remark*** outputs with the option `bWriteThrough=True`
13. Are test parameters constantly read and written
14. Are very large test protocols generated?

See also chapter ***8.12.3 PROVEtech:TA runs very slowly*** as well as chapter ***8.4.8 The database is very slow***

8.6 Problems concerning the AL/RTAE

In this chapter we describe some common problems that can occur when using the AL (Automation Library) and the RTAE (Real-Time Automation Engine).

8.6.1 RTAE link overflow

Problem:

What to do about the error message "Error: RTAE link overflow when writing on node XXX to YYY"?

Solution:

If your RTAE test program runs on model node XXX and accesses signals from node YYY (reading or writing), the data must be transported via the real-time links of your test system. Your test program has exceeded the limits during a write operation.

When writing, the number of write operations (plus the amount of data per write operation) within a model tick is crucial. In one model tick, your RTAE program has written a too large amount of data for another model node.

Possible solutions:

1. Fewer write operations in a model tick (distribute over several ticks)
2. Run the test program on the node where the signals are located
3. Avoid unnecessary write operations (e.g., set signal to the same value again and again in a loop).

See also chapter ***Working with multiple model nodes*** in the Automation Library manual section.

8.7 Problems with CAN (both CAESAR and MCD-3 D)

In this chapter we describe some common problems occurring in CAESAR and in MCD-3 D diagnostics.

8.7.1 TX Error

The exact text of the message depends on the used D-PDU API software. This error means that the ACK bis could not become dominant. It happens when

- The bus is busy by higher priority messages (lower IDs)
- The wiring is not correct (e.g. no 120 Ohm terminating resistor, Hi/Lo wires exchanged, no power supply or wrong baudrate)
- There is no receiver connected to the bus (any receiver sets the ACK on dominant to confirm receiving of the message)

8.7.2 RX Error

The exact text of the message depends on the used D-PDU API software. This error occurs when the request could be sent but the software did not receive an expected response in a defined time span.

Tis can happen when the ECU is not connected, has no power supply or is in a sleep mode or does not react for any other reasons.

Another common cause is the usage of a wrong REQ-ID (ECU does not react) or RES-ID (ECU reacts but tester does not understand the message as an answer), e.g. the connected ECU is not compliant with the ECU data, or the data contain a wrong ID.

Another possibility is running into a timeout. This can happen in case of a long-running routine in the ECU or a very busy CAN bus. In this case temporarily changing the timeout value can be helpful. You find details about setting communication parameters in ***6.1.9 Communication parameters*** (for CAESAR) and in ***6.2.3.12.1 Special case: ComParams (service ProtocolParameterSet)*** (MCD-3 D).

See also ***8.10.5.5 RX-Timeout (PDU_ERR_EVT_RX_ERROR)*** for more hints.

8.8 Problems concerning the eCOM

In the following chapters you find some possible eCOM problems affecting either CAESAR or MCD-3 D diagnosis.

eCOM is the only diagnosis hardware (MVCI) to be used for CAESAR in Windows 7 and newer.

8.8.1 General eCOM problems

The eCOM hardware and the associated software (D-PDU API and SDconnect Toolkit) are quite error-prone. There is no alternative hardware for CAESAR diagnostics (in windows XP and earlier there were also other hardware parts, esp. PartY). In MCD-3 D diagnosis eCOM can be replaced by a Vector hardware with one or more CAN lines. For communication with ECUs which have an Ethernet port (Diagnosis over IP = DoIP), no additional hardware is needed. In both cases (Vector device and Ethernet) a special D-PDU API software is needed.

When PROVEtech:TA runs into problems using eCOM, there can be several different causes. The following chapters describe different problem categories and are helpful to find the cause.

8.8.2 Malfunction between eCOM and PC

Sometimes the Ethernet connection to the eCOM cannot be established.

Possible causes for this malfunction are numerous: a missing power supply for the eCOM, missing or wrong opened ports in the firewall or a missing driver for the USB2LAN adapter.

Possible solutions:

Make sure that the eCOM has a power supply and is connected to the PC. Check furthermore if you have opened the necessary ports for the network connected to eCOM in both directions in the firewall (UDP: 4009, 4010, 4011, 4012; TCP: 4349, 4500). Please note also the correct setting of the network (*private*, *public*, *domain*…). The easiest way to open the ports is to open them via a package provided by the IT. This package contains also the driver for the USB2LAN adapter (you can also find it on the PROVEtech:TA installation medium).

Occasionally it can happen that the set IP address or the set subnet mask causes problems. The recommended IP address for the eCOM is 169.254.255.42 and the recommended subnet mask is 255.255.0.0.

When switched on, the green LED on the eCOM must be permanently burning after some seconds. In this state the eCOM should be accessible from command line by the command *ping 169.254.255.42*.

The correct function of the hardware can be checked by additional applications, sometimes delivered together with the D-PDU API: TkAdministration.exe and IME_D-PDU_API_Tester.exe. Please do not forget to set up the firewall also for all programs as described above. It might happen that these helper programs detect a correct function of the eCOM but PROVEtech:TA gets no connection due to diverging firewall settings.

Important:

Since version 2025 the MCD-3 D diagnostics runs in a separate process. The firewall settings must be adapted for this process. The settings for the TA.exe process are not sufficient any more. In case you use the DTS9, you need the same settings for the DiagnosticsMCD3DExt_DTS64.exe process.
In case you have several rules contradicting each other, the blocking rule has the higher priority.

8.8.3 Malfunction between eCOM and PROVEtech:TA

This kind of malfunction can be recognized by a correct LAN connection showed by Windows, however the status lamp of the eCOM does not blink but glows continuously. This means that the eCOM is not allocated.

Possible reasons for this kind of failure are wrong settings or missing parts of the system components (CAESAR, MCD-3 D server, D-PDU API, SDconnect Toolkit)

Possible solutions:

Make sure that you chose the correct driver in PROVEtech:TA. To use the eCOM choose the driver "Part P (eCOM)" or "MVCI Device" (see chapter ***3.2.7.7 CAESAR diagnostics***).

Open in subdirectory „bin\Caesar\driver“ of the TA program folder the file "slave.ini". In section "[MVCI_Device]" is an entry "RootPath" defining the RDF file, by default: "C:\Users\Public\Documents\PROVEtechTA\caesarpdu\XS D PDU API\ RDF_I+ME_ACTIA_XS.xml"

This file contains the paths to the following files: "PDUAPI_I+ME_ACTIA_XS.dll", "MDF_I+ME_ACTIA_XS.xml" and "CDF_I+ME_ACTIA_XS.xml". Please check if these paths are correct too. By default the path should be: „C:\Users\Public\Documents\PROVEtechTA\caesarpdu\XS D PDU API“

Furthermore make sure that the paths stated in the entries "ToolkitApiDll" and "ToolkitTraceDll" of the "PDUAPI_I+ME_ACTIA_XS.ini" (section "SETTING") are correct and point to existing files:

```
[SETTING]

ToolkitApiDll=
C:\Users\Public\Documents\PROVEtechTA\caesarpdu\SDconnect
Toolkit\toolkitapidll.dll

ToolkitTraceDll=
C:\Users\Public\Documents\PROVEtechTA\caesarpdu\SDconnect Toolkit\toolkit-
trace.dll

[Logging]
LogFilename=<Path>
```

The LogFilename must point to a file in a folder which the current user has write access. After installing the software with admin rights it may happen that this file exists but cannot be opened by the normal user. This sometimes causes crashes of the D-PDU API.

The same applies to the log file entries in the toolkit.ini file – the configuration of the SDconnect Toolkit, placed in the same folder as the mentioned ToolkitApiDll.

RDF file: errors in format or some special characters in this file can cause problems with CAESAR, event when the same file works fine with the MCD-3 D server. We recommend to use a separate RDF file for CAESAR, containing only the eCOM interface definition.

If you use MCD-3 D, you find a detailed configuration instruction in ***8.10.1 Configuring MCD-3 D servers*** and the following chapters.

When installing CAESAR and MCD-3 D, it is strongly recommended always to use the D-PDU API and Toolkit versions supplied by Daimler in the software package together with CAESAR and DTS8, as only this version combination has been tested and approved by Daimler. Mixing versions might lead to unpredictable results.

8.8.4 Malfunction between eCOM and ECU

When a communication with the ECU is not possible, although the LAN connection to the eCOM is correct and after the start of PROVEtech:TA the status lamp of the eCOM blinks properly it is an indicator for this category of malfunctions.

A possible reason is that the eCOM has no power supply via the D-Sub 15 connector.

Possible solution:

The permanent power supply via the D-Sub 15 connector is mandatory for the communication. Please make sure that there is a UBat connection to PIN 8 and a GND connection to PIN 14 respectively PIN 15. Do not use the same power supply you use for the ECU(s), because you can typically control that supply at runtime and switch it on and off. The eCOM needs a permanent supply without any disturbances. An additional supply via the round plug at the back is optional, but might be helpful.

Please check also the CAN connection: the Hi and Lo cables must not be swapped. The connection must be terminated by a 120 Ohm resistor.

8.8.5 eCOM not recognized

Symptom:
After installing DTS9 a warning about non-active eCOM blocks initialization of the diagnostics module. See also chapter ***8.10.3.1.3 Device Explorer blocks initialization***.

Explanation:
The DTS9 setup installs several D PDU-API. The I+ME Actia software brings the mentioned warning when no eCOM is connected and properly configured.

Solution:
If you do not use eCOM hardware, please delete the interface definition from the RDF file.

See chapter ***8.10.1.4 Configuring DTS8 and DTS9*** for details about RDF file.

8.8.6 Conflict between CAESAR and MCD-3 D

When MCD-3 D diagnosis is active, the access to eCOM is blocked for CAESAR and vice versa. Attempts to contact the ECU are stopped by the diagnosis server with a message "*No matching resources to open a channel*" or similar. After that error the server might get into an instable state (depending on the version of the D-PDU API) and must be restarted.

Solution:

Shutdown MCD-3 D Diagnosis before restarting CAESAR. In Settings, tab "Configurations", it is possible to determine which diagnosis platform shall get access to the eCOM hardware. This prevents both systems from time consuming simultaneous attempts to initialize eCOM during program start or configuration change. A switch between CAESAR and MCD-3 D is still possible after initialization.

A concurrent usage of CAESAR and MCD-3 D is possible, but only if both platforms use different hardware (i.e. CAESAR via eCOM and MCD-3 D via Vector device).

8.8.7 Version mismatch between firmware and toolkit

When first connecting the eCOM with newer SDConnect Toolkit software, the eCOM firmware is sometimes updated without asking the user. This can lead to the following quite often occurring problem:

- A colleague borrows my functioning eCOM – it works in his system also.
- Afterwards the eCOM does not work in my system any more.

Reason:
The colleague's SDConnect Toolkit is newer than mine and has flashed the eCOM with newer firmware. The older Toolkit cannot deal with it.

Solution 1:
Manually downgrade the firmware (quite complicated)

Solution 2:
Update your own SDConnect Toolkit.

CAESAR, D PDU-API and Toolkit are delivered in a tested combination, but it is possible to update PDU-API and Toolkit (both together) – in this case we did not register problems so far.

8.8.8 Further problems

In case the status lamp of the eCOM blinks before the start of PROVEtech:TA the failure can be traced back to another program that is allocating the eCOM. Please close this program and restart the Diagnostics module (see chapter ***6.1.2.1 Static commands of the diagnostics dialog*** and ***0***

Control toolbar).

If the status lamp of the eCOM blinks faster than usually, please restart first the eCOM and then the CAESAR or MCD-3 D module, whichever is in use.

After a longer period of inactivity it might happen that the eCOM gets into a sleep mode and does not react. The only way to reactivate it again is to switch its power off and on again, and also to restart the diagnosis module. To avoid this problem during longer test scripts, shutdown the currently diagnosis module for the time it is not needed and restart it again before diagnostics shall be used again. This problem does not happen with other communication hardware (MCD-3 D Diagnostics can be run with Vector devices instead of eCOM).

Further possible causes for failing to contact some ECUs are wrong settings in PROVEtech:TA, i.e. a false baud rate. Please make sure that all your settings within PROVEtech:TA are correct.

Crash at program start: The eCOM software contains a bug which causes a crash when MallocSpy is active. MallocSpy is a function of the operating system for detecting memory leaks. In PROVEtech:TA it can be activated via the file TA.ini.
If you use eCOM with CAESAR or MCD-3 D, please set in TA.ini the parameter ActivateLeakDetection to 0.

8.9 Problems concerning the CAESAR diagnostics

The following chapters describe problems and solutions concerning the CAESAR diagnostics.

8.9.1 No matching resources to open a channel

Problem:

You try to activate an ECU and receive the error message "*CAESAR API error. No matching resources to open a channel, please check your configuration*". This is the most popular and annoying error message in CAESAR which can have very different reasons.

eCOM does not respond:
This can mean, the hardware is damaged or switched off or incorrectly wired, it can have a wrong firmware version, it can be in sleeping mode (after a short power-down or a longer period of inactivity), or the contact is blocked by the Windows firewall.

If the communication worked already but stopped during testing, restarting Caesar will usually help.

If this eCOM did not work yet with the current system, please refer to chapter ***8.8 Problems concerning the eCOM*** and ff.

Wrong data versions:
If the eCOM already worked at your system, but you get the mentioned error message after attaching a new ECU or updating the ECU's data (CBF file): Please check the trace file. If there is a warning before the mentioned error message: "*Required protocol not available for current firmware*", then maybe your old firmware or old GBF cannot deal with the new CBF file. Have you recently added a new CBF file? Updating CAESAR will most probably solve the problem. We recommend to upgrade PROVEtech:TA (it is delivered with the newest CAESAR version). It is also possible to exchange CAESAR files and configuration in a PROVEtech:TA installation. But the version combination you get has not been tested and could lead to some compatibility issues.

A different reason for this error message appearing just for some ECUs may be a wrong interface setting in the PROVEtech:TA tab `Diagnostics`. In the diagnostics panel select the tab "Settings" and then the involved ECU in the diagnostics tree. Verify, that the "Requested Interface" value is conform with your ECU and your hardware environment. The so called "Interface" (CAESAR terminology) consists of 3 components: The hardware name (CANHS, CANLS, KLINE ...), the protocol (UDS, KW2C3PE, KW2000PE ...) and the baud rate.

Other possible problems - You are using Nexiq / Part J / Pass-Thru:
Please read chapter ***8.9.4 Nexiq Device / Part J / Pass-Thru***.

See also:

8.8 Problems concerning the eCOM

8.9.2 No hardware present

Problem:
You get an error message "No hardware present", although the hardware is properly connected and works with MCD-3 D module.

Solution:
The most probable cause is incorrect setting "Hardware present" in the configuration (see chapter ***3.2.7.7 CAESAR diagnostics settings***)

Problem:
At program start or when initializing the CAESAR module you get the message:
Initializing Diagnostics Caesar: Caesar API error. Init of Caesar communication module failed.

Alternatively: when restarting the CAESAR Module while the program already runs, you get:
Caesar Restart Error: Root.Connect

Solution:
The D-DPU API (the interface) cannot be initialized. Probably there are errors in the RDF file. Please see chapter ***8.8.3 Malfunction between eCOM and PROVEtech:TA***.

8.9.3 Diagnostics interface of an ECU

Problem:

In the `Settings` of an ECU, PROVEtech:TA offers only so called “interfaces”, which does not match your hardware requirements. For example, you need a different baud rate. What can I do?

Solution:

PROVEtech:TA takes the offered "interfaces" from the .cbf file of the ECU. If there is no interface, which matches your requirements, you normally need a different .cbf file for your ECU. In the special case of a wrong baud rate, there is an additional workaround:

Before activating your ECU, select the "COMParams" node of the ECU in the diagnostics tree and then "CP_BAUDRATE". Set the field "Write Buffer" to the correct baud rate and then activate the ECU (the field "Write Option" should contain the default selection "static"). You need to do this every time you want to activate the ECU.

8.9.4 Nexiq Device / Part J / Pass-Thru

Problem:

I have plugged in the Nexiq Device or I am using Part J / Pass-Thru, but I do not get a communication with the ECU. I get an error like “No matching resources to open a channel”.

Solution:

This hardware type uses the so-called Pass-Thru technology. This means CAESAR does not directly access the drivers of the hardware, but indirectly via a standardized API. For

the Nexiq device you must install a Pass-Thru DLL beside the Nexiq driver. The name is typically SID32.DLL and is no part of the Nexiq driver package or CAESAR. You should obtain this DLL separately within you organization. For the 32-bit PROVEtech:TA and CAESAR version you need a 32-bit DLL and for the 64-bit version a 64-bit DLL.

Please make sure that the SID32.DLL is registered in Windows. To do this open a command shell with administrator rights, go to the directory of the SID32.DLL and call:

regsvr32.exe SID32.DLL

Additionally configure the according access in CAESAR's slave.ini file (<PROVEtechTADirectory>\bin\Caesar\driver\slave.ini). It is in the section [PartJ]. The following settings were correct for the SID32.DLL we have:

[PartJ]
PassThruVersion="02.02"
DeviceName="SID (SAE->RP1210)"

Attention: Running the PROVEtech:TA setup overwrites the slave.ini file.

To get more information from CAESAR, activate the CAESAR debug level 3 in the settings and reset CAESAR. Open the PROVEtech:TA Log file created by PROVEtech:TA and search for:

Starting Registry Scan for SAE-J2534 PassThru Devices

Below that, you can see what CAESAR searches for (your slave.ini settings):

Searching for PassThru 02.02 vendor and device "SID (SAE->RP1210)" (given by slave.ini)

After that CAESAR lists the found devices after each version (e.g. *== PassThru 02.02 ==*, corresponds to the „PassThruVersion" setting of the slave.ini). The part after "Name:" corresponds to the "DeviceName" setting of the slave.ini.

In the Windows registry you can find the according entries here (depending on 32-bit and 64-bit):

HKEY_LOCAL_MACHINE\SOFTWARE\Wow6432Node\PassThruSupport
HKEY_LOCAL_MACHINE\SOFTWARE\Wow6432Node\PassThruSupport.04.04
HKEY_LOCAL_MACHINE\SOFTWARE\PassThruSupport
HKEY_LOCAL_MACHINE\SOFTWARE\PassThruSupport.04.04

8.9.5 CAESAR Part Y and Part F

Problem:

You are using a combination of Part Y and Part F as diagnostics hardware. Is it necessary to connect the Part F with a power supply?

Solution:

You only have to connect Part F with a power supply, if you are working with a KLINE ECU. With a CAN ECU, there is no need to connect a power supply. Connecting a power supply to Part F in combination with a CAN ECU can even cause problems, there are some CAN ECUs, which do not work, if a power supply is connected to Part F.

8.9.6 TIMEOUTP2CAN

Problem:

You try to activate an ECU or try to execute a diagnostics command and you get error messages like "ComCoordinator:02142:TIMEOUTP2CAN", "ComCoordinator:02136:CANMESSAGENOTSEND" or "ComCoordinator:02120:CANBUSOFF". What is wrong?

Solution:

In most cases the above error messages are caused by problems with the CAN hardware. Check your hardware and your hardware settings. Possible reasons are an incorrect or missing CAN termination, a CAN malfunction, a low power supply or a wrong baud rate. The "TIMEOUTP2CAN" error message means, that the ECU did not respond to a diagnostics message. There can be dozens of reasons for this. Some examples: Low power supply, sleeping ECU, the gateway ECU does not route the messages.

8.9.7 Activation without matching variant

Problem:

You get the error message "Activation without matching variant not allowed". What can I do?

Solution:

The reason for this error message is, that the .cbf file does not contain the correct variant for your ECU. So you need a new .cbf file for your ECU, which contains the correct variant.

As a workaround, you can set the flag "ActivateWithoutVariant" in your ECUs "Settings" node (diagnostics tree) to the value "Activate without matching variant". Use this workaround with great care and only in situations, where you can accept a potential crash of PROVEtech:TA.

8.9.8 Activation of your ECU

Problem:

The activation (opening channel) of your ECU does not work. What should I do?

Solution:

There could be several reasons for this. First you should check the "Requested Interface" of the ECU settings (node "Settings" in the tree). This determines the physical layer of the communication (e.g. CANHS/LS and the baud rate). See ***8.9.3 Diagnostics interface of an ECU*** for details. When PROVEtech:TA complains about not finding the correct variant, the ECU returns an ID that is not specified in your .cbf file. See also ***8.9.7 Activation without matching variant***.

You should also check the "No matching resources to open a channel" error message (see ***8.9.1 No matching resources to open a channel***). Check ***8.9.6 TIMEOUTP2CAN*** for hardware problems.

8.9.9 Missing service, coding fragment, etc.

Problem:

I am missing a service, coding fragment, etc. in PROVEtech:TA. Where do I find it?

Solution:

PROVEtech:TA only displays the nodes supplied via the .cbf file. When something is missing, you should ask your supplier of the .cbf file for it.

8.9.10 Update of CAESAR Firmware

Problem:

When I use a different diagnostics tool or switch back to PROVEtech:TA, the firmware is updated every time. Why?

Solution:

The firmware is part of the CAESAR version contained in the tool. So when two tools use different CAESAR versions, the firmware will be updated every time you switch the tool.

8.9.11 CAESAR release authentication

Problem:

PROVEtech:TA cannot initialize the diagnostics. I get the error message "CAESAR release authentication is not identical with that extracted from configuration matrix file". What is wrong?

Solution:

Your CAESAR version of the c32s.dll and the files in the Caesar\driver and Caesar\gbf directory do not match. Please check out if you use the correct version of c32s.dll and the correct files in these two directories.

8.9.12 .ccf files

Problem:

What are .ccf files?

Solution:

CAESAR coding files (.ccf files) are files with additional coding information. They contain predefined codestring and fragment values, named with a so called "Meaning" and optional with a "PartNumber" and "PartVersion". If available, PROVEtech:TA allows you to access the predefined codestrings and fragments by selecting a "Meaning" or a "PartNumber" and "PartVersion". To use the information of an additional .ccf file for an ECU, you just have to put the .ccf file into the ccf directory of PROVEtech:TA

(the location of this directory is defined in the diagnostics tab of the PROVEtech:TA settings).

8.9.13 Application is hanging after CAESAR call

Problem:

In certain situations, it seems like the whole application hangs when executing a diagnostic command. Why?

Solution:

The diagnostic communication protocols (e.g. KWP2000) define a response message where the ECU can answer, that it is busy and the

response is pending but it will come soon. When the ECU sends this response message cyclic and never sends the real response,

CAESAR waits an endless time for it. The only way to force CAESAR to return from the command is breaking the physical connection

(e.g. unplug the CAN connector). After this CAESAR will generate a timeout after a few seconds.

8.9.14 “Cannot send message” on low speed CAN

Problem:

CAESAR returns an error like "Cannot send message" when you have a low speed CAN connection. Why?

Solution:

This can happen in case the battery power and ground are not connected to the CAESAR CAN hardware.

8.9.15 Coding problems

Problem:

How can I read the coding default values from the .cbf file and then code the ECU with the default values?

Solution:

In the PROVEtech:TA user interface, in diagnostics tree, select "Settings" branch for your ECU. Then go to the data window, and select the entry "InitialReadCoding". Select "Do not read VarCoding automatically". This disables the process, that the CodeStrings of the ECU are read automatically when starting PROVEtech:TA. Now restart PROVEtech:TA, and all CodeStrings in the CodingDomains are filled with their default values. Press Coding. Now the default values are coded in the ECU.

Using testscripts, it is possible to read the CodeStrings via the according "DiagNode" object and write them by calling "Diag.DoECUCoding". You can also change fragmens via "DiagNode" before doing this.

Pitfall:

As soon as "Diag.DoECUCodingByFragment" is called within a test script, the CodeStrings are read from the ECU and thus the default values from the .cbf file are overwritten. So make sure this does not happen prior to writing by "Diag.DoECUCoding".

8.9.16 Channel init failed

Problem:

Read Errors fails with CAESAR API error, Channel init failed. CAESAR error text:

ChannelManager: 03029: ECU responded with a refuse message to the init message. ECU returned "7F 10 22".

What can I do to fix this problem?

Solution:

The ECU refuses correct communication. The ECU or the ECU control description may be erroneous. Please check for recent changes.

Also a malfunction in Part-Y can cause this error. Please exchange the Part-Y and test again.

8.9.17 Application crashes due to data version mismatch

To work properly, CAESAR requires a correct version of CBF and GBF files. When introducing a new ECU in a working system, or when updating the software without updating all files, sometimes version mismatch can lead to different symptoms. The most popular is the error message “No matching resources…” (see chapter ***8.9.1 No matching resources to open a channel***), but also a crash of CAESAR may happen, leading to the whole system being stopped.

Please check the last CAESAR lines in the trace file. If you find a hint like “GPD version too low”, then most probably you have this problem. Updating CBF and GBF files fitting the current CAESAR version should solve the problem.

8.10 Problems concerning the MCD-3 D diagnostics

The following chapters describe problems and solutions concerning the MCD-3 D diagnostics. The chapter ***8.8.1 General eCOM problems*** is also relevant for the MCD-3 D diagnostics.

Please read the chapter ***6.2.4 Button to terminate the MCD-3 D server*** as well.

8.10.1 Configuring MCD-3 D servers

Diagnostics is a very complex technology, due to several software modules delivered by different companies. Installing and configuring the hardware and software needs a lot of time and should be done very thoroughly.

8.10.1.1 Installing MCD-3 D

The MCD-3 D diagnostics module of PROVEtech:TA requires at least one MCD-3 D diagnostics server (aka kernel). To run a 32 or 64 version of PROVEtech:TA you need an appropriate version of the kernel and also the proper version of Java VM (which is used by the kernel).

You can install the kernel before or after installing PROVEtech:TA. The installed kernel is required when creating a diagnostic configuration (see ***3.2.7.8 MCD-3 D diagnostics settings***).

The MCD-3 D servers are COM servers. They are registered in the Windows registry by the setup. It is not possible to register more than one version of a special COM server. Fortunately, PROVEtech:TA does not need the registration of the MCD-3 D server to use it. Therefore it is possible to keep copies of different kernels and use them in different projects.

For setup execution you need admin rights. If you use the kernel(s) only with PROVEtech:TA or need another version on the machine, you can simply copy the kernel's installation directory from another PC and copy or manually create the configuration. For this you do not need admin rights.

8.10.1.1.1 Installing DTS (Softing)

PROVEtech:TA has been tested with DTS9 version 9.01 (64 bit only) and DTS8 version 8.06.027, 8.09.060, 8.12.016, 8.13.029, 8.15.021, 8.16.011, 8.17.010 (starting with 8.15.xxx both 32 and 64 bit versions). Version 8.08.021 is incompatible!
Other compatible versions are supported, but not extensively tested.

The DTS9 is licensed for the PC workstation. The DTS8 is licensed for the MAC address or for a dongle (for this you need a driver). The license file must contain a "Server" or "DTS-COS" entry. A typical Monaco license does not sufficient. You can check (or import) the license with SystemConfiguration, a program installed together with the kernel.

You need also one or more appropriate 32 or 64 bit versions of D-PDU API

SystemConfiguration is used to create one or more projects based on ECU data in smr-d format. If you have ODX/PDX files, they can be converted into smr-d. You need also common data, defined in the Globals smr-d files. They define protocols and jobs common to all ECUs.

SystemConfiguration is also used to configure interfaces (D-PDU API) to enable ECU communication via a special hardware. There are different D-PDU API for eCOM, Vector hardware or Ethernet adapters. You can also configure a simulation interface. This allows you to use PROVEtech:TA to access the ECU data without hardware, e.g. when defining test sctipts. The simulation interface provides also a limited simulation of communication with the ECU.

A Java VM is included in the DTS installation. Other Java VM can be used, but this is not recommended by Softing.

There can be several versions of DTS on one PC, each with an own configuration. Different PROVEtech:TA configurations might use different DTS versions. But it is not possible to switch to another version at runtime due to problems at shutting down DTS which prevent an other version from starting. When changing between DTS versions, PROVEtech:TA must be restarted therefore.

8.10.1.1.2 Installing Prodis and VW kernels

This chapter deals with installing the DSA Prodis and the VW kernel, which is a special version of Prodis. Differences between both are mentioned.

PROVEtech:TA has been tested with Prodis 3.2.2, 3.6.1, 3.8.0 and 3.11.0 (32 and 64 bit) and with VW kernel 11, 12 and 17 (32 and 64 bit). Other compatible versions may also be used, but are not tested.

Prodis requitres a license file bound to the PC. A dongle is not required. The program ProdisLicenseRequest generates a lrq file which must be sent to DSA to get an individual license file.

The employees of the VW Group use the company's installation of the VW kernel with its included license.

Installing the kernel creates two environment variables pointing to the installation path and the config path. If you simply copy a kernel installation without using setup, you must create those variables manually. The names of them are PRODIS_MCD_HOME and PRODIS_MCD_CONFIG (for DSA Prodis) or VW_MCD_HOME and VW_MCD_CONFIG (for VW kernel).

An appropriate Java version (32 or 64 bit) must be installed separately and included in the PATH variable before other Java VMs (if present).

To create projects based on OTX/PDX files a ODX-Converter is offered in the setup.

DSA offers an own D-PDU API (PDU Daemon) for communication via Vector hardware, but any other standard compliant D-PDU API can be used. There is no simulation interface offered, but for offline work (without hardware access and/or hardware license) a D-PDU API can be anyway installed and used for accessing the ECU data, as long as communication is not required.

The 32 and 64 bit versions of the kernel uses the same environment variables and the same configuration files. Therefore using both 32 and 64 versions on the same PC is possible, but limited:

- the used interface as set in MCD3D_SERVER.INI must be defined in both 32 and 64 bit versions of the RDF file
- The PATH variable must be changed before PROVEtech:TA start to include the proper Java version at first (e.g. with a BAT file)

8.10.1.2 Basics

The MCD-3 D Server (also known as kernel) accesses the diagnose hardware through an interface implementing the standard D-PDU API. D-PDU API implementations are offered by several companies (DSA, Softing, I+ME, Vector, Berner&Mattner, Bosch), they run with different hardware.

Diagnostics is depending on several configuration files. The PROVEtech:TA own file DiagnosticsMCD3D.ini determines which MCD-3 D kernel is to be used in any configuration.

A RDF file (XML format) defines one or more interfaces (references to D-PDU API). The interface's software has also configuration options (files or registry). The MCD-3 D Kernel uses specific interfaces from the RDF file according to own configuration.

MCD-3 D works when the following conditions are fulfilled:

- PROVEtech:TA „finds" the kernel. According to the parameters in DiagnosticsMCD3D.ini the correct kernel DLL is being loaded
- The kernel „finds" the RDF file. Either by own configuration, or by a standard entry in the Registry:
  HKEY_LOCAL_MACHINE\SOFTWARE\Wow6432Node\D-PDU API\Root File (for 32 bit applications)
  or:
  HKEY_LOCAL_MACHINE\SOFTWARE\D-PDU API\Root File (for 64 bit applications)
- The RDF contains an interface fitting to the kernel
- The Interface is complete (hardware + driver + configuration files)
- A valid license is present (for some kernels also a dongle and a dongle driver)
- To execute jobs, kernel must have a defined path to JRE. There are different solutions to that issue.

8.10.1.3 Working without hardware

The MCD-3 D module can be used also without diagnostic hardware (MVCI) and ECUs. This is helpful when writing test scripts including diagnostic functions. The MCD-3 D kernel loads all descriptions and definitions from the ODX database and makes it possible to get all names and other information about ECUs, services, jobs and their parameters.

To work without hardware, you do not need a hardware license for PROVEtech:TA. There are significant differences in configuring all the supported MCD-3 D kernels.

8.10.1.3.1 Softing kernel without hardware

This kernel can be only started when a functioning interface can be used. For the purpose of offline work, the DTS supports a simulation interface. This can be set using the SystemConfiguration program. This interface supports also some kind of simulated communication. This can be very helpful as you can access the structure of the ECU's answers. For this, you need a hardware license for PROVEtech:TA, as the application cannot distinguish a simulation from a real hardware.

Please see also ***6.2.2.5 Simulating diagnostic communication*** and refer to Softing documentation for more details.

8.10.1.3.2 DSA and VW kernels without hardware

With some interfaces, this kernel can be started even without a connection to a hardware. It has been successfully tested with the Vector D-PDU API. Only when trying to contact an ECU, the kernel reports a problem. This means, you can simply copy a proper interface (the D-PDU API files) and the according entry in the RDF from a test bench PC, where the communication is correctly set up, to your desktop PC and use MCD-3 D offline, but you have to check if the given D-PDU API is suitable.

Theoretically, a simulation PDU API could be also used, but there are no such programs available at the moment.

8.10.1.4 Configuring DTS8 and DTS9

The typical architecture based on the Softing diagserver with eCOM is described here:

<table>
<tr><td colspan="2">PROVEtech:TA</td><td rowspan="2">Akkodis</td><td rowspan="2">CFG.ini</td></tr>
<tr><td>GUI</td><td>Basic</td></tr>
<tr><td colspan="2">MCD-3 D Interface</td><td>DiagnosticsMCD3D.ini</td></tr>
<tr><td colspan="2">MCD-3 D Kernel</td><td>Softing</td><td>SystemConfig.xml
DtsCconfig.xml
ConfigCharacteristic.xml
RDF_I+ME_ACTIA_XS.xml</td></tr>
<tr><td colspan="2">D-PDU API</td><td>I+ME</td><td>PDUAPI_I+ME_ACTIA_XS.ini</td></tr>
<tr><td colspan="2">SDconnect Toolkit</td><td>I+ME</td><td>toolkit.ini</td></tr>
<tr><td colspan="2">USB2Lan Adapter</td><td>ASIX</td><td></td></tr>
<tr><td colspan="2">eCOM</td><td>I+ME</td><td>Firewall, Network settings</td></tr>
</table>

Separate vs. Common Configuration

The Standard MCD-3 D defines a registry entry referencing a RDF file with all needed D-PDU API interfaces.

This can lead to conflicts if one application changes configurations used also by other applications.

Using one single RDF file is not forbidden and DTS8 can work properly, but in case of problems it is good to understand all the details contained in this chapter to correct the installation.

To avoid conflicts, an own RDF file can be used. Especially I-HiL installations use a separate DTS8 subfolder in the PROVEtech:TA installation directory. In this case an own SystemConfiguration.xml is used to reference a separate configuration folder.

License File

DTS8 is licensed using dongles. The dongle requires a driver (haspdinst.exe).

The dongle must be in the USB port during the start of MCD-3 D.

The license file is valid for a special dongle number and can be limited in time.

The license file lies in the same folder as the currently used DtsConfig.xml. Instead of moving the license there, SystemConfiguration.exe (a Softing config program) should be used. After import of the license the program displays recognized dongle numbers and the lending time (for which the license is valid).

DTS9 has a different licensing. The license is activated via the DTS9 GUI for the PC workstation and does not require a dongle.

SystemConfiguration.exe

This program (in the install folder of the DTS8 kernel) is used to edit the DtsConfig.xml, especially to select the proper interface (D-PDU API), administering the diagnostic data and to import a license.

As long as the program is running, the DtsConfig.xml is locked and PROVEtech:TA (or its diagnostic module) cannot be started.

If some knowledge can be assumed, the DtsConfig.xml can be edited with any text editor.

Trace Files

With DTS and the used D-PDU API, several trace files with different debug levels can be recoded. See chapter ***8.10.2.1 Trace recording with DTS*** for details.

Files and Paths for DTS9

The central configuration file is DtsConfig64.xml. It is located in the folder:

C:\ProgramData\Softing\DTS9\<Version>\COS (versions 9.00-9.02)

Or – since 9.03:
C:\ProgramData\Softing\Diagnostic Tool Set\<Version>

where <Version> hat the format „X.YY“, e.g. „9.01“

To edit the configuration, use the program SystemConfiguration.exe. You can access it by clicking the button “Open configuration…” in the Settings window for MCD-3 D Diagnostics, just beside the server path line.

The main items causing trouble during installation are:

- <JVMLocation> - path to a valid Java VM
- <ProjectPath> - path to the MCD-3 D projects. In this folder you find the smr-d files
- <RdfFilePath> - path to RDF file. If empty, the registry based path is used (see also ***8.10.1.2 Basics***). You find details about interfaces and RDF file further below.
- < InterfaceConfiguration ShortName="eCOM"> - the DtsConfig.xml can contain several interface configurations. Daimler often uses the eCOM, but others are possible also. It is recommended not to change any parameters manually in this part.
  The interface configuration must correspond to an interface definition in the RDF file.

Other items, not described here, may also be interesting and subject to changes, e.g. for storing a MCD-3 D log file.

Characteristics

The folder containing DtsConfig64.xml contains also a subfolder “Characteristics” with more subfolders. These contain OEM specific configurations. E.g. the folder 1D73E455 contains Daimler specific configurations, Java libraries (for ZenZefi interaction) and globals.

Files and Paths for DTS8

Since Version 8.06.027 of the DTS8 the same kernel works with Daimler and non-Daimler ECUs, but has for both cases a separate configuration and uses different file formats. This leads to the following folder structure:

Install folder of DTS8 -> SystemConfig.xml

SystemConfig.xml

Tag <ConfigCharacteristicPath>

usually: C:\ProgramData\Softing\Diagnostic Tool Set\[version no.]\

refers the file ConfigCharacteristic.xml.

Prior versions of DTS8 than 8.06.025 (e.g. the often used 8.02.101) do not use this file, but refer directly to the DtsConfig.xml.

ConfigCharacteristic.xml

Tag <CurrentCharacteristic>

example: Daimler

refers to subfolder Daimler containing the DtsConfig.xml file

In case of non-Daimler data, this subfolder is named Standard.

DtsConfig.xml / DtsConfig64.xml

This is the central configuration file of the DTS (DtsConfig64.xml is used by the 64 bit variant of DTS). The Softing program SystemConfiguration.exe (in the same path as Dts8.dll) is used to edit this file. You can access it by clicking the button “Open configuration…” in the Settings window for MCD-3 D Diagnostics, just beside the server path line.

The main items causing trouble during installation are:

- <JVMLocation> - path to a valid Java VM
- <SystemProperty ShortName="com.softing.StoragePath"> - path to any valid folder with writing right access. Here temporary files for executing jobs are stored.
- <ProjectPath> - path to the MCD-3 D projects. In this folder you find the smr-d files
- <RdfFilePath> - path to RDF file. If empty, the registry based path is used (see also ***8.10.1.2 Basics***). You find details about interfaces and RDF file further below.

- < GlobalSmrFilesPath> - path to "Globals" – common ODX files, defining not single ECUs but protocols, common jobs and services etc.
- < InterfaceConfiguration ShortName="eCOM"> - the DtsConfig(64).xml can contain several interface configurations. Typically Daimler uses the eCOM, but others are possible also. It is recommended not to change any parameters manually in this part.
  The interface configuration must correspond to an interface definition in the RDF file.

Other items, not described here, may also be interesting and subject to changes, e.g. for storing a MCD-3 D log file.

RDF file

The RDF file must contain definitions of all interfaces to be used. In this case the term *interface* means a special hardware (e.g. eCOM, Vector CANcaseXL, Ethernet adapter…) and a fitting D-PDU API software.

DTS initializes all defined interfaces at startup, despite they are in use or not. Therefore we recommend to delete all unnecessary interface definitions to avoid resource conflicts and time-consuming routines

Special case eCOM and the I+ME Actia D-PDU API

DTS8 and DTS9 are often used with eCOM hardware and the I+ME Actia D-PDU API. This combination is very sensitive and in case of problems you get error messages which are not always helpful.

Please see chapter ***8.8 Problems concerning the eCOM*** and following for more details.

Alternative Hardware and D-PDU API by Vector GmbH

Generally the MCD-3 D kernel can run with different underlying software layers. The following configuration has been successfully tested:

<table>
<tr><td colspan="2">PROVEtech:TA</td><td rowspan="2">Akkodis</td><td rowspan="2">CFG.ini</td></tr>
<tr><td>GUI</td><td>Basic</td></tr>
<tr><td colspan="2">MCD-3 D Interface</td><td></td><td>DiagnosticsMCD3D.ini</td></tr>
<tr><td colspan="2">MCD-3 D Kernel</td><td>Softing</td><td>SystemConfig.xml
DtsCconfig.xml
ConfigCharacteristic.xml
RDF.xml</td></tr>
<tr><td colspan="2">D-PDU API</td><td>Vector</td><td></td></tr>
<tr><td colspan="2">CANcase, CANcard, VNxxxx</td><td>Vector</td><td></td></tr>
</table>

Instead of the D-PDU API by I+ME running with eCOM, the D-PDU API by Vector can be used. It requires no toolkits or USB2Lan adapter but simply Vector-CAN-Hardware (CANcaseXL, VN1610 etc.) with Vector driver. It has also no configuration files. Its parameters are stored in the registry under

HKEY_CURENT_USER\Software\Vector\D-PDU API

To use the Vector D-PDU API and hardware, it must be licensed. Please contact Vector (www.vector.de) for details.

There is also another D-PDU API compatible with Vector devices. It is called EDIC and is delevoped by Softing AG. Also this software requires a license. In this case the DTS license must contain also an entry for EDIC.

When using a Vector device, the corresponding D-PDU API software must be referenced in the RDF file and also in the DtsConfig(64).xml file.

Diagnostics Over Ethernet (DoIP)

ECUs having an Ethernet port can be accessed directly via TCP/IP and do not need any specific hardware between ECU and PC. They can be connected directly to the Ethernet port of the PC. You only need a special D-PDU API software for DoIP to be referenced in the RDF file. PROVEtech:TA has been tested with D-PDU API by Berner&Matter, I+ME Actia and Bosch.

More about settings you find in ***3.2.7.8 MCD-3 D diagnostics***

<table>
<tr><td colspan="2">PROVEtech:TA</td><td rowspan="2">Akkodis</td><td rowspan="2">CFG.ini</td></tr>
<tr><td>GUI</td><td>Basic</td></tr>
<tr><td colspan="2">MCD-3 D Interface</td><td></td><td>DiagnosticsMCD3D.ini</td></tr>
<tr><td colspan="2">MCD-3 D Kernel</td><td>Softing</td><td>SystemConfig.xml
DtsCconfig.xml
ConfigCharacteristic.xml
RDF.xml</td></tr>
<tr><td colspan="2">D-PDU API</td><td>Berner&Mattner, I+ME, Softing, Bosch, Vector</td><td></td></tr>
<tr><td colspan="2">Ethernet port</td><td></td><td></td></tr>
</table>

The ethernet chip of the ECU is normally off. To activate it, a voltage between 6V and 16V must be applied to a wakeup line. This can be done manually or by setting a signal by the HiL testbench hardware. If eCOM is in use, it can be used to activate the wakeup

pin by software. PROVEtech:TA does this automatically if eCOM is correctly configured.

Important:
when the wakeup signal is permanently on, the ECU cannot switch to sleep mode. Also for some D-PDU API the signal has to be activated shortly before activating the interface (before 1st ECU contact is being established).

DoIP can be used simultaneously with other channels (e.g. CAN via Vector device or eCOM). To determine which channel to use, select the appropriate LogicalLink.

8.10.1.5 Configuring DSA PRODIS and VW kernel

The typical architecture based on the DSA PRODIS kernel is described here:

<table>
<tr><td colspan="2">PROVEtech:TA</td><td rowspan="2">Akkodis</td><td rowspan="2">CFG.ini</td></tr>
<tr><td>GUI</td><td>Basic</td></tr>
<tr><td colspan="2">MCD-3 D Interface</td><td></td><td>DiagnosticsMCD3D.ini</td></tr>
<tr><td colspan="2">MCD-3 D Kernel</td><td>DSA</td><td>MCD3D_SERVER.ini
D-PDU-API_RDF.xml</td></tr>
<tr><td colspan="2">D-PDU API</td><td>DSA</td><td>VCIDEV.ini</td></tr>
<tr><td colspan="2">CANcaseXL, CANcardXL, VNxxxx</td><td>Vector</td><td></td></tr>
</table>

The VW kernel is a special version of the DSA Prodis kernel. There are no significant differences in architecture or functionality. The following chapter refers to both kernel variants.

Configuration

Which interface (D-PDU API) is used, determines the file MCD3D_SERVER.INI. It lies in the folder defined by the environment variable PRODIS_MCD_CONFIG (VW: VW_MCD_CONFIG). In WinXP this variable is empty and the file can be found in the installation folder (variable PRODIS_MCD_HOME resp. VW_MCD_HOME).

The interface to-be-used is declared in the line:

MCD3D_MVCI_PDU_API_Shortname="*Interface Name*"

The "*Interface Name*" must correspond to the SHORT_NAME of a valid interface definition in the RDF file.

The kernel finds the RDF file via the registry:

HKEY_LOCAL_MACHINE\SOFTWARE\Wow6432Node\D-PDU API\Root File (for 32 bit applications)

or:
HKEY_LOCAL_MACHINE\SOFTWARE\D-PDU API\Root File (for 64 bit applications)

Alternatively, the RDF can be set by the parameter

[MCD3D_BASICS]
MCD3D_MVCI_PDU_API_RDF_FILE="*Path to RDF*"

In the same folder there is another file:MCD3D_PROJECTS.INI. This file defines the path to the diagnostic files (the MCD-3 D projects).

The DSA and VW kernels can be used with any standard D-PDU API. DSA offers also an own version of D-PDU API, supporting, among others, Vector hardware.

The DSA D-PDU API configuration is stored in VCIDEV.INI in a folder determined by the variable DSA_PDU_API_CONFIG. If this is empty, the same folder as for the MCD3D_xxxx.INI files is used.

The D-PDU API is activated by the PDU-Daemon. It happens that the D-PDU API does not respond. In this case the Daemon process should be stopped using the taskbar icon or the task manager. The next MCD-3 D start will restart the Daemon process.

PATH Variable

The Java (JRE, 32-bit version for PROVEtech:TA 32-bit and 64-bit version for PROVEtech:TA 64-bit) required for job execution must be accessible via the PATH-Variable. It must contain the BIN path and the path to jvm.dll (this might be the BIN\CLIENT or BIN\SERVER folder).

Attention! If the proper version of Java cannot be found, the diagnostics process will crash without any useful error message at the attempt to execute any diagnostic job.

In case the PATH variable references several different JRE, the correct version must be the 1st one.

License File

The license is valid for a special PC. It is stored in the installation folder of the kernel. The file name contains the name of the computer, e.g.:

License_PRODIS.MCD_[computer name].lic.

To obtain a license, run ProdisLicenseRequest.exe and send the generated license request file to DSA. When the license file is sent, copy it into the installation folder (admin rights required).

In case of VW kernel, VW employees use a general company license.

Trace Files

See chapter ***8.10.2.2 Trace recording with DSA/VW*** for details of recording trace files.

8.10.2 Trace recording with MCD-3 D servers

In the following chapters you find a short instruction on how to record the most important traces if the diagnostics server and the used D-PDU API module. For full information please refer to the original documentation of the software modules.

8.10.2.1 Trace recording with DTS

When problems arise on the protocol level (hardware connected and recognized, but one or several ECUs have a strange behavior), it is often necessary to deliver trace files to the support (Softing or Daimler/DCP). Trace files can be recorded in different layers: the DTS8/DTS9 itself or the used D-PDU API software. Here is a collection of most popular settings. For more information please refer to the manuals of the used software modules or contact the responsible support.

The following hints refer to files described in chapter ***8.10.1.4 Configuring DTS8 and DTS9***.

DTS trace

Open SystemConfiguration, a program located in the DTS8 folder, switch to “Expert Mode”, select “System settings” and set all needed “Enable trace” entries. Check other entries also – e.g. the trace path etc.

D-PDU API trace with eCOM

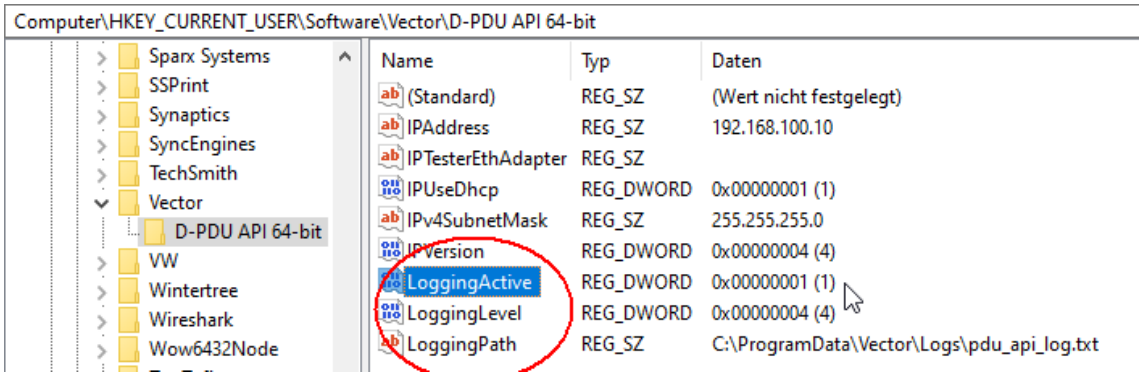
Find the file PDUAPI_I+ME_ACTIA_XS.ini and set the relevant entries in section [Logging].

In the section[Setting] see the entry ToolkitApiDll. It defines the path to the used DLL. In the same path you find a toolkit.ini file. In this file you can additionally activate tracing of this low-level module.

D-PDU API trace with Vector hard- and software

Open the Windows registry. You find all relevant entries in the key:
HKEY_CURRENT_USER\Software\Vector\D-PDU API

or, for 64 bit:
HKEY_CURRENT_USER\Software\Vector\D-PDU API 64-bit



D-PDU API trace from a Bosch DoIP interface

In the installation folder of the D-PDU API you can find the file D_PDU_API_CFG.xml. In this file you find the parameters LogFileDir and LogFileEnabled.

CeBas job traces

According to the installation instructions for Globals 1.2.4the following actions are needed to record CeBas job traces:

Create an environment variable LOG_CONFIG_PATH with the content:

„C:\ProgramData\Softing\Diagnostic Tool Set\JobLogs“

In this folder a text file must be created, named „joblogging.properties“ and with the following content:

```
handlers= java.util.logging.FileHandler

.level= FINEST

java.util.logging.FileHandler.pattern = c:/ProgramData/Softing/Diagnostic Tool Set/JobLogs/
java.util.logging.FileHandler.limit = 5000000
java.util.logging.FileHandler.count = 3
java.util.logging.FileHandler.formatter = com.daimler.odxjobs.common.logging.JobTextFormatter
java.util.logging.FileHandler.append = true
```

8.10.2.2 Trace recording with DSA/VW

Open MCD3D_SERVER.INI (see ***8.10.1.5 Configuring DSA PRODIS and VW kernel***) and set:

```
# (0=off 3=errors 9=traces 15=full)
MCD3D_LogLevel=9
```

```
# relative or absolute log directory
MCD3D_LogDirectory=../log
```

The referenced folder must exist and the current user must have writing access. Otherwise no log files will be created.

In the mentioned folder (in this case it is relative to the configuration folder), e.g.
“C:\Users\Public\DSA\prodismcd\log“ or respectively
“C:\Users\Public\Volkswagen\VW-MCD\Log”
you find the log files.

8.10.3 Known issues with MCD-3 D servers

Please read the chapter ***6.2.4 Button to terminate the MCD-3 D server*** as well.

8.10.3.1 Softing DTS

PROVEtect:TA supports different versions of the Softing DTS kernel. The following problems might occur with not all versions. Any MCD-3 D configuration of PROVEtect:TA can be associated with a different kernel.

8.10.3.1.1 Sporadic MCD-3 D server crashes or deadlocks when using it

In long-running tests it happens that PROVEtech:TA hangs during the execution of diagnostics functions or that the server process crashes. This is due to random failures of the diagnostic server DTS and happens in functions which might have been properly executed many times before and after. They happen randomly, we have no knowledge about what this depends on and how to avoid them, e.g. by changing the scripts. Some testbenches are free from them while others cannot fulfill a single night test.

Most cases happen when calling Dgn.Activate and DgnJob.Execute, but it happens also in other Execute calls (DgnService, DgnCodingDomain…) or even during starting or shutdown of the diagnostics module.

These problems became much more frequent after introducing DoIP and the security functions of Star3.

The DTS developer team at Softing AG was not able to find an ultimate solution for that.

If you encounter this problem, you have the following possibilities:

a) Activate DTS traces (see chapter ***8.10.2.1 Trace recording with DTS***), try to reproduce the issue and send the DTS traces to the Softing support. This information might help Softing to find a solution.

b) Read the chapter ***6.2.4 Button to terminate the MCD-3 D server*** for further instructions.

8.10.3.1.2 License Problems with DTS8

The DTS8 kernel has a dongle based licensing system. The license file must fit the used DTS8 version and the dongle serial number. For every used DTS8 version a separate license is required. The dongle requires a device driver (haspdinst.exe).

The license file is to be imported by SystemConfiguration.exe (in the installation folder of DTS8) and lies usually in the same folder as the used DtsConfig.xml file. This program displays in the *Licenses* page the dongle number(s) and the time for which the license is valid.

The following error message states that the license file „license.bin“ or the dongle with licensed serial no. is not recognized (missing dongle, no contact, or no driver installed):

Access to database file [file name] denied (license is missing).

In case of an invalid license file (wrong version) the message is:

Invalid schema of license file!

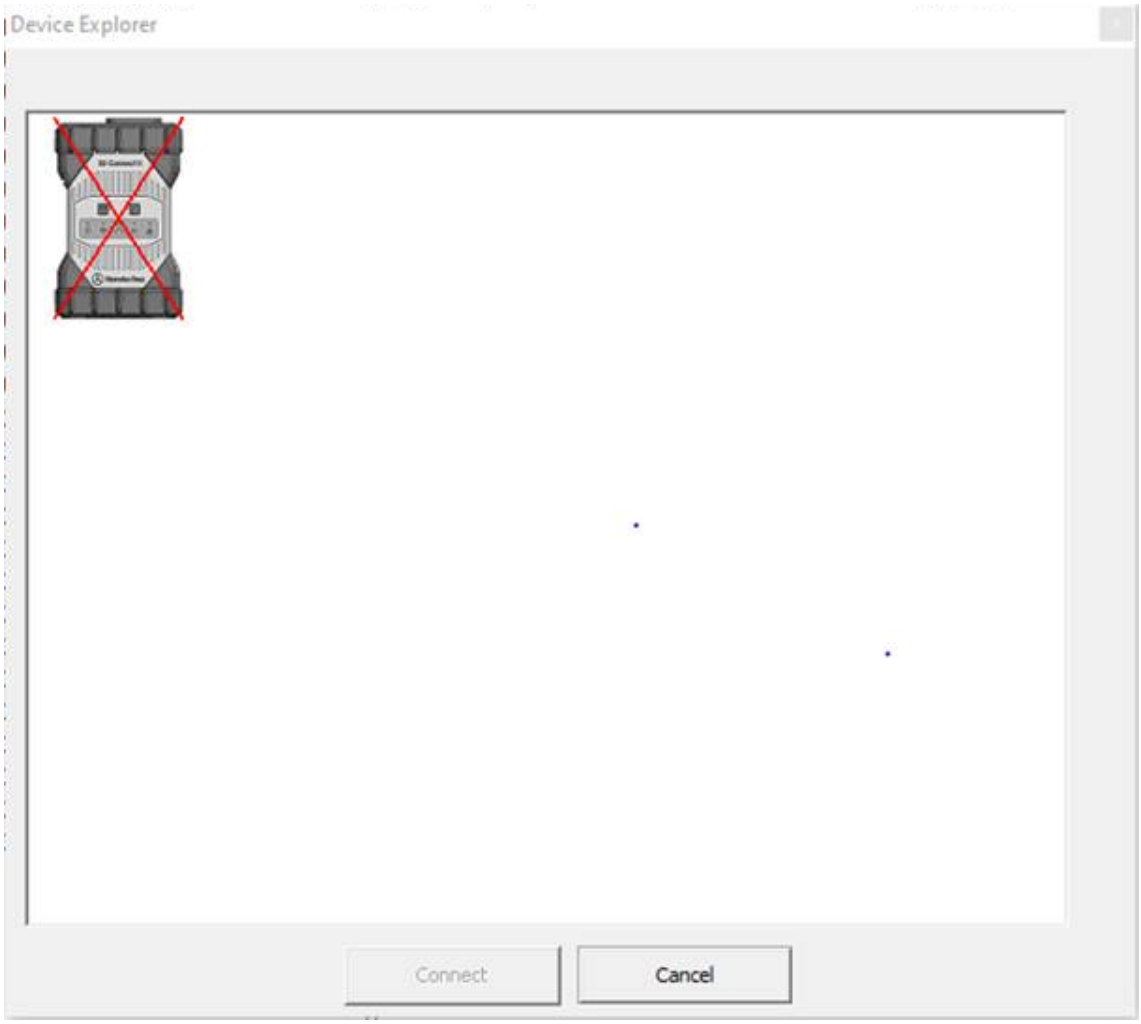
In case of this message there is no eCOM connected or it has no power:

No connected interface with the specified interface id and serial no

For more details see chapter ***8.10.1.4 Configuring DTS8 and DTS9.***

8.10.3.1.3 Device Explorer blocks initialization

After installation of DTS9 there may occur a warning message which blocks the initialization of the start of the program:



This message comes from the I+ME Actia D PDU-API when no eCOM hardware is connected.

If you do not use eCOM, please delete the interface definition from the RDF file. See chapter ***8.10.1.4 Configuring DTS8 and DTS9*** for details about RDF file.

8.10.3.1.4 Conflicts between different versions of D PDU-API

When installing a new D PDU-API (interface), the definition is added to the RDF file without deleting old versions. This can lead to conflicts as DTS initializes every interface contained in the RDF file. Especially EDIC interfaces have this problem – if you have several versions of EDIC, only one of them will properly work.

We recommend to delete all interface definitions which will not be used.

See chapter ***8.10.1.4 Configuring DTS8 and DTS9*** for details about RDF file and interfaces.

8.10.3.1.5 Conflicts With CAESAR Module

Problem:

Initialization of the CAESAR and the MCD-3 D systems leads to a conflict, when both systems use the same diagnosis hardware (eCom). If nothing is specified in the configuration, both interfaces try to exclusively access the hardware. It cannot be determined, which one – CAESAR or MCD-3 D – shall be successful.

Solution:

When using just MCD-3 D diagnosis, the simplest solution is to deactivate CAESAR permanently (file TA.INI, section [Panels], parameter DiagnosticsCaesar=0). This also shortens the initialization time.

If both interfaces are needed and only one hardware part is given, it is recommended to set the initialization priority in the configuration (Settings, tab “Configurations”). This prevents the other interface from trying to connect the hardware. A switch between CAESAR and MCD-3 D is still possible. As soon as one platform is shut down, the other one can be initialized by GUI or by test language.

8.10.3.1.6 Loosing contact with ECU

Problem:

After a contact break (e.g. unintentionally pulled the cable) a re-initialization fails. In case of Ethernet this happens quite often, in contrast to CAN communication.

Solution:

After losing Ethernet contact the MCD-3 D server is often not able to re-establish the contact again. In this case a restart of the MCD-3 D module might help. If this also does not work, the ECU is in an instable state. Pleas switch off the ActivationLine (or the whole ECU), shutdown the MCD-3 D module, switch on the ActivationLine and restart MCD-3 D after a few seconds.

8.10.3.1.7 MCD-3 D System Crashes due to wrong file versions

Problem:

Initialization of the MCD-3 D system (loading a MCD-3 D project) sometimes may lead to a crash, when the project data contains old *Globals* (smr-d files not associated with a special ECU but defining protocols etc.). No error message or log entry gives a hint to the real cause.

This problem is known since version 8.06.011.

Solution:

Exchange the *Globals* in your project with the newest version from the Diagnoseportal. Keep in mind that even one forgotten old file may lead to unpredictable behavior of the server or to a crash.

If this does not help, please contact Softing support directly.

Problem:

Bugs in 3rd party software. The DTS itself, as well as the D-PDU API are not bug-free. If an *unhandled exception* occurs in this software, the whole PROVEtech:TA process crashes. It can be checked by analyzing the dump file, contained in the problem report, created by PROVEtect:TA or the PROVEtech:TA Log window, which software module is responsible for the crash.

Solution:

For the solution of those problems the Daimler Diagnostic Helpdesk DCP is responsible. Our support will forward the report to dcp@mercedes-benz.com in that case.

8.10.3.1.8 Flashing Problems

Problem:

FlashJobs (maybe also other Jobs), working with DTS in version 8.02.101 do not work any more. The message “Internal error” is not helpful in finding the solution.

Cause:

The file „DtsConfig.xml“ contains a parameter needed for job execution. Installation of the DTS8 server leaves the entry empty and it must be changed manually. The parameter:

```
SystemProperty ShortName="com.softing.StoragePath"
```

Must contain a valid folder for which the current user has writing access. The MCD-3 D server stores in this folder temporary files needed for execution of the FlashJob.

Solution:

Open the file „DtsConfig.xml“ in notepad and enter a valid folder into the parameter.

When UpdateVersion is used for installing PROVEtech:TA, this parameter is copied from the old DtsConfig.xml and need not be corrected manually.

8.10.3.1.9 Non-standard Behavior

According to ASAM standard, services with parameters must have the parameters set explicitly by the application before execution or have predefined values in the data.

DTS executes services without ensuring that parameters have values set or default values defined. This can lead to toe following effect:

- An incomplete message is being sent to the ECU. The ECU answers with a 7F message meaning:
  „Incorrect message length or invalid format“
- A message of the proper length is sent, but the parameter bits are set to 0. This can lead to unpredictable effects, depending on the meaning of the parameters.

8.10.3.1.10 Problems Concerning Flashing

In this chapter we describe some common flashing problems.

8.10.3.1.10.1 Flashing Does Not Start

In this case two reasons are quite probable:

- The preconditions are not met. For example it is not possible to flash engine ECUs when the velocity signal is above 0. Ask the ECU developer for details.
- If no ECU can be flashed, there is probably a configuration problem regarding the Java Virtual Machine. Please refer to chapter ***8.10.1 Configuring MCD-3 D servers***.

8.10.3.1.10.2 Flashing Failed

If the flashing started but was interrupted or ended with an error, the cause may lie in disturbing signals on the bus. It is always a good means to stop any bus traffic before flashing.

8.10.3.1.10.3 Loading Flash File Failed

A flash file can be loaded at runtime. If loading a correct flash file ends with the error message “file XXXX not loaded” (or similar), then the file probably lies in a folder with special characters in the name (e.g. the German characters Ä Ö Ü ß). The DTS8 has problems dealing with special characters. Please move the file into another folder with a full path name (beginning with the disk letter) without any special characters.

8.10.3.1.11 Star3 problems

When every Star3 DiagJob results in an error message with a ClassNotFoundException, please check the versions of globals and ZenZefi and also if the property AdditionalClassPath in DtsConfig.xml is properly set according to the installation instruction of the currently used globals.

If you get other errors, please contact MBC hotline dcp@mercedes-benz.com. You may need detailed job logs. See chapter ***8.10.2.1 Trace recording with DTS*** how to activate them.

8.10.3.1.12 Problems with SecureSession

A bug in DTS causes in some cases executing the wrong StartCommunication service. Especially for Star3 ECUs this might be a problem as the SecureSession should be activated. Without this, CeBas jobs as variant coding, cannot be executed.

This bug is corrected in DTS 8.17.010. For former versions you can activate SecureSession either manually or automatically, as described in chapter ***6.2.2.17 Activation***.

8.10.3.2 DSA PRODIS and VW kernel

The VW kernel is a special version of the DSA Prodis kernel. There are no significant differences in architecture or functionality. The following chapter refers to both kernel variants.

8.10.3.2.1 Problems with Java Virtual Machine

Problem:

To execute jobs and services containing Java code, you need a proper JVM. For using the 32-bit PRODIS or VW kernel a 32-bit version of JVM must be installed and for the 64-bit version the 64-bit JVM. The PATH environment variable must contain the paths to „java.exe“ and to „jvm.dll“ before any other JVM paths.

Without a correct link to Java VM, executing a job might cause a system crash.

Solution:

Install a fitting (32 or 64 bit) JVM.
Update the contents of the PATH variable.

8.10.4 Graphical user interface of MCD-3 D diagnostics

8.10.4.1 When starting PROVEtech:TA for the very first time the diagnostics will not start

Problem:

After the installation I have started PROVEtech:TA for the very first time and I have gotten the error message "Dsystem Server selected but not installed".

Solution:

This error occurs when no compatible MCD-3 D server is installed. To use the MCD-3 D diagnostics in PROVEtech:TA please install a compatible MCD-3 D server of your choice.

8.10.4.2 Client is in use

Problem:

The MCD-3 D server shows an error that says that the client is already in use and that it cannot be occupied again.

Solution:

A failure notice of that kind hints that the MCD-3 D server is already occupied by another application. Please close the other application and start the diagnostic module again.

8.10.5 Communication issues with the ECU

8.10.5.1 Execution Of A Specific Service Or Job Fails Or Returns Incorrect Results

If the diagnosis generally runs – contact to the ECU is established, some services can be executed – sometimes there are problems with specific services or jobs when there are incompatibilities between ECU and ODX data or the configuration is incorrect. There can be several reasons:

1. Please check the selected ECU variant. Does it correspond to the actual ECU variant? If the parameter “Check variant on activation” in the ECU request area

is set to “Yes” and the contact is established without error messages, the variant is correct.

2. Incorrect ComParams (see chapter ***6.2.3.12.1 Special case: ComParams (service ProtocolParameterSet)***) might be the reason. Especially it happens often that the ECU response fails to be received before Timeout P2 and the diagnosis server ignores it. This kind of problems is easily identified using CAN traces. The cause for the problem might lie in invalid values in the ODX data, in problems of the ECU itself or a too slow communication hardware (it happens that Timeout P2 is violated with eCOM but the same service works with Vector device), or a large data traffic on the bus.
   A possible short-term solution can be to increase the Timeout P2 parameter.

3. If a service (or job) does not work in PROVEtech:TA, but it works with a different diagnosis software (Monaco for DTS8 or PRODIS.Client for DSA), it suggests diverging configurations. Please ensure that the alternative software uses the same diagnosis server and the same configuration. See chapter ***8.10.1 Configuring MCD-3 D servers*** and ff. for details)

4. If you have problems only with (all or some) jobs, the most probable cause is an invalid reference to the Java Virtual Machine or the parameter StoragePath in DtsConfig(64).xml (in case of DTS server). See chapter ***8.10.1 Configuring MCD-3 D servers*** and ff. for details)

5. If the error still continues to happen only in PROVEtech:TA and you ensured that alternative software is using the same diagnosis server and the same configuration, please send a problem report to PROVEtech Support (provetech@akkodis.com). It is very helpful if you can attach trace files of both cases: the good case with alternative software and the bad case with PROVEtech:TA.

If all above reasons can be excluded, it might be a fault in the diagnosis server, in the ODX data or in the ECU itself. Please contact in that case the responsible support or supplier.

8.10.5.2 Start of a MCD-3 D configuration with PROVEtech:TA fails with “DSA; Database format version file missing for project xy”

Solution:

1. Check whether the project specified in the configuration and the corresponding vehicle information exist in the “projects” folder of your DSA installation.

2. Check whether the specified project and vehicle information in your configuration are spelled correctly.

8.10.5.3 Execution of a service or activation of an ECU fails with “PDU CreateComLogicalLink failed”

Solution:

1. Check whether the Vector CANcaseXL/CANboardXL/CANcardXL/VNxxxx is attached correctly.
2. Restart your PC if the CAN hardware has been attached after the last start of your PC.
3. Check which CAN channel you are using for MCD-3 D diagnostics. This channel must be selected in VCIDEV.ini (.ini file of the DSA MCD-3 D server).

8.10.5.4 TX-Timeout (PDU_ERR_EVT_TX_ERROR)

Solution:

1. Check, whether the CAN bus is connected correctly.
2. Check, whether the communication parameter (COMParams) in ODX are correct. Especially „CP_Baudrate“ and „CP_CanPhysCanId“ are important in this case. You can select and display these communication parameter in the navigation area. Please find more information in chapter ***6.2.3.12.1 Special case: ComParams (service ProtocolParameterSet)***.

See also ***8.7.1 TX Error*** for more hints.

8.10.5.5 RX-Timeout (PDU_ERR_EVT_RX_ERROR)

Solution:

1. Check, whether the communication parameter (COMParams) in ODX are correct. Especially „CP_CanRespUUDTId“, „CP_CanRespUUDTId“ and „CP_CanRespUUDTFormat“ are important in this case.
   You can select and display these communication parameter in the navigation area. Please find more information in chapter ***6.2.3.12.1 Special case: ComParams (service ProtocolParameterSet)*** .
2. Maybe the positive service response in ODX is missing or not available. You can select an display service responses in the navigation area. Please find more information in chapter ***6.2.3.4.1.3 Service***.
3. Maybe the negative service response in ODX is missing or not available. You can select an display service responses in the navigation area. Please find more information in chapter ***6.2.3.4.1.3 Service***.

See also ***8.7.2 RX Error***.

8.10.5.6 '181A;DSA: Data type mismatch for ProtocolParameter […]'

Solution:

Check, whether communication parameter are in use, which are not ISO22900 / ISO 22901 standard conform or, if manufacturer-specific communication parameter are in use. These standards are also available in several versions, which differ in certain areas. Also check, if the D-PDU API version in use supports the standard version you want to use Please find more information in chapter ***6.2.3.4.1.3 Services***.

8.10.5.7 CompuMethod Error (eSYSTEM_COMPU_METHOD_FAILED)

Information:

The MCD-3 D server decodes responses from the ECU. The received messages on the bus are compared with the data in ODX. Does the response from the ECU not fit to the possible ECU responses, MCD-3 D cannot decode the message or parts of the message. In latter case, MCD-3 D returns the error eSYSTEM_COMPU_METHOD_FAILED. Example: The ECU responses with $7F $22 $13. The third byte of the possible service response is either $11 or $12. $13 is not defined.

Solution:

Check, whether the values received from the ECU are defined in ODX for the respective service response. You can select an display service responses in the navigation area. Please find more information in chapter ***6.2.3.4.1.3 Service***. It may get necessary to use an ODX Viewer in order to view values of more complex parameter.

8.10.5.8 Access to diagnostic databases not possible

Problem:

After the last update of PROVEtech:TA the diagnostic module does not start anymore, because the databases could not be accessed.

Solution:

Please make sure that you have proper data. For each version of a MCD-3 D server there is a version of the transformer, to convert the ODX data in the corresponding runtime format. It can happen that already transformed data are used with a newer MCD-3 D server that does not suite the data and so the database is not recognized by

the server. Please convert your ODX files with the new converter and replace the runtime files in the corresponding project.

8.10.5.9 Table entry not found

Problem:

On my request I get the error message "No matching TextTable entry found for value "115.000000"."

Solution:

This error message hints, that the database in the ODX file is not complete for this response. There is no entry for the error 0x73, that was sent by the ECU. The database is lacking an error text for this value. Please check your ODX data and add an entry for this error number of the ECU.

8.10.5.10 Java runtime error after job execution

Problem:

After executing a job an error message is shown in the communication view with a java runtime error message.

Solution:

A java runtime error hints that a job is not executed correctly and it raised a runtime exception. Please check the job in your database for any errors.

8.11 Problems concerning the fault simulation

The following chapters describe problems and solutions concerning the fault simulation.

8.11.1 The ECU does not react as expected

Problem:

After activating a fault like “Short to Ground” or “Short to Battery” the ECU does not react as expected. How can I solve this?

Solution:

It is possible that the fault simulation module has set the fault correctly, but the ECU (electronic control unit) does not respond as expected. For example, no diagnostic error code was set.

In the end, this could mean that you found an ECU bug and that's exactly what you wanted to achieve with your test system.

However, we cannot tell you how your ECU works and how its expected behavior is. On PROVEtech:TA side we simply send commands to the fault simulation unit. It is up to the test system to activate those fault settings correctly.

In case of dSPACE SCALEXIO, it is always important that your .portconfig file fits to the currently running SCALEXIO model.

8.11.2 Error frames in the CAN communication

Problem:

There are error frames in the CAN communication. What is wrong?

Solution:

There could be several reasons that may cause error frames:

- Did you configure the right CAN card?
- You have to twist the CAN cables!
- Are the CAN cables terminated with the right resistor?
- Are all connectors correctly plugged in?
- Is the gage on the CAN bus too low?
- Do you use the correct firmware on the fault simulation hardware?
- There is a problem with the transceiver on the CAN card!
- Are there interferences because of parallel cables?

8.11.3 Critical PROVEtech:TA Message

Problem:

PROVEtech:TA displays a dialog "Critical PROVEtech:TA Message" and some CAN messages. What is this?

Solution in case of a Fault Simulation message:

The dSPACE fault simulation modules report error messages asynchronously via CAN. PROVEtech:TA displays these messages in such a message box. The error itself might have several reasons. A very common reason is a fuse hit on the FIU. This can be a software or a hardware fuse. A hardware fuse needs a replacement, for a software fuse you can just reset the Fault Simulation.

PROVEtech:TA uses the same mechanism for the fuses of the ETAS ES4440 box. This box only uses lead fuses.

Solution in case of a CAESAR diagnostic message:

Your CAESAR license is about to expire. Please get a new license and replace the license file in the “bin” folder of your PROVEtech:TA installation. PROVEtech Support cannot issue CAESAR licenses.

8.12 Other issues

The following chapters describe other problems and solutions.

8.12.1 Transferring a complete PROVEtech:TA configuration

Problem:

You want to transfer a complete PROVEtech:TA configuration to a different computer.

Solution:

There is no automatic solution for that. However, it is not hard to do when taking care about some rules. Please read chapter ***3.2.7 Settings*** for more information on that.

8.12.2 PROVEtech:TA crashes or hangs

Problem:

PROVEtech:TA crashes, a message box pops up containing a message like "PROVEtech:TA has detected a problem" or "Abnormal program termination". When I click "OK" or "Cancel", PROVEtech:TA disappears. Or PROVEtech:TA disappears

without showing any message box before. Or the PROVEtech:TA user interface does not react to any input anymore. What is wrong?

Solution:

When the PROVEtech:TA process produces an access violation (invalid memory access) Windows terminates the process and possibly displays an according message box before that. There can be dozens of reasons for this, in most cases it is a program bug anywhere in the program code. In case the user interface does not respond anymore, the reason very often is a so-called deadlock (two threads wait for each other to finish an action and thus block each other) or an endless loop in the program code. It can also happen, that an action just takes a long time, e.g. executing a diagnostic service. In that case you simply must wait a bit longer. This can also happen if the computer has not enough RAM or a slow processor or when the virus scanner or another software slows down the process. See chapter ***8.12.3 PROVEtech:TA runs very slowly***.

The first thing you should do is to check the PROVEtech:TA Log for error messages (see ***3.6.1 PROVEtech:TA Log***). This can help you and us to isolate the bug. Very often a crash happens or the application hangs in combination with the usage of the diagnostics. In that case we, the PROVEtech:TA developers, cannot solve anything. PROVEtech:TA calls functions of foreign components like a MCD-3 D server or CAESAR and they cause the issue. We do not have the source code and no contact person for these foreign software components. We recommend that you collect hints and log data in that case and contact the suppliers of those foreign components.

In other cases, you can contact the PROVEtech support to analyze the issue and maybe solve it. Therefore, create a problem report via the PROVEtech:TA Log (see ***3.6.1 PROVEtech:TA Log***) using the menu item *Technical Support...* (see ***2.13 Creating a problem report***) directly after the crash or while the PROVEtech:TA main application is hanging and send it to PROVEtech Support (provetech@akkodis.com). It is important that the PROVEtech:TA Log is a separate process. It is still available when PROVEtech:TA has an issue. Close the PROVEtech:TA process (TA.exe) in case it hangs via the Windows Task Manager only after creating the technical report.

8.12.3 PROVEtech:TA runs very slowly

Problem:

PROVEtech:TA runs very slowly, for example when starting the application.

Reason:

A common reason for bad performance is when files are loaded from a network path (e.g., workpage that contains pictures, model files, configuration files, diagnostics files, ...). As network performance fluctuates and sometimes Windows cuts and reconnects network drives, there is waiting time in consequence.

PROVEtech:TA has no influence on integrated third-party software, e.g., regarding the loading of the model or the diagnostic data. PROVEtech:TA is only the frontend for these tools. However, loading even a very complex model should not take more than 5 minutes, and loading extensive diagnostic files usually does not take more than 1 minute.

You can read chapter ***8.5.10 The execution time of the Basic script takes very long*** to analyze the execution time of your Basic script.

In general and with a little practice, one can recognize in the PROVEtech:TA Log (see chapter ***3.6.1 PROVEtech:TA Log***) which operations and function calls caused the performance problems.

Performance problems due to the database connection can occur if, for example, there are firewalls to pass. You can display the execution time of an SQL command in the PROVEtech:TA Log. Set `Show additional debug output` in your PROVEtech:TA Log settings for that (see chapter ***8.4.8 The database is very slow***). You can then also see whether there is activity in PROVEtech:TA.

Please also consider that a completely different circumstance could affect the performance of your computer. One explanation could be that a virus scanner is running in the background, or your computer does not have enough memory, or the hard disk is very slow.

8.12.4 Memory overflow

Problem:

PROVEtech:TA shows an error message like: “Memory allocation of additional ??? MB has failed”.

Reason:

Influenced by the configuration, PROVEtech:TA might need a lot of memory. There are two types of memory: physical available memory and virtual memory.

In case the upper error message pops up, PROVEtech:TA cannot allocate more virtual memory. The error message shows how much virtual memory PROVEtech:TA currently occupies.

This typically happens in combination with large signal information files while loading the model or with large Automation Library protocols and Log/Verify results or large test protocols in common.

Typically the error message pops up after PROVEtech:TA already ran a long time (several days) without a restart.

There can be many reasons and it is hard to analyze. An obvious reason are memory leaks. PROVEtech:TA itself gets developed in a way to produce no memory leaks. However, it is not possible to rule this out. More often 3rd party software modules included in PROVEtech:TA cause memory leaks. As an example this can be diagnostic modules, but also modules included via test programs. If applicable you must demand an according fix from the provider of the module.

Beside memory leaks there can be issues concerning the memory fragmentation. While permanently allocating and freeing memory blocks of different sizes, the memory management gets in trouble as this creates a lot of small holes in the address space. It is not possible to allocate big blocks anymore in this case.

Possible solutions:

1. The size of the page file in Windows should be as large as possible. You can set it in the Windows Control Panel.
2. Restart PROVEtech:TA every day.
3. Deactivate unneeded modules (e.g. fault simulation).
4. An invalid test program could flood the memory (e.g. collections or arrays with too many elements, or use of extremely long strings)
5. Avoid very large test protocols (e.g. larger than 10 MB)
6. Windows shares some resources between all started processes. Terminate any unused process or service.
7. Have a look in the Windows Task Manager (as administrator) if other processes need a lot of memory.
8. See if you can isolate a special test activity that permanently increases the memory usage. If this uses a 3rd party software module, please contact the development of that module. If it only uses PROVEtech:TA standard functionality, please contact the PROVEtech Support (provetech@akkodis.com) and describe it.
9. The Test Language command ***TM.ClearCache*** (see Test Language user manual) clears the Test Manager cache, that means, empties locally saved data that was loaded from the database.

10. Additionally, the current need of working storage is logged in the PROVEtech:TA Log (see ***3.6.1 PROVEtech:TA Log***). This can help narrow down the cause of the memory increase. Look for "RAM" or "VirtualSize" in the trace files. Investigate whether memory usage is increasing gradually or sky-rocketing.

Read also in chapter ***2.10 32-bit and 64-bit***.

The following Basic example code could help you analyzing the problem. It outputs the current memory statistic values. You can adapt it and integrate it in your test scripts to detect changes at runtime.

```
'#Language "WWB-COM"

Option UsesAmbiguousError On
Option Explicit

'Win32 struct
Type PROCESS_MEMORY_COUNTERS
    cb As ULong
    PageFaultCount As ULong
    PeakWorkingSetSize As PortInt
    WorkingSetSize As PortInt
    QuotaPeakPagedPoolUsage As PortInt
    QuotaPagedPoolUsage As PortInt
    QuotaPeakNonPagedPoolUsage As PortInt
    QuotaNonPagedPoolUsage As PortInt
    PagefileUsage As PortInt
    PeakPagefileUsage As PortInt
End Type

'Win32 struct
Type MEMORYSTATUSEX
    dwLength As ULong
    dwMemoryLoad As ULong
    ullTotalPhys As UHuge_
    ullAvailPhys As UHuge_
    ullTotalPageFile As UHuge_
    ullAvailPageFile As UHuge_
    ullTotalVirtual As UHuge_
    ullAvailVirtual As UHuge_
    ullAvailExtendedVirtual As UHuge_
End Type

'Win32 struct
Type THREADENTRY32
    dwSize As ULong
    cntUsage As ULong
    th32ThreadID As ULong
    th32OwnerProcessID As ULong
    tpBasePri As Long
    tpDeltaPri As Long
    dwFlags As ULong
End Type

'Win32 constants
Const GR_GDIOBJECTS&=0
Const GR_GDIOBJECTS_PEAK&=2    'only Win7 or later
Const GR_USEROBJECTS&=1
```

```
Const GR_USEROBJECTS_PEAK&=4   'only Win7 or later

'Win32 functions
Declare Function GetProcessMemoryInfo Lib "psapi.dll" _
    (ByVal process As PortInt, sPmc As PROCESS_MEMORY_COUNTERS, ByVal size As Long) _
        As Long
Declare Function GetCurrentProcess Lib "Kernel32.dll" () As PortInt
Declare Function GetGuiResources Lib "User32.dll" _
    (ByVal process As PortInt, ByVal flags As ULong) As ULong
Declare Function GetProcessHandleCount Lib "Kernel32.dll" _
    (ByVal process As PortInt, ByRef handleCount As ULong) As Long
Declare Function GlobalMemoryStatusEx Lib "Kernel32.dll" _
    (sMs As MEMORYSTATUSEX) As Long
Declare Function CreateToolhelp32Snapshot Lib "Kernel32.dll" _
    (ByVal dwFlags As ULong, ByVal th32ProcessID As ULong) As PortInt
Declare Function CloseHandle Lib "Kernel32.dll" _
    (ByVal handle As PortInt) As Long
Declare Function GetCurrentProcessId Lib "Kernel32.dll" () As ULong
Declare Function Thread32First Lib "Kernel32.dll" _
    (ByVal hSnapshot As PortInt, te As THREADENTRY32) As Long
Declare Function Thread32Next Lib "Kernel32.dll" _
    (ByVal hSnapshot As PortInt, te As THREADENTRY32) As Long

'helper function to get thread count
Public Function GetCurrentProcessThreadCount() As ULong
    Dim sTe As THREADENTRY32
    Dim uCurrProcessId As ULong
    Dim uNumThreads As ULong
    Dim hSnap As PortInt

    hSnap=CreateToolhelp32Snapshot(4, 0)
    If hSnap<>-1 Then
        sTe.dwSize=7*4
        If Thread32First(hSnap, sTe)<>0 Then
            uCurrProcessId=GetCurrentProcessId()
            Do
                If sTe.th32OwnerProcessID=uCurrProcessId Then
                    uNumThreads+=1
                End If
            Loop While Thread32Next(hSnap, sTe)<>0
        End If
        CloseHandle(hSnap)
    End If
    Return uNumThreads
End Function

'converts signed integer to unsigned integer
Function Unsig(lVal As PortInt) As UHuge_
    If lVal>=0 Then Return lVal
    If Win64 Then Err.Raise 1, , "not supported"
    Return lVal+4294967296UH
End Function

'main function which outputs memory/handle values
Sub Main
    Dim sPmc As PROCESS_MEMORY_COUNTERS
    Dim sMs As MEMORYSTATUSEX
    Dim lProcess As PortInt
    Dim uHandleCount As ULong

    'get memory and handle information
    lProcess=GetCurrentProcess()
    GetProcessMemoryInfo(lProcess, sPmc, 8+If(Win64, 8*8, 8*4))
    GetProcessHandleCount(lProcess, uHandleCount)
    sMs.dwLength=2*4+7*8
    GlobalMemoryStatusEx(sMs)

    'output values
```

```
    Debug.Print "PagefileUsage="+Format(Unsig(sPmc.PagefileUsage)/1024/1024, "0.000")+ _
        "MB, WorkingSetSize="+Format(Unsig(sPmc.WorkingSetSize)/1024/1024, "0.000")+ _
        "MB, VirtualUsage="+Format((sMs.ullTotalVirtual-sMs.ullAvailVirtual) _
            /1024/1024, "0.000")+ _
        "MB of "+Format(sMs.ullTotalVirtual/1024/1024, "0.000")+ _
        "MB, UserHandles="+CStr(GetGuiResources(lProcess, GR_USEROBJECTS))+ _
        ", GdiHandles="+CStr(GetGuiResources(lProcess, GR_GDIOBJECTS))+ _
        ", Handles="+CStr(uHandleCount)+ _
        ", Threads="+CStr(GetCurrentProcessThreadCount())
End Sub
```

8.12.5 Screen scaling

Problem:

The PROVEtech:TA graphical user interface looks strange or blurry. Some fields seem to have the wrong size compared to others.

Reasons/possible solutions:

Microsoft added several different variants to Windows to support different screen scaling over the years. The screen resolution – how many pixels a monitor displays – is only a minor part of that. More important for the display is the density of pixels, how many pixels there are per inch. The typical unit for that is DPI (dots per inch). The more pixels there are per inch, the smaller the presentation looks if you do not enlarge it while painting it. Enlarging it while painting is very hard though as there are different elements to draw like text (vectors), pictures (picture dots), lines (line width), window borders, etc.

Windows nowadays has a separate scaling setting per monitor using a percentage number, typically in 25% steps, as each monitor can have a different pixel density. The so-called primary monitor, which has the number 1 in the Windows settings has a special role.

PROVEtech:TA differentiates two modes. In case the primary monitor has a scaling of 225% or less, PROVEtech:TA itself does the scaling based on that setting. In case a monitor has a different scaling than the primary monitor, Windows enlarges or reduces the output on that monitor automatically with strange results sometimes.

In case the primary monitor has a greater scaling than 225%, PROVEtech:TA lets the operating system completely perform the scaling. In that case PROVEtech:TA draws everything at 100% and Windows enlarges the picture depending on the monitor scaling. This enlargement inevitably leads to a slightly blurred display.

Very often the display is particularly strange in case a window stretches across several monitors with different scaling settings. Depending on which monitor covers the biggest part of the window, Windows uses that monitor’s setting as base for the overall scaling.

All in all, it is no good idea to use several monitors with different scaling settings on one Windows computer. We discourage you doing that. You only get a consistent

display if all monitors use the same scaling. Additionally, a high pixel density brings not much benefit for the normal office/lab usage – in opposite to a cell phone where the eyes are much closer to the screen.

Please read as well in chapter ***2.11 System requirements***.



In case you change the scaling setting of a monitor in Windows, you must always log off from Windows and log in again afterwards to get it working correctly. Microsoft tried for several versions to make this logging off and logging in procedure obsolete, but made the situation even worse in some cases. Never trust the output of Windows after changing the scaling. Restarting the applications is not sufficient to take over the scaling changes correctly even if Windows tells you that. Additionally, any tool tip Windows displays for changing the compatibility settings regarding PROVEtech:TA is counterproductive. Never change any compatibility setting for PROVEtech:TA. Reverse such a setting if Windows changed it. Only logging off from Windows and logging in again correctly changes the settings. If you do not do that, the Windows API reports wrong settings to PROVEtech:TA and thus PROVEtech:TA draws incorrect.

8.13 Tips and tricks

Tip 1: Context sensitive help

Set the focus on any PROVEtech:TA window and press F1 to display the according chapter of the manual.

Tip 2: Strip chart control in offline mode

To quickly zoom or move the content of a strip chart control in the offline mode, position the mouse cursor over it and press Ctrl (to move) or Shift (to zoom) plus the left mouse button and move the mouse. You can also use Ctrl+mouse wheel to zoom in/out. The manual describes this in the chapter ***4.4.2 The strip chart and multi-strip chart controls***.

Tip 3: Custom control for the workpage

You need a control for the workpage that is not part of the PROVEtech:TA standard controls? Try the custom control to build up your own control out of elementary controls which are more flexible than the standard ones.

Tip 4: Drag and drop on the workpage

You have a control on the workpage and you want to display the same signal with a different type of control? Drag and drop the existing control onto one of the control type buttons in the signal selection window. This creates a new control at the end of the workpage using the corresponding control type.

Tip 5: Pixel exact positioning on the workpage

To do a pixel exact positioning/sizing of workpage controls press the Shift key while moving the mouse.

Tip 6: Convert data acquisitions

The *Convert Data Acquisition...* can convert data acquisition files between different formats. See chapter ***3.4.1 Converting data acquisitions***.

For importing and exporting, PROVEtech:TA supports the following formats beside its internal format (.hdr):

Action	Format	Processing	Information
Import and Export	ASCII-table (.txt)	Signals only	Format described in the ***Administrator's Guide***.
Import and Export	MATLAB (.mat)	Signals only	Only a container format and PROVEtech:TA uses a specific structure which you can browse within MATLAB.

Import	MF4 (.mf4)	Signals and CAN/LIN/ FlexRay/Ethernet messages	Basically, the following applies to the import of MF4 files: Blocks or combinations of settings that cannot be converted into a usable signal or message are ignored if they do not impede the reading/evaluation of the remaining data. In such cases, corresponding warnings are written to the PROVEtech:TA log. Signals: Only and channel type 0 – fixed sample without record ID; only time master signal; data type only INT, UINT, FLOAT and DOUBLE with little or big endian; conversion: linear, rational or TextTable. Signals with non-equidistant sampling are resampled equidistant, the sampling rate can be configured per signal group in an additional INI file; if it is missing, a dialog pops up for entering the sampling rates. The file must be in the same directory as the .mf4 file and have the same name, but the file extension ".mf4cfg". The file has the following format: [Resample] *One entry per signal group:* *Signal group number = Sample rate in seconds* Example: [Resample] 0 = 0.05 1 = 0.033 In case of a missing entry or sampling rate <=0.0 there is an automatic choice. Messages: Chapter ***4.4.14.2 Handling of Ethernet*** describes the rules for Ethernet messages An additional INI file provides necessary parameters; if it is missing, a dialog pops up for entering the parameters. The file must be in the same directory as the .mf4 file and have the same name, but the file extension ".mf4cfg". The file has the following format: [ChannelMapping] *One entry per channel:*

			Channel name = path to bus description file[optional:|*Cluster*|*Channel*] Example: [ChannelMapping] 1 = C:\Files\MyCanChannel.dbc|INFOCAN|INFOCAN Li2 = C:\Files\MyLinChannel.ldf
Import	CANalyzer ASCII traces (.asc)	Signals and CAN/LIN/ FlexRay/Ethernet messages	Messages optionally converted to signals while importing, but the possibilities are very limited; the time stamps must increase monotonously, for Ethernet channels there are special rules described in chapter ***4.4.14.2 Handling of Ethernet***. An additional INI file provides necessary parameters; if it is missing, a dialog pops up for entering the parameters. The file must be in the same directory as the .asc file and have the same name, but the file extension ".asccfg". The file has the following format: [General] Messages = *1 for importing messages or 0* Signals = *1 for converting messages to signals or 0* SamplingRate = *sampling rate in seconds for signal conversion, default 0.001* [ChannelMapping] *One entry per channel:* *Channel name = path to bus description file*[optional:|*Cluster*|*Channel*] Example: [General] Messages = 1 Signals = 0 SamplingRate = 0.001 [ChannelMapping] 1 = C:\Files\MyCanChannel.dbc|INFOCAN|INFOCAN Li2 = C:\Files\MyLinChannel.ldf

Import	PCAPNG (.pcapng)	Ethernet messages only	The time stamps must increase monotonously, only support for one section in the file, the interfaces must have the type LINKTYPE_ETHERNET, use the same time stamp resolution and the data must reside in enhanced packet blocks, additionally, there are special rules described in chapter ***4.4.14.2 Handling of Ethernet***. An additional INI file provides necessary AUTOSAR-XML file paths for the TCP import; if it is missing, a dialog pops up for entering them. The file must be in the same directory as the .pcapng file and have the same name, but the file extension ".pcapngcfg". The file has the following format: [ChannelMapping] *One entry per channel:* *Channel name = path to AUTOSAR-XML file* Example: [ChannelMapping] MyChannel1 = C:\Files\MyFile1.arxml MyChannel2 = C:\Files\MyFile2.arxml
Export	CANalyzer ASCII traces (.asc)	CAN/LIN/ FlexRay/Ethernet messages only	Chapter ***4.4.14.2 Handling of Ethernet*** describes the rules for Ethernet messages
Export	MDF (.mdf)	Signals only	
Export	MF4 (.mf4)	Signals and CAN/LIN/ FlexRay/Ethernet messages	Chapter ***4.4.14.2 Handling of Ethernet*** describes the rules for Ethernet messages
Export	PCAPNG (.pcapng)	Ethernet messages only	Chapter ***4.4.14.2 Handling of Ethernet*** describes the rules for Ethernet messages
Export	TDM (.tdm)	Signals only	
Export	TDM Streaming (.tdms)	Signals only	

Import and Export	DLT (.dlt)	DLT messages only	While exporting to DLT, the direction (Rx/Tx) information is lost and multiple DLT channels are merged into one as the DLT format does not have channels. The HW time information is mapped to the DLT header time field (seconds from 1/1/1970, UTC).

Additionally, there is a plug-in interface for adding user defined formats. Please contact PROVEtech Support (provetech@akkodis.com) if you are interested in it.

Tip 7: Hotkeys in the Basic editor

The Basic editor supports a lot of hotkeys. You can see some of them in the context menu or in the tooltips of the toolbar or read the manual to get a complete list (see chapter ***5.4.2 The hotkeys***).

Tip 8: Drag and drop of signal names out of the Basic editor

You edit your test program which contains a signal name and you want to see this signal on the workpage? Mark the signal name and use drag and drop or copy and paste to create a new control on the workpage displaying this signal. The control type is the one currently selected in the signal selection window.

Tip 9: Button on the workpage that starts a test

To create a button on the workpage that starts a test, drag and drop or copy and paste the test from the Test Manager tree to the workpage.

Tip 10: Test protocols in HTML

PROVEtech:TA uses the Internet Explorer and Edge on Chromium to display the test protocols. So you can use **System.Remark** to output HTML (see ***5.6.3.3 Writing test protocols in HTML format***). PROVEtech:TA automatically inserts the <pre> tag in front of every protocol. This is good for simple protocols. You can switch back to the normal HTML mode via System.Remark("</pre>").

To use images in your protocol, add the image paths in the configuration dialog (menu item *Application|Settings...*, see ***3.2.7.5 Test Manager*** in the Settings dialog) tab `Test Manager`. PROVEtech:TA automatically copies these files into the protocol directory.

Attention: If you want to design your test protocol in html, you should make sure that the protocols are still readable in Dark Mode operation on different PROVEtech:TA installations.

Tip 11: ECU variants in the CAESAR diagnostics

In the CAESAR diagnostics select `ECU Manager` to see the currently selected variants of all ECUs. You can also change the variants there. This is useful for working offline. Alternatively, you can view and change the variant of a single ECU by selecting the `Settings` node of the ECU in the `All` tab or by selecting the ECU in the `Settings` tab.

Tip 12: .cbf files without matching variant in the CAESAR diagnostics

In the CAESAR diagnostics select `Settings` tab to see and change the setting for allowing the activation of ECUs without the matching variant in the corresponding .cbf files. Normally it is no good idea to activate an ECU without having the correct .cbf file (containing the correct variant), but sometimes you just do not get a correct one.